Category Framework

Release 10.10

The Sage Development Team

Oct 28, 2026

CONTENTS

1 Introduction 1

2 The Sage Category Framework 3

3 Maps and Morphisms 165

4 Individual Categories 267

5 Functorial constructions 1527

6 Examples of parents using categories 1575

7 Internals 1677

8 Indices and Tables 1695

Python Module Index 1697

Index 1701

CHAPTER

ONE

INTRODUCTION

1.1 Sage categories quickref

- `sage.categories.primer?` a primer on Elements, Parents, and Categories
- `sage.categories.tutorial?` a tutorial on Elements, Parents, and Categories
- `Category?` technical background on categories
- `Sets()`, `Semigroups()`, `Algebras(QQ)` some categories
- `SemiGroups().example()??` sample implementation of a semigroup
- `Hom(A, B)`, `End(A, Algebras())` homomorphisms sets
- `tensor`, `cartesian_product` functorial constructions

Module layout:

- *sage.categories.basic* the basic categories
- *sage.categories.all* all categories
- *sage.categories.semigroups* the `Semigroups()` category
- *sage.categories.examples.semigroups* the example of `Semigroups()`
- *sage.categories.homset* morphisms, …
- *sage.categories.map*
- *sage.categories.morphism*
- *sage.categories.functor*
- *sage.categories.cartesian_product* functorial constructions
- *sage.categories.tensor*
- *sage.categories.dual*

CHAPTER

TWO

THE SAGE CATEGORY FRAMEWORK

2.1 Elements, parents, and categories in Sage: a primer

2.1.1 Abstract

The purpose of categories in Sage is to translate the mathematical concept of categories (category of groups, of vector spaces, …) into a concrete software engineering design pattern for:

- organizing and promoting generic code
- fostering consistency across the Sage library (naming conventions, doc, tests)
- embedding more mathematical knowledge into the system

This design pattern is largely inspired from Axiom and its followers (Aldor, Fricas, MuPAD, …). It differs from those by:

- blending in the Magma inspired concept of Parent/Element
- being built on top of (and not into) the standard Python object oriented and class hierarchy mechanism. This did not require changing the language, and could in principle be implemented in any language supporting the creation of new classes dynamically.

The general philosophy is that *Building mathematical information into the system yields more expressive, more conceptual and, at the end, easier to maintain and faster code* (within a programming realm; this would not necessarily apply to specialized libraries like gmp!).

One line pitch for mathematicians

Categories in Sage provide a library of interrelated bookshelves, with each bookshelf containing algorithms, tests, documentation, or some mathematical facts about the objects of a given category (e.g. groups).

One line pitch for programmers

Categories in Sage provide a large hierarchy of abstract classes for mathematical objects. To keep it maintainable, the inheritance information between the classes is not hardcoded but instead reconstructed dynamically from duplication free semantic information.

2.1.2 Introduction: Sage as a library of objects and algorithms

The Sage library, with more than one million lines of code, documentation, and tests, implements:

- Thousands of different kinds of objects (classes):

  Integers, polynomials, matrices, groups, number fields, elliptic curves, permutations, morphisms, languages, … and a few raccoons …

- Tens of thousands methods and functions:

  Arithmetic, integer and polynomial factorization, pattern matching on words, …

Some challenges

- How to organize this library?

  One needs some bookshelves to group together related objects and algorithms.

- How to ensure consistency?

  Similar objects should behave similarly:

```
sage: Permutations(5).cardinality()
120

sage: GL(2,2).cardinality()
↪# needs sage.libs.gap sage.modules
6

sage: A = random_matrix(ZZ, 6, 3, x=7)
↪# needs sage.modules
sage: L = LatticePolytope(A.rows())
↪# needs sage.geometry.polyhedron sage.modules
sage: L.n_points()                  # oops!   # random                    ␣
↪ # needs sage.geometry.polyhedron sage.modules
37
```

```
>>> from sage.all import *
>>> Permutations(Integer(5)).cardinality()
120

>>> GL(Integer(2),Integer(2)).cardinality()                                ␣
↪                 # needs sage.libs.gap sage.modules
6

>>> A = random_matrix(ZZ, Integer(6), Integer(3), x=Integer(7))            ␣
↪                           # needs sage.modules
>>> L = LatticePolytope(A.rows())                                         #␣
↪needs sage.geometry.polyhedron sage.modules
>>> L.n_points()                  # oops!   # random
↪# needs sage.geometry.polyhedron sage.modules
37
```

- How to ensure robustness?
- How to reduce duplication?

  Example: binary powering:

```
sage: m = 3
sage: m^8 == m*m*m*m*m*m*m*m == ((m^2)^2)^2
True
```

```
>>> from sage.all import *
>>> m = Integer(3)
>>> m**Integer(8) == m*m*m*m*m*m*m*m == ((m**Integer(2))**Integer(2))**Integer(2)
True
```

```
sage: # needs sage.modules
sage: m = random_matrix(QQ, 4, algorithm='echelonizable',
....:                   rank=3, upper_bound=60)
sage: m^8 == m*m*m*m*m*m*m*m == ((m^2)^2)^2
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> m = random_matrix(QQ, Integer(4), algorithm='echelonizable',
...                   rank=Integer(3), upper_bound=Integer(60))
>>> m**Integer(8) == m*m*m*m*m*m*m*m == ((m**Integer(2))**Integer(2))**Integer(2)
True
```

We want to implement binary powering only once, as *generic* code that will apply in all cases.

2.1.3 A bit of help from abstract algebra

The hierarchy of categories

What makes binary powering work in the above examples? In both cases, we have *a set* endowed with a *multiplicative binary operation* which is *associative* and which has a unit element. Such a set is called a *monoid*, and binary powering (to a nonnegative power) works generally for any monoid.

Sage knows about monoids:

```
sage: Monoids()
Category of monoids
```

```
>>> from sage.all import *
>>> Monoids()
Category of monoids
```

and sure enough, binary powering is defined there:

```
sage: m._pow_int.__module__                                                 #␣
↪needs sage.modules
'sage.categories.monoids'
```

```
>>> from sage.all import *
>>> m._pow_int.__module__                                                 #␣
↪needs sage.modules
'sage.categories.monoids'
```

That's our bookshelf! And it's used in many places:

```
sage: GL(2, ZZ) in Monoids()                                                #␣
↪needs sage.modules
True
```

(continues on next page)

(continued from previous page)

```
sage: NN in Monoids()
True
```

```
>>> from sage.all import *
>>> GL(Integer(2), ZZ) in Monoids()
↪     # needs sage.modules
True
>>> NN in Monoids()
True
```

For a less trivial bookshelf we can consider euclidean rings: once we know how to do euclidean division in some set R, we can compute gcd's in R generically using the Euclidean algorithm.

We are in fact very lucky: abstract algebra provides us right away with a large and robust set of bookshelves which is the result of centuries of work of mathematicians to identify the important concepts. This includes for example:

```
sage: Sets()
Category of sets

sage: Groups()
Category of groups

sage: Rings()
Category of rings

sage: Fields()
Category of fields

sage: HopfAlgebras(QQ)
Category of Hopf algebras over Rational Field
```

```
>>> from sage.all import *
>>> Sets()
Category of sets

>>> Groups()
Category of groups

>>> Rings()
Category of rings

>>> Fields()
Category of fields

>>> HopfAlgebras(QQ)
Category of Hopf algebras over Rational Field
```

Each of the above is called a *category*. It typically specifies what are the operations on the elements, as well as the axioms satisfied by those operations. For example the category of groups specifies that a group is a set endowed with a binary operation (the multiplication) which is associative and admits a unit and inverses.

Each set in Sage knows which bookshelf of generic algorithms it can use, that is to which category it belongs:

```
sage: G = GL(2, ZZ)                                                    #␣
↪needs sage.modules
sage: G.category()                                                     #␣
↪needs sage.modules
Category of infinite groups
```

```
>>> from sage.all import *
>>> G = GL(Integer(2), ZZ)                                                ␣
↪      # needs sage.modules
>>> G.category()                                                       #␣
↪needs sage.modules
Category of infinite groups
```

In fact a group is a semigroup, and Sage knows about this:

```
sage: Groups().is_subcategory(Semigroups())
True
sage: G in Semigroups()                                                #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> Groups().is_subcategory(Semigroups())
True
>>> G in Semigroups()                                                  #␣
↪needs sage.modules
True
```

Altogether, our group gets algorithms from a bunch of bookshelves:

```
sage: G.categories()                                                   #␣
↪needs sage.modules
[Category of infinite groups, Category of groups, Category of monoids,
 ...,
 Category of magmas,
 Category of infinite sets, ...]
```

```
>>> from sage.all import *
>>> G.categories()                                                     #␣
↪needs sage.modules
[Category of infinite groups, Category of groups, Category of monoids,
 ...,
 Category of magmas,
 Category of infinite sets, ...]
```

Those can be viewed graphically:

```
sage: g = Groups().category_graph()                                    #␣
↪needs sage.graphs
sage: g.set_latex_options(format='dot2tex')                            #␣
↪needs sage.graphs sage.modules sage.plot
sage: view(g)                                  # not tested            #␣
↪needs sage.graphs sage.modules sage.plot
```

```
>>> from sage.all import *
>>> g = Groups().category_graph()                                              #␣
↪needs sage.graphs
>>> g.set_latex_options(format='dot2tex')                                      #␣
↪needs sage.graphs sage.modules sage.plot
>>> view(g)                                  # not tested                     #␣
↪needs sage.graphs sage.modules sage.plot
```

In case `dot2tex` is not available, you can use instead:

```
sage: g.show(vertex_shape=None, figsize=20)                                    #␣
↪needs sage.graphs sage.modules sage.plot
```

```
>>> from sage.all import *
>>> g.show(vertex_shape=None, figsize=Integer(20))                              ␣
↪     # needs sage.graphs sage.modules sage.plot
```

Here is an overview of all categories in Sage:

```
sage: g = sage.categories.category.category_graph()                            #␣
↪needs sage.graphs sage.groups sage.modules
sage: g.set_latex_options(format='dot2tex')                                    #␣
↪needs sage.graphs sage.modules sage.plot
sage: view(g)                                  # not tested                   #␣
↪needs sage.graphs sage.modules sage.plot
```

```
>>> from sage.all import *
>>> g = sage.categories.category.category_graph()                              #␣
↪needs sage.graphs sage.groups sage.modules
>>> g.set_latex_options(format='dot2tex')                                      #␣
↪needs sage.graphs sage.modules sage.plot
>>> view(g)                                  # not tested                     #␣
↪needs sage.graphs sage.modules sage.plot
```

Wrap-up: generic algorithms in Sage are organized in a hierarchy of bookshelves modelled upon the usual hierarchy of categories provided by abstract algebra.

Elements, Parents, Categories

Parent

A *parent* is a Python instance modelling a set of mathematical elements together with its additional (algebraic) structure.

Examples include the ring of integers, the group S_3, the set of prime numbers, the set of linear maps between two given vector spaces, and a given finite semigroup.

These sets are often equipped with additional structure: the set of all integers forms a ring. The main way of encoding this information is specifying which categories a parent belongs to.

It is completely possible to have different Python instances modelling the same set of elements. For example, one might want to consider the ring of integers, or the poset of integers under their standard order, or the poset of integers under divisibility, or the semiring of integers under the operations of maximum and addition. Each of these would be a different instance, belonging to different categories.

For a given model, there should be a unique instance in Sage representing that parent:

```
sage: IntegerRing() is IntegerRing()
True
```

```
>>> from sage.all import *
>>> IntegerRing() is IntegerRing()
True
```

Element

An *element* is a Python instance modelling a mathematical element of a set.

Examples of element include 5 in the integer ring, $x^3 - x$ in the polynomial ring in x over the rationals, $4 + O(3^3)$ in the 3-adics, the transposition (12) in S_3, and the identity morphism in the set of linear maps from $\mathbf{Q}^3$ to $\mathbf{Q}^3$.

Every element in Sage has a parent. The standard idiom in Sage for creating elements is to create their parent, and then provide enough data to define the element:

```
sage: R = PolynomialRing(ZZ, name='x')
sage: R([1,2,3])
3*x^2 + 2*x + 1
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, name='x')
>>> R([Integer(1),Integer(2),Integer(3)])
3*x^2 + 2*x + 1
```

One can also create elements using various methods on the parent and arithmetic of elements:

```
sage: x = R.gen()
sage: 1 + 2*x + 3*x^2
3*x^2 + 2*x + 1
```

```
>>> from sage.all import *
>>> x = R.gen()
>>> Integer(1) + Integer(2)*x + Integer(3)*x**Integer(2)
3*x^2 + 2*x + 1
```

Unlike parents, elements in Sage are not necessarily unique:

```
sage: ZZ(5040) is ZZ(5040)
False
```

```
>>> from sage.all import *
>>> ZZ(Integer(5040)) is ZZ(Integer(5040))
False
```

Many parents model algebraic structures, and their elements support arithmetic operations. One often further wants to do arithmetic by combining elements from different parents: adding together integers and rationals for example. Sage supports this feature using coercion (see `sage.structure.coerce` for more details).

It is possible for a parent to also have simultaneously the structure of an element. Consider for example the monoid of all finite groups, endowed with the Cartesian product operation. Then, every finite group (which is a parent) is also an element of this monoid. This is not yet implemented, and the design details are not yet fixed but experiments are underway in this direction.

Todo

Give a concrete example, typically using `ElementWrapper`.

Category

A *category* is a Python instance modelling a mathematical category.

Examples of categories include the category of finite semigroups, the category of all (Python) objects, the category of **Z**-algebras, and the category of Cartesian products of **Z**-algebras:

```
sage: FiniteSemigroups()
Category of finite semigroups
sage: Objects()
Category of objects
sage: Algebras(ZZ)
Category of algebras over Integer Ring
sage: Algebras(ZZ).CartesianProducts()
Category of Cartesian products of algebras over Integer Ring
```

```
>>> from sage.all import *
>>> FiniteSemigroups()
Category of finite semigroups
>>> Objects()
Category of objects
>>> Algebras(ZZ)
Category of algebras over Integer Ring
>>> Algebras(ZZ).CartesianProducts()
Category of Cartesian products of algebras over Integer Ring
```

Mind the 's' in the names of the categories above; `GroupAlgebra` and `GroupAlgebras` are distinct things.

Every parent belongs to a collection of categories. Moreover, categories are interrelated by the *super categories* relation. For example, the category of rings is a super category of the category of fields, because every field is also a ring.

A category serves two roles:

- to provide a model for the mathematical concept of a category and the associated structures: homsets, morphisms, functorial constructions, axioms.
- to organize and promote generic code, naming conventions, documentation, and tests across similar mathematical structures.

CategoryObject

Objects of a mathematical category are not necessarily parents. Parent has a superclass that provides a means of modeling such.

For example, the category of schemes does not have a faithful forgetful functor to the category of sets, so it does not make sense to talk about schemes as parents.

Morphisms, Homsets

As category theorists will expect, *Morphisms* and *Homsets* will play an ever more important role, as support for them will improve.

Much of the mathematical information in Sage is encoded as relations between elements and their parents, parents and their categories, and categories and their super categories:

```
sage: 1.parent()
Integer Ring

sage: ZZ
Integer Ring

sage: ZZ.category()
Join of Category of euclidean domains
    and Category of noetherian rings
    and Category of infinite enumerated sets
    and Category of metric spaces

sage: ZZ.categories()
[Join of Category of euclidean domains
     and Category of noetherian rings
     and Category of infinite enumerated sets
     and Category of metric spaces,
 Category of euclidean domains, Category of principal ideal domains,
 Category of Dedekind domains,
 Category of unique factorization domains, Category of gcd domains,
 Category of integral domains, Category of domains, ...
 Category of commutative rings, Category of rings, ...
 Category of magmas and additive magmas, ...
 Category of monoids, Category of semigroups,
 Category of commutative magmas, Category of unital magmas, Category of magmas,
 Category of commutative additive groups, ..., Category of additive magmas,
 Category of infinite enumerated sets, Category of enumerated sets,
 Category of infinite sets, Category of metric spaces,
 Category of topological spaces, Category of sets,
 Category of sets with partial maps,
 Category of objects]

sage: g = EuclideanDomains().category_graph()                                  #␣
↪needs sage.graphs
sage: g.set_latex_options(format='dot2tex')                                    #␣
↪needs sage.graphs sage.plot
sage: view(g)                                 # not tested                     #␣
↪needs sage.graphs sage.plot
```

```
>>> from sage.all import *
>>> Integer(1).parent()
Integer Ring

>>> ZZ
Integer Ring

>>> ZZ.category()
Join of Category of euclidean domains
    and Category of noetherian rings
    and Category of infinite enumerated sets
```

(continues on next page)

(continued from previous page)

```
    and Category of metric spaces

>>> ZZ.categories()
[Join of Category of euclidean domains
     and Category of noetherian rings
     and Category of infinite enumerated sets
     and Category of metric spaces,
 Category of euclidean domains, Category of principal ideal domains,
 Category of Dedekind domains,
 Category of unique factorization domains, Category of gcd domains,
 Category of integral domains, Category of domains, ...
 Category of commutative rings, Category of rings, ...
 Category of magmas and additive magmas, ...
 Category of monoids, Category of semigroups,
 Category of commutative magmas, Category of unital magmas, Category of magmas,
 Category of commutative additive groups, ..., Category of additive magmas,
 Category of infinite enumerated sets, Category of enumerated sets,
 Category of infinite sets, Category of metric spaces,
 Category of topological spaces, Category of sets,
 Category of sets with partial maps,
 Category of objects]

>>> g = EuclideanDomains().category_graph()                                    #␣
↪needs sage.graphs
>>> g.set_latex_options(format='dot2tex')                                      #␣
↪needs sage.graphs sage.plot
>>> view(g)                                  # not tested                     #␣
↪needs sage.graphs sage.plot
```

2.1.4 A bit of help from computer science

Hierarchy of classes

How are the bookshelves implemented in practice?

Sage uses the classical design paradigm of Object Oriented Programming (OOP). Its fundamental principle is that any object that a program is to manipulate should be modelled by an *instance* of a *class*. The class implements:

- a *data structure*: which describes how the object is stored,
- *methods*: which describe the operations on the object.

The instance itself contains the data for the given object, according to the specified data structure.

Hence, all the objects mentioned above should be instances of some classes. For example, an integer in Sage is an instance of the class `Integer` (and it knows about it!):

```
sage: i = 12
sage: type(i)
<class 'sage.rings.integer.Integer'>
```

```
>>> from sage.all import *
>>> i = Integer(12)
>>> type(i)
<class 'sage.rings.integer.Integer'>
```

Applying an operation is generally done by *calling a method*:

```
sage: i.factor()
2^2 * 3

sage: # needs sage.symbolic
sage: x = var('x')
sage: p = 6*x^2 + 12*x + 6
sage: type(p)
<class 'sage.symbolic.expression.Expression'>
sage: p.factor()
6*(x + 1)^2

sage: # needs sage.symbolic
sage: R.<x> = PolynomialRing(QQ, sparse=True)
sage: pQ = R(p)
sage: type(pQ)
<class 'sage.rings.polynomial.polynomial_ring.PolynomialRing_field_with_category.
↪element_class'>
sage: pQ.factor()
(6) * (x + 1)^2

sage: # needs sage.symbolic
sage: pZ = ZZ['x'](p)
sage: type(pZ)
<class 'sage.rings.polynomial.polynomial_integer_dense_flint.Polynomial_integer_dense_
↪flint'>
sage: pZ.factor()
2 * 3 * (x + 1)^2
```

```
>>> from sage.all import *
>>> i.factor()
2^2 * 3

>>> # needs sage.symbolic
>>> x = var('x')
>>> p = Integer(6)*x**Integer(2) + Integer(12)*x + Integer(6)
>>> type(p)
<class 'sage.symbolic.expression.Expression'>
>>> p.factor()
6*(x + 1)^2

>>> # needs sage.symbolic
>>> R = PolynomialRing(QQ, sparse=True, names=('x',)); (x,) = R._first_ngens(1)
>>> pQ = R(p)
>>> type(pQ)
<class 'sage.rings.polynomial.polynomial_ring.PolynomialRing_field_with_category.
↪element_class'>
>>> pQ.factor()
(6) * (x + 1)^2

>>> # needs sage.symbolic
>>> pZ = ZZ['x'](p)
```

(continues on next page)

(continued from previous page)

```
>>> type(pZ)
<class 'sage.rings.polynomial.polynomial_integer_dense_flint.Polynomial_integer_dense_
↪flint'>
>>> pZ.factor()
2 * 3 * (x + 1)^2
```

Factoring integers, expressions, or polynomials are distinct tasks, with completely different algorithms. Yet, from a user (or caller) point of view, all those objects can be manipulated alike. This illustrates the OOP concepts of *polymorphism*, *data abstraction*, and *encapsulation*.

Let us be curious, and see where some methods are defined. This can be done by introspection:

```
sage: i._mul_??                    # not tested
```

```
>>> from sage.all import *
>>> i._mul_??                    # not tested
```

For plain Python methods, one can also just ask in which module they are implemented:

```
sage: i._pow_.__module__  # not tested (Issue #24275)
'sage.categories.semigroups'

sage: pQ._mul_.__module__                                                    #␣
↪needs sage.symbolic
'sage.rings.polynomial.polynomial_element_generic'
sage: pQ._pow_.__module__                    # not tested (Issue #24275)     #␣
↪needs sage.symbolic
'sage.categories.semigroups'
```

```
>>> from sage.all import *
>>> i._pow_.__module__  # not tested (Issue #24275)
'sage.categories.semigroups'

>>> pQ._mul_.__module__                                                      #␣
↪needs sage.symbolic
'sage.rings.polynomial.polynomial_element_generic'
>>> pQ._pow_.__module__                    # not tested (Issue #24275)     #␣
↪needs sage.symbolic
'sage.categories.semigroups'
```

We see that integers and polynomials have each their own multiplication method: the multiplication algorithms are indeed unrelated and deeply tied to their respective datastructures. On the other hand, as we have seen above, they share the same powering method because the set $\mathbf{Z}$ of integers, and the set $\mathbf{Q}[x]$ of polynomials are both semigroups. Namely, the class for integers and the class for polynomials both derive from an *abstract class* for semigroup elements, which factors out the *generic* methods like `_pow_`. This illustrates the use of *hierarchy of classes* to share common code between classes having common behaviour.

OOP design is all about isolating the objects that one wants to model together with their operations, and designing an appropriate hierarchy of classes for organizing the code. As we have seen above, the design of the class hierarchy is easy since it can be modelled upon the hierarchy of categories (bookshelves). Here is for example a piece of the hierarchy of classes for an element of a group of permutations:

```
sage: P = Permutations(4)
sage: m = P.an_element()
sage: for cls in m.__class__.mro(): print(cls)
<class 'sage.combinat.permutation.StandardPermutations_n_with_category.element_class'>
<class 'sage.combinat.permutation.StandardPermutations_n.Element'>
<class 'sage.combinat.permutation.Permutation'>
...
<class 'sage.categories.groups.Groups.element_class'>
<class 'sage.categories.monoids.Monoids.element_class'>
...
<class 'sage.categories.semigroups.Semigroups.element_class'>
...
```

```
>>> from sage.all import *
>>> P = Permutations(Integer(4))
>>> m = P.an_element()
>>> for cls in m.__class__.mro(): print(cls)
<class 'sage.combinat.permutation.StandardPermutations_n_with_category.element_class'>
<class 'sage.combinat.permutation.StandardPermutations_n.Element'>
<class 'sage.combinat.permutation.Permutation'>
...
<class 'sage.categories.groups.Groups.element_class'>
<class 'sage.categories.monoids.Monoids.element_class'>
...
<class 'sage.categories.semigroups.Semigroups.element_class'>
...
```

On the top, we see concrete classes that describe the data structure for matrices and provide the operations that are tied to this data structure. Then follow abstract classes that are attached to the hierarchy of categories and provide generic algorithms.

The full hierarchy is best viewed graphically:

```
sage: g = class_graph(m.__class__)                                              #␣
↪needs sage.combinat sage.graphs
sage: g.set_latex_options(format='dot2tex')                                     #␣
↪needs sage.combinat sage.graphs sage.plot
sage: view(g)                                 # not tested                      #␣
↪needs sage.combinat sage.graphs sage.plot
```

```
>>> from sage.all import *
>>> g = class_graph(m.__class__)                                                #␣
↪needs sage.combinat sage.graphs
>>> g.set_latex_options(format='dot2tex')                                       #␣
↪needs sage.combinat sage.graphs sage.plot
>>> view(g)                                 # not tested                        #␣
↪needs sage.combinat sage.graphs sage.plot
```

Parallel hierarchy of classes for parents

Let us recall that we do not just want to compute with elements of mathematical sets, but with the sets themselves:

```
sage: ZZ.one()
1

sage: R = QQ['x,y']
sage: R.krull_dimension()
2
sage: A = R.quotient( R.ideal(x^2 - 2) )
sage: A.krull_dimension() # todo: not implemented
```

```
>>> from sage.all import *
>>> ZZ.one()
1

>>> R = QQ['x,y']
>>> R.krull_dimension()
2
>>> A = R.quotient( R.ideal(x**Integer(2) - Integer(2)) )
>>> A.krull_dimension() # todo: not implemented
```

Here are some typical operations that one may want to carry on various kinds of sets:

- The set of permutations of 5, the set of rational points of an elliptic curve: counting, listing, random generation
- A language (set of words): rationality testing, counting elements, generating series
- A finite semigroup: left/right ideals, center, representation theory
- A vector space, an algebra: Cartesian product, tensor product, quotient

Hence, following the OOP fundamental principle, parents should also be modelled by instances of some (hierarchy of) classes. For example, our group G is an instance of the following class:

```
sage: G = GL(2, ZZ)                                                             #␣
↪needs sage.modules
sage: type(G)                                                                   #␣
↪needs sage.libs.gap sage.modules
<class 'sage.groups.matrix_gps.linear_gap.LinearMatrixGroup_gap_with_category'>
```

```
>>> from sage.all import *
>>> G = GL(Integer(2), ZZ)                                                       ␣
↪     # needs sage.modules
>>> type(G)                                                                    #␣
↪needs sage.libs.gap sage.modules
<class 'sage.groups.matrix_gps.linear_gap.LinearMatrixGroup_gap_with_category'>
```

Here is a piece of the hierarchy of classes above it:

```
sage: for cls in G.__class__.mro(): print(cls)                                  #␣
↪needs sage.libs.gap sage.modules
<class 'sage.groups.matrix_gps.linear_gap.LinearMatrixGroup_gap_with_category'>
...
<class 'sage.categories.groups.Groups.parent_class'>
```

(continues on next page)

(continued from previous page)

```
<class 'sage.categories.monoids.Monoids.parent_class'>
<class 'sage.categories.semigroups.Semigroups.parent_class'>
...
```

```
>>> from sage.all import *
>>> for cls in G.__class__.mro(): print(cls)                                    #␣
↪needs sage.libs.gap sage.modules
<class 'sage.groups.matrix_gps.linear_gap.LinearMatrixGroup_gap_with_category'>
...
<class 'sage.categories.groups.Groups.parent_class'>
<class 'sage.categories.monoids.Monoids.parent_class'>
<class 'sage.categories.semigroups.Semigroups.parent_class'>
...
```

Note that the hierarchy of abstract classes is again attached to categories and parallel to that we had seen for the elements. This is best viewed graphically:

```
sage: # needs sage.combinat sage.graphs sage.modules sage.plot
sage: g = class_graph(m.__class__)
sage: g.relabel(lambda x: x.replace("_",r"\_"))
sage: g.set_latex_options(format='dot2tex')
sage: view(g)                                 # not tested
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules sage.plot
>>> g = class_graph(m.__class__)
>>> g.relabel(lambda x: x.replace("_",r"\_"))
>>> g.set_latex_options(format='dot2tex')
>>> view(g)                                 # not tested
```

Note

This is a progress upon systems like Axiom or MuPAD where a parent is modelled by the class of its elements; this oversimplification leads to confusion between methods on parents and elements, and makes parents special; in particular it prevents potentially interesting constructions like "groups of groups".

2.1.5 Sage categories

Why this business of categories? And to start with, why don't we just have a good old hierarchy of classes `Group`, `Semigroup`, `Magma`, ... ?

Dynamic hierarchy of classes

As we have just seen, when we manipulate groups, we actually manipulate several kinds of objects:

- groups
- group elements
- morphisms between groups
- and even the category of groups itself!

Thus, on the group bookshelf, we want to put generic code for each of the above. We therefore need three, parallel hierarchies of abstract classes:

- Group, Monoid, Semigroup, Magma, …
- GroupElement, MonoidElement, SemigroupElement, MagmaElement, …
- GroupMorphism, MonoidMorphism, SemigroupMorphism, MagmaMorphism, …

(and in fact many more as we will see).

We could implement the above hierarchies as usual:

```
class Group(Monoid):
    # generic methods that apply to all groups

class GroupElement(MonoidElement):
    # generic methods that apply to all group elements

class GroupMorphism(MonoidMorphism):
    # generic methods that apply to all group morphisms
```

And indeed that's how it was done in Sage before 2009, and there are still many traces of this. The drawback of this approach is duplication: the fact that a group is a monoid is repeated three times above!

Instead, Sage now uses the following syntax, where the *Groups* bookshelf is structured into units with *nested classes*:

```
class Groups(Category):

    def super_categories(self):
        return [Monoids(), ...]

    class ParentMethods:
        # generic methods that apply to all groups

    class ElementMethods:
        # generic methods that apply to all group elements

    class MorphismMethods:
        # generic methods that apply to all group morphisms (not yet implemented)

    class SubcategoryMethods:
        # generic methods that apply to all subcategories of Groups()
```

With this syntax, the information that a group is a monoid is specified only once, in the *Category.super_categories()* method. And indeed, when the category of inverse unital magmas was introduced, there was a *single point of truth* to update in order to reflect the fact that a group is an inverse unital magma:

```
sage: Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
```

```
>>> from sage.all import *
>>> Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
```

The price to pay (there is no free lunch) is that some magic is required to construct the actual hierarchy of classes for parents, elements, and morphisms. Namely, `Groups.ElementMethods` should be seen as just a bag of methods, and

the actual class `Groups().element_class` is constructed from it by adding the appropriate super classes according to `Groups().super_categories()`:

```
sage: Groups().element_class
<class 'sage.categories.groups.Groups.element_class'>

sage: Groups().element_class.__bases__
(<class 'sage.categories.monoids.Monoids.element_class'>,
 <class 'sage.categories.magmas.Magmas.Unital.Inverse.element_class'>)
```

```
>>> from sage.all import *
>>> Groups().element_class
<class 'sage.categories.groups.Groups.element_class'>

>>> Groups().element_class.__bases__
(<class 'sage.categories.monoids.Monoids.element_class'>,
 <class 'sage.categories.magmas.Magmas.Unital.Inverse.element_class'>)
```

We now see that the hierarchy of classes for parents and elements is parallel to the hierarchy of categories:

```
sage: Groups().all_super_categories()
[Category of groups,
 Category of monoids,
 Category of semigroups,
 ...
 Category of magmas,
 Category of sets,
 ...]

sage: for cls in Groups().element_class.mro(): print(cls)
<class 'sage.categories.groups.Groups.element_class'>
<class 'sage.categories.monoids.Monoids.element_class'>
<class 'sage.categories.semigroups.Semigroups.element_class'>
...
<class 'sage.categories.magmas.Magmas.element_class'>
...
sage: for cls in Groups().parent_class.mro(): print(cls)
<class 'sage.categories.groups.Groups.parent_class'>
<class 'sage.categories.monoids.Monoids.parent_class'>
<class 'sage.categories.semigroups.Semigroups.parent_class'>
...
<class 'sage.categories.magmas.Magmas.parent_class'>
...
```

```
>>> from sage.all import *
>>> Groups().all_super_categories()
[Category of groups,
 Category of monoids,
 Category of semigroups,
 ...
 Category of magmas,
 Category of sets,
 ...]
```

(continues on next page)

(continued from previous page)

```
>>> for cls in Groups().element_class.mro(): print(cls)
<class 'sage.categories.groups.Groups.element_class'>
<class 'sage.categories.monoids.Monoids.element_class'>
<class 'sage.categories.semigroups.Semigroups.element_class'>
...
<class 'sage.categories.magmas.Magmas.element_class'>
...
>>> for cls in Groups().parent_class.mro(): print(cls)
<class 'sage.categories.groups.Groups.parent_class'>
<class 'sage.categories.monoids.Monoids.parent_class'>
<class 'sage.categories.semigroups.Semigroups.parent_class'>
...
<class 'sage.categories.magmas.Magmas.parent_class'>
...
```

Another advantage of building the hierarchy of classes dynamically is that, for parametrized categories, the hierarchy may depend on the parameters. For example an algebra over **Q** is a **Q**-vector space, but an algebra over **Z** is not (it is just a **Z**-module)!

Note

At this point this whole infrastructure may feel like overdesigning, right? We felt like this too! But we will see later that, once one gets used to it, this approach scales very naturally.

From a computer science point of view, this infrastructure implements, on top of standard multiple inheritance, a dynamic composition mechanism of mixin classes (Wikipedia article Mixin), governed by mathematical properties.

For implementation details on how the hierarchy of classes for parents and elements is constructed, see *Category*.

On the category hierarchy: subcategories and super categories

We have seen above that, for example, the category of sets is a super category of the category of groups. This models the fact that a group can be unambiguously considered as a set by forgetting its group operation. In object-oriented parlance, we want the relation "a group *is a* set", so that groups can directly inherit code implemented on sets.

Formally, a category `Cs()` is a *super category* of a category `Ds()` if Sage considers any object of `Ds()` to be an object of `Cs()`, up to an implicit application of a canonical functor from `Ds()` to `Cs()`. This functor is normally an inclusion of categories or a forgetful functor. Reciprocally, `Ds()` is said to be a *subcategory* of `Cs()`.

> **Warning**
>
> This terminology deviates from the usual mathematical definition of *subcategory* and is subject to change. Indeed, the forgetful functor from the category of groups to the category of sets is not an inclusion of categories, as it is not injective: a given set may admit more than one group structure. See Issue #16183 for more details. The name *supercategory* is also used with a different meaning in certain areas of mathematics.

Categories are instances and have operations

Note that categories themselves are naturally modelled by instances because they can have operations of their own. An important one is:

```
sage: Groups().example()                                                    #␣
↪needs sage.modules
General Linear Group of degree 4 over Rational Field
```

```
>>> from sage.all import *
>>> Groups().example()                                                      #␣
↪needs sage.modules
General Linear Group of degree 4 over Rational Field
```

which gives an example of object of the category. Besides illustrating the category, the example provides a minimal template for implementing a new object in the category:

```
sage: S = Semigroups().example(); S
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().example(); S
An example of a semigroup: the left zero semigroup
```

Its source code can be obtained by introspection:

```
sage: S??                                       # not tested
```

```
>>> from sage.all import *
>>> S??                                       # not tested
```

This example is also typically used for testing generic methods. See *Category.example()* for more.

Other operations on categories include querying the super categories or the axioms satisfied by the operations of a category:

```
sage: Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
sage: Groups().axioms()
frozenset({'Associative', 'Inverse', 'Unital'})
```

```
>>> from sage.all import *
>>> Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
>>> Groups().axioms()
frozenset({'Associative', 'Inverse', 'Unital'})
```

or constructing the intersection of two categories, or the smallest category containing them:

```
sage: Groups() & FiniteSets()
Category of finite groups
sage: Algebras(QQ) | Groups()
Category of monoids
```

```
>>> from sage.all import *
>>> Groups() & FiniteSets()
Category of finite groups
>>> Algebras(QQ) | Groups()
Category of monoids
```

Specifications and generic documentation

Categories do not only contain code but also the specifications of the operations. In particular a list of mandatory and optional methods to be implemented can be found by introspection with:

```
sage: Groups().required_methods()
{'element': {'optional': ['_mul_'], 'required': []},
 'parent': {'optional': [], 'required': ['__contains__']}}
```

```
>>> from sage.all import *
>>> Groups().required_methods()
{'element': {'optional': ['_mul_'], 'required': []},
 'parent': {'optional': [], 'required': ['__contains__']}}
```

Documentation about those methods can be obtained with:

```
sage: G = Groups()
sage: G.element_class._mul_?         # not tested
sage: G.parent_class.one?            # not tested
```

```
>>> from sage.all import *
>>> G = Groups()
>>> G.element_class._mul_?         # not tested
>>> G.parent_class.one?            # not tested
```

See also the `abstract_method()` decorator.

> **Warning**
>
> Well, more precisely, that's how things should be, but there is still some work to do in this direction. For example, the inverse operation is not specified above. Also, we are still missing a good programmatic syntax to specify the input and output types of the methods. Finally, in many cases the implementer must provide at least one of two methods, each having a default implementation using the other one (e.g. listing or iterating for a finite enumerated set); there is currently no good programmatic way to specify this.

Generic tests

Another feature that parents and elements receive from categories is generic tests; their purpose is to check (at least to some extent) that the parent satisfies the required mathematical properties (is my semigroup indeed associative?) and is implemented according to the specifications (does the method `an_element` indeed return an element of the parent?):

```
sage: S = FiniteSemigroups().example(alphabet=('a', 'b'))
sage: TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('a', 'b'))
>>> TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

Tests can be run individually:

```
sage: S._test_associativity()
```

```
>>> from sage.all import *
>>> S._test_associativity()
```

Here is how to access the code of this test:

```
sage: S._test_associativity?? # not tested
```

```
>>> from sage.all import *
>>> S._test_associativity?? # not tested
```

Here is how to run the test on all elements:

```
sage: L = S.list()
sage: S._test_associativity(elements=L)
```

```
>>> from sage.all import *
>>> L = S.list()
>>> S._test_associativity(elements=L)
```

See `TestSuite` for more information.

Let us see what happens when a test fails. Here we redefine the product of S to something definitely not associative:

```
sage: S.product = lambda x, y: S("("+x.value +y.value+")")
```

```
>>> from sage.all import *
>>> S.product = lambda x, y: S("("+x.value +y.value+")")
```

And rerun the test:

```
sage: S._test_associativity(elements=L)
Traceback (most recent call last):
...
  File ".../sage/categories/semigroups.py", line ..., in _test_associativity
    tester.assertTrue((x * y) * z == x * (y * z))
...
AssertionError: '((aa)a)' != '(a(aa))'
```

```
>>> from sage.all import *
>>> S._test_associativity(elements=L)
Traceback (most recent call last):
...
  File ".../sage/categories/semigroups.py", line ..., in _test_associativity
    tester.assertTrue((x * y) * z == x * (y * z))
...
AssertionError: '((aa)a)' != '(a(aa))'
```

We can recover instantly the actual values of `x`, `y`, `z`, that is, a counterexample to the associativity of our broken semigroup, using post mortem introspection with the Python debugger `pdb` (this does not work yet in the notebook):

```
sage: import pdb
sage: pdb.pm()                          # not tested
> /opt/sage-5.11.rc1/local/lib/python/unittest/case.py(424)assertTrue()
-> raise self.failureException(msg)
(Pdb) u
> /opt/sage-5.11.rc1/local/lib/python2.7/site-packages/sage/categories/semigroups.
↪py(145)_test_associativity()
-> tester.assertTrue((x * y) * z == x * (y * z))
(Pdb) p x, y, z
('a', 'a', 'a')
(Pdb) p (x * y) * z
'((aa)a)'
(Pdb) p x * (y * z)
'(a(aa))'
```

```
>>> from sage.all import *
>>> import pdb
>>> pdb.pm()                          # not tested
> /opt/sage-5.11.rc1/local/lib/python/unittest/case.py(424)assertTrue()
-> raise self.failureException(msg)
(Pdb) u
> /opt/sage-5.11.rc1/local/lib/python2.7/site-packages/sage/categories/semigroups.
↪py(145)_test_associativity()
-> tester.assertTrue((x * y) * z == x * (y * z))
(Pdb) p x, y, z
('a', 'a', 'a')
(Pdb) p (x * y) * z
'((aa)a)'
```

(continues on next page)

(continued from previous page)

```
(Pdb) p x * (y * z)
'(a(aa))'
```

Wrap-up

- Categories provide a natural hierarchy of bookshelves to organize not only code, but also specifications and testing tools.
- Everything about, say, algebras with a distinguished basis is gathered in *AlgebrasWithBasis* or its super categories. This includes properties and algorithms for elements, parents, morphisms, but also, as we will see, for constructions like Cartesian products or quotients.
- The mathematical relations between elements, parents, and categories translate dynamically into a traditional hierarchy of classes.
- This design enforces robustness and consistency, which is particularly welcome given that Python is an interpreted language without static type checking.

2.1.6 Case study

In this section, we study an existing parent in detail; a good followup is to go through the *sage.categories.tutorial* or the thematic tutorial on coercion and categories ("How to implement new algebraic structures in Sage") to learn how to implement a new one!

We consider the example of finite semigroup provided by the category:

```
sage: S = FiniteSemigroups().example(); S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
sage: S?                    # not tested
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(); S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
>>> S?                    # not tested
```

Where do all the operations on `S` and its elements come from?

```
sage: x = S('a')
```

```
>>> from sage.all import *
>>> x = S('a')
```

`_repr_` is a technical method which comes with the data structure (`ElementWrapper`); since it's implemented in Cython, we need to use Sage's introspection tools to recover where it's implemented:

```
sage: x._repr_.__module__
'sage.structure.element_wrapper'
sage: sage.misc.sageinspect.sage_getfile(x._repr_)
'.../sage/structure/element_wrapper.pyx'
```

```
>>> from sage.all import *
>>> x._repr_.__module__
'sage.structure.element_wrapper'
>>> sage.misc.sageinspect.sage_getfile(x._repr_)
'.../sage/structure/element_wrapper.pyx'
```

`_pow_int` is a generic method for all finite semigroups:

```
sage: x._pow_int.__module__
'sage.categories.semigroups'
```

```
>>> from sage.all import *
>>> x._pow_int.__module__
'sage.categories.semigroups'
```

`__mul__` is a generic method provided by the *Magmas* category (a *magma* is a set with an inner law $*$, not necessarily associative). If the two arguments are in the same parent, it will call the method `_mul_`, and otherwise let the coercion model try to discover how to do the multiplication:

```
sage: x.__mul__??                                  # not tested
```

```
>>> from sage.all import *
>>> x.__mul__??                                  # not tested
```

Since it is a speed critical method, it is implemented in Cython in a separate file:

```
sage: x._mul_.__module__
'sage.categories.coercion_methods'
```

```
>>> from sage.all import *
>>> x._mul_.__module__
'sage.categories.coercion_methods'
```

`_mul_` is a default implementation, also provided by the *Magmas* category, that delegates the work to the method `product` of the parent (following the advice: if you do not know what to do, ask your parent); it's also a speed critical method:

```
sage: x._mul_??                                    # not tested
sage: x._mul_.__module__
'sage.categories.coercion_methods'
sage: x._mul_.__func__ is Magmas.ElementMethods._mul_parent
True
```

```
>>> from sage.all import *
>>> x._mul_??                                    # not tested
>>> x._mul_.__module__
'sage.categories.coercion_methods'
>>> x._mul_.__func__ is Magmas.ElementMethods._mul_parent
True
```

`product` is a mathematical method implemented by the parent:

```
sage: S.product.__module__
'sage.categories.examples.finite_semigroups'
```

```
>>> from sage.all import *
>>> S.product.__module__
'sage.categories.examples.finite_semigroups'
```

`cayley_graph` is a generic method on the parent, provided by the *FiniteSemigroups* category:

```
sage: S.cayley_graph.__module__
'sage.categories.semigroups'
```

```
>>> from sage.all import *
>>> S.cayley_graph.__module__
'sage.categories.semigroups'
```

`multiplication_table` is a generic method on the parent, provided by the *Magmas* category (it does not require associativity):

```
sage: S.multiplication_table.__module__
'sage.categories.magmas'
```

```
>>> from sage.all import *
>>> S.multiplication_table.__module__
'sage.categories.magmas'
```

Consider now the implementation of the semigroup:

```
sage: S??                                       # not tested
```

```
>>> from sage.all import *
>>> S??                                         # not tested
```

This implementation specifies a data structure for the parents and the elements, and makes a promise: the implemented parent is a finite semigroup. Then it fulfills the promise by implementing the basic operation `product`. It also implements the optional method `semigroup_generators`. In exchange, S and its elements receive generic implementations of all the other operations. S may override any of those by more efficient ones. It may typically implement the element method `is_idempotent` to always return `True`.

A (not yet complete) list of mandatory and optional methods to be implemented can be found by introspection with:

```
sage: FiniteSemigroups().required_methods()
{'element': {'optional': ['_mul_'], 'required': []},
 'parent': {'optional': ['semigroup_generators'],
  'required': ['__contains__']}}
```

```
>>> from sage.all import *
>>> FiniteSemigroups().required_methods()
{'element': {'optional': ['_mul_'], 'required': []},
 'parent': {'optional': ['semigroup_generators'],
  'required': ['__contains__']}}
```

`product` does not appear in the list because a default implementation is provided in term of the method `_mul_` on elements. Of course, at least one of them should be implemented. On the other hand, a default implementation for `__contains__` is provided by `Parent`.

Documentation about those methods can be obtained with:

```
sage: C = FiniteSemigroups().element_class
sage: C._mul_?                                  # not tested
```

```
>>> from sage.all import *
>>> C = FiniteSemigroups().element_class
>>> C._mul_?                                  # not tested
```

See also the `abstract_method()` decorator.

Here is the code for the finite semigroups category:

```
sage: FiniteSemigroups??                        # not tested
```

```
>>> from sage.all import *
>>> FiniteSemigroups??                        # not tested
```

2.1.7 Specifying the category of a parent

Some parent constructors (not enough!) allow to specify the desired category for the parent. This can typically be used to specify additional properties of the parent that we know to hold a priori. For example, permutation groups are by default in the category of finite permutation groups (no surprise):

```
sage: P = PermutationGroup([[(1,2,3)]]); P                                    #␣
↪needs sage.groups
Permutation Group with generators [(1,2,3)]
sage: P.category()                                                            #␣
↪needs sage.groups
Category of finite enumerated permutation groups
```

```
>>> from sage.all import *
>>> P = PermutationGroup([[(Integer(1),Integer(2),Integer(3))]]); P                 ␣
↪                        # needs sage.groups
Permutation Group with generators [(1,2,3)]
>>> P.category()                                                             #␣
↪needs sage.groups
Category of finite enumerated permutation groups
```

In this case, the group is commutative, so we can specify this:

```
sage: P = PermutationGroup([[(1,2,3)]],                                       #␣
↪needs sage.groups
....:                       category=PermutationGroups().Finite().Commutative()); P
Permutation Group with generators [(1,2,3)]
sage: P.category()                                                            #␣
↪needs sage.groups
Category of finite enumerated commutative permutation groups
```

```
>>> from sage.all import *
>>> P = PermutationGroup([[(Integer(1),Integer(2),Integer(3))]],                    ␣
↪                        # needs sage.groups
...                      category=PermutationGroups().Finite().Commutative()); P
Permutation Group with generators [(1,2,3)]
```

(continues on next page)

(continued from previous page)

```
>>> P.category()                                                              #␣
↪needs sage.groups
Category of finite enumerated commutative permutation groups
```

This feature can even be used, typically in experimental code, to add more structure to existing parents, and in particular to add methods for the parents or the elements, without touching the code base:

```
sage: class Foos(Category):
....:     def super_categories(self):
....:         return [PermutationGroups().Finite().Commutative()]
....:     class ParentMethods:
....:         def foo(self): print("foo")
....:     class ElementMethods:
....:         def bar(self): print("bar")

sage: # needs sage.groups
sage: P = PermutationGroup([[(1,2,3)]], category=Foos())
sage: P.foo()
foo
sage: p = P.an_element()
sage: p.bar()
bar
```

```
>>> from sage.all import *
>>> class Foos(Category):
...     def super_categories(self):
...         return [PermutationGroups().Finite().Commutative()]
...     class ParentMethods:
...         def foo(self): print("foo")
...     class ElementMethods:
...         def bar(self): print("bar")

>>> # needs sage.groups
>>> P = PermutationGroup([[(Integer(1),Integer(2),Integer(3))]], category=Foos())
>>> P.foo()
foo
>>> p = P.an_element()
>>> p.bar()
bar
```

In the long run, it would be thinkable to use this idiom to implement forgetful functors; for example the above group could be constructed as a plain set with:

```
sage: P = PermutationGroup([[(1,2,3)]], category=Sets())     # not implemented, needs␣
↪sage.groups
```

```
>>> from sage.all import *
>>> P = PermutationGroup([[(Integer(1),Integer(2),Integer(3))]], category=Sets())
↪# not implemented, needs sage.groups
```

At this stage though, this is still to be explored for robustness and practicality. For now, most parents that accept a category argument only accept a subcategory of the default one.

2.1.8 Scaling further: functorial constructions, axioms, ...

In this section, we explore more advanced features of categories. Along the way, we illustrate that a large hierarchy of categories is desirable to model complicated mathematics, and that scaling to support such a large hierarchy is the driving motivation for the design of the category infrastructure.

Functorial constructions

Sage has support for a certain number of so-called *covariant functorial constructions* which can be used to construct new parents from existing ones while carrying over as much as possible of their algebraic structure. This includes:

- Cartesian products: See *CartesianProductFunctor*.
- Tensor products: See *tensor*.
- Subquotients / quotients / subobjects / isomorphic objects: See:
  - *Sets().Subquotients*,
  - *Sets().Quotients*,
  - *Sets().Subobjects*,
  - *Sets().IsomorphicObjects*
- Dual objects: See *Modules().DualObjects*.
- Algebras, as in group algebras, monoid algebras, ...: See: *Sets.ParentMethods.algebra()*.

Let for example A and B be two parents, and let us construct the Cartesian product $A \times B \times B$:

```
sage: A = AlgebrasWithBasis(QQ).example();     A.rename('A')                    #␣
↪needs sage.combinat sage.modules
sage: B = HopfAlgebrasWithBasis(QQ).example(); B.rename('B')                    #␣
↪needs sage.groups sage.modules
sage: C = cartesian_product([A, B, B]); C                                       #␣
↪needs sage.combinat sage.groups sage.modules
A (+) B (+) B
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example();     A.rename('A')                    #␣
↪needs sage.combinat sage.modules
>>> B = HopfAlgebrasWithBasis(QQ).example(); B.rename('B')                    #␣
↪needs sage.groups sage.modules
>>> C = cartesian_product([A, B, B]); C                                       #␣
↪needs sage.combinat sage.groups sage.modules
A (+) B (+) B
```

In which category should this new parent be? Since A and B are vector spaces, the result is, as a vector space, the direct sum $A \oplus B \oplus B$, hence the notation. Also, since both A and B are monoids, $A \times B \times B$ is naturally endowed with a monoid structure for pointwise multiplication:

```
sage: C in Monoids()                                                            #␣
↪needs sage.combinat sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> C in Monoids()                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.groups sage.modules
True
```

the unit being the Cartesian product of the units of the operands:

```
sage: C.one()                                                                #␣
↪needs sage.combinat sage.groups sage.modules
B[(0, word: )] + B[(1, ())] + B[(2, ())]
sage: cartesian_product([A.one(), B.one(), B.one()])                         #␣
↪needs sage.combinat sage.groups sage.modules
B[(0, word: )] + B[(1, ())] + B[(2, ())]
```

```
>>> from sage.all import *
>>> C.one()                                                                  #␣
↪needs sage.combinat sage.groups sage.modules
B[(0, word: )] + B[(1, ())] + B[(2, ())]
>>> cartesian_product([A.one(), B.one(), B.one()])                           #␣
↪needs sage.combinat sage.groups sage.modules
B[(0, word: )] + B[(1, ())] + B[(2, ())]
```

The pointwise product can be implemented generically for all magmas (i.e. sets endowed with a multiplicative operation) that are constructed as Cartesian products. It's thus implemented in the *Magmas* category:

```
sage: C.product.__module__                                                   #␣
↪needs sage.combinat sage.groups sage.modules
'sage.categories.magmas'
```

```
>>> from sage.all import *
>>> C.product.__module__                                                     #␣
↪needs sage.combinat sage.groups sage.modules
'sage.categories.magmas'
```

More specifically, keeping on using nested classes to structure the code, the product method is put in the nested class *Magmas.CartesianProducts.ParentMethods*:

```
class Magmas(Category):
    class ParentMethods:
        # methods for magmas
    class ElementMethods:
        # methods for elements of magmas
    class CartesianProduct(CartesianProductCategory):
        class ParentMethods:
            # methods for magmas that are constructed as Cartesian products
            def product(self, x, y):
                # ...
        class ElementMethods:
            # ...
```

Note

The support for nested classes in Python is relatively recent. Their intensive use for the category infrastructure did

reveal some glitches in their implementation, in particular around class naming and introspection. Sage currently works around the more annoying ones but some remain visible. See e.g. `sage.misc.test_nested_class`.

Let us now look at the categories of `C`:

```
sage: C.categories()                                                          #␣
↪needs sage.combinat sage.groups sage.modules
[Category of finite dimensional Cartesian products of algebras with basis over␣
↪Rational Field, ...
 Category of Cartesian products of algebras over Rational Field, ...
 Category of Cartesian products of semigroups, Category of semigroups, ...
 Category of Cartesian products of magmas, ..., Category of magmas, ...
 Category of Cartesian products of additive magmas, ..., Category of additive magmas,
 Category of Cartesian products of sets, Category of sets, ...]
```

```
>>> from sage.all import *
>>> C.categories()                                                            #␣
↪needs sage.combinat sage.groups sage.modules
[Category of finite dimensional Cartesian products of algebras with basis over␣
↪Rational Field, ...
 Category of Cartesian products of algebras over Rational Field, ...
 Category of Cartesian products of semigroups, Category of semigroups, ...
 Category of Cartesian products of magmas, ..., Category of magmas, ...
 Category of Cartesian products of additive magmas, ..., Category of additive magmas,
 Category of Cartesian products of sets, Category of sets, ...]
```

This reveals the parallel hierarchy of categories for Cartesian products of semigroups magmas, … We are thus glad that Sage uses its knowledge that a monoid is a semigroup to automatically deduce that a Cartesian product of monoids is a Cartesian product of semigroups, and build the hierarchy of classes for parents and elements accordingly.

In general, the Cartesian product of A and B can potentially be an algebra, a coalgebra, a differential module, and be finite dimensional, or graded, or …. This can only be decided at runtime, by introspection into the properties of A and B; furthermore, the number of possible combinations (e.g. finite dimensional differential algebra) grows exponentially with the number of properties.

Axioms

First examples

We have seen that Sage is aware of the axioms satisfied by, for example, groups:

```
sage: Groups().axioms()
frozenset({'Associative', 'Inverse', 'Unital'})
```

```
>>> from sage.all import *
>>> Groups().axioms()
frozenset({'Associative', 'Inverse', 'Unital'})
```

In fact, the category of groups can be *defined* by stating that a group is a magma, that is a set endowed with an internal binary multiplication, which satisfies the above axioms. Accordingly, we can construct the category of groups from the category of magmas:

```
sage: Magmas().Associative().Unital().Inverse()
Category of groups
```

```
>>> from sage.all import *
>>> Magmas().Associative().Unital().Inverse()
Category of groups
```

In general, we can construct new categories in Sage by specifying the axioms that are satisfied by the operations of the super categories. For example, starting from the category of magmas, we can construct all the following categories just by specifying the axioms satisfied by the multiplication:

```
sage: Magmas()
Category of magmas
sage: Magmas().Unital()
Category of unital magmas
```

```
>>> from sage.all import *
>>> Magmas()
Category of magmas
>>> Magmas().Unital()
Category of unital magmas
```

```
sage: Magmas().Commutative().Unital()
Category of commutative unital magmas
sage: Magmas().Unital().Commutative()
Category of commutative unital magmas
```

```
>>> from sage.all import *
>>> Magmas().Commutative().Unital()
Category of commutative unital magmas
>>> Magmas().Unital().Commutative()
Category of commutative unital magmas
```

```
sage: Magmas().Associative()
Category of semigroups
```

```
>>> from sage.all import *
>>> Magmas().Associative()
Category of semigroups
```

```
sage: Magmas().Associative().Unital()
Category of monoids
```

```
>>> from sage.all import *
>>> Magmas().Associative().Unital()
Category of monoids
```

```
sage: Magmas().Associative().Unital().Commutative()
Category of commutative monoids
```

```
>>> from sage.all import *
>>> Magmas().Associative().Unital().Commutative()
Category of commutative monoids
```

```
sage: Magmas().Associative().Unital().Inverse()
Category of groups
```

```
>>> from sage.all import *
>>> Magmas().Associative().Unital().Inverse()
Category of groups
```

Axioms and categories with axioms

Here, `Associative`, `Unital`, `Commutative` are axioms. In general, any category `Cs` in Sage can declare a new axiom `A`. Then, the *category with axiom* `Cs.A()` models the subcategory of the objects of `Cs` satisfying the axiom `A`. Similarly, for any subcategory `Ds` of `Cs`, `Ds.A()` models the subcategory of the objects of `Ds` satisfying the axiom `A`. In most cases, it's a *full subcategory* (see Wikipedia article Subcategory).

For example, the category of sets defines the `Finite` axiom, and this axiom is available in the subcategory of groups:

```
sage: Sets().Finite()
Category of finite sets
sage: Groups().Finite()
Category of finite groups
```

```
>>> from sage.all import *
>>> Sets().Finite()
Category of finite sets
>>> Groups().Finite()
Category of finite groups
```

The meaning of each axiom is described in the documentation of the corresponding method, which can be obtained as usual by introspection:

```
sage: C = Groups()
sage: C.Finite?                 # not tested
```

```
>>> from sage.all import *
>>> C = Groups()
>>> C.Finite?                 # not tested
```

The purpose of categories with axioms is no different from other categories: to provide bookshelves of code, documentation, mathematical knowledge, tests, for their objects. The extra feature is that, when intersecting categories, axioms are automatically combined together:

```
sage: C = Magmas().Associative() & Magmas().Unital().Inverse() & Sets().Finite(); C
Category of finite groups
sage: sorted(C.axioms())
['Associative', 'Finite', 'Inverse', 'Unital']
```

```
>>> from sage.all import *
>>> C = Magmas().Associative() & Magmas().Unital().Inverse() & Sets().Finite(); C
```

(continues on next page)

(continued from previous page)

```
Category of finite groups
>>> sorted(C.axioms())
['Associative', 'Finite', 'Inverse', 'Unital']
```

For a more advanced example, Sage knows that a ring is a set C endowed with a multiplication which distributes over addition, such that $(C, +)$ is a commutative additive group and $(C, *)$ is a monoid:

```
sage: C = (CommutativeAdditiveGroups() & Monoids()).Distributive(); C
Category of rings

sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Unital']
```

```
>>> from sage.all import *
>>> C = (CommutativeAdditiveGroups() & Monoids()).Distributive(); C
Category of rings

>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Unital']
```

The infrastructure allows for specifying further deduction rules, in order to encode mathematical facts like Wedderburn's theorem:

```
sage: DivisionRings() & Sets().Finite()
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> DivisionRings() & Sets().Finite()
Category of finite enumerated fields
```

Note

When an axiom specifies the properties of some operations in Sage, the notations for those operations are tied to this axiom. For example, as we have seen above, we need two distinct axioms for associativity: the axiom "AdditiveAssociative" is about the properties of the addition $+$, whereas the axiom "Associative" is about the properties of the multiplication $*$.

We are touching here an inherent limitation of the current infrastructure. There is indeed no support for providing generic code that is independent of the notations. In particular, the category hierarchy about additive structures (additive monoids, additive groups, …) is completely duplicated by that for multiplicative structures (monoids, groups, …).

As far as we know, none of the existing computer algebra systems has a good solution for this problem. The difficulty is that this is not only about a single notation but a bunch of operators and methods: `+, -, zero, summation, sum, ...` in one case, `*, /, one, product, prod, factor, ...` in the other. Sharing something between the two hierarchies of categories would only be useful if one could write generic code that applies in both cases; for that one needs to somehow automatically substitute the right operations in the right spots in the code. That's kind of what we are doing manually between e.g. *`AdditiveMagmas.ParentMethods.addition_table()`* and *`Magmas.ParentMethods.multiplication_table()`*, but doing this systematically is a different beast from what we have been doing so far with just usual inheritance.

Single entry point and name space usage

A nice feature of the notation `Cs.A()` is that, from a single entry point (say the category *Magmas* as above), one can explore a whole range of related categories, typically with the help of introspection to discover which axioms are available, and without having to import new Python modules. This feature will be used in Issue #15741 to unclutter the global name space from, for example, the many variants of the category of algebras like:

```
sage: FiniteDimensionalAlgebrasWithBasis(QQ)
Category of finite dimensional algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> FiniteDimensionalAlgebrasWithBasis(QQ)
Category of finite dimensional algebras with basis over Rational Field
```

There will of course be a deprecation step, but it's recommended to prefer right away the more flexible notation:

```
sage: Algebras(QQ).WithBasis().FiniteDimensional()
Category of finite dimensional algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Algebras(QQ).WithBasis().FiniteDimensional()
Category of finite dimensional algebras with basis over Rational Field
```

Design discussion

How far should this be pushed? *Fields* should definitely stay, but should *FiniteGroups* or *DivisionRings* be removed from the global namespace? Do we want to further completely deprecate the notation `FiniteGroups()` in favor of `Groups().Finite()`?

On the potential combinatorial explosion of categories with axioms

Even for a very simple category like `Magmas`, there are about 2^5 potential combinations of the axioms! Think about what this becomes for a category with two operations $+$ and $*$:

```
sage: C = (Magmas() & AdditiveMagmas()).Distributive(); C
Category of distributive magmas and additive magmas

sage: CAA = C.Associative().AdditiveAssociative()
sage: CAA.AdditiveCommutative().AdditiveUnital().AdditiveInverse()
Category of rngs

sage: CAA.AdditiveCommutative().AdditiveUnital().Unital()
Category of semirings

sage: CAA.AdditiveCommutative().AdditiveUnital().AdditiveInverse().Unital()
Category of rings

sage: Rings().Division()
Category of division rings

sage: Rings().Division().Commutative()
Category of fields
```

(continues on next page)

(continued from previous page)

```
sage: Rings().Division().Finite()
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> C = (Magmas() & AdditiveMagmas()).Distributive(); C
Category of distributive magmas and additive magmas

>>> CAA = C.Associative().AdditiveAssociative()
>>> CAA.AdditiveCommutative().AdditiveUnital().AdditiveInverse()
Category of rngs

>>> CAA.AdditiveCommutative().AdditiveUnital().Unital()
Category of semirings

>>> CAA.AdditiveCommutative().AdditiveUnital().AdditiveInverse().Unital()
Category of rings

>>> Rings().Division()
Category of division rings

>>> Rings().Division().Commutative()
Category of fields

>>> Rings().Division().Finite()
Category of finite enumerated fields
```

or for more advanced categories:

```
sage: g = HopfAlgebras(QQ).WithBasis().Graded().Connected().category_graph()          #␣
↪needs sage.graphs
sage: g.set_latex_options(format='dot2tex')                                           #␣
↪needs sage.graphs sage.plot
sage: view(g)                                  # not tested                           #␣
↪needs sage.graphs sage.plot
```

```
>>> from sage.all import *
>>> g = HopfAlgebras(QQ).WithBasis().Graded().Connected().category_graph()          #␣
↪needs sage.graphs
>>> g.set_latex_options(format='dot2tex')                                           #␣
↪needs sage.graphs sage.plot
>>> view(g)                                  # not tested                           #␣
↪needs sage.graphs sage.plot
```

Difference between axioms and regressive covariant functorial constructions

Our running examples here will be the axiom `FiniteDimensional` and the regressive covariant functorial construction `Graded`. Let `Cs` be some subcategory of `Modules`, say the category of modules itself:

```
sage: Cs = Modules(QQ)
```

```
>>> from sage.all import *
>>> Cs = Modules(QQ)
```

Then, `Cs.FiniteDimensional()` (respectively `Cs.Graded()`) is the subcategory of the objects `O` of `Cs` which are finite dimensional (respectively graded).

Let also `Ds` be a subcategory of `Cs`, say:

```
sage: Ds = Algebras(QQ)
```

```
>>> from sage.all import *
>>> Ds = Algebras(QQ)
```

A finite dimensional algebra is also a finite dimensional module:

```
sage: Algebras(QQ).FiniteDimensional().is_subcategory(Modules(QQ).FiniteDimensional())
True
```

```
>>> from sage.all import *
>>> Algebras(QQ).FiniteDimensional().is_subcategory(Modules(QQ).FiniteDimensional())
True
```

Similarly a graded algebra is also a graded module:

```
sage: Algebras(QQ).Graded().is_subcategory( Modules(QQ).Graded() )
True
```

```
>>> from sage.all import *
>>> Algebras(QQ).Graded().is_subcategory( Modules(QQ).Graded() )
True
```

This is the *covariance* property: for `A` an axiom or a covariant functorial construction, if `Ds` is a subcategory of `Cs`, then `Ds.A()` is a subcategory of `Cs.A()`.

What happens if we consider reciprocally an object of `Cs.A()` which is also in `Ds`? A finite dimensional module which is also an algebra is a finite dimensional algebra:

```
sage: Modules(QQ).FiniteDimensional() & Algebras(QQ)
Category of finite dimensional algebras over Rational Field
```

```
>>> from sage.all import *
>>> Modules(QQ).FiniteDimensional() & Algebras(QQ)
Category of finite dimensional algebras over Rational Field
```

On the other hand, a graded module O which is also an algebra is not necessarily a graded algebra! Indeed, the grading on O may not be compatible with the product on O:

```
sage: Modules(QQ).Graded() & Algebras(QQ)
Join of Category of algebras over Rational Field
 and Category of graded vector spaces over Rational Field
```

```
>>> from sage.all import *
>>> Modules(QQ).Graded() & Algebras(QQ)
```

(continues on next page)

(continued from previous page)

```
Join of Category of algebras over Rational Field
 and Category of graded vector spaces over Rational Field
```

The relevant difference between `FiniteDimensional` and `Graded` is that `FiniteDimensional` is a statement about the properties of `O` seen as a module (and thus does not depend on the given category), whereas `Graded` is a statement about the properties of `O` and all its operations in the given category.

In general, if a category satisfies a given axiom, any subcategory also satisfies that axiom. Another formulation is that, for an axiom `A` defined in a super category `Cs` of `Ds`, `Ds.A()` is the intersection of the categories `Ds` and `Cs.A()`:

```
sage: As = Algebras(QQ).FiniteDimensional(); As
Category of finite dimensional algebras over Rational Field
sage: Bs = Algebras(QQ) & Modules(QQ).FiniteDimensional(); As
Category of finite dimensional algebras over Rational Field
sage: As is Bs
True
```

```
>>> from sage.all import *
>>> As = Algebras(QQ).FiniteDimensional(); As
Category of finite dimensional algebras over Rational Field
>>> Bs = Algebras(QQ) & Modules(QQ).FiniteDimensional(); As
Category of finite dimensional algebras over Rational Field
>>> As is Bs
True
```

An immediate consequence is that, as we have already noticed, axioms commute:

```
sage: As = Algebras(QQ).FiniteDimensional().WithBasis(); As
Category of finite dimensional algebras with basis over Rational Field
sage: Bs = Algebras(QQ).WithBasis().FiniteDimensional(); Bs
Category of finite dimensional algebras with basis over Rational Field
sage: As is Bs
True
```

```
>>> from sage.all import *
>>> As = Algebras(QQ).FiniteDimensional().WithBasis(); As
Category of finite dimensional algebras with basis over Rational Field
>>> Bs = Algebras(QQ).WithBasis().FiniteDimensional(); Bs
Category of finite dimensional algebras with basis over Rational Field
>>> As is Bs
True
```

On the other hand, axioms do not necessarily commute with functorial constructions, even if the current printout may missuggest so:

```
sage: As = Algebras(QQ).Graded().WithBasis(); As
Category of graded algebras with basis over Rational Field
sage: Bs = Algebras(QQ).WithBasis().Graded(); Bs
Category of graded algebras with basis over Rational Field
sage: As is Bs
False
```

```
>>> from sage.all import *
>>> As = Algebras(QQ).Graded().WithBasis(); As
Category of graded algebras with basis over Rational Field
>>> Bs = Algebras(QQ).WithBasis().Graded(); Bs
Category of graded algebras with basis over Rational Field
>>> As is Bs
False
```

This is because `Bs` is the category of algebras endowed with basis, which are further graded; in particular the basis must respect the grading (i.e. be made of homogeneous elements). On the other hand, `As` is the category of graded algebras, which are further endowed with some basis; that basis need not respect the grading. In fact `As` is really a join category:

```
sage: type(As)
<class 'sage.categories.category.JoinCategory_with_category'>
sage: As._repr_(as_join=True)
'Join of Category of algebras with basis over Rational Field and Category of graded␣
↪algebras over Rational Field'
```

```
>>> from sage.all import *
>>> type(As)
<class 'sage.categories.category.JoinCategory_with_category'>
>>> As._repr_(as_join=True)
'Join of Category of algebras with basis over Rational Field and Category of graded␣
↪algebras over Rational Field'
```

Todo

Improve the printing of functorial constructions and joins to raise this potentially dangerous ambiguity.

Further reading on axioms

We refer to *sage.categories.category_with_axiom* for how to implement axioms.

Wrap-up

As we have seen, there is a combinatorial explosion of possible classes. Constructing by hand the full class hierarchy would not scale unless one would restrict to a very rigid subset. Even if it was possible to construct automatically the full hierarchy, this would not scale with respect to system resources.

When designing software systems with large hierarchies of abstract classes for business objects, the difficulty is usually to identify a proper set of key concepts. Here we are lucky, as the key concepts have been long identified and are relatively few:

- Operations ($+$, $*$, …)
- Axioms on those operations (associativity, …)
- Constructions (Cartesian products, …)

Better, those concepts are sufficiently well known so that a user can reasonably be expected to be familiar with the concepts that are involved for his own needs.

Instead, the difficulty is concentrated in the huge number of possible combinations, an unpredictable large subset of which being potentially of interest; at the same time, only a small – but moving – subset has code naturally attached to it.

This has led to the current design, where one focuses on writing the relatively few classes for which there is actual code or mathematical information, and lets Sage *compose dynamically and lazily* those building blocks to construct the minimal hierarchy of classes needed for the computation at hand. This allows for the infrastructure to scale smoothly as bookshelves are added, extended, or reorganized.

2.1.9 Writing a new category

Each category C **must** be provided with a method `C.super_categories()` and *can* be provided with a method `C._subcategory_hook_(D)`. Also, it may be needed to insert C into the output of the `super_categories()` method of some other category. This determines the position of C in the category graph.

A category *may* provide methods that can be used by all its objects, respectively by all elements of its objects.

Each category *should* come with a good example in the `sage.categories.examples` package.

Inserting the new category into the category graph

`C.super_categories()` *must* return a list of categories, namely the *immediate* super categories of C. Of course, if you know that your new category C is an immediate super category of some existing category D, then you should also update the method `D.super_categories` to include C.

The immediate super categories of C *should not* be *join categories*. Furthermore, one always should have:

```
Cs().is_subcategory( Category.join(Cs().super_categories()) )

Cs()._cmp_key  >  other._cmp_key  for other in Cs().super_categories()
```

This is checked by *_test_category()*.

In several cases, the category C is directly provided with a generic implementation of `super_categories`; a typical example is when C implements an axiom or a functorial construction; in such a case, C may implement `C.extra_super_categories()` to complement the super categories discovered by the generic implementation. This method needs not return immediate super categories; instead it's usually best to specify the largest super category providing the desired mathematical information. For example, the category *Magmas.Commutative.Algebras* just states that the algebra of a commutative magma is a commutative magma. This is sufficient to let Sage deduce that it's in fact a commutative algebra.

Methods for objects and elements

Different objects of the same category share some algebraic features, and very often these features can be encoded in a method, in a generic way. For example, for every commutative additive monoid, it makes sense to ask for the sum of a list of elements. Sage's category framework allows to provide a generic implementation for all objects of a category.

If you want to provide your new category with generic methods for objects (or elements of objects), then you simply add a nested class called `ParentMethods` (or `ElementMethods`). The methods of that class will automatically become methods of the objects (or the elements). For instance:

```
sage: P.<x,y> = ZZ[]
sage: P.prod([x,y,2])
2*x*y
sage: P.prod.__module__
'sage.categories.monoids'
sage: P.prod.__func__ is raw_getattr(Monoids().ParentMethods, "prod")
True
```

```
>>> from sage.all import *
>>> P = ZZ['x, y']; (x, y,) = P._first_ngens(2)
>>> P.prod([x,y,Integer(2)])
2*x*y
>>> P.prod.__module__
'sage.categories.monoids'
>>> P.prod.__func__ is raw_getattr(Monoids().ParentMethods, "prod")
True
```

We recommend to study the code of one example:

```
sage: C = CommutativeAdditiveMonoids()
sage: C??                                    # not tested
```

```
>>> from sage.all import *
>>> C = CommutativeAdditiveMonoids()
>>> C??                                    # not tested
```

On the order of super categories

The generic method `C.all_super_categories()` determines recursively the list of *all* super categories of C.

The order of the categories in this list does influence the inheritance of methods for parents and elements. Namely, if P is an object in the category C and if C_1 and C_2 are both super categories of C defining some method `foo` in `ParentMethods`, then P will use C_1's version of `foo` if and only if C_1 appears in `C.all_super_categories()` before C_2.

However this must be considered as an *implementation detail*: if C_1 and C_2 are incomparable categories, then the order in which they appear must be mathematically irrelevant: in particular, the methods `foo` in C_1 and C_2 must have the same semantic. Code should not rely on any specific order, as it is subject to later change. Whenever one of the implementations is preferred in some common subcategory of C_1 and C_2, for example for efficiency reasons, the ambiguity should be resolved explicitly by defining a method `foo` in this category. See the method `some_elements` in the code of the category *`FiniteCoxeterGroups`* for an example.

Since Issue #11943, `C.all_super_categories()` is computed by the so-called `C3` algorithm used by Python to compute Method Resolution Order of new-style classes. Thus the order in `C.all_super_categories()`, `C.parent_class.mro()` and `C.element_class.mro()` are guaranteed to be consistent.

Since Issue #13589, the `C3` algorithm is put under control of some total order on categories. This order is not necessarily meaningful, but it guarantees that `C3` always finds a consistent Method Resolution Order. For background, see `sage.misc.c3_controlled`. A visible effect is that the order in which categories are specified in `C.super_categories()`, or in a join category, no longer influences the result of `C.all_super_categories()`.

Subcategory hook (advanced optimization feature)

The default implementation of the method `C.is_subcategory(D)` is to look up whether D appears in `C.all_super_categories()`. However, building the list of all the super categories of C is an expensive operation that is sometimes best avoided. For example, if both C and D are categories defined over a base, but the bases differ, then one knows right away that they can not be subcategories of each other.

When such a short-path is known, one can implement a method `_subcategory_hook_`. Then, `C.is_subcategory(D)` first calls `D._subcategory_hook_(C)`. If this returns `Unknown`, then `C.is_subcategory(D)` tries to find `D` in `C.all_super_categories()`. Otherwise, `C.is_subcategory(D)` returns the result of `D._subcategory_hook_(C)`.

By default, `D._subcategory_hook_(C)` tests whether `issubclass(C.parent_class,D.parent_class)`, which is very often giving the right answer:

```
sage: Rings()._subcategory_hook_(Algebras(QQ))
True
sage: HopfAlgebras(QQ)._subcategory_hook_(Algebras(QQ))
False
sage: Algebras(QQ)._subcategory_hook_(HopfAlgebras(QQ))
True
```

```
>>> from sage.all import *
>>> Rings()._subcategory_hook_(Algebras(QQ))
True
>>> HopfAlgebras(QQ)._subcategory_hook_(Algebras(QQ))
False
>>> Algebras(QQ)._subcategory_hook_(HopfAlgebras(QQ))
True
```

2.2 A subset of sage.categories.all with just the basic categories needed

for sage startup (i.e. to define ZZ, QQ, …).

2.3 Categories

AUTHORS:

- David Kohel, William Stein and Nicolas M. Thiery

Every Sage object lies in a category. Categories in Sage are modeled on the mathematical idea of category, and are distinct from Python classes, which are a programming construct.

In most cases, typing `x.category()` returns the category to which `x` belongs. If `C` is a category and `x` is any object, `C(x)` tries to make an object in `C` from `x`. Checking if `x` belongs to `C` is done as usually by `x in C`.

See *Category* and *sage.categories.primer* for more details.

EXAMPLES:

We create a couple of categories:

```
sage: Sets()
Category of sets
sage: GSets(AbelianGroup([2, 4, 9]))                                    #␣
↪needs sage.groups
Category of G-sets for Multiplicative Abelian group isomorphic to C2 x C4 x C9
sage: Semigroups()
Category of semigroups
sage: VectorSpaces(FiniteField(11))
Category of vector spaces over Finite Field of size 11
sage: Ideals(IntegerRing())
Category of ring ideals in Integer Ring
```

```
>>> from sage.all import *
>>> Sets()
Category of sets
>>> GSets(AbelianGroup([Integer(2), Integer(4), Integer(9)]))                    ␣
↪                        # needs sage.groups
Category of G-sets for Multiplicative Abelian group isomorphic to C2 x C4 x C9
>>> Semigroups()
Category of semigroups
>>> VectorSpaces(FiniteField(Integer(11)))
Category of vector spaces over Finite Field of size 11
>>> Ideals(IntegerRing())
Category of ring ideals in Integer Ring
```

Let's request the category of some objects:

```
sage: V = VectorSpace(RationalField(), 3)                                      #␣
↪needs sage.modules
sage: V.category()                                                             #␣
↪needs sage.modules
Category of finite dimensional vector spaces with basis
 over (number fields and quotient fields and metric spaces)

sage: G = SymmetricGroup(9)                                                    #␣
↪needs sage.groups
sage: G.category()                                                             #␣
↪needs sage.groups
Join of
 Category of finite enumerated permutation groups and
 Category of finite Weyl groups and
 Category of well generated finite irreducible complex reflection groups

sage: P = PerfectMatchings(3)                                                  #␣
↪needs sage.combinat
sage: P.category()                                                             #␣
↪needs sage.combinat
Category of finite enumerated sets
```

```
>>> from sage.all import *
>>> V = VectorSpace(RationalField(), Integer(3))                               ␣
↪     # needs sage.modules
>>> V.category()                                                              #␣
↪needs sage.modules
Category of finite dimensional vector spaces with basis
 over (number fields and quotient fields and metric spaces)

>>> G = SymmetricGroup(Integer(9))                                             ␣
↪     # needs sage.groups
>>> G.category()                                                              #␣
↪needs sage.groups
Join of
 Category of finite enumerated permutation groups and
 Category of finite Weyl groups and
 Category of well generated finite irreducible complex reflection groups
```

(continues on next page)

(continued from previous page)

```
>>> P = PerfectMatchings(Integer(3))                                    ␣
↪     # needs sage.combinat
>>> P.category()                                                       #␣
↪needs sage.combinat
Category of finite enumerated sets
```

Let's check some memberships:

```
sage: V in VectorSpaces(QQ)                                            #␣
↪needs sage.modules
True
sage: V in VectorSpaces(FiniteField(11))                               #␣
↪needs sage.modules
False
sage: G in Monoids()                                                   #␣
↪needs sage.groups
True
sage: P in Rings()                                                     #␣
↪needs sage.combinat
False
```

```
>>> from sage.all import *
>>> V in VectorSpaces(QQ)                                              #␣
↪needs sage.modules
True
>>> V in VectorSpaces(FiniteField(Integer(11)))                        ␣
↪     # needs sage.modules
False
>>> G in Monoids()                                                     #␣
↪needs sage.groups
True
>>> P in Rings()                                                       #␣
↪needs sage.combinat
False
```

For parametrized categories one can use the following shorthand:

```
sage: V in VectorSpaces                                                #␣
↪needs sage.modules
True
sage: G in VectorSpaces                                                #␣
↪needs sage.groups
False
```

```
>>> from sage.all import *
>>> V in VectorSpaces                                                  #␣
↪needs sage.modules
True
>>> G in VectorSpaces                                                  #␣
↪needs sage.groups
False
```

A parent `P` is in a category `C` if `P.category()` is a subcategory of `C`.

Note

Any object of a category should be an instance of `CategoryObject`.

For backward compatibility this is not yet enforced:

```
sage: class A:
....:   def category(self):
....:       return Fields()
sage: A() in Rings()
True
```

```
>>> from sage.all import *
>>> class A:
...   def category(self):
...       return Fields()
>>> A() in Rings()
True
```

By default, the category of an element x of a parent P is the category of all objects of P (this is dubious and may be deprecated):

```
sage: V = VectorSpace(RationalField(), 3)                                   #␣
↪needs sage.modules
sage: v = V.gen(1)                                                          #␣
↪needs sage.modules
sage: v.category()                                                          #␣
↪needs sage.modules
Category of elements of Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> V = VectorSpace(RationalField(), Integer(3))                                ␣
↪    # needs sage.modules
>>> v = V.gen(Integer(1))                                                       ␣
↪    # needs sage.modules
>>> v.category()                                                             #␣
↪needs sage.modules
Category of elements of Vector space of dimension 3 over Rational Field
```

`Category.`**`_test_category_graph`**`(**options)`

Check that the category graph matches with Python's method resolution order.

Note

By Issue #11943, the list of categories returned by `all_super_categories()` is supposed to match with the method resolution order of the parent and element classes. This method checks this.

Note that if `_refine_category_` is called at unexpected times, the invariant might be false. Most commonly, this happens with rings like `Zmod(n)` or `SR`, where a check like `Zmod(n) in Fields()` is needed (which checks the primality of n) to refine their category to be a subcategory of fields.

See also

`CategoryWithParameters._make_named_class_key`

EXAMPLES:

```
sage: C = HopfAlgebrasWithBasis(QQ)
sage: C.parent_class.mro() == [X.parent_class for X in C._all_super_categories] +␣
↪[object]
True
sage: C.element_class.mro() == [X.element_class for X in C._all_super_categories]␣
↪+ [object]
True
sage: TestSuite(C).run()    # indirect doctest
```

```
>>> from sage.all import *
>>> C = HopfAlgebrasWithBasis(QQ)
>>> C.parent_class.mro() == [X.parent_class for X in C._all_super_categories] +␣
↪[object]
True
>>> C.element_class.mro() == [X.element_class for X in C._all_super_categories] +␣
↪[object]
True
>>> TestSuite(C).run()    # indirect doctest
```

`Category.__and__(other)`

Return the intersection of two categories.

This is just a shortcut for *join()*.

EXAMPLES:

```
sage: Sets().Finite() & Rings().Commutative()
Category of finite commutative rings
sage: Monoids() & CommutativeAdditiveMonoids()
Join of Category of monoids and Category of commutative additive monoids
```

```
>>> from sage.all import *
>>> Sets().Finite() & Rings().Commutative()
Category of finite commutative rings
>>> Monoids() & CommutativeAdditiveMonoids()
Join of Category of monoids and Category of commutative additive monoids
```

`Category.__or__(other)`

Return the smallest category containing the two categories.

This is just a shortcut for *meet()*.

EXAMPLES:

```
sage: Algebras(QQ) | Groups()
Category of monoids
```

```
>>> from sage.all import *
>>> Algebras(QQ) | Groups()
Category of monoids
```

class sage.categories.category.**Category**

Bases: UniqueRepresentation, SageObject

The base class for modeling mathematical categories, like for example:

- `Groups()` – the category of groups
- `EuclideanDomains()` – the category of euclidean rings
- `VectorSpaces(QQ)` – the category of vector spaces over the field of rationals

See *sage.categories.primer* for an introduction to categories in Sage, their relevance, purpose, and usage. The documentation below will focus on their implementation.

Technically, a category is an instance of the class *Category* or some of its subclasses. Some categories, like *VectorSpaces*, are parametrized: `VectorSpaces(QQ)` is one of many instances of the class *VectorSpaces*. On the other hand, `EuclideanDomains()` is the single instance of the class *EuclideanDomains*.

Recall that an algebraic structure (say, the ring $\mathbf{Q}[x]$) is modelled in Sage by an object which is called a parent. This object belongs to certain categories (here `EuclideanDomains()` and `Algebras()`). The elements of the ring are themselves objects.

The class of a category (say *EuclideanDomains*) can define simultaneously:

- Operations on the category itself (what is its super categories? its category of morphisms? its dual category?).
- Generic operations on parents in this category, like the ring $\mathbf{Q}[x]$.
- Generic operations on elements of such parents (e. g., the Euclidean algorithm for computing gcds).
- Generic operations on morphisms of this category.

This is achieved as follows:

```
sage: from sage.categories.category import Category
sage: class EuclideanDomains(Category):
....:     # operations on the category itself
....:     def super_categories(self):
....:         [Rings()]
....:
....:     def dummy(self): # TODO: find some good examples
....:          pass
....:
....:     class ParentMethods: # holds the generic operations on parents
....:          # TODO: find a good example of an operation
....:          pass
....:
....:     class ElementMethods:# holds the generic operations on elements
....:          def gcd(x, y):
....:              # Euclid algorithms
....:              pass
....:
....:     class MorphismMethods: # holds the generic operations on morphisms
....:          # TODO: find a good example of an operation
```

(continues on next page)

(continued from previous page)

```
....:         pass
....:
```

```
>>> from sage.all import *
>>> from sage.categories.category import Category
>>> class EuclideanDomains(Category):
...     # operations on the category itself
...     def super_categories(self):
...         [Rings()]
....:
>>>     def dummy(self): # TODO: find some good examples
...         pass
....:
>>>     class ParentMethods: # holds the generic operations on parents
...         # TODO: find a good example of an operation
...         pass
....:
>>>     class ElementMethods:# holds the generic operations on elements
...         def gcd(x, y):
...             # Euclid algorithms
...             pass
....:
>>>     class MorphismMethods: # holds the generic operations on morphisms
...         # TODO: find a good example of an operation
...         pass
....:
```

Note that the nested class `ParentMethods` is merely a container of operations, and does not inherit from anything. Instead, the hierarchy relation is defined once at the level of the categories, and the actual hierarchy of classes is built in parallel from all the `ParentMethods` nested classes, and stored in the attributes `parent_class`. Then, a parent in a category `C` receives the appropriate operations from all the super categories by usual class inheritance from `C.parent_class`.

Similarly, two other hierarchies of classes, for elements and morphisms respectively, are built from all the `ElementMethods` and `MorphismMethods` nested classes.

EXAMPLES:

We define a hierarchy of four categories `As()`, `Bs()`, `Cs()`, `Ds()` with a diamond inheritance. Think for example:

- `As()` – the category of sets
- `Bs()` – the category of additive groups
- `Cs()` – the category of multiplicative monoids
- `Ds()` – the category of rings

```
sage: from sage.categories.category import Category
sage: from sage.misc.lazy_attribute import lazy_attribute
sage: class As (Category):
....:     def super_categories(self):
....:         return []
....:
....:     class ParentMethods:
```

(continues on next page)

(continued from previous page)

```
....:         def fA(self):
....:             return "A"
....:         f = fA

sage: class Bs (Category):
....:     def super_categories(self):
....:         return [As()]
....:
....:     class ParentMethods:
....:         def fB(self):
....:             return "B"

sage: class Cs (Category):
....:     def super_categories(self):
....:         return [As()]
....:
....:     class ParentMethods:
....:         def fC(self):
....:             return "C"
....:         f = fC

sage: class Ds (Category):
....:     def super_categories(self):
....:         return [Bs(),Cs()]
....:
....:     class ParentMethods:
....:         def fD(self):
....:             return "D"
```

```
>>> from sage.all import *
>>> from sage.categories.category import Category
>>> from sage.misc.lazy_attribute import lazy_attribute
>>> class As (Category):
...     def super_categories(self):
...         return []
....:
>>>     class ParentMethods:
...         def fA(self):
...             return "A"
...         f = fA

>>> class Bs (Category):
...     def super_categories(self):
...         return [As()]
....:
>>>     class ParentMethods:
...         def fB(self):
...             return "B"

>>> class Cs (Category):
...     def super_categories(self):
...         return [As()]
```

(continues on next page)

(continued from previous page)

```
....:
>>>     class ParentMethods:
...         def fC(self):
...             return "C"
...         f = fC

>>> class Ds (Category):
...     def super_categories(self):
...         return [Bs(),Cs()]
....:
>>>     class ParentMethods:
...         def fD(self):
...             return "D"
```

Categories should always have unique representation; by Issue #12215, this means that it will be kept in cache, but only if there is still some strong reference to it.

We check this before proceeding:

```
sage: import gc
sage: idAs = id(As())
sage: _ = gc.collect()
sage: n == id(As())
False
sage: a = As()
sage: id(As()) == id(As())
True
sage: As().parent_class == As().parent_class
True
```

```
>>> from sage.all import *
>>> import gc
>>> idAs = id(As())
>>> _ = gc.collect()
>>> n == id(As())
False
>>> a = As()
>>> id(As()) == id(As())
True
>>> As().parent_class == As().parent_class
True
```

We construct a parent in the category `Ds()` (that, is an instance of `Ds().parent_class`), and check that it has access to all the methods provided by all the categories, with the appropriate inheritance order:

```
sage: D = Ds().parent_class()
sage: [ D.fA(), D.fB(), D.fC(), D.fD() ]
['A', 'B', 'C', 'D']
sage: D.f()
'C'
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> D = Ds().parent_class()
>>> [ D.fA(), D.fB(), D.fC(), D.fD() ]
['A', 'B', 'C', 'D']
>>> D.f()
'C'
```

```
sage: C = Cs().parent_class()
sage: [ C.fA(), C.fC() ]
['A', 'C']
sage: C.f()
'C'
```

```
>>> from sage.all import *
>>> C = Cs().parent_class()
>>> [ C.fA(), C.fC() ]
['A', 'C']
>>> C.f()
'C'
```

Here is the parallel hierarchy of classes which has been built automatically, together with the method resolution order (`.mro()`):

```
sage: As().parent_class
<class '__main__.As.parent_class'>
sage: As().parent_class.__bases__
(<... 'object'>,)
sage: As().parent_class.mro()
[<class '__main__.As.parent_class'>, <... 'object'>]
```

```
>>> from sage.all import *
>>> As().parent_class
<class '__main__.As.parent_class'>
>>> As().parent_class.__bases__
(<... 'object'>,)
>>> As().parent_class.mro()
[<class '__main__.As.parent_class'>, <... 'object'>]
```

```
sage: Bs().parent_class
<class '__main__.Bs.parent_class'>
sage: Bs().parent_class.__bases__
(<class '__main__.As.parent_class'>,)
sage: Bs().parent_class.mro()
[<class '__main__.Bs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>]
```

```
>>> from sage.all import *
>>> Bs().parent_class
<class '__main__.Bs.parent_class'>
>>> Bs().parent_class.__bases__
(<class '__main__.As.parent_class'>,)
>>> Bs().parent_class.mro()
```

(continues on next page)

(continued from previous page)

```
[<class '__main__.Bs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>]
```

```
sage: Cs().parent_class
<class '__main__.Cs.parent_class'>
sage: Cs().parent_class.__bases__
(<class '__main__.As.parent_class'>,)
sage: Cs().parent_class.__mro__
(<class '__main__.Cs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>)
```

```
>>> from sage.all import *
>>> Cs().parent_class
<class '__main__.Cs.parent_class'>
>>> Cs().parent_class.__bases__
(<class '__main__.As.parent_class'>,)
>>> Cs().parent_class.__mro__
(<class '__main__.Cs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>)
```

```
sage: Ds().parent_class
<class '__main__.Ds.parent_class'>
sage: Ds().parent_class.__bases__
(<class '__main__.Cs.parent_class'>, <class '__main__.Bs.parent_class'>)
sage: Ds().parent_class.mro()
[<class '__main__.Ds.parent_class'>, <class '__main__.Cs.parent_class'>,
 <class '__main__.Bs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>]
```

```
>>> from sage.all import *
>>> Ds().parent_class
<class '__main__.Ds.parent_class'>
>>> Ds().parent_class.__bases__
(<class '__main__.Cs.parent_class'>, <class '__main__.Bs.parent_class'>)
>>> Ds().parent_class.mro()
[<class '__main__.Ds.parent_class'>, <class '__main__.Cs.parent_class'>,
 <class '__main__.Bs.parent_class'>, <class '__main__.As.parent_class'>, <...
↪'object'>]
```

Note that two categories in the same class need not have the same `super_categories`. For example, `Algebras(QQ)` has `VectorSpaces(QQ)` as super category, whereas `Algebras(ZZ)` only has `Modules(ZZ)` as super category. In particular, the constructed parent class and element class will differ (inheriting, or not, methods specific for vector spaces):

```
sage: Algebras(QQ).parent_class is Algebras(ZZ).parent_class
False
sage: issubclass(Algebras(QQ).parent_class, VectorSpaces(QQ).parent_class)
True
```

```
>>> from sage.all import *
>>> Algebras(QQ).parent_class is Algebras(ZZ).parent_class
```

(continues on next page)

(continued from previous page)

```
False
>>> issubclass(Algebras(QQ).parent_class, VectorSpaces(QQ).parent_class)
True
```

On the other hand, identical hierarchies of classes are, preferably, built only once (e.g. for categories over a base ring):

```
sage: Algebras(GF(5)).parent_class is Algebras(GF(7)).parent_class
True
sage: F = FractionField(ZZ['t'])
sage: Coalgebras(F).parent_class is Coalgebras(FractionField(F['x'])).parent_class
True
```

```
>>> from sage.all import *
>>> Algebras(GF(Integer(5))).parent_class is Algebras(GF(Integer(7))).parent_class
True
>>> F = FractionField(ZZ['t'])
>>> Coalgebras(F).parent_class is Coalgebras(FractionField(F['x'])).parent_class
True
```

We now construct a parent in the usual way:

```
sage: class myparent(Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category=Ds())
....:     def g(self):
....:         return "myparent"
....:     class Element():
....:         pass
sage: D = myparent()
sage: D.__class__
<class '__main__.myparent_with_category'>
sage: D.__class__.__bases__
(<class '__main__.myparent'>, <class '__main__.Ds.parent_class'>)
sage: D.__class__.mro()
[<class '__main__.myparent_with_category'>,
<class '__main__.myparent'>,
<class 'sage.structure.parent.Parent'>,
<class 'sage.structure.category_object.CategoryObject'>,
<class 'sage.structure.sage_object.SageObject'>,
<class '__main__.Ds.parent_class'>,
<class '__main__.Cs.parent_class'>,
<class '__main__.Bs.parent_class'>,
<class '__main__.As.parent_class'>,
<... 'object'>]
sage: D.fA()
'A'
sage: D.fB()
'B'
sage: D.fC()
'C'
sage: D.fD()
```

(continues on next page)

(continued from previous page)

```
'D'
sage: D.f()
'C'
sage: D.g()
'myparent'
```

```
>>> from sage.all import *
>>> class myparent(Parent):
...     def __init__(self):
...         Parent.__init__(self, category=Ds())
...     def g(self):
...         return "myparent"
...     class Element():
...         pass
>>> D = myparent()
>>> D.__class__
<class '__main__.myparent_with_category'>
>>> D.__class__.__bases__
(<class '__main__.myparent'>, <class '__main__.Ds.parent_class'>)
>>> D.__class__.mro()
[<class '__main__.myparent_with_category'>,
<class '__main__.myparent'>,
<class 'sage.structure.parent.Parent'>,
<class 'sage.structure.category_object.CategoryObject'>,
<class 'sage.structure.sage_object.SageObject'>,
<class '__main__.Ds.parent_class'>,
<class '__main__.Cs.parent_class'>,
<class '__main__.Bs.parent_class'>,
<class '__main__.As.parent_class'>,
<... 'object'>]
>>> D.fA()
'A'
>>> D.fB()
'B'
>>> D.fC()
'C'
>>> D.fD()
'D'
>>> D.f()
'C'
>>> D.g()
'myparent'
```

```
sage: D.element_class
<class '__main__.myparent_with_category.element_class'>
sage: D.element_class.mro()
[<class '__main__.myparent_with_category.element_class'>,
<class ...__main__....Element...>,
<class '__main__.Ds.element_class'>,
<class '__main__.Cs.element_class'>,
<class '__main__.Bs.element_class'>,
```

(continues on next page)

(continued from previous page)

```
<class '__main__.As.element_class'>,
<... 'object'>]
```

```
>>> from sage.all import *
>>> D.element_class
<class '__main__.myparent_with_category.element_class'>
>>> D.element_class.mro()
[<class '__main__.myparent_with_category.element_class'>,
<class ...__main__....Element...>,
<class '__main__.Ds.element_class'>,
<class '__main__.Cs.element_class'>,
<class '__main__.Bs.element_class'>,
<class '__main__.As.element_class'>,
<... 'object'>]
```

_super_categories()

The immediate super categories of this category.

This lazy attribute caches the result of the mandatory method *super_categories()* for speed. It also does some mangling (flattening join categories, sorting, …).

Whenever speed matters, developers are advised to use this lazy attribute rather than calling *super_categories()*.

Note

This attribute is likely to eventually become a tuple. When this happens, we might as well use *Category._sort()*, if not *Category._sort_uniq()*.

EXAMPLES:

```
sage: Rings()._super_categories
[Category of rngs, Category of semirings]
```

```
>>> from sage.all import *
>>> Rings()._super_categories
[Category of rngs, Category of semirings]
```

_super_categories_for_classes()

The super categories of this category used for building classes.

This is a close variant of *_super_categories()* used for constructing the list of the bases for *parent_class()*, *element_class()*, and friends. The purpose is ensure that Python will find a proper Method Resolution Order for those classes. For background, see sage.misc.c3_controlled.

See also

CmpKey.

Note

This attribute is calculated as a by-product of computing *_all_super_categories()*.

EXAMPLES:

```
sage: Rings()._super_categories_for_classes
[Category of rngs, Category of semirings]
```

```
>>> from sage.all import *
>>> Rings()._super_categories_for_classes
[Category of rngs, Category of semirings]
```

_all_super_categories()

All the super categories of this category, including this category.

Since Issue #11943, the order of super categories is determined by Python's method resolution order C3 algorithm.

See also

all_super_categories()

Note

this attribute is likely to eventually become a tuple.

Note

this sets *_super_categories_for_classes()* as a side effect

EXAMPLES:

```
sage: C = Rings(); C
Category of rings
sage: C._all_super_categories
[Category of rings, Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = Rings(); C
Category of rings
>>> C._all_super_categories
[Category of rings, Category of rngs, Category of semirings, ...
 Category of monoids, ...
```

(continues on next page)

(continued from previous page)

```
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]
```

_all_super_categories_proper()

All the proper super categories of this category.

Since Issue #11943, the order of super categories is determined by Python's method resolution order C3 algorithm.

See also

all_super_categories()

Note

this attribute is likely to eventually become a tuple.

EXAMPLES:

```
sage: C = Rings(); C
Category of rings
sage: C._all_super_categories_proper
[Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = Rings(); C
Category of rings
>>> C._all_super_categories_proper
[Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]
```

_set_of_super_categories()

The frozen set of all proper super categories of this category.

Note

this is used for speeding up category containment tests.

See also

all_super_categories()

EXAMPLES:

```
sage: sorted(Groups()._set_of_super_categories, key=str)
[Category of inverse unital magmas,
 Category of magmas,
 Category of monoids,
 Category of objects,
 Category of semigroups,
 Category of sets,
 Category of sets with partial maps,
 Category of unital magmas]
sage: sorted(Groups()._set_of_super_categories, key=str)
[Category of inverse unital magmas, Category of magmas, Category of monoids,
 Category of objects, Category of semigroups, Category of sets,
 Category of sets with partial maps, Category of unital magmas]
```

```
>>> from sage.all import *
>>> sorted(Groups()._set_of_super_categories, key=str)
[Category of inverse unital magmas,
 Category of magmas,
 Category of monoids,
 Category of objects,
 Category of semigroups,
 Category of sets,
 Category of sets with partial maps,
 Category of unital magmas]
>>> sorted(Groups()._set_of_super_categories, key=str)
[Category of inverse unital magmas, Category of magmas, Category of monoids,
 Category of objects, Category of semigroups, Category of sets,
 Category of sets with partial maps, Category of unital magmas]
```

_make_named_class(*name*, *method_provider*, *cache=False*, *picklable=True*)

Construction of the parent/element/… class of `self`.

INPUT:

- `name` – string; the name of the class as an attribute of `self` (e.g. "parent_class")
- `method_provider` – string; the name of an attribute of `self` that provides methods for the new class (in addition to those coming from the super categories, e.g. "ParentMethods")
- `cache` – boolean or `ignore_reduction` (default: `False`) (passed down to dynamic_class; for internal use only)
- `picklable` – boolean (default: `True`)

ASSUMPTION:

It is assumed that this method is only called from a lazy attribute whose name coincides with the given `name`.

OUTPUT:

A dynamic class with bases given by the corresponding named classes of `self`'s super_categories, and methods taken from the class `getattr(self,method_provider)`.

Note

- In this default implementation, the reduction data of the named class makes it depend on `self`. Since the result is going to be stored in a lazy attribute of `self` anyway, we may as well disable the caching in `dynamic_class` (hence the default value `cache=False`).
- *CategoryWithParameters* overrides this method so that the same parent/element/... classes can be shared between closely related categories.
- The bases of the named class may also contain the named classes of some indirect super categories, according to *_super_categories_for_classes()*. This is to guarantee that Python will build consistent method resolution orders. For background, see `sage.misc.c3_controlled`.

See also

`CategoryWithParameters._make_named_class`

EXAMPLES:

```
sage: PC = Rings()._make_named_class("parent_class", "ParentMethods"); PC
<class 'sage.categories.rings.Rings.parent_class'>
sage: type(PC)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
sage: PC.__bases__
(<class 'sage.categories.rngs.Rngs.parent_class'>,
 <class 'sage.categories.semirings.Semirings.parent_class'>)
```

```
>>> from sage.all import *
>>> PC = Rings()._make_named_class("parent_class", "ParentMethods"); PC
<class 'sage.categories.rings.Rings.parent_class'>
>>> type(PC)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
>>> PC.__bases__
(<class 'sage.categories.rngs.Rngs.parent_class'>,
 <class 'sage.categories.semirings.Semirings.parent_class'>)
```

Note that, by default, the result is not cached:

```
sage: PC is Rings()._make_named_class("parent_class", "ParentMethods")
False
```

```
>>> from sage.all import *
>>> PC is Rings()._make_named_class("parent_class", "ParentMethods")
False
```

Indeed this method is only meant to construct lazy attributes like `parent_class` which already handle this caching:

```
sage: Rings().parent_class
<class 'sage.categories.rings.Rings.parent_class'>
```

```
>>> from sage.all import *
>>> Rings().parent_class
<class 'sage.categories.rings.Rings.parent_class'>
```

Reduction for pickling also assumes the existence of this lazy attribute:

```
sage: PC._reduction
(<built-in function getattr>, (Category of rings, 'parent_class'))
sage: loads(dumps(PC)) is Rings().parent_class
True
```

```
>>> from sage.all import *
>>> PC._reduction
(<built-in function getattr>, (Category of rings, 'parent_class'))
>>> loads(dumps(PC)) is Rings().parent_class
True
```

repr()

Return the print representation of this category.

EXAMPLES:

```
sage: Sets() # indirect doctest
Category of sets
```

```
>>> from sage.all import *
>>> Sets() # indirect doctest
Category of sets
```

_repr_object_names()

Return the name of the objects of this category.

EXAMPLES:

```
sage: FiniteGroups()._repr_object_names()
'finite groups'
sage: AlgebrasWithBasis(QQ)._repr_object_names()
'algebras with basis over Rational Field'
```

```
>>> from sage.all import *
>>> FiniteGroups()._repr_object_names()
'finite groups'
>>> AlgebrasWithBasis(QQ)._repr_object_names()
'algebras with basis over Rational Field'
```

_test_category(***options*)

Run generic tests on this category.

See also

TestSuite.

EXAMPLES:

```
sage: Sets()._test_category()
```

```
>>> from sage.all import *
>>> Sets()._test_category()
```

Let us now write a couple broken categories:

```
sage: class MyObjects(Category):
....:     pass
sage: MyObjects()._test_category()
Traceback (most recent call last):
...
NotImplementedError: <abstract method super_categories at ...>

sage: class MyObjects(Category):
....:     def super_categories(self):
....:         return tuple()
sage: MyObjects()._test_category()
Traceback (most recent call last):
...
AssertionError: Category of my objects.super_categories() should return a list

sage: class MyObjects(Category):
....:     def super_categories(self):
....:         return []
sage: MyObjects()._test_category()
Traceback (most recent call last):
...
AssertionError: Category of my objects is not a subcategory of Objects()
```

```
>>> from sage.all import *
>>> class MyObjects(Category):
...     pass
>>> MyObjects()._test_category()
Traceback (most recent call last):
...
NotImplementedError: <abstract method super_categories at ...>

>>> class MyObjects(Category):
...     def super_categories(self):
...         return tuple()
>>> MyObjects()._test_category()
Traceback (most recent call last):
...
AssertionError: Category of my objects.super_categories() should return a list
```

(continues on next page)

(continued from previous page)

```
>>> class MyObjects(Category):
...       def super_categories(self):
...           return []
>>> MyObjects()._test_category()
Traceback (most recent call last):
...
AssertionError: Category of my objects is not a subcategory of Objects()
```

_with_axiom(*axiom*)

Return the subcategory of the objects of `self` satisfying the given `axiom`.

Note that this is a private method thus should not be directly used, see below.

INPUT:

- `axiom` – string, the name of an axiom

EXAMPLES:

```
sage: Sets()._with_axiom("Finite")  # not idiomatic
Category of finite sets
sage: Sets().Finite()  # recommended
Category of finite sets

sage: type(Magmas().Finite().Commutative())
<class 'sage.categories.category.JoinCategory_with_category'>
sage: Magmas().Finite().Commutative().super_categories()
[Category of commutative magmas, Category of finite sets]
sage: C = Algebras(QQ).WithBasis().Commutative()
sage: C is Algebras(QQ).Commutative().WithBasis()
True
```

```
>>> from sage.all import *
>>> Sets()._with_axiom("Finite")  # not idiomatic
Category of finite sets
>>> Sets().Finite()  # recommended
Category of finite sets

>>> type(Magmas().Finite().Commutative())
<class 'sage.categories.category.JoinCategory_with_category'>
>>> Magmas().Finite().Commutative().super_categories()
[Category of commutative magmas, Category of finite sets]
>>> C = Algebras(QQ).WithBasis().Commutative()
>>> C is Algebras(QQ).Commutative().WithBasis()
True
```

When `axiom` is not defined for `self`, `self` is returned:

```
sage: Sets()._with_axiom("Associative")
Category of sets
```

```
>>> from sage.all import *
>>> Sets()._with_axiom("Associative")
Category of sets
```

> **Warning**
>
> This may be changed in the future to raise an error.

_with_axiom_as_tuple(*axiom*)

Return a tuple of categories whose join is `self._with_axiom()`.

INPUT:

- `axiom` – string, the name of an axiom

This is a lazy version of *`_with_axiom()`* which is used to avoid recursion loops during join calculations.

> **Note**
>
> The order in the result is irrelevant.

EXAMPLES:

```
sage: Sets()._with_axiom_as_tuple('Finite')
(Category of finite sets,)
sage: Magmas()._with_axiom_as_tuple('Finite')
(Category of magmas, Category of finite sets)
sage: Rings().Division()._with_axiom_as_tuple('Finite')
(Category of division rings,
 Category of finite monoids,
 Category of commutative magmas,
 Category of finite additive groups)
sage: HopfAlgebras(QQ)._with_axiom_as_tuple('FiniteDimensional')
(Category of Hopf algebras over Rational Field,
 Category of finite dimensional vector spaces over Rational Field)
```

```
>>> from sage.all import *
>>> Sets()._with_axiom_as_tuple('Finite')
(Category of finite sets,)
>>> Magmas()._with_axiom_as_tuple('Finite')
(Category of magmas, Category of finite sets)
>>> Rings().Division()._with_axiom_as_tuple('Finite')
(Category of division rings,
 Category of finite monoids,
 Category of commutative magmas,
 Category of finite additive groups)
>>> HopfAlgebras(QQ)._with_axiom_as_tuple('FiniteDimensional')
(Category of Hopf algebras over Rational Field,
 Category of finite dimensional vector spaces over Rational Field)
```

_without_axioms(*named=False*)

Return the category without the axioms that have been added to create it.

INPUT:

- `named` – boolean (default: `False`)

Todo

Improve this explanation.

If `named` is `True`, then this stops at the first category that has an explicit name of its own. See *category_with_axiom.CategoryWithAxiom._without_axioms()*

EXAMPLES:

```
sage: Sets()._without_axioms()
Category of sets
sage: Semigroups()._without_axioms()
Category of magmas
sage: Algebras(QQ).Commutative().WithBasis()._without_axioms()
Category of magmatic algebras over Rational Field
sage: Algebras(QQ).Commutative().WithBasis()._without_axioms(named=True)
Category of algebras over Rational Field
```

```
>>> from sage.all import *
>>> Sets()._without_axioms()
Category of sets
>>> Semigroups()._without_axioms()
Category of magmas
>>> Algebras(QQ).Commutative().WithBasis()._without_axioms()
Category of magmatic algebras over Rational Field
>>> Algebras(QQ).Commutative().WithBasis()._without_axioms(named=True)
Category of algebras over Rational Field
```

static _sort(*categories*)

Return the categories after sorting them decreasingly according to their comparison key.

See also

`CmpKey`

INPUT:

- `categories` – list (or iterable) of non-join categories

OUTPUT: a sorted tuple of categories, possibly with repeats

Note

The auxiliary function `_flatten_categories` used in the test below expects a second argument, which is a type such that instances of that type will be replaced by its super categories. Usually, this type is *JoinCategory*.

EXAMPLES:

```
sage: Category._sort([Sets(), Objects(), Coalgebras(QQ), Monoids(), Sets().
↪Finite()])
(Category of monoids,
```

(continues on next page)

(continued from previous page)

```
 Category of coalgebras over Rational Field,
 Category of finite sets,
 Category of sets,
 Category of objects)
sage: Category._sort([Sets().Finite(), Semigroups().Finite(), Sets().Facade(),
 ↪Magmas().Commutative()])
(Category of finite semigroups,
 Category of commutative magmas,
 Category of finite sets,
 Category of facade sets)
sage: Category._sort(Category._flatten_categories([Sets().Finite(),␣
 ↪Algebras(QQ).WithBasis(), Semigroups().Finite(),
....:                                               Sets().Facade(),␣
 ↪Algebras(QQ).Commutative(), Algebras(QQ).Graded().WithBasis()],
....:                                               sage.categories.category.
 ↪JoinCategory))
(Category of algebras with basis over Rational Field,
 Category of algebras with basis over Rational Field,
 Category of graded algebras over Rational Field,
 Category of commutative algebras over Rational Field,
 Category of finite semigroups,
 Category of finite sets,
 Category of facade sets)
```

```
>>> from sage.all import *
>>> Category._sort([Sets(), Objects(), Coalgebras(QQ), Monoids(), Sets().
 ↪Finite()])
(Category of monoids,
 Category of coalgebras over Rational Field,
 Category of finite sets,
 Category of sets,
 Category of objects)
>>> Category._sort([Sets().Finite(), Semigroups().Finite(), Sets().Facade(),
 ↪Magmas().Commutative()])
(Category of finite semigroups,
 Category of commutative magmas,
 Category of finite sets,
 Category of facade sets)
>>> Category._sort(Category._flatten_categories([Sets().Finite(),␣
 ↪Algebras(QQ).WithBasis(), Semigroups().Finite(),
...                                             Sets().Facade(),␣
 ↪Algebras(QQ).Commutative(), Algebras(QQ).Graded().WithBasis()],
...                                             sage.categories.category.
 ↪JoinCategory))
(Category of algebras with basis over Rational Field,
 Category of algebras with basis over Rational Field,
 Category of graded algebras over Rational Field,
 Category of commutative algebras over Rational Field,
 Category of finite semigroups,
 Category of finite sets,
 Category of facade sets)
```

static _sort_uniq(*categories*)

Return the categories after sorting them and removing redundant categories.

Redundant categories include duplicates and categories which are super categories of other categories in the input.

INPUT:

- `categories` – list (or iterable) of categories

OUTPUT: a sorted tuple of mutually incomparable categories

EXAMPLES:

```
sage: Category._sort_uniq([Rings(), Monoids(), Coalgebras(QQ)])
(Category of rings, Category of coalgebras over Rational Field)
```

```
>>> from sage.all import *
>>> Category._sort_uniq([Rings(), Monoids(), Coalgebras(QQ)])
(Category of rings, Category of coalgebras over Rational Field)
```

Note that, in the above example, `Monoids()` does not appear in the result because it is a super category of `Rings()`.

static __classcall__(**args*, ***options*)

Input mangling for unique representation.

Let `C = Cs(...)` be a category. Since Issue #12895, the class of `C` is a dynamic subclass `Cs_with_category` of `Cs` in order for `C` to inherit code from the `SubcategoryMethods` nested classes of its super categories.

The purpose of this `__classcall__` method is to ensure that reconstructing `C` from its class with `Cs_with_category(...)` actually calls properly `Cs(...)` and gives back `C`.

See also

subcategory_class()

EXAMPLES:

```
sage: A = Algebras(QQ)
sage: A.__class__
<class 'sage.categories.algebras.Algebras_with_category'>
sage: A is Algebras(QQ)
True
sage: A is A.__class__(QQ)
True
```

```
>>> from sage.all import *
>>> A = Algebras(QQ)
>>> A.__class__
<class 'sage.categories.algebras.Algebras_with_category'>
>>> A is Algebras(QQ)
True
>>> A is A.__class__(QQ)
True
```

__init__()

Initialize this category.

EXAMPLES:

```
sage: class SemiprimitiveRings(Category):
....:     def super_categories(self):
....:         return [Rings()]
....:     class ParentMethods:
....:         def jacobson_radical(self):
....:             return self.ideal(0)
sage: C = SemiprimitiveRings()
sage: C
Category of semiprimitive rings
sage: C.__class__
<class '__main__.SemiprimitiveRings_with_category'>
```

```
>>> from sage.all import *
>>> class SemiprimitiveRings(Category):
...     def super_categories(self):
...         return [Rings()]
...     class ParentMethods:
...         def jacobson_radical(self):
...             return self.ideal(Integer(0))
>>> C = SemiprimitiveRings()
>>> C
Category of semiprimitive rings
>>> C.__class__
<class '__main__.SemiprimitiveRings_with_category'>
```

Note

If the default name of the category (built from the name of the class) is not adequate, please implement *_repr_object_names()* to customize it.

Realizations()

Return the category of realizations of the parent `self` or of objects of the category `self`

INPUT:

- `self` – a parent or a concrete category

Note

this *function* is actually inserted as a *method* in the class *Category* (see *Realizations()*). It is defined here for code locality reasons.

EXAMPLES:

The category of realizations of some algebra:

```
sage: Algebras(QQ).Realizations()
Join of Category of algebras over Rational Field
    and Category of realizations of unital magmas
```

```
>>> from sage.all import *
>>> Algebras(QQ).Realizations()
Join of Category of algebras over Rational Field
    and Category of realizations of unital magmas
```

The category of realizations of a given algebra:

```
sage: A = Sets().WithRealizations().example(); A
↪  # needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.Realizations()
↪  # needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field

sage: C = GradedHopfAlgebrasWithBasis(QQ).Realizations(); C
Join of Category of graded Hopf algebras with basis over Rational Field
    and Category of realizations of Hopf algebras over Rational Field
sage: C.super_categories()
[Category of graded Hopf algebras with basis over Rational Field,
 Category of realizations of Hopf algebras over Rational Field]

sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
↪# needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.Realizations()
↪# needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field

>>> C = GradedHopfAlgebrasWithBasis(QQ).Realizations(); C
Join of Category of graded Hopf algebras with basis over Rational Field
    and Category of realizations of Hopf algebras over Rational Field
>>> C.super_categories()
[Category of graded Hopf algebras with basis over Rational Field,
 Category of realizations of Hopf algebras over Rational Field]

>>> TestSuite(C).run()
```

See also

- *Sets().WithRealizations*
- ClasscallMetaclass

Todo

Add an optional argument to allow for:

```
sage: Realizations(A, category=Blahs()) # todo: not implemented
```

```
>>> from sage.all import *
>>> Realizations(A, category=Blahs()) # todo: not implemented
```

WithRealizations()

Return the category of parents in `self` endowed with multiple realizations.

INPUT:

- `self` – a category

See also

- The documentation and code (*sage.categories.examples.with_realizations*) of `Sets().WithRealizations().example()` for more on how to use and implement a parent with several realizations.
- Various use cases:
  - `SymmetricFunctions`
  - `QuasiSymmetricFunctions`
  - `NonCommutativeSymmetricFunctions`
  - `SymmetricFunctionsNonCommutingVariables`
  - `DescentAlgebra`
  - `MoebiusAlgebra`
  - `IwahoriHeckeAlgebra`
  - `ExtendedAffineWeylGroup()`
- The Implementing Algebraic Structures thematic tutorial.
- *sage.categories.realizations*

Note

this *function* is actually inserted as a *method* in the class *Category* (see *WithRealizations()*). It is defined here for code locality reasons.

EXAMPLES:

```
sage: Sets().WithRealizations()
Category of sets with realizations
```

```
>>> from sage.all import *
>>> Sets().WithRealizations()
```

(continues on next page)

(continued from previous page)

```
Category of sets with realizations
```

Parent with realizations

Let us now explain the concept of realizations. A *parent with realizations* is a facade parent (see *Sets.Facade*) admitting multiple concrete realizations where its elements are represented. Consider for example an algebra A which admits several natural bases:

```
sage: A = Sets().WithRealizations().example(); A
↪  # needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
↪# needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

For each such basis B one implements a parent P_B which realizes A with its elements represented by expanding them on the basis B:

```
sage: # needs sage.modules
sage: A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
sage: A.Out()
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
sage: A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
sage: A.an_element()
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
>>> A.Out()
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
>>> A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
>>> A.an_element()
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
```

If B and B' are two bases, then the change of basis from B to B' is implemented by a canonical coercion between P_B and $P_{B'}$:

```
sage: # needs sage.modules
sage: F = A.F(); In = A.In(); Out = A.Out()
sage: i = In.an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
sage: F(i)
7*F[{}] + 3*F[{1}] + 4*F[{2}] + F[{1, 2}]
sage: F.coerce_map_from(Out)
Generic morphism:
```

(continues on next page)

(continued from previous page)

```
  From: The subset algebra of {1, 2, 3} over Rational Field in the Out basis
  To:   The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = A.F(); In = A.In(); Out = A.Out()
>>> i = In.an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
>>> F(i)
7*F[{}] + 3*F[{1}] + 4*F[{2}] + F[{1, 2}]
>>> F.coerce_map_from(Out)
Generic morphism:
  From: The subset algebra of {1, 2, 3} over Rational Field in the Out basis
  To:   The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis
```

allowing for mixed arithmetic:

```
sage: (1 + Out.from_set(1)) * In.from_set(2,3)                                 ␣
↪  # needs sage.modules
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}]
+ 2*Out[{1, 3}] + 4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> (Integer(1) + Out.from_set(Integer(1))) * In.from_set(Integer(2),
↪Integer(3))                                      # needs sage.modules
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}]
+ 2*Out[{1, 3}] + 4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

In our example, there are three realizations:

```
sage: A.realizations()                                                         ␣
↪  # needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

```
>>> from sage.all import *
>>> A.realizations()
↪# needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

Instead of manually defining the shorthands `F`, `In`, and `Out`, as above one can just do:

```
sage: A.inject_shorthands()                                                    ␣
↪  # needs sage.modules
Defining F as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
Defining In as shorthand for
```

(continues on next page)

(continued from previous page)

```
 The subset algebra of {1, 2, 3} over Rational Field in the In basis
Defining Out as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

```
>>> from sage.all import *
>>> A.inject_shorthands()
↪# needs sage.modules
Defining F as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
Defining In as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the In basis
Defining Out as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

Rationale

Besides some goodies described below, the role of A is threefold:

- To provide, as illustrated above, a single entry point for the algebra as a whole: documentation, access to its properties and different realizations, etc.
- To provide a natural location for the initialization of the bases and the coercions between, and other methods that are common to all bases.
- To let other objects refer to A while allowing elements to be represented in any of the realizations.

We now illustrate this second point by defining the polynomial ring with coefficients in A:

```
sage: P = A['x']; P                                                             ␣
↪  # needs sage.modules
Univariate Polynomial Ring in x over
 The subset algebra of {1, 2, 3} over Rational Field
sage: x = P.gen()                                                               ␣
↪  # needs sage.modules
```

```
>>> from sage.all import *
>>> P = A['x']; P
↪# needs sage.modules
Univariate Polynomial Ring in x over
 The subset algebra of {1, 2, 3} over Rational Field
>>> x = P.gen()
↪# needs sage.modules
```

In the following examples, the coefficients turn out to be all represented in the F basis:

```
sage: P.one()                                                                   ␣
↪  # needs sage.modules
F[{}]
sage: (P.an_element() + 1)^2                                                    ␣
↪  # needs sage.modules
F[{}]*x^2 + 2*F[{}]*x + F[{}]
```

```
>>> from sage.all import *
>>> P.one()
↪# needs sage.modules
F[{}]
>>> (P.an_element() + Integer(1))**Integer(2)
↪                    # needs sage.modules
F[{}]*x^2 + 2*F[{}]*x + F[{}]
```

However we can create a polynomial with mixed coefficients, and compute with it:

```
sage: p = P([1, In[{1}], Out[{2}] ]); p
↪  # needs sage.modules
Out[{2}]*x^2 + In[{1}]*x + F[{}]
sage: p^2
↪  # needs sage.modules
Out[{2}]*x^4
+ (-8*In[{}] + 4*In[{1}] + 8*In[{2}] + 4*In[{3}]
   - 4*In[{1, 2}] - 2*In[{1, 3}] - 4*In[{2, 3}] + 2*In[{1, 2, 3}])*x^3
+ (F[{}] + 3*F[{1}] + 2*F[{2}] - 2*F[{1, 2}] - 2*F[{2, 3}] + 2*F[{1, 2, 3}
↪])*x^2
+ (2*F[{}] + 2*F[{1}])*x
+ F[{}]
```

```
>>> from sage.all import *
>>> p = P([Integer(1), In[{Integer(1)}], Out[{Integer(2)}] ]); p
↪                            # needs sage.modules
Out[{2}]*x^2 + In[{1}]*x + F[{}]
>>> p**Integer(2)
↪          # needs sage.modules
Out[{2}]*x^4
+ (-8*In[{}] + 4*In[{1}] + 8*In[{2}] + 4*In[{3}]
   - 4*In[{1, 2}] - 2*In[{1, 3}] - 4*In[{2, 3}] + 2*In[{1, 2, 3}])*x^3
+ (F[{}] + 3*F[{1}] + 2*F[{2}] - 2*F[{1, 2}] - 2*F[{2, 3}] + 2*F[{1, 2, 3}
↪])*x^2
+ (2*F[{}] + 2*F[{1}])*x
+ F[{}]
```

Note how each coefficient involves a single basis which need not be that of the other coefficients. Which basis is used depends on how coercion happened during mixed arithmetic and needs not be deterministic.

One can easily coerce all coefficient to a given basis with:

```
sage: p.map_coefficients(In)
↪  # needs sage.modules
(-4*In[{}] + 2*In[{1}] + 4*In[{2}] + 2*In[{3}]
 - 2*In[{1, 2}] - In[{1, 3}] - 2*In[{2, 3}] + In[{1, 2, 3}])*x^2
+ In[{1}]*x + In[{}]
```

```
>>> from sage.all import *
>>> p.map_coefficients(In)
↪# needs sage.modules
(-4*In[{}] + 2*In[{1}] + 4*In[{2}] + 2*In[{3}]
 - 2*In[{1, 2}] - In[{1, 3}] - 2*In[{2, 3}] + In[{1, 2, 3}])*x^2
```

(continues on next page)

(continued from previous page)

```
+ In[{1}]*x + In[{}]
```

Alas, the natural notation for constructing such polynomials does not yet work:

```
sage: In[{1}] * x
 ↪  # needs sage.modules
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
'The subset algebra of {1, 2, 3} over Rational Field in the In basis'
and 'Univariate Polynomial Ring in x over
The subset algebra of {1, 2, 3} over Rational Field'
```

```
>>> from sage.all import *
>>> In[{Integer(1)}] * x
 ↪         # needs sage.modules
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
'The subset algebra of {1, 2, 3} over Rational Field in the In basis'
and 'Univariate Polynomial Ring in x over
The subset algebra of {1, 2, 3} over Rational Field'
```

The category of realizations of A

The set of all realizations of A, together with the coercion morphisms is a category (whose class inherits from *Category_realization_of_parent*):

```
sage: A.Realizations()
 ↪  # needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A.Realizations()
 ↪# needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

The various parent realizing A belong to this category:

```
sage: A.F() in A.Realizations()
 ↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> A.F() in A.Realizations()
 ↪# needs sage.modules
True
```

A itself is in the category of algebras with realizations:

```
sage: A in Algebras(QQ).WithRealizations()
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Algebras(QQ).WithRealizations()
↪# needs sage.modules
True
```

The (mostly technical) `WithRealizations` categories are the analogs of the `*WithSeveralBases` categories in MuPAD-Combinat. They provide support tools for handling the different realizations and the morphisms between them.

Typically, `VectorSpaces(QQ).FiniteDimensional().WithRealizations()` will eventually be in charge, whenever a coercion $\phi : A \mapsto B$ is registered, to register ϕ^{-1} as coercion $B \mapsto A$ if there is none defined yet. To achieve this, `FiniteDimensionalVectorSpaces` would provide a nested class `WithRealizations` implementing the appropriate logic.

`WithRealizations` is a *regressive covariant functorial construction*. On our example, this simply means that A is automatically in the category of rings with realizations (covariance):

```
sage: A in Rings().WithRealizations()
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Rings().WithRealizations()
↪# needs sage.modules
True
```

and in the category of algebras (regressiveness):

```
sage: A in Algebras(QQ)
↪  # needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Algebras(QQ)
↪# needs sage.modules
True
```

Note

For `C` a category, `C.WithRealizations()` in fact calls `sage.categories.with_realizations.WithRealizations(C)`. The later is responsible for building the hierarchy of the categories with realizations in parallel to that of their base categories, optimizing away those categories that do not provide a `WithRealizations` nested class. See *sage.categories.covariant_functorial_construction* for the technical details.

Note

Design question: currently `WithRealizations` is a regressive construction. That is `self.WithRealizations()` is a subcategory of `self` by default:

```
sage: Algebras(QQ).WithRealizations().super_categories()
[Category of algebras over Rational Field,
 Category of monoids with realizations,
 Category of additive unital additive magmas with realizations]
```

```
>>> from sage.all import *
>>> Algebras(QQ).WithRealizations().super_categories()
[Category of algebras over Rational Field,
 Category of monoids with realizations,
 Category of additive unital additive magmas with realizations]
```

Is this always desirable? For example, `AlgebrasWithBasis(QQ).WithRealizations()` should certainly be a subcategory of `Algebras(QQ)`, but not of `AlgebrasWithBasis(QQ)`. This is because `AlgebrasWithBasis(QQ)` is specifying something about the concrete realization.

additional_structure()

Return whether `self` defines additional structure.

OUTPUT:

- `self` if `self` defines additional structure and `None` otherwise. This default implementation returns `self`.

A category C *defines additional structure* if C-morphisms shall preserve more structure (e.g. operations) than that specified by the super categories of C. For example, the category of magmas defines additional structure, namely the operation $*$ that shall be preserved by magma morphisms. On the other hand the category of rings does not define additional structure: a function between two rings that is both a unital magma morphism and a unital additive magma morphism is automatically a ring morphism.

Formally speaking C *defines additional structure*, if C is *not* a full subcategory of the join of its super categories: the morphisms need to preserve more structure, and thus the homsets are smaller.

By default, a category is considered as defining additional structure, unless it is a *category with axiom*.

EXAMPLES:

Here are some typical structure categories, with the additional structure they define:

```
sage: Sets().additional_structure()
Category of sets
sage: Magmas().additional_structure()          # `*`
Category of magmas
sage: AdditiveMagmas().additional_structure()  # `+`
Category of additive magmas
sage: LeftModules(ZZ).additional_structure()   # left multiplication by scalar
Category of left modules over Integer Ring
sage: Coalgebras(QQ).additional_structure()    # coproduct
Category of coalgebras over Rational Field
sage: Crystals().additional_structure()        # crystal operators
Category of crystals
```

```
>>> from sage.all import *
>>> Sets().additional_structure()
Category of sets
>>> Magmas().additional_structure()          # `*`
Category of magmas
>>> AdditiveMagmas().additional_structure() # `+`
Category of additive magmas
>>> LeftModules(ZZ).additional_structure()  # left multiplication by scalar
Category of left modules over Integer Ring
>>> Coalgebras(QQ).additional_structure()   # coproduct
Category of coalgebras over Rational Field
>>> Crystals().additional_structure()       # crystal operators
Category of crystals
```

On the other hand, the category of semigroups is not a structure category, since its operation $+$ is already defined by the category of magmas:

```
sage: Semigroups().additional_structure()
```

```
>>> from sage.all import *
>>> Semigroups().additional_structure()
```

Most *categories with axiom* don't define additional structure:

```
sage: Sets().Finite().additional_structure()
sage: Rings().Commutative().additional_structure()
sage: Modules(QQ).FiniteDimensional().additional_structure()
sage: from sage.categories.magmatic_algebras import MagmaticAlgebras
sage: MagmaticAlgebras(QQ).Unital().additional_structure()
```

```
>>> from sage.all import *
>>> Sets().Finite().additional_structure()
>>> Rings().Commutative().additional_structure()
>>> Modules(QQ).FiniteDimensional().additional_structure()
>>> from sage.categories.magmatic_algebras import MagmaticAlgebras
>>> MagmaticAlgebras(QQ).Unital().additional_structure()
```

As of Sage 6.4, the only exceptions are the category of unital magmas or the category of unital additive magmas (both define a unit which shall be preserved by morphisms):

```
sage: Magmas().Unital().additional_structure()
Category of unital magmas
sage: AdditiveMagmas().AdditiveUnital().additional_structure()
Category of additive unital additive magmas
```

```
>>> from sage.all import *
>>> Magmas().Unital().additional_structure()
Category of unital magmas
>>> AdditiveMagmas().AdditiveUnital().additional_structure()
Category of additive unital additive magmas
```

Similarly, *functorial construction categories* don't define additional structure, unless the construction is actually defined by their base category. For example, the category of graded modules defines a grading which shall be

preserved by morphisms:

```
sage: Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
```

```
>>> from sage.all import *
>>> Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
```

On the other hand, the category of graded algebras does not define additional structure; indeed an algebra morphism which is also a module morphism is a graded algebra morphism:

```
sage: Algebras(ZZ).Graded().additional_structure()
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Graded().additional_structure()
```

Similarly, morphisms are requested to preserve the structure given by the following constructions:

```
sage: Sets().Quotients().additional_structure()
Category of quotients of sets
sage: Sets().CartesianProducts().additional_structure()
Category of Cartesian products of sets
sage: Modules(QQ).TensorProducts().additional_structure()
```

```
>>> from sage.all import *
>>> Sets().Quotients().additional_structure()
Category of quotients of sets
>>> Sets().CartesianProducts().additional_structure()
Category of Cartesian products of sets
>>> Modules(QQ).TensorProducts().additional_structure()
```

This might change, as we are lacking enough data points to guarantee that this was the correct design decision.

Note

In some cases a category defines additional structure, where the structure can be useful to manipulate morphisms but where, in most use cases, we don't want the morphisms to necessarily preserve it. For example, in the context of finite dimensional vector spaces, having a distinguished basis allows for representing morphisms by matrices; yet considering only morphisms that preserve that distinguished basis would be boring.

In such cases, we might want to eventually have two categories, one where the additional structure is preserved, and one where it's not necessarily preserved (we would need to find an idiom for this).

At this point, a choice is to be made each time, according to the main use cases. Some of those choices are yet to be settled. For example, should by default:

- an euclidean domain morphism preserve euclidean division?

```
sage: EuclideanDomains().additional_structure()
Category of euclidean domains
```

```
>>> from sage.all import *
>>> EuclideanDomains().additional_structure()
Category of euclidean domains
```

- an enumerated set morphism preserve the distinguished enumeration?

```
sage: EnumeratedSets().additional_structure()
```

```
>>> from sage.all import *
>>> EnumeratedSets().additional_structure()
```

- a module with basis morphism preserve the distinguished basis?

```
sage: Modules(QQ).WithBasis().additional_structure()
```

```
>>> from sage.all import *
>>> Modules(QQ).WithBasis().additional_structure()
```

See also

This method together with the methods overloading it provide the basic data to determine, for a given category, the super categories that define some structure (see *structure()*), and to test whether a category is a full subcategory of some other category (see *is_full_subcategory()*). For example, the category of Coxeter groups is not full subcategory of the category of groups since morphisms need to preserve the distinguished generators:

```
sage: CoxeterGroups().is_full_subcategory(Groups())
False
```

```
>>> from sage.all import *
>>> CoxeterGroups().is_full_subcategory(Groups())
False
```

The support for modeling full subcategories has been introduced in Issue #16340.

all_super_categories(*proper=False*)

Return the list of all super categories of this category.

INPUT:

- `proper` – boolean (default: `False`); whether to exclude this category

Since Issue #11943, the order of super categories is determined by Python's method resolution order C3 algorithm.

Note

Whenever speed matters, the developers are advised to use instead the lazy attributes *_all_super_categories()*, *_all_super_categories_proper()*, or *_set_of_super_categories()*, as appropriate. Simply because lazy attributes are much faster than any method.

Note

This is not the same as the concept of super category in mathematics. In fact, this is not even the opposite relation of *is_subcategory()*:

```
sage: A = VectorSpaces(QQ); A
Category of vector spaces over Rational Field
sage: B = VectorSpaces(QQ.category()); B
Category of vector spaces over (number fields and quotient fields and␣
↪metric spaces)
sage: A.is_subcategory(B)
True
sage: B in A.all_super_categories()
False
>>> from sage.all import *
>>> A = VectorSpaces(QQ); A
Category of vector spaces over Rational Field
>>> B = VectorSpaces(QQ.category()); B
Category of vector spaces over (number fields and quotient fields and␣
↪metric spaces)
>>> A.is_subcategory(B)
True
>>> B in A.all_super_categories()
False
```

See also

_test_category_graph()

EXAMPLES:

```
sage: C = Rings(); C
Category of rings
sage: C.all_super_categories()
[Category of rings, Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]

sage: C.all_super_categories(proper = True)
[Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]

sage: Sets().all_super_categories()
[Category of sets, Category of sets with partial maps, Category of objects]
sage: Sets().all_super_categories(proper=True)
[Category of sets with partial maps, Category of objects]
sage: Sets().all_super_categories() is Sets()._all_super_categories
True
sage: Sets().all_super_categories(proper=True) is Sets()._all_super_
↪categories_proper
True
```

```
>>> from sage.all import *
>>> C = Rings(); C
Category of rings
>>> C.all_super_categories()
[Category of rings, Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]

>>> C.all_super_categories(proper = True)
[Category of rngs, Category of semirings, ...
 Category of monoids, ...
 Category of commutative additive groups, ...
 Category of sets, Category of sets with partial maps,
 Category of objects]

>>> Sets().all_super_categories()
[Category of sets, Category of sets with partial maps, Category of objects]
>>> Sets().all_super_categories(proper=True)
[Category of sets with partial maps, Category of objects]
>>> Sets().all_super_categories() is Sets()._all_super_categories
True
>>> Sets().all_super_categories(proper=True) is Sets()._all_super_categories_
↪proper
True
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: Rings.an_instance()
Category of rings
```

```
>>> from sage.all import *
>>> Rings.an_instance()
Category of rings
```

Parametrized categories should overload this default implementation to provide appropriate arguments:

```
sage: Algebras.an_instance()
Category of algebras over Rational Field
sage: Bimodules.an_instance()                                                #␣
↪needs sage.rings.real_mpfr
Category of bimodules over Rational Field on the left
 and Real Field with 53 bits of precision on the right
sage: AlgebraIdeals.an_instance()
Category of algebra ideals
 in Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> Algebras.an_instance()
```

(continues on next page)

(continued from previous page)

```
Category of algebras over Rational Field
>>> Bimodules.an_instance()                                                    #␣
↪needs sage.rings.real_mpfr
Category of bimodules over Rational Field on the left
 and Real Field with 53 bits of precision on the right
>>> AlgebraIdeals.an_instance()
Category of algebra ideals
 in Univariate Polynomial Ring in x over Rational Field
```

axioms()

Return the axioms known to be satisfied by all the objects of `self`.

Technically, this is the set of all the axioms `A` such that, if `Cs` is the category defining `A`, then `self` is a subcategory of `Cs().A()`. Any additional axiom `A` would yield a strict subcategory of `self`, at the very least `self & Cs().A()` where `Cs` is the category defining `A`.

EXAMPLES:

```
sage: Monoids().axioms()
frozenset({'Associative', 'Unital'})
sage: (EnumeratedSets().Infinite() & Sets().Facade()).axioms()
frozenset({'Enumerated', 'Facade', 'Infinite'})
```

```
>>> from sage.all import *
>>> Monoids().axioms()
frozenset({'Associative', 'Unital'})
>>> (EnumeratedSets().Infinite() & Sets().Facade()).axioms()
frozenset({'Enumerated', 'Facade', 'Infinite'})
```

category()

Return the category of this category. So far, all categories are in the category of objects.

EXAMPLES:

```
sage: Sets().category()
Category of objects
sage: VectorSpaces(QQ).category()
Category of objects
```

```
>>> from sage.all import *
>>> Sets().category()
Category of objects
>>> VectorSpaces(QQ).category()
Category of objects
```

category_graph()

Return the graph of all super categories of this category.

EXAMPLES:

```
sage: C = Algebras(QQ)
sage: G = C.category_graph()                                                #␣
↪needs sage.graphs
```

(continues on next page)

(continued from previous page)

```
sage: G.is_directed_acyclic()                                                 #␣
↪needs sage.graphs
True
```

```
>>> from sage.all import *
>>> C = Algebras(QQ)
>>> G = C.category_graph()                                                    #␣
↪needs sage.graphs
>>> G.is_directed_acyclic()                                                   #␣
↪needs sage.graphs
True
```

The girth of a directed acyclic graph is infinite, however, the girth of the underlying undirected graph is 4 in this case:

```
sage: Graph(G).girth()                                                        #␣
↪needs sage.graphs
4
```

```
>>> from sage.all import *
>>> Graph(G).girth()                                                          #␣
↪needs sage.graphs
4
```

element_class()

A common super class for all elements of parents in this category (and its subcategories).

This class contains the methods defined in the nested class `self.ElementMethods` (if it exists), and has as bases the element classes of the super categories of `self`.

See also

- *parent_class()*, *morphism_class()*
- *Category* for details

EXAMPLES:

```
sage: C = Algebras(QQ).element_class; C
<class 'sage.categories.algebras.Algebras.element_class'>
sage: type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).element_class; C
<class 'sage.categories.algebras.Algebras.element_class'>
>>> type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

By Issue #11935, some categories share their element classes. For example, the element class of an algebra only depends on the category of the base. A typical example is the category of algebras over a field versus algebras over a non-field:

```
sage: Algebras(GF(5)).element_class is Algebras(GF(3)).element_class
True
sage: Algebras(QQ).element_class is Algebras(ZZ).element_class
False
sage: Algebras(ZZ['t']).element_class is Algebras(ZZ['t','x']).element_class
True
```

```
>>> from sage.all import *
>>> Algebras(GF(Integer(5))).element_class is Algebras(GF(Integer(3))).
↪element_class
True
>>> Algebras(QQ).element_class is Algebras(ZZ).element_class
False
>>> Algebras(ZZ['t']).element_class is Algebras(ZZ['t','x']).element_class
True
```

These classes are constructed with `__slots__ = ()`, so instances may not have a `__dict__`:

```
sage: E = FiniteEnumeratedSets().element_class
sage: E.__dictoffset__
0
```

```
>>> from sage.all import *
>>> E = FiniteEnumeratedSets().element_class
>>> E.__dictoffset__
0
```

See also

parent_class()

Note

See the note about *_test_category_graph()* regarding Python class hierarchy.

example(**args*, ***keywords*)

Return an object in this category. Most of the time, this is a parent.

This serves three purposes:

- Give a typical example to better explain what the category is all about. (and by the way prove that the category is non empty :-))
- Provide a minimal template for implementing other objects in this category
- Provide an object on which to test generic code implemented by the category

For all those applications, the implementation of the object shall be kept to a strict minimum. The object is therefore not meant to be used for other applications; most of the time a full featured version is available elsewhere in Sage, and should be used instead.

Technical note: by default `FooBar(...).example()` is constructed by looking up `sage.categories.examples.foo_bar.Example` and calling it as `Example()`. Extra positional or named parameters are also

passed down. For a category over base ring, the base ring is further passed down as an optional argument.

Categories are welcome to override this default implementation.

EXAMPLES:

```
sage: Semigroups().example()
An example of a semigroup: the left zero semigroup

sage: Monoids().Subquotients().example()
NotImplemented
```

```
>>> from sage.all import *
>>> Semigroups().example()
An example of a semigroup: the left zero semigroup

>>> Monoids().Subquotients().example()
NotImplemented
```

full_super_categories()

Return the *immediate* full super categories of `self`.

See also

- *super_categories()*
- *is_full_subcategory()*

Warning

The current implementation selects the full subcategories among the immediate super categories of `self`. This assumes that, if $C \subset B \subset A$ is a chain of categories and C is a full subcategory of A, then C is a full subcategory of B and B is a full subcategory of A.

This assumption is guaranteed to hold with the current model and implementation of full subcategories in Sage. However, mathematically speaking, this is too restrictive. This indeed prevents the complete modelling of situations where any A morphism between elements of C automatically preserves the B structure. See below for an example.

EXAMPLES:

A semigroup morphism between two finite semigroups is a finite semigroup morphism:

```
sage: Semigroups().Finite().full_super_categories()
[Category of semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().Finite().full_super_categories()
[Category of semigroups]
```

On the other hand, a semigroup morphism between two monoids is not necessarily a monoid morphism (which must map the unit to the unit):

```
sage: Monoids().super_categories()
[Category of semigroups, Category of unital magmas]
sage: Monoids().full_super_categories()
[Category of unital magmas]
```

```
>>> from sage.all import *
>>> Monoids().super_categories()
[Category of semigroups, Category of unital magmas]
>>> Monoids().full_super_categories()
[Category of unital magmas]
```

Any semigroup morphism between two groups is automatically a monoid morphism (in a group the unit is the unique idempotent, so it has to be mapped to the unit). Yet, due to the limitation of the model advertised above, Sage currently cannot be taught that the category of groups is a full subcategory of the category of semigroups:

```
sage: Groups().full_super_categories()      # todo: not implemented
[Category of monoids, Category of semigroups, Category of inverse unital␣
↪magmas]
sage: Groups().full_super_categories()
[Category of monoids, Category of inverse unital magmas]
```

```
>>> from sage.all import *
>>> Groups().full_super_categories()      # todo: not implemented
[Category of monoids, Category of semigroups, Category of inverse unital␣
↪magmas]
>>> Groups().full_super_categories()
[Category of monoids, Category of inverse unital magmas]
```

is_abelian()

Return whether this category is abelian.

An abelian category is a category satisfying:

- It has a zero object;
- It has all pullbacks and pushouts;
- All monomorphisms and epimorphisms are normal.

Equivalently, one can define an increasing sequence of conditions:

- A category is pre-additive if it is enriched over abelian groups (all homsets are abelian groups and composition is bilinear);
- A pre-additive category is additive if every finite set of objects has a biproduct (we can form direct sums and direct products);
- An additive category is pre-abelian if every morphism has both a kernel and a cokernel;
- A pre-abelian category is abelian if every monomorphism is the kernel of some morphism and every epimorphism is the cokernel of some morphism.

EXAMPLES:

```
sage: Modules(ZZ).is_abelian()
True
```

(continues on next page)

(continued from previous page)

```
sage: FreeModules(ZZ).is_abelian()
False
sage: FreeModules(QQ).is_abelian()
True
sage: CommutativeAdditiveGroups().is_abelian()
True
sage: Semigroups().is_abelian()
Traceback (most recent call last):
...
NotImplementedError: is_abelian
```

```
>>> from sage.all import *
>>> Modules(ZZ).is_abelian()
True
>>> FreeModules(ZZ).is_abelian()
False
>>> FreeModules(QQ).is_abelian()
True
>>> CommutativeAdditiveGroups().is_abelian()
True
>>> Semigroups().is_abelian()
Traceback (most recent call last):
...
NotImplementedError: is_abelian
```

is_full_subcategory(*other*)

Return whether `self` is a full subcategory of `other`.

A subcategory B of a category A is a *full subcategory* if any A-morphism between two objects of B is also a B-morphism (the reciprocal always holds: any B-morphism between two objects of B is an A-morphism).

This is computed by testing whether `self` is a subcategory of `other` and whether they have the same structure, as determined by *structure()* from the result of *additional_structure()* on the super categories.

Warning

A positive answer is guaranteed to be mathematically correct. A negative answer may mean that Sage has not been taught enough information (or can not yet within the current model) to derive this information. See *full_super_categories()* for a discussion.

See also

- *is_subcategory()*
- *full_super_categories()*

EXAMPLES:

```
sage: Magmas().Associative().is_full_subcategory(Magmas())
True
sage: Magmas().Unital().is_full_subcategory(Magmas())
False
sage: Rings().is_full_subcategory(Magmas().Unital() & AdditiveMagmas().
↪AdditiveUnital())
True
```

```
>>> from sage.all import *
>>> Magmas().Associative().is_full_subcategory(Magmas())
True
>>> Magmas().Unital().is_full_subcategory(Magmas())
False
>>> Rings().is_full_subcategory(Magmas().Unital() & AdditiveMagmas().
↪AdditiveUnital())
True
```

Here are two typical examples of false negatives:

```
sage: Groups().is_full_subcategory(Semigroups())
False
sage: Groups().is_full_subcategory(Semigroups()) # todo: not implemented
True
sage: Fields().is_full_subcategory(Rings())
False
sage: Fields().is_full_subcategory(Rings())      # todo: not implemented
True
```

```
>>> from sage.all import *
>>> Groups().is_full_subcategory(Semigroups())
False
>>> Groups().is_full_subcategory(Semigroups()) # todo: not implemented
True
>>> Fields().is_full_subcategory(Rings())
False
>>> Fields().is_full_subcategory(Rings())      # todo: not implemented
True
```

Todo

The latter is a consequence of *EuclideanDomains* currently being a structure category. Is this what we want?

```
sage: EuclideanDomains().is_full_subcategory(Rings())
False
```

```
>>> from sage.all import *
>>> EuclideanDomains().is_full_subcategory(Rings())
False
```

is_subcategory(*c*)

Return `True` if there is a natural forgetful functor from `self` to *c*.

EXAMPLES:

```
sage: AbGrps = CommutativeAdditiveGroups()
sage: Rings().is_subcategory(AbGrps)
True
sage: AbGrps.is_subcategory(Rings())
False
```

```
>>> from sage.all import *
>>> AbGrps = CommutativeAdditiveGroups()
>>> Rings().is_subcategory(AbGrps)
True
>>> AbGrps.is_subcategory(Rings())
False
```

The `is_subcategory` function takes into account the base.

```
sage: M3 = VectorSpaces(FiniteField(3))
sage: M9 = VectorSpaces(FiniteField(9, 'a'))                                #␣
↪needs sage.rings.finite_rings
sage: M3.is_subcategory(M9)                                                 #␣
↪needs sage.rings.finite_rings
False
```

```
>>> from sage.all import *
>>> M3 = VectorSpaces(FiniteField(Integer(3)))
>>> M9 = VectorSpaces(FiniteField(Integer(9), 'a'))                          ␣
↪     # needs sage.rings.finite_rings
>>> M3.is_subcategory(M9)                                                  #␣
↪needs sage.rings.finite_rings
False
```

Join categories are properly handled:

```
sage: CatJ = Category.join((CommutativeAdditiveGroups(), Semigroups()))
sage: Rings().is_subcategory(CatJ)
True
```

```
>>> from sage.all import *
>>> CatJ = Category.join((CommutativeAdditiveGroups(), Semigroups()))
>>> Rings().is_subcategory(CatJ)
True
```

```
sage: V3 = VectorSpaces(FiniteField(3))
sage: POSet = PartiallyOrderedSets()
sage: PoV3 = Category.join((V3, POSet))
sage: A3 = AlgebrasWithBasis(FiniteField(3))
sage: PoA3 = Category.join((A3, POSet))
sage: PoA3.is_subcategory(PoV3)
True
sage: PoV3.is_subcategory(PoV3)
True
```

(continues on next page)

(continued from previous page)

```
sage: PoV3.is_subcategory(PoA3)
False
```

```
>>> from sage.all import *
>>> V3 = VectorSpaces(FiniteField(Integer(3)))
>>> POSet = PartiallyOrderedSets()
>>> PoV3 = Category.join((V3, POSet))
>>> A3 = AlgebrasWithBasis(FiniteField(Integer(3)))
>>> PoA3 = Category.join((A3, POSet))
>>> PoA3.is_subcategory(PoV3)
True
>>> PoV3.is_subcategory(PoV3)
True
>>> PoV3.is_subcategory(PoA3)
False
```

static join(*categories*, *as_list=False*, *ignore_axioms=()*, *axioms=()*)

Return the join of the input categories in the lattice of categories.

At the level of objects and morphisms, this operation corresponds to intersection: the objects and morphisms of a join category are those that belong to all its super categories.

INPUT:

- `categories` – list (or iterable) of categories
- `as_list` – boolean (default: `False`); whether the result should be returned as a list
- `axioms` – tuple of strings; the names of some supplementary axioms

See also

__and__() for a shortcut

EXAMPLES:

```
sage: J = Category.join((Groups(), CommutativeAdditiveMonoids())); J
Join of Category of groups and Category of commutative additive monoids
sage: J.super_categories()
[Category of groups, Category of commutative additive monoids]
sage: J.all_super_categories(proper=True)
[Category of groups, ..., Category of magmas,
 Category of commutative additive monoids, ..., Category of additive magmas,
 Category of sets, ...]
```

```
>>> from sage.all import *
>>> J = Category.join((Groups(), CommutativeAdditiveMonoids())); J
Join of Category of groups and Category of commutative additive monoids
>>> J.super_categories()
[Category of groups, Category of commutative additive monoids]
>>> J.all_super_categories(proper=True)
[Category of groups, ..., Category of magmas,
```

(continues on next page)

(continued from previous page)

```
 Category of commutative additive monoids, ..., Category of additive magmas,
 Category of sets, ...]
```

As a short hand, one can use:

```
sage: Groups() & CommutativeAdditiveMonoids()
Join of Category of groups and Category of commutative additive monoids
```

```
>>> from sage.all import *
>>> Groups() & CommutativeAdditiveMonoids()
Join of Category of groups and Category of commutative additive monoids
```

This is a commutative and associative operation:

```
sage: Groups() & Posets()
Join of Category of groups and Category of posets
sage: Posets() & Groups()
Join of Category of groups and Category of posets

sage: Groups() & (CommutativeAdditiveMonoids() & Posets())
Join of Category of groups
    and Category of commutative additive monoids
    and Category of posets
sage: (Groups() & CommutativeAdditiveMonoids()) & Posets()
Join of Category of groups
    and Category of commutative additive monoids
    and Category of posets
```

```
>>> from sage.all import *
>>> Groups() & Posets()
Join of Category of groups and Category of posets
>>> Posets() & Groups()
Join of Category of groups and Category of posets

>>> Groups() & (CommutativeAdditiveMonoids() & Posets())
Join of Category of groups
    and Category of commutative additive monoids
    and Category of posets
>>> (Groups() & CommutativeAdditiveMonoids()) & Posets()
Join of Category of groups
    and Category of commutative additive monoids
    and Category of posets
```

The join of a single category is the category itself:

```
sage: Category.join([Monoids()])
Category of monoids
```

```
>>> from sage.all import *
>>> Category.join([Monoids()])
Category of monoids
```

Similarly, the join of several mutually comparable categories is the smallest one:

```
sage: Category.join((Sets(), Rings(), Monoids()))
Category of rings
```

```
>>> from sage.all import *
>>> Category.join((Sets(), Rings(), Monoids()))
Category of rings
```

In particular, the unit is the top category *Objects*:

```
sage: Groups() & Objects()
Category of groups
```

```
>>> from sage.all import *
>>> Groups() & Objects()
Category of groups
```

If the optional parameter `as_list` is `True`, this returns the super categories of the join as a list, without constructing the join category itself:

```
sage: Category.join((Groups(), CommutativeAdditiveMonoids()), as_list=True)
[Category of groups, Category of commutative additive monoids]
sage: Category.join((Sets(), Rings(), Monoids()), as_list=True)
[Category of rings]
sage: Category.join((Modules(ZZ), FiniteFields()), as_list=True)
[Category of finite enumerated fields, Category of modules over Integer Ring]
sage: Category.join([], as_list=True)
[]
sage: Category.join([Groups()], as_list=True)
[Category of groups]
sage: Category.join([Groups() & Posets()], as_list=True)
[Category of groups, Category of posets]
```

```
>>> from sage.all import *
>>> Category.join((Groups(), CommutativeAdditiveMonoids()), as_list=True)
[Category of groups, Category of commutative additive monoids]
>>> Category.join((Sets(), Rings(), Monoids()), as_list=True)
[Category of rings]
>>> Category.join((Modules(ZZ), FiniteFields()), as_list=True)
[Category of finite enumerated fields, Category of modules over Integer Ring]
>>> Category.join([], as_list=True)
[]
>>> Category.join([Groups()], as_list=True)
[Category of groups]
>>> Category.join([Groups() & Posets()], as_list=True)
[Category of groups, Category of posets]
```

Support for axiom categories (TODO: put here meaningful examples):

```
sage: Sets().Facade() & Sets().Infinite()
Category of facade infinite sets
sage: Magmas().Infinite() & Sets().Facade()
Category of facade infinite magmas
```

(continues on next page)

(continued from previous page)

```
sage: FiniteSets() & Monoids()
Category of finite monoids
sage: Rings().Commutative() & Sets().Finite()
Category of finite commutative rings
```

```
>>> from sage.all import *
>>> Sets().Facade() & Sets().Infinite()
Category of facade infinite sets
>>> Magmas().Infinite() & Sets().Facade()
Category of facade infinite magmas

>>> FiniteSets() & Monoids()
Category of finite monoids
>>> Rings().Commutative() & Sets().Finite()
Category of finite commutative rings
```

Note that several of the above examples are actually join categories; they are just nicely displayed:

```
sage: AlgebrasWithBasis(QQ) & FiniteSets().Algebras(QQ)
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of finite set algebras over Rational Field

sage: UniqueFactorizationDomains() & Algebras(QQ)
Join of Category of unique factorization domains
    and Category of commutative algebras over Rational Field
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ) & FiniteSets().Algebras(QQ)
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of finite set algebras over Rational Field

>>> UniqueFactorizationDomains() & Algebras(QQ)
Join of Category of unique factorization domains
    and Category of commutative algebras over Rational Field
```

static meet(*categories*)

Return the meet of a list of categories.

INPUT:

- `categories` – a non empty list (or iterable) of categories

See also

__or__() for a shortcut

EXAMPLES:

```
sage: Category.meet([Algebras(ZZ), Algebras(QQ), Groups()])
Category of monoids
```

```
>>> from sage.all import *
>>> Category.meet([Algebras(ZZ), Algebras(QQ), Groups()])
Category of monoids
```

That meet of an empty list should be a category which is a subcategory of all categories, which does not make practical sense:

```
sage: Category.meet([])
Traceback (most recent call last):
...
ValueError: The meet of an empty list of categories is not implemented
```

```
>>> from sage.all import *
>>> Category.meet([])
Traceback (most recent call last):
...
ValueError: The meet of an empty list of categories is not implemented
```

morphism_class()

A common super class for all morphisms between parents in this category (and its subcategories).

This class contains the methods defined in the nested class `self.MorphismMethods` (if it exists), and has as bases the morphism classes of the super categories of `self`.

See also

- *parent_class()*, *element_class()*
- *Category* for details

EXAMPLES:

```
sage: C = Algebras(QQ).morphism_class; C
<class 'sage.categories.algebras.Algebras.morphism_class'>
sage: type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).morphism_class; C
<class 'sage.categories.algebras.Algebras.morphism_class'>
>>> type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

or_subcategory(*category=None*, *join=False*)

Return `category` or `self` if `category` is `None`.

INPUT:

- `category` – a sub category of `self`, tuple/list thereof, or `None`
- `join` – boolean (default: `False`)

OUTPUT: a category

EXAMPLES:

```
sage: Monoids().or_subcategory(Groups())
Category of groups
sage: Monoids().or_subcategory(None)
Category of monoids
```

```
>>> from sage.all import *
>>> Monoids().or_subcategory(Groups())
Category of groups
>>> Monoids().or_subcategory(None)
Category of monoids
```

If category is a list/tuple, then a join category is returned:

```
sage: Monoids().or_subcategory((CommutativeAdditiveMonoids(), Groups()))
Join of Category of groups and Category of commutative additive monoids
```

```
>>> from sage.all import *
>>> Monoids().or_subcategory((CommutativeAdditiveMonoids(), Groups()))
Join of Category of groups and Category of commutative additive monoids
```

If `join` is `False`, an error if raised if category is not a subcategory of `self`:

```
sage: Monoids().or_subcategory(EnumeratedSets())
Traceback (most recent call last):
...
ValueError: Subcategory of `Category of monoids` required;
got `Category of enumerated sets`
```

```
>>> from sage.all import *
>>> Monoids().or_subcategory(EnumeratedSets())
Traceback (most recent call last):
...
ValueError: Subcategory of `Category of monoids` required;
got `Category of enumerated sets`
```

Otherwise, the two categories are joined together:

```
sage: Monoids().or_subcategory(EnumeratedSets(), join=True)
Category of enumerated monoids
```

```
>>> from sage.all import *
>>> Monoids().or_subcategory(EnumeratedSets(), join=True)
Category of enumerated monoids
```

parent_class()

A common super class for all parents in this category (and its subcategories).

This class contains the methods defined in the nested class `self.ParentMethods` (if it exists), and has as bases the parent classes of the super categories of `self`.

See also

- *element_class()*, *morphism_class()*
- *Category* for details

EXAMPLES:

```
sage: C = Algebras(QQ).parent_class; C
<class 'sage.categories.algebras.Algebras.parent_class'>
sage: type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).parent_class; C
<class 'sage.categories.algebras.Algebras.parent_class'>
>>> type(C)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

By Issue #11935, some categories share their parent classes. For example, the parent class of an algebra only depends on the category of the base ring. A typical example is the category of algebras over a finite field versus algebras over a non-field:

```
sage: Algebras(GF(7)).parent_class is Algebras(GF(5)).parent_class
True
sage: Algebras(QQ).parent_class is Algebras(ZZ).parent_class
False
sage: Algebras(ZZ['t']).parent_class is Algebras(ZZ['t','x']).parent_class
True
```

```
>>> from sage.all import *
>>> Algebras(GF(Integer(7))).parent_class is Algebras(GF(Integer(5))).parent_
↪class
True
>>> Algebras(QQ).parent_class is Algebras(ZZ).parent_class
False
>>> Algebras(ZZ['t']).parent_class is Algebras(ZZ['t','x']).parent_class
True
```

See *CategoryWithParameters* for an abstract base class for categories that depend on parameters, even though the parent and element classes only depend on the parent or element classes of its super categories. It is used in *Bimodules*, *Category_over_base* and *sage.categories.category.JoinCategory*.

Note

See the note about *_test_category_graph()* regarding Python class hierarchy.

required_methods()

Return the methods that are required and optional for parents in this category and their elements.

EXAMPLES:

```
sage: Algebras(QQ).required_methods()
{'element': {'optional': ['_add_', '_mul_'], 'required': ['__bool__']},
 'parent': {'optional': ['algebra_generators'], 'required': ['__contains__']}}
```

```
>>> from sage.all import *
>>> Algebras(QQ).required_methods()
{'element': {'optional': ['_add_', '_mul_'], 'required': ['__bool__']},
 'parent': {'optional': ['algebra_generators'], 'required': ['__contains__']}}
```

structure()

Return the structure `self` is endowed with.

This method returns the structure that morphisms in this category shall be preserving. For example, it tells that a ring is a set endowed with a structure of both a unital magma and an additive unital magma which satisfies some further axioms. In other words, a ring morphism is a function that preserves the unital magma and additive unital magma structure.

In practice, this returns the collection of all the super categories of `self` that define some additional structure, as a frozen set.

EXAMPLES:

```
sage: Objects().structure()
frozenset()

sage: def structure(C):
....:     return Category._sort(C.structure())

sage: structure(Sets())
(Category of sets, Category of sets with partial maps)
sage: structure(Magmas())
(Category of magmas, Category of sets, Category of sets with partial maps)
```

```
>>> from sage.all import *
>>> Objects().structure()
frozenset()

>>> def structure(C):
...     return Category._sort(C.structure())

>>> structure(Sets())
(Category of sets, Category of sets with partial maps)
>>> structure(Magmas())
(Category of magmas, Category of sets, Category of sets with partial maps)
```

In the following example, we only list the smallest structure categories to get a more readable output:

```
sage: def structure(C):
....:     return Category._sort_uniq(C.structure())

sage: structure(Magmas())
(Category of magmas,)
sage: structure(Rings())
(Category of unital magmas, Category of additive unital additive magmas)
```

(continues on next page)

(continued from previous page)

```
sage: structure(Fields())
(Category of euclidean domains, Category of noetherian rings)
sage: structure(Algebras(QQ))
(Category of unital magmas,
 Category of right modules over Rational Field,
 Category of left modules over Rational Field)
sage: structure(HopfAlgebras(QQ).Graded().WithBasis().Connected())
(Category of Hopf algebras over Rational Field,
 Category of graded modules over Rational Field)
```

```
>>> from sage.all import *
>>> def structure(C):
...     return Category._sort_uniq(C.structure())

>>> structure(Magmas())
(Category of magmas,)
>>> structure(Rings())
(Category of unital magmas, Category of additive unital additive magmas)
>>> structure(Fields())
(Category of euclidean domains, Category of noetherian rings)
>>> structure(Algebras(QQ))
(Category of unital magmas,
 Category of right modules over Rational Field,
 Category of left modules over Rational Field)
>>> structure(HopfAlgebras(QQ).Graded().WithBasis().Connected())
(Category of Hopf algebras over Rational Field,
 Category of graded modules over Rational Field)
```

This method is used in *is_full_subcategory()* for deciding whether a category is a full subcategory of some other category, and for documentation purposes. It is computed recursively from the result of *additional_structure()* on the super categories of `self`.

subcategory_class()

A common superclass for all subcategories of this category (including this one).

This class derives from `D.subcategory_class` for each super category D of `self`, and includes all the methods from the nested class `self.SubcategoryMethods`, if it exists.

See also

- Issue #12895
- *parent_class()*
- *element_class()*
- *_make_named_class()*

EXAMPLES:

```
sage: cls = Rings().subcategory_class; cls
<class 'sage.categories.rings.Rings.subcategory_class'>
```

(continues on next page)

(continued from previous page)

```
sage: type(cls)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

```
>>> from sage.all import *
>>> cls = Rings().subcategory_class; cls
<class 'sage.categories.rings.Rings.subcategory_class'>
>>> type(cls)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
```

`Rings()` is an instance of this class, as well as all its subcategories:

```
sage: isinstance(Rings(), cls)
True
sage: isinstance(AlgebrasWithBasis(QQ), cls)
True
```

```
>>> from sage.all import *
>>> isinstance(Rings(), cls)
True
>>> isinstance(AlgebrasWithBasis(QQ), cls)
True
```

Note

See the note about *`_test_category_graph()`* regarding Python class hierarchy.

super_categories()

Return the *immediate* super categories of `self`.

OUTPUT: a duplicate-free list of categories

Every category should implement this method.

EXAMPLES:

```
sage: Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
sage: Objects().super_categories()
[]
```

```
>>> from sage.all import *
>>> Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
>>> Objects().super_categories()
[]
```

Note

Since Issue #10963, the order of the categories in the result is irrelevant. For details, see *On the order of super categories*.

> **Note**
>
> Whenever speed matters, developers are advised to use the lazy attribute *_super_categories()* instead of calling this method.

class sage.categories.category.**CategoryWithParameters**

Bases: *Category*

A parametrized category whose parent/element classes depend only on its super categories.

Many categories in Sage are parametrized, like `C = Algebras(K)` which takes a base ring as parameter. In many cases, however, the operations provided by `C` in the parent class and element class depend only on the super categories of `C`. For example, the vector space operations are provided if and only if `K` is a field, since `VectorSpaces(K)` is a super category of `C` only in that case. In such cases, and as an optimization (see Issue #11935), we want to use the same parent and element class for all fields. This is the purpose of this abstract class.

Currently, *JoinCategory*, *Category_over_base* and *Bimodules* inherit from this class.

EXAMPLES:

```
sage: C1 = Algebras(GF(5))
sage: C2 = Algebras(GF(3))
sage: C3 = Algebras(ZZ)
sage: from sage.categories.category import CategoryWithParameters
sage: isinstance(C1, CategoryWithParameters)
True
sage: C1.parent_class is C2.parent_class
True
sage: C1.parent_class is C3.parent_class
False
```

```
>>> from sage.all import *
>>> C1 = Algebras(GF(Integer(5)))
>>> C2 = Algebras(GF(Integer(3)))
>>> C3 = Algebras(ZZ)
>>> from sage.categories.category import CategoryWithParameters
>>> isinstance(C1, CategoryWithParameters)
True
>>> C1.parent_class is C2.parent_class
True
>>> C1.parent_class is C3.parent_class
False
```

Category.**_make_named_class**(*name*, *method_provider*, *cache=False*, *picklable=True*)

Construction of the parent/element/... class of `self`.

INPUT:

- `name` – string; the name of the class as an attribute of `self` (e.g. "parent_class")
- `method_provider` – string; the name of an attribute of `self` that provides methods for the new class (in addition to those coming from the super categories, e.g. "ParentMethods")
- `cache` – boolean or `ignore_reduction` (default: `False`) (passed down to dynamic_class; for internal use only)
- `picklable` – boolean (default: `True`)

ASSUMPTION:

It is assumed that this method is only called from a lazy attribute whose name coincides with the given `name`.

OUTPUT:

A dynamic class with bases given by the corresponding named classes of `self`'s super_categories, and methods taken from the class `getattr(self,method_provider)`.

Note

- In this default implementation, the reduction data of the named class makes it depend on `self`. Since the result is going to be stored in a lazy attribute of `self` anyway, we may as well disable the caching in `dynamic_class` (hence the default value `cache=False`).
- *CategoryWithParameters* overrides this method so that the same parent/element/... classes can be shared between closely related categories.
- The bases of the named class may also contain the named classes of some indirect super categories, according to *_super_categories_for_classes()*. This is to guarantee that Python will build consistent method resolution orders. For background, see `sage.misc.c3_controlled`.

See also

`CategoryWithParameters._make_named_class`

EXAMPLES:

```
sage: PC = Rings()._make_named_class("parent_class", "ParentMethods"); PC
<class 'sage.categories.rings.Rings.parent_class'>
sage: type(PC)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
sage: PC.__bases__
(<class 'sage.categories.rngs.Rngs.parent_class'>,
 <class 'sage.categories.semirings.Semirings.parent_class'>)
```

```
>>> from sage.all import *
>>> PC = Rings()._make_named_class("parent_class", "ParentMethods"); PC
<class 'sage.categories.rings.Rings.parent_class'>
>>> type(PC)
<class 'sage.structure.dynamic_class.DynamicMetaclass'>
>>> PC.__bases__
(<class 'sage.categories.rngs.Rngs.parent_class'>,
 <class 'sage.categories.semirings.Semirings.parent_class'>)
```

Note that, by default, the result is not cached:

```
sage: PC is Rings()._make_named_class("parent_class", "ParentMethods")
False
```

```
>>> from sage.all import *
>>> PC is Rings()._make_named_class("parent_class", "ParentMethods")
False
```

Indeed this method is only meant to construct lazy attributes like `parent_class` which already handle this caching:

```
sage: Rings().parent_class
<class 'sage.categories.rings.Rings.parent_class'>
```

```
>>> from sage.all import *
>>> Rings().parent_class
<class 'sage.categories.rings.Rings.parent_class'>
```

Reduction for pickling also assumes the existence of this lazy attribute:

```
sage: PC._reduction
(<built-in function getattr>, (Category of rings, 'parent_class'))
sage: loads(dumps(PC)) is Rings().parent_class
True
```

```
>>> from sage.all import *
>>> PC._reduction
(<built-in function getattr>, (Category of rings, 'parent_class'))
>>> loads(dumps(PC)) is Rings().parent_class
True
```

class sage.categories.category.**JoinCategory**(*super_categories*, ***kwds*)

Bases: *CategoryWithParameters*

A class for joins of several categories. Do not use directly; see Category.join instead.

EXAMPLES:

```
sage: from sage.categories.category import JoinCategory
sage: J = JoinCategory((Groups(), CommutativeAdditiveMonoids())); J
Join of Category of groups and Category of commutative additive monoids
sage: J.super_categories()
[Category of groups, Category of commutative additive monoids]
sage: J.all_super_categories(proper=True)
[Category of groups, ..., Category of magmas,
 Category of commutative additive monoids, ..., Category of additive magmas,
 Category of sets, Category of sets with partial maps, Category of objects]
```

```
>>> from sage.all import *
>>> from sage.categories.category import JoinCategory
>>> J = JoinCategory((Groups(), CommutativeAdditiveMonoids())); J
Join of Category of groups and Category of commutative additive monoids
>>> J.super_categories()
[Category of groups, Category of commutative additive monoids]
>>> J.all_super_categories(proper=True)
[Category of groups, ..., Category of magmas,
 Category of commutative additive monoids, ..., Category of additive magmas,
 Category of sets, Category of sets with partial maps, Category of objects]
```

By Issue #11935, join categories and categories over base rings inherit from *CategoryWithParameters*. This allows for sharing parent and element classes between similar categories. For example, since group algebras belong to a join category and since the underlying implementation is the same for all finite fields, we have:

```
sage: # needs sage.groups sage.rings.finite_rings
sage: G = SymmetricGroup(10)
sage: A3 = G.algebra(GF(3))
sage: A5 = G.algebra(GF(5))
sage: type(A3.category())
<class 'sage.categories.category.JoinCategory_with_category'>
sage: type(A3) is type(A5)
True
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.rings.finite_rings
>>> G = SymmetricGroup(Integer(10))
>>> A3 = G.algebra(GF(Integer(3)))
>>> A5 = G.algebra(GF(Integer(5)))
>>> type(A3.category())
<class 'sage.categories.category.JoinCategory_with_category'>
>>> type(A3) is type(A5)
True
```

Category.**_repr_object_names**()

Return the name of the objects of this category.

EXAMPLES:

```
sage: FiniteGroups()._repr_object_names()
'finite groups'
sage: AlgebrasWithBasis(QQ)._repr_object_names()
'algebras with basis over Rational Field'
```

```
>>> from sage.all import *
>>> FiniteGroups()._repr_object_names()
'finite groups'
>>> AlgebrasWithBasis(QQ)._repr_object_names()
'algebras with basis over Rational Field'
```

Category.**_repr_**()

Return the print representation of this category.

EXAMPLES:

```
sage: Sets() # indirect doctest
Category of sets
```

```
>>> from sage.all import *
>>> Sets() # indirect doctest
Category of sets
```

Category.**_without_axioms**(*named=False*)

Return the category without the axioms that have been added to create it.

INPUT:

- `named` – boolean (default: `False`)

Todo

Improve this explanation.

If `named` is `True`, then this stops at the first category that has an explicit name of its own. See *category_with_axiom.CategoryWithAxiom._without_axioms()*

EXAMPLES:

```
sage: Sets()._without_axioms()
Category of sets
sage: Semigroups()._without_axioms()
Category of magmas
sage: Algebras(QQ).Commutative().WithBasis()._without_axioms()
Category of magmatic algebras over Rational Field
sage: Algebras(QQ).Commutative().WithBasis()._without_axioms(named=True)
Category of algebras over Rational Field
```

```
>>> from sage.all import *
>>> Sets()._without_axioms()
Category of sets
>>> Semigroups()._without_axioms()
Category of magmas
>>> Algebras(QQ).Commutative().WithBasis()._without_axioms()
Category of magmatic algebras over Rational Field
>>> Algebras(QQ).Commutative().WithBasis()._without_axioms(named=True)
Category of algebras over Rational Field
```

additional_structure()

Return `None`.

Indeed, a join category defines no additional structure.

See also

Category.additional_structure()

EXAMPLES:

```
sage: Modules(ZZ).additional_structure()
```

```
>>> from sage.all import *
>>> Modules(ZZ).additional_structure()
```

is_subcategory(*C*)

Check whether this join category is subcategory of another category `C`.

EXAMPLES:

```
sage: Category.join([Rings(),Modules(QQ)]).is_subcategory(Category.
↪join([Rngs(),Bimodules(QQ,QQ)]))
True
```

```
>>> from sage.all import *
>>> Category.join([Rings(),Modules(QQ)]).is_subcategory(Category.join([Rngs(),
↪Bimodules(QQ,QQ)]))
True
```

super_categories()

Return the immediate super categories, as per *Category.super_categories()*.

EXAMPLES:

```
sage: from sage.categories.category import JoinCategory
sage: JoinCategory((Semigroups(), FiniteEnumeratedSets())).super_categories()
[Category of semigroups, Category of finite enumerated sets]
```

```
>>> from sage.all import *
>>> from sage.categories.category import JoinCategory
>>> JoinCategory((Semigroups(), FiniteEnumeratedSets())).super_categories()
[Category of semigroups, Category of finite enumerated sets]
```

sage.categories.category.**category_graph**(*categories=None*)

Return the graph of the categories in Sage.

INPUT:

- `categories` – list (or iterable) of categories

If `categories` is specified, then the graph contains the mentioned categories together with all their super categories. Otherwise the graph contains (an instance of) each category in *sage.categories.all* (e.g. `Algebras(QQ)` for algebras).

For readability, the names of the category are shortened.

Todo

Further remove the base ring (see also Issue #15801).

EXAMPLES:

```
sage: G = sage.categories.category.category_graph(categories=[Groups()])          #␣
↪needs sage.graphs
sage: G.vertices(sort=True)                                                       #␣
↪needs sage.graphs
['groups', 'inverse unital magmas', 'magmas', 'monoids', 'objects',
 'semigroups', 'sets', 'sets with partial maps', 'unital magmas']
sage: G.plot()                                                                    #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 20 graphics primitives

sage: sage.categories.category.category_graph().plot()                            #␣
↪needs sage.graphs sage.groups sage.plot
Graphics object consisting of ... graphics primitives
```

```
>>> from sage.all import *
>>> G = sage.categories.category.category_graph(categories=[Groups()])        #␣
↪needs sage.graphs
>>> G.vertices(sort=True)                                                     #␣
↪needs sage.graphs
['groups', 'inverse unital magmas', 'magmas', 'monoids', 'objects',
 'semigroups', 'sets', 'sets with partial maps', 'unital magmas']
>>> G.plot()                                                                  #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 20 graphics primitives

>>> sage.categories.category.category_graph().plot()                          #␣
↪needs sage.graphs sage.groups sage.plot
Graphics object consisting of ... graphics primitives
```

sage.categories.category.**category_sample**()

Return a sample of categories.

It is constructed by looking for all concrete category classes declared in `sage.categories.all`, calling *Category.an_instance()* on those and taking all their super categories.

EXAMPLES:

```
sage: from sage.categories.category import category_sample
sage: sorted(category_sample(), key=str)                                      #␣
↪needs sage.groups
[Category of Coxeter groups,
 Category of Dedekind domains,
 Category of G-sets for Symmetric group of order 8! as a permutation group,
 Category of Hecke modules over Rational Field,
 Category of Hopf algebras over Rational Field,
 Category of Hopf algebras with basis over Rational Field,
 Category of Jacobians over Rational Field,
 Category of Lie algebras over Rational Field,
 Category of Weyl groups,
 Category of abelian varieties over Rational Field,
 Category of additive magmas, ...,
 Category of fields, ...,
 Category of graded Hopf algebras with basis over Rational Field, ...,
 Category of modular abelian varieties over Rational Field, ...,
 Category of simplicial complexes, ...,
 Category of vector spaces over Rational Field, ...
```

```
>>> from sage.all import *
>>> from sage.categories.category import category_sample
>>> sorted(category_sample(), key=str)                                        #␣
↪needs sage.groups
[Category of Coxeter groups,
 Category of Dedekind domains,
 Category of G-sets for Symmetric group of order 8! as a permutation group,
 Category of Hecke modules over Rational Field,
 Category of Hopf algebras over Rational Field,
 Category of Hopf algebras with basis over Rational Field,
```

(continues on next page)

(continued from previous page)

```
Category of Jacobians over Rational Field,
Category of Lie algebras over Rational Field,
Category of Weyl groups,
Category of abelian varieties over Rational Field,
Category of additive magmas, ...,
Category of fields, ...,
Category of graded Hopf algebras with basis over Rational Field, ...,
Category of modular abelian varieties over Rational Field, ...,
Category of simplicial complexes, ...,
Category of vector spaces over Rational Field, ...
```

2.4 Axioms

This documentation covers how to implement axioms and proceeds with an overview of the implementation of the axiom infrastructure. It assumes that the reader is familiar with the *category primer*, and in particular its *section about axioms*.

2.4.1 Implementing axioms

Simple case involving a single predefined axiom

Suppose that one wants to provide code (and documentation, tests, …) for the objects of some existing category `Cs()` that satisfy some predefined axiom `A`.

The first step is to open the hood and check whether there already exists a class implementing the category `Cs().A()`. For example, taking `Cs=Semigroups` and the `Finite` axiom, there already exists a class for the category of finite semigroups:

```
sage: Semigroups().Finite()
Category of finite semigroups
sage: type(Semigroups().Finite())
<class 'sage.categories.finite_semigroups.FiniteSemigroups_with_category'>
```

```
>>> from sage.all import *
>>> Semigroups().Finite()
Category of finite semigroups
>>> type(Semigroups().Finite())
<class 'sage.categories.finite_semigroups.FiniteSemigroups_with_category'>
```

In this case, we say that the category of semigroups *implements* the axiom `Finite`, and code about finite semigroups should go in the class *`FiniteSemigroups`* (or, as usual, in its nested classes `ParentMethods`, `ElementMethods`, and so on).

On the other hand, there is no class for the category of infinite semigroups:

```
sage: Semigroups().Infinite()
Category of infinite semigroups
sage: type(Semigroups().Infinite())
<class 'sage.categories.category.JoinCategory_with_category'>
```

```
>>> from sage.all import *
>>> Semigroups().Infinite()
Category of infinite semigroups
```

(continues on next page)

(continued from previous page)

```
>>> type(Semigroups().Infinite())
<class 'sage.categories.category.JoinCategory_with_category'>
```

This category is indeed just constructed as the intersection of the categories of semigroups and of infinite sets respectively:

```
sage: Semigroups().Infinite().super_categories()
[Category of semigroups, Category of infinite sets]
```

```
>>> from sage.all import *
>>> Semigroups().Infinite().super_categories()
[Category of semigroups, Category of infinite sets]
```

In this case, one needs to create a new class to implement the axiom `Infinite` for this category. This boils down to adding a nested class `Semigroups.Infinite` inheriting from *CategoryWithAxiom*.

In the following example, we implement a category `Cs`, with a subcategory for the objects satisfying the `Finite` axiom defined in the super category `Sets` (we will see later on how to *define* new axioms):

```
sage: from sage.categories.category_with_axiom import CategoryWithAxiom
sage: class Cs(Category):
....:     def super_categories(self):
....:         return [Sets()]
....:     class Finite(CategoryWithAxiom):
....:         class ParentMethods:
....:             def foo(self):
....:                 print("I am a method on finite C's")
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import CategoryWithAxiom
>>> class Cs(Category):
...     def super_categories(self):
...         return [Sets()]
...     class Finite(CategoryWithAxiom):
...         class ParentMethods:
...             def foo(self):
...                 print("I am a method on finite C's")
```

```
sage: Cs().Finite()
Category of finite cs
sage: Cs().Finite().super_categories()
[Category of finite sets, Category of cs]
sage: Cs().Finite().all_super_categories()
[Category of finite cs, Category of finite sets,
 Category of cs, Category of sets, ...]
sage: Cs().Finite().axioms()
frozenset({'Finite'})
```

```
>>> from sage.all import *
>>> Cs().Finite()
Category of finite cs
>>> Cs().Finite().super_categories()
[Category of finite sets, Category of cs]
```

(continues on next page)

(continued from previous page)

```
>>> Cs().Finite().all_super_categories()
[Category of finite cs, Category of finite sets,
 Category of cs, Category of sets, ...]
>>> Cs().Finite().axioms()
frozenset({'Finite'})
```

Now a parent declared in the category `Cs().Finite()` inherits from all the methods of finite sets and of finite C's, as desired:

```
sage: P = Parent(category=Cs().Finite())
sage: P.is_finite()             # Provided by Sets.Finite.ParentMethods
True
sage: P.foo()                   # Provided by Cs.Finite.ParentMethods
I am a method on finite C's
```

```
>>> from sage.all import *
>>> P = Parent(category=Cs().Finite())
>>> P.is_finite()             # Provided by Sets.Finite.ParentMethods
True
>>> P.foo()                   # Provided by Cs.Finite.ParentMethods
I am a method on finite C's
```

Note

- This follows the same idiom as for *Covariant Functorial Constructions*.
- From an object oriented point of view, any subcategory `Cs()` of *`Sets`* inherits a `Finite` method. Usually `Cs` could complement this method by overriding it with a method `Cs.Finite` which would make a super call to `Sets.Finite` and then do extra stuff.

  In the above example, `Cs` also wants to complement `Sets.Finite`, though not by doing more stuff, but by providing it with an additional mixin class containing the code for finite `Cs`. To keep the analogy, this mixin class is to be put in `Cs.Finite`.
- By defining the axiom `Finite`, *`Sets`* fixes the semantic of `Cs.Finite()` for all its subcategories `Cs`: namely "the category of `Cs` which are finite as sets". Hence, for example, `Modules.Free.Finite` cannot be used to model the category of free modules of finite rank, even though their traditional name "finite free modules" might suggest it.
- It may come as a surprise that we can actually use the same name `Finite` for the mixin class and for the method defining the axiom; indeed, by default a class does not have a binding behavior and would completely override the method. See the section *Defining a new axiom* for details and the rationale behind it.

  An alternative would have been to give another name to the mixin class, like `FiniteCategory`. However this would have resulted in more namespace pollution, whereas using `Finite` is already clear, explicit, and easier to remember.
- Under the hood, the category `Cs().Finite()` is aware that it has been constructed from the category `Cs()` by adding the axiom `Finite`:

  ```
  sage: Cs().Finite().base_category()
  Category of cs
  sage: Cs().Finite()._axiom
  'Finite'
  ```

```
>>> from sage.all import *
>>> Cs().Finite().base_category()
Category of cs
>>> Cs().Finite()._axiom
'Finite'
```

Over time, the nested class `Cs.Finite` may become large and too cumbersome to keep as a nested subclass of `Cs`. Or the category with axiom may have a name of its own in the literature, like *semigroups* rather than *associative magmas*, or *fields* rather than *commutative division rings*. In this case, the category with axiom can be put elsewhere, typically in a separate file, with just a link from `Cs`:

```
sage: class Cs(Category):
....:     def super_categories(self):
....:         return [Sets()]
sage: class FiniteCs(CategoryWithAxiom):
....:     class ParentMethods:
....:         def foo(self):
....:             print("I am a method on finite C's")
sage: Cs.Finite = FiniteCs
sage: Cs().Finite()
Category of finite cs
```

```
>>> from sage.all import *
>>> class Cs(Category):
...     def super_categories(self):
...         return [Sets()]
>>> class FiniteCs(CategoryWithAxiom):
...     class ParentMethods:
...         def foo(self):
...             print("I am a method on finite C's")
>>> Cs.Finite = FiniteCs
>>> Cs().Finite()
Category of finite cs
```

For a real example, see the code of the class *FiniteGroups* and the link to it in *Groups*. Note that the link is implemented using `LazyImport`; this is highly recommended: it makes sure that *FiniteGroups* is imported after *Groups* it depends upon, and makes it explicit that the class *Groups* can be imported and is fully functional without importing *FiniteGroups*.

Note

Some categories with axioms are created upon Sage's startup. In such a case, one needs to pass the `at_startup=True` option to `LazyImport`, in order to quiet the warning about that lazy import being resolved upon startup. See for example `Sets.Finite`.

This is undoubtedly a code smell. Nevertheless, it is preferable to stick to lazy imports, first to resolve the import order properly, and more importantly as a reminder that the category would be best not constructed upon Sage's startup. This is to spur developers to reduce the number of parents (and therefore categories) that are constructed upon startup. Each `at_startup=True` that will be removed will be a measure of progress in this direction.

Note

In principle, due to a limitation of `LazyImport` with nested classes (see Issue #15648), one should pass the option `as_name` to `LazyImport`:

```
Finite = LazyImport('sage.categories.finite_groups', 'FiniteGroups',
                    as_name='Finite')
```

in order to prevent `Groups.Finite` to keep on reimporting `FiniteGroups`.

Given that passing this option introduces some redundancy and is error prone, the axiom infrastructure includes a little workaround which makes the `as_name` unnecessary in this case.

Making the category with axiom directly callable

If desired, a category with axiom can be constructed directly through its class rather than through its base category:

```
sage: Semigroups()
Category of semigroups
sage: Semigroups() is Magmas().Associative()
True

sage: FiniteGroups()
Category of finite groups
sage: FiniteGroups() is Groups().Finite()
True
```

```
>>> from sage.all import *
>>> Semigroups()
Category of semigroups
>>> Semigroups() is Magmas().Associative()
True

>>> FiniteGroups()
Category of finite groups
>>> FiniteGroups() is Groups().Finite()
True
```

For this notation to work, the class *Semigroups* needs to be aware of the base category class (here, *Magmas*) and of the axiom (here, `Associative`):

```
sage: Semigroups._base_category_class_and_axiom
(<class 'sage.categories.magmas.Magmas'>, 'Associative')
sage: Fields._base_category_class_and_axiom
(<class 'sage.categories.division_rings.DivisionRings'>, 'Commutative')
sage: FiniteGroups._base_category_class_and_axiom
(<class 'sage.categories.groups.Groups'>, 'Finite')
sage: FiniteDimensionalAlgebrasWithBasis._base_category_class_and_axiom
(<class 'sage.categories.algebras_with_basis.AlgebrasWithBasis'>, 'FiniteDimensional')
```

```
>>> from sage.all import *
>>> Semigroups._base_category_class_and_axiom
(<class 'sage.categories.magmas.Magmas'>, 'Associative')
```

(continues on next page)

(continued from previous page)

```
>>> Fields._base_category_class_and_axiom
(<class 'sage.categories.division_rings.DivisionRings'>, 'Commutative')
>>> FiniteGroups._base_category_class_and_axiom
(<class 'sage.categories.groups.Groups'>, 'Finite')
>>> FiniteDimensionalAlgebrasWithBasis._base_category_class_and_axiom
(<class 'sage.categories.algebras_with_basis.AlgebrasWithBasis'>, 'FiniteDimensional')
```

In our example, the attribute `_base_category_class_and_axiom` was set upon calling `Cs().Finite()`, which makes the notation seemingly work:

```
sage: FiniteCs()
Category of finite cs
sage: FiniteCs._base_category_class_and_axiom
(<class '__main__.Cs'>, 'Finite')
sage: FiniteCs._base_category_class_and_axiom_origin
'set by __classget__'
```

```
>>> from sage.all import *
>>> FiniteCs()
Category of finite cs
>>> FiniteCs._base_category_class_and_axiom
(<class '__main__.Cs'>, 'Finite')
>>> FiniteCs._base_category_class_and_axiom_origin
'set by __classget__'
```

But calling `FiniteCs()` right after defining the class would have failed (try it!). In general, one needs to set the attribute explicitly:

```
sage: class FiniteCs(CategoryWithAxiom):
....:     _base_category_class_and_axiom = (Cs, 'Finite')
....:     class ParentMethods:
....:         def foo(self):
....:             print("I am a method on finite C's")
```

```
>>> from sage.all import *
>>> class FiniteCs(CategoryWithAxiom):
...     _base_category_class_and_axiom = (Cs, 'Finite')
...     class ParentMethods:
...         def foo(self):
...             print("I am a method on finite C's")
```

Having to set explicitly this link back from `FiniteCs` to `Cs` introduces redundancy in the code. It would therefore be desirable to have the infrastructure set the link automatically instead (a difficulty is to achieve this while supporting lazy imported categories with axiom).

As a first step, the link is set automatically upon accessing the class from the base category class:

```
sage: Algebras.WithBasis._base_category_class_and_axiom
(<class 'sage.categories.algebras.Algebras'>, 'WithBasis')
sage: Algebras.WithBasis._base_category_class_and_axiom_origin
'set by __classget__'
```

```
>>> from sage.all import *
>>> Algebras.WithBasis._base_category_class_and_axiom
(<class 'sage.categories.algebras.Algebras'>, 'WithBasis')
>>> Algebras.WithBasis._base_category_class_and_axiom_origin
'set by __classget__'
```

Hence, for whatever this notation is worth, one can currently do:

```
sage: Algebras.WithBasis(QQ)
Category of algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Algebras.WithBasis(QQ)
Category of algebras with basis over Rational Field
```

We don't recommend using syntax like `Algebras.WithBasis(QQ)`, as it may eventually be deprecated.

As a second step, Sage tries some obvious heuristics to deduce the link from the name of the category with axiom (see *base_category_class_and_axiom()* for the details). This typically covers the following examples:

```
sage: FiniteCoxeterGroups()
Category of finite Coxeter groups
sage: FiniteCoxeterGroups() is CoxeterGroups().Finite()
True
sage: FiniteCoxeterGroups._base_category_class_and_axiom_origin
'deduced by base_category_class_and_axiom'

sage: FiniteDimensionalAlgebrasWithBasis(QQ)
Category of finite dimensional algebras with basis over Rational Field
sage: FiniteDimensionalAlgebrasWithBasis(QQ) is Algebras(QQ).FiniteDimensional().
↪WithBasis()
True
```

```
>>> from sage.all import *
>>> FiniteCoxeterGroups()
Category of finite Coxeter groups
>>> FiniteCoxeterGroups() is CoxeterGroups().Finite()
True
>>> FiniteCoxeterGroups._base_category_class_and_axiom_origin
'deduced by base_category_class_and_axiom'

>>> FiniteDimensionalAlgebrasWithBasis(QQ)
Category of finite dimensional algebras with basis over Rational Field
>>> FiniteDimensionalAlgebrasWithBasis(QQ) is Algebras(QQ).FiniteDimensional().
↪WithBasis()
True
```

If the heuristic succeeds, the result is guaranteed to be correct. If it fails, typically because the category has a name of its own like *Fields*, the attribute `_base_category_class_and_axiom` should be set explicitly. For more examples, see the code of the classes *Semigroups* or *Fields*.

Note

When printing out a category with axiom, the heuristic determines whether a category has a name of its own by checking out how `_base_category_class_and_axiom` was set:

```
sage: Fields._base_category_class_and_axiom_origin
'hardcoded'
```

```
>>> from sage.all import *
>>> Fields._base_category_class_and_axiom_origin
'hardcoded'
```

See *CategoryWithAxiom._without_axioms()*, *CategoryWithAxiom._repr_object_names_static()*.

In our running example `FiniteCs`, Sage failed to deduce automatically the base category class and axiom because the class `Cs` is not in the standard location `sage.categories.cs`.

Design discussion

The above deduction, based on names, is undoubtedly inelegant. But it's safe (either the result is guaranteed to be correct, or an error is raised), it saves on some redundant information, and it is only used for the simple shorthands like `FiniteGroups()` for `Groups().Finite()`. Finally, most if not all of these shorthands are likely to eventually disappear (see Issue #15741 and the *related discussion in the primer*).

Defining a new axiom

We describe now how to define a new axiom. The first step is to figure out the largest category where the axiom makes sense. For example `Sets` for `Finite`, `Magmas` for `Associative`, or `Modules` for `FiniteDimensional`. Here we define the axiom `Green` for the category `Cs` and its subcategories:

```
sage: from sage.categories.category_with_axiom import CategoryWithAxiom
sage: class Cs(Category):
....:     def super_categories(self):
....:         return [Sets()]
....:     class SubcategoryMethods:
....:         def Green(self):
....:             '<documentation of the axiom Green>'
....:             return self._with_axiom("Green")
....:     class Green(CategoryWithAxiom):
....:         class ParentMethods:
....:             def foo(self):
....:                 print("I am a method on green C's")
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import CategoryWithAxiom
>>> class Cs(Category):
...     def super_categories(self):
...         return [Sets()]
...     class SubcategoryMethods:
...         def Green(self):
...             '<documentation of the axiom Green>'
...             return self._with_axiom("Green")
```

(continues on next page)

(continued from previous page)

```
...     class Green(CategoryWithAxiom):
...         class ParentMethods:
...             def foo(self):
...                 print("I am a method on green C's")
```

With the current implementation, the name of the axiom must also be added to a global container:

```
sage: all_axioms = sage.categories.category_with_axiom.all_axioms
sage: all_axioms += ("Green",)
```

```
>>> from sage.all import *
>>> all_axioms = sage.categories.category_with_axiom.all_axioms
>>> all_axioms += ("Green",)
```

We can now use the axiom as usual:

```
sage: Cs().Green()
Category of green cs

sage: P = Parent(category=Cs().Green())
sage: P.foo()
I am a method on green C's
```

```
>>> from sage.all import *
>>> Cs().Green()
Category of green cs

>>> P = Parent(category=Cs().Green())
>>> P.foo()
I am a method on green C's
```

Compared with our first example, the only newcomer is the method `.Green()` that can be used by any subcategory `Ds()` of `Cs()` to add the axiom `Green`. Note that the expression `Ds().Green` always evaluates to this method, regardless of whether `Ds` has a nested class `Ds.Green` or not (an implementation detail):

```
sage: Cs().Green
<bound method Cs.SubcategoryMethods.Green of Category of cs>
```

```
>>> from sage.all import *
>>> Cs().Green
<bound method Cs.SubcategoryMethods.Green of Category of cs>
```

Thanks to this feature (implemented in *CategoryWithAxiom.__classget__()*), the user is systematically referred to the documentation of this method when doing introspection on `Ds().Green`:

```
sage: C = Cs()
sage: C.Green?              # not tested
sage: Cs().Green.__doc__
'<documentation of the axiom Green>'
```

```
>>> from sage.all import *
>>> C = Cs()
```

(continues on next page)

(continued from previous page)

```
>>> C.Green?              # not tested
>>> Cs().Green.__doc__
'<documentation of the axiom Green>'
```

It is therefore the natural spot for the documentation of the axiom.

Note

The presence of the nested class `Green` in `Cs` is currently mandatory even if it is empty.

Todo

Specify whether or not one should systematically use @cached_method in the definition of the axiom. And make sure all the definition of axioms in Sage are consistent in this respect!

Todo

We could possibly define an @axiom decorator? This could hide two little implementation details: whether or not to make the method a cached method, and the call to _with_axiom(…) under the hood. It could do possibly do some more magic. The gain is not obvious though.

Note

`all_axioms` is only used marginally, for sanity checks and when trying to derive automatically the base category class. The order of the axioms in this tuple also controls the order in which they appear when printing out categories with axioms (see *CategoryWithAxiom._repr_object_names_static()*).

During a Sage session, new axioms should only be added at the *end* of `all_axioms`, as above, so as to not break the cache of `axioms_rank`. Otherwise, they can be inserted statically anywhere in the tuple. For axioms defined within the Sage library, the name is best inserted by editing directly the definition of `all_axioms` in *sage.categories.category_with_axiom*.

Design note

Let us state again that, unlike what the existence of `all_axioms` might suggest, the definition of an axiom is local to a category and its subcategories. In particular, two independent categories `Cs()` and `Ds()` can very well define axioms with the same name and different semantics. As long as the two hierarchies of subcategories don't intersect, this is not a problem. And if they do intersect naturally (that is if one is likely to create a parent belonging to both categories), this probably means that the categories `Cs` and `Ds` are about related enough areas of mathematics that one should clear the ambiguity by having either the same semantic or different names.

This caveat is no different from that of name clashes in hierarchy of classes involving multiple inheritance.

Todo

Explore ways to get rid of this global `all_axioms` tuple, and/or have automatic registration there, and/or having a

register_axiom(…) method.

Special case: defining an axiom depending on several categories

In some cases, the largest category where the axiom makes sense is the intersection of two categories. This is typically the case for axioms specifying compatibility conditions between two otherwise unrelated operations, like `Distributive` which specifies a compatibility between $*$ and $+$. Ideally, we would want the `Distributive` axiom to be defined by:

```
sage: Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas
```

```
>>> from sage.all import *
>>> Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas
```

The current infrastructure does not support this perfectly: indeed, defining an axiom for a category C requires C to have a class of its own; hence a *JoinCategory* as above won't do; we need to implement a new class like *MagmasAndAdditiveMagmas*; furthermore, we cannot yet model the fact that `MagmasAndAdditiveMagmas()` *is* the intersection of `Magmas()` and `AdditiveMagmas()` rather than a mere subcategory:

```
sage: from sage.categories.magmas_and_additive_magmas import MagmasAndAdditiveMagmas
sage: Magmas() & AdditiveMagmas() is MagmasAndAdditiveMagmas()
False
sage: Magmas() & AdditiveMagmas()              # todo: not implemented
Category of magmas and additive magmas
```

```
>>> from sage.all import *
>>> from sage.categories.magmas_and_additive_magmas import MagmasAndAdditiveMagmas
>>> Magmas() & AdditiveMagmas() is MagmasAndAdditiveMagmas()
False
>>> Magmas() & AdditiveMagmas()              # todo: not implemented
Category of magmas and additive magmas
```

Still, there is a workaround to get the natural notations:

```
sage: (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
sage: (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings
```

```
>>> from sage.all import *
>>> (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
>>> (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings
```

The trick is to define `Distributive` as usual in *MagmasAndAdditiveMagmas*, and to add a method *Magmas.SubcategoryMethods.Distributive()* which checks that `self` is a subcategory of both `Magmas()` and `AdditiveMagmas()`, complains if not, and otherwise takes the intersection of `self` with `MagmasAndAdditiveMagmas()` before calling `Distributive`.

The downsides of this workaround are:

- Creation of an otherwise empty class *MagmasAndAdditiveMagmas*.

- Pollution of the namespace of `Magmas()` (and subcategories like `Groups()`) with a method that is irrelevant (but safely complains if called).
- `C._with_axiom('Distributive')` is not strictly equivalent to `C.Distributive()`, which can be unpleasantly surprising:

```
sage: (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings

sage: (Monoids() & CommutativeAdditiveGroups())._with_axiom('Distributive')
Join of Category of monoids and Category of commutative additive groups
```

```
>>> from sage.all import *
>>> (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings

>>> (Monoids() & CommutativeAdditiveGroups())._with_axiom('Distributive')
Join of Category of monoids and Category of commutative additive groups
```

Todo

Other categories that would be better implemented via an axiom depending on a join category include:

- *Algebras*: defining an associative unital algebra as a ring and a module satisfying the suitable compatibility axiom between inner multiplication and multiplication by scalars (bilinearity). Of course this should be implemented at the level of *MagmaticAlgebras*, if not higher.
- *Bialgebras*: defining a bialgebra as an algebra and coalgebra where the coproduct is a morphism for the product.
- *Bimodules*: defining a bimodule as a left and right module where the two actions commute.

Todo

- Design and implement an idiom for the definition of an axiom by a join category.
- Or support more advanced joins, through some hook or registration process to specify that a given category *is* the intersection of two (or more) categories.
- Or at least improve the above workaround to avoid the last issue; this possibly could be achieved using a class `Magmas.Distributive` with a bit of `__classcall__` magic.

Handling multiple axioms, arborescence structure of the code

Prelude

Let us consider the category of magmas, together with two of its axioms, namely `Associative` and `Unital`. An associative magma is a *semigroup* and a unital semigroup is a *monoid*. We have also seen that axioms commute:

```
sage: Magmas().Unital()
Category of unital magmas
sage: Magmas().Associative()
Category of semigroups
sage: Magmas().Associative().Unital()
```

(continues on next page)

(continued from previous page)

```
Category of monoids
sage: Magmas().Unital().Associative()
Category of monoids
```

```
>>> from sage.all import *
>>> Magmas().Unital()
Category of unital magmas
>>> Magmas().Associative()
Category of semigroups
>>> Magmas().Associative().Unital()
Category of monoids
>>> Magmas().Unital().Associative()
Category of monoids
```

At the level of the classes implementing these categories, the following comes as a general naturalization of the previous section:

```
sage: Magmas.Unital
<class 'sage.categories.magmas.Magmas.Unital'>
sage: Magmas.Associative
<class 'sage.categories.semigroups.Semigroups'>
sage: Magmas.Associative.Unital
<class 'sage.categories.monoids.Monoids'>
```

```
>>> from sage.all import *
>>> Magmas.Unital
<class 'sage.categories.magmas.Magmas.Unital'>
>>> Magmas.Associative
<class 'sage.categories.semigroups.Semigroups'>
>>> Magmas.Associative.Unital
<class 'sage.categories.monoids.Monoids'>
```

However, the following may look suspicious at first:

```
sage: Magmas.Unital.Associative
Traceback (most recent call last):
...
AttributeError: type object 'Magmas.Unital' has no attribute 'Associative'...
```

```
>>> from sage.all import *
>>> Magmas.Unital.Associative
Traceback (most recent call last):
...
AttributeError: type object 'Magmas.Unital' has no attribute 'Associative'...
```

The purpose of this section is to explain the design of the code layout and the rationale for this mismatch.

Abstract model

As we have seen in the *Primer*, the objects of a category `Cs()` can usually satisfy, or not, many different axioms. Out of all combinations of axioms, only a small number are relevant in practice, in the sense that we actually want to provide features for the objects satisfying these axioms.

Therefore, in the context of the category class `Cs`, we want to provide the system with a collection $(D_S)_{S\in\mathcal{S}}$ where each S is a subset of the axioms and the corresponding D_S is a class for the subcategory of the objects of `Cs()` satisfying the axioms in S. For example, if `Cs()` is the category of magmas, the pairs (S, D_S) would include:

```
{Associative}                 : Semigroups
{Associative, Unital}         : Monoids
{Associative, Unital, Inverse}: Groups
{Associative, Commutative}    : Commutative Semigroups
{Unital,      Inverse}        : Loops
```

Then, given a subset T of axioms, we want the system to be able to select automatically the relevant classes $(D_S)_{S\in\mathcal{S},S\subset T}$, and build from them a category for the objects of `Cs` satisfying the axioms in T, together with its hierarchy of super categories. If T is in the indexing set $\mathcal{S}$, then the class of the resulting category is directly D_T:

```
sage: C = Magmas().Unital().Inverse().Associative(); C
Category of groups
sage: type(C)
<class 'sage.categories.groups.Groups_with_category'>
```

```
>>> from sage.all import *
>>> C = Magmas().Unital().Inverse().Associative(); C
Category of groups
>>> type(C)
<class 'sage.categories.groups.Groups_with_category'>
```

Otherwise, we get a join category:

```
sage: C = Magmas().Infinite().Unital().Associative(); C
Category of infinite monoids
sage: type(C)
<class 'sage.categories.category.JoinCategory_with_category'>
sage: C.super_categories()
[Category of monoids, Category of infinite sets]
```

```
>>> from sage.all import *
>>> C = Magmas().Infinite().Unital().Associative(); C
Category of infinite monoids
>>> type(C)
<class 'sage.categories.category.JoinCategory_with_category'>
>>> C.super_categories()
[Category of monoids, Category of infinite sets]
```

Concrete model as an arborescence of nested classes

We further want the construction to be efficient and amenable to laziness. This led us to the following design decision: the collection $(D_S)_{S\in\mathcal{S}}$ of classes should be structured as an arborescence (or equivalently a *rooted forest*). The root is `Cs`, corresponding to $S = \emptyset$. Any other class D_S should be the child of a single class $D_{S'}$ where S' is obtained from S by removing a single axiom A. Of course, $D_{S'}$ and A are respectively the base category class and axiom of the category with axiom D_S that we have met in the first section.

At this point, we urge the reader to explore the code of *Magmas* and *DistributiveMagmasAndAdditiveMagmas* and see how the arborescence structure on the categories with axioms is reflected by the nesting of category classes.

Discussion of the design

Performance

Thanks to the arborescence structure on subsets of axioms, constructing the hierarchy of categories and computing intersections can be made efficient with, roughly speaking, a linear/quadratic complexity in the size of the involved category hierarchy multiplied by the number of axioms (see Section *Algorithms*). This is to be put in perspective with the manipulation of arbitrary collections of subsets (aka boolean functions) which can easily raise NP-hard problems.

Furthermore, thanks to its locality, the algorithms can be made suitably lazy: in particular, only the involved category classes need to be imported.

Flexibility

This design also brings in quite some flexibility, with the possibility to support features such as defining new axioms depending on other axioms and deduction rules. See below.

Asymmetry

As we have seen at the beginning of this section, this design introduces an asymmetry. It's not so bad in practice, since in most practical cases, we want to work incrementally. It's for example more natural to describe *FiniteFields* as *Fields* with the axiom `Finite` rather than *Magmas* and *AdditiveMagmas* with all (or at least sufficiently many) of the following axioms:

```
sage: sorted(Fields().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Commutative', 'Distributive',
 'Division', 'NoZeroDivisors', 'Unital']
```

```
>>> from sage.all import *
>>> sorted(Fields().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Commutative', 'Distributive',
 'Division', 'NoZeroDivisors', 'Unital']
```

The main limitation is that the infrastructure currently imposes to be incremental by steps of a single axiom.

In practice, among the roughly 60 categories with axioms that are currently implemented in Sage, most admitted a (rather) natural choice of a base category and single axiom to add. For example, one usually thinks more naturally of a monoid as a semigroup which is unital rather than as a unital magma which is associative. Modeling this asymmetry in the code actually brings a bonus: it is used for printing out categories in a (heuristically) mathematician-friendly way:

```
sage: Magmas().Commutative().Associative()
Category of commutative semigroups
```

```
>>> from sage.all import *
>>> Magmas().Commutative().Associative()
Category of commutative semigroups
```

Only in a few cases is a choice made that feels mathematically arbitrary. This is essentially in the chain of nested classes *distributive_magmas_and_additive_magmas.DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital.Associative*.

Placeholder classes

Given that we can only add a single axiom at a time when implementing a *CategoryWithAxiom*, we need to create a few category classes that are just placeholders. For the worst example, see the chain of nested classes *distributive_magmas_and_additive_magmas.DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital.Associative*.

This is suboptimal, but fits within the scope of the axiom infrastructure which is to reduce a potentially exponential number of placeholder category classes to just a couple.

Note also that, in the above example, it's likely that some of the intermediate classes will grow to non placeholder ones, as people will explore more weaker variants of rings.

Mismatch between the arborescence of nested classes and the hierarchy of categories

The fact that the hierarchy relation between categories is not reflected directly as a relation between the classes may sound suspicious at first! However, as mentioned in the primer, this is actually a big selling point of the axioms infrastructure: by calculating automatically the hierarchy relation between categories with axioms one avoids the nightmare of maintaining it by hand. Instead, only a rather minimal number of links needs to be maintained in the code (one per category with axiom).

Besides, with the flexibility introduced by runtime deduction rules (see below), the hierarchy of categories may depend on the parameters of the categories and not just their class. So it's fine to make it clear from the onset that the two relations do not match.

Evolutivity

At this point, the arborescence structure has to be hardcoded by hand with the annoyances we have seen. This does not preclude, in a future iteration, to design and implement some idiom for categories with axioms that adds several axioms at once to a base category; maybe some variation around:

```
class DistributiveMagmasAndAdditiveMagmas:
    ...

    @category_with_axiom(
        AdditiveAssociative,
        AdditiveCommutative,
        AdditiveUnital,
        AdditiveInverse,
        Associative)
    def _(): return LazyImport('sage.categories.rngs', 'Rngs', at_startup=True)
```

or:

```
register_axiom_category(DistributiveMagmasAndAdditiveMagmas,
                        {AdditiveAssociative,
                         AdditiveCommutative,
                         AdditiveUnital,
                         AdditiveInverse,
                         Associative},
                        'sage.categories.rngs', 'Rngs', at_startup=True)
```

The infrastructure would then be in charge of building the appropriate arborescence under the hood. Or rely on some database (see discussion on Issue #10963, in particular at the end of comment 332).

Axioms defined upon other axioms

Sometimes an axiom can only be defined when some other axiom holds. For example, the axiom `NoZeroDivisors` only makes sense if there is a zero, that is if the axiom `AdditiveUnital` holds. Hence, for the category *MagmasAndAdditiveMagmas*, we consider in the abstract model only those subsets of axioms where the presence of `NoZeroDivisors` implies that of `AdditiveUnital`. We also want the axiom to be only available if meaningful:

```
sage: Rings().NoZeroDivisors()
Category of domains
sage: Rings().Commutative().NoZeroDivisors()
Category of integral domains
sage: Semirings().NoZeroDivisors()
Traceback (most recent call last):
...
AttributeError: 'Semirings_with_category' object has no attribute 'NoZeroDivisors'...
```

```
>>> from sage.all import *
>>> Rings().NoZeroDivisors()
Category of domains
>>> Rings().Commutative().NoZeroDivisors()
Category of integral domains
>>> Semirings().NoZeroDivisors()
Traceback (most recent call last):
...
AttributeError: 'Semirings_with_category' object has no attribute 'NoZeroDivisors'...
```

Concretely, this is to be implemented by defining the new axiom in the (`SubcategoryMethods` nested class of the) appropriate category with axiom. For example the axiom `NoZeroDivisors` would be naturally defined in *AdditiveUnital*.

Note

The axiom `NoZeroDivisors` is currently defined in *Rings*, by simple lack of need for the feature; it should be lifted up as soon as relevant, that is when some code will be available for parents with no zero divisors that are not necessarily rings.

Deduction rules

A similar situation is when an axiom `A` of a category `Cs` implies some other axiom `B`, with the same consequence as above on the subsets of axioms appearing in the abstract model. For example, a division ring necessarily has no zero divisors:

```
sage: 'NoZeroDivisors' in Rings().Division().axioms()
True
sage: 'NoZeroDivisors' in Rings().axioms()
False
```

```
>>> from sage.all import *
>>> 'NoZeroDivisors' in Rings().Division().axioms()
True
>>> 'NoZeroDivisors' in Rings().axioms()
False
```

This deduction rule is implemented by the method *extra_super_categories()*:

```
sage: Rings().Division().extra_super_categories()
(Category of domains,)
```

```
>>> from sage.all import *
>>> Rings().Division().extra_super_categories()
(Category of domains,)
```

In general, this is to be implemented by a method `Cs.A.extra_super_categories` returning a tuple `(Cs().B(),)`, or preferably `(Ds().B(),)` where `Ds` is the category defining the axiom `B`.

This follows the same idiom as for deduction rules about functorial constructions (see *extra_super_categories()*). For example, the fact that a Cartesian product of associative magmas (i.e. of semigroups) is an associative magma is implemented in *Semigroups.CartesianProducts.extra_super_categories()*:

```
sage: Magmas().Associative()
Category of semigroups
sage: Magmas().Associative().CartesianProducts().extra_super_categories()
[Category of semigroups]
```

```
>>> from sage.all import *
>>> Magmas().Associative()
Category of semigroups
>>> Magmas().Associative().CartesianProducts().extra_super_categories()
[Category of semigroups]
```

Similarly, the fact that the algebra of a commutative magma is commutative is implemented in *Magmas.Commutative.Algebras.extra_super_categories()*:

```
sage: Magmas().Commutative().Algebras(QQ).extra_super_categories()
[Category of commutative magmas]
```

```
>>> from sage.all import *
>>> Magmas().Commutative().Algebras(QQ).extra_super_categories()
[Category of commutative magmas]
```

Warning

In some situations this idiom is inapplicable as it would require to implement two classes for the same category. This is the purpose of the next section.

Special case

In the previous examples, the deduction rule only had an influence on the super categories of the category with axiom being constructed. For example, when constructing `Rings().Division()`, the rule *extra_super_categories()* simply adds `Rings().NoZeroDivisors()` as a super category thereof.

In some situations this idiom is inapplicable because a class for the category with axiom under construction already exists elsewhere. Take for example Wedderburn's theorem: any finite division ring is commutative, i.e. is a finite field. In other words, `DivisionRings().Finite()` *coincides* with `Fields().Finite()`:

```
sage: DivisionRings().Finite()
Category of finite enumerated fields
```

(continues on next page)

(continued from previous page)

```
sage: DivisionRings().Finite() is Fields().Finite()
True
```

```
>>> from sage.all import *
>>> DivisionRings().Finite()
Category of finite enumerated fields
>>> DivisionRings().Finite() is Fields().Finite()
True
```

Therefore we cannot create a class `DivisionRings.Finite` to hold the desired `extra_super_categories` method, because there is already a class for this category with axiom, namely `Fields.Finite`.

A natural idiom would be to have `DivisionRings.Finite` be a link to `Fields.Finite` (locally introducing an undirected cycle in the arborescence of nested classes). It would be a bit tricky to implement though, since one would need to detect, upon constructing `DivisionRings().Finite()`, that `DivisionRings.Finite` is actually `Fields.Finite`, in order to construct appropriately `Fields().Finite()`; and reciprocally, upon computing the super categories of `Fields().Finite()`, to not try to add `DivisionRings().Finite()` as a super category.

Instead the current idiom is to have a method `DivisionRings.Finite_extra_super_categories` which mimics the behavior of the would-be `DivisionRings.Finite.extra_super_categories`:

```
sage: DivisionRings().Finite_extra_super_categories()
(Category of commutative magmas,)
```

```
>>> from sage.all import *
>>> DivisionRings().Finite_extra_super_categories()
(Category of commutative magmas,)
```

This idiom is admittedly rudimentary, but consistent with how mathematical facts specifying non trivial inclusion relations between categories are implemented elsewhere in the various `extra_super_categories` methods of axiom categories and covariant functorial constructions. Besides, it gives a natural spot (the docstring of the method) to document and test the modeling of the mathematical fact. Finally, Wedderburn's theorem is arguably a theorem about division rings (in the context of division rings, finiteness implies commutativity) and therefore lives naturally in *DivisionRings*.

An alternative would be to implement the category of finite division rings (i.e. finite fields) in a class `DivisionRings.Finite` rather than `Fields.Finite`:

```
sage: from sage.categories.category_with_axiom import CategoryWithAxiom

sage: class MyDivisionRings(Category):
....:     def super_categories(self):
....:         return [Rings()]

sage: class MyFields(Category):
....:     def super_categories(self):
....:         return [MyDivisionRings()]

sage: class MyFiniteFields(CategoryWithAxiom):
....:     _base_category_class_and_axiom = (MyDivisionRings, "Finite")
....:     def extra_super_categories(self): # Wedderburn's theorem
....:         return [MyFields()]

sage: MyDivisionRings.Finite = MyFiniteFields
```

(continues on next page)

(continued from previous page)

```
sage: MyDivisionRings().Finite()
Category of my finite fields
sage: MyFields().Finite() is MyDivisionRings().Finite()
True
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import CategoryWithAxiom

>>> class MyDivisionRings(Category):
...     def super_categories(self):
...         return [Rings()]

>>> class MyFields(Category):
...     def super_categories(self):
...         return [MyDivisionRings()]

>>> class MyFiniteFields(CategoryWithAxiom):
...     _base_category_class_and_axiom = (MyDivisionRings, "Finite")
...     def extra_super_categories(self): # Wedderburn's theorem
...         return [MyFields()]

>>> MyDivisionRings.Finite = MyFiniteFields

>>> MyDivisionRings().Finite()
Category of my finite fields
>>> MyFields().Finite() is MyDivisionRings().Finite()
True
```

In general, if several categories `C1s()`, `C2s()`, ... are mapped to the same category when applying some axiom `A` (that is `C1s().A() == C2s().A() == ...`), then one should be careful to implement this category in a single class `Cs.A`, and set up methods `extra_super_categories` or `A_extra_super_categories` methods as appropriate. Each such method should return something like `[C2s()]` and not `[C2s().A()]` for the latter would likely lead to an infinite recursion.

Design discussion

Supporting similar deduction rules will be an important feature in the future, with quite a few occurrences already implemented in upcoming issues. For the time being though there is a single occurrence of this idiom outside of the tests. So this would be an easy thing to refactor after Issue #10963 if a better idiom is found.

Larger synthetic examples

We now consider some larger synthetic examples to check that the machinery works as expected. Let us start with a category defining a bunch of axioms, using *axiom()* for conciseness (don't do it for real axioms; they deserve a full documentation!):

```
sage: from sage.categories.category_singleton import Category_singleton
sage: from sage.categories.category_with_axiom import axiom
sage: import sage.categories.category_with_axiom
sage: all_axioms = sage.categories.category_with_axiom.all_axioms
```

(continues on next page)

(continued from previous page)

```
sage: all_axioms += ("B","C","D","E","F")

sage: class As(Category_singleton):
....:     def super_categories(self):
....:         return [Objects()]
....:
....:     class SubcategoryMethods:
....:         B = axiom("B")
....:         C = axiom("C")
....:         D = axiom("D")
....:         E = axiom("E")
....:         F = axiom("F")
....:
....:     class B(CategoryWithAxiom):
....:         pass
....:     class C(CategoryWithAxiom):
....:         pass
....:     class D(CategoryWithAxiom):
....:         pass
....:     class E(CategoryWithAxiom):
....:         pass
....:     class F(CategoryWithAxiom):
....:         pass
```

```
>>> from sage.all import *
>>> from sage.categories.category_singleton import Category_singleton
>>> from sage.categories.category_with_axiom import axiom
>>> import sage.categories.category_with_axiom
>>> all_axioms = sage.categories.category_with_axiom.all_axioms
>>> all_axioms += ("B","C","D","E","F")

>>> class As(Category_singleton):
...     def super_categories(self):
...         return [Objects()]
....:
>>>     class SubcategoryMethods:
...         B = axiom("B")
...         C = axiom("C")
...         D = axiom("D")
...         E = axiom("E")
...         F = axiom("F")
....:
>>>     class B(CategoryWithAxiom):
...         pass
...     class C(CategoryWithAxiom):
...         pass
...     class D(CategoryWithAxiom):
...         pass
...     class E(CategoryWithAxiom):
...         pass
...     class F(CategoryWithAxiom):
...         pass
```

Now we construct a subcategory where, by some theorem of William, axioms `B` and `C` together are equivalent to `E` and `F` together:

```
sage: class A1s(Category_singleton):
....:     def super_categories(self):
....:         return [As()]
....:
....:     class B(CategoryWithAxiom):
....:         def C_extra_super_categories(self):
....:             return [As().E(), As().F()]
....:
....:     class E(CategoryWithAxiom):
....:         def F_extra_super_categories(self):
....:             return [As().B(), As().C()]

sage: A1s().B().C()
Category of e f a1s
```

```
>>> from sage.all import *
>>> class A1s(Category_singleton):
...     def super_categories(self):
...         return [As()]
....:
>>>     class B(CategoryWithAxiom):
...         def C_extra_super_categories(self):
...             return [As().E(), As().F()]
....:
>>>     class E(CategoryWithAxiom):
...         def F_extra_super_categories(self):
...             return [As().B(), As().C()]

>>> A1s().B().C()
Category of e f a1s
```

The axioms `B` and `C` do not show up in the name of the obtained category because, for concision, the printing uses some heuristics to not show axioms that are implied by others. But they are satisfied:

```
sage: sorted(A1s().B().C().axioms())
['B', 'C', 'E', 'F']
```

```
>>> from sage.all import *
>>> sorted(A1s().B().C().axioms())
['B', 'C', 'E', 'F']
```

Note also that this is a join category:

```
sage: type(A1s().B().C())
<class 'sage.categories.category.JoinCategory_with_category'>
sage: A1s().B().C().super_categories()
[Category of e a1s,
 Category of f as,
 Category of b a1s,
 Category of c as]
```

```
>>> from sage.all import *
>>> type(A1s().B().C())
<class 'sage.categories.category.JoinCategory_with_category'>
>>> A1s().B().C().super_categories()
[Category of e a1s,
 Category of f as,
 Category of b a1s,
 Category of c as]
```

As desired, William's theorem holds:

```
sage: A1s().B().C() is A1s().E().F()
True
```

```
>>> from sage.all import *
>>> A1s().B().C() is A1s().E().F()
True
```

and propagates appropriately to subcategories:

```
sage: C =  A1s().E().F().D().B().C()
sage: C is A1s().B().C().E().F().D()  # commutativity
True
sage: C is A1s().E().F().E().F().D()  # William's theorem
True
sage: C is A1s().E().E().F().F().D()  # commutativity
True
sage: C is A1s().E().F().D()          # idempotency
True
sage: C is A1s().D().E().F()
True
```

```
>>> from sage.all import *
>>> C =  A1s().E().F().D().B().C()
>>> C is A1s().B().C().E().F().D()  # commutativity
True
>>> C is A1s().E().F().E().F().D()  # William's theorem
True
>>> C is A1s().E().E().F().F().D()  # commutativity
True
>>> C is A1s().E().F().D()          # idempotency
True
>>> C is A1s().D().E().F()
True
```

In this quick variant, we actually implement the category of `b c a2s`, and choose to do so in `A2s.B.C`:

```
sage: class A2s(Category_singleton):
....:     def super_categories(self):
....:         return [As()]
....:
....:     class B(CategoryWithAxiom):
....:         class C(CategoryWithAxiom):
```

(continues on next page)

(continued from previous page)

```
....:             def extra_super_categories(self):
....:                 return [As().E(), As().F()]
....:
....:     class E(CategoryWithAxiom):
....:         def F_extra_super_categories(self):
....:             return [As().B(), As().C()]


sage: A2s().B().C()
Category of e f a2s
sage: sorted(A2s().B().C().axioms())
['B', 'C', 'E', 'F']
sage: type(A2s().B().C())
<class '__main__.A2s.B.C_with_category'>
```

```
>>> from sage.all import *
>>> class A2s(Category_singleton):
...     def super_categories(self):
...         return [As()]
....:
>>>     class B(CategoryWithAxiom):
...         class C(CategoryWithAxiom):
...             def extra_super_categories(self):
...                 return [As().E(), As().F()]
....:
>>>     class E(CategoryWithAxiom):
...         def F_extra_super_categories(self):
...             return [As().B(), As().C()]


>>> A2s().B().C()
Category of e f a2s
>>> sorted(A2s().B().C().axioms())
['B', 'C', 'E', 'F']
>>> type(A2s().B().C())
<class '__main__.A2s.B.C_with_category'>
```

As desired, William's theorem and its consequences hold:

```
sage: A2s().B().C() is A2s().E().F()
True
sage: C =  A2s().E().F().D().B().C()
sage: C is A2s().B().C().E().F().D()  # commutativity
True
sage: C is A2s().E().F().E().F().D()  # William's theorem
True
sage: C is A2s().E().E().F().F().D()  # commutativity
True
sage: C is A2s().E().F().D()          # idempotency
True
sage: C is A2s().D().E().F()
True
```

```
>>> from sage.all import *
>>> A2s().B().C() is A2s().E().F()
True
>>> C =  A2s().E().F().D().B().C()
>>> C is A2s().B().C().E().F().D()  # commutativity
True
>>> C is A2s().E().F().E().F().D()  # William's theorem
True
>>> C is A2s().E().E().F().F().D()  # commutativity
True
>>> C is A2s().E().F().D()          # idempotency
True
>>> C is A2s().D().E().F()
True
```

Finally, we "accidentally" implement the category of `b c a1s`, both in `A3s.B.C` and `A3s.E.F`:

```
sage: class A3s(Category_singleton):
....:     def super_categories(self):
....:         return [As()]
....:
....:     class B(CategoryWithAxiom):
....:         class C(CategoryWithAxiom):
....:             def extra_super_categories(self):
....:                 return [As().E(), As().F()]
....:
....:     class E(CategoryWithAxiom):
....:         class F(CategoryWithAxiom):
....:             def extra_super_categories(self):
....:                 return [As().B(), As().C()]
```

```
>>> from sage.all import *
>>> class A3s(Category_singleton):
...     def super_categories(self):
...         return [As()]
....:
>>>     class B(CategoryWithAxiom):
...         class C(CategoryWithAxiom):
...             def extra_super_categories(self):
...                 return [As().E(), As().F()]
....:
>>>     class E(CategoryWithAxiom):
...         class F(CategoryWithAxiom):
...             def extra_super_categories(self):
...                 return [As().B(), As().C()]
```

We can still construct, say:

```
sage: A3s().B()
Category of b a3s
sage: A3s().C()
Category of c a3s
```

```
>>> from sage.all import *
>>> A3s().B()
Category of b a3s
>>> A3s().C()
Category of c a3s
```

However,

```
sage: A3s().B().C()           # not tested
```

```
>>> from sage.all import *
>>> A3s().B().C()           # not tested
```

runs into an infinite recursion loop, as `A3s().B().C()` wants to have `A3s().E().F()` as super category and reciprocally.

Todo

The above example violates the specifications (a category should be modelled by at most one class), so it's appropriate that it fails. Yet, the error message could be usefully complemented by some hint at what the source of the problem is (a category implemented in two distinct classes). Leaving a large enough piece of the backtrace would be useful though, so that one can explore where the issue comes from (e.g. with post mortem debugging).

2.4.2 Specifications

After fixing some vocabulary, we summarize here some specifications about categories and axioms.

The lattice of constructible categories

A mathematical category C is *implemented* if there is a class in Sage modelling it; it is *constructible* if it is either implemented, or is the intersection of *implemented* categories; in the latter case it is modelled by a *JoinCategory*. The comparison of two constructible categories with the *Category.is_subcategory()* method is supposed to model the comparison of the corresponding mathematical categories for inclusion of the objects (see *On the category hierarchy: subcategories and super categories* for details). For example:

```
sage: Fields().is_subcategory(Rings())
True
```

```
>>> from sage.all import *
>>> Fields().is_subcategory(Rings())
True
```

However this modelling may be incomplete. It can happen that a mathematical fact implying that a category A is a subcategory of a category B is not implemented. Still, the comparison should endow the set of constructible categories with a poset structure and in fact a lattice structure.

In this lattice, the join of two categories (*Category.join()*) is supposed to model their intersection. Given that we compare categories for inclusion, it would be more natural to call this operation the *meet*; blames go to me (Nicolas) for originally comparing categories by *amount of structure* rather than by *inclusion*. In practice, the join of two categories may be a strict super category of their intersection; first because this intersection might not be constructible; second because Sage might miss some mathematical information to recover the smallest constructible super category of the intersection.

Axioms

We say that an axiom `A` is *defined by* a category `Cs()` if `Cs` defines an appropriate method `Cs.SubcategoryMethods.A`, with the semantic of the axiom specified in the documentation; for any subcategory `Ds()`, `Ds().A()` models the subcategory of the objects of `Ds()` satisfying `A`. In this case, we say that the axiom `A` is *defined for* the category `Ds()`. Furthermore, `Ds` *implements the axiom* `A` if `Ds` has a category with axiom as nested class `Ds.A`. The category `Ds()` *satisfies* the axiom if `Ds()` is a subcategory of `Cs().A()` (meaning that all the objects of `Ds()` are known to satisfy the axiom `A`).

A digression on the structure of fibers when adding an axiom

Consider the application ϕ_A which maps a category to its category of objects satisfying A. Equivalently, ϕ_A is computing the intersection with the defining category with axiom of A. It follows immediately from the latter that ϕ_A is a regressive endomorphism of the lattice of categories. It restricts to a regressive endomorphism `Cs() |-> Cs().A()` on the lattice of constructible categories.

This endomorphism may have non trivial fibers, as in our favorite example: `DivisionRings()` and `Fields()` are in the same fiber for the axiom `Finite`:

```
sage: DivisionRings().Finite() is Fields().Finite()
True
```

```
>>> from sage.all import *
>>> DivisionRings().Finite() is Fields().Finite()
True
```

Consider the intersection S of such a fiber of ϕ_A with the upper set I_A of categories that do not satisfy `A`. The fiber itself is a sublattice. However I_A is not guaranteed to be stable under intersection (though exceptions should be rare). Therefore, there is a priori no guarantee that S would be stable under intersection. Also it's presumably finite, in fact small, but this is not guaranteed either.

Specifications

- Any constructible category `C` should admit a finite number of larger constructible categories.
- The methods `super_categories`, `extra_super_categories`, and friends should always return strict super-categories.

  For example, to specify that a finite division ring is a finite field, `DivisionRings.Finite_extra_super_categories` should not return `Fields().Finite()`! It could possibly return `Fields()`; but it's preferable to return the largest category that contains the relevant information, in this case `Magmas().Commutative()`, and to let the infrastructure apply the derivations.
- The base category of a *`CategoryWithAxiom`* should be an implemented category (i.e. not a *`JoinCategory`*). This is checked by *`CategoryWithAxiom._test_category_with_axiom()`*.
- Arborescent structure: Let `Cs()` be a category, and S be some set of axioms defined in some super categories of `Cs()` but not satisfied by `Cs()`. Suppose we want to provide a category with axiom for the elements of `Cs()` satisfying the axioms in S. Then, there should be a single enumeration `A1, A2, ..., Ak` without repetition of axioms in S such that `Cs.A1.A2....Ak` is an implemented category. Furthermore, every intermediate step `Cs.A1.A2....Ai` with $i \leq k$ should be a category with axiom having `Ai` as axiom and `Cs.A1.A2....Ai-1` as base category class; this base category class should not satisfy `Ai`. In particular, when some axioms of S can be deduced from previous ones by deduction rules, they should not appear in the enumeration `A1, A2, ..., Ak`.
- In particular, if `Cs()` is a category that satisfies some axiom `A` (e.g. from one of its super categories), then it should not implement that axiom. For example, a category class `Cs` can never have a nested class `Cs.A.A`. Similarly, applying the specification recursively, a category satisfying `A` cannot have a nested class `Cs.A1.A2.A3.A` where `A1`, `A2`, `A3` are axioms.

- A category can only implement an axiom if this axiom is defined by some super category. The code has not been systematically checked to support having two super categories defining the same axiom (which should of course have the same semantic). You are welcome to try, at your own risk. :-)
- When a category defines an axiom or functorial construction `A`, this fixes the semantic of `A` for all the subcategories. In particular, if two categories define `A`, then these categories should be independent, and either the semantic of `A` should be the same, or there should be no natural intersection between the two hierarchies of subcategories.
- Any super category of a *CategoryWithParameters* should either be a *CategoryWithParameters* or a *Category_singleton*.
- A *CategoryWithAxiom* having a *Category_singleton* as base category should be a *CategoryWithAxiom_singleton*. This is handled automatically by *CategoryWithAxiom.__init__()* and checked in *CategoryWithAxiom._test_category_with_axiom()*.
- A *CategoryWithAxiom* having a *Category_over_base_ring* as base category should be a *Category_over_base_ring*. This currently has to be handled by hand, using *CategoryWithAxiom_over_base_ring*. This is checked in *CategoryWithAxiom._test_category_with_axiom()*.

Todo

The following specifications would be desirable but are not yet implemented:

- A functorial construction category (Graded, CartesianProducts, …) having a *Category_singleton* as base category should be a *CategoryWithAxiom_singleton*.

  Nothing difficult to implement, but this will need to rework the current "no subclass of a concrete class" assertion test of *__classcall__()*.
- Similarly, a covariant functorial construction category having a *Category_over_base_ring* as base category should be a *Category_over_base_ring*.

The following specification might be desirable, or not:

- A join category involving a *Category_over_base_ring* should be a *Category_over_base_ring*. In the mean time, a `base_ring` method is automatically provided for most of those by *Modules.SubcategoryMethods.base_ring()*.

2.4.3 Design goals

As pointed out in the primer, the main design goal of the axioms infrastructure is to subdue the potential combinatorial explosion of the category hierarchy by letting the developer focus on implementing a few bookshelves for which there is actual code or mathematical information, and let Sage *compose dynamically and lazily* these building blocks to construct the minimal hierarchy of classes needed for the computation at hand. This allows for the infrastructure to scale smoothly as bookshelves are added, extended, or reorganized.

Other design goals include:

- Flexibility in the code layout: the category of, say, finite sets can be implemented either within the Sets category (in a nested class `Sets.Finite`), or in a separate file (typically in a class `FiniteSets` in a lazily imported module sage.categories.finite_sets).
- Single point of truth: a theorem, like Wedderburn's, should be implemented in a single spot.
- Single entry point: for example, from the entry *Rings*, one can explore a whole range of related categories just by applying axioms and constructions:

```
sage: Rings().Commutative().Finite().NoZeroDivisors()
Category of finite integral domains
```

(continues on next page)

(continued from previous page)

```
sage: Rings().Finite().Division()
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> Rings().Commutative().Finite().NoZeroDivisors()
Category of finite integral domains
>>> Rings().Finite().Division()
Category of finite enumerated fields
```

This will allow for progressively getting rid of all the entries like *GradedHopfAlgebrasWithBasis* which are polluting the global name space.

Note that this is not about precluding the existence of multiple natural ways to construct the same category:

```
sage: Groups().Finite()
Category of finite groups
sage: Monoids().Finite().Inverse()
Category of finite groups
sage: Sets().Finite() & Monoids().Inverse()
Category of finite groups
```

```
>>> from sage.all import *
>>> Groups().Finite()
Category of finite groups
>>> Monoids().Finite().Inverse()
Category of finite groups
>>> Sets().Finite() & Monoids().Inverse()
Category of finite groups
```

- Concise idioms for the users (adding axioms, …)
- Concise idioms and well highlighted hierarchy of bookshelves for the developer (especially with code folding)
- Introspection friendly (listing the axioms, recovering the mixins)

Note

The constructor for instances of this class takes as input the base category. Hence, they should in principle be constructed as:

```
sage: FiniteSets(Sets())
Category of finite sets

sage: Sets.Finite(Sets())
Category of finite sets
```

```
>>> from sage.all import *
>>> FiniteSets(Sets())
Category of finite sets

>>> Sets.Finite(Sets())
Category of finite sets
```

None of these idioms are really practical for the user. So instead, this object is to be constructed using any of the

following idioms:

```
sage: Sets()._with_axiom('Finite')
Category of finite sets
sage: FiniteSets()
Category of finite sets
sage: Sets().Finite()
Category of finite sets
```

```
>>> from sage.all import *
>>> Sets()._with_axiom('Finite')
Category of finite sets
>>> FiniteSets()
Category of finite sets
>>> Sets().Finite()
Category of finite sets
```

The later two are implemented using respectively *CategoryWithAxiom.__classcall__()* and *CategoryWithAxiom.__classget__()*.

2.4.4 Upcoming features

Todo

- Implement compatibility axiom / functorial constructions. For example, one would want to have:

  ```
  A.CartesianProducts() & B.CartesianProducts() = (A&B).CartesianProducts()
  ```

- Once full subcategories are implemented (see Issue #10668), make the relevant categories with axioms be such. This can be done systematically for, e.g., the axioms `Associative` or `Commutative`, but not for the axiom `Unital`: a semigroup morphism between two monoids need not preserve the unit.

  Should all full subcategories be implemented in term of axioms?

2.4.5 Algorithms

Computing joins

The workhorse of the axiom infrastructure is the algorithm for computing the join J of a set $C_1, \ldots, C_k$ of categories (see *Category.join()*). Formally, J is defined as the largest constructible category such that $J \subset C_i$ for all i, and $J \subset C.A()$ for every constructible category $C \supset J$ and any axiom A satisfied by J.

The join J is naturally computed as a closure in the lattice of constructible categories: it starts with the C_i's, gathers the set S of all the axioms satisfied by them, and repeatedly adds each axiom A to those categories that do not yet satisfy A using *Category._with_axiom()*. Due to deduction rules or (extra) super categories, new categories or new axioms may appear in the process. The process stops when each remaining category has been combined with each axiom. In practice, only the smallest categories are kept along the way; this is correct because adding an axiom is covariant: `C.A()` is a subcategory of `D.A()` whenever `C` is a subcategory of `D`.

As usual in such closure computations, the result does not depend on the order of execution. Furthermore, given that adding an axiom is an idempotent and regressive operation, the process is guaranteed to stop in a number of steps which is bounded by the number of super categories of J. In particular, it is a finite process.

Todo

Detail this a bit. What could typically go wrong is a situation where, for some category `C1`, `C1.A()` specifies a category `C2` as super category such that `C2.A()` specifies `C3` as super category such that …; this would clearly cause an infinite execution. Note that this situation violates the specifications since `C1.A()` is supposed to be a subcategory of `C2.A()`, … so we would have an infinite increasing chain of constructible categories.

It's reasonable to assume that there is a finite number of axioms defined in the code. There remains to use this assumption to argue that any infinite execution of the algorithm would give rise to such an infinite sequence.

Adding an axiom

Let `Cs` be a category and `A` an axiom defined for this category. To compute `Cs().A()`, there are two cases.

Adding an axiom `A` to a category `Cs()` not implementing it

In this case, `Cs().A()` returns the join of:

- `Cs()`
- `Bs().A()` for every direct super category `Bs()` of `Cs()`
- the categories appearing in `Cs().A_extra_super_categories()`

This is a highly recursive process. In fact, as such, it would run right away into an infinite loop! Indeed, the join of `Cs()` with `Bs().A()` would trigger the construction of `Cs().A()` and reciprocally. To avoid this, the *`Category.join()`* method itself does not use *`Category._with_axiom()`* to add axioms, but its sister *`Category._with_axiom_as_tuple()`*; the latter builds a tuple of categories that should be joined together but leaves the computation of the join to its caller, the master join calculation.

Adding an axiom `A` to a category `Cs()` implementing it

In this case `Cs().A()` simply constructs an instance D of `Cs.A` which models the desired category. The non trivial part is the construction of the super categories of D. Very much like above, this includes:

- `Cs()`
- `Bs().A()` for every super category `Bs()` of `Cs()`
- the categories appearing in `D.extra_super_categories()`

This by itself may not be sufficient, due in particular to deduction rules. On may for example discover a new axiom `A1` satisfied by D, imposing to add `A1` to all of the above categories. Therefore the super categories are computed as the join of the above categories. Up to one twist: as is, the computation of this join would trigger recursively a recalculation of `Cs().A()`! To avoid this, *`Category.join()`* is given an optional argument to specify that the axiom `A` should *not* be applied to `Cs()`.

Sketch of proof of correctness and evaluation of complexity

As we have seen, this is a highly recursive process! In particular, one needs to argue that, as long as the specifications are satisfied, the algorithm won't run in an infinite recursion, in particular in case of deduction rule.

Theorem

Consider the construction of a category C by adding an axiom to a category (or computing of a join). Let H be the hierarchy of implemented categories above C. Let n and m be respectively the number of categories and the number of inheritance edges in H.

Assuming that the specifications are satisfied, the construction of C involves constructing the categories in H exactly once (and no other category), and at most n join calculations. In particular, the time complexity should be, roughly speaking, bounded by n^2. In particular, it's finite.

Remark

It's actually to be expected that the complexity is more of the order of magnitude of $na + m$, where a is the number of axioms satisfied by C. But this is to be checked in detail, in particular due to the many category inclusion tests involved.

The key argument is that *Category.join* cannot call itself recursively without going through the construction of some implemented category. In turn, the construction of some implemented category C only involves constructing strictly smaller categories, and possibly a direct join calculation whose result is strictly smaller than C. This statement is obvious if C implements the `super_categories` method directly, and easy to check for functorial construction categories. It requires a proof for categories with axioms since there is a recursive join involved.

Lemma

Let C be a category implementing an axiom A. Recall that the construction of `C.A()` involves a single direct join calculation for computing the super categories. No other direct join calculation occur, and the calculation involves only implemented categories that are strictly smaller than `C.A()`.

Proof

Let D be a category involved in the join calculation for the super categories of `C.A()`, and assume by induction that D is strictly smaller than `C.A()`. A category E newly constructed from D can come from:

- `D.(extra_)super_categories()`

  In this case, the specifications impose that E should be strictly smaller than D and therefore strictly smaller than C.

- `D.with_axiom_as_tuple('B')` or `D.B_extra_super_categories()` for some axiom B

  In this case, the axiom B is satisfied by some subcategory of `C.A()`, and therefore must be satisfied by `C.A()` itself. Since adding an axiom is a regressive construction, E must be a subcategory of `C.A()`. If there is equality, then E and `C.A()` must have the same class, and therefore, E must be directly constructed as `C.A()`. However the join construction explicitly prevents this call.

Note that a call to `D.with_axiom_as_tuple('B')` does not trigger a direct join calculation; but of course, if D implements B, the construction of the implemented category `E = D.B()` will involve a strictly smaller join calculation.

2.4.6 Conclusion

This is the end of the axioms documentation. Congratulations on having read that far!

2.4.7 Tests

Note

Quite a few categories with axioms are constructed early on during Sage's startup. Therefore, when playing around with the implementation of the axiom infrastructure, it is easy to break Sage. The following sequence of tests is designed to test the infrastructure from the ground up even in a partially broken Sage. Please don't remove the imports!

class sage.categories.category_with_axiom.**Bars**

Bases: *Category_singleton*

A toy singleton category, for testing purposes.

See also

Blahs

Unital_extra_super_categories()

Return extraneous super categories for the unital objects of `self`.

This method specifies that a unital bar is a test object. Thus, the categories of unital bars and of unital test objects coincide.

EXAMPLES:

```
sage: from sage.categories.category_with_axiom import Bars, TestObjects
sage: Bars().Unital_extra_super_categories()
[Category of test objects]
sage: Bars().Unital()
Category of unital test objects
sage: TestObjects().Unital().all_super_categories()
[Category of unital test objects,
 Category of unital blahs,
 Category of test objects,
 Category of bars,
 Category of blahs,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import Bars, TestObjects
>>> Bars().Unital_extra_super_categories()
[Category of test objects]
>>> Bars().Unital()
Category of unital test objects
>>> TestObjects().Unital().all_super_categories()
[Category of unital test objects,
 Category of unital blahs,
 Category of test objects,
 Category of bars,
 Category of blahs,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

super_categories()

class sage.categories.category_with_axiom.**Blahs**

Bases: *Category_singleton*

A toy singleton category, for testing purposes.

This is the root of a hierarchy of mathematically meaningless categories, used for testing Sage's category framework:

- *Bars*
- *TestObjects*
- *DummyObjectsOverBaseRing*

Blue_extra_super_categories()

Illustrates a current limitation in the way to have an axiom imply another one.

Here, we would want `Blue` to imply `Unital`, and to put the class for the category of unital blue blahs in `Blahs.Unital.Blue` rather than `Blahs.Blue`.

This currently fails because `Blahs` is the category where the axiom `Blue` is defined, and the specifications currently impose that a category defining an axiom should also implement it (here in a category with axiom `Blahs.Blue`). In practice, due to this violation of the specifications, the axiom is lost during the join calculation.

Todo

Decide whether we care about this feature. In such a situation, we are not really defining a new axiom, but just defining an axiom as an alias for a couple others, which might not be that useful.

Todo

Improve the infrastructure to detect and report this violation of the specifications, if this is easy. Otherwise, it's not so bad: when defining an axiom A in a category `Cs` the first thing one is supposed to doctest is that `Cs().A()` works. So the problem should not go unnoticed.

class Commutative(*base_category*)

Bases: *CategoryWithAxiom*

class Connected(*base_category*)

Bases: *CategoryWithAxiom*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom*

class Flying(*base_category*)

Bases: *CategoryWithAxiom*

extra_super_categories()

This illustrates a way to have an axiom imply another one.

Here, we want `Flying` to imply `Unital`, and to put the class for the category of unital flying blahs in `Blahs.Flying` rather than `Blahs.Unital.Flying`.

class SubcategoryMethods

Bases: object

Blue()

Commutative()

Connected()

FiniteDimensional()

Flying()

Unital()

class Unital(*base_category*)

Bases: *CategoryWithAxiom*

class Blue(*base_category*)

Bases: *CategoryWithAxiom*

super_categories()

class sage.categories.category_with_axiom.**CategoryWithAxiom**(*base_category*)

Bases: *Category*

An abstract class for categories obtained by adding an axiom to a base category.

See the *category primer*, and in particular its *section about axioms* for an introduction to axioms, and *CategoryWithAxiom* for how to implement axioms and the documentation of the axiom infrastructure.

static __classcall__(**args*, ***options*)

Make `FoosBar(**)` an alias for `Foos(**)._with_axiom("Bar")`.

EXAMPLES:

```
sage: FiniteGroups()
Category of finite groups
sage: ModulesWithBasis(ZZ)
Category of modules with basis over Integer Ring
sage: AlgebrasWithBasis(QQ)
Category of algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> FiniteGroups()
Category of finite groups
>>> ModulesWithBasis(ZZ)
Category of modules with basis over Integer Ring
>>> AlgebrasWithBasis(QQ)
Category of algebras with basis over Rational Field
```

This is relevant when e.g. `Foos(**)` does some non trivial transformations:

```
sage: Modules(QQ) is VectorSpaces(QQ)
True
sage: type(Modules(QQ))
<class 'sage.categories.vector_spaces.VectorSpaces_with_category'>
```

(continues on next page)

(continued from previous page)

```
sage: ModulesWithBasis(QQ) is VectorSpaces(QQ).WithBasis()
True
sage: type(ModulesWithBasis(QQ))
<class 'sage.categories.vector_spaces.VectorSpaces.WithBasis_with_category'>
```

```
>>> from sage.all import *
>>> Modules(QQ) is VectorSpaces(QQ)
True
>>> type(Modules(QQ))
<class 'sage.categories.vector_spaces.VectorSpaces_with_category'>

>>> ModulesWithBasis(QQ) is VectorSpaces(QQ).WithBasis()
True
>>> type(ModulesWithBasis(QQ))
<class 'sage.categories.vector_spaces.VectorSpaces.WithBasis_with_category'>
```

static __classget__(*base_category*, *base_category_class*)

Implement the binding behavior for categories with axioms.

This method implements a binding behavior on category with axioms so that, when a category `Cs` implements an axiom `A` with a nested class `Cs.A`, the expression `Cs().A` evaluates to the method defining the axiom `A` and not the nested class. See those design notes for the rationale behind this behavior.

EXAMPLES:

```
sage: Sets().Infinite()
Category of infinite sets
sage: Sets().Infinite
Cached version of <function ...Infinite at ...>
sage: Sets().Infinite.f == Sets.SubcategoryMethods.Infinite.f
True
```

```
>>> from sage.all import *
>>> Sets().Infinite()
Category of infinite sets
>>> Sets().Infinite
Cached version of <function ...Infinite at ...>
>>> Sets().Infinite.f == Sets.SubcategoryMethods.Infinite.f
True
```

We check that this also works when the class is implemented in a separate file, and lazy imported:

```
sage: Sets().Finite
Cached version of <function ...Finite at ...>
```

```
>>> from sage.all import *
>>> Sets().Finite
Cached version of <function ...Finite at ...>
```

There is no binding behavior when accessing `Finite` or `Infinite` from the class of the category instead of the category itself:

```
sage: Sets.Finite
<class 'sage.categories.finite_sets.FiniteSets'>
sage: Sets.Infinite
<class 'sage.categories.sets_cat.Sets.Infinite'>
```

```
>>> from sage.all import *
>>> Sets.Finite
<class 'sage.categories.finite_sets.FiniteSets'>
>>> Sets.Infinite
<class 'sage.categories.sets_cat.Sets.Infinite'>
```

This method also initializes the attribute `_base_category_class_and_axiom` if not already set:

```
sage: Sets.Infinite._base_category_class_and_axiom
(<class 'sage.categories.sets_cat.Sets'>, 'Infinite')
sage: Sets.Infinite._base_category_class_and_axiom_origin
'set by __classget__'
```

```
>>> from sage.all import *
>>> Sets.Infinite._base_category_class_and_axiom
(<class 'sage.categories.sets_cat.Sets'>, 'Infinite')
>>> Sets.Infinite._base_category_class_and_axiom_origin
'set by __classget__'
```

__init__(*base_category*)

_repr_object_names()

The names of the objects of this category, as used by `_repr_`.

See also

Category._repr_object_names()

EXAMPLES:

```
sage: FiniteSets()._repr_object_names()
'finite sets'
sage: AlgebrasWithBasis(QQ).FiniteDimensional()._repr_object_names()
'finite dimensional algebras with basis over Rational Field'
sage: Monoids()._repr_object_names()
'monoids'
sage: Semigroups().Unital().Finite()._repr_object_names()
'finite monoids'
sage: Algebras(QQ).Commutative()._repr_object_names()
'commutative algebras over Rational Field'
```

```
>>> from sage.all import *
>>> FiniteSets()._repr_object_names()
'finite sets'
>>> AlgebrasWithBasis(QQ).FiniteDimensional()._repr_object_names()
'finite dimensional algebras with basis over Rational Field'
```

(continues on next page)

(continued from previous page)

```
>>> Monoids()._repr_object_names()
'monoids'
>>> Semigroups().Unital().Finite()._repr_object_names()
'finite monoids'
>>> Algebras(QQ).Commutative()._repr_object_names()
'commutative algebras over Rational Field'
```

Note

This is implemented by taking _repr_object_names from self._without_axioms(named=True), and adding the names of the relevant axioms in appropriate order.

static _repr_object_names_static(*category*, *axioms*)

INPUT:

- `base_category` – a category
- `axioms` – list or iterable of strings

EXAMPLES:

```
sage: from sage.categories.category_with_axiom import CategoryWithAxiom
sage: CategoryWithAxiom._repr_object_names_static(Semigroups(), ["Flying",
↪"Blue"])
'flying blue semigroups'
sage: CategoryWithAxiom._repr_object_names_static(Algebras(QQ), ["Flying",
↪"WithBasis", "Blue"])
'flying blue algebras with basis over Rational Field'
sage: CategoryWithAxiom._repr_object_names_static(Algebras(QQ), ["WithBasis"])
'algebras with basis over Rational Field'
sage: CategoryWithAxiom._repr_object_names_static(Sets().Finite().
↪Subquotients(), ["Finite"])
'subquotients of finite sets'
sage: CategoryWithAxiom._repr_object_names_static(Monoids(), ["Unital"])
'monoids'
sage: CategoryWithAxiom._repr_object_names_static(Algebras(QQ['x']['y']), [
↪"Flying", "WithBasis", "Blue"])
'flying blue algebras with basis over Univariate Polynomial Ring in y over␣
↪Univariate Polynomial Ring in x over Rational Field'
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import CategoryWithAxiom
>>> CategoryWithAxiom._repr_object_names_static(Semigroups(), ["Flying", "Blue
↪"])
'flying blue semigroups'
>>> CategoryWithAxiom._repr_object_names_static(Algebras(QQ), ["Flying",
↪"WithBasis", "Blue"])
'flying blue algebras with basis over Rational Field'
>>> CategoryWithAxiom._repr_object_names_static(Algebras(QQ), ["WithBasis"])
'algebras with basis over Rational Field'
>>> CategoryWithAxiom._repr_object_names_static(Sets().Finite().
↪Subquotients(), ["Finite"])
```

(continues on next page)

(continued from previous page)

```
'subquotients of finite sets'
>>> CategoryWithAxiom._repr_object_names_static(Monoids(), ["Unital"])
'monoids'
>>> CategoryWithAxiom._repr_object_names_static(Algebras(QQ['x']['y']), [
↪"Flying", "WithBasis", "Blue"])
'flying blue algebras with basis over Univariate Polynomial Ring in y over␣
↪Univariate Polynomial Ring in x over Rational Field'
```

If the axioms is a set or frozen set, then they are first sorted using *canonicalize_axioms()*:

```
sage: CategoryWithAxiom._repr_object_names_static(Semigroups(), set(["Finite",
↪ "Commutative", "Facade"]))
'facade finite commutative semigroups'
```

```
>>> from sage.all import *
>>> CategoryWithAxiom._repr_object_names_static(Semigroups(), set(["Finite",
↪"Commutative", "Facade"]))
'facade finite commutative semigroups'
```

See also

_repr_object_names()

Note

The logic here is shared between *_repr_object_names()* and `JoinCategory._repr_object_names`

_test_category_with_axiom(***options*)

Run generic tests on this category with axioms.

See also

`TestSuite`.

This check that an axiom category of a *Category_singleton* is a singleton category, and similarwise for *Category_over_base_ring*.

EXAMPLES:

```
sage: Sets().Finite()._test_category_with_axiom()
sage: Modules(ZZ).FiniteDimensional()._test_category_with_axiom()
```

```
>>> from sage.all import *
>>> Sets().Finite()._test_category_with_axiom()
>>> Modules(ZZ).FiniteDimensional()._test_category_with_axiom()
```

_without_axioms(*named=False*)

Return the category without the axioms that have been added to create it.

EXAMPLES:

```
sage: Sets().Finite()._without_axioms()
Category of sets
sage: Monoids().Finite()._without_axioms()
Category of magmas
```

```
>>> from sage.all import *
>>> Sets().Finite()._without_axioms()
Category of sets
>>> Monoids().Finite()._without_axioms()
Category of magmas
```

This is because:

```
sage: Semigroups().Unital() is Monoids()
True
```

```
>>> from sage.all import *
>>> Semigroups().Unital() is Monoids()
True
```

If `named` is `True`, then `_without_axioms` stops at the first category that has an explicit name of its own:

```
sage: Sets().Finite()._without_axioms(named=True)
Category of sets
sage: Monoids().Finite()._without_axioms(named=True)
Category of monoids
```

```
>>> from sage.all import *
>>> Sets().Finite()._without_axioms(named=True)
Category of sets
>>> Monoids().Finite()._without_axioms(named=True)
Category of monoids
```

Technically we test this by checking if the class specifies explicitly the attribute `_base_category_class_and_axiom` by looking up `_base_category_class_and_axiom_origin`.

Some more examples:

```
sage: Algebras(QQ).Commutative()._without_axioms()
Category of magmatic algebras over Rational Field
sage: Algebras(QQ).Commutative()._without_axioms(named=True)
Category of algebras over Rational Field
```

```
>>> from sage.all import *
>>> Algebras(QQ).Commutative()._without_axioms()
Category of magmatic algebras over Rational Field
>>> Algebras(QQ).Commutative()._without_axioms(named=True)
Category of algebras over Rational Field
```

additional_structure()

Return the additional structure defined by `self`.

OUTPUT: `None`

By default, a category with axiom defines no additional structure.

See also

Category.additional_structure().

EXAMPLES:

```
sage: Sets().Finite().additional_structure()
sage: Monoids().additional_structure()
```

```
>>> from sage.all import *
>>> Sets().Finite().additional_structure()
>>> Monoids().additional_structure()
```

axioms()

Return the axioms known to be satisfied by all the objects of `self`.

See also

Category.axioms()

EXAMPLES:

```
sage: C = Sets.Finite(); C
Category of finite sets
sage: C.axioms()
frozenset({'Finite'})

sage: C = Modules(GF(5)).FiniteDimensional(); C
Category of finite dimensional vector spaces over Finite Field of size 5
sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Finite', 'FiniteDimensional']

sage: sorted(FiniteMonoids().Algebras(QQ).axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive',
 'FiniteDimensional', 'Unital', 'WithBasis']
sage: sorted(FiniteMonoids().Algebras(GF(3)).axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Finite',
 'FiniteDimensional', 'Unital', 'WithBasis']

sage: from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
sage: MagmasAndAdditiveMagmas().Distributive().Unital().axioms()
frozenset({'Distributive', 'Unital'})

sage: D = MagmasAndAdditiveMagmas().Distributive()
sage: X = D.AdditiveAssociative().AdditiveCommutative().Associative()
```

(continues on next page)

(continued from previous page)

```
sage: X.Unital().super_categories()[1]
Category of monoids
sage: X.Unital().super_categories()[1] is Monoids()
True
```

```
>>> from sage.all import *
>>> C = Sets.Finite(); C
Category of finite sets
>>> C.axioms()
frozenset({'Finite'})

>>> C = Modules(GF(Integer(5))).FiniteDimensional(); C
Category of finite dimensional vector spaces over Finite Field of size 5
>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Finite', 'FiniteDimensional']

>>> sorted(FiniteMonoids().Algebras(QQ).axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive',
 'FiniteDimensional', 'Unital', 'WithBasis']
>>> sorted(FiniteMonoids().Algebras(GF(Integer(3))).axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Finite',
 'FiniteDimensional', 'Unital', 'WithBasis']

>>> from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
>>> MagmasAndAdditiveMagmas().Distributive().Unital().axioms()
frozenset({'Distributive', 'Unital'})

>>> D = MagmasAndAdditiveMagmas().Distributive()
>>> X = D.AdditiveAssociative().AdditiveCommutative().Associative()
>>> X.Unital().super_categories()[Integer(1)]
Category of monoids
>>> X.Unital().super_categories()[Integer(1)] is Monoids()
True
```

base_category()

Return the base category of `self`.

EXAMPLES:

```
sage: C = Sets.Finite(); C
Category of finite sets
sage: C.base_category()
Category of sets
sage: C._without_axioms()
Category of sets
```

```
>>> from sage.all import *
>>> C = Sets.Finite(); C
```

(continues on next page)

(continued from previous page)

```
Category of finite sets
>>> C.base_category()
Category of sets
>>> C._without_axioms()
Category of sets
```

extra_super_categories()

Return the extra super categories of a category with axiom.

Default implementation which returns `[]`.

EXAMPLES:

```
sage: FiniteSets().extra_super_categories()
[]
```

```
>>> from sage.all import *
>>> FiniteSets().extra_super_categories()
[]
```

super_categories()

Return a list of the (immediate) super categories of `self`, as per *Category.super_categories()*.

This implements the property that if `As` is a subcategory of `Bs`, then the intersection of `As` with `FiniteSets()` is a subcategory of `As` and of the intersection of `Bs` with `FiniteSets()`.

EXAMPLES:

A finite magma is both a magma and a finite set:

```
sage: Magmas().Finite().super_categories()
[Category of magmas, Category of finite sets]
```

```
>>> from sage.all import *
>>> Magmas().Finite().super_categories()
[Category of magmas, Category of finite sets]
```

Variants:

```
sage: Sets().Finite().super_categories()
[Category of sets]

sage: Monoids().Finite().super_categories()
[Category of monoids, Category of finite semigroups]
```

```
>>> from sage.all import *
>>> Sets().Finite().super_categories()
[Category of sets]

>>> Monoids().Finite().super_categories()
[Category of monoids, Category of finite semigroups]
```

EXAMPLES:

class sage.categories.category_with_axiom.**CategoryWithAxiom_over_base_ring**(*base_category*)

Bases: *CategoryWithAxiom*, *Category_over_base_ring*

class sage.categories.category_with_axiom.**CategoryWithAxiom_singleton**(*base_category*)

Bases: *Category_singleton*, *CategoryWithAxiom*

class sage.categories.category_with_axiom.**DummyObjectsOverBaseRing**(*base*, *name=None*)

Bases: *Category_over_base_ring*

A toy singleton category, for testing purposes.

See also

Blahs

class Commutative(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Facade(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Finite(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Finite(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Unital(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Commutative(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class Unital(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

super_categories()

class sage.categories.category_with_axiom.**TestObjects**

Bases: *Category_singleton*

A toy singleton category, for testing purposes.

See also

Blahs

class Commutative(*base_category*)

Bases: *CategoryWithAxiom*

class Facade(*base_category*)

Bases: *CategoryWithAxiom*

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom*

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

class Unital(*base_category*)

Bases: *CategoryWithAxiom*

class Commutative(*base_category*)

Bases: *CategoryWithAxiom*

class Unital(*base_category*)

Bases: *CategoryWithAxiom*

super_categories()

sage.categories.category_with_axiom.**axiom**(*axiom*)

Return a function/method `self -> self._with_axiom(axiom)`.

This can used as a shorthand to define axioms, in particular in the tests below. Usually one will want to attach documentation to an axiom, so the need for such a shorthand in real life might not be that clear, unless we start creating lots of axioms.

In the long run maybe this could evolve into an `@axiom` decorator.

EXAMPLES:

```
sage: from sage.categories.category_with_axiom import axiom
sage: axiom("Finite")(Semigroups())
Category of finite semigroups
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import axiom
>>> axiom("Finite")(Semigroups())
Category of finite semigroups
```

Upon assigning the result to a class this becomes a method:

```
sage: class As:
....:     def _with_axiom(self, axiom): return self, axiom
....:     Finite = axiom("Finite")
sage: As().Finite()
(<__main__.As ... at ...>, 'Finite')
```

```
>>> from sage.all import *
>>> class As:
...     def _with_axiom(self, axiom): return self, axiom
```

(continues on next page)

(continued from previous page)

```
...     Finite = axiom("Finite")
>>> As().Finite()
(<__main__.As ... at ...>, 'Finite')
```

sage.categories.category_with_axiom.**axiom_of_nested_class**(*nested_cls*)

Given a class and a nested axiom class, return the axiom.

EXAMPLES:

This uses some heuristics like checking if the nested_cls carries the name of the axiom, or is built by appending or prepending the name of the axiom to that of the class:

```
sage: from sage.categories.category_with_axiom import TestObjects, axiom_of_
↪nested_class
sage: axiom_of_nested_class(TestObjects, TestObjects.FiniteDimensional)
'FiniteDimensional'
sage: axiom_of_nested_class(TestObjects.FiniteDimensional,
....:                       TestObjects.FiniteDimensional.Finite)
'Finite'
sage: axiom_of_nested_class(Sets, FiniteSets)
'Finite'
sage: axiom_of_nested_class(Algebras, AlgebrasWithBasis)
'WithBasis'
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import TestObjects, axiom_of_nested_
↪class
>>> axiom_of_nested_class(TestObjects, TestObjects.FiniteDimensional)
'FiniteDimensional'
>>> axiom_of_nested_class(TestObjects.FiniteDimensional,
...                       TestObjects.FiniteDimensional.Finite)
'Finite'
>>> axiom_of_nested_class(Sets, FiniteSets)
'Finite'
>>> axiom_of_nested_class(Algebras, AlgebrasWithBasis)
'WithBasis'
```

In all other cases, the nested class should provide an attribute `_base_category_class_and_axiom`:

```
sage: Semigroups._base_category_class_and_axiom
(<class 'sage.categories.magmas.Magmas'>, 'Associative')
sage: axiom_of_nested_class(Magmas, Semigroups)
'Associative'
```

```
>>> from sage.all import *
>>> Semigroups._base_category_class_and_axiom
(<class 'sage.categories.magmas.Magmas'>, 'Associative')
>>> axiom_of_nested_class(Magmas, Semigroups)
'Associative'
```

sage.categories.category_with_axiom.**base_category_class_and_axiom**(*cls*)

Try to deduce the base category and the axiom from the name of `cls`.

The heuristic is to try to decompose the name as the concatenation of the name of a category and the name of an axiom, and looking up that category in the standard location (i.e. in *sage.categories.hopf_algebras* for *HopfAlgebras*, and in *sage.categories.sets_cat* as a special case for *Sets*).

If the heuristic succeeds, the result is guaranteed to be correct. Otherwise, an error is raised.

EXAMPLES:

```
sage: from sage.categories.category_with_axiom import base_category_class_and_
↪axiom, CategoryWithAxiom
sage: base_category_class_and_axiom(FiniteSets)
(<class 'sage.categories.sets_cat.Sets'>, 'Finite')
sage: Sets.Finite
<class 'sage.categories.finite_sets.FiniteSets'>
sage: base_category_class_and_axiom(Sets.Finite)
(<class 'sage.categories.sets_cat.Sets'>, 'Finite')

sage: base_category_class_and_axiom(FiniteDimensionalHopfAlgebrasWithBasis)
(<class 'sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis'>,
 'FiniteDimensional')

sage: base_category_class_and_axiom(HopfAlgebrasWithBasis)
(<class 'sage.categories.hopf_algebras.HopfAlgebras'>, 'WithBasis')
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import base_category_class_and_axiom,
↪ CategoryWithAxiom
>>> base_category_class_and_axiom(FiniteSets)
(<class 'sage.categories.sets_cat.Sets'>, 'Finite')
>>> Sets.Finite
<class 'sage.categories.finite_sets.FiniteSets'>
>>> base_category_class_and_axiom(Sets.Finite)
(<class 'sage.categories.sets_cat.Sets'>, 'Finite')

>>> base_category_class_and_axiom(FiniteDimensionalHopfAlgebrasWithBasis)
(<class 'sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis'>,
 'FiniteDimensional')

>>> base_category_class_and_axiom(HopfAlgebrasWithBasis)
(<class 'sage.categories.hopf_algebras.HopfAlgebras'>, 'WithBasis')
```

Along the way, this does some sanity checks:

```
sage: class FacadeSemigroups(CategoryWithAxiom):
....:     pass
sage: base_category_class_and_axiom(FacadeSemigroups)
Traceback (most recent call last):
...
AssertionError: Missing (lazy import) link
for <class 'sage.categories.semigroups.Semigroups'>
to <class '__main__.FacadeSemigroups'> for axiom Facade?

sage: Semigroups.Facade = FacadeSemigroups
sage: base_category_class_and_axiom(FacadeSemigroups)
(<class 'sage.categories.semigroups.Semigroups'>, 'Facade')
```

```
>>> from sage.all import *
>>> class FacadeSemigroups(CategoryWithAxiom):
...     pass
>>> base_category_class_and_axiom(FacadeSemigroups)
Traceback (most recent call last):
...
AssertionError: Missing (lazy import) link
for <class 'sage.categories.semigroups.Semigroups'>
to <class '__main__.FacadeSemigroups'> for axiom Facade?

>>> Semigroups.Facade = FacadeSemigroups
>>> base_category_class_and_axiom(FacadeSemigroups)
(<class 'sage.categories.semigroups.Semigroups'>, 'Facade')
```

Note

In the following example, we could possibly retrieve `Sets` from the class name. However this cannot be implemented robustly until Issue #9107 is fixed. Anyway this feature has not been needed so far:

```
sage: Sets.Infinite
<class 'sage.categories.sets_cat.Sets.Infinite'>
sage: base_category_class_and_axiom(Sets.Infinite)
Traceback (most recent call last):
...
TypeError: Could not retrieve the base category class and axiom
for <class 'sage.categories.sets_cat.Sets.Infinite'>.
...
```

```
>>> from sage.all import *
>>> Sets.Infinite
<class 'sage.categories.sets_cat.Sets.Infinite'>
>>> base_category_class_and_axiom(Sets.Infinite)
Traceback (most recent call last):
...
TypeError: Could not retrieve the base category class and axiom
for <class 'sage.categories.sets_cat.Sets.Infinite'>.
...
```

sage.categories.category_with_axiom.**uncamelcase**(*s*, *separator=''*)

EXAMPLES:

```
sage: sage.categories.category_with_axiom.uncamelcase("FiniteDimensionalAlgebras")
'finite dimensional algebras'
sage: sage.categories.category_with_axiom.uncamelcase("JTrivialMonoids")
'j trivial monoids'
sage: sage.categories.category_with_axiom.uncamelcase("FiniteDimensionalAlgebras",
↪ "_")
'finite_dimensional_algebras'
```

```
>>> from sage.all import *
>>> sage.categories.category_with_axiom.uncamelcase("FiniteDimensionalAlgebras")
```

(continues on next page)

(continued from previous page)

```
'finite dimensional algebras'
>>> sage.categories.category_with_axiom.uncamelcase("JTrivialMonoids")
'j trivial monoids'
>>> sage.categories.category_with_axiom.uncamelcase("FiniteDimensionalAlgebras",
↪"_")
'finite_dimensional_algebras'
```

2.5 Functors

AUTHORS:

- David Kohel and William Stein
- David Joyner (2005-12-17): examples
- Robert Bradshaw (2007-06-23): Pyrexify
- Simon King (2010-04-30): more examples, several bug fixes, re-implementation of the default call method, making functors applicable to morphisms (not only to objects)
- Simon King (2010-12): Pickling of functors without losing domain and codomain

`sage.categories.functor.`**`ForgetfulFunctor`**(*domain*, *codomain*)

Construct the forgetful function from one category to another.

INPUT:

- `C`, `D` – two categories

OUTPUT:

A functor that returns the corresponding object of `D` for any element of `C`, by forgetting the extra structure.

ASSUMPTION:

The category `C` must be a sub-category of `D`.

EXAMPLES:

```
sage: rings = Rings()
sage: abgrps = CommutativeAdditiveGroups()
sage: F = ForgetfulFunctor(rings, abgrps)
sage: F
The forgetful functor
 from Category of rings
   to Category of commutative additive groups
```

```
>>> from sage.all import *
>>> rings = Rings()
>>> abgrps = CommutativeAdditiveGroups()
>>> F = ForgetfulFunctor(rings, abgrps)
>>> F
The forgetful functor
 from Category of rings
   to Category of commutative additive groups
```

It would be a mistake to call it in opposite order:

```
sage: F = ForgetfulFunctor(abgrps, rings)
Traceback (most recent call last):
...
ValueError: Forgetful functor not supported for domain
Category of commutative additive groups
```

```
>>> from sage.all import *
>>> F = ForgetfulFunctor(abgrps, rings)
Traceback (most recent call last):
...
ValueError: Forgetful functor not supported for domain
Category of commutative additive groups
```

If both categories are equal, the forgetful functor is the same as the identity functor:

```
sage: ForgetfulFunctor(abgrps, abgrps) == IdentityFunctor(abgrps)
True
```

```
>>> from sage.all import *
>>> ForgetfulFunctor(abgrps, abgrps) == IdentityFunctor(abgrps)
True
```

class sage.categories.functor.**ForgetfulFunctor_generic**

Bases: *Functor*

The forgetful functor, i.e., embedding of a subcategory.

NOTE:

Forgetful functors should be created using *ForgetfulFunctor()*, since the init method of this class does not check whether the domain is a subcategory of the codomain.

EXAMPLES:

```
sage: F = ForgetfulFunctor(FiniteFields(), Fields())  # indirect doctest
sage: F
The forgetful functor
 from Category of finite enumerated fields
   to Category of fields
sage: F(GF(3))
Finite Field of size 3
```

```
>>> from sage.all import *
>>> F = ForgetfulFunctor(FiniteFields(), Fields())  # indirect doctest
>>> F
The forgetful functor
 from Category of finite enumerated fields
   to Category of fields
>>> F(GF(Integer(3)))
Finite Field of size 3
```

class sage.categories.functor.**Functor**

Bases: SageObject

A class for functors between two categories.

NOTE:

- In the first place, a functor is given by its domain and codomain, which are both categories.
- When defining a sub-class, the user should not implement a call method. Instead, one should implement three methods, which are composed in the default call method:
  - `_coerce_into_domain(self, x)` – return an object of `self`'s domain, corresponding to `x`, or raise a `TypeError`. - Default: Raise `TypeError` if `x` is not in `self`'s domain.
  - `_apply_functor(self, x)` – apply `self` to an object `x` of `self`'s domain. - Default: Conversion into `self`'s codomain.
  - `_apply_functor_to_morphism(self, f)` – apply `self` to a morphism `f` in `self`'s domain. - Default: Return `self(f.domain()).hom(f,self(f.codomain()))`.

EXAMPLES:

```
sage: rings  = Rings()
sage: abgrps = CommutativeAdditiveGroups()
sage: F = ForgetfulFunctor(rings, abgrps)
sage: F.domain()
Category of rings
sage: F.codomain()
Category of commutative additive groups
sage: from sage.categories.functor import Functor
sage: isinstance(F, Functor)
True
sage: I = IdentityFunctor(abgrps)
sage: I
The identity functor on Category of commutative additive groups
sage: I.domain()
Category of commutative additive groups
sage: isinstance(I, Functor)
True
```

```
>>> from sage.all import *
>>> rings  = Rings()
>>> abgrps = CommutativeAdditiveGroups()
>>> F = ForgetfulFunctor(rings, abgrps)
>>> F.domain()
Category of rings
>>> F.codomain()
Category of commutative additive groups
>>> from sage.categories.functor import Functor
>>> isinstance(F, Functor)
True
>>> I = IdentityFunctor(abgrps)
>>> I
The identity functor on Category of commutative additive groups
>>> I.domain()
Category of commutative additive groups
>>> isinstance(I, Functor)
True
```

Note that by default, an instance of the class Functor is coercion from the domain into the codomain. The above subclasses overloaded this behaviour. Here we illustrate the default:

```
sage: from sage.categories.functor import Functor
sage: F = Functor(Rings(), Fields())
sage: F
Functor from Category of rings to Category of fields
sage: F(ZZ)
Rational Field
sage: F(GF(2))
Finite Field of size 2
```

```
>>> from sage.all import *
>>> from sage.categories.functor import Functor
>>> F = Functor(Rings(), Fields())
>>> F
Functor from Category of rings to Category of fields
>>> F(ZZ)
Rational Field
>>> F(GF(Integer(2)))
Finite Field of size 2
```

Functors are not only about the objects of a category, but also about their morphisms. We illustrate it, again, with the coercion functor from rings to fields.

```
sage: R1.<x> = ZZ[]
sage: R2.<a,b> = QQ[]
sage: f = R1.hom([a + b], R2)
sage: f
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Multivariate Polynomial Ring in a, b over Rational Field
  Defn: x |--> a + b
sage: F(f)
Ring morphism:
  From: Fraction Field of Univariate Polynomial Ring in x over Integer Ring
  To:   Fraction Field of Multivariate Polynomial Ring in a, b over Rational Field
  Defn: x |--> a + b
sage: F(f)(1/x)
1/(a + b)
```

```
>>> from sage.all import *
>>> R1 = ZZ['x']; (x,) = R1._first_ngens(1)
>>> R2 = QQ['a, b']; (a, b,) = R2._first_ngens(2)
>>> f = R1.hom([a + b], R2)
>>> f
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Multivariate Polynomial Ring in a, b over Rational Field
  Defn: x |--> a + b
>>> F(f)
Ring morphism:
  From: Fraction Field of Univariate Polynomial Ring in x over Integer Ring
  To:   Fraction Field of Multivariate Polynomial Ring in a, b over Rational Field
  Defn: x |--> a + b
```

(continues on next page)

(continued from previous page)

```
>>> F(f)(Integer(1)/x)
1/(a + b)
```

We can also apply a polynomial ring construction functor to our homomorphism. The result is a homomorphism that is defined on the base ring:

```
sage: F = QQ['t'].construction()[0]
sage: F
Poly[t]
sage: F(f)
Ring morphism:
  From: Univariate Polynomial Ring in t
        over Univariate Polynomial Ring in x over Integer Ring
  To:   Univariate Polynomial Ring in t
        over Multivariate Polynomial Ring in a, b over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Multivariate Polynomial Ring in a, b over Rational Field
          Defn: x |--> a + b
sage: p = R1['t']('(-x^2 + x)*t^2 + (x^2 - x)*t - 4*x^2 - x + 1')
sage: F(f)(p)
(-a^2 - 2*a*b - b^2 + a + b)*t^2 + (a^2 + 2*a*b + b^2 - a - b)*t
- 4*a^2 - 8*a*b - 4*b^2 - a - b + 1
```

```
>>> from sage.all import *
>>> F = QQ['t'].construction()[Integer(0)]
>>> F
Poly[t]
>>> F(f)
Ring morphism:
  From: Univariate Polynomial Ring in t
        over Univariate Polynomial Ring in x over Integer Ring
  To:   Univariate Polynomial Ring in t
        over Multivariate Polynomial Ring in a, b over Rational Field
  Defn: Induced from base ring by
        Ring morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Multivariate Polynomial Ring in a, b over Rational Field
          Defn: x |--> a + b
>>> p = R1['t']('(-x^2 + x)*t^2 + (x^2 - x)*t - 4*x^2 - x + 1')
>>> F(f)(p)
(-a^2 - 2*a*b - b^2 + a + b)*t^2 + (a^2 + 2*a*b + b^2 - a - b)*t
- 4*a^2 - 8*a*b - 4*b^2 - a - b + 1
```

codomain()

The codomain of `self`.

EXAMPLES:

```
sage: F = ForgetfulFunctor(FiniteFields(), Fields())
sage: F.codomain()
Category of fields
```

```
>>> from sage.all import *
>>> F = ForgetfulFunctor(FiniteFields(), Fields())
>>> F.codomain()
Category of fields
```

domain()

The domain of `self`.

EXAMPLES:

```
sage: F = ForgetfulFunctor(FiniteFields(), Fields())
sage: F.domain()
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> F = ForgetfulFunctor(FiniteFields(), Fields())
>>> F.domain()
Category of finite enumerated fields
```

sage.categories.functor.**IdentityFunctor**(C)

Construct the identity functor of the given category.

INPUT:

- `C` – a category

OUTPUT: the identity functor in `C`

EXAMPLES:

```
sage: rings = Rings()
sage: F = IdentityFunctor(rings)
sage: F(ZZ['x','y']) is ZZ['x','y']
True
```

```
>>> from sage.all import *
>>> rings = Rings()
>>> F = IdentityFunctor(rings)
>>> F(ZZ['x','y']) is ZZ['x','y']
True
```

class sage.categories.functor.**IdentityFunctor_generic**(C)

Bases: *ForgetfulFunctor_generic*

Generic identity functor on any category.

NOTE:

This usually is created using *IdentityFunctor()*.

EXAMPLES:

```
sage: F = IdentityFunctor(Fields()) #indirect doctest
sage: F
The identity functor on Category of fields
sage: F(RR) is RR
```

(continues on next page)

(continued from previous page)

```
True
sage: F(ZZ)
Traceback (most recent call last):
...
TypeError: x (=Integer Ring) is not in Category of fields
```

```
>>> from sage.all import *
>>> F = IdentityFunctor(Fields()) #indirect doctest
>>> F
The identity functor on Category of fields
>>> F(RR) is RR
True
>>> F(ZZ)
Traceback (most recent call last):
...
TypeError: x (=Integer Ring) is not in Category of fields
```

2.6 Implementing a new parent: a tutorial

The easiest approach for implementing a new parent is to start from a close example in sage.categories.examples. Here, we will get through the process of implementing a new finite semigroup, taking as starting point the provided example:

```
sage: S = FiniteSemigroups().example()
sage: S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
```

You may lookup the implementation of this example with:

```
sage: S??                                 # not tested
```

```
>>> from sage.all import *
>>> S??                                 # not tested
```

Or by browsing the source code of *sage.categories.examples.finite_semigroups.LeftRegularBand*.

Copy-paste this code into, say, a cell of the notebook, and replace every occurrence of `FiniteSemigroups().example(...)` in the documentation by `LeftRegularBand`. This will be equivalent to:

```
sage: from sage.categories.examples.finite_semigroups import LeftRegularBand
```

```
>>> from sage.all import *
>>> from sage.categories.examples.finite_semigroups import LeftRegularBand
```

Now, try:

```
sage: S = LeftRegularBand(); S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
```

```
>>> from sage.all import *
>>> S = LeftRegularBand(); S
An example of a finite semigroup: the left regular band generated by ('a', 'b', 'c',
↪'d')
```

and play around with the examples in the documentation of `S` and of *FiniteSemigroups*.

Rename the class to `ShiftSemigroup`, and modify the product to implement the semigroup generated by the given alphabet such that $au = u$ for any u of length 3.

Use `TestSuite` to test the newly implemented semigroup; draw its Cayley graph.

Add another option to the constructor to generalize the construction to any u of length k.

Lookup the Sloane for the sequence of the sizes of those semigroups.

Now implement the commutative monoid of subsets of $\{1, \ldots, n\}$ endowed with union as product. What is its category? What are the extra functionalities available there? Implement iteration and cardinality.

Todo

The tutorial should explain there how to reuse the enumerated set of subsets, and endow it with more structure.

CHAPTER

THREE

MAPS AND MORPHISMS

3.1 Base class for maps

AUTHORS:

- Robert Bradshaw: initial implementation
- Sebastien Besnier (2014-05-5): *FormalCompositeMap* contains a list of Map instead of only two Map. See Issue #16291.
- Sebastian Oehms (2019-01-19): *section()* added to *FormalCompositeMap*. See Issue #27081.

Map.**_make_weak_references**()

Only store weak references to domain and codomain of this map.

Note

This method is internally used on maps that are used for coercions or conversions between parents. Without using this method, some objects would stay alive indefinitely as soon as they are involved in a coercion or conversion.

See also

_make_strong_references()

EXAMPLES:

```
sage: Q = QuadraticField(-5)                                                #␣
↪needs sage.rings.number_field
sage: phi = CDF._internal_convert_map_from(Q)                               #␣
↪needs sage.rings.number_field
```

```
>>> from sage.all import *
>>> Q = QuadraticField(-Integer(5))                                          ␣
↪ # needs sage.rings.number_field
>>> phi = CDF._internal_convert_map_from(Q)                                 # needs␣
↪sage.rings.number_field
```

By Issue #14711, maps used in the coercion and conversion system use *weak* references to domain and codomain, in contrast to other maps:

```
sage: phi.domain                                                              #␣
↪needs sage.rings.number_field
<weakref at ...; to '...NumberField_quadratic_with_category' at ...>
sage: phi._make_strong_references()                                            #␣
↪needs sage.rings.number_field
sage: print(phi.domain)                                                       #␣
↪needs sage.rings.number_field
The constant function (...) -> Number Field in a
with defining polynomial x^2 + 5 with a = 2.236067977499790?*I
```

```
>>> from sage.all import *
>>> phi.domain                                                              # needs␣
↪sage.rings.number_field
<weakref at ...; to '...NumberField_quadratic_with_category' at ...>
>>> phi._make_strong_references()                                           # needs␣
↪sage.rings.number_field
>>> print(phi.domain)                                                       # needs␣
↪sage.rings.number_field
The constant function (...) -> Number Field in a
with defining polynomial x^2 + 5 with a = 2.236067977499790?*I
```

Now, as there is a strong reference, Q cannot be garbage collected:

```
sage: # needs sage.rings.number_field
sage: import gc
sage: _ = gc.collect()
sage: C = Q.__class__.__base__
sage: x = None
sage: numberQuadFields = len([x for x in gc.get_objects()
....:                         if isinstance(x, C)])
sage: del Q, x
sage: _ = gc.collect()
sage: numberQuadFields == len([x for x in gc.get_objects()
....:                          if isinstance(x, C)])
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> import gc
>>> _ = gc.collect()
>>> C = Q.__class__.__base__
>>> x = None
>>> numberQuadFields = len([x for x in gc.get_objects()
...                         if isinstance(x, C)])
>>> del Q, x
>>> _ = gc.collect()
>>> numberQuadFields == len([x for x in gc.get_objects()
...                          if isinstance(x, C)])
True
```

However, if we now make the references weak again, the number field can be garbage collected, which of course makes the map and its parents invalid. This is why *_make_weak_references()* should only be used if one really knows what one is doing:

```
sage: # needs sage.rings.number_field
sage: phi._make_weak_references()
sage: _ = gc.collect()
sage: numberQuadFields == len([x for x in gc.get_objects()
....:                          if isinstance(x, C)]) + 1
True
sage: phi
Defunct map
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> phi._make_weak_references()
>>> _ = gc.collect()
>>> numberQuadFields == len([x for x in gc.get_objects()
...                          if isinstance(x, C)]) + Integer(1)
True
>>> phi
Defunct map
```

Map.**_make_strong_references**()

Store strong references to domain and codomain of this map.

Note

By default, maps keep strong references to domain and codomain, preventing them thus from garbage collection. However, in Sage's coercion system, these strong references are replaced by weak references, since otherwise some objects would stay alive indefinitely as soon as they are involved in a coercion or conversion.

See also

_make_weak_references()

EXAMPLES:

```
sage: Q = QuadraticField(-5)                                                #␣
↪needs sage.rings.number_field
sage: phi = CDF._internal_convert_map_from(Q)                               #␣
↪needs sage.rings.number_field
```

```
>>> from sage.all import *
>>> Q = QuadraticField(-Integer(5))                                          ␣
↪ # needs sage.rings.number_field
>>> phi = CDF._internal_convert_map_from(Q)                                 # needs␣
↪sage.rings.number_field
```

By Issue #14711, maps used in the coercion and conversion system use *weak* references to domain and codomain, in contrast to other maps:

```
sage: phi.domain                                                            #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
<weakref at ...; to '...NumberField_quadratic_with_category' at ...>
sage: phi._make_strong_references()                                          #␣
↪needs sage.rings.number_field
sage: print(phi.domain)                                                     #␣
↪needs sage.rings.number_field
The constant function (...) -> Number Field in a
with defining polynomial x^2 + 5 with a = 2.236067977499790?*I
```

```
>>> from sage.all import *
>>> phi.domain                                                              # needs␣
↪sage.rings.number_field
<weakref at ...; to '...NumberField_quadratic_with_category' at ...>
>>> phi._make_strong_references()                                           # needs␣
↪sage.rings.number_field
>>> print(phi.domain)                                                       # needs␣
↪sage.rings.number_field
The constant function (...) -> Number Field in a
with defining polynomial x^2 + 5 with a = 2.236067977499790?*I
```

Now, as there is a strong reference, Q cannot be garbage collected:

```
sage: # needs sage.rings.number_field
sage: import gc
sage: _ = gc.collect()
sage: C = Q.__class__.__base__
sage: x = None
sage: numberQuadFields = len([x for x in gc.get_objects()
....:                         if isinstance(x, C)])
sage: del Q, x
sage: _ = gc.collect()
sage: numberQuadFields == len([x for x in gc.get_objects()
....:                          if isinstance(x, C)])
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> import gc
>>> _ = gc.collect()
>>> C = Q.__class__.__base__
>>> x = None
>>> numberQuadFields = len([x for x in gc.get_objects()
...                         if isinstance(x, C)])
>>> del Q, x
>>> _ = gc.collect()
>>> numberQuadFields == len([x for x in gc.get_objects()
...                          if isinstance(x, C)])
True
```

However, if we now make the references weak again, the number field can be garbage collected, which of course makes the map and its parents invalid. This is why *_make_weak_references()* should only be used if one really knows what one is doing:

```
sage: # needs sage.rings.number_field
sage: phi._make_weak_references()
sage: _ = gc.collect()
sage: numberQuadFields == len([x for x in gc.get_objects()
....:                          if isinstance(x, C)]) + 1
True
sage: phi
Defunct map
sage: phi._make_strong_references()
Traceback (most recent call last):
...
RuntimeError: The domain of this map became garbage collected
sage: phi.parent()
Traceback (most recent call last):
...
ValueError: This map is in an invalid state, the domain has been garbage collected
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> phi._make_weak_references()
>>> _ = gc.collect()
>>> numberQuadFields == len([x for x in gc.get_objects()
...                          if isinstance(x, C)]) + Integer(1)
True
>>> phi
Defunct map
>>> phi._make_strong_references()
Traceback (most recent call last):
...
RuntimeError: The domain of this map became garbage collected
>>> phi.parent()
Traceback (most recent call last):
...
ValueError: This map is in an invalid state, the domain has been garbage collected
```

class sage.categories.map.**FormalCompositeMap**

Bases: *Map*

Formal composite maps.

A formal composite map is formed by two maps, so that the codomain of the first map is contained in the domain of the second map.

Note

When calling a composite with additional arguments, these arguments are *only* passed to the second underlying map.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: S.<a> = QQ[]
```

(continues on next page)

(continued from previous page)

```
sage: from sage.categories.morphism import SetMorphism
sage: f = SetMorphism(Hom(R, S, Rings()), lambda p: p[0]*a^p.degree())
sage: g = S.hom([2*x])
sage: f*g
Composite map:
  From: Univariate Polynomial Ring in a over Rational Field
  To:   Univariate Polynomial Ring in a over Rational Field
  Defn:   Ring morphism:
          From: Univariate Polynomial Ring in a over Rational Field
          To:   Univariate Polynomial Ring in x over Rational Field
          Defn: a |--> 2*x
        then
          Generic morphism:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Univariate Polynomial Ring in a over Rational Field
sage: g*f
Composite map:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Univariate Polynomial Ring in a over Rational Field
        then
          Ring morphism:
          From: Univariate Polynomial Ring in a over Rational Field
          To:   Univariate Polynomial Ring in x over Rational Field
          Defn: a |--> 2*x
sage: (f*g)(2*a^2+5)
5*a^2
sage: (g*f)(2*x^2+5)
20*x^2
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> S = QQ['a']; (a,) = S._first_ngens(1)
>>> from sage.categories.morphism import SetMorphism
>>> f = SetMorphism(Hom(R, S, Rings()), lambda p: p[Integer(0)]*a**p.degree())
>>> g = S.hom([Integer(2)*x])
>>> f*g
Composite map:
  From: Univariate Polynomial Ring in a over Rational Field
  To:   Univariate Polynomial Ring in a over Rational Field
  Defn:   Ring morphism:
          From: Univariate Polynomial Ring in a over Rational Field
          To:   Univariate Polynomial Ring in x over Rational Field
          Defn: a |--> 2*x
        then
          Generic morphism:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Univariate Polynomial Ring in a over Rational Field
>>> g*f
Composite map:
```

(continues on next page)

(continued from previous page)

```
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Univariate Polynomial Ring in x over Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Univariate Polynomial Ring in a over Rational Field
        then
          Ring morphism:
          From: Univariate Polynomial Ring in a over Rational Field
          To:   Univariate Polynomial Ring in x over Rational Field
          Defn: a |--> 2*x
>>> (f*g)(Integer(2)*a**Integer(2)+Integer(5))
5*a^2
>>> (g*f)(Integer(2)*x**Integer(2)+Integer(5))
20*x^2
```

domains()

Iterate over the domains of the factors of this map.

(This is useful in particular to check for loops in coercion maps.)

See also

Map.domains()

EXAMPLES:

```
sage: f = QQ.coerce_map_from(ZZ)
sage: g = MatrixSpace(QQ, 2, 2).coerce_map_from(QQ)                    #␣
↪needs sage.modules
sage: list((g * f).domains())                                          #␣
↪needs sage.modules
[Integer Ring, Rational Field]
```

```
>>> from sage.all import *
>>> f = QQ.coerce_map_from(ZZ)
>>> g = MatrixSpace(QQ, Integer(2), Integer(2)).coerce_map_from(QQ)          ␣
↪              # needs sage.modules
>>> list((g * f).domains())                                             #␣
↪needs sage.modules
[Integer Ring, Rational Field]
```

first()

Return the first map in the formal composition.

If `self` represents $f_n \circ f_{n-1} \circ \cdots \circ f_1 \circ f_0$, then `self.first()` returns f_0. We have `self == self.then() * self.first()`.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: S.<a> = QQ[]
sage: from sage.categories.morphism import SetMorphism
```

(continues on next page)

(continued from previous page)

```
sage: f = SetMorphism(Hom(R, S, Rings()), lambda p: p[0]*a^p.degree())
sage: g = S.hom([2*x])
sage: fg = f * g
sage: fg.first() == g
True
sage: fg == fg.then() * fg.first()
True
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> S = QQ['a']; (a,) = S._first_ngens(1)
>>> from sage.categories.morphism import SetMorphism
>>> f = SetMorphism(Hom(R, S, Rings()), lambda p: p[Integer(0)]*a**p.degree())
>>> g = S.hom([Integer(2)*x])
>>> fg = f * g
>>> fg.first() == g
True
>>> fg == fg.then() * fg.first()
True
```

is_injective()

Tell whether `self` is injective.

It raises `NotImplementedError` if it cannot be determined.

EXAMPLES:

```
sage: # needs sage.modules
sage: V1 = QQ^2
sage: V2 = QQ^3
sage: phi1 = (QQ^1).hom(Matrix([[1, 1]]), V1)
sage: phi2 = V1.hom(Matrix([[1, 2, 3], [4, 5, 6]]), V2)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V1 = QQ**Integer(2)
>>> V2 = QQ**Integer(3)
>>> phi1 = (QQ**Integer(1)).hom(Matrix([[Integer(1), Integer(1)]]), V1)
>>> phi2 = V1.hom(Matrix([[Integer(1), Integer(2), Integer(3)], [Integer(4),␣
↪Integer(5), Integer(6)]]), V2)
```

If both constituents are injective, the composition is injective:

```
sage: from sage.categories.map import FormalCompositeMap
sage: c1 = FormalCompositeMap(Hom(QQ^1, V2, phi1.category_for()),           #␣
↪needs sage.modules
....:                         phi1, phi2)
sage: c1.is_injective()                                                     #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> from sage.categories.map import FormalCompositeMap
>>> c1 = FormalCompositeMap(Hom(QQ**Integer(1), V2, phi1.category_for()),␣
↪      # needs sage.modules
...                           phi1, phi2)
>>> c1.is_injective()                                                    #␣
↪needs sage.modules
True
```

If it cannot be determined whether the composition is injective, an error is raised:

```
sage: psi1 = V2.hom(Matrix([[1, 2], [3, 4], [5, 6]]), V1)                #␣
↪needs sage.modules
sage: c2 = FormalCompositeMap(Hom(V1, V1, phi2.category_for()),          #␣
↪needs sage.modules
....:                           phi2, psi1)
sage: c2.is_injective()                                                  #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: not enough information to deduce injectivity
```

```
>>> from sage.all import *
>>> psi1 = V2.hom(Matrix([[Integer(1), Integer(2)], [Integer(3), Integer(4)],␣
↪[Integer(5), Integer(6)]]), V1)                    # needs sage.modules
>>> c2 = FormalCompositeMap(Hom(V1, V1, phi2.category_for()),            #␣
↪needs sage.modules
...                           phi2, psi1)
>>> c2.is_injective()                                                    #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: not enough information to deduce injectivity
```

If the first map is surjective and the second map is not injective, then the composition is not injective:

```
sage: psi2 = V1.hom([[1], [1]], QQ^1)                                    #␣
↪needs sage.modules
sage: c3 = FormalCompositeMap(Hom(V2, QQ^1, phi2.category_for()),        #␣
↪needs sage.modules
....:                           psi2, psi1)
sage: c3.is_injective()                                                  #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> psi2 = V1.hom([[Integer(1)], [Integer(1)]], QQ**Integer(1))          ␣
↪                        # needs sage.modules
>>> c3 = FormalCompositeMap(Hom(V2, QQ**Integer(1), phi2.category_for()),␣
↪      # needs sage.modules
...                           psi2, psi1)
>>> c3.is_injective()                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
False
```

is_surjective()

Tell whether `self` is surjective.

It raises `NotImplementedError` if it cannot be determined.

EXAMPLES:

```
sage: from sage.categories.map import FormalCompositeMap
sage: V3 = QQ^3                                                          #␣
↪needs sage.modules
sage: V2 = QQ^2                                                          #␣
↪needs sage.modules
sage: V1 = QQ^1                                                          #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> from sage.categories.map import FormalCompositeMap
>>> V3 = QQ**Integer(3)                                                  ␣
↪      # needs sage.modules
>>> V2 = QQ**Integer(2)                                                  ␣
↪      # needs sage.modules
>>> V1 = QQ**Integer(1)                                                  ␣
↪      # needs sage.modules
```

If both maps are surjective, the composition is surjective:

```
sage: # needs sage.modules
sage: phi32 = V3.hom(Matrix([[1, 2], [3, 4], [5, 6]]), V2)
sage: phi21 = V2.hom(Matrix([[1], [1]]), V1)
sage: c_phi = FormalCompositeMap(Hom(V3, V1, phi32.category_for()),
....:                            phi32, phi21)
sage: c_phi.is_surjective()
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> phi32 = V3.hom(Matrix([[Integer(1), Integer(2)], [Integer(3), Integer(4)],
↪ [Integer(5), Integer(6)]]), V2)
>>> phi21 = V2.hom(Matrix([[Integer(1)], [Integer(1)]]), V1)
>>> c_phi = FormalCompositeMap(Hom(V3, V1, phi32.category_for()),
...                            phi32, phi21)
>>> c_phi.is_surjective()
True
```

If the second map is not surjective, the composition is not surjective:

```
sage: FormalCompositeMap(Hom(V3, V1, phi32.category_for()),              #␣
↪needs sage.modules
....:                    phi32,
```

(continues on next page)

(continued from previous page)

```
....:                     V2.hom(Matrix([[0], [0]]), V1)).is_surjective()
False
```

```
>>> from sage.all import *
>>> FormalCompositeMap(Hom(V3, V1, phi32.category_for()),                #␣
↪needs sage.modules
...                     phi32,
...                     V2.hom(Matrix([[Integer(0)], [Integer(0)]]), V1)).is_
↪surjective()
False
```

If the second map is an isomorphism and the first map is not surjective, then the composition is not surjective:

```
sage: FormalCompositeMap(Hom(V2, V1, phi32.category_for()),                #␣
↪needs sage.modules
....:                     V2.hom(Matrix([[0], [0]]), V1),
....:                     V1.hom(Matrix([[1]]), V1)).is_surjective()
False
```

```
>>> from sage.all import *
>>> FormalCompositeMap(Hom(V2, V1, phi32.category_for()),                #␣
↪needs sage.modules
...                     V2.hom(Matrix([[Integer(0)], [Integer(0)]]), V1),
...                     V1.hom(Matrix([[Integer(1)]]), V1)).is_surjective()
False
```

Otherwise, surjectivity of the composition cannot be determined:

```
sage: FormalCompositeMap(Hom(V2, V1, phi32.category_for()),                #␣
↪needs sage.modules
....:     V2.hom(Matrix([[1, 1], [1, 1]]), V2),
....:     V2.hom(Matrix([[1], [1]]), V1)).is_surjective()
Traceback (most recent call last):
...
NotImplementedError: not enough information to deduce surjectivity
```

```
>>> from sage.all import *
>>> FormalCompositeMap(Hom(V2, V1, phi32.category_for()),                #␣
↪needs sage.modules
...     V2.hom(Matrix([[Integer(1), Integer(1)], [Integer(1), Integer(1)]]),␣
↪V2),
...     V2.hom(Matrix([[Integer(1)], [Integer(1)]]), V1)).is_surjective()
Traceback (most recent call last):
...
NotImplementedError: not enough information to deduce surjectivity
```

section()

Compute a section map from sections of the factors of `self` if they have been implemented.

EXAMPLES:

```
sage: P.<x> = QQ[]
sage: incl = P.coerce_map_from(ZZ)
sage: sect = incl.section(); sect
Composite map:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Integer Ring
  Defn:   Generic map:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Rational Field
        then
          Generic map:
          From: Rational Field
          To:   Integer Ring
sage: p = x + 5; q = x + 2
sage: sect(p-q)
3
```

```
>>> from sage.all import *
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> incl = P.coerce_map_from(ZZ)
>>> sect = incl.section(); sect
Composite map:
  From: Univariate Polynomial Ring in x over Rational Field
  To:   Integer Ring
  Defn:   Generic map:
          From: Univariate Polynomial Ring in x over Rational Field
          To:   Rational Field
        then
          Generic map:
          From: Rational Field
          To:   Integer Ring
>>> p = x + Integer(5); q = x + Integer(2)
>>> sect(p-q)
3
```

the following example has been attached to `_integer_` of `sage.rings.polynomial.polynomial_element.Polynomial` before (see comment there):

```
sage: k = GF(47)
sage: R.<x> = PolynomialRing(k)
sage: R.coerce_map_from(ZZ).section()
Composite map:
  From: Univariate Polynomial Ring in x over Finite Field of size 47
  To:   Integer Ring
  Defn:   Generic map:
          From: Univariate Polynomial Ring in x over Finite Field of size 47
          To:   Finite Field of size 47
        then
          Lifting map:
          From: Finite Field of size 47
          To:   Integer Ring
sage: ZZ(R(45))                 # indirect doctest
45
```

(continues on next page)

(continued from previous page)

```
sage: ZZ(3*x + 45)              # indirect doctest
Traceback (most recent call last):
...
TypeError: 3*x + 45 is not a constant polynomial
```

```
>>> from sage.all import *
>>> k = GF(Integer(47))
>>> R = PolynomialRing(k, names=('x',)); (x,) = R._first_ngens(1)
>>> R.coerce_map_from(ZZ).section()
Composite map:
  From: Univariate Polynomial Ring in x over Finite Field of size 47
  To:   Integer Ring
  Defn:   Generic map:
          From: Univariate Polynomial Ring in x over Finite Field of size 47
          To:   Finite Field of size 47
        then
          Lifting map:
          From: Finite Field of size 47
          To:   Integer Ring
>>> ZZ(R(Integer(45)))                  # indirect doctest
45
>>> ZZ(Integer(3)*x + Integer(45))              # indirect doctest
Traceback (most recent call last):
...
TypeError: 3*x + 45 is not a constant polynomial
```

then()

Return the tail of the list of maps.

If `self` represents $f_n \circ f_{n-1} \circ \cdots \circ f_1 \circ f_0$, then `self.first()` returns $f_n \circ f_{n-1} \circ \cdots \circ f_1$. We have `self == self.then() * self.first()`.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: S.<a> = QQ[]
sage: from sage.categories.morphism import SetMorphism
sage: f = SetMorphism(Hom(R, S, Rings()), lambda p: p[0]*a^p.degree())
sage: g = S.hom([2*x])
sage: (f*g).then() == f
True

sage: f = QQ.coerce_map_from(ZZ)
sage: f = f.extend_domain(ZZ).extend_codomain(QQ)
sage: f.then()
Composite map:
From: Integer Ring
To:   Rational Field
Defn:   Natural morphism:
From: Integer Ring
To:   Rational Field
then
Identity endomorphism of Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> S = QQ['a']; (a,) = S._first_ngens(1)
>>> from sage.categories.morphism import SetMorphism
>>> f = SetMorphism(Hom(R, S, Rings()), lambda p: p[Integer(0)]*a**p.degree())
>>> g = S.hom([Integer(2)*x])
>>> (f*g).then() == f
True

>>> f = QQ.coerce_map_from(ZZ)
>>> f = f.extend_domain(ZZ).extend_codomain(QQ)
>>> f.then()
Composite map:
From: Integer Ring
To:   Rational Field
Defn:   Natural morphism:
From: Integer Ring
To:   Rational Field
then
Identity endomorphism of Rational Field
```

class sage.categories.map.**Map**

Bases: Element

Basic class for all maps.

Note

The call method is of course not implemented in this base class. This must be done in the sub classes, by overloading _call_ and possibly also _call_with_args.

EXAMPLES:

Usually, instances of this class will not be constructed directly, but for example like this:

```
sage: from sage.categories.morphism import SetMorphism
sage: X.<x> = ZZ[]
sage: Y = ZZ
sage: phi = SetMorphism(Hom(X, Y, Rings()), lambda p: p[0])
sage: phi(x^2+2*x-1)
-1
sage: R.<x,y> = QQ[]
sage: f = R.hom([x+y, x-y], R)
sage: f(x^2+2*x-1)
x^2 + 2*x*y + y^2 + 2*x + 2*y - 1
```

```
>>> from sage.all import *
>>> from sage.categories.morphism import SetMorphism
>>> X = ZZ['x']; (x,) = X._first_ngens(1)
>>> Y = ZZ
>>> phi = SetMorphism(Hom(X, Y, Rings()), lambda p: p[Integer(0)])
>>> phi(x**Integer(2)+Integer(2)*x-Integer(1))
```

(continues on next page)

(continued from previous page)

```
-1
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x+y, x-y], R)
>>> f(x**Integer(2)+Integer(2)*x-Integer(1))
x^2 + 2*x*y + y^2 + 2*x + 2*y - 1
```

category_for()

Return the category `self` is a morphism for.

> **Note**
>
> This is different from the category of maps to which this map belongs *as an object*.

EXAMPLES:

```
sage: from sage.categories.morphism import SetMorphism
sage: X.<x> = ZZ[]
sage: Y = ZZ
sage: phi = SetMorphism(Hom(X, Y, Rings()), lambda p: p[0])
sage: phi.category_for()
Category of rings
sage: phi.category()
Category of homsets of unital magmas and additive unital additive magmas
sage: R.<x,y> = QQ[]
sage: f = R.hom([x+y, x-y], R)
sage: f.category_for()
Join of Category of unique factorization domains
 and Category of algebras with basis over
  (number fields and quotient fields and metric spaces)
 and Category of commutative algebras over
  (number fields and quotient fields and metric spaces)
 and Category of infinite sets
sage: f.category()
Category of endsets of unital magmas
 and right modules over (number fields and quotient fields and metric spaces)
 and left modules over (number fields and quotient fields and metric spaces)
```

```
>>> from sage.all import *
>>> from sage.categories.morphism import SetMorphism
>>> X = ZZ['x']; (x,) = X._first_ngens(1)
>>> Y = ZZ
>>> phi = SetMorphism(Hom(X, Y, Rings()), lambda p: p[Integer(0)])
>>> phi.category_for()
Category of rings
>>> phi.category()
Category of homsets of unital magmas and additive unital additive magmas
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x+y, x-y], R)
>>> f.category_for()
Join of Category of unique factorization domains
 and Category of algebras with basis over
```

(continues on next page)

(continued from previous page)

```
  (number fields and quotient fields and metric spaces)
 and Category of commutative algebras over
  (number fields and quotient fields and metric spaces)
 and Category of infinite sets
>>> f.category()
Category of endsets of unital magmas
 and right modules over (number fields and quotient fields and metric spaces)
 and left modules over (number fields and quotient fields and metric spaces)
```

FIXME: find a better name for this method

codomain

domain

domains()

Iterate over the domains of the factors of a (composite) map.

This default implementation simply yields the domain of this map.

See also

FormalCompositeMap.domains()

EXAMPLES:

```
sage: list(QQ.coerce_map_from(ZZ).domains())
[Integer Ring]
```

```
>>> from sage.all import *
>>> list(QQ.coerce_map_from(ZZ).domains())
[Integer Ring]
```

extend_codomain(*new_codomain*)

INPUT:

- `self` – a member of Hom(X, Y)
- `new_codomain` – an object Z such that there is a canonical coercion ϕ in Hom(Y, Z)

OUTPUT:

An element of Hom(X, Z) obtained by composing `self` with ϕ. If no canonical ϕ exists, a `TypeError` is raised.

EXAMPLES:

```
sage: mor = QQ.coerce_map_from(ZZ)
sage: mor.extend_codomain(RDF)
Composite map:
  From: Integer Ring
  To:   Real Double Field
  Defn:   Natural morphism:
          From: Integer Ring
          To:   Rational Field
```

(continues on next page)

(continued from previous page)

```
        then
          Native morphism:
          From: Rational Field
          To:   Real Double Field
sage: mor.extend_codomain(GF(7))
Traceback (most recent call last):
...
TypeError: No coercion from Rational Field to Finite Field of size 7
```

```
>>> from sage.all import *
>>> mor = QQ.coerce_map_from(ZZ)
>>> mor.extend_codomain(RDF)
Composite map:
  From: Integer Ring
  To:   Real Double Field
  Defn:   Natural morphism:
          From: Integer Ring
          To:   Rational Field
        then
          Native morphism:
          From: Rational Field
          To:   Real Double Field
>>> mor.extend_codomain(GF(Integer(7)))
Traceback (most recent call last):
...
TypeError: No coercion from Rational Field to Finite Field of size 7
```

extend_domain(*new_domain*)

INPUT:

- `self` – a member of Hom(Y, Z)
- `new_codomain` – an object X such that there is a canonical coercion ϕ in Hom(X, Y)

OUTPUT:

An element of Hom(X, Z) obtained by composing self with ϕ. If no canonical ϕ exists, a `TypeError` is raised.

EXAMPLES:

```
sage: # needs sage.rings.complex_double
sage: mor = CDF.coerce_map_from(RDF)
sage: mor.extend_domain(QQ)
Composite map:
  From: Rational Field
  To:   Complex Double Field
  Defn:   Native morphism:
          From: Rational Field
          To:   Real Double Field
        then
          Native morphism:
          From: Real Double Field
          To:   Complex Double Field
```

(continues on next page)

(continued from previous page)

```
sage: mor.extend_domain(ZZ['x'])
Traceback (most recent call last):
...
TypeError: No coercion from Univariate Polynomial Ring in x over Integer Ring
to Real Double Field
```

```
>>> from sage.all import *
>>> # needs sage.rings.complex_double
>>> mor = CDF.coerce_map_from(RDF)
>>> mor.extend_domain(QQ)
Composite map:
  From: Rational Field
  To:   Complex Double Field
  Defn:   Native morphism:
          From: Rational Field
          To:   Real Double Field
        then
          Native morphism:
          From: Real Double Field
          To:   Complex Double Field
>>> mor.extend_domain(ZZ['x'])
Traceback (most recent call last):
...
TypeError: No coercion from Univariate Polynomial Ring in x over Integer Ring
to Real Double Field
```

is_surjective()

Tell whether the map is surjective (not implemented in the base class).

parent()

Return the homset containing this map.

Note

The method *_make_weak_references()*, that is used for the maps found by the coercion system, needs to remove the usual strong reference from the coercion map to the homset containing it. As long as the user keeps strong references to domain and codomain of the map, we will be able to reconstruct the homset. However, a strong reference to the coercion map does not prevent the domain from garbage collection!

EXAMPLES:

```
sage: Q = QuadraticField(-5)                                                #␣
↪needs sage.rings.number_field
sage: phi = CDF._internal_convert_map_from(Q)                               #␣
↪needs sage.rings.number_field
sage: print(phi.parent())                                                   #␣
↪needs sage.rings.number_field
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
```

(continues on next page)

(continued from previous page)

```
   to Complex Double Field
```

```
>>> from sage.all import *
>>> Q = QuadraticField(-Integer(5))                                    ␣
↪     # needs sage.rings.number_field
>>> phi = CDF._internal_convert_map_from(Q)                            #␣
↪needs sage.rings.number_field
>>> print(phi.parent())                                                #␣
↪needs sage.rings.number_field
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
   to Complex Double Field
```

We now demonstrate that the reference to the coercion map ϕ does not prevent Q from being garbage collected:

```
sage: import gc
sage: del Q                                                            #␣
↪needs sage.rings.number_field
sage: _ = gc.collect()
sage: phi.parent()                                                     #␣
↪needs sage.rings.number_field
Traceback (most recent call last):
...
ValueError: This map is in an invalid state,
the domain has been garbage collected
```

```
>>> from sage.all import *
>>> import gc
>>> del Q                                                              #␣
↪needs sage.rings.number_field
>>> _ = gc.collect()
>>> phi.parent()                                                       #␣
↪needs sage.rings.number_field
Traceback (most recent call last):
...
ValueError: This map is in an invalid state,
the domain has been garbage collected
```

You can still obtain copies of the maps used by the coercion system with strong references:

```
sage: # needs sage.rings.number_field
sage: Q = QuadraticField(-5)
sage: phi = CDF.convert_map_from(Q)
sage: print(phi.parent())
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
   to Complex Double Field
sage: import gc
sage: del Q
```

(continues on next page)

(continued from previous page)

```
sage: _ = gc.collect()
sage: phi.parent()
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
   to Complex Double Field
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> Q = QuadraticField(-Integer(5))
>>> phi = CDF.convert_map_from(Q)
>>> print(phi.parent())
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
   to Complex Double Field
>>> import gc
>>> del Q
>>> _ = gc.collect()
>>> phi.parent()
Set of field embeddings
 from Number Field in a with defining polynomial x^2 + 5
      with a = 2.236067977499790?*I
   to Complex Double Field
```

post_compose(*left*)

INPUT:

- `self` – a Map in some `Hom(X, Y, category_right)`
- `left` – a Map in some `Hom(Y, Z, category_left)`

Returns the composition of `self` followed by `left` as a morphism in `Hom(X, Z, category)` where `category` is the meet of `category_left` and `category_right`.

Caveat: see the current restrictions on *Category.meet()*

EXAMPLES:

```
sage: from sage.categories.morphism import SetMorphism
sage: X.<x> = ZZ[]
sage: Y = ZZ
sage: Z = QQ
sage: phi_xy = SetMorphism(Hom(X, Y, Rings()), lambda p: p[0])
sage: phi_yz = SetMorphism(Hom(Y, Z, Monoids()), lambda y: QQ(y**2))
sage: phi_xz = phi_xy.post_compose(phi_yz); phi_xz
Composite map:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Integer Ring
        then
          Generic morphism:
```

(continues on next page)

(continued from previous page)

```
          From: Integer Ring
          To:   Rational Field
sage: phi_xz.category_for()
Category of monoids
```

```
>>> from sage.all import *
>>> from sage.categories.morphism import SetMorphism
>>> X = ZZ['x']; (x,) = X._first_ngens(1)
>>> Y = ZZ
>>> Z = QQ
>>> phi_xy = SetMorphism(Hom(X, Y, Rings()), lambda p: p[Integer(0)])
>>> phi_yz = SetMorphism(Hom(Y, Z, Monoids()), lambda y: QQ(y**Integer(2)))
>>> phi_xz = phi_xy.post_compose(phi_yz); phi_xz
Composite map:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Integer Ring
        then
          Generic morphism:
          From: Integer Ring
          To:   Rational Field
>>> phi_xz.category_for()
Category of monoids
```

pre_compose(*right*)

INPUT:

- `self` – a Map in some `Hom(Y, Z, category_left)`
- `left` – a Map in some `Hom(X, Y, category_right)`

Returns the composition of `right` followed by `self` as a morphism in `Hom(X, Z, category)` where `category` is the meet of `category_left` and `category_right`.

EXAMPLES:

```
sage: from sage.categories.morphism import SetMorphism
sage: X.<x> = ZZ[]
sage: Y = ZZ
sage: Z = QQ
sage: phi_xy = SetMorphism(Hom(X, Y, Rings()), lambda p: p[0])
sage: phi_yz = SetMorphism(Hom(Y, Z, Monoids()), lambda y: QQ(y**2))
sage: phi_xz = phi_yz.pre_compose(phi_xy); phi_xz
Composite map:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Integer Ring
        then
          Generic morphism:
          From: Integer Ring
```

(continues on next page)

(continued from previous page)

```
          To:   Rational Field
sage: phi_xz.category_for()
Category of monoids
```

```
>>> from sage.all import *
>>> from sage.categories.morphism import SetMorphism
>>> X = ZZ['x']; (x,) = X._first_ngens(1)
>>> Y = ZZ
>>> Z = QQ
>>> phi_xy = SetMorphism(Hom(X, Y, Rings()), lambda p: p[Integer(0)])
>>> phi_yz = SetMorphism(Hom(Y, Z, Monoids()), lambda y: QQ(y**Integer(2)))
>>> phi_xz = phi_yz.pre_compose(phi_xy); phi_xz
Composite map:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Rational Field
  Defn:   Generic morphism:
          From: Univariate Polynomial Ring in x over Integer Ring
          To:   Integer Ring
        then
          Generic morphism:
          From: Integer Ring
          To:   Rational Field
>>> phi_xz.category_for()
Category of monoids
```

section()

Return a section of `self`.

> **Note**
>
> By default, it returns `None`. You may override it in subclasses.

class sage.categories.map.**Section**

Bases: *Map*

A formal section of a map.

> **Note**
>
> Call methods are not implemented for the base class `Section`.

EXAMPLES:

```
sage: from sage.categories.map import Section
sage: R.<x,y> = ZZ[]
sage: S.<a,b> = QQ[]
sage: f = R.hom([a+b, a-b])
sage: sf = Section(f); sf
Section map:
  From: Multivariate Polynomial Ring in a, b over Rational Field
```

(continues on next page)

(continued from previous page)

```
  To:   Multivariate Polynomial Ring in x, y over Integer Ring
sage: sf(a)
Traceback (most recent call last):
...
NotImplementedError: <class 'sage.categories.map.Section'>
```

```
>>> from sage.all import *
>>> from sage.categories.map import Section
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> S = QQ['a, b']; (a, b,) = S._first_ngens(2)
>>> f = R.hom([a+b, a-b])
>>> sf = Section(f); sf
Section map:
  From: Multivariate Polynomial Ring in a, b over Rational Field
  To:   Multivariate Polynomial Ring in x, y over Integer Ring
>>> sf(a)
Traceback (most recent call last):
...
NotImplementedError: <class 'sage.categories.map.Section'>
```

inverse()

Return inverse of `self`.

sage.categories.map.**is_Map**(*x*)

Auxiliary function: Is the argument a map?

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: f = R.hom([x+y, x-y], R)
sage: from sage.categories.map import is_Map
sage: is_Map(f)
doctest:warning...
DeprecationWarning: The function is_Map is deprecated; use 'isinstance(..., Map)'␣
↪instead.
See https://github.com/sagemath/sage/issues/38103 for details.
True
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x+y, x-y], R)
>>> from sage.categories.map import is_Map
>>> is_Map(f)
doctest:warning...
DeprecationWarning: The function is_Map is deprecated; use 'isinstance(..., Map)'␣
↪instead.
See https://github.com/sagemath/sage/issues/38103 for details.
True
```

sage.categories.map.**unpickle_map**(*_class*, *parent*, *_dict*, *_slots*)

Auxiliary function for unpickling a map.

3.2 Homsets

The class *Hom* is the base class used to represent sets of morphisms between objects of a given category. *Hom* objects are usually "weakly" cached upon creation so that they don't have to be generated over and over but can be garbage collected together with the corresponding objects when these are not strongly ref'ed anymore.

EXAMPLES:

In the following, the *Hom* object is indeed cached:

```
sage: K = GF(17)
sage: H = Hom(ZZ, K)
sage: H
Set of Homomorphisms from Integer Ring to Finite Field of size 17
sage: H is Hom(ZZ, K)
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(17))
>>> H = Hom(ZZ, K)
>>> H
Set of Homomorphisms from Integer Ring to Finite Field of size 17
>>> H is Hom(ZZ, K)
True
```

Nonetheless, garbage collection occurs when the original references are overwritten:

```
sage: # needs sage.libs.pari
sage: for p in prime_range(200):
....:     K = GF(p)
....:     H = Hom(ZZ, K)
sage: import gc
sage: _ = gc.collect()
sage: from sage.rings.finite_rings.finite_field_prime_modn import FiniteField_prime_
↪modn as FF
sage: L = [x for x in gc.get_objects() if isinstance(x, FF)]
sage: len(L)
1
sage: L
[Finite Field of size 199]
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> for p in prime_range(Integer(200)):
...     K = GF(p)
...     H = Hom(ZZ, K)
>>> import gc
>>> _ = gc.collect()
>>> from sage.rings.finite_rings.finite_field_prime_modn import FiniteField_prime_
↪modn as FF
>>> L = [x for x in gc.get_objects() if isinstance(x, FF)]
>>> len(L)
1
>>> L
```

(continues on next page)

(continued from previous page)

```
[Finite Field of size 199]
```

AUTHORS:

- David Kohel and William Stein
- David Joyner (2005-12-17): added examples
- William Stein (2006-01-14): Changed from Homspace to Homset.
- Nicolas M. Thiery (2008-12-): Updated for the new category framework
- Simon King (2011-12): Use a weak cache for homsets
- Simon King (2013-02): added examples

sage.categories.homset.**End**(*X*, *category=None*)

Create the set of endomorphisms of `X` in the category category.

INPUT:

- `X` – anything
- `category` – (optional) category in which to coerce `X`

OUTPUT: a set of endomorphisms in category

EXAMPLES:

```
sage: V = VectorSpace(QQ, 3)                                                #␣
↪needs sage.modules
sage: End(V)                                                                #␣
↪needs sage.modules
Set of Morphisms (Linear Transformations)
 from Vector space of dimension 3 over Rational Field
 to Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> V = VectorSpace(QQ, Integer(3))                                          ␣
↪     # needs sage.modules
>>> End(V)                                                                 #␣
↪needs sage.modules
Set of Morphisms (Linear Transformations)
 from Vector space of dimension 3 over Rational Field
 to Vector space of dimension 3 over Rational Field
```

```
sage: # needs sage.groups
sage: G = AlternatingGroup(3)
sage: S = End(G); S
Set of Morphisms
 from Alternating group of order 3!/2 as a permutation group
 to Alternating group of order 3!/2 as a permutation group
 in Category of finite enumerated permutation groups
sage: S.domain()
Alternating group of order 3!/2 as a permutation group
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = AlternatingGroup(Integer(3))
>>> S = End(G); S
Set of Morphisms
 from Alternating group of order 3!/2 as a permutation group
 to Alternating group of order 3!/2 as a permutation group
 in Category of finite enumerated permutation groups
>>> S.domain()
Alternating group of order 3!/2 as a permutation group
```

To avoid creating superfluous categories, a homset in a category `Cs()` is in the homset category of the lowest full super category `Bs()` of `Cs()` that implements `Bs.Homsets` (or the join thereof if there are several). For example, finite groups form a full subcategory of unital magmas: any unital magma morphism between two finite groups is a finite group morphism. Since finite groups currently implement nothing more than unital magmas about their homsets, we have:

```
sage: # needs sage.groups
sage: G = GL(3, 3)
sage: G.category()
Category of finite groups
sage: H = Hom(G, G)
sage: H.homset_category()
Category of finite groups
sage: H.category()
Category of endsets of unital magmas
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = GL(Integer(3), Integer(3))
>>> G.category()
Category of finite groups
>>> H = Hom(G, G)
>>> H.homset_category()
Category of finite groups
>>> H.category()
Category of endsets of unital magmas
```

Similarly, a ring morphism just needs to preserve addition, multiplication, zero, and one. Accordingly, and since the category of rings implements nothing specific about its homsets, a ring homset is currently constructed in the category of homsets of unital magmas and unital additive magmas:

```
sage: H = Hom(ZZ,ZZ,Rings())
sage: H.category()
Category of endsets of unital magmas and additive unital additive magmas
```

```
>>> from sage.all import *
>>> H = Hom(ZZ,ZZ,Rings())
>>> H.category()
Category of endsets of unital magmas and additive unital additive magmas
```

`sage.categories.homset.`**`Hom`**(*X*, *Y*, *category=None*, *check=True*)

Create the space of homomorphisms from X to Y in the category `category`.

INPUT:

- `X` – an object of a category
- `Y` – an object of a category
- `category` – a category in which the morphisms must be (default: the meet of the categories of `X` and `Y`); both `X` and `Y` must belong to that category
- `check` – boolean (default: `True`); whether to check the input, and in particular that `X` and `Y` belong to `category`.

OUTPUT: a homset in category

EXAMPLES:

```
sage: V = VectorSpace(QQ, 3)                                                #␣
↪needs sage.modules
sage: Hom(V, V)                                                             #␣
↪needs sage.modules
Set of Morphisms (Linear Transformations) from
Vector space of dimension 3 over Rational Field to
Vector space of dimension 3 over Rational Field
sage: G = AlternatingGroup(3)                                               #␣
↪needs sage.groups
sage: Hom(G, G)                                                             #␣
↪needs sage.groups
Set of Morphisms
 from Alternating group of order 3!/2 as a permutation group
   to Alternating group of order 3!/2 as a permutation group
   in Category of finite enumerated permutation groups
sage: Hom(ZZ, QQ, Sets())
Set of Morphisms from Integer Ring to Rational Field in Category of sets

sage: Hom(FreeModule(ZZ, 1), FreeModule(QQ, 1))                             #␣
↪needs sage.modules
Set of Morphisms
 from Ambient free module of rank 1 over the principal ideal domain Integer Ring
   to Vector space of dimension 1 over Rational Field
   in Category of commutative additive groups
sage: Hom(FreeModule(QQ, 1), FreeModule(ZZ, 1))                             #␣
↪needs sage.modules
Set of Morphisms
 from Vector space of dimension 1 over Rational Field
   to Ambient free module of rank 1 over the principal ideal domain Integer Ring
   in Category of commutative additive groups
```

```
>>> from sage.all import *
>>> V = VectorSpace(QQ, Integer(3))                                         ␣
↪     # needs sage.modules
>>> Hom(V, V)                                                              #␣
↪needs sage.modules
Set of Morphisms (Linear Transformations) from
Vector space of dimension 3 over Rational Field to
Vector space of dimension 3 over Rational Field
>>> G = AlternatingGroup(Integer(3))                                        ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.groups
>>> Hom(G, G)                                                                #␣
↪needs sage.groups
Set of Morphisms
 from Alternating group of order 3!/2 as a permutation group
   to Alternating group of order 3!/2 as a permutation group
   in Category of finite enumerated permutation groups
>>> Hom(ZZ, QQ, Sets())
Set of Morphisms from Integer Ring to Rational Field in Category of sets

>>> Hom(FreeModule(ZZ, Integer(1)), FreeModule(QQ, Integer(1)))              ␣
↪              # needs sage.modules
Set of Morphisms
 from Ambient free module of rank 1 over the principal ideal domain Integer Ring
   to Vector space of dimension 1 over Rational Field
   in Category of commutative additive groups
>>> Hom(FreeModule(QQ, Integer(1)), FreeModule(ZZ, Integer(1)))              ␣
↪              # needs sage.modules
Set of Morphisms
 from Vector space of dimension 1 over Rational Field
   to Ambient free module of rank 1 over the principal ideal domain Integer Ring
   in Category of commutative additive groups
```

Here, we test against a memory leak that has been fixed at Issue #11521 by using a weak cache:

```
sage: # needs sage.libs.pari
sage: for p in prime_range(10^3):
....:  K = GF(p)
....:  a = K(0)
sage: import gc
sage: gc.collect()       # random
624
sage: from sage.rings.finite_rings.finite_field_prime_modn import FiniteField_
↪prime_modn as FF
sage: L = [x for x in gc.get_objects() if isinstance(x, FF)]
sage: len(L), L[0]
(1, Finite Field of size 997)
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> for p in prime_range(Integer(10)**Integer(3)):
...  K = GF(p)
...  a = K(Integer(0))
>>> import gc
>>> gc.collect()       # random
624
>>> from sage.rings.finite_rings.finite_field_prime_modn import FiniteField_prime_
↪modn as FF
>>> L = [x for x in gc.get_objects() if isinstance(x, FF)]
>>> len(L), L[Integer(0)]
(1, Finite Field of size 997)
```

To illustrate the choice of the category, we consider the following parents as running examples:

```
sage: X = ZZ; X
Integer Ring
sage: Y = SymmetricGroup(3); Y                                                  #␣
↪needs sage.groups
Symmetric group of order 3! as a permutation group
```

```
>>> from sage.all import *
>>> X = ZZ; X
Integer Ring
>>> Y = SymmetricGroup(Integer(3)); Y                                           ␣
↪     # needs sage.groups
Symmetric group of order 3! as a permutation group
```

By default, the smallest category containing both `X` and `Y`, is used:

```
sage: Hom(X, Y)                                                                 #␣
↪needs sage.groups
Set of Morphisms from Integer Ring
 to Symmetric group of order 3! as a permutation group
 in Category of enumerated monoids
```

```
>>> from sage.all import *
>>> Hom(X, Y)                                                                   #␣
↪needs sage.groups
Set of Morphisms from Integer Ring
 to Symmetric group of order 3! as a permutation group
 in Category of enumerated monoids
```

Otherwise, if `category` is specified, then `category` is used, after checking that `X` and `Y` are indeed in `category`:

```
sage: Hom(X, Y, Magmas())                                                       #␣
↪needs sage.groups
Set of Morphisms
 from Integer Ring
 to Symmetric group of order 3! as a permutation group
 in Category of magmas

sage: Hom(X, Y, Groups())                                                       #␣
↪needs sage.groups
Traceback (most recent call last):
...
ValueError: Integer Ring is not in Category of groups
```

```
>>> from sage.all import *
>>> Hom(X, Y, Magmas())                                                         #␣
↪needs sage.groups
Set of Morphisms
 from Integer Ring
 to Symmetric group of order 3! as a permutation group
 in Category of magmas

>>> Hom(X, Y, Groups())                                                         #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
Traceback (most recent call last):
...
ValueError: Integer Ring is not in Category of groups
```

A parent (or a parent class of a category) may specify how to construct certain homsets by implementing a method `_Hom_(self, codomain, category)`. This method should either construct the requested homset or raise a `TypeError`. This hook is currently mostly used to create homsets in some specific subclass of *Homset* (e.g. `RingHomset_generic`):

```
sage: Hom(QQ,QQ).__class__
<class 'sage.rings.homset.RingHomset_generic_with_category'>
```

```
>>> from sage.all import *
>>> Hom(QQ,QQ).__class__
<class 'sage.rings.homset.RingHomset_generic_with_category'>
```

Do not call this hook directly to create homsets, as it does not handle unique representation:

```
sage: Hom(QQ,QQ) == QQ._Hom_(QQ, category=QQ.category())
True
sage: Hom(QQ,QQ) is QQ._Hom_(QQ, category=QQ.category())
False
```

```
>>> from sage.all import *
>>> Hom(QQ,QQ) == QQ._Hom_(QQ, category=QQ.category())
True
>>> Hom(QQ,QQ) is QQ._Hom_(QQ, category=QQ.category())
False
```

Todo

- Design decision: how much of the homset comes from the category of `X` and `Y`, and how much from the specific `X` and `Y`. In particular, do we need several parent classes depending on `X` and `Y`, or does the difference only lie in the elements (i.e. the morphism), and of course how the parent calls their constructors.
- Specify the protocol for the `_Hom_` hook in case of ambiguity (e.g. if both a parent and some category thereof provide one).

class sage.categories.homset.**Homset**(*X: DomainElementT*, *Y: CodomainElementT*, *category:* Category | *None* = *None*, *base: Ring* | *None* = *None*, *check: bool* = *True*)

Bases: `Set_generic`, `Generic`

The class for collections of morphisms in a category.

EXAMPLES:

```
sage: H = Hom(QQ^2, QQ^3)                                                   #␣
↪needs sage.modules
sage: loads(H.dumps()) is H                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> H = Hom(QQ**Integer(2), QQ**Integer(3))                                    ␣
↪                # needs sage.modules
>>> loads(H.dumps()) is H                                                     #␣
↪needs sage.modules
True
```

Homsets of unique parents are unique as well:

```
sage: H = End(AffineSpace(2, names='x,y'))
sage: loads(dumps(AffineSpace(2, names='x,y'))) is AffineSpace(2, names='x,y')
True
sage: loads(dumps(H)) is H
True
```

```
>>> from sage.all import *
>>> H = End(AffineSpace(Integer(2), names='x,y'))
>>> loads(dumps(AffineSpace(Integer(2), names='x,y'))) is AffineSpace(Integer(2),␣
↪names='x,y')
True
>>> loads(dumps(H)) is H
True
```

Conversely, homsets of non-unique parents are non-unique:

```
sage: P11 = ProductProjectiveSpaces(QQ, [1, 1])
sage: H = End(P11)
sage: loads(dumps(P11)) is ProductProjectiveSpaces(QQ, [1, 1])
False
sage: loads(dumps(P11)) == ProductProjectiveSpaces(QQ, [1, 1])
True
sage: loads(dumps(H)) is H
False
sage: loads(dumps(H)) == H
True
```

```
>>> from sage.all import *
>>> P11 = ProductProjectiveSpaces(QQ, [Integer(1), Integer(1)])
>>> H = End(P11)
>>> loads(dumps(P11)) is ProductProjectiveSpaces(QQ, [Integer(1), Integer(1)])
False
>>> loads(dumps(P11)) == ProductProjectiveSpaces(QQ, [Integer(1), Integer(1)])
True
>>> loads(dumps(H)) is H
False
>>> loads(dumps(H)) == H
True
```

codomain()

Return the codomain of this homset.

EXAMPLES:

```
sage: P.<t> = ZZ[]
sage: f = P.hom([1/2*t])
sage: f.parent().codomain()
Univariate Polynomial Ring in t over Rational Field
sage: f.codomain() is f.parent().codomain()
True
```

```
>>> from sage.all import *
>>> P = ZZ['t']; (t,) = P._first_ngens(1)
>>> f = P.hom([Integer(1)/Integer(2)*t])
>>> f.parent().codomain()
Univariate Polynomial Ring in t over Rational Field
>>> f.codomain() is f.parent().codomain()
True
```

domain()

Return the domain of this homset.

EXAMPLES:

```
sage: P.<t> = ZZ[]
sage: f = P.hom([1/2*t])
sage: f.parent().domain()
Univariate Polynomial Ring in t over Integer Ring
sage: f.domain() is f.parent().domain()
True
```

```
>>> from sage.all import *
>>> P = ZZ['t']; (t,) = P._first_ngens(1)
>>> f = P.hom([Integer(1)/Integer(2)*t])
>>> f.parent().domain()
Univariate Polynomial Ring in t over Integer Ring
>>> f.domain() is f.parent().domain()
True
```

element_class_set_morphism()

A base class for elements of this homset which are also `SetMorphism`, i.e. implemented by mean of a Python function.

This is currently plain `SetMorphism`, without inheritance from categories.

Todo

Refactor during the upcoming homset cleanup.

EXAMPLES:

```
sage: H = Hom(ZZ, ZZ)
sage: H.element_class_set_morphism
<class 'sage.categories.morphism.SetMorphism'>
```

```
>>> from sage.all import *
>>> H = Hom(ZZ, ZZ)
>>> H.element_class_set_morphism
<class 'sage.categories.morphism.SetMorphism'>
```

homset_category()

Return the category that this is a Hom in, i.e., this is typically the category of the domain or codomain object.

EXAMPLES:

```
sage: H = Hom(AlternatingGroup(4), AlternatingGroup(7))                    #␣
↪needs sage.groups
sage: H.homset_category()                                                  #␣
↪needs sage.groups
Category of finite enumerated permutation groups
```

```
>>> from sage.all import *
>>> H = Hom(AlternatingGroup(Integer(4)), AlternatingGroup(Integer(7)))      ␣
↪              # needs sage.groups
>>> H.homset_category()                                                    #␣
↪needs sage.groups
Category of finite enumerated permutation groups
```

identity()

The identity map of this homset.

Note

Of course, this only exists for sets of endomorphisms.

EXAMPLES:

```
sage: H = Hom(QQ,QQ)
sage: H.identity()
Identity endomorphism of Rational Field
sage: H = Hom(ZZ,QQ)
sage: H.identity()
Traceback (most recent call last):
...
TypeError: identity map only defined for endomorphisms; try natural_map()␣
↪instead
sage: H.natural_map()
Natural morphism:
  From: Integer Ring
  To:   Rational Field
```

```
>>> from sage.all import *
>>> H = Hom(QQ,QQ)
>>> H.identity()
Identity endomorphism of Rational Field
>>> H = Hom(ZZ,QQ)
>>> H.identity()
Traceback (most recent call last):
...
TypeError: identity map only defined for endomorphisms; try natural_map()␣
↪instead
>>> H.natural_map()
Natural morphism:
  From: Integer Ring
  To:   Rational Field
```

natural_map()

Return the "natural map" of this homset.

> **Note**
>
> By default, a formal coercion morphism is returned.

EXAMPLES:

```
sage: H = Hom(ZZ['t'],QQ['t'], CommutativeAdditiveGroups())
sage: H.natural_map()
Coercion morphism:
  From: Univariate Polynomial Ring in t over Integer Ring
  To:   Univariate Polynomial Ring in t over Rational Field
sage: H = Hom(QQ['t'], GF(3)['t'])
sage: H.natural_map()
Traceback (most recent call last):
...
TypeError: natural coercion morphism
from Univariate Polynomial Ring in t over Rational Field
to Univariate Polynomial Ring in t over Finite Field of size 3 not defined
```

```
>>> from sage.all import *
>>> H = Hom(ZZ['t'],QQ['t'], CommutativeAdditiveGroups())
>>> H.natural_map()
Coercion morphism:
  From: Univariate Polynomial Ring in t over Integer Ring
  To:   Univariate Polynomial Ring in t over Rational Field
>>> H = Hom(QQ['t'], GF(Integer(3))['t'])
>>> H.natural_map()
Traceback (most recent call last):
...
TypeError: natural coercion morphism
from Univariate Polynomial Ring in t over Rational Field
to Univariate Polynomial Ring in t over Finite Field of size 3 not defined
```

one()

The identity map of this homset.

> **Note**
>
> Of course, this only exists for sets of endomorphisms.

EXAMPLES:

```
sage: K = GaussianIntegers()                                                #␣
↪needs sage.rings.number_field
sage: End(K).one()                                                          #␣
↪needs sage.rings.number_field
Identity endomorphism of Gaussian Integers generated by I
 in Number Field in I with defining polynomial x^2 + 1 with I = 1*I
```

```
>>> from sage.all import *
>>> K = GaussianIntegers()                                                  #␣
↪needs sage.rings.number_field
>>> End(K).one()                                                           #␣
↪needs sage.rings.number_field
Identity endomorphism of Gaussian Integers generated by I
 in Number Field in I with defining polynomial x^2 + 1 with I = 1*I
```

reversed()

Return the corresponding homset, but with the domain and codomain reversed.

EXAMPLES:

```
sage: # needs sage.modules
sage: H = Hom(ZZ^2, ZZ^3); H
Set of Morphisms from Ambient free module of rank 2 over the
 principal ideal domain Integer Ring to Ambient free module of
 rank 3 over the principal ideal domain Integer Ring in Category
 of finite dimensional modules with basis over (euclidean domains
 and noetherian rings and infinite enumerated sets and metric spaces)
sage: type(H)
<class 'sage.modules.free_module_homspace.FreeModuleHomspace_with_category'>
sage: H.reversed()
Set of Morphisms from Ambient free module of rank 3 over the
 principal ideal domain Integer Ring to Ambient free module of
 rank 2 over the principal ideal domain Integer Ring in Category
 of finite dimensional modules with basis over (euclidean domains
 and noetherian rings and infinite enumerated sets and metric spaces)
sage: type(H.reversed())
<class 'sage.modules.free_module_homspace.FreeModuleHomspace_with_category'>
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> H = Hom(ZZ**Integer(2), ZZ**Integer(3)); H
Set of Morphisms from Ambient free module of rank 2 over the
 principal ideal domain Integer Ring to Ambient free module of
```

(continues on next page)

(continued from previous page)

```
 rank 3 over the principal ideal domain Integer Ring in Category
 of finite dimensional modules with basis over (euclidean domains
 and noetherian rings and infinite enumerated sets and metric spaces)
>>> type(H)
<class 'sage.modules.free_module_homspace.FreeModuleHomspace_with_category'>
>>> H.reversed()
Set of Morphisms from Ambient free module of rank 3 over the
 principal ideal domain Integer Ring to Ambient free module of
 rank 2 over the principal ideal domain Integer Ring in Category
 of finite dimensional modules with basis over (euclidean domains
 and noetherian rings and infinite enumerated sets and metric spaces)
>>> type(H.reversed())
<class 'sage.modules.free_module_homspace.FreeModuleHomspace_with_category'>
```

class sage.categories.homset.**HomsetWithBase**(*X*, *Y*, *category=None*, *check=True*, *base=None*)

Bases: *Homset*

sage.categories.homset.**end**(*X*, *f*)

Return `End(X)(f)`, where `f` is data that defines an element of `End(X)`.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: phi = end(R, [x + 1])
sage: phi
Ring endomorphism of Univariate Polynomial Ring in x over Rational Field
  Defn: x |--> x + 1
sage: phi(x^2 + 5)
x^2 + 2*x + 6
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> phi = end(R, [x + Integer(1)])
>>> phi
Ring endomorphism of Univariate Polynomial Ring in x over Rational Field
  Defn: x |--> x + 1
>>> phi(x**Integer(2) + Integer(5))
x^2 + 2*x + 6
```

sage.categories.homset.**hom**(*X*, *Y*, *f*)

Return `Hom(X,Y)(f)`, where `f` is data that defines an element of `Hom(X,Y)`.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: phi = hom(R, QQ, [2])
sage: phi(x^2 + 3)
7
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> phi = hom(R, QQ, [Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> phi(x**Integer(2) + Integer(3))
7
```

3.3 Morphisms

This module defines the base classes of morphisms between objects of a given category.

EXAMPLES:

Typically, a morphism is defined by the images of the generators of the domain.

```
sage: X.<a, b> = ZZ[]
sage: Y.<c> = ZZ[]
sage: X.hom([c, c^2])
Ring morphism:
  From: Multivariate Polynomial Ring in a, b over Integer Ring
  To:   Univariate Polynomial Ring in c over Integer Ring
  Defn: a |--> c
        b |--> c^2
```

```
>>> from sage.all import *
>>> X = ZZ['a, b']; (a, b,) = X._first_ngens(2)
>>> Y = ZZ['c']; (c,) = Y._first_ngens(1)
>>> X.hom([c, c**Integer(2)])
Ring morphism:
  From: Multivariate Polynomial Ring in a, b over Integer Ring
  To:   Univariate Polynomial Ring in c over Integer Ring
  Defn: a |--> c
        b |--> c^2
```

AUTHORS:

- William Stein (2005): initial version
- David Joyner (2005-12-17): added examples
- Robert Bradshaw (2007-06-25): Pyrexification

class sage.categories.morphism.**CallMorphism**

Bases: *Morphism*

class sage.categories.morphism.**FormalCoercionMorphism**

Bases: *Morphism*

class sage.categories.morphism.**IdentityMorphism**

Bases: *Morphism*

is_identity()

Return `True` if this morphism is the identity morphism.

EXAMPLES:

```
sage: E = End(Partitions(5))                                                #␣
↪needs sage.combinat
sage: E.identity().is_identity()                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> E = End(Partitions(Integer(5)))                                    ␣
↪     # needs sage.combinat
>>> E.identity().is_identity()                                        #␣
↪needs sage.combinat
True
```

Check that Issue #15478 is fixed:

```
sage: # needs sage.rings.finite_rings
sage: K.<z> = GF(4)
sage: phi = End(K)([z^2])
sage: R.<t> = K[]
sage: psi = End(R)(phi)
sage: psi.is_identity()
False
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> K = GF(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = End(K)([z**Integer(2)])
>>> R = K['t']; (t,) = R._first_ngens(1)
>>> psi = End(R)(phi)
>>> psi.is_identity()
False
```

is_injective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: Hom(ZZ, ZZ).identity().is_injective()
True
```

```
>>> from sage.all import *
>>> Hom(ZZ, ZZ).identity().is_injective()
True
```

is_surjective()

Return whether this morphism is surjective.

EXAMPLES:

```
sage: Hom(ZZ, ZZ).identity().is_surjective()
True
```

```
>>> from sage.all import *
>>> Hom(ZZ, ZZ).identity().is_surjective()
True
```

section()

Return a section of this morphism.

EXAMPLES:

```
sage: T = Hom(ZZ, ZZ).identity()
sage: T.section() is T
True
```

```
>>> from sage.all import *
>>> T = Hom(ZZ, ZZ).identity()
>>> T.section() is T
True
```

class sage.categories.morphism.**Morphism**

Bases: *Map*

category()

Return the category of the parent of this morphism.

EXAMPLES:

```
sage: R.<t> = ZZ[]
sage: f = R.hom([t**2])
sage: f.category()
Category of endsets of unital magmas and right modules over
 (euclidean domains and noetherian rings
  and infinite enumerated sets and metric spaces)
 and left modules over
 (euclidean domains and noetherian rings
  and infinite enumerated sets and metric spaces)

sage: # needs sage.rings.number_field
sage: K = CyclotomicField(12)
sage: L = CyclotomicField(132)
sage: phi = L._internal_coerce_map_from(K)
sage: phi.category()
Category of homsets of number fields
```

```
>>> from sage.all import *
>>> R = ZZ['t']; (t,) = R._first_ngens(1)
>>> f = R.hom([t**Integer(2)])
>>> f.category()
Category of endsets of unital magmas and right modules over
 (euclidean domains and noetherian rings
  and infinite enumerated sets and metric spaces)
 and left modules over
 (euclidean domains and noetherian rings
  and infinite enumerated sets and metric spaces)

>>> # needs sage.rings.number_field
>>> K = CyclotomicField(Integer(12))
>>> L = CyclotomicField(Integer(132))
>>> phi = L._internal_coerce_map_from(K)
```

(continues on next page)

(continued from previous page)

```
>>> phi.category()
Category of homsets of number fields
```

is_endomorphism()

Return `True` if this morphism is an endomorphism.

EXAMPLES:

```
sage: R.<t> = ZZ[]
sage: f = R.hom([t])
sage: f.is_endomorphism()
True

sage: # needs sage.rings.number_field
sage: K = CyclotomicField(12)
sage: L = CyclotomicField(132)
sage: phi = L._internal_coerce_map_from(K)
sage: phi.is_endomorphism()
False
```

```
>>> from sage.all import *
>>> R = ZZ['t']; (t,) = R._first_ngens(1)
>>> f = R.hom([t])
>>> f.is_endomorphism()
True

>>> # needs sage.rings.number_field
>>> K = CyclotomicField(Integer(12))
>>> L = CyclotomicField(Integer(132))
>>> phi = L._internal_coerce_map_from(K)
>>> phi.is_endomorphism()
False
```

is_identity()

Return `True` if this morphism is the identity morphism.

> **Note**
>
> Implemented only when the domain has a method gens()

EXAMPLES:

```
sage: R.<t> = ZZ[]
sage: f = R.hom([t])
sage: f.is_identity()
True
sage: g = R.hom([t + 1])
sage: g.is_identity()
False
```

```
>>> from sage.all import *
>>> R = ZZ['t']; (t,) = R._first_ngens(1)
>>> f = R.hom([t])
>>> f.is_identity()
True
>>> g = R.hom([t + Integer(1)])
>>> g.is_identity()
False
```

A morphism between two different spaces cannot be the identity:

```
sage: R2.<t2> = QQ[]
sage: h = R.hom([t2])
sage: h.is_identity()
False
```

```
>>> from sage.all import *
>>> R2 = QQ['t2']; (t2,) = R2._first_ngens(1)
>>> h = R.hom([t2])
>>> h.is_identity()
False
```

pushforward(*I*)

register_as_coercion()

Register this morphism as a coercion to Sage's coercion model (see `sage.structure.coerce`).

EXAMPLES:

By default, adding polynomials over different variables triggers an error:

```
sage: X.<x> = ZZ[]
sage: Y.<y> = ZZ[]
sage: x^2 + y
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Univariate Polynomial Ring in x over Integer Ring' and
'Univariate Polynomial Ring in y over Integer Ring'
```

```
>>> from sage.all import *
>>> X = ZZ['x']; (x,) = X._first_ngens(1)
>>> Y = ZZ['y']; (y,) = Y._first_ngens(1)
>>> x**Integer(2) + y
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Univariate Polynomial Ring in x over Integer Ring' and
'Univariate Polynomial Ring in y over Integer Ring'
```

Let us declare a coercion from $\mathbf{Z}[x]$ to $\mathbf{Z}[z]$:

```
sage: Z.<z> = ZZ[]
sage: phi = Hom(X, Z)(z)
```

(continues on next page)

(continued from previous page)

```
sage: phi(x^2+1)
z^2 + 1
sage: phi.register_as_coercion()
```

```
>>> from sage.all import *
>>> Z = ZZ['z']; (z,) = Z._first_ngens(1)
>>> phi = Hom(X, Z)(z)
>>> phi(x**Integer(2)+Integer(1))
z^2 + 1
>>> phi.register_as_coercion()
```

Now we can add elements from $\mathbf{Z}[x]$ and $\mathbf{Z}[z]$, because the elements of the former are allowed to be implicitly coerced into the later:

```
sage: x^2 + z
z^2 + z
```

```
>>> from sage.all import *
>>> x**Integer(2) + z
z^2 + z
```

Caveat: the registration of the coercion must be done before any other coercion is registered or discovered:

```
sage: phi = Hom(X, Z)(z^2)
sage: phi.register_as_coercion()
Traceback (most recent call last):
...
AssertionError: coercion from Univariate Polynomial Ring in x over Integer␣
↪Ring
to Univariate Polynomial Ring in z over Integer Ring
already registered or discovered
```

```
>>> from sage.all import *
>>> phi = Hom(X, Z)(z**Integer(2))
>>> phi.register_as_coercion()
Traceback (most recent call last):
...
AssertionError: coercion from Univariate Polynomial Ring in x over Integer␣
↪Ring
to Univariate Polynomial Ring in z over Integer Ring
already registered or discovered
```

register_as_conversion()

Register this morphism as a conversion to Sage's coercion model.

(see sage.structure.coerce).

EXAMPLES:

Let us declare a conversion from the symmetric group to $\mathbf{Z}$ through the sign map:

```
sage: # needs sage.groups
sage: S = SymmetricGroup(4)
```

(continues on next page)

(continued from previous page)

```
sage: phi = Hom(S, ZZ)(lambda x: ZZ(x.sign()))
sage: x = S.an_element(); x
(2,3,4)
sage: phi(x)
1
sage: phi.register_as_conversion()
sage: ZZ(x)
1
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> S = SymmetricGroup(Integer(4))
>>> phi = Hom(S, ZZ)(lambda x: ZZ(x.sign()))
>>> x = S.an_element(); x
(2,3,4)
>>> phi(x)
1
>>> phi.register_as_conversion()
>>> ZZ(x)
1
```

class sage.categories.morphism.**SetIsomorphism**

Bases: *SetMorphism*

An isomorphism of sets.

INPUT:

- `parent` – a Homset
- `function` – a Python function that takes elements of the domain as input and returns elements of the codomain

EXAMPLES:

```
sage: f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
....:                                              operator.__neg__); f
Generic endomorphism of Integer Ring
sage: f._set_inverse(f)
sage: ~f is f
True
```

```
>>> from sage.all import *
>>> f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
...                                             operator.__neg__); f
Generic endomorphism of Integer Ring
>>> f._set_inverse(f)
>>> ~f is f
True
```

is_injective()

Return whether this morphism is injective.

EXAMPLES:

```
sage: f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
....:                                             operator.__neg__)
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
...                                             operator.__neg__)
>>> f.is_injective()
True
```

is_surjective()

Return whether this morphism is surjective.

EXAMPLES:

```
sage: f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
....:                                             operator.__neg__)
sage: f.is_surjective()
True
```

```
>>> from sage.all import *
>>> f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
...                                             operator.__neg__)
>>> f.is_surjective()
True
```

section()

Return a section of this morphism.

EXAMPLES:

```
sage: f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
....:                                             operator.__neg__)
sage: f._set_inverse(f)
sage: f.section() is f
True
```

```
>>> from sage.all import *
>>> f = sage.categories.morphism.SetIsomorphism(Hom(ZZ, ZZ, Sets()),
...                                             operator.__neg__)
>>> f._set_inverse(f)
>>> f.section() is f
True
```

class sage.categories.morphism.**SetMorphism**

Bases: *Morphism*

INPUT:

- parent – a Homset
- function – a Python function that takes elements of the domain as input and returns elements of the codomain

EXAMPLES:

```
sage: from sage.categories.morphism import SetMorphism
sage: f = SetMorphism(Hom(QQ, ZZ, Sets()), numerator)
sage: f.parent()
Set of Morphisms from Rational Field to Integer Ring in Category of sets
sage: f.domain()
Rational Field
sage: f.codomain()
Integer Ring
sage: TestSuite(f).run()
```

```
>>> from sage.all import *
>>> from sage.categories.morphism import SetMorphism
>>> f = SetMorphism(Hom(QQ, ZZ, Sets()), numerator)
>>> f.parent()
Set of Morphisms from Rational Field to Integer Ring in Category of sets
>>> f.domain()
Rational Field
>>> f.codomain()
Integer Ring
>>> TestSuite(f).run()
```

sage.categories.morphism.**is_Morphism**(*x*)

3.4 Coercion via construction functors

class sage.categories.pushout.**AlgebraicClosureFunctor**

Bases: *ConstructionFunctor*

Algebraic Closure.

EXAMPLES:

```
sage: # needs sage.rings.complex_double sage.rings.number_field
sage: F = CDF.construction()[0]
sage: F(QQ)
Algebraic Field
sage: F(RR)                                                                 #␣
↪needs sage.rings.real_mpfr
Complex Field with 53 bits of precision
sage: F(F(QQ)) is F(QQ)
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.complex_double sage.rings.number_field
>>> F = CDF.construction()[Integer(0)]
>>> F(QQ)
Algebraic Field
>>> F(RR)                                                                   #␣
↪needs sage.rings.real_mpfr
Complex Field with 53 bits of precision
```

(continues on next page)

(continued from previous page)

```
>>> F(F(QQ)) is F(QQ)
True
```

merge(*other*)

Mathematically, Algebraic Closure subsumes Algebraic Extension. However, it seems that people do want to work with algebraic extensions of `RR`. Therefore, we do not merge with algebraic extension.

rank = 3

class sage.categories.pushout.**AlgebraicExtensionFunctor**(*polys*, *names*, *embeddings=None*, *structures=None*, *cyclotomic=None*, *precs=None*, *implementations=None*, * *(Keyword-only parameters separator (PEP 3102))*, *residue=None*, *latex_names=None*, ***kwds*)

Bases: *ConstructionFunctor*

Algebraic extension (univariate polynomial ring modulo principal ideal).

EXAMPLES:

```
sage: x = polygen(QQ, 'x')
sage: K.<a> = NumberField(x^3 + x^2 + 1)                                #␣
↪needs sage.rings.number_field
sage: F = K.construction()[0]                                           #␣
↪needs sage.rings.number_field
sage: F(ZZ['t'])                                                        #␣
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Univariate Polynomial Ring in t over Integer Ring
 with modulus a^3 + a^2 + 1
```

```
>>> from sage.all import *
>>> x = polygen(QQ, 'x')
>>> K = NumberField(x**Integer(3) + x**Integer(2) + Integer(1), names=('a',)); (a,
↪) = K._first_ngens(1)# needs sage.rings.number_field
>>> F = K.construction()[Integer(0)]                                        ␣
↪     # needs sage.rings.number_field
>>> F(ZZ['t'])                                                           #␣
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Univariate Polynomial Ring in t over Integer Ring
 with modulus a^3 + a^2 + 1
```

Note that, even if a field is algebraically closed, the algebraic extension will be constructed as the quotient of a univariate polynomial ring:

```
sage: F(CC)                                                             #␣
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Complex Field with 53 bits of precision
 with modulus a^3 + a^2 + 1.00000000000000
sage: F(RR)                                                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Real Field with 53 bits of precision
 with modulus a^3 + a^2 + 1.00000000000000
```

```
>>> from sage.all import *
>>> F(CC)                                                                    #␣
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Complex Field with 53 bits of precision
 with modulus a^3 + a^2 + 1.00000000000000
>>> F(RR)                                                                    #␣
↪needs sage.rings.number_field
Univariate Quotient Polynomial Ring in a
 over Real Field with 53 bits of precision
 with modulus a^3 + a^2 + 1.00000000000000
```

Note that the construction functor of a number field applied to the integers returns an order (not necessarily maximal) of that field, similar to the behaviour of `ZZ.extension(...)`:

```
sage: F(ZZ)                                                                  #␣
↪needs sage.rings.number_field
Order generated by a in Number Field in a with defining polynomial x^3 + x^2 + 1
```

```
>>> from sage.all import *
>>> F(ZZ)                                                                    #␣
↪needs sage.rings.number_field
Order generated by a in Number Field in a with defining polynomial x^3 + x^2 + 1
```

This also holds for non-absolute number fields:

```
sage: # needs sage.rings.number_field
sage: x = polygen(QQ, 'x')
sage: K.<a,b> = NumberField([x^3 + x^2 + 1, x^2 + x + 1])
sage: F = K.construction()[0]
sage: O = F(ZZ); O
Relative Order
 generated by [(b - 2)*a^2 + (3*b - 1)*a + 3*b + 4, a - b]
 in Number Field in a with defining polynomial x^3 + x^2 + 1
 over its base field
sage: O.ambient() is K
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> x = polygen(QQ, 'x')
>>> K = NumberField([x**Integer(3) + x**Integer(2) + Integer(1), x**Integer(2) +␣
↪x + Integer(1)], names=('a', 'b',)); (a, b,) = K._first_ngens(2)
>>> F = K.construction()[Integer(0)]
>>> O = F(ZZ); O
Relative Order
 generated by [(b - 2)*a^2 + (3*b - 1)*a + 3*b + 4, a - b]
```

(continues on next page)

(continued from previous page)

```
 in Number Field in a with defining polynomial x^3 + x^2 + 1
 over its base field
>>> O.ambient() is K
True
```

Special cases are made for cyclotomic fields and residue fields:

```
sage: # needs sage.rings.number_field
sage: C = CyclotomicField(8)
sage: F, R = C.construction()
sage: F
AlgebraicExtensionFunctor
sage: R
Rational Field
sage: F(R)
Cyclotomic Field of order 8 and degree 4
sage: F(ZZ)
Maximal Order generated by zeta8 in Cyclotomic Field of order 8 and degree 4
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> C = CyclotomicField(Integer(8))
>>> F, R = C.construction()
>>> F
AlgebraicExtensionFunctor
>>> R
Rational Field
>>> F(R)
Cyclotomic Field of order 8 and degree 4
>>> F(ZZ)
Maximal Order generated by zeta8 in Cyclotomic Field of order 8 and degree 4
```

```
sage: # needs sage.rings.number_field
sage: K.<z> = CyclotomicField(7)
sage: P = K.factor(17)[0][0]
sage: k = K.residue_field(P)
sage: F, R = k.construction()
sage: F
AlgebraicExtensionFunctor
sage: R
Cyclotomic Field of order 7 and degree 6
sage: F(R) is k
True
sage: F(ZZ)
Residue field of Integers modulo 17
sage: F(CyclotomicField(49))
Residue field in zbar of Fractional ideal (17)
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> K = CyclotomicField(Integer(7), names=('z',)); (z,) = K._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> P = K.factor(Integer(17))[Integer(0)][Integer(0)]
>>> k = K.residue_field(P)
>>> F, R = k.construction()
>>> F
AlgebraicExtensionFunctor
>>> R
Cyclotomic Field of order 7 and degree 6
>>> F(R) is k
True
>>> F(ZZ)
Residue field of Integers modulo 17
>>> F(CyclotomicField(Integer(49)))
Residue field in zbar of Fractional ideal (17)
```

expand()

Decompose the functor F into sub-functors, whose product returns F.

EXAMPLES:

```
sage: # needs sage.rings.number_field
sage: P.<x> = QQ[]
sage: K.<a> = NumberField(x^3 - 5, embedding=0)
sage: L.<b> = K.extension(x^2 + a)
sage: F, R = L.construction()
sage: prod(F.expand())(R) == L
True
sage: K = NumberField([x^2 - 2, x^2 - 3],'a')
sage: F, R = K.construction()
sage: F
AlgebraicExtensionFunctor
sage: L = F.expand(); L
[AlgebraicExtensionFunctor, AlgebraicExtensionFunctor]
sage: L[-1](QQ)
Number Field in a1 with defining polynomial x^2 - 3
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> K = NumberField(x**Integer(3) - Integer(5), embedding=Integer(0), names=(
↪'a',)); (a,) = K._first_ngens(1)
>>> L = K.extension(x**Integer(2) + a, names=('b',)); (b,) = L._first_ngens(1)
>>> F, R = L.construction()
>>> prod(F.expand())(R) == L
True
>>> K = NumberField([x**Integer(2) - Integer(2), x**Integer(2) - Integer(3)],
↪'a')
>>> F, R = K.construction()
>>> F
AlgebraicExtensionFunctor
>>> L = F.expand(); L
[AlgebraicExtensionFunctor, AlgebraicExtensionFunctor]
>>> L[-Integer(1)](QQ)
```

(continues on next page)

(continued from previous page)

```
Number Field in a1 with defining polynomial x^2 - 3
```

merge(*other*)

Merging with another *AlgebraicExtensionFunctor*.

INPUT:

- `other` – Construction Functor

OUTPUT:

- If `self==other`, `self` is returned.
- If `self` and `other` are simple extensions and both provide an embedding, then it is tested whether one of the number fields provided by the functors coerces into the other; the functor associated with the target of the coercion is returned. Otherwise, the construction functor associated with the pushout of the codomains of the two embeddings is returned, provided that it is a number field.
- If these two extensions are defined by Conway polynomials over finite fields, merges them into a single extension of degree the lcm of the two degrees.
- Otherwise, `None` is returned.

REMARK:

Algebraic extension with embeddings currently only works when applied to the rational field. This is why we use the admittedly strange rule above for merging.

EXAMPLES:

The following demonstrate coercions for finite fields using Conway or pseudo-Conway polynomials:

```
sage: k = GF(3^2, prefix='z'); a = k.gen()                                  #␣
↪needs sage.rings.finite_rings
sage: l = GF(3^3, prefix='z'); b = l.gen()                                  #␣
↪needs sage.rings.finite_rings
sage: a + b  # indirect doctest                                             #␣
↪needs sage.rings.finite_rings
z6^5 + 2*z6^4 + 2*z6^3 + z6^2 + 2*z6 + 1
```

```
>>> from sage.all import *
>>> k = GF(Integer(3)**Integer(2), prefix='z'); a = k.gen()                 ␣
↪               # needs sage.rings.finite_rings
>>> l = GF(Integer(3)**Integer(3), prefix='z'); b = l.gen()                 ␣
↪               # needs sage.rings.finite_rings
>>> a + b  # indirect doctest                                              #␣
↪needs sage.rings.finite_rings
z6^5 + 2*z6^4 + 2*z6^3 + z6^2 + 2*z6 + 1
```

Note that embeddings are compatible in lattices of such finite fields:

```
sage: # needs sage.rings.finite_rings
sage: m = GF(3^5, prefix='z'); c = m.gen()
sage: (a + b) + c == a + (b + c) # indirect doctest
True
sage: from sage.categories.pushout import pushout
sage: n = pushout(k, l)
```

(continues on next page)

(continued from previous page)

```
sage: o = pushout(l, m)
sage: q = pushout(n, o)
sage: q(o(b)) == q(n(b)) # indirect doctest
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> m = GF(Integer(3)**Integer(5), prefix='z'); c = m.gen()
>>> (a + b) + c == a + (b + c) # indirect doctest
True
>>> from sage.categories.pushout import pushout
>>> n = pushout(k, l)
>>> o = pushout(l, m)
>>> q = pushout(n, o)
>>> q(o(b)) == q(n(b)) # indirect doctest
True
```

Coercion is also available for number fields:

```
sage: # needs sage.rings.number_field
sage: P.<x> = QQ[]
sage: L.<b> = NumberField(x^8 - x^4 + 1, embedding=CDF.0)
sage: M1.<c1> = NumberField(x^2 + x + 1, embedding=b^4 - 1)
sage: M2.<c2> = NumberField(x^2 + 1, embedding=-b^6)
sage: M1.coerce_map_from(M2)
sage: M2.coerce_map_from(M1)
sage: c1 + c2; parent(c1 + c2)    #indirect doctest
-b^6 + b^4 - 1
Number Field in b with defining polynomial x^8 - x^4 + 1
 with b = -0.2588190451025208? + 0.9659258262890683?*I
sage: pushout(M1['x'], M2['x'])                                                 #␣
↪needs sage.rings.finite_rings
Univariate Polynomial Ring in x
 over Number Field in b with defining polynomial x^8 - x^4 + 1
  with b = -0.2588190451025208? + 0.9659258262890683?*I
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> L = NumberField(x**Integer(8) - x**Integer(4) + Integer(1), embedding=CDF.
↪gen(0), names=('b',)); (b,) = L._first_ngens(1)
>>> M1 = NumberField(x**Integer(2) + x + Integer(1), embedding=b**Integer(4) -
↪ Integer(1), names=('c1',)); (c1,) = M1._first_ngens(1)
>>> M2 = NumberField(x**Integer(2) + Integer(1), embedding=-b**Integer(6),␣
↪names=('c2',)); (c2,) = M2._first_ngens(1)
>>> M1.coerce_map_from(M2)
>>> M2.coerce_map_from(M1)
>>> c1 + c2; parent(c1 + c2)    #indirect doctest
-b^6 + b^4 - 1
Number Field in b with defining polynomial x^8 - x^4 + 1
 with b = -0.2588190451025208? + 0.9659258262890683?*I
>>> pushout(M1['x'], M2['x'])                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.finite_rings
Univariate Polynomial Ring in x
 over Number Field in b with defining polynomial x^8 - x^4 + 1
  with b = -0.2588190451025208? + 0.9659258262890683?*I
```

In the previous example, the number field `L` becomes the pushout of `M1` and `M2` since both are provided with an embedding into `L`, *and* since `L` is a number field. If two number fields are embedded into a field that is not a numberfield, no merging occurs:

```
sage: # needs sage.rings.complex_double sage.rings.number_field
sage: cbrt2 = CDF(2)^(1/3)
sage: zeta3 = CDF.zeta(3)
sage: K.<a> = NumberField(x^3 - 2, embedding=cbrt2 * zeta3)
sage: L.<b> = NumberField(x^6 - 2, embedding=1.1)
sage: L.coerce_map_from(K)
sage: K.coerce_map_from(L)
sage: pushout(K, L)                                                         #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', Number Field in a with
defining polynomial x^3 - 2 with a = -0.6299605249474365? + 1.091123635971722?
↪*I,
Number Field in b with defining polynomial x^6 - 2 with b = 1.122462048309373?
↪)
```

```
>>> from sage.all import *
>>> # needs sage.rings.complex_double sage.rings.number_field
>>> cbrt2 = CDF(Integer(2))**(Integer(1)/Integer(3))
>>> zeta3 = CDF.zeta(Integer(3))
>>> K = NumberField(x**Integer(3) - Integer(2), embedding=cbrt2 * zeta3,␣
↪names=('a',)); (a,) = K._first_ngens(1)
>>> L = NumberField(x**Integer(6) - Integer(2), embedding=RealNumber('1.1'),␣
↪names=('b',)); (b,) = L._first_ngens(1)
>>> L.coerce_map_from(K)
>>> K.coerce_map_from(L)
>>> pushout(K, L)                                                           #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', Number Field in a with
defining polynomial x^3 - 2 with a = -0.6299605249474365? + 1.091123635971722?
↪*I,
Number Field in b with defining polynomial x^6 - 2 with b = 1.122462048309373?
↪)
```

rank = 3

class sage.categories.pushout.**BlackBoxConstructionFunctor**(*box*)

Bases: *ConstructionFunctor*

Construction functor obtained from any callable object.

EXAMPLES:

```
sage: from sage.categories.pushout import BlackBoxConstructionFunctor

sage: # needs sage.libs.gap
sage: from sage.interfaces.gap import gap
sage: FG = BlackBoxConstructionFunctor(gap)
sage: FG
BlackBoxConstructionFunctor
sage: FG(ZZ)
Integers
sage: FG(ZZ).parent()
Gap
sage: FG == loads(dumps(FG))
True

sage: FS = BlackBoxConstructionFunctor(singular)
sage: FS(QQ['t'])                                                           #␣
↪needs sage.libs.singular
polynomial ring, over a field, global ordering
// coefficients: QQ...
// number of vars : 1
//        block   1 : ordering lp
//                  : names    t
//        block   2 : ordering C
sage: FG == FS                                                              #␣
↪needs sage.libs.gap sage.libs.singular
False
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import BlackBoxConstructionFunctor

>>> # needs sage.libs.gap
>>> from sage.interfaces.gap import gap
>>> FG = BlackBoxConstructionFunctor(gap)
>>> FG
BlackBoxConstructionFunctor
>>> FG(ZZ)
Integers
>>> FG(ZZ).parent()
Gap
>>> FG == loads(dumps(FG))
True

>>> FS = BlackBoxConstructionFunctor(singular)
>>> FS(QQ['t'])                                                             #␣
↪needs sage.libs.singular
polynomial ring, over a field, global ordering
// coefficients: QQ...
// number of vars : 1
//        block   1 : ordering lp
//                  : names    t
//        block   2 : ordering C
>>> FG == FS                                                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.gap sage.libs.singular
False
```

rank = 100

class sage.categories.pushout.**CompletionFunctor**(*p*, *prec*, *extras=None*)

Bases: *ConstructionFunctor*

Completion of a ring with respect to a given prime (including infinity).

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: R = Zp(5)
sage: R
5-adic Ring with capped relative precision 20
sage: F1 = R.construction()[0]
sage: F1
Completion[5, prec=20]
sage: F1(ZZ) is R
True
sage: F1(QQ)
5-adic Field with capped relative precision 20

sage: F2 = RR.construction()[0]
sage: F2
Completion[+Infinity, prec=53]
sage: F2(QQ) is RR
True

sage: P.<x> = ZZ[]
sage: Px = P.completion(x) # currently the only implemented completion of P
sage: Px
Power Series Ring in x over Integer Ring
sage: F3 = Px.construction()[0]
sage: F3(GF(3)['x'])
Power Series Ring in x over Finite Field of size 3
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> R = Zp(Integer(5))
>>> R
5-adic Ring with capped relative precision 20
>>> F1 = R.construction()[Integer(0)]
>>> F1
Completion[5, prec=20]
>>> F1(ZZ) is R
True
>>> F1(QQ)
5-adic Field with capped relative precision 20

>>> F2 = RR.construction()[Integer(0)]
>>> F2
```

(continues on next page)

(continued from previous page)

```
Completion[+Infinity, prec=53]
>>> F2(QQ) is RR
True

>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> Px = P.completion(x) # currently the only implemented completion of P
>>> Px
Power Series Ring in x over Integer Ring
>>> F3 = Px.construction()[Integer(0)]
>>> F3(GF(Integer(3))['x'])
Power Series Ring in x over Finite Field of size 3
```

commutes(*other*)

Completion commutes with fraction fields.

EXAMPLES:

```
sage: F1 = Zp(5).construction()[0]                                              #␣
↪needs sage.rings.padics
sage: F2 = QQ.construction()[0]
sage: F1.commutes(F2)                                                           #␣
↪needs sage.rings.padics
True
```

```
>>> from sage.all import *
>>> F1 = Zp(Integer(5)).construction()[Integer(0)]                                ␣
↪              # needs sage.rings.padics
>>> F2 = QQ.construction()[Integer(0)]
>>> F1.commutes(F2)                                                            #␣
↪needs sage.rings.padics
True
```

merge(*other*)

Two Completion functors are merged, if they are equal. If the precisions of both functors coincide, then a Completion functor is returned that results from updating the `extras` dictionary of `self` by `other.extras`. Otherwise, if the completion is at infinity then merging does not increase the set precision, and if the completion is at a finite prime, merging does not decrease the capped precision.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: R1.<a> = Zp(5, prec=20)[]
sage: R2 = Qp(5, prec=40)
sage: R2(1) + a       # indirect doctest
(1 + O(5^20))*a + 1 + O(5^40)
sage: R3 = RealField(30)
sage: R4 = RealField(50)
sage: R3(1) + R4(1)   # indirect doctest
2.0000000
sage: (R3(1) + R4(1)).parent()
Real Field with 30 bits of precision
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> R1 = Zp(Integer(5), prec=Integer(20))['a']; (a,) = R1._first_ngens(1)
>>> R2 = Qp(Integer(5), prec=Integer(40))
>>> R2(Integer(1)) + a       # indirect doctest
(1 + O(5^20))*a + 1 + O(5^40)
>>> R3 = RealField(Integer(30))
>>> R4 = RealField(Integer(50))
>>> R3(Integer(1)) + R4(Integer(1))    # indirect doctest
2.0000000
>>> (R3(Integer(1)) + R4(Integer(1))).parent()
Real Field with 30 bits of precision
```

rank = 4

class sage.categories.pushout.**CompositeConstructionFunctor**(**args*)

Bases: *ConstructionFunctor*

A Construction Functor composed by other Construction Functors.

INPUT:

- `F1, F2, ...` – a list of Construction Functors. The result is the composition `F1` followed by `F2` followed by ...

EXAMPLES:

```
sage: from sage.categories.pushout import CompositeConstructionFunctor
sage: F = CompositeConstructionFunctor(QQ.construction()[0], ZZ['x'].
↪construction()[0],
....:                                  QQ.construction()[0], ZZ['y'].
↪construction()[0])
sage: F
Poly[y](FractionField(Poly[x](FractionField(...))))
sage: F == loads(dumps(F))
True
sage: F == CompositeConstructionFunctor(*F.all)
True
sage: F(GF(2)['t'])                                                    #␣
↪needs sage.libs.ntl
Univariate Polynomial Ring in y
 over Fraction Field of Univariate Polynomial Ring in x
  over Fraction Field of Univariate Polynomial Ring in t
   over Finite Field of size 2 (using GF2X)
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import CompositeConstructionFunctor
>>> F = CompositeConstructionFunctor(QQ.construction()[Integer(0)], ZZ['x'].
↪construction()[Integer(0)],
...                                  QQ.construction()[Integer(0)], ZZ['y'].
↪construction()[Integer(0)])
>>> F
Poly[y](FractionField(Poly[x](FractionField(...))))
>>> F == loads(dumps(F))
True
```

(continues on next page)

(continued from previous page)

```
>>> F == CompositeConstructionFunctor(*F.all)
True
>>> F(GF(Integer(2))['t'])                                                    ␣
↪      # needs sage.libs.ntl
Univariate Polynomial Ring in y
 over Fraction Field of Univariate Polynomial Ring in x
  over Fraction Field of Univariate Polynomial Ring in t
   over Finite Field of size 2 (using GF2X)
```

expand()

Return expansion of a CompositeConstructionFunctor.

> **Note**
>
> The product over the list of components, as returned by the `expand()` method, is equal to `self`.

EXAMPLES:

```
sage: from sage.categories.pushout import CompositeConstructionFunctor
sage: F = CompositeConstructionFunctor(QQ.construction()[0],
....:                                  ZZ['x'].construction()[0],
....:                                  QQ.construction()[0],
....:                                  ZZ['y'].construction()[0])
sage: F
Poly[y](FractionField(Poly[x](FractionField(...))))
sage: prod(F.expand()) == F
True
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import CompositeConstructionFunctor
>>> F = CompositeConstructionFunctor(QQ.construction()[Integer(0)],
...                                  ZZ['x'].construction()[Integer(0)],
...                                  QQ.construction()[Integer(0)],
...                                  ZZ['y'].construction()[Integer(0)])
>>> F
Poly[y](FractionField(Poly[x](FractionField(...))))
>>> prod(F.expand()) == F
True
```

class sage.categories.pushout.**ConstructionFunctor**

Bases: *Functor*

Base class for construction functors.

A construction functor is a functorial algebraic construction, such as the construction of a matrix ring over a given ring or the fraction field of a given ring.

In addition to the class *Functor*, construction functors provide rules for combining and merging constructions. This is an important part of Sage's coercion model, namely the pushout of two constructions: When a polynomial `p` in a variable `x` with integer coefficients is added to a rational number `q`, then Sage finds that the parents `ZZ['x']` and `QQ` are obtained from `ZZ` by applying a polynomial ring construction respectively the fraction field construction. Each construction functor has an attribute `rank`, and the rank of the polynomial ring construction is higher than

the rank of the fraction field construction. This means that the pushout of `QQ` and `ZZ['x']`, and thus a common parent in which `p` and `q` can be added, is `QQ['x']`, since the construction functor with a lower rank is applied first.

```
sage: F1, R = QQ.construction()
sage: F1
FractionField
sage: R
Integer Ring
sage: F2, R = (ZZ['x']).construction()
sage: F2
Poly[x]
sage: R
Integer Ring
sage: F3 = F2.pushout(F1)
sage: F3
Poly[x](FractionField(...))
sage: F3(R)
Univariate Polynomial Ring in x over Rational Field
sage: from sage.categories.pushout import pushout
sage: P.<x> = ZZ[]
sage: pushout(QQ,P)
Univariate Polynomial Ring in x over Rational Field
sage: ((x+1) + 1/2).parent()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> F1, R = QQ.construction()
>>> F1
FractionField
>>> R
Integer Ring
>>> F2, R = (ZZ['x']).construction()
>>> F2
Poly[x]
>>> R
Integer Ring
>>> F3 = F2.pushout(F1)
>>> F3
Poly[x](FractionField(...))
>>> F3(R)
Univariate Polynomial Ring in x over Rational Field
>>> from sage.categories.pushout import pushout
>>> P = ZZ['x']; (x,) = P._first_ngens(1)
>>> pushout(QQ,P)
Univariate Polynomial Ring in x over Rational Field
>>> ((x+Integer(1)) + Integer(1)/Integer(2)).parent()
Univariate Polynomial Ring in x over Rational Field
```

When composing two construction functors, they are sometimes merged into one, as is the case in the Quotient construction:

```
sage: Q15, R = (ZZ.quo(15*ZZ)).construction()
sage: Q15
```

(continues on next page)

(continued from previous page)

```
QuotientFunctor
sage: Q35, R = (ZZ.quo(35*ZZ)).construction()
sage: Q35
QuotientFunctor
sage: Q15.merge(Q35)
QuotientFunctor
sage: Q15.merge(Q35)(ZZ)
Ring of integers modulo 5
```

```
>>> from sage.all import *
>>> Q15, R = (ZZ.quo(Integer(15)*ZZ)).construction()
>>> Q15
QuotientFunctor
>>> Q35, R = (ZZ.quo(Integer(35)*ZZ)).construction()
>>> Q35
QuotientFunctor
>>> Q15.merge(Q35)
QuotientFunctor
>>> Q15.merge(Q35)(ZZ)
Ring of integers modulo 5
```

Functors can not only be applied to objects, but also to morphisms in the respective categories. For example:

```
sage: P.<x,y> = ZZ[]
sage: F = P.construction()[0]; F
MPoly[x,y]
sage: A.<a,b> = GF(5)[]
sage: f = A.hom([a + b, a - b], A)
sage: F(A)
Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
sage: F(f)
Ring endomorphism of Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in a, b
         over Finite Field of size 5
          Defn: a |--> a + b
                b |--> a - b
sage: F(f)(F(A)(x)*a)
(a + b)*x
```

```
>>> from sage.all import *
>>> P = ZZ['x, y']; (x, y,) = P._first_ngens(2)
>>> F = P.construction()[Integer(0)]; F
MPoly[x,y]
>>> A = GF(Integer(5))['a, b']; (a, b,) = A._first_ngens(2)
>>> f = A.hom([a + b, a - b], A)
>>> F(A)
Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
>>> F(f)
```

(continues on next page)

(continued from previous page)

```
Ring endomorphism of Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in a, b
         over Finite Field of size 5
          Defn: a |--> a + b
                b |--> a - b
>>> F(f)(F(A)(x)*a)
(a + b)*x
```

coercion_reversed = False

common_base(*other_functor*, *self_bases*, *other_bases*)

This function is called by *pushout()* when no common parent is found in the construction tower.

> **Note**
>
> The main use is for multivariate construction functors, which use this function to implement recursion for *pushout()*.

INPUT:

- other_functor – a construction functor
- self_bases – the arguments passed to this functor
- other_bases – the arguments passed to the functor other_functor

OUTPUT:

Nothing, since a CoercionException is raised.

> **Note**
>
> Overload this function in derived class, see e.e. *MultivariateConstructionFunctor*.

commutes(*other*)

Determine whether self commutes with another construction functor.

> **Note**
>
> By default, False is returned in all cases (even if the two functors are the same, since in this case *merge()* will apply anyway). So far there is no construction functor that overloads this method. Anyway, this method only becomes relevant if two construction functors have the same rank.

EXAMPLES:

```
sage: F = QQ.construction()[0]
sage: P = ZZ['t'].construction()[0]
sage: F.commutes(P)
False
sage: P.commutes(F)
```

(continues on next page)

(continued from previous page)

```
False
sage: F.commutes(F)
False
```

```
>>> from sage.all import *
>>> F = QQ.construction()[Integer(0)]
>>> P = ZZ['t'].construction()[Integer(0)]
>>> F.commutes(P)
False
>>> P.commutes(F)
False
>>> F.commutes(F)
False
```

expand()

Decompose `self` into a list of construction functors.

> **Note**
>
> The default is to return the list only containing `self`.

EXAMPLES:

```
sage: F = QQ.construction()[0]
sage: F.expand()
[FractionField]
sage: Q = ZZ.quo(2).construction()[0]
sage: Q.expand()
[QuotientFunctor]
sage: P = ZZ['t'].construction()[0]
sage: FP = F*P
sage: FP.expand()
[FractionField, Poly[t]]
```

```
>>> from sage.all import *
>>> F = QQ.construction()[Integer(0)]
>>> F.expand()
[FractionField]
>>> Q = ZZ.quo(Integer(2)).construction()[Integer(0)]
>>> Q.expand()
[QuotientFunctor]
>>> P = ZZ['t'].construction()[Integer(0)]
>>> FP = F*P
>>> FP.expand()
[FractionField, Poly[t]]
```

merge(*other*)

Merge `self` with another construction functor, or return `None`.

> **Note**
>
> The default is to merge only if the two functors coincide. But this may be overloaded for subclasses, such as the quotient functor.

EXAMPLES:

```
sage: F = QQ.construction()[0]
sage: P = ZZ['t'].construction()[0]
sage: F.merge(F)
FractionField
sage: F.merge(P)
sage: P.merge(F)
sage: P.merge(P)
Poly[t]
```

```
>>> from sage.all import *
>>> F = QQ.construction()[Integer(0)]
>>> P = ZZ['t'].construction()[Integer(0)]
>>> F.merge(F)
FractionField
>>> F.merge(P)
>>> P.merge(F)
>>> P.merge(P)
Poly[t]
```

pushout(*other*)

Composition of two construction functors, ordered by their ranks.

> **Note**
>
> - This method seems not to be used in the coercion model.
> - By default, the functor with smaller rank is applied first.

class sage.categories.pushout.**EquivariantSubobjectConstructionFunctor**(*S*, *action=<built-in function mul>*, *side='left'*, *other_action=None*, *other_side='left'*)

Bases: *ConstructionFunctor*

Constructor for subobjects invariant or equivariant under given semigroup actions.

Let S be a semigroup that - acts on a parent X as $s \cdot x$ (`action`, `side='left'`) or - acts on X as $x \cdot s$ (`action`, `side='right'`), and (possibly trivially) - acts on X as $s * x$ (`other_action`, `other_side='left'`) or - acts on X as $x * s$ (`other_action`, `other_side='right'`).

The S-equivariant subobject is the subobject

$$X^S := \{x \in X : s \cdot x = s * x, \forall s \in S\}$$

when `side = other_side = 'left'` and mutatis mutandis for the other values of `side` and `other_side`.

When `other_action` is trivial, X^S is called the S-invariant subobject.

EXAMPLES:

Monoterm symmetries of a tensor, here only for matrices: row (index 0), column (index 1); the order of the extra element 2 in a permutation determines whether it is a symmetry or an antisymmetry:

```
sage: # needs sage.groups sage.modules
sage: GSym01 = PermutationGroup([[(0,1),(2,),(3,)]]); GSym01
Permutation Group with generators [(0,1)]
sage: GASym01 = PermutationGroup([[(0,1),(2,3)]]); GASym01
Permutation Group with generators [(0,1)(2,3)]
sage: from sage.categories.action import Action
sage: from sage.structure.element import Matrix
sage: class TensorIndexAction(Action):
....:     def _act_(self, g, x):
....:         if isinstance(x, Matrix):
....:             if g(0) == 1:
....:                 if g(2) == 2:
....:                     return x.transpose()
....:                 return -x.transpose()
....:             else:
....:                 return x
....:         raise NotImplementedError
sage: M = matrix([[1, 2], [3, 4]]); M
[1 2]
[3 4]
sage: GSym01_action = TensorIndexAction(GSym01, M.parent())
sage: GASym01_action = TensorIndexAction(GASym01, M.parent())
sage: GSym01_action.act(GSym01.0, M)
[1 3]
[2 4]
sage: GASym01_action.act(GASym01.0, M)
[-1 -3]
[-2 -4]
sage: Sym01 = M.parent().invariant_module(GSym01, action=GSym01_action); Sym01
(Permutation Group with generators [(0,1)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
sage: list(Sym01.basis())
[B[0], B[1], B[2]]
sage: list(Sym01.basis().map(Sym01.lift))
[
[1 0]  [0 1]  [0 0]
[0 0], [1 0], [0 1]
]
sage: ASym01 = M.parent().invariant_module(GASym01, action=GASym01_action)
sage: ASym01
(Permutation Group with generators [(0,1)(2,3)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
sage: list(ASym01.basis())
[B[0]]
sage: list(ASym01.basis().map(ASym01.lift))
[
[ 0  1]
```

(continues on next page)

(continued from previous page)

```
[-1  0]
]
sage: from sage.categories.pushout import pushout
sage: pushout(Sym01, QQ)
(Permutation Group with generators [(0,1)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> GSym01 = PermutationGroup([[(Integer(0),Integer(1)),(Integer(2),),(Integer(3),
↪)]]); GSym01
Permutation Group with generators [(0,1)]
>>> GASym01 = PermutationGroup([[(Integer(0),Integer(1)),(Integer(2),
↪Integer(3))]]); GASym01
Permutation Group with generators [(0,1)(2,3)]
>>> from sage.categories.action import Action
>>> from sage.structure.element import Matrix
>>> class TensorIndexAction(Action):
...     def _act_(self, g, x):
...         if isinstance(x, Matrix):
...             if g(Integer(0)) == Integer(1):
...                 if g(Integer(2)) == Integer(2):
...                     return x.transpose()
...                 return -x.transpose()
...             else:
...                 return x
...         raise NotImplementedError
>>> M = matrix([[Integer(1), Integer(2)], [Integer(3), Integer(4)]]); M
[1 2]
[3 4]
>>> GSym01_action = TensorIndexAction(GSym01, M.parent())
>>> GASym01_action = TensorIndexAction(GASym01, M.parent())
>>> GSym01_action.act(GSym01.gen(0), M)
[1 3]
[2 4]
>>> GASym01_action.act(GASym01.gen(0), M)
[-1 -3]
[-2 -4]
>>> Sym01 = M.parent().invariant_module(GSym01, action=GSym01_action); Sym01
(Permutation Group with generators [(0,1)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
>>> list(Sym01.basis())
[B[0], B[1], B[2]]
>>> list(Sym01.basis().map(Sym01.lift))
[
[1 0]  [0 1]  [0 0]
[0 0], [1 0], [0 1]
]
>>> ASym01 = M.parent().invariant_module(GASym01, action=GASym01_action)
>>> ASym01
(Permutation Group with generators [(0,1)(2,3)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
```

(continues on next page)

(continued from previous page)

```
>>> list(ASym01.basis())
[B[0]]
>>> list(ASym01.basis().map(ASym01.lift))
[
[ 0  1]
[-1  0]
]
>>> from sage.categories.pushout import pushout
>>> pushout(Sym01, QQ)
(Permutation Group with generators [(0,1)])-invariant submodule
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

class sage.categories.pushout.**FractionField**

Bases: *ConstructionFunctor*

Construction functor for fraction fields.

EXAMPLES:

```
sage: F = QQ.construction()[0]
sage: F
FractionField
sage: F.domain()
Category of integral domains
sage: F.codomain()
Category of fields
sage: F(GF(5)) is GF(5)
True
sage: F(ZZ['t'])
Fraction Field of Univariate Polynomial Ring in t over Integer Ring
sage: P.<x,y> = QQ[]
sage: f = P.hom([x+2*y,3*x-y],P)
sage: F(f)
Ring endomorphism of
 Fraction Field of Multivariate Polynomial Ring in x, y over Rational Field
  Defn: x |--> x + 2*y
        y |--> 3*x - y
sage: F(f)(1/x)
1/(x + 2*y)
sage: F == loads(dumps(F))
True
```

```
>>> from sage.all import *
>>> F = QQ.construction()[Integer(0)]
>>> F
FractionField
>>> F.domain()
Category of integral domains
>>> F.codomain()
Category of fields
>>> F(GF(Integer(5))) is GF(Integer(5))
True
>>> F(ZZ['t'])
```

(continues on next page)

(continued from previous page)

```
Fraction Field of Univariate Polynomial Ring in t over Integer Ring
>>> P = QQ['x, y']; (x, y,) = P._first_ngens(2)
>>> f = P.hom([x+Integer(2)*y,Integer(3)*x-y],P)
>>> F(f)
Ring endomorphism of
 Fraction Field of Multivariate Polynomial Ring in x, y over Rational Field
  Defn: x |--> x + 2*y
        y |--> 3*x - y
>>> F(f)(Integer(1)/x)
1/(x + 2*y)
>>> F == loads(dumps(F))
True
```

rank = 5

class sage.categories.pushout.**IdentityConstructionFunctor**

Bases: *ConstructionFunctor*

A construction functor that is the identity functor.

rank = -100

class sage.categories.pushout.**InfinitePolynomialFunctor**(*gens*, *order*, *implementation*)

Bases: *ConstructionFunctor*

A Construction Functor for Infinite Polynomial Rings (see infinite_polynomial_ring).

AUTHOR:

– Simon King

This construction functor is used to provide uniqueness of infinite polynomial rings as parent structures. As usual, the construction functor allows for constructing pushouts.

Another purpose is to avoid name conflicts of variables of the to-be-constructed infinite polynomial ring with variables of the base ring, and moreover to keep the internal structure of an Infinite Polynomial Ring as simple as possible: If variables $v_1, ..., v_n$ of the given base ring generate an *ordered* sub-monoid of the monomials of the ambient Infinite Polynomial Ring, then they are removed from the base ring and merged with the generators of the ambient ring. However, if the orders don't match, an error is raised, since there was a name conflict without merging.

EXAMPLES:

```
sage: A.<a,b> = InfinitePolynomialRing(ZZ['t'])
sage: A.construction()
[InfPoly{[a,b], "lex", "dense"},
 Univariate Polynomial Ring in t over Integer Ring]
sage: type(_[0])
<class 'sage.categories.pushout.InfinitePolynomialFunctor'>
sage: B.<x,y,a_3,a_1> = PolynomialRing(QQ, order='lex')
sage: B.construction()
(MPoly[x,y,a_3,a_1], Rational Field)
sage: A.construction()[0] * B.construction()[0]
InfPoly{[a,b], "lex", "dense"}(MPoly[x,y](...))
```

```
>>> from sage.all import *
>>> A = InfinitePolynomialRing(ZZ['t'], names=('a', 'b',)); (a, b,) = A._first_
↪ngens(2)
>>> A.construction()
[InfPoly{[a,b], "lex", "dense"},
 Univariate Polynomial Ring in t over Integer Ring]
>>> type(_[Integer(0)])
<class 'sage.categories.pushout.InfinitePolynomialFunctor'>
>>> B = PolynomialRing(QQ, order='lex', names=('x', 'y', 'a_3', 'a_1',)); (x, y,␣
↪a_3, a_1,) = B._first_ngens(4)
>>> B.construction()
(MPoly[x,y,a_3,a_1], Rational Field)
>>> A.construction()[Integer(0)] * B.construction()[Integer(0)]
InfPoly{[a,b], "lex", "dense"}(MPoly[x,y](...))
```

Apparently the variables a_1, a_3 of the polynomial ring are merged with the variables $a_0, a_1, a_2, ...$ of the infinite polynomial ring; indeed, they form an ordered sub-structure. However, if the polynomial ring was given a different ordering, merging would not be allowed, resulting in a name conflict:

```
sage: R = PolynomialRing(QQ, names=['x','y','a_3','a_1'])
sage: A.construction()[0] * R.construction()[0]
Traceback (most recent call last):
...
CoercionException: Incompatible term orders lex, degrevlex
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=['x','y','a_3','a_1'])
>>> A.construction()[Integer(0)] * R.construction()[Integer(0)]
Traceback (most recent call last):
...
CoercionException: Incompatible term orders lex, degrevlex
```

In an infinite polynomial ring with generator a_*, the variable a_3 will always be greater than the variable a_1. Hence, the orders are incompatible in the next example as well:

```
sage: R = PolynomialRing(QQ, names=['x','y','a_1','a_3'], order='lex')
sage: A.construction()[0] * R.construction()[0]
Traceback (most recent call last):
...
CoercionException: Overlapping variables (('a', 'b'),['a_1', 'a_3'])
are incompatible
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=['x','y','a_1','a_3'], order='lex')
>>> A.construction()[Integer(0)] * R.construction()[Integer(0)]
Traceback (most recent call last):
...
CoercionException: Overlapping variables (('a', 'b'),['a_1', 'a_3'])
are incompatible
```

Another requirement is that after merging the order of the remaining variables must be unique. This is not the case in the following example, since it is not clear whether the variables x, y should be greater or smaller than the variables b_*:

```
sage: R = PolynomialRing(QQ, names=['a_3','a_1','x','y'], order='lex')
sage: A.construction()[0] * R.construction()[0]
Traceback (most recent call last):
...
CoercionException: Overlapping variables (('a', 'b'),['a_3', 'a_1'])
are incompatible
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=['a_3','a_1','x','y'], order='lex')
>>> A.construction()[Integer(0)] * R.construction()[Integer(0)]
Traceback (most recent call last):
...
CoercionException: Overlapping variables (('a', 'b'),['a_3', 'a_1'])
are incompatible
```

Since the construction functors are actually used to construct infinite polynomial rings, the following result is no surprise:

```
sage: C.<a,b> = InfinitePolynomialRing(B); C
Infinite polynomial ring in a, b
 over Multivariate Polynomial Ring in x, y over Rational Field
```

```
>>> from sage.all import *
>>> C = InfinitePolynomialRing(B, names=('a', 'b',)); (a, b,) = C._first_ngens(2);
↪ C
Infinite polynomial ring in a, b
 over Multivariate Polynomial Ring in x, y over Rational Field
```

There is also an overlap in the next example:

```
sage: X.<w,x,y> = InfinitePolynomialRing(ZZ)
sage: Y.<x,y,z> = InfinitePolynomialRing(QQ)
```

```
>>> from sage.all import *
>>> X = InfinitePolynomialRing(ZZ, names=('w', 'x', 'y',)); (w, x, y,) = X._first_
↪ngens(3)
>>> Y = InfinitePolynomialRing(QQ, names=('x', 'y', 'z',)); (x, y, z,) = Y._first_
↪ngens(3)
```

X and Y have an overlapping generators x_*, y_*. Since the default lexicographic order is used in both rings, it gives rise to isomorphic sub-monoids in both X and Y. They are merged in the pushout, which also yields a common parent for doing arithmetic:

```
sage: P = sage.categories.pushout.pushout(Y,X); P
Infinite polynomial ring in w, x, y, z over Rational Field
sage: w[2]+z[3]
w_2 + z_3
sage: _.parent() is P
True
```

```
>>> from sage.all import *
>>> P = sage.categories.pushout.pushout(Y,X); P
```

(continues on next page)

(continued from previous page)

```
Infinite polynomial ring in w, x, y, z over Rational Field
>>> w[Integer(2)]+z[Integer(3)]
w_2 + z_3
>>> _.parent() is P
True
```

expand()

Decompose the functor F into sub-functors, whose product returns F.

EXAMPLES:

```
sage: A = InfinitePolynomialRing(QQ, ['x','y'], order='degrevlex')
sage: F = A.construction()[0]; F
InfPoly{[x,y], "degrevlex", "dense"}
sage: F.expand()
[InfPoly{[y], "degrevlex", "dense"}, InfPoly{[x], "degrevlex", "dense"}]
sage: A = InfinitePolynomialRing(QQ, ['x','y','z'], order='degrevlex')
sage: F = A.construction()[0]; F
InfPoly{[x,y,z], "degrevlex", "dense"}
sage: F.expand()
[InfPoly{[z], "degrevlex", "dense"},
 InfPoly{[y], "degrevlex", "dense"},
 InfPoly{[x], "degrevlex", "dense"}]
sage: prod(F.expand())==F
True
```

```
>>> from sage.all import *
>>> A = InfinitePolynomialRing(QQ, ['x','y'], order='degrevlex')
>>> F = A.construction()[Integer(0)]; F
InfPoly{[x,y], "degrevlex", "dense"}
>>> F.expand()
[InfPoly{[y], "degrevlex", "dense"}, InfPoly{[x], "degrevlex", "dense"}]
>>> A = InfinitePolynomialRing(QQ, ['x','y','z'], order='degrevlex')
>>> F = A.construction()[Integer(0)]; F
InfPoly{[x,y,z], "degrevlex", "dense"}
>>> F.expand()
[InfPoly{[z], "degrevlex", "dense"},
 InfPoly{[y], "degrevlex", "dense"},
 InfPoly{[x], "degrevlex", "dense"}]
>>> prod(F.expand())==F
True
```

merge(*other*)

Merge two construction functors of infinite polynomial rings, regardless of monomial order and implementation.

The purpose is to have a pushout (and thus, arithmetic) even in cases when the parents are isomorphic as rings, but not as ordered rings.

EXAMPLES:

```
sage: X.<x,y> = InfinitePolynomialRing(QQ, implementation='sparse')
sage: Y.<x,y> = InfinitePolynomialRing(QQ, order='degrevlex')
```

(continues on next page)

(continued from previous page)

```
sage: X.construction()
[InfPoly{[x,y], "lex", "sparse"}, Rational Field]
sage: Y.construction()
[InfPoly{[x,y], "degrevlex", "dense"}, Rational Field]
sage: Y.construction()[0].merge(Y.construction()[0])
InfPoly{[x,y], "degrevlex", "dense"}
sage: y[3] + X(x[2])
x_2 + y_3
sage: _.parent().construction()
[InfPoly{[x,y], "degrevlex", "dense"}, Rational Field]
```

```
>>> from sage.all import *
>>> X = InfinitePolynomialRing(QQ, implementation='sparse', names=('x', 'y',
↪)); (x, y,) = X._first_ngens(2)
>>> Y = InfinitePolynomialRing(QQ, order='degrevlex', names=('x', 'y',)); (x,␣
↪y,) = Y._first_ngens(2)
>>> X.construction()
[InfPoly{[x,y], "lex", "sparse"}, Rational Field]
>>> Y.construction()
[InfPoly{[x,y], "degrevlex", "dense"}, Rational Field]
>>> Y.construction()[Integer(0)].merge(Y.construction()[Integer(0)])
InfPoly{[x,y], "degrevlex", "dense"}
>>> y[Integer(3)] + X(x[Integer(2)])
x_2 + y_3
>>> _.parent().construction()
[InfPoly{[x,y], "degrevlex", "dense"}, Rational Field]
```

rank = 9.5

class sage.categories.pushout.**LaurentPolynomialFunctor**(*var*, *multi_variate=False*)

Bases: *ConstructionFunctor*

Construction functor for Laurent polynomial rings.

EXAMPLES:

```
sage: L.<t> = LaurentPolynomialRing(ZZ)
sage: F = L.construction()[0]
sage: F
LaurentPolynomialFunctor
sage: F(QQ)
Univariate Laurent Polynomial Ring in t over Rational Field
sage: K.<x> = LaurentPolynomialRing(ZZ)
sage: F(K)
Univariate Laurent Polynomial Ring in t
 over Univariate Laurent Polynomial Ring in x over Integer Ring
sage: P.<x,y> = ZZ[]
sage: f = P.hom([x + 2*y, 3*x - y],P)
sage: F(f)
Ring endomorphism of Univariate Laurent Polynomial Ring in t
 over Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer␣
```

(continues on next page)

(continued from previous page)

```
↪Ring
          Defn: x |--> x + 2*y
                y |--> 3*x - y
sage: F(f)(x*F(P).gen()^-2 + y*F(P).gen()^3)
(x + 2*y)*t^-2 + (3*x - y)*t^3
```

```
>>> from sage.all import *
>>> L = LaurentPolynomialRing(ZZ, names=('t',)); (t,) = L._first_ngens(1)
>>> F = L.construction()[Integer(0)]
>>> F
LaurentPolynomialFunctor
>>> F(QQ)
Univariate Laurent Polynomial Ring in t over Rational Field
>>> K = LaurentPolynomialRing(ZZ, names=('x',)); (x,) = K._first_ngens(1)
>>> F(K)
Univariate Laurent Polynomial Ring in t
 over Univariate Laurent Polynomial Ring in x over Integer Ring
>>> P = ZZ['x, y']; (x, y,) = P._first_ngens(2)
>>> f = P.hom([x + Integer(2)*y, Integer(3)*x - y],P)
>>> F(f)
Ring endomorphism of Univariate Laurent Polynomial Ring in t
 over Multivariate Polynomial Ring in x, y over Integer Ring
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in x, y over Integer␣
↪Ring
          Defn: x |--> x + 2*y
                y |--> 3*x - y
>>> F(f)(x*F(P).gen()**-Integer(2) + y*F(P).gen()**Integer(3))
(x + 2*y)*t^-2 + (3*x - y)*t^3
```

merge(*other*)

Two Laurent polynomial construction functors merge if the variable names coincide.

The result is multivariate if one of the arguments is multivariate.

EXAMPLES:

```
sage: from sage.categories.pushout import LaurentPolynomialFunctor
sage: F1 = LaurentPolynomialFunctor('t')
sage: F2 = LaurentPolynomialFunctor('t', multi_variate=True)
sage: F1.merge(F2)
LaurentPolynomialFunctor
sage: F1.merge(F2)(LaurentPolynomialRing(GF(2), 'a'))                          #␣
↪needs sage.modules
Multivariate Laurent Polynomial Ring in a, t over Finite Field of size 2
sage: F1.merge(F1)(LaurentPolynomialRing(GF(2), 'a'))
Univariate Laurent Polynomial Ring in t over
 Univariate Laurent Polynomial Ring in a over Finite Field of size 2
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import LaurentPolynomialFunctor
>>> F1 = LaurentPolynomialFunctor('t')
```

(continues on next page)

(continued from previous page)

```
>>> F2 = LaurentPolynomialFunctor('t', multi_variate=True)
>>> F1.merge(F2)
LaurentPolynomialFunctor
>>> F1.merge(F2)(LaurentPolynomialRing(GF(Integer(2)), 'a'))
↪     # needs sage.modules
Multivariate Laurent Polynomial Ring in a, t over Finite Field of size 2
>>> F1.merge(F1)(LaurentPolynomialRing(GF(Integer(2)), 'a'))
Univariate Laurent Polynomial Ring in t over
 Univariate Laurent Polynomial Ring in a over Finite Field of size 2
```

rank = 9

class sage.categories.pushout.**MatrixFunctor**(*nrows*, *ncols*, *is_sparse=False*)

Bases: *ConstructionFunctor*

A construction functor for matrices over rings.

EXAMPLES:

```
sage: # needs sage.modules
sage: MS = MatrixSpace(ZZ, 2, 3)
sage: F = MS.construction()[0]; F
MatrixFunctor
sage: MS = MatrixSpace(ZZ, 2)
sage: F = MS.construction()[0]; F
MatrixFunctor
sage: P.<x,y> = QQ[]
sage: R = F(P); R
Full MatrixSpace of 2 by 2 dense matrices
 over Multivariate Polynomial Ring in x, y over Rational Field
sage: f = P.hom([x+y, x-y], P); F(f)
Ring endomorphism
 of Full MatrixSpace of 2 by 2 dense matrices
  over Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Induced from base ring by
        Ring endomorphism
         of Multivariate Polynomial Ring in x, y over Rational Field
          Defn: x |--> x + y
                y |--> x - y
sage: M = R([x, y, x*y, x + y])
sage: F(f)(M)
[    x + y     x - y]
[x^2 - y^2       2*x]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> MS = MatrixSpace(ZZ, Integer(2), Integer(3))
>>> F = MS.construction()[Integer(0)]; F
MatrixFunctor
>>> MS = MatrixSpace(ZZ, Integer(2))
>>> F = MS.construction()[Integer(0)]; F
MatrixFunctor
>>> P = QQ['x, y']; (x, y,) = P._first_ngens(2)
```

(continues on next page)

(continued from previous page)

```
>>> R = F(P); R
Full MatrixSpace of 2 by 2 dense matrices
 over Multivariate Polynomial Ring in x, y over Rational Field
>>> f = P.hom([x+y, x-y], P); F(f)
Ring endomorphism
 of Full MatrixSpace of 2 by 2 dense matrices
  over Multivariate Polynomial Ring in x, y over Rational Field
  Defn: Induced from base ring by
        Ring endomorphism
         of Multivariate Polynomial Ring in x, y over Rational Field
          Defn: x |--> x + y
                y |--> x - y
>>> M = R([x, y, x*y, x + y])
>>> F(f)(M)
[    x + y     x - y]
[x^2 - y^2       2*x]
```

merge(*other*)

Merging is only happening if both functors are matrix functors of the same dimension.

The result is sparse if and only if both given functors are sparse.

EXAMPLES:

```
sage: # needs sage.modules
sage: F1 = MatrixSpace(ZZ, 2, 2).construction()[0]
sage: F2 = MatrixSpace(ZZ, 2, 3).construction()[0]
sage: F3 = MatrixSpace(ZZ, 2, 2, sparse=True).construction()[0]
sage: F1.merge(F2)
sage: F1.merge(F3)
MatrixFunctor
sage: F13 = F1.merge(F3)
sage: F13.is_sparse
False
sage: F1.is_sparse
False
sage: F3.is_sparse
True
sage: F3.merge(F3).is_sparse
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F1 = MatrixSpace(ZZ, Integer(2), Integer(2)).construction()[Integer(0)]
>>> F2 = MatrixSpace(ZZ, Integer(2), Integer(3)).construction()[Integer(0)]
>>> F3 = MatrixSpace(ZZ, Integer(2), Integer(2), sparse=True).
↪construction()[Integer(0)]
>>> F1.merge(F2)
>>> F1.merge(F3)
MatrixFunctor
>>> F13 = F1.merge(F3)
>>> F13.is_sparse
False
```

(continues on next page)

(continued from previous page)

```
>>> F1.is_sparse
False
>>> F3.is_sparse
True
>>> F3.merge(F3).is_sparse
True
```

rank = 10

class sage.categories.pushout.**MultiPolynomialFunctor**(*vars*, *term_order*)

Bases: *ConstructionFunctor*

A constructor for multivariate polynomial rings.

EXAMPLES:

```
sage: P.<x,y> = ZZ[]
sage: F = P.construction()[0]; F
MPoly[x,y]
sage: A.<a,b> = GF(5)[]
sage: F(A)
Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
sage: f = A.hom([a+b, a-b], A)
sage: F(f)
Ring endomorphism of Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in a, b over Finite␣
↪Field of size 5
          Defn: a |--> a + b
                b |--> a - b
sage: F(f)(F(A)(x)*a)
(a + b)*x
```

```
>>> from sage.all import *
>>> P = ZZ['x, y']; (x, y,) = P._first_ngens(2)
>>> F = P.construction()[Integer(0)]; F
MPoly[x,y]
>>> A = GF(Integer(5))['a, b']; (a, b,) = A._first_ngens(2)
>>> F(A)
Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
>>> f = A.hom([a+b, a-b], A)
>>> F(f)
Ring endomorphism of Multivariate Polynomial Ring in x, y
 over Multivariate Polynomial Ring in a, b over Finite Field of size 5
  Defn: Induced from base ring by
        Ring endomorphism of Multivariate Polynomial Ring in a, b over Finite␣
↪Field of size 5
          Defn: a |--> a + b
                b |--> a - b
>>> F(f)(F(A)(x)*a)
```

(continues on next page)

(continued from previous page)

```
(a + b)*x
```

expand()

Decompose `self` into a list of construction functors.

EXAMPLES:

```
sage: F = QQ['x,y,z,t'].construction()[0]; F
MPoly[x,y,z,t]
sage: F.expand()
[MPoly[t], MPoly[z], MPoly[y], MPoly[x]]
```

```
>>> from sage.all import *
>>> F = QQ['x,y,z,t'].construction()[Integer(0)]; F
MPoly[x,y,z,t]
>>> F.expand()
[MPoly[t], MPoly[z], MPoly[y], MPoly[x]]
```

Now an actual use case:

```
sage: R.<x,y,z> = ZZ[]
sage: S.<z,t> = QQ[]
sage: x+t
x + t
sage: parent(x+t)
Multivariate Polynomial Ring in x, y, z, t over Rational Field
sage: T.<y,s> = QQ[]
sage: x + s
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Multivariate Polynomial Ring in x, y, z over Integer Ring' and
'Multivariate Polynomial Ring in y, s over Rational Field'
sage: R = PolynomialRing(ZZ, 'x', 50)
sage: S = PolynomialRing(GF(5), 'x', 20)
sage: R.gen(0) + S.gen(0)
2*x0
```

```
>>> from sage.all import *
>>> R = ZZ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> S = QQ['z, t']; (z, t,) = S._first_ngens(2)
>>> x+t
x + t
>>> parent(x+t)
Multivariate Polynomial Ring in x, y, z, t over Rational Field
>>> T = QQ['y, s']; (y, s,) = T._first_ngens(2)
>>> x + s
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Multivariate Polynomial Ring in x, y, z over Integer Ring' and
'Multivariate Polynomial Ring in y, s over Rational Field'
```

(continues on next page)

(continued from previous page)

```
>>> R = PolynomialRing(ZZ, 'x', Integer(50))
>>> S = PolynomialRing(GF(Integer(5)), 'x', Integer(20))
>>> R.gen(Integer(0)) + S.gen(Integer(0))
2*x0
```

merge(*other*)

Merge `self` with another construction functor, or return `None`.

EXAMPLES:

```
sage: F = sage.categories.pushout.MultiPolynomialFunctor(['x','y'], None)
sage: G = sage.categories.pushout.MultiPolynomialFunctor(['t'], None)
sage: F.merge(G) is None
True
sage: F.merge(F)
MPoly[x,y]
```

```
>>> from sage.all import *
>>> F = sage.categories.pushout.MultiPolynomialFunctor(['x','y'], None)
>>> G = sage.categories.pushout.MultiPolynomialFunctor(['t'], None)
>>> F.merge(G) is None
True
>>> F.merge(F)
MPoly[x,y]
```

rank = 9

class sage.categories.pushout.**MultivariateConstructionFunctor**

Bases: *ConstructionFunctor*

An abstract base class for functors that take multiple inputs (e.g. Cartesian products).

common_base(*other_functor*, *self_bases*, *other_bases*)

This function is called by *pushout()* when no common parent is found in the construction tower.

INPUT:

- `other_functor` – a construction functor
- `self_bases` – the arguments passed to this functor
- `other_bases` – the arguments passed to the functor `other_functor`

OUTPUT: a parent

If no common base is found a `sage.structure.coerce_exceptions.CoercionException` is raised.

Note

Overload this function in derived class, see e.g. *MultivariateConstructionFunctor*.

class sage.categories.pushout.**PermutationGroupFunctor**(*gens*, *domain*)

Bases: *ConstructionFunctor*

EXAMPLES:

```
sage: from sage.categories.pushout import PermutationGroupFunctor
sage: PF = PermutationGroupFunctor([PermutationGroupElement([(1,2)])],      #␣
↪needs sage.groups
....:                                [1,2]); PF
PermutationGroupFunctor[(1,2)]
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import PermutationGroupFunctor
>>> PF = PermutationGroupFunctor([PermutationGroupElement([(Integer(1),
↪Integer(2))])],      # needs sage.groups
...                                [Integer(1),Integer(2)]); PF
PermutationGroupFunctor[(1,2)]
```

gens()

EXAMPLES:

```
sage: P1 = PermutationGroup([[(1,2)]])                                      #␣
↪needs sage.groups
sage: PF, P = P1.construction()                                             #␣
↪needs sage.groups
sage: PF.gens()                                                             #␣
↪needs sage.groups
((1,2),)
```

```
>>> from sage.all import *
>>> P1 = PermutationGroup([[(Integer(1),Integer(2))]])                       ␣
↪              # needs sage.groups
>>> PF, P = P1.construction()                                              #␣
↪needs sage.groups
>>> PF.gens()                                                              #␣
↪needs sage.groups
((1,2),)
```

merge(*other*)

Merge `self` with another construction functor, or return `None`.

EXAMPLES:

```
sage: # needs sage.groups
sage: P1 = PermutationGroup([[(1,2)]])
sage: PF1, P = P1.construction()
sage: P2 = PermutationGroup([[(1,3)]])
sage: PF2, P = P2.construction()
sage: PF1.merge(PF2)
PermutationGroupFunctor[(1,2), (1,3)]
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> P1 = PermutationGroup([[(Integer(1),Integer(2))]])
>>> PF1, P = P1.construction()
>>> P2 = PermutationGroup([[(Integer(1),Integer(3))]])
>>> PF2, P = P2.construction()
```

(continues on next page)

(continued from previous page)

```
>>> PF1.merge(PF2)
PermutationGroupFunctor[(1,2), (1,3)]
```

rank = 10

class sage.categories.pushout.**PolynomialFunctor**(*var*, *multi_variate=False*, *sparse=False*, *implementation=None*)

Bases: *ConstructionFunctor*

Construction functor for univariate polynomial rings.

EXAMPLES:

```
sage: P = ZZ['t'].construction()[0]
sage: P(GF(3))
Univariate Polynomial Ring in t over Finite Field of size 3
sage: P == loads(dumps(P))
True
sage: R.<x,y> = GF(5)[]
sage: f = R.hom([x + 2*y, 3*x - y], R)
sage: P(f)((x+y) * P(R).0)
(-x + y)*t
```

```
>>> from sage.all import *
>>> P = ZZ['t'].construction()[Integer(0)]
>>> P(GF(Integer(3)))
Univariate Polynomial Ring in t over Finite Field of size 3
>>> P == loads(dumps(P))
True
>>> R = GF(Integer(5))['x, y']; (x, y,) = R._first_ngens(2)
>>> f = R.hom([x + Integer(2)*y, Integer(3)*x - y], R)
>>> P(f)((x+y) * P(R).gen(0))
(-x + y)*t
```

By Issue #9944, the construction functor distinguishes sparse and dense polynomial rings. Before, the following example failed:

```
sage: R.<x> = PolynomialRing(GF(5), sparse=True)
sage: F, B = R.construction()
sage: F(B) is R
True
sage: S.<x> = PolynomialRing(ZZ)
sage: R.has_coerce_map_from(S)
False
sage: S.has_coerce_map_from(R)
False
sage: S.0 + R.0
2*x
sage: (S.0 + R.0).parent()
Univariate Polynomial Ring in x over Finite Field of size 5
sage: (S.0 + R.0).parent().is_sparse()
False
```

```
>>> from sage.all import *
>>> R = PolynomialRing(GF(Integer(5)), sparse=True, names=('x',)); (x,) = R._
↪first_ngens(1)
>>> F, B = R.construction()
>>> F(B) is R
True
>>> S = PolynomialRing(ZZ, names=('x',)); (x,) = S._first_ngens(1)
>>> R.has_coerce_map_from(S)
False
>>> S.has_coerce_map_from(R)
False
>>> S.gen(0) + R.gen(0)
2*x
>>> (S.gen(0) + R.gen(0)).parent()
Univariate Polynomial Ring in x over Finite Field of size 5
>>> (S.gen(0) + R.gen(0)).parent().is_sparse()
False
```

merge(*other*)

Merge `self` with another construction functor, or return `None`.

> **Note**
>
> Internally, the merging is delegated to the merging of multipolynomial construction functors. But in effect, this does the same as the default implementation, that returns `None` unless the to-be-merged functors coincide.

EXAMPLES:

```
sage: P = ZZ['x'].construction()[0]
sage: Q = ZZ['y','x'].construction()[0]
sage: P.merge(Q)
sage: P.merge(P) is P
True
```

```
>>> from sage.all import *
>>> P = ZZ['x'].construction()[Integer(0)]
>>> Q = ZZ['y','x'].construction()[Integer(0)]
>>> P.merge(Q)
>>> P.merge(P) is P
True
```

rank = 9

class sage.categories.pushout.**QuotientFunctor**(*I*, *names=None*, *as_field=False*, *domain=None*, *codomain=None*, ***kwds*)

Bases: *ConstructionFunctor*

Construction functor for quotient rings.

> **Note**
>
> The functor keeps track of variable names. Optionally, it may keep track of additional properties of the quotient, such as its category or its implementation.

EXAMPLES:

```
sage: P.<x,y> = ZZ[]
sage: Q = P.quo([x^2 + y^2] * P)
sage: F = Q.construction()[0]
sage: F(QQ['x','y'])
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2 + y^2)
sage: F(QQ['x','y']) == QQ['x','y'].quo([x^2 + y^2] * QQ['x','y'])
True
sage: F(QQ['x','y','z'])
Traceback (most recent call last):
...
CoercionException: Cannot apply this quotient functor to
 Multivariate Polynomial Ring in x, y, z over Rational Field
sage: F(QQ['y','z'])                                                    #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
TypeError: Could not find a mapping of the passed element to this ring.
```

```
>>> from sage.all import *
>>> P = ZZ['x, y']; (x, y,) = P._first_ngens(2)
>>> Q = P.quo([x**Integer(2) + y**Integer(2)] * P)
>>> F = Q.construction()[Integer(0)]
>>> F(QQ['x','y'])
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2 + y^2)
>>> F(QQ['x','y']) == QQ['x','y'].quo([x**Integer(2) + y**Integer(2)] * QQ['x','y
↪'])
True
>>> F(QQ['x','y','z'])
Traceback (most recent call last):
...
CoercionException: Cannot apply this quotient functor to
 Multivariate Polynomial Ring in x, y, z over Rational Field
>>> F(QQ['y','z'])                                                      #␣
↪needs sage.rings.finite_rings
Traceback (most recent call last):
...
TypeError: Could not find a mapping of the passed element to this ring.
```

merge(*other*)

Two quotient functors with coinciding names are merged by taking the gcd of their moduli, the meet of their domains, and the join of their codomains.

In particular, if one of the functors being merged knows that the quotient is going to be a field, then the merged functor will return fields as well.

EXAMPLES:

```
sage: # needs sage.libs.pari
sage: P.<x> = QQ[]
sage: Q1 = P.quo([(x^2+1)^2*(x^2-3)])
sage: Q2 = P.quo([(x^2+1)^2*(x^5+3)])
sage: from sage.categories.pushout import pushout
sage: pushout(Q1,Q2)     # indirect doctest
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^4 + 2*x^2 + 1
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> P = QQ['x']; (x,) = P._first_ngens(1)
>>> Q1 = P.quo([(x**Integer(2)+Integer(1))**Integer(2)*(x**Integer(2)-
↪Integer(3))])
>>> Q2 = P.
↪quo([(x**Integer(2)+Integer(1))**Integer(2)*(x**Integer(5)+Integer(3))])
>>> from sage.categories.pushout import pushout
>>> pushout(Q1,Q2)     # indirect doctest
Univariate Quotient Polynomial Ring in xbar over Rational Field
 with modulus x^4 + 2*x^2 + 1
```

The following was fixed in Issue #8800:

```
sage: pushout(GF(5), Integers(5))                                              #␣
↪needs sage.libs.pari
Finite Field of size 5
```

```
>>> from sage.all import *
>>> pushout(GF(Integer(5)), Integers(Integer(5)))                               ␣
↪              # needs sage.libs.pari
Finite Field of size 5
```

rank = 4.5

class sage.categories.pushout.**SubspaceFunctor**(*basis*)

Bases: *ConstructionFunctor*

Constructing a subspace of an ambient free module, given by a basis.

Note

This construction functor keeps track of the basis. It can only be applied to free modules into which this basis coerces.

EXAMPLES:

```
sage: # needs sage.modules
sage: M = ZZ^3
sage: S = M.submodule([(1,2,3), (4,5,6)]); S
Free module of degree 3 and rank 2 over Integer Ring
Echelon basis matrix:
```

(continues on next page)

(continued from previous page)

```
[1 2 3]
[0 3 6]
sage: F = S.construction()[0]
sage: F(GF(2)^3)
Vector space of degree 3 and dimension 2 over Finite Field of size 2
User basis matrix:
[1 0 1]
[0 1 0]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> M = ZZ**Integer(3)
>>> S = M.submodule([(Integer(1),Integer(2),Integer(3)), (Integer(4),Integer(5),
↪Integer(6))]); S
Free module of degree 3 and rank 2 over Integer Ring
Echelon basis matrix:
[1 2 3]
[0 3 6]
>>> F = S.construction()[Integer(0)]
>>> F(GF(Integer(2))**Integer(3))
Vector space of degree 3 and dimension 2 over Finite Field of size 2
User basis matrix:
[1 0 1]
[0 1 0]
```

coercion_reversed = True

merge(*other*)

Two Subspace Functors are merged into a construction functor of the sum of two subspaces.

EXAMPLES:

```
sage: # needs sage.modules
sage: M = GF(5)^3
sage: S1 = M.submodule([(1,2,3),(4,5,6)])
sage: S2 = M.submodule([(2,2,3)])
sage: F1 = S1.construction()[0]
sage: F2 = S2.construction()[0]
sage: F1.merge(F2)
SubspaceFunctor
sage: F1.merge(F2)(GF(5)^3) == S1 + S2
True
sage: F1.merge(F2)(GF(5)['t']^3)
Free module of degree 3 and rank 3
 over Univariate Polynomial Ring in t over Finite Field of size 5
User basis matrix:
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> M = GF(Integer(5))**Integer(3)
>>> S1 = M.submodule([(Integer(1),Integer(2),Integer(3)),(Integer(4),
↪Integer(5),Integer(6))])
>>> S2 = M.submodule([(Integer(2),Integer(2),Integer(3))])
>>> F1 = S1.construction()[Integer(0)]
>>> F2 = S2.construction()[Integer(0)]
>>> F1.merge(F2)
SubspaceFunctor
>>> F1.merge(F2)(GF(Integer(5))**Integer(3)) == S1 + S2
True
>>> F1.merge(F2)(GF(Integer(5))['t']**Integer(3))
Free module of degree 3 and rank 3
 over Univariate Polynomial Ring in t over Finite Field of size 5
User basis matrix:
[1 0 0]
[0 1 0]
[0 0 1]
```

rank = 11

class sage.categories.pushout.**VectorFunctor**(*n=None*, *is_sparse=False*, *inner_product_matrix=None*, ***, *with_basis='standard'*, *basis_keys=None*, *name_mapping=None*, *latex_name_mapping=None*)

Bases: *ConstructionFunctor*

A construction functor for free modules over commutative rings.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = (ZZ^3).construction()[0]
sage: F
VectorFunctor
sage: F(GF(2)['t'])                                                    #␣
↪needs sage.libs.ntl
Ambient free module of rank 3
 over the principal ideal domain Univariate Polynomial Ring in t
  over Finite Field of size 2 (using GF2X)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = (ZZ**Integer(3)).construction()[Integer(0)]
>>> F
VectorFunctor
>>> F(GF(Integer(2))['t'])                                             ␣
↪     # needs sage.libs.ntl
Ambient free module of rank 3
 over the principal ideal domain Univariate Polynomial Ring in t
  over Finite Field of size 2 (using GF2X)
```

merge(*other*)

Two constructors of free modules merge, if the module ranks and the inner products coincide. If both have explicitly given inner product matrices, they must coincide as well.

EXAMPLES:

Two modules without explicitly given inner product allow coercion:

```
sage: M1 = QQ^3                                                                 #␣
↪needs sage.modules
sage: P.<t> = ZZ[]
sage: M2 = FreeModule(P, 3)                                                     #␣
↪needs sage.modules
sage: M1([1,1/2,1/3]) + M2([t,t^2+t,3])      # indirect doctest                 #␣
↪needs sage.modules
(t + 1, t^2 + t + 1/2, 10/3)
```

```
>>> from sage.all import *
>>> M1 = QQ**Integer(3)                                                         ␣
↪      # needs sage.modules
>>> P = ZZ['t']; (t,) = P._first_ngens(1)
>>> M2 = FreeModule(P, Integer(3))                                              ␣
↪     # needs sage.modules
>>> M1([Integer(1),Integer(1)/Integer(2),Integer(1)/Integer(3)]) + M2([t,
↪t**Integer(2)+t,Integer(3)])      # indirect doctest                 # needs␣
↪sage.modules
(t + 1, t^2 + t + 1/2, 10/3)
```

If only one summand has an explicit inner product, the result will be provided with it:

```
sage: M3 = FreeModule(P, 3, inner_product_matrix=Matrix(3, 3, range(9)))     #␣
↪needs sage.modules
sage: M1([1,1/2,1/3]) + M3([t,t^2+t,3])                                         #␣
↪needs sage.modules
(t + 1, t^2 + t + 1/2, 10/3)
sage: (M1([1,1/2,1/3]) + M3([t,t^2+t,3])).parent().inner_product_matrix()    #␣
↪needs sage.modules
[0 1 2]
[3 4 5]
[6 7 8]
```

```
>>> from sage.all import *
>>> M3 = FreeModule(P, Integer(3), inner_product_matrix=Matrix(Integer(3),␣
↪Integer(3), range(Integer(9))))     # needs sage.modules
>>> M1([Integer(1),Integer(1)/Integer(2),Integer(1)/Integer(3)]) + M3([t,
↪t**Integer(2)+t,Integer(3)])                                          # needs␣
↪sage.modules
(t + 1, t^2 + t + 1/2, 10/3)
>>> (M1([Integer(1),Integer(1)/Integer(2),Integer(1)/Integer(3)]) + M3([t,
↪t**Integer(2)+t,Integer(3)])).parent().inner_product_matrix()    # needs␣
↪sage.modules
[0 1 2]
[3 4 5]
[6 7 8]
```

If both summands have an explicit inner product (even if it is the standard inner product), then the products must coincide. The only difference between `M1` and `M4` in the following example is the fact that the default inner product was *explicitly* requested for `M4`. It is therefore not possible to coerce with a different inner

product:

```
sage: # needs sage.modules
sage: M4 = FreeModule(QQ, 3, inner_product_matrix=Matrix(3, 3, 1))
sage: M4 == M1
True
sage: M4.inner_product_matrix() == M1.inner_product_matrix()
True
sage: M4([1,1/2,1/3]) + M3([t,t^2+t,3])      # indirect doctest
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ambient quadratic space of dimension 3 over Rational Field
Inner product matrix:
[1 0 0]
[0 1 0]
[0 0 1]' and
'Ambient free quadratic module of rank 3 over the integral domain
Univariate Polynomial Ring in t over Integer Ring
Inner product matrix:
[0 1 2]
[3 4 5]
[6 7 8]'
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> M4 = FreeModule(QQ, Integer(3), inner_product_matrix=Matrix(Integer(3),␣
↪Integer(3), Integer(1)))
>>> M4 == M1
True
>>> M4.inner_product_matrix() == M1.inner_product_matrix()
True
>>> M4([Integer(1),Integer(1)/Integer(2),Integer(1)/Integer(3)]) + M3([t,
↪t**Integer(2)+t,Integer(3)])      # indirect doctest
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for +:
'Ambient quadratic space of dimension 3 over Rational Field
Inner product matrix:
[1 0 0]
[0 1 0]
[0 0 1]' and
'Ambient free quadratic module of rank 3 over the integral domain
Univariate Polynomial Ring in t over Integer Ring
Inner product matrix:
[0 1 2]
[3 4 5]
[6 7 8]'
```

Names are removed when they conflict:

```
sage: # needs sage.modules
sage: from sage.categories.pushout import VectorFunctor, pushout
```

(continues on next page)

(continued from previous page)

```
sage: M_ZZx = FreeModule(ZZ['x'], 4, with_basis=None, name='M_ZZx')
sage: N_ZZx = FreeModule(ZZ['x'], 4, with_basis=None, name='N_ZZx')
sage: pushout(M_ZZx, QQ)
Rank-4 free module M_ZZx_base_ext
 over the Univariate Polynomial Ring in x over Rational Field
sage: pushout(M_ZZx, N_ZZx)
Rank-4 free module
 over the Univariate Polynomial Ring in x over Integer Ring
sage: pushout(pushout(M_ZZx, N_ZZx), QQ)
Rank-4 free module
 over the Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> from sage.categories.pushout import VectorFunctor, pushout
>>> M_ZZx = FreeModule(ZZ['x'], Integer(4), with_basis=None, name='M_ZZx')
>>> N_ZZx = FreeModule(ZZ['x'], Integer(4), with_basis=None, name='N_ZZx')
>>> pushout(M_ZZx, QQ)
Rank-4 free module M_ZZx_base_ext
 over the Univariate Polynomial Ring in x over Rational Field
>>> pushout(M_ZZx, N_ZZx)
Rank-4 free module
 over the Univariate Polynomial Ring in x over Integer Ring
>>> pushout(pushout(M_ZZx, N_ZZx), QQ)
Rank-4 free module
 over the Univariate Polynomial Ring in x over Rational Field
```

rank = 10

sage.categories.pushout.**construction_tower**(*R*)

An auxiliary function that is used in *pushout()* and *pushout_lattice()*.

INPUT:

- R – an object

OUTPUT:

A constructive description of the object from scratch, by a list of pairs of a construction functor and an object to which the construction functor is to be applied. The first pair is formed by None and the given object.

EXAMPLES:

```
sage: from sage.categories.pushout import construction_tower
sage: construction_tower(MatrixSpace(FractionField(QQ['t']), 2))                #␣
↪needs sage.modules
[(None, Full MatrixSpace of 2 by 2 dense matrices over Fraction Field
         of Univariate Polynomial Ring in t over Rational Field),
 (MatrixFunctor, Fraction Field
                  of Univariate Polynomial Ring in t over Rational Field),
 (FractionField, Univariate Polynomial Ring in t over Rational Field),
 (Poly[t], Rational Field), (FractionField, Integer Ring)]
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import construction_tower
>>> construction_tower(MatrixSpace(FractionField(QQ['t']), Integer(2)))
↪     # needs sage.modules
[(None, Full MatrixSpace of 2 by 2 dense matrices over Fraction Field
        of Univariate Polynomial Ring in t over Rational Field),
 (MatrixFunctor, Fraction Field
                 of Univariate Polynomial Ring in t over Rational Field),
 (FractionField, Univariate Polynomial Ring in t over Rational Field),
 (Poly[t], Rational Field), (FractionField, Integer Ring)]
```

sage.categories.pushout.**expand_tower**(*tower*)

An auxiliary function that is used in *pushout()*.

INPUT:

- tower – a construction tower as returned by *construction_tower()*

OUTPUT: a new construction tower with all the construction functors expanded

EXAMPLES:

```
sage: from sage.categories.pushout import construction_tower, expand_tower
sage: construction_tower(QQ['x,y,z'])
[(None, Multivariate Polynomial Ring in x, y, z over Rational Field),
 (MPoly[x,y,z], Rational Field),
 (FractionField, Integer Ring)]
sage: expand_tower(construction_tower(QQ['x,y,z']))
[(None, Multivariate Polynomial Ring in x, y, z over Rational Field),
 (MPoly[z], Univariate Polynomial Ring in y
            over Univariate Polynomial Ring in x over Rational Field),
 (MPoly[y], Univariate Polynomial Ring in x over Rational Field),
 (MPoly[x], Rational Field),
 (FractionField, Integer Ring)]
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import construction_tower, expand_tower
>>> construction_tower(QQ['x,y,z'])
[(None, Multivariate Polynomial Ring in x, y, z over Rational Field),
 (MPoly[x,y,z], Rational Field),
 (FractionField, Integer Ring)]
>>> expand_tower(construction_tower(QQ['x,y,z']))
[(None, Multivariate Polynomial Ring in x, y, z over Rational Field),
 (MPoly[z], Univariate Polynomial Ring in y
            over Univariate Polynomial Ring in x over Rational Field),
 (MPoly[y], Univariate Polynomial Ring in x over Rational Field),
 (MPoly[x], Rational Field),
 (FractionField, Integer Ring)]
```

sage.categories.pushout.**pushout**(R, S)

Given a pair of objects R and S, try to construct a reasonable object Y and return maps such that canonically $R \leftarrow Y \rightarrow S$.

ALGORITHM:

This incorporates the idea of functors discussed at Sage Days 4. Every object R can be viewed as an initial object and a series of functors (e.g. polynomial, quotient, extension, completion, vector/matrix, etc.). Call the series of increasingly simple objects (with the associated functors) the "tower" of R. The construction method is used to create the tower.

Given two objects R and S, try to find a common initial object Z. If the towers of R and S meet, let Z be their join. Otherwise, see if the top of one coerces naturally into the other.

Now we have an initial object and two ordered lists of functors to apply. We wish to merge these in an unambiguous order, popping elements off the top of one or the other tower as we apply them to Z.

- If the functors are of distinct types, there is an absolute ordering given by the rank attribute. Use this.
- Otherwise:
  - If the tops are equal, we (try to) merge them.
  - If exactly one occurs lower in the other tower, we may unambiguously apply the other (hoping for a later merge).
  - If the tops commute, we can apply either first.
  - Otherwise fail due to ambiguity.

The algorithm assumes by default that when a construction F is applied to an object X, the object $F(X)$ admits a coercion map from X. However, the algorithm can also handle the case where $F(X)$ has a coercion map *to* X instead. In this case, the attribute `coercion_reversed` of the class implementing F should be set to `True`.

EXAMPLES:

Here our "towers" are $R = Complete_7(Frac(\mathbf{Z}))$ and $Frac(Poly_x(\mathbf{Z}))$, which give us $Frac(Poly_x(Complete_7(Frac(\mathbf{Z}))))$:

```
sage: from sage.categories.pushout import pushout
sage: pushout(Qp(7), Frac(ZZ['x']))                                             #␣
↪needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import pushout
>>> pushout(Qp(Integer(7)), Frac(ZZ['x']))                                       ␣
↪     # needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
```

Note we get the same thing with

```
sage: pushout(Zp(7), Frac(QQ['x']))                                             #␣
↪needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
sage: pushout(Zp(7)['x'], Frac(QQ['x']))                                        #␣
↪needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
```

```
>>> from sage.all import *
>>> pushout(Zp(Integer(7)), Frac(QQ['x']))                                    ␣
↪     # needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
>>> pushout(Zp(Integer(7))['x'], Frac(QQ['x']))                               ␣
↪     # needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
```

Note that polynomial variable ordering must be unambiguously determined.

```
sage: pushout(ZZ['x,y,z'], QQ['w,z,t'])
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension',
Multivariate Polynomial Ring in x, y, z over Integer Ring,
Multivariate Polynomial Ring in w, z, t over Rational Field)
sage: pushout(ZZ['x,y,z'], QQ['w,x,z,t'])
Multivariate Polynomial Ring in w, x, y, z, t over Rational Field
```

```
>>> from sage.all import *
>>> pushout(ZZ['x,y,z'], QQ['w,z,t'])
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension',
Multivariate Polynomial Ring in x, y, z over Integer Ring,
Multivariate Polynomial Ring in w, z, t over Rational Field)
>>> pushout(ZZ['x,y,z'], QQ['w,x,z,t'])
Multivariate Polynomial Ring in w, x, y, z, t over Rational Field
```

Some other examples:

```
sage: pushout(Zp(7)['y'], Frac(QQ['t'])['x,y,z'])                           #␣
↪needs sage.rings.padics
Multivariate Polynomial Ring in x, y, z
 over Fraction Field of Univariate Polynomial Ring in t
  over 7-adic Field with capped relative precision 20
sage: pushout(ZZ['x,y,z'], Frac(ZZ['x'])['y'])
Multivariate Polynomial Ring in y, z
 over Fraction Field of Univariate Polynomial Ring in x over Integer Ring
sage: pushout(MatrixSpace(RDF, 2, 2), Frac(ZZ['x']))                        #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices
 over Fraction Field of Univariate Polynomial Ring in x over Real Double Field
sage: pushout(ZZ, MatrixSpace(ZZ[['x']], 3, 3))                             #␣
↪needs sage.modules
Full MatrixSpace of 3 by 3 dense matrices
 over Power Series Ring in x over Integer Ring
sage: pushout(QQ['x,y'], ZZ[['x']])
Univariate Polynomial Ring in y
 over Power Series Ring in x over Rational Field
sage: pushout(Frac(ZZ['x']), QQ[['x']])
```

(continues on next page)

(continued from previous page)

```
Laurent Series Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> pushout(Zp(Integer(7))['y'], Frac(QQ['t'])['x,y,z'])
↪     # needs sage.rings.padics
Multivariate Polynomial Ring in x, y, z
 over Fraction Field of Univariate Polynomial Ring in t
  over 7-adic Field with capped relative precision 20
>>> pushout(ZZ['x,y,z'], Frac(ZZ['x'])['y'])
Multivariate Polynomial Ring in y, z
 over Fraction Field of Univariate Polynomial Ring in x over Integer Ring
>>> pushout(MatrixSpace(RDF, Integer(2), Integer(2)), Frac(ZZ['x']))
↪              # needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices
 over Fraction Field of Univariate Polynomial Ring in x over Real Double Field
>>> pushout(ZZ, MatrixSpace(ZZ[['x']], Integer(3), Integer(3)))
↪              # needs sage.modules
Full MatrixSpace of 3 by 3 dense matrices
 over Power Series Ring in x over Integer Ring
>>> pushout(QQ['x,y'], ZZ[['x']])
Univariate Polynomial Ring in y
 over Power Series Ring in x over Rational Field
>>> pushout(Frac(ZZ['x']), QQ[['x']])
Laurent Series Ring in x over Rational Field
```

A construction with `coercion_reversed=True` (currently only the *SubspaceFunctor* construction) is only applied if it leads to a valid coercion:

```
sage: # needs sage.modules
sage: A = ZZ^2
sage: V = span([[1, 2]], QQ)
sage: P = sage.categories.pushout.pushout(A, V)
sage: P
Vector space of dimension 2 over Rational Field
sage: P.has_coerce_map_from(A)
True

sage: # needs sage.modules
sage: V = (QQ^3).span([[1, 2, 3/4]])
sage: A = ZZ^3
sage: pushout(A, V)
Vector space of dimension 3 over Rational Field
sage: B = A.span([[0, 0, 2/3]])
sage: pushout(B, V)
Vector space of degree 3 and dimension 2 over Rational Field
User basis matrix:
[1 2 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = ZZ**Integer(2)
```

(continues on next page)

(continued from previous page)

```
>>> V = span([[Integer(1), Integer(2)]], QQ)
>>> P = sage.categories.pushout.pushout(A, V)
>>> P
Vector space of dimension 2 over Rational Field
>>> P.has_coerce_map_from(A)
True

>>> # needs sage.modules
>>> V = (QQ**Integer(3)).span([[Integer(1), Integer(2), Integer(3)/Integer(4)]])
>>> A = ZZ**Integer(3)
>>> pushout(A, V)
Vector space of dimension 3 over Rational Field
>>> B = A.span([[Integer(0), Integer(0), Integer(2)/Integer(3)]])
>>> pushout(B, V)
Vector space of degree 3 and dimension 2 over Rational Field
User basis matrix:
[1 2 0]
[0 0 1]
```

Some more tests with `coercion_reversed=True`:

```
sage: from sage.categories.pushout import ConstructionFunctor
sage: class EvenPolynomialRing(type(QQ['x'])):
....:     def __init__(self, base, var):
....:         super().__init__(base, var)
....:         self.register_embedding(base[var])
....:     def __repr__(self):
....:         return "Even Power " + super().__repr__()
....:     def construction(self):
....:         return EvenPolynomialFunctor(), self.base()[self.variable_name()]
....:     def _coerce_map_from_(self, R):
....:         return self.base().has_coerce_map_from(R)
sage: class EvenPolynomialFunctor(ConstructionFunctor):
....:     rank = 10
....:     coercion_reversed = True
....:     def __init__(self):
....:         ConstructionFunctor.__init__(self, Rings(), Rings())
....:     def _apply_functor(self, R):
....:         return EvenPolynomialRing(R.base(), R.variable_name())
sage: pushout(EvenPolynomialRing(QQ, 'x'), ZZ)
Even Power Univariate Polynomial Ring in x over Rational Field
sage: pushout(EvenPolynomialRing(QQ, 'x'), QQ)
Even Power Univariate Polynomial Ring in x over Rational Field
sage: pushout(EvenPolynomialRing(QQ, 'x'), RR)                                  #␣
↪needs sage.rings.real_mpfr
Even Power Univariate Polynomial Ring in x over Real Field with 53 bits of␣
↪precision

sage: pushout(EvenPolynomialRing(QQ, 'x'), ZZ['x'])
Univariate Polynomial Ring in x over Rational Field
sage: pushout(EvenPolynomialRing(QQ, 'x'), QQ['x'])
Univariate Polynomial Ring in x over Rational Field
```

(continues on next page)

(continued from previous page)

```
sage: pushout(EvenPolynomialRing(QQ, 'x'), RR['x'])                             #␣
↪needs sage.rings.real_mpfr
Univariate Polynomial Ring in x over Real Field with 53 bits of precision

sage: pushout(EvenPolynomialRing(QQ, 'x'), EvenPolynomialRing(QQ, 'x'))
Even Power Univariate Polynomial Ring in x over Rational Field
sage: pushout(EvenPolynomialRing(QQ, 'x'), EvenPolynomialRing(RR, 'x'))         #␣
↪needs sage.rings.real_mpfr
Even Power Univariate Polynomial Ring in x over Real Field with 53 bits of␣
↪precision

sage: pushout(EvenPolynomialRing(QQ, 'x')^2, RR^2)                              #␣
↪needs sage.modules sage.rings.real_mpfr
Ambient free module of rank 2
 over the principal ideal domain Even Power Univariate Polynomial Ring in x
  over Real Field with 53 bits of precision
sage: pushout(EvenPolynomialRing(QQ, 'x')^2, RR['x']^2)                         #␣
↪needs sage.modules sage.rings.real_mpfr
Ambient free module of rank 2
 over the principal ideal domain Univariate Polynomial Ring in x
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import ConstructionFunctor
>>> class EvenPolynomialRing(type(QQ['x'])):
...     def __init__(self, base, var):
...         super().__init__(base, var)
...         self.register_embedding(base[var])
...     def __repr__(self):
...         return "Even Power " + super().__repr__()
...     def construction(self):
...         return EvenPolynomialFunctor(), self.base()[self.variable_name()]
...     def _coerce_map_from_(self, R):
...         return self.base().has_coerce_map_from(R)
>>> class EvenPolynomialFunctor(ConstructionFunctor):
...     rank = Integer(10)
...     coercion_reversed = True
...     def __init__(self):
...         ConstructionFunctor.__init__(self, Rings(), Rings())
...     def _apply_functor(self, R):
...         return EvenPolynomialRing(R.base(), R.variable_name())
>>> pushout(EvenPolynomialRing(QQ, 'x'), ZZ)
Even Power Univariate Polynomial Ring in x over Rational Field
>>> pushout(EvenPolynomialRing(QQ, 'x'), QQ)
Even Power Univariate Polynomial Ring in x over Rational Field
>>> pushout(EvenPolynomialRing(QQ, 'x'), RR)                                    #␣
↪needs sage.rings.real_mpfr
Even Power Univariate Polynomial Ring in x over Real Field with 53 bits of␣
↪precision

>>> pushout(EvenPolynomialRing(QQ, 'x'), ZZ['x'])
Univariate Polynomial Ring in x over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> pushout(EvenPolynomialRing(QQ, 'x'), QQ['x'])
Univariate Polynomial Ring in x over Rational Field
>>> pushout(EvenPolynomialRing(QQ, 'x'), RR['x'])                              #␣
↪needs sage.rings.real_mpfr
Univariate Polynomial Ring in x over Real Field with 53 bits of precision

>>> pushout(EvenPolynomialRing(QQ, 'x'), EvenPolynomialRing(QQ, 'x'))
Even Power Univariate Polynomial Ring in x over Rational Field
>>> pushout(EvenPolynomialRing(QQ, 'x'), EvenPolynomialRing(RR, 'x'))          #␣
↪needs sage.rings.real_mpfr
Even Power Univariate Polynomial Ring in x over Real Field with 53 bits of␣
↪precision

>>> pushout(EvenPolynomialRing(QQ, 'x')**Integer(2), RR**Integer(2))              ␣
↪                # needs sage.modules sage.rings.real_mpfr
Ambient free module of rank 2
 over the principal ideal domain Even Power Univariate Polynomial Ring in x
  over Real Field with 53 bits of precision
>>> pushout(EvenPolynomialRing(QQ, 'x')**Integer(2), RR['x']**Integer(2))         ␣
↪                # needs sage.modules sage.rings.real_mpfr
Ambient free module of rank 2
 over the principal ideal domain Univariate Polynomial Ring in x
 over Real Field with 53 bits of precision
```

Some more tests related to univariate/multivariate constructions. We consider a generalization of polynomial rings, where in addition to the coefficient ring C we also specify an additive monoid E for the exponents of the indeterminate. In particular, the elements of such a parent are given by

$$\sum_{i=0}^{I} c_i X^{e_i}$$

with $c_i \in C$ and $e_i \in E$. We define

```
sage: class GPolynomialRing(Parent):
....:     def __init__(self, coefficients, var, exponents):
....:         self.coefficients = coefficients
....:         self.var = var
....:         self.exponents = exponents
....:         super().__init__(category=Rings())
....:     def _repr_(self):
....:         return 'Generalized Polynomial Ring in %s^(%s) over %s' % (
....:                self.var, self.exponents, self.coefficients)
....:     def construction(self):
....:         return GPolynomialFunctor(self.var, self.exponents), self.
↪coefficients
....:     def _coerce_map_from_(self, R):
....:         return self.coefficients.has_coerce_map_from(R)
```

```
>>> from sage.all import *
>>> class GPolynomialRing(Parent):
...     def __init__(self, coefficients, var, exponents):
...         self.coefficients = coefficients
```

(continues on next page)

(continued from previous page)

```
...         self.var = var
...         self.exponents = exponents
...         super().__init__(category=Rings())
...     def _repr_(self):
...         return 'Generalized Polynomial Ring in %s^(%s) over %s' % (
...                self.var, self.exponents, self.coefficients)
...     def construction(self):
...         return GPolynomialFunctor(self.var, self.exponents), self.coefficients
...     def _coerce_map_from_(self, R):
...         return self.coefficients.has_coerce_map_from(R)
```

and

```
sage: class GPolynomialFunctor(ConstructionFunctor):
....:     rank = 10
....:     def __init__(self, var, exponents):
....:         self.var = var
....:         self.exponents = exponents
....:         ConstructionFunctor.__init__(self, Rings(), Rings())
....:     def _repr_(self):
....:         return 'GPoly[%s^(%s)]' % (self.var, self.exponents)
....:     def _apply_functor(self, coefficients):
....:         return GPolynomialRing(coefficients, self.var, self.exponents)
....:     def merge(self, other):
....:         if isinstance(other, GPolynomialFunctor) and self.var == other.var:
....:             exponents = pushout(self.exponents, other.exponents)
....:             return GPolynomialFunctor(self.var, exponents)
```

```
>>> from sage.all import *
>>> class GPolynomialFunctor(ConstructionFunctor):
...     rank = Integer(10)
...     def __init__(self, var, exponents):
...         self.var = var
...         self.exponents = exponents
...         ConstructionFunctor.__init__(self, Rings(), Rings())
...     def _repr_(self):
...         return 'GPoly[%s^(%s)]' % (self.var, self.exponents)
...     def _apply_functor(self, coefficients):
...         return GPolynomialRing(coefficients, self.var, self.exponents)
...     def merge(self, other):
...         if isinstance(other, GPolynomialFunctor) and self.var == other.var:
...             exponents = pushout(self.exponents, other.exponents)
...             return GPolynomialFunctor(self.var, exponents)
```

We can construct a parent now in two different ways:

```
sage: GPolynomialRing(QQ, 'X', ZZ)
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
sage: GP_ZZ = GPolynomialFunctor('X', ZZ); GP_ZZ
GPoly[X^(Integer Ring)]
sage: GP_ZZ(QQ)
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
```

```
>>> from sage.all import *
>>> GPolynomialRing(QQ, 'X', ZZ)
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
>>> GP_ZZ = GPolynomialFunctor('X', ZZ); GP_ZZ
GPoly[X^(Integer Ring)]
>>> GP_ZZ(QQ)
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
```

Since the construction

```
sage: GP_ZZ(QQ).construction()
(GPoly[X^(Integer Ring)], Rational Field)
```

```
>>> from sage.all import *
>>> GP_ZZ(QQ).construction()
(GPoly[X^(Integer Ring)], Rational Field)
```

uses the coefficient ring, we have the usual coercion with respect to this parameter:

```
sage: pushout(GP_ZZ(ZZ), GP_ZZ(QQ))
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
sage: pushout(GP_ZZ(ZZ['t']), GP_ZZ(QQ))
Generalized Polynomial Ring in X^(Integer Ring)
  over Univariate Polynomial Ring in t over Rational Field
sage: pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(ZZ['b,c']))
Generalized Polynomial Ring in X^(Integer Ring)
  over Multivariate Polynomial Ring in a, b, c over Integer Ring
sage: pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(QQ['b,c']))
Generalized Polynomial Ring in X^(Integer Ring)
  over Multivariate Polynomial Ring in a, b, c over Rational Field
sage: pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(ZZ['c,d']))
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', ...)
```

```
>>> from sage.all import *
>>> pushout(GP_ZZ(ZZ), GP_ZZ(QQ))
Generalized Polynomial Ring in X^(Integer Ring) over Rational Field
>>> pushout(GP_ZZ(ZZ['t']), GP_ZZ(QQ))
Generalized Polynomial Ring in X^(Integer Ring)
  over Univariate Polynomial Ring in t over Rational Field
>>> pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(ZZ['b,c']))
Generalized Polynomial Ring in X^(Integer Ring)
  over Multivariate Polynomial Ring in a, b, c over Integer Ring
>>> pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(QQ['b,c']))
Generalized Polynomial Ring in X^(Integer Ring)
  over Multivariate Polynomial Ring in a, b, c over Rational Field
>>> pushout(GP_ZZ(ZZ['a,b']), GP_ZZ(ZZ['c,d']))
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', ...)
```

```
sage: GP_QQ = GPolynomialFunctor('X', QQ)
sage: pushout(GP_ZZ(ZZ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Integer Ring
sage: pushout(GP_QQ(ZZ), GP_ZZ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Integer Ring
```

```
>>> from sage.all import *
>>> GP_QQ = GPolynomialFunctor('X', QQ)
>>> pushout(GP_ZZ(ZZ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Integer Ring
>>> pushout(GP_QQ(ZZ), GP_ZZ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Integer Ring
```

```
sage: GP_ZZt = GPolynomialFunctor('X', ZZ['t'])
sage: pushout(GP_ZZt(ZZ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Integer Ring
```

```
>>> from sage.all import *
>>> GP_ZZt = GPolynomialFunctor('X', ZZ['t'])
>>> pushout(GP_ZZt(ZZ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Integer Ring
```

```
sage: pushout(GP_ZZ(ZZ), GP_QQ(QQ))
Generalized Polynomial Ring in X^(Rational Field) over Rational Field
sage: pushout(GP_ZZ(QQ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Rational Field
sage: pushout(GP_ZZt(QQ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Rational Field
sage: pushout(GP_ZZt(ZZ), GP_QQ(QQ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Rational Field
sage: pushout(GP_ZZt(ZZ['a,b']), GP_QQ(ZZ['c,d']))
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', ...)
sage: pushout(GP_ZZt(ZZ['a,b']), GP_QQ(ZZ['b,c']))
Generalized Polynomial Ring
 in X^(Univariate Polynomial Ring in t over Rational Field)
  over Multivariate Polynomial Ring in a, b, c over Integer Ring
```

```
>>> from sage.all import *
>>> pushout(GP_ZZ(ZZ), GP_QQ(QQ))
Generalized Polynomial Ring in X^(Rational Field) over Rational Field
>>> pushout(GP_ZZ(QQ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Rational Field) over Rational Field
>>> pushout(GP_ZZt(QQ), GP_QQ(ZZ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> pushout(GP_ZZt(ZZ), GP_QQ(QQ))
Generalized Polynomial Ring in X^(Univariate Polynomial Ring in t
  over Rational Field) over Rational Field
>>> pushout(GP_ZZt(ZZ['a,b']), GP_QQ(ZZ['c,d']))
Traceback (most recent call last):
...
CoercionException: ('Ambiguous Base Extension', ...)
>>> pushout(GP_ZZt(ZZ['a,b']), GP_QQ(ZZ['b,c']))
Generalized Polynomial Ring
 in X^(Univariate Polynomial Ring in t over Rational Field)
  over Multivariate Polynomial Ring in a, b, c over Integer Ring
```

Some tests with Cartesian products:

```
sage: from sage.sets.cartesian_product import CartesianProduct
sage: A = CartesianProduct((ZZ['x'], QQ['y'], QQ['z']),
....:                      Sets().CartesianProducts())
sage: B = CartesianProduct((ZZ['x'], ZZ['y'], ZZ['t']['z']),
....:                      Sets().CartesianProducts())
sage: A.construction()
(The cartesian_product functorial construction,
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over Rational Field))
sage: pushout(A, B)
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over
   Univariate Polynomial Ring in t over Rational Field)
sage: pushout(ZZ, cartesian_product([ZZ, QQ]))
Traceback (most recent call last):
...
CoercionException: 'NoneType' object is not iterable
```

```
>>> from sage.all import *
>>> from sage.sets.cartesian_product import CartesianProduct
>>> A = CartesianProduct((ZZ['x'], QQ['y'], QQ['z']),
...                      Sets().CartesianProducts())
>>> B = CartesianProduct((ZZ['x'], ZZ['y'], ZZ['t']['z']),
...                      Sets().CartesianProducts())
>>> A.construction()
(The cartesian_product functorial construction,
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over Rational Field))
>>> pushout(A, B)
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over
   Univariate Polynomial Ring in t over Rational Field)
```

(continues on next page)

(continued from previous page)

```
>>> pushout(ZZ, cartesian_product([ZZ, QQ]))
Traceback (most recent call last):
...
CoercionException: 'NoneType' object is not iterable
```

```
sage: from sage.categories.pushout import PolynomialFunctor
sage: from sage.sets.cartesian_product import CartesianProduct
sage: class CartesianProductPoly(CartesianProduct):
....:     def __init__(self, polynomial_rings):
....:         sort = sorted(polynomial_rings,
....:                       key=lambda P: P.variable_name())
....:         super().__init__(sort, Sets().CartesianProducts())
....:     def vars(self):
....:         return tuple(P.variable_name()
....:                      for P in self.cartesian_factors())
....:     def _pushout_(self, other):
....:         if isinstance(other, CartesianProductPoly):
....:             s_vars = self.vars()
....:             o_vars = other.vars()
....:             if s_vars == o_vars:
....:                 return
....:             return pushout(CartesianProductPoly(
....:                     self.cartesian_factors() +
....:                     tuple(f for f in other.cartesian_factors()
....:                           if f.variable_name() not in s_vars)),
....:                 CartesianProductPoly(
....:                     other.cartesian_factors() +
....:                     tuple(f for f in self.cartesian_factors()
....:                           if f.variable_name() not in o_vars)))
....:         C = other.construction()
....:         if C is None:
....:             return
....:         elif isinstance(C[0], PolynomialFunctor):
....:             return pushout(self, CartesianProductPoly((other,)))
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import PolynomialFunctor
>>> from sage.sets.cartesian_product import CartesianProduct
>>> class CartesianProductPoly(CartesianProduct):
...     def __init__(self, polynomial_rings):
...         sort = sorted(polynomial_rings,
...                       key=lambda P: P.variable_name())
...         super().__init__(sort, Sets().CartesianProducts())
...     def vars(self):
...         return tuple(P.variable_name()
...                      for P in self.cartesian_factors())
...     def _pushout_(self, other):
...         if isinstance(other, CartesianProductPoly):
...             s_vars = self.vars()
...             o_vars = other.vars()
...             if s_vars == o_vars:
```

(continues on next page)

(continued from previous page)

```
...                 return
...             return pushout(CartesianProductPoly(
...                     self.cartesian_factors() +
...                     tuple(f for f in other.cartesian_factors()
...                           if f.variable_name() not in s_vars)),
...                 CartesianProductPoly(
...                     other.cartesian_factors() +
...                     tuple(f for f in self.cartesian_factors()
...                           if f.variable_name() not in o_vars)))
...         C = other.construction()
...         if C is None:
...             return
...         elif isinstance(C[Integer(0)], PolynomialFunctor):
...             return pushout(self, CartesianProductPoly((other,)))
```

```
sage: pushout(CartesianProductPoly((ZZ['x'],)),
....:         CartesianProductPoly((ZZ['y'],)))
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring)
sage: pushout(CartesianProductPoly((ZZ['x'], ZZ['y'])),
....:         CartesianProductPoly((ZZ['x'], ZZ['z'])))
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring,
  Univariate Polynomial Ring in z over Integer Ring)
sage: pushout(CartesianProductPoly((QQ['a,b']['x'], QQ['y'])),                #␣
↪needs sage.symbolic
....:         CartesianProductPoly((ZZ['b,c']['x'], SR['z'])))
The Cartesian product of
 (Univariate Polynomial Ring in x over
    Multivariate Polynomial Ring in a, b, c over Rational Field,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over Symbolic Ring)
```

```
>>> from sage.all import *
>>> pushout(CartesianProductPoly((ZZ['x'],)),
...         CartesianProductPoly((ZZ['y'],)))
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring)
>>> pushout(CartesianProductPoly((ZZ['x'], ZZ['y'])),
...         CartesianProductPoly((ZZ['x'], ZZ['z'])))
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring,
  Univariate Polynomial Ring in z over Integer Ring)
>>> pushout(CartesianProductPoly((QQ['a,b']['x'], QQ['y'])),                  #␣
↪needs sage.symbolic
...         CartesianProductPoly((ZZ['b,c']['x'], SR['z'])))
The Cartesian product of
```

(continues on next page)

(continued from previous page)

```
 (Univariate Polynomial Ring in x over
    Multivariate Polynomial Ring in a, b, c over Rational Field,
  Univariate Polynomial Ring in y over Rational Field,
  Univariate Polynomial Ring in z over Symbolic Ring)
```

```
sage: pushout(CartesianProductPoly((ZZ['x'],)), ZZ['y'])
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring)
sage: pushout(QQ['b,c']['y'], CartesianProductPoly((ZZ['a,b']['x'],)))
The Cartesian product of
 (Univariate Polynomial Ring in x over
    Multivariate Polynomial Ring in a, b over Integer Ring,
  Univariate Polynomial Ring in y over
    Multivariate Polynomial Ring in b, c over Rational Field)
```

```
>>> from sage.all import *
>>> pushout(CartesianProductPoly((ZZ['x'],)), ZZ['y'])
The Cartesian product of
 (Univariate Polynomial Ring in x over Integer Ring,
  Univariate Polynomial Ring in y over Integer Ring)
>>> pushout(QQ['b,c']['y'], CartesianProductPoly((ZZ['a,b']['x'],)))
The Cartesian product of
 (Univariate Polynomial Ring in x over
    Multivariate Polynomial Ring in a, b over Integer Ring,
  Univariate Polynomial Ring in y over
    Multivariate Polynomial Ring in b, c over Rational Field)
```

```
sage: pushout(CartesianProductPoly((ZZ['x'],)), ZZ)
Traceback (most recent call last):
...
CoercionException: No common base ("join") found for
The cartesian_product functorial construction(...) and None(Integer Ring):
(Multivariate) functors are incompatible.
```

```
>>> from sage.all import *
>>> pushout(CartesianProductPoly((ZZ['x'],)), ZZ)
Traceback (most recent call last):
...
CoercionException: No common base ("join") found for
The cartesian_product functorial construction(...) and None(Integer Ring):
(Multivariate) functors are incompatible.
```

AUTHORS:

- Robert Bradshaw
- Peter Bruin
- Simon King
- Daniel Krenn
- David Roe

sage.categories.pushout.**pushout_lattice**(*R*, *S*)

Given a pair of objects R and S, try to construct a reasonable object Y and return maps such that canonically $R \leftarrow Y \rightarrow S$.

ALGORITHM:

This is based on the model that arose from much discussion at Sage Days 4. Going up the tower of constructions of R and S (e.g. the reals come from the rationals come from the integers), try to find a common parent, and then try to fill in a lattice with these two towers as sides with the top as the common ancestor and the bottom will be the desired ring.

See the code for a specific worked-out example.

EXAMPLES:

```
sage: from sage.categories.pushout import pushout_lattice
sage: A, B = pushout_lattice(Qp(7), Frac(ZZ['x']))                          #␣
↪needs sage.rings.padics
sage: A.codomain()                                                          #␣
↪needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
sage: A.codomain() is B.codomain()                                          #␣
↪needs sage.rings.padics
True
sage: A, B = pushout_lattice(ZZ, MatrixSpace(ZZ[['x']], 3, 3))             #␣
↪needs sage.modules
sage: B                                                                     #␣
↪needs sage.modules
Identity endomorphism of Full MatrixSpace of 3 by 3 dense matrices
 over Power Series Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> from sage.categories.pushout import pushout_lattice
>>> A, B = pushout_lattice(Qp(Integer(7)), Frac(ZZ['x']))                      ␣
↪     # needs sage.rings.padics
>>> A.codomain()                                                            #␣
↪needs sage.rings.padics
Fraction Field of Univariate Polynomial Ring in x
 over 7-adic Field with capped relative precision 20
>>> A.codomain() is B.codomain()                                            #␣
↪needs sage.rings.padics
True
>>> A, B = pushout_lattice(ZZ, MatrixSpace(ZZ[['x']], Integer(3), Integer(3)))  ␣
↪              # needs sage.modules
>>> B                                                                       #␣
↪needs sage.modules
Identity endomorphism of Full MatrixSpace of 3 by 3 dense matrices
 over Power Series Ring in x over Integer Ring
```

AUTHOR:

- Robert Bradshaw

sage.categories.pushout.**type_to_parent**(*P*)

An auxiliary function that is used in *pushout()*.

INPUT:

- `P` – a type

OUTPUT: a Sage parent structure corresponding to the given type

CHAPTER

FOUR

INDIVIDUAL CATEGORIES

4.1 Group, ring, etc. actions on objects

The terminology and notation used is suggestive of groups acting on sets, but this framework can be used for modules, algebras, etc.

A group action $G \times S \to S$ is a functor from G to Sets.

Warning

An *Action* object only keeps a weak reference to the underlying set which is acted upon. This decision was made in Issue #715 in order to allow garbage collection within the coercion framework (this is where actions are mainly used) and avoid memory leaks.

```
sage: from sage.categories.action import Action
sage: class P: pass
sage: A = Action(P(),P())
sage: import gc
sage: _ = gc.collect()
sage: A
<repr(<sage.categories.action.Action at 0x...>) failed:
 RuntimeError: This action acted on a set that became garbage collected>
```

```
>>> from sage.all import *
>>> from sage.categories.action import Action
>>> class P: pass
>>> A = Action(P(),P())
>>> import gc
>>> _ = gc.collect()
>>> A
<repr(<sage.categories.action.Action at 0x...>) failed:
 RuntimeError: This action acted on a set that became garbage collected>
```

To avoid garbage collection of the underlying set, it is sufficient to create a strong reference to it before the action is created.

```
sage: _ = gc.collect()
sage: from sage.categories.action import Action
sage: class P: pass
sage: q = P()
sage: A = Action(P(),q)
sage: gc.collect()
0
sage: A
Left action by <__main__.P ... at ...> on <__main__.P ... at ...>
```

```
>>> from sage.all import *
>>> _ = gc.collect()
>>> from sage.categories.action import Action
>>> class P: pass
>>> q = P()
>>> A = Action(P(),q)
>>> gc.collect()
0
>>> A
Left action by <__main__.P ... at ...> on <__main__.P ... at ...>
```

AUTHOR:

- Robert Bradshaw: initial version

class sage.categories.action.**Action**

Bases: *Functor*

The action of `G` on `S`.

INPUT:

- `G` – a parent or Python type
- `S` – a parent or Python type
- `is_left` – boolean (default: `True`); whether elements of `G` are on the left
- `op` – (default: `None`) operation. This is not used by *Action* itself, but other classes may use it

G

act(*g*, *x*)

This is a consistent interface for acting on `x` by `g`, regardless of whether it's a left or right action.

If needed, `g` and `x` are converted to the correct parent.

EXAMPLES:

```
sage: R.<x> = ZZ []
sage: from sage.structure.coerce_actions import IntegerMulAction
sage: A = IntegerMulAction(ZZ, R, True)   # Left action
sage: A.act(5, x)
5*x
sage: A.act(int(5), x)
5*x
sage: A = IntegerMulAction(ZZ, R, False)  # Right action
sage: A.act(5, x)
5*x
sage: A.act(int(5), x)
5*x
```

```
>>> from sage.all import *
>>> R = ZZ ['x']; (x,) = R._first_ngens(1)
>>> from sage.structure.coerce_actions import IntegerMulAction
>>> A = IntegerMulAction(ZZ, R, True)   # Left action
```

(continues on next page)

(continued from previous page)

```
>>> A.act(Integer(5), x)
5*x
>>> A.act(int(Integer(5)), x)
5*x
>>> A = IntegerMulAction(ZZ, R, False)  # Right action
>>> A.act(Integer(5), x)
5*x
>>> A.act(int(Integer(5)), x)
5*x
```

actor()

codomain()

domain()

is_left()

left_domain()

op

operation()

right_domain()

class sage.categories.action.**ActionEndomorphism**

Bases: *Morphism*

The endomorphism defined by the action of one element.

EXAMPLES:

```
sage: A = ZZ['x'].get_action(QQ, self_on_left=False, op=operator.mul)
sage: A
Left scalar multiplication by Rational Field
 on Univariate Polynomial Ring in x over Integer Ring
sage: A(1/2)
Action of 1/2 on Univariate Polynomial Ring in x over Integer Ring
under Left scalar multiplication by Rational Field on Univariate
Polynomial Ring in x over Integer Ring.
```

```
>>> from sage.all import *
>>> A = ZZ['x'].get_action(QQ, self_on_left=False, op=operator.mul)
>>> A
Left scalar multiplication by Rational Field
 on Univariate Polynomial Ring in x over Integer Ring
>>> A(Integer(1)/Integer(2))
Action of 1/2 on Univariate Polynomial Ring in x over Integer Ring
under Left scalar multiplication by Rational Field on Univariate
Polynomial Ring in x over Integer Ring.
```

class sage.categories.action.**InverseAction**

Bases: *Action*

An action that acts as the inverse of the given action.

EXAMPLES:

```
sage: V = QQ^3                                                              #␣
↪needs sage.modules
sage: v = V((1, 2, 3))                                                      #␣
↪needs sage.modules
sage: cm = get_coercion_model()

sage: # needs sage.modules
sage: a = cm.get_action(V, QQ, operator.mul)
sage: a
Right scalar multiplication by Rational Field
 on Vector space of dimension 3 over Rational Field
sage: ~a
Right inverse action by Rational Field
 on Vector space of dimension 3 over Rational Field
sage: (~a)(v, 1/3)
(3, 6, 9)

sage: # needs sage.modules
sage: b = cm.get_action(QQ, V, operator.mul)
sage: b
Left scalar multiplication by Rational Field
 on Vector space of dimension 3 over Rational Field
sage: ~b
Left inverse action by Rational Field
 on Vector space of dimension 3 over Rational Field
sage: (~b)(1/3, v)
(3, 6, 9)

sage: c = cm.get_action(ZZ, list, operator.mul)
sage: c
Left action by Integer Ring on <... 'list'>
sage: ~c
Traceback (most recent call last):
...
TypeError: no inverse defined for Left action by Integer Ring on <... 'list'>
```

```
>>> from sage.all import *
>>> V = QQ**Integer(3)                                                      ␣
↪      # needs sage.modules
>>> v = V((Integer(1), Integer(2), Integer(3)))                             ␣
↪                        # needs sage.modules
>>> cm = get_coercion_model()

>>> # needs sage.modules
>>> a = cm.get_action(V, QQ, operator.mul)
>>> a
Right scalar multiplication by Rational Field
 on Vector space of dimension 3 over Rational Field
>>> ~a
Right inverse action by Rational Field
 on Vector space of dimension 3 over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> (~a)(v, Integer(1)/Integer(3))
(3, 6, 9)

>>> # needs sage.modules
>>> b = cm.get_action(QQ, V, operator.mul)
>>> b
Left scalar multiplication by Rational Field
 on Vector space of dimension 3 over Rational Field
>>> ~b
Left inverse action by Rational Field
 on Vector space of dimension 3 over Rational Field
>>> (~b)(Integer(1)/Integer(3), v)
(3, 6, 9)

>>> c = cm.get_action(ZZ, list, operator.mul)
>>> c
Left action by Integer Ring on <... 'list'>
>>> ~c
Traceback (most recent call last):
...
TypeError: no inverse defined for Left action by Integer Ring on <... 'list'>
```

codomain()

class sage.categories.action.**PrecomposedAction**

Bases: *Action*

A precomposed action first applies given maps, and then applying an action to the return values of the maps.

EXAMPLES:

We demonstrate that an example discussed on Issue #14711 did not become a problem:

```
sage: # needs sage.libs.flint sage.modular
sage: E = ModularSymbols(11).2
sage: s = E.modular_symbol_rep()
sage: del E,s
sage: import gc
sage: _ = gc.collect()
sage: E = ModularSymbols(11).2
sage: v = E.manin_symbol_rep()
sage: c,x = v[0]
sage: y = x.modular_symbol_rep()
sage: coercion_model.get_action(QQ, parent(y), op=operator.mul)
Left scalar multiplication by Rational Field
 on Abelian Group of all Formal Finite Sums over Rational Field
 with precomposition on right by Coercion map:
  From: Abelian Group of all Formal Finite Sums over Integer Ring
  To:   Abelian Group of all Formal Finite Sums over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.libs.flint sage.modular
>>> E = ModularSymbols(Integer(11)).gen(2)
```

(continues on next page)

(continued from previous page)

```
>>> s = E.modular_symbol_rep()
>>> del E,s
>>> import gc
>>> _ = gc.collect()
>>> E = ModularSymbols(Integer(11)).gen(2)
>>> v = E.manin_symbol_rep()
>>> c,x = v[Integer(0)]
>>> y = x.modular_symbol_rep()
>>> coercion_model.get_action(QQ, parent(y), op=operator.mul)
Left scalar multiplication by Rational Field
 on Abelian Group of all Formal Finite Sums over Rational Field
 with precomposition on right by Coercion map:
  From: Abelian Group of all Formal Finite Sums over Integer Ring
  To:   Abelian Group of all Formal Finite Sums over Rational Field
```

codomain()

domain()

left_precomposition

The left map to precompose with, or `None` if there is no left precomposition map.

right_precomposition

The right map to precompose with, or `None` if there is no right precomposition map.

4.2 Additive groups

class sage.categories.additive_groups.**AdditiveGroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of additive groups.

An *additive group* is a set with an internal binary operation $+$ which is associative, admits a zero, and where every element can be negated.

EXAMPLES:

```
sage: from sage.categories.additive_groups import AdditiveGroups
sage: from sage.categories.additive_monoids import AdditiveMonoids
sage: AdditiveGroups()
Category of additive groups
sage: AdditiveGroups().super_categories()
[Category of additive inverse additive unital additive magmas,
 Category of additive monoids]
sage: AdditiveGroups().all_super_categories()
[Category of additive groups,
 Category of additive inverse additive unital additive magmas,
 Category of additive monoids,
 Category of additive unital additive magmas,
 Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
```

(continues on next page)

(continued from previous page)

```
 Category of objects]

sage: AdditiveGroups().axioms()
frozenset({'AdditiveAssociative', 'AdditiveInverse', 'AdditiveUnital'})
sage: AdditiveGroups() is AdditiveMonoids().AdditiveInverse()
True
```

```
>>> from sage.all import *
>>> from sage.categories.additive_groups import AdditiveGroups
>>> from sage.categories.additive_monoids import AdditiveMonoids
>>> AdditiveGroups()
Category of additive groups
>>> AdditiveGroups().super_categories()
[Category of additive inverse additive unital additive magmas,
 Category of additive monoids]
>>> AdditiveGroups().all_super_categories()
[Category of additive groups,
 Category of additive inverse additive unital additive magmas,
 Category of additive monoids,
 Category of additive unital additive magmas,
 Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

>>> AdditiveGroups().axioms()
frozenset({'AdditiveAssociative', 'AdditiveInverse', 'AdditiveUnital'})
>>> AdditiveGroups() is AdditiveMonoids().AdditiveInverse()
True
```

AdditiveCommutative

alias of *CommutativeAdditiveGroups*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

group()

Return the underlying group of the group algebra.

EXAMPLES:

```
sage: GroupAlgebras(QQ).example(GL(3, GF(11))).group()
↪ # needs sage.groups sage.modules
General Linear Group of degree 3 over Finite Field of size 11
sage: SymmetricGroup(10).algebra(QQ).group()
↪ # needs sage.combinat sage.groups sage.modules
Symmetric group of order 10! as a permutation group
```

```
>>> from sage.all import *
>>> GroupAlgebras(QQ).example(GL(Integer(3), GF(Integer(11)))).group()␣
↪                  # needs sage.groups sage.modules
General Linear Group of degree 3 over Finite Field of size 11
>>> SymmetricGroup(Integer(10)).algebra(QQ).group()                    ␣
↪         # needs sage.combinat sage.groups sage.modules
Symmetric group of order 10! as a permutation group
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

extra_super_categories()

Implement Maschke's theorem.

In characteristic 0 all finite group algebras are semisimple.

EXAMPLES:

```
sage: FiniteGroups().Algebras(QQ).is_subcategory(Algebras(QQ).
↪Semisimple())
True
sage: FiniteGroups().Algebras(FiniteField(7)).is_
↪subcategory(Algebras(FiniteField(7)).Semisimple())
False
sage: FiniteGroups().Algebras(ZZ).is_subcategory(Algebras(ZZ).
↪Semisimple())
False
sage: FiniteGroups().Algebras(Fields()).is_
↪subcategory(Algebras(Fields()).Semisimple())
False

sage: Cat = CommutativeAdditiveGroups().Finite()
sage: Cat.Algebras(QQ).is_subcategory(Algebras(QQ).Semisimple())
True
sage: Cat.Algebras(GF(7)).is_subcategory(Algebras(GF(7)).Semisimple())
False
sage: Cat.Algebras(ZZ).is_subcategory(Algebras(ZZ).Semisimple())
False
sage: Cat.Algebras(Fields()).is_subcategory(Algebras(Fields()).
↪Semisimple())
False
```

```
>>> from sage.all import *
>>> FiniteGroups().Algebras(QQ).is_subcategory(Algebras(QQ).
↪Semisimple())
True
>>> FiniteGroups().Algebras(FiniteField(Integer(7))).is_
↪subcategory(Algebras(FiniteField(Integer(7))).Semisimple())
```

(continues on next page)

(continued from previous page)

```
False
>>> FiniteGroups().Algebras(ZZ).is_subcategory(Algebras(ZZ).
↪Semisimple())
False
>>> FiniteGroups().Algebras(Fields()).is_
↪subcategory(Algebras(Fields()).Semisimple())
False

>>> Cat = CommutativeAdditiveGroups().Finite()
>>> Cat.Algebras(QQ).is_subcategory(Algebras(QQ).Semisimple())
True
>>> Cat.Algebras(GF(Integer(7))).is_
↪subcategory(Algebras(GF(Integer(7))).Semisimple())
False
>>> Cat.Algebras(ZZ).is_subcategory(Algebras(ZZ).Semisimple())
False
>>> Cat.Algebras(Fields()).is_subcategory(Algebras(Fields()).
↪Semisimple())
False
```

4.3 Additive magmas

class sage.categories.additive_magmas.**AdditiveMagmas**

Bases: *Category_singleton*

The category of additive magmas.

An additive magma is a set endowed with a binary operation $+$.

EXAMPLES:

```
sage: AdditiveMagmas()
Category of additive magmas
sage: AdditiveMagmas().super_categories()
[Category of sets]
sage: AdditiveMagmas().all_super_categories()
[Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> AdditiveMagmas()
Category of additive magmas
>>> AdditiveMagmas().super_categories()
[Category of sets]
>>> AdditiveMagmas().all_super_categories()
[Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

The following axioms are defined by this category:

```
sage: AdditiveMagmas().AdditiveAssociative()
Category of additive semigroups
sage: AdditiveMagmas().AdditiveUnital()
Category of additive unital additive magmas
sage: AdditiveMagmas().AdditiveCommutative()
Category of additive commutative additive magmas
sage: AdditiveMagmas().AdditiveUnital().AdditiveInverse()
Category of additive inverse additive unital additive magmas
sage: C = AdditiveMagmas().AdditiveAssociative().AdditiveCommutative(); C
Category of commutative additive semigroups
sage: C.AdditiveUnital()
Category of commutative additive monoids
sage: C.AdditiveUnital().AdditiveInverse()
Category of commutative additive groups
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveAssociative()
Category of additive semigroups
>>> AdditiveMagmas().AdditiveUnital()
Category of additive unital additive magmas
>>> AdditiveMagmas().AdditiveCommutative()
Category of additive commutative additive magmas
>>> AdditiveMagmas().AdditiveUnital().AdditiveInverse()
Category of additive inverse additive unital additive magmas
>>> C = AdditiveMagmas().AdditiveAssociative().AdditiveCommutative(); C
Category of commutative additive semigroups
>>> C.AdditiveUnital()
Category of commutative additive monoids
>>> C.AdditiveUnital().AdditiveInverse()
Category of commutative additive groups
```

AdditiveAssociative

alias of *AdditiveSemigroups*

class AdditiveCommutative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

extra_super_categories()

Implement the fact that the algebra of a commutative additive magmas is commutative.

EXAMPLES:

```
sage: C = AdditiveMagmas().AdditiveCommutative().Algebras(QQ)
sage: C.extra_super_categories()
[Category of commutative magmas]

sage: C.super_categories()
[Category of additive magma algebras over Rational Field,
 Category of commutative magmas]
```

```
>>> from sage.all import *
>>> C = AdditiveMagmas().AdditiveCommutative().Algebras(QQ)
>>> C.extra_super_categories()
[Category of commutative magmas]

>>> C.super_categories()
[Category of additive magma algebras over Rational Field,
 Category of commutative magmas]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of commutative additive magmas is a commutative additive magma.

EXAMPLES:

```
sage: C = AdditiveMagmas().AdditiveCommutative().CartesianProducts()
sage: C.extra_super_categories()
[Category of additive commutative additive magmas]
sage: C.axioms()
frozenset({'AdditiveCommutative'})
```

```
>>> from sage.all import *
>>> C = AdditiveMagmas().AdditiveCommutative().CartesianProducts()
>>> C.extra_super_categories()
[Category of additive commutative additive magmas]
>>> C.axioms()
frozenset({'AdditiveCommutative'})
```

class AdditiveUnital(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class AdditiveInverse(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ElementMethods

Bases: object

extra_super_categories()

Implement the fact that a Cartesian product of additive magmas with inverses is an additive magma with inverse.

EXAMPLES:

```
sage: C = AdditiveMagmas().AdditiveUnital().AdditiveInverse().
 ↪CartesianProducts()
sage: C.extra_super_categories()
[Category of additive inverse additive unital additive magmas]
sage: sorted(C.axioms())
['AdditiveInverse', 'AdditiveUnital']
```

```
>>> from sage.all import *
>>> C = AdditiveMagmas().AdditiveUnital().AdditiveInverse().
↪CartesianProducts()
>>> C.extra_super_categories()
[Category of additive inverse additive unital additive magmas]
>>> sorted(C.axioms())
['AdditiveInverse', 'AdditiveUnital']
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

one_basis()

Return the zero of this additive magma, which index the one of this algebra, as per *one_basis()*.

EXAMPLES:

```
sage: # needs sage.modules
sage: S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid:
 the free commutative monoid generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(ZZ)
sage: A.one_basis()
0
sage: A.one()
B[0]
sage: A(3)
3*B[0]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid:
 the free commutative monoid generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(ZZ)
>>> A.one_basis()
0
>>> A.one()
B[0]
>>> A(Integer(3))
3*B[0]
```

extra_super_categories()

EXAMPLES:

```
sage: C = AdditiveMagmas().AdditiveUnital().Algebras(QQ)
sage: C.extra_super_categories()
[Category of unital magmas]

sage: C.super_categories()
[Category of additive magma algebras over Rational Field,
 Category of unital algebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = AdditiveMagmas().AdditiveUnital().Algebras(QQ)
>>> C.extra_super_categories()
[Category of unital magmas]

>>> C.super_categories()
[Category of additive magma algebras over Rational Field,
 Category of unital algebras with basis over Rational Field]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ParentMethods

Bases: object

zero()

Return the zero of this group.

EXAMPLES:

```
sage: GF(8, 'x').cartesian_product(GF(5)).zero()                #␣
↪needs sage.rings.finite_rings
(0, 0)
```

```
>>> from sage.all import *
>>> GF(Integer(8), 'x').cartesian_product(GF(Integer(5))).zero()    ␣
↪           # needs sage.rings.finite_rings
(0, 0)
```

extra_super_categories()

Implement the fact that a Cartesian product of unital additive magmas is a unital additive magma.

EXAMPLES:

```
sage: C = AdditiveMagmas().AdditiveUnital().CartesianProducts()
sage: C.extra_super_categories()
[Category of additive unital additive magmas]
sage: C.axioms()
frozenset({'AdditiveUnital'})
```

```
>>> from sage.all import *
>>> C = AdditiveMagmas().AdditiveUnital().CartesianProducts()
>>> C.extra_super_categories()
[Category of additive unital additive magmas]
>>> C.axioms()
frozenset({'AdditiveUnital'})
```

class ElementMethods

Bases: object

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class ParentMethods

Bases: object

zero()

EXAMPLES:

```
sage: R = QQ['x']
sage: H = Hom(ZZ, R, AdditiveMagmas().AdditiveUnital())
sage: f = H.zero()
sage: f
Generic morphism:
  From: Integer Ring
  To:   Univariate Polynomial Ring in x over Rational Field
sage: f(3)
0
sage: f(3) is R.zero()
True
```

```
>>> from sage.all import *
>>> R = QQ['x']
>>> H = Hom(ZZ, R, AdditiveMagmas().AdditiveUnital())
>>> f = H.zero()
>>> f
Generic morphism:
  From: Integer Ring
  To:   Univariate Polynomial Ring in x over Rational Field
>>> f(Integer(3))
0
>>> f(Integer(3)) is R.zero()
True
```

extra_super_categories()

Implement the fact that a homset between two unital additive magmas is a unital additive magma.

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveUnital().Homsets().extra_super_
↪categories()
[Category of additive unital additive magmas]
sage: AdditiveMagmas().AdditiveUnital().Homsets().super_categories()
[Category of additive unital additive magmas, Category of homsets]
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveUnital().Homsets().extra_super_
↪categories()
[Category of additive unital additive magmas]
>>> AdditiveMagmas().AdditiveUnital().Homsets().super_categories()
[Category of additive unital additive magmas, Category of homsets]
```

class ParentMethods

Bases: object

is_empty()

Return whether this set is empty.

Since this set is an additive magma it has a zero element and hence is not empty. This method thus always returns False.

EXAMPLES:

```
sage: # needs sage.modules
sage: A = AdditiveAbelianGroup([3, 3])
sage: A in AdditiveMagmas()
True
sage: A.is_empty()
False

sage: B = CommutativeAdditiveMonoids().example()
sage: B.is_empty()
False
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = AdditiveAbelianGroup([Integer(3), Integer(3)])
>>> A in AdditiveMagmas()
True
>>> A.is_empty()
False

>>> B = CommutativeAdditiveMonoids().example()
>>> B.is_empty()
False
```

zero()

Return the zero of this additive magma, that is the unique neutral element for $+$.

The default implementation is to coerce `0` into `self`.

It is recommended to override this method because the coercion from the integers:
- is not always meaningful (except for 0), and
- often uses `self.zero()` otherwise.

EXAMPLES:

```
sage: S = CommutativeAdditiveMonoids().example()
sage: S.zero()
0
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveMonoids().example()
>>> S.zero()
0
```

class SubcategoryMethods

Bases: `object`

AdditiveInverse()

Return the full subcategory of the additive inverse objects of `self`.

An inverse *additive magma* is a *unital additive magma* such that every element admits both an additive inverse on the left and on the right. Such an additive magma is also called an *additive loop*.

See also

Wikipedia article Inverse_element, Wikipedia article Quasigroup

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveUnital().AdditiveInverse()
Category of additive inverse additive unital additive magmas
sage: from sage.categories.additive_monoids import AdditiveMonoids
sage: AdditiveMonoids().AdditiveInverse()
Category of additive groups
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveUnital().AdditiveInverse()
Category of additive inverse additive unital additive magmas
>>> from sage.categories.additive_monoids import AdditiveMonoids
>>> AdditiveMonoids().AdditiveInverse()
Category of additive groups
```

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

class ParentMethods

Bases: object

zero()

Return the zero of this unital additive magma.

This default implementation returns the zero of the realization of `self` given by *a_realization()*.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.zero.__module__                                             #␣
↪needs sage.modules
'sage.categories.additive_magmas'
sage: A.zero()                                                      #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.zero.__module__                                               #␣
↪needs sage.modules
'sage.categories.additive_magmas'
>>> A.zero()                                                        #␣
↪needs sage.modules
0
```

additional_structure()

Return whether `self` is a structure category.

> **See also**
>
> *Category.additional_structure()*

The category of unital additive magmas defines the zero as additional structure, and this zero shall be preserved by morphisms.

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveUnital().additional_structure()
Category of additive unital additive magmas
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveUnital().additional_structure()
Category of additive unital additive magmas
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: `object`

algebra_generators()

The generators of this algebra, as per *MagmaticAlgebras.ParentMethods.algebra_generators()*.

They correspond to the generators of the additive semigroup.

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: A.algebra_generators()                                            #␣
↪needs sage.modules
Family (B[a], B[b], B[c], B[d])
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> A.algebra_generators()                                              #␣
↪needs sage.modules
Family (B[a], B[b], B[c], B[d])
```

Todo

This doctest does not actually test this method, but rather the method of the same name for `AdditiveSemigroups`. Find a better doctest!

product_on_basis(*g1*, *g2*)

Product, on basis elements, as per *MagmaticAlgebras.WithBasis.ParentMethods.product_on_basis()*.

The product of two basis elements is induced by the addition of the corresponding elements of the group.

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: a, b, c, d = A.algebra_generators()                               #␣
↪needs sage.modules
sage: a * d * b                                                         #␣
↪needs sage.modules
B[a + b + d]
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> a, b, c, d = A.algebra_generators()                                 #␣
↪needs sage.modules
>>> a * d * b                                                           #␣
↪needs sage.modules
B[a + b + d]
```

Todo

This doctest does not actually test this method, but rather the method of the same name for `AdditiveSemigroups`. Find a better doctest!

extra_super_categories()

EXAMPLES:

```
sage: AdditiveMagmas().Algebras(QQ).extra_super_categories()
[Category of magmatic algebras with basis over Rational Field]

sage: AdditiveMagmas().Algebras(QQ).super_categories()
[Category of magmatic algebras with basis over Rational Field,
 Category of set algebras over Rational Field]
```

```
sage: from sage.all import *
sage: AdditiveMagmas().Algebras(QQ).extra_super_categories()
[Category of magmatic algebras with basis over Rational Field]

sage: AdditiveMagmas().Algebras(QQ).super_categories()
[Category of magmatic algebras with basis over Rational Field,
 Category of set algebras over Rational Field]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ElementMethods

Bases: object

extra_super_categories()

Implement the fact that a Cartesian product of additive magmas is an additive magma.

EXAMPLES:

```
sage: C = AdditiveMagmas().CartesianProducts()
sage: C.extra_super_categories()
[Category of additive magmas]
sage: C.super_categories()
[Category of additive magmas, Category of Cartesian products of sets]
sage: C.axioms()
frozenset()
```

```
sage: from sage.all import *
sage: C = AdditiveMagmas().CartesianProducts()
sage: C.extra_super_categories()
[Category of additive magmas]
sage: C.super_categories()
[Category of additive magmas, Category of Cartesian products of sets]
sage: C.axioms()
frozenset()
```

class ElementMethods

Bases: object

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

extra_super_categories()

Implement the fact that a homset between two magmas is a magma.

EXAMPLES:

```
sage: AdditiveMagmas().Homsets().extra_super_categories()
[Category of additive magmas]
sage: AdditiveMagmas().Homsets().super_categories()
[Category of additive magmas, Category of homsets]
```

```
sage: from sage.all import *
sage: AdditiveMagmas().Homsets().extra_super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of additive magmas]
>>> AdditiveMagmas().Homsets().super_categories()
[Category of additive magmas, Category of homsets]
```

class ParentMethods

Bases: object

addition_table(*names='letters'*, *elements=None*)

Return a table describing the addition operation.

Note

The order of the elements in the row and column headings is equal to the order given by the table's column_keys() method. The association can also be retrieved with the translation() method.

INPUT:

- names – the type of names used:
  - 'letters' – lowercase ASCII letters are used for a base 26 representation of the elements' positions in the list given by column_keys(), padded to a common width with leading 'a's.
  - 'digits' – base 10 representation of the elements' positions in the list given by column_keys(), padded to a common width with leading zeros.
  - 'elements' – the string representations of the elements themselves.
  - a list - a list of strings, where the length of the list equals the number of elements.
- elements – (default: None) A list of elements of the additive magma, in forms that can be coerced into the structure, eg. their string representations. This may be used to impose an alternate ordering on the elements, perhaps when this is used in the context of a particular structure. The default is to use whatever ordering the S.list method returns. Or the elements can be a subset which is closed under the operation. In particular, this can be used when the base set is infinite.

OUTPUT:

The addition table as an object of the class OperationTable which defines several methods for manipulating and displaying the table. See the documentation there for full details to supplement the documentation here.

EXAMPLES:

All that is required is that an algebraic structure has an addition defined.The default is to represent elements as lowercase ASCII letters.

```
sage: R = IntegerModRing(5)
sage: R.addition_table()                                                #␣
↪needs sage.modules
+  a b c d e
 +----------
a| a b c d e
b| b c d e a
c| c d e a b
d| d e a b c
e| e a b c d
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> R.addition_table()                                                    #␣
↪needs sage.modules
+  a b c d e
 +----------
a| a b c d e
b| b c d e a
c| c d e a b
d| d e a b c
e| e a b c d
```

The `names` argument allows displaying the elements in different ways. Requesting `elements` will use the representation of the elements of the set. Requesting `digits` will include leading zeros as padding.

```
sage: R = IntegerModRing(11)
sage: P = R.addition_table(names='elements'); P                           #␣
↪needs sage.modules
 +   0  1  2  3  4  5  6  7  8  9 10
  +---------------------------------
 0|  0  1  2  3  4  5  6  7  8  9 10
 1|  1  2  3  4  5  6  7  8  9 10  0
 2|  2  3  4  5  6  7  8  9 10  0  1
 3|  3  4  5  6  7  8  9 10  0  1  2
 4|  4  5  6  7  8  9 10  0  1  2  3
 5|  5  6  7  8  9 10  0  1  2  3  4
 6|  6  7  8  9 10  0  1  2  3  4  5
 7|  7  8  9 10  0  1  2  3  4  5  6
 8|  8  9 10  0  1  2  3  4  5  6  7
 9|  9 10  0  1  2  3  4  5  6  7  8
10| 10  0  1  2  3  4  5  6  7  8  9

sage: T = R.addition_table(names='digits'); T                             #␣
↪needs sage.modules
+  00 01 02 03 04 05 06 07 08 09 10
  +---------------------------------
00| 00 01 02 03 04 05 06 07 08 09 10
01| 01 02 03 04 05 06 07 08 09 10 00
02| 02 03 04 05 06 07 08 09 10 00 01
03| 03 04 05 06 07 08 09 10 00 01 02
04| 04 05 06 07 08 09 10 00 01 02 03
05| 05 06 07 08 09 10 00 01 02 03 04
06| 06 07 08 09 10 00 01 02 03 04 05
07| 07 08 09 10 00 01 02 03 04 05 06
08| 08 09 10 00 01 02 03 04 05 06 07
09| 09 10 00 01 02 03 04 05 06 07 08
10| 10 00 01 02 03 04 05 06 07 08 09
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(11))
>>> P = R.addition_table(names='elements'); P                             #␣
↪needs sage.modules
 +   0  1  2  3  4  5  6  7  8  9 10
  +---------------------------------
```

(continues on next page)

(continued from previous page)

```
 0|  0  1  2  3  4  5  6  7  8  9 10
 1|  1  2  3  4  5  6  7  8  9 10  0
 2|  2  3  4  5  6  7  8  9 10  0  1
 3|  3  4  5  6  7  8  9 10  0  1  2
 4|  4  5  6  7  8  9 10  0  1  2  3
 5|  5  6  7  8  9 10  0  1  2  3  4
 6|  6  7  8  9 10  0  1  2  3  4  5
 7|  7  8  9 10  0  1  2  3  4  5  6
 8|  8  9 10  0  1  2  3  4  5  6  7
 9|  9 10  0  1  2  3  4  5  6  7  8
10| 10  0  1  2  3  4  5  6  7  8  9

>>> T = R.addition_table(names='digits'); T                                #␣
↪needs sage.modules
+  00 01 02 03 04 05 06 07 08 09 10
  +--------------------------------
00| 00 01 02 03 04 05 06 07 08 09 10
01| 01 02 03 04 05 06 07 08 09 10 00
02| 02 03 04 05 06 07 08 09 10 00 01
03| 03 04 05 06 07 08 09 10 00 01 02
04| 04 05 06 07 08 09 10 00 01 02 03
05| 05 06 07 08 09 10 00 01 02 03 04
06| 06 07 08 09 10 00 01 02 03 04 05
07| 07 08 09 10 00 01 02 03 04 05 06
08| 08 09 10 00 01 02 03 04 05 06 07
09| 09 10 00 01 02 03 04 05 06 07 08
10| 10 00 01 02 03 04 05 06 07 08 09
```

Specifying the elements in an alternative order can provide more insight into how the operation behaves.

```
sage: S = IntegerModRing(7)
sage: elts = [0, 3, 6, 2, 5, 1, 4]
sage: S.addition_table(elements=elts)                                       #␣
↪needs sage.modules
+  a b c d e f g
 +--------------
a| a b c d e f g
b| b c d e f g a
c| c d e f g a b
d| d e f g a b c
e| e f g a b c d
f| f g a b c d e
g| g a b c d e f
```

```
>>> from sage.all import *
>>> S = IntegerModRing(Integer(7))
>>> elts = [Integer(0), Integer(3), Integer(6), Integer(2), Integer(5),␣
↪Integer(1), Integer(4)]
>>> S.addition_table(elements=elts)                                         #␣
↪needs sage.modules
+  a b c d e f g
 +--------------
```

(continues on next page)

(continued from previous page)

```
a| a b c d e f g
b| b c d e f g a
c| c d e f g a b
d| d e f g a b c
e| e f g a b c d
f| f g a b c d e
g| g a b c d e f
```

The `elements` argument can be used to provide a subset of the elements of the structure. The subset must be closed under the operation. Elements need only be in a form that can be coerced into the set. The `names` argument can also be used to request that the elements be represented with their usual string representation.

```
sage: T = IntegerModRing(12)
sage: elts = [0, 3, 6, 9]
sage: T.addition_table(names='elements', elements=elts)                    #␣
↪needs sage.modules
+  0 3 6 9
 +--------
0| 0 3 6 9
3| 3 6 9 0
6| 6 9 0 3
9| 9 0 3 6
```

```
>>> from sage.all import *
>>> T = IntegerModRing(Integer(12))
>>> elts = [Integer(0), Integer(3), Integer(6), Integer(9)]
>>> T.addition_table(names='elements', elements=elts)                    #␣
↪needs sage.modules
+  0 3 6 9
 +--------
0| 0 3 6 9
3| 3 6 9 0
6| 6 9 0 3
9| 9 0 3 6
```

The table returned can be manipulated in various ways. See the documentation for `OperationTable` for more comprehensive documentation.

```
sage: # needs sage.modules
sage: R = IntegerModRing(3)
sage: T = R.addition_table()
sage: T.column_keys()
(0, 1, 2)
sage: sorted(T.translation().items())
[('a', 0), ('b', 1), ('c', 2)]
sage: T.change_names(['x', 'y', 'z'])
sage: sorted(T.translation().items())
[('x', 0), ('y', 1), ('z', 2)]
sage: T
+  x y z
 +------
```

(continues on next page)

(continued from previous page)

```
x| x y z
y| y z x
z| z x y
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> R = IntegerModRing(Integer(3))
>>> T = R.addition_table()
>>> T.column_keys()
(0, 1, 2)
>>> sorted(T.translation().items())
[('a', 0), ('b', 1), ('c', 2)]
>>> T.change_names(['x', 'y', 'z'])
>>> sorted(T.translation().items())
[('x', 0), ('y', 1), ('z', 2)]
>>> T
+  x y z
 +------
x| x y z
y| y z x
z| z x y
```

summation(*x*, *y*)

Return the sum of `x` and `y`.

The binary addition operator of this additive magma.

INPUT:

- `x`, `y` – elements of this additive magma

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example()
sage: (a,b,c,d) = S.additive_semigroup_generators()
sage: S.summation(a, b)
a + b
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example()
>>> (a,b,c,d) = S.additive_semigroup_generators()
>>> S.summation(a, b)
a + b
```

A parent in `AdditiveMagmas()` must either implement *`summation()`* in the parent class or `_add_` in the element class. By default, the addition method on elements `x._add_(y)` calls `S.summation(x, y)`, and reciprocally.

As a bonus effect, `S.summation` by itself models the binary function from `S` to `S`:

```
sage: bin = S.summation
sage: bin(a,b)
a + b
```

```
>>> from sage.all import *
>>> bin = S.summation
>>> bin(a,b)
a + b
```

Here, `S.summation` is just a bound method. Whenever possible, it is recommended to enrich `S.summation` with extra mathematical structure. Lazy attributes can come handy for this.

Todo

Add an example.

summation_from_element_class_add(x, y)

alias of *summation()*.

class SubcategoryMethods

Bases: `object`

AdditiveAssociative()

Return the full subcategory of the additive associative objects of `self`.

An *additive magma* M is *associative* if, for all $x, y, z \in M$,

$$x + (y + z) = (x + y) + z$$

See also

Wikipedia article Associative_property

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveAssociative()
Category of additive semigroups
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveAssociative()
Category of additive semigroups
```

AdditiveCommutative()

Return the full subcategory of the commutative objects of `self`.

An *additive magma* M is *commutative* if, for all $x, y \in M$,

$$x + y = y + x$$

See also

Wikipedia article Commutative_property

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveCommutative()
Category of additive commutative additive magmas
sage: C = AdditiveMagmas().AdditiveAssociative().AdditiveUnital()
sage: C.AdditiveCommutative()
Category of commutative additive monoids
sage: C.AdditiveCommutative() is CommutativeAdditiveMonoids()
True
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveCommutative()
Category of additive commutative additive magmas
>>> C = AdditiveMagmas().AdditiveAssociative().AdditiveUnital()
>>> C.AdditiveCommutative()
Category of commutative additive monoids
>>> C.AdditiveCommutative() is CommutativeAdditiveMonoids()
True
```

AdditiveUnital()

Return the subcategory of the unital objects of `self`.

An *additive magma* M is *unital* if it admits an element 0, called *neutral element*, such that for all $x \in M$,

$$0 + x = x + 0 = x$$

This element is necessarily unique, and should be provided as `M.zero()`.

See also

Wikipedia article Unital_magma#unital

EXAMPLES:

```
sage: AdditiveMagmas().AdditiveUnital()
Category of additive unital additive magmas
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: AdditiveSemigroups().AdditiveUnital()
Category of additive monoids
sage: CommutativeAdditiveMonoids().AdditiveUnital()
Category of commutative additive monoids
```

```
>>> from sage.all import *
>>> AdditiveMagmas().AdditiveUnital()
Category of additive unital additive magmas
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> AdditiveSemigroups().AdditiveUnital()
Category of additive monoids
>>> CommutativeAdditiveMonoids().AdditiveUnital()
Category of commutative additive monoids
```

super_categories()

EXAMPLES:

```
sage: AdditiveMagmas().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> AdditiveMagmas().super_categories()
[Category of sets]
```

4.4 Additive monoids

class sage.categories.additive_monoids.**AdditiveMonoids**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of additive monoids.

An *additive monoid* is a unital *additive semigroup*, that is a set endowed with a binary operation + which is associative and admits a zero (see Wikipedia article Monoid).

EXAMPLES:

```
sage: from sage.categories.additive_monoids import AdditiveMonoids
sage: C = AdditiveMonoids(); C
Category of additive monoids
sage: C.super_categories()
[Category of additive unital additive magmas, Category of additive semigroups]
sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveUnital']
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: C is AdditiveSemigroups().AdditiveUnital()
True
```

```
>>> from sage.all import *
>>> from sage.categories.additive_monoids import AdditiveMonoids
>>> C = AdditiveMonoids(); C
Category of additive monoids
>>> C.super_categories()
[Category of additive unital additive magmas, Category of additive semigroups]
>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveUnital']
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> C is AdditiveSemigroups().AdditiveUnital()
True
```

AdditiveCommutative

alias of *CommutativeAdditiveMonoids*

AdditiveInverse

alias of *AdditiveGroups*

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

extra_super_categories()

Implement the fact that a homset between two monoids is associative.

EXAMPLES:

```
sage: from sage.categories.additive_monoids import AdditiveMonoids
sage: AdditiveMonoids().Homsets().extra_super_categories()
[Category of additive semigroups]
sage: AdditiveMonoids().Homsets().super_categories()
[Category of homsets of additive unital additive magmas, Category of␣
↪additive monoids]
```

```
>>> from sage.all import *
>>> from sage.categories.additive_monoids import AdditiveMonoids
>>> AdditiveMonoids().Homsets().extra_super_categories()
[Category of additive semigroups]
>>> AdditiveMonoids().Homsets().super_categories()
[Category of homsets of additive unital additive magmas, Category of␣
↪additive monoids]
```

Todo

This could be deduced from *AdditiveSemigroups.Homsets.extra_super_categories()*.
See comment in *Objects.SubcategoryMethods.Homsets()*.

class ParentMethods

Bases: object

sum(*args*)

Return the sum of the elements in `args`, as an element of `self`.

INPUT:

- `args` – list (or iterable) of elements of `self`

EXAMPLES:

```
sage: S = CommutativeAdditiveMonoids().example()
sage: (a,b,c,d) = S.additive_semigroup_generators()
sage: S.sum((a,b,a,c,a,b))
3*a + 2*b + c
sage: S.sum(())
0
sage: S.sum(()).parent() == S
True
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveMonoids().example()
>>> (a,b,c,d) = S.additive_semigroup_generators()
>>> S.sum((a,b,a,c,a,b))
3*a + 2*b + c
>>> S.sum(())
0
>>> S.sum(()).parent() == S
True
```

4.5 Additive semigroups

class sage.categories.additive_semigroups.**AdditiveSemigroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of additive semigroups.

An *additive semigroup* is an associative *additive magma*, that is a set endowed with an operation $+$ which is associative.

EXAMPLES:

```
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: C = AdditiveSemigroups(); C
Category of additive semigroups
sage: C.super_categories()
[Category of additive magmas]
sage: C.all_super_categories()
[Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

sage: C.axioms()
frozenset({'AdditiveAssociative'})
sage: C is AdditiveMagmas().AdditiveAssociative()
True
```

```
>>> from sage.all import *
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> C = AdditiveSemigroups(); C
Category of additive semigroups
>>> C.super_categories()
[Category of additive magmas]
>>> C.all_super_categories()
[Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

>>> C.axioms()
frozenset({'AdditiveAssociative'})
>>> C is AdditiveMagmas().AdditiveAssociative()
True
```

AdditiveCommutative

alias of *CommutativeAdditiveSemigroups*

AdditiveUnital

alias of *AdditiveMonoids*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

algebra_generators()

Return the generators of this algebra, as per *MagmaticAlgebras.ParentMethods.algebra_generators()*.

They correspond to the generators of the additive semigroup.

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: A.algebra_generators()                                            #␣
↪needs sage.modules
Family (B[a], B[b], B[c], B[d])
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> A.algebra_generators()                                              #␣
↪needs sage.modules
Family (B[a], B[b], B[c], B[d])
```

product_on_basis(*g1*, *g2*)

Product, on basis elements, as per *MagmaticAlgebras.WithBasis.ParentMethods.product_on_basis()*.

The product of two basis elements is induced by the addition of the corresponding elements of the group.

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: a, b, c, d = A.algebra_generators()                               #␣
↪needs sage.modules
sage: b * d * c                                                         #␣
↪needs sage.modules
B[b + c + d]
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup:
 the free commutative semigroup generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ)                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
>>> a, b, c, d = A.algebra_generators()                                    #␣
↪needs sage.modules
>>> b * d * c                                                              #␣
↪needs sage.modules
B[b + c + d]
```

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: AdditiveSemigroups().Algebras(QQ).extra_super_categories()
[Category of semigroups]
sage: CommutativeAdditiveSemigroups().Algebras(QQ).super_categories()
[Category of additive semigroup algebras over Rational Field,
 Category of additive commutative additive magma algebras over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> AdditiveSemigroups().Algebras(QQ).extra_super_categories()
[Category of semigroups]
>>> CommutativeAdditiveSemigroups().Algebras(QQ).super_categories()
[Category of additive semigroup algebras over Rational Field,
 Category of additive commutative additive magma algebras over Rational␣
↪Field]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of additive semigroups is an additive semigroup.

EXAMPLES:

```
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: C = AdditiveSemigroups().CartesianProducts()
sage: C.extra_super_categories()
[Category of additive semigroups]
sage: C.axioms()
frozenset({'AdditiveAssociative'})
```

```
>>> from sage.all import *
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> C = AdditiveSemigroups().CartesianProducts()
>>> C.extra_super_categories()
[Category of additive semigroups]
>>> C.axioms()
frozenset({'AdditiveAssociative'})
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

extra_super_categories()

Implement the fact that a homset between two semigroups is a semigroup.

EXAMPLES:

```
sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: AdditiveSemigroups().Homsets().extra_super_categories()
[Category of additive semigroups]
sage: AdditiveSemigroups().Homsets().super_categories()
[Category of homsets of additive magmas, Category of additive semigroups]
```

```
>>> from sage.all import *
>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> AdditiveSemigroups().Homsets().extra_super_categories()
[Category of additive semigroups]
>>> AdditiveSemigroups().Homsets().super_categories()
[Category of homsets of additive magmas, Category of additive semigroups]
```

class ParentMethods

Bases: object

4.6 Affine Weyl groups

class sage.categories.affine_weyl_groups.**AffineWeylGroups**

Bases: *Category_singleton*

The category of affine Weyl groups.

Todo

add a description of this category

See also

- Wikipedia article Affine_weyl_group
- *WeylGroups*, WeylGroup()

EXAMPLES:

```
sage: C = AffineWeylGroups(); C
Category of affine Weyl groups
sage: C.super_categories()
[Category of infinite Weyl groups]

sage: C.example()
NotImplemented
sage: W = WeylGroup(["A", 4, 1]); W                                    #␣
↪needs sage.combinat sage.groups
Weyl Group of type ['A', 4, 1] (as a matrix group acting on the root space)
```

(continues on next page)

(continued from previous page)

```
sage: W.category()                                                          #␣
↪needs sage.combinat sage.groups
Category of irreducible affine Weyl groups
```

```
>>> from sage.all import *
>>> C = AffineWeylGroups(); C
Category of affine Weyl groups
>>> C.super_categories()
[Category of infinite Weyl groups]

>>> C.example()
NotImplemented
>>> W = WeylGroup(["A", Integer(4), Integer(1)]); W                         ␣
↪              # needs sage.combinat sage.groups
Weyl Group of type ['A', 4, 1] (as a matrix group acting on the root space)
>>> W.category()                                                            #␣
↪needs sage.combinat sage.groups
Category of irreducible affine Weyl groups
```

class ElementMethods

Bases: object

affine_grassmannian_to_core()

Bijection between affine Grassmannian elements of type $A_k^{(1)}$ and $(k+1)$-cores.

INPUT:

- self – an affine Grassmannian element of some affine Weyl group of type $A_k^{(1)}$

Recall that an element w of an affine Weyl group is affine Grassmannian if all its all reduced words end in 0, see *is_affine_grassmannian()*.

OUTPUT:

- a $(k+1)$-core

See also

affine_grassmannian_to_partition()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 2, 1])
sage: w = W.from_reduced_word([0,2,1,0])
sage: la = w.affine_grassmannian_to_core(); la
[4, 2]
sage: type(la)
<class 'sage.combinat.core.Cores_length_with_category.element_class'>
sage: la.to_grassmannian() == w
True

sage: w = W.from_reduced_word([0,2,1])                                      #␣
↪needs sage.combinat sage.groups
sage: w.affine_grassmannian_to_core()                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.groups
Traceback (most recent call last):
...
ValueError: this only works on type 'A' affine Grassmannian elements
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(2), Integer(1)])
>>> w = W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)])
>>> la = w.affine_grassmannian_to_core(); la
[4, 2]
>>> type(la)
<class 'sage.combinat.core.Cores_length_with_category.element_class'>
>>> la.to_grassmannian() == w
True

>>> w = W.from_reduced_word([Integer(0),Integer(2),Integer(1)])          ␣
↪                          # needs sage.combinat sage.groups
>>> w.affine_grassmannian_to_core()                                      #␣
↪needs sage.combinat sage.groups
Traceback (most recent call last):
...
ValueError: this only works on type 'A' affine Grassmannian elements
```

affine_grassmannian_to_partition()

Bijection between affine Grassmannian elements of type $A_k^{(1)}$ and k-bounded partitions.

INPUT:

- `self` is affine Grassmannian element of the affine Weyl group of type $A_k^{(1)}$ (i.e. all reduced words end in 0)

OUTPUT: k-bounded partition

See also

affine_grassmannian_to_core()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: k = 2
sage: W = WeylGroup(['A', k, 1])
sage: w = W.from_reduced_word([0,2,1,0])
sage: la = w.affine_grassmannian_to_partition(); la
[2, 2]
sage: la.from_kbounded_to_grassmannian(k) == w
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> k = Integer(2)
>>> W = WeylGroup(['A', k, Integer(1)])
```

(continues on next page)

(continued from previous page)

```
>>> w = W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)])
>>> la = w.affine_grassmannian_to_partition(); la
[2, 2]
>>> la.from_kbounded_to_grassmannian(k) == w
True
```

is_affine_grassmannian()

Test whether `self` is affine Grassmannian.

An element of an affine Weyl group is *affine Grassmannian* if any of the following equivalent properties holds:

- all reduced words for `self` end with 0.
- `self` is the identity, or 0 is its single right descent.
- `self` is a minimal coset representative for W / cl W.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3, 1])
sage: w = W.from_reduced_word([2,1,0])
sage: w.is_affine_grassmannian()
True
sage: w = W.from_reduced_word([2,0])
sage: w.is_affine_grassmannian()
False
sage: W.one().is_affine_grassmannian()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3), Integer(1)])
>>> w = W.from_reduced_word([Integer(2),Integer(1),Integer(0)])
>>> w.is_affine_grassmannian()
True
>>> w = W.from_reduced_word([Integer(2),Integer(0)])
>>> w.is_affine_grassmannian()
False
>>> W.one().is_affine_grassmannian()
True
```

class ParentMethods

Bases: `object`

affine_grassmannian_elements_of_given_length(k)

Return the affine Grassmannian elements of length k.

This is returned as a finite enumerated set.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3, 1])                                    #␣
↪needs sage.combinat sage.groups
sage: [x.reduced_word()                                             #␣
↪needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
....:  for x in W.affine_grassmannian_elements_of_given_length(3)]
[[2, 1, 0], [3, 1, 0], [2, 3, 0]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3), Integer(1)])
↪              # needs sage.combinat sage.groups
>>> [x.reduced_word()
↪needs sage.combinat sage.groups
...  for x in W.affine_grassmannian_elements_of_given_length(Integer(3))]
[[2, 1, 0], [3, 1, 0], [2, 3, 0]]
```

See also

AffineWeylGroups.ElementMethods.is_affine_grassmannian()

special_node()

Return the distinguished special node of the underlying Dynkin diagram.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3, 1])
↪needs sage.combinat sage.groups
sage: W.special_node()
↪needs sage.combinat sage.groups
0
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3), Integer(1)])
↪              # needs sage.combinat sage.groups
>>> W.special_node()
↪needs sage.combinat sage.groups
0
```

additional_structure()

Return `None`.

Indeed, the category of affine Weyl groups defines no additional structure: affine Weyl groups are a special class of Weyl groups.

See also

Category.additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: AffineWeylGroups().additional_structure()
```

```
>>> from sage.all import *
>>> AffineWeylGroups().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: AffineWeylGroups().super_categories()
[Category of infinite Weyl groups]
```

```
>>> from sage.all import *
>>> AffineWeylGroups().super_categories()
[Category of infinite Weyl groups]
```

4.7 Algebra ideals

class sage.categories.algebra_ideals.**AlgebraIdeals**(*A*)

Bases: *Category_ideal*

The category of two-sided ideals in a fixed algebra A.

EXAMPLES:

```
sage: AlgebraIdeals(QQ['a'])
Category of algebra ideals in Univariate Polynomial Ring in a over Rational Field
```

```
>>> from sage.all import *
>>> AlgebraIdeals(QQ['a'])
Category of algebra ideals in Univariate Polynomial Ring in a over Rational Field
```

Todo

- Add support for non commutative rings (this is currently not supported by the subcategory *AlgebraModules*).
- Make `AlgebraIdeals(R)`, return `CommutativeAlgebraIdeals(R)` when `R` is commutative.
- If useful, implement `AlgebraLeftIdeals` and `AlgebraRightIdeals` of which `AlgebraIdeals` would be a subcategory.

algebra()

EXAMPLES:

```
sage: AlgebraIdeals(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> AlgebraIdeals(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

super_categories()

The category of algebra modules should be a super category of this category.

However, since algebra modules are currently only available over commutative rings, we have to omit it if our ring is non-commutative.

EXAMPLES:

```
sage: AlgebraIdeals(QQ['x']).super_categories()
[Category of algebra modules
  over Univariate Polynomial Ring in x over Rational Field]
sage: C = AlgebraIdeals(FreeAlgebra(QQ, 2, 'a,b'))                          #␣
↪needs sage.combinat sage.modules
sage: C.super_categories()                                                  #␣
↪needs sage.combinat sage.modules
[]
```

```
>>> from sage.all import *
>>> AlgebraIdeals(QQ['x']).super_categories()
[Category of algebra modules
  over Univariate Polynomial Ring in x over Rational Field]
>>> C = AlgebraIdeals(FreeAlgebra(QQ, Integer(2), 'a,b'))                   ␣
↪     # needs sage.combinat sage.modules
>>> C.super_categories()                                                   #␣
↪needs sage.combinat sage.modules
[]
```

4.8 Algebra modules

class sage.categories.algebra_modules.**AlgebraModules**(A)

Bases: *Category_module*

The category of modules over a fixed algebra A.

EXAMPLES:

```
sage: AlgebraModules(QQ['a'])
Category of algebra modules over Univariate Polynomial Ring in a over Rational␣
↪Field
sage: AlgebraModules(QQ['a']).super_categories()
[Category of modules over Univariate Polynomial Ring in a over Rational Field]
```

```
>>> from sage.all import *
>>> AlgebraModules(QQ['a'])
Category of algebra modules over Univariate Polynomial Ring in a over Rational␣
↪Field
>>> AlgebraModules(QQ['a']).super_categories()
[Category of modules over Univariate Polynomial Ring in a over Rational Field]
```

Note: as of now, A is required to be commutative, ensuring that the categories of left and right modules are isomorphic. Feedback and use cases for potential generalizations to the non commutative case are welcome.

algebra()

EXAMPLES:

```
sage: AlgebraModules(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> AlgebraModules(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: AlgebraModules.an_instance()
Category of algebra modules over Univariate Polynomial Ring in x over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> AlgebraModules.an_instance()
Category of algebra modules over Univariate Polynomial Ring in x over␣
↪Rational Field
```

super_categories()

EXAMPLES:

```
sage: AlgebraModules(QQ['x']).super_categories()
[Category of modules over Univariate Polynomial Ring in x over Rational Field]
```

```
>>> from sage.all import *
>>> AlgebraModules(QQ['x']).super_categories()
[Category of modules over Univariate Polynomial Ring in x over Rational Field]
```

4.9 Algebras

AUTHORS:

- David Kohel & William Stein (2005): initial revision
- Nicolas M. Thiery (2008-2011): rewrote for the category framework

class sage.categories.algebras.**Algebras**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of associative and unital algebras over a given base ring.

An associative and unital algebra over a ring R is a module over R which is itself a ring.

Warning

Algebras will be eventually be replaced by *magmatic_algebras.MagmaticAlgebras* for consistency with e.g. Wikipedia article Algebras which assumes neither associativity nor the existence of a unit (see Issue #15043).

Todo

Should R be a commutative ring?

EXAMPLES:

```
sage: Algebras(ZZ)
Category of algebras over Integer Ring
sage: sorted(Algebras(ZZ).super_categories(), key=str)
[Category of associative algebras over Integer Ring,
 Category of rings,
 Category of unital algebras over Integer Ring]
```

```
>>> from sage.all import *
>>> Algebras(ZZ)
Category of algebras over Integer Ring
>>> sorted(Algebras(ZZ).super_categories(), key=str)
[Category of associative algebras over Integer Ring,
 Category of rings,
 Category of unital algebras over Integer Ring]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of algebras constructed as Cartesian products of algebras.

This construction gives the direct product of algebras. See discussion on:

- http://groups.google.fr/group/sage-devel/browse_thread/thread/35a72b1d0a2fc77a/348f42ae77a66d16#348f42ae77a66d16
- Wikipedia article Direct_product

extra_super_categories()

A Cartesian product of algebras is endowed with a natural algebra structure.

EXAMPLES:

```
sage: C = Algebras(QQ).CartesianProducts()
sage: C.extra_super_categories()
[Category of algebras over Rational Field]
sage: sorted(C.super_categories(), key=str)
[Category of Cartesian products of monoids,
 Category of Cartesian products of unital algebras over Rational Field,
 Category of algebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).CartesianProducts()
>>> C.extra_super_categories()
[Category of algebras over Rational Field]
>>> sorted(C.super_categories(), key=str)
[Category of Cartesian products of monoids,
 Category of Cartesian products of unital algebras over Rational Field,
 Category of algebras over Rational Field]
```

Commutative

alias of *CommutativeAlgebras*

class DualObjects(*category*, **args*)

Bases: *DualObjectsCategory*

extra_super_categories()

Return the dual category.

EXAMPLES:

The category of algebras over the Rational Field is dual to the category of coalgebras over the same field:

```
sage: C = Algebras(QQ)
sage: C.dual()
Category of duals of algebras over Rational Field
sage: C.dual().extra_super_categories()
[Category of coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = Algebras(QQ)
>>> C.dual()
Category of duals of algebras over Rational Field
>>> C.dual().extra_super_categories()
[Category of coalgebras over Rational Field]
```

Warning

This is only correct in certain cases (finite dimension, …). See Issue #15647.

class ElementMethods

Bases: object

Filtered

alias of *FilteredAlgebras*

Graded

alias of *GradedAlgebras*

class ParentMethods

Bases: object

characteristic()

Return the characteristic of this algebra, which is the same as the characteristic of its base ring.

EXAMPLES:

```
sage: # needs sage.modules
sage: ZZ.characteristic()
0
sage: A = GF(7^3, 'a')
↪  # needs sage.rings.finite_rings
sage: A.characteristic()
↪  # needs sage.rings.finite_rings
7
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> ZZ.characteristic()
0
>>> A = GF(Integer(7)**Integer(3), 'a')                                    ␣
↪                   # needs sage.rings.finite_rings
>>> A.characteristic()
↪# needs sage.rings.finite_rings
7
```

has_standard_involution()

Return `True` if the algebra has a standard involution and `False` otherwise.

This algorithm follows Algorithm 2.10 from John Voight's *Identifying the Matrix Ring*. Currently the only type of algebra this will work for is a quaternion algebra. Though this function seems redundant, once algebras have more functionality, in particular have a method to construct a basis, this algorithm will have more general purpose.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: B = QuaternionAlgebra(2)
sage: B.has_standard_involution()
True
sage: R.<x> = PolynomialRing(QQ)
sage: K.<u> = NumberField(x**2 - 2)                                       ␣
↪  # needs sage.rings.number_field
sage: A = QuaternionAlgebra(K, -2, 5)                                     ␣
↪  # needs sage.rings.number_field
sage: A.has_standard_involution()                                         ␣
↪  # needs sage.rings.number_field
True
sage: L.<a,b> = FreeAlgebra(QQ, 2)
sage: L.has_standard_involution()
Traceback (most recent call last):
...
NotImplementedError: has_standard_involution is not implemented for this␣
↪algebra
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> B = QuaternionAlgebra(Integer(2))
>>> B.has_standard_involution()
True
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> K = NumberField(x**Integer(2) - Integer(2), names=('u',)); (u,) = K._
↪first_ngens(1)# needs sage.rings.number_field
>>> A = QuaternionAlgebra(K, -Integer(2), Integer(5))                     ␣
↪                   # needs sage.rings.number_field
>>> A.has_standard_involution()
↪# needs sage.rings.number_field
True
>>> L = FreeAlgebra(QQ, Integer(2), names=('a', 'b',)); (a, b,) = L._
```

(continues on next page)

(continued from previous page)

```
↪first_ngens(2)
>>> L.has_standard_involution()
Traceback (most recent call last):
...
NotImplementedError: has_standard_involution is not implemented for this␣
↪algebra
```

class Quotients(*category*, **args*)

Bases: *QuotientsCategory*

class ParentMethods

Bases: object

algebra_generators()

Return algebra generators for self.

This implementation retracts the algebra generators from the ambient algebra.

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: S = A.semisimple_quotient()
sage: S.algebra_generators()
Finite family {'x': B['x'], 'y': B['y'], 'a': 0, 'b': 0}
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> S = A.semisimple_quotient()
>>> S.algebra_generators()
Finite family {'x': B['x'], 'y': B['y'], 'a': 0, 'b': 0}
```

Todo

this could possibly remove the elements that retract to zero.

Semisimple

alias of *SemisimpleAlgebras*

class SubcategoryMethods

Bases: object

Semisimple()

Return the subcategory of semisimple objects of self.

> **Note**
>
> This mimics the syntax of axioms for a smooth transition if `Semisimple` becomes one.

EXAMPLES:

```
sage: Algebras(QQ).Semisimple()
Category of semisimple algebras over Rational Field
sage: Algebras(QQ).WithBasis().FiniteDimensional().Semisimple()
Category of finite dimensional semisimple algebras with basis over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> Algebras(QQ).Semisimple()
Category of semisimple algebras over Rational Field
>>> Algebras(QQ).WithBasis().FiniteDimensional().Semisimple()
Category of finite dimensional semisimple algebras with basis over␣
↪Rational Field
```

Supercommutative()

Return the full subcategory of the supercommutative objects of `self`.

This is shorthand for creating the corresponding super category.

EXAMPLES:

```
sage: Algebras(ZZ).Supercommutative()
Category of supercommutative algebras over Integer Ring
sage: Algebras(ZZ).WithBasis().Supercommutative()
Category of supercommutative super algebras with basis over Integer Ring

sage: Cat = Algebras(ZZ).Supercommutative()
sage: Cat is Algebras(ZZ).Super().Supercommutative()
True
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Supercommutative()
Category of supercommutative algebras over Integer Ring
>>> Algebras(ZZ).WithBasis().Supercommutative()
Category of supercommutative super algebras with basis over Integer Ring

>>> Cat = Algebras(ZZ).Supercommutative()
>>> Cat is Algebras(ZZ).Super().Supercommutative()
True
```

Super

alias of *SuperAlgebras*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

class ElementMethods

Bases: `object`

class ParentMethods

Bases: object

extra_super_categories()

EXAMPLES:

```
sage: Algebras(QQ).TensorProducts().extra_super_categories()
[Category of algebras over Rational Field]
sage: Algebras(QQ).TensorProducts().super_categories()
[Category of algebras over Rational Field,
 Category of tensor products of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).TensorProducts().extra_super_categories()
[Category of algebras over Rational Field]
>>> Algebras(QQ).TensorProducts().super_categories()
[Category of algebras over Rational Field,
 Category of tensor products of vector spaces over Rational Field]
```

Meaning: a tensor product of algebras is an algebra

WithBasis

alias of *AlgebrasWithBasis*

4.10 Algebras With Basis

class sage.categories.algebras_with_basis.**AlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of algebras with a distinguished basis.

EXAMPLES:

```
sage: C = AlgebrasWithBasis(QQ); C
Category of algebras with basis over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of unital algebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(QQ); C
Category of algebras with basis over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of unital algebras with basis over Rational Field]
```

We construct a typical parent in this category, and do some computations with it:

```
sage: # needs sage.combinat sage.modules
sage: A = C.example(); A
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
sage: A.category()
```

(continues on next page)

(continued from previous page)

```
Category of algebras with basis over Rational Field
sage: A.one_basis()
word:
sage: A.one()
B[word: ]
sage: A.base_ring()
Rational Field
sage: A.basis().keys()
Finite words over {'a', 'b', 'c'}
sage: (a,b,c) = A.algebra_generators()
sage: a^3, b^2
(B[word: aaa], B[word: bb])
sage: a * c * b
B[word: acb]
sage: A.product
<bound method MagmaticAlgebras.WithBasis.ParentMethods._product_from_product_on_
↪basis_multiply of
 An example of an algebra with basis:
  the free algebra on the generators ('a', 'b', 'c') over Rational Field>
sage: A.product(a * b, b)
B[word: abb]
sage: TestSuite(A).run(verbose=True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_characteristic() . . . pass
running ._test_construction() . . . pass
running ._test_distributivity() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_monomial_coefficients() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
```

(continues on next page)

(continued from previous page)

```
sage: A.__class__
<class 'sage.categories.examples.algebras_with_basis.FreeAlgebra_with_category'>
sage: A.element_class
<class 'sage.categories.examples.algebras_with_basis.FreeAlgebra_with_category.
↪element_class'>
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = C.example(); A
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
>>> A.category()
Category of algebras with basis over Rational Field
>>> A.one_basis()
word:
>>> A.one()
B[word: ]
>>> A.base_ring()
Rational Field
>>> A.basis().keys()
Finite words over {'a', 'b', 'c'}
>>> (a,b,c) = A.algebra_generators()
>>> a**Integer(3), b**Integer(2)
(B[word: aaa], B[word: bb])
>>> a * c * b
B[word: acb]
>>> A.product
<bound method MagmaticAlgebras.WithBasis.ParentMethods._product_from_product_on_
↪basis_multiply of
 An example of an algebra with basis:
  the free algebra on the generators ('a', 'b', 'c') over Rational Field>
>>> A.product(a * b, b)
B[word: abb]
>>> TestSuite(A).run(verbose=True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_characteristic() . . . pass
running ._test_construction() . . . pass
running ._test_distributivity() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_monomial_coefficients() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
```

(continues on next page)

(continued from previous page)

```
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
>>> A.__class__
<class 'sage.categories.examples.algebras_with_basis.FreeAlgebra_with_category'>
>>> A.element_class
<class 'sage.categories.examples.algebras_with_basis.FreeAlgebra_with_category.
↪element_class'>
```

Please see the source code of A (with `A??`) for how to implement other algebras with basis.

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of algebras with basis, constructed as Cartesian products of algebras with basis.

Note: this construction give the direct products of algebras with basis. See comment in *Algebras.CartesianProducts*

class ParentMethods

Bases: object

one()

one_from_cartesian_product_of_one_basis()

Return the one of this Cartesian product of algebras, as per `Monoids.ParentMethods.one`

It is constructed as the Cartesian product of the ones of the summands, using their *one_basis()* methods.

This implementation does not require multiplication by scalars nor calling cartesian_product. This might help keeping things as lazy as possible upon initialization.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
sage: A.one_basis()
word:
sage: B = cartesian_product((A, A, A))
sage: B.one_from_cartesian_product_of_one_basis()
B[(0, word: )] + B[(1, word: )] + B[(2, word: )]
sage: B.one()
B[(0, word: )] + B[(1, word: )] + B[(2, word: )]
```

(continues on next page)

(continued from previous page)

```
sage: cartesian_product([SymmetricGroupAlgebra(QQ, 3),                #␣
↪needs sage.combinat sage.groups sage.modules
....:                       SymmetricGroupAlgebra(QQ, 4)]).one()
B[(0, [1, 2, 3])] + B[(1, [1, 2, 3, 4])]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
>>> A.one_basis()
word:
>>> B = cartesian_product((A, A, A))
>>> B.one_from_cartesian_product_of_one_basis()
B[(0, word: )] + B[(1, word: )] + B[(2, word: )]
>>> B.one()
B[(0, word: )] + B[(1, word: )] + B[(2, word: )]

>>> cartesian_product([SymmetricGroupAlgebra(QQ, Integer(3)),          ␣
↪    # needs sage.combinat sage.groups sage.modules
...                       SymmetricGroupAlgebra(QQ, Integer(4))]).one()
B[(0, [1, 2, 3])] + B[(1, [1, 2, 3, 4])]
```

extra_super_categories()

A Cartesian product of algebras with basis is endowed with a natural algebra with basis structure.

EXAMPLES:

```
sage: AlgebrasWithBasis(QQ).CartesianProducts().extra_super_categories()
[Category of algebras with basis over Rational Field]
sage: AlgebrasWithBasis(QQ).CartesianProducts().super_categories()
[Category of algebras with basis over Rational Field,
 Category of Cartesian products of algebras over Rational Field,
 Category of Cartesian products of vector spaces with basis over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).CartesianProducts().extra_super_categories()
[Category of algebras with basis over Rational Field]
>>> AlgebrasWithBasis(QQ).CartesianProducts().super_categories()
[Category of algebras with basis over Rational Field,
 Category of Cartesian products of algebras over Rational Field,
 Category of Cartesian products of vector spaces with basis over Rational␣
↪Field]
```

class ElementMethods

Bases: object

Filtered

alias of *FilteredAlgebrasWithBasis*

FiniteDimensional

alias of *FiniteDimensionalAlgebrasWithBasis*

Graded

alias of *GradedAlgebrasWithBasis*

class ParentMethods

Bases: object

hochschild_complex(*M*)

Return the Hochschild complex of `self` with coefficients in `M`.

See also

HochschildComplex

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: A = algebras.DifferentialWeyl(R)                                  #␣
↪needs sage.modules
sage: H = A.hochschild_complex(A)                                       #␣
↪needs sage.modules

sage: # needs sage.combinat sage.groups sage.modules
sage: SGA = SymmetricGroupAlgebra(QQ, 3)
sage: T = SGA.trivial_representation()
sage: H = SGA.hochschild_complex(T)
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> A = algebras.DifferentialWeyl(R)                                    #␣
↪needs sage.modules
>>> H = A.hochschild_complex(A)                                         #␣
↪needs sage.modules

>>> # needs sage.combinat sage.groups sage.modules
>>> SGA = SymmetricGroupAlgebra(QQ, Integer(3))
>>> T = SGA.trivial_representation()
>>> H = SGA.hochschild_complex(T)
```

one()

Return the multiplicative unit element.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                               #␣
↪needs sage.combinat sage.modules
sage: A.one_basis()                                                     #␣
↪needs sage.combinat sage.modules
word:
sage: A.one()                                                           #␣
↪needs sage.combinat sage.modules
B[word: ]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                    # needs␣
↪sage.combinat sage.modules
>>> A.one_basis()                                                          # needs␣
↪sage.combinat sage.modules
word:
>>> A.one()                                                                # needs␣
↪sage.combinat sage.modules
B[word: ]
```

Super

alias of *SuperAlgebrasWithBasis*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of algebras with basis constructed by tensor product of algebras with basis

class ElementMethods

Bases: object

Implement operations on elements of tensor products of algebras with basis

class ParentMethods

Bases: object

implements operations on tensor products of algebras with basis

one_basis()

Return the index of the one of this tensor product of algebras, as per AlgebrasWithBasis.ParentMethods.one_basis

It is the tuple whose operands are the indices of the ones of the operands, as returned by their *one_basis()* methods.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
sage: A.one_basis()
word:
sage: B = tensor((A, A, A))
sage: B.one_basis()
(word: , word: , word: )
sage: B.one()
B[word: ] # B[word: ] # B[word: ]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
>>> A.one_basis()
word:
```

(continues on next page)

(continued from previous page)

```
>>> B = tensor((A, A, A))
>>> B.one_basis()
(word: , word: , word: )
>>> B.one()
B[word: ] # B[word: ] # B[word: ]
```

product_on_basis(*t1*, *t2*)

The product of the algebra on the basis, as per `AlgebrasWithBasis.ParentMethods.product_on_basis`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
sage: (a,b,c) = A.algebra_generators()
sage: x = tensor((a, b, c)); x
B[word: a] # B[word: b] # B[word: c]
sage: y = tensor((c, b, a)); y
B[word: c] # B[word: b] # B[word: a]
sage: x * y
B[word: ac] # B[word: bb] # B[word: ca]
sage: x = tensor(((a + 2*b), c)); x
B[word: a] # B[word: c] + 2*B[word: b] # B[word: c]
sage: y = tensor((c, a)) + 1; y
B[word: ] # B[word: ] + B[word: c] # B[word: a]
sage: x * y
B[word: a] # B[word: c] + B[word: ac] # B[word: ca]
 + 2*B[word: b] # B[word: c] + 2*B[word: bc] # B[word: ca]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example(); A
An example of an algebra with basis: the free algebra
 on the generators ('a', 'b', 'c') over Rational Field
>>> (a,b,c) = A.algebra_generators()
>>> x = tensor((a, b, c)); x
B[word: a] # B[word: b] # B[word: c]
>>> y = tensor((c, b, a)); y
B[word: c] # B[word: b] # B[word: a]
>>> x * y
B[word: ac] # B[word: bb] # B[word: ca]
>>> x = tensor(((a + Integer(2)*b), c)); x
B[word: a] # B[word: c] + 2*B[word: b] # B[word: c]
>>> y = tensor((c, a)) + Integer(1); y
B[word: ] # B[word: ] + B[word: c] # B[word: a]
>>> x * y
B[word: a] # B[word: c] + B[word: ac] # B[word: ca]
 + 2*B[word: b] # B[word: c] + 2*B[word: bc] # B[word: ca]
```

TODO: optimize this implementation!

extra_super_categories()

EXAMPLES:

```
sage: AlgebrasWithBasis(QQ).TensorProducts().extra_super_categories()
[Category of algebras with basis over Rational Field]
sage: AlgebrasWithBasis(QQ).TensorProducts().super_categories()
[Category of algebras with basis over Rational Field,
 Category of tensor products of algebras over Rational Field,
 Category of tensor products of vector spaces with basis over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).TensorProducts().extra_super_categories()
[Category of algebras with basis over Rational Field]
>>> AlgebrasWithBasis(QQ).TensorProducts().super_categories()
[Category of algebras with basis over Rational Field,
 Category of tensor products of algebras over Rational Field,
 Category of tensor products of vector spaces with basis over Rational␣
↪Field]
```

example(*alphabet=('a', 'b', 'c')*)

Return an example of algebra with basis.

EXAMPLES:

```
sage: AlgebrasWithBasis(QQ).example()                                          #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).example()                                            #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
```

An other set of generators can be specified as optional argument:

```
sage: AlgebrasWithBasis(QQ).example((1,2,3))                                   #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators (1, 2, 3) over Rational Field
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).example((Integer(1),Integer(2),Integer(3)))          ␣
↪                        # needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators (1, 2, 3) over Rational Field
```

4.11 Anderson motives

AUTHOR:

- Xavier Caruso, Antoine Leudière (2025-11): initial version

class sage.categories.anderson_motives.**AndersonMotives**(*category*)

Bases: *OreModules*

The category of Anderson motives.

> **See also**
>
> *sage.categories.drinfeld_modules.DrinfeldModules*, sage.rings.function_field.drinfeld_modules.anderson_motive

A_field()

Return the underlying A-field of this category.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.A_field()
Finite Field in z of size 3^3 over its base
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.A_field()
Finite Field in z of size 3^3 over its base
```

Endsets()

Return the category of endsets.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())

sage: from sage.categories.homsets import Homsets
sage: C.Endsets() is Homsets().Endsets()
True
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())

>>> from sage.categories.homsets import Homsets
>>> C.Endsets() is Homsets().Endsets()
True
```

Homsets()

Return the category of homsets.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())

sage: from sage.categories.homsets import Homsets
sage: C.Homsets() is Homsets()
True
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())

>>> from sage.categories.homsets import Homsets
>>> C.Homsets() is Homsets()
True
```

class ParentMethods

Bases: object

A_field()

Return the underlying A-field of this Anderson motive.

This is an instance of the class sage.rings.ring_extension.RingExtension_generic.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: M.A_field()
Finite Field in z of size 5^12 over its base
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
>>> M.A_field()
Finite Field in z of size 5^12 over its base
```

base()

Return the base ring over which this Anderson motive is defined.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: M.base()
Univariate Polynomial Ring in T over Finite Field in z of size 5^12
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
>>> M.base()
Univariate Polynomial Ring in T over Finite Field in z of size 5^12
```

base_ring()

alias of *base()*.

characteristic()

Return the characteristic of the underlying A-field.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: M.characteristic()
T^6 + (4*z2 + 3)*T^5 + T^4 + (3*z2 + 1)*T^3 + T^2 + (4*z2 + 1)*T + z2
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
```

(continues on next page)

(continued from previous page)

```
>>> M.characteristic()
T^6 + (4*z2 + 3)*T^5 + T^4 + (3*z2 + 1)*T^3 + T^2 + (4*z2 + 1)*T + z2
```

function_ring()

Return the underlying function ring of this Anderson motive.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: M.function_ring()
Univariate Polynomial Ring in T over Finite Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
>>> M.function_ring()
Univariate Polynomial Ring in T over Finite Field in z2 of size 5^2
```

ore_polring()

Return the Ore polynomial ring over which this Anderson motive is defined.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: M.ore_polring()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base␣
↪ring
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
>>> M.ore_polring()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base␣
↪ring
```

ore_variable()

Return the generator of the Ore polynomial ring over which this Anderson motive is defined.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: M = phi.anderson_motive()
sage: tau = M.ore_variable()
sage: tau
τ
sage: tau.parent()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base␣
↪ring
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> M = phi.anderson_motive()
>>> tau = M.ore_variable()
>>> tau
τ
>>> tau.parent()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base␣
↪ring
```

base()

Return the base over which the Anderson motives in this category are defined.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.base()
Univariate Polynomial Ring in T over Finite Field in z of size 3^3
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.base()
Univariate Polynomial Ring in T over Finite Field in z of size 3^3
```

characteristic()

Return the characteristic of the underlying A-field.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.divisor()
T + 2*z
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.divisor()
T + 2*z
```

divisor()

Return the polynomial $T - z$ if T denotes the generator of the function ring A and z is the image of T in the A-field.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.divisor()
T + 2*z
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.divisor()
T + 2*z
```

function_ring()

Return the underlying function ring of this category.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 3
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 3
```

object (*tau=None*, *names=None*)

Return the object in this category with τ-action given by the matrix `tau`.

INPUT:

- `tau` – a matrix or `None` (default: `None`); if `None`, return the trivial Anderson module in this category
- `names` – a matrix or `None` (default: `None`); if `None`, return the trivial Anderson module in this category

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())

sage: M = C.object()
sage: M
Anderson motive of rank 1 over Univariate Polynomial Ring in T over Finite␣
↪Field in z of size 3^3
sage: M.matrix()
[1]
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())

>>> M = C.object()
>>> M
Anderson motive of rank 1 over Univariate Polynomial Ring in T over Finite␣
↪Field in z of size 3^3
>>> M.matrix()
[1]
```

```
sage: tau = matrix(2, 2, [[T, 1], [z, 1]])
sage: N = C.object(tau)
sage: N.matrix()
[T 1]
[z 1]
```

```
>>> from sage.all import *
>>> tau = matrix(Integer(2), Integer(2), [[T, Integer(1)], [z, Integer(1)]])
>>> N = C.object(tau)
>>> N.matrix()
[T 1]
[z 1]
```

ore_polring()

Return the Ore polynomial ring over which the Anderson motives in this category are defined.

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: C = AndersonMotives(phi.category())
sage: C.ore_polring()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base ring
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> C = AndersonMotives(phi.category())
>>> C.ore_polring()
Ore Polynomial Ring in τ over Univariate Polynomial Ring in T
over Finite Field in z of size 5^12 twisted by T |--> T, with map of base ring
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.anderson_motives import AndersonMotives
sage: A.<T> = GF(3)[]
sage: K.<z> = GF(3^3)
sage: phi = DrinfeldModule(A, [z, z^2, z^3])
sage: C = AndersonMotives(phi.category())
sage: C.super_categories()
[Category of Ore modules over Univariate Polynomial Ring in T over Finite␣
↪Field in z of size 3^3 twisted by T |--> T, with map of base ring]
```

```
>>> from sage.all import *
>>> from sage.categories.anderson_motives import AndersonMotives
>>> A = GF(Integer(3))['T']; (T,) = A._first_ngens(1)
>>> K = GF(Integer(3)**Integer(3), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(2), z**Integer(3)])
>>> C = AndersonMotives(phi.category())
>>> C.super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of Ore modules over Univariate Polynomial Ring in T over Finite␣
↪Field in z of size 3^3 twisted by T |--> T, with map of base ring]
```

4.12 Aperiodic semigroups

class sage.categories.aperiodic_semigroups.**AperiodicSemigroups**(*base_category*)

Bases: *CategoryWithAxiom*

extra_super_categories()

Implement the fact that an aperiodic semigroup is H-trivial.

EXAMPLES:

```
sage: Semigroups().Aperiodic().extra_super_categories()
[Category of h trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().Aperiodic().extra_super_categories()
[Category of h trivial semigroups]
```

4.13 Associative algebras

class sage.categories.associative_algebras.**AssociativeAlgebras**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of associative algebras over a given base ring.

An associative algebra over a ring R is a module over R which is also a not necessarily unital ring.

Warning

Until Issue #15043 is implemented, *Algebras* is the category of associative unital algebras; thus, unlike the name suggests, *AssociativeAlgebras* is not a subcategory of *Algebras* but of *MagmaticAlgebras*.

EXAMPLES:

```
sage: from sage.categories.associative_algebras import AssociativeAlgebras
sage: C = AssociativeAlgebras(ZZ); C
Category of associative algebras over Integer Ring
```

```
>>> from sage.all import *
>>> from sage.categories.associative_algebras import AssociativeAlgebras
>>> C = AssociativeAlgebras(ZZ); C
Category of associative algebras over Integer Ring
```

Unital

alias of *Algebras*

4.14 Bialgebras

class sage.categories.bialgebras.**Bialgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of bialgebras.

EXAMPLES:

```
sage: Bialgebras(ZZ)
Category of bialgebras over Integer Ring
sage: Bialgebras(ZZ).super_categories()
[Category of algebras over Integer Ring, Category of coalgebras over Integer Ring]
```

```
>>> from sage.all import *
>>> Bialgebras(ZZ)
Category of bialgebras over Integer Ring
>>> Bialgebras(ZZ).super_categories()
[Category of algebras over Integer Ring, Category of coalgebras over Integer Ring]
```

class ElementMethods

Bases: object

is_grouplike()

Return whether self is a grouplike element.

EXAMPLES:

```
sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.modules
sage: s([5]).is_grouplike()                                                 #␣
↪needs lrcalc_python sage.modules
False
sage: s([]).is_grouplike()                                                  #␣
↪needs lrcalc_python sage.modules
True
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.modules
>>> s([Integer(5)]).is_grouplike()                                          ␣
↪     # needs lrcalc_python sage.modules
False
>>> s([]).is_grouplike()                                                  #␣
↪needs lrcalc_python sage.modules
True
```

is_primitive()

Return whether self is a primitive element.

EXAMPLES:

```
sage: # needs sage.modules
sage: s = SymmetricFunctions(QQ).schur()
```

(continues on next page)

(continued from previous page)

```
sage: s([5]).is_primitive()                                                 #␣
↪needs lrcalc_python
False
sage: p = SymmetricFunctions(QQ).powersum()
sage: p([5]).is_primitive()
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> s = SymmetricFunctions(QQ).schur()
>>> s([Integer(5)]).is_primitive()                                          ␣
↪     # needs lrcalc_python
False
>>> p = SymmetricFunctions(QQ).powersum()
>>> p([Integer(5)]).is_primitive()
True
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

WithBasis

alias of *BialgebrasWithBasis*

additional_structure()

Return `None`.

Indeed, the category of bialgebras defines no additional structure: a morphism of coalgebras and of algebras between two bialgebras is a bialgebra morphism.

See also

Category.additional_structure()

Todo

This category should be a *CategoryWithAxiom*.

EXAMPLES:

```
sage: Bialgebras(QQ).additional_structure()
```

```
>>> from sage.all import *
>>> Bialgebras(QQ).additional_structure()
```

super_categories()

EXAMPLES:

```
sage: Bialgebras(QQ).super_categories()
[Category of algebras over Rational Field, Category of coalgebras over␣
↪Rational Field]
```

```
>>> from sage.all import *
>>> Bialgebras(QQ).super_categories()
[Category of algebras over Rational Field, Category of coalgebras over␣
↪Rational Field]
```

4.15 Bialgebras with basis

class sage.categories.bialgebras_with_basis.**BialgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of bialgebras with a distinguished basis.

EXAMPLES:

```
sage: C = BialgebrasWithBasis(QQ); C
Category of bialgebras with basis over Rational Field

sage: sorted(C.super_categories(), key=str)
[Category of algebras with basis over Rational Field,
 Category of bialgebras over Rational Field,
 Category of coalgebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = BialgebrasWithBasis(QQ); C
Category of bialgebras with basis over Rational Field

>>> sorted(C.super_categories(), key=str)
[Category of algebras with basis over Rational Field,
 Category of bialgebras over Rational Field,
 Category of coalgebras with basis over Rational Field]
```

class ElementMethods

Bases: object

convolution_power_of_id(*n*)

Compute the n-th convolution power of the identity morphism Id on `self`.

INPUT:

- `n` – nonnegative integer

OUTPUT: the image of `self` under the convolution power Id^{*n}

Note

In the literature, this is also called a Hopf power or Sweedler power, cf. [AL2015].

See also

sage.categories.bialgebras_with_basis.BialgebrasWithBasis.ParentMethods.convolution_product()

Todo

Remove dependency on `modules_with_basis` methods.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: h = SymmetricFunctions(QQ).h()
sage: h[5].convolution_power_of_id(2)
2*h[3, 2] + 2*h[4, 1] + 2*h[5]
sage: h[5].plethysm(2*h[1])
2*h[3, 2] + 2*h[4, 1] + 2*h[5]
sage: h([]).convolution_power_of_id(0)
h[]
sage: h([]).convolution_power_of_id(1)
h[]
sage: h[3,2].convolution_power_of_id(0)
0
sage: h[3,2].convolution_power_of_id(1)
h[3, 2]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> h = SymmetricFunctions(QQ).h()
>>> h[Integer(5)].convolution_power_of_id(Integer(2))
2*h[3, 2] + 2*h[4, 1] + 2*h[5]
>>> h[Integer(5)].plethysm(Integer(2)*h[Integer(1)])
2*h[3, 2] + 2*h[4, 1] + 2*h[5]
>>> h([]).convolution_power_of_id(Integer(0))
h[]
>>> h([]).convolution_power_of_id(Integer(1))
h[]
>>> h[Integer(3),Integer(2)].convolution_power_of_id(Integer(0))
0
>>> h[Integer(3),Integer(2)].convolution_power_of_id(Integer(1))
h[3, 2]
```

```
sage: S = NonCommutativeSymmetricFunctions(QQ).S()                    #␣
↪needs sage.combinat sage.modules
sage: S[4].convolution_power_of_id(5)                                 #␣
↪needs sage.combinat sage.modules
5*S[1, 1, 1, 1] + 10*S[1, 1, 2] + 10*S[1, 2, 1]
 + 10*S[1, 3] + 10*S[2, 1, 1] + 10*S[2, 2] + 10*S[3, 1] + 5*S[4]
```

```
>>> from sage.all import *
>>> S = NonCommutativeSymmetricFunctions(QQ).S()                      #␣
↪needs sage.combinat sage.modules
>>> S[Integer(4)].convolution_power_of_id(Integer(5))                  ␣
↪              # needs sage.combinat sage.modules
5*S[1, 1, 1, 1] + 10*S[1, 1, 2] + 10*S[1, 2, 1]
 + 10*S[1, 3] + 10*S[2, 1, 1] + 10*S[2, 2] + 10*S[3, 1] + 5*S[4]
```

```
sage: m = SymmetricFunctionsNonCommutingVariables(QQ).m()                #␣
↪needs sage.combinat sage.graphs sage.modules
sage: m[[1,3],[2]].convolution_power_of_id(-2)                           #␣
↪needs sage.combinat sage.graphs sage.modules
3*m{{1}, {2, 3}} + 3*m{{1, 2}, {3}} + 6*m{{1, 2, 3}} - 2*m{{1, 3}, {2}}
```

```
>>> from sage.all import *
>>> m = SymmetricFunctionsNonCommutingVariables(QQ).m()                  #␣
↪needs sage.combinat sage.graphs sage.modules
>>> m[[Integer(1),Integer(3)],[Integer(2)]].convolution_power_of_id(-
↪Integer(2))                          # needs sage.combinat sage.graphs␣
↪sage.modules
3*m{{1}, {2, 3}} + 3*m{{1, 2}, {3}} + 6*m{{1, 2, 3}} - 2*m{{1, 3}, {2}}
```

convolution_product(**maps*)

Return the image of `self` under the convolution product (map) of the maps.

Let A and B be bialgebras over a commutative ring R. Given maps $f_i : A \to B$ for $1 \leq i < n$, define the convolution product

$$(f_1 * f_2 * \cdots * f_n) := \mu^{(n-1)} \circ (f_1 \otimes f_2 \otimes \cdots \otimes f_n) \circ \Delta^{(n-1)},$$

where $\Delta^{(k)} := \left(\Delta \otimes \mathrm{Id}^{\otimes(k-1)}\right) \circ \Delta^{(k-1)}$, with $\Delta^{(1)} = \Delta$ (the ordinary coproduct in A) and $\Delta^{(0)} = \mathrm{Id}$; and with $\mu^{(k)} := \mu \circ \left(\mu^{(k-1)} \otimes \mathrm{Id}\right)$ and $\mu^{(1)} = \mu$ (the ordinary product in B). See [Swe1969].

(In the literature, one finds, e.g., $\Delta^{(2)}$ for what we denote above as $\Delta^{(1)}$. See [KMN2012].)

INPUT:

- `maps` – any number $n \geq 0$ of linear maps $f_1, f_2, \ldots, f_n$ on `self.parent()`; or a single `list` or `tuple` of such maps

OUTPUT: the convolution product of `maps` applied to `self`

AUTHORS:

- Amy Pang - 12 June 2015 - Sage Days 65

Todo

Remove dependency on `modules_with_basis` methods.

EXAMPLES:

We compute convolution products of the identity and antipode maps on Schur functions:

```
sage: # needs sage.combinat sage.modules
sage: Id = lambda x: x
sage: Antipode = lambda x: x.antipode()
sage: s = SymmetricFunctions(QQ).schur()
sage: s[3].convolution_product(Id, Id)                                   #␣
↪needs lrcalc_python
2*s[2, 1] + 4*s[3]
sage: s[3,2].convolution_product(Id) == s[3,2]
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Id = lambda x: x
>>> Antipode = lambda x: x.antipode()
>>> s = SymmetricFunctions(QQ).schur()
>>> s[Integer(3)].convolution_product(Id, Id)                                  ␣
↪     # needs lrcalc_python
2*s[2, 1] + 4*s[3]
>>> s[Integer(3),Integer(2)].convolution_product(Id) == s[Integer(3),
↪Integer(2)]
True
```

The method accepts multiple arguments, or a single argument consisting of a list of maps:

```
sage: s[3,2].convolution_product(Id, Id)                                    #␣
↪needs lrcalc_python sage.combinat sage.modules
2*s[2, 1, 1, 1] + 6*s[2, 2, 1] + 6*s[3, 1, 1] + 12*s[3, 2] + 6*s[4, 1] +␣
↪2*s[5]
sage: s[3,2].convolution_product([Id, Id])                                  #␣
↪needs lrcalc_python sage.combinat sage.modules
2*s[2, 1, 1, 1] + 6*s[2, 2, 1] + 6*s[3, 1, 1] + 12*s[3, 2] + 6*s[4, 1] +␣
↪2*s[5]
```

```
>>> from sage.all import *
>>> s[Integer(3),Integer(2)].convolution_product(Id, Id)                       ␣
↪              # needs lrcalc_python sage.combinat sage.modules
2*s[2, 1, 1, 1] + 6*s[2, 2, 1] + 6*s[3, 1, 1] + 12*s[3, 2] + 6*s[4, 1] +␣
↪2*s[5]
>>> s[Integer(3),Integer(2)].convolution_product([Id, Id])                     ␣
↪              # needs lrcalc_python sage.combinat sage.modules
2*s[2, 1, 1, 1] + 6*s[2, 2, 1] + 6*s[3, 1, 1] + 12*s[3, 2] + 6*s[4, 1] +␣
↪2*s[5]
```

We test the defining property of the antipode morphism; namely, that the antipode is the inverse of the identity map in the convolution algebra whose identity element is the composition of the counit and unit:

```
sage: (s[3,2].convolution_product()                                         #␣
↪needs lrcalc_python sage.combinat sage.modules
....:   == s[3,2].convolution_product(Antipode, Id)
....:   == s[3,2].convolution_product(Id, Antipode))
True
```

```
>>> from sage.all import *
>>> (s[Integer(3),Integer(2)].convolution_product()                            ␣
↪              # needs lrcalc_python sage.combinat sage.modules
...   == s[Integer(3),Integer(2)].convolution_product(Antipode, Id)
...   == s[Integer(3),Integer(2)].convolution_product(Id, Antipode))
True
```

```
sage: Psi = NonCommutativeSymmetricFunctions(QQ).Psi()                      #␣
↪needs sage.combinat sage.modules
sage: Psi[2,1].convolution_product(Id, Id, Id)                              #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
3*Psi[1, 2] + 6*Psi[2, 1]
sage: (Psi[5,1] - Psi[1,5]).convolution_product(Id, Id, Id)              #␣
↪needs sage.combinat sage.modules
-3*Psi[1, 5] + 3*Psi[5, 1]
```

```
>>> from sage.all import *
>>> Psi = NonCommutativeSymmetricFunctions(QQ).Psi()                    #␣
↪needs sage.combinat sage.modules
>>> Psi[Integer(2),Integer(1)].convolution_product(Id, Id, Id)           ␣
↪              # needs sage.combinat sage.modules
3*Psi[1, 2] + 6*Psi[2, 1]
>>> (Psi[Integer(5),Integer(1)] - Psi[Integer(1),Integer(5)]).convolution_
↪product(Id, Id, Id)              # needs sage.combinat sage.modules
-3*Psi[1, 5] + 3*Psi[5, 1]
```

```
sage: # needs sage.combinat sage.groups sage.modules
sage: G = SymmetricGroup(3)
sage: QG = GroupAlgebra(G, QQ)
sage: x = QG.sum_of_terms([(p, p.length())
....:                      for p in Permutations(3)]); x
[1, 3, 2] + [2, 1, 3] + 2*[2, 3, 1] + 2*[3, 1, 2] + 3*[3, 2, 1]
sage: x.convolution_product(Id, Id)
5*[1, 2, 3] + 2*[2, 3, 1] + 2*[3, 1, 2]
sage: x.convolution_product(Id, Id, Id)
4*[1, 2, 3] + [1, 3, 2] + [2, 1, 3] + 3*[3, 2, 1]
sage: x.convolution_product([Id] * 6)
9*[1, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> G = SymmetricGroup(Integer(3))
>>> QG = GroupAlgebra(G, QQ)
>>> x = QG.sum_of_terms([(p, p.length())
...                      for p in Permutations(Integer(3))]); x
[1, 3, 2] + [2, 1, 3] + 2*[2, 3, 1] + 2*[3, 1, 2] + 3*[3, 2, 1]
>>> x.convolution_product(Id, Id)
5*[1, 2, 3] + 2*[2, 3, 1] + 2*[3, 1, 2]
>>> x.convolution_product(Id, Id, Id)
4*[1, 2, 3] + [1, 3, 2] + [2, 1, 3] + 3*[3, 2, 1]
>>> x.convolution_product([Id] * Integer(6))
9*[1, 2, 3]
```

class ParentMethods

Bases: object

convolution_product(**maps*)

Return the convolution product (a map) of the given maps.

Let A and B be bialgebras over a commutative ring R. Given maps $f_i : A \to B$ for $1 \leq i < n$, define the convolution product

$$(f_1 * f_2 * \cdots * f_n) := \mu^{(n-1)} \circ (f_1 \otimes f_2 \otimes \cdots \otimes f_n) \circ \Delta^{(n-1)},$$

where $\Delta^{(k)} := \left(\Delta \otimes \mathrm{Id}^{\otimes(k-1)}\right) \circ \Delta^{(k-1)}$, with $\Delta^{(1)} = \Delta$ (the ordinary coproduct in A) and $\Delta^{(0)} = \mathrm{Id}$; and with $\mu^{(k)} := \mu \circ \left(\mu^{(k-1)} \otimes \mathrm{Id}\right)$ and $\mu^{(1)} = \mu$ (the ordinary product in B). See [Swe1969].

(In the literature, one finds, e.g., $\Delta^{(2)}$ for what we denote above as $\Delta^{(1)}$. See [KMN2012].)

INPUT:

- `maps` – any number $n \geq 0$ of linear maps $f_1, f_2, \ldots, f_n$ on `self`; or a single `list` or `tuple` of such maps

OUTPUT:

- the new map $f_1 * f_2 * \cdots * f_2$ representing their convolution product

See also

sage.categories.bialgebras_with_basis.BialgebrasWithBasis.ElementMethods.convolution_product()

AUTHORS:

- Aaron Lauve - 12 June 2015 - Sage Days 65

Todo

Remove dependency on `modules_with_basis` methods.

EXAMPLES:

We construct some maps: the identity, the antipode and projection onto the homogeneous component of degree 2:

```
sage: Id = lambda x: x
sage: Antipode = lambda x: x.antipode()
sage: Proj2 = lambda x: x.parent().sum_of_terms([(m, c) for (m, c) in x␣
↪if m.size() == 2])
```

```
>>> from sage.all import *
>>> Id = lambda x: x
>>> Antipode = lambda x: x.antipode()
>>> Proj2 = lambda x: x.parent().sum_of_terms([(m, c) for (m, c) in x if␣
↪m.size() == Integer(2)])
```

Compute the convolution product of the identity with itself and with the projection `Proj2` on the Hopf algebra of non-commutative symmetric functions:

```
sage: # needs sage.combinat sage.modules
sage: R = NonCommutativeSymmetricFunctions(QQ).ribbon()
sage: T = R.convolution_product([Id, Id])
sage: [T(R(comp)) for comp in Compositions(3)]
[4*R[1, 1, 1] + R[1, 2] + R[2, 1],
 2*R[1, 1, 1] + 4*R[1, 2] + 2*R[2, 1] + 2*R[3],
 2*R[1, 1, 1] + 2*R[1, 2] + 4*R[2, 1] + 2*R[3],
 R[1, 2] + R[2, 1] + 4*R[3]]
sage: T = R.convolution_product(Proj2, Id)
sage: [T(R([i])) for i in range(1, 5)]
[0, R[2], R[2, 1] + R[3], R[2, 2] + R[4]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> R = NonCommutativeSymmetricFunctions(QQ).ribbon()
>>> T = R.convolution_product([Id, Id])
>>> [T(R(comp)) for comp in Compositions(Integer(3))]
[4*R[1, 1, 1] + R[1, 2] + R[2, 1],
 2*R[1, 1, 1] + 4*R[1, 2] + 2*R[2, 1] + 2*R[3],
 2*R[1, 1, 1] + 2*R[1, 2] + 4*R[2, 1] + 2*R[3],
 R[1, 2] + R[2, 1] + 4*R[3]]
>>> T = R.convolution_product(Proj2, Id)
>>> [T(R([i])) for i in range(Integer(1), Integer(5))]
[0, R[2], R[2, 1] + R[3], R[2, 2] + R[4]]
```

Compute the convolution product of no maps on the Hopf algebra of symmetric functions in non-commuting variables. This is the composition of the counit with the unit:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: m = SymmetricFunctionsNonCommutingVariables(QQ).m()
sage: T = m.convolution_product()
sage: [T(m(lam))
....:  for lam in SetPartitions(0).list() + SetPartitions(2).list()]
[m{}, 0, 0]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> m = SymmetricFunctionsNonCommutingVariables(QQ).m()
>>> T = m.convolution_product()
>>> [T(m(lam))
...  for lam in SetPartitions(Integer(0)).list() +␣
↪SetPartitions(Integer(2)).list()]
[m{}, 0, 0]
```

Compute the convolution product of the projection `Proj2` with the identity on the Hopf algebra of symmetric functions in non-commuting variables:

```
sage: T = m.convolution_product(Proj2, Id)                                  #␣
↪needs sage.combinat sage.graphs sage.modules
sage: [T(m(lam)) for lam in SetPartitions(3)]                               #␣
↪needs sage.combinat sage.graphs sage.modules
[0,
 m{{1, 2}, {3}} + m{{1, 2, 3}},
 m{{1, 2}, {3}} + m{{1, 2, 3}},
 m{{1, 2}, {3}} + m{{1, 2, 3}},
 3*m{{1}, {2}, {3}} + 3*m{{1}, {2, 3}} + 3*m{{1, 3}, {2}}]
```

```
>>> from sage.all import *
>>> T = m.convolution_product(Proj2, Id)                                  #␣
↪needs sage.combinat sage.graphs sage.modules
>>> [T(m(lam)) for lam in SetPartitions(Integer(3))]                        ␣
↪     # needs sage.combinat sage.graphs sage.modules
[0,
 m{{1, 2}, {3}} + m{{1, 2, 3}},
 m{{1, 2}, {3}} + m{{1, 2, 3}},
```

(continues on next page)

(continued from previous page)

```
m{{1, 2}, {3}} + m{{1, 2, 3}},
3*m{{1}, {2}, {3}} + 3*m{{1}, {2, 3}} + 3*m{{1, 3}, {2}}]
```

Compute the convolution product of the antipode with itself and the identity map on group algebra of the symmetric group:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: G = SymmetricGroup(3)
sage: QG = GroupAlgebra(G, QQ)
sage: x = QG.sum_of_terms(
....:          [(p, p.number_of_peaks() + p.number_of_inversions())
....:           for p in Permutations(3)]
....:      ); x
2*[1, 3, 2] + [2, 1, 3] + 3*[2, 3, 1] + 2*[3, 1, 2] + 3*[3, 2, 1]
sage: T = QG.convolution_product(Antipode, Antipode, Id)
sage: T(x)
2*[1, 3, 2] + [2, 1, 3] + 2*[2, 3, 1] + 3*[3, 1, 2] + 3*[3, 2, 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> G = SymmetricGroup(Integer(3))
>>> QG = GroupAlgebra(G, QQ)
>>> x = QG.sum_of_terms(
...          [(p, p.number_of_peaks() + p.number_of_inversions())
...           for p in Permutations(Integer(3))]
...      ); x
2*[1, 3, 2] + [2, 1, 3] + 3*[2, 3, 1] + 2*[3, 1, 2] + 3*[3, 2, 1]
>>> T = QG.convolution_product(Antipode, Antipode, Id)
>>> T(x)
2*[1, 3, 2] + [2, 1, 3] + 2*[2, 3, 1] + 3*[3, 1, 2] + 3*[3, 2, 1]
```

4.16 Bimodules

class sage.categories.bimodules.**Bimodules**(*left_base*, *right_base*, *name=None*)

Bases: *CategoryWithParameters*

The category of (R, S)-bimodules.

For R and S rings, a (R, S)-bimodule X is a left R-module and right S-module such that the left and right actions commute: $r * (x * s) = (r * x) * s$.

EXAMPLES:

```
sage: Bimodules(QQ, ZZ)
Category of bimodules over Rational Field on the left and Integer Ring on the␣
↪right
sage: Bimodules(QQ, ZZ).super_categories()
[Category of left modules over Rational Field, Category of right modules over␣
↪Integer Ring]
```

```
>>> from sage.all import *
>>> Bimodules(QQ, ZZ)
```

(continues on next page)

(continued from previous page)

```
Category of bimodules over Rational Field on the left and Integer Ring on the␣
↪right
>>> Bimodules(QQ, ZZ).super_categories()
[Category of left modules over Rational Field, Category of right modules over␣
↪Integer Ring]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

additional_structure()

Return None.

Indeed, the category of bimodules defines no additional structure: a left and right module morphism between two bimodules is a bimodule morphism.

See also

Category.additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: Bimodules(QQ, ZZ).additional_structure()
```

```
>>> from sage.all import *
>>> Bimodules(QQ, ZZ).additional_structure()
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: Bimodules.an_instance()                                                  #␣
↪needs sage.rings.real_mpfr
Category of bimodules over Rational Field on the left and Real Field with 53␣
↪bits of precision on the right
```

```
>>> from sage.all import *
>>> Bimodules.an_instance()                                                    #␣
↪needs sage.rings.real_mpfr
Category of bimodules over Rational Field on the left and Real Field with 53␣
↪bits of precision on the right
```

left_base_ring()

Return the left base ring over which elements of this category are defined.

EXAMPLES:

```
sage: Bimodules(QQ, ZZ).left_base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> Bimodules(QQ, ZZ).left_base_ring()
Rational Field
```

right_base_ring()

Return the right base ring over which elements of this category are defined.

EXAMPLES:

```
sage: Bimodules(QQ, ZZ).right_base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> Bimodules(QQ, ZZ).right_base_ring()
Integer Ring
```

super_categories()

EXAMPLES:

```
sage: Bimodules(QQ, ZZ).super_categories()
[Category of left modules over Rational Field, Category of right modules over␣
↪Integer Ring]
```

```
>>> from sage.all import *
>>> Bimodules(QQ, ZZ).super_categories()
[Category of left modules over Rational Field, Category of right modules over␣
↪Integer Ring]
```

4.17 Category of chain complexes

class sage.categories.chain_complexes.**ChainComplexes**(*base*, *name=None*)

Bases: *Category_module*

The category of all chain complexes over a base ring.

EXAMPLES:

```
sage: ChainComplexes(RationalField())
Category of chain complexes over Rational Field
sage: ChainComplexes(Integers(9))
Category of chain complexes over Ring of integers modulo 9
```

```
>>> from sage.all import *
>>> ChainComplexes(RationalField())
Category of chain complexes over Rational Field
>>> ChainComplexes(Integers(Integer(9)))
Category of chain complexes over Ring of integers modulo 9
```

class ParentMethods

Bases: object

differential(**args*, ***kwargs*)

Return the differentials (or boundary maps) of the chain complex.

EXAMPLES:

```
sage: C = ChainComplex({0: matrix(ZZ, 2, 3, [3, 0, 0, 0, 0, 0])})        #␣
↪needs sage.modules
sage: C.differential(0)                                                   #␣
↪needs sage.modules
[3 0 0]
[0 0 0]
```

```
>>> from sage.all import *
>>> C = ChainComplex({Integer(0): matrix(ZZ, Integer(2), Integer(3),␣
↪[Integer(3), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0)])})        # needs sage.modules
>>> C.differential(Integer(0))                                             ␣
↪      # needs sage.modules
[3 0 0]
[0 0 0]
```

```
sage: A.<x,y,z> = GradedCommutativeAlgebra(QQ, degrees=(2, 2, 3))        #␣
↪needs sage.combinat sage.libs.singular sage.modules
sage: C = A.cdg_algebra({z: x*y})                                         #␣
↪needs sage.combinat sage.libs.singular sage.modules
sage: C.differential()                                                    #␣
↪needs sage.combinat sage.modules
Differential of Commutative Differential Graded Algebra with
 generators ('x', 'y', 'z') in degrees (2, 2, 3) over Rational Field
  Defn: x --> 0
        y --> 0
        z --> x*y
```

```
>>> from sage.all import *
>>> A = GradedCommutativeAlgebra(QQ, degrees=(Integer(2), Integer(2),␣
↪Integer(3)), names=('x', 'y', 'z',)); (x, y, z,) = A._first_ngens(3)#␣
↪needs sage.combinat sage.libs.singular sage.modules
>>> C = A.cdg_algebra({z: x*y})                                           #␣
↪needs sage.combinat sage.libs.singular sage.modules
>>> C.differential()                                                      #␣
↪needs sage.combinat sage.modules
Differential of Commutative Differential Graded Algebra with
 generators ('x', 'y', 'z') in degrees (2, 2, 3) over Rational Field
  Defn: x --> 0
        y --> 0
        z --> x*y
```

homology(*n=None*)

Return the homology of the chain complex.

INPUT:

- n – (default: None) degree of the homology; if none is provided, the direct sum homology will be used

EXAMPLES:

```
sage: # needs sage.modules
sage: C = ChainComplex({0: matrix(ZZ, 2, 3, [3, 0, 0, 0, 0, 0])})
sage: C.homology(0)
Z x Z
sage: C.homology(1)
Z x C3
sage: C.homology(2)
0
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> C = ChainComplex({Integer(0): matrix(ZZ, Integer(2), Integer(3),␣
↪[Integer(3), Integer(0), Integer(0), Integer(0), Integer(0),␣
↪Integer(0)])})
>>> C.homology(Integer(0))
Z x Z
>>> C.homology(Integer(1))
Z x C3
>>> C.homology(Integer(2))
0
```

```
sage: # needs sage.combinat sage.modules
sage: A.<x,y,z> = GradedCommutativeAlgebra(QQ, degrees=(2, 2, 3))        #␣
↪needs sage.libs.singular
sage: C = A.cdg_algebra({z: x*y})                                        #␣
↪needs sage.libs.singular
sage: C.homology(0)
Free module generated by {[1]} over Rational Field
sage: C.homology(1)
Free module generated by {} over Rational Field
sage: C.homology(2)
Free module generated by {[x], [y]} over Rational Field
sage: C.homology(3)
Free module generated by {} over Rational Field
sage: C.homology(4)
Free module generated by {[x^2], [y^2]} over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = GradedCommutativeAlgebra(QQ, degrees=(Integer(2), Integer(2),␣
↪Integer(3)), names=('x', 'y', 'z',)); (x, y, z,) = A._first_ngens(3)#␣
↪needs sage.libs.singular
>>> C = A.cdg_algebra({z: x*y})                                          #␣
↪needs sage.libs.singular
>>> C.homology(Integer(0))
Free module generated by {[1]} over Rational Field
>>> C.homology(Integer(1))
Free module generated by {} over Rational Field
>>> C.homology(Integer(2))
```

(continues on next page)

(continued from previous page)

```
Free module generated by {[x], [y]} over Rational Field
>>> C.homology(Integer(3))
Free module generated by {} over Rational Field
>>> C.homology(Integer(4))
Free module generated by {[x^2], [y^2]} over Rational Field
```

lift_from_homology(*x*)

Lift the homology element `x` to the corresponding module.

EXAMPLES:

```
sage: # needs sage.symbolic
sage: E3 = EuclideanSpace(3)
sage: C = E3.de_rham_complex()
sage: one = C.homology().one()
sage: C.lift_from_homology(one)
Mixed differential form one on the Euclidean space E^3
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> E3 = EuclideanSpace(Integer(3))
>>> C = E3.de_rham_complex()
>>> one = C.homology().one()
>>> C.lift_from_homology(one)
Mixed differential form one on the Euclidean space E^3
```

reduce_to_homology(*x*, *n=None*)

Reduce a cycle to the corresponding quotient in homology.

INPUT:

- `x` – a cycle
- `n` – (default: `None`) degree of the homology; if none is provided, the direct sum homology will be used

EXAMPLES:

```
sage: # needs sage.symbolic
sage: E3 = EuclideanSpace(3)
sage: C = E3.de_rham_complex()
sage: one = C.one()
sage: C.reduce_to_homology(one)
[one]
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> E3 = EuclideanSpace(Integer(3))
>>> C = E3.de_rham_complex()
>>> one = C.one()
>>> C.reduce_to_homology(one)
[one]
```

super_categories()

EXAMPLES:

```
sage: ChainComplexes(Integers(9)).super_categories()
[Category of modules over Ring of integers modulo 9]
```

```
>>> from sage.all import *
>>> ChainComplexes(Integers(Integer(9))).super_categories()
[Category of modules over Ring of integers modulo 9]
```

class sage.categories.chain_complexes.**HomologyFunctor**(*domain*, *n=None*)

Bases: *Functor*

Homology functor.

INPUT:

- `domain` – must be a category of chain complexes
- `n` – (default: `None`) degree of the homology; if none is provided, the direct sum homology will be used

EXAMPLES:

```
sage: C = ChainComplex({0: matrix(ZZ, 2, 3, [3, 0, 0, 0, 0, 0])})                #␣
↪needs sage.modules
sage: H = HomologyFunctor(ChainComplexes(ZZ), 1)
sage: H(C)                                                                       #␣
↪needs sage.modules
Z x C3
```

```
>>> from sage.all import *
>>> C = ChainComplex({Integer(0): matrix(ZZ, Integer(2), Integer(3), [Integer(3),␣
↪Integer(0), Integer(0), Integer(0), Integer(0), Integer(0)])})                  #␣
↪needs sage.modules
>>> H = HomologyFunctor(ChainComplexes(ZZ), Integer(1))
>>> H(C)                                                                         #␣
↪needs sage.modules
Z x C3
```

```
sage: A.<x,y,z> = GradedCommutativeAlgebra(QQ, degrees=(2, 2, 3))                #␣
↪needs sage.combinat sage.libs.singular sage.modules
sage: C = A.cdg_algebra({z: x*y})                                                #␣
↪needs sage.combinat sage.libs.singular sage.modules
sage: H = HomologyFunctor(ChainComplexes(QQ), 2)
sage: H(C)                                                                       #␣
↪needs sage.combinat sage.modules
Free module generated by {[x], [y]} over Rational Field
```

```
>>> from sage.all import *
>>> A = GradedCommutativeAlgebra(QQ, degrees=(Integer(2), Integer(2), Integer(3)),
↪ names=('x', 'y', 'z',)); (x, y, z,) = A._first_ngens(3)# needs sage.combinat␣
↪sage.libs.singular sage.modules
>>> C = A.cdg_algebra({z: x*y})                                                  #␣
↪needs sage.combinat sage.libs.singular sage.modules
>>> H = HomologyFunctor(ChainComplexes(QQ), Integer(2))
>>> H(C)                                                                         #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
Free module generated by {[x], [y]} over Rational Field
```

Applying to a chain map:

```
sage: # needs sage.graphs sage.modules
sage: S = simplicial_complexes.Sphere(1); S
Minimal triangulation of the 1-sphere
sage: SCC = S.chain_complex()
sage: SCC.differential()
{0: [],
 1: [-1 -1  0]
    [ 1  0 -1]
    [ 0  1  1],
 2: []}
sage: f = {0: zero_matrix(ZZ,3,3), 1: zero_matrix(ZZ,3,3)}
sage: G = Hom(SCC, SCC)
sage: x = G(f)
sage: H = HomologyFunctor(ChainComplexes(ZZ), 1)
sage: H(SCC)
Z
sage: H(x)
Generic morphism:
  From: Z
  To:   Z
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> S = simplicial_complexes.Sphere(Integer(1)); S
Minimal triangulation of the 1-sphere
>>> SCC = S.chain_complex()
>>> SCC.differential()
{0: [],
 1: [-1 -1  0]
    [ 1  0 -1]
    [ 0  1  1],
 2: []}
>>> f = {Integer(0): zero_matrix(ZZ,Integer(3),Integer(3)), Integer(1): zero_
↪matrix(ZZ,Integer(3),Integer(3))}
>>> G = Hom(SCC, SCC)
>>> x = G(f)
>>> H = HomologyFunctor(ChainComplexes(ZZ), Integer(1))
>>> H(SCC)
Z
>>> H(x)
Generic morphism:
  From: Z
  To:   Z
```

4.18 Classical Crystals

class sage.categories.classical_crystals.**ClassicalCrystals**

Bases: *Category_singleton*

The category of classical crystals, that is crystals of finite Cartan type.

EXAMPLES:

```
sage: C = ClassicalCrystals()
sage: C
Category of classical crystals
sage: C.super_categories()
[Category of regular crystals,
 Category of finite crystals,
 Category of highest weight crystals]
sage: C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> C = ClassicalCrystals()
>>> C
Category of classical crystals
>>> C.super_categories()
[Category of regular crystals,
 Category of finite crystals,
 Category of highest weight crystals]
>>> C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

class ElementMethods

Bases: object

lusztig_involution()

Return the Lusztig involution on the classical highest weight crystal `self`.

The Lusztig involution on a finite-dimensional highest weight crystal $B(\lambda)$ of highest weight λ maps the highest weight vector to the lowest weight vector and the Kashiwara operator f_i to e_{i^*}, where i^* is defined as $\alpha_{i^*} = -w_0(\alpha_i)$. Here w_0 is the longest element of the Weyl group acting on the i-th simple root α_i.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',3],shape=[2,1])
sage: b = B(rows=[[1,2],[4]])
sage: b.lusztig_involution()
[[1, 4], [3]]
sage: b.to_tableau().schuetzenberger_involution(n=4)
[[1, 4], [3]]

sage: all(b.lusztig_involution().to_tableau() == b.to_tableau().
↪schuetzenberger_involution(n=4) for b in B)
True

sage: B = crystals.Tableaux(['D',4],shape=[1])
sage: [[b,b.lusztig_involution()] for b in B]
```

(continues on next page)

(continued from previous page)

```
[[[[1]], [[-1]]], [[[2]], [[-2]]], [[[3]], [[-3]]], [[[4]], [[-4]]], [[[-
↪4]],
[[4]]], [[[-3]], [[3]]], [[[-2]], [[2]]], [[[-1]], [[1]]]]

sage: B = crystals.Tableaux(['D',3],shape=[1])
sage: [[b,b.lusztig_involution()] for b in B]
[[[[1]], [[-1]]], [[[2]], [[-2]]], [[[3]], [[3]]], [[[-3]], [[-3]]],
[[[-2]], [[2]]], [[[-1]], [[1]]]]

sage: C = CartanType(['E',6])
sage: La = C.root_system().weight_lattice().fundamental_weights()
sage: T = crystals.HighestWeight(La[1])
sage: t = T[3]; t
[(-4, 2, 5)]
sage: t.lusztig_involution()
[(-2, -3, 4)]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(3)],shape=[Integer(2),Integer(1)])
>>> b = B(rows=[[Integer(1),Integer(2)],[Integer(4)]])
>>> b.lusztig_involution()
[[1, 4], [3]]
>>> b.to_tableau().schuetzenberger_involution(n=Integer(4))
[[1, 4], [3]]

>>> all(b.lusztig_involution().to_tableau() == b.to_tableau().
↪schuetzenberger_involution(n=Integer(4)) for b in B)
True

>>> B = crystals.Tableaux(['D',Integer(4)],shape=[Integer(1)])
>>> [[b,b.lusztig_involution()] for b in B]
[[[[1]], [[-1]]], [[[2]], [[-2]]], [[[3]], [[-3]]], [[[4]], [[-4]]], [[[-
↪4]],
[[4]]], [[[-3]], [[3]]], [[[-2]], [[2]]], [[[-1]], [[1]]]]

>>> B = crystals.Tableaux(['D',Integer(3)],shape=[Integer(1)])
>>> [[b,b.lusztig_involution()] for b in B]
[[[[1]], [[-1]]], [[[2]], [[-2]]], [[[3]], [[3]]], [[[-3]], [[-3]]],
[[[-2]], [[2]]], [[[-1]], [[1]]]]

>>> C = CartanType(['E',Integer(6)])
>>> La = C.root_system().weight_lattice().fundamental_weights()
>>> T = crystals.HighestWeight(La[Integer(1)])
>>> t = T[Integer(3)]; t
[(-4, 2, 5)]
>>> t.lusztig_involution()
[(-2, -3, 4)]
```

class ParentMethods

Bases: object

cardinality()

Return the number of elements of the crystal, using Weyl's dimension formula on each connected com-

ponent.

EXAMPLES:

```
sage: C = ClassicalCrystals().example(5)
sage: C.cardinality()
6
```

```
>>> from sage.all import *
>>> C = ClassicalCrystals().example(Integer(5))
>>> C.cardinality()
6
```

character(*R=None*)

Return the character of this crystal.

INPUT:

- `R` – a `WeylCharacterRing` (default: the default `WeylCharacterRing` for this Cartan type)

Returns the character of `self` as an element of `R`.

EXAMPLES:

```
sage: C = crystals.Tableaux("A2", shape=[2,1])
sage: chi = C.character(); chi
A2(2,1,0)

sage: T = crystals.TensorProduct(C,C)
sage: chiT = T.character(); chiT
A2(2,2,2) + 2*A2(3,2,1) + A2(3,3,0) + A2(4,1,1) + A2(4,2,0)
sage: chiT == chi^2
True
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux("A2", shape=[Integer(2),Integer(1)])
>>> chi = C.character(); chi
A2(2,1,0)

>>> T = crystals.TensorProduct(C,C)
>>> chiT = T.character(); chiT
A2(2,2,2) + 2*A2(3,2,1) + A2(3,3,0) + A2(4,1,1) + A2(4,2,0)
>>> chiT == chi**Integer(2)
True
```

One may specify an alternate `WeylCharacterRing`:

```
sage: R = WeylCharacterRing("A2", style='coroots')
sage: chiT = T.character(R); chiT
A2(0,0) + 2*A2(1,1) + A2(0,3) + A2(3,0) + A2(2,2)
sage: chiT in R
True
```

```
>>> from sage.all import *
>>> R = WeylCharacterRing("A2", style='coroots')
>>> chiT = T.character(R); chiT
```

(continues on next page)

(continued from previous page)

```
A2(0,0) + 2*A2(1,1) + A2(0,3) + A2(3,0) + A2(2,2)
>>> chiT in R
True
```

It should have the same Cartan type and use the same realization of the weight lattice as `self`:

```
sage: R = WeylCharacterRing("A3", style='coroots')
sage: T.character(R)
Traceback (most recent call last):
...
ValueError: Weyl character ring does not have the right Cartan type
```

```
>>> from sage.all import *
>>> R = WeylCharacterRing("A3", style='coroots')
>>> T.character(R)
Traceback (most recent call last):
...
ValueError: Weyl character ring does not have the right Cartan type
```

demazure_character(*w*, *f=None*)

Return the Demazure character associated to `w`.

INPUT:

- `w` – an element of the ambient weight lattice realization of the crystal, or a reduced word, or an element in the associated Weyl group

OPTIONAL:

- `f` – a function from the crystal to a module

This is currently only supported for crystals whose underlying weight space is the ambient space.

The Demazure character is obtained by applying the Demazure operator D_w (see *sage.categories.regular_crystals.RegularCrystals.ParentMethods.demazure_operator()*) to the highest weight element of the classical crystal. The simple Demazure operators D_i (see *sage.categories.regular_crystals.RegularCrystals.ElementMethods.demazure_operator_simple()*) do not braid on the level of crystals, but on the level of characters they do. That is why it makes sense to input `w` either as a weight, a reduced word, or as an element of the underlying Weyl group.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape = [2,1])
sage: e = T.weight_lattice_realization().basis()
sage: weight = e[0] + 2*e[2]
sage: weight.reduced_word()
[2, 1]
sage: T.demazure_character(weight)
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x1*x3^2

sage: T = crystals.Tableaux(['A',3],shape=[2,1])
sage: T.demazure_character([1,2,3])
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x2^2*x3
sage: W = WeylGroup(['A',3])
sage: w = W.from_reduced_word([1,2,3])
sage: T.demazure_character(w)
```

(continues on next page)

(continued from previous page)

```
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x2^2*x3

sage: T = crystals.Tableaux(['B',2], shape = [2])
sage: e = T.weight_lattice_realization().basis()
sage: weight = -2*e[1]
sage: T.demazure_character(weight)
x1^2 + x1*x2 + x2^2 + x1 + x2 + x1/x2 + 1/x2 + 1/x2^2 + 1

sage: T = crystals.Tableaux("B2",shape=[1/2,1/2])
sage: b2=WeylCharacterRing("B2",base_ring=QQ).ambient()
sage: T.demazure_character([1,2],f=lambda x:b2(x.weight()))
b2(-1/2,1/2) + b2(1/2,-1/2) + b2(1/2,1/2)
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape = [Integer(2),
↪Integer(1)])
>>> e = T.weight_lattice_realization().basis()
>>> weight = e[Integer(0)] + Integer(2)*e[Integer(2)]
>>> weight.reduced_word()
[2, 1]
>>> T.demazure_character(weight)
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x1*x3^2

>>> T = crystals.Tableaux(['A',Integer(3)],shape=[Integer(2),Integer(1)])
>>> T.demazure_character([Integer(1),Integer(2),Integer(3)])
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x2^2*x3
>>> W = WeylGroup(['A',Integer(3)])
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(3)])
>>> T.demazure_character(w)
x1^2*x2 + x1*x2^2 + x1^2*x3 + x1*x2*x3 + x2^2*x3

>>> T = crystals.Tableaux(['B',Integer(2)], shape = [Integer(2)])
>>> e = T.weight_lattice_realization().basis()
>>> weight = -Integer(2)*e[Integer(1)]
>>> T.demazure_character(weight)
x1^2 + x1*x2 + x2^2 + x1 + x2 + x1/x2 + 1/x2 + 1/x2^2 + 1

>>> T = crystals.Tableaux("B2",shape=[Integer(1)/Integer(2),Integer(1)/
↪Integer(2)])
>>> b2=WeylCharacterRing("B2",base_ring=QQ).ambient()
>>> T.demazure_character([Integer(1),Integer(2)],f=lambda x:b2(x.
↪weight()))
b2(-1/2,1/2) + b2(1/2,-1/2) + b2(1/2,1/2)
```

REFERENCES:

- [De1974]
- [Ma2009]

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of classical crystals constructed by tensor product of classical crystals.

extra_super_categories()

EXAMPLES:

```
sage: ClassicalCrystals().TensorProducts().extra_super_categories()
[Category of classical crystals]
```

```
>>> from sage.all import *
>>> ClassicalCrystals().TensorProducts().extra_super_categories()
[Category of classical crystals]
```

additional_structure()

Return None.

Indeed, the category of classical crystals defines no additional structure: it only states that its objects are $U_q(\mathfrak{g})$-crystals, where $\mathfrak{g}$ is of finite type.

See also

Category.additional_structure()

EXAMPLES:

```
sage: ClassicalCrystals().additional_structure()
```

```
>>> from sage.all import *
>>> ClassicalCrystals().additional_structure()
```

example(*n=3*)

Return an example of highest weight crystals, as per *Category.example()*.

EXAMPLES:

```
sage: B = ClassicalCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> B = ClassicalCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

super_categories()

EXAMPLES:

```
sage: ClassicalCrystals().super_categories()
[Category of regular crystals,
 Category of finite crystals,
 Category of highest weight crystals]
```

```
>>> from sage.all import *
>>> ClassicalCrystals().super_categories()
[Category of regular crystals,
 Category of finite crystals,
 Category of highest weight crystals]
```

4.19 Coalgebras

class sage.categories.coalgebras.**Coalgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of coalgebras.

EXAMPLES:

```
sage: Coalgebras(QQ)
Category of coalgebras over Rational Field
sage: Coalgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> Coalgebras(QQ)
Category of coalgebras over Rational Field
>>> Coalgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

class Cocommutative(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of cocommutative coalgebras.

class DualObjects(*category*, **args*)

Bases: *DualObjectsCategory*

extra_super_categories()

Return the dual category.

EXAMPLES:

The category of coalgebras over the Rational Field is dual to the category of algebras over the same field:

```
sage: C = Coalgebras(QQ)
sage: C.dual()
Category of duals of coalgebras over Rational Field
sage: C.dual().super_categories() # indirect doctest
[Category of algebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> C = Coalgebras(QQ)
>>> C.dual()
Category of duals of coalgebras over Rational Field
>>> C.dual().super_categories() # indirect doctest
[Category of algebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

Warning

This is only correct in certain cases (finite dimension, …). See Issue #15647.

class ElementMethods

Bases: object

coproduct()

Return the coproduct of self.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
sage: [a,b] = A.algebra_generators()
sage: a, a.coproduct()
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
sage: b, b.coproduct()
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
>>> [a,b] = A.algebra_generators()
>>> a, a.coproduct()
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
>>> b, b.coproduct()
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

counit()

Return the counit of self.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
sage: [a,b] = A.algebra_generators()
sage: a, a.counit()
(B[(1,2,3)], 1)
sage: b, b.counit()
(B[(1,3)], 1)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
>>> [a,b] = A.algebra_generators()
```

(continues on next page)

(continued from previous page)

```
>>> a, a.counit()
(B[(1,2,3)], 1)
>>> b, b.counit()
(B[(1,3)], 1)
```

class Filtered(*base_category*)

Bases: *FilteredModulesCategory*

Category of filtered coalgebras.

Graded

alias of *GradedCoalgebras*

class ParentMethods

Bases: object

coproduct(*x*)

Return the coproduct of x.

Eventually, there will be a default implementation, delegating to the overloading mechanism and forcing the conversion back

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
sage: [a,b] = A.algebra_generators()
sage: a, A.coproduct(a)
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
sage: b, A.coproduct(b)
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
>>> [a,b] = A.algebra_generators()
>>> a, A.coproduct(a)
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
>>> b, A.coproduct(b)
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

counit(*x*)

Return the counit of x.

Eventually, there will be a default implementation, delegating to the overloading mechanism and forcing the conversion back

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
sage: [a,b] = A.algebra_generators()
sage: a, A.counit(a)
(B[(1,2,3)], 1)
sage: b, A.counit(b)
(B[(1,3)], 1)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6 as a permutation group
 over Rational Field
>>> [a,b] = A.algebra_generators()
>>> a, A.counit(a)
(B[(1,2,3)], 1)
>>> b, A.counit(b)
(B[(1,3)], 1)
```

TODO: implement some tests of the axioms of coalgebras, bialgebras and Hopf algebras using the counit.

class Realizations(*category*, **args*)

Bases: *RealizationsCategory*

class ParentMethods

Bases: object

coproduct_by_coercion(*x*)

Return the coproduct by coercion if `coproduct_by_basis` is not implemented.

EXAMPLES:

```
sage: # needs sage.modules
sage: Sym = SymmetricFunctions(QQ)
sage: m = Sym.monomial()
sage: f = m[2,1]
sage: f.coproduct.__module__
'sage.categories.coalgebras'
sage: m.coproduct_on_basis
NotImplemented
sage: m.coproduct == m.coproduct_by_coercion
True
sage: f.coproduct()                                                    #␣
↪needs lrcalc_python
m[] # m[2, 1] + m[1] # m[2] + m[2] # m[1] + m[2, 1] # m[]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Sym = SymmetricFunctions(QQ)
>>> m = Sym.monomial()
```

(continues on next page)

(continued from previous page)

```
>>> f = m[Integer(2),Integer(1)]
>>> f.coproduct.__module__
'sage.categories.coalgebras'
>>> m.coproduct_on_basis
NotImplemented
>>> m.coproduct == m.coproduct_by_coercion
True
>>> f.coproduct()                                                    #␣
↪needs lrcalc_python
m[] # m[2, 1] + m[1] # m[2] + m[2] # m[1] + m[2, 1] # m[]
```

```
sage: # needs sage.modules
sage: N = NonCommutativeSymmetricFunctions(QQ)
sage: R = N.ribbon()
sage: R.coproduct_by_coercion.__module__
'sage.categories.coalgebras'
sage: R.coproduct_on_basis
NotImplemented
sage: R.coproduct == R.coproduct_by_coercion
True
sage: R[1].coproduct()
R[] # R[1] + R[1] # R[]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> N = NonCommutativeSymmetricFunctions(QQ)
>>> R = N.ribbon()
>>> R.coproduct_by_coercion.__module__
'sage.categories.coalgebras'
>>> R.coproduct_on_basis
NotImplemented
>>> R.coproduct == R.coproduct_by_coercion
True
>>> R[Integer(1)].coproduct()
R[] # R[1] + R[1] # R[]
```

counit_by_coercion(*x*)

Return the counit of `x` if `counit_by_basis` is not implemented.

EXAMPLES:

```
sage: # needs lrcalc_python sage.modules
sage: sp = SymmetricFunctions(QQ).sp()
sage: sp.an_element()
2*sp[] + 2*sp[1] + 3*sp[2]
sage: sp.counit(sp.an_element())
2
sage: o = SymmetricFunctions(QQ).o()
sage: o.an_element()
2*o[] + 2*o[1] + 3*o[2]
sage: o.counit(o.an_element())
-1
```

```
>>> from sage.all import *
>>> # needs lrcalc_python sage.modules
>>> sp = SymmetricFunctions(QQ).sp()
>>> sp.an_element()
2*sp[] + 2*sp[1] + 3*sp[2]
>>> sp.counit(sp.an_element())
2
>>> o = SymmetricFunctions(QQ).o()
>>> o.an_element()
2*o[] + 2*o[1] + 3*o[2]
>>> o.counit(o.an_element())
-1
```

class SubcategoryMethods

Bases: object

Cocommutative()

Return the full subcategory of the cocommutative objects of self.

A coalgebra C is said to be *cocommutative* if

$$\Delta(c) = \sum_{(c)} c_{(1)} \otimes c_{(2)} = \sum_{(c)} c_{(2)} \otimes c_{(1)}$$

in Sweedler's notation for all $c \in C$.

EXAMPLES:

```
sage: C1 = Coalgebras(ZZ).Cocommutative().WithBasis(); C1
Category of cocommutative coalgebras with basis over Integer Ring
sage: C2 = Coalgebras(ZZ).WithBasis().Cocommutative()
sage: C1 is C2
True
sage: BialgebrasWithBasis(QQ).Cocommutative()
Category of cocommutative bialgebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> C1 = Coalgebras(ZZ).Cocommutative().WithBasis(); C1
Category of cocommutative coalgebras with basis over Integer Ring
>>> C2 = Coalgebras(ZZ).WithBasis().Cocommutative()
>>> C1 is C2
True
>>> BialgebrasWithBasis(QQ).Cocommutative()
Category of cocommutative bialgebras with basis over Rational Field
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

class SubcategoryMethods

Bases: object

Supercocommutative()

Return the full subcategory of the supercocommutative objects of self.

EXAMPLES:

```
sage: Coalgebras(ZZ).WithBasis().Super().Supercocommutative()
Category of supercocommutative super coalgebras with basis over␣
↪Integer Ring
sage: BialgebrasWithBasis(QQ).Super().Supercocommutative()
Join of Category of super algebras with basis over Rational Field
 and Category of super bialgebras over Rational Field
 and Category of super coalgebras with basis over Rational Field
 and Category of supercocommutative super coalgebras over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> Coalgebras(ZZ).WithBasis().Super().Supercocommutative()
Category of supercocommutative super coalgebras with basis over␣
↪Integer Ring
>>> BialgebrasWithBasis(QQ).Super().Supercocommutative()
Join of Category of super algebras with basis over Rational Field
 and Category of super bialgebras over Rational Field
 and Category of super coalgebras with basis over Rational Field
 and Category of supercocommutative super coalgebras over Rational␣
↪Field
```

class Supercocommutative(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of supercocommutative coalgebras.

extra_super_categories()

EXAMPLES:

```
sage: Coalgebras(ZZ).Super().extra_super_categories()
[Category of graded coalgebras over Integer Ring]
sage: Coalgebras(ZZ).Super().super_categories()
[Category of graded coalgebras over Integer Ring,
 Category of super modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Coalgebras(ZZ).Super().extra_super_categories()
[Category of graded coalgebras over Integer Ring]
>>> Coalgebras(ZZ).Super().super_categories()
[Category of graded coalgebras over Integer Ring,
 Category of super modules over Integer Ring]
```

Compare this with the situation for bialgebras:

```
sage: Bialgebras(ZZ).Super().extra_super_categories()
[]
sage: Bialgebras(ZZ).Super().super_categories()
[Category of super algebras over Integer Ring,
 Category of super coalgebras over Integer Ring]
```

```
>>> from sage.all import *
>>> Bialgebras(ZZ).Super().extra_super_categories()
[]
```

(continues on next page)

(continued from previous page)

```
>>> Bialgebras(ZZ).Super().super_categories()
[Category of super algebras over Integer Ring,
 Category of super coalgebras over Integer Ring]
```

The category of bialgebras does not occur in these results, since super bialgebras are not bialgebras.

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

class ElementMethods

Bases: object

class ParentMethods

Bases: object

extra_super_categories()

EXAMPLES:

```
sage: Coalgebras(QQ).TensorProducts().extra_super_categories()
[Category of coalgebras over Rational Field]
sage: Coalgebras(QQ).TensorProducts().super_categories()
[Category of tensor products of vector spaces over Rational Field,
 Category of coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> Coalgebras(QQ).TensorProducts().extra_super_categories()
[Category of coalgebras over Rational Field]
>>> Coalgebras(QQ).TensorProducts().super_categories()
[Category of tensor products of vector spaces over Rational Field,
 Category of coalgebras over Rational Field]
```

Meaning: a tensor product of coalgebras is a coalgebra

WithBasis

alias of *CoalgebrasWithBasis*

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

class ParentMethods

Bases: object

coproduct(*x*)

Return the coproduct of `x`.

EXAMPLES:

```
sage: # needs sage.modules
sage: N = NonCommutativeSymmetricFunctions(QQ)
sage: S = N.complete()
sage: N.coproduct.__module__
'sage.categories.coalgebras'
sage: N.coproduct(S[2])
S[] # S[2] + S[1] # S[1] + S[2] # S[]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> N = NonCommutativeSymmetricFunctions(QQ)
>>> S = N.complete()
>>> N.coproduct.__module__
'sage.categories.coalgebras'
>>> N.coproduct(S[Integer(2)])
S[] # S[2] + S[1] # S[1] + S[2] # S[]
```

counit(*x*)

Return the counit of `x`.

EXAMPLES:

```
sage: # needs sage.modules
sage: Sym = SymmetricFunctions(QQ)
sage: s = Sym.schur()
sage: f = s[2,1]
sage: f.counit.__module__
'sage.categories.coalgebras'
sage: f.counit()
0
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Sym = SymmetricFunctions(QQ)
>>> s = Sym.schur()
>>> f = s[Integer(2),Integer(1)]
>>> f.counit.__module__
'sage.categories.coalgebras'
>>> f.counit()
0
```

```
sage: # needs sage.modules
sage: N = NonCommutativeSymmetricFunctions(QQ)
sage: N.counit.__module__
'sage.categories.coalgebras'
sage: N.counit(N.one())
1
sage: x = N.an_element(); x
2*S[] + 2*S[1] + 3*S[1, 1]
sage: N.counit(x)
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> N = NonCommutativeSymmetricFunctions(QQ)
>>> N.counit.__module__
'sage.categories.coalgebras'
>>> N.counit(N.one())
1
>>> x = N.an_element(); x
```

(continues on next page)

(continued from previous page)

```
2*S[] + 2*S[1] + 3*S[1, 1]
>>> N.counit(x)
2
```

super_categories()

EXAMPLES:

```
sage: Coalgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> Coalgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

4.20 Coalgebras with basis

class sage.categories.coalgebras_with_basis.**CoalgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of coalgebras with a distinguished basis.

EXAMPLES:

```
sage: CoalgebrasWithBasis(ZZ)
Category of coalgebras with basis over Integer Ring
sage: sorted(CoalgebrasWithBasis(ZZ).super_categories(), key=str)
[Category of coalgebras over Integer Ring,
 Category of modules with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> CoalgebrasWithBasis(ZZ)
Category of coalgebras with basis over Integer Ring
>>> sorted(CoalgebrasWithBasis(ZZ).super_categories(), key=str)
[Category of coalgebras over Integer Ring,
 Category of modules with basis over Integer Ring]
```

class ElementMethods

Bases: object

coproduct_iterated(*n=1*)

Apply n coproducts to self.

Todo

Remove dependency on modules_with_basis methods.

EXAMPLES:

```
sage: Psi = NonCommutativeSymmetricFunctions(QQ).Psi()                    #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
sage: Psi[2,2].coproduct_iterated(0)                                    #␣
↪needs sage.combinat sage.modules
Psi[2, 2]
sage: Psi[2,2].coproduct_iterated(2)                                    #␣
↪needs sage.combinat sage.modules
Psi[] # Psi[] # Psi[2, 2] + 2*Psi[] # Psi[2] # Psi[2]
 + Psi[] # Psi[2, 2] # Psi[] + 2*Psi[2] # Psi[] # Psi[2]
 + 2*Psi[2] # Psi[2] # Psi[] + Psi[2, 2] # Psi[] # Psi[]
```

```
>>> from sage.all import *
>>> Psi = NonCommutativeSymmetricFunctions(QQ).Psi()                    #␣
↪needs sage.combinat sage.modules
>>> Psi[Integer(2),Integer(2)].coproduct_iterated(Integer(0))           ␣
↪                       # needs sage.combinat sage.modules
Psi[2, 2]
>>> Psi[Integer(2),Integer(2)].coproduct_iterated(Integer(2))           ␣
↪                       # needs sage.combinat sage.modules
Psi[] # Psi[] # Psi[2, 2] + 2*Psi[] # Psi[2] # Psi[2]
 + Psi[] # Psi[2, 2] # Psi[] + 2*Psi[2] # Psi[] # Psi[2]
 + 2*Psi[2] # Psi[2] # Psi[] + Psi[2, 2] # Psi[] # Psi[]
```

class Filtered(*base_category*)

Bases: *FilteredModulesCategory*

Category of filtered coalgebras.

Graded

alias of *GradedCoalgebrasWithBasis*

class ParentMethods

Bases: object

coproduct()

If *coproduct_on_basis()* is available, construct the coproduct morphism from `self` to `self` ⊗ `self` by extending it by linearity. Otherwise, use *coproduct_by_coercion()*, if available.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
sage: a, b = A.algebra_generators()
sage: a, A.coproduct(a)
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
sage: b, A.coproduct(b)
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
```

(continues on next page)

(continued from previous page)

```
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
>>> a, b = A.algebra_generators()
>>> a, A.coproduct(a)
(B[(1,2,3)], B[(1,2,3)] # B[(1,2,3)])
>>> b, A.coproduct(b)
(B[(1,3)], B[(1,3)] # B[(1,3)])
```

coproduct_on_basis(*i*)

The coproduct of the algebra on the basis (optional).

INPUT:

- `i` – the indices of an element of the basis of `self`

Returns the coproduct of the corresponding basis elements If implemented, the coproduct of the algebra is defined from it by linearity.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example(); A                                #␣
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
sage: a, b = A._group.gens()                                                    #␣
↪needs sage.groups sage.modules
sage: A.coproduct_on_basis(a)                                                   #␣
↪needs sage.groups sage.modules
B[(1,2,3)] # B[(1,2,3)]
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example(); A                                  #␣
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
>>> a, b = A._group.gens()                                                     #␣
↪needs sage.groups sage.modules
>>> A.coproduct_on_basis(a)                                                    #␣
↪needs sage.groups sage.modules
B[(1,2,3)] # B[(1,2,3)]
```

counit()

If *counit_on_basis()* is available, construct the counit morphism from `self` to `self` ⊗ `self` by extending it by linearity

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
sage: a, b = A.algebra_generators()
```

(continues on next page)

(continued from previous page)

```
sage: a, A.counit(a)
(B[(1,2,3)], 1)
sage: b, A.counit(b)
(B[(1,3)], 1)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
>>> a, b = A.algebra_generators()
>>> a, A.counit(a)
(B[(1,2,3)], 1)
>>> b, A.counit(b)
(B[(1,3)], 1)
```

counit_on_basis(*i*)

The counit of the algebra on the basis (optional).

INPUT:

- `i` – the indices of an element of the basis of `self`

Returns the counit of the corresponding basis elements If implemented, the counit of the algebra is defined from it by linearity.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example(); A                            #␣
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
sage: a, b = A._group.gens()                                                #␣
↪needs sage.groups sage.modules
sage: A.counit_on_basis(a)                                                  #␣
↪needs sage.groups sage.modules
1
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example(); A                              #␣
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
  as a permutation group over Rational Field
>>> a, b = A._group.gens()                                                  #␣
↪needs sage.groups sage.modules
>>> A.counit_on_basis(a)                                                    #␣
↪needs sage.groups sage.modules
1
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

extra_super_categories()

EXAMPLES:

```
sage: C = Coalgebras(ZZ).WithBasis().Super()
sage: sorted(C.super_categories(), key=str)  # indirect doctest
[Category of graded coalgebras with basis over Integer Ring,
 Category of super coalgebras over Integer Ring,
 Category of super modules with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> C = Coalgebras(ZZ).WithBasis().Super()
>>> sorted(C.super_categories(), key=str)  # indirect doctest
[Category of graded coalgebras with basis over Integer Ring,
 Category of super coalgebras over Integer Ring,
 Category of super modules with basis over Integer Ring]
```

4.21 Commutative additive groups

class sage.categories.commutative_additive_groups.**CommutativeAdditiveGroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*, *AbelianCategory*

The category of abelian groups, i.e. additive abelian monoids where each element has an inverse.

EXAMPLES:

```
sage: C = CommutativeAdditiveGroups(); C
Category of commutative additive groups
sage: C.super_categories()
[Category of additive groups, Category of commutative additive monoids]
sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse', 'AdditiveUnital
↪']
sage: C is CommutativeAdditiveMonoids().AdditiveInverse()
True
sage: from sage.categories.additive_groups import AdditiveGroups
sage: C is AdditiveGroups().AdditiveCommutative()
True
```

```
>>> from sage.all import *
>>> C = CommutativeAdditiveGroups(); C
Category of commutative additive groups
>>> C.super_categories()
[Category of additive groups, Category of commutative additive monoids]
>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse', 'AdditiveUnital
↪']
>>> C is CommutativeAdditiveMonoids().AdditiveInverse()
True
>>> from sage.categories.additive_groups import AdditiveGroups
>>> C is AdditiveGroups().AdditiveCommutative()
True
```

> **Note**
>
> This category is currently empty. It's left there for backward compatibility and because it is likely to grow in the future.

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ElementMethods

Bases: object

additive_order()

Return the additive order of this element.

EXAMPLES:

```
sage: G = cartesian_product([Zmod(3), Zmod(6), Zmod(5)])
sage: G((1,1,1)).additive_order()
30
sage: any((i * G((1,1,1))).is_zero() for i in range(1,30))
False
sage: 30 * G((1,1,1))
(0, 0, 0)

sage: G = cartesian_product([ZZ, ZZ])
sage: G((0,0)).additive_order()
1
sage: G((0,1)).additive_order()
+Infinity

sage: # needs sage.rings.finite_rings
sage: K = GF(9)
sage: H = cartesian_product([
....:     cartesian_product([Zmod(2), Zmod(9)]), K])
sage: z = H(((1,2), K.gen()))
sage: z.additive_order()
18
```

```
>>> from sage.all import *
>>> G = cartesian_product([Zmod(Integer(3)), Zmod(Integer(6)),␣
↪Zmod(Integer(5))])
>>> G((Integer(1),Integer(1),Integer(1))).additive_order()
30
>>> any((i * G((Integer(1),Integer(1),Integer(1)))).is_zero() for i in␣
↪range(Integer(1),Integer(30)))
False
>>> Integer(30) * G((Integer(1),Integer(1),Integer(1)))
(0, 0, 0)

>>> G = cartesian_product([ZZ, ZZ])
```

(continues on next page)

(continued from previous page)

```
>>> G((Integer(0),Integer(0))).additive_order()
1
>>> G((Integer(0),Integer(1))).additive_order()
+Infinity

>>> # needs sage.rings.finite_rings
>>> K = GF(Integer(9))
>>> H = cartesian_product([
...     cartesian_product([Zmod(Integer(2)), Zmod(Integer(9))]), K])
>>> z = H(((Integer(1),Integer(2)), K.gen()))
>>> z.additive_order()
18
```

4.22 Commutative additive monoids

class sage.categories.commutative_additive_monoids.**CommutativeAdditiveMonoids**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of commutative additive monoids, that is abelian additive semigroups with a unit

EXAMPLES:

```
sage: C = CommutativeAdditiveMonoids(); C
Category of commutative additive monoids
sage: C.super_categories()
[Category of additive monoids, Category of commutative additive semigroups]
sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveUnital']
sage: C is AdditiveMagmas().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital()
True
```

```
>>> from sage.all import *
>>> C = CommutativeAdditiveMonoids(); C
Category of commutative additive monoids
>>> C.super_categories()
[Category of additive monoids, Category of commutative additive semigroups]
>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveUnital']
>>> C is AdditiveMagmas().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital()
True
```

> **Note**
>
> This category is currently empty and only serves as a place holder to make `C.example()` work.

4.23 Commutative additive semigroups

class sage.categories.commutative_additive_semigroups.**CommutativeAdditiveSemigroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of additive abelian semigroups, i.e. sets with an associative and abelian operation +.

EXAMPLES:

```
sage: C = CommutativeAdditiveSemigroups(); C
Category of commutative additive semigroups
sage: C.example()
An example of a commutative semigroup: the free commutative semigroup generated␣
↪by ('a', 'b', 'c', 'd')

sage: sorted(C.super_categories(), key=str)
[Category of additive commutative additive magmas,
 Category of additive semigroups]
sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative']
sage: C is AdditiveMagmas().AdditiveAssociative().AdditiveCommutative()
True
```

```
>>> from sage.all import *
>>> C = CommutativeAdditiveSemigroups(); C
Category of commutative additive semigroups
>>> C.example()
An example of a commutative semigroup: the free commutative semigroup generated␣
↪by ('a', 'b', 'c', 'd')

>>> sorted(C.super_categories(), key=str)
[Category of additive commutative additive magmas,
 Category of additive semigroups]
>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative']
>>> C is AdditiveMagmas().AdditiveAssociative().AdditiveCommutative()
True
```

> **Note**
>
> This category is currently empty and only serves as a place holder to make `C.example()` work.

4.24 Commutative algebra ideals

class sage.categories.commutative_algebra_ideals.**CommutativeAlgebraIdeals**(A)

Bases: *Category_ideal*

The category of ideals in a fixed commutative algebra A.

EXAMPLES:

```
sage: C = CommutativeAlgebraIdeals(QQ['x'])
sage: C
Category of commutative algebra ideals in
 Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> C = CommutativeAlgebraIdeals(QQ['x'])
>>> C
Category of commutative algebra ideals in
 Univariate Polynomial Ring in x over Rational Field
```

algebra()

EXAMPLES:

```
sage: CommutativeAlgebraIdeals(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> CommutativeAlgebraIdeals(QQ['x']).algebra()
Univariate Polynomial Ring in x over Rational Field
```

super_categories()

EXAMPLES:

```
sage: CommutativeAlgebraIdeals(QQ['x']).super_categories()
[Category of algebra ideals in Univariate Polynomial Ring in x over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> CommutativeAlgebraIdeals(QQ['x']).super_categories()
[Category of algebra ideals in Univariate Polynomial Ring in x over Rational␣
↪Field]
```

4.25 Commutative algebras

class sage.categories.commutative_algebras.**CommutativeAlgebras**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of commutative algebras with unit over a given base ring.

EXAMPLES:

```
sage: M = CommutativeAlgebras(GF(19))
sage: M
Category of commutative algebras over Finite Field of size 19
sage: CommutativeAlgebras(QQ).super_categories()
[Category of algebras over Rational Field, Category of commutative rings]
```

```
>>> from sage.all import *
>>> M = CommutativeAlgebras(GF(Integer(19)))
>>> M
Category of commutative algebras over Finite Field of size 19
>>> CommutativeAlgebras(QQ).super_categories()
[Category of algebras over Rational Field, Category of commutative rings]
```

This is just a shortcut for:

```
sage: Algebras(QQ).Commutative()
Category of commutative algebras over Rational Field
```

```
>>> from sage.all import *
>>> Algebras(QQ).Commutative()
Category of commutative algebras over Rational Field
```

Todo

- product (= Cartesian product)
- coproduct (= tensor product over base ring)

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of commutative algebras constructed by tensor product of commutative algebras.

extra_super_categories()

EXAMPLES:

```
sage: Algebras(QQ).Commutative().TensorProducts().extra_super_categories()
[Category of commutative rings]
sage: Algebras(QQ).Commutative().TensorProducts().super_categories()
[Category of tensor products of algebras over Rational Field,
 Category of commutative algebras over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).Commutative().TensorProducts().extra_super_categories()
[Category of commutative rings]
>>> Algebras(QQ).Commutative().TensorProducts().super_categories()
[Category of tensor products of algebras over Rational Field,
 Category of commutative algebras over Rational Field]
```

4.26 Commutative ring ideals

class sage.categories.commutative_ring_ideals.**CommutativeRingIdeals**(R)

Bases: *Category_ideal*

The category of ideals in a fixed commutative ring.

EXAMPLES:

```
sage: C = CommutativeRingIdeals(IntegerRing())
sage: C
Category of commutative ring ideals in Integer Ring
```

```
>>> from sage.all import *
>>> C = CommutativeRingIdeals(IntegerRing())
>>> C
Category of commutative ring ideals in Integer Ring
```

super_categories()

EXAMPLES:

```
sage: CommutativeRingIdeals(ZZ).super_categories()
[Category of ring ideals in Integer Ring]
```

```
>>> from sage.all import *
>>> CommutativeRingIdeals(ZZ).super_categories()
[Category of ring ideals in Integer Ring]
```

4.27 Commutative rings

class sage.categories.commutative_rings.**CommutativeRings**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of commutative rings.

commutative rings with unity, i.e. rings with commutative * and a multiplicative identity

EXAMPLES:

```
sage: C = CommutativeRings(); C
Category of commutative rings
sage: C.super_categories()
[Category of rings, Category of commutative monoids]
```

```
>>> from sage.all import *
>>> C = CommutativeRings(); C
Category of commutative rings
>>> C.super_categories()
[Category of rings, Category of commutative monoids]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Let Sage knows that Cartesian products of commutative rings is a commutative ring.

EXAMPLES:

```
sage: CommutativeRings().Commutative().CartesianProducts().extra_super_
↪categories()
[Category of commutative rings]
sage: cartesian_product([ZZ, Zmod(34),
....:                   QQ, GF(5)]) in CommutativeRings()
True
```

```
>>> from sage.all import *
>>> CommutativeRings().Commutative().CartesianProducts().extra_super_
↪categories()
[Category of commutative rings]
>>> cartesian_product([ZZ, Zmod(Integer(34)),
...                    QQ, GF(Integer(5))]) in CommutativeRings()
True
```

class ElementMethods

Bases: object

is_square(*root=False*)

Return whether or not the ring element `self` is a square. If the optional argument root is `True`, then also return the square root (or `None`, if it is not a square).

INPUT:

- `root` – boolean (default: `False`); whether or not to also return a square root

OUTPUT:

- boolean; whether or not a square
- object; (optional) an actual square root if found, and `None` otherwise

EXAMPLES:

```
sage: R.<x> = PolynomialRing(QQ)
sage: f = 12*(x+1)^2 * (x+3)^2
sage: f.is_square()
False
sage: f.is_square(root=True)
(False, None)
sage: h = f/3
sage: h.is_square()
True
sage: h.is_square(root=True)
(True, 2*x^2 + 8*x + 6)
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> f = Integer(12)*(x+Integer(1))**Integer(2) *␣
↪(x+Integer(3))**Integer(2)
>>> f.is_square()
False
>>> f.is_square(root=True)
(False, None)
```

(continues on next page)

(continued from previous page)

```
>>> h = f/Integer(3)
>>> h.is_square()
True
>>> h.is_square(root=True)
(True, 2*x^2 + 8*x + 6)
```

Note

This is the is_square implementation for general commutative ring elements. It's implementation is to raise a `NotImplementedError`. The function definition is here to show what functionality is expected and provide a general framework.

sqrt (*extend=True*, *all=False*, *name=None*)

Compute the square root.

INPUT:

- `extend` – boolean (default: `True`); whether to make a ring extension containing a square root if `self` is not a square
- `all` – boolean (default: `False`); whether to return a list of all square roots or just a square root
- `name` – required when `extend=True` and `self` is not a square; this will be the name of the generator of the extension

OUTPUT:

- if `all=False`, a square root; raises an error if `extend=False` and `self` is not a square
- if `all=True`, a list of all the square roots (empty if `extend=False` and `self` is not a square)

ALGORITHM:

It uses `is_square(root=true)` for the hard part of the work, the rest is just wrapper code.

EXAMPLES:

```
sage: # needs sage.libs.pari
sage: R.<x> = ZZ[]
sage: (x^2).sqrt()
x
sage: f = x^2 - 4*x + 4; f.sqrt(all=True)
[x - 2, -x + 2]
sage: sqrtx = x.sqrt(name='y'); sqrtx
y
sage: sqrtx^2
x
sage: x.sqrt(all=true, name='y')
[y, -y]
sage: x.sqrt(extend=False, all=True)
[]
sage: x.sqrt()
Traceback (most recent call last):
...
TypeError: Polynomial is not a square. You must specify the name
of the square root when using the default extend = True
sage: x.sqrt(extend=False)
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: trying to take square root of non-square x with extend = False
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> (x**Integer(2)).sqrt()
x
>>> f = x**Integer(2) - Integer(4)*x + Integer(4); f.sqrt(all=True)
[x - 2, -x + 2]
>>> sqrtx = x.sqrt(name='y'); sqrtx
y
>>> sqrtx**Integer(2)
x
>>> x.sqrt(all=true, name='y')
[y, -y]
>>> x.sqrt(extend=False, all=True)
[]
>>> x.sqrt()
Traceback (most recent call last):
...
TypeError: Polynomial is not a square. You must specify the name
of the square root when using the default extend = True
>>> x.sqrt(extend=False)
Traceback (most recent call last):
...
ValueError: trying to take square root of non-square x with extend = False
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

Check that Sage knows that Cartesian products of finite commutative rings is a finite commutative ring.

EXAMPLES:

```
sage: cartesian_product([Zmod(34),
....:                    GF(5)]) in Rings().Commutative().Finite()
True
```

```
>>> from sage.all import *
>>> cartesian_product([Zmod(Integer(34)),
...                    GF(Integer(5))]) in Rings().Commutative().Finite()
True
```

class ParentMethods

Bases: object

cyclotomic_cosets(*q*, *cosets=None*)

Return the (multiplicative) orbits of `q` in the ring.

Let R be a finite commutative ring. The group of invertible elements R^* in R gives rise to a group action on R by multiplication. An orbit of the subgroup generated by an invertible element q is called a *q-cyclotomic coset* (since in a finite ring, each invertible element is a root of unity).

These cosets arise in the theory of minimal polynomials of finite fields, duadic codes and combinatorial designs. Fix a primitive element z of $GF(q^k)$. The minimal polynomial of z^s over $GF(q)$ is given by

$$M_s(x) = \prod_{i \in C_s} (x - z^i),$$

where C_s is the q-cyclotomic coset mod n containing s, $n = q^k - 1$.

Note

When $R = \mathbf{Z}/n\mathbf{Z}$ the smallest element of each coset is sometimes called a *coset leader*. This function returns sorted lists so that the coset leader will always be the first element of the coset.

INPUT:

- `q` – an invertible element of the ring
- `cosets` – an optional lists of elements of `self`. If provided, the function only return the list of cosets that contain some element from `cosets`.

OUTPUT: list of lists

EXAMPLES:

```
sage: Zmod(11).cyclotomic_cosets(2)
[[0], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]
sage: Zmod(15).cyclotomic_cosets(2)
[[0], [1, 2, 4, 8], [3, 6, 9, 12], [5, 10], [7, 11, 13, 14]]
```

```
>>> from sage.all import *
>>> Zmod(Integer(11)).cyclotomic_cosets(Integer(2))
[[0], [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]]
>>> Zmod(Integer(15)).cyclotomic_cosets(Integer(2))
[[0], [1, 2, 4, 8], [3, 6, 9, 12], [5, 10], [7, 11, 13, 14]]
```

Since the group of invertible elements of a finite field is cyclic, the set of squares is a particular case of cyclotomic coset:

```
sage: # needs sage.rings.finite_rings
sage: K = GF(25, 'z')
sage: a = K.multiplicative_generator()
sage: K.cyclotomic_cosets(a**2, cosets=[1])
[[1, 2, 3, 4, z + 1, z + 3,
  2*z + 1, 2*z + 2, 3*z + 3,
  3*z + 4, 4*z + 2, 4*z + 4]]
sage: sorted(b for b in K if not b.is_zero() and b.is_square())
[1, 2, 3, 4, z + 1, z + 3,
 2*z + 1, 2*z + 2, 3*z + 3,
 3*z + 4, 4*z + 2, 4*z + 4]
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> K = GF(Integer(25), 'z')
>>> a = K.multiplicative_generator()
>>> K.cyclotomic_cosets(a**Integer(2), cosets=[Integer(1)])
```

(continues on next page)

(continued from previous page)

```
[[1, 2, 3, 4, z + 1, z + 3,
  2*z + 1, 2*z + 2, 3*z + 3,
  3*z + 4, 4*z + 2, 4*z + 4]]
>>> sorted(b for b in K if not b.is_zero() and b.is_square())
[1, 2, 3, 4, z + 1, z + 3,
 2*z + 1, 2*z + 2, 3*z + 3,
 3*z + 4, 4*z + 2, 4*z + 4]
```

We compute some examples of minimal polynomials:

```
sage: # needs sage.rings.finite_rings
sage: K = GF(27, 'z')
sage: a = K.multiplicative_generator()
sage: R.<X> = PolynomialRing(K, 'X')
sage: a.minimal_polynomial('X')
X^3 + 2*X + 1

sage: cyc3 = Zmod(26).cyclotomic_cosets(3, cosets=[1]); cyc3
[[1, 3, 9]]
sage: prod(X - a**i for i in cyc3[0])                                   #␣
↪needs sage.rings.finite_rings
X^3 + 2*X + 1
sage: (a**7).minimal_polynomial('X')                                    #␣
↪needs sage.rings.finite_rings
X^3 + X^2 + 2*X + 1
sage: cyc7 = Zmod(26).cyclotomic_cosets(3, cosets=[7]); cyc7
[[7, 11, 21]]
sage: prod(X - a**i for i in cyc7[0])                                   #␣
↪needs sage.rings.finite_rings
X^3 + X^2 + 2*X + 1
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> K = GF(Integer(27), 'z')
>>> a = K.multiplicative_generator()
>>> R = PolynomialRing(K, 'X', names=('X',)); (X,) = R._first_ngens(1)
>>> a.minimal_polynomial('X')
X^3 + 2*X + 1

>>> cyc3 = Zmod(Integer(26)).cyclotomic_cosets(Integer(3),␣
↪cosets=[Integer(1)]); cyc3
[[1, 3, 9]]
>>> prod(X - a**i for i in cyc3[Integer(0)])                            ␣
↪    # needs sage.rings.finite_rings
X^3 + 2*X + 1
>>> (a**Integer(7)).minimal_polynomial('X')                             ␣
↪    # needs sage.rings.finite_rings
X^3 + X^2 + 2*X + 1
>>> cyc7 = Zmod(Integer(26)).cyclotomic_cosets(Integer(3),␣
↪cosets=[Integer(7)]); cyc7
[[7, 11, 21]]
>>> prod(X - a**i for i in cyc7[Integer(0)])                            ␣
```

(continues on next page)

(continued from previous page)

```
↪    # needs sage.rings.finite_rings
X^3 + X^2 + 2*X + 1
```

Cyclotomic cosets of fields are useful in combinatorial design theory to provide so called difference families (see Wikipedia article Difference_set and `difference_family`). This is illustrated on the following examples:

```
sage: K = GF(5)
sage: a = K.multiplicative_generator()                                  #␣
↪needs sage.libs.pari
sage: H = K.cyclotomic_cosets(a**2, cosets=[1, 2]); H                   #␣
↪needs sage.rings.finite_rings
[[1, 4], [2, 3]]
sage: sorted(x - y for D in H for x in D for y in D if x != y)          #␣
↪needs sage.rings.finite_rings
[1, 2, 3, 4]

sage: K = GF(37)
sage: a = K.multiplicative_generator()                                  #␣
↪needs sage.libs.pari
sage: H = K.cyclotomic_cosets(a**4, cosets=[1]); H                      #␣
↪needs sage.rings.finite_rings
[[1, 7, 9, 10, 12, 16, 26, 33, 34]]
sage: sorted(x - y for D in H for x in D for y in D if x != y)          #␣
↪needs sage.rings.finite_rings
[1, 1, 2, 2, 3, 3, 4, 4, 5, 5, ..., 33, 34, 34, 35, 35, 36, 36]
```

```
>>> from sage.all import *
>>> K = GF(Integer(5))
>>> a = K.multiplicative_generator()                                    #␣
↪needs sage.libs.pari
>>> H = K.cyclotomic_cosets(a**Integer(2), cosets=[Integer(1),␣
↪Integer(2)]); H               # needs sage.rings.finite_rings
[[1, 4], [2, 3]]
>>> sorted(x - y for D in H for x in D for y in D if x != y)          #␣
↪needs sage.rings.finite_rings
[1, 2, 3, 4]

>>> K = GF(Integer(37))
>>> a = K.multiplicative_generator()                                    #␣
↪needs sage.libs.pari
>>> H = K.cyclotomic_cosets(a**Integer(4), cosets=[Integer(1)]); H    ␣
↪             # needs sage.rings.finite_rings
[[1, 7, 9, 10, 12, 16, 26, 33, 34]]
>>> sorted(x - y for D in H for x in D for y in D if x != y)          #␣
↪needs sage.rings.finite_rings
[1, 1, 2, 2, 3, 3, 4, 4, 5, 5, ..., 33, 34, 34, 35, 35, 36, 36]
```

The method `cyclotomic_cosets` works on any finite commutative ring:

```
sage: R = cartesian_product([GF(7), Zmod(14)])
sage: a = R((3,5))
```

(continues on next page)

(continued from previous page)

```
sage: R.cyclotomic_cosets((3,5), [(1,1)])
[[(1, 1), (2, 11), (3, 5), (4, 9), (5, 3), (6, 13)]]
```

```
>>> from sage.all import *
>>> R = cartesian_product([GF(Integer(7)), Zmod(Integer(14))])
>>> a = R((Integer(3),Integer(5)))
>>> R.cyclotomic_cosets((Integer(3),Integer(5)), [(Integer(1),
↪Integer(1))])
[[(1, 1), (2, 11), (3, 5), (4, 9), (5, 3), (6, 13)]]
```

extra_super_categories()

Let Sage know that finite commutative rings are Noetherian.

EXAMPLES:

```
sage: CommutativeRings().Finite().extra_super_categories()
[Category of noetherian rings]
```

```
>>> from sage.all import *
>>> CommutativeRings().Finite().extra_super_categories()
[Category of noetherian rings]
```

class ParentMethods

Bases: object

derivation(*arg=None*, *twist=None*)

Return the twisted or untwisted derivation over this ring specified by `arg`.

Note

A twisted derivation with respect to θ (or a θ-derivation for short) is an additive map d satisfying the following axiom for all x, y in the domain:

$$d(xy) = \theta(x)d(y) + d(x)y.$$

INPUT:

- `arg` – (optional) a generator or a list of coefficients that defines the derivation
- `twist` – (optional) the twisting homomorphism

EXAMPLES:

```
sage: R.<x,y,z> = QQ[]
sage: R.derivation()
↪  # needs sage.modules
d/dx
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> R.derivation()
↪# needs sage.modules
d/dx
```

In that case, `arg` could be a generator:

```
sage: R.derivation(y)
↪  # needs sage.modules
d/dy
```

```
>>> from sage.all import *
>>> R.derivation(y)
↪# needs sage.modules
d/dy
```

or a list of coefficients:

```
sage: R.derivation([1,2,3])
↪  # needs sage.modules
d/dx + 2*d/dy + 3*d/dz
```

```
>>> from sage.all import *
>>> R.derivation([Integer(1),Integer(2),Integer(3)])
↪                              # needs sage.modules
d/dx + 2*d/dy + 3*d/dz
```

It is not possible to define derivations with respect to a polynomial which is not a variable:

```
sage: R.derivation(x^2)
↪  # needs sage.modules
Traceback (most recent call last):
...
ValueError: unable to create the derivation
```

```
>>> from sage.all import *
>>> R.derivation(x**Integer(2))
↪           # needs sage.modules
Traceback (most recent call last):
...
ValueError: unable to create the derivation
```

Here is an example with twisted derivations:

```
sage: R.<x,y,z> = QQ[]
sage: theta = R.hom([x^2, y^2, z^2])
sage: f = R.derivation(twist=theta); f
↪  # needs sage.modules
0
sage: f.parent()
↪  # needs sage.modules
Module of twisted derivations over Multivariate Polynomial Ring in x, y, z
 over Rational Field (twisting morphism: x |--> x^2, y |--> y^2, z |--> z^
↪2)
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> theta = R.hom([x**Integer(2), y**Integer(2), z**Integer(2)])
>>> f = R.derivation(twist=theta); f
```

(continues on next page)

(continued from previous page)

```
↪# needs sage.modules
0
>>> f.parent()
↪# needs sage.modules
Module of twisted derivations over Multivariate Polynomial Ring in x, y, z
 over Rational Field (twisting morphism: x |--> x^2, y |--> y^2, z |--> z^
↪2)
```

Specifying a scalar, the returned twisted derivation is the corresponding multiple of $\theta - id$:

```
sage: R.derivation(1, twist=theta)                                          ␣
↪  # needs sage.modules
[x |--> x^2, y |--> y^2, z |--> z^2] - id
sage: R.derivation(x, twist=theta)                                          ␣
↪  # needs sage.modules
x*([x |--> x^2, y |--> y^2, z |--> z^2] - id)
```

```
>>> from sage.all import *
>>> R.derivation(Integer(1), twist=theta)                                   ␣
↪         # needs sage.modules
[x |--> x^2, y |--> y^2, z |--> z^2] - id
>>> R.derivation(x, twist=theta)
↪# needs sage.modules
x*([x |--> x^2, y |--> y^2, z |--> z^2] - id)
```

derivation_module(*codomain=None*, *twist=None*)

Return the module of derivations over this ring.

INPUT:

- `codomain` – an algebra over this ring or a ring homomorphism whose domain is this ring or `None` (default: `None`); if it is a morphism, the codomain of derivations will be the codomain of the morphism viewed as an algebra over `self` through the given morphism; if `None`, the codomain will be this ring
- `twist` – a morphism from this ring to `codomain` or `None` (default: `None`); if `None`, the coercion map from this ring to `codomain` will be used

Note

A twisted derivation with respect to θ (or a θ-derivation for short) is an additive map d satisfying the following axiom for all x, y in the domain:

$$d(xy) = \theta(x)d(y) + d(x)y.$$

EXAMPLES:

```
sage: R.<x,y,z> = QQ[]
sage: M = R.derivation_module(); M                                          ␣
↪  # needs sage.modules
Module of derivations over
 Multivariate Polynomial Ring in x, y, z over Rational Field
sage: M.gens()                                                              ␣
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.modules
(d/dx, d/dy, d/dz)
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> M = R.derivation_module(); M
↪# needs sage.modules
Module of derivations over
 Multivariate Polynomial Ring in x, y, z over Rational Field
>>> M.gens()
↪# needs sage.modules
(d/dx, d/dy, d/dz)
```

We can specify a different codomain:

```
sage: K = R.fraction_field()
sage: M = R.derivation_module(K); M                                             ␣
↪  # needs sage.modules
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Fraction Field of
      Multivariate Polynomial Ring in x, y, z over Rational Field
sage: M.gen() / x                                                               ␣
↪  # needs sage.modules
1/x*d/dx
```

```
>>> from sage.all import *
>>> K = R.fraction_field()
>>> M = R.derivation_module(K); M
↪# needs sage.modules
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Fraction Field of
      Multivariate Polynomial Ring in x, y, z over Rational Field
>>> M.gen() / x
↪# needs sage.modules
1/x*d/dx
```

Here is an example with a non-canonical defining morphism:

```
sage: ev = R.hom([QQ(0), QQ(1), QQ(2)])
sage: ev
Ring morphism:
  From: Multivariate Polynomial Ring in x, y, z over Rational Field
  To:   Rational Field
  Defn: x |--> 0
        y |--> 1
        z |--> 2
sage: M = R.derivation_module(ev)                                               ␣
↪  # needs sage.modules
sage: M                                                                         ␣
↪  # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Rational Field
```

```
>>> from sage.all import *
>>> ev = R.hom([QQ(Integer(0)), QQ(Integer(1)), QQ(Integer(2))])
>>> ev
Ring morphism:
  From: Multivariate Polynomial Ring in x, y, z over Rational Field
  To:   Rational Field
  Defn: x |--> 0
        y |--> 1
        z |--> 2
>>> M = R.derivation_module(ev)
↪# needs sage.modules
>>> M
↪# needs sage.modules
Module of derivations
 from Multivariate Polynomial Ring in x, y, z over Rational Field
   to Rational Field
```

Elements in M acts as derivations at $(0, 1, 2)$:

```
sage: # needs sage.modules
sage: Dx = M.gen(0); Dx
d/dx
sage: Dy = M.gen(1); Dy
d/dy
sage: Dz = M.gen(2); Dz
d/dz
sage: f = x^2 + y^2 + z^2
sage: Dx(f)  # = 2*x evaluated at (0,1,2)
0
sage: Dy(f)  # = 2*y evaluated at (0,1,2)
2
sage: Dz(f)  # = 2*z evaluated at (0,1,2)
4
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Dx = M.gen(Integer(0)); Dx
d/dx
>>> Dy = M.gen(Integer(1)); Dy
d/dy
>>> Dz = M.gen(Integer(2)); Dz
d/dz
>>> f = x**Integer(2) + y**Integer(2) + z**Integer(2)
>>> Dx(f)  # = 2*x evaluated at (0,1,2)
0
>>> Dy(f)  # = 2*y evaluated at (0,1,2)
2
>>> Dz(f)  # = 2*z evaluated at (0,1,2)
```

(continues on next page)

(continued from previous page)

```
4
```

An example with a twisting homomorphism:

```
sage: theta = R.hom([x^2, y^2, z^2])
sage: M = R.derivation_module(twist=theta); M                                  ␣
↪  # needs sage.modules
Module of twisted derivations over Multivariate Polynomial Ring in x, y, z
 over Rational Field (twisting morphism: x |--> x^2, y |--> y^2, z |--> z^
↪2)
```

```
>>> from sage.all import *
>>> theta = R.hom([x**Integer(2), y**Integer(2), z**Integer(2)])
>>> M = R.derivation_module(twist=theta); M
↪# needs sage.modules
Module of twisted derivations over Multivariate Polynomial Ring in x, y, z
 over Rational Field (twisting morphism: x |--> x^2, y |--> y^2, z |--> z^
↪2)
```

See also

derivation()

extension(*poly*, *name=None*, *names=None*, ***kwds*)

Algebraically extend `self` by taking the quotient `self[x] / (poly(x))`.

INPUT:

- `poly` – a polynomial whose coefficients are coercible into `self`
- `name` – (optional) name for the root of `poly`

Note

Using this method on an algebraically complete field k does *not* return the field k; the construction `self[x] / (poly(x))` is done anyway.

EXAMPLES:

```
sage: R = QQ['x']
sage: y = polygen(R)
sage: R.extension(y^2 - 5, 'a')                                                ␣
↪  # needs sage.libs.pari
Univariate Quotient Polynomial Ring in a over
 Univariate Polynomial Ring in x over Rational Field with modulus a^2 - 5
```

```
>>> from sage.all import *
>>> R = QQ['x']
>>> y = polygen(R)
>>> R.extension(y**Integer(2) - Integer(5), 'a')                               ␣
↪                    # needs sage.libs.pari
Univariate Quotient Polynomial Ring in a over
 Univariate Polynomial Ring in x over Rational Field with modulus a^2 - 5
```

```
sage: # needs sage.rings.finite_rings
sage: P.<x> = PolynomialRing(GF(5))
sage: F.<a> = GF(5).extension(x^2 - 2)
sage: P.<t> = F[]
sage: R.<b> = F.extension(t^2 - a); R
Univariate Quotient Polynomial Ring in b over
 Finite Field in a of size 5^2 with modulus b^2 + 4*a

sage: R.<t> = QQ[]
sage: Integers(8).extension(t^2 - 3)
Univariate Quotient Polynomial Ring in t
 over Ring of integers modulo 8 with modulus t^2 + 5
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> P = PolynomialRing(GF(Integer(5)), names=('x',)); (x,) = P._first_
↪ngens(1)
>>> F = GF(Integer(5)).extension(x**Integer(2) - Integer(2), names=('a',
↪)); (a,) = F._first_ngens(1)
>>> P = F['t']; (t,) = P._first_ngens(1)
>>> R = F.extension(t**Integer(2) - a, names=('b',)); (b,) = R._first_
↪ngens(1); R
Univariate Quotient Polynomial Ring in b over
 Finite Field in a of size 5^2 with modulus b^2 + 4*a

>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> Integers(Integer(8)).extension(t**Integer(2) - Integer(3))
Univariate Quotient Polynomial Ring in t
 over Ring of integers modulo 8 with modulus t^2 + 5
```

fraction_field()

Return the fraction field of `self`.

EXAMPLES:

```
sage: R = Integers(389)['x,y']
sage: Frac(R)
Fraction Field of Multivariate Polynomial Ring in x, y over Ring of␣
↪integers modulo 389
sage: R.fraction_field()
Fraction Field of Multivariate Polynomial Ring in x, y over Ring of␣
↪integers modulo 389
```

```
>>> from sage.all import *
>>> R = Integers(Integer(389))['x,y']
>>> Frac(R)
Fraction Field of Multivariate Polynomial Ring in x, y over Ring of␣
↪integers modulo 389
>>> R.fraction_field()
Fraction Field of Multivariate Polynomial Ring in x, y over Ring of␣
↪integers modulo 389
```

frobenius_endomorphism(*n=1*)

Return the Frobenius endomorphism.

INPUT:

- `n` – nonnegative integer (default: 1)

OUTPUT:

The n-th power of the absolute arithmetic Frobenius endomorphism on this commutative ring.

EXAMPLES:

```
sage: K.<u> = PowerSeriesRing(GF(5))
sage: Frob = K.frobenius_endomorphism(); Frob
Frobenius endomorphism x |--> x^5 of Power Series Ring in u
 over Finite Field of size 5
sage: Frob(u)
u^5
```

```
>>> from sage.all import *
>>> K = PowerSeriesRing(GF(Integer(5)), names=('u',)); (u,) = K._first_
↪ngens(1)
>>> Frob = K.frobenius_endomorphism(); Frob
Frobenius endomorphism x |--> x^5 of Power Series Ring in u
 over Finite Field of size 5
>>> Frob(u)
u^5
```

We can specify a power:

```
sage: f = K.frobenius_endomorphism(2); f
Frobenius endomorphism x |--> x^(5^2) of Power Series Ring in u
 over Finite Field of size 5
sage: f(1+u)
1 + u^25
```

```
>>> from sage.all import *
>>> f = K.frobenius_endomorphism(Integer(2)); f
Frobenius endomorphism x |--> x^(5^2) of Power Series Ring in u
 over Finite Field of size 5
>>> f(Integer(1)+u)
1 + u^25
```

is_commutative()

Return whether the ring is commutative.

The answer is `True` only if the category is a sub-category of `CommutativeRings`.

It is recommended to use instead `R in Rings().Commutative()`.

EXAMPLES:

```
sage: QQ.is_commutative()
True
sage: QQ['x,y,z'].is_commutative()
True
```

```
>>> from sage.all import *
>>> QQ.is_commutative()
True
>>> QQ['x,y,z'].is_commutative()
True
```

krull_dimension()

Return the Krull dimension of this commutative ring.

The Krull dimension is the length of the longest ascending chain of prime ideals.

This raises `NotImplementedError` by default for generic commutative rings.

Fields and PIDs, with Krull dimension equal to 0 and 1, respectively, have naive implementations of `krull_dimension`. Orders in number fields also have Krull dimension 1.

EXAMPLES:

Some polynomial rings:

```
sage: T.<x,y> = PolynomialRing(QQ,2); T
Multivariate Polynomial Ring in x, y over Rational Field
sage: T.krull_dimension()
2
sage: U.<x,y,z> = PolynomialRing(ZZ,3); U
Multivariate Polynomial Ring in x, y, z over Integer Ring
sage: U.krull_dimension()
4
```

```
>>> from sage.all import *
>>> T = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = T._
↪first_ngens(2); T
Multivariate Polynomial Ring in x, y over Rational Field
>>> T.krull_dimension()
2
>>> U = PolynomialRing(ZZ,Integer(3), names=('x', 'y', 'z',)); (x, y, z,)␣
↪= U._first_ngens(3); U
Multivariate Polynomial Ring in x, y, z over Integer Ring
>>> U.krull_dimension()
4
```

All orders in number fields have Krull dimension 1, including non-maximal orders:

```
sage: # needs sage.rings.number_field
sage: K.<i> = QuadraticField(-1)
sage: R = K.order(2*i); R
Order of conductor 2 generated by 2*i
 in Number Field in i with defining polynomial x^2 + 1 with i = 1*I
sage: R.is_maximal()
False
sage: R.krull_dimension()
1
sage: R = K.maximal_order(); R
Gaussian Integers generated by i in Number Field in i
 with defining polynomial x^2 + 1 with i = 1*I
```

(continues on next page)

(continued from previous page)

```
sage: R.krull_dimension()
1
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> K = QuadraticField(-Integer(1), names=('i',)); (i,) = K._first_
↪ngens(1)
>>> R = K.order(Integer(2)*i); R
Order of conductor 2 generated by 2*i
 in Number Field in i with defining polynomial x^2 + 1 with i = 1*I
>>> R.is_maximal()
False
>>> R.krull_dimension()
1
>>> R = K.maximal_order(); R
Gaussian Integers generated by i in Number Field in i
 with defining polynomial x^2 + 1 with i = 1*I
>>> R.krull_dimension()
1
```

over(*base=None*, *gen=None*, *gens=None*, *name=None*, *names=None*)

Return this ring, considered as an extension of `base`.

INPUT:

- `base` – a commutative ring or a morphism or `None` (default: `None`); the base of this extension or its defining morphism
- `gen` – a generator of this extension (over its base) or `None` (default: `None`)
- `gens` – list of generators of this extension (over its base) or `None` (default: `None`)
- `name` – a variable name or `None` (default: `None`)
- `names` – list or a tuple of variable names or `None` (default: `None`)

EXAMPLES:

We construct an extension of finite fields:

```
sage: # needs sage.rings.finite_rings
sage: F = GF(5^2)
sage: k = GF(5^4)
sage: z4 = k.gen()
sage: K = k.over(F); K                                                  #␣
↪needs sage.modules
Field in z4 with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
```

```
>>> from sage.all import *
>>> # needs sage.rings.finite_rings
>>> F = GF(Integer(5)**Integer(2))
>>> k = GF(Integer(5)**Integer(4))
>>> z4 = k.gen()
>>> K = k.over(F); K                                                  #␣
↪needs sage.modules
Field in z4 with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
```

If not explicitly given, the default generator of the top ring (here k) is used and the same name is kept:

```
sage: K.gen()                                                               #␣
↪needs sage.modules sage.rings.finite_rings
z4
sage: K(z4)                                                                 #␣
↪needs sage.modules sage.rings.finite_rings
z4
```

```
>>> from sage.all import *
>>> K.gen()                                                                 #␣
↪needs sage.modules sage.rings.finite_rings
z4
>>> K(z4)                                                                   #␣
↪needs sage.modules sage.rings.finite_rings
z4
```

However, it is possible to specify another generator and/or another name. For example:

```
sage: # needs sage.modules sage.rings.finite_rings
sage: Ka = k.over(F, name='a'); Ka
Field in a with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
sage: Ka.gen()
a
sage: Ka(z4)
a

sage: # needs sage.modules sage.rings.finite_rings
sage: Kb = k.over(F, gen=-z4+1, name='b')
sage: Kb
Field in b with defining polynomial x^2 + z2*x + 4 over its base
sage: Kb.gen()
b
sage: Kb(-z4+1)
b
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.rings.finite_rings
>>> Ka = k.over(F, name='a'); Ka
Field in a with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
>>> Ka.gen()
a
>>> Ka(z4)
a

>>> # needs sage.modules sage.rings.finite_rings
>>> Kb = k.over(F, gen=-z4+Integer(1), name='b')
>>> Kb
Field in b with defining polynomial x^2 + z2*x + 4 over its base
>>> Kb.gen()
b
```

(continues on next page)

(continued from previous page)

```
>>> Kb(-z4+Integer(1))
b
```

Note that the shortcut `K.<a>` is also available:

```
sage: KKa.<a> = k.over(F)                                                   #␣
↪needs sage.modules sage.rings.finite_rings
sage: KKa is Ka                                                             #␣
↪needs sage.modules sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> KKa = k.over(F, names=('a',)); (a,) = KKa._first_ngens(1)# needs sage.
↪modules sage.rings.finite_rings
>>> KKa is Ka                                                               #␣
↪needs sage.modules sage.rings.finite_rings
True
```

Building an extension on top of another extension is allowed:

```
sage: L = GF(5^12).over(K); L                                               #␣
↪needs sage.modules sage.rings.finite_rings
Field in z12 with defining polynomial
 x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
sage: L.base_ring()                                                         #␣
↪needs sage.modules sage.rings.finite_rings
Field in z4 with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
```

```
>>> from sage.all import *
>>> L = GF(Integer(5)**Integer(12)).over(K); L                              ␣
↪               # needs sage.modules sage.rings.finite_rings
Field in z12 with defining polynomial
 x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base
>>> L.base_ring()                                                           #␣
↪needs sage.modules sage.rings.finite_rings
Field in z4 with defining polynomial
 x^2 + (4*z2 + 3)*x + z2 over its base
```

The successive bases of an extension are accessible via the method `sage.rings.ring_extension.RingExtension_generic.bases()`:

```
sage: L.bases()                                                             #␣
↪needs sage.modules sage.rings.finite_rings
[Field in z12 with defining polynomial
  x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base,
 Field in z4 with defining polynomial
  x^2 + (4*z2 + 3)*x + z2 over its base,
 Finite Field in z2 of size 5^2]
```

```
>>> from sage.all import *
>>> L.bases()                                                               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules sage.rings.finite_rings
[Field in z12 with defining polynomial
  x^3 + (1 + (4*z2 + 2)*z4)*x^2 + (2 + 2*z4)*x - z4 over its base,
 Field in z4 with defining polynomial
  x^2 + (4*z2 + 3)*x + z2 over its base,
 Finite Field in z2 of size 5^2]
```

When `base` is omitted, the canonical base of the ring is used:

```
sage: S.<x> = QQ[]
sage: E = S.over(); E                                                    #␣
↪needs sage.modules
Univariate Polynomial Ring in x over Rational Field over its base
sage: E.base_ring()                                                      #␣
↪needs sage.modules
Rational Field
```

```
>>> from sage.all import *
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> E = S.over(); E                                                      #␣
↪needs sage.modules
Univariate Polynomial Ring in x over Rational Field over its base
>>> E.base_ring()                                                        #␣
↪needs sage.modules
Rational Field
```

Here is an example where `base` is a defining morphism:

```
sage: # needs sage.modules sage.rings.number_field
sage: k.<a> = QQ.extension(x^2 - 2)
sage: l.<b> = QQ.extension(x^4 - 2)
sage: f = k.hom([b^2])
sage: L = l.over(f)
sage: L
Field in b with defining polynomial x^2 - a over its base
sage: L.base_ring()
Number Field in a with defining polynomial x^2 - 2
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.rings.number_field
>>> k = QQ.extension(x**Integer(2) - Integer(2), names=('a',)); (a,) = k._
↪first_ngens(1)
>>> l = QQ.extension(x**Integer(4) - Integer(2), names=('b',)); (b,) = l._
↪first_ngens(1)
>>> f = k.hom([b**Integer(2)])
>>> L = l.over(f)
>>> L
Field in b with defining polynomial x^2 - a over its base
>>> L.base_ring()
Number Field in a with defining polynomial x^2 - 2
```

Similarly, one can create a tower of extensions:

```
sage: # needs sage.modules sage.rings.number_field
sage: K = k.over()
sage: L = l.over(Hom(K, l)(f)); L
Field in b with defining polynomial x^2 - a over its base
sage: L.base_ring()
Field in a with defining polynomial x^2 - 2 over its base
sage: L.bases()
[Field in b with defining polynomial x^2 - a over its base,
 Field in a with defining polynomial x^2 - 2 over its base,
 Rational Field]
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.rings.number_field
>>> K = k.over()
>>> L = l.over(Hom(K, l)(f)); L
Field in b with defining polynomial x^2 - a over its base
>>> L.base_ring()
Field in a with defining polynomial x^2 - 2 over its base
>>> L.bases()
[Field in b with defining polynomial x^2 - a over its base,
 Field in a with defining polynomial x^2 - 2 over its base,
 Rational Field]
```

4.28 Complete Discrete Valuation Rings (CDVR) and Fields (CDVF)

class sage.categories.complete_discrete_valuation.**CompleteDiscreteValuationFields**

Bases: *Category_singleton*

The category of complete discrete valuation fields.

EXAMPLES:

```
sage: Zp(7) in CompleteDiscreteValuationFields()                                #␣
↪needs sage.rings.padics
False
sage: QQ in CompleteDiscreteValuationFields()
False
sage: LaurentSeriesRing(QQ, 'u') in CompleteDiscreteValuationFields()
True
sage: Qp(7) in CompleteDiscreteValuationFields()                                #␣
↪needs sage.rings.padics
True
sage: TestSuite(CompleteDiscreteValuationFields()).run()
```

```
>>> from sage.all import *
>>> Zp(Integer(7)) in CompleteDiscreteValuationFields()                         ␣
↪     # needs sage.rings.padics
False
>>> QQ in CompleteDiscreteValuationFields()
False
>>> LaurentSeriesRing(QQ, 'u') in CompleteDiscreteValuationFields()
True
```

(continues on next page)

(continued from previous page)

```
>>> Qp(Integer(7)) in CompleteDiscreteValuationFields()                    ␣
↪     # needs sage.rings.padics
True
>>> TestSuite(CompleteDiscreteValuationFields()).run()
```

class ElementMethods

Bases: object

denominator()

Return the denominator of this element normalized as a power of the uniformizer

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: K = Qp(7)
sage: x = K(1/21)
sage: x.denominator()
7 + O(7^21)
sage: x = K(7)
sage: x.denominator()
1 + O(7^20)
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> K = Qp(Integer(7))
>>> x = K(Integer(1)/Integer(21))
>>> x.denominator()
7 + O(7^21)
>>> x = K(Integer(7))
>>> x.denominator()
1 + O(7^20)
```

Note that the denominator lives in the ring of integers:

```
sage: x.denominator().parent()                                              #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> x.denominator().parent()                                                #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 20
```

When the denominator is indistinguishable from 0 and the precision on the input is $O(p^n)$, the return value is 1 if n is nonnegative and $p^(-n)$ otherwise:

```
sage: # needs sage.rings.padics
sage: x = K(0, 5); x
O(7^5)
sage: x.denominator()
1 + O(7^20)
sage: x = K(0, -5); x
O(7^-5)
```

(continues on next page)

(continued from previous page)

```
sage: x.denominator()
7^5 + O(7^25)
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> x = K(Integer(0), Integer(5)); x
O(7^5)
>>> x.denominator()
1 + O(7^20)
>>> x = K(Integer(0), -Integer(5)); x
O(7^-5)
>>> x.denominator()
7^5 + O(7^25)
```

numerator()

Return the numerator of this element, normalized in such a way that $x = x.numerator()/x.denominator()$ always holds true.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: K = Qp(7, 5)
sage: x = K(1/21)
sage: x.numerator()
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + O(7^5)
sage: x == x.numerator() / x.denominator()
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> K = Qp(Integer(7), Integer(5))
>>> x = K(Integer(1)/Integer(21))
>>> x.numerator()
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + O(7^5)
>>> x == x.numerator() / x.denominator()
True
```

Note that the numerator lives in the ring of integers:

```
sage: x.numerator().parent()                                                #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 5
```

```
>>> from sage.all import *
>>> x.numerator().parent()                                                  #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 5
```

valuation()

Return the valuation of this element.

EXAMPLES:

```
sage: K = Qp(7)                                                          #␣
↪needs sage.rings.padics
sage: x = K(7); x                                                        #␣
↪needs sage.rings.padics
7 + O(7^21)
sage: x.valuation()                                                      #␣
↪needs sage.rings.padics
1
```

```
>>> from sage.all import *
>>> K = Qp(Integer(7))                                                   ␣
↪     # needs sage.rings.padics
>>> x = K(Integer(7)); x                                                ␣
↪     # needs sage.rings.padics
7 + O(7^21)
>>> x.valuation()                                                      #␣
↪needs sage.rings.padics
1
```

super_categories()

EXAMPLES:

```
sage: CompleteDiscreteValuationFields().super_categories()
[Category of discrete valuation fields]
```

```
>>> from sage.all import *
>>> CompleteDiscreteValuationFields().super_categories()
[Category of discrete valuation fields]
```

class sage.categories.complete_discrete_valuation.**CompleteDiscreteValuationRings**

Bases: *Category_singleton*

The category of complete discrete valuation rings.

EXAMPLES:

```
sage: Zp(7) in CompleteDiscreteValuationRings()                          #␣
↪needs sage.rings.padics
True
sage: QQ in CompleteDiscreteValuationRings()
False
sage: QQ[['u']] in CompleteDiscreteValuationRings()
True
sage: Qp(7) in CompleteDiscreteValuationRings()                          #␣
↪needs sage.rings.padics
False
sage: TestSuite(CompleteDiscreteValuationRings()).run()
```

```
>>> from sage.all import *
>>> Zp(Integer(7)) in CompleteDiscreteValuationRings()                   ␣
↪     # needs sage.rings.padics
True
>>> QQ in CompleteDiscreteValuationRings()
```

(continues on next page)

(continued from previous page)

```
False
>>> QQ[['u']] in CompleteDiscreteValuationRings()
True
>>> Qp(Integer(7)) in CompleteDiscreteValuationRings()
↪     # needs sage.rings.padics
False
>>> TestSuite(CompleteDiscreteValuationRings()).run()
```

class ElementMethods

Bases: object

denominator()

Return the denominator of this element normalized as a power of the uniformizer

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: K = Qp(7)
sage: x = K(1/21)
sage: x.denominator()
7 + O(7^21)
sage: x = K(7)
sage: x.denominator()
1 + O(7^20)
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> K = Qp(Integer(7))
>>> x = K(Integer(1)/Integer(21))
>>> x.denominator()
7 + O(7^21)
>>> x = K(Integer(7))
>>> x.denominator()
1 + O(7^20)
```

Note that the denominator lives in the ring of integers:

```
sage: x.denominator().parent()                                              #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 20
```

```
>>> from sage.all import *
>>> x.denominator().parent()                                                #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 20
```

When the denominator is indistinguishable from 0 and the precision on the input is $O(p^n)$, the return value is 1 if n is nonnegative and $p^(- n)$ otherwise:

```
sage: # needs sage.rings.padics
sage: x = K(0, 5); x
O(7^5)
sage: x.denominator()
```

(continues on next page)

(continued from previous page)

```
1 + O(7^20)
sage: x = K(0, -5); x
O(7^-5)
sage: x.denominator()
7^5 + O(7^25)
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> x = K(Integer(0), Integer(5)); x
O(7^5)
>>> x.denominator()
1 + O(7^20)
>>> x = K(Integer(0), -Integer(5)); x
O(7^-5)
>>> x.denominator()
7^5 + O(7^25)
```

lift_to_precision(*absprec=None*)

Return another element of the same parent with absolute precision at least `absprec`, congruent to this element modulo the precision of this element.

INPUT:

- `absprec` – integer or `None` (default: `None`); the absolute precision of the result. If `None`, lifts to the maximum precision allowed.

Note

If setting `absprec` that high would violate the precision cap, raises a precision error. Note that the new digits will not necessarily be zero.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: R = ZpCA(17)
sage: R(-1, 2).lift_to_precision(10)
16 + 16*17 + O(17^10)
sage: R(1, 15).lift_to_precision(10)
1 + O(17^15)
sage: R(1, 15).lift_to_precision(30)
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
sage: (R(-1, 2).lift_to_precision().precision_absolute()
....:   == R.precision_cap())
True

sage: R = Zp(5); c = R(17, 3); c.lift_to_precision(8)                    #␣
↪needs sage.rings.padics
2 + 3*5 + O(5^8)
sage: c.lift_to_precision().precision_relative() == R.precision_cap()    #␣
↪needs sage.rings.padics
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> R = ZpCA(Integer(17))
>>> R(-Integer(1), Integer(2)).lift_to_precision(Integer(10))
16 + 16*17 + O(17^10)
>>> R(Integer(1), Integer(15)).lift_to_precision(Integer(10))
1 + O(17^15)
>>> R(Integer(1), Integer(15)).lift_to_precision(Integer(30))
Traceback (most recent call last):
...
PrecisionError: precision higher than allowed by the precision cap
>>> (R(-Integer(1), Integer(2)).lift_to_precision().precision_absolute()
...   == R.precision_cap())
True

>>> R = Zp(Integer(5)); c = R(Integer(17), Integer(3)); c.lift_to_
↪precision(Integer(8))                    # needs sage.rings.padics
2 + 3*5 + O(5^8)
>>> c.lift_to_precision().precision_relative() == R.precision_cap()   #␣
↪needs sage.rings.padics
True
```

numerator()

Return the numerator of this element, normalized in such a way that $x = x.numerator()/x.denominator()$ always holds true.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: K = Qp(7, 5)
sage: x = K(1/21)
sage: x.numerator()
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + O(7^5)
sage: x == x.numerator() / x.denominator()
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> K = Qp(Integer(7), Integer(5))
>>> x = K(Integer(1)/Integer(21))
>>> x.numerator()
5 + 4*7 + 4*7^2 + 4*7^3 + 4*7^4 + O(7^5)
>>> x == x.numerator() / x.denominator()
True
```

Note that the numerator lives in the ring of integers:

```
sage: x.numerator().parent()                                              #␣
↪needs sage.rings.padics
7-adic Ring with capped relative precision 5
```

```
>>> from sage.all import *
>>> x.numerator().parent()                                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.padics
7-adic Ring with capped relative precision 5
```

valuation()

Return the valuation of this element.

EXAMPLES:

```
sage: R = Zp(7)                                                          #␣
↪needs sage.rings.padics
sage: x = R(7); x                                                        #␣
↪needs sage.rings.padics
7 + O(7^21)
sage: x.valuation()                                                      #␣
↪needs sage.rings.padics
1
```

```
>>> from sage.all import *
>>> R = Zp(Integer(7))                                                   ␣
↪     # needs sage.rings.padics
>>> x = R(Integer(7)); x                                                 ␣
↪     # needs sage.rings.padics
7 + O(7^21)
>>> x.valuation()                                                      #␣
↪needs sage.rings.padics
1
```

super_categories()

EXAMPLES:

```
sage: CompleteDiscreteValuationRings().super_categories()
[Category of discrete valuation rings]
```

```
>>> from sage.all import *
>>> CompleteDiscreteValuationRings().super_categories()
[Category of discrete valuation rings]
```

4.29 Complex reflection groups

class sage.categories.complex_reflection_groups.**ComplexReflectionGroups**

Bases: *Category_singleton*

The category of complex reflection groups.

Let V be a complex vector space. A *complex reflection* is an element of $\mathrm{GL}(V)$ fixing a hyperplane pointwise and acting by multiplication by a root of unity on a complementary line.

A *complex reflection group* is a group W that is (isomorphic to) a subgroup of some general linear group $\mathrm{GL}(V)$ generated by a distinguished set of complex reflections.

The dimension of V is the *rank* of W.

For a comprehensive treatment of complex reflection groups and many definitions and theorems used here, we refer to [LT2009]. See also Wikipedia article Reflection_group.

See also

`ReflectionGroup()` for usage examples of this category.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups()
Category of complex reflection groups
sage: ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
sage: ComplexReflectionGroups().all_super_categories()
[Category of complex reflection groups,
 Category of complex reflection or generalized Coxeter groups,
 Category of groups,
 Category of monoids,
 Category of finitely generated semigroups,
 Category of semigroups,
 Category of finitely generated magmas,
 Category of inverse unital magmas,
 Category of unital magmas,
 Category of magmas,
 Category of enumerated sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import ComplexReflectionGroups
>>> ComplexReflectionGroups()
Category of complex reflection groups
>>> ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
>>> ComplexReflectionGroups().all_super_categories()
[Category of complex reflection groups,
 Category of complex reflection or generalized Coxeter groups,
 Category of groups,
 Category of monoids,
 Category of finitely generated semigroups,
 Category of semigroups,
 Category of finitely generated magmas,
 Category of inverse unital magmas,
 Category of unital magmas,
 Category of magmas,
 Category of enumerated sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

An example of a reflection group:

```
sage: W = ComplexReflectionGroups().example(); W                                #␣
↪needs sage.combinat
5-colored permutations of size 3
```

```
>>> from sage.all import *
>>> W = ComplexReflectionGroups().example(); W                                  #␣
↪needs sage.combinat
5-colored permutations of size 3
```

`W` is in the category of complex reflection groups:

```
sage: W in ComplexReflectionGroups()                                            #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> W in ComplexReflectionGroups()                                              #␣
↪needs sage.combinat
True
```

Finite

alias of *FiniteComplexReflectionGroups*

class ParentMethods

Bases: `object`

rank()

Return the rank of `self`.

The rank of `self` is the dimension of the smallest faithfull reflection representation of `self`.

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: W.rank()
3
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> W.rank()
3
```

additional_structure()

Return `None`.

Indeed, all the structure complex reflection groups have in addition to groups (simple reflections, ...) is already defined in the super category.

> **See also**
>
> *Category.additional_structure()*

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().additional_structure()
```

example()

Return an example of a complex reflection group.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().example()                                   #␣
↪needs sage.combinat
5-colored permutations of size 3
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().example()                                     #␣
↪needs sage.combinat
5-colored permutations of size 3
```

super_categories()

Return the super categories of `self`.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

4.30 Common category for Generalized Coxeter Groups or Complex Reflection Groups

ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.**_mul_**(*other*)

Return the product of `self` and `other`.

This default implementation computes a reduced word of `other` using *reduced_word()*, and applies the corresponding simple reflections on `self` using *apply_simple_reflections()*.

EXAMPLES:

```
sage: W = FiniteCoxeterGroups().example(); W
The 5-th dihedral group of order 10
sage: w = W.an_element()
sage: w
(1, 2)
sage: w._mul_(w)
(1, 2, 1, 2)
sage: w._mul_(w)._mul_(w)
(2, 1, 2, 1)
```

```
>>> from sage.all import *
>>> W = FiniteCoxeterGroups().example(); W
The 5-th dihedral group of order 10
>>> w = W.an_element()
>>> w
(1, 2)
>>> w._mul_(w)
(1, 2, 1, 2)
>>> w._mul_(w)._mul_(w)
(2, 1, 2, 1)
```

This method is called when computing `self * other`:

```
sage: w * w
(1, 2, 1, 2)
```

```
>>> from sage.all import *
>>> w * w
(1, 2, 1, 2)
```

class sage.categories.complex_reflection_or_generalized_coxeter_groups.**ComplexReflectionOrGeneralizedCoxeterGroups**

Bases: *Category_singleton*

The category of complex reflection groups or generalized Coxeter groups.

Finite Coxeter groups can be defined equivalently as groups generated by reflections, or by presentations. Over the last decades, the theory has been generalized in both directions, leading to the study of (finite) complex reflection groups on the one hand, and (finite) generalized Coxeter groups on the other hand. Many of the features remain similar, yet, in the current state of the art, there is no general theory covering both directions.

This is reflected by the name of this category which is about factoring out the common code, tests, and declarations.

A group in this category has:

- A distinguished finite set of generators $(s_i)_I$, called *simple reflections*. The set I is called the *index set*. The name "reflection" is somewhat of an abuse as they can have higher order; still, they are all of finite order: $s_i^k = 1$ for some k.
- A collection of *distinguished reflections* which are the conjugates of the simple reflections. For complex reflection groups, they are in one-to-one correspondence with the reflection hyperplanes and share the same index set.
- A collection of *reflections* which are the conjugates of all the non trivial powers of the simple reflections.

The usual notions of reduced words, length, irreducibility, etc., can be canonically defined from the above.

The following methods must be implemented:

- *ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods.index_set()*
- *ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods.simple_reflection()*

Optionally one can define analog methods for distinguished reflections and reflections (see below).

At least one of the following methods must be implemented:

- *ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.apply_simple_reflection()*
- *ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.apply_simple_reflection_left()*
- *ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.apply_simple_reflection_right()*
- *ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods._mul_()*

It's recommended to implement either `_mul_` or both `apply_simple_reflection_left` and `apply_simple_reflection_right`.

See also

- *ComplexReflectionGroups*
- *GeneralizedCoxeterGroups*

EXAMPLES:

```
sage: from sage.categories.complex_reflection_or_generalized_coxeter_groups␣
↪import ComplexReflectionOrGeneralizedCoxeterGroups
sage: C = ComplexReflectionOrGeneralizedCoxeterGroups(); C
Category of complex reflection or generalized Coxeter groups
sage: C.super_categories()
[Category of finitely generated enumerated groups]

sage: C.required_methods()
{'element': {'optional': ['reflection_length'],
             'required': []},
  'parent': {'optional': ['distinguished_reflection', 'hyperplane_index_set',
                          'irreducible_components',
                          'reflection', 'reflection_index_set'],
             'required':  ['__contains__', 'index_set']}}
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_or_generalized_coxeter_groups import␣
↪ComplexReflectionOrGeneralizedCoxeterGroups
>>> C = ComplexReflectionOrGeneralizedCoxeterGroups(); C
Category of complex reflection or generalized Coxeter groups
>>> C.super_categories()
[Category of finitely generated enumerated groups]

>>> C.required_methods()
{'element': {'optional': ['reflection_length'],
             'required': []},
  'parent': {'optional': ['distinguished_reflection', 'hyperplane_index_set',
                          'irreducible_components',
                          'reflection', 'reflection_index_set'],
             'required':  ['__contains__', 'index_set']}}
```

class ElementMethods

Bases: object

apply_conjugation_by_simple_reflection(*i*)

Conjugate `self` by the `i`-th simple reflection.

EXAMPLES:

```
sage: W = WeylGroup(['A',3])                                                    #␣
↪needs sage.rings.number_field
sage: w = W.from_reduced_word([3,1,2,1])                                        #␣
↪needs sage.rings.number_field
sage: w.apply_conjugation_by_simple_reflection(1).reduced_word()                #␣
↪needs sage.rings.number_field
[3, 2]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)])                                             ␣
↪     # needs sage.rings.number_field
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),
↪Integer(1)])                                    # needs sage.rings.number_
↪field
>>> w.apply_conjugation_by_simple_reflection(Integer(1)).reduced_word()         ␣
↪     # needs sage.rings.number_field
[3, 2]
```

apply_reflections(*word*, *side='right'*, *word_type='all'*)

Return the result of the (left/right) multiplication of `self` by `word`.

INPUT:

- `word` – a sequence of indices of reflections
- `side` – (default: `'right'`) indicates multiplying from left or right
- `word_type` – (default: `'all'`): either `'simple'`, `'distinguished'`, or `'all'`

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: W.one().apply_reflections([1])
```

(continues on next page)

(continued from previous page)

```
(1,4)(2,3)(5,6)
sage: W.one().apply_reflections([2])
(1,3)(2,5)(4,6)
sage: W.one().apply_reflections([2,1])
(1,2,6)(3,4,5)


sage: W = CoxeterGroups().example()
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.apply_reflections([0,1], word_type='simple')
(2, 3, 1, 0)
sage: w
(1, 2, 3, 0)
sage: w.apply_reflections([0,1], side='left', word_type='simple')
(0, 1, 3, 2)

sage: # needs sage.rings.number_field
sage: W = WeylGroup("A3", prefix='s')
sage: w = W.an_element(); w
s1*s2*s3
sage: AS = W.domain()
sage: r1 = AS.roots()[4]; r1
(0, 1, 0, -1)
sage: r2 = AS.roots()[5]; r2
(0, 0, 1, -1)
sage: w.apply_reflections([r1, r2], word_type='all')
s1

sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: W.one().apply_reflections([1],   word_type='distinguished')
(1,4)(2,3)(5,6)
sage: W.one().apply_reflections([2],   word_type='distinguished')
(1,3)(2,5)(4,6)
sage: W.one().apply_reflections([3],   word_type='distinguished')
(1,5)(2,4)(3,6)
sage: W.one().apply_reflections([2,1], word_type='distinguished')
(1,2,6)(3,4,5)

sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3), hyperplane_index_set=['A','B','C']); W
Irreducible real reflection group of rank 2 and type A2
sage: W.one().apply_reflections(['A'], word_type='distinguished')
(1,4)(2,3)(5,6)
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> W.one().apply_reflections([Integer(1)])
(1,4)(2,3)(5,6)
>>> W.one().apply_reflections([Integer(2)])
```

(continues on next page)

(continued from previous page)

```
(1,3)(2,5)(4,6)
>>> W.one().apply_reflections([Integer(2),Integer(1)])
(1,2,6)(3,4,5)


>>> W = CoxeterGroups().example()
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.apply_reflections([Integer(0),Integer(1)], word_type='simple')
(2, 3, 1, 0)
>>> w
(1, 2, 3, 0)
>>> w.apply_reflections([Integer(0),Integer(1)], side='left', word_type=
↪'simple')
(0, 1, 3, 2)

>>> # needs sage.rings.number_field
>>> W = WeylGroup("A3", prefix='s')
>>> w = W.an_element(); w
s1*s2*s3
>>> AS = W.domain()
>>> r1 = AS.roots()[Integer(4)]; r1
(0, 1, 0, -1)
>>> r2 = AS.roots()[Integer(5)]; r2
(0, 0, 1, -1)
>>> w.apply_reflections([r1, r2], word_type='all')
s1

>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> W.one().apply_reflections([Integer(1)],   word_type='distinguished')
(1,4)(2,3)(5,6)
>>> W.one().apply_reflections([Integer(2)],   word_type='distinguished')
(1,3)(2,5)(4,6)
>>> W.one().apply_reflections([Integer(3)],   word_type='distinguished')
(1,5)(2,4)(3,6)
>>> W.one().apply_reflections([Integer(2),Integer(1)], word_type=
↪'distinguished')
(1,2,6)(3,4,5)

>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), hyperplane_
↪index_set=['A','B','C']); W
Irreducible real reflection group of rank 2 and type A2
>>> W.one().apply_reflections(['A'], word_type='distinguished')
(1,4)(2,3)(5,6)
```

apply_simple_reflection(*i*, *side='right'*)

Return `self` multiplied by the simple reflection `s[i]`.

INPUT:

- `i` – an element of the index set
- `side` – (default: `'right'`) `'left'` or `'right'`

This default implementation simply calls *apply_simple_reflection_left()* or *apply_simple_reflection_right()*.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.apply_simple_reflection(0, side='left')
(0, 2, 3, 1)
sage: w.apply_simple_reflection(1, side='left')
(2, 1, 3, 0)
sage: w.apply_simple_reflection(2, side='left')
(1, 3, 2, 0)
sage: w.apply_simple_reflection(0, side='right')
(2, 1, 3, 0)
sage: w.apply_simple_reflection(1, side='right')
(1, 3, 2, 0)
sage: w.apply_simple_reflection(2, side='right')
(1, 2, 0, 3)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.apply_simple_reflection(Integer(0), side='left')
(0, 2, 3, 1)
>>> w.apply_simple_reflection(Integer(1), side='left')
(2, 1, 3, 0)
>>> w.apply_simple_reflection(Integer(2), side='left')
(1, 3, 2, 0)
>>> w.apply_simple_reflection(Integer(0), side='right')
(2, 1, 3, 0)
>>> w.apply_simple_reflection(Integer(1), side='right')
(1, 3, 2, 0)
>>> w.apply_simple_reflection(Integer(2), side='right')
(1, 2, 0, 3)
```

By default, `side` is `'right'`:

```
sage: w.apply_simple_reflection(0)
(2, 1, 3, 0)
```

```
>>> from sage.all import *
>>> w.apply_simple_reflection(Integer(0))
(2, 1, 3, 0)
```

Some tests with a complex reflection group:

```
sage: # needs sage.combinat
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: W = ComplexReflectionGroups().example(); W
5-colored permutations of size 3
```

(continues on next page)

(continued from previous page)

```
sage: w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
sage: w.apply_simple_reflection(1, side='left')
[[0, 1, 0], [1, 3, 2]]
sage: w.apply_simple_reflection(2, side='left')
[[1, 0, 0], [3, 2, 1]]
sage: w.apply_simple_reflection(3, side='left')
[[1, 0, 1], [3, 1, 2]]
sage: w.apply_simple_reflection(1, side='right')
[[1, 0, 0], [3, 2, 1]]
sage: w.apply_simple_reflection(2, side='right')
[[1, 0, 0], [2, 1, 3]]
sage: w.apply_simple_reflection(3, side='right')
[[2, 0, 0], [3, 1, 2]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> W = ComplexReflectionGroups().example(); W
5-colored permutations of size 3
>>> w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
>>> w.apply_simple_reflection(Integer(1), side='left')
[[0, 1, 0], [1, 3, 2]]
>>> w.apply_simple_reflection(Integer(2), side='left')
[[1, 0, 0], [3, 2, 1]]
>>> w.apply_simple_reflection(Integer(3), side='left')
[[1, 0, 1], [3, 1, 2]]
>>> w.apply_simple_reflection(Integer(1), side='right')
[[1, 0, 0], [3, 2, 1]]
>>> w.apply_simple_reflection(Integer(2), side='right')
[[1, 0, 0], [2, 1, 3]]
>>> w.apply_simple_reflection(Integer(3), side='right')
[[2, 0, 0], [3, 1, 2]]
```

apply_simple_reflection_left(*i*)

Return `self` multiplied by the simple reflection `s[i]` on the left.

This low level method is used intensively. Coxeter groups are encouraged to override this straightforward implementation whenever a faster approach exists.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.apply_simple_reflection_left(0)
(0, 2, 3, 1)
sage: w.apply_simple_reflection_left(1)
(2, 1, 3, 0)
sage: w.apply_simple_reflection_left(2)
(1, 3, 2, 0)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.apply_simple_reflection_left(Integer(0))
(0, 2, 3, 1)
>>> w.apply_simple_reflection_left(Integer(1))
(2, 1, 3, 0)
>>> w.apply_simple_reflection_left(Integer(2))
(1, 3, 2, 0)
```

EXAMPLES:

```
sage: # needs sage.combinat
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: W = ComplexReflectionGroups().example()
sage: w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
sage: w.apply_simple_reflection_left(1)
[[0, 1, 0], [1, 3, 2]]
sage: w.apply_simple_reflection_left(2)
[[1, 0, 0], [3, 2, 1]]
sage: w.apply_simple_reflection_left(3)
[[1, 0, 1], [3, 1, 2]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> W = ComplexReflectionGroups().example()
>>> w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
>>> w.apply_simple_reflection_left(Integer(1))
[[0, 1, 0], [1, 3, 2]]
>>> w.apply_simple_reflection_left(Integer(2))
[[1, 0, 0], [3, 2, 1]]
>>> w.apply_simple_reflection_left(Integer(3))
[[1, 0, 1], [3, 1, 2]]
```

apply_simple_reflection_right(*i*)

Return `self` multiplied by the simple reflection `s[i]` on the right.

This low level method is used intensively. Coxeter groups are encouraged to override this straightforward implementation whenever a faster approach exists.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.apply_simple_reflection_right(0)
(2, 1, 3, 0)
sage: w.apply_simple_reflection_right(1)
```

(continues on next page)

(continued from previous page)

```
(1, 3, 2, 0)
sage: w.apply_simple_reflection_right(2)
(1, 2, 0, 3)

sage: # needs sage.combinat
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: W = ComplexReflectionGroups().example()
sage: w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
sage: w.apply_simple_reflection_right(1)
[[1, 0, 0], [3, 2, 1]]
sage: w.apply_simple_reflection_right(2)
[[1, 0, 0], [2, 1, 3]]
sage: w.apply_simple_reflection_right(3)
[[2, 0, 0], [3, 1, 2]]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.apply_simple_reflection_right(Integer(0))
(2, 1, 3, 0)
>>> w.apply_simple_reflection_right(Integer(1))
(1, 3, 2, 0)
>>> w.apply_simple_reflection_right(Integer(2))
(1, 2, 0, 3)

>>> # needs sage.combinat
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> W = ComplexReflectionGroups().example()
>>> w = W.an_element(); w
[[1, 0, 0], [3, 1, 2]]
>>> w.apply_simple_reflection_right(Integer(1))
[[1, 0, 0], [3, 2, 1]]
>>> w.apply_simple_reflection_right(Integer(2))
[[1, 0, 0], [2, 1, 3]]
>>> w.apply_simple_reflection_right(Integer(3))
[[2, 0, 0], [3, 1, 2]]
```

apply_simple_reflections(*word*, *side='right'*, *type='simple'*)

Return the result of the (left/right) multiplication of `self` by `word`.

INPUT:

- `word` – a sequence of indices of simple reflections
- `side` – (default: `'right'`) indicates multiplying from left or right

This is a specialized implementation of *apply_reflections()* for the simple reflections. The rationale for its existence are:

- It can take advantage of `apply_simple_reflection`, which often is less expensive than computing a product.
- It reduced burden on implementations that would want to provide an optimized version of this method.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.apply_simple_reflections([0,1])
(2, 3, 1, 0)
sage: w
(1, 2, 3, 0)
sage: w.apply_simple_reflections([0,1],side='left')
(0, 1, 3, 2)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.apply_simple_reflections([Integer(0),Integer(1)])
(2, 3, 1, 0)
>>> w
(1, 2, 3, 0)
>>> w.apply_simple_reflections([Integer(0),Integer(1)],side='left')
(0, 1, 3, 2)
```

is_reflection()

Return whether `self` is a reflection.

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4))
sage: [t.is_reflection() for t in W.reflections()]
[True, True, True, True, True, True]
sage: len([t for t in W.reflections() if t.is_reflection()])
6
sage: W = ReflectionGroup((2,1,3))
sage: [t.is_reflection() for t in W.reflections()]
[True, True, True, True, True, True, True, True, True]
sage: len([t for t in W.reflections() if t.is_reflection()])
9
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))
>>> [t.is_reflection() for t in W.reflections()]
[True, True, True, True, True, True]
>>> len([t for t in W.reflections() if t.is_reflection()])
6
>>> W = ReflectionGroup((Integer(2),Integer(1),Integer(3)))
>>> [t.is_reflection() for t in W.reflections()]
[True, True, True, True, True, True, True, True, True]
>>> len([t for t in W.reflections() if t.is_reflection()])
9
```

reflection_length()

Return the reflection length of `self`.

This is the minimal length of a factorization of `self` into reflections.

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,2))
sage: sorted([t.reflection_length() for t in W])
[0, 1]
sage: W = ReflectionGroup((2,1,2))
sage: sorted([t.reflection_length() for t in W])
[0, 1, 1, 1, 1, 2, 2, 2]
sage: W = ReflectionGroup((3,1,2))
sage: sorted([t.reflection_length() for t in W])
[0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
sage: W = ReflectionGroup((2,2,2))
sage: sorted([t.reflection_length() for t in W])
[0, 1, 1, 2]
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(2)))
>>> sorted([t.reflection_length() for t in W])
[0, 1]
>>> W = ReflectionGroup((Integer(2),Integer(1),Integer(2)))
>>> sorted([t.reflection_length() for t in W])
[0, 1, 1, 1, 1, 2, 2, 2]
>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(2)))
>>> sorted([t.reflection_length() for t in W])
[0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
>>> W = ReflectionGroup((Integer(2),Integer(2),Integer(2)))
>>> sorted([t.reflection_length() for t in W])
[0, 1, 1, 2]
```

class Irreducible(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: `object`

irreducible_components()

Return a list containing all irreducible components of `self` as finite reflection groups.

EXAMPLES:

```
sage: W = ColoredPermutations(4, 3)                                    #␣
↪needs sage.combinat
sage: W.irreducible_components()                                       #␣
↪needs sage.combinat
[4-colored permutations of size 3]
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(4), Integer(3))                    ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs sage.combinat
>>> W.irreducible_components()                                    #␣
↪needs sage.combinat
[4-colored permutations of size 3]
```

class ParentMethods

Bases: object

distinguished_reflection(*i*)

Return the i-th distinguished reflection of `self`.

INPUT:

- `i` – an element of the index set of the distinguished reflections

See also

- *distinguished_reflections()*
- *hyperplane_index_set()*

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4),
....:                     hyperplane_index_set=('a','b','c','d','e','f'))
sage: for i in W.hyperplane_index_set():
....:     print('%s %s'%(i, W.distinguished_reflection(i)))
a (1,7)(2,4)(5,6)(8,10)(11,12)
b (1,4)(2,8)(3,5)(7,10)(9,11)
c (2,5)(3,9)(4,6)(8,11)(10,12)
d (1,8)(2,7)(3,6)(4,10)(9,12)
e (1,6)(2,9)(3,8)(5,11)(7,12)
f (1,11)(3,10)(4,9)(5,7)(6,12)
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)),
...                     hyperplane_index_set=('a','b','c','d','e','f'))
>>> for i in W.hyperplane_index_set():
...     print('%s %s'%(i, W.distinguished_reflection(i)))
a (1,7)(2,4)(5,6)(8,10)(11,12)
b (1,4)(2,8)(3,5)(7,10)(9,11)
c (2,5)(3,9)(4,6)(8,11)(10,12)
d (1,8)(2,7)(3,6)(4,10)(9,12)
e (1,6)(2,9)(3,8)(5,11)(7,12)
f (1,11)(3,10)(4,9)(5,7)(6,12)
```

distinguished_reflections()

Return a finite family containing the distinguished reflections of `self`, indexed by *hyperplane_index_set()*.

A *distinguished reflection* is a conjugate of a simple reflection. For a Coxeter group, reflections and distinguished reflections coincide. For a Complex reflection groups this is a reflection acting on the

complement of the fixed hyperplane H as $\exp(2\pi i/n)$, where n is the order of the reflection subgroup fixing H.

See also

- *distinguished_reflection()*
- *hyperplane_index_set()*

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: distinguished_reflections = W.distinguished_reflections()
sage: for index in sorted(distinguished_reflections.keys()):
....:     print('%s %s'%(index, distinguished_reflections[index]))
1 (1,4)(2,3)(5,6)
2 (1,3)(2,5)(4,6)
3 (1,5)(2,4)(3,6)
sage: W = ReflectionGroup((1,1,3), hyperplane_index_set=['a','b','c'])
sage: distinguished_reflections = W.distinguished_reflections()
sage: for index in sorted(distinguished_reflections.keys()):
....:     print('%s %s'%(index, distinguished_reflections[index]))
a (1,4)(2,3)(5,6)
b (1,3)(2,5)(4,6)
c (1,5)(2,4)(3,6)
sage: W = ReflectionGroup((3,1,1))
sage: distinguished_reflections = W.distinguished_reflections()
sage: for index in sorted(distinguished_reflections.keys()):
....:     print('%s %s'%(index, distinguished_reflections[index]))
1 (1,2,3)
sage: W = ReflectionGroup((1,1,3), (3,1,2))
sage: distinguished_reflections = W.distinguished_reflections()
sage: for index in sorted(distinguished_reflections.keys()):
....:     print('%s %s'%(index, distinguished_reflections[index]))
1 (1,6)(2,5)(7,8)
2 (1,5)(2,7)(6,8)
3 (3,9,15)(4,10,16)(12,17,23)(14,18,24)(20,25,29)(21,22,26)(27,28,30)
4 (3,11)(4,12)(9,13)(10,14)(15,19)(16,20)(17,21)(18,22)(23,27)(24,28)(25,
↪26)(29,30)
5 (1,7)(2,6)(5,8)
6 (3,19)(4,25)(9,11)(10,17)(12,28)(13,15)(14,30)(16,18)(20,27)(21,29)(22,
↪23)(24,26)
7 (4,21,27)(10,22,28)(11,13,19)(12,14,20)(16,26,30)(17,18,25)(23,24,29)
8 (3,13)(4,24)(9,19)(10,29)(11,15)(12,26)(14,21)(16,23)(17,30)(18,27)(20,
↪22)(25,28)
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> distinguished_reflections = W.distinguished_reflections()
>>> for index in sorted(distinguished_reflections.keys()):
```

(continues on next page)

(continued from previous page)

```
...       print('%s %s'%(index, distinguished_reflections[index]))
1  (1,4)(2,3)(5,6)
2  (1,3)(2,5)(4,6)
3  (1,5)(2,4)(3,6)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), hyperplane_
↪index_set=['a','b','c'])
>>> distinguished_reflections = W.distinguished_reflections()
>>> for index in sorted(distinguished_reflections.keys()):
...       print('%s %s'%(index, distinguished_reflections[index]))
a  (1,4)(2,3)(5,6)
b  (1,3)(2,5)(4,6)
c  (1,5)(2,4)(3,6)
>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(1)))
>>> distinguished_reflections = W.distinguished_reflections()
>>> for index in sorted(distinguished_reflections.keys()):
...       print('%s %s'%(index, distinguished_reflections[index]))
1  (1,2,3)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), (Integer(3),
↪Integer(1),Integer(2)))
>>> distinguished_reflections = W.distinguished_reflections()
>>> for index in sorted(distinguished_reflections.keys()):
...       print('%s %s'%(index, distinguished_reflections[index]))
1  (1,6)(2,5)(7,8)
2  (1,5)(2,7)(6,8)
3  (3,9,15)(4,10,16)(12,17,23)(14,18,24)(20,25,29)(21,22,26)(27,28,30)
4  (3,11)(4,12)(9,13)(10,14)(15,19)(16,20)(17,21)(18,22)(23,27)(24,28)(25,
↪26)(29,30)
5  (1,7)(2,6)(5,8)
6  (3,19)(4,25)(9,11)(10,17)(12,28)(13,15)(14,30)(16,18)(20,27)(21,29)(22,
↪23)(24,26)
7  (4,21,27)(10,22,28)(11,13,19)(12,14,20)(16,26,30)(17,18,25)(23,24,29)
8  (3,13)(4,24)(9,19)(10,29)(11,15)(12,26)(14,21)(16,23)(17,30)(18,27)(20,
↪22)(25,28)
```

from_reduced_word(*word*, *word_type='simple'*)

Return an element of `self` from its (reduced) word.

INPUT:

- `word` – list (or iterable) of elements of the index set of `self` (resp. of the distinguished or of all reflections)
- `word_type` – (default: `'simple'`): either `'simple'`, `'distinguished'`, or `'all'`

If `word` is $[i_1, i_2, \ldots, i_k]$, then this returns the corresponding product of simple reflections $s_{i_1} s_{i_2} \cdots s_{i_k}$.

If `word_type` is `'distinguished'` (resp. `'all'`), then the product of the distinguished reflections (resp. all reflections) is returned.

Note

The main use case is for constructing elements from reduced words, hence the name of this method. However, the input word need *not* be reduced.

See also

- *index_set()*
- *reflection_index_set()*
- *hyperplane_index_set()*
- *apply_simple_reflections()*
- *reduced_word()*
- CoxeterGroups.ParentMethods._test_reduced_word

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: s = W.simple_reflections()
sage: W.from_reduced_word([0,2,0,1])
(0, 3, 1, 2)
sage: W.from_reduced_word((0,2,0,1))
(0, 3, 1, 2)
sage: s[0]*s[2]*s[0]*s[1]
(0, 3, 1, 2)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> s = W.simple_reflections()
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(0),Integer(1)])
(0, 3, 1, 2)
>>> W.from_reduced_word((Integer(0),Integer(2),Integer(0),Integer(1)))
(0, 3, 1, 2)
>>> s[Integer(0)]*s[Integer(2)]*s[Integer(0)]*s[Integer(1)]
(0, 3, 1, 2)
```

We now experiment with the different values for `word_type` for the colored symmetric group:

```
sage: # needs sage.combinat
sage: W = ColoredPermutations(1,4)
sage: W.from_reduced_word([1,2,1,2,1,2])
[[0, 0, 0, 0], [1, 2, 3, 4]]
sage: W.from_reduced_word([1, 2, 3]).reduced_word()
[1, 2, 3]

sage: # needs sage.rings.number_field
sage: W = WeylGroup("A3", prefix='s')
sage: AS = W.domain()
sage: r1 = AS.roots()[4]; r1
(0, 1, 0, -1)
sage: r2 = AS.roots()[5]; r2
(0, 0, 1, -1)
sage: W.from_reduced_word([r1, r2], word_type='all')
s3*s2
```

(continues on next page)

(continued from previous page)

```
sage: W = WeylGroup("G2", prefix='s')                                    #␣
↪needs sage.rings.number_field
sage: W.from_reduced_word(W.domain().positive_roots(),                   #␣
↪needs sage.rings.number_field
....:                      word_type='all')
s1*s2

sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4))
sage: W.from_reduced_word([1,2,3], word_type='all').reduced_word()
[1, 2, 3]
sage: W.from_reduced_word([1,2,3], word_type='all').reduced_word_in_
↪reflections()
[1, 2, 3]
sage: W.from_reduced_word([1,2,3]).reduced_word_in_reflections()
[1, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> W = ColoredPermutations(Integer(1),Integer(4))
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(2),
↪Integer(1),Integer(2)])
[[0, 0, 0, 0], [1, 2, 3, 4]]
>>> W.from_reduced_word([Integer(1), Integer(2), Integer(3)]).reduced_
↪word()
[1, 2, 3]

>>> # needs sage.rings.number_field
>>> W = WeylGroup("A3", prefix='s')
>>> AS = W.domain()
>>> r1 = AS.roots()[Integer(4)]; r1
(0, 1, 0, -1)
>>> r2 = AS.roots()[Integer(5)]; r2
(0, 0, 1, -1)
>>> W.from_reduced_word([r1, r2], word_type='all')
s3*s2

>>> W = WeylGroup("G2", prefix='s')                                      #␣
↪needs sage.rings.number_field
>>> W.from_reduced_word(W.domain().positive_roots(),                     #␣
↪needs sage.rings.number_field
...                      word_type='all')
s1*s2

>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(3)], word_type='all
↪').reduced_word()
[1, 2, 3]
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(3)], word_type='all
↪').reduced_word_in_reflections()
```

(continues on next page)

(continued from previous page)

```
[1, 2, 3]
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(3)]).reduced_word_
↪in_reflections()
[1, 2, 3]
```

group_generators()

Return the simple reflections of `self`, as distinguished group generators.

See also

- *simple_reflections()*
- *Groups.ParentMethods.group_generators()*
- *Semigroups.ParentMethods.semigroup_generators()*

EXAMPLES:

```
sage: D10 = FiniteCoxeterGroups().example(10)
sage: D10.group_generators()
Finite family {1: (1,), 2: (2,)}
sage: SymmetricGroup(5).group_generators()
Finite family {1: (1,2), 2: (2,3), 3: (3,4), 4: (4,5)}

sage: W = ColoredPermutations(3,2)                                    #␣
↪needs sage.combinat
sage: W.group_generators()                                            #␣
↪needs sage.combinat
Finite family {1: [[0, 0],
                   [2, 1]],
               2: [[0, 1],
                   [1, 2]]}
```

```
>>> from sage.all import *
>>> D10 = FiniteCoxeterGroups().example(Integer(10))
>>> D10.group_generators()
Finite family {1: (1,), 2: (2,)}
>>> SymmetricGroup(Integer(5)).group_generators()
Finite family {1: (1,2), 2: (2,3), 3: (3,4), 4: (4,5)}

>>> W = ColoredPermutations(Integer(3),Integer(2))                     ␣
↪              # needs sage.combinat
>>> W.group_generators()                                             #␣
↪needs sage.combinat
Finite family {1: [[0, 0],
                   [2, 1]],
               2: [[0, 1],
                   [1, 2]]}
```

The simple reflections are also semigroup generators, even for an infinite group:

```
sage: W = WeylGroup(["A",2,1])                                          #␣
↪needs sage.rings.number_field
sage: W.semigroup_generators()                                          #␣
↪needs sage.rings.number_field
Finite family {0: [-1  1  1]
                  [ 0  1  0]
                  [ 0  0  1],
               1: [ 1  0  0]
                  [ 1 -1  1]
                  [ 0  0  1],
               2: [ 1  0  0]
                  [ 0  1  0]
                  [ 1  1 -1]}
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A",Integer(2),Integer(1)])                          ␣
↪              # needs sage.rings.number_field
>>> W.semigroup_generators()                                           #␣
↪needs sage.rings.number_field
Finite family {0: [-1  1  1]
                  [ 0  1  0]
                  [ 0  0  1],
               1: [ 1  0  0]
                  [ 1 -1  1]
                  [ 0  0  1],
               2: [ 1  0  0]
                  [ 0  1  0]
                  [ 1  1 -1]}
```

hyperplane_index_set()

Return the index set of the distinguished reflections of `self`.

This is also the index set of the reflection hyperplanes of `self`, hence the name. This name is slightly abusive since the concept of reflection hyperplanes is not defined for all generalized Coxeter groups. However for all practical purposes this is only used for complex reflection groups, and there this is the desirable name.

See also

- *distinguished_reflection()*
- *distinguished_reflections()*

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4))
sage: W.hyperplane_index_set()
(1, 2, 3, 4, 5, 6)
sage: W = ReflectionGroup((1,1,4), hyperplane_index_set=[1,3,'asdf',7,9,
↪11])
sage: W.hyperplane_index_set()
```

(continues on next page)

(continued from previous page)

```
(1, 3, 'asdf', 7, 9, 11)
sage: W = ReflectionGroup((1,1,4),
....:                     hyperplane_index_set=('a','b','c','d','e','f'))
sage: W.hyperplane_index_set()
('a', 'b', 'c', 'd', 'e', 'f')
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))
>>> W.hyperplane_index_set()
(1, 2, 3, 4, 5, 6)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), hyperplane_
↪index_set=[Integer(1),Integer(3),'asdf',Integer(7),Integer(9),
↪Integer(11)])
>>> W.hyperplane_index_set()
(1, 3, 'asdf', 7, 9, 11)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)),
...                     hyperplane_index_set=('a','b','c','d','e','f'))
>>> W.hyperplane_index_set()
('a', 'b', 'c', 'd', 'e', 'f')
```

index_set()

Return the index set of (the simple reflections of) `self`, as a list (or iterable).

See also

- *simple_reflection()*
- *simple_reflections()*

EXAMPLES:

```
sage: W = CoxeterGroups().Finite().example(); W
The 5-th dihedral group of order 10
sage: W.index_set()
(1, 2)

sage: W = ColoredPermutations(1, 4)                                    #␣
↪needs sage.combinat
sage: W.index_set()                                                    #␣
↪needs sage.combinat
(1, 2, 3)

sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4), index_set=[1,3,'asdf'])
sage: W.index_set()
(1, 3, 'asdf')
sage: W = ReflectionGroup((1,1,4), index_set=('a','b','c'))
sage: W.index_set()
('a', 'b', 'c')
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().Finite().example(); W
The 5-th dihedral group of order 10
>>> W.index_set()
(1, 2)

>>> W = ColoredPermutations(Integer(1), Integer(4))                    ␣
↪              # needs sage.combinat
>>> W.index_set()                                                      #␣
↪needs sage.combinat
(1, 2, 3)

>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), index_
↪set=[Integer(1),Integer(3),'asdf'])
>>> W.index_set()
(1, 3, 'asdf')
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), index_set=('a
↪','b','c'))
>>> W.index_set()
('a', 'b', 'c')
```

irreducible_component_index_sets()

Return a list containing the index sets of the irreducible components of `self` as finite reflection groups.

EXAMPLES:

```
sage: # needs sage.graphs
sage: W = ReflectionGroup([1,1,3], [3,1,3], 4); W       # optional - gap3
Reducible complex reflection group of rank 7 and type A2 x G(3,1,3) x ST4
sage: sorted(W.irreducible_component_index_sets())      # optional - gap3
[[1, 2], [3, 4, 5], [6, 7]]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> W = ReflectionGroup([Integer(1),Integer(1),Integer(3)], [Integer(3),
↪Integer(1),Integer(3)], Integer(4)); W       # optional - gap3
Reducible complex reflection group of rank 7 and type A2 x G(3,1,3) x ST4
>>> sorted(W.irreducible_component_index_sets())      # optional - gap3
[[1, 2], [3, 4, 5], [6, 7]]
```

ALGORITHM:

Take the connected components of the graph on the index set with edges `(i,j)`, where `s[i]` and `s[j]` do not commute.

irreducible_components()

Return the irreducible components of `self` as finite reflection groups.

EXAMPLES:

```
sage: # needs sage.graphs
sage: W = ReflectionGroup([1,1,3], [3,1,3], 4)          # optional - gap3
sage: W.irreducible_components()                        # optional - gap3
```

(continues on next page)

(continued from previous page)

```
[Irreducible real reflection group of rank 2 and type A2,
 Irreducible complex reflection group of rank 3 and type G(3,1,3),
 Irreducible complex reflection group of rank 2 and type ST4]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> W = ReflectionGroup([Integer(1),Integer(1),Integer(3)], [Integer(3),
↪Integer(1),Integer(3)], Integer(4))          # optional - gap3
>>> W.irreducible_components()                          # optional - gap3
[Irreducible real reflection group of rank 2 and type A2,
 Irreducible complex reflection group of rank 3 and type G(3,1,3),
 Irreducible complex reflection group of rank 2 and type ST4]
```

is_irreducible()

Return `True` if `self` is irreducible.

EXAMPLES:

```
sage: # needs sage.graphs
sage: W = ColoredPermutations(1,3); W                                    #␣
↪needs sage.combinat
1-colored permutations of size 3
sage: W.is_irreducible()                                                 #␣
↪needs sage.combinat
True
sage: W = ReflectionGroup((1,1,3),(2,1,3)); W           # optional - gap3
Reducible real reflection group of rank 5 and type A2 x B3
sage: W.is_irreducible()                                # optional - gap3
False
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> W = ColoredPermutations(Integer(1),Integer(3)); W                    ␣
↪              # needs sage.combinat
1-colored permutations of size 3
>>> W.is_irreducible()                                                  #␣
↪needs sage.combinat
True
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)),(Integer(2),
↪Integer(1),Integer(3))); W           # optional - gap3
Reducible real reflection group of rank 5 and type A2 x B3
>>> W.is_irreducible()                                # optional - gap3
False
```

is_reducible()

Return `True` if `self` is not irreducible.

EXAMPLES:

```
sage: # needs sage.graphs
sage: W = ColoredPermutations(1,3); W                                    #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
1-colored permutations of size 3
sage: W.is_reducible()                                                  #␣
↪needs sage.combinat
False
sage: W = ReflectionGroup((1,1,3), (2,1,3)); W          # optional - gap3
Reducible real reflection group of rank 5 and type A2 x B3
sage: W.is_reducible()                                  # optional - gap3
True
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> W = ColoredPermutations(Integer(1),Integer(3)); W                    ␣
↪              # needs sage.combinat
1-colored permutations of size 3
>>> W.is_reducible()                                                    #␣
↪needs sage.combinat
False
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), (Integer(2),
↪Integer(1),Integer(3))); W          # optional - gap3
Reducible real reflection group of rank 5 and type A2 x B3
>>> W.is_reducible()                                    # optional - gap3
True
```

number_of_irreducible_components()

Return the number of irreducible components of `self`.

EXAMPLES:

```
sage: # needs sage.graphs
sage: SymmetricGroup(3).number_of_irreducible_components()
1
sage: ColoredPermutations(1,3).number_of_irreducible_components()       #␣
↪needs sage.combinat
1
sage: ReflectionGroup((1,1,3),(2,1,3)).number_of_irreducible_components()␣
↪  # optional - gap3
2
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> SymmetricGroup(Integer(3)).number_of_irreducible_components()
1
>>> ColoredPermutations(Integer(1),Integer(3)).number_of_irreducible_
↪components()        # needs sage.combinat
1
>>> ReflectionGroup((Integer(1),Integer(1),Integer(3)),(Integer(2),
↪Integer(1),Integer(3))).number_of_irreducible_components()    # optional␣
↪- gap3
2
```

number_of_simple_reflections()

Return the number of simple reflections of `self`.

EXAMPLES:

```
sage: # needs sage.combinat
sage: W = ColoredPermutations(1,3)
sage: W.number_of_simple_reflections()
2
sage: W = ColoredPermutations(2,3)
sage: W.number_of_simple_reflections()
3
sage: W = ColoredPermutations(4,3)
sage: W.number_of_simple_reflections()
3

sage: W = ReflectionGroup((4,2,3))                                  #␣
↪optional - gap3
sage: W.number_of_simple_reflections()                              #␣
↪optional - gap3
4
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> W = ColoredPermutations(Integer(1),Integer(3))
>>> W.number_of_simple_reflections()
2
>>> W = ColoredPermutations(Integer(2),Integer(3))
>>> W.number_of_simple_reflections()
3
>>> W = ColoredPermutations(Integer(4),Integer(3))
>>> W.number_of_simple_reflections()
3

>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))                ␣
↪               # optional - gap3
>>> W.number_of_simple_reflections()                            # optional -
↪ gap3
4
```

reflection(i)

Return the i-th reflection of `self`.

For i in $1, \ldots, N$, this gives the i-th reflection of `self`.

See also

- *reflection_index_set()*
- *reflections()*

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,4))                          # optional - gap3
sage: for i in W.reflection_index_set():                    # optional - gap3
....:     print('%s %s'%(i, W.reflection(i)))
```

(continues on next page)

(continued from previous page)

```
1  (1,7)(2,4)(5,6)(8,10)(11,12)
2  (1,4)(2,8)(3,5)(7,10)(9,11)
3  (2,5)(3,9)(4,6)(8,11)(10,12)
4  (1,8)(2,7)(3,6)(4,10)(9,12)
5  (1,6)(2,9)(3,8)(5,11)(7,12)
6  (1,11)(3,10)(4,9)(5,7)(6,12)
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))
↪       # optional - gap3
>>> for i in W.reflection_index_set():                # optional - gap3
...     print('%s %s'%(i, W.reflection(i)))
1  (1,7)(2,4)(5,6)(8,10)(11,12)
2  (1,4)(2,8)(3,5)(7,10)(9,11)
3  (2,5)(3,9)(4,6)(8,11)(10,12)
4  (1,8)(2,7)(3,6)(4,10)(9,12)
5  (1,6)(2,9)(3,8)(5,11)(7,12)
6  (1,11)(3,10)(4,9)(5,7)(6,12)
```

reflection_index_set()

Return the index set of the reflections of `self`.

See also

- *reflection()*
- *reflections()*

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,4))
sage: W.reflection_index_set()
(1, 2, 3, 4, 5, 6)
sage: W = ReflectionGroup((1,1,4), reflection_index_set=[1,3,'asdf',7,9,
↪11])
sage: W.reflection_index_set()
(1, 3, 'asdf', 7, 9, 11)
sage: W = ReflectionGroup((1,1,4), reflection_index_set=('a','b','c','d',
↪'e','f'))
sage: W.reflection_index_set()
('a', 'b', 'c', 'd', 'e', 'f')
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))
>>> W.reflection_index_set()
(1, 2, 3, 4, 5, 6)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), reflection_
↪index_set=[Integer(1),Integer(3),'asdf',Integer(7),Integer(9),
↪Integer(11)])
```

(continues on next page)

(continued from previous page)

```
>>> W.reflection_index_set()
(1, 3, 'asdf', 7, 9, 11)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), reflection_
↪index_set=('a','b','c','d','e','f'))
>>> W.reflection_index_set()
('a', 'b', 'c', 'd', 'e', 'f')
```

reflections()

Return a finite family containing the reflections of `self`, indexed by *reflection_index_set()*.

See also

- *reflection()*
- *reflection_index_set()*

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: reflections = W.reflections()
sage: for index in sorted(reflections.keys()):
....:     print('%s %s'%(index, reflections[index]))
1 (1,4)(2,3)(5,6)
2 (1,3)(2,5)(4,6)
3 (1,5)(2,4)(3,6)
sage: W = ReflectionGroup((1,1,3),
....:                     reflection_index_set=['a','b','c'])
sage: reflections = W.reflections()
sage: for index in sorted(reflections.keys()):
....:     print('%s %s'%(index, reflections[index]))
a (1,4)(2,3)(5,6)
b (1,3)(2,5)(4,6)
c (1,5)(2,4)(3,6)
sage: W = ReflectionGroup((3,1,1))
sage: reflections = W.reflections()
sage: for index in sorted(reflections.keys()):
....:     print('%s %s'%(index, reflections[index]))
1 (1,2,3)
2 (1,3,2)
sage: W = ReflectionGroup((1,1,3), (3,1,2))
sage: reflections = W.reflections()
sage: for index in sorted(reflections.keys()):
....:     print('%s %s'%(index, reflections[index]))
1 (1,6)(2,5)(7,8)
2 (1,5)(2,7)(6,8)
3 (3,9,15)(4,10,16)(12,17,23)(14,18,24)(20,25,29)(21,22,26)(27,28,30)
4 (3,11)(4,12)(9,13)(10,14)(15,19)(16,20)(17,21)(18,22)(23,27)(24,28)(25,
↪26)(29,30)
5 (1,7)(2,6)(5,8)
6 (3,19)(4,25)(9,11)(10,17)(12,28)(13,15)(14,30)(16,18)(20,27)(21,29)(22,
```

(continues on next page)

(continued from previous page)

```
↪23)(24,26)
7 (4,21,27)(10,22,28)(11,13,19)(12,14,20)(16,26,30)(17,18,25)(23,24,29)
8 (3,13)(4,24)(9,19)(10,29)(11,15)(12,26)(14,21)(16,23)(17,30)(18,27)(20,
↪22)(25,28)
9 (3,15,9)(4,16,10)(12,23,17)(14,24,18)(20,29,25)(21,26,22)(27,30,28)
10 (4,27,21)(10,28,22)(11,19,13)(12,20,14)(16,30,26)(17,25,18)(23,29,24)
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> reflections = W.reflections()
>>> for index in sorted(reflections.keys()):
...     print('%s %s'%(index, reflections[index]))
1 (1,4)(2,3)(5,6)
2 (1,3)(2,5)(4,6)
3 (1,5)(2,4)(3,6)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)),
...                     reflection_index_set=['a','b','c'])
>>> reflections = W.reflections()
>>> for index in sorted(reflections.keys()):
...     print('%s %s'%(index, reflections[index]))
a (1,4)(2,3)(5,6)
b (1,3)(2,5)(4,6)
c (1,5)(2,4)(3,6)
>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(1)))
>>> reflections = W.reflections()
>>> for index in sorted(reflections.keys()):
...     print('%s %s'%(index, reflections[index]))
1 (1,2,3)
2 (1,3,2)
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), (Integer(3),
↪Integer(1),Integer(2)))
>>> reflections = W.reflections()
>>> for index in sorted(reflections.keys()):
...     print('%s %s'%(index, reflections[index]))
1 (1,6)(2,5)(7,8)
2 (1,5)(2,7)(6,8)
3 (3,9,15)(4,10,16)(12,17,23)(14,18,24)(20,25,29)(21,22,26)(27,28,30)
4 (3,11)(4,12)(9,13)(10,14)(15,19)(16,20)(17,21)(18,22)(23,27)(24,28)(25,
↪26)(29,30)
5 (1,7)(2,6)(5,8)
6 (3,19)(4,25)(9,11)(10,17)(12,28)(13,15)(14,30)(16,18)(20,27)(21,29)(22,
↪23)(24,26)
7 (4,21,27)(10,22,28)(11,13,19)(12,14,20)(16,26,30)(17,18,25)(23,24,29)
8 (3,13)(4,24)(9,19)(10,29)(11,15)(12,26)(14,21)(16,23)(17,30)(18,27)(20,
↪22)(25,28)
9 (3,15,9)(4,16,10)(12,23,17)(14,24,18)(20,29,25)(21,26,22)(27,30,28)
10 (4,27,21)(10,28,22)(11,19,13)(12,20,14)(16,30,26)(17,25,18)(23,29,24)
```

semigroup_generators()

alias of *group_generators()*.

simple_reflection(*i*)

Return the i-th simple reflection s_i of `self`.

INPUT:

- `i` – an element from the index set

See also

- *index_set()*
- *simple_reflections()*

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: W.simple_reflection(1)
(0, 2, 1, 3)
sage: s = W.simple_reflections()
sage: s[1]
(0, 2, 1, 3)

sage: W = ReflectionGroup((1,1,4), index_set=[1,3,'asdf'])       #␣
↪optional - gap3
sage: for i in W.index_set():                                    #␣
↪optional - gap3
....:     print('%s %s'%(i, W.simple_reflection(i)))
1 (1,7)(2,4)(5,6)(8,10)(11,12)
3 (1,4)(2,8)(3,5)(7,10)(9,11)
asdf (2,5)(3,9)(4,6)(8,11)(10,12)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> W.simple_reflection(Integer(1))
(0, 2, 1, 3)
>>> s = W.simple_reflections()
>>> s[Integer(1)]
(0, 2, 1, 3)

>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)), index_
↪set=[Integer(1),Integer(3),'asdf'])       # optional - gap3
>>> for i in W.index_set():                                    # optional -
↪ gap3
...     print('%s %s'%(i, W.simple_reflection(i)))
1 (1,7)(2,4)(5,6)(8,10)(11,12)
3 (1,4)(2,8)(3,5)(7,10)(9,11)
asdf (2,5)(3,9)(4,6)(8,11)(10,12)
```

simple_reflection_orders()

Return the orders of the simple reflections.

EXAMPLES:

```
sage: # needs sage.rings.number_field
sage: W = WeylGroup(['B',3])
sage: W.simple_reflection_orders()
[2, 2, 2]
sage: W = CoxeterGroup(['C',4])
sage: W.simple_reflection_orders()
[2, 2, 2, 2]

sage: SymmetricGroup(5).simple_reflection_orders()
[2, 2, 2, 2]

sage: C = ColoredPermutations(4, 3)                                    #␣
↪needs sage.combinat
sage: C.simple_reflection_orders()                                     #␣
↪needs sage.combinat
[2, 2, 4]
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> W = WeylGroup(['B',Integer(3)])
>>> W.simple_reflection_orders()
[2, 2, 2]
>>> W = CoxeterGroup(['C',Integer(4)])
>>> W.simple_reflection_orders()
[2, 2, 2, 2]

>>> SymmetricGroup(Integer(5)).simple_reflection_orders()
[2, 2, 2, 2]

>>> C = ColoredPermutations(Integer(4), Integer(3))                     ␣
↪              # needs sage.combinat
>>> C.simple_reflection_orders()                                      #␣
↪needs sage.combinat
[2, 2, 4]
```

simple_reflections()

Return the simple reflections $(s_i)_{i \in I}$ of `self` as a family indexed by *index_set()*.

See also

- *simple_reflection()*
- *index_set()*

EXAMPLES:

For the symmetric group, we recognize the simple transpositions:

```
sage: W = SymmetricGroup(4); W
Symmetric group of order 4! as a permutation group
sage: s = W.simple_reflections(); s
Finite family {1: (1,2), 2: (2,3), 3: (3,4)}
```

(continues on next page)

(continued from previous page)

```
sage: s[1]
(1,2)
sage: s[2]
(2,3)
sage: s[3]
(3,4)
```

```
>>> from sage.all import *
>>> W = SymmetricGroup(Integer(4)); W
Symmetric group of order 4! as a permutation group
>>> s = W.simple_reflections(); s
Finite family {1: (1,2), 2: (2,3), 3: (3,4)}
>>> s[Integer(1)]
(1,2)
>>> s[Integer(2)]
(2,3)
>>> s[Integer(3)]
(3,4)
```

Here are the simple reflections for a colored symmetric group and a reflection group:

```
sage: W = ColoredPermutations(1,3)                                    #␣
↪needs sage.combinat
sage: W.simple_reflections()                                          #␣
↪needs sage.combinat
Finite family {1: [[0, 0, 0], [2, 1, 3]], 2: [[0, 0, 0], [1, 3, 2]]}

sage: W = ReflectionGroup((1,1,3), index_set=['a','b'])               #␣
↪optional - gap3
sage: W.simple_reflections()                                          #␣
↪optional - gap3
Finite family {'a': (1,4)(2,3)(5,6), 'b': (1,3)(2,5)(4,6)}
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                    ␣
↪              # needs sage.combinat
>>> W.simple_reflections()                                           #␣
↪needs sage.combinat
Finite family {1: [[0, 0, 0], [2, 1, 3]], 2: [[0, 0, 0], [1, 3, 2]]}

>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)), index_set=['a
↪','b'])          # optional - gap3
>>> W.simple_reflections()                                        # optional -
↪ gap3
Finite family {'a': (1,4)(2,3)(5,6), 'b': (1,3)(2,5)(4,6)}
```

This default implementation uses *index_set()* and *simple_reflection()*.

some_elements()

Implement *Sets.ParentMethods.some_elements()* by returning some typical elements of `self`.

The result is currently composed of the simple reflections together with the unit and the result of *an_element()*.

EXAMPLES:

```
sage: W = WeylGroup(['A',3])                                                    #␣
↪needs sage.rings.number_field
sage: W.some_elements()                                                         #␣
↪needs sage.rings.number_field
[
[0 1 0 0]  [1 0 0 0]  [1 0 0 0]  [1 0 0 0]  [0 0 0 1]
[1 0 0 0]  [0 0 1 0]  [0 1 0 0]  [0 1 0 0]  [1 0 0 0]
[0 0 1 0]  [0 1 0 0]  [0 0 0 1]  [0 0 1 0]  [0 1 0 0]
[0 0 0 1], [0 0 0 1], [0 0 1 0], [0 0 0 1], [0 0 1 0]
]

sage: W = ColoredPermutations(1,4)                                              #␣
↪needs sage.combinat
sage: W.some_elements()                                                         #␣
↪needs sage.combinat
[[[0, 0, 0, 0], [2, 1, 3, 4]],
 [[0, 0, 0, 0], [1, 3, 2, 4]],
 [[0, 0, 0, 0], [1, 2, 4, 3]],
 [[0, 0, 0, 0], [1, 2, 3, 4]],
 [[0, 0, 0, 0], [4, 1, 2, 3]]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)])                                             ␣
↪     # needs sage.rings.number_field
>>> W.some_elements()                                                          #␣
↪needs sage.rings.number_field
[
[0 1 0 0]  [1 0 0 0]  [1 0 0 0]  [1 0 0 0]  [0 0 0 1]
[1 0 0 0]  [0 0 1 0]  [0 1 0 0]  [0 1 0 0]  [1 0 0 0]
[0 0 1 0]  [0 1 0 0]  [0 0 0 1]  [0 0 1 0]  [0 1 0 0]
[0 0 0 1], [0 0 0 1], [0 0 1 0], [0 0 0 1], [0 0 1 0]
]

>>> W = ColoredPermutations(Integer(1),Integer(4))                              ␣
↪              # needs sage.combinat
>>> W.some_elements()                                                          #␣
↪needs sage.combinat
[[[0, 0, 0, 0], [2, 1, 3, 4]],
 [[0, 0, 0, 0], [1, 3, 2, 4]],
 [[0, 0, 0, 0], [1, 2, 4, 3]],
 [[0, 0, 0, 0], [1, 2, 3, 4]],
 [[0, 0, 0, 0], [4, 1, 2, 3]]]
```

class SubcategoryMethods

Bases: object

Irreducible()

Return the full subcategory of irreducible objects of `self`.

A complex reflection group, or generalized Coxeter group is *reducible* if its simple reflections can be split in two sets X and Y such that the elements of X commute with that of Y. In particular, the group is then direct product of $\langle X\rangle$ and $\langle Y\rangle$. It's *irreducible* otherwise.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().Irreducible()
Category of irreducible complex reflection groups
sage: CoxeterGroups().Irreducible()
Category of irreducible Coxeter groups
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().Irreducible()
Category of irreducible complex reflection groups
>>> CoxeterGroups().Irreducible()
Category of irreducible Coxeter groups
```

super_categories()

Return the super categories of `self`.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

4.31 Coxeter Group Algebras

class sage.categories.coxeter_group_algebras.**CoxeterGroupAlgebras**(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: `object`

demazure_lusztig_eigenvectors(*q1*, *q2*)

Return the family of eigenvectors for the Cherednik operators.

INPUT:

- `self` – a finite Coxeter group W
- `q1`, `q2` – two elements of the ground ring K

The affine Hecke algebra $H_{q_1,q_2}(\tilde{W})$ acts on the group algebra of W through the Demazure-Lusztig operators T_i. Its Cherednik operators Y^λ can be simultaneously diagonalized as long as q_1/q_2 is not a small root of unity [HST2008].

This method returns the family of joint eigenvectors, indexed by W.

See also

- *demazure_lusztig_operators()*
- sage.combinat.root_system.hecke_algebra_representation.CherednikOperatorsEigenvectors

EXAMPLES:

```
sage: W = WeylGroup(["B",2])
sage: W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
↪word())
sage: K = QQ['q1,q2'].fraction_field()
sage: q1, q2 = K.gens()
sage: KW = W.algebra(K)
sage: E = KW.demazure_lusztig_eigenvectors(q1,q2)
sage: E.keys()                                                          #␣
↪needs sage.rings.number_field
Weyl Group of type ['B', 2] (as a matrix group acting on the ambient␣
↪space)
sage: w = W.an_element()
sage: E[w]                                                              #␣
↪needs sage.rings.number_field
(q2/(-q1+q2))*2121 - (q2/(-q1+q2))*121 - 212 + 12
```

```
>>> from sage.all import *
>>> W = WeylGroup(["B",Integer(2)])
>>> W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
↪word())
>>> K = QQ['q1,q2'].fraction_field()
>>> q1, q2 = K.gens()
>>> KW = W.algebra(K)
>>> E = KW.demazure_lusztig_eigenvectors(q1,q2)
>>> E.keys()                                                            #␣
↪needs sage.rings.number_field
Weyl Group of type ['B', 2] (as a matrix group acting on the ambient␣
↪space)
>>> w = W.an_element()
>>> E[w]                                                                #␣
↪needs sage.rings.number_field
(q2/(-q1+q2))*2121 - (q2/(-q1+q2))*121 - 212 + 12
```

demazure_lusztig_operator_on_basis(*w*, *i*, *q1*, *q2*, *side='right'*)

Return the result of applying the i-th Demazure Lusztig operator on `w`.

INPUT:

- `w` – an element of the Coxeter group
- `i` – an element of the index set
- `q1`, `q2` – two elements of the ground ring
- `bar` – boolean (default: `False`)

See *demazure_lusztig_operators()* for details.

EXAMPLES:

```
sage: W = WeylGroup(["B",3])
sage: W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
↪word())
sage: K = QQ['q1,q2']
sage: q1, q2 = K.gens()
sage: KW = W.algebra(K)
sage: w = W.an_element()
sage: KW.demazure_lusztig_operator_on_basis(w, 0, q1, q2)
-q2*323123 + (q1+q2)*123
sage: KW.demazure_lusztig_operator_on_basis(w, 1, q1, q2)
q1*1231
sage: KW.demazure_lusztig_operator_on_basis(w, 2, q1, q2)
q1*1232
sage: KW.demazure_lusztig_operator_on_basis(w, 3, q1, q2)
(q1+q2)*123 - q2*12
```

```
>>> from sage.all import *
>>> W = WeylGroup(["B",Integer(3)])
>>> W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
↪word())
>>> K = QQ['q1,q2']
>>> q1, q2 = K.gens()
>>> KW = W.algebra(K)
>>> w = W.an_element()
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(0), q1, q2)
-q2*323123 + (q1+q2)*123
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(1), q1, q2)
q1*1231
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(2), q1, q2)
q1*1232
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(3), q1, q2)
(q1+q2)*123 - q2*12
```

At $q_1 = 1$ and $q_2 = 0$ we recover the action of the isobaric divided differences π_i:

```
sage: KW.demazure_lusztig_operator_on_basis(w, 0, 1, 0)
123
sage: KW.demazure_lusztig_operator_on_basis(w, 1, 1, 0)
1231
sage: KW.demazure_lusztig_operator_on_basis(w, 2, 1, 0)
1232
sage: KW.demazure_lusztig_operator_on_basis(w, 3, 1, 0)
123
```

```
>>> from sage.all import *
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(0), Integer(1),␣
↪Integer(0))
123
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(1), Integer(1),␣
↪Integer(0))
1231
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(2), Integer(1),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(0))
1232
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(3), Integer(1),␣
↪Integer(0))
123
```

At $q_1 = 1$ and $q_2 = -1$ we recover the action of the simple reflection s_i:

```
sage: KW.demazure_lusztig_operator_on_basis(w, 0, 1, -1)
323123
sage: KW.demazure_lusztig_operator_on_basis(w, 1, 1, -1)
1231
sage: KW.demazure_lusztig_operator_on_basis(w, 2, 1, -1)
1232
sage: KW.demazure_lusztig_operator_on_basis(w, 3, 1, -1)
12
```

```
>>> from sage.all import *
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(0), Integer(1), -
↪Integer(1))
323123
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(1), Integer(1), -
↪Integer(1))
1231
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(2), Integer(1), -
↪Integer(1))
1232
>>> KW.demazure_lusztig_operator_on_basis(w, Integer(3), Integer(1), -
↪Integer(1))
12
```

demazure_lusztig_operators(*q1*, *q2*, *side='right'*, *affine=True*)

Return the Demazure Lusztig operators acting on `self`.

INPUT:

- `q1`, `q2` – two elements of the ground ring K
- `side` – `'left'` or `'right'` (default: `'right'`); which side to act upon
- `affine` – boolean (default: `True`)

The Demazure-Lusztig operator T_i is the linear map $R \to R$ obtained by interpolating between the simple projection π_i (see *simple_projection()*) and the simple reflection s_i so that T_i has eigenvalues q_1 and q_2:

$$(q_1 + q_2)\pi_i - q_2 s_i.$$

The Demazure-Lusztig operators give the usual representation of the operators T_i of the q_1, q_2 Hecke algebra associated to the Coxeter group.

For a finite Coxeter group, and if `affine=True`, the Demazure-Lusztig operators $T_1, \ldots, T_n$ are completed by T_0 to implement the level 0 action of the affine Hecke algebra.

EXAMPLES:

```
sage: W = WeylGroup(["B",3])
sage: W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
```

(continues on next page)

(continued from previous page)

```
↪word())
sage: K = QQ['q1,q2']
sage: q1, q2 = K.gens()
sage: KW = W.algebra(K)
sage: T = KW.demazure_lusztig_operators(q1, q2, affine=True)
sage: x = KW.monomial(W.an_element()); x
123
sage: T[0](x)
-q2*323123 + (q1+q2)*123
sage: T[1](x)
q1*1231
sage: T[2](x)
q1*1232
sage: T[3](x)
(q1+q2)*123 - q2*12

sage: T._test_relations()
```

```
>>> from sage.all import *
>>> W = WeylGroup(["B",Integer(3)])
>>> W.element_class._repr_=lambda x: "".join(str(i) for i in x.reduced_
↪word())
>>> K = QQ['q1,q2']
>>> q1, q2 = K.gens()
>>> KW = W.algebra(K)
>>> T = KW.demazure_lusztig_operators(q1, q2, affine=True)
>>> x = KW.monomial(W.an_element()); x
123
>>> T[Integer(0)](x)
-q2*323123 + (q1+q2)*123
>>> T[Integer(1)](x)
q1*1231
>>> T[Integer(2)](x)
q1*1232
>>> T[Integer(3)](x)
(q1+q2)*123 - q2*12

>>> T._test_relations()
```

Note

For a finite Weyl group W, the level 0 action of the affine Weyl group $\tilde{W}$ only depends on the Coxeter diagram of the affinization, not its Dynkin diagram. Hence it is possible to explore all cases using only untwisted affinizations.

4.32 Coxeter Groups

class sage.categories.coxeter_groups.**CoxeterGroups**

Bases: *Category_singleton*

The category of Coxeter groups.

A *Coxeter group* is a group W with a distinguished (finite) family of involutions $(s_i)_{i \in I}$, called the *simple reflections*, subject to relations of the form $(s_i s_j)^{m_{i,j}} = 1$.

I is the *index set* of W and $|I|$ is the *rank* of W.

See Wikipedia article Coxeter_group for details.

EXAMPLES:

```
sage: C = CoxeterGroups(); C
Category of Coxeter groups
sage: C.super_categories()
[Category of generalized Coxeter groups]

sage: W = C.example(); W
The symmetric group on {0, ..., 3}

sage: W.simple_reflections()
Finite family {0: (1, 0, 2, 3), 1: (0, 2, 1, 3), 2: (0, 1, 3, 2)}
```

```
>>> from sage.all import *
>>> C = CoxeterGroups(); C
Category of Coxeter groups
>>> C.super_categories()
[Category of generalized Coxeter groups]

>>> W = C.example(); W
The symmetric group on {0, ..., 3}

>>> W.simple_reflections()
Finite family {0: (1, 0, 2, 3), 1: (0, 2, 1, 3), 2: (0, 1, 3, 2)}
```

Here are some further examples:

```
sage: FiniteCoxeterGroups().example()
The 5-th dihedral group of order 10
sage: FiniteWeylGroups().example()
The symmetric group on {0, ..., 3}
sage: WeylGroup(["B", 3])                                                    #␣
↪needs sage.combinat sage.groups
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)

sage: S4 = SymmetricGroup(4); S4                                             #␣
↪needs sage.groups
Symmetric group of order 4! as a permutation group
sage: S4 in CoxeterGroups().Finite()                                         #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> FiniteCoxeterGroups().example()
The 5-th dihedral group of order 10
>>> FiniteWeylGroups().example()
```

(continues on next page)

(continued from previous page)

```
The symmetric group on {0, ..., 3}
>>> WeylGroup(["B", Integer(3)])
↪     # needs sage.combinat sage.groups
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)

>>> S4 = SymmetricGroup(Integer(4)); S4
↪     # needs sage.groups
Symmetric group of order 4! as a permutation group
>>> S4 in CoxeterGroups().Finite()
↪needs sage.groups
True
```

Those will eventually be also in this category:

```
sage: DihedralGroup(5)
↪needs sage.groups
Dihedral group of order 10 as a permutation group
```

```
>>> from sage.all import *
>>> DihedralGroup(Integer(5))
↪     # needs sage.groups
Dihedral group of order 10 as a permutation group
```

Todo

add a demo of usual computations on Coxeter groups.

See also

- sage.combinat.root_system.root_system
- *WeylGroups*
- *GeneralizedCoxeterGroups*

Warning

It is assumed that morphisms in this category preserve the distinguished choice of simple reflections. In particular, subobjects in this category are parabolic subgroups. In this sense, this category might be better named `Coxeter Systems`. In the long run we might want to have two distinct categories, one for Coxeter groups (with morphisms being just group morphisms) and one for Coxeter systems:

```
sage: CoxeterGroups().is_full_subcategory(Groups())
False
sage: from sage.categories.generalized_coxeter_groups import
↪GeneralizedCoxeterGroups
sage: CoxeterGroups().is_full_subcategory(GeneralizedCoxeterGroups())
True
```

```
>>> from sage.all import *
>>> CoxeterGroups().is_full_subcategory(Groups())
False
>>> from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
>>> CoxeterGroups().is_full_subcategory(GeneralizedCoxeterGroups())
True
```

Algebras

alias of *CoxeterGroupAlgebras*

class ElementMethods

Bases: object

absolute_chain()

Return a (saturated) chain in absolute order from 1 to self.

See also

absolute_chain_reflections()

EXAMPLES:

```
sage: W = CoxeterGroup(['A', 2, 1])
sage: (r, s, t) = W.simple_reflections()
sage: (r * s * r * t).absolute_chain()
[
[1 0 0]  [ 0 -1  2]  [ 2  1 -2]
[0 1 0]  [-1  0  2]  [ 1  2 -2]
[0 0 1], [ 0  0  1], [ 1  1 -1]
]
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A', Integer(2), Integer(1)])
>>> (r, s, t) = W.simple_reflections()
>>> (r * s * r * t).absolute_chain()
[
[1 0 0]  [ 0 -1  2]  [ 2  1 -2]
[0 1 0]  [-1  0  2]  [ 1  2 -2]
[0 0 1], [ 0  0  1], [ 1  1 -1]
]
```

absolute_chain_reflections()

Return a list of reflection which, when (left) multiplied in order, give self.

This method is based on Theorem 1.1 in [Dy2001], combined with the strong exchange condition. As an example, if W is a type A_2 Coxeter group with simple reflections a, b, then the absolute chain reflections for the element $w = ab$ is the list $[a, aba]$ as $w = (aba)a = ab$.

See also

absolute_length(), *absolute_chain()*

EXAMPLES:

```
sage: W = CoxeterGroup(["A",2,1])
sage: W.one().absolute_chain_reflections()
[]
sage: (r, s, t) = W.simple_reflections()
sage: r.absolute_chain_reflections()
[
[-1  1  1]
[ 0  1  0]
[ 0  0  1]
]
sage: (r * s).absolute_chain_reflections()
[
[-1  1  1]  [ 0 -1  2]
[ 0  1  0]  [-1  0  2]
[ 0  0  1], [ 0  0  1]
]
sage: (r * s * r * t).absolute_chain_reflections()
[
[ 0 -1  2]  [-1 -2  4]
[-1  0  2]  [-2 -1  4]
[ 0  0  1], [-1 -1  3]
]
sage: W = CoxeterGroup(['A', 3, 1])
sage: (r, s, t, u) = W.simple_reflections()
sage: (r * s * t * u).absolute_chain_reflections()
[
[-1  1  0  1]  [ 0 -1  1  1]  [ 0  0 -1  2]  [-3  2  0  2]
[ 0  1  0  0]  [-1  0  1  1]  [-1  1 -1  2]  [-2  2  0  1]
[ 0  0  1  0]  [ 0  0  1  0]  [-1  0  0  2]  [-2  1  1  1]
[ 0  0  0  1], [ 0  0  0  1], [ 0  0  0  1], [-2  1  0  2]
]
sage: (r * s * t * u * s).absolute_chain_reflections()
[
[-1  1  0  1]  [ 0  0 -1  2]  [-3  2  0  2]
[ 0  1  0  0]  [-1  1 -1  2]  [-2  2  0  1]
[ 0  0  1  0]  [-1  0  0  2]  [-2  1  1  1]
[ 0  0  0  1], [ 0  0  0  1], [-2  1  0  2]
]
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(["A",Integer(2),Integer(1)])
>>> W.one().absolute_chain_reflections()
[]
>>> (r, s, t) = W.simple_reflections()
>>> r.absolute_chain_reflections()
[
```

(continues on next page)

(continued from previous page)

```
[-1  1  1]
[ 0  1  0]
[ 0  0  1]
]
>>> (r * s).absolute_chain_reflections()
[
[-1  1  1]  [ 0 -1  2]
[ 0  1  0]  [-1  0  2]
[ 0  0  1], [ 0  0  1]
]
>>> (r * s * r * t).absolute_chain_reflections()
[
[ 0 -1  2]  [-1 -2  4]
[-1  0  2]  [-2 -1  4]
[ 0  0  1], [-1 -1  3]
]
>>> W = CoxeterGroup(['A', Integer(3), Integer(1)])
>>> (r, s, t, u) = W.simple_reflections()
>>> (r * s * t * u).absolute_chain_reflections()
[
[-1  1  0  1]  [ 0 -1  1  1]  [ 0  0 -1  2]  [-3  2  0  2]
[ 0  1  0  0]  [-1  0  1  1]  [-1  1 -1  2]  [-2  2  0  1]
[ 0  0  1  0]  [ 0  0  1  0]  [-1  0  0  2]  [-2  1  1  1]
[ 0  0  0  1], [ 0  0  0  1], [ 0  0  0  1], [-2  1  0  2]
]
>>> (r * s * t * u * s).absolute_chain_reflections()
[
[-1  1  0  1]  [ 0  0 -1  2]  [-3  2  0  2]
[ 0  1  0  0]  [-1  1 -1  2]  [-2  2  0  1]
[ 0  0  1  0]  [-1  0  0  2]  [-2  1  1  1]
[ 0  0  0  1], [ 0  0  0  1], [-2  1  0  2]
]
```

absolute_covers()

Return the list of covers of `self` in absolute order.

See also

absolute_length()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
sage: s = W.simple_reflections()
sage: w0 = s[1]
sage: w1 = s[1]*s[2]*s[3]
sage: list(w0.absolute_covers())
[
[0 0 1 0]  [0 1 0 0]  [0 1 0 0]  [0 0 0 1]  [0 1 0 0]
[1 0 0 0]  [1 0 0 0]  [0 0 1 0]  [1 0 0 0]  [0 0 0 1]
```

(continues on next page)

(continued from previous page)

```
[0 1 0 0]  [0 0 0 1]  [1 0 0 0]  [0 0 1 0]  [0 0 1 0]
[0 0 0 1], [0 0 1 0], [0 0 0 1], [0 1 0 0], [1 0 0 0]
]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> s = W.simple_reflections()
>>> w0 = s[Integer(1)]
>>> w1 = s[Integer(1)]*s[Integer(2)]*s[Integer(3)]
>>> list(w0.absolute_covers())
[
[0 0 1 0]  [0 1 0 0]  [0 1 0 0]  [0 0 0 1]  [0 1 0 0]
[1 0 0 0]  [1 0 0 0]  [0 0 1 0]  [1 0 0 0]  [0 0 0 1]
[0 1 0 0]  [0 0 0 1]  [1 0 0 0]  [0 0 1 0]  [0 0 1 0]
[0 0 0 1], [0 0 1 0], [0 0 0 1], [0 1 0 0], [1 0 0 0]
]
```

absolute_le(*other*)

Return whether `self` is smaller than `other` in the absolute order.

A general reflection is an element of the form ws_iw^{-1}, where s_i is a simple reflection. The absolute order is defined analogously to the weak order but using general reflections rather than just simple reflections.

This partial order can be used to define noncrossing partitions associated with this Coxeter group.

See also

absolute_length()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
sage: s = W.simple_reflections()
sage: w0 = s[1]
sage: w1 = s[1]*s[2]*s[3]
sage: w0.absolute_le(w1)
True
sage: w1.absolute_le(w0)
False
sage: w1.absolute_le(w1)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> s = W.simple_reflections()
>>> w0 = s[Integer(1)]
>>> w1 = s[Integer(1)]*s[Integer(2)]*s[Integer(3)]
>>> w0.absolute_le(w1)
True
```

(continues on next page)

(continued from previous page)

```
>>> w1.absolute_le(w0)
False
>>> w1.absolute_le(w1)
True
```

absolute_length()

Return the absolute length of `self`.

The absolute length is the length of the shortest expression of the element as a product of reflections. In general, we use Theorem 1.1 in [Dy2001].

See also

absolute_le(), *absolute_chain()*

EXAMPLES:

```
sage: W = WeylGroup(["A", 3])                                          #␣
↪needs sage.combinat sage.groups
sage: s = W.simple_reflections()                                       #␣
↪needs sage.combinat sage.groups
sage: (s[1]*s[2]*s[3]).absolute_length()                               #␣
↪needs sage.combinat sage.groups
3

sage: W = SymmetricGroup(4)                                            #␣
↪needs sage.groups
sage: s = W.simple_reflections()                                       #␣
↪needs sage.groups
sage: (s[3]*s[2]*s[1]).absolute_length()                               #␣
↪needs sage.combinat sage.groups
3

sage: W = CoxeterGroup(["A",2,1])
sage: (r, s, t) = W.simple_reflections()
sage: (r * s * r * t).absolute_length()
2
sage: W.one().absolute_length()
0
sage: r.absolute_length()
1
sage: (r * s).absolute_length()
2
sage: (r * s * r).absolute_length()
1
sage: W = CoxeterGroup(['A', 3, 1])
sage: (r, s, t, u) = W.simple_reflections()
sage: (r * s * t * u).absolute_length()
4
sage: (r * s * t * u * s).absolute_length()
3
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])                                    ␣
↪     # needs sage.combinat sage.groups
>>> s = W.simple_reflections()                                          #␣
↪needs sage.combinat sage.groups
>>> (s[Integer(1)]*s[Integer(2)]*s[Integer(3)]).absolute_length()       ␣
↪                        # needs sage.combinat sage.groups
3

>>> W = SymmetricGroup(Integer(4))                                      ␣
↪     # needs sage.groups
>>> s = W.simple_reflections()                                          #␣
↪needs sage.groups
>>> (s[Integer(3)]*s[Integer(2)]*s[Integer(1)]).absolute_length()       ␣
↪                        # needs sage.combinat sage.groups
3

>>> W = CoxeterGroup(["A",Integer(2),Integer(1)])
>>> (r, s, t) = W.simple_reflections()
>>> (r * s * r * t).absolute_length()
2
>>> W.one().absolute_length()
0
>>> r.absolute_length()
1
>>> (r * s).absolute_length()
2
>>> (r * s * r).absolute_length()
1
>>> W = CoxeterGroup(['A', Integer(3), Integer(1)])
>>> (r, s, t, u) = W.simple_reflections()
>>> (r * s * t * u).absolute_length()
4
>>> (r * s * t * u * s).absolute_length()
3
```

apply_demazure_product(*element*, *side='right'*, *length_increasing=True*)

Return the Demazure or 0-Hecke product of `self` with another Coxeter group element.

See *CoxeterGroups.ParentMethods.simple_projections()*.

INPUT:

- **element – either an element of the same Coxeter** group as `self` or a tuple or a list (such as a reduced word) of elements from the index set of the Coxeter group.
- **side – `'left'` or `'right'` (default: `'right'`); the** side of `self` on which the element should be applied. If `side` is 'left' then the operation is applied on the left.
- **length_increasing – boolean (default: `True`)** whether to act length increasingly or decreasingly

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['C', 4], prefix='s')
```

(continues on next page)

(continued from previous page)

```
sage: v = W.from_reduced_word([1,2,3,4,3,1])
sage: v.apply_demazure_product([1,3,4,3,3])
s4*s1*s2*s3*s4*s3*s1
sage: v.apply_demazure_product([1,3,4,3], side='left')
s3*s4*s1*s2*s3*s4*s2*s3*s1
sage: v.apply_demazure_product((1,3,4,3), side='left')
s3*s4*s1*s2*s3*s4*s2*s3*s1
sage: v.apply_demazure_product(v)
s2*s3*s4*s1*s2*s3*s4*s2*s3*s2*s1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['C', Integer(4)], prefix='s')
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(3),Integer(1)])
>>> v.apply_demazure_product([Integer(1),Integer(3),Integer(4),Integer(3),
↪Integer(3)])
s4*s1*s2*s3*s4*s3*s1
>>> v.apply_demazure_product([Integer(1),Integer(3),Integer(4),
↪Integer(3)], side='left')
s3*s4*s1*s2*s3*s4*s2*s3*s1
>>> v.apply_demazure_product((Integer(1),Integer(3),Integer(4),
↪Integer(3)), side='left')
s3*s4*s1*s2*s3*s4*s2*s3*s1
>>> v.apply_demazure_product(v)
s2*s3*s4*s1*s2*s3*s4*s2*s3*s2*s1
```

apply_simple_projection(*i*, *side='right'*, *length_increasing=True*)

Return the result of the application of the simple projection π_i (resp. $\overline{\pi}_i$) on `self`.

INPUT:

- `i` – an element of the index set of the Coxeter group
- `side` – `'left'` or `'right'` (default: `'right'`)
- `length_increasing` – boolean (default: `True`); specifying the direction of the projection

See *CoxeterGroups.ParentMethods.simple_projections()* for the definition of the simple projections.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.an_element()
sage: w
(1, 2, 3, 0)
sage: w.apply_simple_projection(2)
(1, 2, 3, 0)
sage: w.apply_simple_projection(2, length_increasing=False)
(1, 2, 0, 3)

sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['C', 4], prefix='s')
sage: v = W.from_reduced_word([1,2,3,4,3,1])
sage: v
s1*s2*s3*s4*s3*s1
```

(continues on next page)

(continued from previous page)

```
sage: v.apply_simple_projection(2)
s1*s2*s3*s4*s3*s1*s2
sage: v.apply_simple_projection(2, side='left')
s1*s2*s3*s4*s3*s1
sage: v.apply_simple_projection(1, length_increasing=False)
s1*s2*s3*s4*s3
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.an_element()
>>> w
(1, 2, 3, 0)
>>> w.apply_simple_projection(Integer(2))
(1, 2, 3, 0)
>>> w.apply_simple_projection(Integer(2), length_increasing=False)
(1, 2, 0, 3)

>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['C', Integer(4)], prefix='s')
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(4),
↪Integer(3),Integer(1)])
>>> v
s1*s2*s3*s4*s3*s1
>>> v.apply_simple_projection(Integer(2))
s1*s2*s3*s4*s3*s1*s2
>>> v.apply_simple_projection(Integer(2), side='left')
s1*s2*s3*s4*s3*s1
>>> v.apply_simple_projection(Integer(1), length_increasing=False)
s1*s2*s3*s4*s3
```

binary_factorizations (*predicate=The constant function (...) -> True*)

Return the set of all the factorizations $self = uv$ such that $l(self) = l(u) + l(v)$.

Iterating through this set is Constant Amortized Time (counting arithmetic operations in the Coxeter group as constant time) complexity, and memory linear in the length of `self`.

One can pass as optional argument a predicate p such that $p(u)$ implies $p(u')$ for any u left factor of `self` and u' left factor of u. Then this returns only the factorizations $self = uv$ such $p(u)$ holds.

EXAMPLES:

We construct the set of all factorizations of the maximal element of the group:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3])
sage: s = W.simple_reflections()
sage: w0 = W.from_reduced_word([1,2,3,1,2,1])
sage: w0.binary_factorizations().cardinality()
24
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3)])
```

(continues on next page)

(continued from previous page)

```
>>> s = W.simple_reflections()
>>> w0 = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(1),
↪Integer(2),Integer(1)])
>>> w0.binary_factorizations().cardinality()
24
```

The same number of factorizations, by bounded length:

```
sage: [w0.binary_factorizations(                                        #␣
↪needs sage.combinat sage.groups
....:       lambda u: u.length() <= l
....:   ).cardinality()
....:   for l in [-1,0,1,2,3,4,5,6]]
[0, 1, 4, 9, 15, 20, 23, 24]
```

```
>>> from sage.all import *
>>> [w0.binary_factorizations(                                          #␣
↪needs sage.combinat sage.groups
...       lambda u: u.length() <= l
...   ).cardinality()
...   for l in [-Integer(1),Integer(0),Integer(1),Integer(2),Integer(3),
↪Integer(4),Integer(5),Integer(6)]]
[0, 1, 4, 9, 15, 20, 23, 24]
```

The number of factorizations of the elements just below the maximal element:

```
sage: [(s[i]*w0).binary_factorizations().cardinality()                  #␣
↪needs sage.combinat sage.groups
....:   for i in [1,2,3]]
[12, 12, 12]
sage: w0.binary_factorizations(lambda u: False).cardinality()           #␣
↪needs sage.combinat sage.groups
0
```

```
>>> from sage.all import *
>>> [(s[i]*w0).binary_factorizations().cardinality()                    #␣
↪needs sage.combinat sage.groups
...   for i in [Integer(1),Integer(2),Integer(3)]]
[12, 12, 12]
>>> w0.binary_factorizations(lambda u: False).cardinality()             #␣
↪needs sage.combinat sage.groups
0
```

bruhat_le(*other*)

Return whether `self` <= `other` in the Bruhat order.

INPUT:

- `other` – an element of the same Coxeter group

OUTPUT: boolean

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
sage: u = W.from_reduced_word([1,2,1])
sage: v = W.from_reduced_word([1,2,3,2,1])
sage: u.bruhat_le(u)
True
sage: u.bruhat_le(v)
True
sage: v.bruhat_le(u)
False
sage: v.bruhat_le(v)
True
sage: s = W.simple_reflections()
sage: s[1].bruhat_le(W.one())
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> u = W.from_reduced_word([Integer(1),Integer(2),Integer(1)])
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(2),
↪Integer(1)])
>>> u.bruhat_le(u)
True
>>> u.bruhat_le(v)
True
>>> v.bruhat_le(u)
False
>>> v.bruhat_le(v)
True
>>> s = W.simple_reflections()
>>> s[Integer(1)].bruhat_le(W.one())
False
```

The implementation uses the equivalent condition that any reduced word for `other` contains a reduced word for `self` as subword. See Stembridge, A short derivation of the Möbius function for the Bruhat order. J. Algebraic Combinatoric 25 (2007), no. 2, 141–148, Proposition 1.1.

Complexity: $O(l * c)$, where l is the minimum of the lengths of u and of v, and c is the cost of the low level methods *first_descent()*, *has_descent()*, *apply_simple_reflection()*), etc. Those are typically $O(n)$, where n is the rank of the Coxeter group.

bruhat_lower_covers()

Return all elements that `self` covers in (strong) Bruhat order.

If `w = self` has a descent at i, then the elements that w covers are exactly $\{ws_i, u_1s_i, u_2s_i, ..., u_js_i\}$, where the u_k are elements that ws_i covers that also do not have a descent at i.

EXAMPLES:

```
sage: W = WeylGroup(["A", 3])                                                #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,2,3])                                       #␣
↪needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: print([v.reduced_word() for v in w.bruhat_lower_covers()])          #␣
↪needs sage.combinat sage.groups
[[3, 2], [2, 3]]

sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
sage: print([v.reduced_word()
....:        for v in W.simple_reflection(1).bruhat_lower_covers()])
[[]]
sage: print([v.reduced_word()
....:        for v in W.one().bruhat_lower_covers()])
[]
sage: W = WeylGroup(["B", 4, 1])
sage: w = W.from_reduced_word([0,2])
sage: print([v.reduced_word() for v in w.bruhat_lower_covers()])
[[2], [0]]
sage: W = WeylGroup("A3", prefix='s', implementation='permutation')
sage: s1, s2, s3 = W.simple_reflections()
sage: (s1*s2*s3*s1).bruhat_lower_covers()
[s2*s1*s3, s1*s2*s1, s1*s2*s3]
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])                                      ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(3)])           ␣
↪                          # needs sage.combinat sage.groups
>>> print([v.reduced_word() for v in w.bruhat_lower_covers()])          #␣
↪needs sage.combinat sage.groups
[[3, 2], [2, 3]]

>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> print([v.reduced_word()
...        for v in W.simple_reflection(Integer(1)).bruhat_lower_
↪covers()])
[[]]
>>> print([v.reduced_word()
...        for v in W.one().bruhat_lower_covers()])
[]
>>> W = WeylGroup(["B", Integer(4), Integer(1)])
>>> w = W.from_reduced_word([Integer(0),Integer(2)])
>>> print([v.reduced_word() for v in w.bruhat_lower_covers()])
[[2], [0]]
>>> W = WeylGroup("A3", prefix='s', implementation='permutation')
>>> s1, s2, s3 = W.simple_reflections()
>>> (s1*s2*s3*s1).bruhat_lower_covers()
[s2*s1*s3, s1*s2*s1, s1*s2*s3]
```

We now show how to construct the Bruhat poset:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
```

(continues on next page)

(continued from previous page)

```
sage: covers = tuple([u, v]
....:                for v in W for u in v.bruhat_lower_covers())
sage: P = Poset((W, covers), cover_relations=True)                    #␣
↪needs sage.graphs
sage: P.show()                                                        #␣
↪needs sage.graphs sage.plot
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> covers = tuple([u, v]
...                for v in W for u in v.bruhat_lower_covers())
>>> P = Poset((W, covers), cover_relations=True)                      #␣
↪needs sage.graphs
>>> P.show()                                                          #␣
↪needs sage.graphs sage.plot
```

Alternatively, one can just use:

```
sage: P = W.bruhat_poset()                                            #␣
↪needs sage.combinat sage.graphs sage.groups
```

```
>>> from sage.all import *
>>> P = W.bruhat_poset()                                              #␣
↪needs sage.combinat sage.graphs sage.groups
```

The algorithm is taken from Stembridge's 'coxeter/weyl' package for Maple.

bruhat_lower_covers_reflections()

Return all 2-tuples of lower_covers and reflections (`v`, `r`) where `v` is covered by `self` and `r` is the reflection such that `self` = `v` `r`.

ALGORITHM:

See *bruhat_lower_covers()*

EXAMPLES:

```
sage: W = WeylGroup(['A', 3], prefix='s')                             #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,1,2,1])                              #␣
↪needs sage.combinat sage.groups
sage: w.bruhat_lower_covers_reflections()                             #␣
↪needs sage.combinat sage.groups
[(s1*s2*s1, s1*s2*s3*s2*s1), (s3*s2*s1, s2), (s3*s1*s2, s1)]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)], prefix='s')                      ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),
↪Integer(1)])                                  # needs sage.combinat sage.
↪groups
>>> w.bruhat_lower_covers_reflections()                               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.groups
[(s1*s2*s1, s1*s2*s3*s2*s1), (s3*s2*s1, s2), (s3*s1*s2, s1)]
```

bruhat_upper_covers()

Return all elements that cover `self` in (strong) Bruhat order.

The algorithm works recursively, using the 'inverse' of the method described for lower covers *bruhat_lower_covers()*. Namely, it runs through all i in the index set. Let w equal `self`. If w has no right descent i, then ws_i is a cover; if w has a decent at i, then $u_j s_i$ is a cover of w where u_j is a cover of ws_i.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3, 1], prefix='s')                                #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([1,2,1])                                      #␣
↪needs sage.combinat sage.groups
sage: w.bruhat_upper_covers()                                               #␣
↪needs sage.combinat sage.groups
[s1*s2*s1*s0, s1*s2*s0*s1, s0*s1*s2*s1, s3*s1*s2*s1, s2*s3*s1*s2,␣
↪s1*s2*s3*s1]

sage: W = WeylGroup(['A', 3])                                               #␣
↪needs sage.combinat sage.groups
sage: w = W.long_element()                                                  #␣
↪needs sage.combinat sage.groups
sage: w.bruhat_upper_covers()                                               #␣
↪needs sage.combinat sage.groups
[]

sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3])
sage: w = W.from_reduced_word([1,2,1])
sage: S = [v for v in W if w in v.bruhat_lower_covers()]
sage: C = w.bruhat_upper_covers()
sage: set(S) == set(C)
True
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3), Integer(1)], prefix='s')                ␣
↪              # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1)])             ␣
↪                        # needs sage.combinat sage.groups
>>> w.bruhat_upper_covers()                                                 #␣
↪needs sage.combinat sage.groups
[s1*s2*s1*s0, s1*s2*s0*s1, s0*s1*s2*s1, s3*s1*s2*s1, s2*s3*s1*s2,␣
↪s1*s2*s3*s1]

>>> W = WeylGroup(['A', Integer(3)])                                        ␣
↪     # needs sage.combinat sage.groups
>>> w = W.long_element()                                                    #␣
↪needs sage.combinat sage.groups
>>> w.bruhat_upper_covers()                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.groups
[]

>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3)])
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1)])
>>> S = [v for v in W if w in v.bruhat_lower_covers()]
>>> C = w.bruhat_upper_covers()
>>> set(S) == set(C)
True
```

bruhat_upper_covers_reflections()

Return all 2-tuples of covers and reflections (v, r) where v covers self and r is the reflection such that self = v r.

ALGORITHM:

See *bruhat_upper_covers()*

EXAMPLES:

```
sage: W = WeylGroup(['A', 4], prefix='s')                                  #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,1,2,1])                                   #␣
↪needs sage.combinat sage.groups
sage: w.bruhat_upper_covers_reflections()                                  #␣
↪needs sage.combinat sage.groups
[(s1*s2*s3*s2*s1, s3), (s2*s3*s1*s2*s1, s2*s3*s2),
 (s3*s4*s1*s2*s1, s4), (s4*s3*s1*s2*s1, s1*s2*s3*s4*s3*s2*s1)]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(4)], prefix='s')                           ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),
↪Integer(1)])                                  # needs sage.combinat sage.
↪groups
>>> w.bruhat_upper_covers_reflections()                                    #␣
↪needs sage.combinat sage.groups
[(s1*s2*s3*s2*s1, s3), (s2*s3*s1*s2*s1, s2*s3*s2),
 (s3*s4*s1*s2*s1, s4), (s4*s3*s1*s2*s1, s1*s2*s3*s4*s3*s2*s1)]
```

canonical_matrix()

Return the matrix of self in the canonical faithful representation.

This is an n-dimension real faithful essential representation, where n is the number of generators of the Coxeter group. Note that this is not always the most natural matrix representation, for instance in type A_n.

EXAMPLES:

```
sage: W = WeylGroup(["A", 3])                                              #␣
↪needs sage.combinat sage.groups
sage: s = W.simple_reflections()                                           #␣
↪needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: (s[1]*s[2]*s[3]).canonical_matrix()                                   #␣
↪needs sage.combinat sage.groups
[ 0  0 -1]
[ 1  0 -1]
[ 0  1 -1]
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])                                        ␣
↪     # needs sage.combinat sage.groups
>>> s = W.simple_reflections()                                             #␣
↪needs sage.combinat sage.groups
>>> (s[Integer(1)]*s[Integer(2)]*s[Integer(3)]).canonical_matrix()          ␣
↪                        # needs sage.combinat sage.groups
[ 0  0 -1]
[ 1  0 -1]
[ 0  1 -1]
```

coset_representative(*index_set*, *side='right'*)

Return the unique shortest element of the Coxeter group W which is in the same left (resp. right) coset as `self`, with respect to the parabolic subgroup W_I.

INPUT:

- `index_set` – a subset (or iterable) of the nodes of the Dynkin diagram
- `side` – `'left'` or `'right'`

EXAMPLES:

```
sage: W = CoxeterGroups().example(5)
sage: s = W.simple_reflections()
sage: w = s[2]*s[1]*s[3]
sage: w.coset_representative([]).reduced_word()
[2, 3, 1]
sage: w.coset_representative([1]).reduced_word()
[2, 3]
sage: w.coset_representative([1,2]).reduced_word()
[2, 3]
sage: w.coset_representative([1,3]               ).reduced_word()
[2]
sage: w.coset_representative([2,3]               ).reduced_word()
[2, 1]
sage: w.coset_representative([1,2,3]             ).reduced_word()
[]
sage: w.coset_representative([],      side='left').reduced_word()
[2, 3, 1]
sage: w.coset_representative([1],     side='left').reduced_word()
[2, 3, 1]
sage: w.coset_representative([1,2],   side='left').reduced_word()
[3]
sage: w.coset_representative([1,3],   side='left').reduced_word()
[2, 3, 1]
sage: w.coset_representative([2,3],   side='left').reduced_word()
[1]
sage: w.coset_representative([1,2,3], side='left').reduced_word()
```

(continues on next page)

(continued from previous page)

```
[]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(Integer(5))
>>> s = W.simple_reflections()
>>> w = s[Integer(2)]*s[Integer(1)]*s[Integer(3)]
>>> w.coset_representative([]).reduced_word()
[2, 3, 1]
>>> w.coset_representative([Integer(1)]).reduced_word()
[2, 3]
>>> w.coset_representative([Integer(1),Integer(2)]).reduced_word()
[2, 3]
>>> w.coset_representative([Integer(1),Integer(3)]               ).
↪reduced_word()
[2]
>>> w.coset_representative([Integer(2),Integer(3)]               ).
↪reduced_word()
[2, 1]
>>> w.coset_representative([Integer(1),Integer(2),Integer(3)]            ␣
↪).reduced_word()
[]
>>> w.coset_representative([],      side='left').reduced_word()
[2, 3, 1]
>>> w.coset_representative([Integer(1)],     side='left').reduced_word()
[2, 3, 1]
>>> w.coset_representative([Integer(1),Integer(2)],   side='left').
↪reduced_word()
[3]
>>> w.coset_representative([Integer(1),Integer(3)],   side='left').
↪reduced_word()
[2, 3, 1]
>>> w.coset_representative([Integer(2),Integer(3)],   side='left').
↪reduced_word()
[1]
>>> w.coset_representative([Integer(1),Integer(2),Integer(3)], side='left
↪').reduced_word()
[]
```

cover_reflections(*side='right'*)

Return the set of reflections `t` such that `self t` covers `self`.

If `side` is 'left', `t self` covers `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 4], prefix='s')
sage: w = W.from_reduced_word([3,1,2,1])
sage: w.cover_reflections()
[s3, s2*s3*s2, s4, s1*s2*s3*s4*s3*s2*s1]
sage: w.cover_reflections(side='left')
[s4, s2, s1*s2*s1, s3*s4*s3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(4)], prefix='s')
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(1)])
>>> w.cover_reflections()
[s3, s2*s3*s2, s4, s1*s2*s3*s4*s3*s2*s1]
>>> w.cover_reflections(side='left')
[s4, s2, s1*s2*s1, s3*s4*s3]
```

coxeter_sorting_word(*c*)

Return the `c`-sorting word of `self`.

For a Coxeter element c and an element w, the c-sorting word of w is the lexicographic minimal reduced expression of w in the infinite word c^∞.

INPUT:

- `c` – a Coxeter element

OUTPUT: the `c`-sorting word of `self` as a list of integers

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: c = W.from_reduced_word([0,2,1])
sage: w = W.from_reduced_word([1,2,1,0,1])
sage: w.coxeter_sorting_word(c)
[2, 1, 2, 0, 1]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> c = W.from_reduced_word([Integer(0),Integer(2),Integer(1)])
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(0),
↪Integer(1)])
>>> w.coxeter_sorting_word(c)
[2, 1, 2, 0, 1]
```

deodhar_factor_element(*w*, *index_set*)

Return Deodhar's Bruhat order factoring element.

INPUT:

- `w` – an element of the same Coxeter group `W` as `self`
- `index_set` – a subset of Dynkin nodes defining a parabolic subgroup `W'` of `W`

It is assumed that `v = self` and `w` are minimum length coset representatives for `W/W'` such that `v` $\leq$ `w` in Bruhat order.

OUTPUT:

Deodhar's element `f(v,w)` is the unique element of `W'` such that, for all `v'` and `w'` in `W'`, `vv'` $\leq$ `ww'` in `W` if and only if `v'` $\leq$ `f(v,w) * w'` in `W'` where `*` is the Demazure product.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 5], prefix='s')
sage: v = W.from_reduced_word([5])
sage: w = W.from_reduced_word([4,5,2,3,1,2])
sage: v.deodhar_factor_element(w, [1,3,4])
```

(continues on next page)

(continued from previous page)

```
s3*s1
sage: W = WeylGroup(['C', 2])
sage: w = W.from_reduced_word([2,1])
sage: w.deodhar_factor_element(W.from_reduced_word([2]),[1])
Traceback (most recent call last):
...
ValueError: [2, 1] is not of minimum length in its coset
for the parabolic subgroup with index set [1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(5)], prefix='s')
>>> v = W.from_reduced_word([Integer(5)])
>>> w = W.from_reduced_word([Integer(4),Integer(5),Integer(2),Integer(3),
↪Integer(1),Integer(2)])
>>> v.deodhar_factor_element(w, [Integer(1),Integer(3),Integer(4)])
s3*s1
>>> W = WeylGroup(['C', Integer(2)])
>>> w = W.from_reduced_word([Integer(2),Integer(1)])
>>> w.deodhar_factor_element(W.from_reduced_word([Integer(2)]),
↪[Integer(1)])
Traceback (most recent call last):
...
ValueError: [2, 1] is not of minimum length in its coset
for the parabolic subgroup with index set [1]
```

REFERENCES:

- [Deo1987a]

deodhar_lift_down(*w*, *index_set*)

Letting `v = self`, given a Bruhat relation `v W' ≥ w W'` among cosets with respect to the subgroup `W'` given by the Dynkin node subset `index_set`, returns the Bruhat-maximum lift `x` of `wW'` such that `v ≥ x`.

INPUT:

- `w` – an element of the same Coxeter group `W` as `self`
- `index_set` – a subset of Dynkin nodes defining a parabolic subgroup `W'`

OUTPUT:

The unique Bruhat-maximum element `x` in `W` such that `x W' = w W'` and `v ≥ x`.

See also

sage.categories.coxeter_groups.CoxeterGroups.ElementMethods.deodhar_lift_up()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3], prefix='s')
sage: v = W.from_reduced_word([1,2,3,2])
sage: w = W.from_reduced_word([3,2])
```

(continues on next page)

(continued from previous page)

```
sage: v.deodhar_lift_down(w, [3])
s2*s3*s2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3)], prefix='s')
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(2)])
>>> w = W.from_reduced_word([Integer(3),Integer(2)])
>>> v.deodhar_lift_down(w, [Integer(3)])
s2*s3*s2
```

deodhar_lift_up(*w*, *index_set*)

Letting `v = self`, given a Bruhat relation `v W' ≤ w W'` among cosets with respect to the subgroup `W'` given by the Dynkin node subset `index_set`, returns the Bruhat-minimum lift `x` of `wW'` such that `v ≤ x`.

INPUT:

- `w` – an element of the same Coxeter group `W` as `self`
- `index_set` – a subset of Dynkin nodes defining a parabolic subgroup `W'`

OUTPUT:

The unique Bruhat-minimum element `x` in `W` such that `x W' = w W'` and `v ≤ x`.

See also

sage.categories.coxeter_groups.CoxeterGroups.ElementMethods.deodhar_lift_down()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3], prefix='s')
sage: v = W.from_reduced_word([1,2,3])
sage: w = W.from_reduced_word([1,3,2])
sage: v.deodhar_lift_up(w, [3])
s1*s2*s3*s2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3)], prefix='s')
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3)])
>>> w = W.from_reduced_word([Integer(1),Integer(3),Integer(2)])
>>> v.deodhar_lift_up(w, [Integer(3)])
s1*s2*s3*s2
```

descents(*side='right'*, *index_set=None*, *positive=False*)

Return the descents of self, as a list of elements of the index_set.

INPUT:

- `index_set` – a subset (as a list or iterable) of the nodes of the Dynkin diagram; (default: all of them)
- `side` – `'left'` or `'right'` (default: `'right'`)
- `positive` – boolean (default: `False`)

The `index_set` option can be used to restrict to the parabolic subgroup indexed by `index_set`.

If positive is `True`, then returns the non-descents instead

Todo

find a better name for `positive`: complement? non_descent?

Caveat: the return type may change to some other iterable (tuple, …) in the future. Please use keyword arguments also, as the order of the arguments may change as well.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0]*s[1]
sage: w.descents()
[1]
sage: w = s[0]*s[2]
sage: w.descents()
[0, 2]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)]*s[Integer(1)]
>>> w.descents()
[1]
>>> w = s[Integer(0)]*s[Integer(2)]
>>> w.descents()
[0, 2]
```

Todo

side, index_set, positive

first_descent(*side='right'*, *index_set=None*, *positive=False*)

Return the first left (resp. right) descent of self, as an element of `index_set`, or `None` if there is none.

See *descents()* for a description of the options.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[2]*s[0]
sage: w.first_descent()
0
sage: w = s[0]*s[2]
sage: w.first_descent()
0
sage: w = s[0]*s[1]
```

(continues on next page)

(continued from previous page)

```
sage: w.first_descent()
1
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(2)]*s[Integer(0)]
>>> w.first_descent()
0
>>> w = s[Integer(0)]*s[Integer(2)]
>>> w.first_descent()
0
>>> w = s[Integer(0)]*s[Integer(1)]
>>> w.first_descent()
1
```

has_descent(*i*, *side='right'*, *positive=False*)

Return whether i is a (left/right) descent of `self`.

See *descents()* for a description of the options.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0] * s[1] * s[2]
sage: w.has_descent(2)
True
sage: [ w.has_descent(i)                for i in [0,1,2] ]
[False, False, True]
sage: [ w.has_descent(i, side='left')   for i in [0,1,2] ]
[True, False, False]
sage: [ w.has_descent(i, positive=True) for i in [0,1,2] ]
[True, True, False]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)] * s[Integer(1)] * s[Integer(2)]
>>> w.has_descent(Integer(2))
True
>>> [ w.has_descent(i)                for i in [Integer(0),Integer(1),
↪Integer(2)] ]
[False, False, True]
>>> [ w.has_descent(i, side='left')   for i in [Integer(0),Integer(1),
↪Integer(2)] ]
[True, False, False]
>>> [ w.has_descent(i, positive=True) for i in [Integer(0),Integer(1),
↪Integer(2)] ]
[True, True, False]
```

This default implementation delegates the work to *has_left_descent()* and *has_right_descent()*.

has_full_support()

Return whether `self` has full support.

An element is said to have full support if its support contains all simple reflections.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.from_reduced_word([1,2,1])
sage: w.has_full_support()
False
sage: w = W.from_reduced_word([1,2,1,0,1])
sage: w.has_full_support()
True
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1)])
>>> w.has_full_support()
False
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(0),
↪Integer(1)])
>>> w.has_full_support()
True
```

has_left_descent(i)

Return whether i is a left descent of `self`.

This default implementation uses that a left descent of w is a right descent of w^{-1}.

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.has_left_descent(0)
True
sage: w.has_left_descent(1)
False
sage: w.has_left_descent(2)
False
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.has_left_descent(Integer(0))
True
>>> w.has_left_descent(Integer(1))
False
>>> w.has_left_descent(Integer(2))
False
```

has_right_descent(*i*)

Return whether i is a right descent of `self`.

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: w = W.an_element(); w
(1, 2, 3, 0)
sage: w.has_right_descent(0)
False
sage: w.has_right_descent(1)
False
sage: w.has_right_descent(2)
True
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> w = W.an_element(); w
(1, 2, 3, 0)
>>> w.has_right_descent(Integer(0))
False
>>> w.has_right_descent(Integer(1))
False
>>> w.has_right_descent(Integer(2))
True
```

inversions_as_reflections()

Return the set of reflections `r` such that `self r < self`.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3], prefix='s')                                   #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,1,2,1])                                    #␣
↪needs sage.combinat sage.groups
sage: w.inversions_as_reflections()                                         #␣
↪needs sage.combinat sage.groups
[s1, s1*s2*s1, s2, s1*s2*s3*s2*s1]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)], prefix='s')                            ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),
↪Integer(1)])                                  # needs sage.combinat sage.
↪groups
>>> w.inversions_as_reflections()                                           #␣
↪needs sage.combinat sage.groups
[s1, s1*s2*s1, s2, s1*s2*s3*s2*s1]
```

is_coxeter_sortable(*c*, *sorting_word=None*)

Return whether `self` is `c`-sortable.

Given a Coxeter element c, an element w is c-sortable if its c-sorting word decomposes into a sequence of weakly decreasing subwords of c.

INPUT:

- `c` – a Coxeter element
- `sorting_word` – sorting word (default: `None`); used to not recompute the c-sorting word if already computed

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: c = W.from_reduced_word([0,2,1])
sage: w = W.from_reduced_word([1,2,1,0,1])
sage: w.coxeter_sorting_word(c)
[2, 1, 2, 0, 1]
sage: w.is_coxeter_sortable(c)
False
sage: w = W.from_reduced_word([0,2,1,0,2])
sage: w.coxeter_sorting_word(c)
[2, 0, 1, 2, 0]
sage: w.is_coxeter_sortable(c)
True

sage: W = CoxeterGroup(['A', 3])                                    #␣
↪needs sage.combinat sage.groups
sage: c = W.from_reduced_word([1,2,3])                              #␣
↪needs sage.combinat sage.groups
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> c = W.from_reduced_word([Integer(0),Integer(2),Integer(1)])
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(0),
↪Integer(1)])
>>> w.coxeter_sorting_word(c)
[2, 1, 2, 0, 1]
>>> w.is_coxeter_sortable(c)
False
>>> w = W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0),
↪Integer(2)])
>>> w.coxeter_sorting_word(c)
[2, 0, 1, 2, 0]
>>> w.is_coxeter_sortable(c)
True

>>> W = CoxeterGroup(['A', Integer(3)])                             ␣
↪     # needs sage.combinat sage.groups
>>> c = W.from_reduced_word([Integer(1),Integer(2),Integer(3)])     ␣
↪                        # needs sage.combinat sage.groups
```

Number of c-sortable elements in A_3 (Catalan number):

```
sage: len([w for w in W if w.is_coxeter_sortable(c)])               #␣
↪needs sage.rings.number_field
14
```

```
>>> from sage.all import *
>>> len([w for w in W if w.is_coxeter_sortable(c)])                    #␣
↪needs sage.rings.number_field
14
```

is_fully_commutative()

Check if `self` is a fully-commutative element.

We use the characterization that an element w in a Coxeter system (W, S) is fully-commutative if and only if for every pair of generators $s, t \in S$ for which $m(s,t) > 2$, no reduced word of w contains the 'braid' word $sts\ldots$ of length $m(s,t)$ as a contiguous subword. See [Ste1996].

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = CoxeterGroup(['A', 3])
sage: len([1 for w in W if w.is_fully_commutative()])
14
sage: W = CoxeterGroup(['B', 3])
sage: len([1 for w in W if w.is_fully_commutative()])
24
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = CoxeterGroup(['A', Integer(3)])
>>> len([Integer(1) for w in W if w.is_fully_commutative()])
14
>>> W = CoxeterGroup(['B', Integer(3)])
>>> len([Integer(1) for w in W if w.is_fully_commutative()])
24
```

is_grassmannian(*side='right'*)

Return whether `self` is Grassmannian.

INPUT:

- `side` – `'left'` or `'right'` (default: `'right'`)

An element is Grassmannian if it has at most one descent on the right (resp. on the left).

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: s = W.simple_reflections()
sage: W.one().is_grassmannian()
True
sage: s[1].is_grassmannian()
True
sage: (s[1]*s[2]).is_grassmannian()
True
sage: (s[0]*s[1]).is_grassmannian()
True
sage: (s[1]*s[2]*s[1]).is_grassmannian()
False
```

(continues on next page)

(continued from previous page)

```
sage: (s[0]*s[2]*s[1]).is_grassmannian(side='left')
False
sage: (s[0]*s[2]*s[1]).is_grassmannian(side='right')
True
sage: (s[0]*s[2]*s[1]).is_grassmannian()
True
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> s = W.simple_reflections()
>>> W.one().is_grassmannian()
True
>>> s[Integer(1)].is_grassmannian()
True
>>> (s[Integer(1)]*s[Integer(2)]).is_grassmannian()
True
>>> (s[Integer(0)]*s[Integer(1)]).is_grassmannian()
True
>>> (s[Integer(1)]*s[Integer(2)]*s[Integer(1)]).is_grassmannian()
False

>>> (s[Integer(0)]*s[Integer(2)]*s[Integer(1)]).is_grassmannian(side='left
↪')
False
>>> (s[Integer(0)]*s[Integer(2)]*s[Integer(1)]).is_grassmannian(side=
↪'right')
True
>>> (s[Integer(0)]*s[Integer(2)]*s[Integer(1)]).is_grassmannian()
True
```

kazhdan_lusztig_cell(*side='left'*)

Compute the left, right, or two-sided Kazhdan-Lusztig cell containing the element `self` depending on the specified `side`.

Let C' denote the Kazhdan-Lusztig C'-basis of the Iwahori-Hecke algebra H of a Coxeter system (W, S). Two elements x, y of the Coxeter group W are said to lie in the same left Kazhdan-Lusztig cell if there exist sequences $x = w_1, w_2, \ldots, w_k = y$ and $y = u_1, u_2, \ldots, u_l = x$ such that for all $1 \leq i < k$ and all $1 \leq j < l$, there exist some Coxeter generators $s, t \in S$ for which $C'_{w_{i+1}}$ appears in $C'_s C'_{w_i}$ and $C'_{u_{j+1}}$ appears in $C'_s C'_{u_j}$ in H. Right and two-sided Kazhdan-Lusztig cells of W are defined similarly; see [Lus2013].

In this function, we compute products in the C' basis by using `IwahoriHeckeAlgebra.Cp`. As mentioned in that class, installing the optional package `coxeter3` is recommended (though not required) before using this function because the package speeds up product computations that are sometimes computationally infeasible without it.

INPUT:

- `w` – an element of `self`
- `side` – (default: `'left'`) the kind of cell to compute; must be either `'left'`, `'right'`, or `'two-sided'`

EXAMPLES:

We compute the left cell of the generator s_1 in type A_3 in three different implementations of the Coxeter

group. Note that the choice of implementation affects the representation of elements in the output cell but not the method used for the cell computation:

```
sage: W = CoxeterGroup('A3', implementation='permutation')                #␣
↪needs sage.combinat sage.groups
sage: s1, s2, s3 = W.simple_reflections()                                 #␣
↪needs sage.combinat sage.groups
sage: s1.kazhdan_lusztig_cell()                                           #␣
↪needs sage.combinat sage.groups
{(1,2,3,12)(4,5,10,11)(6,7,8,9),
 (1,2,10)(3,6,5)(4,7,8)(9,12,11),
 (1,7)(2,4)(5,6)(8,10)(11,12)}
```

```
>>> from sage.all import *
>>> W = CoxeterGroup('A3', implementation='permutation')                #␣
↪needs sage.combinat sage.groups
>>> s1, s2, s3 = W.simple_reflections()                                 #␣
↪needs sage.combinat sage.groups
>>> s1.kazhdan_lusztig_cell()                                           #␣
↪needs sage.combinat sage.groups
{(1,2,3,12)(4,5,10,11)(6,7,8,9),
 (1,2,10)(3,6,5)(4,7,8)(9,12,11),
 (1,7)(2,4)(5,6)(8,10)(11,12)}
```

The cell computation uses the optional package `coxeter3` in the background if available to speed up the computation, even in the different implementations:

```
sage: # optional - coxeter3, needs sage.combinat sage.groups sage.modules
sage: W = WeylGroup('A3', prefix='s')
sage: s1,s2,s3 = W.simple_reflections()
sage: s1.kazhdan_lusztig_cell()
{s3*s2*s1, s2*s1, s1}
sage: W = CoxeterGroup('A3', implementation='coxeter3')
sage: s1,s2,s3 = W.simple_reflections()
sage: s1.kazhdan_lusztig_cell()
{[1], [2, 1], [3, 2, 1]}
```

```
>>> from sage.all import *
>>> # optional - coxeter3, needs sage.combinat sage.groups sage.modules
>>> W = WeylGroup('A3', prefix='s')
>>> s1,s2,s3 = W.simple_reflections()
>>> s1.kazhdan_lusztig_cell()
{s3*s2*s1, s2*s1, s1}
>>> W = CoxeterGroup('A3', implementation='coxeter3')
>>> s1,s2,s3 = W.simple_reflections()
>>> s1.kazhdan_lusztig_cell()
{[1], [2, 1], [3, 2, 1]}
```

Next, we compute a right cell and a two-sided cell in A_3:

```
sage: # optional - coxeter3, needs sage.combinat sage.groups sage.modules
sage: W = CoxeterGroup('A3', implementation='coxeter3')
sage: s1,s2,s3 = W.simple_reflections()
```

(continues on next page)

(continued from previous page)

```
sage: w = s1 * s3
sage: w.kazhdan_lusztig_cell(side='right')
{[1, 3], [1, 3, 2]}
sage: w.kazhdan_lusztig_cell(side='two-sided')
{[1, 3], [1, 3, 2], [2, 1, 3], [2, 1, 3, 2]}
```

```
>>> from sage.all import *
>>> # optional - coxeter3, needs sage.combinat sage.groups sage.modules
>>> W = CoxeterGroup('A3', implementation='coxeter3')
>>> s1,s2,s3 = W.simple_reflections()
>>> w = s1 * s3
>>> w.kazhdan_lusztig_cell(side='right')
{[1, 3], [1, 3, 2]}
>>> w.kazhdan_lusztig_cell(side='two-sided')
{[1, 3], [1, 3, 2], [2, 1, 3], [2, 1, 3, 2]}
```

Some slightly longer computations in B_4:

```
sage: # optional - coxeter3, needs sage.combinat sage.groups sage.modules
sage: W = CoxeterGroup('B4', implementation='coxeter3')
sage: s1,s2,s3,s4 = W.simple_reflections()
sage: s1.kazhdan_lusztig_cell(side='right')     # long time (4 seconds)
{[1],
 [1, 2],
 [1, 2, 3],
 [1, 2, 3, 4],
 [1, 2, 3, 4, 3],
 [1, 2, 3, 4, 3, 2],
 [1, 2, 3, 4, 3, 2, 1]}
sage: (s4*s2*s3*s4).kazhdan_lusztig_cell(side='two-sided')      # long␣
↪time (8 seconds)
{[2, 3, 1],
 [2, 3, 1, 2],
 [2, 3, 4, 1],
 [2, 3, 4, 1, 2],
 [2, 3, 4, 1, 2, 3],
 [2, 3, 4, 1, 2, 3, 4],
 [2, 3, 4, 3, 1],
 [2, 3, 4, 3, 1, 2],
 ...
 [4, 3, 4, 2, 3, 4, 1, 2, 3, 4]}
```

```
>>> from sage.all import *
>>> # optional - coxeter3, needs sage.combinat sage.groups sage.modules
>>> W = CoxeterGroup('B4', implementation='coxeter3')
>>> s1,s2,s3,s4 = W.simple_reflections()
>>> s1.kazhdan_lusztig_cell(side='right')     # long time (4 seconds)
{[1],
 [1, 2],
 [1, 2, 3],
 [1, 2, 3, 4],
 [1, 2, 3, 4, 3],
```

(continues on next page)

(continued from previous page)

```
 [1, 2, 3, 4, 3, 2],
 [1, 2, 3, 4, 3, 2, 1]}
>>> (s4*s2*s3*s4).kazhdan_lusztig_cell(side='two-sided')      # long time␣
↪(8 seconds)
{[2, 3, 1],
 [2, 3, 1, 2],
 [2, 3, 4, 1],
 [2, 3, 4, 1, 2],
 [2, 3, 4, 1, 2, 3],
 [2, 3, 4, 1, 2, 3, 4],
 [2, 3, 4, 3, 1],
 [2, 3, 4, 3, 1, 2],
 ...
 [4, 3, 4, 2, 3, 4, 1, 2, 3, 4]}
```

left_inversions_as_reflections()

Return the set of reflections `r` such that `r self` < `self`.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3], prefix='s')                                  #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,1,2,1])                                   #␣
↪needs sage.combinat sage.groups
sage: w.left_inversions_as_reflections()                                   #␣
↪needs sage.combinat sage.groups
[s1, s3, s1*s2*s3*s2*s1, s2*s3*s2]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)], prefix='s')                           ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),
↪Integer(1)])                                  # needs sage.combinat sage.
↪groups
>>> w.left_inversions_as_reflections()                                     #␣
↪needs sage.combinat sage.groups
[s1, s3, s1*s2*s3*s2*s1, s2*s3*s2]
```

length()

Return the length of `self`.

This is the minimal length of a product of simple reflections giving `self`.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s1 = W.simple_reflection(1)
sage: s2 = W.simple_reflection(2)
sage: s1.length()
1
sage: (s1*s2).length()
2
sage: W = CoxeterGroups().example()
```

(continues on next page)

(continued from previous page)

```
sage: s = W.simple_reflections()
sage: w = s[0]*s[1]*s[0]
sage: w.length()
3
sage: W = CoxeterGroups().example()
sage: R.<x> = ZZ[]
sage: s = sum(x^w.length() for w in W)
sage: p = prod(sum(x^i for i in range(j)) for j in range(1, 5))
sage: s - p
0
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s1 = W.simple_reflection(Integer(1))
>>> s2 = W.simple_reflection(Integer(2))
>>> s1.length()
1
>>> (s1*s2).length()
2
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)]*s[Integer(1)]*s[Integer(0)]
>>> w.length()
3
>>> W = CoxeterGroups().example()
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> s = sum(x**w.length() for w in W)
>>> p = prod(sum(x**i for i in range(j)) for j in range(Integer(1),␣
↪Integer(5)))
>>> s - p
0
```

See also

reduced_word()

Todo

Should use reduced_word_iterator (or reverse_iterator)

lower_cover_reflections(*side='right'*)

Return the reflections `t` such that `self` covers `self t`.

If `side` is 'left', `self` covers `t self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 3],prefix='s')
sage: w = W.from_reduced_word([3,1,2,1])
```

(continues on next page)

(continued from previous page)

```
sage: w.lower_cover_reflections()
[s1*s2*s3*s2*s1, s2, s1]
sage: w.lower_cover_reflections(side='left')
[s2*s3*s2, s3, s1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(3)],prefix='s')
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(1)])
>>> w.lower_cover_reflections()
[s1*s2*s3*s2*s1, s2, s1]
>>> w.lower_cover_reflections(side='left')
[s2*s3*s2, s3, s1]
```

lower_covers(*side='right'*, *index_set=None*)

Return all elements that `self` covers in weak order.

INPUT:

- `side` – `'left'` or `'right'` (default: `'right'`)
- `index_set` – list of indices or `None`

OUTPUT: list

EXAMPLES:

```
sage: W = WeylGroup(['A', 3])                                                  #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,2,1])                                         #␣
↪needs sage.combinat sage.groups
sage: [x.reduced_word() for x in w.lower_covers()]                             #␣
↪needs sage.combinat sage.groups
[[3, 2]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)])                                            ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(1)])                 ␣
↪                          # needs sage.combinat sage.groups
>>> [x.reduced_word() for x in w.lower_covers()]                              #␣
↪needs sage.combinat sage.groups
[[3, 2]]
```

To obtain covers for left weak order, set the option side to 'left':

```
sage: [x.reduced_word() for x in w.lower_covers(side='left')]                  #␣
↪needs sage.combinat sage.groups
[[2, 1]]
sage: w = W.from_reduced_word([3,2,3,1])                                       #␣
↪needs sage.combinat sage.groups
sage: [x.reduced_word() for x in w.lower_covers()]                             #␣
↪needs sage.combinat sage.groups
[[2, 3, 2], [3, 2, 1]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.lower_covers(side='left')]           #␣
↪needs sage.combinat sage.groups
[[2, 1]]
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(3),
↪Integer(1)])                                  # needs sage.combinat sage.
↪groups
>>> [x.reduced_word() for x in w.lower_covers()]                      #␣
↪needs sage.combinat sage.groups
[[2, 3, 2], [3, 2, 1]]
```

Covers w.r.t. a parabolic subgroup are obtained with the option `index_set`:

```
sage: [x.reduced_word() for x in w.lower_covers(index_set=[1,2])]       #␣
↪needs sage.combinat sage.groups
[[2, 3, 2]]
sage: [x.reduced_word() for x in w.lower_covers(side='left')]           #␣
↪needs sage.combinat sage.groups
[[3, 2, 1], [2, 3, 1]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.lower_covers(index_set=[Integer(1),
↪Integer(2)])]       # needs sage.combinat sage.groups
[[2, 3, 2]]
>>> [x.reduced_word() for x in w.lower_covers(side='left')]           #␣
↪needs sage.combinat sage.groups
[[3, 2, 1], [2, 3, 1]]
```

min_demazure_product_greater(*element*)

Find the unique Bruhat-minimum element `u` such that `v` $\leq$ `w` * `u` where `v` is `self`, `w` is `element` and `*` is the Demazure product.

INPUT:

- `element` – is either an element of the same Coxeter group as `self` or a list (such as a reduced word) of elements from the index set of the Coxeter group

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 4], prefix='s')
sage: v = W.from_reduced_word([2,3,4,1,2])
sage: u = W.from_reduced_word([2,3,2,1])
sage: v.min_demazure_product_greater(u)
s4*s2
sage: v.min_demazure_product_greater([2,3,2,1])
s4*s2
sage: v.min_demazure_product_greater((2,3,2,1))
s4*s2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(4)], prefix='s')
>>> v = W.from_reduced_word([Integer(2),Integer(3),Integer(4),Integer(1),
↪Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> u = W.from_reduced_word([Integer(2),Integer(3),Integer(2),Integer(1)])
>>> v.min_demazure_product_greater(u)
s4*s2
>>> v.min_demazure_product_greater([Integer(2),Integer(3),Integer(2),
↪Integer(1)])
s4*s2
>>> v.min_demazure_product_greater((Integer(2),Integer(3),Integer(2),
↪Integer(1)))
s4*s2
```

reduced_word()

Return a reduced word for `self`.

This is a word $[i_1, i_2, \ldots, i_k]$ of minimal length such that $s_{i_1} s_{i_2} \cdots s_{i_k} =$ self, where the s_i are the simple reflections.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0]*s[1]*s[2]
sage: w.reduced_word()
[0, 1, 2]
sage: w = s[0]*s[2]
sage: w.reduced_word()
[2, 0]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)]*s[Integer(1)]*s[Integer(2)]
>>> w.reduced_word()
[0, 1, 2]
>>> w = s[Integer(0)]*s[Integer(2)]
>>> w.reduced_word()
[2, 0]
```

See also

- *reduced_words()*, *reduced_word_reverse_iterator()*,
- *length()*, *reduced_word_graph()*

reduced_word_graph()

Return the reduced word graph of `self`.

The reduced word graph of an element w in a Coxeter group is the graph whose vertices are the reduced words for w (see *reduced_word()* for a definition of this term), and which has an m-colored edge between two reduced words x and y whenever x and y differ by exactly one length-m braid move (with $m \geq 2$).

This graph is always connected (a theorem due to Tits) and has no multiple edges.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.groups
sage: W = WeylGroup(['A', 3], prefix='s')
sage: w0 = W.long_element()
sage: G = w0.reduced_word_graph()
sage: G.n_vertices()
16
sage: len(w0.reduced_words())
16
sage: G.n_edges()
18
sage: len([e for e in G.edges(sort=False) if e[2] == 2])
10
sage: len([e for e in G.edges(sort=False) if e[2] == 3])
8
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.groups
>>> W = WeylGroup(['A', Integer(3)], prefix='s')
>>> w0 = W.long_element()
>>> G = w0.reduced_word_graph()
>>> G.n_vertices()
16
>>> len(w0.reduced_words())
16
>>> G.n_edges()
18
>>> len([e for e in G.edges(sort=False) if e[Integer(2)] == Integer(2)])
10
>>> len([e for e in G.edges(sort=False) if e[Integer(2)] == Integer(3)])
8
```

See also

reduced_words(), *reduced_word_reverse_iterator()*, *length()*, *reduced_word()*

reduced_word_reverse_iterator()

Return a reverse iterator on a reduced word for `self`.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: sigma = s[0]*s[1]*s[2]
sage: rI=sigma.reduced_word_reverse_iterator()
sage: [i for i in rI]
[2, 1, 0]
sage: s[0]*s[1]*s[2]==sigma
True
sage: sigma.length()
3
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> sigma = s[Integer(0)]*s[Integer(1)]*s[Integer(2)]
>>> rI=sigma.reduced_word_reverse_iterator()
>>> [i for i in rI]
[2, 1, 0]
>>> s[Integer(0)]*s[Integer(1)]*s[Integer(2)]==sigma
True
>>> sigma.length()
3
```

See also

reduced_word()

Default implementation: recursively remove the first right descent until the identity is reached (see *first_descent()* and *apply_simple_reflection()*).

reduced_words()

Return all reduced words for `self`.

See *reduced_word()* for the definition of a reduced word.

The algorithm uses the Matsumoto property that any two reduced expressions are related by braid relations, see Theorem 3.3.1(ii) in [BB2005].

See also

braid_orbit()

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0] * s[2]
sage: sorted(w.reduced_words())                                             #␣
↪needs sage.graphs sage.modules
[[0, 2], [2, 0]]

sage: W = WeylGroup(['E', 6])                                               #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([2,3,4,2])                                    #␣
↪needs sage.combinat sage.groups
sage: sorted(w.reduced_words())                                             #␣
↪needs sage.combinat sage.groups
[[2, 3, 4, 2], [3, 2, 4, 2], [3, 4, 2, 4]]

sage: # optional - gap3, needs sage.combinat sage.groups
sage: W = ReflectionGroup(['A',3],
....:                     index_set=["AA","BB","5"])
sage: w = W.long_element()
```

(continues on next page)

(continued from previous page)

```
sage: w.reduced_words()
[['BB', '5', 'AA', 'BB', '5', 'AA'],
 ['5', 'BB', '5', 'AA', 'BB', '5'],
 ['BB', 'AA', 'BB', '5', 'BB', 'AA'],
 ['AA', '5', 'BB', 'AA', '5', 'BB'],
 ['5', 'AA', 'BB', 'AA', '5', 'BB'],
 ['AA', 'BB', '5', 'AA', 'BB', 'AA'],
 ['AA', 'BB', 'AA', '5', 'BB', 'AA'],
 ['AA', 'BB', '5', 'BB', 'AA', 'BB'],
 ['BB', 'AA', '5', 'BB', 'AA', '5'],
 ['BB', '5', 'AA', 'BB', 'AA', '5'],
 ['AA', '5', 'BB', '5', 'AA', 'BB'],
 ['5', 'BB', 'AA', '5', 'BB', '5'],
 ['5', 'BB', 'AA', 'BB', '5', 'BB'],
 ['5', 'AA', 'BB', '5', 'AA', 'BB'],
 ['BB', '5', 'BB', 'AA', 'BB', '5'],
 ['BB', 'AA', '5', 'BB', '5', 'AA']]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)] * s[Integer(2)]
>>> sorted(w.reduced_words())                                                #␣
↪needs sage.graphs sage.modules
[[0, 2], [2, 0]]

>>> W = WeylGroup(['E', Integer(6)])                                          ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(2),Integer(3),Integer(4),
↪Integer(2)])                                    # needs sage.combinat sage.
↪groups
>>> sorted(w.reduced_words())                                                #␣
↪needs sage.combinat sage.groups
[[2, 3, 4, 2], [3, 2, 4, 2], [3, 4, 2, 4]]

>>> # optional - gap3, needs sage.combinat sage.groups
>>> W = ReflectionGroup(['A',Integer(3)],
...                     index_set=["AA","BB","5"])
>>> w = W.long_element()
>>> w.reduced_words()
[['BB', '5', 'AA', 'BB', '5', 'AA'],
 ['5', 'BB', '5', 'AA', 'BB', '5'],
 ['BB', 'AA', 'BB', '5', 'BB', 'AA'],
 ['AA', '5', 'BB', 'AA', '5', 'BB'],
 ['5', 'AA', 'BB', 'AA', '5', 'BB'],
 ['AA', 'BB', '5', 'AA', 'BB', 'AA'],
 ['AA', 'BB', 'AA', '5', 'BB', 'AA'],
 ['AA', 'BB', '5', 'BB', 'AA', 'BB'],
 ['BB', 'AA', '5', 'BB', 'AA', '5'],
 ['BB', '5', 'AA', 'BB', 'AA', '5'],
 ['AA', '5', 'BB', '5', 'AA', 'BB'],
 ['5', 'BB', 'AA', '5', 'BB', '5'],
```

(continues on next page)

(continued from previous page)

```
['5', 'BB', 'AA', 'BB', '5', 'BB'],
['5', 'AA', 'BB', '5', 'AA', 'BB'],
['BB', '5', 'BB', 'AA', 'BB', '5'],
['BB', 'AA', '5', 'BB', '5', 'AA']]
```

Todo

The result should be full featured finite enumerated set (e.g., counting can be done much faster than iterating).

See also

reduced_word(), *reduced_word_reverse_iterator()*, *length()*, *reduced_word_graph()*

reduced_words_iter()

Iterate over all reduced words for `self`.

See *reduced_word()* for the definition of a reduced word.

The algorithm uses the Matsumoto property that any two reduced expressions are related by braid relations, see Theorem 3.3.1(ii) in [BB2005].

See also

braid_orbit_iter()

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0] * s[2]
sage: sorted(w.reduced_words_iter())                                    #␣
↪needs sage.combinat sage.graphs
[[0, 2], [2, 0]]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)] * s[Integer(2)]
>>> sorted(w.reduced_words_iter())                                      #␣
↪needs sage.combinat sage.graphs
[[0, 2], [2, 0]]
```

reflection_length()

Return the reflection length of `self`.

The reflection length is the length of the shortest expression of the element as a product of reflections.

See also

absolute_length()

EXAMPLES:

```
sage: W = WeylGroup(['A', 3])                                                  #␣
↪needs sage.combinat sage.groups
sage: s = W.simple_reflections()                                               #␣
↪needs sage.combinat sage.groups
sage: (s[1]*s[2]*s[3]).reflection_length()                                     #␣
↪needs sage.combinat sage.groups
3

sage: W = SymmetricGroup(4)                                                    #␣
↪needs sage.groups
sage: s = W.simple_reflections()                                               #␣
↪needs sage.groups
sage: (s[3]*s[2]*s[3]).reflection_length()                                     #␣
↪needs sage.combinat sage.groups
1
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)])                                           ␣
↪     # needs sage.combinat sage.groups
>>> s = W.simple_reflections()                                                #␣
↪needs sage.combinat sage.groups
>>> (s[Integer(1)]*s[Integer(2)]*s[Integer(3)]).reflection_length()           ␣
↪                        # needs sage.combinat sage.groups
3

>>> W = SymmetricGroup(Integer(4))                                             ␣
↪     # needs sage.groups
>>> s = W.simple_reflections()                                                #␣
↪needs sage.groups
>>> (s[Integer(3)]*s[Integer(2)]*s[Integer(3)]).reflection_length()           ␣
↪                        # needs sage.combinat sage.groups
1
```

support()

Return the support of `self`, that is the simple reflections that appear in the reduced expressions of `self`.

OUTPUT: the support of `self` as a set of integers

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: w = W.from_reduced_word([1,2,1])
sage: w.support()
{1, 2}
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
```

(continues on next page)

(continued from previous page)

```
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1)])
>>> w.support()
{1, 2}
```

upper_covers(*side='right'*, *index_set=None*)

Return all elements that cover `self` in weak order.

INPUT:

- `side` – `'left'` or `'right'` (default: `'right'`)
- `index_set` – list of indices or `None`

OUTPUT: list

EXAMPLES:

```
sage: W = WeylGroup(['A', 3])                                                  #␣
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([2,3])                                           #␣
↪needs sage.combinat sage.groups
sage: [x.reduced_word() for x in w.upper_covers()]                             #␣
↪needs sage.combinat sage.groups
[[2, 3, 1], [2, 3, 2]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)])                                            ␣
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(2),Integer(3)])                            ␣
↪              # needs sage.combinat sage.groups
>>> [x.reduced_word() for x in w.upper_covers()]                              #␣
↪needs sage.combinat sage.groups
[[2, 3, 1], [2, 3, 2]]
```

To obtain covers for left weak order, set the option `side` to 'left':

```
sage: [x.reduced_word() for x in w.upper_covers(side='left')]                  #␣
↪needs sage.combinat sage.groups
[[1, 2, 3], [2, 3, 2]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.upper_covers(side='left')]                  #␣
↪needs sage.combinat sage.groups
[[1, 2, 3], [2, 3, 2]]
```

Covers w.r.t. a parabolic subgroup are obtained with the option `index_set`:

```
sage: [x.reduced_word() for x in w.upper_covers(index_set=[1])]                #␣
↪needs sage.combinat sage.groups
[[2, 3, 1]]
sage: [x.reduced_word()                                                        #␣
↪needs sage.combinat sage.groups
....:  for x in w.upper_covers(side='left', index_set=[1])]
[[1, 2, 3]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.upper_covers(index_set=[Integer(1)])]
↪     # needs sage.combinat sage.groups
[[2, 3, 1]]
>>> [x.reduced_word()
↪needs sage.combinat sage.groups
...  for x in w.upper_covers(side='left', index_set=[Integer(1)])]
[[1, 2, 3]]
```

weak_covers(*side='right'*, *index_set=None*, *positive=False*)

Return all elements that `self` covers in weak order.

INPUT:

- `side` – `'left'` or `'right'` (default: `'right'`)
- `positive` – boolean (default: `False`)
- `index_set` – list of indices or None

OUTPUT: list

EXAMPLES:

```
sage: W = WeylGroup(['A', 3])
↪needs sage.combinat sage.groups
sage: w = W.from_reduced_word([3,2,1])
↪needs sage.combinat sage.groups
sage: [x.reduced_word() for x in w.weak_covers()]
↪needs sage.combinat sage.groups
[[3, 2]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3)])
↪     # needs sage.combinat sage.groups
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(1)])
↪                        # needs sage.combinat sage.groups
>>> [x.reduced_word() for x in w.weak_covers()]
↪needs sage.combinat sage.groups
[[3, 2]]
```

To obtain instead elements that cover self, set `positive=True`:

```
sage: [x.reduced_word() for x in w.weak_covers(positive=True)]
↪needs sage.combinat sage.groups
[[3, 1, 2, 1], [2, 3, 2, 1]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.weak_covers(positive=True)]
↪needs sage.combinat sage.groups
[[3, 1, 2, 1], [2, 3, 2, 1]]
```

To obtain covers for left weak order, set the option side to 'left':

```
sage: # needs sage.combinat sage.groups
sage: [x.reduced_word() for x in w.weak_covers(side='left')]
[[2, 1]]
```

(continues on next page)

(continued from previous page)

```
sage: w = W.from_reduced_word([3,2,3,1])
sage: [x.reduced_word() for x in w.weak_covers()]
[[2, 3, 2], [3, 2, 1]]
sage: [x.reduced_word() for x in w.weak_covers(side='left')]
[[3, 2, 1], [2, 3, 1]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> [x.reduced_word() for x in w.weak_covers(side='left')]
[[2, 1]]
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(3),Integer(1)])
>>> [x.reduced_word() for x in w.weak_covers()]
[[2, 3, 2], [3, 2, 1]]
>>> [x.reduced_word() for x in w.weak_covers(side='left')]
[[3, 2, 1], [2, 3, 1]]
```

Covers w.r.t. a parabolic subgroup are obtained with the option `index_set`:

```
sage: [x.reduced_word() for x in w.weak_covers(index_set=[1,2])]        #␣
↪needs sage.combinat sage.groups
[[2, 3, 2]]
```

```
>>> from sage.all import *
>>> [x.reduced_word() for x in w.weak_covers(index_set=[Integer(1),
↪Integer(2)])]        # needs sage.combinat sage.groups
[[2, 3, 2]]
```

weak_le(*other*, *side='right'*)

Perform the comparison between `self` and `other` in weak (Bruhat) order.

INPUT:

- `other` – an element of the same Coxeter group
- `side` – string (default: `'right'`); `'left'` or `'right'`

OUTPUT: boolean

This returns whether $u \leq v$, where u is `self` and v is `other`, in left (resp. right) weak order, that is if v can be obtained from u by length increasing multiplication by simple reflections on the left (resp. right).

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(["A", 3])
sage: u = W.from_reduced_word([1,2])
sage: v = W.from_reduced_word([1,2,3,2])
sage: u.weak_le(u)
True
sage: u.weak_le(v)
True
sage: v.weak_le(u)
False
sage: v.weak_le(v)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(["A", Integer(3)])
>>> u = W.from_reduced_word([Integer(1),Integer(2)])
>>> v = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(2)])
>>> u.weak_le(u)
True
>>> u.weak_le(v)
True
>>> v.weak_le(u)
False
>>> v.weak_le(v)
True
```

Comparison for left weak order is achieved with the option `side`:

```
sage: u.weak_le(v, side='left')                                             #␣
↪needs sage.combinat sage.groups
False
```

```
>>> from sage.all import *
>>> u.weak_le(v, side='left')                                               #␣
↪needs sage.combinat sage.groups
False
```

The implementation uses the equivalent condition that any reduced word for u is a right (resp. left) prefix of some reduced word for v.

Complexity: $O(l * c)$, where l is the minimum of the lengths of u and of v, and c is the cost of the low level methods *first_descent()*, *has_descent()*, *apply_simple_reflection()*), etc. Those are typically $O(n)$, where n is the rank of the Coxeter group.

We now run consistency tests with permutations:

```
sage: W = WeylGroup(["A", 3])                                               #␣
↪needs sage.combinat sage.groups
sage: P4 = Permutations(4)
sage: def P4toW(w): return W.from_reduced_word(w.reduced_word())
sage: for u in P4:                # long time (5s on sage.math, 2011),␣
↪needs sage.combinat sage.groups
....:     for v in P4:
....:         assert u.permutohedron_lequal(v) == P4toW(u).weak_
↪le(P4toW(v))
....:         assert u.permutohedron_lequal(v, side='left') == P4toW(u).
↪weak_le(P4toW(v), side='left')
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])                                        ␣
↪     # needs sage.combinat sage.groups
>>> P4 = Permutations(Integer(4))
>>> def P4toW(w): return W.from_reduced_word(w.reduced_word())
>>> for u in P4:                # long time (5s on sage.math, 2011), needs␣
↪sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
...       for v in P4:
...           assert u.permutohedron_lequal(v) == P4toW(u).weak_le(P4toW(v))
...           assert u.permutohedron_lequal(v, side='left') == P4toW(u).
↪weak_le(P4toW(v), side='left')
```

Finite

alias of *FiniteCoxeterGroups*

class ParentMethods

Bases: object

braid_group_as_finitely_presented_group()

Return the associated braid group.

EXAMPLES:

```
sage: W = CoxeterGroup(['A', 2])                                          #␣
↪needs sage.combinat sage.groups
sage: W.braid_group_as_finitely_presented_group()                         #␣
↪needs sage.combinat sage.groups
Finitely presented group < S1, S2 | S1*S2*S1*S2^-1*S1^-1*S2^-1 >

sage: W = WeylGroup(['B', 2])                                             #␣
↪needs sage.combinat sage.groups
sage: W.braid_group_as_finitely_presented_group()                         #␣
↪needs sage.combinat sage.groups
Finitely presented group < S1, S2 | (S1*S2)^2*(S1^-1*S2^-1)^2 >

sage: W = ReflectionGroup(['B',3], index_set=["AA","BB","5"])  # optional␣
↪- gap3
sage: W.braid_group_as_finitely_presented_group()              # optional␣
↪- gap3
Finitely presented group < SAA, SBB, S5 |
 (SAA*SBB)^2*(SAA^-1*SBB^-1)^2, SAA*S5*SAA^-1*S5^-1,
 SBB*S5*SBB*S5^-1*SBB^-1*S5^-1 >
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A', Integer(2)])                                    ␣
↪     # needs sage.combinat sage.groups
>>> W.braid_group_as_finitely_presented_group()                          #␣
↪needs sage.combinat sage.groups
Finitely presented group < S1, S2 | S1*S2*S1*S2^-1*S1^-1*S2^-1 >

>>> W = WeylGroup(['B', Integer(2)])                                       ␣
↪     # needs sage.combinat sage.groups
>>> W.braid_group_as_finitely_presented_group()                          #␣
↪needs sage.combinat sage.groups
Finitely presented group < S1, S2 | (S1*S2)^2*(S1^-1*S2^-1)^2 >

>>> W = ReflectionGroup(['B',Integer(3)], index_set=["AA","BB","5"])  #␣
↪optional - gap3
>>> W.braid_group_as_finitely_presented_group()              # optional -␣
↪gap3
```

(continues on next page)

(continued from previous page)

```
Finitely presented group < SAA, SBB, S5 |
 (SAA*SBB)^2*(SAA^-1*SBB^-1)^2, SAA*S5*SAA^-1*S5^-1,
 SBB*S5*SBB*S5^-1*SBB^-1*S5^-1 >
```

braid_orbit(*word*)

Return the braid orbit of a word `word` of indices.

The input word does not need to be a reduced expression of an element.

INPUT:

- `word` – a list (or iterable) of indices in `self.index_set()`

OUTPUT:

a list of all lists that can be obtained from `word` by replacements of braid relations

See *braid_relations()* for the definition of braid relations.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: s = W.simple_reflections()
sage: w = s[0] * s[1] * s[2] * s[1]
sage: word = w.reduced_word(); word
[0, 1, 2, 1]

sage: sorted(W.braid_orbit(word))                                          #␣
↪needs sage.combinat sage.graphs
[[0, 1, 2, 1], [0, 2, 1, 2], [2, 0, 1, 2]]

sage: sorted(W.braid_orbit([2,1,1,2,1]))                                   #␣
↪needs sage.combinat sage.graphs
[[1, 2, 1, 1, 2], [2, 1, 1, 2, 1], [2, 1, 2, 1, 2], [2, 2, 1, 2, 2]]

sage: # optional - gap3
sage: W = ReflectionGroup(['A',3], index_set=["AA","BB","5"])
sage: w = W.long_element()
sage: W.braid_orbit(w.reduced_word())
[['BB', '5', 'AA', 'BB', '5', 'AA'],
 ['5', 'BB', '5', 'AA', 'BB', '5'],
 ['BB', 'AA', 'BB', '5', 'BB', 'AA'],
 ['AA', '5', 'BB', 'AA', '5', 'BB'],
 ['5', 'AA', 'BB', 'AA', '5', 'BB'],
 ['AA', 'BB', '5', 'AA', 'BB', 'AA'],
 ['AA', 'BB', 'AA', '5', 'BB', 'AA'],
 ['AA', 'BB', '5', 'BB', 'AA', 'BB'],
 ['BB', 'AA', '5', 'BB', 'AA', '5'],
 ['BB', '5', 'AA', 'BB', 'AA', '5'],
 ['AA', '5', 'BB', '5', 'AA', 'BB'],
 ['5', 'BB', 'AA', '5', 'BB', '5'],
 ['5', 'BB', 'AA', 'BB', '5', 'BB'],
 ['5', 'AA', 'BB', '5', 'AA', 'BB'],
 ['BB', '5', 'BB', 'AA', 'BB', '5'],
 ['BB', 'AA', '5', 'BB', '5', 'AA']]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> s = W.simple_reflections()
>>> w = s[Integer(0)] * s[Integer(1)] * s[Integer(2)] * s[Integer(1)]
>>> word = w.reduced_word(); word
[0, 1, 2, 1]

>>> sorted(W.braid_orbit(word))                                    #␣
↪needs sage.combinat sage.graphs
[[0, 1, 2, 1], [0, 2, 1, 2], [2, 0, 1, 2]]

>>> sorted(W.braid_orbit([Integer(2),Integer(1),Integer(1),Integer(2),
↪Integer(1)]))                                  # needs sage.combinat sage.
↪graphs
[[1, 2, 1, 1, 2], [2, 1, 1, 2, 1], [2, 1, 2, 1, 2], [2, 2, 1, 2, 2]]

>>> # optional - gap3
>>> W = ReflectionGroup(['A',Integer(3)], index_set=["AA","BB","5"])
>>> w = W.long_element()
>>> W.braid_orbit(w.reduced_word())
[['BB', '5', 'AA', 'BB', '5', 'AA'],
 ['5', 'BB', '5', 'AA', 'BB', '5'],
 ['BB', 'AA', 'BB', '5', 'BB', 'AA'],
 ['AA', '5', 'BB', 'AA', '5', 'BB'],
 ['5', 'AA', 'BB', 'AA', '5', 'BB'],
 ['AA', 'BB', '5', 'AA', 'BB', 'AA'],
 ['AA', 'BB', 'AA', '5', 'BB', 'AA'],
 ['AA', 'BB', '5', 'BB', 'AA', 'BB'],
 ['BB', 'AA', '5', 'BB', 'AA', '5'],
 ['BB', '5', 'AA', 'BB', 'AA', '5'],
 ['AA', '5', 'BB', '5', 'AA', 'BB'],
 ['5', 'BB', 'AA', '5', 'BB', '5'],
 ['5', 'BB', 'AA', 'BB', '5', 'BB'],
 ['5', 'AA', 'BB', '5', 'AA', 'BB'],
 ['BB', '5', 'BB', 'AA', 'BB', '5'],
 ['BB', 'AA', '5', 'BB', '5', 'AA']]
```

Todo

The result should be full featured finite enumerated set (e.g., counting can be done much faster than iterating).

See also

reduced_words()

braid_orbit_iter(*word*)

Iterate over the braid orbit of a word `word` of indices.

The input word does not need to be a reduced expression of an element.

INPUT:

- `word` – list (or iterable) of indices in `self.index_set()`

OUTPUT:

all lists that can be obtained from `word` by replacements of braid relations

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: sorted(W.braid_orbit_iter([0, 1, 2, 1]))                          #␣
↪needs sage.combinat sage.graphs
[[0, 1, 2, 1], [0, 2, 1, 2], [2, 0, 1, 2]]
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> sorted(W.braid_orbit_iter([Integer(0), Integer(1), Integer(2),␣
↪Integer(1)]))                          # needs sage.combinat sage.graphs
[[0, 1, 2, 1], [0, 2, 1, 2], [2, 0, 1, 2]]
```

braid_relations()

Return the braid relations of `self` as a list of reduced words of the braid relations.

EXAMPLES:

```
sage: W = WeylGroup(["A", 2])                                           #␣
↪needs sage.combinat sage.groups
sage: W.braid_relations()                                               #␣
↪needs sage.combinat sage.groups
[[[1, 2, 1], [2, 1, 2]]]

sage: W = WeylGroup(["B", 3])                                           #␣
↪needs sage.combinat sage.groups
sage: W.braid_relations()                                               #␣
↪needs sage.combinat sage.groups
[[[1, 2, 1], [2, 1, 2]], [[1, 3], [3, 1]], [[2, 3, 2, 3], [3, 2, 3, 2]]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(2)])                                    ␣
↪     # needs sage.combinat sage.groups
>>> W.braid_relations()                                                #␣
↪needs sage.combinat sage.groups
[[[1, 2, 1], [2, 1, 2]]]

>>> W = WeylGroup(["B", Integer(3)])                                    ␣
↪     # needs sage.combinat sage.groups
>>> W.braid_relations()                                                #␣
↪needs sage.combinat sage.groups
[[[1, 2, 1], [2, 1, 2]], [[1, 3], [3, 1]], [[2, 3, 2, 3], [3, 2, 3, 2]]]
```

bruhat_graph(*x=None*, *y=None*, *edge_labels=False*)

Return the Bruhat graph as a directed graph, with an edge $u \to v$ if and only if $u < v$ in the Bruhat order, and $u = r \cdot v$.

The Bruhat graph $\Gamma(x, y)$, defined if $x \leq y$ in the Bruhat order, has as its vertices the Bruhat interval $\{t | x \leq t \leq y\}$, and as its edges are the pairs (u, v) such that $u = r \cdot v$ where r is a reflection, that is, a conjugate of a simple reflection.

REFERENCES:

Carrell, The Bruhat graph of a Coxeter group, a conjecture of Deodhar, and rational smoothness of Schubert varieties. Algebraic groups and their generalizations: classical methods (University Park, PA, 1991), 53–61, Proc. Sympos. Pure Math., 56, Part 1, Amer. Math. Soc., Providence, RI, 1994.

EXAMPLES:

```
sage: W = CoxeterGroup(['H', 3])                                        #␣
↪needs sage.combinat sage.graphs sage.groups
sage: G = W.bruhat_graph(); G                                           #␣
↪needs sage.combinat sage.graphs sage.groups
Digraph on 120 vertices

sage: # needs sage.combinat sage.graphs sage.groups
sage: W = CoxeterGroup(['A', 2, 1])
sage: s1, s2, s3 = W.simple_reflections()
sage: W.bruhat_graph(s1, s1*s3*s2*s3)
Digraph on 6 vertices
sage: W.bruhat_graph(s1, s3*s2*s3)
Digraph on 0 vertices

sage: W = WeylGroup("A3", prefix='s')                                   #␣
↪needs sage.combinat sage.graphs sage.groups
sage: s1, s2, s3 = W.simple_reflections()                               #␣
↪needs sage.combinat sage.graphs sage.groups
sage: G = W.bruhat_graph(s1*s3, s1*s2*s3*s2*s1); G                      #␣
↪needs sage.combinat sage.graphs sage.groups
Digraph on 10 vertices
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['H', Integer(3)])                                  ␣
↪     # needs sage.combinat sage.graphs sage.groups
>>> G = W.bruhat_graph(); G                                            #␣
↪needs sage.combinat sage.graphs sage.groups
Digraph on 120 vertices

>>> # needs sage.combinat sage.graphs sage.groups
>>> W = CoxeterGroup(['A', Integer(2), Integer(1)])
>>> s1, s2, s3 = W.simple_reflections()
>>> W.bruhat_graph(s1, s1*s3*s2*s3)
Digraph on 6 vertices
>>> W.bruhat_graph(s1, s3*s2*s3)
Digraph on 0 vertices

>>> W = WeylGroup("A3", prefix='s')                                    #␣
↪needs sage.combinat sage.graphs sage.groups
>>> s1, s2, s3 = W.simple_reflections()                                #␣
↪needs sage.combinat sage.graphs sage.groups
>>> G = W.bruhat_graph(s1*s3, s1*s2*s3*s2*s1); G                       #␣
↪needs sage.combinat sage.graphs sage.groups
Digraph on 10 vertices
```

Check that the graph has the correct number of edges (see Issue #17744):

```
sage: len(G.edges(sort=False))                                                  #␣
↪needs sage.combinat sage.graphs sage.groups
16
```

```
>>> from sage.all import *
>>> len(G.edges(sort=False))                                                    #␣
↪needs sage.combinat sage.graphs sage.groups
16
```

bruhat_interval(*x*, *y*)

Return the list of `t` such that `x <= t <= y`.

EXAMPLES:

```
sage: W = WeylGroup("A3", prefix='s')                                           #␣
↪needs sage.combinat sage.groups
sage: s1, s2, s3 = W.simple_reflections()                                       #␣
↪needs sage.combinat sage.groups
sage: W.bruhat_interval(s2, s1*s3*s2*s1*s3)                                     #␣
↪needs sage.combinat sage.groups
[s1*s2*s3*s2*s1, s2*s3*s2*s1, s3*s1*s2*s1, s1*s2*s3*s1,
 s1*s2*s3*s2, s3*s2*s1, s2*s3*s1, s2*s3*s2, s1*s2*s1,
 s3*s1*s2, s1*s2*s3, s2*s1, s3*s2, s2*s3, s1*s2, s2]

sage: W = WeylGroup(['A', 2, 1], prefix='s')                                    #␣
↪needs sage.combinat sage.groups
sage: s0, s1, s2 = W.simple_reflections()                                       #␣
↪needs sage.combinat sage.groups
sage: W.bruhat_interval(1, s0*s1*s2)                                            #␣
↪needs sage.combinat sage.groups
[s0*s1*s2, s1*s2, s0*s2, s0*s1, s2, s1, s0, 1]
```

```
>>> from sage.all import *
>>> W = WeylGroup("A3", prefix='s')                                             #␣
↪needs sage.combinat sage.groups
>>> s1, s2, s3 = W.simple_reflections()                                         #␣
↪needs sage.combinat sage.groups
>>> W.bruhat_interval(s2, s1*s3*s2*s1*s3)                                       #␣
↪needs sage.combinat sage.groups
[s1*s2*s3*s2*s1, s2*s3*s2*s1, s3*s1*s2*s1, s1*s2*s3*s1,
 s1*s2*s3*s2, s3*s2*s1, s2*s3*s1, s2*s3*s2, s1*s2*s1,
 s3*s1*s2, s1*s2*s3, s2*s1, s3*s2, s2*s3, s1*s2, s2]

>>> W = WeylGroup(['A', Integer(2), Integer(1)], prefix='s')                   ␣
↪              # needs sage.combinat sage.groups
>>> s0, s1, s2 = W.simple_reflections()                                         #␣
↪needs sage.combinat sage.groups
>>> W.bruhat_interval(Integer(1), s0*s1*s2)                                     ␣
↪     # needs sage.combinat sage.groups
[s0*s1*s2, s1*s2, s0*s2, s0*s1, s2, s1, s0, 1]
```

bruhat_interval_poset(*x*, *y*, *facade=False*)

Return the poset of the Bruhat interval between `x` and `y` in Bruhat order.

EXAMPLES:

```
sage: W = WeylGroup("A3", prefix='s')                                   #␣
↪needs sage.combinat sage.groups
sage: s1, s2, s3 = W.simple_reflections()                               #␣
↪needs sage.combinat sage.groups
sage: W.bruhat_interval_poset(s2, s1*s3*s2*s1*s3)                       #␣
↪needs sage.combinat sage.groups
Finite poset containing 16 elements

sage: W = WeylGroup(['A', 2, 1], prefix='s')                            #␣
↪needs sage.combinat sage.groups
sage: s0, s1, s2 = W.simple_reflections()                               #␣
↪needs sage.combinat sage.groups
sage: W.bruhat_interval_poset(1, s0*s1*s2)                              #␣
↪needs sage.combinat sage.groups
Finite poset containing 8 elements
```

```
>>> from sage.all import *
>>> W = WeylGroup("A3", prefix='s')                                     #␣
↪needs sage.combinat sage.groups
>>> s1, s2, s3 = W.simple_reflections()                                 #␣
↪needs sage.combinat sage.groups
>>> W.bruhat_interval_poset(s2, s1*s3*s2*s1*s3)                         #␣
↪needs sage.combinat sage.groups
Finite poset containing 16 elements

>>> W = WeylGroup(['A', Integer(2), Integer(1)], prefix='s')              ␣
↪              # needs sage.combinat sage.groups
>>> s0, s1, s2 = W.simple_reflections()                                 #␣
↪needs sage.combinat sage.groups
>>> W.bruhat_interval_poset(Integer(1), s0*s1*s2)                         ␣
↪     # needs sage.combinat sage.groups
Finite poset containing 8 elements
```

canonical_representation()

Return the canonical faithful representation of `self`.

See also

To obtain the underlying module with the action, use *reflection_representation()*.

EXAMPLES:

```
sage: W = WeylGroup("A3")                                               #␣
↪needs sage.combinat sage.groups
sage: W.canonical_representation()                                      #␣
↪needs sage.combinat sage.groups
Finite Coxeter group over Integer Ring with Coxeter matrix:
[1 3 2]
[3 1 3]
[2 3 1]
```

```
>>> from sage.all import *
>>> W = WeylGroup("A3")                                                    #␣
↪needs sage.combinat sage.groups
>>> W.canonical_representation()                                           #␣
↪needs sage.combinat sage.groups
Finite Coxeter group over Integer Ring with Coxeter matrix:
[1 3 2]
[3 1 3]
[2 3 1]
```

coxeter_diagram()

Return the Coxeter diagram of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.groups
sage: W = CoxeterGroup(['H', 3], implementation='reflection')
sage: G = W.coxeter_diagram(); G
Graph on 3 vertices
sage: G.edges(sort=True)
[(1, 2, 3), (2, 3, 5)]
sage: CoxeterGroup(G) is W
True
sage: G = Graph([(0, 1, 3), (1, 2, oo)])
sage: W = CoxeterGroup(G)
sage: W.coxeter_diagram() == G
True
sage: CoxeterGroup(W.coxeter_diagram()) is W
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.groups
>>> W = CoxeterGroup(['H', Integer(3)], implementation='reflection')
>>> G = W.coxeter_diagram(); G
Graph on 3 vertices
>>> G.edges(sort=True)
[(1, 2, 3), (2, 3, 5)]
>>> CoxeterGroup(G) is W
True
>>> G = Graph([(Integer(0), Integer(1), Integer(3)), (Integer(1),␣
↪Integer(2), oo)])
>>> W = CoxeterGroup(G)
>>> W.coxeter_diagram() == G
True
>>> CoxeterGroup(W.coxeter_diagram()) is W
True
```

coxeter_element()

Return a Coxeter element.

The result is the product of the simple reflections, in some order.

Note

This implementation is shared with well generated complex reflection groups. It would be nicer to put it in some joint super category; however, in the current state of the art, there is none where it is clear that this is the right construction for obtaining a Coxeter element.

In this context, this is an element having a regular eigenvector (a vector not contained in any reflection hyperplane of `self`).

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: CoxeterGroup(['A', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['B', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['D', 4]).coxeter_element().reduced_word()
[1, 2, 4, 3]
sage: CoxeterGroup(['F', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['E', 8]).coxeter_element().reduced_word()
[1, 3, 2, 4, 5, 6, 7, 8]
sage: CoxeterGroup(['H', 3]).coxeter_element().reduced_word()
[1, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> CoxeterGroup(['A', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['B', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['D', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 4, 3]
>>> CoxeterGroup(['F', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['E', Integer(8)]).coxeter_element().reduced_word()
[1, 3, 2, 4, 5, 6, 7, 8]
>>> CoxeterGroup(['H', Integer(3)]).coxeter_element().reduced_word()
[1, 2, 3]
```

This method is also used for well generated finite complex reflection groups:

```
sage: W = ReflectionGroup((1,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3]

sage: W = ReflectionGroup((2,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

sage: W = ReflectionGroup((4,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]
```

(continues on next page)

(continued from previous page)

```
sage: W = ReflectionGroup((4,4,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3]

>>> W = ReflectionGroup((Integer(2),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

>>> W = ReflectionGroup((Integer(4),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

>>> W = ReflectionGroup((Integer(4),Integer(4),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]
```

coxeter_matrix()

Return the Coxeter matrix associated to `self`.

EXAMPLES:

```
sage: G = WeylGroup(['A', 3])                                          #␣
↪needs sage.combinat sage.groups
sage: G.coxeter_matrix()                                               #␣
↪needs sage.combinat sage.groups
[1 3 2]
[3 1 3]
[2 3 1]
```

```
>>> from sage.all import *
>>> G = WeylGroup(['A', Integer(3)])                                    ␣
↪     # needs sage.combinat sage.groups
>>> G.coxeter_matrix()                                                 #␣
↪needs sage.combinat sage.groups
[1 3 2]
[3 1 3]
[2 3 1]
```

coxeter_type()

Return the Coxeter type of `self`.

EXAMPLES:

```
sage: W = CoxeterGroup(['H', 3])                                        #␣
↪needs sage.combinat sage.groups
sage: W.coxeter_type()                                                  #␣
↪needs sage.combinat sage.groups
Coxeter type of ['H', 3]
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['H', Integer(3)])                                  ␣
↪     # needs sage.combinat sage.groups
>>> W.coxeter_type()                                                   #␣
↪needs sage.combinat sage.groups
Coxeter type of ['H', 3]
```

demazure_product(Q)

Return the Demazure product of the list `Q` in `self`.

INPUT:

- `Q` – list of elements from the index set of `self`

This returns the Coxeter group element that represents the composition of 0-Hecke or Demazure operators.

See *CoxeterGroups.ParentMethods.simple_projections()*.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = WeylGroup(['A', 2])
sage: w = W.demazure_product([2,2,1])
sage: w.reduced_word()
[2, 1]
sage: w = W.demazure_product([2,1,2,1,2])
sage: w.reduced_word()
[1, 2, 1]

sage: W = WeylGroup(['B', 2])                                           #␣
↪needs sage.combinat sage.groups
sage: w = W.demazure_product([2,1,2,1,2])                               #␣
↪needs sage.combinat sage.groups
sage: w.reduced_word()                                                  #␣
↪needs sage.combinat sage.groups
[2, 1, 2, 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = WeylGroup(['A', Integer(2)])
>>> w = W.demazure_product([Integer(2),Integer(2),Integer(1)])
>>> w.reduced_word()
[2, 1]
>>> w = W.demazure_product([Integer(2),Integer(1),Integer(2),Integer(1),
↪Integer(2)])
>>> w.reduced_word()
[1, 2, 1]

>>> W = WeylGroup(['B', Integer(2)])                                     ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.combinat sage.groups
>>> w = W.demazure_product([Integer(2),Integer(1),Integer(2),Integer(1),
↪Integer(2)])                                  # needs sage.combinat sage.
↪groups
>>> w.reduced_word()                                                    #␣
↪needs sage.combinat sage.groups
[2, 1, 2, 1]
```

elements_of_length(*n*)

Return all elements of length n.

EXAMPLES:

```
sage: A = AffinePermutationGroup(['A', 2, 1])                           #␣
↪needs sage.combinat
sage: [len(list(A.elements_of_length(i))) for i in [0..5]]              #␣
↪needs sage.combinat
[1, 3, 6, 9, 12, 15]

sage: W = CoxeterGroup(['H', 3])                                        #␣
↪needs sage.combinat sage.groups
sage: [len(list(W.elements_of_length(i))) for i in range(4)]            #␣
↪needs sage.combinat sage.groups
[1, 3, 5, 7]

sage: W = CoxeterGroup(['A', 2])                                        #␣
↪needs sage.combinat sage.groups
sage: [len(list(W.elements_of_length(i))) for i in range(6)]            #␣
↪needs sage.combinat sage.groups
[1, 2, 2, 1, 0, 0]
```

```
>>> from sage.all import *
>>> A = AffinePermutationGroup(['A', Integer(2), Integer(1)])           ␣
↪              # needs sage.combinat
>>> [len(list(A.elements_of_length(i))) for i in (ellipsis_
↪range(Integer(0),Ellipsis,Integer(5)))]              # needs sage.
↪combinat
[1, 3, 6, 9, 12, 15]

>>> W = CoxeterGroup(['H', Integer(3)])                                 ␣
↪     # needs sage.combinat sage.groups
>>> [len(list(W.elements_of_length(i))) for i in range(Integer(4))]     ␣
↪     # needs sage.combinat sage.groups
[1, 3, 5, 7]

>>> W = CoxeterGroup(['A', Integer(2)])                                 ␣
↪     # needs sage.combinat sage.groups
>>> [len(list(W.elements_of_length(i))) for i in range(Integer(6))]     ␣
↪     # needs sage.combinat sage.groups
[1, 2, 2, 1, 0, 0]
```

fully_commutative_elements()

Return the set of fully commutative elements in this Coxeter group.

See also

FullyCommutativeElements

EXAMPLES:

```
sage: CoxeterGroup(['A', 3]).fully_commutative_elements()                #␣
↪needs sage.combinat sage.groups
Fully commutative elements of
 Finite Coxeter group over Integer Ring with Coxeter matrix:
[1 3 2]
[3 1 3]
[2 3 1]
```

```
>>> from sage.all import *
>>> CoxeterGroup(['A', Integer(3)]).fully_commutative_elements()          ␣
↪     # needs sage.combinat sage.groups
Fully commutative elements of
 Finite Coxeter group over Integer Ring with Coxeter matrix:
[1 3 2]
[3 1 3]
[2 3 1]
```

grassmannian_elements(*side='right'*)

Return the left or right Grassmannian elements of `self` as an enumerated set.

INPUT:

- `side` – (default: `'right'`) `'left'` or `'right'`

EXAMPLES:

```
sage: S = CoxeterGroups().example()
sage: G = S.grassmannian_elements()
sage: G.cardinality()
12
sage: G.list()
[(0, 1, 2, 3), (1, 0, 2, 3), (0, 2, 1, 3), (0, 1, 3, 2),
 (2, 0, 1, 3), (1, 2, 0, 3), (0, 3, 1, 2), (0, 2, 3, 1),
 (3, 0, 1, 2), (1, 3, 0, 2), (1, 2, 3, 0), (2, 3, 0, 1)]
sage: sorted(tuple(w.descents()) for w in G)
[(), (0,), (0,), (0,), (1,), (1,), (1,), (1,), (1,), (2,), (2,), (2,)]
sage: G = S.grassmannian_elements(side = "left")
sage: G.cardinality()
12
sage: sorted(tuple(w.descents(side = "left")) for w in G)
[(), (0,), (0,), (0,), (1,), (1,), (1,), (1,), (1,), (2,), (2,), (2,)]
```

```
>>> from sage.all import *
>>> S = CoxeterGroups().example()
>>> G = S.grassmannian_elements()
>>> G.cardinality()
12
>>> G.list()
[(0, 1, 2, 3), (1, 0, 2, 3), (0, 2, 1, 3), (0, 1, 3, 2),
```

(continues on next page)

(continued from previous page)

```
 (2, 0, 1, 3), (1, 2, 0, 3), (0, 3, 1, 2), (0, 2, 3, 1),
 (3, 0, 1, 2), (1, 3, 0, 2), (1, 2, 3, 0), (2, 3, 0, 1)]
>>> sorted(tuple(w.descents()) for w in G)
[(), (0,), (0,), (0,), (1,), (1,), (1,), (1,), (1,), (2,), (2,), (2,)]
>>> G = S.grassmannian_elements(side = "left")
>>> G.cardinality()
12
>>> sorted(tuple(w.descents(side = "left")) for w in G)
[(), (0,), (0,), (0,), (1,), (1,), (1,), (1,), (1,), (2,), (2,), (2,)]
```

index_set()

Return the index set of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = CoxeterGroup([[1,3],[3,1]])
sage: W.index_set()
(1, 2)
sage: W = CoxeterGroup([[1,3],[3,1]], index_set=['x', 'y'])
sage: W.index_set()
('x', 'y')
sage: W = CoxeterGroup(['H', 3])
sage: W.index_set()
(1, 2, 3)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = CoxeterGroup([[Integer(1),Integer(3)],[Integer(3),Integer(1)]])
>>> W.index_set()
(1, 2)
>>> W = CoxeterGroup([[Integer(1),Integer(3)],[Integer(3),Integer(1)]],␣
↪index_set=['x', 'y'])
>>> W.index_set()
('x', 'y')
>>> W = CoxeterGroup(['H', Integer(3)])
>>> W.index_set()
(1, 2, 3)
```

is_commutative()

Return whether this Coxeter group is commutative.

EXAMPLES:

```
sage: W = CoxeterGroup(['B', 3])
sage: W.is_commutative()
False
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['B', Integer(3)])
>>> W.is_commutative()
False
```

kazhdan_lusztig_cells(*side='left'*)

Compute the left, right, or two-sided Kazhdan-Lusztig cells of `self` if `self` is finite.

The cells are computed by using *kazhdan_lusztig_cell()*.

As detailed there, installation of the optional package `coxeter3` is recommended (though not required) before using this function as it speeds up the computation.

INPUT:

- `side` – (default: `'left'`) either `'left'`, `'right'`, or `'two-sided'`

EXAMPLES:

We compute the right cells in the Coxeter group of type A_2 below. Note that each Coxeter group may be created with multiple implementations, namely, 'reflection' (default), 'permutation', 'matrix', or 'coxeter3'. The choice of implementation affects the representation of elements in the output cells but not the method used for the cell computation:

```
sage: # needs sage.combinat sage.groups
sage: W = CoxeterGroup('A2')
sage: KL_cells = W.kazhdan_lusztig_cells(side='right')
sage: set([tuple(sorted(C, key=lambda w: w.reduced_word()))
....:       for C in KL_cells])
{(
[-1  1]  [ 0 -1]
[ 0  1], [ 1 -1]
),
 (
[ 0 -1]
[-1  0]
),
 (
[1 0]
[0 1]
),
 (
[ 1  0]  [-1  1]
[ 1 -1], [-1  0]
)}
sage: len(KL_cells)
4

sage: W = CoxeterGroup('A2', implementation='permutation')              #␣
↪needs sage.combinat sage.groups
sage: len(W.kazhdan_lusztig_cells(side='right'))                        #␣
↪needs sage.combinat sage.groups
4
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = CoxeterGroup('A2')
>>> KL_cells = W.kazhdan_lusztig_cells(side='right')
>>> set([tuple(sorted(C, key=lambda w: w.reduced_word()))
...       for C in KL_cells])
{(
[-1  1]  [ 0 -1]
```

(continues on next page)

(continued from previous page)

```
[ 0  1], [ 1 -1]
),
 (
[ 0 -1]
[-1  0]
),
 (
[1 0]
[0 1]
),
 (
[ 1  0]  [-1  1]
[ 1 -1], [-1  0]
)}
>>> len(KL_cells)
4

>>> W = CoxeterGroup('A2', implementation='permutation')                #␣
↪needs sage.combinat sage.groups
>>> len(W.kazhdan_lusztig_cells(side='right'))                          #␣
↪needs sage.combinat sage.groups
4
```

We compute the left cells in the Coxeter group of type A_3 below. If the optional package `coxeter3` is installed, it runs in the background even if the group is not created with the `'coxeter3'` implementation:

```
sage: # optional - coxeter3, needs sage.combinat sage.groups sage.libs.
↪gap sage.modules sage.rings.number_field
sage: W = CoxeterGroup('A3', implementation='coxeter3')
sage: KL_cells = W.kazhdan_lusztig_cells()
sage: set([tuple(sorted(C)) for C in KL_cells])
{([],),
 ([1], [2, 1], [3, 2, 1]),
 ([1, 2], [2], [3, 2]),
 ([1, 2, 1], [1, 3, 2, 1], [2, 1, 3, 2, 1]),
 ([1, 2, 1, 3], [1, 2, 3, 2, 1], [2, 3, 2, 1]),
 ([1, 2, 1, 3, 2], [1, 2, 3, 2], [2, 3, 2]),
 ([1, 2, 1, 3, 2, 1],),
 ([1, 2, 3], [2, 3], [3]),
 ([1, 3], [2, 1, 3]),
 ([1, 3, 2], [2, 1, 3, 2])}
sage: len(KL_cells)
10
sage: W = CoxeterGroup('A3', implementation='permutation')
sage: len(W.kazhdan_lusztig_cells())
10
```

```
>>> from sage.all import *
>>> # optional - coxeter3, needs sage.combinat sage.groups sage.libs.gap␣
↪sage.modules sage.rings.number_field
>>> W = CoxeterGroup('A3', implementation='coxeter3')
>>> KL_cells = W.kazhdan_lusztig_cells()
```

(continues on next page)

(continued from previous page)

```
>>> set([tuple(sorted(C)) for C in KL_cells])
{([],),
 ([1], [2, 1], [3, 2, 1]),
 ([1, 2], [2], [3, 2]),
 ([1, 2, 1], [1, 3, 2, 1], [2, 1, 3, 2, 1]),
 ([1, 2, 1, 3], [1, 2, 3, 2, 1], [2, 3, 2, 1]),
 ([1, 2, 1, 3, 2], [1, 2, 3, 2], [2, 3, 2]),
 ([1, 2, 1, 3, 2, 1],),
 ([1, 2, 3], [2, 3], [3]),
 ([1, 3], [2, 1, 3]),
 ([1, 3, 2], [2, 1, 3, 2])}
>>> len(KL_cells)
10
>>> W = CoxeterGroup('A3', implementation='permutation')
>>> len(W.kazhdan_lusztig_cells())
10
```

Computing the two-sided cells in B_3:

```
sage: # optional - coxeter3, needs sage.combinat sage.groups sage.libs.
↪gap sage.modules sage.rings.number_field
sage: W = CoxeterGroup('B3', implementation='coxeter3')
sage: b3_cells = W.kazhdan_lusztig_cells('two-sided')
sage: len(b3_cells)
6
sage: set([tuple(sorted(C))
....:      for C in W.kazhdan_lusztig_cells()])
{([],),
 ([1], [1, 2, 3, 2, 1], [2, 1], [2, 3, 2, 1], [3, 2, 1]),
 ([1, 2], [1, 2, 3, 2], [2], [2, 3, 2], [3, 2]),
 ([1, 2, 3], [2, 3], [3], [3, 2, 3]),
 ([2, 1, 2], [2, 3, 2, 1, 2], [3, 2, 1, 2]),
 ([2, 1, 2, 3], [2, 3, 2, 1, 2, 3], [3, 2, 1, 2, 3]),
 ([2, 1, 2, 3, 2], [2, 3, 2, 1, 2, 3, 2], [3, 2, 1, 2, 3, 2]),
 ([2, 1, 2, 3, 2, 1],
  [2, 3, 2, 1, 2, 3, 2, 1],
  [3, 2, 1, 2, 3, 2, 1],
  [3, 2, 3, 2, 1, 2]),
 ([2, 3, 1], [3, 1], [3, 2, 3, 1]),
 ([2, 3, 1, 2], [3, 1, 2], [3, 2, 3, 1, 2]),
 ([2, 3, 1, 2, 3], [3, 1, 2, 3], [3, 2, 3, 1, 2, 3]),
 ([2, 3, 1, 2, 3, 2],
  [3, 1, 2, 3, 2],
  [3, 2, 3, 1, 2, 3, 2],
  [3, 2, 3, 2],
  [3, 2, 3, 2, 1, 2, 3, 2]),
 ([2, 3, 1, 2, 3, 2, 1],
  [3, 1, 2, 3, 2, 1],
  [3, 2, 3, 1, 2, 3, 2, 1],
  [3, 2, 3, 2, 1],
  [3, 2, 3, 2, 1, 2, 3]),
 ([3, 2, 3, 2, 1, 2, 3, 2, 1],)}
```

```
>>> from sage.all import *
>>> # optional - coxeter3, needs sage.combinat sage.groups sage.libs.gap␣
↪sage.modules sage.rings.number_field
>>> W = CoxeterGroup('B3', implementation='coxeter3')
>>> b3_cells = W.kazhdan_lusztig_cells('two-sided')
>>> len(b3_cells)
6
>>> set([tuple(sorted(C))
...     for C in W.kazhdan_lusztig_cells()])
{([],),
 ([1], [1, 2, 3, 2, 1], [2, 1], [2, 3, 2, 1], [3, 2, 1]),
 ([1, 2], [1, 2, 3, 2], [2], [2, 3, 2], [3, 2]),
 ([1, 2, 3], [2, 3], [3], [3, 2, 3]),
 ([2, 1, 2], [2, 3, 2, 1, 2], [3, 2, 1, 2]),
 ([2, 1, 2, 3], [2, 3, 2, 1, 2, 3], [3, 2, 1, 2, 3]),
 ([2, 1, 2, 3, 2], [2, 3, 2, 1, 2, 3, 2], [3, 2, 1, 2, 3, 2]),
 ([2, 1, 2, 3, 2, 1],
  [2, 3, 2, 1, 2, 3, 2, 1],
  [3, 2, 1, 2, 3, 2, 1],
  [3, 2, 3, 2, 1, 2]),
 ([2, 3, 1], [3, 1], [3, 2, 3, 1]),
 ([2, 3, 1, 2], [3, 1, 2], [3, 2, 3, 1, 2]),
 ([2, 3, 1, 2, 3], [3, 1, 2, 3], [3, 2, 3, 1, 2, 3]),
 ([2, 3, 1, 2, 3, 2],
  [3, 1, 2, 3, 2],
  [3, 2, 3, 1, 2, 3, 2],
  [3, 2, 3, 2],
  [3, 2, 3, 2, 1, 2, 3, 2]),
 ([2, 3, 1, 2, 3, 2, 1],
  [3, 1, 2, 3, 2, 1],
  [3, 2, 3, 1, 2, 3, 2, 1],
  [3, 2, 3, 2, 1],
  [3, 2, 3, 2, 1, 2, 3]),
 ([3, 2, 3, 2, 1, 2, 3, 2, 1],)}
```

random_element_of_length(n)

Return a random element of length `n` in `self`.

Starts at the identity, then chooses an upper cover at random.

Not very uniform: actually constructs a uniformly random reduced word of length n. Thus we most likely get elements with lots of reduced words!

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: A = AffinePermutationGroup(['A', 7, 1])
sage: p = A.random_element_of_length(10)
sage: p in A
True
sage: p.length() == 10
True

sage: # needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: W = CoxeterGroup(['A', 4])
sage: p = W.random_element_of_length(5)
sage: p in W
True
sage: p.length() == 5
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> A = AffinePermutationGroup(['A', Integer(7), Integer(1)])
>>> p = A.random_element_of_length(Integer(10))
>>> p in A
True
>>> p.length() == Integer(10)
True

>>> # needs sage.combinat sage.groups
>>> W = CoxeterGroup(['A', Integer(4)])
>>> p = W.random_element_of_length(Integer(5))
>>> p in W
True
>>> p.length() == Integer(5)
True
```

reflection_representation(*base_ring=None*, *side='left'*)

Return the reflection representation of `self`.

This is also the canonical faithful representation of a Coxeter group.

INPUT:

- `base_ring` – (optional) the base ring; the default is the base ring of *canonical_representation()*
- `side` – ignored

EXAMPLES:

```
sage: W = CoxeterGroup(['D', 4])
sage: W.reflection_representation()
Reflection representation of Finite Coxeter group over
 Integer Ring with Coxeter matrix:
[1 3 2 2]
[3 1 3 3]
[2 3 1 2]
[2 3 2 1]

sage: W = CoxeterGroup(['I', 13])
sage: W.reflection_representation()
Reflection representation of Finite Coxeter group over
 Universal Cyclotomic Field with Coxeter matrix:
[ 1 13]
[13  1]

sage: W = WeylGroup(["B", 3, 1])
sage: W.reflection_representation(QQ)
```

(continues on next page)

(continued from previous page)

```
Reflection representation of Weyl Group of type ['B', 3, 1]
 (as a matrix group acting on the root space)
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['D', Integer(4)])
>>> W.reflection_representation()
Reflection representation of Finite Coxeter group over
 Integer Ring with Coxeter matrix:
[1 3 2 2]
[3 1 3 3]
[2 3 1 2]
[2 3 2 1]

>>> W = CoxeterGroup(['I', Integer(13)])
>>> W.reflection_representation()
Reflection representation of Finite Coxeter group over
 Universal Cyclotomic Field with Coxeter matrix:
[ 1 13]
[13  1]

>>> W = WeylGroup(["B", Integer(3), Integer(1)])
>>> W.reflection_representation(QQ)
Reflection representation of Weyl Group of type ['B', 3, 1]
 (as a matrix group acting on the root space)
```

sign_representation(*base_ring=None*)

Return the sign representation of `self` over `base_ring`.

INPUT:

- `base_ring` – (optional) the base ring; the default is **Z**

EXAMPLES:

```
sage: W = WeylGroup(['D', 4])                                                  #␣
↪needs sage.combinat sage.groups
sage: W.sign_representation(QQ)                                                #␣
↪needs sage.combinat sage.groups
Sign representation of
 Weyl Group of type ['D', 4] (as a matrix group acting on the ambient␣
↪space)
 over Rational Field

sage: # optional - gap3
sage: W = CoxeterGroup(['B',3], implementation="coxeter3")
sage: W.sign_representation()
Sign representation of Coxeter group of type ['B', 3]
 implemented by Coxeter3 over Integer Ring
```

```
>>> from sage.all import *
>>> W = WeylGroup(['D', Integer(4)])                                           ␣
↪     # needs sage.combinat sage.groups
>>> W.sign_representation(QQ)                                                 #␣
↪needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
Sign representation of
 Weyl Group of type ['D', 4] (as a matrix group acting on the ambient␣
↪space)
 over Rational Field

>>> # optional - gap3
>>> W = CoxeterGroup(['B',Integer(3)], implementation="coxeter3")
>>> W.sign_representation()
Sign representation of Coxeter group of type ['B', 3]
 implemented by Coxeter3 over Integer Ring
```

simple_projection(*i*, *side='right'*, *length_increasing=True*)

Return the simple projection π_i (or $\overline{\pi}_i$ if `length_increasing` is `False`).

INPUT:

- `i` – an element of the index set of `self`

See *simple_projections()* for the options and for the definition of the simple projections.

EXAMPLES:

```
sage: W = CoxeterGroups().example()
sage: W
The symmetric group on {0, ..., 3}
sage: s = W.simple_reflections()
sage: sigma = W.an_element()
sage: sigma
(1, 2, 3, 0)
sage: u0 = W.simple_projection(0)
sage: d0 = W.simple_projection(0, length_increasing=False)
sage: sigma.length()
3
sage: pi=sigma*s[0]
sage: pi.length()
4
sage: u0(sigma)
(2, 1, 3, 0)
sage: pi
(2, 1, 3, 0)
sage: u0(pi)
(2, 1, 3, 0)
sage: d0(sigma)
(1, 2, 3, 0)
sage: d0(pi)
(1, 2, 3, 0)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example()
>>> W
The symmetric group on {0, ..., 3}
>>> s = W.simple_reflections()
>>> sigma = W.an_element()
>>> sigma
(1, 2, 3, 0)
```

(continues on next page)

(continued from previous page)

```
>>> u0 = W.simple_projection(Integer(0))
>>> d0 = W.simple_projection(Integer(0), length_increasing=False)
>>> sigma.length()
3
>>> pi=sigma*s[Integer(0)]
>>> pi.length()
4
>>> u0(sigma)
(2, 1, 3, 0)
>>> pi
(2, 1, 3, 0)
>>> u0(pi)
(2, 1, 3, 0)
>>> d0(sigma)
(1, 2, 3, 0)
>>> d0(pi)
(1, 2, 3, 0)
```

simple_projections(*side='right'*, *length_increasing=True*)

Return the family of simple projections, also known as 0-Hecke or Demazure operators.

INPUT:

- `self` – a Coxeter group W
- `side` – `'left'` or `'right'` (default: `'right'`)
- `length_increasing` – boolean (default: `True`); whether the operator increases or decreases length

This returns the simple projections of W, as a family.

To each simple reflection s_i of W, corresponds a *simple projection* π_i from W to W defined by:

$\pi_i(w) = ws_i$ if i is not a descent of w $\pi_i(w) = w$ otherwise.

The simple projections $(\pi_i)_{i \in I}$ move elements down the right permutohedron, toward the maximal element. They satisfy the same braid relations as the simple reflections, but are idempotents $\pi_i^2 = \pi$ not involutions $s_i^2 = 1$. As such, the simple projections generate the 0-Hecke monoid.

By symmetry, one can also define the projections $(\overline{\pi}_i)_{i \in I}$ (when the option `length_increasing` is False):

$\overline{\pi}_i(w) = ws_i$ if i is a descent of w $\overline{\pi}_i(w) = w$ otherwise.

as well as the analogues acting on the left (when the option `side` is 'left').

EXAMPLES:

```
sage: W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
sage: s = W.simple_reflections()
sage: sigma = W.an_element(); sigma
(1, 2, 3, 0)
sage: pi = W.simple_projections(); pi
Finite family {0: <function ...<lambda> at ...>,
               1: <function ...<lambda> at ...>,
               2: <function ...<lambda> ...>}
sage: pi[1](sigma)
(1, 3, 2, 0)
sage: W.simple_projection(1)(sigma)
(1, 3, 2, 0)
```

```
>>> from sage.all import *
>>> W = CoxeterGroups().example(); W
The symmetric group on {0, ..., 3}
>>> s = W.simple_reflections()
>>> sigma = W.an_element(); sigma
(1, 2, 3, 0)
>>> pi = W.simple_projections(); pi
Finite family {0: <function ...<lambda> at ...>,
               1: <function ...<lambda> at ...>,
               2: <function ...<lambda> ...>}
>>> pi[Integer(1)](sigma)
(1, 3, 2, 0)
>>> W.simple_projection(Integer(1))(sigma)
(1, 3, 2, 0)
```

standard_coxeter_elements()

Return all standard Coxeter elements in `self`.

This is the set of all elements in `self` obtained from any product of the simple reflections in `self`.

Note

- `self` is assumed to be well-generated.
- This works even beyond real reflection groups, but the conjugacy class is not unique and we only obtain one such class.

EXAMPLES:

```
sage: W = ReflectionGroup(4)                  # optional - gap3
sage: sorted(W.standard_coxeter_elements())   # optional - gap3
[(1,7,6,12,23,20)(2,8,17,24,9,5)(3,16,10,19,15,21)(4,14,11,22,18,13),
 (1,10,4,12,21,22)(2,11,19,24,13,3)(5,15,7,17,16,23)(6,18,8,20,14,9)]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup(Integer(4))                  # optional - gap3
>>> sorted(W.standard_coxeter_elements())   # optional - gap3
[(1,7,6,12,23,20)(2,8,17,24,9,5)(3,16,10,19,15,21)(4,14,11,22,18,13),
 (1,10,4,12,21,22)(2,11,19,24,13,3)(5,15,7,17,16,23)(6,18,8,20,14,9)]
```

weak_order_ideal(*predicate*, *side='right'*, *category=None*)

Return a weak order ideal defined by a predicate.

INPUT:

- `predicate` – a predicate on the elements of `self` defining an weak order ideal in `self`
- `side` – `'left'` or `'right'` (default: `'right'`)

OUTPUT: an enumerated set

EXAMPLES:

```
sage: D6 = FiniteCoxeterGroups().example(5)
sage: I = D6.weak_order_ideal(predicate=lambda w: w.length() <= 3)
sage: I.cardinality()
```

(continues on next page)

(continued from previous page)

```
7
sage: list(I)
[(), (1,), (2,), (1, 2), (2, 1), (1, 2, 1), (2, 1, 2)]
```

```
>>> from sage.all import *
>>> D6 = FiniteCoxeterGroups().example(Integer(5))
>>> I = D6.weak_order_ideal(predicate=lambda w: w.length() <= Integer(3))
>>> I.cardinality()
7
>>> list(I)
[(), (1,), (2,), (1, 2), (2, 1), (1, 2, 1), (2, 1, 2)]
```

We now consider an infinite Coxeter group:

```
sage: W = WeylGroup(["A",1,1])                                          #␣
↪needs sage.groups sage.rings.number_field
sage: I = W.weak_order_ideal(predicate=lambda w: w.length() <= 2)       #␣
↪needs sage.groups sage.rings.number_field
sage: list(iter(I))                                                     #␣
↪needs sage.groups sage.rings.number_field
[
[1 0]  [-1  2]  [ 1  0]  [ 3 -2]  [-1  2]
[0 1], [ 0  1], [ 2 -1], [ 2 -1], [-2  3]
]
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A",Integer(1),Integer(1)])                          ␣
↪              # needs sage.groups sage.rings.number_field
>>> I = W.weak_order_ideal(predicate=lambda w: w.length() <= Integer(2)) ␣
↪     # needs sage.groups sage.rings.number_field
>>> list(iter(I))                                                      #␣
↪needs sage.groups sage.rings.number_field
[
[1 0]  [-1  2]  [ 1  0]  [ 3 -2]  [-1  2]
[0 1], [ 0  1], [ 2 -1], [ 2 -1], [-2  3]
]
```

Even when the result is finite, some features of *FiniteEnumeratedSets* are not available:

```
sage: I.cardinality()  # todo: not implemented
5
sage: list(I)          # todo: not implemented
```

```
>>> from sage.all import *
>>> I.cardinality()  # todo: not implemented
5
>>> list(I)          # todo: not implemented
```

unless this finiteness is explicitly specified:

```
sage: I = W.weak_order_ideal(predicate=lambda w: w.length() <= 2,       #␣
↪needs sage.groups sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
....:                          category=FiniteEnumeratedSets())
sage: I.cardinality()                                                    #␣
↪needs sage.groups sage.rings.number_field
5
sage: list(I)                                                            #␣
↪needs sage.groups sage.rings.number_field
[
[1 0]  [-1  2]  [ 1  0]  [ 3 -2]  [-1  2]
[0 1], [ 0  1], [ 2 -1], [ 2 -1], [-2  3]
]
```

```
>>> from sage.all import *
>>> I = W.weak_order_ideal(predicate=lambda w: w.length() <= Integer(2), ␣
↪     # needs sage.groups sage.rings.number_field
...                         category=FiniteEnumeratedSets())
>>> I.cardinality()                                                      #␣
↪needs sage.groups sage.rings.number_field
5
>>> list(I)                                                              #␣
↪needs sage.groups sage.rings.number_field
[
[1 0]  [-1  2]  [ 1  0]  [ 3 -2]  [-1  2]
[0 1], [ 0  1], [ 2 -1], [ 2 -1], [-2  3]
]
```

Background

The weak order is returned as a `RecursivelyEnumeratedSet_forest`. This is achieved by assigning to each element $u1$ of the ideal a single ancestor $u = u1s_i$, where i is the smallest descent of u.

This allows for iterating through the elements in roughly Constant Amortized Time and constant memory (taking the operations and size of the generated objects as constants).

additional_structure()

Return `None`.

Indeed, all the structure Coxeter groups have in addition to groups (simple reflections, …) is already defined in the super category.

See also

Category.additional_structure()

EXAMPLES:

```
sage: CoxeterGroups().additional_structure()
```

```
>>> from sage.all import *
>>> CoxeterGroups().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: CoxeterGroups().super_categories()
[Category of generalized Coxeter groups]
```

```
>>> from sage.all import *
>>> CoxeterGroups().super_categories()
[Category of generalized Coxeter groups]
```

4.33 Crystals

class sage.categories.crystals.**CrystalHomset**(*X*, *Y*, *category=None*)

Bases: *Homset*

The set of crystal morphisms from one crystal to another.

An $U_q(\mathfrak{g})$ I-crystal morphism $\Psi : B \to C$ is a map $\Psi : B \cup \{0\} \to C \cup \{0\}$ such that:

- $\Psi(0) = 0$.
- If $b \in B$ and $\Psi(b) \in C$, then $\mathrm{wt}(\Psi(b)) = \mathrm{wt}(b)$, $\varepsilon_i(\Psi(b)) = \varepsilon_i(b)$, and $\varphi_i(\Psi(b)) = \varphi_i(b)$ for all $i \in I$.
- If $b, b' \in B$, $\Psi(b), \Psi(b') \in C$ and $f_i b = b'$, then $f_i \Psi(b) = \Psi(b')$ and $\Psi(b) = e_i \Psi(b')$ for all $i \in I$.

If the Cartan type is unambiguous, it is suppressed from the notation.

We can also generalize the definition of a crystal morphism by considering a map of σ of the (now possibly different) Dynkin diagrams corresponding to B and C along with scaling factors $\gamma_i \in \mathbf{Z}$ for $i \in I$. Let σ_i denote the orbit of i under σ. We write objects for B as X with corresponding objects of C as $\widehat{X}$. Then a *virtual* crystal morphism Ψ is a map such that the following holds:

- $\Psi(0) = 0$.
- If $b \in B$ and $\Psi(b) \in C$, then for all $j \in \sigma_i$:

$$\varepsilon_i(b) = \frac{1}{\gamma_j}\widehat{\varepsilon}_j(\Psi(b)), \quad \varphi_i(b) = \frac{1}{\gamma_j}\widehat{\varphi}_j(\Psi(b)), \quad \mathrm{wt}(\Psi(b)) = \sum_i c_i \sum_{j \in \sigma_i} \gamma_j \widehat{\Lambda}_j,$$

where $\mathrm{wt}(b) = \sum_i c_i \Lambda_i$.

- If $b, b' \in B$, $\Psi(b), \Psi(b') \in C$ and $f_i b = b'$, then independent of the ordering of σ_i we have:

$$\Psi(b') = e_i \Psi(b) = \prod_{j \in \sigma_i} \widehat{e}_j^{\gamma_i} \Psi(b), \quad \Psi(b') = f_i \Psi(b) = \prod_{j \in \sigma_i} \widehat{f}_j^{\gamma_i} \Psi(b).$$

If $\gamma_i = 1$ for all $i \in I$ and the Dynkin diagrams are the same, then we call Ψ a *twisted* crystal morphism.

INPUT:

- `X` – the domain
- `Y` – the codomain
- `category` – (optional) the category of the crystal morphisms

See also

For the construction of an element of the homset, see *CrystalMorphismByGenerators* and *crystal_morphism()*.

EXAMPLES:

We begin with the natural embedding of $B(2\Lambda_1)$ into $B(\Lambda_1) \otimes B(\Lambda_1)$ in type A_1:

```
sage: B = crystals.Tableaux(['A',1], shape=[2])
sage: F = crystals.Tableaux(['A',1], shape=[1])
sage: T = crystals.TensorProduct(F, F)
sage: v = T.highest_weight_vectors()[0]; v
[[[1]], [[1]]]
sage: H = Hom(B, T)
sage: psi = H([v])
sage: b = B.highest_weight_vector(); b
[[1, 1]]
sage: psi(b)
[[[1]], [[1]]]
sage: b.f(1)
[[1, 2]]
sage: psi(b.f(1))
[[[1]], [[2]]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(1)], shape=[Integer(2)])
>>> F = crystals.Tableaux(['A',Integer(1)], shape=[Integer(1)])
>>> T = crystals.TensorProduct(F, F)
>>> v = T.highest_weight_vectors()[Integer(0)]; v
[[[1]], [[1]]]
>>> H = Hom(B, T)
>>> psi = H([v])
>>> b = B.highest_weight_vector(); b
[[1, 1]]
>>> psi(b)
[[[1]], [[1]]]
>>> b.f(Integer(1))
[[1, 2]]
>>> psi(b.f(Integer(1)))
[[[1]], [[2]]]
```

We now look at the decomposition of $B(\Lambda_1) \otimes B(\Lambda_1)$ into $B(2\Lambda_1) \oplus B(0)$:

```
sage: B0 = crystals.Tableaux(['A',1], shape=[])
sage: D = crystals.DirectSum([B, B0])
sage: H = Hom(T, D)
sage: psi = H(D.module_generators)
sage: psi
['A', 1] Crystal morphism:
  From: Full tensor product of the crystals
   [The crystal of tableaux of type ['A', 1] and shape(s) [[1]],
    The crystal of tableaux of type ['A', 1] and shape(s) [[1]]]
```

(continues on next page)

(continued from previous page)

```
  To:   Direct sum of the crystals Family
   (The crystal of tableaux of type ['A', 1] and shape(s) [[2]],
    The crystal of tableaux of type ['A', 1] and shape(s) [[]])
  Defn: [[[1]], [[1]]] |--> [[1, 1]]
        [[[2]], [[1]]] |--> []
sage: psi.is_isomorphism()
True
```

```
>>> from sage.all import *
>>> B0 = crystals.Tableaux(['A',Integer(1)], shape=[])
>>> D = crystals.DirectSum([B, B0])
>>> H = Hom(T, D)
>>> psi = H(D.module_generators)
>>> psi
['A', 1] Crystal morphism:
  From: Full tensor product of the crystals
   [The crystal of tableaux of type ['A', 1] and shape(s) [[1]],
    The crystal of tableaux of type ['A', 1] and shape(s) [[1]]]
  To:   Direct sum of the crystals Family
   (The crystal of tableaux of type ['A', 1] and shape(s) [[2]],
    The crystal of tableaux of type ['A', 1] and shape(s) [[]])
  Defn: [[[1]], [[1]]] |--> [[1, 1]]
        [[[2]], [[1]]] |--> []
>>> psi.is_isomorphism()
True
```

We can always construct the trivial morphism which sends everything to 0:

```
sage: Binf = crystals.infinity.Tableaux(['B', 2])
sage: B = crystals.Tableaux(['B',2], shape=[1])
sage: H = Hom(Binf, B)
sage: psi = H(lambda x: None)
sage: psi(Binf.highest_weight_vector())
```

```
>>> from sage.all import *
>>> Binf = crystals.infinity.Tableaux(['B', Integer(2)])
>>> B = crystals.Tableaux(['B',Integer(2)], shape=[Integer(1)])
>>> H = Hom(Binf, B)
>>> psi = H(lambda x: None)
>>> psi(Binf.highest_weight_vector())
```

For Kirillov-Reshetikhin crystals, we consider the map to the corresponding classical crystal:

```
sage: K = crystals.KirillovReshetikhin(['D',4,1], 2,1)
sage: B = K.classical_decomposition()
sage: H = Hom(K, B)
sage: psi = H(lambda x: x.lift(), cartan_type=['D',4])
sage: L = [psi(mg) for mg in K.module_generators]; L
[[], [[1], [2]]]
sage: all(x.parent() == B for x in L)
True
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)], Integer(2),
↪Integer(1))
>>> B = K.classical_decomposition()
>>> H = Hom(K, B)
>>> psi = H(lambda x: x.lift(), cartan_type=['D',Integer(4)])
>>> L = [psi(mg) for mg in K.module_generators]; L
[[], [[1], [2]]]
>>> all(x.parent() == B for x in L)
True
```

Next we consider a type D_4 crystal morphism where we twist by $3 \leftrightarrow 4$:

```
sage: B = crystals.Tableaux(['D',4], shape=[1])
sage: H = Hom(B, B)
sage: d = {1:1, 2:2, 3:4, 4:3}
sage: psi = H(B.module_generators, automorphism=d)
sage: b = B.highest_weight_vector()
sage: b.f_string([1,2,3])
[[4]]
sage: b.f_string([1,2,4])
[[-4]]
sage: psi(b.f_string([1,2,3]))
[[-4]]
sage: psi(b.f_string([1,2,4]))
[[4]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1)])
>>> H = Hom(B, B)
>>> d = {Integer(1):Integer(1), Integer(2):Integer(2), Integer(3):Integer(4),␣
↪Integer(4):Integer(3)}
>>> psi = H(B.module_generators, automorphism=d)
>>> b = B.highest_weight_vector()
>>> b.f_string([Integer(1),Integer(2),Integer(3)])
[[4]]
>>> b.f_string([Integer(1),Integer(2),Integer(4)])
[[-4]]
>>> psi(b.f_string([Integer(1),Integer(2),Integer(3)]))
[[-4]]
>>> psi(b.f_string([Integer(1),Integer(2),Integer(4)]))
[[4]]
```

We construct the natural virtual embedding of a type B_3 into a type D_4 crystal:

```
sage: B = crystals.Tableaux(['B',3], shape=[1])
sage: C = crystals.Tableaux(['D',4], shape=[2])
sage: H = Hom(B, C)
sage: psi = H(C.module_generators)
sage: psi
['B', 3] -> ['D', 4] Virtual Crystal morphism:
  From: The crystal of tableaux of type ['B', 3] and shape(s) [[1]]
  To:   The crystal of tableaux of type ['D', 4] and shape(s) [[2]]
```

(continues on next page)

(continued from previous page)

```
  Defn: [[1]] |--> [[1, 1]]
sage: for b in B: print("{} |--> {}".format(b, psi(b)))
[[1]] |--> [[1, 1]]
[[2]] |--> [[2, 2]]
[[3]] |--> [[3, 3]]
[[0]] |--> [[3, -3]]
[[-3]] |--> [[-3, -3]]
[[-2]] |--> [[-2, -2]]
[[-1]] |--> [[-1, -1]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['B',Integer(3)], shape=[Integer(1)])
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2)])
>>> H = Hom(B, C)
>>> psi = H(C.module_generators)
>>> psi
['B', 3] -> ['D', 4] Virtual Crystal morphism:
  From: The crystal of tableaux of type ['B', 3] and shape(s) [[1]]
  To:   The crystal of tableaux of type ['D', 4] and shape(s) [[2]]
  Defn: [[1]] |--> [[1, 1]]
>>> for b in B: print("{} |--> {}".format(b, psi(b)))
[[1]] |--> [[1, 1]]
[[2]] |--> [[2, 2]]
[[3]] |--> [[3, 3]]
[[0]] |--> [[3, -3]]
[[-3]] |--> [[-3, -3]]
[[-2]] |--> [[-2, -2]]
[[-1]] |--> [[-1, -1]]
```

Element

alias of *CrystalMorphismByGenerators*

class sage.categories.crystals.**CrystalMorphism**(*parent*, *cartan_type=None*, *virtualization=None*, *scaling_factors=None*)

Bases: *Morphism*

A crystal morphism.

INPUT:

- `parent` – a homset
- `cartan_type` – (optional) a Cartan type; the default is the Cartan type of the domain
- `virtualization` – (optional) a dictionary whose keys are in the index set of the domain and whose values are lists of entries in the index set of the codomain
- `scaling_factors` – (optional) a dictionary whose keys are in the index set of the domain and whose values are scaling factors for the weight, ε and φ

cartan_type()

Return the Cartan type of `self`.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: psi = Hom(B, B).an_element()
sage: psi.cartan_type()
['A', 2]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> psi = Hom(B, B).an_element()
>>> psi.cartan_type()
['A', 2]
```

is_injective()

Return if `self` is an injective crystal morphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: psi = Hom(B, B).an_element()
sage: psi.is_injective()
False
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> psi = Hom(B, B).an_element()
>>> psi.is_injective()
False
```

is_surjective()

Check if `self` is a surjective crystal morphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['C',2], shape=[1,1])
sage: C = crystals.Tableaux(['C',2], ([2,1], [1,1]))
sage: psi = B.crystal_morphism(C.module_generators[1:], codomain=C)
sage: psi.is_surjective()
False
sage: im_gens = [None, B.module_generators[0]]
sage: psi = C.crystal_morphism(im_gens, codomain=B)
sage: psi.is_surjective()
True

sage: C = crystals.Tableaux(['A',2], shape=[2,1])
sage: B = crystals.infinity.Tableaux(['A',2])
sage: La = RootSystem(['A',2]).weight_lattice().fundamental_weights()
sage: W = crystals.elementary.T(['A',2], La[1]+La[2])
sage: T = W.tensor(B)
sage: mg = T(W.module_generators[0], B.module_generators[0])
sage: psi = Hom(C,T)([mg])
sage: psi.is_surjective()
False
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['C',Integer(2)], shape=[Integer(1),Integer(1)])
>>> C = crystals.Tableaux(['C',Integer(2)], ([Integer(2),Integer(1)],␣
↪[Integer(1),Integer(1)]))
>>> psi = B.crystal_morphism(C.module_generators[Integer(1):], codomain=C)
>>> psi.is_surjective()
False
>>> im_gens = [None, B.module_generators[Integer(0)]]
>>> psi = C.crystal_morphism(im_gens, codomain=B)
>>> psi.is_surjective()
True

>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> B = crystals.infinity.Tableaux(['A',Integer(2)])
>>> La = RootSystem(['A',Integer(2)]).weight_lattice().fundamental_weights()
>>> W = crystals.elementary.T(['A',Integer(2)], La[Integer(1)]+La[Integer(2)])
>>> T = W.tensor(B)
>>> mg = T(W.module_generators[Integer(0)], B.module_generators[Integer(0)])
>>> psi = Hom(C,T)([mg])
>>> psi.is_surjective()
False
```

scaling_factors()

Return the scaling factors γ_i.

EXAMPLES:

```
sage: B = crystals.Tableaux(['B',3], shape=[1])
sage: C = crystals.Tableaux(['D',4], shape=[2])
sage: psi = B.crystal_morphism(C.module_generators)
sage: psi.scaling_factors()
Finite family {1: 2, 2: 2, 3: 1}
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['B',Integer(3)], shape=[Integer(1)])
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2)])
>>> psi = B.crystal_morphism(C.module_generators)
>>> psi.scaling_factors()
Finite family {1: 2, 2: 2, 3: 1}
```

virtualization()

Return the virtualization sets σ_i.

EXAMPLES:

```
sage: B = crystals.Tableaux(['B',3], shape=[1])
sage: C = crystals.Tableaux(['D',4], shape=[2])
sage: psi = B.crystal_morphism(C.module_generators)
sage: psi.virtualization()
Finite family {1: (1,), 2: (2,), 3: (3, 4)}
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['B',Integer(3)], shape=[Integer(1)])
```

(continues on next page)

(continued from previous page)

```
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2)])
>>> psi = B.crystal_morphism(C.module_generators)
>>> psi.virtualization()
Finite family {1: (1,), 2: (2,), 3: (3, 4)}
```

class sage.categories.crystals.**CrystalMorphismByGenerators**(*parent*, *on_gens*, *cartan_type=None*, *virtualization=None*, *scaling_factors=None*, *gens=None*, *check=True*)

Bases: *CrystalMorphism*

A crystal morphism defined by a set of generators which create a virtual crystal inside the codomain.

INPUT:

- `parent` – a homset
- `on_gens` – a function or list that determines the image of the generators (if given a list, then this uses the order of the generators of the domain) of the domain under `self`
- `cartan_type` – (optional) a Cartan type; the default is the Cartan type of the domain
- `virtualization` – (optional) a dictionary whose keys are in the index set of the domain and whose values are lists of entries in the index set of the codomain
- `scaling_factors` – (optional) a dictionary whose keys are in the index set of the domain and whose values are scaling factors for the weight, ε and φ
- `gens` – (optional) a finite list of generators to define the morphism; the default is to use the highest weight vectors of the crystal
- `check` – boolean (default: `True`); check if the crystal morphism is valid

See also

sage.categories.crystals.Crystals.ParentMethods.crystal_morphism()

im_gens()

Return the image of the generators of `self` as a tuple.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: F = crystals.Tableaux(['A',2], shape=[1])
sage: T = crystals.TensorProduct(F, F, F)
sage: H = Hom(T, B)
sage: b = B.highest_weight_vector()
sage: psi = H((None, b, b, None), generators=T.highest_weight_vectors())
sage: psi.im_gens()
(None, [[1, 1], [2]], [[1, 1], [2]], None)
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> F = crystals.Tableaux(['A',Integer(2)], shape=[Integer(1)])
>>> T = crystals.TensorProduct(F, F, F)
```

(continues on next page)

(continued from previous page)

```
>>> H = Hom(T, B)
>>> b = B.highest_weight_vector()
>>> psi = H((None, b, b, None), generators=T.highest_weight_vectors())
>>> psi.im_gens()
(None, [[1, 1], [2]], [[1, 1], [2]], None)
```

image()

Return the image of `self` in the codomain as a `Subcrystal`.

> **Warning**
>
> This assumes that `self` is a strict crystal morphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['B',3], shape=[1])
sage: C = crystals.Tableaux(['D',4], shape=[2])
sage: H = Hom(B, C)
sage: psi = H(C.module_generators)
sage: psi.image()
Virtual crystal of The crystal of tableaux of type ['D', 4] and shape(s)␣
↪[[2]] of type ['B', 3]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['B',Integer(3)], shape=[Integer(1)])
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2)])
>>> H = Hom(B, C)
>>> psi = H(C.module_generators)
>>> psi.image()
Virtual crystal of The crystal of tableaux of type ['D', 4] and shape(s)␣
↪[[2]] of type ['B', 3]
```

to_module_generator(x)

Return a generator `mg` and a path of e_i and f_i operations to `mg`.

OUTPUT: a tuple consisting of:

- a module generator,
- a list of `'e'` and `'f'` to denote which operation, and
- a list of matching indices.

EXAMPLES:

```
sage: B = crystals.elementary.Elementary(['A',2], 2)
sage: psi = B.crystal_morphism(B.module_generators)
sage: psi.to_module_generator(B(4))
(0, ['f', 'f', 'f', 'f'], [2, 2, 2, 2])
sage: psi.to_module_generator(B(-2))
(0, ['e', 'e'], [2, 2])
```

```
>>> from sage.all import *
>>> B = crystals.elementary.Elementary(['A',Integer(2)], Integer(2))
>>> psi = B.crystal_morphism(B.module_generators)
>>> psi.to_module_generator(B(Integer(4)))
(0, ['f', 'f', 'f', 'f'], [2, 2, 2, 2])
>>> psi.to_module_generator(B(-Integer(2)))
(0, ['e', 'e'], [2, 2])
```

class sage.categories.crystals.**Crystals**

Bases: *Category_singleton*

The category of crystals.

See sage.combinat.crystals.crystals for an introduction to crystals.

EXAMPLES:

```
sage: C = Crystals()
sage: C
Category of crystals
sage: C.super_categories()
[Category of... enumerated sets]
sage: C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> C = Crystals()
>>> C
Category of crystals
>>> C.super_categories()
[Category of... enumerated sets]
>>> C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

Parents in this category should implement the following methods:

- either an attribute `_cartan_type` or a method `cartan_type`
- `module_generators`: a list (or container) of distinct elements which generate the crystal using f_i

Furthermore, their elements `x` should implement the following methods:

- `x.e(i)` (returning $e_i(x)$)
- `x.f(i)` (returning $f_i(x)$)
- `x.epsilon(i)` (returning $\varepsilon_i(x)$)
- `x.phi(i)` (returning $\varphi_i(x)$)

EXAMPLES:

```
sage: from sage.misc.abstract_method import abstract_methods_of_class
sage: abstract_methods_of_class(Crystals().element_class)
{'optional': [], 'required': ['e', 'epsilon', 'f', 'phi', 'weight']}
```

```
>>> from sage.all import *
>>> from sage.misc.abstract_method import abstract_methods_of_class
>>> abstract_methods_of_class(Crystals().element_class)
{'optional': [], 'required': ['e', 'epsilon', 'f', 'phi', 'weight']}
```

class ElementMethods

Bases: object

Epsilon()

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(0).Epsilon()
(0, 0, 0, 0, 0, 0)
sage: C(1).Epsilon()
(0, 0, 0, 0, 0, 0)
sage: C(2).Epsilon()
(1, 0, 0, 0, 0, 0)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(0)).Epsilon()
(0, 0, 0, 0, 0, 0)
>>> C(Integer(1)).Epsilon()
(0, 0, 0, 0, 0, 0)
>>> C(Integer(2)).Epsilon()
(1, 0, 0, 0, 0, 0)
```

Phi()

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(0).Phi()
(0, 0, 0, 0, 0, 0)
sage: C(1).Phi()
(1, 0, 0, 0, 0, 0)
sage: C(2).Phi()
(1, 1, 0, 0, 0, 0)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(0)).Phi()
(0, 0, 0, 0, 0, 0)
>>> C(Integer(1)).Phi()
(1, 0, 0, 0, 0, 0)
>>> C(Integer(2)).Phi()
(1, 1, 0, 0, 0, 0)
```

all_paths_to_highest_weight(*index_set=None*)

Iterate over all paths to the highest weight from `self` with respect to `index_set`.

INPUT:

- `index_set` – (optional) a subset of the index set of `self`

EXAMPLES:

```
sage: B = crystals.infinity.Tableaux("A2")
sage: b0 = B.highest_weight_vector()
sage: b = b0.f_string([1, 2, 1, 2])
sage: L = b.all_paths_to_highest_weight()
sage: list(L)
[[2, 1, 2, 1], [2, 2, 1, 1]]

sage: Y = crystals.infinity.GeneralizedYoungWalls(3)
sage: y0 = Y.highest_weight_vector()
sage: y = y0.f_string([0, 1, 2, 3, 2, 1, 0])
sage: list(y.all_paths_to_highest_weight())
[[0, 1, 2, 3, 2, 1, 0],
 [0, 1, 3, 2, 2, 1, 0],
 [0, 3, 1, 2, 2, 1, 0],
 [0, 3, 2, 1, 1, 0, 2],
 [0, 3, 2, 1, 1, 2, 0]]

sage: B = crystals.Tableaux("A3", shape=[4,2,1])
sage: b0 = B.highest_weight_vector()
sage: b = b0.f_string([1, 1, 2, 3])
sage: list(b.all_paths_to_highest_weight())
[[1, 3, 2, 1], [3, 1, 2, 1], [3, 2, 1, 1]]
```

```
>>> from sage.all import *
>>> B = crystals.infinity.Tableaux("A2")
>>> b0 = B.highest_weight_vector()
>>> b = b0.f_string([Integer(1), Integer(2), Integer(1), Integer(2)])
>>> L = b.all_paths_to_highest_weight()
>>> list(L)
[[2, 1, 2, 1], [2, 2, 1, 1]]

>>> Y = crystals.infinity.GeneralizedYoungWalls(Integer(3))
>>> y0 = Y.highest_weight_vector()
>>> y = y0.f_string([Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(2), Integer(1), Integer(0)])
>>> list(y.all_paths_to_highest_weight())
[[0, 1, 2, 3, 2, 1, 0],
 [0, 1, 3, 2, 2, 1, 0],
 [0, 3, 1, 2, 2, 1, 0],
 [0, 3, 2, 1, 1, 0, 2],
 [0, 3, 2, 1, 1, 2, 0]]

>>> B = crystals.Tableaux("A3", shape=[Integer(4),Integer(2),Integer(1)])
>>> b0 = B.highest_weight_vector()
>>> b = b0.f_string([Integer(1), Integer(1), Integer(2), Integer(3)])
>>> list(b.all_paths_to_highest_weight())
[[1, 3, 2, 1], [3, 1, 2, 1], [3, 2, 1, 1]]
```

cartan_type()

Return the Cartan type associated to `self`.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: C(1).cartan_type()
['A', 5]
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> C(Integer(1)).cartan_type()
['A', 5]
```

e (*i*)

Return e_i of `self` if it exists or `None` otherwise.

This method should be implemented by the element class of the crystal.

EXAMPLES:

```
sage: C = Crystals().example(5)
sage: x = C[2]; x
3
sage: x.e(1), x.e(2), x.e(3)
(None, 2, None)
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(5))
>>> x = C[Integer(2)]; x
3
>>> x.e(Integer(1)), x.e(Integer(2)), x.e(Integer(3))
(None, 2, None)
```

e_string (*list*)

Applies $e_{i_r} \cdots e_{i_1}$ to `self` for `list` as $[i_1, ..., i_r]$

EXAMPLES:

```
sage: C = crystals.Letters(['A',3])
sage: b = C(3)
sage: b.e_string([2,1])
1
sage: b.e_string([1,2])
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(3)])
>>> b = C(Integer(3))
>>> b.e_string([Integer(2),Integer(1)])
1
>>> b.e_string([Integer(1),Integer(2)])
```

epsilon (*i*)

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).epsilon(1)
0
```

(continues on next page)

(continued from previous page)

```
sage: C(2).epsilon(1)
1
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).epsilon(Integer(1))
0
>>> C(Integer(2)).epsilon(Integer(1))
1
```

f(*i*)

Return f_i of `self` if it exists or `None` otherwise.

This method should be implemented by the element class of the crystal.

EXAMPLES:

```
sage: C = Crystals().example(5)
sage: x = C[1]; x
2
sage: x.f(1), x.f(2), x.f(3)
(None, 3, None)
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(5))
>>> x = C[Integer(1)]; x
2
>>> x.f(Integer(1)), x.f(Integer(2)), x.f(Integer(3))
(None, 3, None)
```

f_string(*list*)

Applies $f_{i_r} \cdots f_{i_1}$ to `self` for `list` as $[i_1, ..., i_r]$

EXAMPLES:

```
sage: C = crystals.Letters(['A',3])
sage: b = C(1)
sage: b.f_string([1,2])
3
sage: b.f_string([2,1])
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(3)])
>>> b = C(Integer(1))
>>> b.f_string([Integer(1),Integer(2)])
3
>>> b.f_string([Integer(2),Integer(1)])
```

index_set()

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).index_set()
(1, 2, 3, 4, 5)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).index_set()
(1, 2, 3, 4, 5)
```

is_highest_weight(*index_set=None*)

Return `True` if `self` is a highest weight.

Specifying the option `index_set` to be a subset I of the index set of the underlying crystal, finds all highest weight vectors for arrows in I.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).is_highest_weight()
True
sage: C(2).is_highest_weight()
False
sage: C(2).is_highest_weight(index_set = [2,3,4,5])
True
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).is_highest_weight()
True
>>> C(Integer(2)).is_highest_weight()
False
>>> C(Integer(2)).is_highest_weight(index_set = [Integer(2),Integer(3),
↪Integer(4),Integer(5)])
True
```

is_lowest_weight(*index_set=None*)

Return `True` if `self` is a lowest weight.

Specifying the option `index_set` to be a subset I of the index set of the underlying crystal, finds all lowest weight vectors for arrows in I.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).is_lowest_weight()
False
sage: C(6).is_lowest_weight()
True
sage: C(4).is_lowest_weight(index_set = [1,3])
True
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).is_lowest_weight()
```

(continues on next page)

(continued from previous page)

```
False
>>> C(Integer(6)).is_lowest_weight()
True
>>> C(Integer(4)).is_lowest_weight(index_set = [Integer(1),Integer(3)])
True
```

phi(i)

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).phi(1)
1
sage: C(2).phi(1)
0
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).phi(Integer(1))
1
>>> C(Integer(2)).phi(Integer(1))
0
```

phi_minus_epsilon(i)

Return $\varphi_i - \varepsilon_i$ of `self`.

There are sometimes better implementations using the weight for this. It is used for reflections along a string.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).phi_minus_epsilon(1)
1
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).phi_minus_epsilon(Integer(1))
1
```

s(i)

Return the reflection of `self` along its i-string.

EXAMPLES:

```
sage: C = crystals.Tableaux(['A',2], shape=[2,1])
sage: b = C(rows=[[1,1],[3]])
sage: b.s(1)
[[2, 2], [3]]
sage: b = C(rows=[[1,2],[3]])
sage: b.s(2)
[[1, 2], [3]]
sage: T = crystals.Tableaux(['A',2],shape=[4])
sage: t = T(rows=[[1,2,2,2]])
```

(continues on next page)

(continued from previous page)

```
sage: t.s(1)
[[1, 1, 1, 2]]
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> b = C(rows=[[Integer(1),Integer(1)],[Integer(3)]])
>>> b.s(Integer(1))
[[2, 2], [3]]
>>> b = C(rows=[[Integer(1),Integer(2)],[Integer(3)]])
>>> b.s(Integer(2))
[[1, 2], [3]]
>>> T = crystals.Tableaux(['A',Integer(2)],shape=[Integer(4)])
>>> t = T(rows=[[Integer(1),Integer(2),Integer(2),Integer(2)]])
>>> t.s(Integer(1))
[[1, 1, 1, 2]]
```

subcrystal(*index_set=None*, *max_depth=inf*, *direction='both'*, *contained=None*, *cartan_type=None*, *category=None*)

Construct the subcrystal generated by `self` using e_i and/or f_i for all i in `index_set`.

INPUT:

- `index_set` – (default: `None`) the index set; if `None` then use the index set of the crystal
- `max_depth` – (default: infinity) the maximum depth to build
- `direction` – (default: `'both'`) the direction to build the subcrystal; it can be one of the following:
  - `'both'` – using both e_i and f_i
  - `'upper'` – using e_i
  - `'lower'` – using f_i
- `contained` – (optional) a set (or function) defining the containment in the subcrystal
- `cartan_type` – (optional) specify the Cartan type of the subcrystal
- `category` – (optional) specify the category of the subcrystal

See also

- *Crystals.ParentMethods.subcrystal()*

EXAMPLES:

```
sage: C = crystals.KirillovReshetikhin(['A',3,1], 1, 2)
sage: elt = C(1,4)
sage: list(elt.subcrystal(index_set=[1,3]))
[[[1, 4]], [[2, 4]], [[1, 3]], [[2, 3]]]
sage: list(elt.subcrystal(index_set=[1,3], max_depth=1))
[[[1, 4]], [[2, 4]], [[1, 3]]]
sage: list(elt.subcrystal(index_set=[1,3], direction='upper'))
[[[1, 4]], [[1, 3]]]
sage: list(elt.subcrystal(index_set=[1,3], direction='lower'))
[[[1, 4]], [[2, 4]]]
```

```
>>> from sage.all import *
>>> C = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],␣
↪Integer(1), Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> elt = C(Integer(1),Integer(4))
>>> list(elt.subcrystal(index_set=[Integer(1),Integer(3)]))
[[[1, 4]], [[2, 4]], [[1, 3]], [[2, 3]]]
>>> list(elt.subcrystal(index_set=[Integer(1),Integer(3)], max_
↪depth=Integer(1)))
[[[1, 4]], [[2, 4]], [[1, 3]]]
>>> list(elt.subcrystal(index_set=[Integer(1),Integer(3)], direction=
↪'upper'))
[[[1, 4]], [[1, 3]]]
>>> list(elt.subcrystal(index_set=[Integer(1),Integer(3)], direction=
↪'lower'))
[[[1, 4]], [[2, 4]]]
```

tensor(**elts*)

Return the tensor product of `self` with the crystal elements `elts`.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 3])
sage: B = crystals.infinity.Tableaux(['A', 3])
sage: c = C[0]
sage: b = B.highest_weight_vector()
sage: t = c.tensor(c, b)
sage: ascii_art(t)
          1  1  1
1 # 1 #   2  2
          3
sage: tensor([c, c, b]) == t
True
sage: ascii_art(tensor([b, b, c]))
  1  1  1     1  1  1
  2  2    #   2  2    # 1
  3          3
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(3)])
>>> B = crystals.infinity.Tableaux(['A', Integer(3)])
>>> c = C[Integer(0)]
>>> b = B.highest_weight_vector()
>>> t = c.tensor(c, b)
>>> ascii_art(t)
          1  1  1
1 # 1 #   2  2
          3
>>> tensor([c, c, b]) == t
True
>>> ascii_art(tensor([b, b, c]))
  1  1  1     1  1  1
  2  2    #   2  2    # 1
  3          3
```

to_highest_weight(*index_set=None*)

Return the highest weight element u and a list $[i_1, ..., i_k]$ such that `self` $= f_{i_1} ... f_{i_k} u$, where $i_1, ..., i_k$

are elements in `index_set`.

By default the `index_set` is assumed to be the full index set of `self`.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',3], shape = [1])
sage: t = T(rows = [[3]])
sage: t.to_highest_weight()
[[[1]], [2, 1]]
sage: T = crystals.Tableaux(['A',3], shape = [2,1])
sage: t = T(rows = [[1,2],[4]])
sage: t.to_highest_weight()
[[[1, 1], [2]], [1, 3, 2]]
sage: t.to_highest_weight(index_set = [3])
[[[1, 2], [3]], [3]]
sage: K = crystals.KirillovReshetikhin(['A',3,1],2,1)
sage: t = K(rows=[[2],[3]]); t.to_highest_weight(index_set=[1])
[[[1], [3]], [1]]
sage: t.to_highest_weight()
Traceback (most recent call last):
...
ValueError: this is not a highest weight crystal
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(3)], shape = [Integer(1)])
>>> t = T(rows = [[Integer(3)]])
>>> t.to_highest_weight()
[[[1]], [2, 1]]
>>> T = crystals.Tableaux(['A',Integer(3)], shape = [Integer(2),
↪Integer(1)])
>>> t = T(rows = [[Integer(1),Integer(2)],[Integer(4)]])
>>> t.to_highest_weight()
[[[1, 1], [2]], [1, 3, 2]]
>>> t.to_highest_weight(index_set = [Integer(3)])
[[[1, 2], [3]], [3]]
>>> K = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],
↪Integer(2),Integer(1))
>>> t = K(rows=[[Integer(2)],[Integer(3)]]); t.to_highest_weight(index_
↪set=[Integer(1)])
[[[1], [3]], [1]]
>>> t.to_highest_weight()
Traceback (most recent call last):
...
ValueError: this is not a highest weight crystal
```

to_lowest_weight(*index_set=None*)

Return the lowest weight element u and a list $[i_1, ..., i_k]$ such that `self` $= e_{i_1} ... e_{i_k} u$, where $i_1, ..., i_k$ are elements in `index_set`.

By default the `index_set` is assumed to be the full index set of `self`.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',3], shape = [1])
sage: t = T(rows = [[3]])
sage: t.to_lowest_weight()
[[[4]], [3]]
sage: T = crystals.Tableaux(['A',3], shape = [2,1])
sage: t = T(rows = [[1,2],[4]])
sage: t.to_lowest_weight()
[[[3, 4], [4]], [1, 2, 2, 3]]
sage: t.to_lowest_weight(index_set = [3])
[[[1, 2], [4]], []]
sage: K = crystals.KirillovReshetikhin(['A',3,1],2,1)
sage: t = K.module_generator(); t
[[1], [2]]
sage: t.to_lowest_weight(index_set=[1,2,3])
[[[3], [4]], [2, 1, 3, 2]]
sage: t.to_lowest_weight()
Traceback (most recent call last):
...
ValueError: this is not a highest weight crystal
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(3)], shape = [Integer(1)])
>>> t = T(rows = [[Integer(3)]])
>>> t.to_lowest_weight()
[[[4]], [3]]
>>> T = crystals.Tableaux(['A',Integer(3)], shape = [Integer(2),
↪Integer(1)])
>>> t = T(rows = [[Integer(1),Integer(2)],[Integer(4)]])
>>> t.to_lowest_weight()
[[[3, 4], [4]], [1, 2, 2, 3]]
>>> t.to_lowest_weight(index_set = [Integer(3)])
[[[1, 2], [4]], []]
>>> K = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],
↪Integer(2),Integer(1))
>>> t = K.module_generator(); t
[[1], [2]]
>>> t.to_lowest_weight(index_set=[Integer(1),Integer(2),Integer(3)])
[[[3], [4]], [2, 1, 3, 2]]
>>> t.to_lowest_weight()
Traceback (most recent call last):
...
ValueError: this is not a highest weight crystal
```

weight()

Return the weight of this crystal element.

This method should be implemented by the element class of the crystal.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).weight()
(1, 0, 0, 0, 0, 0)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).weight()
(1, 0, 0, 0, 0, 0)
```

Finite

alias of *FiniteCrystals*

class MorphismMethods

Bases: object

is_embedding()

Check if self is an injective crystal morphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['C',2], shape=[1,1])
sage: C = crystals.Tableaux(['C',2], ([2,1], [1,1]))
sage: psi = B.crystal_morphism(C.module_generators[1:], codomain=C)
sage: psi.is_embedding()
True

sage: C = crystals.Tableaux(['A',2], shape=[2,1])
sage: B = crystals.infinity.Tableaux(['A',2])
sage: La = RootSystem(['A',2]).weight_lattice().fundamental_weights()
sage: W = crystals.elementary.T(['A',2], La[1]+La[2])
sage: T = W.tensor(B)
sage: mg = T(W.module_generators[0], B.module_generators[0])
sage: psi = Hom(C,T)([mg])
sage: psi.is_embedding()
True
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['C',Integer(2)], shape=[Integer(1),Integer(1)])
>>> C = crystals.Tableaux(['C',Integer(2)], ([Integer(2),Integer(1)],␣
↪[Integer(1),Integer(1)]))
>>> psi = B.crystal_morphism(C.module_generators[Integer(1):], codomain=C)
>>> psi.is_embedding()
True

>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> B = crystals.infinity.Tableaux(['A',Integer(2)])
>>> La = RootSystem(['A',Integer(2)]).weight_lattice().fundamental_
↪weights()
>>> W = crystals.elementary.T(['A',Integer(2)],␣
↪La[Integer(1)]+La[Integer(2)])
>>> T = W.tensor(B)
>>> mg = T(W.module_generators[Integer(0)], B.module_
↪generators[Integer(0)])
>>> psi = Hom(C,T)([mg])
>>> psi.is_embedding()
True
```

is_isomorphism()

Check if `self` is a crystal isomorphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['C',2], shape=[1,1])
sage: C = crystals.Tableaux(['C',2], ([2,1], [1,1]))
sage: psi = B.crystal_morphism(C.module_generators[1:], codomain=C)
sage: psi.is_isomorphism()
False
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['C',Integer(2)], shape=[Integer(1),Integer(1)])
>>> C = crystals.Tableaux(['C',Integer(2)], ([Integer(2),Integer(1)],␣
↪[Integer(1),Integer(1)]))
>>> psi = B.crystal_morphism(C.module_generators[Integer(1):], codomain=C)
>>> psi.is_isomorphism()
False
```

is_strict()

Check if `self` is a strict crystal morphism.

EXAMPLES:

```
sage: B = crystals.Tableaux(['C',2], shape=[1,1])
sage: C = crystals.Tableaux(['C',2], ([2,1], [1,1]))
sage: psi = B.crystal_morphism(C.module_generators[1:], codomain=C)
sage: psi.is_strict()
True
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['C',Integer(2)], shape=[Integer(1),Integer(1)])
>>> C = crystals.Tableaux(['C',Integer(2)], ([Integer(2),Integer(1)],␣
↪[Integer(1),Integer(1)]))
>>> psi = B.crystal_morphism(C.module_generators[Integer(1):], codomain=C)
>>> psi.is_strict()
True
```

class ParentMethods

Bases: `object`

Lambda()

Return the fundamental weights in the weight lattice realization for the root system associated with the crystal

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: C.Lambda()
Finite family {1: (1, 0, 0, 0, 0, 0), 2: (1, 1, 0, 0, 0, 0), 3: (1, 1, 1,␣
↪0, 0, 0), 4: (1, 1, 1, 1, 0, 0), 5: (1, 1, 1, 1, 1, 0)}
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
```

(continues on next page)

(continued from previous page)

```
>>> C.Lambda()
Finite family {1: (1, 0, 0, 0, 0, 0), 2: (1, 1, 0, 0, 0, 0), 3: (1, 1, 1,␣
↪0, 0, 0), 4: (1, 1, 1, 1, 0, 0), 5: (1, 1, 1, 1, 1, 0)}
```

an_element()

Return an element of `self`.

sage: C = crystals.Letters([‘A’, 5]) sage: C.an_element() 1

cartan_type()

Return the Cartan type of the crystal.

EXAMPLES:

```
sage: C = crystals.Letters(['A',2])
sage: C.cartan_type()
['A', 2]
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(2)])
>>> C.cartan_type()
['A', 2]
```

connected_components()

Return the connected components of `self` as subcrystals.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(B,C)
sage: T.connected_components()
[Subcrystal of Full tensor product of the crystals
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]],
 Subcrystal of Full tensor product of the crystals
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]],
 Subcrystal of Full tensor product of the crystals
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(B,C)
>>> T.connected_components()
[Subcrystal of Full tensor product of the crystals
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]],
 Subcrystal of Full tensor product of the crystals
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]],
 Subcrystal of Full tensor product of the crystals
```

(continues on next page)

(continued from previous page)

```
 [The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
  The crystal of letters for type ['A', 2]]]
```

connected_components_generators()

Return a tuple of generators for each of the connected components of `self`.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(B,C)
sage: T.connected_components_generators()
([[[1, 1], [2]], 1], [[[1, 2], [2]], 1], [[[1, 2], [3]], 1])
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(B,C)
>>> T.connected_components_generators()
([[[1, 1], [2]], 1], [[[1, 2], [2]], 1], [[[1, 2], [3]], 1])
```

crystal_morphism(*on_gens*, *codomain=None*, *cartan_type=None*, *index_set=None*, *generators=None*, *automorphism=None*, *virtualization=None*, *scaling_factors=None*, *category=None*, *check=True*)

Construct a crystal morphism from `self` to another crystal `codomain`.

INPUT:

- `on_gens` – a function or list that determines the image of the generators (if given a list, then this uses the order of the generators of the domain) of `self` under the crystal morphism
- `codomain` – (default: `self`) the codomain of the morphism
- `cartan_type` – (optional) the Cartan type of the morphism; the default is the Cartan type of `self`
- `index_set` – (optional) the index set of the morphism; the default is the index set of the Cartan type
- `generators` – (optional) the generators to define the morphism; the default is the generators of `self`
- `automorphism` – (optional) the automorphism to perform the twisting
- `virtualization` – (optional) a dictionary whose keys are in the index set of the domain and whose values are lists of entries in the index set of the codomain; the default is the identity dictionary
- `scaling_factors` – (optional) a dictionary whose keys are in the index set of the domain and whose values are scaling factors for the weight, ε and φ; the default are all scaling factors to be one
- `category` – (optional) the category for the crystal morphism; the default is the category of *Crystals*.
- `check` – boolean (default: `True`); check if the crystal morphism is valid

See also

For more examples, see *sage.categories.crystals.CrystalHomset*.

EXAMPLES:

We construct the natural embedding of a crystal using tableaux into the tensor product of single boxes via the reading word:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: F = crystals.Tableaux(['A',2], shape=[1])
sage: T = crystals.TensorProduct(F, F, F)
sage: mg = T.highest_weight_vectors()[2]; mg
[[[1]], [[2]], [[1]]]
sage: psi = B.crystal_morphism([mg], codomain=T); psi
['A', 2] Crystal morphism:
  From: The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]]
  To:   Full tensor product of the crystals
         [The crystal of tableaux of type ['A', 2] and shape(s) [[1]],
          The crystal of tableaux of type ['A', 2] and shape(s) [[1]],
          The crystal of tableaux of type ['A', 2] and shape(s) [[1]]]
  Defn: [[1, 1], [2]] |--> [[[1]], [[2]], [[1]]]
sage: b = B.module_generators[0]
sage: b.pp()
  1  1
  2
sage: psi(b)
[[[1]], [[2]], [[1]]]
sage: psi(b.f(2))
[[[1]], [[3]], [[1]]]
sage: psi(b.f_string([2,1,1]))
[[[2]], [[3]], [[2]]]
sage: lw = b.to_lowest_weight()[0]
sage: lw.pp()
  2  3
  3
sage: psi(lw)
[[[3]], [[3]], [[2]]]
sage: psi(lw) == mg.to_lowest_weight()[0]
True
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> F = crystals.Tableaux(['A',Integer(2)], shape=[Integer(1)])
>>> T = crystals.TensorProduct(F, F, F)
>>> mg = T.highest_weight_vectors()[Integer(2)]; mg
[[[1]], [[2]], [[1]]]
>>> psi = B.crystal_morphism([mg], codomain=T); psi
['A', 2] Crystal morphism:
  From: The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]]
  To:   Full tensor product of the crystals
         [The crystal of tableaux of type ['A', 2] and shape(s) [[1]],
          The crystal of tableaux of type ['A', 2] and shape(s) [[1]],
          The crystal of tableaux of type ['A', 2] and shape(s) [[1]]]
  Defn: [[1, 1], [2]] |--> [[[1]], [[2]], [[1]]]
>>> b = B.module_generators[Integer(0)]
>>> b.pp()
  1  1
  2
>>> psi(b)
[[[1]], [[2]], [[1]]]
```

(continues on next page)

(continued from previous page)

```
>>> psi(b.f(Integer(2)))
[[[1]], [[3]], [[1]]]
>>> psi(b.f_string([Integer(2),Integer(1),Integer(1)]))
[[[2]], [[3]], [[2]]]
>>> lw = b.to_lowest_weight()[Integer(0)]
>>> lw.pp()
  2  3
  3
>>> psi(lw)
[[[3]], [[3]], [[2]]]
>>> psi(lw) == mg.to_lowest_weight()[Integer(0)]
True
```

We now take the other isomorphic highest weight component in the tensor product:

```
sage: mg = T.highest_weight_vectors()[1]; mg
[[[2]], [[1]], [[1]]]
sage: psi = B.crystal_morphism([mg], codomain=T)
sage: psi(lw)
[[[3]], [[2]], [[3]]]
```

```
>>> from sage.all import *
>>> mg = T.highest_weight_vectors()[Integer(1)]; mg
[[[2]], [[1]], [[1]]]
>>> psi = B.crystal_morphism([mg], codomain=T)
>>> psi(lw)
[[[3]], [[2]], [[3]]]
```

We construct a crystal morphism of classical crystals using a Kirillov-Reshetikhin crystal:

```
sage: B = crystals.Tableaux(['D', 4], shape=[1,1])
sage: K = crystals.KirillovReshetikhin(['D',4,1], 2,2)
sage: K.module_generators
[[], [[1], [2]], [[1, 1], [2, 2]]]
sage: v = K.module_generators[1]
sage: psi = B.crystal_morphism([v], codomain=K, category=FiniteCrystals())
sage: psi
['D', 4] -> ['D', 4, 1] Virtual Crystal morphism:
  From: The crystal of tableaux of type ['D', 4] and shape(s) [[1, 1]]
  To:   Kirillov-Reshetikhin crystal of type ['D', 4, 1] with (r,s)=(2,2)
  Defn: [[1], [2]] |--> [[1], [2]]
sage: b = B.module_generators[0]
sage: psi(b)
[[1], [2]]
sage: psi(b.to_lowest_weight()[0])
[[-2], [-1]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['D', Integer(4)], shape=[Integer(1),
↪Integer(1)])
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2),Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> K.module_generators
[[], [[1], [2]], [[1, 1], [2, 2]]]
>>> v = K.module_generators[Integer(1)]
>>> psi = B.crystal_morphism([v], codomain=K, category=FiniteCrystals())
>>> psi
['D', 4] -> ['D', 4, 1] Virtual Crystal morphism:
  From: The crystal of tableaux of type ['D', 4] and shape(s) [[1, 1]]
  To:   Kirillov-Reshetikhin crystal of type ['D', 4, 1] with (r,s)=(2,2)
  Defn: [[1], [2]] |--> [[1], [2]]
>>> b = B.module_generators[Integer(0)]
>>> psi(b)
[[1], [2]]
>>> psi(b.to_lowest_weight()[Integer(0)])
[[-2], [-1]]
```

We can define crystal morphisms using a different set of generators. For example, we construct an example using the lowest weight vector:

```
sage: B = crystals.Tableaux(['A',2], shape=[1])
sage: La = RootSystem(['A',2]).weight_lattice().fundamental_weights()
sage: T = crystals.elementary.T(['A',2], La[2])
sage: Bp = T.tensor(B)
sage: C = crystals.Tableaux(['A',2], shape=[2,1])
sage: x = C.module_generators[0].f_string([1,2])
sage: psi = Bp.crystal_morphism([x], generators=Bp.lowest_weight_
↪vectors())
sage: psi(Bp.highest_weight_vector())
[[1, 1], [2]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(1)])
>>> La = RootSystem(['A',Integer(2)]).weight_lattice().fundamental_
↪weights()
>>> T = crystals.elementary.T(['A',Integer(2)], La[Integer(2)])
>>> Bp = T.tensor(B)
>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> x = C.module_generators[Integer(0)].f_string([Integer(1),Integer(2)])
>>> psi = Bp.crystal_morphism([x], generators=Bp.lowest_weight_vectors())
>>> psi(Bp.highest_weight_vector())
[[1, 1], [2]]
```

We can also use a dictionary to specify the generators and their images:

```
sage: psi = Bp.crystal_morphism({Bp.lowest_weight_vectors()[0]: x})
sage: psi(Bp.highest_weight_vector())
[[1, 1], [2]]
```

```
>>> from sage.all import *
>>> psi = Bp.crystal_morphism({Bp.lowest_weight_vectors()[Integer(0)]: x})
>>> psi(Bp.highest_weight_vector())
[[1, 1], [2]]
```

We construct a twisted crystal morphism induced from the diagram automorphism of type $A_3^{(1)}$:

```
sage: La = RootSystem(['A',3,1]).weight_lattice(extended=True).
↪fundamental_weights()
sage: B0 = crystals.GeneralizedYoungWalls(3, La[0])
sage: B1 = crystals.GeneralizedYoungWalls(3, La[1])
sage: phi = B0.crystal_morphism(B1.module_generators, automorphism={0:1,␣
↪1:2, 2:3, 3:0})
sage: phi
['A', 3, 1] Twisted Crystal morphism:
  From: Highest weight crystal of generalized Young walls of Cartan type [
↪'A', 3, 1] and highest weight Lambda[0]
  To:   Highest weight crystal of generalized Young walls of Cartan type [
↪'A', 3, 1] and highest weight Lambda[1]
  Defn: [] |--> []
sage: x = B0.module_generators[0].f_string([0,1,2,3]); x
[[0, 3], [1], [2]]
sage: phi(x)
[[], [1, 0], [2], [3]]
```

```
>>> from sage.all import *
>>> La = RootSystem(['A',Integer(3),Integer(1)]).weight_
↪lattice(extended=True).fundamental_weights()
>>> B0 = crystals.GeneralizedYoungWalls(Integer(3), La[Integer(0)])
>>> B1 = crystals.GeneralizedYoungWalls(Integer(3), La[Integer(1)])
>>> phi = B0.crystal_morphism(B1.module_generators, automorphism=
↪{Integer(0):Integer(1), Integer(1):Integer(2), Integer(2):Integer(3),␣
↪Integer(3):Integer(0)})
>>> phi
['A', 3, 1] Twisted Crystal morphism:
  From: Highest weight crystal of generalized Young walls of Cartan type [
↪'A', 3, 1] and highest weight Lambda[0]
  To:   Highest weight crystal of generalized Young walls of Cartan type [
↪'A', 3, 1] and highest weight Lambda[1]
  Defn: [] |--> []
>>> x = B0.module_generators[Integer(0)].f_string([Integer(0),Integer(1),
↪Integer(2),Integer(3)]); x
[[0, 3], [1], [2]]
>>> phi(x)
[[], [1, 0], [2], [3]]
```

We construct a virtual crystal morphism from type G_2 into type D_4:

```
sage: D = crystals.Tableaux(['D',4], shape=[1,1])
sage: G = crystals.Tableaux(['G',2], shape=[1])
sage: psi = G.crystal_morphism(D.module_generators,
....:                          virtualization={1:[2],2:[1,3,4]},
....:                          scaling_factors={1:1, 2:1})
sage: for x in G:
....:     ascii_art(x, psi(x), sep='  |-->  ')
....:     print("")
          1
  1  |-->  2

          1
```

(continues on next page)

(continued from previous page)

```
  2   |-->    3

              2
  3   |-->   -3

              3
  0   |-->   -3

              3
 -3   |-->   -2

             -3
 -2   |-->   -1

             -2
 -1   |-->   -1
```

```
>>> from sage.all import *
>>> D = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1),Integer(1)])
>>> G = crystals.Tableaux(['G',Integer(2)], shape=[Integer(1)])
>>> psi = G.crystal_morphism(D.module_generators,
...                          virtualization={Integer(1):[Integer(2)],
↪Integer(2):[Integer(1),Integer(3),Integer(4)]},
...                          scaling_factors={Integer(1):Integer(1),␣
↪Integer(2):Integer(1)})
>>> for x in G:
...     ascii_art(x, psi(x), sep='  |-->  ')
...     print("")
            1
  1   |-->    2
<BLANKLINE>
            1
  2   |-->    3
<BLANKLINE>
            2
  3   |-->   -3
<BLANKLINE>
            3
  0   |-->   -3
<BLANKLINE>
            3
 -3   |-->   -2
<BLANKLINE>
           -3
 -2   |-->   -1
<BLANKLINE>
           -2
 -1   |-->   -1
```

digraph(*subset=None*, *index_set=None*)

Return the `DiGraph` associated to `self`.

INPUT:

- `subset` – (optional) a subset of vertices for which the digraph should be constructed
- `index_set` – (optional) the index set to draw arrows

EXAMPLES:

```
sage: C = Crystals().example(5)
sage: C.digraph()
Digraph on 6 vertices
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(5))
>>> C.digraph()
Digraph on 6 vertices
```

The edges of the crystal graph are by default colored using blue for edge 1, red for edge 2, and green for edge 3:

```
sage: C = Crystals().example(3)
sage: G = C.digraph()
sage: view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(3))
>>> G = C.digraph()
>>> view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

One may also overwrite the colors:

```
sage: C = Crystals().example(3)
sage: G = C.digraph()
sage: G.set_latex_options(color_by_label = {1:"red", 2:"purple", 3:"blue"}
↪)
sage: view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(3))
>>> G = C.digraph()
>>> G.set_latex_options(color_by_label = {Integer(1):"red", Integer(2):
↪"purple", Integer(3):"blue"})
>>> view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

Or one may add colors to yet unspecified edges:

```
sage: C = Crystals().example(4)
sage: G = C.digraph()
sage: C.cartan_type()._index_set_coloring[4]="purple"
sage: view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(4))
>>> G = C.digraph()
>>> C.cartan_type()._index_set_coloring[Integer(4)]="purple"
>>> view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

Here is an example of how to take the top part up to a given depth of an infinite dimensional crystal:

```
sage: C = CartanType(['C',2,1])
sage: La = C.root_system().weight_lattice().fundamental_weights()
sage: T = crystals.HighestWeight(La[0])
sage: S = T.subcrystal(max_depth=3)
sage: G = T.digraph(subset=S); G
Digraph on 5 vertices
sage: G.vertices(sort=True, key=str)
[(-Lambda[0] + 2*Lambda[1] - delta,),
 (1/2*Lambda[0] + Lambda[1] - Lambda[2] - 1/2*delta, -1/2*Lambda[0] +␣
↪Lambda[1] - 1/2*delta),
 (1/2*Lambda[0] - Lambda[1] + Lambda[2] - 1/2*delta, -1/2*Lambda[0] +␣
↪Lambda[1] - 1/2*delta),
 (Lambda[0] - 2*Lambda[1] + 2*Lambda[2] - delta,),
 (Lambda[0],)]
```

```
>>> from sage.all import *
>>> C = CartanType(['C',Integer(2),Integer(1)])
>>> La = C.root_system().weight_lattice().fundamental_weights()
>>> T = crystals.HighestWeight(La[Integer(0)])
>>> S = T.subcrystal(max_depth=Integer(3))
>>> G = T.digraph(subset=S); G
Digraph on 5 vertices
>>> G.vertices(sort=True, key=str)
[(-Lambda[0] + 2*Lambda[1] - delta,),
 (1/2*Lambda[0] + Lambda[1] - Lambda[2] - 1/2*delta, -1/2*Lambda[0] +␣
↪Lambda[1] - 1/2*delta),
 (1/2*Lambda[0] - Lambda[1] + Lambda[2] - 1/2*delta, -1/2*Lambda[0] +␣
↪Lambda[1] - 1/2*delta),
 (Lambda[0] - 2*Lambda[1] + 2*Lambda[2] - delta,),
 (Lambda[0],)]
```

Here is a way to construct a picture of a Demazure crystal using the `subset` option:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = B.highest_weight_vector()
sage: D = B.demazure_subcrystal(t, [2,1])
sage: list(D)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
 [[1, 3], [2]], [[1, 3], [3]]]
sage: view(D)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
```

(continues on next page)

(continued from previous page)

```
>>> t = B.highest_weight_vector()
>>> D = B.demazure_subcrystal(t, [Integer(2),Integer(1)])
>>> list(D)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
 [[1, 3], [2]], [[1, 3], [3]]]
>>> view(D)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

We can also choose to display particular arrows using the `index_set` option:

```
sage: C = crystals.KirillovReshetikhin(['D',4,1], 2, 1)
sage: G = C.digraph(index_set=[1,3])
sage: len(G.edges(sort=False))
20
sage: view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> C = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2), Integer(1))
>>> G = C.digraph(index_set=[Integer(1),Integer(3)])
>>> len(G.edges(sort=False))
20
>>> view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

Todo

Add more tests.

direct_sum(X)

Return the direct sum of `self` with `X`.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = crystals.Letters(['A',2])
sage: B.direct_sum(C)
Direct sum of the crystals Family
(The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
 The crystal of letters for type ['A', 2])
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = crystals.Letters(['A',Integer(2)])
>>> B.direct_sum(C)
Direct sum of the crystals Family
(The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
 The crystal of letters for type ['A', 2])
```

As a shorthand, we can use `+`:

```
sage: B + C
Direct sum of the crystals Family
(The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
 The crystal of letters for type ['A', 2])
```

```
>>> from sage.all import *
>>> B + C
Direct sum of the crystals Family
(The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]],
 The crystal of letters for type ['A', 2])
```

dot_tex()

Return a dot_tex string representation of `self`.

EXAMPLES:

```
sage: C = crystals.Letters(['A',2])
sage: C.dot_tex()
'digraph G { \n  node [ shape=plaintext ];\n  N_0 [ label = " ", texlbl =
↪"$1$" ];\n  N_1 [ label = " ", texlbl = "$2$" ];\n  N_2 [ label = " ",␣
↪texlbl = "$3$" ];\n  N_0 -> N_1 [ label = " ", texlbl = "1" ];\n  N_1 ->
↪ N_2 [ label = " ", texlbl = "2" ];\n}'
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(2)])
>>> C.dot_tex()
'digraph G { \n  node [ shape=plaintext ];\n  N_0 [ label = " ", texlbl =
↪"$1$" ];\n  N_1 [ label = " ", texlbl = "$2$" ];\n  N_2 [ label = " ",␣
↪texlbl = "$3$" ];\n  N_0 -> N_1 [ label = " ", texlbl = "1" ];\n  N_1 ->
↪ N_2 [ label = " ", texlbl = "2" ];\n}'
```

index_set()

Return the index set of the Dynkin diagram underlying the crystal.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: C.index_set()
(1, 2, 3, 4, 5)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> C.index_set()
(1, 2, 3, 4, 5)
```

is_connected()

Return `True` if `self` is a connected crystal.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(B,C)
```

(continues on next page)

(continued from previous page)

```
sage: B.is_connected()
True
sage: T.is_connected()
False
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(B,C)
>>> B.is_connected()
True
>>> T.is_connected()
False
```

latex(***options*)

Return the crystal graph as a latex string. This can be exported to a file with `self.latex_file('filename')`.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2],shape=[1])
sage: T._latex_()
'...tikzpicture...'
sage: view(T) # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)],shape=[Integer(1)])
>>> T._latex_()
'...tikzpicture...'
>>> view(T) # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

One can for example also color the edges using the following options:

```
sage: T = crystals.Tableaux(['A',2],shape=[1])
sage: T._latex_(color_by_label={0:"black", 1:"red", 2:"blue"})
'...tikzpicture...'
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)],shape=[Integer(1)])
>>> T._latex_(color_by_label={Integer(0):"black", Integer(1):"red",␣
↪Integer(2):"blue"})
'...tikzpicture...'
```

latex_file(*filename*)

Export a file, suitable for pdflatex, to `filename`.

This requires a proper installation of `dot2tex`. For more information see the documentation for `self.latex()`.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: fn = tmp_filename(ext='.tex')
sage: C.latex_file(fn)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> fn = tmp_filename(ext='.tex')
>>> C.latex_file(fn)
```

metapost(*filename*, *thicklines=False*, *labels=True*, *scaling_factor=1.0*, *tallness=1.0*)

Export a file, suitable for MetaPost, to `filename`.

Root operators $e(1)$ or $f(1)$ move along red lines, $e(2)$ or $f(2)$ along green. The highest weight is in the lower left. Vertices with the same weight are kept close together. The concise labels on the nodes are strings introduced by Berenstein and Zelevinsky and Littelmann; see Littelmann's paper Cones, Crystals, Patterns, sections 5 and 6.

For Cartan types B2 or C2, the pattern has the form

$a_2 a_3 a_4 a_1$

where $c * a_2 = a_3 = 2 * a_4 = 0$ and $a_1 = 0$, with $c = 2$ for B2, $c = 1$ for C2. Applying $e(2)$ a_1 times, $e(1)$ a_2 times, $e(2)$ a_3 times, $e(1)$ a_4 times returns to the highest weight. (Observe that Littelmann writes the roots in opposite of the usual order, so our $e(1)$ is his $e(2)$ for these Cartan types.) For type A2, the pattern has the form

$a_3 a_2 a_1$

where applying $e(1)$ a_3 times, $e(2)$ a_2 times then $e(1)$ a_1 times returns to the highest weight. These data determine the vertex and may be translated into a Gelfand-Tsetlin pattern or tableau.

INPUT:

- `filename` – name of the output file, e.g., `'filename.mp'`
- `thicklines` – boolean (default: `True`); for thicker edges
- `labels` – boolean (default: `False`); whether to suppress labeling of the vertices
- `scaling_factor` – (default: `1.0`) increasing or decreasing the scaling factor changes the size of the image
- `tallness` – (default: `1.0`) increasing makes the image taller without increasing the width

EXAMPLES:

```
sage: C = crystals.Letters(['A', 2])
sage: C.metapost(tmp_filename())
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(2)])
>>> C.metapost(tmp_filename())
```

```
sage: C = crystals.Letters(['A', 5])
sage: C.metapost(tmp_filename())
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
```

(continues on next page)

(continued from previous page)

```
>>> C.metapost(tmp_filename())
Traceback (most recent call last):
...
NotImplementedError
```

number_of_connected_components()

Return the number of connected components of `self`.

EXAMPLES:

```
sage: B = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(B,C)
sage: T.number_of_connected_components()
3
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(B,C)
>>> T.number_of_connected_components()
3
```

plot(***options*)

Return the plot of `self` as a directed graph.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: print(C.plot())
Graphics object consisting of 17 graphics primitives
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> print(C.plot())
Graphics object consisting of 17 graphics primitives
```

plot3d(***options*)

Return the 3-dimensional plot of `self` as a directed graph.

EXAMPLES:

```
sage: C = crystals.KirillovReshetikhin(['A',3,1],2,1)
sage: print(C.plot3d())
Graphics3d Object
```

```
>>> from sage.all import *
>>> C = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],
↪Integer(2),Integer(1))
>>> print(C.plot3d())
Graphics3d Object
```

subcrystal(*index_set=None*, *generators=None*, *max_depth=inf*, *direction='both'*, *contained=None*, *virtualization=None*, *scaling_factors=None*, *cartan_type=None*, *category=None*)

Construct the subcrystal from `generators` using e_i and/or f_i for all i in `index_set`.

INPUT:

- `index_set` – (default: `None`) the index set; if `None` then use the index set of the crystal
- `generators` – (default: `None`) the list of generators; if `None` then use the module generators of the crystal
- `max_depth` – (default: infinity) the maximum depth to build
- `direction` – (default: `'both'`) the direction to build the subcrystal; it can be one of the following:
  - `'both'` – using both e_i and f_i
  - `'upper'` – using e_i
  - `'lower'` – using f_i
- `contained` – (optional) a set or function defining the containment in the subcrystal
- `virtualization`, `scaling_factors` – (optional) dictionaries whose key i corresponds to the sets σ_i and γ_i respectively used to define virtual crystals; see `VirtualCrystal`
- `cartan_type` – (optional) specify the Cartan type of the subcrystal
- `category` – (optional) specify the category of the subcrystal

EXAMPLES:

```
sage: C = crystals.KirillovReshetikhin(['A',3,1], 1, 2)
sage: S = list(C.subcrystal(index_set=[1,2])); S
[[[1, 1]], [[1, 2]], [[2, 2]], [[1, 3]], [[2, 3]], [[3, 3]]]
sage: C.cardinality()
10
sage: len(S)
6
sage: list(C.subcrystal(index_set=[1,3], generators=[C(1,4)]))
[[[1, 4]], [[2, 4]], [[1, 3]], [[2, 3]]]
sage: list(C.subcrystal(index_set=[1,3], generators=[C(1,4)], max_
↪depth=1))
[[[1, 4]], [[2, 4]], [[1, 3]]]
sage: list(C.subcrystal(index_set=[1,3], generators=[C(1,4)], direction=
↪'upper'))
[[[1, 4]], [[1, 3]]]
sage: list(C.subcrystal(index_set=[1,3], generators=[C(1,4)], direction=
↪'lower'))
[[[1, 4]], [[2, 4]]]

sage: G = C.subcrystal(index_set=[1,2,3]).digraph()
sage: GA = crystals.Tableaux('A3', shape=[2]).digraph()
sage: G.is_isomorphic(GA, edge_labels=True)
True
```

```
>>> from sage.all import *
>>> C = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],␣
↪Integer(1), Integer(2))
>>> S = list(C.subcrystal(index_set=[Integer(1),Integer(2)])); S
[[[1, 1]], [[1, 2]], [[2, 2]], [[1, 3]], [[2, 3]], [[3, 3]]]
>>> C.cardinality()
10
>>> len(S)
6
```

(continues on next page)

(continued from previous page)

```
>>> list(C.subcrystal(index_set=[Integer(1),Integer(3)],␣
↪generators=[C(Integer(1),Integer(4))]))
[[[1, 4]], [[2, 4]], [[1, 3]], [[2, 3]]]
>>> list(C.subcrystal(index_set=[Integer(1),Integer(3)],␣
↪generators=[C(Integer(1),Integer(4))], max_depth=Integer(1)))
[[[1, 4]], [[2, 4]], [[1, 3]]]
>>> list(C.subcrystal(index_set=[Integer(1),Integer(3)],␣
↪generators=[C(Integer(1),Integer(4))], direction='upper'))
[[[1, 4]], [[1, 3]]]
>>> list(C.subcrystal(index_set=[Integer(1),Integer(3)],␣
↪generators=[C(Integer(1),Integer(4))], direction='lower'))
[[[1, 4]], [[2, 4]]]

>>> G = C.subcrystal(index_set=[Integer(1),Integer(2),Integer(3)]).
↪digraph()
>>> GA = crystals.Tableaux('A3', shape=[Integer(2)]).digraph()
>>> G.is_isomorphic(GA, edge_labels=True)
True
```

We construct the subcrystal which contains the necessary data to construct the corresponding dual equivalence graph:

```
sage: C = crystals.Tableaux(['A',5], shape=[3,3])
sage: is_wt0 = lambda x: all(x.epsilon(i) == x.phi(i) for i in x.parent().
↪index_set())
sage: def check(x):
....:     if is_wt0(x):
....:         return True
....:     for i in x.parent().index_set()[:-1]:
....:         L = [x.e(i), x.e_string([i,i+1]), x.f(i), x.f_string([i,
↪i+1])]
....:         if any(y is not None and is_wt0(y) for y in L):
....:             return True
....:     return False
sage: wt0 = [x for x in C if is_wt0(x)]
sage: S = C.subcrystal(contained=check, generators=wt0)
sage: S.module_generators[0]
[[1, 3, 5], [2, 4, 6]]
sage: S.module_generators[0].e(2).e(3).f(2).f(3)
[[1, 2, 5], [3, 4, 6]]
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['A',Integer(5)], shape=[Integer(3),Integer(3)])
>>> is_wt0 = lambda x: all(x.epsilon(i) == x.phi(i) for i in x.parent().
↪index_set())
>>> def check(x):
...     if is_wt0(x):
...         return True
...     for i in x.parent().index_set()[:-Integer(1)]:
...         L = [x.e(i), x.e_string([i,i+Integer(1)]), x.f(i), x.f_
↪string([i,i+Integer(1)])]
...         if any(y is not None and is_wt0(y) for y in L):
```

(continues on next page)

(continued from previous page)

```
...             return True
...     return False
>>> wt0 = [x for x in C if is_wt0(x)]
>>> S = C.subcrystal(contained=check, generators=wt0)
>>> S.module_generators[Integer(0)]
[[1, 3, 5], [2, 4, 6]]
>>> S.module_generators[Integer(0)].e(Integer(2)).e(Integer(3)).
↪f(Integer(2)).f(Integer(3))
[[1, 2, 5], [3, 4, 6]]
```

An example of a type B_2 virtual crystal inside of a type A_3 ambient crystal:

```
sage: A = crystals.Tableaux(['A',3], shape=[2,1,1])
sage: S = A.subcrystal(virtualization={1:[1,3], 2:[2]},
....:                  scaling_factors={1:1,2:1}, cartan_type=['B',2])
sage: B = crystals.Tableaux(['B',2], shape=[1])
sage: S.digraph().is_isomorphic(B.digraph(), edge_labels=True)
True
```

```
>>> from sage.all import *
>>> A = crystals.Tableaux(['A',Integer(3)], shape=[Integer(2),Integer(1),
↪Integer(1)])
>>> S = A.subcrystal(virtualization={Integer(1):[Integer(1),Integer(3)],␣
↪Integer(2):[Integer(2)]},
...                  scaling_factors={Integer(1):Integer(1),
↪Integer(2):Integer(1)}, cartan_type=['B',Integer(2)])
>>> B = crystals.Tableaux(['B',Integer(2)], shape=[Integer(1)])
>>> S.digraph().is_isomorphic(B.digraph(), edge_labels=True)
True
```

tensor(**crystals*, ***options*)

Return the tensor product of `self` with the crystals `B`.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 3])
sage: B = crystals.infinity.Tableaux(['A', 3])
sage: T = C.tensor(C, B); T
Full tensor product of the crystals
 [The crystal of letters for type ['A', 3],
  The crystal of letters for type ['A', 3],
  The infinity crystal of tableaux of type ['A', 3]]
sage: tensor([C, C, B]) is T
True

sage: C = crystals.Letters(['A',2])
sage: T = C.tensor(C, C, generators=[[C(2),C(1),C(1)],[C(1),C(2),C(1)]]);␣
↪T
The tensor product of the crystals
 [The crystal of letters for type ['A', 2],
  The crystal of letters for type ['A', 2],
  The crystal of letters for type ['A', 2]]
```

(continues on next page)

(continued from previous page)

```
sage: T.module_generators
([2, 1, 1], [1, 2, 1])
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(3)])
>>> B = crystals.infinity.Tableaux(['A', Integer(3)])
>>> T = C.tensor(C, B); T
Full tensor product of the crystals
 [The crystal of letters for type ['A', 3],
  The crystal of letters for type ['A', 3],
  The infinity crystal of tableaux of type ['A', 3]]
>>> tensor([C, C, B]) is T
True

>>> C = crystals.Letters(['A',Integer(2)])
>>> T = C.tensor(C, C, generators=[[C(Integer(2)),C(Integer(1)),
↪C(Integer(1))],[C(Integer(1)),C(Integer(2)),C(Integer(1))]]); T
The tensor product of the crystals
 [The crystal of letters for type ['A', 2],
  The crystal of letters for type ['A', 2],
  The crystal of letters for type ['A', 2]]
>>> T.module_generators
([2, 1, 1], [1, 2, 1])
```

weight_lattice_realization()

Return the weight lattice realization used to express weights in `self`.

This default implementation uses the ambient space of the root system for (non relabelled) finite types and the weight lattice otherwise. This is a legacy from when ambient spaces were partially implemented, and may be changed in the future.

For affine types, this returns the extended weight lattice by default.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: C.weight_lattice_realization()
Ambient space of the Root system of type ['A', 5]
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: K.weight_lattice_realization()
Weight lattice of the Root system of type ['A', 2, 1]
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> C.weight_lattice_realization()
Ambient space of the Root system of type ['A', 5]
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.weight_lattice_realization()
Weight lattice of the Root system of type ['A', 2, 1]
```

class SubcategoryMethods

Bases: `object`

Methods for all subcategories.

TensorProducts()

Return the full subcategory of objects of `self` constructed as tensor products.

See also

- *tensor.TensorProductsCategory*
- *RegressiveCovariantConstructionCategory*.

EXAMPLES:

```
sage: HighestWeightCrystals().TensorProducts()
Category of tensor products of highest weight crystals
```

```
>>> from sage.all import *
>>> HighestWeightCrystals().TensorProducts()
Category of tensor products of highest weight crystals
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of crystals constructed by tensor product of crystals.

extra_super_categories()

EXAMPLES:

```
sage: Crystals().TensorProducts().extra_super_categories()
[Category of crystals]
```

```
>>> from sage.all import *
>>> Crystals().TensorProducts().extra_super_categories()
[Category of crystals]
```

example(*choice='highwt'*, ***kwds*)

Return an example of a crystal, as per *Category.example()*.

INPUT:

- `choice` – string (default: `'highwt'`); can be either `'highwt'` for the highest weight crystal of type A, or `'naive'` for an example of a broken crystal
- `**kwds` – keyword arguments passed onto the constructor for the chosen crystal

EXAMPLES:

```
sage: Crystals().example(choice='highwt', n=5)
Highest weight crystal of type A_5 of highest weight omega_1
sage: Crystals().example(choice='naive')
A broken crystal, defined by digraph, of dimension five.
```

```
>>> from sage.all import *
>>> Crystals().example(choice='highwt', n=Integer(5))
Highest weight crystal of type A_5 of highest weight omega_1
```

(continues on next page)

(continued from previous page)

```
>>> Crystals().example(choice='naive')
A broken crystal, defined by digraph, of dimension five.
```

super_categories()

EXAMPLES:

```
sage: Crystals().super_categories()
[Category of enumerated sets]
```

```
>>> from sage.all import *
>>> Crystals().super_categories()
[Category of enumerated sets]
```

4.34 CW Complexes

class sage.categories.cw_complexes.**CWComplexes**

Bases: *Category_singleton*

The category of CW complexes.

A CW complex is a Closure-finite cell complex in the Weak topology.

REFERENCES:

- Wikipedia article CW_complex

Note

The notion of "finite" is that the number of cells is finite.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: C = CWComplexes(); C
Category of CW complexes
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> C = CWComplexes(); C
Category of CW complexes
```

Compact_extra_super_categories()

Return extraneous super categories for `CWComplexes().Compact()`.

A compact CW complex is finite, see Proposition A.1 in [Hat2002].

Todo

Fix the name of finite CW complexes.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: CWComplexes().Compact() # indirect doctest
Category of finite finite dimensional CW complexes
sage: CWComplexes().Compact() is CWComplexes().Finite()
True
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> CWComplexes().Compact() # indirect doctest
Category of finite finite dimensional CW complexes
>>> CWComplexes().Compact() is CWComplexes().Finite()
True
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom*

The category of connected CW complexes.

class ElementMethods

Bases: object

dimension()

Return the dimension of self.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: X.an_element().dimension()
2
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> X.an_element().dimension()
2
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

Category of finite CW complexes.

A finite CW complex is a CW complex with a finite number of cells.

class ParentMethods

Bases: object

dimension()

Return the dimension of self.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: X.dimension()
2
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> X.dimension()
2
```

extra_super_categories()

Return the extra super categories of `self`.

A finite CW complex is a compact finite-dimensional CW complex.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: C = CWComplexes().Finite()
sage: C.extra_super_categories()
[Category of finite dimensional CW complexes,
 Category of compact topological spaces]
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> C = CWComplexes().Finite()
>>> C.extra_super_categories()
[Category of finite dimensional CW complexes,
 Category of compact topological spaces]
```

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom*

Category of finite dimensional CW complexes.

class ParentMethods

Bases: object

cells()

Return the cells of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: C = X.cells()
sage: sorted((d, C[d]) for d in C.keys())
[(0, (0-cell v,)),
 (1, (0-cell e1, 0-cell e2)),
 (2, (2-cell f,))]
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> C = X.cells()
>>> sorted((d, C[d]) for d in C.keys())
[(0, (0-cell v,)),
 (1, (0-cell e1, 0-cell e2)),
 (2, (2-cell f,))]
```

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: X.dimension()
2
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> X.dimension()
2
```

class SubcategoryMethods

Bases: `object`

Connected()

Return the full subcategory of the connected objects of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: CWComplexes().Connected()
Category of connected CW complexes
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> CWComplexes().Connected()
Category of connected CW complexes
```

FiniteDimensional()

Return the full subcategory of the finite dimensional objects of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: C = CWComplexes().FiniteDimensional(); C
Category of finite dimensional CW complexes
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> C = CWComplexes().FiniteDimensional(); C
Category of finite dimensional CW complexes
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: CWComplexes().super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> CWComplexes().super_categories()
[Category of topological spaces]
```

4.35 Dedekind Domains

class sage.categories.dedekind_domains.**DedekindDomains**

Bases: *Category*

The category of Dedekind domains.

A Dedekind domain is a Noetherian integral domain of Krull dimension one that is integrally closed in its field of fractions.

EXAMPLES:

```
sage: C = DedekindDomains(); C
Category of Dedekind domains
sage: C.super_categories()
[Category of integral domains]
```

```
>>> from sage.all import *
>>> C = DedekindDomains(); C
Category of Dedekind domains
>>> C.super_categories()
[Category of integral domains]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

integral_closure()

Return self since Dedekind domains are integrally closed.

EXAMPLES:

```
sage: # needs sage.rings.number_field
sage: x = polygen(ZZ, 'x')
sage: K = NumberField(x^2 + 1, 's')
sage: OK = K.ring_of_integers()
sage: OK.integral_closure()
Gaussian Integers generated by s in Number Field in s
 with defining polynomial x^2 + 1
sage: OK.integral_closure() == OK
True

sage: QQ.integral_closure() == QQ
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(1), 's')
>>> OK = K.ring_of_integers()
>>> OK.integral_closure()
Gaussian Integers generated by s in Number Field in s
 with defining polynomial x^2 + 1
>>> OK.integral_closure() == OK
True

>>> QQ.integral_closure() == QQ
True
```

is_integrally_closed()

Return `True` since Dedekind domains are integrally closed.

EXAMPLES:

The following are examples of Dedekind domains:

```
sage: ZZ.is_integrally_closed()
True
sage: x = polygen(ZZ, 'x')
sage: K = NumberField(x^2 + 1, 's')
 ↪  # needs sage.rings.number_field
sage: OK = K.ring_of_integers()
 ↪  # needs sage.rings.number_field
sage: OK.is_integrally_closed()
 ↪  # needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> ZZ.is_integrally_closed()
True
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(1), 's')
 ↪                  # needs sage.rings.number_field
>>> OK = K.ring_of_integers()
 ↪# needs sage.rings.number_field
>>> OK.is_integrally_closed()
 ↪# needs sage.rings.number_field
True
```

These, however, are not Dedekind domains:

```
sage: QQ.is_integrally_closed()
True
sage: S = ZZ[sqrt(5)]; S.is_integrally_closed()
 ↪  # needs sage.rings.number_field sage.symbolic
False
sage: T.<x,y> = PolynomialRing(QQ, 2); T
Multivariate Polynomial Ring in x, y over Rational Field
```

(continues on next page)

(continued from previous page)

```
sage: T.is_integral_domain()
True
```

```
>>> from sage.all import *
>>> QQ.is_integrally_closed()
True
>>> S = ZZ[sqrt(Integer(5))]; S.is_integrally_closed()
↪          # needs sage.rings.number_field sage.symbolic
False
>>> T = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = T._
↪first_ngens(2); T
Multivariate Polynomial Ring in x, y over Rational Field
>>> T.is_integral_domain()
True
```

is_noetherian()

Return `True` since Dedekind domains are Noetherian.

EXAMPLES:

The integers, **Z**, and rings of integers of number fields are Dedekind domains:

```
sage: ZZ.is_noetherian()
True
sage: x = polygen(ZZ, 'x')
sage: K = NumberField(x^2 + 1, 's')
↪  # needs sage.rings.number_field
sage: OK = K.ring_of_integers()
↪  # needs sage.rings.number_field
sage: OK.is_noetherian()
↪  # needs sage.rings.number_field
True
sage: QQ.is_noetherian()
True
```

```
>>> from sage.all import *
>>> ZZ.is_noetherian()
True
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(1), 's')
↪                    # needs sage.rings.number_field
>>> OK = K.ring_of_integers()
↪# needs sage.rings.number_field
>>> OK.is_noetherian()
↪# needs sage.rings.number_field
True
>>> QQ.is_noetherian()
True
```

krull_dimension()

Return 1 since Dedekind domains have Krull dimension 1.

EXAMPLES:

The following are examples of Dedekind domains:

```
sage: ZZ.krull_dimension()
1
sage: x = polygen(ZZ, 'x')
sage: K = NumberField(x^2 + 1, 's')
↪  # needs sage.rings.number_field
sage: OK = K.ring_of_integers()
↪  # needs sage.rings.number_field
sage: OK.krull_dimension()
↪  # needs sage.rings.number_field
1
```

```
>>> from sage.all import *
>>> ZZ.krull_dimension()
1
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(1), 's')
↪                  # needs sage.rings.number_field
>>> OK = K.ring_of_integers()
↪# needs sage.rings.number_field
>>> OK.krull_dimension()
↪# needs sage.rings.number_field
1
```

super_categories()

EXAMPLES:

```
sage: DedekindDomains().super_categories()
[Category of integral domains]
```

```
>>> from sage.all import *
>>> DedekindDomains().super_categories()
[Category of integral domains]
```

4.36 Discrete Valuation Rings (DVR) and Fields (DVF)

class sage.categories.discrete_valuation.**DiscreteValuationFields**

Bases: *Category_singleton*

The category of discrete valuation fields.

EXAMPLES:

```
sage: Qp(7) in DiscreteValuationFields()
↪needs sage.rings.padics
True
sage: TestSuite(DiscreteValuationFields()).run()
```

```
>>> from sage.all import *
>>> Qp(Integer(7)) in DiscreteValuationFields()
↪     # needs sage.rings.padics
```

(continues on next page)

(continued from previous page)

```
True
>>> TestSuite(DiscreteValuationFields()).run()
```

class ElementMethods

Bases: object

valuation()

Return the valuation of this element.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: x = Qp(5)(50)
sage: x.valuation()
2
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> x = Qp(Integer(5))(Integer(50))
>>> x.valuation()
2
```

class ParentMethods

Bases: object

residue_field()

Return the residue field of the ring of integers of this discrete valuation field.

EXAMPLES:

```
sage: Qp(5).residue_field()                                                 #␣
↪needs sage.rings.padics
Finite Field of size 5

sage: K.<u> = LaurentSeriesRing(QQ)
sage: K.residue_field()
Rational Field
```

```
>>> from sage.all import *
>>> Qp(Integer(5)).residue_field()                                          ␣
↪     # needs sage.rings.padics
Finite Field of size 5

>>> K = LaurentSeriesRing(QQ, names=('u',)); (u,) = K._first_ngens(1)
>>> K.residue_field()
Rational Field
```

uniformizer()

Return a uniformizer of this ring.

EXAMPLES:

```
sage: Qp(5).uniformizer()                                                    #␣
↪needs sage.rings.padics
5 + O(5^21)
```

```
>>> from sage.all import *
>>> Qp(Integer(5)).uniformizer()                                             ␣
↪     # needs sage.rings.padics
5 + O(5^21)
```

super_categories()

EXAMPLES:

```
sage: DiscreteValuationFields().super_categories()
[Category of fields]
```

```
>>> from sage.all import *
>>> DiscreteValuationFields().super_categories()
[Category of fields]
```

class sage.categories.discrete_valuation.**DiscreteValuationRings**

Bases: *Category_singleton*

The category of discrete valuation rings.

EXAMPLES:

```
sage: GF(7)[['x']] in DiscreteValuationRings()
True
sage: TestSuite(DiscreteValuationRings()).run()
```

```
>>> from sage.all import *
>>> GF(Integer(7))[['x']] in DiscreteValuationRings()
True
>>> TestSuite(DiscreteValuationRings()).run()
```

class ElementMethods

Bases: object

euclidean_degree()

Return the Euclidean degree of this element.

gcd(*other*)

Return the greatest common divisor of `self` and `other`, normalized so that it is a power of the distinguished uniformizer.

is_unit()

Return `True` if `self` is invertible.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: x = Zp(5)(50)
sage: x.is_unit()
False
```

(continues on next page)

(continued from previous page)

```
sage: # needs sage.rings.padics
sage: x = Zp(7)(50)
sage: x.is_unit()
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> x = Zp(Integer(5))(Integer(50))
>>> x.is_unit()
False

>>> # needs sage.rings.padics
>>> x = Zp(Integer(7))(Integer(50))
>>> x.is_unit()
True
```

lcm(*other*)

Return the least common multiple of `self` and `other`, normalized so that it is a power of the distinguished uniformizer.

quo_rem(*other*)

Return the quotient and remainder for Euclidean division of `self` by `other`.

EXAMPLES:

```
sage: R.<q> = GF(5)[[]]
sage: (q^2 + q).quo_rem(q)
(1 + q, 0)
sage: (q + 1).quo_rem(q^2)
(0, 1 + q)
```

```
>>> from sage.all import *
>>> R = GF(Integer(5))[['q']]; (q,) = R._first_ngens(1)
>>> (q**Integer(2) + q).quo_rem(q)
(1 + q, 0)
>>> (q + Integer(1)).quo_rem(q**Integer(2))
(0, 1 + q)
```

valuation()

Return the valuation of this element.

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: x = Zp(5)(50)
sage: x.valuation()
2
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> x = Zp(Integer(5))(Integer(50))
```

(continues on next page)

(continued from previous page)

```
>>> x.valuation()
2
```

class ParentMethods

Bases: object

residue_field()

Return the residue field of this ring.

EXAMPLES:

```
sage: Zp(5).residue_field()                                                 #␣
↪needs sage.rings.padics
Finite Field of size 5

sage: K.<u> = QQ[[]]
sage: K.residue_field()
Rational Field
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).residue_field()                                          ␣
↪     # needs sage.rings.padics
Finite Field of size 5

>>> K = QQ[['u']]; (u,) = K._first_ngens(1)
>>> K.residue_field()
Rational Field
```

uniformizer()

Return a uniformizer of this ring.

EXAMPLES:

```
sage: Zp(5).uniformizer()                                                   #␣
↪needs sage.rings.padics
5 + O(5^21)

sage: K.<u> = QQ[[]]
sage: K.uniformizer()
u
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).uniformizer()                                            ␣
↪     # needs sage.rings.padics
5 + O(5^21)

>>> K = QQ[['u']]; (u,) = K._first_ngens(1)
>>> K.uniformizer()
u
```

super_categories()

EXAMPLES:

```
sage: DiscreteValuationRings().super_categories()
[Category of euclidean domains]
```

```
>>> from sage.all import *
>>> DiscreteValuationRings().super_categories()
[Category of euclidean domains]
```

4.37 Distributive Magmas and Additive Magmas

class sage.categories.distributive_magmas_and_additive_magmas.**DistributiveMagmasAndAdditiveMagmas**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of sets $(S, +, *)$ with $*$ distributing on $+$.

This is similar to a ring, but $+$ and $*$ are only required to be (additive) magmas.

EXAMPLES:

```
sage: from sage.categories.distributive_magmas_and_additive_magmas import␣
↪DistributiveMagmasAndAdditiveMagmas
sage: C = DistributiveMagmasAndAdditiveMagmas(); C
Category of distributive magmas and additive magmas
sage: C.super_categories()
[Category of magmas and additive magmas]
```

```
>>> from sage.all import *
>>> from sage.categories.distributive_magmas_and_additive_magmas import␣
↪DistributiveMagmasAndAdditiveMagmas
>>> C = DistributiveMagmasAndAdditiveMagmas(); C
Category of distributive magmas and additive magmas
>>> C.super_categories()
[Category of magmas and additive magmas]
```

class AdditiveAssociative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class AdditiveCommutative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class AdditiveUnital(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class Associative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

AdditiveInverse

alias of *Rngs*

Unital

alias of *Semirings*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of magmas distributing over additive magmas is a magma distributing over an additive magma.

EXAMPLES:

```
sage: C = (Magmas() & AdditiveMagmas()).Distributive().CartesianProducts()
sage: C.extra_super_categories()
[Category of distributive magmas and additive magmas]
sage: C.axioms()
frozenset({'Distributive'})
```

```
>>> from sage.all import *
>>> C = (Magmas() & AdditiveMagmas()).Distributive().CartesianProducts()
>>> C.extra_super_categories()
[Category of distributive magmas and additive magmas]
>>> C.axioms()
frozenset({'Distributive'})
```

class ParentMethods

Bases: object

4.38 Division rings

class sage.categories.division_rings.**DivisionRings**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of division rings.

A division ring (or skew field) is a not necessarily commutative ring where all nonzero elements have multiplicative inverses

EXAMPLES:

```
sage: DivisionRings()
Category of division rings
sage: DivisionRings().super_categories()
[Category of domains]
```

```
>>> from sage.all import *
>>> DivisionRings()
Category of division rings
>>> DivisionRings().super_categories()
[Category of domains]
```

Commutative

alias of *Fields*

class ElementMethods

Bases: object

Finite_extra_super_categories()

Return extraneous super categories for `DivisionRings().Finite()`.

EXAMPLES:

Any field is a division ring:

```
sage: Fields().is_subcategory(DivisionRings())
True
```

```
>>> from sage.all import *
>>> Fields().is_subcategory(DivisionRings())
True
```

This methods specifies that, by Weddeburn theorem, the reciprocal holds in the finite case: a finite division ring is commutative and thus a field:

```
sage: DivisionRings().Finite_extra_super_categories()
(Category of commutative magmas,)
sage: DivisionRings().Finite()
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> DivisionRings().Finite_extra_super_categories()
(Category of commutative magmas,)
>>> DivisionRings().Finite()
Category of finite enumerated fields
```

Warning

This is not implemented in `DivisionRings.Finite.extra_super_categories` because the categories of finite division rings and of finite fields coincide. See the section *Deduction rules* in the documentation of axioms.

class ParentMethods

Bases: `object`

extra_super_categories()

Return the *Domains* category.

This method specifies that a division ring has no zero divisors, i.e. is a domain.

See also

The *Deduction rules* section in the documentation of axioms

EXAMPLES:

```
sage: DivisionRings().extra_super_categories()
(Category of domains,)
sage: "NoZeroDivisors" in DivisionRings().axioms()
True
```

```
>>> from sage.all import *
>>> DivisionRings().extra_super_categories()
(Category of domains,)
>>> "NoZeroDivisors" in DivisionRings().axioms()
True
```

4.39 Domains

class sage.categories.domains.**Domains**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of domains.

A domain (or non-commutative integral domain), is a ring, not necessarily commutative, with no nonzero zero divisors.

EXAMPLES:

```
sage: C = Domains(); C
Category of domains
sage: C.super_categories()
[Category of rings]
sage: C is Rings().NoZeroDivisors()
True
```

```
>>> from sage.all import *
>>> C = Domains(); C
Category of domains
>>> C.super_categories()
[Category of rings]
>>> C is Rings().NoZeroDivisors()
True
```

Commutative

alias of *IntegralDomains*

class ElementMethods

Bases: object

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: Domains().super_categories()
[Category of rings]
```

```
>>> from sage.all import *
>>> Domains().super_categories()
[Category of rings]
```

4.40 Drinfeld modules over a base

This module provides the class *sage.categories.drinfeld_modules.DrinfeldModules*.

AUTHORS:

- Antoine Leudière (2022-04)
- Xavier Caruso (2022-06)

class sage.categories.drinfeld_modules.**DrinfeldModules**(*base_morphism*, *name='τ'*)

Bases: *Category_over_base_ring*

This class implements the category of Drinfeld $\mathbf{F}_q[T]$-modules on a given base field.

Let $\mathbf{F}_q[T]$ be a polynomial ring with coefficients in a finite field $\mathbf{F}_q$ and let K be a field. Fix a ring morphism $\gamma : \mathbf{F}_q[T] \to K$; we say that K is an $\mathbf{F}_q[T]$-field. Let $K\{\tau\}$ be the ring of Ore polynomials with coefficients in K, whose multiplication is given by the rule $\tau\lambda = \lambda^q\tau$ for any $\lambda \in K$.

The extension $K/\mathbf{F}_q[T]$ (represented as an instance of the class sage.rings.ring_extension.RingExtension_generic) is the *base field* of the category; its defining morphism γ is called the *base morphism*.

The monic polynomial that generates the kernel of γ is called the $\mathbf{F}_q[T]$-*characteristic*, or *function-field characteristic*, of the base field. We say that $\mathbf{F}_q[T]$ is the *function ring* of the category; $K\{\tau\}$ is the *Ore polynomial ring*. The constant coefficient of the category is the image of T under the base morphism.

Construction

Generally, Drinfeld modules objects are created before their category, and the category is retrieved as an attribute of the Drinfeld module:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C
Category of Drinfeld modules over Finite Field in z of size 11^4
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z + Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C
Category of Drinfeld modules over Finite Field in z of size 11^4
```

The output tells the user that the category is only defined by its base.

Properties of the category

The base field is retrieved using the method *base()*:

```
sage: C.base()
Finite Field in z of size 11^4
```

```
>>> from sage.all import *
>>> C.base()
Finite Field in z of size 11^4
```

Equivalently, one can use *base_morphism()* to retrieve the base morphism:

```
sage: C.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field of size 11
  To:   Finite Field in z of size 11^4
  Defn: T |--> z^3 + 7*z^2 + 6*z + 10
```

```
>>> from sage.all import *
>>> C.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field of size 11
  To:   Finite Field in z of size 11^4
  Defn: T |--> z^3 + 7*z^2 + 6*z + 10
```

The so-called constant coefficient — which is the same for all Drinfeld modules in the category — is simply the image of T by the base morphism:

```
sage: C.constant_coefficient()
z^3 + 7*z^2 + 6*z + 10
sage: C.base_morphism()(T) == C.constant_coefficient()
True
```

```
>>> from sage.all import *
>>> C.constant_coefficient()
z^3 + 7*z^2 + 6*z + 10
>>> C.base_morphism()(T) == C.constant_coefficient()
True
```

Similarly, the function ring-characteristic of the category is either 0 or the unique monic polynomial in $\mathbf{F}_q[T]$ that generates the kernel of the base:

```
sage: C.characteristic()
T^2 + 7*T + 2
sage: C.base_morphism()(C.characteristic())
0
```

```
>>> from sage.all import *
>>> C.characteristic()
T^2 + 7*T + 2
>>> C.base_morphism()(C.characteristic())
0
```

The base field, base morphism, function ring and Ore polynomial ring are the same for the category and its objects:

```
sage: C.base() is phi.base()
True
sage: C.base_morphism() is phi.base_morphism()
True
```

(continues on next page)

(continued from previous page)

```
sage: C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 11
sage: C.function_ring() is phi.function_ring()
True

sage: C.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z of size 11^4 twisted by z |--> z^
↪11
sage: C.ore_polring() is phi.ore_polring()
True
```

```
>>> from sage.all import *
>>> C.base() is phi.base()
True
>>> C.base_morphism() is phi.base_morphism()
True

>>> C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 11
>>> C.function_ring() is phi.function_ring()
True

>>> C.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z of size 11^4 twisted by z |--> z^
↪11
>>> C.ore_polring() is phi.ore_polring()
True
```

Creating Drinfeld module objects from the category

Calling *object()* with an Ore polynomial creates a Drinfeld module object in the category whose generator is the input:

```
sage: psi = C.object([p_root, 1])
sage: psi
Drinfeld module defined by T |--> τ + z^3 + 7*z^2 + 6*z + 10
sage: psi.category() is C
True
```

```
>>> from sage.all import *
>>> psi = C.object([p_root, Integer(1)])
>>> psi
Drinfeld module defined by T |--> τ + z^3 + 7*z^2 + 6*z + 10
>>> psi.category() is C
True
```

Of course, the constant coefficient of the input must be the same as the category:

```
sage: C.object([z, 1])
Traceback (most recent call last):
```

(continues on next page)

(continued from previous page)

```
...
ValueError: constant coefficient must equal that of the category
```

```
>>> from sage.all import *
>>> C.object([z, Integer(1)])
Traceback (most recent call last):
...
ValueError: constant coefficient must equal that of the category
```

It is also possible to create a random object in the category. The input is the desired rank:

```
sage: rho = C.random_object(2)
sage: rho  # random
Drinfeld module defined by T |--> (7*z^3 + 7*z^2 + 10*z + 2)*t^2 + (9*z^3 + 5*z^2␣
↪+ 2*z + 7)*t + z^3 + 7*z^2 + 6*z + 10
sage: rho.rank() == 2
True
sage: rho.category() is C
True
```

```
>>> from sage.all import *
>>> rho = C.random_object(Integer(2))
>>> rho  # random
Drinfeld module defined by T |--> (7*z^3 + 7*z^2 + 10*z + 2)*t^2 + (9*z^3 + 5*z^2␣
↪+ 2*z + 7)*t + z^3 + 7*z^2 + 6*z + 10
>>> rho.rank() == Integer(2)
True
>>> rho.category() is C
True
```

A_field()

Return the underlying A-field of this category, viewed as an algebra over the function ring A.

This is an instance of the class `sage.rings.ring_extension.RingExtension_generic`.

Note

This method has the same behavior as *base()*.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: C = phi.category()
sage: C.A_field()
Finite Field in z of size 5^12 over its base
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
```

(continues on next page)

(continued from previous page)

```
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> C = phi.category()
>>> C.A_field()
Finite Field in z of size 5^12 over its base
```

Endsets()

Return the category of endsets.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()

sage: from sage.categories.homsets import Homsets
sage: C.Endsets() is Homsets().Endsets()
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()

>>> from sage.categories.homsets import Homsets
>>> C.Endsets() is Homsets().Endsets()
True
```

Homsets()

Return the category of homsets.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()

sage: from sage.categories.homsets import Homsets
sage: C.Homsets() is Homsets()
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()

>>> from sage.categories.homsets import Homsets
>>> C.Homsets() is Homsets()
True
```

class ParentMethods

Bases: object

A_field()

Return the underlying A-field of this Drinfeld module, viewed as an algebra over the function ring A.

This is an instance of the class sage.rings.ring_extension.RingExtension_generic.

Note

This method has the same behavior as *base()*.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(6)
sage: phi = DrinfeldModule(A, [z, z^3, z^5])
sage: phi.A_field()
Finite Field in z of size 5^12 over its base
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z',)); (z,) = K._first_ngens(1)
>>> phi = DrinfeldModule(A, [z, z**Integer(3), z**Integer(5)])
>>> phi.A_field()
Finite Field in z of size 5^12 over its base
```

base()

Return the field over which this Drinfeld module is defined.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
```

(continues on next page)

(continued from previous page)

```
sage: phi.base()
Finite Field in z12 of size 5^12
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.base()
Finite Field in z12 of size 5^12
```

The base can be infinite:

```
sage: sigma = DrinfeldModule(A, [T, 1])
sage: sigma.base()
Fraction Field of Univariate Polynomial Ring in T over Finite Field in z2␣
↪of size 5^2
```

```
>>> from sage.all import *
>>> sigma = DrinfeldModule(A, [T, Integer(1)])
>>> sigma.base()
Fraction Field of Univariate Polynomial Ring in T over Finite Field in z2␣
↪of size 5^2
```

base_morphism()

Return the base morphism of this Drinfeld module.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: phi.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field in z2 of size 5^
↪2
  To:   Finite Field in z12 of size 5^12
  Defn: T |--> 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
```

(continues on next page)

(continued from previous page)

```
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field in z2 of size 5^
↪2
  To:   Finite Field in z12 of size 5^12
  Defn: T |--> 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
```

The base field can be infinite:

```
sage: sigma = DrinfeldModule(A, [T, 1])
sage: sigma.base_morphism()
Coercion map:
  From: Univariate Polynomial Ring in T over Finite Field in z2 of size 5^
↪2
  To:   Fraction Field of Univariate Polynomial Ring in T over Finite␣
↪Field in z2 of size 5^2
```

```
>>> from sage.all import *
>>> sigma = DrinfeldModule(A, [T, Integer(1)])
>>> sigma.base_morphism()
Coercion map:
  From: Univariate Polynomial Ring in T over Finite Field in z2 of size 5^
↪2
  To:   Fraction Field of Univariate Polynomial Ring in T over Finite␣
↪Field in z2 of size 5^2
```

base_over_constants_field()

Return the base field, seen as an extension over the constants field $\mathbf{F}_q$.

This is an instance of the class `sage.rings.ring_extension.RingExtension_generic`.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: phi.base_over_constants_field()
Field in z12 with defining polynomial x^6 + (4*z2 + 3)*x^5 + x^4 + (3*z2␣
↪+ 1)*x^3 + x^2 + (4*z2 + 1)*x + z2 over its base
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.base_over_constants_field()
Field in z12 with defining polynomial x^6 + (4*z2 + 3)*x^5 + x^4 + (3*z2␣
↪+ 1)*x^3 + x^2 + (4*z2 + 1)*x + z2 over its base
```

base_ring()

alias of *base()*.

characteristic()

Return the function ring-characteristic.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: phi.characteristic()
T^2 + (4*z2 + 2)*T + 2
sage: phi.base_morphism()(phi.characteristic())
0
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.characteristic()
T^2 + (4*z2 + 2)*T + 2
>>> phi.base_morphism()(phi.characteristic())
0
```

```
sage: B.<Y> = Fq[]
sage: psi = DrinfeldModule(A, [B(1), 0, 0, 1])
sage: psi.characteristic()
Traceback (most recent call last):
...
NotImplementedError: function ring characteristic not implemented in this␣
↪case
```

```
>>> from sage.all import *
>>> B = Fq['Y']; (Y,) = B._first_ngens(1)
>>> psi = DrinfeldModule(A, [B(Integer(1)), Integer(0), Integer(0),␣
↪Integer(1)])
>>> psi.characteristic()
Traceback (most recent call last):
...
NotImplementedError: function ring characteristic not implemented in this␣
↪case
```

constant_coefficient()

Return the constant coefficient of the generator of this Drinfeld module.

OUTPUT: an element in the base field

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: phi.constant_coefficient() == p_root
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.constant_coefficient() == p_root
True
```

Let $\mathbf{F}_q[T]$ be the function ring, and let γ be the base of the Drinfeld module. The constant coefficient is $\gamma(T)$:

```
sage: C = phi.category()
sage: F = C.A_field()
sage: F(T) == phi.constant_coefficient()
True
```

```
>>> from sage.all import *
>>> C = phi.category()
>>> F = C.A_field()
>>> F(T) == phi.constant_coefficient()
True
```

Naturally, two Drinfeld modules in the same category have the same constant coefficient:

```
sage: t = phi.ore_polring().gen()
sage: psi = C.object(phi.constant_coefficient() + t^3)
sage: psi
Drinfeld module defined by T |--> τ^3 + 2*z12^11 + 2*z12^10 + z12^9 +␣
↪3*z12^8 + z12^7 + 2*z12^5 + 2*z12^4 + 3*z12^3 + z12^2 + 2*z12
```

```
>>> from sage.all import *
>>> t = phi.ore_polring().gen()
>>> psi = C.object(phi.constant_coefficient() + t**Integer(3))
>>> psi
Drinfeld module defined by T |--> τ^3 + 2*z12^11 + 2*z12^10 + z12^9 +␣
↪3*z12^8 + z12^7 + 2*z12^5 + 2*z12^4 + 3*z12^3 + z12^2 + 2*z12
```

Reciprocally, it is impossible to create two Drinfeld modules in this category if they do not share the same constant coefficient:

```
sage: rho = C.object(phi.constant_coefficient() + 1 + t^3)
Traceback (most recent call last):
...
ValueError: constant coefficient must equal that of the category
```

```
>>> from sage.all import *
>>> rho = C.object(phi.constant_coefficient() + Integer(1) +␣
↪t**Integer(3))
Traceback (most recent call last):
...
ValueError: constant coefficient must equal that of the category
```

function_ring()

Return the function ring of this Drinfeld module.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: phi.function_ring()
Univariate Polynomial Ring in T over Finite Field in z2 of size 5^2
sage: phi.function_ring() is A
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> phi.function_ring()
Univariate Polynomial Ring in T over Finite Field in z2 of size 5^2
>>> phi.function_ring() is A
True
```

ore_polring()

Return the Ore polynomial ring of this Drinfeld module.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])
sage: S = phi.ore_polring()
sage: S
Ore Polynomial Ring in τ over Finite Field in z12 of size 5^12 twisted by␣
↪z12 |--> z12^(5^2)
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])
>>> S = phi.ore_polring()
>>> S
Ore Polynomial Ring in τ over Finite Field in z12 of size 5^12 twisted by␣
↪z12 |--> z12^(5^2)
```

The Ore polynomial ring can also be retrieved from the category of the Drinfeld module:

```
sage: S is phi.category().ore_polring()
True
```

```
>>> from sage.all import *
>>> S is phi.category().ore_polring()
True
```

The generator of the Drinfeld module is in the Ore polynomial ring:

```
sage: phi(T) in S
True
```

```
>>> from sage.all import *
>>> phi(T) in S
True
```

ore_variable()

Return the variable of the Ore polynomial ring of this Drinfeld module.

EXAMPLES:

```
sage: Fq = GF(25)
sage: A.<T> = Fq[]
sage: K.<z12> = Fq.extension(6)
sage: p_root = 2*z12^11 + 2*z12^10 + z12^9 + 3*z12^8 + z12^7 + 2*z12^5 +␣
↪2*z12^4 + 3*z12^3 + z12^2 + 2*z12
sage: phi = DrinfeldModule(A, [p_root, z12^3, z12^5])

sage: phi.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z12 of size 5^12 twisted by␣
↪z12 |--> z12^(5^2)
sage: phi.ore_variable()
τ
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(25))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(6), names=('z12',)); (z12,) = K._first_
↪ngens(1)
>>> p_root = Integer(2)*z12**Integer(11) + Integer(2)*z12**Integer(10) +␣
↪z12**Integer(9) + Integer(3)*z12**Integer(8) + z12**Integer(7) +␣
↪Integer(2)*z12**Integer(5) + Integer(2)*z12**Integer(4) +␣
↪Integer(3)*z12**Integer(3) + z12**Integer(2) + Integer(2)*z12
>>> phi = DrinfeldModule(A, [p_root, z12**Integer(3), z12**Integer(5)])

>>> phi.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z12 of size 5^12 twisted by␣
↪z12 |--> z12^(5^2)
>>> phi.ore_variable()
τ
```

base_morphism()

Return the base morphism of the category.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field of size 11
```

(continues on next page)

(continued from previous page)

```
  To:   Finite Field in z of size 11^4
  Defn: T |--> z^3 + 7*z^2 + 6*z + 10

sage: C.constant_coefficient() == C.base_morphism()(T)
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.base_morphism()
Ring morphism:
  From: Univariate Polynomial Ring in T over Finite Field of size 11
  To:   Finite Field in z of size 11^4
  Defn: T |--> z^3 + 7*z^2 + 6*z + 10

>>> C.constant_coefficient() == C.base_morphism()(T)
True
```

base_over_constants_field()

Return the base field, seen as an extension over the constants field $\mathbf{F}_q$.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.base_over_constants_field()
Field in z with defining polynomial x^4 + 8*x^2 + 10*x + 2 over its base
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.base_over_constants_field()
Field in z with defining polynomial x^4 + 8*x^2 + 10*x + 2 over its base
```

characteristic()

Return the function ring-characteristic.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.characteristic()
T^2 + 7*T + 2
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.characteristic()
T^2 + 7*T + 2
```

```
sage: psi = DrinfeldModule(A, [T, 1])
sage: C = psi.category()
sage: C.characteristic()
0
```

```
>>> from sage.all import *
>>> psi = DrinfeldModule(A, [T, Integer(1)])
>>> C = psi.category()
>>> C.characteristic()
0
```

constant_coefficient()

Return the constant coefficient of the category.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.constant_coefficient()
z^3 + 7*z^2 + 6*z + 10
sage: C.constant_coefficient() == C.A_field()(T)
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.constant_coefficient()
z^3 + 7*z^2 + 6*z + 10
>>> C.constant_coefficient() == C.A_field()(T)
True
```

function_ring()

Return the function ring of the category.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 11
sage: C.function_ring() is A
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.function_ring()
Univariate Polynomial Ring in T over Finite Field of size 11
>>> C.function_ring() is A
True
```

object(*gen*)

Return a Drinfeld module object in the category whose generator is the input.

INPUT:

- `gen` – the generator of the Drinfeld module, given as an Ore polynomial or a list of coefficients

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: psi = DrinfeldModule(A, [p_root, 1])
sage: C = psi.category()

sage: phi = C.object([p_root, 0, 1])
```

(continues on next page)

(continued from previous page)

```
sage: phi
Drinfeld module defined by T |--> τ^2 + z^3 + 7*z^2 + 6*z + 10
sage: t = phi.ore_polring().gen()
sage: C.object(t^2 + z^3 + 7*z^2 + 6*z + 10) is phi
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> psi = DrinfeldModule(A, [p_root, Integer(1)])
>>> C = psi.category()

>>> phi = C.object([p_root, Integer(0), Integer(1)])
>>> phi
Drinfeld module defined by T |--> τ^2 + z^3 + 7*z^2 + 6*z + 10
>>> t = phi.ore_polring().gen()
>>> C.object(t**Integer(2) + z**Integer(3) + Integer(7)*z**Integer(2) +␣
↪Integer(6)*z + Integer(10)) is phi
True
```

ore_polring()

Return the Ore polynomial ring of the category.

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z of size 11^4 twisted by z |-->
↪ z^11
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
>>> C.ore_polring()
Ore Polynomial Ring in τ over Finite Field in z of size 11^4 twisted by z |-->
↪ z^11
```

random_object(*rank*)

Return a random Drinfeld module in the category with given rank.

INPUT:

- `rank` – integer; the rank of the Drinfeld module

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()

sage: psi = C.random_object(3)  # random
Drinfeld module defined by T |--> (6*z^3 + 4*z^2 + 10*z + 9)*t^3 + (4*z^3 +␣
↪8*z^2 + 8*z)*t^2 + (10*z^3 + 3*z^2 + 6*z)*t + z^3 + 7*z^2 + 6*z + 10
sage: psi.rank() == 3
True
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()

>>> psi = C.random_object(Integer(3))  # random
Drinfeld module defined by T |--> (6*z^3 + 4*z^2 + 10*z + 9)*t^3 + (4*z^3 +␣
↪8*z^2 + 8*z)*t^2 + (10*z^3 + 3*z^2 + 6*z)*t + z^3 + 7*z^2 + 6*z + 10
>>> psi.rank() == Integer(3)
True
```

super_categories()

EXAMPLES:

```
sage: Fq = GF(11)
sage: A.<T> = Fq[]
sage: K.<z> = Fq.extension(4)
sage: p_root = z^3 + 7*z^2 + 6*z + 10
sage: phi = DrinfeldModule(A, [p_root, 0, 0, 1])
sage: C = phi.category()
sage: C.super_categories()
[Category of objects]
```

```
>>> from sage.all import *
>>> Fq = GF(Integer(11))
>>> A = Fq['T']; (T,) = A._first_ngens(1)
>>> K = Fq.extension(Integer(4), names=('z',)); (z,) = K._first_ngens(1)
>>> p_root = z**Integer(3) + Integer(7)*z**Integer(2) + Integer(6)*z +␣
↪Integer(10)
>>> phi = DrinfeldModule(A, [p_root, Integer(0), Integer(0), Integer(1)])
>>> C = phi.category()
```

(continues on next page)

(continued from previous page)

```
>>> C.super_categories()
[Category of objects]
```

4.41 Enumerated sets

EnumeratedSets.ParentMethods.**__iter__**()

An iterator for the enumerated set.

`iter(self)` allows the combinatorial class to be treated as an iterable. This is the default implementation from the category `EnumeratedSets()`; it just goes through the iterator of the set to count the number of objects.

By decreasing order of priority, the second column of the following array shows which method is used to define `__iter__`, when the methods of the first column are overloaded:

Needed methods	Default `__iterator` provided
`first` and `next`	`_iterator_from_next`
`unrank`	`_iterator_from_unrank`
`list`	`_iterator_from_next`

It is also possible to override `__iter__` method itself. Then the methods of the first column are defined using `__iter__`

If none of these are provided, this raises a `NotImplementedError`.

EXAMPLES:

We start with an example where nothing is implemented:

```
sage: class broken(UniqueRepresentation, Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category = EnumeratedSets())
sage: it = iter(broken()); [next(it), next(it), next(it)]
Traceback (most recent call last):
...
NotImplementedError: iterator called but not implemented
```

```
>>> from sage.all import *
>>> class broken(UniqueRepresentation, Parent):
...     def __init__(self):
...         Parent.__init__(self, category = EnumeratedSets())
>>> it = iter(broken()); [next(it), next(it), next(it)]
Traceback (most recent call last):
...
NotImplementedError: iterator called but not implemented
```

Here is what happens when `first` and `next` are implemented:

```
sage: class set_first_next(UniqueRepresentation, Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category = EnumeratedSets())
....:     def first(self):
```

(continues on next page)

(continued from previous page)

```
....:             return 0
....:         def next(self, elt):
....:             return elt+1
sage: it = iter(set_first_next()); [next(it), next(it), next(it)]
[0, 1, 2]
```

```
>>> from sage.all import *
>>> class set_first_next(UniqueRepresentation, Parent):
...       def __init__(self):
...           Parent.__init__(self, category = EnumeratedSets())
...       def first(self):
...           return Integer(0)
...       def next(self, elt):
...           return elt+Integer(1)
>>> it = iter(set_first_next()); [next(it), next(it), next(it)]
[0, 1, 2]
```

Let us try with `unrank`:

```
sage: class set_unrank(UniqueRepresentation, Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category = EnumeratedSets())
....:     def unrank(self, i):
....:         return i + 5
sage: it = iter(set_unrank()); [next(it), next(it), next(it)]
[5, 6, 7]
```

```
>>> from sage.all import *
>>> class set_unrank(UniqueRepresentation, Parent):
...       def __init__(self):
...           Parent.__init__(self, category = EnumeratedSets())
...       def unrank(self, i):
...           return i + Integer(5)
>>> it = iter(set_unrank()); [next(it), next(it), next(it)]
[5, 6, 7]
```

Let us finally try with `list`:

```
sage: class set_list(UniqueRepresentation, Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category = EnumeratedSets())
....:     def list(self):
....:         return [5, 6, 7]
sage: it = iter(set_list()); [next(it), next(it), next(it)]
[5, 6, 7]
```

```
>>> from sage.all import *
>>> class set_list(UniqueRepresentation, Parent):
...       def __init__(self):
...           Parent.__init__(self, category = EnumeratedSets())
...       def list(self):
...           return [Integer(5), Integer(6), Integer(7)]
```

(continues on next page)

(continued from previous page)

```
>>> it = iter(set_list()); [next(it), next(it), next(it)]
[5, 6, 7]
```

class sage.categories.enumerated_sets.**EnumeratedSets**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of enumerated sets.

An *enumerated set* is a *finite* or *countable* set or multiset S together with a canonical enumeration of its elements; conceptually, this is very similar to an immutable list. The main difference lies in the names and the return type of the methods, and of course the fact that the list of elements is not supposed to be expanded in memory. Whenever possible one should use one of the two sub-categories *FiniteEnumeratedSets* or *InfiniteEnumeratedSets*.

The purpose of this category is threefold:

- to fix a common interface for all these sets;
- to provide a bunch of default implementations;
- to provide consistency tests.

The standard methods for an enumerated set `S` are:

- `S.cardinality()` – the number of elements of the set. This is the equivalent for `len` on a list except that the return value is specified to be a Sage `Integer` or `infinity`, instead of a Python `int`.
- `iter(S)` – an iterator for the elements of the set;
- `S.list()` – a fresh list of the elements of the set, when possible; raises a `NotImplementedError` if the list is predictably too large to be expanded in memory.
- `S.tuple()` – a tuple of the elements of the set, when possible; raises a `NotImplementedError` if the tuple is predictably too large to be expanded in memory.
- `S.unrank(n)` – the `n`-th element of the set when `n` is a sage `Integer`. This is the equivalent for `l[n]` on a list.
- `S.rank(e)` – the position of the element `e` in the set; This is equivalent to `l.index(e)` for a list except that the return value is specified to be a Sage `Integer`, instead of a Python `int`.
- `S.first()` – the first object of the set; it is equivalent to `S.unrank(0)`.
- `S.next(e)` – the object of the set which follows `e`; it is equivalent to `S.unrank(S.rank(e) + 1)`.
- `S.random_element()` – a random generator for an element of the set. Unless otherwise stated, and for finite enumerated sets, the probability is uniform.

For examples and tests see:

- `FiniteEnumeratedSets().example()`
- `InfiniteEnumeratedSets().example()`

EXAMPLES:

```
sage: EnumeratedSets()
Category of enumerated sets
sage: EnumeratedSets().super_categories()
[Category of sets]
sage: EnumeratedSets().all_super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of enumerated sets, Category of sets,
 Category of sets with partial maps, Category of objects]
```

```
>>> from sage.all import *
>>> EnumeratedSets()
Category of enumerated sets
>>> EnumeratedSets().super_categories()
[Category of sets]
>>> EnumeratedSets().all_super_categories()
[Category of enumerated sets, Category of sets,
 Category of sets with partial maps, Category of objects]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ParentMethods

Bases: object

first()

Return the first element.

EXAMPLES:

```
sage: cartesian_product([ZZ]*10).first()
(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

```
>>> from sage.all import *
>>> cartesian_product([ZZ]*Integer(10)).first()
(0, 0, 0, 0, 0, 0, 0, 0, 0, 0)
```

class ElementMethods

Bases: object

rank()

Return the rank of `self` in its parent.

See also *EnumeratedSets.ElementMethods.rank()*

EXAMPLES:

```
sage: F = FiniteSemigroups().example(('a','b','c'))
sage: L = list(F)
sage: L[7].rank()
7
sage: all(x.rank() == i for i,x in enumerate(L))
True
```

```
>>> from sage.all import *
>>> F = FiniteSemigroups().example(('a','b','c'))
>>> L = list(F)
>>> L[Integer(7)].rank()
7
>>> all(x.rank() == i for i,x in enumerate(L))
True
```

Finite

alias of *FiniteEnumeratedSets*

Infinite

alias of *InfiniteEnumeratedSets*

class ParentMethods

Bases: object

first()

The "first" element of self.

self.first() returns the first element of the set self. This is a generic implementation from the category EnumeratedSets() which can be used when the method __iter__ is provided.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.first() # indirect doctest
1
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C.first() # indirect doctest
1
```

is_empty()

Return whether this set is empty.

EXAMPLES:

```
sage: F = FiniteEnumeratedSet([1,2,3])
sage: F.is_empty()
False
sage: F = FiniteEnumeratedSet([])
sage: F.is_empty()
True
```

```
>>> from sage.all import *
>>> F = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> F.is_empty()
False
>>> F = FiniteEnumeratedSet([])
>>> F.is_empty()
True
```

iterator_range(*start=None*, *stop=None*, *step=None*)

Iterate over the range of elements of self starting at start, ending at stop, and stepping by step.

See also

unrank(), unrank_range()

EXAMPLES:

```
sage: # needs sage.combinat
sage: P = Partitions()
sage: list(P.iterator_range(stop=5))
[[], [1], [2], [1, 1], [3]]
sage: list(P.iterator_range(0, 5))
[[], [1], [2], [1, 1], [3]]
sage: list(P.iterator_range(3, 5))
[[1, 1], [3]]
sage: list(P.iterator_range(3, 10))
[[1, 1], [3], [2, 1], [1, 1, 1], [4], [3, 1], [2, 2]]
sage: list(P.iterator_range(3, 10, 2))
[[1, 1], [2, 1], [4], [2, 2]]
sage: it = P.iterator_range(3)
sage: [next(it) for x in range(10)]
[[1, 1],
 [3], [2, 1], [1, 1, 1],
 [4], [3, 1], [2, 2], [2, 1, 1], [1, 1, 1, 1],
 [5]]
sage: it = P.iterator_range(3, step=2)
sage: [next(it) for x in range(5)]
[[1, 1],
 [2, 1],
 [4], [2, 2], [1, 1, 1, 1]]
sage: next(P.iterator_range(stop=-3))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
sage: next(P.iterator_range(start=-3))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> P = Partitions()
>>> list(P.iterator_range(stop=Integer(5)))
[[], [1], [2], [1, 1], [3]]
>>> list(P.iterator_range(Integer(0), Integer(5)))
[[], [1], [2], [1, 1], [3]]
>>> list(P.iterator_range(Integer(3), Integer(5)))
[[1, 1], [3]]
>>> list(P.iterator_range(Integer(3), Integer(10)))
[[1, 1], [3], [2, 1], [1, 1, 1], [4], [3, 1], [2, 2]]
>>> list(P.iterator_range(Integer(3), Integer(10), Integer(2)))
[[1, 1], [2, 1], [4], [2, 2]]
>>> it = P.iterator_range(Integer(3))
>>> [next(it) for x in range(Integer(10))]
[[1, 1],
 [3], [2, 1], [1, 1, 1],
 [4], [3, 1], [2, 2], [2, 1, 1], [1, 1, 1, 1],
 [5]]
>>> it = P.iterator_range(Integer(3), step=Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> [next(it) for x in range(Integer(5))]
[[1, 1],
 [2, 1],
 [4], [2, 2], [1, 1, 1, 1]]
>>> next(P.iterator_range(stop=-Integer(3)))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
>>> next(P.iterator_range(start=-Integer(3)))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

list()

Return a list of the elements of `self`.

The elements of set `x` are created and cached on the first call of `x.list()`. Then each call of `x.list()` returns a new list from the cached result. Thus in looping, it may be better to do `for e in x:`, not `for e in x.list():`.

If `x` is not known to be finite, then an exception is raised.

EXAMPLES:

```
sage: (GF(3)^2).list()                                                  #␣
↪needs sage.modules
[(0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (2, 1), (0, 2), (1, 2), (2, 2)]
sage: R = Integers(11)
sage: R.list()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sage: l = R.list(); l
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sage: l.remove(0); l
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
sage: R.list()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

sage: C = FiniteEnumeratedSets().example()
sage: C.list()
[1, 2, 3]
```

```
>>> from sage.all import *
>>> (GF(Integer(3))**Integer(2)).list()                                 ␣
↪                # needs sage.modules
[(0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (2, 1), (0, 2), (1, 2), (2, 2)]
>>> R = Integers(Integer(11))
>>> R.list()
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> l = R.list(); l
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> l.remove(Integer(0)); l
[1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
>>> R.list()
```

(continues on next page)

(continued from previous page)

```
[0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10]

>>> C = FiniteEnumeratedSets().example()
>>> C.list()
[1, 2, 3]
```

map(*f*, *name*, *is_injective=None*)

Return the image $\{f(x)|x \in \text{self}\}$ of this enumerated set by f, as an enumerated set.

INPUT:

- `is_injective` – boolean (default: `True`); whether to assume that f is injective

EXAMPLES:

```
sage: R = Compositions(4).map(attrcall('partial_sums')); R
Image of Compositions of 4 by The map *.partial_sums()
 from Compositions of 4
sage: R.cardinality()
8
sage: R.list()
[[1, 2, 3, 4], [1, 2, 4], [1, 3, 4], [1, 4], [2, 3, 4], [2, 4], [3, 4],␣
↪[4]]
sage: [r for r in R]
[[1, 2, 3, 4], [1, 2, 4], [1, 3, 4], [1, 4], [2, 3, 4], [2, 4], [3, 4],␣
↪[4]]
sage: R.category()
Category of finite enumerated subobjects of sets
```

```
>>> from sage.all import *
>>> R = Compositions(Integer(4)).map(attrcall('partial_sums')); R
Image of Compositions of 4 by The map *.partial_sums()
 from Compositions of 4
>>> R.cardinality()
8
>>> R.list()
[[1, 2, 3, 4], [1, 2, 4], [1, 3, 4], [1, 4], [2, 3, 4], [2, 4], [3, 4],␣
↪[4]]
>>> [r for r in R]
[[1, 2, 3, 4], [1, 2, 4], [1, 3, 4], [1, 4], [2, 3, 4], [2, 4], [3, 4],␣
↪[4]]
>>> R.category()
Category of finite enumerated subobjects of sets
```

Warning

If the function is not injective, then there may be repeated elements:

```
sage: P = Compositions(4)
sage: P.list()
[[1, 1, 1, 1], [1, 1, 2], [1, 2, 1], [1, 3], [2, 1, 1], [2, 2], [3, 1],␣
↪[4]]
sage: P.map(attrcall('major_index')).list()
[6, 3, 4, 1, 5, 2, 3, 0]
```

```
>>> from sage.all import *
>>> P = Compositions(Integer(4))
>>> P.list()
[[1, 1, 1, 1], [1, 1, 2], [1, 2, 1], [1, 3], [2, 1, 1], [2, 2], [3, 1],␣
↪[4]]
>>> P.map(attrcall('major_index')).list()
[6, 3, 4, 1, 5, 2, 3, 0]
```

Pass `is_injective=False` to get a correct result in this case:

```
sage: P.map(attrcall('major_index'), is_injective=False).list()
[6, 3, 4, 1, 5, 2, 0]
```

```
>>> from sage.all import *
>>> P.map(attrcall('major_index'), is_injective=False).list()
[6, 3, 4, 1, 5, 2, 0]
```

next(*obj*)

The "next" element after `obj` in `self`.

`self.next(e)` returns the element of the set `self` which follows `e`. This is a generic implementation from the category `EnumeratedSets()` which can be used when the method `__iter__` is provided.

Remark: this is the default (brute force) implementation of the category `EnumeratedSets()`. Its complexity is $O(r)$, where r is the rank of `obj`.

EXAMPLES:

```
sage: C = InfiniteEnumeratedSets().example()
sage: C._next_from_iterator(10) # indirect doctest
11
```

```
>>> from sage.all import *
>>> C = InfiniteEnumeratedSets().example()
>>> C._next_from_iterator(Integer(10)) # indirect doctest
11
```

TODO: specify the behavior when `obj` is not in `self`.

random_element()

Return a random element in `self`.

Unless otherwise stated, and for finite enumerated sets, the probability is uniform.

This is a generic implementation from the category `EnumeratedSets()`. It raises a `NotImplementedError` since one does not know whether the set is finite.

EXAMPLES:

```
sage: class broken(UniqueRepresentation, Parent):
....:  def __init__(self):
....:      Parent.__init__(self, category = EnumeratedSets())
sage: broken().random_element()
Traceback (most recent call last):
...
NotImplementedError: unknown cardinality
```

```
>>> from sage.all import *
>>> class broken(UniqueRepresentation, Parent):
...  def __init__(self):
...      Parent.__init__(self, category = EnumeratedSets())
>>> broken().random_element()
Traceback (most recent call last):
...
NotImplementedError: unknown cardinality
```

rank(*x*)

The rank of an element of `self`.

`self.rank(x)` returns the rank of x, that is its position in the enumeration of `self`. This is an integer between `0` and `n-1` where `n` is the cardinality of `self`, or None if x is not in `self`.

This is the default (brute force) implementation from the category `EnumeratedSets()` which can be used when the method `__iter__` is provided. Its complexity is $O(r)$, where r is the rank of `obj`. For infinite enumerated sets, this won't terminate when x is not in `self`

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: list(C)
[1, 2, 3]
sage: C.rank(3) # indirect doctest
2
sage: C.rank(5) # indirect doctest
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> list(C)
[1, 2, 3]
>>> C.rank(Integer(3)) # indirect doctest
2
>>> C.rank(Integer(5)) # indirect doctest
```

some_elements()

Return some elements in `self`.

See `TestSuite` for a typical use case.

This is a generic implementation from the category `EnumeratedSets()` which can be used when the method `__iter__` is provided. It returns an iterator for up to the first 100 elements of `self`

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: list(C.some_elements()) # indirect doctest
[1, 2, 3]
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> list(C.some_elements()) # indirect doctest
[1, 2, 3]
```

tuple()

Return a tuple of the elements of `self`.

The tuple of elements of `x` is created and cached on the first call of `x.tuple()`. Each following call of `x.tuple()` returns the same tuple.

For looping, it may be better to do `for e in x:`, not `for e in x.tuple():`.

If `x` is not known to be finite, then an exception is raised.

EXAMPLES:

```
sage: (GF(3)^2).tuple()                                                    #␣
↪needs sage.modules
((0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (2, 1), (0, 2), (1, 2), (2, 2))
sage: R = Integers(11)
sage: l = R.tuple(); l
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
sage: l is R.tuple()
True
```

```
>>> from sage.all import *
>>> (GF(Integer(3))**Integer(2)).tuple()                                   ␣
↪               # needs sage.modules
((0, 0), (1, 0), (2, 0), (0, 1), (1, 1), (2, 1), (0, 2), (1, 2), (2, 2))
>>> R = Integers(Integer(11))
>>> l = R.tuple(); l
(0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10)
>>> l is R.tuple()
True
```

unrank(*r*)

The `r`-th element of `self`.

`self.unrank(r)` returns the `r`-th element of `self`, where `r` is an integer between `0` and `n-1` where `n` is the cardinality of `self`.

This is the default (brute force) implementation from the category `EnumeratedSets()` which can be used when the method `__iter__` is provided. Its complexity is $O(r)$, where r is the rank of `obj`.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.unrank(2) # indirect doctest
3
sage: C._unrank_from_iterator(5)
Traceback (most recent call last):
...
ValueError: the rank must be in the range from 0 to 2
sage: ZZ._unrank_from_iterator(-1)
Traceback (most recent call last):
...
ValueError: the rank must be greater than or equal to 0
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
```

(continues on next page)

(continued from previous page)

```
>>> C.unrank(Integer(2)) # indirect doctest
3
>>> C._unrank_from_iterator(Integer(5))
Traceback (most recent call last):
...
ValueError: the rank must be in the range from 0 to 2
>>> ZZ._unrank_from_iterator(-Integer(1))
Traceback (most recent call last):
...
ValueError: the rank must be greater than or equal to 0
```

unrank_range(*start=None*, *stop=None*, *step=None*)

Return the range of elements of `self` starting at `start`, ending at `stop`, and stepping by `step`.

See also

`unrank()`, `iterator_range()`

EXAMPLES:

```
sage: # needs sage.combinat
sage: P = Partitions()
sage: P.unrank_range(stop=5)
[[], [1], [2], [1, 1], [3]]
sage: P.unrank_range(0, 5)
[[], [1], [2], [1, 1], [3]]
sage: P.unrank_range(3, 5)
[[1, 1], [3]]
sage: P.unrank_range(3, 10)
[[1, 1], [3], [2, 1], [1, 1, 1], [4], [3, 1], [2, 2]]
sage: P.unrank_range(3, 10, 2)
[[1, 1], [2, 1], [4], [2, 2]]
sage: P.unrank_range(3)
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
sage: P.unrank_range(stop=-3)
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
sage: P.unrank_range(start=-3)
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> P = Partitions()
>>> P.unrank_range(stop=Integer(5))
[[], [1], [2], [1, 1], [3]]
>>> P.unrank_range(Integer(0), Integer(5))
```

(continues on next page)

(continued from previous page)

```
[[], [1], [2], [1, 1], [3]]
>>> P.unrank_range(Integer(3), Integer(5))
[[1, 1], [3]]
>>> P.unrank_range(Integer(3), Integer(10))
[[1, 1], [3], [2, 1], [1, 1, 1], [4], [3, 1], [2, 2]]
>>> P.unrank_range(Integer(3), Integer(10), Integer(2))
[[1, 1], [2, 1], [4], [2, 2]]
>>> P.unrank_range(Integer(3))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
>>> P.unrank_range(stop=-Integer(3))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
>>> P.unrank_range(start=-Integer(3))
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

additional_structure()

Return `None`.

Indeed, morphisms of enumerated sets are not required to preserve the enumeration.

See also

Category.additional_structure()

EXAMPLES:

```
sage: EnumeratedSets().additional_structure()
```

```
>>> from sage.all import *
>>> EnumeratedSets().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: EnumeratedSets().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> EnumeratedSets().super_categories()
[Category of sets]
```

4.42 Euclidean domains

AUTHORS:

- Teresa Gomez-Diaz (2008): initial version

- Julian Rueth (2013-09-13): added euclidean degree, quotient remainder, and their tests

class sage.categories.euclidean_domains.**EuclideanDomains**

Bases: *Category_singleton*

The category of constructive euclidean domains, i.e., one can divide producing a quotient and a remainder where the remainder is either zero or its *ElementMethods.euclidean_degree()* is smaller than the divisor.

EXAMPLES:

```
sage: EuclideanDomains()
Category of euclidean domains
sage: EuclideanDomains().super_categories()
[Category of principal ideal domains]
```

```
>>> from sage.all import *
>>> EuclideanDomains()
Category of euclidean domains
>>> EuclideanDomains().super_categories()
[Category of principal ideal domains]
```

class ElementMethods

Bases: object

euclidean_degree()

Return the degree of this element as an element of a Euclidean domain, i.e., for elements a, b the euclidean degree f satisfies the usual properties:

1. if b is not zero, then there are elements q and r such that $a = bq + r$ with $r = 0$ or $f(r) < f(b)$
2. if a, b are not zero, then $f(a) \leq f(ab)$

Note

The name euclidean_degree was chosen because the euclidean function has different names in different contexts, e.g., absolute value for integers, degree for polynomials.

OUTPUT:

For nonzero elements, a natural number. For the zero element, this might raise an exception or produce some other output, depending on the implementation.

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: x.euclidean_degree()
1
sage: ZZ.one().euclidean_degree()
1
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> x.euclidean_degree()
1
>>> ZZ.one().euclidean_degree()
1
```

gcd(*other*)

Return the greatest common divisor of this element and `other`.

INPUT:

- `other` – an element in the same ring as `self`

ALGORITHM:

Algorithm 3.2.1 in [Coh1993].

EXAMPLES:

```
sage: R.<x> = PolynomialRing(QQ, sparse=True)
sage: EuclideanDomains().element_class.gcd(x,x+1)
-1
```

```
>>> from sage.all import *
>>> R = PolynomialRing(QQ, sparse=True, names=('x',)); (x,) = R._first_
↪ngens(1)
>>> EuclideanDomains().element_class.gcd(x,x+Integer(1))
-1
```

quo_rem(*other*)

Return the quotient and remainder of the division of this element by the nonzero element `other`.

INPUT:

- `other` – an element in the same euclidean domain

OUTPUT: a pair of elements

EXAMPLES:

```
sage: R.<x> = QQ[]
sage: x.quo_rem(x)
(1, 0)
```

```
>>> from sage.all import *
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> x.quo_rem(x)
(1, 0)
```

class ParentMethods

Bases: `object`

gcd_free_basis(*elts*)

Compute a set of coprime elements that can be used to express the elements of `elts`.

INPUT:

- `elts` – a sequence of elements of `self`

OUTPUT:

A GCD-free basis (also called a coprime base) of `elts`; that is, a set of pairwise relatively prime elements of `self` such that any element of `elts` can be written as a product of elements of the set.

ALGORITHM:

Naive implementation of the algorithm described in Section 4.8 of Bach & Shallit [BS1996].

EXAMPLES:

```
sage: ZZ.gcd_free_basis([1])
[]
sage: ZZ.gcd_free_basis([4, 30, 14, 49])
[2, 15, 7]

sage: Pol.<x> = QQ[]
sage: sorted(Pol.gcd_free_basis([
....:     (x+1)^3*(x+2)^3*(x+3), (x+1)*(x+2)*(x+3),
....:     (x+1)*(x+2)*(x+4)]))
[x + 3, x + 4, x^2 + 3*x + 2]
```

```
>>> from sage.all import *
>>> ZZ.gcd_free_basis([Integer(1)])
[]
>>> ZZ.gcd_free_basis([Integer(4), Integer(30), Integer(14), Integer(49)])
[2, 15, 7]

>>> Pol = QQ['x']; (x,) = Pol._first_ngens(1)
>>> sorted(Pol.gcd_free_basis([
...     ␣
↪(x+Integer(1))**Integer(3)*(x+Integer(2))**Integer(3)*(x+Integer(3)),␣
↪(x+Integer(1))*(x+Integer(2))*(x+Integer(3)),
...     (x+Integer(1))*(x+Integer(2))*(x+Integer(4))]))
[x + 3, x + 4, x^2 + 3*x + 2]
```

is_euclidean_domain()

Return `True`, since this in an object of the category of Euclidean domains.

EXAMPLES:

```
sage: Parent(QQ,category=EuclideanDomains()).is_euclidean_domain()
True
```

```
>>> from sage.all import *
>>> Parent(QQ,category=EuclideanDomains()).is_euclidean_domain()
True
```

super_categories()

EXAMPLES:

```
sage: EuclideanDomains().super_categories()
[Category of principal ideal domains]
```

```
>>> from sage.all import *
>>> EuclideanDomains().super_categories()
[Category of principal ideal domains]
```

4.43 Fields

class sage.categories.fields.**Fields**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of (commutative) fields, i.e. commutative rings where all nonzero elements have multiplicative inverses

EXAMPLES:

```
sage: K = Fields()
sage: K
Category of fields
sage: Fields().super_categories()
[Category of euclidean domains,
 Category of division rings,
 Category of noetherian rings]

sage: K(IntegerRing())
Rational Field
sage: K(PolynomialRing(GF(3), 'x'))
Fraction Field of Univariate Polynomial Ring in x over
Finite Field of size 3
sage: K(RealField())                                                    #␣
↪needs sage.rings.real_mpfr
Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> K = Fields()
>>> K
Category of fields
>>> Fields().super_categories()
[Category of euclidean domains,
 Category of division rings,
 Category of noetherian rings]

>>> K(IntegerRing())
Rational Field
>>> K(PolynomialRing(GF(Integer(3)), 'x'))
Fraction Field of Univariate Polynomial Ring in x over
Finite Field of size 3
>>> K(RealField())                                                      #␣
↪needs sage.rings.real_mpfr
Real Field with 53 bits of precision
```

class ElementMethods

Bases: object

euclidean_degree()

Return the degree of this element as an element of a Euclidean domain.

In a field, this returns 0 for all but the zero element (for which it is undefined).

EXAMPLES:

```
sage: QQ.one().euclidean_degree()
0
```

```
>>> from sage.all import *
>>> QQ.one().euclidean_degree()
0
```

factor()

Return a factorization of `self`.

Since `self` is either a unit or zero, this function is trivial.

EXAMPLES:

```
sage: x = GF(7)(5)
sage: x.factor()
5
sage: RR(0).factor()                                                    #␣
↪needs sage.rings.real_mpfr
Traceback (most recent call last):
...
ArithmeticError: factorization of 0.000000000000000 is not defined
```

```
>>> from sage.all import *
>>> x = GF(Integer(7))(Integer(5))
>>> x.factor()
5
>>> RR(Integer(0)).factor()                                                ␣
↪     # needs sage.rings.real_mpfr
Traceback (most recent call last):
...
ArithmeticError: factorization of 0.000000000000000 is not defined
```

gcd(*other*)

Greatest common divisor.

Note

Since we are in a field and the greatest common divisor is only determined up to a unit, it is correct to either return zero or one. Note that fraction fields of unique factorization domains provide a more sophisticated gcd.

EXAMPLES:

```
sage: K = GF(5)
sage: K(2).gcd(K(1))
1
sage: K(0).gcd(K(0))
0
sage: all(x.gcd(y) == (0 if x == 0 and y == 0 else 1)
....:     for x in K for y in K)
True
```

```
>>> from sage.all import *
>>> K = GF(Integer(5))
```

(continues on next page)

(continued from previous page)

```
>>> K(Integer(2)).gcd(K(Integer(1)))
1
>>> K(Integer(0)).gcd(K(Integer(0)))
0
>>> all(x.gcd(y) == (Integer(0) if x == Integer(0) and y == Integer(0)␣
↪else Integer(1))
...     for x in K for y in K)
True
```

For field of characteristic zero, the gcd of integers is considered as if they were elements of the integer ring:

```
sage: gcd(15.0,12.0)                                                    #␣
↪needs sage.rings.real_mpfr
3.00000000000000
```

```
>>> from sage.all import *
>>> gcd(RealNumber('15.0'),RealNumber('12.0'))                          ␣
↪                        # needs sage.rings.real_mpfr
3.00000000000000
```

But for other floating point numbers, the gcd is just 0.0 or 1.0:

```
sage: gcd(3.2, 2.18)                                                    #␣
↪needs sage.rings.real_mpfr
1.00000000000000

sage: gcd(0.0, 0.0)                                                     #␣
↪needs sage.rings.real_mpfr
0.000000000000000
```

```
>>> from sage.all import *
>>> gcd(RealNumber('3.2'), RealNumber('2.18'))                          ␣
↪                        # needs sage.rings.real_mpfr
1.00000000000000

>>> gcd(RealNumber('0.0'), RealNumber('0.0'))                           ␣
↪                        # needs sage.rings.real_mpfr
0.000000000000000
```

AUTHOR:

- Simon King (2011-02) – Issue #10771
- Vincent Delecroix (2015) – Issue #17671

inverse_of_unit()

Return the inverse of this element.

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: NumberField(x^7 + 2, 'a')(2).inverse_of_unit()                    #␣
↪needs sage.rings.number_field
1/2
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> NumberField(x**Integer(7) + Integer(2), 'a')(Integer(2)).inverse_of_
↪unit()                    # needs sage.rings.number_field
1/2
```

Trying to invert the zero element typically raises a `ZeroDivisionError`:

```
sage: QQ(0).inverse_of_unit()
Traceback (most recent call last):
...
ZeroDivisionError: rational division by zero
```

```
>>> from sage.all import *
>>> QQ(Integer(0)).inverse_of_unit()
Traceback (most recent call last):
...
ZeroDivisionError: rational division by zero
```

To catch that exception in a way that also works for non-units in more general rings, use something like:

```
sage: try:
....:     QQ(0).inverse_of_unit()
....: except ArithmeticError:
....:     pass
```

```
>>> from sage.all import *
>>> try:
...     QQ(Integer(0)).inverse_of_unit()
... except ArithmeticError:
...     pass
```

Also note that some "fields" allow one to invert the zero element:

```
sage: RR(0).inverse_of_unit()
+infinity
```

```
>>> from sage.all import *
>>> RR(Integer(0)).inverse_of_unit()
+infinity
```

is_unit()

Return `True` if `self` has a multiplicative inverse.

EXAMPLES:

```
sage: QQ(2).is_unit()
True
sage: QQ(0).is_unit()
False
```

```
>>> from sage.all import *
>>> QQ(Integer(2)).is_unit()
```

(continues on next page)

(continued from previous page)

```
True
>>> QQ(Integer(0)).is_unit()
False
```

lcm(*other*)

Least common multiple.

> **Note**
>
> Since we are in a field and the least common multiple is only determined up to a unit, it is correct to either return zero or one. Note that fraction fields of unique factorization domains provide a more sophisticated lcm.

EXAMPLES:

```
sage: GF(2)(1).lcm(GF(2)(0))
0
sage: GF(2)(1).lcm(GF(2)(1))
1
```

```
>>> from sage.all import *
>>> GF(Integer(2))(Integer(1)).lcm(GF(Integer(2))(Integer(0)))
0
>>> GF(Integer(2))(Integer(1)).lcm(GF(Integer(2))(Integer(1)))
1
```

For field of characteristic zero, the lcm of integers is considered as if they were elements of the integer ring:

```
sage: lcm(15.0, 12.0)                                                       #␣
↪needs sage.rings.real_mpfr
60.0000000000000
```

```
>>> from sage.all import *
>>> lcm(RealNumber('15.0'), RealNumber('12.0'))                              ␣
↪                          # needs sage.rings.real_mpfr
60.0000000000000
```

But for others floating point numbers, it is just 0.0 or 1.0:

```
sage: lcm(3.2, 2.18)                                                        #␣
↪needs sage.rings.real_mpfr
1.00000000000000

sage: lcm(0.0, 0.0)                                                         #␣
↪needs sage.rings.real_mpfr
0.000000000000000
```

```
>>> from sage.all import *
>>> lcm(RealNumber('3.2'), RealNumber('2.18'))                               ␣
```

(continues on next page)

(continued from previous page)

```
↪                          # needs sage.rings.real_mpfr
1.00000000000000

>>> lcm(RealNumber('0.0'), RealNumber('0.0'))
↪                          # needs sage.rings.real_mpfr
0.000000000000000
```

AUTHOR:

- Simon King (2011-02) – Issue #10771
- Vincent Delecroix (2015) – Issue #17671

quo_rem(*other*)

Return the quotient with remainder of the division of this element by `other`.

INPUT:

- `other` – an element of the field

EXAMPLES:

```
sage: f,g = QQ(1), QQ(2)
sage: f.quo_rem(g)
(1/2, 0)
```

```
>>> from sage.all import *
>>> f,g = QQ(Integer(1)), QQ(Integer(2))
>>> f.quo_rem(g)
(1/2, 0)
```

xgcd(*other*)

Compute the extended gcd of `self` and `other`.

INPUT:

- `other` – an element with the same parent as `self`

OUTPUT:

A tuple `(r, s, t)` of elements in the parent of `self` such that `r = s * self + t * other`. Since the computations are done over a field, `r` is zero if `self` and `other` are zero, and one otherwise.

AUTHORS:

- Julian Rueth (2012-10-19): moved here from `sage.structure.element.FieldElement`

EXAMPLES:

```
sage: K = GF(5)
sage: K(2).xgcd(K(1))
(1, 3, 0)
sage: K(0).xgcd(K(4))
(1, 0, 4)
sage: K(1).xgcd(K(1))
(1, 1, 0)
sage: GF(5)(0).xgcd(GF(5)(0))
(0, 0, 0)
```

```
>>> from sage.all import *
>>> K = GF(Integer(5))
>>> K(Integer(2)).xgcd(K(Integer(1)))
```

(continues on next page)

(continued from previous page)

```
(1, 3, 0)
>>> K(Integer(0)).xgcd(K(Integer(4)))
(1, 0, 4)
>>> K(Integer(1)).xgcd(K(Integer(1)))
(1, 1, 0)
>>> GF(Integer(5))(Integer(0)).xgcd(GF(Integer(5))(Integer(0)))
(0, 0, 0)
```

The xgcd of nonzero floating point numbers will be a triple of floating points. But if the input are two integral floating points the result is a floating point version of the standard gcd on **Z**:

```
sage: xgcd(12.0, 8.0)                                                          #␣
↪needs sage.rings.real_mpfr
(4.00000000000000, 1.00000000000000, -1.00000000000000)

sage: xgcd(3.1, 2.98714)                                                       #␣
↪needs sage.rings.real_mpfr
(1.00000000000000, 0.322580645161290, 0.000000000000000)

sage: xgcd(0.0, 1.1)                                                           #␣
↪needs sage.rings.real_mpfr
(1.00000000000000, 0.000000000000000, 0.909090909090909)
```

```
>>> from sage.all import *
>>> xgcd(RealNumber('12.0'), RealNumber('8.0'))                                ␣
↪                          # needs sage.rings.real_mpfr
(4.00000000000000, 1.00000000000000, -1.00000000000000)

>>> xgcd(RealNumber('3.1'), RealNumber('2.98714'))                             ␣
↪                          # needs sage.rings.real_mpfr
(1.00000000000000, 0.322580645161290, 0.000000000000000)

>>> xgcd(RealNumber('0.0'), RealNumber('1.1'))                                 ␣
↪                          # needs sage.rings.real_mpfr
(1.00000000000000, 0.000000000000000, 0.909090909090909)
```

Finite

alias of *FiniteFields*

class ParentMethods

Bases: `object`

algebraic_closure()

Return the algebraic closure of `self`.

> **Note**
>
> This is only implemented for certain classes of field.

EXAMPLES:

```
sage: K = PolynomialRing(QQ,'x').fraction_field(); K
Fraction Field of Univariate Polynomial Ring in x over Rational Field
sage: K.algebraic_closure()
Traceback (most recent call last):
...
NotImplementedError: algebraic closures of general fields not implemented
```

```
>>> from sage.all import *
>>> K = PolynomialRing(QQ,'x').fraction_field(); K
Fraction Field of Univariate Polynomial Ring in x over Rational Field
>>> K.algebraic_closure()
Traceback (most recent call last):
...
NotImplementedError: algebraic closures of general fields not implemented
```

an_embedding(K)

Return some embedding of this field into another field K, and raise a `ValueError` if none exists.

EXAMPLES:

```
sage: GF(2).an_embedding(GF(4))
Ring morphism:
  From: Finite Field of size 2
  To:   Finite Field in z2 of size 2^2
  Defn: 1 |--> 1
sage: GF(4).an_embedding(GF(8))
Traceback (most recent call last):
...
ValueError: no embedding from Finite Field in z2 of size 2^2 to Finite␣
↪Field in z3 of size 2^3
sage: GF(4).an_embedding(GF(16))
Ring morphism:
  From: Finite Field in z2 of size 2^2
  To:   Finite Field in z4 of size 2^4
  Defn: z2 |--> z4^2 + z4
```

```
>>> from sage.all import *
>>> GF(Integer(2)).an_embedding(GF(Integer(4)))
Ring morphism:
  From: Finite Field of size 2
  To:   Finite Field in z2 of size 2^2
  Defn: 1 |--> 1
>>> GF(Integer(4)).an_embedding(GF(Integer(8)))
Traceback (most recent call last):
...
ValueError: no embedding from Finite Field in z2 of size 2^2 to Finite␣
```

(continues on next page)

(continued from previous page)

```
↪Field in z3 of size 2^3
>>> GF(Integer(4)).an_embedding(GF(Integer(16)))
Ring morphism:
  From: Finite Field in z2 of size 2^2
  To:   Finite Field in z4 of size 2^4
  Defn: z2 |--> z4^2 + z4
```

```
sage: CyclotomicField(5).an_embedding(QQbar)
Coercion map:
  From: Cyclotomic Field of order 5 and degree 4
  To:   Algebraic Field
sage: CyclotomicField(3).an_embedding(CyclotomicField(7))
Traceback (most recent call last):
...
ValueError: no embedding from Cyclotomic Field of order 3 and degree 2 to␣
↪Cyclotomic Field of order 7 and degree 6
sage: CyclotomicField(3).an_embedding(CyclotomicField(6))
Generic morphism:
  From: Cyclotomic Field of order 3 and degree 2
  To:   Cyclotomic Field of order 6 and degree 2
  Defn: zeta3 -> zeta6 - 1
```

```
>>> from sage.all import *
>>> CyclotomicField(Integer(5)).an_embedding(QQbar)
Coercion map:
  From: Cyclotomic Field of order 5 and degree 4
  To:   Algebraic Field
>>> CyclotomicField(Integer(3)).an_embedding(CyclotomicField(Integer(7)))
Traceback (most recent call last):
...
ValueError: no embedding from Cyclotomic Field of order 3 and degree 2 to␣
↪Cyclotomic Field of order 7 and degree 6
>>> CyclotomicField(Integer(3)).an_embedding(CyclotomicField(Integer(6)))
Generic morphism:
  From: Cyclotomic Field of order 3 and degree 2
  To:   Cyclotomic Field of order 6 and degree 2
  Defn: zeta3 -> zeta6 - 1
```

divides(*x*, *y*, *coerce=True*)

Return `True` if `x` divides `y` in this field.

This is usually `True` in a field!.

If `coerce` is `True` (the default), first coerce `x` and `y` into `self`.

EXAMPLES:

```
sage: QQ.divides(2, 3/4)
True
sage: QQ.divides(0, 5)
False
```

```
>>> from sage.all import *
>>> QQ.divides(Integer(2), Integer(3)/Integer(4))
True
>>> QQ.divides(Integer(0), Integer(5))
False
```

fraction_field()

Return the *fraction field* of `self`, which is `self`.

EXAMPLES:

Since fields are their own field of fractions, we simply get the original field in return:

```
sage: QQ.fraction_field()
Rational Field
sage: RR.fraction_field()
↪  # needs sage.rings.real_mpfr
Real Field with 53 bits of precision
sage: CC.fraction_field()
↪  # needs sage.rings.real_mpfr
Complex Field with 53 bits of precision

sage: x = polygen(ZZ, 'x')
sage: F = NumberField(x^2 + 1, 'i')
↪  # needs sage.rings.number_field
sage: F.fraction_field()
↪  # needs sage.rings.number_field
Number Field in i with defining polynomial x^2 + 1
```

```
>>> from sage.all import *
>>> QQ.fraction_field()
Rational Field
>>> RR.fraction_field()
↪# needs sage.rings.real_mpfr
Real Field with 53 bits of precision
>>> CC.fraction_field()
↪# needs sage.rings.real_mpfr
Complex Field with 53 bits of precision

>>> x = polygen(ZZ, 'x')
>>> F = NumberField(x**Integer(2) + Integer(1), 'i')
↪                  # needs sage.rings.number_field
>>> F.fraction_field()
↪# needs sage.rings.number_field
Number Field in i with defining polynomial x^2 + 1
```

ideal(*gens*, ***kwds*)

Return the ideal generated by `gens`.

INPUT:

- an element or a list/tuple/sequence of elements, the generators

Any named arguments are ignored.

EXAMPLES:

```
sage: QQ.ideal(2)
Principal ideal (1) of Rational Field
sage: QQ.ideal(0)
Principal ideal (0) of Rational Field
```

```
>>> from sage.all import *
>>> QQ.ideal(Integer(2))
Principal ideal (1) of Rational Field
>>> QQ.ideal(Integer(0))
Principal ideal (0) of Rational Field
```

integral_closure()

Return this field, since fields are integrally closed in their fraction field.

EXAMPLES:

```
sage: QQ.integral_closure()
Rational Field
sage: Frac(ZZ['x,y']).integral_closure()
Fraction Field of Multivariate Polynomial Ring in x, y
over Integer Ring
```

```
>>> from sage.all import *
>>> QQ.integral_closure()
Rational Field
>>> Frac(ZZ['x,y']).integral_closure()
Fraction Field of Multivariate Polynomial Ring in x, y
over Integer Ring
```

is_field(*proof=True*)

Return `True` as `self` is a field.

EXAMPLES:

```
sage: QQ.is_field()
True
sage: Parent(QQ,category=Fields()).is_field()
True

sage: Frac(ZZ['x,y']).is_field()
True
```

```
>>> from sage.all import *
>>> QQ.is_field()
True
>>> Parent(QQ,category=Fields()).is_field()
True

>>> Frac(ZZ['x,y']).is_field()
True
```

is_integrally_closed()

Return whether `self` is integrally closed.

For every field F, F is its own field of fractions. Therefore every element of F is integral over F.

EXAMPLES:

```
sage: QQ.is_integrally_closed()
True
sage: QQbar.is_integrally_closed()                                          #␣
↪needs sage.rings.number_field
True
sage: Z5 = GF(5); Z5
Finite Field of size 5
sage: Z5.is_integrally_closed()
True
sage: Frac(ZZ['x,y']).is_integrally_closed()
True
```

```
>>> from sage.all import *
>>> QQ.is_integrally_closed()
True
>>> QQbar.is_integrally_closed()                                            #␣
↪needs sage.rings.number_field
True
>>> Z5 = GF(Integer(5)); Z5
Finite Field of size 5
>>> Z5.is_integrally_closed()
True
>>> Frac(ZZ['x,y']).is_integrally_closed()
True
```

is_perfect()

Return whether this field is perfect, i.e., its characteristic is $p = 0$ or every element has a p-th root.

EXAMPLES:

```
sage: QQ.is_perfect()
True
sage: GF(2).is_perfect()
True
sage: FunctionField(GF(2), 'x').is_perfect()
False
```

```
>>> from sage.all import *
>>> QQ.is_perfect()
True
>>> GF(Integer(2)).is_perfect()
True
>>> FunctionField(GF(Integer(2)), 'x').is_perfect()
False
```

krull_dimension()

Return the Krull dimension of this field, which is 0.

EXAMPLES:

```
sage: QQ.krull_dimension()
0
sage: Frac(QQ['x,y']).krull_dimension()
0
```

```
>>> from sage.all import *
>>> QQ.krull_dimension()
0
>>> Frac(QQ['x,y']).krull_dimension()
0
```

prime_subfield()

Return the prime subfield of `self`.

EXAMPLES:

```
sage: k = GF(9, 'a')                                                    ␣
↪  # needs sage.rings.finite_rings
sage: k.prime_subfield()                                                ␣
↪  # needs sage.rings.finite_rings
Finite Field of size 3
```

```
>>> from sage.all import *
>>> k = GF(Integer(9), 'a')                                             ␣
↪          # needs sage.rings.finite_rings
>>> k.prime_subfield()
↪# needs sage.rings.finite_rings
Finite Field of size 3
```

vector_space(**args*, ***kwds*)

Give an isomorphism of this field with a vector space over a subfield.

This method is an alias for `free_module`, which may have more documentation.

INPUT:

- `base` – a subfield or morphism into this field (defaults to the base field)
- `basis` – a basis of the field as a vector space over the subfield; if not given, one is chosen automatically
- `map` – whether to return maps from and to the vector space

OUTPUT:

- `V` – a vector space over `base`
- `from_V` – an isomorphism from `V` to this field
- `to_V` – the inverse isomorphism from this field to `V`

EXAMPLES:

```
sage: # needs sage.rings.padics
sage: K.<a> = Qq(125)
sage: V, fr, to = K.vector_space()
sage: v = V([1, 2, 3])
sage: fr(v, 7)
(3*a^2 + 2*a + 1) + O(5^7)
```

```
>>> from sage.all import *
>>> # needs sage.rings.padics
>>> K = Qq(Integer(125), names=('a',)); (a,) = K._first_ngens(1)
>>> V, fr, to = K.vector_space()
>>> v = V([Integer(1), Integer(2), Integer(3)])
>>> fr(v, Integer(7))
(3*a^2 + 2*a + 1) + O(5^7)
```

extra_super_categories()

EXAMPLES:

```
sage: Fields().extra_super_categories()
[Category of euclidean domains, Category of noetherian rings]
```

```
>>> from sage.all import *
>>> Fields().extra_super_categories()
[Category of euclidean domains, Category of noetherian rings]
```

4.44 Filtered Algebras

class sage.categories.filtered_algebras.**FilteredAlgebras**(*base_category*)

Bases: *FilteredModulesCategory*

The category of filtered algebras.

An algebra A over a commutative ring R is *filtered* if A is endowed with a structure of a filtered R-module (whose underlying R-module structure is identical with that of the R-algebra A) such that the indexing set I (typically $I = \mathbf{N}$) is also an additive abelian monoid, the unity 1 of A belongs to F_0, and we have $F_i \cdot F_j \subseteq F_{i+j}$ for all $i, j \in I$.

EXAMPLES:

```
sage: Algebras(ZZ).Filtered()
Category of filtered algebras over Integer Ring
sage: Algebras(ZZ).Filtered().super_categories()
[Category of algebras over Integer Ring,
 Category of filtered modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Filtered()
Category of filtered algebras over Integer Ring
>>> Algebras(ZZ).Filtered().super_categories()
[Category of algebras over Integer Ring,
 Category of filtered modules over Integer Ring]
```

REFERENCES:

- Wikipedia article Filtered_algebra

class ParentMethods

Bases: object

graded_algebra()

Return the associated graded algebra to `self`.

Todo

Implement a version of the associated graded algebra which does not require `self` to have a distinguished basis.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: A.graded_algebra()
Graded Algebra of An example of a filtered algebra with basis:
 the universal enveloping algebra of
 Lie algebra of RR^3 with cross product over Integer Ring
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> A.graded_algebra()
Graded Algebra of An example of a filtered algebra with basis:
 the universal enveloping algebra of
 Lie algebra of RR^3 with cross product over Integer Ring
```

4.45 Filtered Algebras With Basis

A filtered algebra with basis over a commutative ring R is a filtered algebra over R endowed with the structure of a filtered module with basis (with the same underlying filtered-module structure). See *FilteredAlgebras* and *FilteredModulesWithBasis* for these two notions.

class sage.categories.filtered_algebras_with_basis.**FilteredAlgebrasWithBasis**(*base_category*)

Bases: *FilteredModulesCategory*

The category of filtered algebras with a distinguished homogeneous basis.

A filtered algebra with basis over a commutative ring R is a filtered algebra over R endowed with the structure of a filtered module with basis (with the same underlying filtered-module structure). See *FilteredAlgebras* and *FilteredModulesWithBasis* for these two notions.

EXAMPLES:

```
sage: C = AlgebrasWithBasis(ZZ).Filtered(); C
Category of filtered algebras with basis over Integer Ring
sage: sorted(C.super_categories(), key=str)
[Category of algebras with basis over Integer Ring,
 Category of filtered algebras over Integer Ring,
 Category of filtered modules with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(ZZ).Filtered(); C
Category of filtered algebras with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of algebras with basis over Integer Ring,
 Category of filtered algebras over Integer Ring,
 Category of filtered modules with basis over Integer Ring]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

from_graded_conversion()

Return the inverse of the canonical R-module isomorphism $A \to \operatorname{gr} A$ induced by the basis of A (where $A = \text{```}self\text{`}$). This inverse is an isomorphism $\operatorname{gr} A \to A$.

This is an isomorphism of R-modules, not of algebras. See the class documentation AssociatedGradedAlgebra.

See also

to_graded_conversion()

EXAMPLES:

```
sage: A = Algebras(QQ).WithBasis().Filtered().example()
sage: p = A.an_element() + A.algebra_generators()['x'] + 2; p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
sage: q = A.to_graded_conversion()(p)
sage: A.from_graded_conversion()(q) == p
True
sage: q.parent() is A.graded_algebra()
True
```

```
>>> from sage.all import *
>>> A = Algebras(QQ).WithBasis().Filtered().example()
>>> p = A.an_element() + A.algebra_generators()['x'] + Integer(2); p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
>>> q = A.to_graded_conversion()(p)
>>> A.from_graded_conversion()(q) == p
True
>>> q.parent() is A.graded_algebra()
True
```

graded_algebra()

Return the associated graded algebra to self.

See AssociatedGradedAlgebra for the definition and the properties of this.

If the filtered algebra self with basis is called A, then this method returns $\operatorname{gr} A$. The method *to_graded_conversion()* returns the canonical R-module isomorphism $A \to \operatorname{gr} A$ induced by the basis of A, and the method *from_graded_conversion()* returns the inverse of this isomorphism. The method *projection()* projects elements of A onto $\operatorname{gr} A$ according to their place in the filtration on A.

Warning

When not overridden, this method returns the default implementation of an associated graded algebra – namely, AssociatedGradedAlgebra(self), where AssociatedGradedAlgebra is

AssociatedGradedAlgebra. But many instances of *FilteredAlgebrasWithBasis* override this method, as the associated graded algebra often is (isomorphic) to a simpler object (for instance, the associated graded algebra of a graded algebra can be identified with the graded algebra itself). Generic code that uses associated graded algebras (such as the code of the *induced_graded_map()* method below) should make sure to only communicate with them via the *to_graded_conversion()*, *from_graded_conversion()*, and *projection()* methods (in particular, do not expect there to be a conversion from `self` to `self.graded_algebra()`; this currently does not work for Clifford algebras). Similarly, when overriding *graded_algebra()*, make sure to accordingly redefine these three methods, unless their definitions below still apply to your case (this will happen whenever the basis of your *graded_algebra()* has the same indexing set as `self`, and the partition of this indexing set according to degree is the same as for `self`).

Todo

Maybe the thing about the conversion from `self` to `self.graded_algebra()` on the Clifford at least could be made to work? (I would still warn the user against ASSUMING that it must work – as there is probably no way to guarantee it in all cases, and we shouldn't require users to mess with element constructors.)

EXAMPLES:

```
sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: A.graded_algebra()
Graded Algebra of An example of a filtered algebra with basis:
 the universal enveloping algebra of
 Lie algebra of RR^3 with cross product over Integer Ring
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> A.graded_algebra()
Graded Algebra of An example of a filtered algebra with basis:
 the universal enveloping algebra of
 Lie algebra of RR^3 with cross product over Integer Ring
```

induced_graded_map(*other*, *f*)

Return the graded linear map between the associated graded algebras of `self` and `other` canonically induced by the filtration-preserving map `f : self -> other`.

Let A and B be two filtered algebras with basis, and let $(F_i)_{i \in I}$ and $(G_i)_{i \in I}$ be their filtrations. Let $f : A \to B$ be a linear map which preserves the filtration (i.e., satisfies $f(F_i) \subseteq G_i$ for all $i \in I$). Then, there is a canonically defined graded linear map $\operatorname{gr} f : \operatorname{gr} A \to \operatorname{gr} B$ which satisfies

$$(\operatorname{gr} f)(p_i(a)) = p_i(f(a)) \qquad \text{for all } i \in I \text{ and } a \in F_i,$$

where the p_i on the left hand side is the canonical projection from F_i onto the i-th graded component of $\operatorname{gr} A$, while the p_i on the right hand side is the canonical projection from G_i onto the i-th graded component of $\operatorname{gr} B$.

INPUT:

- `other` – a filtered algebra with basis
- `f` – a filtration-preserving linear map from `self` to `other` (can be given as a morphism or as a function)

OUTPUT: the graded linear map gr f

EXAMPLES:

Example 1.

We start with the universal enveloping algebra of the Lie algebra $\mathbf{R}^3$ (with the cross product serving as Lie bracket):

```
sage: A = AlgebrasWithBasis(QQ).Filtered().example(); A
An example of a filtered algebra with basis: the
 universal enveloping algebra of Lie algebra of RR^3
 with cross product over Rational Field
sage: M = A.indices(); M
Free abelian monoid indexed by {'x', 'y', 'z'}
sage: x,y,z = [A.basis()[M.gens()[i]] for i in "xyz"]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).Filtered().example(); A
An example of a filtered algebra with basis: the
 universal enveloping algebra of Lie algebra of RR^3
 with cross product over Rational Field
>>> M = A.indices(); M
Free abelian monoid indexed by {'x', 'y', 'z'}
>>> x,y,z = [A.basis()[M.gens()[i]] for i in "xyz"]
```

Let us define a stupid filtered map from `A` to itself:

```
sage: def map_on_basis(m):
....:     d = m.dict()
....:     i = d.get('x', 0); j = d.get('y', 0); k = d.get('z', 0)
....:     g = (y ** (i+j)) * (z ** k)
....:     if i > 0:
....:         g += i * (x ** (i-1)) * (y ** j) * (z ** k)
....:     return g
sage: f = A.module_morphism(on_basis=map_on_basis,
....:                       codomain=A)
sage: f(x)
U['y'] + 1
sage: f(x*y*z)
U['y']^2*U['z'] + U['y']*U['z']
sage: f(x*x*y*z)
U['y']^3*U['z'] + 2*U['x']*U['y']*U['z']
sage: f(A.one())
1
sage: f(y*z)
U['y']*U['z']
```

```
>>> from sage.all import *
>>> def map_on_basis(m):
...     d = m.dict()
...     i = d.get('x', Integer(0)); j = d.get('y', Integer(0)); k = d.get(
↪'z', Integer(0))
...     g = (y ** (i+j)) * (z ** k)
...     if i > Integer(0):
```

(continues on next page)

(continued from previous page)

```
...         g += i * (x ** (i-Integer(1))) * (y ** j) * (z ** k)
...     return g
>>> f = A.module_morphism(on_basis=map_on_basis,
...                       codomain=A)
>>> f(x)
U['y'] + 1
>>> f(x*y*z)
U['y']^2*U['z'] + U['y']*U['z']
>>> f(x*x*y*z)
U['y']^3*U['z'] + 2*U['x']*U['y']*U['z']
>>> f(A.one())
1
>>> f(y*z)
U['y']*U['z']
```

(There is nothing here that is peculiar to this universal enveloping algebra; we are only using its module structure, and we could just as well be using a polynomial algebra in its stead.)

We now compute gr f

```
sage: grA = A.graded_algebra(); grA
Graded Algebra of An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Rational Field
sage: xx, yy, zz = [A.to_graded_conversion()(i) for i in [x, y, z]]
sage: xx+yy*zz
bar(U['y']*U['z']) + bar(U['x'])
sage: grf = A.induced_graded_map(A, f); grf
Generic endomorphism of Graded Algebra of An example
 of a filtered algebra with basis: the universal
 enveloping algebra of Lie algebra of RR^3 with cross
 product over Rational Field
sage: grf(xx)
bar(U['y'])
sage: grf(xx*yy*zz)
bar(U['y']^2*U['z'])
sage: grf(xx*xx*yy*zz)
bar(U['y']^3*U['z'])
sage: grf(grA.one())
bar(1)
sage: grf(yy*zz)
bar(U['y']*U['z'])
sage: grf(yy*zz-2*yy)
bar(U['y']*U['z']) - 2*bar(U['y'])
```

```
>>> from sage.all import *
>>> grA = A.graded_algebra(); grA
Graded Algebra of An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Rational Field
>>> xx, yy, zz = [A.to_graded_conversion()(i) for i in [x, y, z]]
>>> xx+yy*zz
```

(continues on next page)

(continued from previous page)

```
bar(U['y']*U['z']) + bar(U['x'])
>>> grf = A.induced_graded_map(A, f); grf
Generic endomorphism of Graded Algebra of An example
 of a filtered algebra with basis: the universal
 enveloping algebra of Lie algebra of RR^3 with cross
 product over Rational Field
>>> grf(xx)
bar(U['y'])
>>> grf(xx*yy*zz)
bar(U['y']^2*U['z'])
>>> grf(xx*xx*yy*zz)
bar(U['y']^3*U['z'])
>>> grf(grA.one())
bar(1)
>>> grf(yy*zz)
bar(U['y']*U['z'])
>>> grf(yy*zz-Integer(2)*yy)
bar(U['y']*U['z']) - 2*bar(U['y'])
```

Example 2.

We shall now construct gr f for a different map f out of the same `A`; the new map f will lead into a graded algebra already, namely into the algebra of symmetric functions:

```
sage: # needs sage.combinat sage.modules
sage: h = SymmetricFunctions(QQ).h()
sage: def map_on_basis(m):  # redefining map_on_basis
....:     d = m.dict()
....:     i = d.get('x', 0); j = d.get('y', 0); k = d.get('z', 0)
....:     g = (h[1] ** i) * (h[2] ** (j // 2) * (h[3] ** (k // 3)))
....:     g += i * (h[1] ** (i+j+k))
....:     return g
sage: f = A.module_morphism(on_basis=map_on_basis,
....:                       codomain=h)  # redefining f
sage: f(x)
2*h[1]
sage: f(y)
h[]
sage: f(z)
h[]
sage: f(y**2)
h[2]
sage: f(x**2)
3*h[1, 1]
sage: f(x*y*z)
h[1] + h[1, 1, 1]
sage: f(x*x*y*y*z)
2*h[1, 1, 1, 1, 1] + h[2, 1, 1]
sage: f(A.one())
h[]
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> # needs sage.combinat sage.modules
>>> h = SymmetricFunctions(QQ).h()
>>> def map_on_basis(m):  # redefining map_on_basis
...     d = m.dict()
...     i = d.get('x', Integer(0)); j = d.get('y', Integer(0)); k = d.get(
↪'z', Integer(0))
...     g = (h[Integer(1)] ** i) * (h[Integer(2)] ** (j // Integer(2)) *␣
↪(h[Integer(3)] ** (k // Integer(3))))
...     g += i * (h[Integer(1)] ** (i+j+k))
...     return g
>>> f = A.module_morphism(on_basis=map_on_basis,
...                       codomain=h)  # redefining f
>>> f(x)
2*h[1]
>>> f(y)
h[]
>>> f(z)
h[]
>>> f(y**Integer(2))
h[2]
>>> f(x**Integer(2))
3*h[1, 1]
>>> f(x*y*z)
h[1] + h[1, 1, 1]
>>> f(x*x*y*y*z)
2*h[1, 1, 1, 1, 1] + h[2, 1, 1]
>>> f(A.one())
h[]
```

The algebra `h` of symmetric functions in the h-basis is already graded, so its associated graded algebra is implemented as itself:

```
sage: # needs sage.combinat sage.modules
sage: grh = h.graded_algebra(); grh is h
True
sage: grf = A.induced_graded_map(h, f); grf
Generic morphism:
  From: Graded Algebra of An example of a filtered
   algebra with basis: the universal enveloping
   algebra of Lie algebra of RR^3 with cross
   product over Rational Field
  To:   Symmetric Functions over Rational Field
   in the homogeneous basis
sage: grf(xx)
2*h[1]
sage: grf(yy)
0
sage: grf(zz)
0
sage: grf(yy**2)
h[2]
sage: grf(xx**2)
```

(continues on next page)

(continued from previous page)

```
3*h[1, 1]
sage: grf(xx*yy*zz)
h[1, 1, 1]
sage: grf(xx*xx*yy*yy*zz)
2*h[1, 1, 1, 1, 1]
sage: grf(grA.one())
h[]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> grh = h.graded_algebra(); grh is h
True
>>> grf = A.induced_graded_map(h, f); grf
Generic morphism:
  From: Graded Algebra of An example of a filtered
   algebra with basis: the universal enveloping
   algebra of Lie algebra of RR^3 with cross
   product over Rational Field
  To:   Symmetric Functions over Rational Field
   in the homogeneous basis
>>> grf(xx)
2*h[1]
>>> grf(yy)
0
>>> grf(zz)
0
>>> grf(yy**Integer(2))
h[2]
>>> grf(xx**Integer(2))
3*h[1, 1]
>>> grf(xx*yy*zz)
h[1, 1, 1]
>>> grf(xx*xx*yy*yy*zz)
2*h[1, 1, 1, 1, 1]
>>> grf(grA.one())
h[]
```

Example 3.

After having had a graded algebra as the codomain, let us try to have one as the domain instead. Our new `f` will go from `h` to `A`:

```
sage: # needs sage.combinat sage.modules
sage: def map_on_basis(lam):  # redefining map_on_basis
....:     return x ** (sum(lam)) + y ** (len(lam))
sage: f = h.module_morphism(on_basis=map_on_basis,
....:                       codomain=A)  # redefining f
sage: f(h[1])
U['x'] + U['y']
sage: f(h[2])
U['x']^2 + U['y']
sage: f(h[1, 1])
```

(continues on next page)

(continued from previous page)

```
U['x']^2 + U['y']^2
sage: f(h[2, 2])
U['x']^4 + U['y']^2
sage: f(h[3, 2, 1])
U['x']^6 + U['y']^3
sage: f(h.one())
2
sage: grf = h.induced_graded_map(A, f); grf
Generic morphism:
  From: Symmetric Functions over Rational Field
   in the homogeneous basis
  To:   Graded Algebra of An example of a filtered
   algebra with basis: the universal enveloping
   algebra of Lie algebra of RR^3 with cross
   product over Rational Field
sage: grf(h[1])
bar(U['x']) + bar(U['y'])
sage: grf(h[2])
bar(U['x']^2)
sage: grf(h[1, 1])
bar(U['x']^2) + bar(U['y']^2)
sage: grf(h[2, 2])
bar(U['x']^4)
sage: grf(h[3, 2, 1])
bar(U['x']^6)
sage: grf(h.one())
2*bar(1)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> def map_on_basis(lam):  # redefining map_on_basis
...     return x ** (sum(lam)) + y ** (len(lam))
>>> f = h.module_morphism(on_basis=map_on_basis,
...                       codomain=A)  # redefining f
>>> f(h[Integer(1)])
U['x'] + U['y']
>>> f(h[Integer(2)])
U['x']^2 + U['y']
>>> f(h[Integer(1), Integer(1)])
U['x']^2 + U['y']^2
>>> f(h[Integer(2), Integer(2)])
U['x']^4 + U['y']^2
>>> f(h[Integer(3), Integer(2), Integer(1)])
U['x']^6 + U['y']^3
>>> f(h.one())
2
>>> grf = h.induced_graded_map(A, f); grf
Generic morphism:
  From: Symmetric Functions over Rational Field
   in the homogeneous basis
  To:   Graded Algebra of An example of a filtered
   algebra with basis: the universal enveloping
```

(continues on next page)

(continued from previous page)

```
   algebra of Lie algebra of RR^3 with cross
   product over Rational Field
>>> grf(h[Integer(1)])
bar(U['x']) + bar(U['y'])
>>> grf(h[Integer(2)])
bar(U['x']^2)
>>> grf(h[Integer(1), Integer(1)])
bar(U['x']^2) + bar(U['y']^2)
>>> grf(h[Integer(2), Integer(2)])
bar(U['x']^4)
>>> grf(h[Integer(3), Integer(2), Integer(1)])
bar(U['x']^6)
>>> grf(h.one())
2*bar(1)
```

Example 4.

The construct gr f also makes sense when f is a filtration-preserving map between graded algebras.

```
sage: # needs sage.combinat sage.modules
sage: def map_on_basis(lam):  # redefining map_on_basis
....:     return h[lam] + h[len(lam)]
sage: f = h.module_morphism(on_basis=map_on_basis,
....:                       codomain=h)  # redefining f
sage: f(h[1])
2*h[1]
sage: f(h[2])
h[1] + h[2]
sage: f(h[1, 1])
h[1, 1] + h[2]
sage: f(h[2, 1])
h[2] + h[2, 1]
sage: f(h.one())
2*h[]
sage: grf = h.induced_graded_map(h, f); grf
Generic endomorphism of Symmetric Functions over Rational
 Field in the homogeneous basis
sage: grf(h[1])
2*h[1]
sage: grf(h[2])
h[2]
sage: grf(h[1, 1])
h[1, 1] + h[2]
sage: grf(h[2, 1])
h[2, 1]
sage: grf(h.one())
2*h[]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> def map_on_basis(lam):  # redefining map_on_basis
...     return h[lam] + h[len(lam)]
```

(continues on next page)

(continued from previous page)

```
>>> f = h.module_morphism(on_basis=map_on_basis,
...                       codomain=h)  # redefining f
>>> f(h[Integer(1)])
2*h[1]
>>> f(h[Integer(2)])
h[1] + h[2]
>>> f(h[Integer(1), Integer(1)])
h[1, 1] + h[2]
>>> f(h[Integer(2), Integer(1)])
h[2] + h[2, 1]
>>> f(h.one())
2*h[]
>>> grf = h.induced_graded_map(h, f); grf
Generic endomorphism of Symmetric Functions over Rational
 Field in the homogeneous basis
>>> grf(h[Integer(1)])
2*h[1]
>>> grf(h[Integer(2)])
h[2]
>>> grf(h[Integer(1), Integer(1)])
h[1, 1] + h[2]
>>> grf(h[Integer(2), Integer(1)])
h[2, 1]
>>> grf(h.one())
2*h[]
```

Example 5.

For another example, let us compute gr f for a map f between two Clifford algebras:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(ZZ, 2, [1,2,3])
sage: B = CliffordAlgebra(Q, names=['u','v']); B
The Clifford algebra of the Quadratic form in 2
 variables over Integer Ring with coefficients:
[ 1 2 ]
[ * 3 ]
sage: m = Matrix(ZZ, [[1, 2], [1, -1]])
sage: f = B.lift_module_morphism(m, names=['x','y'])
sage: A = f.domain(); A
The Clifford algebra of the Quadratic form in 2
 variables over Integer Ring with coefficients:
[ 6 0 ]
[ * 3 ]
sage: x, y = A.gens()
sage: f(x)
u + v
sage: f(y)
2*u - v
sage: f(x**2)
6
sage: f(x*y)
-3*u*v + 3
```

(continues on next page)

(continued from previous page)

```
sage: grA = A.graded_algebra(); grA
The exterior algebra of rank 2 over Integer Ring
sage: A.to_graded_conversion()(x)
x
sage: A.to_graded_conversion()(y)
y
sage: A.to_graded_conversion()(x*y)
x*y
sage: u = A.to_graded_conversion()(x*y+1); u
x*y + 1
sage: A.from_graded_conversion()(u)
x*y + 1
sage: A.projection(2)(x*y+1)
x*y
sage: A.projection(1)(x+2*y-2)
x + 2*y
sage: grf = A.induced_graded_map(B, f); grf
Generic morphism:
  From: The exterior algebra of rank 2 over Integer Ring
  To:   The exterior algebra of rank 2 over Integer Ring
sage: grf(A.to_graded_conversion()(x))
u + v
sage: grf(A.to_graded_conversion()(y))
2*u - v
sage: grf(A.to_graded_conversion()(x**2))
6
sage: grf(A.to_graded_conversion()(x*y))
-3*u*v
sage: grf(grA.one())
1
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(ZZ, Integer(2), [Integer(1),Integer(2),Integer(3)])
>>> B = CliffordAlgebra(Q, names=['u','v']); B
The Clifford algebra of the Quadratic form in 2
 variables over Integer Ring with coefficients:
[ 1 2 ]
[ * 3 ]
>>> m = Matrix(ZZ, [[Integer(1), Integer(2)], [Integer(1), -Integer(1)]])
>>> f = B.lift_module_morphism(m, names=['x','y'])
>>> A = f.domain(); A
The Clifford algebra of the Quadratic form in 2
 variables over Integer Ring with coefficients:
[ 6 0 ]
[ * 3 ]
>>> x, y = A.gens()
>>> f(x)
u + v
>>> f(y)
2*u - v
>>> f(x**Integer(2))
```

(continues on next page)

(continued from previous page)

```
6
>>> f(x*y)
-3*u*v + 3
>>> grA = A.graded_algebra(); grA
The exterior algebra of rank 2 over Integer Ring
>>> A.to_graded_conversion()(x)
x
>>> A.to_graded_conversion()(y)
y
>>> A.to_graded_conversion()(x*y)
x*y
>>> u = A.to_graded_conversion()(x*y+Integer(1)); u
x*y + 1
>>> A.from_graded_conversion()(u)
x*y + 1
>>> A.projection(Integer(2))(x*y+Integer(1))
x*y
>>> A.projection(Integer(1))(x+Integer(2)*y-Integer(2))
x + 2*y
>>> grf = A.induced_graded_map(B, f); grf
Generic morphism:
  From: The exterior algebra of rank 2 over Integer Ring
  To:   The exterior algebra of rank 2 over Integer Ring
>>> grf(A.to_graded_conversion()(x))
u + v
>>> grf(A.to_graded_conversion()(y))
2*u - v
>>> grf(A.to_graded_conversion()(x**Integer(2)))
6
>>> grf(A.to_graded_conversion()(x*y))
-3*u*v
>>> grf(grA.one())
1
```

projection(*i*)

Return the i-th projection $p_i : F_i \to G_i$ (in the notations of the class documentation `AssociatedGradedAlgebra`, where $A = \text{```}self\text{`}$).

This method actually does not return the map p_i itself, but an extension of p_i to the whole R-module A. This extension is the composition of the R-module isomorphism $A \to \operatorname{gr} A$ with the canonical projection of the graded R-module $\operatorname{gr} A$ onto its i-th graded component G_i. The codomain of this map is $\operatorname{gr} A$, although its actual image is G_i. The map p_i is obtained from this map by restricting its domain to F_i and its image to G_i.

EXAMPLES:

```
sage: A = Algebras(QQ).WithBasis().Filtered().example()
sage: p = A.an_element() + A.algebra_generators()['x'] + 2; p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
sage: q = A.projection(7)(p); q
bar(U['x']^2*U['y']^2*U['z']^3)
sage: q.parent() is A.graded_algebra()
True
```

(continues on next page)

(continued from previous page)

```
sage: A.projection(8)(p)
0
```

```
>>> from sage.all import *
>>> A = Algebras(QQ).WithBasis().Filtered().example()
>>> p = A.an_element() + A.algebra_generators()['x'] + Integer(2); p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
>>> q = A.projection(Integer(7))(p); q
bar(U['x']^2*U['y']^2*U['z']^3)
>>> q.parent() is A.graded_algebra()
True
>>> A.projection(Integer(8))(p)
0
```

to_graded_conversion()

Return the canonical R-module isomorphism $A \to \operatorname{gr} A$ induced by the basis of A (where $A =$ ``*self*`).

This is an isomorphism of R-modules, not of algebras. See the class documentation `Associated-GradedAlgebra`.

See also

from_graded_conversion()

EXAMPLES:

```
sage: A = Algebras(QQ).WithBasis().Filtered().example()
sage: p = A.an_element() + A.algebra_generators()['x'] + 2; p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
sage: q = A.to_graded_conversion()(p); q
bar(U['x']^2*U['y']^2*U['z']^3) + 3*bar(U['x'])
 + 3*bar(U['y']) + 3*bar(1)
sage: q.parent() is A.graded_algebra()
True
```

```
>>> from sage.all import *
>>> A = Algebras(QQ).WithBasis().Filtered().example()
>>> p = A.an_element() + A.algebra_generators()['x'] + Integer(2); p
U['x']^2*U['y']^2*U['z']^3 + 3*U['x'] + 3*U['y'] + 3
>>> q = A.to_graded_conversion()(p); q
bar(U['x']^2*U['y']^2*U['z']^3) + 3*bar(U['x'])
 + 3*bar(U['y']) + 3*bar(1)
>>> q.parent() is A.graded_algebra()
True
```

4.46 Filtered Hopf algebras with basis

class sage.categories.filtered_hopf_algebras_with_basis.**FilteredHopfAlgebrasWithBasis**(*base_category*)

Bases: *FilteredModulesCategory*

The category of filtered Hopf algebras with a distinguished basis.

A filtered Hopf algebra with basis over a commutative ring R is a filtered Hopf algebra over R (that is, a Hopf algebra equipped with a module filtration such that all structure maps of the Hopf algebra respect the filtration) equipped with an R-module basis that makes it a filtered R-module with basis (see *FilteredModulesWithBasis* for the notion of a filtered module with basis).

EXAMPLES:

```
sage: C = HopfAlgebrasWithBasis(ZZ).Filtered(); C
Category of filtered Hopf algebras with basis over Integer Ring
sage: C.super_categories()
[Category of Hopf algebras with basis over Integer Ring,
 Category of filtered algebras with basis over Integer Ring,
 Category of filtered coalgebras with basis over Integer Ring]

sage: C is HopfAlgebras(ZZ).WithBasis().Filtered()
True
sage: C is HopfAlgebras(ZZ).Filtered().WithBasis()
False
```

```
>>> from sage.all import *
>>> C = HopfAlgebrasWithBasis(ZZ).Filtered(); C
Category of filtered Hopf algebras with basis over Integer Ring
>>> C.super_categories()
[Category of Hopf algebras with basis over Integer Ring,
 Category of filtered algebras with basis over Integer Ring,
 Category of filtered coalgebras with basis over Integer Ring]

>>> C is HopfAlgebras(ZZ).WithBasis().Filtered()
True
>>> C is HopfAlgebras(ZZ).Filtered().WithBasis()
False
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ElementMethods

Bases: object

class ParentMethods

Bases: object

antipode(*elem*)

Return the antipode of `self` applied to `elem`.

antipode_on_basis(*index*)

The antipode on the basis element indexed by `index`.

INPUT:

- `index` – an element of the index set

For a filtered connected Hopf algebra, we can define an antipode recursively by

$$S(x) := -\sum_{x^L \neq x} S(x^L) \times x^R + \epsilon(x)$$

for all x, using the Sweedler notation. Also, $S(x) = x$ for all x with $|x| = 0$.

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

super_categories()

EXAMPLES:

```
sage: HopfAlgebrasWithBasis(QQ).Filtered().WithRealizations().super_
↪categories()
[Join of Category of Hopf algebras over Rational Field
     and Category of filtered algebras over Rational Field
     and Category of filtered coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> HopfAlgebrasWithBasis(QQ).Filtered().WithRealizations().super_
↪categories()
[Join of Category of Hopf algebras over Rational Field
     and Category of filtered algebras over Rational Field
     and Category of filtered coalgebras over Rational Field]
```

4.47 Filtered Modules

A *filtered module* over a ring R with a totally ordered indexing set I (typically $I = \mathbf{N}$) is an R-module M equipped with a family $(F_i)_{i \in I}$ of R-submodules satisfying $F_i \subseteq F_j$ for all $i, j \in I$ having $i \leq j$, and $M = \bigcup_{i \in I} F_i$. This family is called a *filtration* of the given module M.

Todo

Implement a notion for decreasing filtrations: where $F_j \subseteq F_i$ when $i \leq j$.

Todo

Implement filtrations for all concrete categories.

Todo

Implement gr as a functor.

class sage.categories.filtered_modules.**FilteredModules**(*base_category*)

Bases: *FilteredModulesCategory*

The category of filtered modules over a given ring R.

A *filtered module* over a ring R with a totally ordered indexing set I (typically $I = \mathbf{N}$) is an R-module M equipped with a family $(F_i)_{i \in I}$ of R-submodules satisfying $F_i \subseteq F_j$ for all $i, j \in I$ having $i \leq j$, and $M = \bigcup_{i \in I} F_i$. This family is called a *filtration* of the given module M.

EXAMPLES:

```
sage: Modules(ZZ).Filtered()
Category of filtered modules over Integer Ring
sage: Modules(ZZ).Filtered().super_categories()
[Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).Filtered()
Category of filtered modules over Integer Ring
>>> Modules(ZZ).Filtered().super_categories()
[Category of modules over Integer Ring]
```

REFERENCES:

- Wikipedia article Filtration_(mathematics)

class Connected(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class SubcategoryMethods

Bases: object

Connected()

Return the full subcategory of the connected objects of self.

A filtered R-module M with filtration $(F_0, F_1, F_2, \ldots)$ (indexed by $\mathbf{N}$) is said to be *connected* if F_0 is isomorphic to R.

EXAMPLES:

```
sage: Modules(ZZ).Filtered().Connected()
Category of filtered connected modules over Integer Ring
sage: Coalgebras(QQ).Filtered().Connected()
Category of filtered connected coalgebras over Rational Field
sage: AlgebrasWithBasis(QQ).Filtered().Connected()
Category of filtered connected algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).Filtered().Connected()
Category of filtered connected modules over Integer Ring
>>> Coalgebras(QQ).Filtered().Connected()
Category of filtered connected coalgebras over Rational Field
>>> AlgebrasWithBasis(QQ).Filtered().Connected()
Category of filtered connected algebras with basis over Rational Field
```

extra_super_categories()

Add *VectorSpaces* to the super categories of self if the base ring is a field.

EXAMPLES:

```
sage: Modules(QQ).Filtered().is_subcategory(VectorSpaces(QQ))
True
sage: Modules(ZZ).Filtered().extra_super_categories()
[]
```

```
>>> from sage.all import *
>>> Modules(QQ).Filtered().is_subcategory(VectorSpaces(QQ))
True
>>> Modules(ZZ).Filtered().extra_super_categories()
[]
```

This makes sure that `Modules(QQ).Filtered()` returns an instance of *FilteredModules* and not a join category of an instance of this class and of `VectorSpaces(QQ)`:

```
sage: type(Modules(QQ).Filtered())
<class 'sage.categories.vector_spaces.VectorSpaces.Filtered_with_category'>
```

```
>>> from sage.all import *
>>> type(Modules(QQ).Filtered())
<class 'sage.categories.vector_spaces.VectorSpaces.Filtered_with_category'>
```

Todo

Get rid of this workaround once there is a more systematic approach for the alias `Modules(QQ)` -> `VectorSpaces(QQ)`. Probably the latter should be a category with axiom, and covariant constructions should play well with axioms.

class sage.categories.filtered_modules.**FilteredModulesCategory**(*base_category*)

Bases: *RegressiveCovariantConstructionCategory*, *Category_over_base_ring*

EXAMPLES:

```
sage: C = Algebras(QQ).Filtered()
sage: C
Category of filtered algebras over Rational Field
sage: C.base_category()
Category of algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of filtered vector spaces over Rational Field]

sage: AlgebrasWithBasis(QQ).Filtered().base_ring()
Rational Field
sage: HopfAlgebrasWithBasis(QQ).Filtered().base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).Filtered()
>>> C
Category of filtered algebras over Rational Field
>>> C.base_category()
Category of algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of filtered vector spaces over Rational Field]
```

(continues on next page)

(continued from previous page)

```
>>> AlgebrasWithBasis(QQ).Filtered().base_ring()
Rational Field
>>> HopfAlgebrasWithBasis(QQ).Filtered().base_ring()
Rational Field
```

4.48 Filtered Modules With Basis

A *filtered module with basis* over a ring R means (for the purpose of this code) a filtered R-module M with filtration $(F_i)_{i \in I}$ (typically $I = \mathbf{N}$) endowed with a basis $(b_j)_{j \in J}$ of M and a partition $J = \bigsqcup_{i \in I} J_i$ of the set J (it is allowed that some J_i are empty) such that for every $n \in I$, the subfamily $(b_j)_{j \in U_n}$, where $U_n = \bigcup_{i \leq n} J_i$, is a basis of the R-submodule F_n.

For every $i \in I$, the R-submodule of M spanned by $(b_j)_{j \in J_i}$ is called the *i-th graded component* (aka the *i-th homogeneous component*) of the filtered module with basis M; the elements of this submodule are referred to as *homogeneous elements of degree i*.

See the class documentation `FilteredModulesWithBasis` for further details.

class sage.categories.filtered_modules_with_basis.**FilteredModulesWithBasis**(*base_category*)

Bases: `FilteredModulesCategory`

The category of filtered modules with a distinguished basis.

A *filtered module with basis* over a ring R means (for the purpose of this code) a filtered R-module M with filtration $(F_i)_{i \in I}$ (typically $I = \mathbf{N}$) endowed with a basis $(b_j)_{j \in J}$ of M and a partition $J = \bigsqcup_{i \in I} J_i$ of the set J (it is allowed that some J_i are empty) such that for every $n \in I$, the subfamily $(b_j)_{j \in U_n}$, where $U_n = \bigcup_{i \leq n} J_i$, is a basis of the R-submodule F_n.

For every $i \in I$, the R-submodule of M spanned by $(b_j)_{j \in J_i}$ is called the *i-th graded component* (aka the *i-th homogeneous component*) of the filtered module with basis M; the elements of this submodule are referred to as *homogeneous elements of degree i*. The R-module M is the direct sum of its i-th graded components over all $i \in I$, and thus becomes a graded R-module with basis. Conversely, any graded R-module with basis canonically becomes a filtered R-module with basis (by defining $F_n = \bigoplus_{i \leq n} G_i$ where G_i is the i-th graded component, and defining J_i as the indexing set of the basis of the i-th graded component). Hence, the notion of a filtered R-module with basis is equivalent to the notion of a graded R-module with basis.

However, the *category* of filtered R-modules with basis is not the category of graded R-modules with basis. Indeed, the *morphisms* of filtered R-modules with basis are defined to be morphisms of R-modules which send each F_n of the domain to the corresponding F_n of the target; in contrast, the morphisms of graded R-modules with basis must preserve each homogeneous component. Also, the notion of a filtered algebra with basis differs from that of a graded algebra with basis.

Note

Currently, to make use of the functionality of this class, an instance of `FilteredModulesWithBasis` should fulfill the contract of a `CombinatorialFreeModule` (most likely by inheriting from it). It should also have the indexing set J encoded as its `_indices` attribute, and `_indices.subset(size=i)` should yield the subset J_i (as an iterable). If the latter conditions are not satisfied, then `basis()` must be overridden.

Note

One should implement a `degree_on_basis` method in the parent class in order to fully utilize the methods of this category. This might become a required abstract method in the future.

EXAMPLES:

```
sage: C = ModulesWithBasis(ZZ).Filtered(); C
Category of filtered modules with basis over Integer Ring
sage: sorted(C.super_categories(), key=str)
[Category of filtered modules over Integer Ring,
 Category of modules with basis over Integer Ring]
sage: C is ModulesWithBasis(ZZ).Filtered()
True
```

```
>>> from sage.all import *
>>> C = ModulesWithBasis(ZZ).Filtered(); C
Category of filtered modules with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of filtered modules over Integer Ring,
 Category of modules with basis over Integer Ring]
>>> C is ModulesWithBasis(ZZ).Filtered()
True
```

class ElementMethods

Bases: `object`

degree()

alias of *homogeneous_degree()*.

degree_on_basis(*m*)

Return the degree of the basis element indexed by `m` in `self`.

EXAMPLES:

```
sage: A = GradedModulesWithBasis(QQ).example()                              #␣
↪needs sage.combinat sage.modules
sage: A.degree_on_basis(Partition((2,1)))                                   #␣
↪needs sage.combinat sage.modules
3
sage: A.degree_on_basis(Partition((4,2,1,1,1,1)))                           #␣
↪needs sage.combinat sage.modules
10
```

```
>>> from sage.all import *
>>> A = GradedModulesWithBasis(QQ).example()                                #␣
↪needs sage.combinat sage.modules
>>> A.degree_on_basis(Partition((Integer(2),Integer(1))))                    ␣
↪              # needs sage.combinat sage.modules
3
>>> A.degree_on_basis(Partition((Integer(4),Integer(2),Integer(1),
↪Integer(1),Integer(1),Integer(1))))                          # needs sage.
↪combinat sage.modules
10
```

homogeneous_component(*n*)

Return the homogeneous component of degree `n` of the element `self`.

Let m be an element of a filtered R-module M with basis. Then, m can be uniquely written in the form $m = \sum_{i \in I} m_i$, where each m_i is a homogeneous element of degree i. For $n \in I$, we define the homogeneous component of degree n of the element m to be m_n.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
sage: x.homogeneous_component(-1)
0
sage: x.homogeneous_component(0)
2*P[]
sage: x.homogeneous_component(1)
2*P[1]
sage: x.homogeneous_component(2)
3*P[2]
sage: x.homogeneous_component(3)
0

sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Graded().example()
sage: x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
sage: x.homogeneous_component(-1)
0
sage: x.homogeneous_component(0)
2*P[]
sage: x.homogeneous_component(1)
2*P[1]
sage: x.homogeneous_component(2)
3*P[2]
sage: x.homogeneous_component(3)
0

sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: G = A.algebra_generators()
sage: g = A.an_element() - 2 * G['x'] * G['y']; g
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
sage: g.homogeneous_component(-1)
0
sage: g.homogeneous_component(0)
1
sage: g.homogeneous_component(2)
-2*U['x']*U['y']
sage: g.homogeneous_component(5)
0
sage: g.homogeneous_component(7)
U['x']^2*U['y']^2*U['z']^3
```

(continues on next page)

(continued from previous page)

```
sage: g.homogeneous_component(8)
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
>>> x.homogeneous_component(-Integer(1))
0
>>> x.homogeneous_component(Integer(0))
2*P[]
>>> x.homogeneous_component(Integer(1))
2*P[1]
>>> x.homogeneous_component(Integer(2))
3*P[2]
>>> x.homogeneous_component(Integer(3))
0

>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Graded().example()
>>> x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
>>> x.homogeneous_component(-Integer(1))
0
>>> x.homogeneous_component(Integer(0))
2*P[]
>>> x.homogeneous_component(Integer(1))
2*P[1]
>>> x.homogeneous_component(Integer(2))
3*P[2]
>>> x.homogeneous_component(Integer(3))
0

>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> G = A.algebra_generators()
>>> g = A.an_element() - Integer(2) * G['x'] * G['y']; g
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
>>> g.homogeneous_component(-Integer(1))
0
>>> g.homogeneous_component(Integer(0))
1
>>> g.homogeneous_component(Integer(2))
-2*U['x']*U['y']
>>> g.homogeneous_component(Integer(5))
0
>>> g.homogeneous_component(Integer(7))
U['x']^2*U['y']^2*U['z']^3
>>> g.homogeneous_component(Integer(8))
0
```

homogeneous_degree()

The degree of a nonzero homogeneous element `self` in the filtered module.

Note

This raises an error if the element is not homogeneous. To compute the maximum of the degrees of the homogeneous summands of a (not necessarily homogeneous) element, use *maximal_degree()* instead.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: x = A(Partition((3,2,1)))
sage: y = A(Partition((4,4,1)))
sage: z = A(Partition((2,2,2)))
sage: x.degree()
6
sage: (x + 2*z).degree()
6
sage: (y - x).degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> x = A(Partition((Integer(3),Integer(2),Integer(1))))
>>> y = A(Partition((Integer(4),Integer(4),Integer(1))))
>>> z = A(Partition((Integer(2),Integer(2),Integer(2))))
>>> x.degree()
6
>>> (x + Integer(2)*z).degree()
6
>>> (y - x).degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

An example in a graded algebra:

```
sage: # needs sage.combinat sage.modules
sage: S = NonCommutativeSymmetricFunctions(QQ).S()
sage: x, y = S[2], S[3]
sage: x.homogeneous_degree()
2
sage: (x^3 + 4*y^2).homogeneous_degree()
6
sage: ((1 + x)^3).homogeneous_degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> S = NonCommutativeSymmetricFunctions(QQ).S()
>>> x, y = S[Integer(2)], S[Integer(3)]
>>> x.homogeneous_degree()
2
>>> (x**Integer(3) + Integer(4)*y**Integer(2)).homogeneous_degree()
6
>>> ((Integer(1) + x)**Integer(3)).homogeneous_degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

Let us now test a filtered algebra (but remember that the notion of homogeneity now depends on the choice of a basis):

```
sage: A = AlgebrasWithBasis(QQ).Filtered().example()
sage: x,y,z = A.algebra_generators()
sage: (x*y).homogeneous_degree()
2
sage: (y*x).homogeneous_degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
sage: A.one().homogeneous_degree()
0
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).Filtered().example()
>>> x,y,z = A.algebra_generators()
>>> (x*y).homogeneous_degree()
2
>>> (y*x).homogeneous_degree()
Traceback (most recent call last):
...
ValueError: element is not homogeneous
>>> A.one().homogeneous_degree()
0
```

is_homogeneous()

Return whether the element `self` is homogeneous.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: x = A(Partition((3,2,1)))
sage: y = A(Partition((4,4,1)))
sage: z = A(Partition((2,2,2)))
sage: (3*x).is_homogeneous()
True
sage: (x - y).is_homogeneous()
False
```

(continues on next page)

(continued from previous page)

```
sage: (x+2*z).is_homogeneous()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> x = A(Partition((Integer(3),Integer(2),Integer(1))))
>>> y = A(Partition((Integer(4),Integer(4),Integer(1))))
>>> z = A(Partition((Integer(2),Integer(2),Integer(2))))
>>> (Integer(3)*x).is_homogeneous()
True
>>> (x - y).is_homogeneous()
False
>>> (x+Integer(2)*z).is_homogeneous()
True
```

Here is an example with a graded algebra:

```
sage: # needs sage.combinat sage.modules
sage: S = NonCommutativeSymmetricFunctions(QQ).S()
sage: x, y = S[2], S[3]
sage: (3*x).is_homogeneous()
True
sage: (x^3 - y^2).is_homogeneous()
True
sage: ((x + y)^2).is_homogeneous()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> S = NonCommutativeSymmetricFunctions(QQ).S()
>>> x, y = S[Integer(2)], S[Integer(3)]
>>> (Integer(3)*x).is_homogeneous()
True
>>> (x**Integer(3) - y**Integer(2)).is_homogeneous()
True
>>> ((x + y)**Integer(2)).is_homogeneous()
False
```

Let us now test a filtered algebra (but remember that the notion of homogeneity now depends on the choice of a basis, or at least on a definition of homogeneous components):

```
sage: A = AlgebrasWithBasis(QQ).Filtered().example()
sage: x,y,z = A.algebra_generators()
sage: (x*y).is_homogeneous()
True
sage: (y*x).is_homogeneous()
False
sage: A.one().is_homogeneous()
True
sage: A.zero().is_homogeneous()
True
```

(continues on next page)

(continued from previous page)

```
sage: (A.one()+x).is_homogeneous()
False
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).Filtered().example()
>>> x,y,z = A.algebra_generators()
>>> (x*y).is_homogeneous()
True
>>> (y*x).is_homogeneous()
False
>>> A.one().is_homogeneous()
True
>>> A.zero().is_homogeneous()
True
>>> (A.one()+x).is_homogeneous()
False
```

maximal_degree()

The maximum of the degrees of the homogeneous components of `self`.

This is also the smallest i such that `self` belongs to F_i. Hence, it does not depend on the basis of the parent of `self`.

See also

homogeneous_degree()

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: x = A(Partition((3,2,1)))
sage: y = A(Partition((4,4,1)))
sage: z = A(Partition((2,2,2)))
sage: x.maximal_degree()
6
sage: (x + 2*z).maximal_degree()
6
sage: (y - x).maximal_degree()
9
sage: (3*z).maximal_degree()
6
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> x = A(Partition((Integer(3),Integer(2),Integer(1))))
>>> y = A(Partition((Integer(4),Integer(4),Integer(1))))
>>> z = A(Partition((Integer(2),Integer(2),Integer(2))))
>>> x.maximal_degree()
6
```

(continues on next page)

(continued from previous page)

```
>>> (x + Integer(2)*z).maximal_degree()
6
>>> (y - x).maximal_degree()
9
>>> (Integer(3)*z).maximal_degree()
6
```

Now, we test this on a graded algebra:

```
sage: # needs sage.combinat sage.modules
sage: S = NonCommutativeSymmetricFunctions(QQ).S()
sage: x, y = S[2], S[3]
sage: x.maximal_degree()
2
sage: (x^3 + 4*y^2).maximal_degree()
6
sage: ((1 + x)^3).maximal_degree()
6
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> S = NonCommutativeSymmetricFunctions(QQ).S()
>>> x, y = S[Integer(2)], S[Integer(3)]
>>> x.maximal_degree()
2
>>> (x**Integer(3) + Integer(4)*y**Integer(2)).maximal_degree()
6
>>> ((Integer(1) + x)**Integer(3)).maximal_degree()
6
```

Let us now test a filtered algebra:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).Filtered().example()
sage: x,y,z = A.algebra_generators()
sage: (x*y).maximal_degree()
2
sage: (y*x).maximal_degree()
2
sage: A.one().maximal_degree()
0
sage: A.zero().maximal_degree()
Traceback (most recent call last):
...
ValueError: the zero element does not have a well-defined degree
sage: (A.one()+x).maximal_degree()
1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).Filtered().example()
>>> x,y,z = A.algebra_generators()
```

(continues on next page)

(continued from previous page)

```
>>> (x*y).maximal_degree()
2
>>> (y*x).maximal_degree()
2
>>> A.one().maximal_degree()
0
>>> A.zero().maximal_degree()
Traceback (most recent call last):
...
ValueError: the zero element does not have a well-defined degree
>>> (A.one()+x).maximal_degree()
1
```

truncate(*n*)

Return the sum of the homogeneous components of degree strictly less than `n` of `self`.

See *homogeneous_component()* for the notion of a homogeneous component.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
sage: x.truncate(0)
0
sage: x.truncate(1)
2*P[]
sage: x.truncate(2)
2*P[] + 2*P[1]
sage: x.truncate(3)
2*P[] + 2*P[1] + 3*P[2]

sage: # needs sage.combinat sage.modules
sage: A = ModulesWithBasis(ZZ).Graded().example()
sage: x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
sage: x.truncate(0)
0
sage: x.truncate(1)
2*P[]
sage: x.truncate(2)
2*P[] + 2*P[1]
sage: x.truncate(3)
2*P[] + 2*P[1] + 3*P[2]

sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: G = A.algebra_generators()
sage: g = A.an_element() - 2 * G['x'] * G['y']; g
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
sage: g.truncate(-1)
0
```

(continues on next page)

(continued from previous page)

```
sage: g.truncate(0)
0
sage: g.truncate(2)
2*U['x'] + 3*U['y'] + 1
sage: g.truncate(3)
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
sage: g.truncate(5)
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
sage: g.truncate(7)
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
sage: g.truncate(8)
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
>>> x.truncate(Integer(0))
0
>>> x.truncate(Integer(1))
2*P[]
>>> x.truncate(Integer(2))
2*P[] + 2*P[1]
>>> x.truncate(Integer(3))
2*P[] + 2*P[1] + 3*P[2]

>>> # needs sage.combinat sage.modules
>>> A = ModulesWithBasis(ZZ).Graded().example()
>>> x = A.an_element(); x
2*P[] + 2*P[1] + 3*P[2]
>>> x.truncate(Integer(0))
0
>>> x.truncate(Integer(1))
2*P[]
>>> x.truncate(Integer(2))
2*P[] + 2*P[1]
>>> x.truncate(Integer(3))
2*P[] + 2*P[1] + 3*P[2]

>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> G = A.algebra_generators()
>>> g = A.an_element() - Integer(2) * G['x'] * G['y']; g
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
>>> g.truncate(-Integer(1))
0
>>> g.truncate(Integer(0))
0
>>> g.truncate(Integer(2))
2*U['x'] + 3*U['y'] + 1
```

(continues on next page)

(continued from previous page)

```
>>> g.truncate(Integer(3))
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
>>> g.truncate(Integer(5))
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
>>> g.truncate(Integer(7))
-2*U['x']*U['y'] + 2*U['x'] + 3*U['y'] + 1
>>> g.truncate(Integer(8))
U['x']^2*U['y']^2*U['z']^3 - 2*U['x']*U['y']
 + 2*U['x'] + 3*U['y'] + 1
```

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: object

hilbert_series(*prec=None*)

Return the Hilbert series of `self` as a polynomial.

Let R be a commutative ring (with unit). Let $M = \bigcup_{n=0}^{\infty} M_n$ be a filtered R-module. The *Hilbert series* of M is the formal power series

$$H(t) = \sum_{n=0}^{\infty} \ell(M_n/M_{n-1}) t^n,$$

where $\ell(N)$ is the *length* of N, which is the longest chain of submodules (over R), and by convention $M_{-1} = \{0\}$. By the assumptions of the category, M_n/M_{n-1} is a free R-module, and so $\ell(M_n/M_{n-1})$ is equal to the rank of M_n/M_{n-1}.

EXAMPLES:

```
sage: OS = hyperplane_arrangements.braid(3).orlik_solomon_algebra(QQ)
sage: OS.hilbert_series()
2*t^2 + 3*t + 1

sage: OS = matroids.Uniform(5, 3).orlik_solomon_algebra(ZZ)
sage: OS.hilbert_series()
t^3 + 3*t^2 + 3*t + 1

sage: OS = matroids.PG(2, 3).orlik_solomon_algebra(ZZ['x','y'])
sage: OS.hilbert_series()
27*t^3 + 39*t^2 + 13*t + 1
```

```
>>> from sage.all import *
>>> OS = hyperplane_arrangements.braid(Integer(3)).orlik_solomon_
↪algebra(QQ)
>>> OS.hilbert_series()
2*t^2 + 3*t + 1

>>> OS = matroids.Uniform(Integer(5), Integer(3)).orlik_solomon_
↪algebra(ZZ)
>>> OS.hilbert_series()
t^3 + 3*t^2 + 3*t + 1
```

(continues on next page)

(continued from previous page)

```
>>> OS = matroids.PG(Integer(2), Integer(3)).orlik_solomon_algebra(ZZ[
↪'x','y'])
>>> OS.hilbert_series()
27*t^3 + 39*t^2 + 13*t + 1
```

class ParentMethods

Bases: object

basis(*d=None*)

Return the basis for (the d-th homogeneous component of) self.

INPUT:

- d – (default: None) nonnegative integer or None

OUTPUT:

If d is None, returns the basis of the module. Otherwise, returns the basis of the homogeneous component of degree d (i.e., the subfamily of the basis of the whole module which consists only of the basis vectors lying in $F_d \setminus \bigcup_{i<d} F_i$).

The basis is always returned as a family.

EXAMPLES:

```
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: A.basis(4)
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions of the integer 4}
```

```
>>> from sage.all import *
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> A.basis(Integer(4))
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions of the integer 4}
```

Without arguments, the full basis is returned:

```
sage: A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
sage: A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
```

```
>>> from sage.all import *
>>> A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
>>> A.basis()
```

(continues on next page)

(continued from previous page)

```
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
```

Checking this method on a filtered algebra. Note that this will typically raise a `NotImplementedError` when this feature is not implemented.

```
sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: A.basis(4)
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> A.basis(Integer(4))
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

Without arguments, the full basis is returned:

```
sage: A.basis()
Lazy family (Term map from Free abelian monoid indexed by
 {'x', 'y', 'z'} to An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Integer Ring(i))_{i in
 Free abelian monoid indexed by {'x', 'y', 'z'}}
```

```
>>> from sage.all import *
>>> A.basis()
Lazy family (Term map from Free abelian monoid indexed by
 {'x', 'y', 'z'} to An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Integer Ring(i))_{i in
 Free abelian monoid indexed by {'x', 'y', 'z'}}
```

An example with a graded algebra:

```
sage: E.<x,y> = ExteriorAlgebra(QQ)
sage: E.basis()
Lazy family (Term map from Subsets of {0,1} to
 The exterior algebra of rank 2 over Rational Field(i))_{i in
 Subsets of {0,1}}
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = E._first_
↪ngens(2)
>>> E.basis()
Lazy family (Term map from Subsets of {0,1} to
 The exterior algebra of rank 2 over Rational Field(i))_{i in
 Subsets of {0,1}}
```

from_graded_conversion()

Return the inverse of the canonical R-module isomorphism $A \to \operatorname{gr} A$ induced by the basis of A (where $A = \text{```}self\text{`}$). This inverse is an isomorphism $\operatorname{gr} A \to A$.

This is an isomorphism of R-modules. See the class documentation AssociatedGradedAlgebra.

See also

to_graded_conversion()

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = Modules(QQ).WithBasis().Filtered().example()
sage: p = -2 * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
sage: q = A.to_graded_conversion()(p); q
-4*Bbar[[]] - 4*Bbar[[1]] - 6*Bbar[[2]]
sage: A.from_graded_conversion()(q) == p
True
sage: q.parent() is A.graded_algebra()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = Modules(QQ).WithBasis().Filtered().example()
>>> p = -Integer(2) * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
>>> q = A.to_graded_conversion()(p); q
-4*Bbar[[]] - 4*Bbar[[1]] - 6*Bbar[[2]]
>>> A.from_graded_conversion()(q) == p
True
>>> q.parent() is A.graded_algebra()
True
```

graded_algebra()

Return the associated graded module to `self`.

See AssociatedGradedAlgebra for the definition and the properties of this.

If the filtered module `self` with basis is called A, then this method returns $\operatorname{gr} A$. The method *to_graded_conversion()* returns the canonical R-module isomorphism $A \to \operatorname{gr} A$ induced by the basis of A, and the method *from_graded_conversion()* returns the inverse of this isomorphism. The method *projection()* projects elements of A onto $\operatorname{gr} A$ according to their place in the filtration on A.

Warning

When not overridden, this method returns the default implementation of an associated graded module – namely, `AssociatedGradedAlgebra(self)`, where `AssociatedGradedAlgebra` is AssociatedGradedAlgebra. But some instances of *FilteredModulesWithBasis* override this method, as the associated graded module often is (isomorphic) to a simpler object (for instance, the associated graded module of a graded module can be identified with the graded

module itself). Generic code that uses associated graded modules (such as the code of the *induced_graded_map()* method below) should make sure to only communicate with them via the *to_graded_conversion()*, *from_graded_conversion()* and *projection()* methods (in particular, do not expect there to be a conversion from `self` to `self.graded_algebra()`; this currently does not work for Clifford algebras). Similarly, when overriding *graded_algebra()*, make sure to accordingly redefine these three methods, unless their definitions below still apply to your case (this will happen whenever the basis of your *graded_algebra()* has the same indexing set as `self`, and the partition of this indexing set according to degree is the same as for `self`).

EXAMPLES:

```
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: A.graded_algebra()
Graded Module of An example of a filtered module with basis:
 the free module on partitions over Integer Ring
```

```
>>> from sage.all import *
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> A.graded_algebra()
Graded Module of An example of a filtered module with basis:
 the free module on partitions over Integer Ring
```

hilbert_series(*prec=None*)

Return the Hilbert series of `self`.

Let R be a commutative ring (with unit). Let $M = \bigcup_{n=0}^{\infty} M_n$ be a filtered R-module. The *Hilbert series* of M is the formal power series

$$H(t) = \sum_{n=0}^{\infty} \ell(M_n/M_{n-1}) t^n,$$

where $\ell(N)$ is the *length* of N, which is the longest chain of submodules (over R), and by convention $M_{-1} = \{0\}$. By the assumptions of the category, M_n/M_{n-1} is a free R-module, and so $\ell(M_n/M_{n-1})$ is equal to the rank of M_n/M_{n-1}.

INPUT:

- `prec` – (default: ∞) the precision

OUTPUT:

If the precision is finite, then this returns an element in the *PowerSeriesRing()* over `ZZ`. Otherwise it returns an element in the *LazyPowerSeriesRing* over `ZZ`.

EXAMPLES:

```
sage: A = GradedModulesWithBasis(ZZ).example()
sage: A.hilbert_series()
1 + t + 2*t^2 + 3*t^3 + 5*t^4 + 7*t^5 + 11*t^6 + O(t^7)
sage: A.hilbert_series(10)
1 + t + 2*t^2 + 3*t^3 + 5*t^4 + 7*t^5 + 11*t^6 + 15*t^7 + 22*t^8 + 30*t^9␣
↪+ O(t^10)
```

```
>>> from sage.all import *
>>> A = GradedModulesWithBasis(ZZ).example()
>>> A.hilbert_series()
```

(continues on next page)

(continued from previous page)

```
1 + t + 2*t^2 + 3*t^3 + 5*t^4 + 7*t^5 + 11*t^6 + O(t^7)
>>> A.hilbert_series(Integer(10))
1 + t + 2*t^2 + 3*t^3 + 5*t^4 + 7*t^5 + 11*t^6 + 15*t^7 + 22*t^8 + 30*t^9␣
↪+ O(t^10)
```

homogeneous_component(*d*)

Return the `d`-th homogeneous component of `self`.

EXAMPLES:

```
sage: A = GradedModulesWithBasis(ZZ).example()
sage: A.homogeneous_component(4)
Degree 4 homogeneous component of An example of a graded module
 with basis: the free module on partitions over Integer Ring
```

```
>>> from sage.all import *
>>> A = GradedModulesWithBasis(ZZ).example()
>>> A.homogeneous_component(Integer(4))
Degree 4 homogeneous component of An example of a graded module
 with basis: the free module on partitions over Integer Ring
```

homogeneous_component_basis(*d*)

Return a basis for the `d`-th homogeneous component of `self`.

EXAMPLES:

```
sage: A = GradedModulesWithBasis(ZZ).example()
sage: A.homogeneous_component_basis(4)
Lazy family (Term map
 from Partitions
   to An example of a graded module with basis: the free module
      on partitions over Integer Ring(i))_{i in Partitions of the integer␣
↪4}

sage: # needs sage.modules
sage: cat = GradedModulesWithBasis(ZZ)
sage: C = CombinatorialFreeModule(ZZ, ['a', 'b'], category=cat)
sage: C.degree_on_basis = lambda x: 1 if x == 'a' else 2
sage: C.homogeneous_component_basis(1)
Finite family {'a': B['a']}
sage: C.homogeneous_component_basis(2)
Finite family {'b': B['b']}
```

```
>>> from sage.all import *
>>> A = GradedModulesWithBasis(ZZ).example()
>>> A.homogeneous_component_basis(Integer(4))
Lazy family (Term map
 from Partitions
   to An example of a graded module with basis: the free module
      on partitions over Integer Ring(i))_{i in Partitions of the integer␣
↪4}
```

(continues on next page)

(continued from previous page)

```
>>> # needs sage.modules
>>> cat = GradedModulesWithBasis(ZZ)
>>> C = CombinatorialFreeModule(ZZ, ['a', 'b'], category=cat)
>>> C.degree_on_basis = lambda x: Integer(1) if x == 'a' else Integer(2)
>>> C.homogeneous_component_basis(Integer(1))
Finite family {'a': B['a']}
>>> C.homogeneous_component_basis(Integer(2))
Finite family {'b': B['b']}
```

induced_graded_map(*other*, *f*)

Return the graded linear map between the associated graded modules of `self` and `other` canonically induced by the filtration-preserving map `f : self -> other`.

Let A and B be two filtered modules with basis, and let $(F_i)_{i \in I}$ and $(G_i)_{i \in I}$ be their filtrations. Let $f : A \to B$ be a linear map which preserves the filtration (i.e., satisfies $f(F_i) \subseteq G_i$ for all $i \in I$). Then, there is a canonically defined graded linear map $\operatorname{gr} f : \operatorname{gr} A \to \operatorname{gr} B$ which satisfies

$$(\operatorname{gr} f)(p_i(a)) = p_i(f(a)) \qquad \text{for all } i \in I \text{ and } a \in F_i,$$

where the p_i on the left hand side is the canonical projection from F_i onto the i-th graded component of $\operatorname{gr} A$, while the p_i on the right hand side is the canonical projection from G_i onto the i-th graded component of $\operatorname{gr} B$.

INPUT:

- `other` – a filtered algebra with basis
- `f` – a filtration-preserving linear map from `self` to `other` (can be given as a morphism or as a function)

OUTPUT: the graded linear map $\operatorname{gr} f$

EXAMPLES:

Example 1.

We start with the free **Q**-module with basis the set of all partitions:

```
sage: A = Modules(QQ).WithBasis().Filtered().example(); A                #␣
↪needs sage.combinat sage.modules
An example of a filtered module with basis: the free module
 on partitions over Rational Field
sage: M = A.indices(); M                                                 #␣
↪needs sage.combinat sage.modules
Partitions
sage: p1, p2, p21, p321 = [A.basis()[Partition(i)]                       #␣
↪needs sage.combinat sage.modules
....:                      for i in [[1], [2], [2,1], [3,2,1]]]
```

```
>>> from sage.all import *
>>> A = Modules(QQ).WithBasis().Filtered().example(); A                  #␣
↪needs sage.combinat sage.modules
An example of a filtered module with basis: the free module
 on partitions over Rational Field
>>> M = A.indices(); M                                                  #␣
↪needs sage.combinat sage.modules
Partitions
```

(continues on next page)

(continued from previous page)

```
>>> p1, p2, p21, p321 = [A.basis()[Partition(i)]                                #␣
↪needs sage.combinat sage.modules
...                     for i in [[Integer(1)], [Integer(2)],␣
↪[Integer(2),Integer(1)], [Integer(3),Integer(2),Integer(1)]]]
```

Let us define a map from `A` to itself which acts on the basis by sending every partition λ to the sum of the conjugates of all partitions μ for which λ/μ is a horizontal strip:

```
sage: # needs sage.combinat sage.modules
sage: def map_on_basis(lam):
....:     def mus(k):
....:         return lam.remove_horizontal_border_strip(k)
....:     return A.sum_of_monomials([Partition(mu).conjugate()
....:                                for k in range(sum(lam) + 1)
....:                                for mu in mus(k)])
sage: f = A.module_morphism(on_basis=map_on_basis,
....:                       codomain=A)
sage: f(p1)
P[] + P[1]
sage: f(p2)
P[] + P[1] + P[1, 1]
sage: f(p21)
P[1] + P[1, 1] + P[2] + P[2, 1]
sage: f(p21 - p1)
-P[] + P[1, 1] + P[2] + P[2, 1]
sage: f(p321)
P[2, 1] + P[2, 1, 1] + P[2, 2] + P[2, 2, 1]
 + P[3, 1] + P[3, 1, 1] + P[3, 2] + P[3, 2, 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> def map_on_basis(lam):
...     def mus(k):
...         return lam.remove_horizontal_border_strip(k)
...     return A.sum_of_monomials([Partition(mu).conjugate()
...                                for k in range(sum(lam) + Integer(1))
...                                for mu in mus(k)])
>>> f = A.module_morphism(on_basis=map_on_basis,
...                       codomain=A)
>>> f(p1)
P[] + P[1]
>>> f(p2)
P[] + P[1] + P[1, 1]
>>> f(p21)
P[1] + P[1, 1] + P[2] + P[2, 1]
>>> f(p21 - p1)
-P[] + P[1, 1] + P[2] + P[2, 1]
>>> f(p321)
P[2, 1] + P[2, 1, 1] + P[2, 2] + P[2, 2, 1]
 + P[3, 1] + P[3, 1, 1] + P[3, 2] + P[3, 2, 1]
```

We now compute gr f

```
sage: # needs sage.combinat sage.modules
sage: grA = A.graded_algebra(); grA
Graded Module of An example of a filtered module with basis:
 the free module on partitions over Rational Field
sage: pp1, pp2, pp21, pp321 = [A.to_graded_conversion()(i)
....:                          for i in [p1, p2, p21, p321]]
sage: pp2 + 4 * pp21
Bbar[[2]] + 4*Bbar[[2, 1]]
sage: grf = A.induced_graded_map(A, f); grf
Generic endomorphism of Graded Module of
 An example of a filtered module with basis:
  the free module on partitions over Rational Field
sage: grf(pp1)
Bbar[[1]]
sage: grf(pp2 + 4 * pp21)
Bbar[[1, 1]] + 4*Bbar[[2, 1]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> grA = A.graded_algebra(); grA
Graded Module of An example of a filtered module with basis:
 the free module on partitions over Rational Field
>>> pp1, pp2, pp21, pp321 = [A.to_graded_conversion()(i)
...                          for i in [p1, p2, p21, p321]]
>>> pp2 + Integer(4) * pp21
Bbar[[2]] + 4*Bbar[[2, 1]]
>>> grf = A.induced_graded_map(A, f); grf
Generic endomorphism of Graded Module of
 An example of a filtered module with basis:
  the free module on partitions over Rational Field
>>> grf(pp1)
Bbar[[1]]
>>> grf(pp2 + Integer(4) * pp21)
Bbar[[1, 1]] + 4*Bbar[[2, 1]]
```

Example 2.

We shall now construct gr f for a different map f out of the same `A`; the new map f will lead into a graded algebra already, namely into the algebra of symmetric functions:

```
sage: # needs sage.combinat sage.modules
sage: h = SymmetricFunctions(QQ).h()
sage: def map_on_basis(lam):  # redefining map_on_basis
....:     def mus(k):
....:         return lam.remove_horizontal_border_strip(k)
....:     return h.sum_of_monomials([Partition(mu).conjugate()
....:                                for k in range(sum(lam) + 1)
....:                                for mu in mus(k)])
sage: f = A.module_morphism(on_basis=map_on_basis,
....:                       codomain=h)  # redefining f
sage: f(p1)
h[] + h[1]
sage: f(p2)
```

(continues on next page)

(continued from previous page)

```
h[] + h[1] + h[1, 1]
sage: f(A.zero())
0
sage: f(p2 - 3*p1)
-2*h[] - 2*h[1] + h[1, 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> h = SymmetricFunctions(QQ).h()
>>> def map_on_basis(lam):  # redefining map_on_basis
...     def mus(k):
...         return lam.remove_horizontal_border_strip(k)
...     return h.sum_of_monomials([Partition(mu).conjugate()
...                                for k in range(sum(lam) + Integer(1))
...                                for mu in mus(k)])
>>> f = A.module_morphism(on_basis=map_on_basis,
...                       codomain=h)  # redefining f
>>> f(p1)
h[] + h[1]
>>> f(p2)
h[] + h[1] + h[1, 1]
>>> f(A.zero())
0
>>> f(p2 - Integer(3)*p1)
-2*h[] - 2*h[1] + h[1, 1]
```

The algebra `h` of symmetric functions in the h-basis is already graded, so its associated graded algebra is implemented as itself:

```
sage: # needs sage.combinat sage.modules
sage: grh = h.graded_algebra(); grh is h
True
sage: grf = A.induced_graded_map(h, f); grf
Generic morphism:
  From: Graded Module of An example of a filtered
   module with basis: the free module on partitions
   over Rational Field
  To:   Symmetric Functions over Rational Field
   in the homogeneous basis
sage: grf(pp1)
h[1]
sage: grf(pp2)
h[1, 1]
sage: grf(pp321)
h[3, 2, 1]
sage: grf(pp2 - 3*pp1)
-3*h[1] + h[1, 1]
sage: grf(pp21)
h[2, 1]
sage: grf(grA.zero())
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> grh = h.graded_algebra(); grh is h
True
>>> grf = A.induced_graded_map(h, f); grf
Generic morphism:
  From: Graded Module of An example of a filtered
   module with basis: the free module on partitions
   over Rational Field
  To:   Symmetric Functions over Rational Field
   in the homogeneous basis
>>> grf(pp1)
h[1]
>>> grf(pp2)
h[1, 1]
>>> grf(pp321)
h[3, 2, 1]
>>> grf(pp2 - Integer(3)*pp1)
-3*h[1] + h[1, 1]
>>> grf(pp21)
h[2, 1]
>>> grf(grA.zero())
0
```

Example 3.

After having had a graded module as the codomain, let us try to have one as the domain instead. Our new `f` will go from `h` to `A`:

```
sage: # needs sage.combinat sage.modules
sage: def map_on_basis(lam):  # redefining map_on_basis
....:     def mus(k):
....:         return lam.remove_horizontal_border_strip(k)
....:     return A.sum_of_monomials([Partition(mu).conjugate()
....:                                for k in range(sum(lam) + 1)
....:                                for mu in mus(k)])
sage: f = h.module_morphism(on_basis=map_on_basis,
....:                       codomain=A)  # redefining f
sage: f(h[1])
P[] + P[1]
sage: f(h[2])
P[] + P[1] + P[1, 1]
sage: f(h[1, 1])
P[1] + P[2]
sage: f(h[2, 2])
P[1, 1] + P[2, 1] + P[2, 2]
sage: f(h[3, 2, 1])
P[2, 1] + P[2, 1, 1] + P[2, 2] + P[2, 2, 1]
 + P[3, 1] + P[3, 1, 1] + P[3, 2] + P[3, 2, 1]
sage: f(h.one())
P[]
sage: grf = h.induced_graded_map(A, f); grf
Generic morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Symmetric Functions over Rational Field
   in the homogeneous basis
  To:   Graded Module of An example of a filtered
   module with basis: the free module on partitions
   over Rational Field
sage: grf(h[1])
Bbar[[1]]
sage: grf(h[2])
Bbar[[1, 1]]
sage: grf(h[1, 1])
Bbar[[2]]
sage: grf(h[2, 2])
Bbar[[2, 2]]
sage: grf(h[3, 2, 1])
Bbar[[3, 2, 1]]
sage: grf(h.one())
Bbar[[]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> def map_on_basis(lam):  # redefining map_on_basis
...     def mus(k):
...         return lam.remove_horizontal_border_strip(k)
...     return A.sum_of_monomials([Partition(mu).conjugate()
...                                for k in range(sum(lam) + Integer(1))
...                                for mu in mus(k)])
>>> f = h.module_morphism(on_basis=map_on_basis,
...                       codomain=A)  # redefining f
>>> f(h[Integer(1)])
P[] + P[1]
>>> f(h[Integer(2)])
P[] + P[1] + P[1, 1]
>>> f(h[Integer(1), Integer(1)])
P[1] + P[2]
>>> f(h[Integer(2), Integer(2)])
P[1, 1] + P[2, 1] + P[2, 2]
>>> f(h[Integer(3), Integer(2), Integer(1)])
P[2, 1] + P[2, 1, 1] + P[2, 2] + P[2, 2, 1]
 + P[3, 1] + P[3, 1, 1] + P[3, 2] + P[3, 2, 1]
>>> f(h.one())
P[]
>>> grf = h.induced_graded_map(A, f); grf
Generic morphism:
  From: Symmetric Functions over Rational Field
   in the homogeneous basis
  To:   Graded Module of An example of a filtered
   module with basis: the free module on partitions
   over Rational Field
>>> grf(h[Integer(1)])
Bbar[[1]]
>>> grf(h[Integer(2)])
Bbar[[1, 1]]
```

(continues on next page)

(continued from previous page)

```
>>> grf(h[Integer(1), Integer(1)])
Bbar[[2]]
>>> grf(h[Integer(2), Integer(2)])
Bbar[[2, 2]]
>>> grf(h[Integer(3), Integer(2), Integer(1)])
Bbar[[3, 2, 1]]
>>> grf(h.one())
Bbar[[]]
```

Example 4.

The construct gr f also makes sense when f is a filtration-preserving map between graded modules.

```
sage: # needs sage.combinat sage.modules
sage: def map_on_basis(lam):  # redefining map_on_basis
....:     def mus(k):
....:         return lam.remove_horizontal_border_strip(k)
....:     return h.sum_of_monomials([Partition(mu).conjugate()
....:                                for k in range(sum(lam) + 1)
....:                                for mu in mus(k)])
sage: f = h.module_morphism(on_basis=map_on_basis,
....:                       codomain=h)  # redefining f
sage: f(h[1])
h[] + h[1]
sage: f(h[2])
h[] + h[1] + h[1, 1]
sage: f(h[1, 1])
h[1] + h[2]
sage: f(h[2, 1])
h[1] + h[1, 1] + h[2] + h[2, 1]
sage: f(h.one())
h[]
sage: grf = h.induced_graded_map(h, f); grf
Generic endomorphism of
 Symmetric Functions over Rational Field in the homogeneous basis
sage: grf(h[1])
h[1]
sage: grf(h[2])
h[1, 1]
sage: grf(h[1, 1])
h[2]
sage: grf(h[2, 1])
h[2, 1]
sage: grf(h.one())
h[]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> def map_on_basis(lam):  # redefining map_on_basis
...     def mus(k):
...         return lam.remove_horizontal_border_strip(k)
...     return h.sum_of_monomials([Partition(mu).conjugate()
```

(continues on next page)

(continued from previous page)

```
...                                 for k in range(sum(lam) + Integer(1))
...                                 for mu in mus(k)])
>>> f = h.module_morphism(on_basis=map_on_basis,
...                       codomain=h)  # redefining f
>>> f(h[Integer(1)])
h[] + h[1]
>>> f(h[Integer(2)])
h[] + h[1] + h[1, 1]
>>> f(h[Integer(1), Integer(1)])
h[1] + h[2]
>>> f(h[Integer(2), Integer(1)])
h[1] + h[1, 1] + h[2] + h[2, 1]
>>> f(h.one())
h[]
>>> grf = h.induced_graded_map(h, f); grf
Generic endomorphism of
 Symmetric Functions over Rational Field in the homogeneous basis
>>> grf(h[Integer(1)])
h[1]
>>> grf(h[Integer(2)])
h[1, 1]
>>> grf(h[Integer(1), Integer(1)])
h[2]
>>> grf(h[Integer(2), Integer(1)])
h[2, 1]
>>> grf(h.one())
h[]
```

projection(i)

Return the i-th projection $p_i : F_i \to G_i$ (in the notations of the class documentation Associated-GradedAlgebra, where $A =$ ``$self$`).

This method actually does not return the map p_i itself, but an extension of p_i to the whole R-module A. This extension is the composition of the R-module isomorphism $A \to \mathrm{gr}\, A$ with the canonical projection of the graded R-module $\mathrm{gr}\, A$ onto its i-th graded component G_i. The codomain of this map is $\mathrm{gr}\, A$, although its actual image is G_i. The map p_i is obtained from this map by restricting its domain to F_i and its image to G_i.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = Modules(ZZ).WithBasis().Filtered().example()
sage: p = -2 * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
sage: q = A.projection(2)(p); q
-6*Bbar[[2]]
sage: q.parent() is A.graded_algebra()
True
sage: A.projection(3)(p)
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> A = Modules(ZZ).WithBasis().Filtered().example()
>>> p = -Integer(2) * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
>>> q = A.projection(Integer(2))(p); q
-6*Bbar[[2]]
>>> q.parent() is A.graded_algebra()
True
>>> A.projection(Integer(3))(p)
0
```

to_graded_conversion()

Return the canonical R-module isomorphism $A \to \mathrm{gr}\, A$ induced by the basis of A (where $A =$ ``*self*`).

This is an isomorphism of R-modules. See the class documentation AssociatedGradedAlgebra.

See also

from_graded_conversion()

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = Modules(QQ).WithBasis().Filtered().example()
sage: p = -2 * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
sage: q = A.to_graded_conversion()(p); q
-4*Bbar[[]] - 4*Bbar[[1]] - 6*Bbar[[2]]
sage: q.parent() is A.graded_algebra()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = Modules(QQ).WithBasis().Filtered().example()
>>> p = -Integer(2) * A.an_element(); p
-4*P[] - 4*P[1] - 6*P[2]
>>> q = A.to_graded_conversion()(p); q
-4*Bbar[[]] - 4*Bbar[[1]] - 6*Bbar[[2]]
>>> q.parent() is A.graded_algebra()
True
```

class Subobjects(*category*, **args*)

Bases: *SubobjectsCategory*

class ElementMethods

Bases: object

degree()

Return the degree of self.

EXAMPLES:

```
sage: # needs sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z, y])
sage: B = S.basis()
sage: [B[0].lift(), B[1].lift(), B[2].lift()]
[x, y, x*y - y*z]
sage: B[0].degree()
1
sage: B[1].degree()
1
sage: (B[0] + 3*B[1]).degree()
1
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z, y])
>>> B = S.basis()
>>> [B[Integer(0)].lift(), B[Integer(1)].lift(), B[Integer(2)].lift()]
[x, y, x*y - y*z]
>>> B[Integer(0)].degree()
1
>>> B[Integer(1)].degree()
1
>>> (B[Integer(0)] + Integer(3)*B[Integer(1)]).degree()
1
```

The degree of inhomogeneous elements is not defined (following the behavior of the exterior algebra):

```
sage: (B[0] + B[2]).degree()                                                #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

```
>>> from sage.all import *
>>> (B[Integer(0)] + B[Integer(2)]).degree()                                ␣
↪             # needs sage.modules
Traceback (most recent call last):
...
ValueError: element is not homogeneous
```

We can still get the maximal degree:

```
sage: (B[0] + B[2]).maximal_degree()                                        #␣
↪needs sage.modules
2
```

```
>>> from sage.all import *
>>> (B[Integer(0)] + B[Integer(2)]).maximal_degree()                        ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs sage.modules
2
```

maximal_degree()

The maximum of the degrees of the homogeneous components of `self`.

This is also the smallest i such that `self` belongs to F_i. Hence, it does not depend on the basis of the parent of `self`.

See also

homogeneous_degree()

EXAMPLES:

```
sage: # needs sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: F = E.submodule([x + 1, x*y - 1])
sage: B = F.basis()
sage: [B[0].lift(), B[1].lift()]
[-x*y + 1, x*y + x]
sage: B[0].maximal_degree()
2
sage: B[1].maximal_degree()
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> F = E.submodule([x + Integer(1), x*y - Integer(1)])
>>> B = F.basis()
>>> [B[Integer(0)].lift(), B[Integer(1)].lift()]
[-x*y + 1, x*y + x]
>>> B[Integer(0)].maximal_degree()
2
>>> B[Integer(1)].maximal_degree()
2
```

class ParentMethods

Bases: `object`

degree_on_basis(*m*)

Return the degree of the basis element indexed by `m` in `self`.

EXAMPLES:

```
sage: # needs sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z, y])
sage: B = S.basis()
sage: [B[0].lift(), B[1].lift(), B[2].lift()]
```

(continues on next page)

(continued from previous page)

```
[x, y, x*y - y*z]
sage: S.degree_on_basis(0)
1
sage: S.degree_on_basis(1)
1
sage: S.degree_on_basis(2)
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z, y])
>>> B = S.basis()
>>> [B[Integer(0)].lift(), B[Integer(1)].lift(), B[Integer(2)].lift()]
[x, y, x*y - y*z]
>>> S.degree_on_basis(Integer(0))
1
>>> S.degree_on_basis(Integer(1))
1
>>> S.degree_on_basis(Integer(2))
2
```

4.49 Finite Complex Reflection Groups

class sage.categories.finite_complex_reflection_groups.**FiniteComplexReflectionGroups**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite complex reflection groups.

See *ComplexReflectionGroups* for the definition of complex reflection group. In the finite case, most of the information about the group can be recovered from its *degrees* and *codegrees*, and to a lesser extent to the explicit realization as subgroup of $GL(V)$. Hence the most important optional methods to implement are:

- *degrees()*,
- *codegrees()*,
- *to_matrix()*.

Finite complex reflection groups are completely classified. In particular, if the group is irreducible, then it's uniquely determined by its degrees and codegrees and whether it's reflection representation is *primitive* or not (see [LT2009] Chapter 2.1 for the definition of primitive).

See also

Wikipedia article Complex_reflection_groups

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().Finite()
Category of finite complex reflection groups
sage: ComplexReflectionGroups().Finite().super_categories()
[Category of complex reflection groups,
 Category of finite groups,
 Category of finite finitely generated semigroups]
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import ComplexReflectionGroups
>>> ComplexReflectionGroups().Finite()
Category of finite complex reflection groups
>>> ComplexReflectionGroups().Finite().super_categories()
[Category of complex reflection groups,
 Category of finite groups,
 Category of finite finitely generated semigroups]
```

An example of a finite reflection group:

```
sage: W = ComplexReflectionGroups().Finite().example(); W        # optional - gap3
Reducible real reflection group of rank 4 and type A2 x B2

sage: W.reflections()                                            # optional - gap3
Finite family {1: (1,8)(2,5)(9,12), 2: (1,5)(2,9)(8,12),
               3: (3,10)(4,7)(11,14), 4: (3,6)(4,11)(10,13),
               5: (1,9)(2,8)(5,12), 6: (4,14)(6,13)(7,11),
               7: (3,13)(6,10)(7,14)}
```

```
>>> from sage.all import *
>>> W = ComplexReflectionGroups().Finite().example(); W        # optional - gap3
Reducible real reflection group of rank 4 and type A2 x B2

>>> W.reflections()                                            # optional - gap3
Finite family {1: (1,8)(2,5)(9,12), 2: (1,5)(2,9)(8,12),
               3: (3,10)(4,7)(11,14), 4: (3,6)(4,11)(10,13),
               5: (1,9)(2,8)(5,12), 6: (4,14)(6,13)(7,11),
               7: (3,13)(6,10)(7,14)}
```

`W` is in the category of complex reflection groups:

```
sage: W in ComplexReflectionGroups().Finite()                    # optional - gap3
True
```

```
>>> from sage.all import *
>>> W in ComplexReflectionGroups().Finite()                    # optional - gap3
True
```

class ElementMethods

Bases: `object`

character_value()

Return the value at `self` of the character of the reflection representation given by *to_matrix()*.

EXAMPLES:

```
sage: W = ColoredPermutations(1,3); W                                          #␣
↪needs sage.combinat
1-colored permutations of size 3
sage: [t.character_value() for t in W]                                         #␣
↪needs sage.combinat sage.groups
[3, 1, 1, 0, 0, 1]
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3)); W                          ␣
↪              # needs sage.combinat
1-colored permutations of size 3
>>> [t.character_value() for t in W]                                          #␣
↪needs sage.combinat sage.groups
[3, 1, 1, 0, 0, 1]
```

Note that this could be a different (faithful) representation than that given by the corresponding root system:

```
sage: W = ReflectionGroup((1,1,3)); W      # optional - gap3
Irreducible real reflection group of rank 2 and type A2
sage: [t.character_value() for t in W]     # optional - gap3
[2, 0, 0, -1, -1, 0]

sage: W = ColoredPermutations(2,2); W                                          #␣
↪needs sage.combinat
2-colored permutations of size 2
sage: [t.character_value() for t in W]                                         #␣
↪needs sage.combinat sage.groups
[2, 0, 0, -2, 0, 0, 0, 0]

sage: W = ColoredPermutations(3,1); W                                          #␣
↪needs sage.combinat
3-colored permutations of size 1
sage: [t.character_value() for t in W]                                         #␣
↪needs sage.combinat sage.groups
[1, zeta3, -zeta3 - 1]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3))); W      #␣
↪optional - gap3
Irreducible real reflection group of rank 2 and type A2
>>> [t.character_value() for t in W]     # optional - gap3
[2, 0, 0, -1, -1, 0]

>>> W = ColoredPermutations(Integer(2),Integer(2)); W                          ␣
↪              # needs sage.combinat
2-colored permutations of size 2
>>> [t.character_value() for t in W]                                          #␣
↪needs sage.combinat sage.groups
[2, 0, 0, -2, 0, 0, 0, 0]
```

(continues on next page)

(continued from previous page)

```
>>> W = ColoredPermutations(Integer(3),Integer(1)); W
↪              # needs sage.combinat
3-colored permutations of size 1
>>> [t.character_value() for t in W]                                    #␣
↪needs sage.combinat sage.groups
[1, zeta3, -zeta3 - 1]
```

reflection_length(*in_unitary_group=False*)

Return the reflection length of `self`.

This is the minimal numbers of reflections needed to obtain `self`.

INPUT:

- `in_unitary_group` – boolean (default: `False`); if `True`, the reflection length is computed in the unitary group which is the dimension of the move space of `self`

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,3))                # optional - gap3
sage: sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 2, 2]

sage: W = ReflectionGroup((2,1,2))                # optional - gap3
sage: sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 1, 2, 2, 2]

sage: W = ReflectionGroup((2,2,2))                # optional - gap3
sage: sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 2]

sage: W = ReflectionGroup((3,1,2))                # optional - gap3
sage: sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))              ␣
↪ # optional - gap3
>>> sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 2, 2]

>>> W = ReflectionGroup((Integer(2),Integer(1),Integer(2)))              ␣
↪ # optional - gap3
>>> sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 1, 2, 2, 2]

>>> W = ReflectionGroup((Integer(2),Integer(2),Integer(2)))              ␣
↪ # optional - gap3
>>> sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 2]

>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(2)))              ␣
↪ # optional - gap3
>>> sorted([t.reflection_length() for t in W])  # optional - gap3
[0, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]
```

to_matrix()

Return the matrix presentation of `self` acting on a vector space V.

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,3))          # optional - gap3
sage: [t.to_matrix() for t in W]            # optional - gap3
[
[1 0]  [ 1  1]  [-1  0]  [-1 -1]  [ 0  1]  [ 0 -1]
[0 1], [ 0 -1], [ 1  1], [ 1  0], [-1 -1], [-1  0]
]

sage: W = ColoredPermutations(1,3)                                    #␣
↪needs sage.combinat
sage: [t.to_matrix() for t in W]                                      #␣
↪needs sage.combinat sage.groups
[
[1 0 0]  [1 0 0]  [0 1 0]  [0 0 1]  [0 1 0]  [0 0 1]
[0 1 0]  [0 0 1]  [1 0 0]  [1 0 0]  [0 0 1]  [0 1 0]
[0 0 1], [0 1 0], [0 0 1], [0 1 0], [1 0 0], [1 0 0]
]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))          #␣
↪optional - gap3
>>> [t.to_matrix() for t in W]            # optional - gap3
[
[1 0]  [ 1  1]  [-1  0]  [-1 -1]  [ 0  1]  [ 0 -1]
[0 1], [ 0 -1], [ 1  1], [ 1  0], [-1 -1], [-1  0]
]

>>> W = ColoredPermutations(Integer(1),Integer(3))                     ␣
↪              # needs sage.combinat
>>> [t.to_matrix() for t in W]                                       #␣
↪needs sage.combinat sage.groups
[
[1 0 0]  [1 0 0]  [0 1 0]  [0 0 1]  [0 1 0]  [0 0 1]
[0 1 0]  [0 0 1]  [1 0 0]  [1 0 0]  [0 0 1]  [0 1 0]
[0 0 1], [0 1 0], [0 0 1], [0 1 0], [1 0 0], [1 0 0]
]
```

A different representation is given by the colored permutations:

```
sage: W = ColoredPermutations(3, 1)                                   #␣
↪needs sage.combinat
sage: [t.to_matrix() for t in W]                                      #␣
↪needs sage.combinat sage.groups
[[1], [zeta3], [-zeta3 - 1]]
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(3), Integer(1))                    ␣
↪              # needs sage.combinat
>>> [t.to_matrix() for t in W]                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.groups
[[1], [zeta3], [-zeta3 - 1]]
```

class Irreducible(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: object

absolute_order_ideal(*gens=None*, *in_unitary_group=True*, *return_lengths=False*)

Return all elements in `self` below given elements in the absolute order of `self`.

This order is defined by

$$\omega \leq_R \tau \Leftrightarrow \ell_R(\omega) + \ell_R(\omega^{-1}\tau) = \ell_R(\tau),$$

where ℓ_R denotes the reflection length.

This is, if `in_unitary_group` is `False`, then

$$\ell_R(w) = \min\{\ell : w = r_1 \cdots r_\ell, r_i \in R\},$$

and otherwise

$$\ell_R(w) = \dim \operatorname{im}(w - 1).$$

Note

If `gens` are not given, `self` is assumed to be well-generated.

INPUT:

- `gens` – (default: `None`) if one or more elements are given, the order ideal in the absolute order generated by `gens` is returned. Otherwise, the standard Coxeter element is used as unique maximal element.
- `in_unitary_group` – (default: `True`) determines the length function used to compute the order. For real groups, both possible orders coincide, and for complex non-real groups, the order in the unitary group is much faster to compute.
- `return_lengths` – (default: `False`) whether or not to also return the lengths of the elements

EXAMPLES:

```
sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: sorted(w.reduced_word()
....:        for w in W.absolute_order_ideal())
[[], [1], [1, 2], [1, 2, 1], [2]]
sage: sorted(w.reduced_word()
....:        for w in W.absolute_order_ideal(W.from_reduced_word([2,
↪1])))
[[], [1], [1, 2, 1], [2], [2, 1]]
sage: sorted(w.reduced_word()
....:        for w in W.absolute_order_ideal(W.from_reduced_word([2])))
[[], [2]]
```

(continues on next page)

(continued from previous page)

```
sage: W = CoxeterGroup(['A', 3])                                        #␣
↪needs sage.combinat sage.groups
sage: len(list(W.absolute_order_ideal()))                               #␣
↪needs sage.combinat sage.groups
14

sage: W = CoxeterGroup(['A', 2])                                        #␣
↪needs sage.combinat sage.groups
sage: for w, l in W.absolute_order_ideal(return_lengths=True):     #␣
↪needs sage.combinat sage.groups
....:     print(w.reduced_word(), l)
[1, 2] 2
[1, 2, 1] 1
[2] 1
[1] 1
[] 0
```

```
>>> from sage.all import *
>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> sorted(w.reduced_word()
...        for w in W.absolute_order_ideal())
[[], [1], [1, 2], [1, 2, 1], [2]]
>>> sorted(w.reduced_word()
...        for w in W.absolute_order_ideal(W.from_reduced_
↪word([Integer(2),Integer(1)])))
[[], [1], [1, 2, 1], [2], [2, 1]]
>>> sorted(w.reduced_word()
...        for w in W.absolute_order_ideal(W.from_reduced_
↪word([Integer(2)])))
[[], [2]]

>>> W = CoxeterGroup(['A', Integer(3)])                                 ␣
↪    # needs sage.combinat sage.groups
>>> len(list(W.absolute_order_ideal()))                               #␣
↪needs sage.combinat sage.groups
14

>>> W = CoxeterGroup(['A', Integer(2)])                                 ␣
↪    # needs sage.combinat sage.groups
>>> for w, l in W.absolute_order_ideal(return_lengths=True):     #␣
↪needs sage.combinat sage.groups
...     print(w.reduced_word(), l)
[1, 2] 2
[1, 2, 1] 1
[2] 1
[1] 1
[] 0
```

absolute_poset(*in_unitary_group=False*)

Return the poset induced by the absolute order of `self` as a finite lattice.

INPUT:

- `in_unitary_group` – boolean (default: `False`); if `False`, the relation is given by `\sigma \leq \tau` if $l_R(\sigma)+l_R(\sigma^{-1}\tau) = l_R(\tau)$ If `True`, the relation is given by $\sigma \leq \tau$ if $\dim(\mathrm{Fix}(\sigma))+\dim(\mathrm{Fix}(\sigma^{-1}\tau)) = \dim(\mathrm{Fix}(\tau))$

See also

noncrossing_partition_lattice()

EXAMPLES:

```
sage: P = ReflectionGroup((1,1,3)).absolute_poset(); P       # optional␣
↪- gap3
Finite poset containing 6 elements

sage: sorted(w.reduced_word() for w in P)                    # optional␣
↪- gap3
[[], [1], [1, 2], [1, 2, 1], [2], [2, 1]]

sage: W = ReflectionGroup(4); W                              # optional␣
↪- gap3
Irreducible complex reflection group of rank 2 and type ST4
sage: W.absolute_poset()                                     # optional␣
↪- gap3
Finite poset containing 24 elements
```

```
>>> from sage.all import *
>>> P = ReflectionGroup((Integer(1),Integer(1),Integer(3))).absolute_
↪poset(); P      # optional - gap3
Finite poset containing 6 elements

>>> sorted(w.reduced_word() for w in P)                    # optional -␣
↪gap3
[[], [1], [1, 2], [1, 2, 1], [2], [2, 1]]

>>> W = ReflectionGroup(Integer(4)); W                              #␣
↪optional - gap3
Irreducible complex reflection group of rank 2 and type ST4
>>> W.absolute_poset()                                     # optional -␣
↪gap3
Finite poset containing 24 elements
```

coxeter_number()

Return the Coxeter number of an irreducible reflection group.

This is defined as $\frac{N+N^*}{n}$ where N is the number of reflections, N^* is the number of reflection hyperplanes, and n is the rank of `self`.

EXAMPLES:

```
sage: W = ReflectionGroup(31)           # optional - gap3
sage: W.coxeter_number()                # optional - gap3
30
```

```
>>> from sage.all import *
>>> W = ReflectionGroup(Integer(31))           # optional - gap3
>>> W.coxeter_number()                 # optional - gap3
30
```

generalized_noncrossing_partitions(*m*, *c=None*, *positive=False*)

Return the set of all chains of length m in the noncrossing partition lattice of `self`, see *noncrossing_partition_lattice()*.

Note

`self` is assumed to be well-generated.

INPUT:

- `c` – (default: `None`) if an element c in `self` is given, it is used as the maximal element in the interval
- `positive` – boolean (default: `False`); if `True`, only those generalized noncrossing partitions of full support are returned

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,3))                                  # optional␣
↪- gap3

sage: chains = W.generalized_noncrossing_partitions(2)              # optional␣
↪- gap3
sage: sorted([w.reduced_word() for w in chain]                      # optional␣
↪- gap3
....:         for chain in chains)
[[[], [], [1, 2]],
 [[], [1], [2]],
 [[], [1, 2], []],
 [[], [1, 2, 1], [1]],
 [[], [2], [1, 2, 1]],
 [[1], [], [2]],
 [[1], [2], []],
 [[1, 2], [], []],
 [[1, 2, 1], [], [1]],
 [[1, 2, 1], [1], []],
 [[2], [], [1, 2, 1]],
 [[2], [1, 2, 1], []]]

sage: chains = W.generalized_noncrossing_partitions(2,              # optional␣
↪- gap3
....:                                               positive=True)
sage: sorted([w.reduced_word() for w in chain]                      # optional␣
↪- gap3
....:         for chain in chains)
[[[], [1, 2], []],
 [[], [1, 2, 1], [1]],
 [[1], [2], []],
 [[1, 2], [], []],
 [[1, 2, 1], [], [1]],
```

(continues on next page)

(continued from previous page)

```
 [[1, 2, 1], [1], []],
 [[2], [1, 2, 1], []]]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))           ␣
↪              # optional - gap3

>>> chains = W.generalized_noncrossing_partitions(Integer(2))      #␣
↪optional - gap3
>>> sorted([w.reduced_word() for w in chain]              # optional -␣
↪gap3
...         for chain in chains)
[[[], [], [1, 2]],
 [[], [1], [2]],
 [[], [1, 2], []],
 [[], [1, 2, 1], [1]],
 [[], [2], [1, 2, 1]],
 [[1], [], [2]],
 [[1], [2], []],
 [[1, 2], [], []],
 [[1, 2, 1], [], [1]],
 [[1, 2, 1], [1], []],
 [[2], [], [1, 2, 1]],
 [[2], [1, 2, 1], []]]

>>> chains = W.generalized_noncrossing_partitions(Integer(2),      #␣
↪optional - gap3
...                 positive=True)
>>> sorted([w.reduced_word() for w in chain]              # optional -␣
↪gap3
...         for chain in chains)
[[[], [1, 2], []],
 [[], [1, 2, 1], [1]],
 [[1], [2], []],
 [[1, 2], [], []],
 [[1, 2, 1], [], [1]],
 [[1, 2, 1], [1], []],
 [[2], [1, 2, 1], []]]
```

noncrossing_partition_lattice(*c=None*, *L=None*, *in_unitary_group=True*)

Return the interval $[1, c]$ in the absolute order of `self` as a finite lattice.

See also

absolute_order_ideal()

INPUT:

- `c` – (default: `None`) if an element `c` in `self` is given, it is used as the maximal element in the interval
- `L` – (default: `None`) if a subset `L` (must be hashable!) of `self` is given, it is used as the underlying set (only cover relations are checked).

- `in_unitary_group` – boolean (default: `False`); if `False`, the relation is given by $\sigma \leq \tau$ if $l_R(\sigma) + l_R(\sigma^{-1}\tau) = l_R(\tau)$; if `True`, the relation is given by $\sigma \leq \tau$ if $\dim(\mathrm{Fix}(\sigma)) + \dim(\mathrm{Fix}(\sigma^{-1}\tau)) = \dim(\mathrm{Fix}(\tau))$

Note

If `L` is given, the parameter `c` is ignored.

EXAMPLES:

```
sage: W = SymmetricGroup(4)                                                 #␣
↪needs sage.groups
sage: W.noncrossing_partition_lattice()                                     #␣
↪needs sage.combinat sage.groups
Finite lattice containing 14 elements

sage: W = WeylGroup(['G', 2])                                               #␣
↪needs sage.combinat sage.groups
sage: W.noncrossing_partition_lattice()                                     #␣
↪needs sage.combinat sage.groups
Finite lattice containing 8 elements

sage: # optional - gap3
sage: W = ReflectionGroup((1,1,3))
sage: sorted(w.reduced_word()
....:        for w in W.noncrossing_partition_lattice())
[[], [1], [1, 2], [1, 2, 1], [2]]
sage: c21 = W.from_reduced_word([2,1])
sage: sorted(w.reduced_word()
....:        for w in W.noncrossing_partition_lattice(c21))
[[], [1], [1, 2, 1], [2], [2, 1]]
sage: c2 = W.from_reduced_word([2])
sage: sorted(w.reduced_word()
....:        for w in W.noncrossing_partition_lattice(c2))
[[], [2]]
```

```
>>> from sage.all import *
>>> W = SymmetricGroup(Integer(4))                                          ␣
↪    # needs sage.groups
>>> W.noncrossing_partition_lattice()                                      #␣
↪needs sage.combinat sage.groups
Finite lattice containing 14 elements

>>> W = WeylGroup(['G', Integer(2)])                                        ␣
↪    # needs sage.combinat sage.groups
>>> W.noncrossing_partition_lattice()                                      #␣
↪needs sage.combinat sage.groups
Finite lattice containing 8 elements

>>> # optional - gap3
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))
>>> sorted(w.reduced_word()
...        for w in W.noncrossing_partition_lattice())
```

(continues on next page)

(continued from previous page)

```
[[], [1], [1, 2], [1, 2, 1], [2]]
>>> c21 = W.from_reduced_word([Integer(2),Integer(1)])
>>> sorted(w.reduced_word()
...         for w in W.noncrossing_partition_lattice(c21))
[[], [1], [1, 2, 1], [2], [2, 1]]
>>> c2 = W.from_reduced_word([Integer(2)])
>>> sorted(w.reduced_word()
...         for w in W.noncrossing_partition_lattice(c2))
[[], [2]]
```

example()

Return an example of an irreducible complex reflection group.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: C = ComplexReflectionGroups().Finite().Irreducible()
sage: C.example()                              # optional - gap3
Irreducible complex reflection group of rank 3 and type G(4,2,3)
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> C = ComplexReflectionGroups().Finite().Irreducible()
>>> C.example()                               # optional - gap3
Irreducible complex reflection group of rank 3 and type G(4,2,3)
```

class ParentMethods

Bases: object

base_change_matrix()

Return the base change from the standard basis of the vector space of `self` to the basis given by the independent roots of `self`.

Todo

For non-well-generated groups there is a conflict with construction of the matrix for an element.

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,3))          # optional - gap3
sage: W.base_change_matrix()                # optional - gap3
[1 0]
[0 1]

sage: W = ReflectionGroup(23)               # optional - gap3
sage: W.base_change_matrix()                # optional - gap3
[1 0 0]
[0 1 0]
[0 0 1]
```

(continues on next page)

(continued from previous page)

```
sage: W = ReflectionGroup((3,1,2))          # optional - gap3
sage: W.base_change_matrix()                # optional - gap3
[1 0]
[1 1]

sage: W = ReflectionGroup((4,2,2))          # optional - gap3
sage: W.base_change_matrix()                # optional - gap3
[   1    0]
[E(4)    1]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))          #␣
↪optional - gap3
>>> W.base_change_matrix()                # optional - gap3
[1 0]
[0 1]

>>> W = ReflectionGroup(Integer(23))               # optional - gap3
>>> W.base_change_matrix()                # optional - gap3
[1 0 0]
[0 1 0]
[0 0 1]

>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(2)))          #␣
↪optional - gap3
>>> W.base_change_matrix()                # optional - gap3
[1 0]
[1 1]

>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(2)))          #␣
↪optional - gap3
>>> W.base_change_matrix()                # optional - gap3
[   1    0]
[E(4)    1]
```

cardinality()

Return the cardinality of `self`.

It is given by the product of the degrees of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = ColoredPermutations(1,3)
sage: W.cardinality()
6
sage: W = ColoredPermutations(2,3)
sage: W.cardinality()
48
sage: W = ColoredPermutations(4,3)
sage: W.cardinality()
384
```

(continues on next page)

(continued from previous page)

```
sage: # optional - gap3, needs sage.combinat sage.groups
sage: W = ReflectionGroup((4,2,3))
sage: W.cardinality()
192
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = ColoredPermutations(Integer(1),Integer(3))
>>> W.cardinality()
6
>>> W = ColoredPermutations(Integer(2),Integer(3))
>>> W.cardinality()
48
>>> W = ColoredPermutations(Integer(4),Integer(3))
>>> W.cardinality()
384

>>> # optional - gap3, needs sage.combinat sage.groups
>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))
>>> W.cardinality()
192
```

codegrees()

Return the codegrees of `self`.

OUTPUT: a tuple of Sage integers

EXAMPLES:

```
sage: W = ColoredPermutations(1,4)                                          #␣
↪needs sage.combinat
sage: W.codegrees()                                                         #␣
↪needs sage.combinat
(2, 1, 0)

sage: W = ColoredPermutations(3,3)                                          #␣
↪needs sage.combinat
sage: W.codegrees()                                                         #␣
↪needs sage.combinat
(6, 3, 0)

sage: W = ReflectionGroup(31)               # optional - gap3
sage: W.codegrees()                         # optional - gap3
(28, 16, 12, 0)
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(4))                          ␣
↪              # needs sage.combinat
>>> W.codegrees()                                                          #␣
↪needs sage.combinat
(2, 1, 0)
```

(continues on next page)

(continued from previous page)

```
>>> W = ColoredPermutations(Integer(3),Integer(3))                    ␣
↪              # needs sage.combinat
>>> W.codegrees()                                                     #␣
↪needs sage.combinat
(6, 3, 0)

>>> W = ReflectionGroup(Integer(31))              # optional - gap3
>>> W.codegrees()                           # optional - gap3
(28, 16, 12, 0)
```

degrees()

Return the degrees of `self`.

OUTPUT: a tuple of Sage integers

EXAMPLES:

```
sage: W = ColoredPermutations(1,4)                                     #␣
↪needs sage.combinat
sage: W.degrees()                                                      #␣
↪needs sage.combinat
(2, 3, 4)

sage: W = ColoredPermutations(3,3)                                     #␣
↪needs sage.combinat
sage: W.degrees()                                                      #␣
↪needs sage.combinat
(3, 6, 9)

sage: W = ReflectionGroup(31)               # optional - gap3
sage: W.degrees()                           # optional - gap3
(8, 12, 20, 24)
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(4))                    ␣
↪              # needs sage.combinat
>>> W.degrees()                                                       #␣
↪needs sage.combinat
(2, 3, 4)

>>> W = ColoredPermutations(Integer(3),Integer(3))                    ␣
↪              # needs sage.combinat
>>> W.degrees()                                                       #␣
↪needs sage.combinat
(3, 6, 9)

>>> W = ReflectionGroup(Integer(31))              # optional - gap3
>>> W.degrees()                           # optional - gap3
(8, 12, 20, 24)
```

is_real()

Return whether `self` is real.

A complex reflection group is *real* if it is isomorphic to a reflection group in $GL(V)$ over a real vector space V. Equivalently its character table has real entries.

This implementation uses the following statement: an irreducible complex reflection group is real if and only if 2 is a degree of `self` with multiplicity one. Hence, in general we just need to compare the number of occurrences of 2 as degree of `self` and the number of irreducible components.

EXAMPLES:

```
sage: W = ColoredPermutations(1,3)                                          #␣
↪needs sage.combinat
sage: W.is_real()                                                           #␣
↪needs sage.combinat
True

sage: W = ColoredPermutations(4,3)                                          #␣
↪needs sage.combinat
sage: W.is_real()                                                           #␣
↪needs sage.combinat sage.graphs
False
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                           ␣
↪              # needs sage.combinat
>>> W.is_real()                                                            #␣
↪needs sage.combinat
True

>>> W = ColoredPermutations(Integer(4),Integer(3))                           ␣
↪              # needs sage.combinat
>>> W.is_real()                                                            #␣
↪needs sage.combinat sage.graphs
False
```

Todo

Add an example of non real finite complex reflection group that is generated by order 2 reflections.

is_well_generated()

Return whether `self` is well-generated.

A finite complex reflection group is *well generated* if the number of its simple reflections coincides with its rank.

See also

WellGenerated()

Note

- All finite real reflection groups are well generated.

- The complex reflection groups of type $G(r, 1, n)$ and of type $G(r, r, n)$ are well generated.
- The complex reflection groups of type $G(r, p, n)$ with $1 < p < r$ are *not* well generated.
- The direct product of two well generated finite complex reflection group is still well generated.

EXAMPLES:

```
sage: W = ColoredPermutations(1,3)                                          #␣
↪needs sage.combinat
sage: W.is_well_generated()                                                 #␣
↪needs sage.combinat
True

sage: W = ColoredPermutations(4,3)                                          #␣
↪needs sage.combinat
sage: W.is_well_generated()                                                 #␣
↪needs sage.combinat
True

sage: # optional - gap3, needs sage.combinat sage.groups
sage: W = ReflectionGroup((4,2,3))
sage: W.is_well_generated()
False
sage: W = ReflectionGroup((4,4,3))
sage: W.is_well_generated()
True
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                          ␣
↪              # needs sage.combinat
>>> W.is_well_generated()                                                  #␣
↪needs sage.combinat
True

>>> W = ColoredPermutations(Integer(4),Integer(3))                          ␣
↪              # needs sage.combinat
>>> W.is_well_generated()                                                  #␣
↪needs sage.combinat
True

>>> # optional - gap3, needs sage.combinat sage.groups
>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))
>>> W.is_well_generated()
False
>>> W = ReflectionGroup((Integer(4),Integer(4),Integer(3)))
>>> W.is_well_generated()
True
```

milnor_fiber_poset()

Return the Milnor fiber poset of `self`.

The *Milnor fiber poset* of a finite complex reflection group W is defined as the poset of (right) standard cosets gW_J, where J is a subset of the index set I of W, ordered by reverse inclusion. This is

conjecturally a meet semilattice if and only if W is well-generated.

EXAMPLES:

```
sage: # needs sage.combinat
sage: W = ColoredPermutations(3, 2)
sage: P = W.milnor_fiber_poset()
sage: P
Finite meet-semilattice containing 34 elements
sage: R.<x> = ZZ[]
sage: sum(x**P.rank(elt) for elt in P)
18*x^2 + 15*x + 1

sage: # optional - gap3
sage: W = ReflectionGroup(4)
sage: P = W.milnor_fiber_poset(); P
Finite meet-semilattice containing 41 elements
sage: sum(x**P.rank(elt) for elt in P)
24*x^2 + 16*x + 1

sage: # optional - gap3
sage: W = ReflectionGroup([4,2,2])
sage: W.is_well_generated()
False
sage: P = W.milnor_fiber_poset(); P
Finite poset containing 47 elements
sage: sum(x**P.rank(elt) for elt in P)
16*x^3 + 24*x^2 + 6*x + 1
sage: P.is_meet_semilattice()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> W = ColoredPermutations(Integer(3), Integer(2))
>>> P = W.milnor_fiber_poset()
>>> P
Finite meet-semilattice containing 34 elements
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> sum(x**P.rank(elt) for elt in P)
18*x^2 + 15*x + 1

>>> # optional - gap3
>>> W = ReflectionGroup(Integer(4))
>>> P = W.milnor_fiber_poset(); P
Finite meet-semilattice containing 41 elements
>>> sum(x**P.rank(elt) for elt in P)
24*x^2 + 16*x + 1

>>> # optional - gap3
>>> W = ReflectionGroup([Integer(4),Integer(2),Integer(2)])
>>> W.is_well_generated()
False
>>> P = W.milnor_fiber_poset(); P
Finite poset containing 47 elements
```

(continues on next page)

(continued from previous page)

```
>>> sum(x**P.rank(elt) for elt in P)
16*x^3 + 24*x^2 + 6*x + 1
>>> P.is_meet_semilattice()
False
```

number_of_reflection_hyperplanes()

Return the number of reflection hyperplanes of `self`.

This is also the number of distinguished reflections. For real groups, this coincides with the number of reflections.

This implementation uses that it is given by the sum of the codegrees of `self` plus its rank.

See also

number_of_reflections()

EXAMPLES:

```
sage: # needs sage.combinat
sage: W = ColoredPermutations(1,3)
sage: W.number_of_reflection_hyperplanes()
3
sage: W = ColoredPermutations(2,3)
sage: W.number_of_reflection_hyperplanes()
9
sage: W = ColoredPermutations(4,3)
sage: W.number_of_reflection_hyperplanes()
15

sage: # optional - gap3, needs sage.combinat
sage: W = ReflectionGroup((4,2,3))
sage: W.number_of_reflection_hyperplanes()
15
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> W = ColoredPermutations(Integer(1),Integer(3))
>>> W.number_of_reflection_hyperplanes()
3
>>> W = ColoredPermutations(Integer(2),Integer(3))
>>> W.number_of_reflection_hyperplanes()
9
>>> W = ColoredPermutations(Integer(4),Integer(3))
>>> W.number_of_reflection_hyperplanes()
15

>>> # optional - gap3, needs sage.combinat
>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))
>>> W.number_of_reflection_hyperplanes()
15
```

number_of_reflections()

Return the number of reflections of `self`.

For real groups, this coincides with the number of reflection hyperplanes.

This implementation uses that it is given by the sum of the degrees of `self` minus its rank.

See also

number_of_reflection_hyperplanes()

EXAMPLES:

```
sage: [SymmetricGroup(i).number_of_reflections()                          #␣
↪needs sage.groups sage.rings.number_field
....:  for i in range(int(8))]
[0, 0, 1, 3, 6, 10, 15, 21]

sage: # needs sage.combinat sage.groups
sage: W = ColoredPermutations(1,3)
sage: W.number_of_reflections()
3
sage: W = ColoredPermutations(2,3)
sage: W.number_of_reflections()
9
sage: W = ColoredPermutations(4,3)
sage: W.number_of_reflections()
21
sage: W = ReflectionGroup((4,2,3))       # optional - gap3
sage: W.number_of_reflections()          # optional - gap3
15
```

```
>>> from sage.all import *
>>> [SymmetricGroup(i).number_of_reflections()                           #␣
↪needs sage.groups sage.rings.number_field
...  for i in range(int(Integer(8)))]
[0, 0, 1, 3, 6, 10, 15, 21]

>>> # needs sage.combinat sage.groups
>>> W = ColoredPermutations(Integer(1),Integer(3))
>>> W.number_of_reflections()
3
>>> W = ColoredPermutations(Integer(2),Integer(3))
>>> W.number_of_reflections()
9
>>> W = ColoredPermutations(Integer(4),Integer(3))
>>> W.number_of_reflections()
21
>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))       #␣
↪optional - gap3
>>> W.number_of_reflections()          # optional - gap3
15
```

rank()

Return the rank of `self`.

The rank of `self` is the dimension of the smallest faithfull reflection representation of `self`.

This default implementation uses that the rank is the number of *degrees()*.

See also

rank()

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: W = ColoredPermutations(1,3)
sage: W.rank()
2
sage: W = ColoredPermutations(2,3)
sage: W.rank()
3
sage: W = ColoredPermutations(4,3)
sage: W.rank()
3

sage: # optional - gap3, needs sage.combinat sage.groups
sage: W = ReflectionGroup((4,2,3))
sage: W.rank()
3
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> W = ColoredPermutations(Integer(1),Integer(3))
>>> W.rank()
2
>>> W = ColoredPermutations(Integer(2),Integer(3))
>>> W.rank()
3
>>> W = ColoredPermutations(Integer(4),Integer(3))
>>> W.rank()
3

>>> # optional - gap3, needs sage.combinat sage.groups
>>> W = ReflectionGroup((Integer(4),Integer(2),Integer(3)))
>>> W.rank()
3
```

class SubcategoryMethods

Bases: `object`

WellGenerated()

Return the full subcategory of well-generated objects of `self`.

A finite complex generated group is *well generated* if it is isomorphic to a subgroup of the general linear group GL_n generated by n reflections.

See also

is_well_generated()

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: C = ComplexReflectionGroups().Finite().WellGenerated(); C
Category of well generated finite complex reflection groups
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> C = ComplexReflectionGroups().Finite().WellGenerated(); C
Category of well generated finite complex reflection groups
```

Here is an example of a finite well-generated complex reflection group:

```
sage: W = C.example(); W                    # optional - gap3
Reducible complex reflection group of rank 4 and type A2 x G(3,1,2)
```

```
>>> from sage.all import *
>>> W = C.example(); W                    # optional - gap3
Reducible complex reflection group of rank 4 and type A2 x G(3,1,2)
```

All finite Coxeter groups are well generated:

```
sage: CoxeterGroups().Finite().is_subcategory(C)
True
sage: SymmetricGroup(3) in C                                                #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> CoxeterGroups().Finite().is_subcategory(C)
True
>>> SymmetricGroup(Integer(3)) in C                                         ␣
↪     # needs sage.groups
True
```

Note

The category of well generated finite complex reflection groups is currently implemented as an axiom. See discussion on Issue #11187. This may be a bit of overkill. Still it's nice to have a full subcategory.

class WellGenerated(*base_category*)

Bases: *CategoryWithAxiom*

class Irreducible(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite irreducible well-generated finite complex reflection groups.

class ParentMethods

Bases: object

catalan_number(*positive=False*, *polynomial=False*)

Return the Catalan number associated to `self`.

It is defined by

$$\prod_{i=1}^{n} \frac{d_i + h}{d_i},$$

where $d_1, \ldots, d_n$ are the degrees and where h is the Coxeter number. See [Ar2006] for further information.

INPUT:

- `positive` – boolean (default: `False`); if `True`, return instead the positive Catalan number
- `polynomial` – boolean (default: `False`); if `True`, return instead the q-analogue as a polynomial in q

Note

- For the symmetric group S_n, it reduces to the Catalan number $\frac{1}{n+1}\binom{2n}{n}$.
- The Catalan numbers for $G(r, 1, n)$ all coincide for $r > 1$.

EXAMPLES:

```
sage: [ColoredPermutations(1,n).catalan_number()                #␣
↪needs sage.combinat
....:  for n in [3,4,5]]
[5, 14, 42]

sage: [ColoredPermutations(2,n).catalan_number()                #␣
↪needs sage.combinat
....:  for n in [3,4,5]]
[20, 70, 252]

sage: [ReflectionGroup((2,2,n)).catalan_number()  # optional - gap3
....:  for n in [3,4,5]]
[14, 50, 182]
```

```
>>> from sage.all import *
>>> [ColoredPermutations(Integer(1),n).catalan_number()            ␣
↪  # needs sage.combinat
...  for n in [Integer(3),Integer(4),Integer(5)]]
[5, 14, 42]

>>> [ColoredPermutations(Integer(2),n).catalan_number()            ␣
↪  # needs sage.combinat
...  for n in [Integer(3),Integer(4),Integer(5)]]
[20, 70, 252]
```

(continues on next page)

(continued from previous page)

```
>>> [ReflectionGroup((Integer(2),Integer(2),n)).catalan_number()  #␣
↪optional - gap3
...  for n in [Integer(3),Integer(4),Integer(5)]]
[14, 50, 182]
```

coxeter_number()

Return the Coxeter number of a well-generated, irreducible reflection group. This is defined to be the order of a regular element in `self`, and is equal to the highest degree of `self`.

See also

Irreducible

Note

This method overwrites the more general method for complex reflection groups since the expression given here is quicker to compute.

EXAMPLES:

```
sage: W = ColoredPermutations(1,3)                                    #␣
↪needs sage.combinat
sage: W.coxeter_number()                                              #␣
↪needs sage.combinat
3

sage: W = ColoredPermutations(4,3)                                    #␣
↪needs sage.combinat
sage: W.coxeter_number()                                              #␣
↪needs sage.combinat
12

sage: W = ReflectionGroup((4,4,3))  # optional - gap3
sage: W.coxeter_number()            # optional - gap3
8
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                    ␣
↪           # needs sage.combinat
>>> W.coxeter_number()                                               #␣
↪needs sage.combinat
3

>>> W = ColoredPermutations(Integer(4),Integer(3))                    ␣
↪           # needs sage.combinat
>>> W.coxeter_number()                                               #␣
↪needs sage.combinat
12
```

(continues on next page)

(continued from previous page)

```
>>> W = ReflectionGroup((Integer(4),Integer(4),Integer(3)))   #␣
↪optional - gap3
>>> W.coxeter_number()            # optional - gap3
8
```

fuss_catalan_number(*m*, *positive=False*, *polynomial=False*)

Return the `m`-th Fuss-Catalan number associated to `self`.

This is defined by

$$\prod_{i=1}^{n} \frac{d_i + mh}{d_i},$$

where $d_1, \ldots, d_n$ are the degrees and h is the Coxeter number.

INPUT:

- `positive` – boolean (default: `False`); if `True`, return instead the positive Fuss-Catalan number
- `polynomial` – boolean (default: `False`); if `True`, return instead the q-analogue as a polynomial in q

See [Ar2006] for further information.

Note

- For the symmetric group S_n, it reduces to the Fuss-Catalan number $\frac{1}{mn+1}\binom{(m+1)n}{n}$.
- The Fuss-Catalan numbers for $G(r, 1, n)$ all coincide for $r > 1$.

EXAMPLES:

```
sage: W = ColoredPermutations(1,3)                                  #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                   #␣
↪needs sage.combinat
[5, 12, 22]

sage: W = ColoredPermutations(1,4)                                  #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                   #␣
↪needs sage.combinat
[14, 55, 140]

sage: W = ColoredPermutations(1,5)                                  #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                   #␣
↪needs sage.combinat
[42, 273, 969]

sage: W = ColoredPermutations(2,2)                                  #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                   #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
[6, 15, 28]

sage: W = ColoredPermutations(2,3)                                    #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                     #␣
↪needs sage.combinat
[20, 84, 220]

sage: W = ColoredPermutations(2,4)                                    #␣
↪needs sage.combinat
sage: [W.fuss_catalan_number(i) for i in [1,2,3]]                     #␣
↪needs sage.combinat
[70, 495, 1820]
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[5, 12, 22]

>>> W = ColoredPermutations(Integer(1),Integer(4))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[14, 55, 140]

>>> W = ColoredPermutations(Integer(1),Integer(5))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[42, 273, 969]

>>> W = ColoredPermutations(Integer(2),Integer(2))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[6, 15, 28]

>>> W = ColoredPermutations(Integer(2),Integer(3))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[20, 84, 220]

>>> W = ColoredPermutations(Integer(2),Integer(4))                    ␣
↪           # needs sage.combinat
>>> [W.fuss_catalan_number(i) for i in [Integer(1),Integer(2),
↪Integer(3)]]               # needs sage.combinat
[70, 495, 1820]
```

number_of_reflections_of_full_support()

Return the number of reflections with full support.

EXAMPLES:

```
sage: W = Permutations(4)
sage: W.number_of_reflections_of_full_support()
1

sage: W = ColoredPermutations(1,4)                                  #␣
↪needs sage.combinat
sage: W.number_of_reflections_of_full_support()
1

sage: W = CoxeterGroup("B3")                                        #␣
↪needs sage.combinat sage.groups
sage: W.number_of_reflections_of_full_support()                     #␣
↪needs sage.combinat sage.groups
3

sage: W = ColoredPermutations(3,3)                                  #␣
↪needs sage.combinat
sage: W.number_of_reflections_of_full_support()                     #␣
↪needs sage.combinat
3
```

```
>>> from sage.all import *
>>> W = Permutations(Integer(4))
>>> W.number_of_reflections_of_full_support()
1

>>> W = ColoredPermutations(Integer(1),Integer(4))                  ␣
↪            # needs sage.combinat
>>> W.number_of_reflections_of_full_support()
1

>>> W = CoxeterGroup("B3")                                        #␣
↪needs sage.combinat sage.groups
>>> W.number_of_reflections_of_full_support()                     #␣
↪needs sage.combinat sage.groups
3

>>> W = ColoredPermutations(Integer(3),Integer(3))                  ␣
↪            # needs sage.combinat
>>> W.number_of_reflections_of_full_support()                     #␣
↪needs sage.combinat
3
```

rational_catalan_number(*p*, *polynomial=False*)

Return the `p`-th rational Catalan number associated to `self`.

It is defined by

$$\prod_{i=1}^{n} \frac{p + (p(d_i - 1)) \mod h)}{d_i},$$

where $d_1, \ldots, d_n$ are the degrees and h is the Coxeter number. See [STW2016] for this formula.

INPUT:

- `polynomial` – boolean (default: `False`); if `True`, return instead the q-analogue as a polynomial in q

EXAMPLES:

```
sage: W = ColoredPermutations(1,3)                                    #␣
↪needs sage.combinat
sage: [W.rational_catalan_number(p) for p in [5,7,8]]                 #␣
↪needs sage.combinat
[7, 12, 15]

sage: W = ColoredPermutations(2,2)                                    #␣
↪needs sage.combinat
sage: [W.rational_catalan_number(p) for p in [7,9,11]]                #␣
↪needs sage.combinat
[10, 15, 21]
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(1),Integer(3))                    ␣
↪           # needs sage.combinat
>>> [W.rational_catalan_number(p) for p in [Integer(5),Integer(7),
↪Integer(8)]]           # needs sage.combinat
[7, 12, 15]

>>> W = ColoredPermutations(Integer(2),Integer(2))                    ␣
↪           # needs sage.combinat
>>> [W.rational_catalan_number(p) for p in [Integer(7),Integer(9),
↪Integer(11)]]          # needs sage.combinat
[10, 15, 21]
```

example()

Return an example of an irreducible well-generated complex reflection group.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: C = ComplexReflectionGroups().Finite().WellGenerated().
↪Irreducible()
sage: C.example()                                                     #␣
↪needs sage.combinat
4-colored permutations of size 3
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> C = ComplexReflectionGroups().Finite().WellGenerated().
↪Irreducible()
>>> C.example()                                                       #␣
↪needs sage.combinat
4-colored permutations of size 3
```

class ParentMethods

Bases: object

coxeter_element()

Return a Coxeter element.

The result is the product of the simple reflections, in some order.

> **Note**
>
> This implementation is shared with well generated complex reflection groups. It would be nicer to put it in some joint super category; however, in the current state of the art, there is none where it is clear that this is the right construction for obtaining a Coxeter element.
>
> In this context, this is an element having a regular eigenvector (a vector not contained in any reflection hyperplane of self).

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: CoxeterGroup(['A', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['B', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['D', 4]).coxeter_element().reduced_word()
[1, 2, 4, 3]
sage: CoxeterGroup(['F', 4]).coxeter_element().reduced_word()
[1, 2, 3, 4]
sage: CoxeterGroup(['E', 8]).coxeter_element().reduced_word()
[1, 3, 2, 4, 5, 6, 7, 8]
sage: CoxeterGroup(['H', 3]).coxeter_element().reduced_word()
[1, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> CoxeterGroup(['A', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['B', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['D', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 4, 3]
>>> CoxeterGroup(['F', Integer(4)]).coxeter_element().reduced_word()
[1, 2, 3, 4]
>>> CoxeterGroup(['E', Integer(8)]).coxeter_element().reduced_word()
[1, 3, 2, 4, 5, 6, 7, 8]
>>> CoxeterGroup(['H', Integer(3)]).coxeter_element().reduced_word()
[1, 2, 3]
```

This method is also used for well generated finite complex reflection groups:

```
sage: W = ReflectionGroup((1,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3]
```

(continues on next page)

(continued from previous page)

```
sage: W = ReflectionGroup((2,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

sage: W = ReflectionGroup((4,1,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

sage: W = ReflectionGroup((4,4,4))           # optional - gap3
sage: W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3]

>>> W = ReflectionGroup((Integer(2),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

>>> W = ReflectionGroup((Integer(4),Integer(1),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]

>>> W = ReflectionGroup((Integer(4),Integer(4),Integer(4)))           #␣
↪optional - gap3
>>> W.coxeter_element().reduced_word()     # optional - gap3
[1, 2, 3, 4]
```

coxeter_elements()

Return the (unique) conjugacy class in `self` containing all Coxeter elements.

A Coxeter element is an element that has an eigenvalue $e^{2\pi i/h}$ where h is the Coxeter number.

In case of finite Coxeter groups, these are exactly the elements that are conjugate to one (or, equivalently, all) standard Coxeter element, this is, to an element that is the product of the simple generators in some order.

See also

standard_coxeter_elements()

EXAMPLES:

```
sage: W = ReflectionGroup((1,1,3))      # optional - gap3
sage: sorted(c.reduced_word()           # optional - gap3
....:        for c in W.coxeter_elements())
```

(continues on next page)

(continued from previous page)

```
[[1, 2], [2, 1]]

sage: W = ReflectionGroup((1,1,4))      # optional - gap3
sage: sorted(c.reduced_word()           # optional - gap3
....:        for c in W.coxeter_elements())
[[1, 2, 1, 3, 2], [1, 2, 3], [1, 3, 2],
 [2, 1, 3], [2, 1, 3, 2, 1], [3, 2, 1]]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(3)))      #␣
↪optional - gap3
>>> sorted(c.reduced_word()           # optional - gap3
...        for c in W.coxeter_elements())
[[1, 2], [2, 1]]

>>> W = ReflectionGroup((Integer(1),Integer(1),Integer(4)))      #␣
↪optional - gap3
>>> sorted(c.reduced_word()           # optional - gap3
...        for c in W.coxeter_elements())
[[1, 2, 1, 3, 2], [1, 2, 3], [1, 3, 2],
 [2, 1, 3], [2, 1, 3, 2, 1], [3, 2, 1]]
```

is_well_generated()

Return `True` as `self` is well-generated.

EXAMPLES:

```
sage: W = ReflectionGroup((3,1,2))      # optional - gap3
sage: W.is_well_generated()             # optional - gap3
True
```

```
>>> from sage.all import *
>>> W = ReflectionGroup((Integer(3),Integer(1),Integer(2)))      #␣
↪optional - gap3
>>> W.is_well_generated()             # optional - gap3
True
```

milnor_fiber_complex()

Return the Milnor fiber complex of `self`.

The *Milnor fiber complex* of a finite well-generated complex reflection group W is the simplicial complex whose face poset is given by *milnor_fiber_poset()*. When W is an irreducible Shephard group, it is also an equivariant strong deformation retract of the Milnor fiber $f_1^{-1}(1)$, where $f_1 : V \to \mathbf{C}$ is the polynomial invariant of smallest degree acting on the reflection representation V.

When W is a Coxeter group, this is isomorphic to the Coxeter complex of W.

EXAMPLES:

```
sage: W = ColoredPermutations(3, 2)                                  #␣
↪needs sage.combinat
sage: C = W.milnor_fiber_complex()                                   #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
sage: C.homology()                                                          #␣
↪needs sage.combinat
{0: 0, 1: Z x Z x Z x Z}

sage: W = ReflectionGroup(5)                    # optional - gap3
sage: C = W.milnor_fiber_complex()              # optional - gap3
sage: C.homology()                              # optional - gap3
{0: 0, 1: Z^25}
```

```
>>> from sage.all import *
>>> W = ColoredPermutations(Integer(3), Integer(2))                         ␣
↪             # needs sage.combinat
>>> C = W.milnor_fiber_complex()                                           #␣
↪needs sage.combinat
>>> C.homology()                                                           #␣
↪needs sage.combinat
{0: 0, 1: Z x Z x Z x Z}

>>> W = ReflectionGroup(Integer(5))                    # optional - gap3
>>> C = W.milnor_fiber_complex()              # optional - gap3
>>> C.homology()                              # optional - gap3
{0: 0, 1: Z^25}
```

standard_coxeter_elements()

Return all standard Coxeter elements in `self`.

This is the set of all elements in `self` obtained from any product of the simple reflections in `self`.

Note

- `self` is assumed to be well-generated.
- This works even beyond real reflection groups, but the conjugacy class is not unique and we only obtain one such class.

EXAMPLES:

```
sage: W = ReflectionGroup(4)                    # optional - gap3
sage: sorted(W.standard_coxeter_elements())   # optional - gap3
[(1,7,6,12,23,20)(2,8,17,24,9,5)(3,16,10,19,15,21)(4,14,11,22,18,13),
 (1,10,4,12,21,22)(2,11,19,24,13,3)(5,15,7,17,16,23)(6,18,8,20,14,9)]
```

```
>>> from sage.all import *
>>> W = ReflectionGroup(Integer(4))                    # optional - gap3
>>> sorted(W.standard_coxeter_elements())   # optional - gap3
[(1,7,6,12,23,20)(2,8,17,24,9,5)(3,16,10,19,15,21)(4,14,11,22,18,13),
 (1,10,4,12,21,22)(2,11,19,24,13,3)(5,15,7,17,16,23)(6,18,8,20,14,9)]
```

example()

Return an example of a well-generated complex reflection group.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: C = ComplexReflectionGroups().Finite().WellGenerated()
sage: C.example()                              # optional - gap3
Reducible complex reflection group of rank 4 and type A2 x G(3,1,2)
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> C = ComplexReflectionGroups().Finite().WellGenerated()
>>> C.example()                               # optional - gap3
Reducible complex reflection group of rank 4 and type A2 x G(3,1,2)
```

example()

Return an example of a complex reflection group.

EXAMPLES:

```
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
sage: ComplexReflectionGroups().Finite().example()          # optional - gap3
Reducible real reflection group of rank 4 and type A2 x B2
```

```
>>> from sage.all import *
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups
>>> ComplexReflectionGroups().Finite().example()          # optional - gap3
Reducible real reflection group of rank 4 and type A2 x B2
```

4.50 Finite Coxeter Groups

class sage.categories.finite_coxeter_groups.**FiniteCoxeterGroups**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite Coxeter groups.

EXAMPLES:

```
sage: CoxeterGroups.Finite()
Category of finite Coxeter groups
sage: FiniteCoxeterGroups().super_categories()
[Category of finite generalized Coxeter groups,
 Category of Coxeter groups]

sage: G = CoxeterGroups().Finite().example()
sage: G.cayley_graph(side = "right").plot()
Graphics object consisting of 40 graphics primitives
```

```
>>> from sage.all import *
>>> CoxeterGroups.Finite()
Category of finite Coxeter groups
>>> FiniteCoxeterGroups().super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of finite generalized Coxeter groups,
 Category of Coxeter groups]

>>> G = CoxeterGroups().Finite().example()
>>> G.cayley_graph(side = "right").plot()
Graphics object consisting of 40 graphics primitives
```

Here are some further examples:

```
sage: WeylGroups().Finite().example()
The symmetric group on {0, ..., 3}

sage: WeylGroup(["B", 3])
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)
```

```
>>> from sage.all import *
>>> WeylGroups().Finite().example()
The symmetric group on {0, ..., 3}

>>> WeylGroup(["B", Integer(3)])
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)
```

Those other examples will eventually be also in this category:

```
sage: SymmetricGroup(4)
Symmetric group of order 4! as a permutation group
sage: DihedralGroup(5)
Dihedral group of order 10 as a permutation group
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(4))
Symmetric group of order 4! as a permutation group
>>> DihedralGroup(Integer(5))
Dihedral group of order 10 as a permutation group
```

class ElementMethods

Bases: object

absolute_length()

Return the absolute length of self.

The absolute length is the length of the shortest expression of the element as a product of reflections. For finite Coxeter groups, the absolute length is the codimension of the 1-eigenspace of the element (Lemmas 1-3 in [Car1972a]).

For permutations in the symmetric groups, the absolute length is the size minus the number of its disjoint cycles.

See also

absolute_le()

EXAMPLES:

```
sage: W = WeylGroup(["A", 3])                                                  #␣
↪needs sage.combinat sage.groups
sage: s = W.simple_reflections()                                               #␣
↪needs sage.combinat sage.groups
sage: (s[1]*s[2]*s[3]).absolute_length()                                       #␣
↪needs sage.combinat sage.groups
3

sage: W = SymmetricGroup(4)                                                    #␣
↪needs sage.groups
sage: s = W.simple_reflections()                                               #␣
↪needs sage.groups
sage: (s[3]*s[2]*s[1]).absolute_length()                                       #␣
↪needs sage.combinat sage.groups
3
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])                                           ␣
↪     # needs sage.combinat sage.groups
>>> s = W.simple_reflections()                                                #␣
↪needs sage.combinat sage.groups
>>> (s[Integer(1)]*s[Integer(2)]*s[Integer(3)]).absolute_length()              ␣
↪                        # needs sage.combinat sage.groups
3

>>> W = SymmetricGroup(Integer(4))                                             ␣
↪     # needs sage.groups
>>> s = W.simple_reflections()                                                #␣
↪needs sage.groups
>>> (s[Integer(3)]*s[Integer(2)]*s[Integer(1)]).absolute_length()              ␣
↪                        # needs sage.combinat sage.groups
3
```

bruhat_upper_covers()

Return all the elements that cover `self` in Bruhat order.

EXAMPLES:

```
sage: W = WeylGroup(["A",4])
sage: w = W.from_reduced_word([3,2])
sage: print([v.reduced_word() for v in w.bruhat_upper_covers()])
[[4, 3, 2], [3, 4, 2], [2, 3, 2], [3, 1, 2], [3, 2, 1]]

sage: W = WeylGroup(["B",6])
sage: w = W.from_reduced_word([1,2,1,4,5])
sage: C = w.bruhat_upper_covers()
sage: len(C)
9
sage: print([v.reduced_word() for v in C])
[[6, 4, 5, 1, 2, 1], [4, 5, 6, 1, 2, 1], [3, 4, 5, 1, 2, 1], [2, 3, 4, 5,␣
↪1, 2],
[1, 2, 3, 4, 5, 1], [4, 5, 4, 1, 2, 1], [4, 5, 3, 1, 2, 1], [4, 5, 2, 3,␣
↪1, 2],
```

(continues on next page)

(continued from previous page)

```
[4, 5, 1, 2, 3, 1]]
sage: ww = W.from_reduced_word([5,6,5])
sage: CC = ww.bruhat_upper_covers()
sage: print([v.reduced_word() for v in CC])
[[6, 5, 6, 5], [4, 5, 6, 5], [5, 6, 4, 5], [5, 6, 5, 4], [5, 6, 5, 3], [5,
↪ 6, 5, 2],
[5, 6, 5, 1]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A",Integer(4)])
>>> w = W.from_reduced_word([Integer(3),Integer(2)])
>>> print([v.reduced_word() for v in w.bruhat_upper_covers()])
[[4, 3, 2], [3, 4, 2], [2, 3, 2], [3, 1, 2], [3, 2, 1]]

>>> W = WeylGroup(["B",Integer(6)])
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(4),
↪Integer(5)])
>>> C = w.bruhat_upper_covers()
>>> len(C)
9
>>> print([v.reduced_word() for v in C])
[[6, 4, 5, 1, 2, 1], [4, 5, 6, 1, 2, 1], [3, 4, 5, 1, 2, 1], [2, 3, 4, 5,␣
↪1, 2],
[1, 2, 3, 4, 5, 1], [4, 5, 4, 1, 2, 1], [4, 5, 3, 1, 2, 1], [4, 5, 2, 3,␣
↪1, 2],
[4, 5, 1, 2, 3, 1]]
>>> ww = W.from_reduced_word([Integer(5),Integer(6),Integer(5)])
>>> CC = ww.bruhat_upper_covers()
>>> print([v.reduced_word() for v in CC])
[[6, 5, 6, 5], [4, 5, 6, 5], [5, 6, 4, 5], [5, 6, 5, 4], [5, 6, 5, 3], [5,
↪ 6, 5, 2],
[5, 6, 5, 1]]
```

Recursive algorithm: write w for `self`. If i is a non-descent of w, then the covers of w are exactly $\{ws_i, u_1 s_i, u_2 s_i, ..., u_j s_i\}$, where the u_k are those covers of ws_i that have a descent at i.

covered_reflections_subgroup()

Return the subgroup of W generated by the conjugates by w of the simple reflections indexed by right descents of w.

This is used to compute the shard intersection order on W.

EXAMPLES:

```
sage: W = CoxeterGroup(['A',3], base_ring=ZZ)
sage: len(W.long_element().covered_reflections_subgroup())
24
sage: s = W.simple_reflection(1)
sage: Gs = s.covered_reflections_subgroup()
sage: len(Gs)
2
sage: s in [u.lift() for u in Gs]
True
```

(continues on next page)

(continued from previous page)

```
sage: len(W.one().covered_reflections_subgroup())
1
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A',Integer(3)], base_ring=ZZ)
>>> len(W.long_element().covered_reflections_subgroup())
24
>>> s = W.simple_reflection(Integer(1))
>>> Gs = s.covered_reflections_subgroup()
>>> len(Gs)
2
>>> s in [u.lift() for u in Gs]
True
>>> len(W.one().covered_reflections_subgroup())
1
```

coxeter_knuth_graph()

Return the Coxeter-Knuth graph of type A.

The Coxeter-Knuth graph of type A is generated by the Coxeter-Knuth relations which are given by $aa+1a \sim a+1aa+1$, $abc \sim acb$ if $b < a < c$ and $abc \sim bac$ if $a < c < b$.

EXAMPLES:

```
sage: W = WeylGroup(['A',4], prefix='s')
sage: w = W.from_reduced_word([1,2,1,3,2])
sage: D = w.coxeter_knuth_graph()
sage: D.vertices(sort=True)
[(1, 2, 1, 3, 2),
(1, 2, 3, 1, 2),
(2, 1, 2, 3, 2),
(2, 1, 3, 2, 3),
(2, 3, 1, 2, 3)]
sage: D.edges(sort=True)
[((1, 2, 1, 3, 2), (1, 2, 3, 1, 2), None),
((1, 2, 1, 3, 2), (2, 1, 2, 3, 2), None),
((2, 1, 2, 3, 2), (2, 1, 3, 2, 3), None),
((2, 1, 3, 2, 3), (2, 3, 1, 2, 3), None)]

sage: w = W.from_reduced_word([1,3])
sage: D = w.coxeter_knuth_graph()
sage: D.vertices(sort=True)
[(1, 3), (3, 1)]
sage: D.edges(sort=False)
[]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(4)], prefix='s')
>>> w = W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(3),
↪Integer(2)])
>>> D = w.coxeter_knuth_graph()
>>> D.vertices(sort=True)
```

(continues on next page)

(continued from previous page)

```
[(1, 2, 1, 3, 2),
(1, 2, 3, 1, 2),
(2, 1, 2, 3, 2),
(2, 1, 3, 2, 3),
(2, 3, 1, 2, 3)]
>>> D.edges(sort=True)
[((1, 2, 1, 3, 2), (1, 2, 3, 1, 2), None),
((1, 2, 1, 3, 2), (2, 1, 2, 3, 2), None),
((2, 1, 2, 3, 2), (2, 1, 3, 2, 3), None),
((2, 1, 3, 2, 3), (2, 3, 1, 2, 3), None)]

>>> w = W.from_reduced_word([Integer(1),Integer(3)])
>>> D = w.coxeter_knuth_graph()
>>> D.vertices(sort=True)
[(1, 3), (3, 1)]
>>> D.edges(sort=False)
[]
```

coxeter_knuth_neighbor(*w*)

Return the Coxeter-Knuth (oriented) neighbors of the reduced word w of `self`.

INPUT:

- `w` – reduced word of `self`

The Coxeter-Knuth relations are given by $aa + 1a \sim a + 1aa + 1$, $abc \sim acb$ if $b < a < c$ and $abc \sim bac$ if $a < c < b$. This method returns all neighbors of `w` under the Coxeter-Knuth relations oriented from left to right.

EXAMPLES:

```
sage: W = WeylGroup(['A',4], prefix='s')
sage: word = [1,2,1,3,2]
sage: w = W.from_reduced_word(word)
sage: w.coxeter_knuth_neighbor(word)
{(1, 2, 3, 1, 2), (2, 1, 2, 3, 2)}

sage: word = [1,2,1,3,2,4,3]
sage: w = W.from_reduced_word(word)
sage: w.coxeter_knuth_neighbor(word)
{(1, 2, 1, 3, 4, 2, 3), (1, 2, 3, 1, 2, 4, 3), (2, 1, 2, 3, 2, 4, 3)}
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(4)], prefix='s')
>>> word = [Integer(1),Integer(2),Integer(1),Integer(3),Integer(2)]
>>> w = W.from_reduced_word(word)
>>> w.coxeter_knuth_neighbor(word)
{(1, 2, 3, 1, 2), (2, 1, 2, 3, 2)}

>>> word = [Integer(1),Integer(2),Integer(1),Integer(3),Integer(2),
↪Integer(4),Integer(3)]
>>> w = W.from_reduced_word(word)
>>> w.coxeter_knuth_neighbor(word)
{(1, 2, 1, 3, 4, 2, 3), (1, 2, 3, 1, 2, 4, 3), (2, 1, 2, 3, 2, 4, 3)}
```

is_coxeter_element()

Return whether this is a Coxeter element.

This is, whether `self` has an eigenvalue $e^{2\pi i/h}$ where h is the Coxeter number.

See also

coxeter_elements()

EXAMPLES:

```
sage: W = CoxeterGroup(['A',2])
sage: c = prod(W.gens())
sage: c.is_coxeter_element()
True
sage: W.one().is_coxeter_element()
False

sage: W = WeylGroup(['G', 2])
sage: c = prod(W.gens())
sage: c.is_coxeter_element()
True
sage: W.one().is_coxeter_element()
False
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A',Integer(2)])
>>> c = prod(W.gens())
>>> c.is_coxeter_element()
True
>>> W.one().is_coxeter_element()
False

>>> W = WeylGroup(['G', Integer(2)])
>>> c = prod(W.gens())
>>> c.is_coxeter_element()
True
>>> W.one().is_coxeter_element()
False
```

class ParentMethods

Bases: `object`

Ambiguity resolution: the implementation of `some_elements` is preferable to that of *FiniteGroups*. The same holds for `__iter__`, although a breadth first search would be more natural; at least this maintains backward compatibility after Issue #13589.

bhz_poset()

Return the Bergeron-Hohlweg-Zabrocki partial order on the Coxeter group.

This is a partial order on the elements of a finite Coxeter group W, which is distinct from the Bruhat order, the weak order and the shard intersection order. It was defined in [BHZ2005].

This partial order is not a lattice, as there is no unique maximal element. It can be succinctly defined as follows.

Let u and v be two elements of the Coxeter group W. Let $S(u)$ be the support of u. Then $u \leq v$ if and only if $v_{S(u)} = u$ (here $v = v^I v_I$ denotes the usual parabolic decomposition with respect to the standard parabolic subgroup W_I).

See also

bruhat_poset(), *shard_poset()*, *weak_poset()*

EXAMPLES:

```
sage: W = CoxeterGroup(['A',3], base_ring=ZZ)
sage: P = W.bhz_poset(); P
Finite poset containing 24 elements
sage: P.number_of_relations()
103
sage: P.chain_polynomial()
34*q^4 + 90*q^3 + 79*q^2 + 24*q + 1
sage: len(P.maximal_elements())
13
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A',Integer(3)], base_ring=ZZ)
>>> P = W.bhz_poset(); P
Finite poset containing 24 elements
>>> P.number_of_relations()
103
>>> P.chain_polynomial()
34*q^4 + 90*q^3 + 79*q^2 + 24*q + 1
>>> len(P.maximal_elements())
13
```

bruhat_poset(*facade=False*)

Return the Bruhat poset of `self`.

See also

bhz_poset(), *shard_poset()*, *weak_poset()*

EXAMPLES:

```
sage: W = WeylGroup(["A", 2])
sage: P = W.bruhat_poset()
sage: P
Finite poset containing 6 elements
sage: P.show()
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(2)])
>>> P = W.bruhat_poset()
>>> P
Finite poset containing 6 elements
>>> P.show()
```

Here are some typical operations on this poset:

```
sage: W = WeylGroup(["A", 3])
sage: P = W.bruhat_poset()
sage: u = W.from_reduced_word([3,1])
sage: v = W.from_reduced_word([3,2,1,2,3])
sage: P(u) <= P(v)
True
sage: len(P.interval(P(u), P(v)))
10
sage: P.is_join_semilattice()
False
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(3)])
>>> P = W.bruhat_poset()
>>> u = W.from_reduced_word([Integer(3),Integer(1)])
>>> v = W.from_reduced_word([Integer(3),Integer(2),Integer(1),Integer(2),
↪Integer(3)])
>>> P(u) <= P(v)
True
>>> len(P.interval(P(u), P(v)))
10
>>> P.is_join_semilattice()
False
```

By default, the elements of P are aware that they belong to P:

```
sage: P.an_element().parent()
Finite poset containing 24 elements
```

```
>>> from sage.all import *
>>> P.an_element().parent()
Finite poset containing 24 elements
```

If instead one wants the elements to be plain elements of the Coxeter group, one can use the `facade` option:

```
sage: P = W.bruhat_poset(facade = True)
sage: P.an_element().parent()
Weyl Group of type ['A', 3] (as a matrix group acting on the ambient␣
↪space)
```

```
>>> from sage.all import *
>>> P = W.bruhat_poset(facade = True)
>>> P.an_element().parent()
Weyl Group of type ['A', 3] (as a matrix group acting on the ambient␣
↪space)
```

See also

`Poset()` for more on posets and facade posets.

Todo

- Use the symmetric group in the examples (for nicer output), and print the edges for a stronger test.
- The constructed poset should be lazy, in order to handle large / infinite Coxeter groups.

cambrian_lattice(*c*, *on_roots=False*)

Return the c-Cambrian lattice on delta sequences.

See arXiv 1503.00710 and arXiv math/0611106.

Delta sequences are certain 2-colored minimal factorizations of c into reflections.

INPUT:

- `c` – a standard Coxeter element in `self` (as a tuple, or as an element of `self`)
- `on_roots` – boolean (default: `False`); if `on_roots` is `True`, the lattice is realized on roots rather than on reflections. In order for this to work, the ElementMethod `reflection_to_root` must be available.

EXAMPLES:

```
sage: CoxeterGroup(["A", 2]).cambrian_lattice((1,2))
Finite lattice containing 5 elements

sage: CoxeterGroup(["B", 2]).cambrian_lattice((1,2))
Finite lattice containing 6 elements

sage: CoxeterGroup(["G", 2]).cambrian_lattice((1,2))
Finite lattice containing 8 elements
```

```
>>> from sage.all import *
>>> CoxeterGroup(["A", Integer(2)]).cambrian_lattice((Integer(1),
↪Integer(2)))
Finite lattice containing 5 elements

>>> CoxeterGroup(["B", Integer(2)]).cambrian_lattice((Integer(1),
↪Integer(2)))
Finite lattice containing 6 elements

>>> CoxeterGroup(["G", Integer(2)]).cambrian_lattice((Integer(1),
↪Integer(2)))
Finite lattice containing 8 elements
```

codegrees()

Return the codegrees of the Coxeter group.

These are just the degrees minus 2.

EXAMPLES:

```
sage: CoxeterGroup(['A', 4]).codegrees()
(0, 1, 2, 3)
sage: CoxeterGroup(['B', 4]).codegrees()
(0, 2, 4, 6)
```

(continues on next page)

(continued from previous page)

```
sage: CoxeterGroup(['D', 4]).codegrees()
(0, 2, 2, 4)
sage: CoxeterGroup(['F', 4]).codegrees()
(0, 4, 6, 10)
sage: CoxeterGroup(['E', 8]).codegrees()
(0, 6, 10, 12, 16, 18, 22, 28)
sage: CoxeterGroup(['H', 3]).codegrees()
(0, 4, 8)

sage: WeylGroup([["A",3], ["A",3], ["B",2]]).codegrees()
(0, 1, 2, 0, 1, 2, 0, 2)
```

```
>>> from sage.all import *
>>> CoxeterGroup(['A', Integer(4)]).codegrees()
(0, 1, 2, 3)
>>> CoxeterGroup(['B', Integer(4)]).codegrees()
(0, 2, 4, 6)
>>> CoxeterGroup(['D', Integer(4)]).codegrees()
(0, 2, 2, 4)
>>> CoxeterGroup(['F', Integer(4)]).codegrees()
(0, 4, 6, 10)
>>> CoxeterGroup(['E', Integer(8)]).codegrees()
(0, 6, 10, 12, 16, 18, 22, 28)
>>> CoxeterGroup(['H', Integer(3)]).codegrees()
(0, 4, 8)

>>> WeylGroup([["A",Integer(3)], ["A",Integer(3)], ["B",Integer(2)]]).
↪codegrees()
(0, 1, 2, 0, 1, 2, 0, 2)
```

coxeter_complex()

Return the Coxeter complex of `self`.

Let W be a Coxeter group, and let X be the corresponding Tits cone, which is constructed as the W orbit of the fundamental chamber in the reflection representation. The *Coxeter complex* of W is the simplicial complex $(X \setminus \{0\})/\mathbf{R}_{>0}$. The face poset of this simplicial complex is given by the *coxeter_poset()*. When W is a finite group, then the Coxeter complex is homeomorphic to an $(n-1)$-dimensional sphere, where n is the rank of W.

EXAMPLES:

```
sage: W = CoxeterGroup(['A', 3])
sage: C = W.coxeter_complex()
sage: C
Simplicial complex with 14 vertices and 24 facets
sage: C.homology()
{0: 0, 1: 0, 2: Z}

sage: W = WeylGroup(['B', 3])
sage: C = W.coxeter_complex()
sage: C
Simplicial complex with 26 vertices and 48 facets
sage: C.homology()
```

(continues on next page)

(continued from previous page)

```
{0: 0, 1: 0, 2: Z}

sage: W = CoxeterGroup(['I', 7])
sage: C = W.coxeter_complex()
sage: C
Simplicial complex with 14 vertices and 14 facets
sage: C.homology()
{0: 0, 1: Z}

sage: W = CoxeterGroup(['H', 3])
sage: C = W.coxeter_complex()
sage: C
Simplicial complex with 62 vertices and 120 facets
sage: C.homology()
{0: 0, 1: 0, 2: Z}

sage: # optional - gap3
sage: W = CoxeterGroup(['H', 3], implementation='permutation')
sage: C = W.coxeter_complex()
sage: C
Simplicial complex with 62 vertices and 120 facets
sage: C.homology()
{0: 0, 1: 0, 2: Z}
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A', Integer(3)])
>>> C = W.coxeter_complex()
>>> C
Simplicial complex with 14 vertices and 24 facets
>>> C.homology()
{0: 0, 1: 0, 2: Z}

>>> W = WeylGroup(['B', Integer(3)])
>>> C = W.coxeter_complex()
>>> C
Simplicial complex with 26 vertices and 48 facets
>>> C.homology()
{0: 0, 1: 0, 2: Z}

>>> W = CoxeterGroup(['I', Integer(7)])
>>> C = W.coxeter_complex()
>>> C
Simplicial complex with 14 vertices and 14 facets
>>> C.homology()
{0: 0, 1: Z}

>>> W = CoxeterGroup(['H', Integer(3)])
>>> C = W.coxeter_complex()
>>> C
Simplicial complex with 62 vertices and 120 facets
>>> C.homology()
{0: 0, 1: 0, 2: Z}
```

(continues on next page)

(continued from previous page)

```
>>> # optional - gap3
>>> W = CoxeterGroup(['H', Integer(3)], implementation='permutation')
>>> C = W.coxeter_complex()
>>> C
Simplicial complex with 62 vertices and 120 facets
>>> C.homology()
{0: 0, 1: 0, 2: Z}
```

coxeter_poset()

Return the Coxeter poset of `self`.

Let W be a Coxeter group. The *Coxeter poset* is defined as the set of (right) standard cosets gW_J, where J is a subset of the index set I of W, ordered by reverse inclusion.

This is equal to the face poset of the *Coxeter complex*.

EXAMPLES:

```
sage: W = CoxeterGroup(['A', 3])
sage: P = W.coxeter_poset()
sage: P
Finite meet-semilattice containing 75 elements
sage: P.rank()
3

sage: W = WeylGroup(['B', 3])
sage: P = W.coxeter_poset()
sage: P
Finite meet-semilattice containing 147 elements
sage: P.rank()
3

sage: W = CoxeterGroup(['I', 7])
sage: P = W.coxeter_poset()
sage: P
Finite meet-semilattice containing 29 elements
sage: P.rank()
2

sage: W = CoxeterGroup(['H', 3])
sage: P = W.coxeter_poset()
sage: P
Finite meet-semilattice containing 363 elements
sage: P.rank()
3

sage: # optional - gap3
sage: W = CoxeterGroup(['H', 3], implementation='permutation')
sage: P = W.coxeter_poset()
sage: P
Finite meet-semilattice containing 363 elements
sage: P.rank()
3
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A', Integer(3)])
>>> P = W.coxeter_poset()
>>> P
Finite meet-semilattice containing 75 elements
>>> P.rank()
3

>>> W = WeylGroup(['B', Integer(3)])
>>> P = W.coxeter_poset()
>>> P
Finite meet-semilattice containing 147 elements
>>> P.rank()
3

>>> W = CoxeterGroup(['I', Integer(7)])
>>> P = W.coxeter_poset()
>>> P
Finite meet-semilattice containing 29 elements
>>> P.rank()
2

>>> W = CoxeterGroup(['H', Integer(3)])
>>> P = W.coxeter_poset()
>>> P
Finite meet-semilattice containing 363 elements
>>> P.rank()
3

>>> # optional - gap3
>>> W = CoxeterGroup(['H', Integer(3)], implementation='permutation')
>>> P = W.coxeter_poset()
>>> P
Finite meet-semilattice containing 363 elements
>>> P.rank()
3
```

degrees()

Return the degrees of the Coxeter group.

The output is an increasing tuple of integers.

EXAMPLES:

```
sage: CoxeterGroup(['A', 4]).degrees()
(2, 3, 4, 5)
sage: CoxeterGroup(['B', 4]).degrees()
(2, 4, 6, 8)
sage: CoxeterGroup(['D', 4]).degrees()
(2, 4, 4, 6)
sage: CoxeterGroup(['F', 4]).degrees()
(2, 6, 8, 12)
sage: CoxeterGroup(['E', 8]).degrees()
(2, 8, 12, 14, 18, 20, 24, 30)
```

(continues on next page)

(continued from previous page)

```
sage: CoxeterGroup(['H', 3]).degrees()
(2, 6, 10)

sage: WeylGroup([["A",3], ["A",3], ["B",2]]).degrees()
(2, 3, 4, 2, 3, 4, 2, 4)
```

```
>>> from sage.all import *
>>> CoxeterGroup(['A', Integer(4)]).degrees()
(2, 3, 4, 5)
>>> CoxeterGroup(['B', Integer(4)]).degrees()
(2, 4, 6, 8)
>>> CoxeterGroup(['D', Integer(4)]).degrees()
(2, 4, 4, 6)
>>> CoxeterGroup(['F', Integer(4)]).degrees()
(2, 6, 8, 12)
>>> CoxeterGroup(['E', Integer(8)]).degrees()
(2, 8, 12, 14, 18, 20, 24, 30)
>>> CoxeterGroup(['H', Integer(3)]).degrees()
(2, 6, 10)

>>> WeylGroup([["A",Integer(3)], ["A",Integer(3)], ["B",Integer(2)]]).
↪degrees()
(2, 3, 4, 2, 3, 4, 2, 4)
```

inversion_sequence(*word*)

Return the inversion sequence corresponding to the `word` in indices of simple generators of `self`.

If `word` corresponds to $[w_0, w_1, ...w_k]$, the output is $[w_0, w_0w_1w_0, \ldots, w_0w_1 \cdots w_k \cdots w_1w_0]$.

INPUT:

- `word` – a word in the indices of the simple generators of `self`

EXAMPLES:

```
sage: CoxeterGroup(["A", 2]).inversion_sequence([1,2,1])
[
[-1  1]  [ 0 -1]  [ 1  0]
[ 0  1], [-1  0], [ 1 -1]
]

sage: [t.reduced_word() for t in CoxeterGroup(["A",3]).inversion_
↪sequence([2,1,3,2,1,3])]
[[2], [1, 2, 1], [2, 3, 2], [1, 2, 3, 2, 1], [3], [1]]
```

```
>>> from sage.all import *
>>> CoxeterGroup(["A", Integer(2)]).inversion_sequence([Integer(1),
↪Integer(2),Integer(1)])
[
[-1  1]  [ 0 -1]  [ 1  0]
[ 0  1], [-1  0], [ 1 -1]
]

>>> [t.reduced_word() for t in CoxeterGroup(["A",Integer(3)]).inversion_
```

(continues on next page)

(continued from previous page)

```
↪sequence([Integer(2),Integer(1),Integer(3),Integer(2),Integer(1),
↪Integer(3)])]
[[2], [1, 2, 1], [2, 3, 2], [1, 2, 3, 2, 1], [3], [1]]
```

is_real()

Return `True` since `self` is a real reflection group.

EXAMPLES:

```
sage: CoxeterGroup(['F',4]).is_real()
True
sage: CoxeterGroup(['H',4]).is_real()
True
```

```
>>> from sage.all import *
>>> CoxeterGroup(['F',Integer(4)]).is_real()
True
>>> CoxeterGroup(['H',Integer(4)]).is_real()
True
```

long_element(*index_set=None*, *as_word=False*)

Return the longest element of `self`, or of the parabolic subgroup corresponding to the given `index_set`.

INPUT:

- `index_set` – a subset (as a list or iterable) of the nodes of the Dynkin diagram; (default: all of them)
- `as_word` – boolean (default: `False`); if `True`, then return instead a reduced decomposition of the longest element

Should this method be called maximal_element? longest_element?

EXAMPLES:

```
sage: D10 = FiniteCoxeterGroups().example(10)
sage: D10.long_element()
(1, 2, 1, 2, 1, 2, 1, 2, 1, 2)
sage: D10.long_element([1])
(1,)
sage: D10.long_element([2])
(2,)
sage: D10.long_element([])
()

sage: D7 = FiniteCoxeterGroups().example(7)
sage: D7.long_element()
(1, 2, 1, 2, 1, 2, 1)
```

```
>>> from sage.all import *
>>> D10 = FiniteCoxeterGroups().example(Integer(10))
>>> D10.long_element()
(1, 2, 1, 2, 1, 2, 1, 2, 1, 2)
>>> D10.long_element([Integer(1)])
(1,)
>>> D10.long_element([Integer(2)])
```

(continues on next page)

(continued from previous page)

```
(2,)
>>> D10.long_element([])
()

>>> D7 = FiniteCoxeterGroups().example(Integer(7))
>>> D7.long_element()
(1, 2, 1, 2, 1, 2, 1)
```

One can require instead a reduced word for w0:

```
sage: A3 = CoxeterGroup(['A', 3])
sage: A3.long_element(as_word=True)
[1, 2, 1, 3, 2, 1]
```

```
>>> from sage.all import *
>>> A3 = CoxeterGroup(['A', Integer(3)])
>>> A3.long_element(as_word=True)
[1, 2, 1, 3, 2, 1]
```

m_cambrian_lattice(*c*, *m=1*, *on_roots=False*)

Return the m-Cambrian lattice on m-delta sequences.

See arXiv 1503.00710 and arXiv math/0611106.

The m-delta sequences are certain m-colored minimal factorizations of c into reflections.

INPUT:

- `c` – a Coxeter element of `self` (as a tuple, or as an element of `self`)
- `m` – positive integer (default: 1)
- `on_roots` – boolean (default: `False`); if `on_roots` is `True`, the lattice is realized on roots rather than on reflections. In order for this to work, the ElementMethod `reflection_to_root` must be available.

EXAMPLES:

```
sage: CoxeterGroup(["A",2]).m_cambrian_lattice((1,2))
Finite lattice containing 5 elements

sage: CoxeterGroup(["A",2]).m_cambrian_lattice((1,2),2)
Finite lattice containing 12 elements
```

```
>>> from sage.all import *
>>> CoxeterGroup(["A",Integer(2)]).m_cambrian_lattice((Integer(1),
↪Integer(2)))
Finite lattice containing 5 elements

>>> CoxeterGroup(["A",Integer(2)]).m_cambrian_lattice((Integer(1),
↪Integer(2)),Integer(2))
Finite lattice containing 12 elements
```

permutahedron(*point=None*, *base_ring=None*)

Return the permutahedron of `self`.

This is the convex hull of the point `point` in the weight basis under the action of `self` on the underlying vector space V.

See also

The corresponding `permutahedron` implementations for concrete reflection groups.

INPUT:

- `point` – (optional) a point given by its coordinates in the weight basis (default: $(1, 1, 1, \ldots)$)
- `base_ring` – (optional) the base ring of the polytope

Note

The result is expressed in the root basis coordinates.

Note

If function is too slow, switching the base ring to `RDF` will almost certainly speed things up.

EXAMPLES:

```
sage: W = CoxeterGroup(['H',3], base_ring=RDF)
sage: W.permutahedron()
doctest:warning
...
UserWarning: This polyhedron data is numerically complicated; cdd could␣
↪not convert between the inexact V and H representation without loss of␣
↪data. The resulting object might show inconsistencies.
A 3-dimensional polyhedron in RDF^3 defined as the convex hull of 120␣
↪vertices

sage: W = CoxeterGroup(['I',7])
sage: W.permutahedron()
A 2-dimensional polyhedron in AA^2 defined as the convex hull of 14␣
↪vertices
sage: W.permutahedron(base_ring=RDF)
A 2-dimensional polyhedron in RDF^2 defined as the convex hull of 14␣
↪vertices

sage: W = ReflectionGroup(['A',3])                          # optional -␣
↪gap3
sage: W.permutahedron()                                     # optional -␣
↪gap3
A 3-dimensional polyhedron in QQ^3 defined as the convex hull
of 24 vertices

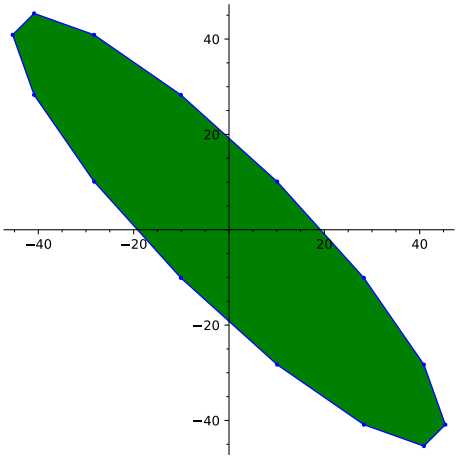
sage: W = ReflectionGroup(['A',3],['B',2])                  # optional -␣
↪gap3
sage: W.permutahedron()                                     # optional -␣
↪gap3
A 5-dimensional polyhedron in QQ^5 defined as the convex hull of 192␣
↪vertices
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['H',Integer(3)], base_ring=RDF)
>>> W.permutahedron()
doctest:warning
...
UserWarning: This polyhedron data is numerically complicated; cdd could␣
↪not convert between the inexact V and H representation without loss of␣
↪data. The resulting object might show inconsistencies.
A 3-dimensional polyhedron in RDF^3 defined as the convex hull of 120␣
↪vertices

>>> W = CoxeterGroup(['I',Integer(7)])
>>> W.permutahedron()
A 2-dimensional polyhedron in AA^2 defined as the convex hull of 14␣
↪vertices
>>> W.permutahedron(base_ring=RDF)
A 2-dimensional polyhedron in RDF^2 defined as the convex hull of 14␣
↪vertices

>>> W = ReflectionGroup(['A',Integer(3)])                              #␣
↪optional - gap3
>>> W.permutahedron()                                          # optional -␣
↪gap3
A 3-dimensional polyhedron in QQ^3 defined as the convex hull
of 24 vertices

>>> W = ReflectionGroup(['A',Integer(3)],['B',Integer(2)])                ␣
↪  # optional - gap3
>>> W.permutahedron()                                          # optional -␣
↪gap3
A 5-dimensional polyhedron in QQ^5 defined as the convex hull of 192␣
↪vertices
```



reflections_from_w0()

Return the reflections of `self` using the inversion set of `w_0`.

EXAMPLES:

```
sage: WeylGroup(['A',2]).reflections_from_w0()
[
[0 1 0]  [0 0 1]  [1 0 0]
[1 0 0]  [0 1 0]  [0 0 1]
[0 0 1], [1 0 0], [0 1 0]
]

sage: WeylGroup(['A',3]).reflections_from_w0()
[
[0 1 0 0]  [0 0 1 0]  [1 0 0 0]  [0 0 0 1]  [1 0 0 0]  [1 0 0 0]
[1 0 0 0]  [0 1 0 0]  [0 0 1 0]  [0 1 0 0]  [0 0 0 1]  [0 1 0 0]
[0 0 1 0]  [1 0 0 0]  [0 1 0 0]  [0 0 1 0]  [0 0 1 0]  [0 0 0 1]
[0 0 0 1], [0 0 0 1], [0 0 0 1], [1 0 0 0], [0 1 0 0], [0 0 1 0]
]
```

```
>>> from sage.all import *
>>> WeylGroup(['A',Integer(2)]).reflections_from_w0()
[
[0 1 0]  [0 0 1]  [1 0 0]
[1 0 0]  [0 1 0]  [0 0 1]
[0 0 1], [1 0 0], [0 1 0]
]

>>> WeylGroup(['A',Integer(3)]).reflections_from_w0()
[
[0 1 0 0]  [0 0 1 0]  [1 0 0 0]  [0 0 0 1]  [1 0 0 0]  [1 0 0 0]
[1 0 0 0]  [0 1 0 0]  [0 0 1 0]  [0 1 0 0]  [0 0 0 1]  [0 1 0 0]
[0 0 1 0]  [1 0 0 0]  [0 1 0 0]  [0 0 1 0]  [0 0 1 0]  [0 0 0 1]
[0 0 0 1], [0 0 0 1], [0 0 0 1], [1 0 0 0], [0 1 0 0], [0 0 1 0]
]
```

shard_poset(*side='right'*)

Return the shard intersection order attached to W.

This is a lattice structure on W, introduced in [Rea2009]. It contains the noncrossing partition lattice, as the induced lattice on the subset of c-sortable elements.

The partial order is given by simultaneous inclusion of inversion sets and subgroups attached to every element.

The precise description used here can be found in [STW2018].

Another implementation for the symmetric groups is available as `shard_poset()`.

See also

bhz_poset(), *bruhat_poset()*, *weak_poset()*

EXAMPLES:

```
sage: W = CoxeterGroup(['A',3], base_ring=ZZ)
sage: SH = W.shard_poset(); SH
Finite lattice containing 24 elements
```

(continues on next page)

(continued from previous page)

```
sage: SH.is_graded()
True
sage: SH.characteristic_polynomial()
q^3 - 11*q^2 + 23*q - 13
sage: SH.f_polynomial()
34*q^3 + 22*q^2 + q
```

```
>>> from sage.all import *
>>> W = CoxeterGroup(['A',Integer(3)], base_ring=ZZ)
>>> SH = W.shard_poset(); SH
Finite lattice containing 24 elements
>>> SH.is_graded()
True
>>> SH.characteristic_polynomial()
q^3 - 11*q^2 + 23*q - 13
>>> SH.f_polynomial()
34*q^3 + 22*q^2 + q
```

w0()

Return the longest element of `self`.

This attribute is deprecated, use *long_element()* instead.

EXAMPLES:

```
sage: D8 = FiniteCoxeterGroups().example(8)
sage: D8.w0
(1, 2, 1, 2, 1, 2, 1, 2)
sage: D3 = FiniteCoxeterGroups().example(3)
sage: D3.w0
(1, 2, 1)
```

```
>>> from sage.all import *
>>> D8 = FiniteCoxeterGroups().example(Integer(8))
>>> D8.w0
(1, 2, 1, 2, 1, 2, 1, 2)
>>> D3 = FiniteCoxeterGroups().example(Integer(3))
>>> D3.w0
(1, 2, 1)
```

weak_lattice(*side='right'*, *facade=False*)

alias of *weak_poset()*.

weak_poset(*side='right'*, *facade=False*)

Return the left (resp. right) poset for weak order.

INPUT:

- `side` – "left", "right", or "twosided" (default: `'right'`)
- `facade` – boolean (default: `False`)

In this poset, u is smaller than v if some reduced word of u is a right (resp. left) factor of some reduced word of v.

See also

bhz_poset(), *bruhat_poset()*, *shard_poset()*

EXAMPLES:

```
sage: W = WeylGroup(["A", 2])
sage: P = W.weak_poset()
sage: P
Finite lattice containing 6 elements
sage: P.show()
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A", Integer(2)])
>>> P = W.weak_poset()
>>> P
Finite lattice containing 6 elements
>>> P.show()
```

This poset is in fact a lattice:

```
sage: W = WeylGroup(["B", 3])
sage: P = W.weak_poset(side = "left")
sage: P.is_lattice()
True
```

```
>>> from sage.all import *
>>> W = WeylGroup(["B", Integer(3)])
>>> P = W.weak_poset(side = "left")
>>> P.is_lattice()
True
```

so this method has an alias *weak_lattice()*:

```
sage: W.weak_lattice(side = "left") is W.weak_poset(side = "left")
True
```

```
>>> from sage.all import *
>>> W.weak_lattice(side = "left") is W.weak_poset(side = "left")
True
```

As a bonus feature, one can create the left-right weak poset:

```
sage: W = WeylGroup(["A",2])
sage: P = W.weak_poset(side = "twosided")
sage: P.show()
sage: len(P.hasse_diagram().edges(sort=False))
8
```

```
>>> from sage.all import *
>>> W = WeylGroup(["A",Integer(2)])
>>> P = W.weak_poset(side = "twosided")
```

(continues on next page)

(continued from previous page)

```
>>> P.show()
>>> len(P.hasse_diagram().edges(sort=False))
8
```

This is the transitive closure of the union of left and right order. In this poset, u is smaller than v if some reduced word of u is a factor of some reduced word of v. Note that this is not a lattice:

```
sage: P.is_lattice()
False
```

```
>>> from sage.all import *
>>> P.is_lattice()
False
```

By default, the elements of P are aware of that they belong to P:

```
sage: P.an_element().parent()
Finite poset containing 6 elements
```

```
>>> from sage.all import *
>>> P.an_element().parent()
Finite poset containing 6 elements
```

If instead one wants the elements to be plain elements of the Coxeter group, one can use the `facade` option:

```
sage: P = W.weak_poset(facade = True)
sage: P.an_element().parent()
Weyl Group of type ['A', 2] (as a matrix group acting on the ambient␣
↪space)
```

```
>>> from sage.all import *
>>> P = W.weak_poset(facade = True)
>>> P.an_element().parent()
Weyl Group of type ['A', 2] (as a matrix group acting on the ambient␣
↪space)
```

See also

`Poset()` for more on posets and facade posets.

Todo

- Use the symmetric group in the examples (for nicer output), and print the edges for a stronger test.
- The constructed poset should be lazy, in order to handle large / infinite Coxeter groups.

extra_super_categories()

EXAMPLES:

```
sage: CoxeterGroups().Finite().super_categories()
[Category of finite generalized Coxeter groups,
 Category of Coxeter groups]
```

```
>>> from sage.all import *
>>> CoxeterGroups().Finite().super_categories()
[Category of finite generalized Coxeter groups,
 Category of Coxeter groups]
```

4.51 Finite Crystals

class sage.categories.finite_crystals.**FiniteCrystals**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite crystals.

EXAMPLES:

```
sage: C = FiniteCrystals()
sage: C
Category of finite crystals
sage: C.super_categories()
[Category of crystals, Category of finite enumerated sets]
sage: C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> C = FiniteCrystals()
>>> C
Category of finite crystals
>>> C.super_categories()
[Category of crystals, Category of finite enumerated sets]
>>> C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of finite crystals constructed by tensor product of finite crystals.

extra_super_categories()

EXAMPLES:

```
sage: FiniteCrystals().TensorProducts().extra_super_categories()
[Category of finite crystals]
```

```
>>> from sage.all import *
>>> FiniteCrystals().TensorProducts().extra_super_categories()
[Category of finite crystals]
```

example(*n=3*)

Return an example of highest weight crystals, as per *Category.example()*.

EXAMPLES:

```
sage: B = FiniteCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> B = FiniteCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

extra_super_categories()

EXAMPLES:

```
sage: FiniteCrystals().extra_super_categories()
[Category of finite enumerated sets]
```

```
>>> from sage.all import *
>>> FiniteCrystals().extra_super_categories()
[Category of finite enumerated sets]
```

4.52 Finite dimensional algebras with basis

Todo

Quotients of polynomial rings.

Quotients in general.

Matrix rings.

REFERENCES:

- [CR1962]

class sage.categories.finite_dimensional_algebras_with_basis.**FiniteDimensionalAlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of finite dimensional algebras with a distinguished basis.

These algebras are assumed to be associative and unital.

EXAMPLES:

```
sage: C = FiniteDimensionalAlgebrasWithBasis(QQ); C
Category of finite dimensional algebras with basis over Rational Field
sage: C.super_categories()
[Category of algebras with basis over Rational Field,
 Category of finite dimensional magmatic algebras with basis over Rational Field]
sage: C.example()                                                          #␣
↪needs sage.modules
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
```

```
>>> from sage.all import *
>>> C = FiniteDimensionalAlgebrasWithBasis(QQ); C
Category of finite dimensional algebras with basis over Rational Field
>>> C.super_categories()
[Category of algebras with basis over Rational Field,
 Category of finite dimensional magmatic algebras with basis over Rational Field]
>>> C.example()                                                              #␣
↪needs sage.modules
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
```

class Cellular(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Cellular algebras.

Let R be a commutative ring. A R-algebra A is a *cellular algebra* if it has a *cell datum*, which is a tuple (Λ, i, M, C), where Λ is finite poset with order $\geq$, if $\mu \in \Lambda$ then $T(\mu)$ is a finite set and

$$C\colon \coprod_{\mu\in\Lambda} T(\mu) \times T(\mu) \longrightarrow A; (\mu, s, t) \mapsto c_{st}^{\mu} \text{ is an injective map}$$

such that the following holds:

- The set $\{c_{st}^{\mu} \mid \mu \in \Lambda, s, t \in T(\mu)\}$ is a basis of A.
- If $a \in A$ and $\mu \in \Lambda, s, t \in T(\mu)$ then:

$$ac_{st}^{\mu} = \sum_{u\in T(\mu)} r_a(s,u)c_{ut}^{\mu} \pmod{A^{>\mu}},$$

  where $A^{>\mu}$ is spanned by

$$\{c_{ab}^{\nu} \mid \nu > \mu \text{ and } a, b \in T(\nu)\}.$$

  Moreover, the scalar $r_a(s,u)$ depends only on a, s and u and, in particular, is independent of t.

- The map $\iota\colon A \longrightarrow A; c_{st}^{\mu} \mapsto c_{ts}^{\mu}$ is an algebra anti-isomorphism.

A *cellular basis* for A is any basis of the form $\{c_{st}^{\mu} \mid \mu \in \Lambda, s, t \in T(\mu)\}$.

Note that in particular, the scalars $r_a(u,s)$ in the second condition do not depend on t.

REFERENCES:

- [GrLe1996]
- [KX1998]
- [Mat1999]
- Wikipedia article Cellular_algebra
- http://webusers.imj-prg.fr/~bernhard.keller/ictp2006/lecturenotes/xi.pdf

class ElementMethods

Bases: object

cellular_involution()

Return the cellular involution on `self`.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S = SymmetricGroupAlgebra(QQ, 4)
sage: elt = S([3,1,2,4])
sage: ci = elt.cellular_involution(); ci
7/48*[1, 3, 2, 4] + 49/48*[2, 3, 1, 4]
 - 1/48*[3, 1, 2, 4] - 7/48*[3, 2, 1, 4]
sage: ci.cellular_involution()
[3, 1, 2, 4]
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S = SymmetricGroupAlgebra(QQ, Integer(4))
>>> elt = S([Integer(3),Integer(1),Integer(2),Integer(4)])
>>> ci = elt.cellular_involution(); ci
7/48*[1, 3, 2, 4] + 49/48*[2, 3, 1, 4]
 - 1/48*[3, 1, 2, 4] - 7/48*[3, 2, 1, 4]
>>> ci.cellular_involution()
[3, 1, 2, 4]
```

class ParentMethods

Bases: `object`

cell_module(*mu*, ***kwds*)

Return the cell module indexed by `mu`.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 3)                                  #␣
↪needs sage.groups sage.modules
sage: S.cell_module(Partition([2,1]))                                   #␣
↪needs sage.combinat sage.groups sage.modules
Cell module indexed by [2, 1] of Cellular basis of
 Symmetric group algebra of order 3 over Rational Field
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(3))                            ␣
↪    # needs sage.groups sage.modules
>>> S.cell_module(Partition([Integer(2),Integer(1)]))                    ␣
↪             # needs sage.combinat sage.groups sage.modules
Cell module indexed by [2, 1] of Cellular basis of
 Symmetric group algebra of order 3 over Rational Field
```

cell_module_indices(*mu*)

Return the indices of the cell module of `self` indexed by `mu` .

This is the finite set $M(\lambda)$.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 3)                                  #␣
↪needs sage.groups sage.modules
sage: S.cell_module_indices([2,1])                                      #␣
↪needs sage.groups sage.modules
Standard tableaux of shape [2, 1]
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(3))                           ␣
↪    # needs sage.groups sage.modules
>>> S.cell_module_indices([Integer(2),Integer(1)])                      ␣
↪             # needs sage.groups sage.modules
Standard tableaux of shape [2, 1]
```

cell_poset()

Return the cell poset of `self`.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 4)                                  #␣
↪needs sage.groups sage.modules
sage: S.cell_poset()                                                    #␣
↪needs sage.groups sage.modules
Finite poset containing 5 elements
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(4))                           ␣
↪    # needs sage.groups sage.modules
>>> S.cell_poset()                                                     #␣
↪needs sage.groups sage.modules
Finite poset containing 5 elements
```

cells()

Return the cells of `self`.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 3)                                  #␣
↪needs sage.groups sage.modules
sage: dict(S.cells())                                                   #␣
↪needs sage.groups sage.modules
{[1, 1, 1]: Standard tableaux of shape [1, 1, 1],
 [2, 1]: Standard tableaux of shape [2, 1],
 [3]: Standard tableaux of shape [3]}
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(3))                           ␣
↪    # needs sage.groups sage.modules
>>> dict(S.cells())                                                    #␣
↪needs sage.groups sage.modules
{[1, 1, 1]: Standard tableaux of shape [1, 1, 1],
 [2, 1]: Standard tableaux of shape [2, 1],
 [3]: Standard tableaux of shape [3]}
```

cellular_basis()

Return the cellular basis of `self`.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 3)                                  #␣
↪needs sage.groups sage.modules
sage: S.cellular_basis()                                                #␣
↪needs sage.groups sage.modules
Cellular basis of Symmetric group algebra of order 3
 over Rational Field
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(3))                           ␣
↪    # needs sage.groups sage.modules
>>> S.cellular_basis()                                                 #␣
↪needs sage.groups sage.modules
Cellular basis of Symmetric group algebra of order 3
 over Rational Field
```

cellular_involution(x)

Return the cellular involution of `x` in `self`.

EXAMPLES:

```
sage: S = SymmetricGroupAlgebra(QQ, 3)                                  #␣
↪needs sage.groups sage.modules
sage: for b in S.basis(): b, S.cellular_involution(b)                   #␣
↪needs sage.groups sage.modules
([1, 2, 3], [1, 2, 3])
([1, 3, 2], 49/48*[1, 3, 2] + 7/48*[2, 3, 1]
            - 7/48*[3, 1, 2] - 1/48*[3, 2, 1])
([2, 1, 3], [2, 1, 3])
([2, 3, 1], -7/48*[1, 3, 2] - 1/48*[2, 3, 1]
             + 49/48*[3, 1, 2] + 7/48*[3, 2, 1])
([3, 1, 2], 7/48*[1, 3, 2] + 49/48*[2, 3, 1]
             - 1/48*[3, 1, 2] - 7/48*[3, 2, 1])
([3, 2, 1], -1/48*[1, 3, 2] - 7/48*[2, 3, 1]
             + 7/48*[3, 1, 2] + 49/48*[3, 2, 1])
```

```
>>> from sage.all import *
>>> S = SymmetricGroupAlgebra(QQ, Integer(3))                           ␣
↪    # needs sage.groups sage.modules
>>> for b in S.basis(): b, S.cellular_involution(b)                     #␣
↪needs sage.groups sage.modules
([1, 2, 3], [1, 2, 3])
([1, 3, 2], 49/48*[1, 3, 2] + 7/48*[2, 3, 1]
            - 7/48*[3, 1, 2] - 1/48*[3, 2, 1])
([2, 1, 3], [2, 1, 3])
([2, 3, 1], -7/48*[1, 3, 2] - 1/48*[2, 3, 1]
             + 49/48*[3, 1, 2] + 7/48*[3, 2, 1])
([3, 1, 2], 7/48*[1, 3, 2] + 49/48*[2, 3, 1]
             - 1/48*[3, 1, 2] - 7/48*[3, 2, 1])
```

(continues on next page)

(continued from previous page)

```
([3, 2, 1], -1/48*[1, 3, 2] - 7/48*[2, 3, 1]
             + 7/48*[3, 1, 2] + 49/48*[3, 2, 1])
```

simple_module_parameterization()

Return a parameterization of the simple modules of `self`.

The set of simple modules are parameterized by $\lambda \in \Lambda$ such that the cell module bilinear form $\Phi_\lambda \neq 0$.

EXAMPLES:

```
sage: # needs sage.modules
sage: TL = TemperleyLiebAlgebra(5, 30, QQ)  # semisimple
sage: len(TL.radical_basis())
0
sage: TL.simple_module_parameterization()
(1, 3, 5)

sage: # needs sage.modules
sage: TL = TemperleyLiebAlgebra(5, 1, QQ)  # not semisimple
sage: len(TL.radical_basis())
24
sage: TL.simple_module_parameterization()
(1, 3, 5)

sage: # needs sage.modules
sage: TL = TemperleyLiebAlgebra(6, 30, QQ)  # semisimple
sage: all(TL.cell_module(la).dimension()
....:     == TL.cell_module(la).simple_module().dimension()
....:     for la in TL.simple_module_parameterization())
True
sage: TL.simple_module_parameterization()
(0, 2, 4, 6)

sage: # needs sage.modules
sage: TL = TemperleyLiebAlgebra(6, 0, QQ)  # not semisimple
sage: TL.simple_module_parameterization()
(2, 4, 6)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> TL = TemperleyLiebAlgebra(Integer(5), Integer(30), QQ)  #␣
↪semisimple
>>> len(TL.radical_basis())
0
>>> TL.simple_module_parameterization()
(1, 3, 5)

>>> # needs sage.modules
>>> TL = TemperleyLiebAlgebra(Integer(5), Integer(1), QQ)  # not␣
↪semisimple
>>> len(TL.radical_basis())
24
```

(continues on next page)

(continued from previous page)

```
>>> TL.simple_module_parameterization()
(1, 3, 5)

>>> # needs sage.modules
>>> TL = TemperleyLiebAlgebra(Integer(6), Integer(30), QQ)  #␣
↪semisimple
>>> all(TL.cell_module(la).dimension()
...      == TL.cell_module(la).simple_module().dimension()
...      for la in TL.simple_module_parameterization())
True
>>> TL.simple_module_parameterization()
(0, 2, 4, 6)

>>> # needs sage.modules
>>> TL = TemperleyLiebAlgebra(Integer(6), Integer(0), QQ)  # not␣
↪semisimple
>>> TL.simple_module_parameterization()
(2, 4, 6)
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of cellular algebras constructed by tensor product of cellular algebras.

class ParentMethods

Bases: object

cell_module_indices(*mu*)

Return the indices of the cell module of `self` indexed by `mu` .

This is the finite set $M(\lambda)$.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S2 = SymmetricGroupAlgebra(QQ, 2)
sage: S3 = SymmetricGroupAlgebra(QQ, 3)
sage: T = S2.tensor(S3)
sage: T.cell_module_indices(([1,1], [2,1]))
The Cartesian product of (Standard tableaux of shape [1, 1],
                          Standard tableaux of shape [2, 1])
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S2 = SymmetricGroupAlgebra(QQ, Integer(2))
>>> S3 = SymmetricGroupAlgebra(QQ, Integer(3))
>>> T = S2.tensor(S3)
>>> T.cell_module_indices(([Integer(1),Integer(1)], [Integer(2),
↪Integer(1)]))
The Cartesian product of (Standard tableaux of shape [1, 1],
                          Standard tableaux of shape [2, 1])
```

cell_poset()

Return the cell poset of `self`.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S2 = SymmetricGroupAlgebra(QQ, 2)
sage: S3 = SymmetricGroupAlgebra(QQ, 3)
sage: T = S2.tensor(S3)
sage: T.cell_poset()                                                    #␣
↪needs sage.combinat sage.graphs
Finite poset containing 6 elements
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S2 = SymmetricGroupAlgebra(QQ, Integer(2))
>>> S3 = SymmetricGroupAlgebra(QQ, Integer(3))
>>> T = S2.tensor(S3)
>>> T.cell_poset()                                                      #␣
↪needs sage.combinat sage.graphs
Finite poset containing 6 elements
```

cellular_involution()

Return the image of the cellular involution of the basis element indexed by `i`.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S2 = SymmetricGroupAlgebra(QQ, 2)
sage: S3 = SymmetricGroupAlgebra(QQ, 3)
sage: T = S2.tensor(S3)
sage: for b in T.basis(): b, T.cellular_involution(b)
([1, 2] # [1, 2, 3], [1, 2] # [1, 2, 3])
([1, 2] # [1, 3, 2],
 49/48*[1, 2] # [1, 3, 2] + 7/48*[1, 2] # [2, 3, 1]
  - 7/48*[1, 2] # [3, 1, 2] - 1/48*[1, 2] # [3, 2, 1])
([1, 2] # [2, 1, 3], [1, 2] # [2, 1, 3])
([1, 2] # [2, 3, 1],
 -7/48*[1, 2] # [1, 3, 2] - 1/48*[1, 2] # [2, 3, 1]
  + 49/48*[1, 2] # [3, 1, 2] + 7/48*[1, 2] # [3, 2, 1])
([1, 2] # [3, 1, 2],
 7/48*[1, 2] # [1, 3, 2] + 49/48*[1, 2] # [2, 3, 1]
  - 1/48*[1, 2] # [3, 1, 2] - 7/48*[1, 2] # [3, 2, 1])
([1, 2] # [3, 2, 1],
 -1/48*[1, 2] # [1, 3, 2] - 7/48*[1, 2] # [2, 3, 1]
  + 7/48*[1, 2] # [3, 1, 2] + 49/48*[1, 2] # [3, 2, 1])
([2, 1] # [1, 2, 3], [2, 1] # [1, 2, 3])
([2, 1] # [1, 3, 2],
 49/48*[2, 1] # [1, 3, 2] + 7/48*[2, 1] # [2, 3, 1]
  - 7/48*[2, 1] # [3, 1, 2] - 1/48*[2, 1] # [3, 2, 1])
([2, 1] # [2, 1, 3], [2, 1] # [2, 1, 3])
([2, 1] # [2, 3, 1],
 -7/48*[2, 1] # [1, 3, 2] - 1/48*[2, 1] # [2, 3, 1]
  + 49/48*[2, 1] # [3, 1, 2] + 7/48*[2, 1] # [3, 2, 1])
([2, 1] # [3, 1, 2],
 7/48*[2, 1] # [1, 3, 2] + 49/48*[2, 1] # [2, 3, 1]
  - 1/48*[2, 1] # [3, 1, 2] - 7/48*[2, 1] # [3, 2, 1])
```

(continues on next page)

(continued from previous page)

```
([2, 1] # [3, 2, 1],
 -1/48*[2, 1] # [1, 3, 2] - 7/48*[2, 1] # [2, 3, 1]
  + 7/48*[2, 1] # [3, 1, 2] + 49/48*[2, 1] # [3, 2, 1])
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S2 = SymmetricGroupAlgebra(QQ, Integer(2))
>>> S3 = SymmetricGroupAlgebra(QQ, Integer(3))
>>> T = S2.tensor(S3)
>>> for b in T.basis(): b, T.cellular_involution(b)
([1, 2] # [1, 2, 3], [1, 2] # [1, 2, 3])
([1, 2] # [1, 3, 2],
 49/48*[1, 2] # [1, 3, 2] + 7/48*[1, 2] # [2, 3, 1]
  - 7/48*[1, 2] # [3, 1, 2] - 1/48*[1, 2] # [3, 2, 1])
([1, 2] # [2, 1, 3], [1, 2] # [2, 1, 3])
([1, 2] # [2, 3, 1],
 -7/48*[1, 2] # [1, 3, 2] - 1/48*[1, 2] # [2, 3, 1]
  + 49/48*[1, 2] # [3, 1, 2] + 7/48*[1, 2] # [3, 2, 1])
([1, 2] # [3, 1, 2],
 7/48*[1, 2] # [1, 3, 2] + 49/48*[1, 2] # [2, 3, 1]
  - 1/48*[1, 2] # [3, 1, 2] - 7/48*[1, 2] # [3, 2, 1])
([1, 2] # [3, 2, 1],
 -1/48*[1, 2] # [1, 3, 2] - 7/48*[1, 2] # [2, 3, 1]
  + 7/48*[1, 2] # [3, 1, 2] + 49/48*[1, 2] # [3, 2, 1])
([2, 1] # [1, 2, 3], [2, 1] # [1, 2, 3])
([2, 1] # [1, 3, 2],
 49/48*[2, 1] # [1, 3, 2] + 7/48*[2, 1] # [2, 3, 1]
  - 7/48*[2, 1] # [3, 1, 2] - 1/48*[2, 1] # [3, 2, 1])
([2, 1] # [2, 1, 3], [2, 1] # [2, 1, 3])
([2, 1] # [2, 3, 1],
 -7/48*[2, 1] # [1, 3, 2] - 1/48*[2, 1] # [2, 3, 1]
  + 49/48*[2, 1] # [3, 1, 2] + 7/48*[2, 1] # [3, 2, 1])
([2, 1] # [3, 1, 2],
 7/48*[2, 1] # [1, 3, 2] + 49/48*[2, 1] # [2, 3, 1]
  - 1/48*[2, 1] # [3, 1, 2] - 7/48*[2, 1] # [3, 2, 1])
([2, 1] # [3, 2, 1],
 -1/48*[2, 1] # [1, 3, 2] - 7/48*[2, 1] # [2, 3, 1]
  + 7/48*[2, 1] # [3, 1, 2] + 49/48*[2, 1] # [3, 2, 1])
```

extra_super_categories()

Tensor products of cellular algebras are cellular.

EXAMPLES:

```
sage: cat = Algebras(QQ).FiniteDimensional().WithBasis()
sage: cat.Cellular().TensorProducts().extra_super_categories()
[Category of finite dimensional cellular algebras with basis
 over Rational Field]
```

```
>>> from sage.all import *
>>> cat = Algebras(QQ).FiniteDimensional().WithBasis()
>>> cat.Cellular().TensorProducts().extra_super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of finite dimensional cellular algebras with basis
 over Rational Field]
```

class ElementMethods

Bases: object

on_left_matrix(*base_ring=None*, *action=<built-in function mul>*, *side='left'*)

alias of *to_matrix()*.

to_matrix(*base_ring=None*, *action=<built-in function mul>*, *side='left'*)

Return the matrix of the action of `self` on the algebra.

INPUT:

- `base_ring` – the base ring for the matrix to be constructed
- `action` – a bivariate function (default: `operator.mul()`)
- `side` – `'left'` or `'right'` (default: `'left'`)

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: QS3 = SymmetricGroupAlgebra(QQ, 3)
sage: a = QS3([2,1,3])
sage: a.to_matrix(side='left')
[0 0 1 0 0 0]
[0 0 0 0 1 0]
[1 0 0 0 0 0]
[0 0 0 0 0 1]
[0 1 0 0 0 0]
[0 0 0 1 0 0]
sage: a.to_matrix(side='right')
[0 0 1 0 0 0]
[0 0 0 1 0 0]
[1 0 0 0 0 0]
[0 1 0 0 0 0]
[0 0 0 0 0 1]
[0 0 0 0 1 0]
sage: a.to_matrix(base_ring=RDF, side='left')
[0.0 0.0 1.0 0.0 0.0 0.0]
[0.0 0.0 0.0 0.0 1.0 0.0]
[1.0 0.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 0.0 0.0 1.0]
[0.0 1.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 1.0 0.0 0.0]
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> QS3 = SymmetricGroupAlgebra(QQ, Integer(3))
>>> a = QS3([Integer(2),Integer(1),Integer(3)])
>>> a.to_matrix(side='left')
[0 0 1 0 0 0]
[0 0 0 0 1 0]
[1 0 0 0 0 0]
[0 0 0 0 0 1]
[0 1 0 0 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 0 1 0 0]
>>> a.to_matrix(side='right')
[0 0 1 0 0 0]
[0 0 0 1 0 0]
[1 0 0 0 0 0]
[0 1 0 0 0 0]
[0 0 0 0 0 1]
[0 0 0 0 1 0]
>>> a.to_matrix(base_ring=RDF, side='left')
[0.0 0.0 1.0 0.0 0.0 0.0]
[0.0 0.0 0.0 0.0 1.0 0.0]
[1.0 0.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 0.0 0.0 1.0]
[0.0 1.0 0.0 0.0 0.0 0.0]
[0.0 0.0 0.0 1.0 0.0 0.0]
```

AUTHORS: Mike Hansen, …

class ParentMethods

Bases: object

cartan_invariants_matrix()

Return the Cartan invariants matrix of the algebra.

OUTPUT: a matrix of nonnegative integers

Todo

Both the implementation and the documentation assume that the algebra is split over its base field.

Let A be this finite-dimensional algebra and $(S_i)_{i\in I}$ be representatives of the right simple modules of A. Note that their adjoints S_i^* are representatives of the left simple modules.

Let $(P_i^L)_{i\in I}$ and $(P_i^R)_{i\in I}$ be respectively representatives of the corresponding indecomposable projective left and right modules of A. In particular, we assume that the indexing is consistent so that $S_i^* = \operatorname{top} P_i^L$ and $S_i = \operatorname{top} P_i^R$.

The *Cartan invariant matrix* $(C_{i,j})_{i,j\in I}$ is a matrix of nonnegative integers that encodes much of the representation theory of A; namely:

- $C_{i,j}$ counts how many times $S_i^* \otimes S_j$ appears as composition factor of A seen as a bimodule over itself;
- $C_{i,j} = \dim Hom_A(P_j^R, P_i^R)$;
- $C_{i,j}$ counts how many times S_j appears as composition factor of P_i^R;
- $C_{i,j} = \dim Hom_A(P_i^L, P_j^L)$;
- $C_{i,j}$ counts how many times S_i^* appears as composition factor of P_j^L.

In the commutative case, the Cartan invariant matrix is diagonal. In the context of solving systems of multivariate polynomial equations of dimension zero, A is the quotient of the polynomial ring by the ideal generated by the equations, the simple modules correspond to the roots, and the numbers $C_{i,i}$ give the multiplicities of those roots.

Note

For simplicity, the current implementation assumes that the index set I is of the form $\{0, \ldots, n-1\}$. Better indexations will be possible in the future.

ALGORITHM:

The Cartan invariant matrix of A is computed from the dimension of the summands of its Peirce decomposition.

See also

- *peirce_decomposition()*
- *isotypic_projective_modules()*

EXAMPLES:

For a semisimple algebra, in particular for group algebras in characteristic zero, the Cartan invariants matrix is the identity:

```
sage: A3 = SymmetricGroup(3).algebra(QQ)                                    #␣
↪needs sage.combinat sage.groups sage.modules
sage: A3.cartan_invariants_matrix()                                         #␣
↪needs sage.combinat sage.groups sage.modules
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> A3 = SymmetricGroup(Integer(3)).algebra(QQ)                              ␣
↪     # needs sage.combinat sage.groups sage.modules
>>> A3.cartan_invariants_matrix()                                          #␣
↪needs sage.combinat sage.groups sage.modules
[1 0 0]
[0 1 0]
[0 0 1]
```

For the path algebra of a quiver, the Cartan invariants matrix counts the number of paths between two vertices:

```
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example()
sage: A.cartan_invariants_matrix()                                          #␣
↪needs sage.modules sage.rings.number_field
[1 2]
[0 1]
```

```
>>> from sage.all import *
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example()
>>> A.cartan_invariants_matrix()                                           #␣
↪needs sage.modules sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
[1 2]
[0 1]
```

In the commutative case, the Cartan invariant matrix is diagonal:

```
sage: Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid: the integers modulo 12
sage: A = Z12.algebra(QQ)                                                   #␣
↪needs sage.modules
sage: A.cartan_invariants_matrix()                                          #␣
↪needs sage.modules sage.rings.number_field
[1 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0]
[0 0 2 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0]
[0 0 0 0 2 0 0 0 0]
[0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 2 0]
[0 0 0 0 0 0 0 0 1]
```

```
>>> from sage.all import *
>>> Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid: the integers modulo 12
>>> A = Z12.algebra(QQ)                                                     #␣
↪needs sage.modules
>>> A.cartan_invariants_matrix()                                            #␣
↪needs sage.modules sage.rings.number_field
[1 0 0 0 0 0 0 0 0]
[0 1 0 0 0 0 0 0 0]
[0 0 2 0 0 0 0 0 0]
[0 0 0 1 0 0 0 0 0]
[0 0 0 0 2 0 0 0 0]
[0 0 0 0 0 1 0 0 0]
[0 0 0 0 0 0 1 0 0]
[0 0 0 0 0 0 0 2 0]
[0 0 0 0 0 0 0 0 1]
```

With the algebra of the 0-Hecke monoid:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: from sage.monoids.hecke_monoid import HeckeMonoid
sage: A = HeckeMonoid(SymmetricGroup(4)).algebra(QQ)
sage: A.cartan_invariants_matrix()                                          #␣
↪needs sage.rings.number_field
[1 0 0 0 0 0 0 0]
[0 2 1 0 1 1 0 0]
[0 1 1 0 1 0 0 0]
[0 0 0 1 0 1 1 0]
[0 1 1 0 1 0 0 0]
[0 1 0 1 0 2 1 0]
[0 0 0 1 0 1 1 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 0 0 0 0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> from sage.monoids.hecke_monoid import HeckeMonoid
>>> A = HeckeMonoid(SymmetricGroup(Integer(4))).algebra(QQ)
>>> A.cartan_invariants_matrix()                                    #␣
↪needs sage.rings.number_field
[1 0 0 0 0 0 0 0]
[0 2 1 0 1 1 0 0]
[0 1 1 0 1 0 0 0]
[0 0 0 1 0 1 1 0]
[0 1 1 0 1 0 0 0]
[0 1 0 1 0 2 1 0]
[0 0 0 1 0 1 1 0]
[0 0 0 0 0 0 0 1]
```

center()

Return the center of `self`.

See also

center_basis()

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: center = A.center(); center
Center of An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: center in Algebras(QQ).WithBasis().FiniteDimensional().Commutative()
True
sage: center.dimension()
1
sage: center.basis()
Finite family {0: B[0]}
sage: center.ambient() is A
True
sage: [c.lift() for c in center.basis()]
[x + y]
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
```

(continues on next page)

(continued from previous page)

```
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> center = A.center(); center
Center of An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> center in Algebras(QQ).WithBasis().FiniteDimensional().Commutative()
True
>>> center.dimension()
1
>>> center.basis()
Finite family {0: B[0]}
>>> center.ambient() is A
True
>>> [c.lift() for c in center.basis()]
[x + y]
```

The center of a semisimple algebra is semisimple:

```
sage: A = DihedralGroup(6).algebra(QQ)                                  #␣
↪needs sage.groups sage.modules
sage: A.center() in Algebras(QQ).Semisimple()                           #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> A = DihedralGroup(Integer(6)).algebra(QQ)                            ␣
↪     # needs sage.groups sage.modules
>>> A.center() in Algebras(QQ).Semisimple()                            #␣
↪needs sage.groups sage.modules
True
```

Todo

- Pickling by construction, as `A.center()`?
- Lazy evaluation of `_repr_`

center_basis()

Return a basis of the center of `self`.

OUTPUT: list of elements of `self`

See also

center()

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
```

(continues on next page)

(continued from previous page)

```
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: A.center_basis()
(x + y,)
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> A.center_basis()
(x + y,)
```

ideal_submodule(*gens*, *side='left'*, *category=None*, *algorithm=None*, **args*, ***opts*)

Return the `side` ideal of `self` generated by `gens` as a submodule.

INPUT:

- `gens` – iterable containing elements of `self` or a single element of `self`
- `side` – string; either `'left'` or `'right'` or `'twosided'`
- `algorithm` – string (optional); must be one of the following:
  - `"generators"` – generate the ideal by successively enlarging the basis by multiplying by the generators of the algebra
  - `"basis" -- multiply all elements in ``gens` by the basis of `self`

If `algorithm` is not specified, if the dimension of `self` is ≤ 10, then this uses the `"basis"` algorithm and otherwise it uses the `"generators"` algorithm.

Todo

This is not generally compatible with the implementation of the ideals. This method should be folded into the `ideal` method after the corresponding classes are refactored to be compatible.

EXAMPLES:

```
sage: scoeffs = {('a','e'): {'a':1}, ('b','e'): {'a':1, 'b':1},
....:            ('c','d'): {'a':1}, ('c','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: MS = MatrixSpace(QQ, 5)
sage: gens = [bg.adjoint_matrix() for bg in L.lie_algebra_generators()]
sage: I = MS.ideal_submodule(gens)
sage: I.dimension()
25

sage: I2 = MS.ideal_submodule(gens, algorithm="basis")
sage: I2.dimension()
25
sage: I.is_equal_subspace(I2)
True
```

```
>>> from sage.all import *
>>> scoeffs = {('a','e'): {'a':Integer(1)}, ('b','e'): {'a':Integer(1), 'b
↪':Integer(1)},
...             ('c','d'): {'a':Integer(1)}, ('c','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> MS = MatrixSpace(QQ, Integer(5))
>>> gens = [bg.adjoint_matrix() for bg in L.lie_algebra_generators()]
>>> I = MS.ideal_submodule(gens)
>>> I.dimension()
25

>>> I2 = MS.ideal_submodule(gens, algorithm="basis")
>>> I2.dimension()
25
>>> I.is_equal_subspace(I2)
True
```

We demonstrate the ideal property by using the `reduce()` method to check for containment:

```
sage: E.<w,x,y,z> = ExteriorAlgebra(QQ)
sage: gens = E.algebra_generators()
sage: elts = [x*y + z]
sage: LI = E.ideal_submodule(elts, side="left")
sage: all(LI.reduce(g * b.lift()) == 0 for b in LI.basis() for g in gens)
True
sage: all(LI.reduce(b.lift() * g) == 0 for b in LI.basis() for g in gens)
False
sage: RI = E.ideal_submodule(elts, side="right")
sage: all(RI.reduce(g * b.lift()) == 0 for b in RI.basis() for g in gens)
False
sage: all(RI.reduce(b.lift() * g) == 0 for b in RI.basis() for g in gens)
True
sage: TI = E.ideal_submodule(elts, side="twosided")
sage: all(TI.reduce(g * b.lift()) == 0 for b in TI.basis() for g in gens)
True
sage: all(TI.reduce(b.lift() * g) == 0 for b in TI.basis() for g in gens)
True
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('w', 'x', 'y', 'z',)); (w, x, y, z,) =␣
↪E._first_ngens(4)
>>> gens = E.algebra_generators()
>>> elts = [x*y + z]
>>> LI = E.ideal_submodule(elts, side="left")
>>> all(LI.reduce(g * b.lift()) == Integer(0) for b in LI.basis() for g␣
↪in gens)
True
>>> all(LI.reduce(b.lift() * g) == Integer(0) for b in LI.basis() for g␣
↪in gens)
False
>>> RI = E.ideal_submodule(elts, side="right")
>>> all(RI.reduce(g * b.lift()) == Integer(0) for b in RI.basis() for g␣
```

(continues on next page)

(continued from previous page)

```
↪in gens)
False
>>> all(RI.reduce(b.lift() * g) == Integer(0) for b in RI.basis() for g␣
↪in gens)
True
>>> TI = E.ideal_submodule(elts, side="twosided")
>>> all(TI.reduce(g * b.lift()) == Integer(0) for b in TI.basis() for g␣
↪in gens)
True
>>> all(TI.reduce(b.lift() * g) == Integer(0) for b in TI.basis() for g␣
↪in gens)
True
```

idempotent_lift(x)

Lift an idempotent of the semisimple quotient into an idempotent of `self`.

Let A be this finite dimensional algebra and π be the projection $A \to \overline{A}$ on its semisimple quotient. Let $\overline{x}$ be an idempotent of $\overline{A}$, and x any lift thereof in A. This returns an idempotent e of A such that $\pi(e) = \pi(x)$ and e is a polynomial in x.

INPUT:

- `x` – an element of A that projects on an idempotent $\overline{x}$ of the semisimple quotient of A. Alternatively one may give as input the idempotent $\overline{x}$, in which case some lift thereof will be taken for x.

OUTPUT: the idempotent e of `self`

ALGORITHM:

Iterate the formula $1 - (1 - x^2)^2$ until having an idempotent.

See [CR1962] for correctness and termination proofs.

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example()
sage: S = A.semisimple_quotient()
sage: A.idempotent_lift(S.basis()['x'])
x
sage: A.idempotent_lift(A.basis()['y'])
y
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example()
>>> S = A.semisimple_quotient()
>>> A.idempotent_lift(S.basis()['x'])
x
>>> A.idempotent_lift(A.basis()['y'])
y
```

A less trivial example:

```
sage: B = DescentAlgebra(QQ, 4).B()
sage: a = 1/8*B[[1, 1, 1, 1]] - 1/2*B[[2, 1, 1]] + 1/2*B[[2, 2]]
sage: a ** 2 == a  # not idempotent, but idempotent mod radical
```

(continues on next page)

(continued from previous page)

```
False
sage: al = B.idempotent_lift(a)
sage: al ** 2 == al
True
sage: (a - al) ** 2
0
```

```
>>> from sage.all import *
>>> B = DescentAlgebra(QQ, Integer(4)).B()
>>> a = Integer(1)/Integer(8)*B[[Integer(1), Integer(1), Integer(1),␣
↪Integer(1)]] - Integer(1)/Integer(2)*B[[Integer(2), Integer(1),␣
↪Integer(1)]] + Integer(1)/Integer(2)*B[[Integer(2), Integer(2)]]
>>> a ** Integer(2) == a  # not idempotent, but idempotent mod radical
False
>>> al = B.idempotent_lift(a)
>>> al ** Integer(2) == al
True
>>> (a - al) ** Integer(2)
0
```

is_commutative()

Return whether `self` is a commutative algebra.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S4 = SymmetricGroupAlgebra(QQ, 4)
sage: S4.is_commutative()
False
sage: S2 = SymmetricGroupAlgebra(QQ, 2)
sage: S2.is_commutative()
True
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S4 = SymmetricGroupAlgebra(QQ, Integer(4))
>>> S4.is_commutative()
False
>>> S2 = SymmetricGroupAlgebra(QQ, Integer(2))
>>> S2.is_commutative()
True
```

is_identity_decomposition_into_orthogonal_idempotents(*l*)

Return whether `l` is a decomposition of the identity into orthogonal idempotents.

INPUT:

- `l` – list or iterable of elements of `self`

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
```

(continues on next page)

(continued from previous page)

```
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: x,y,a,b = A.algebra_generators(); x,y,a,b
(x, y, a, b)
sage: A.is_identity_decomposition_into_orthogonal_idempotents([A.one()])
True
sage: A.is_identity_decomposition_into_orthogonal_idempotents([x, y])
True
sage: A.is_identity_decomposition_into_orthogonal_idempotents([x + a, y -␣
↪a])
True
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> x,y,a,b = A.algebra_generators(); x,y,a,b
(x, y, a, b)
>>> A.is_identity_decomposition_into_orthogonal_idempotents([A.one()])
True
>>> A.is_identity_decomposition_into_orthogonal_idempotents([x, y])
True
>>> A.is_identity_decomposition_into_orthogonal_idempotents([x + a, y -␣
↪a])
True
```

Here the idempotents do not sum up to 1:

```
sage: A.is_identity_decomposition_into_orthogonal_idempotents([x])       ␣
↪          # needs sage.graphs sage.modules
False
```

```
>>> from sage.all import *
>>> A.is_identity_decomposition_into_orthogonal_idempotents([x])         ␣
↪        # needs sage.graphs sage.modules
False
```

Here $1 + x$ and $-x$ are neither idempotent nor orthogonal:

```
sage: A.is_identity_decomposition_into_orthogonal_idempotents([1 + x, -
↪x])          # needs sage.graphs sage.modules
False
```

```
>>> from sage.all import *
>>> A.is_identity_decomposition_into_orthogonal_idempotents([Integer(1) +␣
↪x, -x])          # needs sage.graphs sage.modules
False
```

With the algebra of the 0-Hecke monoid:

```
sage: # needs sage.combinat sage.groups sage.modules sage.rings.number_
↪field
sage: from sage.monoids.hecke_monoid import HeckeMonoid
sage: A = HeckeMonoid(SymmetricGroup(4)).algebra(QQ)
sage: idempotents = A.orthogonal_idempotents_central_mod_radical()
sage: A.is_identity_decomposition_into_orthogonal_idempotents(idempotents)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules sage.rings.number_field
>>> from sage.monoids.hecke_monoid import HeckeMonoid
>>> A = HeckeMonoid(SymmetricGroup(Integer(4))).algebra(QQ)
>>> idempotents = A.orthogonal_idempotents_central_mod_radical()
>>> A.is_identity_decomposition_into_orthogonal_idempotents(idempotents)
True
```

Here are some more counterexamples:

1. Some orthogonal elements summing to 1 but not being idempotent:

```
sage: # needs sage.libs.pari sage.modules
sage: class PQAlgebra(CombinatorialFreeModule):
....:     def __init__(self, F, p):
....:         # Construct the quotient algebra F[x] / p,
....:         # where p is a univariate polynomial.
....:         R = parent(p); x = R.gen()
....:         I = R.ideal(p)
....:         self._xbar = R.quotient(I).gen()
....:         basis_keys = [self._xbar**i for i in range(p.degree())]
....:         CombinatorialFreeModule.__init__(self, F, basis_keys,
....:                 category=Algebras(F).FiniteDimensional().
↪WithBasis())
....:     def x(self):
....:         return self(self._xbar)
....:     def one(self):
....:         return self.basis()[self.base_ring().one()]
....:     def product_on_basis(self, w1, w2):
....:         return self.from_vector(vector(w1*w2))
sage: R.<x> = PolynomialRing(QQ)
sage: A = PQAlgebra(QQ, x**3 - x**2 + x + 1); y = A.x()
sage: a, b = y, 1 - y
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
False
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari sage.modules
>>> class PQAlgebra(CombinatorialFreeModule):
...     def __init__(self, F, p):
...         # Construct the quotient algebra F[x] / p,
...         # where p is a univariate polynomial.
...         R = parent(p); x = R.gen()
...         I = R.ideal(p)
...         self._xbar = R.quotient(I).gen()
```

(continues on next page)

(continued from previous page)

```
...         basis_keys = [self._xbar**i for i in range(p.degree())]
...         CombinatorialFreeModule.__init__(self, F, basis_keys,
...                 category=Algebras(F).FiniteDimensional().
↪WithBasis())
...     def x(self):
...         return self(self._xbar)
...     def one(self):
...         return self.basis()[self.base_ring().one()]
...     def product_on_basis(self, w1, w2):
...         return self.from_vector(vector(w1*w2))
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> A = PQAlgebra(QQ, x**Integer(3) - x**Integer(2) + x + Integer(1));␣
↪y = A.x()
>>> a, b = y, Integer(1) - y
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
False
```

For comparison:

```
sage: # needs sage.libs.pari sage.modules
sage: A = PQAlgebra(QQ, x**2 - x); y = A.x()
sage: a, b = y, 1-y
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
True
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a, A.
↪zero(), b))
True
sage: A = PQAlgebra(QQ, x**3 - x**2 + x - 1); y = A.x()
sage: a = (y**2 + 1) / 2
sage: b = 1 - a
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
True
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari sage.modules
>>> A = PQAlgebra(QQ, x**Integer(2) - x); y = A.x()
>>> a, b = y, Integer(1)-y
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
True
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a, A.
↪zero(), b))
True
>>> A = PQAlgebra(QQ, x**Integer(3) - x**Integer(2) + x - Integer(1));␣
↪y = A.x()
>>> a = (y**Integer(2) + Integer(1)) / Integer(2)
>>> b = Integer(1) - a
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a, b))
True
```

2. Some idempotents summing to 1 but not orthogonal:

```
sage: # needs sage.libs.pari sage.modules
sage: R.<x> = PolynomialRing(GF(2))
```

(continues on next page)

(continued from previous page)

```
sage: A = PQAlgebra(GF(2), x)
sage: a = A.one()
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a,))
True
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a, a,␣
↪a))
False
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari sage.modules
>>> R = PolynomialRing(GF(Integer(2)), names=('x',)); (x,) = R._first_
↪ngens(1)
>>> A = PQAlgebra(GF(Integer(2)), x)
>>> a = A.one()
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a,))
True
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a, a, a))
False
```

3. Some orthogonal idempotents not summing to the identity:

```
sage: # needs sage.libs.pari sage.modules
sage: A.is_identity_decomposition_into_orthogonal_idempotents((a,a))
False
sage: A.is_identity_decomposition_into_orthogonal_idempotents(())
False
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari sage.modules
>>> A.is_identity_decomposition_into_orthogonal_idempotents((a,a))
False
>>> A.is_identity_decomposition_into_orthogonal_idempotents(())
False
```

isotypic_projective_modules(*side='left'*)

Return the isotypic projective `side` `self`-modules.

Todo

The current implementation assumes that the algebra is split over its base field.

Let P_i be representatives of the indecomposable projective `side`-modules of this finite dimensional algebra A, and S_i be the associated simple modules.

The regular `side` representation of A can be decomposed as a direct sum $A = \bigoplus_i Q_i$ where each Q_i is an isotypic projective module; namely Q_i is the direct sum of $\dim S_i$ copies of the indecomposable projective module P_i. This decomposition is not unique.

The isotypic projective modules are constructed as $Q_i = e_i A$, where the $(e_i)_i$ is the decomposition of the identity into orthogonal idempotents obtained by lifting the central orthogonal idempotents of the semisimple quotient of A.

INPUT:

- `side` – `'left'` or `'right'` (default: `'left'`)

OUTPUT: list of subspaces of `self`

EXAMPLES:

```
sage: # needs sage.graphs sage.modules sage.rings.number_field
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: Q = A.isotypic_projective_modules(side='left'); Q
[Free module generated by {0} over Rational Field,
 Free module generated by {0, 1, 2} over Rational Field]
sage: [[x.lift() for x in Qi.basis()]
....:  for Qi in Q]
[[x],
 [y, a, b]]
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules sage.rings.number_field
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> Q = A.isotypic_projective_modules(side='left'); Q
[Free module generated by {0} over Rational Field,
 Free module generated by {0, 1, 2} over Rational Field]
>>> [[x.lift() for x in Qi.basis()]
...  for Qi in Q]
[[x],
 [y, a, b]]
```

We check that the sum of the dimensions of the isotypic projective modules is the dimension of `self`:

```
sage: sum([Qi.dimension() for Qi in Q]) == A.dimension()                #␣
↪needs sage.graphs sage.modules sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> sum([Qi.dimension() for Qi in Q]) == A.dimension()                  #␣
↪needs sage.graphs sage.modules sage.rings.number_field
True
```

See also

- *orthogonal_idempotents_central_mod_radical()*
- *peirce_decomposition()*

orthogonal_idempotents_central_mod_radical()

Return a family of orthogonal idempotents of `self` that project on the central orthogonal idempotents of the semisimple quotient.

Todo

The implementation assumes that the algebra is split over its base field.

OUTPUT:

- a list of orthogonal idempotents obtained by lifting the central orthogonal idempotents of the semisimple quotient.

ALGORITHM:

The orthogonal idempotents of A are obtained by lifting the central orthogonal idempotents of the semisimple quotient $\overline{A}$.

Namely, let $(\overline{f_i})$ be the central orthogonal idempotents of the semisimple quotient of A. We recursively construct orthogonal idempotents of A by the following procedure: assuming $(f_i)_{i<n}$ is a set of already constructed orthogonal idempotents, we construct f_k by idempotent lifting of $(1-f)g(1-f)$, where g is any lift of $\overline{e_k}$ and $f = \sum_{i<k} f_i$.

See [CR1962] for correctness and termination proofs.

See also

- *central_orthogonal_idempotents()*
- *idempotent_lift()*

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: A.orthogonal_idempotents_central_mod_radical()                    #␣
↪needs sage.rings.number_field
(x, y)
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> A.orthogonal_idempotents_central_mod_radical()                      #␣
↪needs sage.rings.number_field
(x, y)
```

```
sage: # needs sage.modules sage.rings.number_field
sage: Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid: the integers modulo 12
sage: A = Z12.algebra(QQ)
sage: idempotents = A.orthogonal_idempotents_central_mod_radical()
sage: sorted(idempotents, key=str)
```

(continues on next page)

(continued from previous page)

```
[-B[0] + 1/2*B[4] + 1/2*B[8],
 1/2*B[4] - 1/2*B[8],
 1/2*B[9] + 1/2*B[3] - B[0],
 1/2*B[9] - 1/2*B[3],
 1/4*B[1] + 1/4*B[11] - 1/4*B[5] - 1/4*B[7],
 1/4*B[1] - 1/2*B[9] + 1/4*B[5] - 1/4*B[7] + 1/2*B[3] - 1/4*B[11],
 1/4*B[1] - 1/2*B[9] - 1/2*B[3] + 1/4*B[11] + 1/4*B[5] + 1/4*B[7] + B[0] -
↪ 1/2*B[4] - 1/2*B[8],
 1/4*B[1] - 1/4*B[5] + 1/4*B[7] - 1/4*B[11] - 1/2*B[4] + 1/2*B[8],
 B[0]]
sage: sum(idempotents) == 1
True
sage: all(e*e == e for e in idempotents)
True
sage: all(e*f == 0 and f*e == 0
....:     for e in idempotents for f in idempotents if e != f)
True
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.rings.number_field
>>> Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid: the integers modulo 12
>>> A = Z12.algebra(QQ)
>>> idempotents = A.orthogonal_idempotents_central_mod_radical()
>>> sorted(idempotents, key=str)
[-B[0] + 1/2*B[4] + 1/2*B[8],
 1/2*B[4] - 1/2*B[8],
 1/2*B[9] + 1/2*B[3] - B[0],
 1/2*B[9] - 1/2*B[3],
 1/4*B[1] + 1/4*B[11] - 1/4*B[5] - 1/4*B[7],
 1/4*B[1] - 1/2*B[9] + 1/4*B[5] - 1/4*B[7] + 1/2*B[3] - 1/4*B[11],
 1/4*B[1] - 1/2*B[9] - 1/2*B[3] + 1/4*B[11] + 1/4*B[5] + 1/4*B[7] + B[0] -
↪ 1/2*B[4] - 1/2*B[8],
 1/4*B[1] - 1/4*B[5] + 1/4*B[7] - 1/4*B[11] - 1/2*B[4] + 1/2*B[8],
 B[0]]
>>> sum(idempotents) == Integer(1)
True
>>> all(e*e == e for e in idempotents)
True
>>> all(e*f == Integer(0) and f*e == Integer(0)
...     for e in idempotents for f in idempotents if e != f)
True
```

This is best tested with:

```
sage: A.is_identity_decomposition_into_orthogonal_
↪idempotents(idempotents)          # needs sage.graphs sage.modules sage.
↪rings.number_field
True
```

```
>>> from sage.all import *
>>> A.is_identity_decomposition_into_orthogonal_idempotents(idempotents) ␣
```

(continues on next page)

(continued from previous page)

```
↪        # needs sage.graphs sage.modules sage.rings.number_field
True
```

We construct orthogonal idempotents for the algebra of the 0-Hecke monoid:

```
sage: # needs sage.combinat sage.graphs sage.groups sage.modules
sage: from sage.monoids.hecke_monoid import HeckeMonoid
sage: A = HeckeMonoid(SymmetricGroup(4)).algebra(QQ)
sage: idempotents = A.orthogonal_idempotents_central_mod_radical()
sage: A.is_identity_decomposition_into_orthogonal_idempotents(idempotents)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.groups sage.modules
>>> from sage.monoids.hecke_monoid import HeckeMonoid
>>> A = HeckeMonoid(SymmetricGroup(Integer(4))).algebra(QQ)
>>> idempotents = A.orthogonal_idempotents_central_mod_radical()
>>> A.is_identity_decomposition_into_orthogonal_idempotents(idempotents)
True
```

peirce_decomposition(*idempotents=None*, *check=True*)

Return a Peirce decomposition of `self`.

Let $(e_i)_i$ be a collection of orthogonal idempotents of A with sum 1. The *Peirce decomposition* of A is the decomposition of A into the direct sum of the subspaces $e_i A e_j$.

With the default collection of orthogonal idempotents, one has

$$\dim e_i A e_j = C_{i,j} \dim S_i \dim S_j$$

where $(S_i)_i$ are the simple modules of A and $(C_{i,j})_{i,j}$ is the Cartan invariants matrix.

INPUT:

- `idempotents` – list of orthogonal idempotents $(e_i)_{i=0,\dots,n}$ of the algebra that sum to 1 (default: the idempotents returned by *orthogonal_idempotents_central_mod_radical()*)
- `check` – boolean (default: `True`); whether to check that the idempotents are indeed orthogonal and idempotent and sum to 1

OUTPUT:

A list of lists l such that `l[i][j]` is the subspace $e_i A e_j$.

See also

- *orthogonal_idempotents_central_mod_radical()*
- *cartan_invariants_matrix()*

EXAMPLES:

```
sage: # needs sage.graphs sage.groups sage.modules sage.rings.number_field
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
```

(continues on next page)

(continued from previous page)

```
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: A.orthogonal_idempotents_central_mod_radical()
(x, y)
sage: decomposition = A.peirce_decomposition(); decomposition
[[Free module generated by {0} over Rational Field,
  Free module generated by {0, 1} over Rational Field],
 [Free module generated by {} over Rational Field,
  Free module generated by {0} over Rational Field]]
sage: [ [[x.lift() for x in decomposition[i][j].basis()]
....:    for j in range(2)]
....:   for i in range(2)]
[[[x], [a, b]],
 [[], [y]]]
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups sage.modules sage.rings.number_field
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> A.orthogonal_idempotents_central_mod_radical()
(x, y)
>>> decomposition = A.peirce_decomposition(); decomposition
[[Free module generated by {0} over Rational Field,
  Free module generated by {0, 1} over Rational Field],
 [Free module generated by {} over Rational Field,
  Free module generated by {0} over Rational Field]]
>>> [ [[x.lift() for x in decomposition[i][j].basis()]
...    for j in range(Integer(2))]
...   for i in range(Integer(2))]
[[[x], [a, b]],
 [[], [y]]]
```

We recover that the group algebra of the symmetric group S_4 is a block matrix algebra:

```
sage: # needs sage.groups sage.modules sage.rings.number_field
sage: A = SymmetricGroup(4).algebra(QQ)
sage: decomposition = A.peirce_decomposition()  # long time
sage: [[decomposition[i][j].dimension()         # long time (4s)
....:   for j in range(len(decomposition))]
....:  for i in range(len(decomposition))]
[[9, 0, 0, 0, 0],
 [0, 9, 0, 0, 0],
 [0, 0, 4, 0, 0],
 [0, 0, 0, 1, 0],
 [0, 0, 0, 0, 1]]
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules sage.rings.number_field
>>> A = SymmetricGroup(Integer(4)).algebra(QQ)
>>> decomposition = A.peirce_decomposition()  # long time
>>> [[decomposition[i][j].dimension()         # long time (4s)
```

(continues on next page)

(continued from previous page)

```
...    for j in range(len(decomposition))]
...   for i in range(len(decomposition))]
[[9, 0, 0, 0, 0],
 [0, 9, 0, 0, 0],
 [0, 0, 4, 0, 0],
 [0, 0, 0, 1, 0],
 [0, 0, 0, 0, 1]]
```

The dimension of each block is d^2, where d is the dimension of the corresponding simple module of S_4. The latter are given by:

```
sage: [p.standard_tableaux().cardinality() for p in Partitions(4)]      #␣
↪needs sage.combinat
[1, 3, 2, 3, 1]
```

```
>>> from sage.all import *
>>> [p.standard_tableaux().cardinality() for p in Partitions(Integer(4))]␣
↪      # needs sage.combinat
[1, 3, 2, 3, 1]
```

peirce_summand(*ei*, *ej*)

Return the Peirce decomposition summand $e_i A e_j$.

INPUT:

- `self` – an algebra A
- `ei`, `ej` – two idempotents of A

OUTPUT: $e_i A e_j$, as a subspace of A

See also

- *peirce_decomposition()*
- *principal_ideal()*

EXAMPLES:

```
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example()
sage: idemp = A.orthogonal_idempotents_central_mod_radical()            #␣
↪needs sage.rings.number_field
sage: A.peirce_summand(idemp[0], idemp[1])                              #␣
↪needs sage.rings.number_field
Free module generated by {0, 1} over Rational Field
sage: A.peirce_summand(idemp[1], idemp[0])                              #␣
↪needs sage.rings.number_field
Free module generated by {} over Rational Field
```

```
>>> from sage.all import *
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example()
>>> idemp = A.orthogonal_idempotents_central_mod_radical()            #␣
↪needs sage.rings.number_field
>>> A.peirce_summand(idemp[Integer(0)], idemp[Integer(1)])              ␣
```

(continues on next page)

(continued from previous page)

```
↪              # needs sage.rings.number_field
Free module generated by {0, 1} over Rational Field
>>> A.peirce_summand(idemp[Integer(1)], idemp[Integer(0)])
↪              # needs sage.rings.number_field
Free module generated by {} over Rational Field
```

We recover the 2×2 block of $\mathbf{Q}[S_4]$ corresponding to the unique simple module of dimension 2 of the symmetric group S_4:

```
sage: # needs sage.combinat sage.groups sage.rings.number_field
sage: A4 = SymmetricGroup(4).algebra(QQ)
sage: e = A4.central_orthogonal_idempotents()[2]
sage: A4.peirce_summand(e, e)
Free module generated by {0, 1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.rings.number_field
>>> A4 = SymmetricGroup(Integer(4)).algebra(QQ)
>>> e = A4.central_orthogonal_idempotents()[Integer(2)]
>>> A4.peirce_summand(e, e)
Free module generated by {0, 1, 2, 3} over Rational Field
```

principal_ideal(*a*, *side='left'*, **args*, ***opts*)

Construct the `side` principal ideal generated by `a`.

INPUT:

- `a` – an element
- `side` – `left` (default) or `right` or `twosided`
- `coerce` – ignored, for compatibility with categories

EXAMPLES:

In order to highlight the difference between left and right principal ideals, our first example deals with a noncommutative algebra:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: x, y, a, b = A.basis()
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> x, y, a, b = A.basis()
```

In this algebra, multiplication on the right by x annihilates all basis elements but x:

```
sage: x*x, y*x, a*x, b*x                                                    #␣
↪needs sage.graphs sage.modules
```

(continues on next page)

(continued from previous page)

```
(x, 0, 0, 0)
```

```
>>> from sage.all import *
>>> x*x, y*x, a*x, b*x                                                    #␣
↪needs sage.graphs sage.modules
(x, 0, 0, 0)
```

so the left ideal generated by x is one-dimensional:

```
sage: Ax = A.principal_ideal(x, side='left'); Ax                          #␣
↪needs sage.graphs sage.modules
Free module generated by {0} over Rational Field
sage: [B.lift() for B in Ax.basis()]                                      #␣
↪needs sage.graphs sage.modules
[x]
```

```
>>> from sage.all import *
>>> Ax = A.principal_ideal(x, side='left'); Ax                            #␣
↪needs sage.graphs sage.modules
Free module generated by {0} over Rational Field
>>> [B.lift() for B in Ax.basis()]                                        #␣
↪needs sage.graphs sage.modules
[x]
```

Multiplication on the left by x annihilates only x and fixes the other basis elements:

```
sage: x*x, x*y, x*a, x*b                                                  #␣
↪needs sage.graphs sage.modules
(x, 0, a, b)
```

```
>>> from sage.all import *
>>> x*x, x*y, x*a, x*b                                                    #␣
↪needs sage.graphs sage.modules
(x, 0, a, b)
```

so the right ideal generated by x is 3-dimensional:

```
sage: xA = A.principal_ideal(x, side='right'); xA                         #␣
↪needs sage.graphs sage.modules
Free module generated by {0, 1, 2} over Rational Field
sage: [B.lift() for B in xA.basis()]                                      #␣
↪needs sage.graphs sage.modules
[x, a, b]
```

```
>>> from sage.all import *
>>> xA = A.principal_ideal(x, side='right'); xA                           #␣
↪needs sage.graphs sage.modules
Free module generated by {0, 1, 2} over Rational Field
>>> [B.lift() for B in xA.basis()]                                        #␣
↪needs sage.graphs sage.modules
[x, a, b]
```

For another example:

```
sage: A = MatrixSpace(QQ, 2)
sage: A.basis()
Finite family {(0, 0): [1 0]
[0 0], (0, 1): [0 1]
[0 0], (1, 0): [0 0]
[1 0], (1, 1): [0 0]
[0 1]}
sage: e = list(A.basis())
sage: [b.lift() for b in A.principal_ideal(e[0], side="left").basis()]
[
[1 0]  [0 0]
[0 0], [1 0]
]
sage: [b.lift() for b in A.principal_ideal(e[0], side="right").basis()]
[
[1 0]  [0 1]
[0 0], [0 0]
]
sage: [b.lift() for b in A.principal_ideal(e[0], side="twosided").basis()]
[
[1 0]  [0 1]  [0 0]  [0 0]
[0 0], [0 0], [1 0], [0 1]
]
```

```
>>> from sage.all import *
>>> A = MatrixSpace(QQ, Integer(2))
>>> A.basis()
Finite family {(0, 0): [1 0]
[0 0], (0, 1): [0 1]
[0 0], (1, 0): [0 0]
[1 0], (1, 1): [0 0]
[0 1]}
>>> e = list(A.basis())
>>> [b.lift() for b in A.principal_ideal(e[Integer(0)], side="left").
↪basis()]
[
[1 0]  [0 0]
[0 0], [1 0]
]
>>> [b.lift() for b in A.principal_ideal(e[Integer(0)], side="right").
↪basis()]
[
[1 0]  [0 1]
[0 0], [0 0]
]
>>> [b.lift() for b in A.principal_ideal(e[Integer(0)], side="twosided").
↪basis()]
[
[1 0]  [0 1]  [0 0]  [0 0]
[0 0], [0 0], [1 0], [0 1]
]
```

See also

- *peirce_summand()*

radical()

Return the Jacobson radical of `self`.

This uses *radical_basis()*, whose default implementation handles algebras over fields of characteristic zero or fields of characteristic p in which we can compute $x^{1/p}$.

See also

radical_basis(), *semisimple_quotient()*

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: radical = A.radical(); radical
Radical of An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> radical = A.radical(); radical
Radical of An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
```

The radical is an ideal of A, and thus a finite dimensional non unital associative algebra:

```
sage: # needs sage.graphs sage.modules
sage: from sage.categories.associative_algebras import AssociativeAlgebras
sage: radical in AssociativeAlgebras(QQ).WithBasis().FiniteDimensional()
True
sage: radical in Algebras(QQ)
False

sage: # needs sage.graphs sage.modules
sage: radical.dimension()
2
sage: radical.basis()
Finite family {0: B[0], 1: B[1]}
```

(continues on next page)

(continued from previous page)

```
sage: radical.ambient() is A
True
sage: [c.lift() for c in radical.basis()]
[a, b]
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> from sage.categories.associative_algebras import AssociativeAlgebras
>>> radical in AssociativeAlgebras(QQ).WithBasis().FiniteDimensional()
True
>>> radical in Algebras(QQ)
False

>>> # needs sage.graphs sage.modules
>>> radical.dimension()
2
>>> radical.basis()
Finite family {0: B[0], 1: B[1]}
>>> radical.ambient() is A
True
>>> [c.lift() for c in radical.basis()]
[a, b]
```

Todo

- Tell Sage that the radical is in fact an ideal;
- Pickling by construction, as `A.center()`;
- Lazy evaluation of `_repr_`.

radical_basis()

Return a basis of the Jacobson radical of this algebra.

Note

This implementation handles algebras over fields of characteristic zero (using Dixon's lemma) or fields of characteristic p in which we can compute $x^{1/p}$ [FR1985], [Eb1989].

OUTPUT: list of elements of `self`

See also

radical(), *Algebras.Semisimple*

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
```

(continues on next page)

(continued from previous page)

```
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: A.radical_basis()
(a, b)
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> A.radical_basis()
(a, b)
```

We construct the group algebra of the Klein Four-Group over the rationals:

```
sage: A = KleinFourGroup().algebra(QQ)                                  #␣
↪needs sage.groups sage.modules
```

```
>>> from sage.all import *
>>> A = KleinFourGroup().algebra(QQ)                                    #␣
↪needs sage.groups sage.modules
```

This algebra belongs to the category of finite dimensional algebras over the rationals:

```
sage: A in Algebras(QQ).FiniteDimensional().WithBasis()                 #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> A in Algebras(QQ).FiniteDimensional().WithBasis()                   #␣
↪needs sage.groups sage.modules
True
```

Since the field has characteristic 0, Maschke's Theorem tells us that the group algebra is semisimple. So its radical is the zero ideal:

```
sage: A in Algebras(QQ).Semisimple()                                    #␣
↪needs sage.groups sage.modules
True
sage: A.radical_basis()                                                 #␣
↪needs sage.groups sage.modules
()
```

```
>>> from sage.all import *
>>> A in Algebras(QQ).Semisimple()                                      #␣
↪needs sage.groups sage.modules
True
>>> A.radical_basis()                                                   #␣
↪needs sage.groups sage.modules
()
```

Let's work instead over a field of characteristic 2:

```
sage: A = KleinFourGroup().algebra(GF(2))                                   #␣
↪needs sage.groups sage.modules
sage: A in Algebras(GF(2)).Semisimple()                                     #␣
↪needs sage.groups sage.modules
False
sage: A.radical_basis()                                                     #␣
↪needs sage.groups sage.modules
(() + (1,2)(3,4), (3,4) + (1,2)(3,4), (1,2) + (1,2)(3,4))
```

```
>>> from sage.all import *
>>> A = KleinFourGroup().algebra(GF(Integer(2)))                            ␣
↪     # needs sage.groups sage.modules
>>> A in Algebras(GF(Integer(2))).Semisimple()                              ␣
↪     # needs sage.groups sage.modules
False
>>> A.radical_basis()                                                      #␣
↪needs sage.groups sage.modules
(() + (1,2)(3,4), (3,4) + (1,2)(3,4), (1,2) + (1,2)(3,4))
```

We now implement the algebra $A = K[x]/(x^p - 1)$, where K is a finite field of characteristic p, and check its radical; alas, we currently need to wrap A to make it a proper *ModulesWithBasis*:

```
sage: # needs sage.modules
sage: class AnAlgebra(CombinatorialFreeModule):
....:     def __init__(self, F):
....:         R.<x> = PolynomialRing(F)
....:         I = R.ideal(x**F.characteristic()-F.one())
....:         self._xbar = R.quotient(I).gen()
....:         basis_keys = [self._xbar**i for i in range(F.
↪characteristic())]
....:         CombinatorialFreeModule.__init__(self, F, basis_keys,
....:                 category=Algebras(F).FiniteDimensional().
↪WithBasis())
....:     def one(self):
....:         return self.basis()[self.base_ring().one()]
....:     def product_on_basis(self, w1, w2):
....:         return self.from_vector(vector(w1*w2))
sage: AnAlgebra(GF(3)).radical_basis()                                      #␣
↪needs sage.libs.pari
(B[1] + 2*B[xbar^2], B[xbar] + 2*B[xbar^2])
sage: AnAlgebra(GF(16,'a')).radical_basis()                                 #␣
↪needs sage.rings.finite_rings
(B[1] + B[xbar],)
sage: AnAlgebra(GF(49,'a')).radical_basis()                                 #␣
↪needs sage.rings.finite_rings
(B[1] + 6*B[xbar^6], B[xbar] + 6*B[xbar^6], B[xbar^2] + 6*B[xbar^6],
 B[xbar^3] + 6*B[xbar^6], B[xbar^4] + 6*B[xbar^6], B[xbar^5] + 6*B[xbar^
↪6])
```

```
>>> from sage.all import *
>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> class AnAlgebra(CombinatorialFreeModule):
...     def __init__(self, F):
...         R = PolynomialRing(F, names=('x',)); (x,) = R._first_ngens(1)
...         I = R.ideal(x**F.characteristic()-F.one())
...         self._xbar = R.quotient(I).gen()
...         basis_keys = [self._xbar**i for i in range(F.
↪characteristic())]
...         CombinatorialFreeModule.__init__(self, F, basis_keys,
...                 category=Algebras(F).FiniteDimensional().WithBasis())
...     def one(self):
...         return self.basis()[self.base_ring().one()]
...     def product_on_basis(self, w1, w2):
...         return self.from_vector(vector(w1*w2))
>>> AnAlgebra(GF(Integer(3))).radical_basis()                              ␣
↪     # needs sage.libs.pari
(B[1] + 2*B[xbar^2], B[xbar] + 2*B[xbar^2])
>>> AnAlgebra(GF(Integer(16),'a')).radical_basis()                         ␣
↪     # needs sage.rings.finite_rings
(B[1] + B[xbar],)
>>> AnAlgebra(GF(Integer(49),'a')).radical_basis()                         ␣
↪     # needs sage.rings.finite_rings
(B[1] + 6*B[xbar^6], B[xbar] + 6*B[xbar^6], B[xbar^2] + 6*B[xbar^6],
 B[xbar^3] + 6*B[xbar^6], B[xbar^4] + 6*B[xbar^6], B[xbar^5] + 6*B[xbar^
↪6])
```

We compute the radical basis in a subalgebra using the inherited product:

```
sage: # needs sage.modules
sage: scoeffs = {('a','e'): {'a':1}, ('b','e'): {'a':1, 'b':1},
....:            ('c','d'): {'a':1}, ('c','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: MS = MatrixSpace(QQ, 5)
sage: A = MS.subalgebra([bg.adjoint_matrix() for bg in L.lie_algebra_
↪generators()])
sage: A.radical_basis()
(B[1], B[2], B[3], B[4], B[5])
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> scoeffs = {('a','e'): {'a':Integer(1)}, ('b','e'): {'a':Integer(1), 'b
↪':Integer(1)},
...            ('c','d'): {'a':Integer(1)}, ('c','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> MS = MatrixSpace(QQ, Integer(5))
>>> A = MS.subalgebra([bg.adjoint_matrix() for bg in L.lie_algebra_
↪generators()])
>>> A.radical_basis()
(B[1], B[2], B[3], B[4], B[5])
```

semisimple_quotient()

Return the semisimple quotient of `self`.

This is the quotient of `self` by its radical.

See also

radical()

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: a,b,x,y = sorted(A.basis())
sage: S = A.semisimple_quotient(); S
Semisimple quotient of An example of a finite dimensional algebra with␣
↪basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: S in Algebras(QQ).Semisimple()
True
sage: S.basis()
Finite family {'x': B['x'], 'y': B['y']}
sage: xs,ys = sorted(S.basis())
sage: (xs + ys) * xs
B['x']
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> A = Algebras(QQ).FiniteDimensional().WithBasis().example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> a,b,x,y = sorted(A.basis())
>>> S = A.semisimple_quotient(); S
Semisimple quotient of An example of a finite dimensional algebra with␣
↪basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> S in Algebras(QQ).Semisimple()
True
>>> S.basis()
Finite family {'x': B['x'], 'y': B['y']}
>>> xs,ys = sorted(S.basis())
>>> (xs + ys) * xs
B['x']
```

Sanity check: the semisimple quotient of the n-th descent algebra of the symmetric group is of dimension the number of partitions of n:

```
sage: [ DescentAlgebra(QQ,n).B().semisimple_quotient().dimension()      #␣
↪needs sage.combinat sage.groups sage.modules
....:   for n in range(6) ]
```

(continues on next page)

(continued from previous page)

```
[1, 1, 2, 3, 5, 7]
sage: [Partitions(n).cardinality() for n in range(10)]                    #␣
↪needs sage.combinat
[1, 1, 2, 3, 5, 7, 11, 15, 22, 30]
```

```
>>> from sage.all import *
>>> [ DescentAlgebra(QQ,n).B().semisimple_quotient().dimension()       #␣
↪needs sage.combinat sage.groups sage.modules
...   for n in range(Integer(6)) ]
[1, 1, 2, 3, 5, 7]
>>> [Partitions(n).cardinality() for n in range(Integer(10))]              ␣
↪     # needs sage.combinat
[1, 1, 2, 3, 5, 7, 11, 15, 22, 30]
```

Todo

- Pickling by construction, as `A.semisimple_quotient()`?
- Lazy evaluation of `_repr_`

subalgebra(*gens*, *category=None*, *order=None*, **args*, ***opts*)

Return the subalgebra of `self` generated by `gens`.

INPUT:

- `gens` – iterable containing elements of `self`
- `category` – (optional) a subcategory of finite dimensional algebras with basis
- `order` – (optional) iterable defining an ordering of the basis elements of `self`

EXAMPLES:

```
sage: # needs sage.modules
sage: scoeffs = {('a','e'): {'a':1}, ('b','e'): {'a':1, 'b':1},
....:            ('c','d'): {'a':1}, ('c','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: MS = MatrixSpace(QQ, 5)
sage: A = MS.subalgebra([bg.adjoint_matrix() for bg in L.lie_algebra_
↪generators()])
sage: A.dimension()
7

sage: # needs sage.modules
sage: L.<x,y,z> = LieAlgebra(GF(3), {('x','z'): {'x':1, 'y':1}, ('y','z
↪'): {'y':1}})
sage: MS = MatrixSpace(L.base_ring(), L.dimension())
sage: gens = [b.adjoint_matrix() for b in L.basis()]
sage: A = MS.subalgebra(gens)
sage: A.dimension()
5

sage: WA = SignedPermutations(3).algebra(QQ)
sage: s1,s2,s3 = WA.algebra_generators()
sage: J2 = s1
```

(continues on next page)

(continued from previous page)

```
sage: J3 = s2 + s1*s2*s1
sage: J4 = (s3 + s2*s3*s2 + s3*s2*s3 + s1*s2*s3*s2*s1
....:       + s1*s3*s2*s3*s1 + s2*s1*s3*s2*s3*s1*s2)
sage: SA = WA.subalgebra([J2, J3, J4])
sage: SA.dimension()
14
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> scoeffs = {('a','e'): {'a':Integer(1)}, ('b','e'): {'a':Integer(1), 'b
↪':Integer(1)},
...             ('c','d'): {'a':Integer(1)}, ('c','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> MS = MatrixSpace(QQ, Integer(5))
>>> A = MS.subalgebra([bg.adjoint_matrix() for bg in L.lie_algebra_
↪generators()])
>>> A.dimension()
7

>>> # needs sage.modules
>>> L = LieAlgebra(GF(Integer(3)), {('x','z'): {'x':Integer(1), 'y
↪':Integer(1)}, ('y','z'): {'y':Integer(1)}}, names=('x', 'y', 'z',));␣
↪(x, y, z,) = L._first_ngens(3)
>>> MS = MatrixSpace(L.base_ring(), L.dimension())
>>> gens = [b.adjoint_matrix() for b in L.basis()]
>>> A = MS.subalgebra(gens)
>>> A.dimension()
5

>>> WA = SignedPermutations(Integer(3)).algebra(QQ)
>>> s1,s2,s3 = WA.algebra_generators()
>>> J2 = s1
>>> J3 = s2 + s1*s2*s1
>>> J4 = (s3 + s2*s3*s2 + s3*s2*s3 + s1*s2*s3*s2*s1
...       + s1*s3*s2*s3*s1 + s2*s1*s3*s2*s3*s1*s2)
>>> SA = WA.subalgebra([J2, J3, J4])
>>> SA.dimension()
14
```

class SubcategoryMethods

Bases: object

Cellular()

Return the full subcategory of the cellular objects of self.

See also

Wikipedia article Cellular_algebra

EXAMPLES:

```
sage: Algebras(QQ).FiniteDimensional().WithBasis().Cellular()
Category of finite dimensional cellular algebras with basis
 over Rational Field
```

```
>>> from sage.all import *
>>> Algebras(QQ).FiniteDimensional().WithBasis().Cellular()
Category of finite dimensional cellular algebras with basis
 over Rational Field
```

4.53 Finite dimensional bialgebras with basis

sage.categories.finite_dimensional_bialgebras_with_basis.**FiniteDimensionalBialgebrasWithBasis**(*base_ring*

The category of finite dimensional bialgebras with a distinguished basis.

EXAMPLES:

```
sage: C = FiniteDimensionalBialgebrasWithBasis(QQ); C
Category of finite dimensional bialgebras with basis over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of finite dimensional algebras with basis over Rational Field]
sage: C is Bialgebras(QQ).WithBasis().FiniteDimensional()
True
```

```
>>> from sage.all import *
>>> C = FiniteDimensionalBialgebrasWithBasis(QQ); C
Category of finite dimensional bialgebras with basis over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of finite dimensional algebras with basis over Rational Field]
>>> C is Bialgebras(QQ).WithBasis().FiniteDimensional()
True
```

4.54 Finite dimensional coalgebras with basis

sage.categories.finite_dimensional_coalgebras_with_basis.**FiniteDimensionalCoalgebrasWithBasis**(*base_ring*

The category of finite dimensional coalgebras with a distinguished basis.

EXAMPLES:

```
sage: C = FiniteDimensionalCoalgebrasWithBasis(QQ); C
Category of finite dimensional coalgebras with basis over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of coalgebras with basis over Rational Field,
 Category of finite dimensional vector spaces with basis over Rational Field]
sage: C is Coalgebras(QQ).WithBasis().FiniteDimensional()
True
```

```
>>> from sage.all import *
>>> C = FiniteDimensionalCoalgebrasWithBasis(QQ); C
```

(continues on next page)

(continued from previous page)

```
Category of finite dimensional coalgebras with basis over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of coalgebras with basis over Rational Field,
 Category of finite dimensional vector spaces with basis over Rational Field]
>>> C is Coalgebras(QQ).WithBasis().FiniteDimensional()
True
```

4.55 Finite Dimensional Graded Lie Algebras With Basis

AUTHORS:

- Eero Hakavuori (2018-08-16): initial version

class sage.categories.finite_dimensional_graded_lie_algebras_with_basis.**FiniteDimensionalGradedLieAlg**

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional graded Lie algebras with a basis.

A grading of a Lie algebra $\mathfrak{g}$ is a direct sum decomposition $\mathfrak{g} = \bigoplus_i V_i$ such that $[V_i, V_j] \subset V_{i+j}$.

EXAMPLES:

```
sage: C = LieAlgebras(ZZ).WithBasis().FiniteDimensional().Graded(); C
Category of finite dimensional graded Lie algebras with basis over Integer Ring
sage: C.super_categories()
[Category of graded Lie algebras with basis over Integer Ring,
 Category of finite dimensional filtered modules with basis over Integer Ring,
 Category of finite dimensional Lie algebras with basis over Integer Ring]

sage: C is LieAlgebras(ZZ).WithBasis().FiniteDimensional().Graded()
True
```

```
>>> from sage.all import *
>>> C = LieAlgebras(ZZ).WithBasis().FiniteDimensional().Graded(); C
Category of finite dimensional graded Lie algebras with basis over Integer Ring
>>> C.super_categories()
[Category of graded Lie algebras with basis over Integer Ring,
 Category of finite dimensional filtered modules with basis over Integer Ring,
 Category of finite dimensional Lie algebras with basis over Integer Ring]

>>> C is LieAlgebras(ZZ).WithBasis().FiniteDimensional().Graded()
True
```

class ParentMethods

Bases: object

homogeneous_component_as_submodule(*d*)

Return the d-th homogeneous component of self as a submodule.

EXAMPLES:

```
sage: C = LieAlgebras(QQ).WithBasis().Graded()
sage: C = C.FiniteDimensional().Stratified().Nilpotent()
sage: L = LieAlgebra(QQ, {('x','y'): {'z': 1}},                          #␣
↪needs sage.combinat sage.modules
....:                        nilpotent=True, category=C)
sage: L.homogeneous_component_as_submodule(2)                            #␣
↪needs sage.combinat sage.modules
Sparse vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[0 0 1]
```

```
>>> from sage.all import *
>>> C = LieAlgebras(QQ).WithBasis().Graded()
>>> C = C.FiniteDimensional().Stratified().Nilpotent()
>>> L = LieAlgebra(QQ, {('x','y'): {'z': Integer(1)}},                    ␣
↪     # needs sage.combinat sage.modules
...                        nilpotent=True, category=C)
>>> L.homogeneous_component_as_submodule(Integer(2))                      ␣
↪     # needs sage.combinat sage.modules
Sparse vector space of degree 3 and dimension 1 over Rational Field
Basis matrix:
[0 0 1]
```

class Stratified(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional stratified Lie algebras with a basis.

A stratified Lie algebra is a graded Lie algebra that is generated as a Lie algebra by its homogeneous component of degree 1. That is to say, for a graded Lie algebra $L = \bigoplus_{k=1}^{M} L_k$, we have $L_{k+1} = [L_1, L_k]$.

EXAMPLES:

```
sage: C = LieAlgebras(QQ).WithBasis().Graded().Stratified().
↪FiniteDimensional()
sage: C
Category of finite dimensional stratified Lie algebras with basis over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> C = LieAlgebras(QQ).WithBasis().Graded().Stratified().FiniteDimensional()
>>> C
Category of finite dimensional stratified Lie algebras with basis over␣
↪Rational Field
```

A finite-dimensional stratified Lie algebra is nilpotent:

```
sage: C is C.Nilpotent()
True
```

```
>>> from sage.all import *
>>> C is C.Nilpotent()
True
```

class ParentMethods

Bases: object

degree_on_basis(*m*)

Return the degree of the basis element indexed by m.

If the degrees of the basis elements are not defined, they will be computed. By assumption the stratification $L_1 \oplus \cdots \oplus L_s$ of self is such that each component L_k is spanned by some subset of the basis.

The degree of a basis element X is therefore the largest index k such that $X \in L_k \oplus \cdots \oplus L_s$. The space $L_k \oplus \cdots \oplus L_s$ is by assumption the k-th term of the lower central series.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: C = LieAlgebras(QQ).WithBasis().Graded()
sage: C = C.FiniteDimensional().Stratified().Nilpotent()
sage: sc = {('X','Y'): {'Z': 1}}
sage: L.<X,Y,Z> = LieAlgebra(QQ, sc, nilpotent=True, category=C)
sage: L.degree_on_basis(X.leading_support())
1
sage: X.degree()
1
sage: Y.degree()
1
sage: L[X, Y]
Z
sage: Z.degree()
2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> C = LieAlgebras(QQ).WithBasis().Graded()
>>> C = C.FiniteDimensional().Stratified().Nilpotent()
>>> sc = {('X','Y'): {'Z': Integer(1)}}
>>> L = LieAlgebra(QQ, sc, nilpotent=True, category=C, names=('X', 'Y',
↪ 'Z',)); (X, Y, Z,) = L._first_ngens(3)
>>> L.degree_on_basis(X.leading_support())
1
>>> X.degree()
1
>>> Y.degree()
1
>>> L[X, Y]
Z
>>> Z.degree()
2
```

4.56 Finite dimensional Hopf algebras with basis

class sage.categories.finite_dimensional_hopf_algebras_with_basis.**FiniteDimensionalHopfAlgebrasWithBa**

Bases: *CategoryWithAxiom_over_base_ring*

The category of finite dimensional Hopf algebras with a distinguished basis.

EXAMPLES:

```
sage: FiniteDimensionalHopfAlgebrasWithBasis(QQ)
Category of finite dimensional Hopf algebras with basis over Rational Field
sage: FiniteDimensionalHopfAlgebrasWithBasis(QQ).super_categories()
[Category of Hopf algebras with basis over Rational Field,
 Category of finite dimensional algebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> FiniteDimensionalHopfAlgebrasWithBasis(QQ)
Category of finite dimensional Hopf algebras with basis over Rational Field
>>> FiniteDimensionalHopfAlgebrasWithBasis(QQ).super_categories()
[Category of Hopf algebras with basis over Rational Field,
 Category of finite dimensional algebras with basis over Rational Field]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

4.57 Finite Dimensional Lie Algebras With Basis

AUTHORS:

- Travis Scrimshaw (07-15-2013): Initial implementation

class sage.categories.finite_dimensional_lie_algebras_with_basis.**FiniteDimensionalLieAlgebrasWithBasi**

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional Lie algebras with a basis.

Todo

Many of these tests should use non-abelian Lie algebras and need to be added after Issue #16820.

class ElementMethods

Bases: object

adjoint_matrix(*sparse=False*)

Return the matrix of the adjoint action of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.an_element().adjoint_matrix()
```

(continues on next page)

(continued from previous page)

```
[0 0 0]
[0 0 0]
[0 0 0]
sage: L.an_element().adjoint_matrix(sparse=True).is_sparse()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.an_element().adjoint_matrix()
[0 0 0]
[0 0 0]
[0 0 0]
>>> L.an_element().adjoint_matrix(sparse=True).is_sparse()
True
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: x.adjoint_matrix()
[0 1]
[0 0]
sage: y.adjoint_matrix()
[-1  0]
[ 0  0]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> x.adjoint_matrix()
[0 1]
[0 0]
>>> y.adjoint_matrix()
[-1  0]
[ 0  0]
```

We verify that this forms a representation:

```
sage: # needs sage.combinat sage.modules
sage: sl3 = lie_algebras.sl(QQ, 3)
sage: e1, e2 = sl3.e(1), sl3.e(2)
sage: e12 = e1.bracket(e2)
sage: E1, E2 = e1.adjoint_matrix(), e2.adjoint_matrix()
sage: E1 * E2 - E2 * E1 == e12.adjoint_matrix()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> sl3 = lie_algebras.sl(QQ, Integer(3))
>>> e1, e2 = sl3.e(Integer(1)), sl3.e(Integer(2))
>>> e12 = e1.bracket(e2)
```

(continues on next page)

(continued from previous page)

```
>>> E1, E2 = e1.adjoint_matrix(), e2.adjoint_matrix()
>>> E1 * E2 - E2 * E1 == e12.adjoint_matrix()
True
```

to_vector(*sparse=False*, *order=None*)

Return the vector in `g.module()` corresponding to the element `self` of `g` (where `g` is the parent of `self`).

Implement this if you implement `g.module()`. See *module()* for how this is to be done.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.an_element().to_vector()
(0, 0, 0)
sage: L.an_element().to_vector(sparse=True)
(0, 0, 0)

sage: # needs sage.combinat sage.groupssage.modules
sage: D = DescentAlgebra(QQ, 4).D()
sage: L = LieAlgebra(associative=D)
sage: L.an_element().to_vector()
(1, 1, 1, 1, 1, 1, 1, 1)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.an_element().to_vector()
(0, 0, 0)
>>> L.an_element().to_vector(sparse=True)
(0, 0, 0)

>>> # needs sage.combinat sage.groupssage.modules
>>> D = DescentAlgebra(QQ, Integer(4)).D()
>>> L = LieAlgebra(associative=D)
>>> L.an_element().to_vector()
(1, 1, 1, 1, 1, 1, 1, 1)
```

Nilpotent

alias of *FiniteDimensionalNilpotentLieAlgebrasWithBasis*

class ParentMethods

Bases: `object`

as_finite_dimensional_algebra()

Return `self` as a `FiniteDimensionalAlgebra`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.cross_product(QQ)
sage: x, y, z = L.basis()
sage: F = L.as_finite_dimensional_algebra()
```

(continues on next page)

(continued from previous page)

```
sage: X, Y, Z = F.basis()
sage: x.bracket(y)
Z
sage: X * Y
Z
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.cross_product(QQ)
>>> x, y, z = L.basis()
>>> F = L.as_finite_dimensional_algebra()
>>> X, Y, Z = F.basis()
>>> x.bracket(y)
Z
>>> X * Y
Z
```

casimir_element(*order=2*, *UEA=None*, *force_generic=False*, *basis=False*)

Return a Casimir element of order `order` in the universal enveloping algebra of `self`.

A *Casimir element* of order k is a distinguished basis element for the center of $U(\mathfrak{g})$ of homogeneous degree k (that is, it is an element of $U_k \setminus U_{k-1}$, where $\{U_i\}_{i=0}^{\infty}$ is the natural filtration of $U(\mathfrak{g})$). When $\mathfrak{g}$ is a simple Lie algebra, then this spans $Z(U(\mathfrak{g}))_k$.

INPUT:

- `order` – (default: `2`) the order of the Casimir element
- `UEA` – (optional) the universal enveloping algebra implementation to return the result in
- `force_generic` – boolean (default: `False`); if `True` for the quadratic order, then this uses the default algorithm (otherwise this is ignored)
- `basis` – boolean (default: `False`); if `True`, this returns a basis of all Casimir elements of order `order` as a list

ALGORITHM:

For the quadratic order (i.e., `order=2`), then this uses K^{ij}, the inverse of the Killing form matrix, to compute $C_{(2)} = \sum_{i,j} K^{ij} X_i \cdots X_j$, where $\{X_1, \ldots, X_n\}$ is a basis for $\mathfrak{g}$. Otherwise this solves the system of equations

$$f_{aj}^{b}\kappa^{jc\cdots d} + f_{aj}^{c}\kappa^{cj\cdots d} \cdots + f_{aj}^{d}\kappa^{bc\cdots j}$$

for the symmetric tensor $\kappa^{i_1\cdots i_k}$, where k is the `order`. This system comes from $[X_i, C_{(k)}] = 0$ with

$$C_{(k)} = \sum_{i_1,\ldots,i_k}^{n} \kappa^{i_1\cdots i_k} X_{i_1} \cdots X_{i_k}.$$

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['A', 1])
sage: C = L.casimir_element(); C
1/8*b1^2 + 1/2*b0*b2 - 1/4*b1
sage: U = L.universal_enveloping_algebra()
sage: all(g * C == C * g for g in U.gens())
True
```

(continues on next page)

(continued from previous page)

```
sage: U = L.pbw_basis()
sage: C = L.casimir_element(UEA=U); C
1/2*PBW[alpha[1]]*PBW[-alpha[1]] + 1/8*PBW[alphacheck[1]]^2
 - 1/4*PBW[alphacheck[1]]
sage: all(g * C == C * g for g in U.algebra_generators())
True

sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['B', 2])
sage: U = L.pbw_basis()
sage: C = L.casimir_element(UEA=U)
sage: all(g * C == C * g for g in U.algebra_generators())
True

sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['C', 3])
sage: U = L.pbw_basis()
sage: C = L.casimir_element(UEA=U)
sage: all(g * C == C * g for g in U.algebra_generators())
True

sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['A', 1])
sage: C4 = L.casimir_element(order=4, UEA=L.pbw_basis()); C4
4*PBW[alpha[1]]^2*PBW[-alpha[1]]^2
 + 2*PBW[alpha[1]]*PBW[alphacheck[1]]^2*PBW[-alpha[1]]
 + 1/4*PBW[alphacheck[1]]^4 - PBW[alphacheck[1]]^3
 - 4*PBW[alpha[1]]*PBW[-alpha[1]] + 2*PBW[alphacheck[1]]
sage: all(g * C4 == C4 * g for g in L.pbw_basis().algebra_generators())
True

sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.Heisenberg(QQ, 2)
sage: L.casimir_element()
0

sage: # needs sage.combinat sage.modules
sage: g = LieAlgebra(QQ, cartan_type=['D',2])
sage: U = g.pbw_basis()
sage: U.casimir_element(2, basis=True)
[2*PBW[alpha[2]]*PBW[-alpha[2]] + 1/2*PBW[alphacheck[2]]^2 -␣
↪PBW[alphacheck[2]],
 2*PBW[alpha[1]]*PBW[-alpha[1]] + 1/2*PBW[alphacheck[1]]^2 -␣
↪PBW[alphacheck[1]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['A', Integer(1)])
>>> C = L.casimir_element(); C
1/8*b1^2 + 1/2*b0*b2 - 1/4*b1
>>> U = L.universal_enveloping_algebra()
>>> all(g * C == C * g for g in U.gens())
```

(continues on next page)

(continued from previous page)

```
True
>>> U = L.pbw_basis()
>>> C = L.casimir_element(UEA=U); C
1/2*PBW[alpha[1]]*PBW[-alpha[1]] + 1/8*PBW[alphacheck[1]]^2
 - 1/4*PBW[alphacheck[1]]
>>> all(g * C == C * g for g in U.algebra_generators())
True

>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['B', Integer(2)])
>>> U = L.pbw_basis()
>>> C = L.casimir_element(UEA=U)
>>> all(g * C == C * g for g in U.algebra_generators())
True

>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['C', Integer(3)])
>>> U = L.pbw_basis()
>>> C = L.casimir_element(UEA=U)
>>> all(g * C == C * g for g in U.algebra_generators())
True

>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['A', Integer(1)])
>>> C4 = L.casimir_element(order=Integer(4), UEA=L.pbw_basis()); C4
4*PBW[alpha[1]]^2*PBW[-alpha[1]]^2
 + 2*PBW[alpha[1]]*PBW[alphacheck[1]]^2*PBW[-alpha[1]]
 + 1/4*PBW[alphacheck[1]]^4 - PBW[alphacheck[1]]^3
 - 4*PBW[alpha[1]]*PBW[-alpha[1]] + 2*PBW[alphacheck[1]]
>>> all(g * C4 == C4 * g for g in L.pbw_basis().algebra_generators())
True

>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.Heisenberg(QQ, Integer(2))
>>> L.casimir_element()
0

>>> # needs sage.combinat sage.modules
>>> g = LieAlgebra(QQ, cartan_type=['D',Integer(2)])
>>> U = g.pbw_basis()
>>> U.casimir_element(Integer(2), basis=True)
[2*PBW[alpha[2]]*PBW[-alpha[2]] + 1/2*PBW[alphacheck[2]]^2 -␣
↪PBW[alphacheck[2]],
 2*PBW[alpha[1]]*PBW[-alpha[1]] + 1/2*PBW[alphacheck[1]]^2 -␣
↪PBW[alphacheck[1]]]
```

Todo

Use the symmetry of the tensor to reduce the number of equations and/or variables to solve.

center()

Return the center of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: Z = L.center(); Z
An example of a finite dimensional Lie algebra with basis: the
 3-dimensional abelian Lie algebra over Rational Field
sage: Z.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> Z = L.center(); Z
An example of a finite dimensional Lie algebra with basis: the
 3-dimensional abelian Lie algebra over Rational Field
>>> Z.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

centralizer(S)

Return the centralizer of `S` in `self`.

INPUT:

- `S` – a subalgebra of `self` or a list of elements that represent generators for a subalgebra

See also

centralizer_basis()

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: S = L.centralizer([a + b, 2*a + c]); S
An example of a finite dimensional Lie algebra with basis:
 the 3-dimensional abelian Lie algebra over Rational Field
sage: S.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> S = L.centralizer([a + b, Integer(2)*a + c]); S
An example of a finite dimensional Lie algebra with basis:
 the 3-dimensional abelian Lie algebra over Rational Field
>>> S.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

centralizer_basis(*S*)

Return a basis of the centralizer of `S` in `self`.

INPUT:

- `S` – a subalgebra of `self` or a list of elements that represent generators for a subalgebra

See also

centralizer()

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: L.centralizer_basis([a + b, 2*a + c])
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]

sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(QQ, 2)
sage: H.centralizer_basis(H)
[z]

sage: # needs sage.combinat sage.groups sage.modules
sage: D = DescentAlgebra(QQ, 4).D()
sage: L = LieAlgebra(associative=D)
sage: L.centralizer_basis(L)
[D{},
 D{1} + D{1, 2} + D{2, 3} + D{3},
 D{1, 2, 3} + D{1, 3} + D{2}]
sage: D.center_basis()
(D{},
 D{1} + D{1, 2} + D{2, 3} + D{3},
 D{1, 2, 3} + D{1, 3} + D{2})

sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.centralizer_basis([a, c])
[a, b, c]
```

(continues on next page)

(continued from previous page)

```
sage: L.centralizer_basis([a, e])
[c]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> L.centralizer_basis([a + b, Integer(2)*a + c])
[(1, 0, 0), (0, 1, 0), (0, 0, 1)]

>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(QQ, Integer(2))
>>> H.centralizer_basis(H)
[z]

>>> # needs sage.combinat sage.groups sage.modules
>>> D = DescentAlgebra(QQ, Integer(4)).D()
>>> L = LieAlgebra(associative=D)
>>> L.centralizer_basis(L)
[D{},
 D{1} + D{1, 2} + D{2, 3} + D{3},
 D{1, 2, 3} + D{1, 3} + D{2}]
>>> D.center_basis()
(D{},
 D{1} + D{1, 2} + D{2, 3} + D{3},
 D{1, 2, 3} + D{1, 3} + D{2})

>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.centralizer_basis([a, c])
[a, b, c]
>>> L.centralizer_basis([a, e])
[c]
```

chevalley_eilenberg_complex(*M=None*, *dual=False*, *sparse=True*, *ncpus=None*)

Return the Chevalley-Eilenberg complex of `self`.

Let $\mathfrak{g}$ be a Lie algebra and M be a right $\mathfrak{g}$-module. The *Chevalley-Eilenberg complex* is the chain complex on

$$C_\bullet(\mathfrak{g}, M) = M \otimes \bigwedge\nolimits^\bullet \mathfrak{g},$$

where the differential is given by

$$d(m \otimes g_1 \wedge \cdots \wedge g_p) = \sum_{i=1}^{p} (-1)^{i+1} (m g_i) \otimes g_1 \wedge \cdots \wedge \hat{g}_i \wedge \cdots \wedge g_p + \sum_{1 \le i < j \le p} (-1)^{i+j} m \otimes [g_i, g_j] \wedge g_1 \wedge \cdots \wedge \hat{g}_i \wedge$$

INPUT:

- `M` – (default: the trivial 1-dimensional module) one of the following:
  - a module M with an action of `self`

  - a dictionary whose keys are basis elements and values are matrices representing a Lie algebra homomorphism defining the representation
- `dual` – boolean (default: `False`); if `True`, causes the dual of the complex to be computed
- `sparse` – boolean (default: `True`); whether to use sparse or dense matrices
- `ncpus` – (optional) how many cpus to use

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.sl(ZZ, 2)
sage: C = L.chevalley_eilenberg_complex(); C
Chain complex with at most 4 nonzero terms over Integer Ring
sage: ascii_art(C)
                          [-2  0  0]       [0]
                          [ 0  1  0]       [0]
            [0 0 0]       [ 0  0 -2]       [0]
 0 <-- C_0 <-------- C_1 <----------- C_2 <---- C_3 <-- 0

sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['C',2])
sage: C = L.chevalley_eilenberg_complex()  # long time
sage: [C.free_module_rank(i) for i in range(11)]  # long time
[1, 10, 45, 120, 210, 252, 210, 120, 45, 10, 1]

sage: # needs sage.combinat sage.modules
sage: g = lie_algebras.sl(QQ, 2)
sage: E, F, H = g.basis()
sage: n = g.subalgebra([F, H])
sage: ascii_art(n.chevalley_eilenberg_complex())
                         [ 0]
            [0 0]        [-2]
 0 <-- C_0 <------ C_1 <----- C_2 <-- 0

sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'y':1}})
sage: f = ({x: Matrix([[1,0],[0,0]]), y: Matrix([[0,1],[0,0]])})
sage: C = L.chevalley_eilenberg_complex(f); C
Chain complex with at most 3 nonzero terms over Rational Field
sage: ascii_art(C)
                            [ 0 -1]
                            [ 2  0]
            [1 0 0 1]       [ 0  0]
            [0 0 0 0]       [ 0  1]
 0 <-- C_0 <---------- C_1 <-------- C_2 <-- 0

sage: ascii_art(L.chevalley_eilenberg_complex(f, sparse=False))
                            [ 0 -1]
                            [ 2  0]
            [1 0 0 1]       [ 0  0]
            [0 0 0 0]       [ 0  1]
 0 <-- C_0 <---------- C_1 <-------- C_2 <-- 0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.sl(ZZ, Integer(2))
```

(continues on next page)

(continued from previous page)

```
>>> C = L.chevalley_eilenberg_complex(); C
Chain complex with at most 4 nonzero terms over Integer Ring
>>> ascii_art(C)
                          [-2  0  0]       [0]
                          [ 0  1  0]       [0]
            [0 0 0]       [ 0  0 -2]       [0]
 0 <-- C_0 <-------- C_1 <---------- C_2 <---- C_3 <-- 0

>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['C',Integer(2)])
>>> C = L.chevalley_eilenberg_complex()  # long time
>>> [C.free_module_rank(i) for i in range(Integer(11))]  # long time
[1, 10, 45, 120, 210, 252, 210, 120, 45, 10, 1]

>>> # needs sage.combinat sage.modules
>>> g = lie_algebras.sl(QQ, Integer(2))
>>> E, F, H = g.basis()
>>> n = g.subalgebra([F, H])
>>> ascii_art(n.chevalley_eilenberg_complex())
                        [ 0]
            [0 0]       [-2]
 0 <-- C_0 <------ C_1 <----- C_2 <-- 0

>>> L = LieAlgebra(QQ, {('x','y'): {'y':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> f = ({x: Matrix([[Integer(1),Integer(0)],[Integer(0),Integer(0)]]),␣
↪y: Matrix([[Integer(0),Integer(1)],[Integer(0),Integer(0)]])})
>>> C = L.chevalley_eilenberg_complex(f); C
Chain complex with at most 3 nonzero terms over Rational Field
>>> ascii_art(C)
                            [ 0 -1]
                            [ 2  0]
            [1 0 0 1]       [ 0  0]
            [0 0 0 0]       [ 0  1]
 0 <-- C_0 <---------- C_1 <-------- C_2 <-- 0

>>> ascii_art(L.chevalley_eilenberg_complex(f, sparse=False))
                            [ 0 -1]
                            [ 2  0]
            [1 0 0 1]       [ 0  0]
            [0 0 0 0]       [ 0  1]
 0 <-- C_0 <---------- C_1 <-------- C_2 <-- 0
```

REFERENCES:

- Wikipedia article Lie_algebra_cohomology#Chevalley-Eilenberg_complex
- [Wei1994] Chapter 7

cohomology(*deg=None*, *M=None*, *sparse=True*, *ncpus=None*)

Return the Lie algebra cohomology of `self`.

The Lie algebra cohomology is the cohomology of the Chevalley-Eilenberg cochain complex (which is the dual of the Chevalley-Eilenberg chain complex).

Let $\mathfrak{g}$ be a Lie algebra and M a left $\mathfrak{g}$-module. It is known that $H^0(\mathfrak{g}; M)$ is the subspace of $\mathfrak{g}$-invariants

of M:

$$H^0(\mathfrak{g}; M) = M^{\mathfrak{g}} = \{m \in M \mid gm = 0 \text{ for all } g \in \mathfrak{g}\}.$$

Additionally, $H^1(\mathfrak{g}; M)$ is the space of derivations $\mathfrak{g} \to M$ modulo the space of inner derivations, and $H^2(\mathfrak{g}; M)$ is the space of equivalence classes of Lie algebra extensions of $\mathfrak{g}$ by M.

INPUT:

- `deg` – the degree of the homology (optional)
- `M` – (default: the trivial module) a right module of `self`
- `sparse` – boolean (default: `True`); whether to use sparse matrices for the Chevalley-Eilenberg chain complex
- `ncpus` – (optional) how many cpus to use when computing the Chevalley-Eilenberg chain complex

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.so(QQ, 4)
sage: L.cohomology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 0 over Rational Field,
 2: Vector space of dimension 0 over Rational Field,
 3: Vector space of dimension 2 over Rational Field,
 4: Vector space of dimension 0 over Rational Field,
 5: Vector space of dimension 0 over Rational Field,
 6: Vector space of dimension 1 over Rational Field}

sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.Heisenberg(QQ, 2)
sage: L.cohomology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 4 over Rational Field,
 2: Vector space of dimension 5 over Rational Field,
 3: Vector space of dimension 5 over Rational Field,
 4: Vector space of dimension 4 over Rational Field,
 5: Vector space of dimension 1 over Rational Field}

sage: # needs sage.combinat sage.modules
sage: d = {('x', 'y'): {'y': 2}}
sage: L.<x,y> = LieAlgebra(ZZ, d)
sage: L.cohomology()
{0: Z, 1: Z, 2: C2}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.so(QQ, Integer(4))
>>> L.cohomology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 0 over Rational Field,
 2: Vector space of dimension 0 over Rational Field,
 3: Vector space of dimension 2 over Rational Field,
 4: Vector space of dimension 0 over Rational Field,
 5: Vector space of dimension 0 over Rational Field,
 6: Vector space of dimension 1 over Rational Field}
```

(continues on next page)

(continued from previous page)

```
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.Heisenberg(QQ, Integer(2))
>>> L.cohomology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 4 over Rational Field,
 2: Vector space of dimension 5 over Rational Field,
 3: Vector space of dimension 5 over Rational Field,
 4: Vector space of dimension 4 over Rational Field,
 5: Vector space of dimension 1 over Rational Field}

>>> # needs sage.combinat sage.modules
>>> d = {('x', 'y'): {'y': Integer(2)}}
>>> L = LieAlgebra(ZZ, d, names=('x', 'y',)); (x, y,) = L._first_ngens(2)
>>> L.cohomology()
{0: Z, 1: Z, 2: C2}
```

See also

chevalley_eilenberg_complex()

REFERENCES:

- Wikipedia article Lie_algebra_cohomology

derivations_basis()

Return a basis for the Lie algebra of derivations of `self` as matrices.

A derivation D of an algebra is an endomorphism of A such that

$$D([a, b]) = [D(a), b] + [a, D(b)]$$

for all $a, b \in A$. The set of all derivations form a Lie algebra.

EXAMPLES:

We construct the derivations of the Heisenberg Lie algebra:

```
sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(QQ, 1)
sage: H.derivations_basis()
(
[1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]  [0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]  [1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]
[0 0 1], [0 0 0], [0 0 0], [0 0 1], [1 0 0], [0 1 0]
)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(QQ, Integer(1))
>>> H.derivations_basis()
(
[1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]  [0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]  [1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 1], [0 0 0], [0 0 0], [0 0 1], [1 0 0], [0 1 0]
)
```

We construct the derivations of $\mathfrak{sl}_2$:

```
sage: # needs sage.combinat sage.modules
sage: sl2 = lie_algebras.sl(QQ, 2)
sage: sl2.derivations_basis()
(
[ 1  0  0]  [  0   1   0]  [ 0  0  0]
[ 0  0  0]  [  0   0 -1/2]  [ 1  0  0]
[ 0  0 -1], [  0   0   0], [ 0 -2  0]
)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> sl2 = lie_algebras.sl(QQ, Integer(2))
>>> sl2.derivations_basis()
(
[ 1  0  0]  [  0   1   0]  [ 0  0  0]
[ 0  0  0]  [  0   0 -1/2]  [ 1  0  0]
[ 0  0 -1], [  0   0   0], [ 0 -2  0]
)
```

We verify these are derivations:

```
sage: # needs sage.combinat sage.modules
sage: D = [sl2.module_morphism(matrix=M, codomain=sl2)
....:      for M in sl2.derivations_basis()]
sage: all(d(a.bracket(b)) == d(a).bracket(b) + a.bracket(d(b))
....:     for a in sl2.basis() for b in sl2.basis() for d in D)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> D = [sl2.module_morphism(matrix=M, codomain=sl2)
...      for M in sl2.derivations_basis()]
>>> all(d(a.bracket(b)) == d(a).bracket(b) + a.bracket(d(b))
...     for a in sl2.basis() for b in sl2.basis() for d in D)
True
```

REFERENCES:

Wikipedia article Derivation_(differential_algebra)

derived_series()

Return the derived series $(\mathfrak{g}^{(i)})_i$ of `self` where the rightmost $\mathfrak{g}^{(k)} = \mathfrak{g}^{(k+1)} = \cdots$.

We define the derived series of a Lie algebra $\mathfrak{g}$ recursively by $\mathfrak{g}^{(0)} := \mathfrak{g}$ and

$$\mathfrak{g}^{(k+1)} = [\mathfrak{g}^{(k)}, \mathfrak{g}^{(k)}]$$

and recall that $\mathfrak{g}^{(k)} \supseteq \mathfrak{g}^{(k+1)}$. Alternatively we can express this as

$$\mathfrak{g} \supseteq [\mathfrak{g}, \mathfrak{g}] \supseteq \left[[\mathfrak{g}, \mathfrak{g}], [\mathfrak{g}, \mathfrak{g}]\right] \supseteq \left[\left[[\mathfrak{g}, \mathfrak{g}], [\mathfrak{g}, \mathfrak{g}]\right], \left[[\mathfrak{g}, \mathfrak{g}], [\mathfrak{g}, \mathfrak{g}]\right]\right] \supseteq \cdots.$$

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.derived_series()
(An example of a finite dimensional Lie algebra with basis:
    the 3-dimensional abelian Lie algebra over Rational Field,
 An example of a finite dimensional Lie algebra with basis:
    the 0-dimensional abelian Lie algebra over Rational Field
    with basis matrix: [])
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.derived_series()
(An example of a finite dimensional Lie algebra with basis:
    the 3-dimensional abelian Lie algebra over Rational Field,
 An example of a finite dimensional Lie algebra with basis:
    the 0-dimensional abelian Lie algebra over Rational Field
    with basis matrix: [])
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.derived_series()
(Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal (x) of Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal () of Lie algebra on 2 generators (x, y) over Rational Field)

sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.derived_series()
(Lie algebra on 5 generators (a, b, c, d, e) over Rational Field,
 Ideal (a, b, c) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field,
 Ideal () of Lie algebra on 5 generators (a, b, c, d, e) over Rational␣
↪Field)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.derived_series()
(Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal (x) of Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal () of Lie algebra on 2 generators (x, y) over Rational Field)

>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
```

(continues on next page)

(continued from previous page)

```
>>> L.derived_series()
(Lie algebra on 5 generators (a, b, c, d, e) over Rational Field,
 Ideal (a, b, c) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field,
 Ideal () of Lie algebra on 5 generators (a, b, c, d, e) over Rational␣
↪Field)
```

derived_subalgebra()

Return the derived subalgebra of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.derived_subalgebra()
An example of a finite dimensional Lie algebra with basis:
 the 0-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
[]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.derived_subalgebra()
An example of a finite dimensional Lie algebra with basis:
 the 0-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
[]
```

If `self` is semisimple, then the derived subalgebra is `self`:

```
sage: # needs sage.combinat sage.modules
sage: sl3 = LieAlgebra(QQ, cartan_type=['A', 2])
sage: sl3.derived_subalgebra()
Lie algebra of ['A', 2] in the Chevalley basis
sage: sl3 is sl3.derived_subalgebra()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> sl3 = LieAlgebra(QQ, cartan_type=['A', Integer(2)])
>>> sl3.derived_subalgebra()
Lie algebra of ['A', 2] in the Chevalley basis
>>> sl3 is sl3.derived_subalgebra()
True
```

faithful_representation(*algorithm=None*)

Return a faithful representation of `self`.

By Ado's and Iwasawa's theorems, every finite dimensional Lie algebra has a faithful finite dimensional representation.

INPUT:

- `algorithm` – one of the following depending on the classification of the Lie algebra:

  Nilpotent:
  – `'regular'` – use the universal enveloping algebra quotient `FaithfulRepresentationNilpotentPBW`
  – `'minimal'` – construct the minimal representation (for precise details, see the documentation of `FaithfulRepresentationNilpotentPBW`)
  Solvable:
  – Not implemented
  Semisimple:
  – Not implemented
  General case
  – `'generic'` – generic algorithm (only implemented currently for positive characteristic)

Note that the algorithm for any more generic cases can be used in the specialized cases. For instance, using `'generic'` for any Lie algebra (e.g., even if nilpotent) will use the generic implementation.

EXAMPLES:

```
sage: H2 = lie_algebras.Heisenberg(QQ, 2)
sage: H2.is_nilpotent()
True
sage: F = H2.faithful_representation(); F
Faithful 16 dimensional representation of
 Heisenberg algebra of rank 2 over Rational Field
sage: M = H2.faithful_representation(algorithm='minimal'); M
Minimal faithful representation of
 Heisenberg algebra of rank 2 over Rational Field
sage: M.dimension()
4
sage: H2.faithful_representation(algorithm='invalid')
Traceback (most recent call last):
...
ValueError: invalid algorithm 'invalid'

sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.is_nilpotent()
False
sage: L.is_solvable()
True
sage: L.faithful_representation()
Traceback (most recent call last):
...
NotImplementedError: only implemented for nilpotent Lie algebras

sage: sl3 = LieAlgebra(QQ, cartan_type=['A', 2])
sage: sl3.is_semisimple()
True
sage: sl3.faithful_representation()
Traceback (most recent call last):
...
NotImplementedError: only implemented for nilpotent Lie algebras
```

```
>>> from sage.all import *
>>> H2 = lie_algebras.Heisenberg(QQ, Integer(2))
>>> H2.is_nilpotent()
True
>>> F = H2.faithful_representation(); F
Faithful 16 dimensional representation of
 Heisenberg algebra of rank 2 over Rational Field
>>> M = H2.faithful_representation(algorithm='minimal'); M
Minimal faithful representation of
 Heisenberg algebra of rank 2 over Rational Field
>>> M.dimension()
4
>>> H2.faithful_representation(algorithm='invalid')
Traceback (most recent call last):
...
ValueError: invalid algorithm 'invalid'

>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.is_nilpotent()
False
>>> L.is_solvable()
True
>>> L.faithful_representation()
Traceback (most recent call last):
...
NotImplementedError: only implemented for nilpotent Lie algebras

>>> sl3 = LieAlgebra(QQ, cartan_type=['A', Integer(2)])
>>> sl3.is_semisimple()
True
>>> sl3.faithful_representation()
Traceback (most recent call last):
...
NotImplementedError: only implemented for nilpotent Lie algebras
```

from_vector(*v*, *order=None*)

Return the element of `self` corresponding to the vector `v` in `self.module()`.

Implement this if you implement *module()*; see the documentation of *module()* for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: u = L.from_vector(vector(QQ, (1, 0, 0))); u                        #␣
↪needs sage.modules
(1, 0, 0)
sage: parent(u) is L                                                     #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> u = L.from_vector(vector(QQ, (Integer(1), Integer(0), Integer(0))));␣
↪u                          # needs sage.modules
(1, 0, 0)
>>> parent(u) is L                                                     #␣
↪needs sage.modules
True
```

homology(*deg=None*, *M=None*, *sparse=True*, *ncpus=None*)

Return the Lie algebra homology of `self`.

The Lie algebra homology is the homology of the Chevalley-Eilenberg chain complex.

INPUT:

- `deg` – the degree of the homology (optional)
- `M` – (default: the trivial module) a right module of `self`
- `sparse` – boolean (default: `True`); whether to use sparse matrices for the Chevalley-Eilenberg chain complex
- `ncpus` – (optional) how many cpus to use when computing the Chevalley-Eilenberg chain complex

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.cross_product(QQ)
sage: L.homology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 0 over Rational Field,
 2: Vector space of dimension 0 over Rational Field,
 3: Vector space of dimension 1 over Rational Field}

sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.pwitt(GF(5), 5)
sage: L.homology()
{0: Vector space of dimension 1 over Finite Field of size 5,
 1: Vector space of dimension 0 over Finite Field of size 5,
 2: Vector space of dimension 1 over Finite Field of size 5,
 3: Vector space of dimension 1 over Finite Field of size 5,
 4: Vector space of dimension 0 over Finite Field of size 5,
 5: Vector space of dimension 1 over Finite Field of size 5}

sage: # needs sage.combinat sage.modules
sage: d = {('x', 'y'): {'y': 2}}
sage: L.<x,y> = LieAlgebra(ZZ, d)
sage: L.homology()
{0: Z, 1: Z x C2, 2: 0}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.cross_product(QQ)
```

(continues on next page)

(continued from previous page)

```
>>> L.homology()
{0: Vector space of dimension 1 over Rational Field,
 1: Vector space of dimension 0 over Rational Field,
 2: Vector space of dimension 0 over Rational Field,
 3: Vector space of dimension 1 over Rational Field}

>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.pwitt(GF(Integer(5)), Integer(5))
>>> L.homology()
{0: Vector space of dimension 1 over Finite Field of size 5,
 1: Vector space of dimension 0 over Finite Field of size 5,
 2: Vector space of dimension 1 over Finite Field of size 5,
 3: Vector space of dimension 1 over Finite Field of size 5,
 4: Vector space of dimension 0 over Finite Field of size 5,
 5: Vector space of dimension 1 over Finite Field of size 5}

>>> # needs sage.combinat sage.modules
>>> d = {('x', 'y'): {'y': Integer(2)}}
>>> L = LieAlgebra(ZZ, d, names=('x', 'y',)); (x, y,) = L._first_ngens(2)
>>> L.homology()
{0: Z, 1: Z x C2, 2: 0}
```

See also

chevalley_eilenberg_complex()

hypercenter()

Return the hypercenter of `self`.

EXAMPLES:

```
sage: SGA3 = SymmetricGroup(3).algebra(QQ)
sage: L = LieAlgebra(associative=SGA3)
sage: L.hypercenter()
Ideal ((), (1,2,3) + (1,3,2), (2,3) + (1,2) + (1,3)) of
 Lie algebra of Symmetric group algebra of order 3
 over Rational Field

sage: L = lie_algebras.Heisenberg(QQ, 3)
sage: L.hypercenter()
Heisenberg algebra of rank 3 over Rational Field
```

```
>>> from sage.all import *
>>> SGA3 = SymmetricGroup(Integer(3)).algebra(QQ)
>>> L = LieAlgebra(associative=SGA3)
>>> L.hypercenter()
Ideal ((), (1,2,3) + (1,3,2), (2,3) + (1,2) + (1,3)) of
 Lie algebra of Symmetric group algebra of order 3
 over Rational Field

>>> L = lie_algebras.Heisenberg(QQ, Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> L.hypercenter()
Heisenberg algebra of rank 3 over Rational Field
```

ideal(**gens*, ***kwds*)

Return the ideal of `self` generated by `gens`.

INPUT:

- `gens` – list of generators of the ideal
- `category` – (optional) a subcategory of subobjects of finite dimensional Lie algebras with basis

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(QQ, 2)
sage: p1,p2,q1,q2,z = H.basis()
sage: I = H.ideal([p1 - p2, q1 - q2])
sage: I.basis().list()
[-p1 + p2, -q1 + q2, z]
sage: I.reduce(p1 + p2 + q1 + q2 + z)
2*p1 + 2*q1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(QQ, Integer(2))
>>> p1,p2,q1,q2,z = H.basis()
>>> I = H.ideal([p1 - p2, q1 - q2])
>>> I.basis().list()
[-p1 + p2, -q1 + q2, z]
>>> I.reduce(p1 + p2 + q1 + q2 + z)
2*p1 + 2*q1
```

Passing an extra category to an ideal:

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y,z> = LieAlgebra(QQ, abelian=True)
sage: C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
sage: C = C.Subobjects().Graded().Stratified()
sage: I = L.ideal(x, y, category=C)
sage: I.homogeneous_component_basis(1).list()
[x, y]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, abelian=True, names=('x', 'y', 'z',)); (x, y, z,)␣
↪= L._first_ngens(3)
>>> C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
>>> C = C.Subobjects().Graded().Stratified()
>>> I = L.ideal(x, y, category=C)
>>> I.homogeneous_component_basis(Integer(1)).list()
[x, y]
```

inner_derivations_basis()

Return a basis for the Lie algebra of inner derivations of `self` as matrices.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(QQ, 1)
sage: H.inner_derivations_basis()
(
[0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]
[1 0 0], [0 1 0]
)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(QQ, Integer(1))
>>> H.inner_derivations_basis()
(
[0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]
[1 0 0], [0 1 0]
)
```

is_abelian()

Return if `self` is an abelian Lie algebra.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.is_abelian()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.is_abelian()
True
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.is_abelian()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.is_abelian()
False
```

is_ideal(A)

Return if `self` is an ideal of `A`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: I = L.ideal([2*a - c, b + c])
sage: I.is_ideal(L)
True

sage: L.<x,y> = LieAlgebra(QQ, {('x','y'):{'x':1}})                    #␣
↪needs sage.combinat sage.modules
sage: L.is_ideal(L)                                                    #␣
↪needs sage.combinat sage.modules
True

sage: F = LieAlgebra(QQ, 'F', representation='polynomial')              #␣
↪needs sage.combinat sage.modules
sage: L.is_ideal(F)                                                    #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
NotImplementedError: A must be a finite dimensional Lie algebra
 with basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> I = L.ideal([Integer(2)*a - c, b + c])
>>> I.is_ideal(L)
True

>>> L = LieAlgebra(QQ, {('x','y'):{'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)# needs sage.combinat sage.modules
>>> L.is_ideal(L)                                                      #␣
↪needs sage.combinat sage.modules
True

>>> F = LieAlgebra(QQ, 'F', representation='polynomial')               #␣
↪needs sage.combinat sage.modules
>>> L.is_ideal(F)                                                      #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
NotImplementedError: A must be a finite dimensional Lie algebra
 with basis
```

is_nilpotent()

Return if `self` is a nilpotent Lie algebra.

A Lie algebra is nilpotent if the lower central series eventually becomes 0.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.is_nilpotent()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.is_nilpotent()
True
```

is_semisimple()

Return if `self` if a semisimple Lie algebra.

A Lie algebra is semisimple if the solvable radical is zero. In characteristic 0, this is equivalent to saying the Killing form is non-degenerate.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.is_semisimple()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.is_semisimple()
False
```

Positive characteristic examples:

```
sage: L.<x,y,z> = LieAlgebra(GF(3), {('x','z'): {'x':1, 'y':1}, ('y','z
↪'): {'y':1}})
sage: L.is_semisimple()
False

sage: sp4 = LieAlgebra(GF(3), cartan_type=['C',2])
sage: sp4.killing_form_matrix().det()
0
sage: sp4.solvable_radical_basis()  # long time
()
sage: sp4.is_semisimple()  # long time
True
```

```
>>> from sage.all import *
>>> L = LieAlgebra(GF(Integer(3)), {('x','z'): {'x':Integer(1), 'y
↪':Integer(1)}, ('y','z'): {'y':Integer(1)}}, names=('x', 'y', 'z',));␣
↪(x, y, z,) = L._first_ngens(3)
>>> L.is_semisimple()
False

>>> sp4 = LieAlgebra(GF(Integer(3)), cartan_type=['C',Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> sp4.killing_form_matrix().det()
0
>>> sp4.solvable_radical_basis()  # long time
()
>>> sp4.is_semisimple()  # long time
True
```

is_solvable()

Return if `self` is a solvable Lie algebra.

A Lie algebra is solvable if the derived series eventually becomes 0.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.is_solvable()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.is_solvable()
True
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.is_solvable()           # not implemented
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.is_solvable()           # not implemented
False
```

killing_form(x, y)

Return the Killing form on `x` and `y`, where `x` and `y` are two elements of `self`.

The Killing form is defined as

$$\langle x \mid y \rangle = \operatorname{tr}(\operatorname{ad}_x \circ \operatorname{ad}_y).$$

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
sage: L.killing_form(a, b)                                               #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> L.killing_form(a, b)                                               #␣
↪needs sage.modules
0
```

killing_form_matrix()

Return the matrix of the Killing form of `self`.

The rows and the columns of this matrix are indexed by the elements of the basis of `self` (in the order provided by *basis()*).

EXAMPLES:

```
sage: # needs sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.killing_form_matrix()
[0 0 0]
[0 0 0]
[0 0 0]
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example(0)
sage: m = L.killing_form_matrix(); m
[]
sage: parent(m)
Full MatrixSpace of 0 by 0 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.killing_form_matrix()
[0 0 0]
[0 0 0]
[0 0 0]
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().
↪example(Integer(0))
>>> m = L.killing_form_matrix(); m
[]
>>> parent(m)
Full MatrixSpace of 0 by 0 dense matrices over Rational Field
```

killing_matrix(x, y)

Return the Killing matrix of `x` and `y`, where `x` and `y` are two elements of `self`.

The Killing matrix is defined as the matrix corresponding to the action of $\mathrm{ad}_x \circ \mathrm{ad}_y$ in the basis of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
sage: L.killing_matrix(a, b)                                                #␣
↪needs sage.modules
[0 0 0]
[0 0 0]
[0 0 0]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> L.killing_matrix(a, b)                                             #␣
↪needs sage.modules
[0 0 0]
[0 0 0]
[0 0 0]
```

```
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})                       #␣
↪needs sage.combinat sage.modules
sage: L.killing_matrix(y, x)                                               #␣
↪needs sage.combinat sage.modules
[ 0 -1]
[ 0  0]
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)# needs sage.combinat sage.modules
>>> L.killing_matrix(y, x)                                                 #␣
↪needs sage.combinat sage.modules
[ 0 -1]
[ 0  0]
```

lower_central_series (*submodule=False*)

Return the lower central series $(\mathfrak{g}_i)_i$ of `self` where the rightmost $\mathfrak{g}_k = \mathfrak{g}_{k+1} = \cdots$.

INPUT:

- `submodule` – boolean (default: `False`); if `True`, then the result is given as submodules of `self`

We define the lower central series of a Lie algebra $\mathfrak{g}$ recursively by $\mathfrak{g}_0 := \mathfrak{g}$ and

$$\mathfrak{g}_{k+1} = [\mathfrak{g}, \mathfrak{g}_k]$$

and recall that $\mathfrak{g}_k \supseteq \mathfrak{g}_{k+1}$. Alternatively we can express this as

$$\mathfrak{g} \supseteq [\mathfrak{g}, \mathfrak{g}] \supseteq [[\mathfrak{g}, \mathfrak{g}], \mathfrak{g}] \supseteq \Big[[[\mathfrak{g}, \mathfrak{g}], \mathfrak{g}], \mathfrak{g}\Big] \supseteq \cdots .$$

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.derived_series()
(An example of a finite dimensional Lie algebra with basis:
  the 3-dimensional abelian Lie algebra over Rational Field,
```

(continues on next page)

(continued from previous page)

```
 An example of a finite dimensional Lie algebra with basis:
  the 0-dimensional abelian Lie algebra over Rational Field
   with basis matrix: [])
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.derived_series()
(An example of a finite dimensional Lie algebra with basis:
  the 3-dimensional abelian Lie algebra over Rational Field,
 An example of a finite dimensional Lie algebra with basis:
  the 0-dimensional abelian Lie algebra over Rational Field
   with basis matrix: [])
```

The lower central series as submodules:

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.lower_central_series(submodule=True)
(Sparse vector space of dimension 2 over Rational Field,
 Vector space of degree 2 and dimension 1 over Rational Field
  Basis matrix: [1 0])
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.lower_central_series(submodule=True)
(Sparse vector space of dimension 2 over Rational Field,
 Vector space of degree 2 and dimension 1 over Rational Field
  Basis matrix: [1 0])
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.lower_central_series()
(Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal (x) of Lie algebra on 2 generators (x, y) over Rational Field)

sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.lower_central_series()
(Lie algebra on 5 generators (a, b, c, d, e) over Rational Field,
 Ideal (a, b, c) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field,
 Ideal (a, b) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.lower_central_series()
(Lie algebra on 2 generators (x, y) over Rational Field,
 Ideal (x) of Lie algebra on 2 generators (x, y) over Rational Field)

>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.lower_central_series()
(Lie algebra on 5 generators (a, b, c, d, e) over Rational Field,
 Ideal (a, b, c) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field,
 Ideal (a, b) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field)
```

module(*R=None*)

Return a dense free module associated to `self` over `R`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L._dense_free_module()                                             #␣
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L._dense_free_module()                                             #␣
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

morphism(*on_generators*, *codomain=None*, *base_map=None*, *check=True*)

Return a Lie algebra morphism defined by images of a Lie generating subset of `self`.

INPUT:

- `on_generators` – dictionary `{X: Y}` of the images Y in `codomain` of elements X of `domain`
- `codomain` – a Lie algebra (optional); this is inferred from the values of `on_generators` if not given
- `base_map` – a homomorphism from the base ring to something coercing into the codomain
- `check` – boolean (default: `True`); if `False` the values on the Lie brackets implied by `on_generators` will not be checked for contradictory values

Note

The keys of `on_generators` need to generate `domain` as a Lie algebra.

See also

sage.algebras.lie_algebras.morphism.LieAlgebraMorphism_from_generators

EXAMPLES:

A quotient type Lie algebra morphism

```
sage: # needs sage.combinat sage.modules
sage: L.<X,Y,Z,W> = LieAlgebra(QQ, {('X','Y'): {'Z': 1},
....:                              ('X','Z'): {'W': 1}})
sage: K.<A,B> = LieAlgebra(QQ, abelian=True)
sage: L.morphism({X: A, Y: B})
Lie algebra morphism:
  From: Lie algebra on 4 generators (X, Y, Z, W) over Rational Field
  To:   Abelian Lie algebra on 2 generators (A, B) over Rational Field
  Defn: X |--> A
        Y |--> B
        Z |--> 0
        W |--> 0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('X','Y'): {'Z': Integer(1)},
...                                 ('X','Z'): {'W': Integer(1)}}, names=('X
↪', 'Y', 'Z', 'W',)); (X, Y, Z, W,) = L._first_ngens(4)
>>> K = LieAlgebra(QQ, abelian=True, names=('A', 'B',)); (A, B,) = K._
↪first_ngens(2)
>>> L.morphism({X: A, Y: B})
Lie algebra morphism:
  From: Lie algebra on 4 generators (X, Y, Z, W) over Rational Field
  To:   Abelian Lie algebra on 2 generators (A, B) over Rational Field
  Defn: X |--> A
        Y |--> B
        Z |--> 0
        W |--> 0
```

The reverse map $A \mapsto X$, $B \mapsto Y$ does not define a Lie algebra morphism, since $[A, B] = 0$, but $[X, Y] \neq 0$:

```
sage: # needs sage.combinat sage.modules
sage: K.morphism({A:X, B: Y})
Traceback (most recent call last):
...
ValueError: this does not define a Lie algebra morphism;
 contradictory values for brackets of length 2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> K.morphism({A:X, B: Y})
Traceback (most recent call last):
...
ValueError: this does not define a Lie algebra morphism;
```

(continues on next page)

(continued from previous page)

```
contradictory values for brackets of length 2
```

However, it is still possible to create a morphism that acts nontrivially on the coefficients, even though it's not a Lie algebra morphism (since it isn't linear):

```
sage: # needs sage.combinat sage.modules sage.rings.number_fields
sage: R.<x> = ZZ[]
sage: K.<i> = NumberField(x^2 + 1)
sage: cc = K.hom([-i])
sage: L.<X,Y,Z,W> = LieAlgebra(K, {('X','Y'): {'Z': 1},
....:                              ('X','Z'): {'W': 1}})
sage: M.<A,B> = LieAlgebra(K, abelian=True)
sage: phi = L.morphism({X: A, Y: B}, base_map=cc)
sage: phi(X)
A
sage: phi(i*X)
-i*A
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules sage.rings.number_fields
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> K = NumberField(x**Integer(2) + Integer(1), names=('i',)); (i,) = K._
↪first_ngens(1)
>>> cc = K.hom([-i])
>>> L = LieAlgebra(K, {('X','Y'): {'Z': Integer(1)},
...                    ('X','Z'): {'W': Integer(1)}}, names=('X
↪', 'Y', 'Z', 'W',)); (X, Y, Z, W,) = L._first_ngens(4)
>>> M = LieAlgebra(K, abelian=True, names=('A', 'B',)); (A, B,) = M._
↪first_ngens(2)
>>> phi = L.morphism({X: A, Y: B}, base_map=cc)
>>> phi(X)
A
>>> phi(i*X)
-i*A
```

nilradical()

Return the nilradical of `self`.

The *nilradical* of a Lie algebra L is the largest nilpotent ideal of L.

See also

nilradical_basis()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.solvable_radical()
```

(continues on next page)

(continued from previous page)

```
Ideal (a, b, c, d, e) of
 Lie algebra on 5 generators (a, b, c, d, e) over Rational Field
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.solvable_radical()
Ideal (a, b, c, d, e) of
 Lie algebra on 5 generators (a, b, c, d, e) over Rational Field
```

nilradical_basis()

Return a basis of the nilradical of `self`.

See also

nilradical()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.nilradical_basis()
(a, b, c)
sage: L.is_nilpotent()
False

sage: sl3 = LieAlgebra(QQ, cartan_type=['A',2])
sage: sl3.nilradical_basis()
()

sage: scoeffs = {('a','e'): {'a':1}, ('b','e'): {'a':1,'b':1},
....:            ('c','d'): {'a':1}, ('c','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.nilradical_basis()
(a, b, c, d)
sage: L.is_solvable()
True
sage: L.is_nilpotent()
False

sage: K1 = L.quotient([a])
sage: K1.nilradical_basis()
(b, c, d)

sage: SL = L.subalgebra([a,b,c,d]); SL
Subalgebra generated by (a, b, c, d) of
```

(continues on next page)

(continued from previous page)

```
 Lie algebra on 5 generators (a, b, c, d, e) over Rational Field
sage: SL.nilradical_basis()
(a, b, c, d)

sage: scoeffs = {('x','z'): {'x':1, 'y':1}, ('y','z'): {'y':1}}
sage: L.<x,y,z> = LieAlgebra(GF(3), scoeffs)
sage: L.nilradical_basis()
(x, y)
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.nilradical_basis()
(a, b, c)
>>> L.is_nilpotent()
False

>>> sl3 = LieAlgebra(QQ, cartan_type=['A',Integer(2)])
>>> sl3.nilradical_basis()
()

>>> scoeffs = {('a','e'): {'a':Integer(1)}, ('b','e'): {'a':Integer(1),'b
↪':Integer(1)},
...            ('c','d'): {'a':Integer(1)}, ('c','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.nilradical_basis()
(a, b, c, d)
>>> L.is_solvable()
True
>>> L.is_nilpotent()
False

>>> K1 = L.quotient([a])
>>> K1.nilradical_basis()
(b, c, d)

>>> SL = L.subalgebra([a,b,c,d]); SL
Subalgebra generated by (a, b, c, d) of
 Lie algebra on 5 generators (a, b, c, d, e) over Rational Field
>>> SL.nilradical_basis()
(a, b, c, d)

>>> scoeffs = {('x','z'): {'x':Integer(1), 'y':Integer(1)}, ('y','z'): {'y
↪':Integer(1)}}
>>> L = LieAlgebra(GF(Integer(3)), scoeffs, names=('x', 'y', 'z',)); (x,␣
↪y, z,) = L._first_ngens(3)
>>> L.nilradical_basis()
(x, y)
```

We check against the generic algorithm:

```
sage: L.<x,y,z> = LieAlgebra(QQ, {('x','z'): {'x':1,'y':1}, ('y','z'): {'y
↪':1}})
sage: L.nilradical_basis()
(x, y)

sage: dim = L.dimension()
sage: MS = MatrixSpace(L.base_ring(), dim)
sage: gens = [b.adjoint_matrix() for b in L.basis()]
sage: A = MS.subalgebra(gens)
sage: RB = A.radical_basis()
sage: mat = matrix(L.base_ring(),
....:              [g._vector_() for g in gens]
....:              + [A.lift(r)._vector_() for r in RB])
sage: tuple([L.from_vector(w) for v in mat.right_kernel().basis()
....:                if (w := v[:dim])])
(x, y)
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, {('x','z'): {'x':Integer(1),'y':Integer(1)}, ('y',
↪'z'): {'y':Integer(1)}}, names=('x', 'y', 'z',)); (x, y, z,) = L._first_
↪ngens(3)
>>> L.nilradical_basis()
(x, y)

>>> dim = L.dimension()
>>> MS = MatrixSpace(L.base_ring(), dim)
>>> gens = [b.adjoint_matrix() for b in L.basis()]
>>> A = MS.subalgebra(gens)
>>> RB = A.radical_basis()
>>> mat = matrix(L.base_ring(),
...              [g._vector_() for g in gens]
...              + [A.lift(r)._vector_() for r in RB])
>>> tuple([L.from_vector(w) for v in mat.right_kernel().basis()
...                if (w := v[:dim])])
(x, y)
```

A positive characteristic example:

```
sage: scoeffs = {('x','z'): {'x':1,'y':1}, ('y','z'): {'y':1}}
sage: L.<x,y,z> = LieAlgebra(GF(3), scoeffs)
sage: L.nilradical_basis()
(x, y)
```

```
>>> from sage.all import *
>>> scoeffs = {('x','z'): {'x':Integer(1),'y':Integer(1)}, ('y','z'): {'y
↪':Integer(1)}}
>>> L = LieAlgebra(GF(Integer(3)), scoeffs, names=('x', 'y', 'z',)); (x,␣
↪y, z,) = L._first_ngens(3)
>>> L.nilradical_basis()
(x, y)
```

normalizer(S)

Return the normalizer of `S` in `self`.

INPUT:

- `S` – a subalgebra of `self` or a list of elements that represent generators for a subalgebra

See also

normalizer_basis()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.normalizer([a, e])
Subalgebra generated by (b, c) of Lie algebra on
 5 generators (a, b, c, d, e) over Rational Field
sage: L.normalizer([a, c, e])
Subalgebra generated by (b, c, d) of Lie algebra on
 5 generators (a, b, c, d, e) over Rational Field
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.normalizer([a, e])
Subalgebra generated by (b, c) of Lie algebra on
 5 generators (a, b, c, d, e) over Rational Field
>>> L.normalizer([a, c, e])
Subalgebra generated by (b, c, d) of Lie algebra on
 5 generators (a, b, c, d, e) over Rational Field
```

normalizer_basis(*S*)

Return a basis of the normalizer of `S` in `self`.

INPUT:

- `S` – a subalgebra of `self` or a list of elements that represent generators for a subalgebra

See also

normalizer()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.normalizer_basis([a, e])
```

(continues on next page)

(continued from previous page)

```
[b, c]

sage: S = L.subalgebra([a, e])
sage: L.normalizer_basis(S)
[a, b, c, e]
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.normalizer_basis([a, e])
[b, c]

>>> S = L.subalgebra([a, e])
>>> L.normalizer_basis(S)
[a, b, c, e]
```

When the subalgebra is the ambient Lie algebra, we return the basis of the ambient Lie algebra:

```
sage: L.normalizer_basis(L)
Finite family {'a': a, 'b': b, 'c': c, 'd': d, 'e': e}
sage: L.normalizer_basis([a, b, c, a, d + e, a + e])
Finite family {'a': a, 'b': b, 'c': c, 'd': d, 'e': e}
```

```
>>> from sage.all import *
>>> L.normalizer_basis(L)
Finite family {'a': a, 'b': b, 'c': c, 'd': d, 'e': e}
>>> L.normalizer_basis([a, b, c, a, d + e, a + e])
Finite family {'a': a, 'b': b, 'c': c, 'd': d, 'e': e}
```

product_space(*L*, *submodule=False*)

Return the product space `[self, L]`.

INPUT:

- `L` – a Lie subalgebra of `self`
- `submodule` – boolean (default: `False`); if `True`, then the result is forced to be a submodule of `self`

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a,b,c = L.lie_algebra_generators()
sage: X = L.subalgebra([a, b + c])
sage: L.product_space(X)
An example of a finite dimensional Lie algebra with basis:
 the 0-dimensional abelian Lie algebra over Rational Field
  with basis matrix: []
sage: Y = L.subalgebra([a, 2*b - c])
sage: X.product_space(Y)
An example of a finite dimensional Lie algebra with basis:
```

(continues on next page)

(continued from previous page)

```
 the 0-dimensional abelian Lie algebra over Rational Field
  with basis matrix: []
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a,b,c = L.lie_algebra_generators()
>>> X = L.subalgebra([a, b + c])
>>> L.product_space(X)
An example of a finite dimensional Lie algebra with basis:
 the 0-dimensional abelian Lie algebra over Rational Field
  with basis matrix: []
>>> Y = L.subalgebra([a, Integer(2)*b - c])
>>> X.product_space(Y)
An example of a finite dimensional Lie algebra with basis:
 the 0-dimensional abelian Lie algebra over Rational Field
  with basis matrix: []
```

```
sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(ZZ, 4)
sage: Hp = H.product_space(H, submodule=True).basis()
sage: [H.from_vector(v) for v in Hp]
[z]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(ZZ, Integer(4))
>>> Hp = H.product_space(H, submodule=True).basis()
>>> [H.from_vector(v) for v in Hp]
[z]
```

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'):{'x':1}})
sage: Lp = L.product_space(L)    # not implemented
sage: Lp                         # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
sage: Lp.product_space(L)        # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
sage: L.product_space(Lp)        # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
sage: Lp.product_space(Lp)       # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: ()
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'):{'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> Lp = L.product_space(L)    # not implemented
>>> Lp                         # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
>>> Lp.product_space(L)        # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
>>> L.product_space(Lp)        # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: (x,)
>>> Lp.product_space(Lp)       # not implemented
Subalgebra generated of
 Lie algebra on 2 generators (x, y) over Rational Field
 with basis: ()
```

quotient(*I*, *names=None*, *category=None*)

Return the quotient of `self` by the ideal `I`.

A quotient Lie algebra.

INPUT:

- `I` – an ideal or a list of generators of the ideal
- `names` – (optional) string or list of strings; names for the basis elements of the quotient. If `names` is a string, the basis will be named `names_1`,…,``names_n``.

EXAMPLES:

The Engel Lie algebra as a quotient of the free nilpotent Lie algebra of step 3 with 2 generators:

```
sage: # needs sage.combinat sage.modules
sage: L.<X,Y,Z,W,U> = LieAlgebra(QQ, 2, step=3)
sage: E = L.quotient(U); E
Lie algebra quotient L/I of dimension 4 over Rational Field where
 L: Free Nilpotent Lie algebra of rank 2 and step 3 over Rational Field
 I: Ideal (U)
sage: E.basis().list()
[X, Y, Z, W]
sage: E(X).bracket(E(Y))
Z
sage: Y.bracket(Z)
-U
sage: E(Y).bracket(E(Z))
0
sage: E(U)
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> L = LieAlgebra(QQ, Integer(2), step=Integer(3), names=('X', 'Y', 'Z',
↪'W', 'U',)); (X, Y, Z, W, U,) = L._first_ngens(5)
>>> E = L.quotient(U); E
Lie algebra quotient L/I of dimension 4 over Rational Field where
 L: Free Nilpotent Lie algebra of rank 2 and step 3 over Rational Field
 I: Ideal (U)
>>> E.basis().list()
[X, Y, Z, W]
>>> E(X).bracket(E(Y))
Z
>>> Y.bracket(Z)
-U
>>> E(Y).bracket(E(Z))
0
>>> E(U)
0
```

Quotients when the base ring is not a field are not implemented:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.Heisenberg(ZZ, 1)
sage: L.quotient(L.an_element())
Traceback (most recent call last):
...
NotImplementedError: quotients over non-fields not implemented
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.Heisenberg(ZZ, Integer(1))
>>> L.quotient(L.an_element())
Traceback (most recent call last):
...
NotImplementedError: quotients over non-fields not implemented
```

solvable_radical()

Return the solvable radical of `self`.

The *solvable radical* of a Lie algebra L is the largest solvable ideal of L.

See also

solvable_radical_basis()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.solvable_radical()
Ideal (a, b, c, d, e) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.solvable_radical()
Ideal (a, b, c, d, e) of Lie algebra on 5 generators (a, b, c, d, e) over␣
↪Rational Field
```

solvable_radical_basis()

Return a basis of the solvable radical of `self`.

See also

solvable_radical()

EXAMPLES:

```
sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.solvable_radical_basis()
(a, b, c, d, e)
sage: L.is_solvable()
True

sage: sl3 = LieAlgebra(QQ, cartan_type=['A',2])
sage: sl3.solvable_radical_basis()
()

sage: L.<x,y,z> = LieAlgebra(QQ, {('x','z'): {'x':1,'y':1}, ('y','z'): {'y
↪':1}})
sage: S = L.subalgebra([x, y])
sage: S.solvable_radical_basis()
(x, y)
sage: S.is_solvable()
True
```

```
>>> from sage.all import *
>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.solvable_radical_basis()
(a, b, c, d, e)
>>> L.is_solvable()
True

>>> sl3 = LieAlgebra(QQ, cartan_type=['A',Integer(2)])
```

(continues on next page)

(continued from previous page)

```
>>> sl3.solvable_radical_basis()
()

>>> L = LieAlgebra(QQ, {('x','z'): {'x':Integer(1),'y':Integer(1)}, ('y',
↪'z'): {'y':Integer(1)}}, names=('x', 'y', 'z',)); (x, y, z,) = L._first_
↪ngens(3)
>>> S = L.subalgebra([x, y])
>>> S.solvable_radical_basis()
(x, y)
>>> S.is_solvable()
True
```

Positive characteristic examples:

```
sage: scoeffs = {('x','z'): {'x':1,'y':1}, ('y','z'): {'y':1}}
sage: L.<x,y,z> = LieAlgebra(GF(3), scoeffs)
sage: L.solvable_radical_basis()
(x, y, z)
sage: sl3 = LieAlgebra(GF(3), cartan_type=['A',2])
sage: sl3.solvable_radical_basis()
(2*h1 + h2,)
```

```
>>> from sage.all import *
>>> scoeffs = {('x','z'): {'x':Integer(1),'y':Integer(1)}, ('y','z'): {'y
↪':Integer(1)}}
>>> L = LieAlgebra(GF(Integer(3)), scoeffs, names=('x', 'y', 'z',)); (x,␣
↪y, z,) = L._first_ngens(3)
>>> L.solvable_radical_basis()
(x, y, z)
>>> sl3 = LieAlgebra(GF(Integer(3)), cartan_type=['A',Integer(2)])
>>> sl3.solvable_radical_basis()
(2*h1 + h2,)
```

structure_coefficients(*include_zeros=False*)

Return the structure coefficients of `self`.

INPUT:

- `include_zeros` – boolean (default: `False`); if `True`, then include the $[x, y] = 0$ pairs in the output

OUTPUT:

A dictionary whose keys are pairs of basis indices (i, j) with $i < j$, and whose values are the corresponding *elements* $[b_i, b_j]$ in the Lie algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.structure_coefficients()                                         #␣
↪needs sage.modules
Finite family {}
sage: L.structure_coefficients(True)                                     #␣
↪needs sage.modules
Finite family {(0, 1): (0, 0, 0), (0, 2): (0, 0, 0), (1, 2): (0, 0, 0)}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L.structure_coefficients()                                         #␣
↪needs sage.modules
Finite family {}
>>> L.structure_coefficients(True)                                     #␣
↪needs sage.modules
Finite family {(0, 1): (0, 0, 0), (0, 2): (0, 0, 0), (1, 2): (0, 0, 0)}
```

```
sage: # needs sage.combinat sage.groups sage.modules
sage: G = SymmetricGroup(3)
sage: S = GroupAlgebra(G, QQ)
sage: L = LieAlgebra(associative=S)
sage: L.structure_coefficients()
Finite family {((2,3), (1,2)): (1,2,3) - (1,3,2),
               ((2,3), (1,3)): -(1,2,3) + (1,3,2),
               ((1,2,3), (2,3)): -(1,2) + (1,3),
               ((1,2,3), (1,2)): (2,3) - (1,3),
               ((1,2,3), (1,3)): -(2,3) + (1,2),
               ((1,3,2), (2,3)): (1,2) - (1,3),
               ((1,3,2), (1,2)): -(2,3) + (1,3),
               ((1,3,2), (1,3)): (2,3) - (1,2),
               ((1,3), (1,2)): -(1,2,3) + (1,3,2)}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> G = SymmetricGroup(Integer(3))
>>> S = GroupAlgebra(G, QQ)
>>> L = LieAlgebra(associative=S)
>>> L.structure_coefficients()
Finite family {((2,3), (1,2)): (1,2,3) - (1,3,2),
               ((2,3), (1,3)): -(1,2,3) + (1,3,2),
               ((1,2,3), (2,3)): -(1,2) + (1,3),
               ((1,2,3), (1,2)): (2,3) - (1,3),
               ((1,2,3), (1,3)): -(2,3) + (1,2),
               ((1,3,2), (2,3)): (1,2) - (1,3),
               ((1,3,2), (1,2)): -(2,3) + (1,3),
               ((1,3,2), (1,3)): (2,3) - (1,2),
               ((1,3), (1,2)): -(1,2,3) + (1,3,2)}
```

subalgebra(**gens*, ***kwds*)

Return the subalgebra of `self` generated by `gens`.

INPUT:

- `gens` – list of generators of the subalgebra
- `category` – (optional) a subcategory of subobjects of finite dimensional Lie algebras with basis

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: H = lie_algebras.Heisenberg(QQ, 2)
sage: p1,p2,q1,q2,z = H.basis()
sage: S = H.subalgebra([p1, q1])
```

(continues on next page)

(continued from previous page)

```
sage: S.basis().list()
[p1, q1, z]
sage: S.basis_matrix()
[1 0 0 0 0]
[0 0 1 0 0]
[0 0 0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> H = lie_algebras.Heisenberg(QQ, Integer(2))
>>> p1,p2,q1,q2,z = H.basis()
>>> S = H.subalgebra([p1, q1])
>>> S.basis().list()
[p1, q1, z]
>>> S.basis_matrix()
[1 0 0 0 0]
[0 0 1 0 0]
[0 0 0 0 1]
```

Passing an extra category to a subalgebra:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, 3, step=2)
sage: x,y,z = L.homogeneous_component_basis(1)
sage: C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
sage: C = C.Subobjects().Graded().Stratified()
sage: S = L.subalgebra([x, y], category=C)
sage: S.homogeneous_component_basis(2).list()
[X_12]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, Integer(3), step=Integer(2))
>>> x,y,z = L.homogeneous_component_basis(Integer(1))
>>> C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
>>> C = C.Subobjects().Graded().Stratified()
>>> S = L.subalgebra([x, y], category=C)
>>> S.homogeneous_component_basis(Integer(2)).list()
[X_12]
```

universal_commutative_algebra()

Return the universal commutative algebra associated to `self`.

Let I be the index set of the basis of `self`. Let $\mathcal{P} = \{P_{a,i,j}\}_{a,i,j \in I}$ denote the universal polynomials of a Lie algebra L. The *universal commutative algebra* associated to L is the quotient ring $R[X_{ij}]_{i,j \in I}/(\mathcal{P})$.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: A = L.universal_commutative_algebra()
sage: a, b, c, d = A.gens()
sage: a, b, c, d
```

(continues on next page)

(continued from previous page)

```
(X00bar, X01bar, 0, X11bar)
sage: a*d - a
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> A = L.universal_commutative_algebra()
>>> a, b, c, d = A.gens()
>>> a, b, c, d
(X00bar, X01bar, 0, X11bar)
>>> a*d - a
0
```

universal_polynomials()

Return the family of universal polynomials of `self`.

The *universal polynomials* of a Lie algebra L with basis $\{e_i\}_{i \in I}$ and structure coefficients $[e_i, e_j] = \tau_{ij}^a e_a$ is given by

$$P_{aij} = \sum_{u \in I} \tau_{ij}^u X_{au} - \sum_{s,t \in I} \tau_{st}^a X_{si} X_{tj},$$

where $a, i, j \in I$.

REFERENCES:

- [AM2020]

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.universal_polynomials()
Finite family {('x', 'x', 'y'): X01*X10 - X00*X11 + X00,
               ('y', 'x', 'y'): X10}

sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['A',1])
sage: list(L.universal_polynomials())
[-2*X01*X10 + 2*X00*X11 - 2*X00,
 -2*X02*X10 + 2*X00*X12 + X01,
 -2*X02*X11 + 2*X01*X12 - 2*X02,
 X01*X20 - X00*X21 - 2*X10,
 X02*X20 - X00*X22 + X11,
 X02*X21 - X01*X22 - 2*X12,
 -2*X11*X20 + 2*X10*X21 - 2*X20,
 -2*X12*X20 + 2*X10*X22 + X21,
 -2*X12*X21 + 2*X11*X22 - 2*X22]

sage: # long time, needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type=['B', 2])
sage: al = RootSystem(['B', 2]).root_lattice().simple_roots()
sage: k = list(L.basis().keys())[0]
```

(continues on next page)

(continued from previous page)

```
sage: UP = L.universal_polynomials()
sage: len(UP)
450
sage: UP[al[2], al[1], -al[1]]
X0_7*X4_1 - X0_1*X4_7 - 2*X0_7*X5_1 + 2*X0_1*X5_7 + X2_7*X7_1
 - X2_1*X7_7 - X3_7*X8_1 + X3_1*X8_7 + X0_4
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.universal_polynomials()
Finite family {('x', 'x', 'y'): X01*X10 - X00*X11 + X00,
               ('y', 'x', 'y'): X10}

>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['A',Integer(1)])
>>> list(L.universal_polynomials())
[-2*X01*X10 + 2*X00*X11 - 2*X00,
 -2*X02*X10 + 2*X00*X12 + X01,
 -2*X02*X11 + 2*X01*X12 - 2*X02,
 X01*X20 - X00*X21 - 2*X10,
 X02*X20 - X00*X22 + X11,
 X02*X21 - X01*X22 - 2*X12,
 -2*X11*X20 + 2*X10*X21 - 2*X20,
 -2*X12*X20 + 2*X10*X22 + X21,
 -2*X12*X21 + 2*X11*X22 - 2*X22]

>>> # long time, needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type=['B', Integer(2)])
>>> al = RootSystem(['B', Integer(2)]).root_lattice().simple_roots()
>>> k = list(L.basis().keys())[Integer(0)]
>>> UP = L.universal_polynomials()
>>> len(UP)
450
>>> UP[al[Integer(2)], al[Integer(1)], -al[Integer(1)]]
X0_7*X4_1 - X0_1*X4_7 - 2*X0_7*X5_1 + 2*X0_1*X5_7 + X2_7*X7_1
 - X2_1*X7_7 - X3_7*X8_1 + X3_1*X8_7 + X0_4
```

upper_central_series()

Return the upper central series $(Z_i(\mathfrak{g}))_i$ of `self` where the rightmost $Z_k(\mathfrak{g}) = Z_{k+1}(\mathfrak{g}) = \cdots$.

The *upper central series* of a Lie algebra $\mathfrak{g}$ is defined recursively by $Z_0(\mathfrak{g}) := Z(\mathfrak{g})$ and

$$Z_{k+1}(\mathfrak{g})/Z_k(\mathfrak{g}) = Z(\mathfrak{g}/Z_k(\mathfrak{g}),$$

and recall that $Z(\mathfrak{g})$ is the *center()* of $\mathfrak{g}$.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.upper_central_series()
[An example of a finite dimensional Lie algebra with basis:
```

(continues on next page)

(continued from previous page)

```
 the 3-dimensional abelian Lie algebra over Rational Field]

sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'x':1}})
sage: L.upper_central_series()
[Ideal () of Lie algebra on 2 generators (x, y) over Rational Field]

sage: scoeffs = {('a','d'): {'a':1}, ('a','e'): {'b':-1},
....:            ('b','d'): {'b':1}, ('b','e'): {'a':1},
....:            ('d','e'): {'c':1}}
sage: L.<a,b,c,d,e> = LieAlgebra(QQ, scoeffs)
sage: L.upper_central_series()
[Ideal (c) of Lie algebra on 5 generators (a, b, c, d, e) over Rational␣
↪Field]

sage: L = lie_algebras.Heisenberg(QQ, 3)
sage: L.upper_central_series()
[Ideal (z) of Heisenberg algebra of rank 3 over Rational Field,
 Heisenberg algebra of rank 3 over Rational Field]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.upper_central_series()
[An example of a finite dimensional Lie algebra with basis:
 the 3-dimensional abelian Lie algebra over Rational Field]

>>> L = LieAlgebra(QQ, {('x','y'): {'x':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> L.upper_central_series()
[Ideal () of Lie algebra on 2 generators (x, y) over Rational Field]

>>> scoeffs = {('a','d'): {'a':Integer(1)}, ('a','e'): {'b':-Integer(1)},
...            ('b','d'): {'b':Integer(1)}, ('b','e'): {'a':Integer(1)},
...            ('d','e'): {'c':Integer(1)}}
>>> L = LieAlgebra(QQ, scoeffs, names=('a', 'b', 'c', 'd', 'e',)); (a, b,␣
↪c, d, e,) = L._first_ngens(5)
>>> L.upper_central_series()
[Ideal (c) of Lie algebra on 5 generators (a, b, c, d, e) over Rational␣
↪Field]

>>> L = lie_algebras.Heisenberg(QQ, Integer(3))
>>> L.upper_central_series()
[Ideal (z) of Heisenberg algebra of rank 3 over Rational Field,
 Heisenberg algebra of rank 3 over Rational Field]
```

class Subobjects(*category*, **args*)

Bases: *SubobjectsCategory*

A category for subalgebras of a finite dimensional Lie algebra with basis.

class ParentMethods

Bases: object

ambient()

Return the ambient Lie algebra of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
sage: L = C.example()
sage: a, b, c = L.lie_algebra_generators()
sage: S = L.subalgebra([2*a + b, b + c])
sage: S.ambient() == L
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
>>> L = C.example()
>>> a, b, c = L.lie_algebra_generators()
>>> S = L.subalgebra([Integer(2)*a + b, b + c])
>>> S.ambient() == L
True
```

basis_matrix()

Return the basis matrix of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
sage: L = C.example()
sage: a, b, c = L.lie_algebra_generators()
sage: S = L.subalgebra([2*a + b, b + c])
sage: S.basis_matrix()
[   1    0 -1/2]
[   0    1    1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
>>> L = C.example()
>>> a, b, c = L.lie_algebra_generators()
>>> S = L.subalgebra([Integer(2)*a + b, b + c])
>>> S.basis_matrix()
[   1    0 -1/2]
[   0    1    1]
```

reduce(X)

Reduce an element of the ambient Lie algebra modulo the ideal `self`.

INPUT:

- `X` – an element of the ambient Lie algebra

OUTPUT:

An element Y of the ambient Lie algebra that is contained in a fixed complementary submodule V to `self` such that $X = Y$ mod `self`.

When the base ring of `self` is a field, the complementary submodule V is spanned by the elements of the basis that are not the leading supports of the basis of `self`.

EXAMPLES:

An example reduction in a 6 dimensional Lie algebra:

```
sage: sc = {('a','b'): {'d': 1}, ('a','c'): {'e': 1},
....:       ('b','c'): {'f': 1}}
sage: L.<a,b,c,d,e,f> = LieAlgebra(QQ, sc)
sage: I = L.ideal(c)
sage: I.reduce(a + b + c + d + e + f)
a + b + d
```

```
>>> from sage.all import *
>>> sc = {('a','b'): {'d': Integer(1)}, ('a','c'): {'e': Integer(1)},
...       ('b','c'): {'f': Integer(1)}}
>>> L = LieAlgebra(QQ, sc, names=('a', 'b', 'c', 'd', 'e', 'f',)); (a,␣
↪b, c, d, e, f,) = L._first_ngens(6)
>>> I = L.ideal(c)
>>> I.reduce(a + b + c + d + e + f)
a + b + d
```

The reduction of an element is zero if and only if the element belongs to the subalgebra:

```
sage: I.reduce(c + e)
0
sage: c + e in I
True
```

```
>>> from sage.all import *
>>> I.reduce(c + e)
0
>>> c + e in I
True
```

Over non-fields, the complementary submodule may not be spanned by a subset of the basis of the ambient Lie algebra:

```
sage: L.<X,Y,Z> = LieAlgebra(ZZ, {('X','Y'): {'Z': 3}})
sage: I = L.ideal(Y)
sage: I.basis()
Finite family {'Y': Y, 'Z': 3*Z}
sage: I.reduce(3*Z)
0
sage: I.reduce(Y + 14*Z)
2*Z
```

```
>>> from sage.all import *
>>> L = LieAlgebra(ZZ, {('X','Y'): {'Z': Integer(3)}}, names=('X', 'Y',
↪ 'Z',)); (X, Y, Z,) = L._first_ngens(3)
>>> I = L.ideal(Y)
>>> I.basis()
Finite family {'Y': Y, 'Z': 3*Z}
```

(continues on next page)

(continued from previous page)

```
>>> I.reduce(Integer(3)*Z)
0
>>> I.reduce(Y + Integer(14)*Z)
2*Z
```

We can reduce elements of a subalgebra A by an ideal B (Issue #40137):

```
sage: L.<a,b,c,d> = LieAlgebra(QQ, {('a','b'): {'c': 1, 'd':1}, ('a','c
↪'): {'b':1}})
sage: A = L.ideal([b, c, d])
sage: B = L.ideal([c+d])
sage: [B.reduce(v) for v in A.basis()]
[0, c, -c]
sage: A.basis()
Finite family {'b': b, 'c': c, 'd': d}
sage: B.basis()
Finite family {'b': b, 'd': c + d}
sage: all(B.reduce(v).parent() is A for v in A.basis())
True
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, {('a','b'): {'c': Integer(1), 'd':Integer(1)}, (
↪'a','c'): {'b':Integer(1)}}, names=('a', 'b', 'c', 'd',)); (a, b, c,␣
↪d,) = L._first_ngens(4)
>>> A = L.ideal([b, c, d])
>>> B = L.ideal([c+d])
>>> [B.reduce(v) for v in A.basis()]
[0, c, -c]
>>> A.basis()
Finite family {'b': b, 'c': c, 'd': d}
>>> B.basis()
Finite family {'b': b, 'd': c + d}
>>> all(B.reduce(v).parent() is A for v in A.basis())
True
```

example (*n=3*)

Return an example of a finite dimensional Lie algebra with basis as per *Category.example*.

EXAMPLES:

```
sage: C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
sage: C.example()                                                          #␣
↪needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 3-dimensional abelian Lie algebra over Rational Field
```

```
>>> from sage.all import *
>>> C = LieAlgebras(QQ).FiniteDimensional().WithBasis()
>>> C.example()                                                            #␣
↪needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 3-dimensional abelian Lie algebra over Rational Field
```

Other dimensions can be specified as an optional argument:

```
sage: C.example(5)                                                          #␣
↪needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 5-dimensional abelian Lie algebra over Rational Field
```

```
>>> from sage.all import *
>>> C.example(Integer(5))                                                   ␣
↪     # needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 5-dimensional abelian Lie algebra over Rational Field
```

4.58 Finite dimensional modules with basis

class sage.categories.finite_dimensional_modules_with_basis.**FiniteDimensionalModulesWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of finite dimensional modules with a distinguished basis.

EXAMPLES:

```
sage: C = FiniteDimensionalModulesWithBasis(ZZ); C
Category of finite dimensional modules with basis over Integer Ring
sage: sorted(C.super_categories(), key=str)
[Category of finite dimensional modules over Integer Ring,
 Category of modules with basis over Integer Ring]
sage: C is Modules(ZZ).WithBasis().FiniteDimensional()
True
```

```
>>> from sage.all import *
>>> C = FiniteDimensionalModulesWithBasis(ZZ); C
Category of finite dimensional modules with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of finite dimensional modules over Integer Ring,
 Category of modules with basis over Integer Ring]
>>> C is Modules(ZZ).WithBasis().FiniteDimensional()
True
```

class ElementMethods

Bases: `object`

dense_coefficient_list(*order=None*)

Return a list of *all* coefficients of `self`.

By default, this list is ordered in the same way as the indexing set of the basis of the parent of `self`.

INPUT:

- `order` – (optional) an ordering of the basis indexing set

EXAMPLES:

```
sage: # needs sage.modules
sage: v = vector([0, -1, -3])
sage: v.dense_coefficient_list()
[0, -1, -3]
sage: v.dense_coefficient_list([2,1,0])
[-3, -1, 0]
sage: sorted(v.coefficients())
[-3, -1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> v = vector([Integer(0), -Integer(1), -Integer(3)])
>>> v.dense_coefficient_list()
[0, -1, -3]
>>> v.dense_coefficient_list([Integer(2),Integer(1),Integer(0)])
[-3, -1, 0]
>>> sorted(v.coefficients())
[-3, -1]
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class Endset(*base_category*)

Bases: *CategoryWithAxiom*

class ElementMethods

Bases: object

characteristic_polynomial()

Return the characteristic polynomial of this endomorphism.

characteristic_polynomial() and *charpoly()* are the same method.

INPUT:

- var – variable

EXAMPLES:

```
sage: # needs sage.modules
sage: V = ZZ^2; phi = V.hom([V.0 + V.1, 2*V.1])
sage: phi.characteristic_polynomial()
x^2 - 3*x + 2
sage: phi.charpoly()
x^2 - 3*x + 2
sage: phi.matrix().charpoly()
x^2 - 3*x + 2
sage: phi.charpoly('T')
T^2 - 3*T + 2
sage: W = CombinatorialFreeModule(ZZ, ['x', 'y'])
sage: M = matrix(ZZ, [[1, 0], [1, 2]])
sage: psi = W.module_morphism(matrix=M, codomain=W)
sage: psi.charpoly()
x^2 - 3*x + 2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V = ZZ**Integer(2); phi = V.hom([V.gen(0) + V.gen(1),␣
↪Integer(2)*V.gen(1)])
>>> phi.characteristic_polynomial()
x^2 - 3*x + 2
>>> phi.charpoly()
x^2 - 3*x + 2
>>> phi.matrix().charpoly()
x^2 - 3*x + 2
>>> phi.charpoly('T')
T^2 - 3*T + 2
>>> W = CombinatorialFreeModule(ZZ, ['x', 'y'])
>>> M = matrix(ZZ, [[Integer(1), Integer(0)], [Integer(1),␣
↪Integer(2)]])
>>> psi = W.module_morphism(matrix=M, codomain=W)
>>> psi.charpoly()
x^2 - 3*x + 2
```

charpoly()

alias of *characteristic_polynomial()*.

det()

alias of *determinant()*.

determinant()

Return the determinant of this endomorphism.

determinant() and *det()* are the same method.

EXAMPLES:

```
sage: # needs sage.modules
sage: V = ZZ^2; phi = V.hom([V.0 + V.1, 2*V.1])
sage: phi.determinant()
2
sage: phi.det()
2
sage: W = CombinatorialFreeModule(ZZ, ['x', 'y'])
sage: M = matrix(ZZ, [[1, 0], [1, 2]])
sage: psi = W.module_morphism(matrix=M, codomain=W)
sage: psi.det()
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V = ZZ**Integer(2); phi = V.hom([V.gen(0) + V.gen(1),␣
↪Integer(2)*V.gen(1)])
>>> phi.determinant()
2
>>> phi.det()
2
>>> W = CombinatorialFreeModule(ZZ, ['x', 'y'])
>>> M = matrix(ZZ, [[Integer(1), Integer(0)], [Integer(1),␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(2)]])
>>> psi = W.module_morphism(matrix=M, codomain=W)
>>> psi.det()
2
```

fcp()

Return the factorization of the characteristic polynomial.

INPUT:

- `var` – variable

EXAMPLES:

```
sage: # needs sage.modules
sage: V = ZZ^2; phi = V.hom([V.0 + V.1, 2*V.1])
sage: phi.fcp()                                                             ␣
↪   # needs sage.libs.pari
(x - 2) * (x - 1)
sage: phi.fcp('T')                                                          ␣
↪   # needs sage.libs.pari
(T - 2) * (T - 1)
sage: W = CombinatorialFreeModule(ZZ, ['x', 'y'])
sage: M = matrix(ZZ, [[1, 0], [1, 2]])
sage: psi = W.module_morphism(matrix=M, codomain=W)
sage: psi.fcp()                                                             ␣
↪   # needs sage.libs.pari
(x - 2) * (x - 1)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V = ZZ**Integer(2); phi = V.hom([V.gen(0) + V.gen(1),␣
↪Integer(2)*V.gen(1)])
>>> phi.fcp()                                                               ␣
↪ # needs sage.libs.pari
(x - 2) * (x - 1)
>>> phi.fcp('T')                                                            ␣
↪ # needs sage.libs.pari
(T - 2) * (T - 1)
>>> W = CombinatorialFreeModule(ZZ, ['x', 'y'])
>>> M = matrix(ZZ, [[Integer(1), Integer(0)], [Integer(1),␣
↪Integer(2)]])
>>> psi = W.module_morphism(matrix=M, codomain=W)
>>> psi.fcp()                                                               ␣
↪ # needs sage.libs.pari
(x - 2) * (x - 1)
```

minimal_polynomial()

Return the minimal polynomial of this endomorphism.

minimal_polynomial() and *minpoly()* are the same method.

INPUT:

- `var` – string (default: `'x'`); a variable name

EXAMPLES:

Compute the minimal polynomial, and check it.

```
sage: # needs sage.modules
sage: V = GF(7)^3
sage: H = V.Hom(V)([[0,1,2], [-1,0,3], [2,4,1]]); H
Vector space morphism represented by the matrix:
[0 1 2]
[6 0 3]
[2 4 1]
Domain:   Vector space of dimension 3 over Finite Field of size 7
Codomain: Vector space of dimension 3 over Finite Field of size 7
sage: H.minpoly()
 ↪   # needs sage.libs.pari
x^3 + 6*x^2 + 6*x + 1
sage: H.minimal_polynomial()
 ↪   # needs sage.libs.pari
x^3 + 6*x^2 + 6*x + 1
sage: H^3 + (H^2)*6 + H*6 + 1
Vector space morphism represented by the matrix:
[0 0 0]
[0 0 0]
[0 0 0]
Domain:   Vector space of dimension 3 over Finite Field of size 7
Codomain: Vector space of dimension 3 over Finite Field of size 7

sage: # needs sage.modules sage.rings.finite_rings
sage: k = GF(9, 'c')
sage: V = CombinatorialFreeModule(k, ['x', 'y', 'z', 'w'])
sage: A = matrix(k, 4, [1,1,0,0, 0,1,0,0, 0,0,5,0, 0,0,0,5])
sage: phi = V.module_morphism(matrix=A, codomain=V)
sage: factor(phi.minpoly())
(x + 1) * (x + 2)^2
sage: A.minpoly()(A) == 0
True
sage: factor(phi.charpoly())
(x + 1)^2 * (x + 2)^2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V = GF(Integer(7))**Integer(3)
>>> H = V.Hom(V)([[Integer(0),Integer(1),Integer(2)], [-Integer(1),
↪Integer(0),Integer(3)], [Integer(2),Integer(4),Integer(1)]]); H
Vector space morphism represented by the matrix:
[0 1 2]
[6 0 3]
[2 4 1]
Domain:   Vector space of dimension 3 over Finite Field of size 7
Codomain: Vector space of dimension 3 over Finite Field of size 7
>>> H.minpoly()
 ↪ # needs sage.libs.pari
x^3 + 6*x^2 + 6*x + 1
>>> H.minimal_polynomial()
 ↪ # needs sage.libs.pari
```

(continues on next page)

(continued from previous page)

```
x^3 + 6*x^2 + 6*x + 1
>>> H**Integer(3) + (H**Integer(2))*Integer(6) + H*Integer(6) +␣
↪Integer(1)
Vector space morphism represented by the matrix:
[0 0 0]
[0 0 0]
[0 0 0]
Domain:   Vector space of dimension 3 over Finite Field of size 7
Codomain: Vector space of dimension 3 over Finite Field of size 7

>>> # needs sage.modules sage.rings.finite_rings
>>> k = GF(Integer(9), 'c')
>>> V = CombinatorialFreeModule(k, ['x', 'y', 'z', 'w'])
>>> A = matrix(k, Integer(4), [Integer(1),Integer(1),Integer(0),
↪Integer(0), Integer(0),Integer(1),Integer(0),Integer(0),␣
↪Integer(0),Integer(0),Integer(5),Integer(0), Integer(0),Integer(0),
↪Integer(0),Integer(5)])
>>> phi = V.module_morphism(matrix=A, codomain=V)
>>> factor(phi.minpoly())
(x + 1) * (x + 2)^2
>>> A.minpoly()(A) == Integer(0)
True
>>> factor(phi.charpoly())
(x + 1)^2 * (x + 2)^2
```

minpoly()

alias of *minimal_polynomial()*.

trace()

Return the trace of this endomorphism.

EXAMPLES:

```
sage: # needs sage.modules
sage: V = ZZ^2; phi = V.hom([V.0 + V.1, 2*V.1])
sage: phi.trace()
3
sage: W = CombinatorialFreeModule(ZZ, ['x', 'y'])
sage: M = matrix(ZZ, [[1, 0], [1, 2]])
sage: psi = W.module_morphism(matrix=M, codomain=W)
sage: psi.trace()
3
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> V = ZZ**Integer(2); phi = V.hom([V.gen(0) + V.gen(1),␣
↪Integer(2)*V.gen(1)])
>>> phi.trace()
3
>>> W = CombinatorialFreeModule(ZZ, ['x', 'y'])
>>> M = matrix(ZZ, [[Integer(1), Integer(0)], [Integer(1),␣
↪Integer(2)]])
```

(continues on next page)

(continued from previous page)

```
>>> psi = W.module_morphism(matrix=M, codomain=W)
>>> psi.trace()
3
```

class MorphismMethods

Bases: object

image()

Return the image of self as a submodule of the codomain.

EXAMPLES:

```
sage: SGA = SymmetricGroupAlgebra(QQ, 3)                                    #␣
↪needs sage.groups sage.modules
sage: f = SGA.module_morphism(lambda x: SGA(x**2), codomain=SGA)            #␣
↪needs sage.groups sage.modules
sage: f.image()                                                             #␣
↪needs sage.groups sage.modules
Free module generated by {0, 1, 2} over Rational Field
```

```
>>> from sage.all import *
>>> SGA = SymmetricGroupAlgebra(QQ, Integer(3))                              ␣
↪     # needs sage.groups sage.modules
>>> f = SGA.module_morphism(lambda x: SGA(x**Integer(2)), codomain=SGA)      ␣
↪     # needs sage.groups sage.modules
>>> f.image()                                                               #␣
↪needs sage.groups sage.modules
Free module generated by {0, 1, 2} over Rational Field
```

image_basis()

Return a basis for the image of self in echelon form.

EXAMPLES:

```
sage: SGA = SymmetricGroupAlgebra(QQ, 3)                                    #␣
↪needs sage.groups sage.modules
sage: f = SGA.module_morphism(lambda x: SGA(x**2), codomain=SGA)            #␣
↪needs sage.groups sage.modules
sage: f.image_basis()                                                       #␣
↪needs sage.groups sage.modules
([1, 2, 3], [2, 3, 1], [3, 1, 2])
```

```
>>> from sage.all import *
>>> SGA = SymmetricGroupAlgebra(QQ, Integer(3))                              ␣
↪     # needs sage.groups sage.modules
>>> f = SGA.module_morphism(lambda x: SGA(x**Integer(2)), codomain=SGA)      ␣
↪     # needs sage.groups sage.modules
>>> f.image_basis()                                                         #␣
↪needs sage.groups sage.modules
([1, 2, 3], [2, 3, 1], [3, 1, 2])
```

kernel()

Return the kernel of self as a submodule of the domain.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: SGA = SymmetricGroupAlgebra(QQ, 3)
sage: f = SGA.module_morphism(lambda x: SGA(x**2), codomain=SGA)
sage: K = f.kernel()
sage: K
Free module generated by {0, 1, 2} over Rational Field
sage: K.ambient()
Symmetric group algebra of order 3 over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> SGA = SymmetricGroupAlgebra(QQ, Integer(3))
>>> f = SGA.module_morphism(lambda x: SGA(x**Integer(2)), codomain=SGA)
>>> K = f.kernel()
>>> K
Free module generated by {0, 1, 2} over Rational Field
>>> K.ambient()
Symmetric group algebra of order 3 over Rational Field
```

kernel_basis()

Return a basis of the kernel of `self` in echelon form.

EXAMPLES:

```
sage: SGA = SymmetricGroupAlgebra(QQ, 3)                                    #␣
↪needs sage.groups sage.modules
sage: f = SGA.module_morphism(lambda x: SGA(x**2), codomain=SGA)            #␣
↪needs sage.groups sage.modules
sage: f.kernel_basis()                                                      #␣
↪needs sage.groups sage.modules
([1, 2, 3] - [3, 2, 1], [1, 3, 2] - [3, 2, 1], [2, 1, 3] - [3, 2, 1])
```

```
>>> from sage.all import *
>>> SGA = SymmetricGroupAlgebra(QQ, Integer(3))                             ␣
↪     # needs sage.groups sage.modules
>>> f = SGA.module_morphism(lambda x: SGA(x**Integer(2)), codomain=SGA)     ␣
↪     # needs sage.groups sage.modules
>>> f.kernel_basis()                                                      #␣
↪needs sage.groups sage.modules
([1, 2, 3] - [3, 2, 1], [1, 3, 2] - [3, 2, 1], [2, 1, 3] - [3, 2, 1])
```

matrix(*base_ring=None*, *side='left'*)

Return the matrix of this morphism in the distinguished bases of the domain and codomain.

INPUT:

- `base_ring` – a ring (default: `None`, meaning the base ring of the codomain)
- `side` – `'left'` or `'right'` (default: `'left'`)

If `side` is "left", this morphism is considered as acting on the left; i.e. each column of the matrix represents the image of an element of the basis of the domain.

The order of the rows and columns matches with the order in which the bases are enumerated.

See also

sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods.module_morphism()

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(ZZ, [1,2]); x = X.basis()
sage: Y = CombinatorialFreeModule(ZZ, [3,4]); y = Y.basis()
sage: phi = X.module_morphism(on_basis={1: y[3] + 3*y[4],
....:                                   2: 2*y[3] + 5*y[4]}.__getitem__,
....:                         codomain=Y)
sage: phi.matrix()
[1 2]
[3 5]
sage: phi.matrix(side='right')
[1 3]
[2 5]

sage: phi.matrix().parent()                                                 #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
sage: phi.matrix(QQ).parent()                                               #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2)]); x = X.
↪basis()
>>> Y = CombinatorialFreeModule(ZZ, [Integer(3),Integer(4)]); y = Y.
↪basis()
>>> phi = X.module_morphism(on_basis={Integer(1): y[Integer(3)] +␣
↪Integer(3)*y[Integer(4)],
...                                   Integer(2):␣
↪Integer(2)*y[Integer(3)] + Integer(5)*y[Integer(4)]}.__getitem__,
...                           codomain=Y)
>>> phi.matrix()
[1 2]
[3 5]
>>> phi.matrix(side='right')
[1 3]
[2 5]

>>> phi.matrix().parent()                                                 #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Integer Ring
>>> phi.matrix(QQ).parent()                                               #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

The resulting matrix is immutable:

```
sage: phi.matrix().is_mutable()                                                #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> phi.matrix().is_mutable()                                                  #␣
↪needs sage.modules
False
```

The zero morphism has a zero matrix:

```
sage: Hom(X, Y).zero().matrix()                                                #␣
↪needs sage.modules
[0 0]
[0 0]
```

```
>>> from sage.all import *
>>> Hom(X, Y).zero().matrix()                                                  #␣
↪needs sage.modules
[0 0]
[0 0]
```

Todo

Add support for morphisms where the codomain has a different base ring than the domain:

```
sage: Y = CombinatorialFreeModule(QQ, [3,4]); y = Y.basis()                    #␣
↪needs sage.modules
sage: phi = X.module_morphism(on_basis={1: y[3] + 3*y[4],                      #␣
↪needs sage.modules
....:                                   2: 2*y[3] + 5/2*y[4]}.__getitem_
↪_,
....:                           codomain=Y)
sage: phi.matrix().parent()          # not implemented                         #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> Y = CombinatorialFreeModule(QQ, [Integer(3),Integer(4)]); y = Y.
↪basis()          # needs sage.modules
>>> phi = X.module_morphism(on_basis={Integer(1): y[Integer(3)] +␣
↪Integer(3)*y[Integer(4)],            # needs sage.modules
...                                   Integer(2):␣
↪Integer(2)*y[Integer(3)] + Integer(5)/Integer(2)*y[Integer(4)]}.__
↪getitem__,
...                           codomain=Y)
>>> phi.matrix().parent()          # not implemented                          #␣
↪needs sage.modules
Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

This currently does not work because, in this case, the morphism is just in the category of commutative additive groups (i.e. the intersection of the categories of modules over $\mathbf{Z}$ and over $\mathbf{Q}$):

```
sage: phi.parent().homset_category()                                    #␣
↪needs sage.modules
Category of commutative additive semigroups
sage: phi.parent().homset_category()        # not implemented, needs␣
↪sage.modules
Category of finite dimensional modules with basis over Integer Ring
```

```
>>> from sage.all import *
>>> phi.parent().homset_category()                                      #␣
↪needs sage.modules
Category of commutative additive semigroups
>>> phi.parent().homset_category()        # not implemented, needs sage.
↪modules
Category of finite dimensional modules with basis over Integer Ring
```

class ParentMethods

Bases: object

annihilator(*S*, *action=<built-in function mul>*, *side='right'*, *category=None*)

Return the annihilator of a finite set.

INPUT:

- `S` – a finite set
- `action` – a function (default: `operator.mul`)
- `side` – `'left'` or `'right'` (default: `'right'`)
- `category` – a category

Assumptions:

- `action` takes elements of `self` as first argument and elements of `S` as second argument;
- The codomain is any vector space, and `action` is linear on its first argument; typically it is bilinear;
- If `side` is 'left', this is reversed.

OUTPUT:

The subspace of the elements x of `self` such that `action(x,s) = 0` for all $s \in S$. If `side` is 'left' replace the above equation by `action(s,x) = 0`.

If `self` is a ring, `action` an action of `self` on a module M and S is a subset of M, we recover the Wikipedia article Annihilator_%28ring_theory%29. Similarly this can be used to compute torsion or orthogonals.

See also

annihilator_basis() for lots of examples.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = FiniteDimensionalAlgebrasWithBasis(QQ).example(); F
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: x, y, a, b = F.basis()
sage: A = F.annihilator([a + 3*b + 2*y]); A
```

(continues on next page)

(continued from previous page)

```
Free module generated by {0} over Rational Field
sage: [b.lift() for b in A.basis()]
[-1/2*a - 3/2*b + x]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = FiniteDimensionalAlgebrasWithBasis(QQ).example(); F
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> x, y, a, b = F.basis()
>>> A = F.annihilator([a + Integer(3)*b + Integer(2)*y]); A
Free module generated by {0} over Rational Field
>>> [b.lift() for b in A.basis()]
[-1/2*a - 3/2*b + x]
```

The category can be used to specify other properties of this subspace, like that this is a subalgebra:

```
sage: # needs sage.modules
sage: center = F.annihilator(F.basis(), F.bracket,
....:                        category=Algebras(QQ).Subobjects())
sage: (e,) = center.basis()
sage: e.lift()
x + y
sage: e * e == e
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> center = F.annihilator(F.basis(), F.bracket,
...                        category=Algebras(QQ).Subobjects())
>>> (e,) = center.basis()
>>> e.lift()
x + y
>>> e * e == e
True
```

Taking annihilator is order reversing for inclusion:

```
sage: # needs sage.modules
sage: A   = F.annihilator([]);    A  .rename('A')
sage: Ax  = F.annihilator([x]);   Ax .rename('Ax')
sage: Ay  = F.annihilator([y]);   Ay .rename('Ay')
sage: Axy = F.annihilator([x,y]); Axy.rename('Axy')
sage: P = Poset(([A, Ax, Ay, Axy], attrcall("is_submodule")))           #␣
↪needs sage.graphs
sage: sorted(P.cover_relations(), key=str)                              #␣
↪needs sage.graphs
[[Ax, A], [Axy, Ax], [Axy, Ay], [Ay, A]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> A   = F.annihilator([]);    A  .rename('A')
>>> Ax  = F.annihilator([x]);   Ax .rename('Ax')
>>> Ay  = F.annihilator([y]);   Ay .rename('Ay')
>>> Axy = F.annihilator([x,y]); Axy.rename('Axy')
>>> P = Poset(([A, Ax, Ay, Axy], attrcall("is_submodule")))           #␣
↪needs sage.graphs
>>> sorted(P.cover_relations(), key=str)                              #␣
↪needs sage.graphs
[[Ax, A], [Axy, Ax], [Axy, Ay], [Ay, A]]
```

annihilator_basis(*S*, *action=<built-in function mul>*, *side='right'*)

Return a basis of the annihilator of a finite set of elements.

INPUT:

- `S` – a finite set of objects
- `action` – a function (default: `operator.mul`)
- `side` – `'left'` or `'right'` (default: `'right'`): on which side of `self` the elements of S acts

See *annihilator()* for the assumptions and definition of the annihilator.

EXAMPLES:

By default, the action is the standard $*$ operation. So our first example is about an algebra:

```
sage: # needs sage.graphs sage.modules
sage: F = FiniteDimensionalAlgebrasWithBasis(QQ).example(); F
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: x,y,a,b = F.basis()
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> F = FiniteDimensionalAlgebrasWithBasis(QQ).example(); F
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> x,y,a,b = F.basis()
```

In this algebra, multiplication on the right by x annihilates all basis elements but x:

```
sage: x*x, y*x, a*x, b*x                                               #␣
↪needs sage.graphs sage.modules
(x, 0, 0, 0)
```

```
>>> from sage.all import *
>>> x*x, y*x, a*x, b*x                                                 #␣
↪needs sage.graphs sage.modules
(x, 0, 0, 0)
```

So the annihilator is the subspace spanned by y, a, and b:

```
sage: F.annihilator_basis([x])                                         #␣
↪needs sage.graphs sage.modules
(y, a, b)
```

```
>>> from sage.all import *
>>> F.annihilator_basis([x])                                                #␣
↪needs sage.graphs sage.modules
(y, a, b)
```

The same holds for a and b:

```
sage: x*a, y*a, a*a, b*a                                                   #␣
↪needs sage.graphs sage.modules
(a, 0, 0, 0)
sage: F.annihilator_basis([a])                                             #␣
↪needs sage.graphs sage.modules
(y, a, b)
```

```
>>> from sage.all import *
>>> x*a, y*a, a*a, b*a                                                     #␣
↪needs sage.graphs sage.modules
(a, 0, 0, 0)
>>> F.annihilator_basis([a])                                               #␣
↪needs sage.graphs sage.modules
(y, a, b)
```

On the other hand, y annihilates only x:

```
sage: F.annihilator_basis([y])                                             #␣
↪needs sage.graphs sage.modules
(x,)
```

```
>>> from sage.all import *
>>> F.annihilator_basis([y])                                               #␣
↪needs sage.graphs sage.modules
(x,)
```

Here is a non trivial annihilator:

```
sage: F.annihilator_basis([a + 3*b + 2*y])                                 #␣
↪needs sage.graphs sage.modules
(-1/2*a - 3/2*b + x,)
```

```
>>> from sage.all import *
>>> F.annihilator_basis([a + Integer(3)*b + Integer(2)*y])                  ␣
↪              # needs sage.graphs sage.modules
(-1/2*a - 3/2*b + x,)
```

Let's check it:

```
sage: (-1/2*a - 3/2*b + x) * (a + 3*b + 2*y)                               #␣
↪needs sage.graphs sage.modules
0
```

```
>>> from sage.all import *
>>> (-Integer(1)/Integer(2)*a - Integer(3)/Integer(2)*b + x) * (a +␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3)*b + Integer(2)*y)                                    # needs sage.
↪graphs sage.modules
0
```

Doing the same calculations on the left exchanges the roles of x and y:

```
sage: # needs sage.graphs sage.modules
sage: F.annihilator_basis([y], side='left')
(x, a, b)
sage: F.annihilator_basis([a], side='left')
(x, a, b)
sage: F.annihilator_basis([b], side='left')
(x, a, b)
sage: F.annihilator_basis([x], side='left')
(y,)
sage: F.annihilator_basis([a + 3*b + 2*x], side='left')
(-1/2*a - 3/2*b + y,)
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> F.annihilator_basis([y], side='left')
(x, a, b)
>>> F.annihilator_basis([a], side='left')
(x, a, b)
>>> F.annihilator_basis([b], side='left')
(x, a, b)
>>> F.annihilator_basis([x], side='left')
(y,)
>>> F.annihilator_basis([a + Integer(3)*b + Integer(2)*x], side='left')
(-1/2*a - 3/2*b + y,)
```

By specifying an inner product, this method can be used to compute the orthogonal of a subspace:

```
sage: # needs sage.graphs sage.modules
sage: x,y,a,b = F.basis()
sage: def scalar(u, v):
....:     return vector([sum(u[i]*v[i] for i in F.basis().keys())])
sage: F.annihilator_basis([x + y, a + b], scalar)
(x - y, a - b)
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> x,y,a,b = F.basis()
>>> def scalar(u, v):
...     return vector([sum(u[i]*v[i] for i in F.basis().keys())])
>>> F.annihilator_basis([x + y, a + b], scalar)
(x - y, a - b)
```

By specifying the standard Lie bracket as action, one can compute the commutator of a subspace of F:

```
sage: F.annihilator_basis([a + b], action=F.bracket)                #␣
↪needs sage.graphs sage.modules
```

(continues on next page)

(continued from previous page)

```
(x + y, a, b)
```

```
>>> from sage.all import *
>>> F.annihilator_basis([a + b], action=F.bracket)                    #␣
↪needs sage.graphs sage.modules
(x + y, a, b)
```

In particular one can compute a basis of the center of the algebra. In our example, it is reduced to the identity:

```
sage: F.annihilator_basis(F.algebra_generators(), action=F.bracket)     #␣
↪needs sage.graphs sage.modules
(x + y,)
```

```
>>> from sage.all import *
>>> F.annihilator_basis(F.algebra_generators(), action=F.bracket)     #␣
↪needs sage.graphs sage.modules
(x + y,)
```

But see also *FiniteDimensionalAlgebrasWithBasis.ParentMethods.center_basis()*.

echelon_form(*elements*, *row_reduced=False*, *order=None*)

Return a basis in echelon form of the subspace spanned by a finite set of elements.

INPUT:

- `elements` – list or finite iterable of elements of `self`
- `row_reduced` – boolean (default: `False`); whether to compute the basis for the row reduced echelon form
- `order` – (optional) either something that can be converted into a tuple or a key function

OUTPUT:

A list of elements of `self` whose expressions as vectors form a matrix in echelon form. If `base_ring` is specified, then the calculation is achieved in this base ring.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, range(3), prefix='x')
sage: x = X.basis()
sage: V = X.echelon_form([x[0]-x[1], x[0]-x[2], x[1]-x[2]]); V
[x[0] - x[2], x[1] - x[2]]
sage: matrix(list(map(vector, V)))
[ 1  0 -1]
[ 0  1 -1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, range(Integer(3)), prefix='x')
>>> x = X.basis()
>>> V = X.echelon_form([x[Integer(0)]-x[Integer(1)], x[Integer(0)]-
↪x[Integer(2)], x[Integer(1)]-x[Integer(2)]]); V
[x[0] - x[2], x[1] - x[2]]
>>> matrix(list(map(vector, V)))
```

(continues on next page)

(continued from previous page)

```
[ 1  0 -1]
[ 0  1 -1]
```

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(ZZ, [1,2,3,4])
sage: B = F.basis()
sage: elements = [B[1]-17*B[2]+6*B[3], B[1]-17*B[2]+B[4]]
sage: F.echelon_form(elements)
[B[1] - 17*B[2] + B[4], 6*B[3] - B[4]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2),Integer(3),
...Integer(4)])
>>> B = F.basis()
>>> elements = [B[Integer(1)]-
...Integer(17)*B[Integer(2)]+Integer(6)*B[Integer(3)], B[Integer(1)]-
...Integer(17)*B[Integer(2)]+B[Integer(4)]]
>>> F.echelon_form(elements)
[B[1] - 17*B[2] + B[4], 6*B[3] - B[4]]
```

```
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])                    #␣
...needs sage.modules
sage: a,b,c = F.basis()                                                 #␣
...needs sage.modules
sage: F.echelon_form([8*a+b+10*c, -3*a+b-c, a-b-c])                     #␣
...needs sage.modules
[B['a'] + B['c'], B['b'] + 2*B['c']]
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])                      #␣
...needs sage.modules
>>> a,b,c = F.basis()                                                   #␣
...needs sage.modules
>>> F.echelon_form([Integer(8)*a+b+Integer(10)*c, -Integer(3)*a+b-c, a-b-
...c])                      # needs sage.modules
[B['a'] + B['c'], B['b'] + 2*B['c']]
```

```
sage: R.<x,y> = QQ[]
sage: C = CombinatorialFreeModule(R, range(3), prefix='x')              #␣
...needs sage.modules
sage: x = C.basis()                                                     #␣
...needs sage.modules
sage: C.echelon_form([x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]])       #␣
...needs sage.modules sage.rings.function_field
[x[0] - x[2], x[1] - x[2]]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> C = CombinatorialFreeModule(R, range(Integer(3)), prefix='x')       ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.modules
>>> x = C.basis()                                                    #␣
↪needs sage.modules
>>> C.echelon_form([x[Integer(0)] - x[Integer(1)],␣
↪Integer(2)*x[Integer(1)] - Integer(2)*x[Integer(2)], x[Integer(0)] -␣
↪x[Integer(2)]])       # needs sage.modules sage.rings.function_field
[x[0] - x[2], x[1] - x[2]]
```

```
sage: M = MatrixSpace(QQ, 3, 3)                                       #␣
↪needs sage.modules
sage: A = M([[0, 0, 2], [0, 0, 0], [0, 0, 0]])                        #␣
↪needs sage.modules
sage: M.echelon_form([A, A])                                          #␣
↪needs sage.modules
[
[0 0 1]
[0 0 0]
[0 0 0]
]
```

```
>>> from sage.all import *
>>> M = MatrixSpace(QQ, Integer(3), Integer(3))                       ␣
↪              # needs sage.modules
>>> A = M([[Integer(0), Integer(0), Integer(2)], [Integer(0), Integer(0),␣
↪Integer(0)], [Integer(0), Integer(0), Integer(0)]])                  ␣
↪     # needs sage.modules
>>> M.echelon_form([A, A])                                           #␣
↪needs sage.modules
[
[0 0 1]
[0 0 0]
[0 0 0]
]
```

from_vector(*vector*, *order=None*, *coerce=True*)

Build an element of `self` from a vector.

EXAMPLES:

```
sage: # needs sage.modules
sage: p_mult = matrix([[0,0,0], [0,0,-1], [0,0,0]])
sage: q_mult = matrix([[0,0,1], [0,0,0], [0,0,0]])
sage: A = algebras.FiniteDimensional(
....:         QQ, [p_mult, q_mult, matrix(QQ, 3, 3)], 'p,q,z')
sage: A.from_vector(vector([1,0,2]))
p + 2*z
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> p_mult = matrix([[Integer(0),Integer(0),Integer(0)], [Integer(0),
↪Integer(0),-Integer(1)], [Integer(0),Integer(0),Integer(0)]])
>>> q_mult = matrix([[Integer(0),Integer(0),Integer(1)], [Integer(0),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0),Integer(0)], [Integer(0),Integer(0),Integer(0)]])
>>> A = algebras.FiniteDimensional(
...          QQ, [p_mult, q_mult, matrix(QQ, Integer(3), Integer(3))], 'p,
↪q,z')
>>> A.from_vector(vector([Integer(1),Integer(0),Integer(2)]))
p + 2*z
```

gens()

Return the generators of `self`.

OUTPUT: a tuple containing the basis elements of `self`

EXAMPLES:

```
sage: F = CombinatorialFreeModule(ZZ, ['a', 'b', 'c'])                    #␣
↪needs sage.modules
sage: F.gens()                                                            #␣
↪needs sage.modules
(B['a'], B['b'], B['c'])
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(ZZ, ['a', 'b', 'c'])                     #␣
↪needs sage.modules
>>> F.gens()                                                             #␣
↪needs sage.modules
(B['a'], B['b'], B['c'])
```

invariant_module(*S*, *action=<built-in function mul>*, *action_on_basis=None*, *side='left'*, ***kwargs*)

Return the submodule of `self` invariant under the action of `S`.

For a semigroup S acting on a module M, the invariant submodule is given by

$$M^S = \{m \in M : s \cdot m = m, \forall s \in S\}.$$

INPUT:

- `S` – a finitely-generated semigroup
- `action` – a function (default: `operator.mul`)
- `side` – `'left'` or `'right'` (default: `'right'`); which side of `self` the elements of `S` acts
- `action_on_basis` – (optional) define the action of `S` on the basis of `self`

OUTPUT: `FiniteDimensionalInvariantModule`

EXAMPLES:

We build the invariant module of the permutation representation of the symmetric group:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: G = SymmetricGroup(3); G.rename('S3')
sage: M = FreeModule(ZZ, [1,2,3], prefix='M'); M.rename('M')
sage: action = lambda g, x: M.term(g(x))
sage: I = M.invariant_module(G, action_on_basis=action); I
(S3)-invariant submodule of M
sage: I.basis()
Finite family {0: B[0]}
sage: [I.lift(b) for b in I.basis()]
```

(continues on next page)

(continued from previous page)

```
[M[1] + M[2] + M[3]]
sage: G.rename(); M.rename()  # reset the names
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> G = SymmetricGroup(Integer(3)); G.rename('S3')
>>> M = FreeModule(ZZ, [Integer(1),Integer(2),Integer(3)], prefix='M'); M.
↪rename('M')
>>> action = lambda g, x: M.term(g(x))
>>> I = M.invariant_module(G, action_on_basis=action); I
(S3)-invariant submodule of M
>>> I.basis()
Finite family {0: B[0]}
>>> [I.lift(b) for b in I.basis()]
[M[1] + M[2] + M[3]]
>>> G.rename(); M.rename()  # reset the names
```

We can construct the invariant module of any module that has an action of `S`. In this example, we consider the dihedral group $G = D_4$ and the subgroup $H < G$ of all rotations. We construct the H-invariant module of the group algebra $\mathbf{Q}[G]$:

```
sage: # needs sage.groups
sage: G = groups.permutation.Dihedral(4)
sage: H = G.subgroup(G.gen(0))
sage: H
Subgroup generated by [(1,2,3,4)]
 of (Dihedral group of order 8 as a permutation group)
sage: H.cardinality()
4

sage: # needs sage.groups sage.modules
sage: A = G.algebra(QQ)
sage: I = A.invariant_module(H)
sage: [I.lift(b) for b in I.basis()]
[() + (1,2,3,4) + (1,3)(2,4) + (1,4,3,2),
 (2,4) + (1,2)(3,4) + (1,3) + (1,4)(2,3)]
sage: all(h * I.lift(b) == I.lift(b)
....:     for b in I.basis() for h in H)
True
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = groups.permutation.Dihedral(Integer(4))
>>> H = G.subgroup(G.gen(Integer(0)))
>>> H
Subgroup generated by [(1,2,3,4)]
 of (Dihedral group of order 8 as a permutation group)
>>> H.cardinality()
4

>>> # needs sage.groups sage.modules
>>> A = G.algebra(QQ)
```

(continues on next page)

(continued from previous page)

```
>>> I = A.invariant_module(H)
>>> [I.lift(b) for b in I.basis()]
[() + (1,2,3,4) + (1,3)(2,4) + (1,4,3,2),
 (2,4) + (1,2)(3,4) + (1,3) + (1,4)(2,3)]
>>> all(h * I.lift(b) == I.lift(b)
...      for b in I.basis() for h in H)
True
```

twisted_invariant_module(*G*, *chi*, *action=<built-in function mul>*, *action_on_basis=None*, *side='left'*, ***kwargs*)

Create the isotypic component of the action of `G` on `self` with irreducible character given by `chi`.

See also

-FiniteDimensionalTwistedInvariantModule

INPUT:

- `G` – a finitely-generated group
- `chi` – list/tuple of character values or an instance of `ClassFunction`
- `action` – a function (default: `operator.mul`)
- `action_on_basis` – (optional) define the action of `g` on the basis of `self`
- `side` – `'left'` or `'right'` (default: `'right'`); which side of `self` the elements of `S` acts

OUTPUT: `FiniteDimensionalTwistedInvariantModule`

EXAMPLES:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: M = CombinatorialFreeModule(QQ, [1,2,3])
sage: G = SymmetricGroup(3)
sage: def action(g, x): return(M.term(g(x)))  # permute coordinates
sage: T = M.twisted_invariant_module(G, [2,0,-1],
....:                                action_on_basis=action)
sage: import __main__; __main__.action = action
sage: TestSuite(T).run()
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> M = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3)])
>>> G = SymmetricGroup(Integer(3))
>>> def action(g, x): return(M.term(g(x)))  # permute coordinates
>>> T = M.twisted_invariant_module(G, [Integer(2),Integer(0),-Integer(1)],
...                                action_on_basis=action)
>>> import __main__; __main__.action = action
>>> TestSuite(T).run()
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

Implement the fact that a (finite) tensor product of finite dimensional modules is a finite dimensional module.

EXAMPLES:

```
sage: C = ModulesWithBasis(ZZ).FiniteDimensional().TensorProducts()
sage: C.extra_super_categories()
[Category of finite dimensional modules with basis over Integer Ring]
sage: C.FiniteDimensional()
Category of tensor products of
 finite dimensional modules with basis over Integer Ring
```

```
>>> from sage.all import *
>>> C = ModulesWithBasis(ZZ).FiniteDimensional().TensorProducts()
>>> C.extra_super_categories()
[Category of finite dimensional modules with basis over Integer Ring]
>>> C.FiniteDimensional()
Category of tensor products of
 finite dimensional modules with basis over Integer Ring
```

4.59 Finite Dimensional Nilpotent Lie Algebras With Basis

AUTHORS:

- Eero Hakavuori (2018-08-16): initial version

class sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis.**FiniteDimensionalNilpotent**

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional nilpotent Lie algebras with basis.

class ParentMethods

Bases: object

is_nilpotent()

Return True since self is nilpotent.

EXAMPLES:

```
sage: L = LieAlgebra(QQ, {('x','y'): {'z': 1}}, nilpotent=True)          #␣
↪needs sage.combinat sage.modules
sage: L.is_nilpotent()                                                    #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, {('x','y'): {'z': Integer(1)}}, nilpotent=True)   ␣
↪     # needs sage.combinat sage.modules
>>> L.is_nilpotent()                                                     #␣
↪needs sage.combinat sage.modules
True
```

lie_group(*name='G'*, ***kwds*)

Return the Lie group associated to self.

INPUT:

- name – string (default: 'G'); the name (symbol) given to the Lie group

EXAMPLES:

We define the Heisenberg group:

```
sage: L = lie_algebras.Heisenberg(QQ, 1)                                    #␣
↪needs sage.combinat sage.modules
sage: G = L.lie_group('G'); G                                               #␣
↪needs sage.combinat sage.modules sage.symbolic
Lie group G of Heisenberg algebra of rank 1 over Rational Field
```

```
>>> from sage.all import *
>>> L = lie_algebras.Heisenberg(QQ, Integer(1))                              ␣
↪     # needs sage.combinat sage.modules
>>> G = L.lie_group('G'); G                                                 #␣
↪needs sage.combinat sage.modules sage.symbolic
Lie group G of Heisenberg algebra of rank 1 over Rational Field
```

We test multiplying elements of the group:

```
sage: # needs sage.combinat sage.modules sage.symbolic
sage: p, q, z = L.basis()
sage: g = G.exp(p); g
exp(p1)
sage: h = G.exp(q); h
exp(q1)
sage: g * h
exp(p1 + q1 + 1/2*z)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules sage.symbolic
>>> p, q, z = L.basis()
>>> g = G.exp(p); g
exp(p1)
>>> h = G.exp(q); h
exp(q1)
>>> g * h
exp(p1 + q1 + 1/2*z)
```

We extend an element of the Lie algebra to a left-invariant vector field:

```
sage: X = G.left_invariant_extension(2*p + 3*q, name='X'); X                #␣
↪needs sage.combinat sage.modules sage.symbolic
Vector field X on the Lie group G of
 Heisenberg algebra of rank 1 over Rational Field
sage: X.at(G.one()).display()                                               #␣
↪needs sage.combinat sage.modules sage.symbolic
X = 2 ∂/∂x_0 + 3 ∂/∂x_1
sage: X.display()                                                           #␣
↪needs sage.combinat sage.modules sage.symbolic
X = 2 ∂/∂x_0 + 3 ∂/∂x_1 + (3/2*x_0 - x_1) ∂/∂x_2
```

```
>>> from sage.all import *
>>> X = G.left_invariant_extension(Integer(2)*p + Integer(3)*q, name='X');
```

(continues on next page)

(continued from previous page)

```
↪ X            # needs sage.combinat sage.modules sage.symbolic
Vector field X on the Lie group G of
 Heisenberg algebra of rank 1 over Rational Field
>>> X.at(G.one()).display()                                          #␣
↪needs sage.combinat sage.modules sage.symbolic
X = 2 ∂/∂x_0 + 3 ∂/∂x_1
>>> X.display()                                                      #␣
↪needs sage.combinat sage.modules sage.symbolic
X = 2 ∂/∂x_0 + 3 ∂/∂x_1 + (3/2*x_0 - x_1) ∂/∂x_2
```

See also

NilpotentLieGroup

step()

Return the nilpotency step of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, {('X','Y'): {'Z': 1}}, nilpotent=True)
sage: L.step()
2
sage: sc = {('X','Y'): {'Z': 1}, ('X','Z'): {'W': 1}}
sage: LieAlgebra(QQ, sc, nilpotent=True).step()
3
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, {('X','Y'): {'Z': Integer(1)}}, nilpotent=True)
>>> L.step()
2
>>> sc = {('X','Y'): {'Z': Integer(1)}, ('X','Z'): {'W': Integer(1)}}
>>> LieAlgebra(QQ, sc, nilpotent=True).step()
3
```

4.60 Finite dimensional semisimple algebras with basis

class sage.categories.finite_dimensional_semisimple_algebras_with_basis.**FiniteDimensionalSemisimpleAl**

Bases: *CategoryWithAxiom_over_base_ring*

The category of finite dimensional semisimple algebras with a distinguished basis.

EXAMPLES:

```
sage: from sage.categories.finite_dimensional_semisimple_algebras_with_basis␣
↪import FiniteDimensionalSemisimpleAlgebrasWithBasis
sage: C = FiniteDimensionalSemisimpleAlgebrasWithBasis(QQ); C
Category of finite dimensional semisimple algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.finite_dimensional_semisimple_algebras_with_basis import␣
↪FiniteDimensionalSemisimpleAlgebrasWithBasis
>>> C = FiniteDimensionalSemisimpleAlgebrasWithBasis(QQ); C
Category of finite dimensional semisimple algebras with basis over Rational Field
```

This category is best constructed as:

```
sage: D = Algebras(QQ).Semisimple().FiniteDimensional().WithBasis(); D
Category of finite dimensional semisimple algebras with basis over Rational Field
sage: D is C
True
```

```
>>> from sage.all import *
>>> D = Algebras(QQ).Semisimple().FiniteDimensional().WithBasis(); D
Category of finite dimensional semisimple algebras with basis over Rational Field
>>> D is C
True
```

class Commutative(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: object

central_orthogonal_idempotents()

Return the central orthogonal idempotents of this semisimple commutative algebra.

Those idempotents form a maximal decomposition of the identity into primitive orthogonal idempotents.

Todo

The implementation assumes that the algebra is split over its base field.

OUTPUT: list of orthogonal idempotents of `self`

EXAMPLES:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: A4 = SymmetricGroup(4).algebra(QQ)
sage: Z4 = A4.center()
sage: idempotents = Z4.central_orthogonal_idempotents()
sage: idempotents
(1/24*B[0] + 1/24*B[1] + 1/24*B[2] + 1/24*B[3] + 1/24*B[4],
 3/8*B[0] + 1/8*B[1] - 1/8*B[2] - 1/8*B[4],
 1/6*B[0] + 1/6*B[2] - 1/12*B[3],
 3/8*B[0] - 1/8*B[1] - 1/8*B[2] + 1/8*B[4],
 1/24*B[0] - 1/24*B[1] + 1/24*B[2] + 1/24*B[3] - 1/24*B[4])
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> A4 = SymmetricGroup(Integer(4)).algebra(QQ)
```

(continues on next page)

(continued from previous page)

```
>>> Z4 = A4.center()
>>> idempotents = Z4.central_orthogonal_idempotents()
>>> idempotents
(1/24*B[0] + 1/24*B[1] + 1/24*B[2] + 1/24*B[3] + 1/24*B[4],
 3/8*B[0] + 1/8*B[1] - 1/8*B[2] - 1/8*B[4],
 1/6*B[0] + 1/6*B[2] - 1/12*B[3],
 3/8*B[0] - 1/8*B[1] - 1/8*B[2] + 1/8*B[4],
 1/24*B[0] - 1/24*B[1] + 1/24*B[2] + 1/24*B[3] - 1/24*B[4])
```

Lifting those idempotents from the center, we recognize among them the sum and alternating sum of all permutations:

```
sage: [e.lift() for e in idempotents]                                    #␣
↪needs sage.combinat sage.groups sage.modules
[1/24*() + 1/24*(3,4) + 1/24*(2,3) + 1/24*(2,3,4) + 1/24*(2,4,3)
 + 1/24*(2,4) + 1/24*(1,2) + 1/24*(1,2)(3,4) + 1/24*(1,2,3)
 + 1/24*(1,2,3,4) + 1/24*(1,2,4,3) + 1/24*(1,2,4) + 1/24*(1,3,2)
 + 1/24*(1,3,4,2) + 1/24*(1,3) + 1/24*(1,3,4) + 1/24*(1,3)(2,4)
 + 1/24*(1,3,2,4) + 1/24*(1,4,3,2) + 1/24*(1,4,2) + 1/24*(1,4,3)
 + 1/24*(1,4) + 1/24*(1,4,2,3) + 1/24*(1,4)(2,3),
 ...,
 1/24*() - 1/24*(3,4) - 1/24*(2,3) + 1/24*(2,3,4) + 1/24*(2,4,3)
 - 1/24*(2,4) - 1/24*(1,2) + 1/24*(1,2)(3,4) + 1/24*(1,2,3)
 - 1/24*(1,2,3,4) - 1/24*(1,2,4,3) + 1/24*(1,2,4) + 1/24*(1,3,2)
 - 1/24*(1,3,4,2) - 1/24*(1,3) + 1/24*(1,3,4) + 1/24*(1,3)(2,4)
 - 1/24*(1,3,2,4) - 1/24*(1,4,3,2) + 1/24*(1,4,2) + 1/24*(1,4,3)
 - 1/24*(1,4) - 1/24*(1,4,2,3) + 1/24*(1,4)(2,3)]
```

```
>>> from sage.all import *
>>> [e.lift() for e in idempotents]                                      #␣
↪needs sage.combinat sage.groups sage.modules
[1/24*() + 1/24*(3,4) + 1/24*(2,3) + 1/24*(2,3,4) + 1/24*(2,4,3)
 + 1/24*(2,4) + 1/24*(1,2) + 1/24*(1,2)(3,4) + 1/24*(1,2,3)
 + 1/24*(1,2,3,4) + 1/24*(1,2,4,3) + 1/24*(1,2,4) + 1/24*(1,3,2)
 + 1/24*(1,3,4,2) + 1/24*(1,3) + 1/24*(1,3,4) + 1/24*(1,3)(2,4)
 + 1/24*(1,3,2,4) + 1/24*(1,4,3,2) + 1/24*(1,4,2) + 1/24*(1,4,3)
 + 1/24*(1,4) + 1/24*(1,4,2,3) + 1/24*(1,4)(2,3),
 ...,
 1/24*() - 1/24*(3,4) - 1/24*(2,3) + 1/24*(2,3,4) + 1/24*(2,4,3)
 - 1/24*(2,4) - 1/24*(1,2) + 1/24*(1,2)(3,4) + 1/24*(1,2,3)
 - 1/24*(1,2,3,4) - 1/24*(1,2,4,3) + 1/24*(1,2,4) + 1/24*(1,3,2)
 - 1/24*(1,3,4,2) - 1/24*(1,3) + 1/24*(1,3,4) + 1/24*(1,3)(2,4)
 - 1/24*(1,3,2,4) - 1/24*(1,4,3,2) + 1/24*(1,4,2) + 1/24*(1,4,3)
 - 1/24*(1,4) - 1/24*(1,4,2,3) + 1/24*(1,4)(2,3)]
```

We check that they indeed form a decomposition of the identity of Z_4 into orthogonal idempotents:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: Z4.is_identity_decomposition_into_orthogonal_
↪idempotents(idempotents)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> Z4.is_identity_decomposition_into_orthogonal_
↪idempotents(idempotents)
True
```

class ParentMethods

Bases: object

central_orthogonal_idempotents()

Return a maximal list of central orthogonal idempotents of self.

Todo

The implementation assumes that the algebra is split over its base field.

Central orthogonal idempotents of an algebra A are idempotents $(e_1, \ldots, e_n)$ in the center of A such that $e_i e_j = 0$ whenever $i \neq j$.

With the maximality condition, they sum up to 1 and are uniquely determined (up to order).

EXAMPLES:

For the algebra of the (abelian) alternating group A_3, we recover three idempotents corresponding to the three one-dimensional representations V_i on which $(1, 2, 3)$ acts on V_i as multiplication by the i-th power of a cube root of unity:

```
sage: # needs sage.groups sage.rings.number_field
sage: R = CyclotomicField(3)
sage: A3 = AlternatingGroup(3).algebra(R)
sage: idempotents = A3.central_orthogonal_idempotents()
sage: idempotents
(1/3*() + 1/3*(1,2,3) + 1/3*(1,3,2),
 1/3*() - (1/3*zeta3+1/3)*(1,2,3) - (-1/3*zeta3)*(1,3,2),
 1/3*() - (-1/3*zeta3)*(1,2,3) - (1/3*zeta3+1/3)*(1,3,2))
sage: A3.is_identity_decomposition_into_orthogonal_
↪idempotents(idempotents)
True
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.rings.number_field
>>> R = CyclotomicField(Integer(3))
>>> A3 = AlternatingGroup(Integer(3)).algebra(R)
>>> idempotents = A3.central_orthogonal_idempotents()
>>> idempotents
(1/3*() + 1/3*(1,2,3) + 1/3*(1,3,2),
 1/3*() - (1/3*zeta3+1/3)*(1,2,3) - (-1/3*zeta3)*(1,3,2),
 1/3*() - (-1/3*zeta3)*(1,2,3) - (1/3*zeta3+1/3)*(1,3,2))
>>> A3.is_identity_decomposition_into_orthogonal_idempotents(idempotents)
True
```

For the semisimple quotient of a quiver algebra, we recover the vertices of the quiver:

```
sage: # needs sage.graphs sage.modules sage.rings.number_field
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver (containing
the arrows a:x->y and b:x->y) over Rational Field
sage: Aquo = A.semisimple_quotient()
sage: Aquo.central_orthogonal_idempotents()
(B['x'], B['y'])
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules sage.rings.number_field
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver (containing
the arrows a:x->y and b:x->y) over Rational Field
>>> Aquo = A.semisimple_quotient()
>>> Aquo.central_orthogonal_idempotents()
(B['x'], B['y'])
```

radical_basis(***keywords*)

Return a basis of the Jacobson radical of this algebra.

- keywords – for compatibility; ignored

OUTPUT: the empty list since this algebra is semisimple

EXAMPLES:

```
sage: A = SymmetricGroup(4).algebra(QQ)                                    #␣
↪needs sage.combinat sage.groups sage.modules
sage: A.radical_basis()                                                    #␣
↪needs sage.combinat sage.groups sage.modules
()
```

```
>>> from sage.all import *
>>> A = SymmetricGroup(Integer(4)).algebra(QQ)                              ␣
↪     # needs sage.combinat sage.groups sage.modules
>>> A.radical_basis()                                                     #␣
↪needs sage.combinat sage.groups sage.modules
()
```

4.61 Finite Enumerated Sets

FiniteEnumeratedSets.ParentMethods.**_cardinality_from_iterator**(**ignored_args*, ***ignored_kwds*)

Return the cardinality of `self`.

This brute force implementation of *cardinality()* iterates through the elements of `self` to count them.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example(); C
An example of a finite enumerated set: {1,2,3}
sage: C._cardinality_from_iterator()
3
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example(); C
An example of a finite enumerated set: {1,2,3}
>>> C._cardinality_from_iterator()
3
```

FiniteEnumeratedSets.ParentMethods.**_test_enumerated_set_iter_cardinality**(***options*)

Check that the methods *cardinality()* and *__iter__()* are consistent. Also checks that *cardinality()* returns an `Integer`.

For efficiency reasons, those tests are not run if *cardinality()* is *_cardinality_from_iterator()*, or if `self` is too big.

See also

`TestSuite`.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C._test_enumerated_set_iter_cardinality()
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C._test_enumerated_set_iter_cardinality()
```

Let us now break the class:

```
sage: from sage.categories.examples.finite_enumerated_sets import Example
sage: class CCls(Example):
....:     def cardinality(self):
....:         return 4
sage: CC = CCls()
sage: CC._test_enumerated_set_iter_cardinality()
Traceback (most recent call last):
...
AssertionError: 4 != 3
```

```
>>> from sage.all import *
>>> from sage.categories.examples.finite_enumerated_sets import Example
>>> class CCls(Example):
...     def cardinality(self):
...         return Integer(4)
>>> CC = CCls()
>>> CC._test_enumerated_set_iter_cardinality()
Traceback (most recent call last):
...
AssertionError: 4 != 3
```

class sage.categories.finite_enumerated_sets.**FiniteEnumeratedSets**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite enumerated sets.

EXAMPLES:

```
sage: FiniteEnumeratedSets()
Category of finite enumerated sets
sage: FiniteEnumeratedSets().super_categories()
[Category of enumerated sets, Category of finite sets]
sage: FiniteEnumeratedSets().all_super_categories()
[Category of finite enumerated sets,
 Category of enumerated sets,
 Category of finite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> FiniteEnumeratedSets()
Category of finite enumerated sets
>>> FiniteEnumeratedSets().super_categories()
[Category of enumerated sets, Category of finite sets]
>>> FiniteEnumeratedSets().all_super_categories()
[Category of finite enumerated sets,
 Category of enumerated sets,
 Category of finite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

Todo

`sage.combinat.debruijn_sequence.DeBruijnSequences` should not inherit from this class. If that is solved, then *`FiniteEnumeratedSets`* shall be turned into a subclass of *`Category_singleton`*.

class CartesianProducts(*category*, **args*)

Bases: *`CartesianProductsCategory`*

class ParentMethods

Bases: `object`

cardinality()

Return the cardinality of `self`.

EXAMPLES:

```
sage: E = FiniteEnumeratedSet([1,2,3])
sage: C = cartesian_product([E, SymmetricGroup(4)])                    #␣
↪needs sage.groups
sage: C.cardinality()                                                  #␣
↪needs sage.groups
72

sage: E = FiniteEnumeratedSet([])
sage: C = cartesian_product([E, ZZ, QQ])
sage: C.cardinality()
```

(continues on next page)

(continued from previous page)

```
0

sage: C = cartesian_product([ZZ, QQ])
sage: C.cardinality()
+Infinity

sage: cartesian_product([GF(5), Permutations(10)]).cardinality()
18144000
sage: cartesian_product([GF(71)]*20).cardinality() == 71**20
True
```

```
>>> from sage.all import *
>>> E = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> C = cartesian_product([E, SymmetricGroup(Integer(4))])
↪    # needs sage.groups
>>> C.cardinality()                                                #␣
↪needs sage.groups
72

>>> E = FiniteEnumeratedSet([])
>>> C = cartesian_product([E, ZZ, QQ])
>>> C.cardinality()
0

>>> C = cartesian_product([ZZ, QQ])
>>> C.cardinality()
+Infinity

>>> cartesian_product([GF(Integer(5)), Permutations(Integer(10))]).
↪cardinality()
18144000
>>> cartesian_product([GF(Integer(71))]*Integer(20)).cardinality() ==␣
↪Integer(71)**Integer(20)
True
```

last()

Return the last element.

EXAMPLES:

```
sage: C = cartesian_product([Zmod(42), Partitions(10),              #␣
↪needs sage.combinat
....:                         IntegerRange(5)])
sage: C.last()                                                      #␣
↪needs sage.combinat
(41, [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], 4)
```

```
>>> from sage.all import *
>>> C = cartesian_product([Zmod(Integer(42)), Partitions(Integer(10)),␣
↪             # needs sage.combinat
...                           IntegerRange(Integer(5))])
>>> C.last()                                                        #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
(41, [1, 1, 1, 1, 1, 1, 1, 1, 1, 1], 4)
```

random_element(**args*)

Return a random element of this Cartesian product.

The extra arguments are passed down to each of the factors of the Cartesian product.

EXAMPLES:

```
sage: C = cartesian_product([Permutations(10)]*5)
sage: C.random_element()            # random
([2, 9, 4, 7, 1, 8, 6, 10, 5, 3],
 [8, 6, 5, 7, 1, 4, 9, 3, 10, 2],
 [5, 10, 3, 8, 2, 9, 1, 4, 7, 6],
 [9, 6, 10, 3, 2, 1, 5, 8, 7, 4],
 [8, 5, 2, 9, 10, 3, 7, 1, 4, 6])

sage: C = cartesian_product([ZZ]*10)
sage: c1 = C.random_element()
sage: c1                    # random
(3, 1, 4, 1, 1, -3, 0, -4, -17, 2)
sage: c2 = C.random_element(4,7)
sage: c2                    # random
(6, 5, 6, 4, 5, 6, 6, 4, 5, 5)
sage: all(4 <= i <= 7 for i in c2)
True
```

```
>>> from sage.all import *
>>> C = cartesian_product([Permutations(Integer(10))]*Integer(5))
>>> C.random_element()            # random
([2, 9, 4, 7, 1, 8, 6, 10, 5, 3],
 [8, 6, 5, 7, 1, 4, 9, 3, 10, 2],
 [5, 10, 3, 8, 2, 9, 1, 4, 7, 6],
 [9, 6, 10, 3, 2, 1, 5, 8, 7, 4],
 [8, 5, 2, 9, 10, 3, 7, 1, 4, 6])

>>> C = cartesian_product([ZZ]*Integer(10))
>>> c1 = C.random_element()
>>> c1                    # random
(3, 1, 4, 1, 1, -3, 0, -4, -17, 2)
>>> c2 = C.random_element(Integer(4),Integer(7))
>>> c2                    # random
(6, 5, 6, 4, 5, 6, 6, 4, 5, 5)
>>> all(Integer(4) <= i <= Integer(7) for i in c2)
True
```

rank(*x*)

Return the rank of an element of this Cartesian product.

The *rank* of `x` is its position in the enumeration. It is an integer between `0` and `n-1` where `n` is the cardinality of this set.

See also

- *EnumeratedSets.ParentMethods.rank()*
- *unrank()*

EXAMPLES:

```
sage: C = cartesian_product([GF(2), GF(11), GF(7)])
sage: C.rank(C((1,2,5)))
96
sage: C.rank(C((0,0,0)))
0

sage: for c in C: print(C.rank(c))
0
1
2
3
4
5
...
150
151
152
153

sage: # needs sage.combinat
sage: F1 = FiniteEnumeratedSet('abcdefgh')
sage: F2 = IntegerRange(250)
sage: F3 = Partitions(20)
sage: C = cartesian_product([F1, F2, F3])
sage: c = C(('a', 86, [7,5,4,4]))
sage: C.rank(c)
54213
sage: C.unrank(54213)
('a', 86, [7, 5, 4, 4])
```

```
>>> from sage.all import *
>>> C = cartesian_product([GF(Integer(2)), GF(Integer(11)),␣
↪GF(Integer(7))])
>>> C.rank(C((Integer(1),Integer(2),Integer(5))))
96
>>> C.rank(C((Integer(0),Integer(0),Integer(0))))
0

>>> for c in C: print(C.rank(c))
0
1
2
3
4
```

(continues on next page)

(continued from previous page)

```
5
...
150
151
152
153

>>> # needs sage.combinat
>>> F1 = FiniteEnumeratedSet('abcdefgh')
>>> F2 = IntegerRange(Integer(250))
>>> F3 = Partitions(Integer(20))
>>> C = cartesian_product([F1, F2, F3])
>>> c = C(('a', Integer(86), [Integer(7),Integer(5),Integer(4),
↪Integer(4)]))
>>> C.rank(c)
54213
>>> C.unrank(Integer(54213))
('a', 86, [7, 5, 4, 4])
```

unrank(i)

Return the i-th element of this Cartesian product.

INPUT:

- `i` – integer between 0 and $n-1$ where n is the cardinality of this set

See also

- *EnumeratedSets.ParentMethods.unrank()*
- *rank()*

EXAMPLES:

```
sage: C = cartesian_product([GF(3), GF(11), GF(7), GF(5)])
sage: c = C.unrank(123); c
(0, 3, 3, 3)
sage: C.rank(c)
123

sage: c = C.unrank(857); c
(2, 2, 3, 2)
sage: C.rank(c)
857

sage: C.unrank(2500)
Traceback (most recent call last):
...
IndexError: index i (=2) is greater than the cardinality
```

```
>>> from sage.all import *
>>> C = cartesian_product([GF(Integer(3)), GF(Integer(11)),␣
↪GF(Integer(7)), GF(Integer(5))])
```

(continues on next page)

(continued from previous page)

```
>>> c = C.unrank(Integer(123)); c
(0, 3, 3, 3)
>>> C.rank(c)
123

>>> c = C.unrank(Integer(857)); c
(2, 2, 3, 2)
>>> C.rank(c)
857

>>> C.unrank(Integer(2500))
Traceback (most recent call last):
...
IndexError: index i (=2) is greater than the cardinality
```

extra_super_categories()

A Cartesian product of finite enumerated sets is a finite enumerated set.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().CartesianProducts()
sage: C.extra_super_categories()
[Category of finite enumerated sets]
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().CartesianProducts()
>>> C.extra_super_categories()
[Category of finite enumerated sets]
```

class IsomorphicObjects(*category*, **args*)

Bases: *IsomorphicObjectsCategory*

class ParentMethods

Bases: object

cardinality()

Return the cardinality of `self` which is the same as that of the ambient set `self` is isomorphic to.

EXAMPLES:

```
sage: A = FiniteEnumeratedSets().IsomorphicObjects().example(); A
The image by some isomorphism of An example of a finite enumerated␣
↪set: {1,2,3}
sage: A.cardinality()
3
```

```
>>> from sage.all import *
>>> A = FiniteEnumeratedSets().IsomorphicObjects().example(); A
The image by some isomorphism of An example of a finite enumerated␣
↪set: {1,2,3}
>>> A.cardinality()
3
```

example()

Return an example of isomorphic object of a finite enumerated set, as per *Category.example*.

EXAMPLES:

```
sage: FiniteEnumeratedSets().IsomorphicObjects().example()
The image by some isomorphism of An example of a finite enumerated set:
↪{1,2,3}
```

```
>>> from sage.all import *
>>> FiniteEnumeratedSets().IsomorphicObjects().example()
The image by some isomorphism of An example of a finite enumerated set:
↪{1,2,3}
```

class ParentMethods

Bases: object

cardinality(**ignored_args*, ***ignored_kwds*)

Return the cardinality of self.

This brute force implementation of *cardinality()* iterates through the elements of self to count them.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example(); C
An example of a finite enumerated set: {1,2,3}
sage: C._cardinality_from_iterator()
3
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example(); C
An example of a finite enumerated set: {1,2,3}
>>> C._cardinality_from_iterator()
3
```

iterator_range(*start=None*, *stop=None*, *step=None*)

Iterate over the range of elements of self starting at start, ending at stop, and stepping by step.

See also

unrank(), unrank_range()

EXAMPLES:

```
sage: F = FiniteEnumeratedSet([1,2,3])
sage: list(F.iterator_range(1))
[2, 3]
sage: list(F.iterator_range(stop=2))
[1, 2]
sage: list(F.iterator_range(stop=2, step=2))
[1]
sage: list(F.iterator_range(start=1, step=2))
```

(continues on next page)

(continued from previous page)

```
[2]
sage: list(F.iterator_range(start=1, stop=2))
[2]
sage: list(F.iterator_range(start=0, stop=1))
[1]
sage: list(F.iterator_range(start=0, stop=3, step=2))
[1, 3]
sage: list(F.iterator_range(stop=-1))
[1, 2]

sage: F = FiniteEnumeratedSet([1,2,3,4])
sage: list(F.iterator_range(start=1, stop=3))
[2, 3]
sage: list(F.iterator_range(stop=10))
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> F = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> list(F.iterator_range(Integer(1)))
[2, 3]
>>> list(F.iterator_range(stop=Integer(2)))
[1, 2]
>>> list(F.iterator_range(stop=Integer(2), step=Integer(2)))
[1]
>>> list(F.iterator_range(start=Integer(1), step=Integer(2)))
[2]
>>> list(F.iterator_range(start=Integer(1), stop=Integer(2)))
[2]
>>> list(F.iterator_range(start=Integer(0), stop=Integer(1)))
[1]
>>> list(F.iterator_range(start=Integer(0), stop=Integer(3),␣
↪step=Integer(2)))
[1, 3]
>>> list(F.iterator_range(stop=-Integer(1)))
[1, 2]

>>> F = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3),Integer(4)])
>>> list(F.iterator_range(start=Integer(1), stop=Integer(3)))
[2, 3]
>>> list(F.iterator_range(stop=Integer(10)))
[1, 2, 3, 4]
```

last()

The last element of `self`.

`self.last()` returns the last element of `self`.

This is the default (brute force) implementation from the category `FiniteEnumeratedSet()` which can be used when the method `__iter__` is provided. Its complexity is $O(n)$ where n is the size of `self`.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.last()
3
sage: C._last_from_iterator()
3
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C.last()
3
>>> C._last_from_iterator()
3
```

random_element()

A random element in `self`.

`self.random_element()` returns a random element in `self` with uniform probability.

This is the default implementation from the category `EnumeratedSet()` which uses the method `unrank`.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: n = C.random_element()
sage: n in C
True

sage: n = C._random_element_from_unrank()
sage: n in C
True
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> n = C.random_element()
>>> n in C
True

>>> n = C._random_element_from_unrank()
>>> n in C
True
```

tuple()

Return a `tuple` of the elements of `self`.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.tuple()
(1, 2, 3)
sage: C.tuple() is C.tuple()
True
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C.tuple()
(1, 2, 3)
>>> C.tuple() is C.tuple()
True
```

unrank_range(*start=None*, *stop=None*, *step=None*)

Return the range of elements of `self` starting at `start`, ending at `stop`, and stepping by `step`.

See also `unrank()`.

EXAMPLES:

```
sage: F = FiniteEnumeratedSet([1,2,3])
sage: F.unrank_range(1)
[2, 3]
sage: F.unrank_range(stop=2)
[1, 2]
sage: F.unrank_range(stop=2, step=2)
[1]
sage: F.unrank_range(start=1, step=2)
[2]
sage: F.unrank_range(stop=-1)
[1, 2]

sage: F = FiniteEnumeratedSet([1,2,3,4])
sage: F.unrank_range(stop=10)
[1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> F = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> F.unrank_range(Integer(1))
[2, 3]
>>> F.unrank_range(stop=Integer(2))
[1, 2]
>>> F.unrank_range(stop=Integer(2), step=Integer(2))
[1]
>>> F.unrank_range(start=Integer(1), step=Integer(2))
[2]
>>> F.unrank_range(stop=-Integer(1))
[1, 2]

>>> F = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3),Integer(4)])
>>> F.unrank_range(stop=Integer(10))
[1, 2, 3, 4]
```

4.62 Finite fields

class sage.categories.finite_fields.**FiniteFields**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite fields.

EXAMPLES:

```
sage: K = FiniteFields(); K
Category of finite enumerated fields
```

```
>>> from sage.all import *
>>> K = FiniteFields(); K
Category of finite enumerated fields
```

A finite field is a finite monoid with the structure of a field; it is currently assumed to be enumerated:

```
sage: K.super_categories()
[Category of fields,
 Category of finite commutative rings,
 Category of finite enumerated sets]
```

```
>>> from sage.all import *
>>> K.super_categories()
[Category of fields,
 Category of finite commutative rings,
 Category of finite enumerated sets]
```

Some examples of membership testing and coercion:

```
sage: FiniteField(17) in K
True
sage: RationalField() in K
False
sage: K(RationalField())
Traceback (most recent call last):
...
TypeError: unable to canonically associate a finite field to Rational Field
```

```
>>> from sage.all import *
>>> FiniteField(Integer(17)) in K
True
>>> RationalField() in K
False
>>> K(RationalField())
Traceback (most recent call last):
...
TypeError: unable to canonically associate a finite field to Rational Field
```

class ElementMethods

Bases: `object`

is_square()

Test if the element is a square or has a square root element.

OUTPUT:

`True` if the element is a square `False` if not

EXAMPLES:

```
sage: S.<x> = GF(5)[]
sage: f = S.irreducible_element(20)
sage: k.<y> = S.quotient_ring(f)
sage: k in Fields()
True
sage: k(2).is_square()
True
sage: k.quadratic_nonresidue().is_square()
False
```

```
>>> from sage.all import *
>>> S = GF(Integer(5))['x']; (x,) = S._first_ngens(1)
>>> f = S.irreducible_element(Integer(20))
>>> k = S.quotient_ring(f, names=('y',)); (y,) = k._first_ngens(1)
>>> k in Fields()
True
>>> k(Integer(2)).is_square()
True
>>> k.quadratic_nonresidue().is_square()
False
```

sqrt (*all=False*, *algorithm='tonelli'*)

Return the square root of the element if it exists.

INPUT:

- `all` – boolean (default: `False`); whether to return a list of all square roots or just a square root
- `algorithm` – string (default: 'tonelli'); the algorithm to use among `'tonelli'`, `'cipolla'`. Tonelli is typically faster but has a worse worst-case complexity than Cipolla. In particular, if the field cardinality minus 1 is highly divisible by 2 and has a large odd factor then Cipolla may perform better.

OUTPUT:

- if `all=False`, a square root; raises an error if the element is not a square
- if `all=True`, a tuple of all distinct square roots. This tuple can have length 0, 1, or 2 depending on how many distinct square roots the element has.

EXAMPLES:

```
sage: S.<x> = GF(5)[]
sage: f = S.irreducible_element(20)
sage: k.<y> = S.quotient_ring(f)
sage: k in Fields()
True
sage: k(2).is_square()
True
sage: k(2).sqrt()^2 == k(2)
True
sage: my_sqrts = k(4).sqrt(all=True)
sage: len(k(4).sqrt(all=True))
2
sage: 2 in my_sqrts
True
sage: 3 in my_sqrts
True
sage: k.quadratic_nonresidue().sqrt()
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: element is not a square
sage: k.quadratic_nonresidue().sqrt(all=True)
()
```

```
>>> from sage.all import *
>>> S = GF(Integer(5))['x']; (x,) = S._first_ngens(1)
>>> f = S.irreducible_element(Integer(20))
>>> k = S.quotient_ring(f, names=('y',)); (y,) = k._first_ngens(1)
>>> k in Fields()
True
>>> k(Integer(2)).is_square()
True
>>> k(Integer(2)).sqrt()**Integer(2) == k(Integer(2))
True
>>> my_sqrts = k(Integer(4)).sqrt(all=True)
>>> len(k(Integer(4)).sqrt(all=True))
2
>>> Integer(2) in my_sqrts
True
>>> Integer(3) in my_sqrts
True
>>> k.quadratic_nonresidue().sqrt()
Traceback (most recent call last):
...
ValueError: element is not a square
>>> k.quadratic_nonresidue().sqrt(all=True)
()
```

Here is an example where changing the algorithm results in a faster square root:

```
sage: p = 141 * 2^141 + 1
sage: S.<x> = GF(p)[]
sage: f = S.irreducible_element(2)
sage: k.<y> = S.quotient_ring(f)
sage: k in Fields()
True
sage: k(2).sqrt(algorithm="cipolla")^2 == k(2)
True
```

```
>>> from sage.all import *
>>> p = Integer(141) * Integer(2)**Integer(141) + Integer(1)
>>> S = GF(p)['x']; (x,) = S._first_ngens(1)
>>> f = S.irreducible_element(Integer(2))
>>> k = S.quotient_ring(f, names=('y',)); (y,) = k._first_ngens(1)
>>> k in Fields()
True
>>> k(Integer(2)).sqrt(algorithm="cipolla")**Integer(2) == k(Integer(2))
True
```

ALGORITHM:

The algorithms used come from chapter 7 of [BS1996]. Let $q = p^n$ be the order of the finite field, let a be the finite field element that we wish to find the square root of.

- If $p = 2$ then a is always a square, and the square root of $\sqrt{a} = a^{q/2}$.
- If $q \equiv 3 \pmod 4$ then if a is a square $\sqrt{a} = a^{\frac{q+1}{4}}$
- For all other cases we use the algorithm given by the `algorithm` parameter.

square_root(*all=False*, *algorithm='tonelli'*)

alias of *sqrt()*.

class ParentMethods

Bases: `object`

is_perfect()

Return whether this field is perfect, i.e., every element has a p-th root. Always returns `True` since finite fields are perfect.

EXAMPLES:

```
sage: GF(2).is_perfect()
True
```

```
>>> from sage.all import *
>>> GF(Integer(2)).is_perfect()
True
```

quadratic_nonresidue()

Return a random non square element of the finite field

OUTPUT:

A non-square element of the finite field; raises an error if the finite field is of even order.

EXAMPLES:

```
sage: k = GF((3, 10))
sage: k.quadratic_nonresidue().is_square()
False
sage: k = GF((2, 10))
sage: k in Fields()  # to let k be a finite field
True
sage: k.quadratic_nonresidue()
Traceback (most recent call last):
...
ValueError: there are no non-squares in finite fields of even order
```

```
>>> from sage.all import *
>>> k = GF((Integer(3), Integer(10)))
>>> k.quadratic_nonresidue().is_square()
False
>>> k = GF((Integer(2), Integer(10)))
>>> k in Fields()  # to let k be a finite field
True
>>> k.quadratic_nonresidue()
Traceback (most recent call last):
...
ValueError: there are no non-squares in finite fields of even order
```

zeta(*n=None*)

Return an element of multiplicative order `n` in this finite field. If there is no such element, raise `ValueError`.

> **Warning**
>
> In general, this returns an arbitrary element of the correct order. There are no compatibility guarantees: `F.zeta(9)^3` may not be equal to `F.zeta(3)`.

EXAMPLES:

```
sage: k = GF(7)
sage: k.zeta()
3
sage: k.zeta().multiplicative_order()
6
sage: k.zeta(3)
2
sage: k.zeta(3).multiplicative_order()
3
sage: k = GF(49, 'a')
sage: k.zeta().multiplicative_order()
48
sage: k.zeta(6)
3
sage: k.zeta(5)
Traceback (most recent call last):
...
ValueError: no 5th root of unity in Finite Field in a of size 7^2
```

```
>>> from sage.all import *
>>> k = GF(Integer(7))
>>> k.zeta()
3
>>> k.zeta().multiplicative_order()
6
>>> k.zeta(Integer(3))
2
>>> k.zeta(Integer(3)).multiplicative_order()
3
>>> k = GF(Integer(49), 'a')
>>> k.zeta().multiplicative_order()
48
>>> k.zeta(Integer(6))
3
>>> k.zeta(Integer(5))
Traceback (most recent call last):
...
ValueError: no 5th root of unity in Finite Field in a of size 7^2
```

Even more examples:

```
sage: GF(9,'a').zeta_order()
8
sage: GF(9,'a').zeta()
a
sage: GF(9,'a').zeta(4)
a + 1
sage: GF(9,'a').zeta()^2
a + 1
```

```
>>> from sage.all import *
>>> GF(Integer(9),'a').zeta_order()
8
>>> GF(Integer(9),'a').zeta()
a
>>> GF(Integer(9),'a').zeta(Integer(4))
a + 1
>>> GF(Integer(9),'a').zeta()**Integer(2)
a + 1
```

This works even in very large finite fields, provided that `n` can be factored (see Issue #25203):

```
sage: k.<a> = GF(2^2000)
sage: p = 887794514874294500114604143902514703409869050359101317733635669
↪4165175273101819380 01
sage: z = k.zeta(p)
sage: z
a^1999 + a^1996 + a^1995 + a^1994 + ... + a^7 + a^5 + a^4 + 1
sage: z ^ p
1
```

```
>>> from sage.all import *
>>> k = GF(Integer(2)**Integer(2000), names=('a',)); (a,) = k._first_
↪ngens(1)
>>> p = Integer(887794514874294500114604143902514703409869050359101317733
↪6356944165175273101819380 01)
>>> z = k.zeta(p)
>>> z
a^1999 + a^1996 + a^1995 + a^1994 + ... + a^7 + a^5 + a^4 + 1
>>> z ** p
1
```

zeta_order()

Return the order of the distinguished root of unity in `self`.

EXAMPLES:

```
sage: GF(9,'a').zeta_order()
8
sage: GF(9,'a').zeta()
a
sage: GF(9,'a').zeta().multiplicative_order()
8
```

```
>>> from sage.all import *
>>> GF(Integer(9),'a').zeta_order()
8
>>> GF(Integer(9),'a').zeta()
a
>>> GF(Integer(9),'a').zeta().multiplicative_order()
8
```

extra_super_categories()

Any finite field is assumed to be endowed with an enumeration.

4.63 Finite groups

class sage.categories.finite_groups.**FiniteGroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite (multiplicative) groups.

EXAMPLES:

```
sage: C = FiniteGroups(); C
Category of finite groups
sage: C.super_categories()
[Category of finite monoids, Category of groups]
sage: C.example()
General Linear Group of degree 2 over Finite Field of size 3
```

```
>>> from sage.all import *
>>> C = FiniteGroups(); C
Category of finite groups
>>> C.super_categories()
[Category of finite monoids, Category of groups]
>>> C.example()
General Linear Group of degree 2 over Finite Field of size 3
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

extra_super_categories()

Implement Maschke's theorem.

In characteristic 0 all finite group algebras are semisimple.

EXAMPLES:

```
sage: FiniteGroups().Algebras(QQ).is_subcategory(Algebras(QQ).
↪Semisimple())
True
sage: FiniteGroups().Algebras(FiniteField(7)).is_
↪subcategory(Algebras(FiniteField(7)).Semisimple())
False
```

(continues on next page)

(continued from previous page)

```
sage: FiniteGroups().Algebras(ZZ).is_subcategory(Algebras(ZZ).
↪Semisimple())
False
sage: FiniteGroups().Algebras(Fields()).is_subcategory(Algebras(Fields()).
↪Semisimple())
False

sage: Cat = CommutativeAdditiveGroups().Finite()
sage: Cat.Algebras(QQ).is_subcategory(Algebras(QQ).Semisimple())
True
sage: Cat.Algebras(GF(7)).is_subcategory(Algebras(GF(7)).Semisimple())
False
sage: Cat.Algebras(ZZ).is_subcategory(Algebras(ZZ).Semisimple())
False
sage: Cat.Algebras(Fields()).is_subcategory(Algebras(Fields()).
↪Semisimple())
False
```

```
>>> from sage.all import *
>>> FiniteGroups().Algebras(QQ).is_subcategory(Algebras(QQ).Semisimple())
True
>>> FiniteGroups().Algebras(FiniteField(Integer(7))).is_
↪subcategory(Algebras(FiniteField(Integer(7))).Semisimple())
False
>>> FiniteGroups().Algebras(ZZ).is_subcategory(Algebras(ZZ).Semisimple())
False
>>> FiniteGroups().Algebras(Fields()).is_subcategory(Algebras(Fields()).
↪Semisimple())
False

>>> Cat = CommutativeAdditiveGroups().Finite()
>>> Cat.Algebras(QQ).is_subcategory(Algebras(QQ).Semisimple())
True
>>> Cat.Algebras(GF(Integer(7))).is_subcategory(Algebras(GF(Integer(7))).
↪Semisimple())
False
>>> Cat.Algebras(ZZ).is_subcategory(Algebras(ZZ).Semisimple())
False
>>> Cat.Algebras(Fields()).is_subcategory(Algebras(Fields()).Semisimple())
False
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

cardinality()

Return the cardinality of self, as per *cardinality()*.

This default implementation calls *order()* if available, and otherwise resorts to *_cardinality_from_iterator()*. This is for backward compatibility only. Finite groups should override this method instead of *order()*.

EXAMPLES:

We need to use a finite group which uses this default implementation of cardinality:

```
sage: G = groups.misc.SemimonomialTransformation(GF(5), 3); G           #␣
↪needs sage.rings.number_field
Semimonomial transformation group over Finite Field of size 5 of degree 3
sage: G.cardinality.__module__                                          #␣
↪needs sage.rings.number_field
'sage.categories.finite_groups'
sage: G.cardinality()                                                   #␣
↪needs sage.rings.number_field
384
```

```
>>> from sage.all import *
>>> G = groups.misc.SemimonomialTransformation(GF(Integer(5)),␣
↪Integer(3)); G           # needs sage.rings.number_field
Semimonomial transformation group over Finite Field of size 5 of degree 3
>>> G.cardinality.__module__                                            #␣
↪needs sage.rings.number_field
'sage.categories.finite_groups'
>>> G.cardinality()                                                     #␣
↪needs sage.rings.number_field
384
```

cayley_graph_disabled(*connecting_set=None*)

AUTHORS:

- Bobby Moretti (2007-08-10)
- Robert Miller (2008-05-01): editing

conjugacy_classes()

Return a list with all the conjugacy classes of the group.

This will eventually be a fall-back method for groups not defined over GAP. Right now, it just raises a `NotImplementedError`, until we include a non-GAP way of listing the conjugacy classes representatives.

EXAMPLES:

```
sage: from sage.groups.group import FiniteGroup
sage: G = FiniteGroup()
sage: G.conjugacy_classes()
Traceback (most recent call last):
...
NotImplementedError: Listing the conjugacy classes for group <sage.groups.
↪group.FiniteGroup object at ...> is not implemented
```

```
>>> from sage.all import *
>>> from sage.groups.group import FiniteGroup
>>> G = FiniteGroup()
>>> G.conjugacy_classes()
Traceback (most recent call last):
...
NotImplementedError: Listing the conjugacy classes for group <sage.groups.
↪group.FiniteGroup object at ...> is not implemented
```

conjugacy_classes_representatives()

Return a list of the conjugacy classes representatives of the group.

EXAMPLES:

```
sage: G = SymmetricGroup(3)
sage: G.conjugacy_classes_representatives()                                 #␣
↪needs sage.combinat
[(), (1,2), (1,2,3)]
```

```
>>> from sage.all import *
>>> G = SymmetricGroup(Integer(3))
>>> G.conjugacy_classes_representatives()                                 #␣
↪needs sage.combinat
[(), (1,2), (1,2,3)]
```

monoid_generators()

Return monoid generators for `self`.

For finite groups, the group generators are also monoid generators. Hence, this default implementation calls *group_generators()*.

EXAMPLES:

```
sage: A = AlternatingGroup(4)
sage: A.monoid_generators()
Family ((1,2,3), (2,3,4))
```

```
>>> from sage.all import *
>>> A = AlternatingGroup(Integer(4))
>>> A.monoid_generators()
Family ((1,2,3), (2,3,4))
```

semigroup_generators()

Return semigroup generators for `self`.

For finite groups, the group generators are also semigroup generators. Hence, this default implementation calls *group_generators()*.

EXAMPLES:

```
sage: A = AlternatingGroup(4)
sage: A.semigroup_generators()
Family ((1,2,3), (2,3,4))
```

```
>>> from sage.all import *
>>> A = AlternatingGroup(Integer(4))
>>> A.semigroup_generators()
Family ((1,2,3), (2,3,4))
```

some_elements()

Return some elements of `self`.

EXAMPLES:

```
sage: A = AlternatingGroup(4)
sage: A.some_elements()
Family ((1,2,3), (2,3,4))
```

```
>>> from sage.all import *
>>> A = AlternatingGroup(Integer(4))
>>> A.some_elements()
Family ((1,2,3), (2,3,4))
```

example()

Return an example of finite group, as per *Category.example()*.

EXAMPLES:

```
sage: G = FiniteGroups().example(); G
General Linear Group of degree 2 over Finite Field of size 3
```

```
>>> from sage.all import *
>>> G = FiniteGroups().example(); G
General Linear Group of degree 2 over Finite Field of size 3
```

4.64 Finite lattice posets

class sage.categories.finite_lattice_posets.**FiniteLatticePosets**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite lattices, i.e. finite partially ordered sets which are also lattices.

EXAMPLES:

```
sage: FiniteLatticePosets()
Category of finite lattice posets
sage: FiniteLatticePosets().super_categories()
[Category of lattice posets,
 Category of finite posets,
 Category of bounded posets]
sage: FiniteLatticePosets().example()
NotImplemented
```

```
>>> from sage.all import *
>>> FiniteLatticePosets()
Category of finite lattice posets
>>> FiniteLatticePosets().super_categories()
[Category of lattice posets,
 Category of finite posets,
 Category of bounded posets]
>>> FiniteLatticePosets().example()
NotImplemented
```

See also

FinitePosets, *LatticePosets*, FiniteLatticePoset

class ParentMethods

Bases: object

irreducibles_poset()

Return the poset of meet- or join-irreducibles of the lattice.

A *join-irreducible* element of a lattice is an element with exactly one lower cover. Dually a *meet-irreducible* element has exactly one upper cover.

This is the smallest poset with completion by cuts being isomorphic to the lattice. As a special case this returns one-element poset from one-element lattice.

See also

completion_by_cuts().

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: L = LatticePoset({1: [2, 3, 4], 2: [5, 6], 3: [5],
....:                   4: [6], 5: [9, 7], 6: [9, 8], 7: [10],
....:                   8: [10], 9: [10], 10: [11]})
sage: L_ = L.irreducibles_poset()
sage: sorted(L_)
[2, 3, 4, 7, 8, 9, 10, 11]
sage: L_.completion_by_cuts().is_isomorphic(L)
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> L = LatticePoset({Integer(1): [Integer(2), Integer(3), Integer(4)],␣
↪Integer(2): [Integer(5), Integer(6)], Integer(3): [Integer(5)],
...                   Integer(4): [Integer(6)], Integer(5): [Integer(9),␣
↪Integer(7)], Integer(6): [Integer(9), Integer(8)], Integer(7):␣
↪[Integer(10)],
...                   Integer(8): [Integer(10)], Integer(9):␣
↪[Integer(10)], Integer(10): [Integer(11)]})
>>> L_ = L.irreducibles_poset()
>>> sorted(L_)
[2, 3, 4, 7, 8, 9, 10, 11]
>>> L_.completion_by_cuts().is_isomorphic(L)
True
```

is_lattice_morphism(*f*, *codomain*)

Return whether f is a morphism of posets from self to codomain.

A map $f : P \to Q$ is a poset morphism if

$$x \leq y \Rightarrow f(x) \leq f(y)$$

for all $x, y \in P$.

INPUT:

- `f` – a function from `self` to `codomain`
- `codomain` – a lattice

EXAMPLES:

We build the boolean lattice of $\{2, 2, 3\}$ and the lattice of divisors of 60, and check that the map $b \mapsto 5 \prod_{x \in b} x$ is a morphism of lattices:

```
sage: D = LatticePoset((divisors(60), attrcall("divides")))                 #␣
↪needs sage.graphs sage.modules
sage: B = LatticePoset((Subsets([2,2,3]), attrcall("issubset")))            #␣
↪needs sage.graphs sage.modules
sage: def f(b): return D(5*prod(b))
sage: B.is_lattice_morphism(f, D)                                           #␣
↪needs sage.graphs sage.modules
True
```

```
>>> from sage.all import *
>>> D = LatticePoset((divisors(Integer(60)), attrcall("divides")))          ␣
↪     # needs sage.graphs sage.modules
>>> B = LatticePoset((Subsets([Integer(2),Integer(2),Integer(3)]),␣
↪attrcall("issubset")))        # needs sage.graphs sage.modules
>>> def f(b): return D(Integer(5)*prod(b))
>>> B.is_lattice_morphism(f, D)                                             #␣
↪needs sage.graphs sage.modules
True
```

We construct the boolean lattice B_2:

```
sage: B = posets.BooleanLattice(2)                                          #␣
↪needs sage.graphs
sage: B.cover_relations()                                                   #␣
↪needs sage.graphs
[[0, 1], [0, 2], [1, 3], [2, 3]]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(2))                                   ␣
↪     # needs sage.graphs
>>> B.cover_relations()                                                     #␣
↪needs sage.graphs
[[0, 1], [0, 2], [1, 3], [2, 3]]
```

And the same lattice with new top and bottom elements numbered respectively -1 and 3:

```
sage: G = DiGraph({-1:[0], 0:[1,2], 1:[3], 2:[3], 3:[4]})                  #␣
↪needs sage.graphs
sage: L = LatticePoset(G)                                                   #␣
↪needs sage.graphs sage.modules
sage: L.cover_relations()                                                   #␣
↪needs sage.graphs sage.modules
[[-1, 0], [0, 1], [0, 2], [1, 3], [2, 3], [3, 4]]
```

(continues on next page)

(continued from previous page)

```
sage: f = {B(0): L(0), B(1): L(1), B(2): L(2), B(3): L(3)}.__getitem__  #␣
↪needs sage.graphs sage.modules
sage: B.is_lattice_morphism(f, L)                                       #␣
↪needs sage.graphs sage.modules
True

sage: f = {B(0): L(-1),B(1): L(1), B(2): L(2), B(3): L(3)}.__getitem__  #␣
↪needs sage.graphs sage.modules
sage: B.is_lattice_morphism(f, L)                                       #␣
↪needs sage.graphs sage.modules
False

sage: f = {B(0): L(0), B(1): L(1), B(2): L(2), B(3): L(4)}.__getitem__  #␣
↪needs sage.graphs sage.modules
sage: B.is_lattice_morphism(f, L)                                       #␣
↪needs sage.graphs sage.modules
False
```

```
>>> from sage.all import *
>>> G = DiGraph({-Integer(1):[Integer(0)], Integer(0):[Integer(1),
↪Integer(2)], Integer(1):[Integer(3)], Integer(2):[Integer(3)],␣
↪Integer(3):[Integer(4)]})                # needs sage.graphs
>>> L = LatticePoset(G)                                                 #␣
↪needs sage.graphs sage.modules
>>> L.cover_relations()                                                 #␣
↪needs sage.graphs sage.modules
[[-1, 0], [0, 1], [0, 2], [1, 3], [2, 3], [3, 4]]

>>> f = {B(Integer(0)): L(Integer(0)), B(Integer(1)): L(Integer(1)),␣
↪B(Integer(2)): L(Integer(2)), B(Integer(3)): L(Integer(3))}.__getitem__␣
↪ # needs sage.graphs sage.modules
>>> B.is_lattice_morphism(f, L)                                         #␣
↪needs sage.graphs sage.modules
True

>>> f = {B(Integer(0)): L(-Integer(1)),B(Integer(1)): L(Integer(1)),␣
↪B(Integer(2)): L(Integer(2)), B(Integer(3)): L(Integer(3))}.__getitem__␣
↪ # needs sage.graphs sage.modules
>>> B.is_lattice_morphism(f, L)                                         #␣
↪needs sage.graphs sage.modules
False

>>> f = {B(Integer(0)): L(Integer(0)), B(Integer(1)): L(Integer(1)),␣
↪B(Integer(2)): L(Integer(2)), B(Integer(3)): L(Integer(4))}.__getitem__␣
↪ # needs sage.graphs sage.modules
>>> B.is_lattice_morphism(f, L)                                         #␣
↪needs sage.graphs sage.modules
False
```

See also

is_poset_morphism()

join_irreducibles()

Return the join-irreducible elements of this finite lattice.

A *join-irreducible element* of `self` is an element x that is not minimal and that can not be written as the join of two elements different from x.

EXAMPLES:

```
sage: L = LatticePoset({0:[1,2],1:[3],2:[3,4],3:[5],4:[5]})                #␣
↪needs sage.graphs sage.modules
sage: L.join_irreducibles()                                                #␣
↪needs sage.graphs sage.modules
[1, 2, 4]
```

```
>>> from sage.all import *
>>> L = LatticePoset({Integer(0):[Integer(1),Integer(2)],
↪Integer(1):[Integer(3)],Integer(2):[Integer(3),Integer(4)],
↪Integer(3):[Integer(5)],Integer(4):[Integer(5)]})              # needs␣
↪sage.graphs sage.modules
>>> L.join_irreducibles()                                                  #␣
↪needs sage.graphs sage.modules
[1, 2, 4]
```

See also

- Dual function: *meet_irreducibles()*
- Other: `double_irreducibles()`, *join_irreducibles_poset()*

join_irreducibles_poset()

Return the poset of join-irreducible elements of this finite lattice.

A *join-irreducible element* of `self` is an element x that is not minimal and can not be written as the join of two elements different from x.

EXAMPLES:

```
sage: L = LatticePoset({0:[1,2,3],1:[4],2:[4],3:[4]})                      #␣
↪needs sage.graphs sage.modules
sage: L.join_irreducibles_poset()                                          #␣
↪needs sage.graphs sage.modules
Finite poset containing 3 elements
```

```
>>> from sage.all import *
>>> L = LatticePoset({Integer(0):[Integer(1),Integer(2),Integer(3)],
↪Integer(1):[Integer(4)],Integer(2):[Integer(4)],Integer(3):[Integer(4)]}
↪)                    # needs sage.graphs sage.modules
>>> L.join_irreducibles_poset()                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.graphs sage.modules
Finite poset containing 3 elements
```

See also

- Dual function: *meet_irreducibles_poset()*
- Other: *join_irreducibles()*

meet_irreducibles()

Return the meet-irreducible elements of this finite lattice.

A *meet-irreducible element* of `self` is an element x that is not maximal and that can not be written as the meet of two elements different from x.

EXAMPLES:

```
sage: L = LatticePoset({0:[1,2],1:[3],2:[3,4],3:[5],4:[5]})                #␣
↪needs sage.graphs sage.modules
sage: L.meet_irreducibles()                                                #␣
↪needs sage.graphs sage.modules
[1, 3, 4]
```

```
>>> from sage.all import *
>>> L = LatticePoset({Integer(0):[Integer(1),Integer(2)],
↪Integer(1):[Integer(3)],Integer(2):[Integer(3),Integer(4)],
↪Integer(3):[Integer(5)],Integer(4):[Integer(5)]})                # needs␣
↪sage.graphs sage.modules
>>> L.meet_irreducibles()                                                  #␣
↪needs sage.graphs sage.modules
[1, 3, 4]
```

See also

- Dual function: *join_irreducibles()*
- Other: double_irreducibles(), *meet_irreducibles_poset()*

meet_irreducibles_poset()

Return the poset of join-irreducible elements of the lattice.

A *meet-irreducible element* of `self` is an element x that is not maximal and can not be written as the meet of two elements different from x.

EXAMPLES:

```
sage: L = LatticePoset({0:[1,2,3],1:[4],2:[4],3:[4]})                      #␣
↪needs sage.graphs sage.modules
sage: L.meet_irreducibles_poset()                                          #␣
↪needs sage.graphs sage.modules
Finite poset containing 3 elements
```

```
>>> from sage.all import *
>>> L = LatticePoset({Integer(0):[Integer(1),Integer(2),Integer(3)],
↪Integer(1):[Integer(4)],Integer(2):[Integer(4)],Integer(3):[Integer(4)]}
↪)                    # needs sage.graphs sage.modules
>>> L.meet_irreducibles_poset()                                          #␣
↪needs sage.graphs sage.modules
Finite poset containing 3 elements
```

See also

- Dual function: *join_irreducibles_poset()*
- Other: *meet_irreducibles()*

extra_super_categories()

Implement the fact that a finite lattice is a bounded poset.

EXAMPLES:

```
sage: FiniteLatticePosets().super_categories()
[Category of lattice posets,
 Category of finite posets,
 Category of bounded posets]
```

```
>>> from sage.all import *
>>> FiniteLatticePosets().super_categories()
[Category of lattice posets,
 Category of finite posets,
 Category of bounded posets]
```

4.65 Finite monoids

class sage.categories.finite_monoids.**FiniteMonoids**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite (multiplicative) *monoids*.

A finite monoid is a *finite sets* endowed with an associative unital binary operation $*$.

EXAMPLES:

```
sage: FiniteMonoids()
Category of finite monoids
sage: FiniteMonoids().super_categories()
[Category of monoids, Category of finite semigroups]
```

```
>>> from sage.all import *
>>> FiniteMonoids()
Category of finite monoids
>>> FiniteMonoids().super_categories()
[Category of monoids, Category of finite semigroups]
```

class ElementMethods

Bases: object

pseudo_order()

Return the pair $[k, j]$ with k minimal and $0 \leq j < k$ such that `self^k == self^j`.

Note that j is uniquely determined.

EXAMPLES:

```
sage: M = FiniteMonoids().example(); M
An example of a finite multiplicative monoid: the integers modulo 12

sage: x = M(2)
sage: [ x^i for i in range(7) ]
[1, 2, 4, 8, 4, 8, 4]
sage: x.pseudo_order()
[4, 2]

sage: x = M(3)
sage: [ x^i for i in range(7) ]
[1, 3, 9, 3, 9, 3, 9]
sage: x.pseudo_order()
[3, 1]

sage: x = M(4)
sage: [ x^i for i in range(7) ]
[1, 4, 4, 4, 4, 4, 4]
sage: x.pseudo_order()
[2, 1]

sage: x = M(5)
sage: [ x^i for i in range(7) ]
[1, 5, 1, 5, 1, 5, 1]
sage: x.pseudo_order()
[2, 0]
```

```
>>> from sage.all import *
>>> M = FiniteMonoids().example(); M
An example of a finite multiplicative monoid: the integers modulo 12

>>> x = M(Integer(2))
>>> [ x**i for i in range(Integer(7)) ]
[1, 2, 4, 8, 4, 8, 4]
>>> x.pseudo_order()
[4, 2]

>>> x = M(Integer(3))
>>> [ x**i for i in range(Integer(7)) ]
[1, 3, 9, 3, 9, 3, 9]
>>> x.pseudo_order()
[3, 1]

>>> x = M(Integer(4))
```

(continues on next page)

(continued from previous page)

```
>>> [ x**i for i in range(Integer(7)) ]
[1, 4, 4, 4, 4, 4, 4]
>>> x.pseudo_order()
[2, 1]

>>> x = M(Integer(5))
>>> [ x**i for i in range(Integer(7)) ]
[1, 5, 1, 5, 1, 5, 1]
>>> x.pseudo_order()
[2, 0]
```

Todo

more appropriate name? see, for example, Jean-Eric Pin's lecture notes on semigroups.

class ParentMethods

Bases: object

nerve()

The nerve (classifying space) of this monoid.

OUTPUT:

the nerve BG (if G denotes this monoid), as a simplicial set. The k-dimensional simplices of this object are indexed by products of k elements in the monoid:

$$a_1 * a_2 * \cdots * a_k$$

The 0-th face of this is obtained by deleting a_1, and the k-th face is obtained by deleting a_k. The other faces are obtained by multiplying elements: the 1st face is

$$(a1 * a_2) * \cdots * a_k$$

and so on. See Wikipedia article Nerve_(category_theory), which describes the construction of the nerve as a simplicial set.

A simplex in this simplicial set will be degenerate if in the corresponding product of k elements, one of those elements is the identity. So we only need to keep track of the products of non-identity elements. Similarly, if a product $a_{i-1}a_i$ is the identity element, then the corresponding face of the simplex will be a degenerate simplex.

EXAMPLES:

The nerve (classifying space) of the cyclic group of order 2 is infinite-dimensional real projective space.

```
sage: Sigma2 = groups.permutation.Cyclic(2)                                 #␣
↪needs sage.groups
sage: BSigma2 = Sigma2.nerve()                                              #␣
↪needs sage.graphs sage.groups
sage: BSigma2.cohomology(4, base_ring=GF(2))                                #␣
↪needs sage.graphs sage.groups sage.modules
Vector space of dimension 1 over Finite Field of size 2
```

```
>>> from sage.all import *
>>> Sigma2 = groups.permutation.Cyclic(Integer(2))                    ␣
↪     # needs sage.groups
>>> BSigma2 = Sigma2.nerve()                                          #␣
↪needs sage.graphs sage.groups
>>> BSigma2.cohomology(Integer(4), base_ring=GF(Integer(2)))          ␣
↪              # needs sage.graphs sage.groups sage.modules
Vector space of dimension 1 over Finite Field of size 2
```

The k-simplices of the nerve are named after the chains of k non-unit elements to be multiplied. The group Σ_2 has two elements, written `()` (the identity element) and `(1,2)` in Sage. So the 1-cells and 2-cells in $B\Sigma_2$ are:

```
sage: BSigma2.n_cells(1)                                              #␣
↪needs sage.graphs sage.groups
[(1,2)]
sage: BSigma2.n_cells(2)                                              #␣
↪needs sage.graphs sage.groups
[(1,2) * (1,2)]
```

```
>>> from sage.all import *
>>> BSigma2.n_cells(Integer(1))                                       ␣
↪     # needs sage.graphs sage.groups
[(1,2)]
>>> BSigma2.n_cells(Integer(2))                                       ␣
↪     # needs sage.graphs sage.groups
[(1,2) * (1,2)]
```

Another construction of the group, with different names for its elements:

```
sage: # needs sage.groups sage.rings.number_field
sage: C2 = groups.misc.MultiplicativeAbelian([2])
sage: BC2 = C2.nerve()
sage: BC2.n_cells(0)
[1]
sage: BC2.n_cells(1)
[f]
sage: BC2.n_cells(2)
[f * f]
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.rings.number_field
>>> C2 = groups.misc.MultiplicativeAbelian([Integer(2)])
>>> BC2 = C2.nerve()
>>> BC2.n_cells(Integer(0))
[1]
>>> BC2.n_cells(Integer(1))
[f]
>>> BC2.n_cells(Integer(2))
[f * f]
```

With mod p coefficients, $B\Sigma_p$ should have its first nonvanishing homology group in dimension p:

```
sage: Sigma3 = groups.permutation.Symmetric(3)                          #␣
↪needs sage.groups
sage: BSigma3 = Sigma3.nerve()                                          #␣
↪needs sage.graphs sage.groups
sage: BSigma3.homology(range(4), base_ring=GF(3))                       #␣
↪needs sage.graphs sage.groups
{0: Vector space of dimension 0 over Finite Field of size 3,
 1: Vector space of dimension 0 over Finite Field of size 3,
 2: Vector space of dimension 0 over Finite Field of size 3,
 3: Vector space of dimension 1 over Finite Field of size 3}
```

```
>>> from sage.all import *
>>> Sigma3 = groups.permutation.Symmetric(Integer(3))                   ␣
↪     # needs sage.groups
>>> BSigma3 = Sigma3.nerve()                                           #␣
↪needs sage.graphs sage.groups
>>> BSigma3.homology(range(Integer(4)), base_ring=GF(Integer(3)))       ␣
↪              # needs sage.graphs sage.groups
{0: Vector space of dimension 0 over Finite Field of size 3,
 1: Vector space of dimension 0 over Finite Field of size 3,
 2: Vector space of dimension 0 over Finite Field of size 3,
 3: Vector space of dimension 1 over Finite Field of size 3}
```

Note that we can construct the n-skeleton for $B\Sigma_2$ for relatively large values of n, while for $B\Sigma_3$, the complexes get large pretty quickly:

```
sage: # needs sage.graphs sage.groups
sage: Sigma2.nerve().n_skeleton(14)
Simplicial set with 15 non-degenerate simplices
sage: BSigma3 = Sigma3.nerve()
sage: BSigma3.n_skeleton(3)
Simplicial set with 156 non-degenerate simplices
sage: BSigma3.n_skeleton(4)
Simplicial set with 781 non-degenerate simplices
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> Sigma2.nerve().n_skeleton(Integer(14))
Simplicial set with 15 non-degenerate simplices
>>> BSigma3 = Sigma3.nerve()
>>> BSigma3.n_skeleton(Integer(3))
Simplicial set with 156 non-degenerate simplices
>>> BSigma3.n_skeleton(Integer(4))
Simplicial set with 781 non-degenerate simplices
```

Finally, note that the classifying space of the order p cyclic group is smaller than that of the symmetric group on p letters, and its first homology group appears earlier:

```
sage: # needs sage.graphs sage.groups sage.rings.number_field
sage: C3 = groups.misc.MultiplicativeAbelian([3])
sage: list(C3)
[1, f, f^2]
sage: BC3 = C3.nerve()
```

(continues on next page)

(continued from previous page)

```
sage: BC3.n_cells(1)
[f, f^2]
sage: BC3.n_cells(2)
[f * f, f * f^2, f^2 * f, f^2 * f^2]
sage: len(BSigma3.n_cells(2))
25
sage: len(BC3.n_cells(3))
8
sage: len(BSigma3.n_cells(3))
125
sage: BC3.homology(range(4), base_ring=GF(3))
{0: Vector space of dimension 0 over Finite Field of size 3,
 1: Vector space of dimension 1 over Finite Field of size 3,
 2: Vector space of dimension 1 over Finite Field of size 3,
 3: Vector space of dimension 1 over Finite Field of size 3}
sage: BC5 = groups.permutation.Cyclic(5).nerve()
sage: BC5.homology(range(4), base_ring=GF(5))
{0: Vector space of dimension 0 over Finite Field of size 5,
 1: Vector space of dimension 1 over Finite Field of size 5,
 2: Vector space of dimension 1 over Finite Field of size 5,
 3: Vector space of dimension 1 over Finite Field of size 5}
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups sage.rings.number_field
>>> C3 = groups.misc.MultiplicativeAbelian([Integer(3)])
>>> list(C3)
[1, f, f^2]
>>> BC3 = C3.nerve()
>>> BC3.n_cells(Integer(1))
[f, f^2]
>>> BC3.n_cells(Integer(2))
[f * f, f * f^2, f^2 * f, f^2 * f^2]
>>> len(BSigma3.n_cells(Integer(2)))
25
>>> len(BC3.n_cells(Integer(3)))
8
>>> len(BSigma3.n_cells(Integer(3)))
125
>>> BC3.homology(range(Integer(4)), base_ring=GF(Integer(3)))
{0: Vector space of dimension 0 over Finite Field of size 3,
 1: Vector space of dimension 1 over Finite Field of size 3,
 2: Vector space of dimension 1 over Finite Field of size 3,
 3: Vector space of dimension 1 over Finite Field of size 3}
>>> BC5 = groups.permutation.Cyclic(Integer(5)).nerve()
>>> BC5.homology(range(Integer(4)), base_ring=GF(Integer(5)))
{0: Vector space of dimension 0 over Finite Field of size 5,
 1: Vector space of dimension 1 over Finite Field of size 5,
 2: Vector space of dimension 1 over Finite Field of size 5,
 3: Vector space of dimension 1 over Finite Field of size 5}
```

rhodes_radical_congruence(*base_ring=None*)

Return the Rhodes radical congruence of the semigroup.

The Rhodes radical congruence is the congruence induced on S by the map $S \to kS \to kS/radkS$ with k a field.

INPUT:

- `base_ring` – (default: **Q**) a field

OUTPUT:

- A list of couples (m, n) with $m \neq n$ in the lexicographic order for the enumeration of the monoid `self`.

EXAMPLES:

```
sage: M = Monoids().Finite().example()
sage: M.rhodes_radical_congruence()                                    #␣
↪needs sage.modules
[(0, 6), (2, 8), (4, 10)]

sage: # needs sage.combinat sage.groups sage.modules
sage: from sage.monoids.hecke_monoid import HeckeMonoid
sage: H3 = HeckeMonoid(SymmetricGroup(3))
sage: H3.repr_element_method(style='reduced')
sage: H3.rhodes_radical_congruence()
[([1, 2], [2, 1]), ([1, 2], [1, 2, 1]), ([2, 1], [1, 2, 1])]
```

```
>>> from sage.all import *
>>> M = Monoids().Finite().example()
>>> M.rhodes_radical_congruence()                                      #␣
↪needs sage.modules
[(0, 6), (2, 8), (4, 10)]

>>> # needs sage.combinat sage.groups sage.modules
>>> from sage.monoids.hecke_monoid import HeckeMonoid
>>> H3 = HeckeMonoid(SymmetricGroup(Integer(3)))
>>> H3.repr_element_method(style='reduced')
>>> H3.rhodes_radical_congruence()
[([1, 2], [2, 1]), ([1, 2], [1, 2, 1]), ([2, 1], [1, 2, 1])]
```

By Maschke's theorem, every group algebra over **Q** is semisimple hence the Rhodes radical of a group must be trivial:

```
sage: SymmetricGroup(3).rhodes_radical_congruence()                    #␣
↪needs sage.combinat sage.groups sage.modules
[]
sage: DihedralGroup(10).rhodes_radical_congruence()                    #␣
↪needs sage.groups sage.modules
[]
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(3)).rhodes_radical_congruence()             ␣
↪     # needs sage.combinat sage.groups sage.modules
[]
>>> DihedralGroup(Integer(10)).rhodes_radical_congruence()             ␣
↪     # needs sage.groups sage.modules
[]
```

REFERENCES:

- [Rho69]

4.66 Finite Permutation Groups

class sage.categories.finite_permutation_groups.**FinitePermutationGroups**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite permutation groups, i.e. groups concretely represented as groups of permutations acting on a finite set.

It is currently assumed that any finite permutation group comes endowed with a distinguished finite set of generators (method `group_generators`); this is the case for all the existing implementations in Sage.

EXAMPLES:

```
sage: C = PermutationGroups().Finite(); C
Category of finite enumerated permutation groups
sage: C.super_categories()
[Category of permutation groups,
 Category of finite groups,
 Category of finite finitely generated semigroups]

sage: C.example()
Dihedral group of order 6 as a permutation group
```

```
>>> from sage.all import *
>>> C = PermutationGroups().Finite(); C
Category of finite enumerated permutation groups
>>> C.super_categories()
[Category of permutation groups,
 Category of finite groups,
 Category of finite finitely generated semigroups]

>>> C.example()
Dihedral group of order 6 as a permutation group
```

class ElementMethods

Bases: `object`

class ParentMethods

Bases: `object`

cycle_index(*parent=None*)

Return the *cycle index* of `self`.

INPUT:

- `self` – a permutation group G
- `parent` – a free module with basis indexed by partitions, or behave as such, with a `term` and `sum` method (default: the symmetric functions over the rational field in the p basis)

The *cycle index* of a permutation group G (Wikipedia article Cycle_index) is a gadget counting the elements of G by cycle type, averaged over the group:

$$P = \frac{1}{|G|} \sum_{g \in G} p_{\text{cycle type}(g)}$$

EXAMPLES:

Among the permutations of the symmetric group S_4, there is the identity, 6 cycles of length 2, 3 products of two cycles of length 2, 8 cycles of length 3, and 6 cycles of length 4:

```
sage: S4 = SymmetricGroup(4)
sage: P = S4.cycle_index()                                                  #␣
↪needs sage.combinat
sage: 24 * P                                                                #␣
↪needs sage.combinat
p[1, 1, 1, 1] + 6*p[2, 1, 1] + 3*p[2, 2] + 8*p[3, 1] + 6*p[4]
```

```
>>> from sage.all import *
>>> S4 = SymmetricGroup(Integer(4))
>>> P = S4.cycle_index()                                                    #␣
↪needs sage.combinat
>>> Integer(24) * P                                                         ␣
↪     # needs sage.combinat
p[1, 1, 1, 1] + 6*p[2, 1, 1] + 3*p[2, 2] + 8*p[3, 1] + 6*p[4]
```

If $l = (l_1, \ldots, l_k)$ is a partition, `|G| P[l]` is the number of elements of G with cycles of length $(p_1, \ldots, p_k)$:

```
sage: 24 * P[ Partition([3,1]) ]                                            #␣
↪needs sage.combinat
8
```

```
>>> from sage.all import *
>>> Integer(24) * P[ Partition([Integer(3),Integer(1)]) ]                   ␣
↪                       # needs sage.combinat
8
```

The cycle index plays an important role in the enumeration of objects modulo the action of a group (Pólya enumeration), via the use of symmetric functions and plethysms. It is therefore encoded as a symmetric function, expressed in the powersum basis:

```
sage: P.parent()                                                            #␣
↪needs sage.combinat
Symmetric Functions over Rational Field in the powersum basis
```

```
>>> from sage.all import *
>>> P.parent()                                                              #␣
↪needs sage.combinat
Symmetric Functions over Rational Field in the powersum basis
```

This symmetric function can have some nice properties; for example, for the symmetric group S_n, we get the complete symmetric function h_n:

```
sage: S = SymmetricFunctions(QQ); h = S.h()
sage: h( P )                                                                #␣
↪needs sage.combinat
h[4]
```

```
>>> from sage.all import *
>>> S = SymmetricFunctions(QQ); h = S.h()
```

(continues on next page)

(continued from previous page)

```
>>> h( P )                                                                    #␣
↪needs sage.combinat
h[4]
```

Todo

Add some simple examples of Pólya enumeration, once it will be easy to expand symmetric functions on any alphabet.

Here are the cycle indices of some permutation groups:

```
sage: 6 * CyclicPermutationGroup(6).cycle_index()                             #␣
↪needs sage.combinat
p[1, 1, 1, 1, 1, 1] + p[2, 2, 2] + 2*p[3, 3] + 2*p[6]

sage: 60 * AlternatingGroup(5).cycle_index()                                  #␣
↪needs sage.combinat
p[1, 1, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] + 24*p[5]

sage: for G in TransitiveGroups(5):                # long time
....:     G.cardinality() * G.cycle_index()
p[1, 1, 1, 1, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 5*p[2, 2, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 5*p[2, 2, 1] + 10*p[4, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] + 24*p[5]
p[1, 1, 1, 1, 1] + 10*p[2, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] +␣
↪20*p[3, 2] + 30*p[4, 1] + 24*p[5]
```

```
>>> from sage.all import *
>>> Integer(6) * CyclicPermutationGroup(Integer(6)).cycle_index()            ␣
↪               # needs sage.combinat
p[1, 1, 1, 1, 1, 1] + p[2, 2, 2] + 2*p[3, 3] + 2*p[6]

>>> Integer(60) * AlternatingGroup(Integer(5)).cycle_index()                 ␣
↪               # needs sage.combinat
p[1, 1, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] + 24*p[5]

>>> for G in TransitiveGroups(Integer(5)):                # long time
...     G.cardinality() * G.cycle_index()
p[1, 1, 1, 1, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 5*p[2, 2, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 5*p[2, 2, 1] + 10*p[4, 1] + 4*p[5]
p[1, 1, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] + 24*p[5]
p[1, 1, 1, 1, 1] + 10*p[2, 1, 1, 1] + 15*p[2, 2, 1] + 20*p[3, 1, 1] +␣
↪20*p[3, 2] + 30*p[4, 1] + 24*p[5]
```

Permutation groups with arbitrary domains are supported (see Issue #22765):

```
sage: G = PermutationGroup([['b','c','a']], domain=['a','b','c'])
sage: G.cycle_index()                                                         #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
1/3*p[1, 1, 1] + 2/3*p[3]
```

```
>>> from sage.all import *
>>> G = PermutationGroup([['b','c','a']], domain=['a','b','c'])
>>> G.cycle_index()                                                    #␣
↪needs sage.combinat
1/3*p[1, 1, 1] + 2/3*p[3]
```

One may specify another parent for the result:

```
sage: # needs sage.combinat
sage: F = CombinatorialFreeModule(QQ, Partitions())
sage: P = CyclicPermutationGroup(6).cycle_index(parent=F)
sage: 6 * P
B[[1, 1, 1, 1, 1, 1]] + B[[2, 2, 2]] + 2*B[[3, 3]] + 2*B[[6]]
sage: P.parent() is F
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> F = CombinatorialFreeModule(QQ, Partitions())
>>> P = CyclicPermutationGroup(Integer(6)).cycle_index(parent=F)
>>> Integer(6) * P
B[[1, 1, 1, 1, 1, 1]] + B[[2, 2, 2]] + 2*B[[3, 3]] + 2*B[[6]]
>>> P.parent() is F
True
```

This parent should be a module with basis indexed by partitions:

```
sage: CyclicPermutationGroup(6).cycle_index(parent = QQ)
Traceback (most recent call last):
  ...
ValueError: `parent` should be a module with basis indexed by partitions
```

```
>>> from sage.all import *
>>> CyclicPermutationGroup(Integer(6)).cycle_index(parent = QQ)
Traceback (most recent call last):
  ...
ValueError: `parent` should be a module with basis indexed by partitions
```

REFERENCES:

- [Ke1991]

AUTHORS:

- Nicolas Borie and Nicolas M. Thiéry

profile(*n*, *using_polya=True*)

Return the value in n of the profile of the group `self`.

Optional argument `using_polya` allows to change the default method.

INPUT:

- `n` – nonnegative integer

- `using_polya` – boolean (default: `True`); if `True`, the computation uses Pólya enumeration (and all values of the profile are cached, so this should be the method used in case several of them are needed); if `False`, uses the GAP interface to compute the orbit.

OUTPUT:

- A nonnegative integer that is the number of orbits of n-subsets under the action induced by `self` on the subsets of its domain (i.e. the value of the profile of `self` in n)

See also

- *profile_series()*

EXAMPLES:

```
sage: C6 = CyclicPermutationGroup(6)
sage: C6.profile(2)                                                    #␣
↪needs sage.combinat
3
sage: C6.profile(3)                                                    #␣
↪needs sage.combinat
4
sage: D8 = DihedralGroup(8)
sage: D8.profile(4, using_polya=False)
8
```

```
>>> from sage.all import *
>>> C6 = CyclicPermutationGroup(Integer(6))
>>> C6.profile(Integer(2))                                             ␣
↪     # needs sage.combinat
3
>>> C6.profile(Integer(3))                                             ␣
↪     # needs sage.combinat
4
>>> D8 = DihedralGroup(Integer(8))
>>> D8.profile(Integer(4), using_polya=False)
8
```

profile_polynomial(*variable='z'*)

alias of *profile_series()*.

profile_series(*variable='z'*)

Return the (finite) generating series of the (finite) profile of the group.

The profile of a permutation group G is the counting function that maps each nonnegative integer n onto the number of orbits of the action induced by G on the n-subsets of its domain. If f is the profile of G, f(n) is thus the number of orbits of n-subsets of G.

INPUT:

- `variable` – a variable, or variable name as a string (default: $'z'$)

OUTPUT:

- A polynomial in `variable` with nonnegative integer coefficients. By default, a polynomial in z over ZZ.

See also

- *profile()*

EXAMPLES:

```
sage: # needs sage.combinat
sage: C8 = CyclicPermutationGroup(8)
sage: C8.profile_series()
z^8 + z^7 + 4*z^6 + 7*z^5 + 10*z^4 + 7*z^3 + 4*z^2 + z + 1
sage: D8 = DihedralGroup(8)
sage: poly_D8 = D8.profile_series()
sage: poly_D8
z^8 + z^7 + 4*z^6 + 5*z^5 + 8*z^4 + 5*z^3 + 4*z^2 + z + 1
sage: poly_D8.parent()
Univariate Polynomial Ring in z over Rational Field
sage: D8.profile_series(variable='y')
y^8 + y^7 + 4*y^6 + 5*y^5 + 8*y^4 + 5*y^3 + 4*y^2 + y + 1
sage: u = var('u')
sage: D8.profile_series(u).parent()
Symbolic Ring
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> C8 = CyclicPermutationGroup(Integer(8))
>>> C8.profile_series()
z^8 + z^7 + 4*z^6 + 7*z^5 + 10*z^4 + 7*z^3 + 4*z^2 + z + 1
>>> D8 = DihedralGroup(Integer(8))
>>> poly_D8 = D8.profile_series()
>>> poly_D8
z^8 + z^7 + 4*z^6 + 5*z^5 + 8*z^4 + 5*z^3 + 4*z^2 + z + 1
>>> poly_D8.parent()
Univariate Polynomial Ring in z over Rational Field
>>> D8.profile_series(variable='y')
y^8 + y^7 + 4*y^6 + 5*y^5 + 8*y^4 + 5*y^3 + 4*y^2 + y + 1
>>> u = var('u')
>>> D8.profile_series(u).parent()
Symbolic Ring
```

example()

Return an example of finite permutation group, as per *Category.example()*.

EXAMPLES:

```
sage: G = FinitePermutationGroups().example(); G
Dihedral group of order 6 as a permutation group
```

```
>>> from sage.all import *
>>> G = FinitePermutationGroups().example(); G
Dihedral group of order 6 as a permutation group
```

extra_super_categories()

Any permutation group is assumed to be endowed with a finite set of generators.

4.67 Finite posets

Here is some terminology used in this file:

- An *order filter* (or *upper set*) of a poset P is a subset S of P such that if $x \leq y$ and $x \in S$ then $y \in S$.
- An *order ideal* (or *lower set*) of a poset P is a subset S of P such that if $x \leq y$ and $y \in S$ then $x \in S$.

class sage.categories.finite_posets.**FinitePosets**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite posets i.e. finite sets with a partial order structure.

EXAMPLES:

```
sage: FinitePosets()
Category of finite posets
sage: FinitePosets().super_categories()
[Category of posets, Category of finite sets]
sage: FinitePosets().example()
NotImplemented
```

```
>>> from sage.all import *
>>> FinitePosets()
Category of finite posets
>>> FinitePosets().super_categories()
[Category of posets, Category of finite sets]
>>> FinitePosets().example()
NotImplemented
```

See also

Posets, Poset()

class ParentMethods

Bases: object

antichains()

Return all antichains of self.

EXAMPLES:

```
sage: A = posets.PentagonPoset().antichains(); A                              #␣
↪needs sage.modules
Set of antichains of Finite lattice containing 5 elements
sage: list(A)                                                                 #␣
↪needs sage.modules
[[], [0], [1], [1, 2], [1, 3], [2], [3], [4]]
```

```
>>> from sage.all import *
>>> A = posets.PentagonPoset().antichains(); A                                #␣
↪needs sage.modules
Set of antichains of Finite lattice containing 5 elements
>>> list(A)                                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
[[], [0], [1], [1, 2], [1, 3], [2], [3], [4]]
```

birational_free_labelling(*linear_extension=None*, *prefix='x'*, *base_field=None*, *reduced=False*, *addvars=None*, *labels=None*, *min_label=None*, *max_label=None*)

Return the birational free labelling of `self`.

Let us hold back defining this, and introduce birational toggles and birational rowmotion first. These notions have been introduced in [EP2013] as generalizations of the notions of toggles (*order_ideal_toggle()*) and *rowmotion* on order ideals of a finite poset. They have been studied further in [GR2013].

Let $\mathbf{K}$ be a field, and P be a finite poset. Let $\widehat{P}$ denote the poset obtained from P by adding a new element 1 which is greater than all existing elements of P, and a new element 0 which is smaller than all existing elements of P and 1. Now, a $\mathbf{K}$-*labelling of* P will mean any function from $\widehat{P}$ to $\mathbf{K}$. The image of an element v of $\widehat{P}$ under this labelling will be called the *label* of this labelling at v. The set of all $\mathbf{K}$-labellings of P is clearly $\mathbf{K}^{\widehat{P}}$.

For any $v \in P$, we now define a rational map $T_v : \mathbf{K}^{\widehat{P}} \dashrightarrow \mathbf{K}^{\widehat{P}}$ as follows: For every $f \in \mathbf{K}^{\widehat{P}}$, the image $T_v f$ should send every element $u \in \widehat{P}$ distinct from v to $f(u)$ (so the labels at all $u \neq v$ don't change), while v is sent to

$$\frac{1}{f(v)} \cdot \frac{\sum_{u \lessdot v} f(u)}{\sum_{u \gtrdot v} \frac{1}{f(u)}}$$

(both sums are over all $u \in \widehat{P}$ satisfying the respectively given conditions). Here, $\lessdot$ and $\gtrdot$ mean (respectively) "covered by" and "covers", interpreted with respect to the poset $\widehat{P}$. This rational map T_v is an involution and is called the *(birational) v-toggle*; see *birational_toggle()* for its implementation.

Now, *birational rowmotion* is defined as the composition $T_{v_1} \circ T_{v_2} \circ \cdots \circ T_{v_n}$, where $(v_1, v_2, \ldots, v_n)$ is a linear extension of P (written as a linear ordering of the elements of P). This is a rational map $\mathbf{K}^{\widehat{P}} \dashrightarrow \mathbf{K}^{\widehat{P}}$ which does not depend on the choice of the linear extension; it is denoted by R. See *birational_rowmotion()* for its implementation.

The definitions of birational toggles and birational rowmotion extend to the case of $\mathbf{K}$ being any semifield rather than necessarily a field (although it becomes less clear what constitutes a rational map in this generality). The most useful case is that of the `tropical semiring`, in which case birational rowmotion relates to classical constructions such as promotion of rectangular semistandard Young tableaux (page 5 of [EP2013b] and future work, via the related notion of birational *promotion*) and rowmotion on order ideals of the poset ([EP2013]).

The *birational free labelling* is a special labelling defined for every finite poset P and every linear extension $(v_1, v_2, \ldots, v_n)$ of P. It is given by sending every element v_i in P to x_i, sending the element 0 of $\widehat{P}$ to a, and sending the element 1 of $\widehat{P}$ to b, where the ground field $\mathbf{K}$ is the field of rational functions in $n+2$ indeterminates $a, x_1, x_2, \ldots, x_n, b$ over $\mathbb{Q}$.

In Sage, a labelling f of a poset P is encoded as a 4-tuple $(\mathbf{K}, d, u, v)$, where $\mathbf{K}$ is the ground field of the labelling (i. e., its target), d is the dictionary containing the values of f at the elements of P (the keys being the respective elements of P), u is the label of f at 0, and v is the label of f at 1.

Warning

The dictionary d is labelled by the elements of P. If P is a poset with `facade` option set to `False`, these might not be what they seem to be! (For instance, if `P == Poset({1: [2, 3]}, facade=False)`, then the value of d at 1 has to be accessed by `d[P(1)]`, not by `d[1]`.)

> **Warning**
>
> Dictionaries are mutable. They do compare correctly, but are not hashable and need to be cloned to avoid spooky action at a distance. Be careful!

INPUT:

- `linear_extension` – (default: the default linear extension of `self`) a linear extension of `self` (as a linear extension or as a list), or more generally a list of all elements of all elements of `self` each occurring exactly once
- `prefix` – (default: `'x'`) the prefix to name the indeterminates corresponding to the elements of `self` in the labelling (so, setting it to `'frog'` will result in these indeterminates being called `frog1, frog2, ..., frogn` rather than `x1, x2, ..., xn`).
- `base_field` – (default: `QQ`) the base field to be used instead of $\mathbf{Q}$ to define the rational function field over; this is not going to be the base field of the labelling, because the latter will have indeterminates adjoined!
- `reduced` – boolean (default: `False`); if set to `True`, the result will be the *reduced* birational free labelling, which differs from the regular one by having 0 and 1 both sent to 1 instead of a and b (the indeterminates a and b then also won't appear in the ground field)
- `addvars` – (default: `''`) a string containing names of extra variables to be adjoined to the ground field (these don't have an effect on the labels)
- `labels` – (default: `'x'`) either a function that takes an element of the poset and returns a name for the indeterminate corresponding to that element, or a string containing a comma-separated list of indeterminates that will be assigned to elements in the order of `linear_extension`. If the list contains more indeterminates than needed, the excess will be ignored. If it contains too few, then the needed indeterminates will be constructed from `prefix`.
- `min_label` – (default: `'a'`) a string to be used as the label for the element 0 of $\widehat{P}$
- `max_label` – (default: `'b'`) a string to be used as the label for the element 1 of $\widehat{P}$

OUTPUT:

The birational free labelling of the poset `self` and the linear extension `linear_extension`. Or, if `reduced` is set to `True`, the reduced birational free labelling.

EXAMPLES:

We construct the birational free labelling on a simple poset:

```
sage: P = Poset({1: [2, 3]})
sage: l = P.birational_free_labelling(); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(l[1].items())
[(1, x1), (2, x2), (3, x3)]

sage: l = P.birational_free_labelling(linear_extension=[1, 3, 2]); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(l[1].items())
[(1, x1), (2, x3), (3, x2)]
```

(continues on next page)

(continued from previous page)

```
sage: l = P.birational_free_labelling(linear_extension=[1, 3, 2],
....:                                 reduced=True, addvars="spam, eggs");
↪ l
(Fraction Field of Multivariate Polynomial Ring
  in x1, x2, x3, spam, eggs over Rational Field,
 {...},
 1,
 1)
sage: sorted(l[1].items())
[(1, x1), (2, x3), (3, x2)]

sage: l = P.birational_free_labelling(linear_extension=[1, 3, 2],
....:                                 prefix='wut', reduced=True,
....:                                 addvars="spam, eggs"); l
(Fraction Field of Multivariate Polynomial Ring
  in wut1, wut2, wut3, spam, eggs over Rational Field,
 {...},
 1,
 1)
sage: sorted(l[1].items())
[(1, wut1), (2, wut3), (3, wut2)]

sage: l = P.birational_free_labelling(linear_extension=[1, 3, 2],
....:                                 reduced=False, addvars="spam, eggs
↪"); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b, spam, eggs over Rational Field,
 {...},
 a,
 b)
sage: sorted(l[1].items())
[(1, x1), (2, x3), (3, x2)]
sage: l[1][2]
x3
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(2), Integer(3)]})
>>> l = P.birational_free_labelling(); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
>>> sorted(l[Integer(1)].items())
[(1, x1), (2, x2), (3, x3)]

>>> l = P.birational_free_labelling(linear_extension=[Integer(1),␣
↪Integer(3), Integer(2)]); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
```

(continues on next page)

(continued from previous page)

```
 a,
 b)
>>> sorted(l[Integer(1)].items())
[(1, x1), (2, x3), (3, x2)]

>>> l = P.birational_free_labelling(linear_extension=[Integer(1),␣
↪Integer(3), Integer(2)],
...                                 reduced=True, addvars="spam, eggs"); l
(Fraction Field of Multivariate Polynomial Ring
  in x1, x2, x3, spam, eggs over Rational Field,
 {...},
 1,
 1)
>>> sorted(l[Integer(1)].items())
[(1, x1), (2, x3), (3, x2)]

>>> l = P.birational_free_labelling(linear_extension=[Integer(1),␣
↪Integer(3), Integer(2)],
...                                 prefix='wut', reduced=True,
...                                 addvars="spam, eggs"); l
(Fraction Field of Multivariate Polynomial Ring
  in wut1, wut2, wut3, spam, eggs over Rational Field,
 {...},
 1,
 1)
>>> sorted(l[Integer(1)].items())
[(1, wut1), (2, wut3), (3, wut2)]

>>> l = P.birational_free_labelling(linear_extension=[Integer(1),␣
↪Integer(3), Integer(2)],
...                                 reduced=False, addvars="spam, eggs");␣
↪l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b, spam, eggs over Rational Field,
 {...},
 a,
 b)
>>> sorted(l[Integer(1)].items())
[(1, x1), (2, x3), (3, x2)]
>>> l[Integer(1)][Integer(2)]
x3
```

Illustrating labelling with a function:

```
sage: P = posets.ChainPoset(2).product(posets.ChainPoset(2))              #␣
↪needs sage.modules
sage: def x_label(e):
....:     return 'x_' + str(e[0]) + str(e[1])
sage: l = P.birational_free_labelling(labels=x_label)
sage: sorted(l[1].items())
[((0, 0), x_00), ((0, 1), x_01), ((1, 0), x_10), ((1, 1), x_11)]
sage: l[2]
```

(continues on next page)

(continued from previous page)

```
a
```

```
>>> from sage.all import *
>>> P = posets.ChainPoset(Integer(2)).product(posets.
↪ChainPoset(Integer(2)))             # needs sage.modules
>>> def x_label(e):
...     return 'x_' + str(e[Integer(0)]) + str(e[Integer(1)])
>>> l = P.birational_free_labelling(labels=x_label)
>>> sorted(l[Integer(1)].items())
[((0, 0), x_00), ((0, 1), x_01), ((1, 0), x_10), ((1, 1), x_11)]
>>> l[Integer(2)]
a
```

The same, but with `min_label` and `max_label` provided:

```
sage: P = posets.ChainPoset(2).product(posets.ChainPoset(2))             #␣
↪needs sage.modules
sage: l = P.birational_free_labelling(labels=x_label,
....:                                 min_label='lambda', max_label='mu')
sage: sorted(l[1].items())
[((0, 0), x_00), ((0, 1), x_01), ((1, 0), x_10), ((1, 1), x_11)]
sage: l[2]
lambda
sage: l[3]
mu
```

```
>>> from sage.all import *
>>> P = posets.ChainPoset(Integer(2)).product(posets.
↪ChainPoset(Integer(2)))             # needs sage.modules
>>> l = P.birational_free_labelling(labels=x_label,
...                                 min_label='lambda', max_label='mu')
>>> sorted(l[Integer(1)].items())
[((0, 0), x_00), ((0, 1), x_01), ((1, 0), x_10), ((1, 1), x_11)]
>>> l[Integer(2)]
lambda
>>> l[Integer(3)]
mu
```

Illustrating labelling with a comma separated list of labels:

```
sage: l = P.birational_free_labelling(labels='w,x,y,z')
sage: sorted(l[1].items())
[((0, 0), w), ((0, 1), x), ((1, 0), y), ((1, 1), z)]
sage: l = P.birational_free_labelling(labels='w,x,y,z,m')
sage: sorted(l[1].items())
[((0, 0), w), ((0, 1), x), ((1, 0), y), ((1, 1), z)]
sage: l = P.birational_free_labelling(labels='w')
sage: sorted(l[1].items())
[((0, 0), w), ((0, 1), x1), ((1, 0), x2), ((1, 1), x3)]
```

```
>>> from sage.all import *
>>> l = P.birational_free_labelling(labels='w,x,y,z')
```

(continues on next page)

(continued from previous page)

```
>>> sorted(l[Integer(1)].items())
[((0, 0), w), ((0, 1), x), ((1, 0), y), ((1, 1), z)]
>>> l = P.birational_free_labelling(labels='w,x,y,z,m')
>>> sorted(l[Integer(1)].items())
[((0, 0), w), ((0, 1), x), ((1, 0), y), ((1, 1), z)]
>>> l = P.birational_free_labelling(labels='w')
>>> sorted(l[Integer(1)].items())
[((0, 0), w), ((0, 1), x1), ((1, 0), x2), ((1, 1), x3)]
```

Illustrating the warning about facade:

```
sage: P = Poset({1: [2, 3]}, facade=False)
sage: l = P.birational_free_labelling(linear_extension=[1, 3, 2],
....:                                 reduced=False,
....:                                 addvars="spam, eggs"); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b, spam, eggs over Rational Field,
 {...},
 a,
 b)
sage: l[1][2]
Traceback (most recent call last):
...
KeyError: 2
sage: l[1][P(2)]
x3
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(2), Integer(3)]}, facade=False)
>>> l = P.birational_free_labelling(linear_extension=[Integer(1),␣
↪Integer(3), Integer(2)],
...                                 reduced=False,
...                                 addvars="spam, eggs"); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b, spam, eggs over Rational Field,
 {...},
 a,
 b)
>>> l[Integer(1)][Integer(2)]
Traceback (most recent call last):
...
KeyError: 2
>>> l[Integer(1)][P(Integer(2))]
x3
```

Another poset:

```
sage: # needs sage.modules
sage: P = posets.SSTPoset([2,1])
sage: lext = sorted(P)
sage: l = P.birational_free_labelling(linear_extension=lext,
....:                                 addvars='ohai'); l
```

(continues on next page)

(continued from previous page)

```
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, x4, x5, x6, x7, x8, b, ohai over Rational Field,
 {...},
 a,
 b)
sage: sorted(l[1].items())
[([[1, 1], [2]], x1), ([[1, 1], [3]], x2),
 ([[1, 2], [2]], x3), ([[1, 2], [3]], x4),
 ([[1, 3], [2]], x5), ([[1, 3], [3]], x6),
 ([[2, 2], [3]], x7), ([[2, 3], [3]], x8)]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> P = posets.SSTPoset([Integer(2),Integer(1)])
>>> lext = sorted(P)
>>> l = P.birational_free_labelling(linear_extension=lext,
...                                 addvars='ohai'); l
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, x4, x5, x6, x7, x8, b, ohai over Rational Field,
 {...},
 a,
 b)
>>> sorted(l[Integer(1)].items())
[([[1, 1], [2]], x1), ([[1, 1], [3]], x2),
 ([[1, 2], [2]], x3), ([[1, 2], [3]], x4),
 ([[1, 3], [2]], x5), ([[1, 3], [3]], x6),
 ([[2, 2], [3]], x7), ([[2, 3], [3]], x8)]
```

See *birational_rowmotion()*, *birational_toggle()* and *birational_toggles()* for more substantial examples of what one can do with the birational free labelling.

birational_rowmotion(*labelling*)

Return the result of applying birational rowmotion to the **K**-labelling `labelling` of the poset `self`.

See the documentation of *birational_free_labelling()* for a definition of birational rowmotion and **K**-labellings and for an explanation of how **K**-labellings are to be encoded to be understood by Sage. This implementation allows **K** to be a semifield, not just a field. Birational rowmotion is only a rational map, so an exception (most likely, `ZeroDivisionError`) will be thrown if the denominator is zero.

INPUT:

- `labelling` – a **K**-labelling of `self` in the sense as defined in the documentation of *birational_free_labelling()*

OUTPUT:

The image of the **K**-labelling f under birational rowmotion.

EXAMPLES:

```
sage: P = Poset({1: [2, 3], 2: [4], 3: [4]})
sage: lex = [1, 2, 3, 4]
sage: t = P.birational_free_labelling(linear_extension=lex); t
(Fraction Field of Multivariate Polynomial Ring in a, x1, x2, x3, x4, b␣
↪over Rational Field,
 {...},
```

(continues on next page)

(continued from previous page)

```
 a,
 b)
sage: sorted(t[1].items())
[(1, x1), (2, x2), (3, x3), (4, x4)]
sage: t = P.birational_rowmotion(t); t
(Fraction Field of Multivariate Polynomial Ring in a, x1, x2, x3, x4, b␣
↪over Rational Field,
 {...},
 a,
 b)
sage: sorted(t[1].items())
[(1, a*b/x4), (2, (x1*x2*b + x1*x3*b)/(x2*x4)),
 (3, (x1*x2*b + x1*x3*b)/(x3*x4)), (4, (x2*b + x3*b)/x4)]
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)]})
>>> lex = [Integer(1), Integer(2), Integer(3), Integer(4)]
>>> t = P.birational_free_labelling(linear_extension=lex); t
(Fraction Field of Multivariate Polynomial Ring in a, x1, x2, x3, x4, b␣
↪over Rational Field,
 {...},
 a,
 b)
>>> sorted(t[Integer(1)].items())
[(1, x1), (2, x2), (3, x3), (4, x4)]
>>> t = P.birational_rowmotion(t); t
(Fraction Field of Multivariate Polynomial Ring in a, x1, x2, x3, x4, b␣
↪over Rational Field,
 {...},
 a,
 b)
>>> sorted(t[Integer(1)].items())
[(1, a*b/x4), (2, (x1*x2*b + x1*x3*b)/(x2*x4)),
 (3, (x1*x2*b + x1*x3*b)/(x3*x4)), (4, (x2*b + x3*b)/x4)]
```

A result of [GR2013] states that applying birational rowmotion $n + m$ times to a $\mathbf{K}$-labelling f of the poset $[n] \times [m]$ gives back f. Let us check this:

```
sage: def test_rectangle_periodicity(n, m, k):
....:     P = posets.ChainPoset(n).product(posets.ChainPoset(m))
....:     t0 = P.birational_free_labelling(P)
....:     t = t0
....:     for i in range(k):
....:         t = P.birational_rowmotion(t)
....:     return t == t0
sage: test_rectangle_periodicity(2, 2, 4)                                #␣
↪needs sage.modules
True
sage: test_rectangle_periodicity(2, 2, 2)                                #␣
↪needs sage.modules
False
```

(continues on next page)

(continued from previous page)

```
sage: test_rectangle_periodicity(2, 3, 5)       # long time                #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> def test_rectangle_periodicity(n, m, k):
...     P = posets.ChainPoset(n).product(posets.ChainPoset(m))
...     t0 = P.birational_free_labelling(P)
...     t = t0
...     for i in range(k):
...         t = P.birational_rowmotion(t)
...     return t == t0
>>> test_rectangle_periodicity(Integer(2), Integer(2), Integer(4))       ␣
↪                       # needs sage.modules
True
>>> test_rectangle_periodicity(Integer(2), Integer(2), Integer(2))       ␣
↪                       # needs sage.modules
False
>>> test_rectangle_periodicity(Integer(2), Integer(3), Integer(5))
↪# long time             # needs sage.modules
True
```

While computations with the birational free labelling quickly run out of memory due to the complexity of the rational functions involved, it is computationally cheap to check properties of birational rowmotion on examples in the tropical semiring:

```
sage: def test_rectangle_periodicity_tropical(n, m, k):
....:     P = posets.ChainPoset(n).product(posets.ChainPoset(m))
....:     TT = TropicalSemiring(ZZ)
....:     t0 = (TT, {v: TT(randint(0, 99)) for v in P}, TT(0), TT(124))
....:     t = t0
....:     for i in range(k):
....:         t = P.birational_rowmotion(t)
....:     return t == t0
sage: test_rectangle_periodicity_tropical(7, 6, 13)                       #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> def test_rectangle_periodicity_tropical(n, m, k):
...     P = posets.ChainPoset(n).product(posets.ChainPoset(m))
...     TT = TropicalSemiring(ZZ)
...     t0 = (TT, {v: TT(randint(Integer(0), Integer(99))) for v in P},␣
↪TT(Integer(0)), TT(Integer(124)))
...     t = t0
...     for i in range(k):
...         t = P.birational_rowmotion(t)
...     return t == t0
>>> test_rectangle_periodicity_tropical(Integer(7), Integer(6),␣
↪Integer(13))                     # needs sage.modules
True
```

Tropicalization is also what relates birational rowmotion to classical rowmotion on order ideals. In fact, if T denotes the `tropical semiring` of $\mathbf{Z}$ and P is a finite poset, then we can define an embedding ϕ from the set $J(P)$ of all order ideals of P into the set $T^{\widehat{P}}$ of all T-labellings of P by sending every $I \in J(P)$ to the indicator function of I extended by the value 1 at the element 0 and the value 0 at the element 1. This map ϕ has the property that $R \circ \phi = \phi \circ r$, where R denotes birational rowmotion, and r denotes *classical rowmotion* on $J(P)$. An example:

```
sage: P = posets.IntegerPartitions(5)
sage: TT = TropicalSemiring(ZZ)
sage: def indicator_labelling(I):
....:     # send order ideal `I` to a `T`-labelling of `P`.
....:     dct = {v: TT(v in I) for v in P}
....:     return (TT, dct, TT(1), TT(0))
sage: all(indicator_labelling(P.rowmotion(I))                          #␣
↪needs sage.modules
....:     == P.birational_rowmotion(indicator_labelling(I))
....:     for I in P.order_ideals_lattice(facade=True))
True
```

```
>>> from sage.all import *
>>> P = posets.IntegerPartitions(Integer(5))
>>> TT = TropicalSemiring(ZZ)
>>> def indicator_labelling(I):
...     # send order ideal `I` to a `T`-labelling of `P`.
...     dct = {v: TT(v in I) for v in P}
...     return (TT, dct, TT(Integer(1)), TT(Integer(0)))
>>> all(indicator_labelling(P.rowmotion(I))                            #␣
↪needs sage.modules
...     == P.birational_rowmotion(indicator_labelling(I))
...     for I in P.order_ideals_lattice(facade=True))
True
```

birational_toggle(*v*, *labelling*)

Return the result of applying the birational v-toggle T_v to the $\mathbf{K}$-labelling `labelling` of the poset `self`.

See the documentation of *birational_free_labelling()* for a definition of this toggle and of $\mathbf{K}$-labellings as well as an explanation of how $\mathbf{K}$-labellings are to be encoded to be understood by Sage. This implementation allows $\mathbf{K}$ to be a semifield, not just a field. The birational v-toggle is only a rational map, so an exception (most likely, `ZeroDivisionError`) will be thrown if the denominator is zero.

INPUT:

- `v` – an element of `self` (must have `self` as parent if `self` is a `facade=False` poset)
- `labelling` – a $\mathbf{K}$-labelling of `self` in the sense as defined in the documentation of *birational_free_labelling()*

OUTPUT:

The $\mathbf{K}$-labelling $T_v f$ of `self`, where f is `labelling`.

EXAMPLES:

Let us start with the birational free labelling of the "V"-poset (the three-element poset with Hasse diagram looking like a "V"):

```
sage: V = Poset({1: [2, 3]})
sage: s = V.birational_free_labelling(); s
```

(continues on next page)

(continued from previous page)

```
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(s[1].items())
[(1, x1), (2, x2), (3, x3)]
```

```
>>> from sage.all import *
>>> V = Poset({Integer(1): [Integer(2), Integer(3)]})
>>> s = V.birational_free_labelling(); s
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
>>> sorted(s[Integer(1)].items())
[(1, x1), (2, x2), (3, x3)]
```

The image of s under the 1-toggle T_1 is:

```
sage: s1 = V.birational_toggle(1, s); s1
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(s1[1].items())
[(1, a*x2*x3/(x1*x2 + x1*x3)), (2, x2), (3, x3)]
```

```
>>> from sage.all import *
>>> s1 = V.birational_toggle(Integer(1), s); s1
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
>>> sorted(s1[Integer(1)].items())
[(1, a*x2*x3/(x1*x2 + x1*x3)), (2, x2), (3, x3)]
```

Now let us apply the 2-toggle T_2 (to the old `s`):

```
sage: s2 = V.birational_toggle(2, s); s2
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(s2[1].items())
[(1, x1), (2, x1*b/x2), (3, x3)]
```

```
>>> from sage.all import *
>>> s2 = V.birational_toggle(Integer(2), s); s2
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
>>> sorted(s2[Integer(1)].items())
[(1, x1), (2, x1*b/x2), (3, x3)]
```

On the other hand, we can also apply T_2 to the image of s under T_1:

```
sage: s12 = V.birational_toggle(2, s1); s12
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
sage: sorted(s12[1].items())
[(1, a*x2*x3/(x1*x2 + x1*x3)), (2, a*x3*b/(x1*x2 + x1*x3)), (3, x3)]
```

```
>>> from sage.all import *
>>> s12 = V.birational_toggle(Integer(2), s1); s12
(Fraction Field of Multivariate Polynomial Ring
  in a, x1, x2, x3, b over Rational Field,
 {...},
 a,
 b)
>>> sorted(s12[Integer(1)].items())
[(1, a*x2*x3/(x1*x2 + x1*x3)), (2, a*x3*b/(x1*x2 + x1*x3)), (3, x3)]
```

Each toggle is an involution:

```
sage: all( V.birational_toggle(i, V.birational_toggle(i, s)) == s
....:      for i in V )
True
```

```
>>> from sage.all import *
>>> all( V.birational_toggle(i, V.birational_toggle(i, s)) == s
...      for i in V )
True
```

We can also start with a less generic labelling:

```
sage: t = (QQ, {1: 3, 2: 6, 3: 7}, 2, 10)
sage: t1 = V.birational_toggle(1, t); t1
(Rational Field, {...}, 2, 10)
sage: sorted(t1[1].items())
[(1, 28/13), (2, 6), (3, 7)]
sage: t13 = V.birational_toggle(3, t1); t13
(Rational Field, {...}, 2, 10)
sage: sorted(t13[1].items())
[(1, 28/13), (2, 6), (3, 40/13)]
```

```
>>> from sage.all import *
>>> t = (QQ, {Integer(1): Integer(3), Integer(2): Integer(6), Integer(3):␣
↪Integer(7)}, Integer(2), Integer(10))
>>> t1 = V.birational_toggle(Integer(1), t); t1
(Rational Field, {...}, 2, 10)
>>> sorted(t1[Integer(1)].items())
[(1, 28/13), (2, 6), (3, 7)]
>>> t13 = V.birational_toggle(Integer(3), t1); t13
(Rational Field, {...}, 2, 10)
>>> sorted(t13[Integer(1)].items())
[(1, 28/13), (2, 6), (3, 40/13)]
```

However, labellings have to be sufficiently generic, lest denominators vanish:

```
sage: t = (QQ, {1: 3, 2: 5, 3: -5}, 1, 15)
sage: t1 = V.birational_toggle(1, t)
Traceback (most recent call last):
...
ZeroDivisionError: rational division by zero
```

```
>>> from sage.all import *
>>> t = (QQ, {Integer(1): Integer(3), Integer(2): Integer(5), Integer(3):␣
↪-Integer(5)}, Integer(1), Integer(15))
>>> t1 = V.birational_toggle(Integer(1), t)
Traceback (most recent call last):
...
ZeroDivisionError: rational division by zero
```

We don't get into zero-division issues in the tropical semiring (unless the zero of the tropical semiring appears in the labelling):

```
sage: TT = TropicalSemiring(QQ)
sage: t = (TT, {1: TT(2), 2: TT(4), 3: TT(1)}, TT(6), TT(0))
sage: t1 = V.birational_toggle(1, t); t1
(Tropical semiring over Rational Field, {...}, 6, 0)
sage: sorted(t1[1].items())
[(1, 8), (2, 4), (3, 1)]
sage: t12 = V.birational_toggle(2, t1); t12
(Tropical semiring over Rational Field, {...}, 6, 0)
sage: sorted(t12[1].items())
[(1, 8), (2, 4), (3, 1)]
sage: t123 = V.birational_toggle(3, t12); t123
(Tropical semiring over Rational Field, {...}, 6, 0)
sage: sorted(t123[1].items())
[(1, 8), (2, 4), (3, 7)]
```

```
>>> from sage.all import *
>>> TT = TropicalSemiring(QQ)
>>> t = (TT, {Integer(1): TT(Integer(2)), Integer(2): TT(Integer(4)),␣
↪Integer(3): TT(Integer(1))}, TT(Integer(6)), TT(Integer(0)))
>>> t1 = V.birational_toggle(Integer(1), t); t1
(Tropical semiring over Rational Field, {...}, 6, 0)
>>> sorted(t1[Integer(1)].items())
```

(continues on next page)

(continued from previous page)

```
[(1, 8), (2, 4), (3, 1)]
>>> t12 = V.birational_toggle(Integer(2), t1); t12
(Tropical semiring over Rational Field, {...}, 6, 0)
>>> sorted(t12[Integer(1)].items())
[(1, 8), (2, 4), (3, 1)]
>>> t123 = V.birational_toggle(Integer(3), t12); t123
(Tropical semiring over Rational Field, {...}, 6, 0)
>>> sorted(t123[Integer(1)].items())
[(1, 8), (2, 4), (3, 7)]
```

We turn to more interesting posets. Here is the 6-element poset arising from the weak order on S_3:

```
sage: P = posets.SymmetricGroupWeakOrderPoset(3)
sage: sorted(list(P))
['123', '132', '213', '231', '312', '321']
sage: t = (TT, {'123': TT(4), '132': TT(2), '213': TT(3),
....:           '231': TT(1), '321': TT(1), '312': TT(2)}, TT(7), TT(1))
sage: t1 = P.birational_toggle('123', t); t1
(Tropical semiring over Rational Field, {...}, 7, 1)
sage: sorted(t1[1].items())
[('123', 6), ('132', 2), ('213', 3), ('231', 1), ('312', 2), ('321', 1)]
sage: t13 = P.birational_toggle('213', t1); t13
(Tropical semiring over Rational Field, {...}, 7, 1)
sage: sorted(t13[1].items())
[('123', 6), ('132', 2), ('213', 4), ('231', 1), ('312', 2), ('321', 1)]
```

```
>>> from sage.all import *
>>> P = posets.SymmetricGroupWeakOrderPoset(Integer(3))
>>> sorted(list(P))
['123', '132', '213', '231', '312', '321']
>>> t = (TT, {'123': TT(Integer(4)), '132': TT(Integer(2)), '213':␣
↪TT(Integer(3)),
...           '231': TT(Integer(1)), '321': TT(Integer(1)), '312':␣
↪TT(Integer(2))}, TT(Integer(7)), TT(Integer(1)))
>>> t1 = P.birational_toggle('123', t); t1
(Tropical semiring over Rational Field, {...}, 7, 1)
>>> sorted(t1[Integer(1)].items())
[('123', 6), ('132', 2), ('213', 3), ('231', 1), ('312', 2), ('321', 1)]
>>> t13 = P.birational_toggle('213', t1); t13
(Tropical semiring over Rational Field, {...}, 7, 1)
>>> sorted(t13[Integer(1)].items())
[('123', 6), ('132', 2), ('213', 4), ('231', 1), ('312', 2), ('321', 1)]
```

Let us verify on this example some basic properties of toggles. First of all, again let us check that T_v is an involution for every v:

```
sage: all( P.birational_toggle(v, P.birational_toggle(v, t)) == t
....:      for v in P )
True
```

```
>>> from sage.all import *
>>> all( P.birational_toggle(v, P.birational_toggle(v, t)) == t
```

(continues on next page)

(continued from previous page)

```
...       for v in P )
True
```

Furthermore, two toggles T_v and T_w commute unless one of v or w covers the other:

```
sage: all( P.covers(v, w) or P.covers(w, v)
....:       or P.birational_toggle(v, P.birational_toggle(w, t))
....:          == P.birational_toggle(w, P.birational_toggle(v, t))
....:       for v in P for w in P )
True
```

```
>>> from sage.all import *
>>> all( P.covers(v, w) or P.covers(w, v)
...       or P.birational_toggle(v, P.birational_toggle(w, t))
...          == P.birational_toggle(w, P.birational_toggle(v, t))
...       for v in P for w in P )
True
```

birational_toggles(*vs*, *labelling*)

Return the result of applying a sequence of birational toggles (specified by `vs`) to the **K**-labelling `labelling` of the poset `self`.

See the documentation of *birational_free_labelling()* for a definition of birational toggles and **K**-labellings and for an explanation of how **K**-labellings are to be encoded to be understood by Sage. This implementation allows **K** to be a semifield, not just a field. The birational v-toggle is only a rational map, so an exception (most likely, `ZeroDivisionError`) will be thrown if the denominator is zero.

INPUT:

- `vs` – an iterable comprising elements of `self` (which must have `self` as parent if `self` is a `facade=False` poset)
- `labelling` – a **K**-labelling of `self` in the sense as defined in the documentation of *birational_free_labelling()*

OUTPUT:

The **K**-labelling $T_{v_n} T_{v_{n-1}} \cdots T_{v_1} f$ of `self`, where f is `labelling` and $(v_1, v_2, \ldots, v_n)$ is `vs` (written as list).

EXAMPLES:

```
sage: P = posets.SymmetricGroupBruhatOrderPoset(3)
sage: sorted(list(P))
['123', '132', '213', '231', '312', '321']
sage: TT = TropicalSemiring(ZZ)
sage: t = (TT, {'123': TT(4), '132': TT(2), '213': TT(3),
....:          '231': TT(1), '321': TT(1), '312': TT(2)}, TT(7), TT(1))
sage: tA = P.birational_toggles(['123', '231', '312'], t); tA
(Tropical semiring over Integer Ring, {...}, 7, 1)
sage: sorted(tA[1].items())
[('123', 6), ('132', 2), ('213', 3), ('231', 2), ('312', 1), ('321', 1)]
sage: tAB = P.birational_toggles(['132', '213', '321'], tA); tAB
(Tropical semiring over Integer Ring, {...}, 7, 1)
sage: sorted(tAB[1].items())
[('123', 6), ('132', 6), ('213', 5), ('231', 2), ('312', 1), ('321', 1)]
```

(continues on next page)

(continued from previous page)

```
sage: P = Poset({1: [2, 3], 2: [4], 3: [4]})
sage: Qx = PolynomialRing(QQ, 'x').fraction_field()
sage: x = Qx.gen()
sage: t = (Qx, {1: 1, 2: x, 3: (x+1)/x, 4: x^2}, 1, 1)
sage: t1 = P.birational_toggles((i for i in range(1, 5)), t); t1
(Fraction Field of Univariate Polynomial Ring in x over Rational Field,
 {...},
 1,
 1)
sage: sorted(t1[1].items())
[(1, (x^2 + x)/(x^2 + x + 1)),
 (2, (x^3 + x^2)/(x^2 + x + 1)),
 (3, x^4/(x^2 + x + 1)), (4, 1)]
sage: t2 = P.birational_toggles(reversed(range(1, 5)), t)
sage: sorted(t2[1].items())
[(1, 1/x^2), (2, (x^2 + x + 1)/x^4), (3, (x^2 + x + 1)/(x^3 + x^2)),
 (4, (x^2 + x + 1)/x^3)]
```

```
>>> from sage.all import *
>>> P = posets.SymmetricGroupBruhatOrderPoset(Integer(3))
>>> sorted(list(P))
['123', '132', '213', '231', '312', '321']
>>> TT = TropicalSemiring(ZZ)
>>> t = (TT, {'123': TT(Integer(4)), '132': TT(Integer(2)), '213':␣
↪TT(Integer(3)),
...           '231': TT(Integer(1)), '321': TT(Integer(1)), '312':␣
↪TT(Integer(2))}, TT(Integer(7)), TT(Integer(1)))
>>> tA = P.birational_toggles(['123', '231', '312'], t); tA
(Tropical semiring over Integer Ring, {...}, 7, 1)
>>> sorted(tA[Integer(1)].items())
[('123', 6), ('132', 2), ('213', 3), ('231', 2), ('312', 1), ('321', 1)]
>>> tAB = P.birational_toggles(['132', '213', '321'], tA); tAB
(Tropical semiring over Integer Ring, {...}, 7, 1)
>>> sorted(tAB[Integer(1)].items())
[('123', 6), ('132', 6), ('213', 5), ('231', 2), ('312', 1), ('321', 1)]

>>> P = Poset({Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)]})
>>> Qx = PolynomialRing(QQ, 'x').fraction_field()
>>> x = Qx.gen()
>>> t = (Qx, {Integer(1): Integer(1), Integer(2): x, Integer(3):␣
↪(x+Integer(1))/x, Integer(4): x**Integer(2)}, Integer(1), Integer(1))
>>> t1 = P.birational_toggles((i for i in range(Integer(1), Integer(5))),␣
↪t); t1
(Fraction Field of Univariate Polynomial Ring in x over Rational Field,
 {...},
 1,
 1)
>>> sorted(t1[Integer(1)].items())
[(1, (x^2 + x)/(x^2 + x + 1)),
 (2, (x^3 + x^2)/(x^2 + x + 1)),
 (3, x^4/(x^2 + x + 1)), (4, 1)]
```

(continues on next page)

(continued from previous page)

```
>>> t2 = P.birational_toggles(reversed(range(Integer(1), Integer(5))), t)
>>> sorted(t2[Integer(1)].items())
[(1, 1/x^2), (2, (x^2 + x + 1)/x^4), (3, (x^2 + x + 1)/(x^3 + x^2)),
 (4, (x^2 + x + 1)/x^3)]
```

Facade set to `False` works:

```
sage: P = Poset({'x': ['y', 'w'], 'y': ['z'], 'w': ['z']}, facade=False)
sage: lex = ['x', 'y', 'w', 'z']
sage: t = P.birational_free_labelling(linear_extension=lex)
sage: sorted(P.birational_toggles([P('x'), P('y')], t)[1].items())
[(x, a*x2*x3/(x1*x2 + x1*x3)), (y, a*x3*x4/(x1*x2 + x1*x3)), (w, x3), (z,␣
↪x4)]
```

```
>>> from sage.all import *
>>> P = Poset({'x': ['y', 'w'], 'y': ['z'], 'w': ['z']}, facade=False)
>>> lex = ['x', 'y', 'w', 'z']
>>> t = P.birational_free_labelling(linear_extension=lex)
>>> sorted(P.birational_toggles([P('x'), P('y')], t)[Integer(1)].items())
[(x, a*x2*x3/(x1*x2 + x1*x3)), (y, a*x3*x4/(x1*x2 + x1*x3)), (w, x3), (z,␣
↪x4)]
```

directed_subsets(*direction*)

Return the order filters (resp. order ideals) of `self`, as lists.

If `direction` is `'up'`, returns the order filters (upper sets).

If `direction` is `'down'`, returns the order ideals (lower sets).

INPUT:

- `direction` – `'up'` or `'down'`

EXAMPLES:

```
sage: P = Poset((divisors(12), attrcall("divides")), facade=True)
sage: A = P.directed_subsets('up')
sage: sorted(list(A))                                                       #␣
↪needs sage.modules
[[], [1, 2, 4, 3, 6, 12], [2, 4, 3, 6, 12], [2, 4, 6, 12], [3, 6, 12],
 [4, 3, 6, 12], [4, 6, 12], [4, 12], [6, 12], [12]]
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")), facade=True)
>>> A = P.directed_subsets('up')
>>> sorted(list(A))                                                         #␣
↪needs sage.modules
[[], [1, 2, 4, 3, 6, 12], [2, 4, 3, 6, 12], [2, 4, 6, 12], [3, 6, 12],
 [4, 3, 6, 12], [4, 6, 12], [4, 12], [6, 12], [12]]
```

is_lattice()

Return whether the poset is a lattice.

A poset is a lattice if all pairs of elements have both a least upper bound ("join") and a greatest lower bound ("meet") in the poset.

EXAMPLES:

```
sage: P = Poset([[1, 3, 2], [4], [4, 5, 6], [6], [7], [7], [7], []])
sage: P.is_lattice()                                                    #␣
↪needs sage.modules
True

sage: P = Poset([[1, 2], [3], [3], []])
sage: P.is_lattice()                                                    #␣
↪needs sage.modules
True

sage: P = Poset({0: [2, 3], 1: [2, 3]})
sage: P.is_lattice()
False

sage: P = Poset({1: [2, 3, 4], 2: [5, 6], 3: [5, 7], 4: [6, 7], 5: [8, 9],
....:            6: [8, 10], 7: [9, 10], 8: [11], 9: [11], 10: [11]})
sage: P.is_lattice()                                                    #␣
↪needs sage.modules
False
```

```
>>> from sage.all import *
>>> P = Poset([[Integer(1), Integer(3), Integer(2)], [Integer(4)],␣
↪[Integer(4), Integer(5), Integer(6)], [Integer(6)], [Integer(7)],␣
↪[Integer(7)], [Integer(7)], []])
>>> P.is_lattice()                                                      #␣
↪needs sage.modules
True

>>> P = Poset([[Integer(1), Integer(2)], [Integer(3)], [Integer(3)], []])
>>> P.is_lattice()                                                      #␣
↪needs sage.modules
True

>>> P = Poset({Integer(0): [Integer(2), Integer(3)], Integer(1):␣
↪[Integer(2), Integer(3)]})
>>> P.is_lattice()
False

>>> P = Poset({Integer(1): [Integer(2), Integer(3), Integer(4)],␣
↪Integer(2): [Integer(5), Integer(6)], Integer(3): [Integer(5),␣
↪Integer(7)], Integer(4): [Integer(6), Integer(7)], Integer(5):␣
↪[Integer(8), Integer(9)],
...            Integer(6): [Integer(8), Integer(10)], Integer(7):␣
↪[Integer(9), Integer(10)], Integer(8): [Integer(11)], Integer(9):␣
↪[Integer(11)], Integer(10): [Integer(11)]})
>>> P.is_lattice()                                                      #␣
↪needs sage.modules
False
```

See also

- Weaker properties: `is_join_semilattice()`, `is_meet_semilattice()`

is_poset_isomorphism(*f*, *codomain*)

Return whether f is an isomorphism of posets from `self` to `codomain`.

INPUT:

- `f` – a function from `self` to `codomain`
- `codomain` – a poset

EXAMPLES:

We build the poset D of divisors of 30, and check that it is isomorphic to the boolean lattice B of the subsets of $\{2, 3, 5\}$ ordered by inclusion, via the reverse function $f : B \to D, b \mapsto \prod_{x \in b} x$:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: B = Poset(([frozenset(s) for s in Subsets([2,3,5])],
....:           attrcall("issubset")))
sage: def f(b): return D(prod(b))
sage: B.is_poset_isomorphism(f, D)
True
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> B = Poset(([frozenset(s) for s in Subsets([Integer(2),Integer(3),
↪Integer(5)])],
...           attrcall("issubset")))
>>> def f(b): return D(prod(b))
>>> B.is_poset_isomorphism(f, D)
True
```

On the other hand, f is not an isomorphism to the chain of divisors of 30, ordered by usual comparison:

```
sage: P = Poset((divisors(30), operator.le))
sage: def f(b): return P(prod(b))
sage: B.is_poset_isomorphism(f, P)
False
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(30)), operator.le))
>>> def f(b): return P(prod(b))
>>> B.is_poset_isomorphism(f, P)
False
```

A non surjective case:

```
sage: B = Poset(([frozenset(s) for s in Subsets([2,3])],
....:           attrcall("issubset")))
sage: def f(b): return D(prod(b))
sage: B.is_poset_isomorphism(f, D)
False
```

```
>>> from sage.all import *
>>> B = Poset(([frozenset(s) for s in Subsets([Integer(2),Integer(3)])],
...           attrcall("issubset")))
```

(continues on next page)

(continued from previous page)

```
>>> def f(b): return D(prod(b))
>>> B.is_poset_isomorphism(f, D)
False
```

A non injective case:

```
sage: B = Poset(([frozenset(s) for s in Subsets([2,3,5,6])],
....:            attrcall("issubset")))
sage: def f(b): return D(gcd(prod(b), 30))
sage: B.is_poset_isomorphism(f, D)
False
```

```
>>> from sage.all import *
>>> B = Poset(([frozenset(s) for s in Subsets([Integer(2),Integer(3),
↪Integer(5),Integer(6)])],
...            attrcall("issubset")))
>>> def f(b): return D(gcd(prod(b), Integer(30)))
>>> B.is_poset_isomorphism(f, D)
False
```

Note

since `D` and `B` are not facade posets, `f` is responsible for the conversions between integers and subsets to elements of `D` and `B` and back.

See also

FiniteLatticePosets.ParentMethods.is_lattice_morphism()

is_poset_morphism(*f*, *codomain*)

Return whether f is a morphism of posets from `self` to `codomain`, that is

$$x \leq y \Longrightarrow f(x) \leq f(y)$$

for all x and y in `self`.

INPUT:

- `f` – a function from `self` to `codomain`
- `codomain` – a poset

EXAMPLES:

We build the boolean lattice of the subsets of $\{2, 3, 5, 6\}$ and the lattice of divisors of 30, and check that the map $b \mapsto \gcd(\prod_{x \in b} x, 30)$ is a morphism of posets:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: B = Poset(([frozenset(s) for s in Subsets([2,3,5,6])],
....:            attrcall("issubset")))
sage: def f(b): return D(gcd(prod(b), 30))
sage: B.is_poset_morphism(f, D)
True
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> B = Poset(([frozenset(s) for s in Subsets([Integer(2),Integer(3),
↪Integer(5),Integer(6)])],
...            attrcall("issubset")))
>>> def f(b): return D(gcd(prod(b), Integer(30)))
>>> B.is_poset_morphism(f, D)
True
```

Note

since `D` and `B` are not facade posets, `f` is responsible for the conversions between integers and subsets to elements of `D` and `B` and back.

f is also a morphism of posets to the chain of divisors of 30, ordered by usual comparison:

```
sage: P = Poset((divisors(30), operator.le))
sage: def f(b): return P(gcd(prod(b), 30))
sage: B.is_poset_morphism(f, P)
True
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(30)), operator.le))
>>> def f(b): return P(gcd(prod(b), Integer(30)))
>>> B.is_poset_morphism(f, P)
True
```

FIXME: should this be `is_order_preserving_morphism`?

See also

is_poset_isomorphism()

is_self_dual()

Return whether the poset is *self-dual*.

A poset is self-dual if it is isomorphic to its dual poset.

EXAMPLES:

```
sage: P = Poset({1: [3, 4], 2: [3, 4]})
sage: P.is_self_dual()
True

sage: P = Poset({1: [2, 3]})
sage: P.is_self_dual()
False
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(3), Integer(4)], Integer(2):␣
↪[Integer(3), Integer(4)]})
```

(continues on next page)

(continued from previous page)

```
>>> P.is_self_dual()
True

>>> P = Poset({Integer(1): [Integer(2), Integer(3)]})
>>> P.is_self_dual()
False
```

See also

- Stronger properties: `is_orthocomplemented()` (for lattices)
- Other: `dual()`

order_filter_generators (*filter*)

Generators for an order filter.

INPUT:

- `filter` – an order filter of `self`, as a list (or iterable)

EXAMPLES:

```
sage: P = Poset((Subsets([1,2,3]), attrcall("issubset")))
sage: I = P.order_filter([Set([1,2]), Set([2,3]), Set([1])])
sage: sorted(sorted(p) for p in I)
[[1], [1, 2], [1, 2, 3], [1, 3], [2, 3]]
sage: gen = P.order_filter_generators(I)
sage: sorted(sorted(p) for p in gen)
[[1], [2, 3]]
```

```
>>> from sage.all import *
>>> P = Poset((Subsets([Integer(1),Integer(2),Integer(3)]), attrcall(
↪"issubset")))
>>> I = P.order_filter([Set([Integer(1),Integer(2)]), Set([Integer(2),
↪Integer(3)]), Set([Integer(1)])])
>>> sorted(sorted(p) for p in I)
[[1], [1, 2], [1, 2, 3], [1, 3], [2, 3]]
>>> gen = P.order_filter_generators(I)
>>> sorted(sorted(p) for p in gen)
[[1], [2, 3]]
```

See also

order_ideal_generators()

order_ideal_complement_generators (*antichain*, *direction='up'*)

Return the Panyushev complement of the antichain `antichain`.

Given an antichain A of a poset P, the Panyushev complement of A is defined to be the antichain consisting of the minimal elements of the order filter B, where B is the (set-theoretic) complement of the order ideal of P generated by A.

Setting the optional keyword variable `direction` to `'down'` leads to the inverse Panyushev complement being computed instead of the Panyushev complement. The inverse Panyushev complement of an antichain A is the antichain whose Panyushev complement is A. It can be found as the antichain consisting of the maximal elements of the order ideal C, where C is the (set-theoretic) complement of the order filter of P generated by A.

panyushev_complement() is an alias for this method.

Panyushev complementation is related (actually, isomorphic) to rowmotion (*rowmotion()*).

INPUT:

- `antichain` – an antichain of `self`, as a list (or iterable), or, more generally, generators of an order ideal (resp. order filter)
- `direction` – `'up'` or `'down'` (default: `'up'`)

OUTPUT:

- the generating antichain of the complement order filter (resp. order ideal) of the order ideal (resp. order filter) generated by the antichain `antichain`

EXAMPLES:

```
sage: P = Poset( ( [1,2,3], [ [1,3], [2,3] ] ) )
sage: P.order_ideal_complement_generators([1])
{2}
sage: P.order_ideal_complement_generators([3])
set()
sage: P.order_ideal_complement_generators([1,2])
{3}
sage: P.order_ideal_complement_generators([1,2,3])
set()

sage: P.order_ideal_complement_generators([1], direction='down')
{2}
sage: P.order_ideal_complement_generators([3], direction='down')
{1, 2}
sage: P.order_ideal_complement_generators([1,2], direction='down')
set()
sage: P.order_ideal_complement_generators([1,2,3], direction='down')
set()
```

```
>>> from sage.all import *
>>> P = Poset( ( [Integer(1),Integer(2),Integer(3)], [ [Integer(1),
↪Integer(3)], [Integer(2),Integer(3)] ] ) )
>>> P.order_ideal_complement_generators([Integer(1)])
{2}
>>> P.order_ideal_complement_generators([Integer(3)])
set()
>>> P.order_ideal_complement_generators([Integer(1),Integer(2)])
{3}
>>> P.order_ideal_complement_generators([Integer(1),Integer(2),
↪Integer(3)])
set()

>>> P.order_ideal_complement_generators([Integer(1)], direction='down')
{2}
>>> P.order_ideal_complement_generators([Integer(3)], direction='down')
{1, 2}
```

(continues on next page)

(continued from previous page)

```
>>> P.order_ideal_complement_generators([Integer(1),Integer(2)],␣
↪direction='down')
set()
>>> P.order_ideal_complement_generators([Integer(1),Integer(2),
↪Integer(3)], direction='down')
set()
```

Warning

This is a brute force implementation, building the order ideal generated by the antichain, and searching for order filter generators of its complement

order_ideal_generators(*ideal*, *direction='down'*)

Return the antichain of (minimal) generators of the order ideal (resp. order filter) `ideal`.

INPUT:

- `ideal` – an order ideal I (resp. order filter) of `self`, as a list (or iterable); this should be an order ideal if `direction` is set to `'down'`, and an order filter if `direction` is set to `'up'`
- `direction` – `'up'` or `'down'` (default: `'down'`)

The antichain of (minimal) generators of an order ideal I in a poset P is the set of all minimal elements of P. In the case of an order filter, the definition is similar, but with "maximal" used instead of "minimal".

EXAMPLES:

We build the boolean lattice of all subsets of $\{1, 2, 3\}$ ordered by inclusion, and compute an order ideal there:

```
sage: P = Poset((Subsets([1,2,3]), attrcall("issubset")))
sage: I = P.order_ideal([Set([1,2]), Set([2,3]), Set([1])])
sage: sorted(sorted(p) for p in I)
[[], [1], [1, 2], [2], [2, 3], [3]]
```

```
>>> from sage.all import *
>>> P = Poset((Subsets([Integer(1),Integer(2),Integer(3)]), attrcall(
↪"issubset")))
>>> I = P.order_ideal([Set([Integer(1),Integer(2)]), Set([Integer(2),
↪Integer(3)]), Set([Integer(1)])])
>>> sorted(sorted(p) for p in I)
[[], [1], [1, 2], [2], [2, 3], [3]]
```

Then, we retrieve the generators of this ideal:

```
sage: gen = P.order_ideal_generators(I)
sage: sorted(sorted(p) for p in gen)
[[1, 2], [2, 3]]
```

```
>>> from sage.all import *
>>> gen = P.order_ideal_generators(I)
>>> sorted(sorted(p) for p in gen)
[[1, 2], [2, 3]]
```

If `direction` is `'up'`, then this instead computes the minimal generators for an order filter:

```
sage: I = P.order_filter([Set([1,2]), Set([2,3]), Set([1])])
sage: sorted(sorted(p) for p in I)
[[1], [1, 2], [1, 2, 3], [1, 3], [2, 3]]
sage: gen = P.order_ideal_generators(I, direction='up')
sage: sorted(sorted(p) for p in gen)
[[1], [2, 3]]
```

```
>>> from sage.all import *
>>> I = P.order_filter([Set([Integer(1),Integer(2)]), Set([Integer(2),
↪Integer(3)]), Set([Integer(1)])])
>>> sorted(sorted(p) for p in I)
[[1], [1, 2], [1, 2, 3], [1, 3], [2, 3]]
>>> gen = P.order_ideal_generators(I, direction='up')
>>> sorted(sorted(p) for p in gen)
[[1], [2, 3]]
```

Complexity: $O(n + m)$ where n is the cardinality of I, and m the number of upper covers of elements of I.

order_ideals_lattice(*as_ideals=True*, *facade=None*)

Return the lattice of order ideals of a poset `self`, ordered by inclusion.

The lattice of order ideals of a poset P is usually denoted by $J(P)$. Its underlying set is the set of order ideals of P, and its partial order is given by inclusion.

The order ideals of P are in a canonical bijection with the antichains of P. The bijection maps every order ideal to the antichain formed by its maximal elements. By setting the `as_ideals` keyword variable to `False`, one can make this method apply this bijection before returning the lattice.

INPUT:

- `as_ideals` – boolean (default: `True`); if `True` returns a poset on the set of order ideals, otherwise on the set of antichains
- `facade` – boolean or `None` (default); whether to return a facade lattice or not. By default return facade lattice if the poset is a facade poset.

EXAMPLES:

```
sage: # needs sage.modules
sage: P = posets.PentagonPoset()
sage: P.cover_relations()
[[0, 1], [0, 2], [1, 4], [2, 3], [3, 4]]
sage: J = P.order_ideals_lattice(); J
Finite lattice containing 8 elements
sage: sorted(sorted(e) for e in J)
 [[], [0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4], [0, 2], [0,␣
↪2, 3]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> P = posets.PentagonPoset()
>>> P.cover_relations()
[[0, 1], [0, 2], [1, 4], [2, 3], [3, 4]]
>>> J = P.order_ideals_lattice(); J
Finite lattice containing 8 elements
>>> sorted(sorted(e) for e in J)
```

(continues on next page)

(continued from previous page)

```
 [[], [0], [0, 1], [0, 1, 2], [0, 1, 2, 3], [0, 1, 2, 3, 4], [0, 2], [0,␣
↪2, 3]]
```

As a lattice on antichains:

```
sage: J2 = P.order_ideals_lattice(False); J2                                #␣
↪needs sage.modules
Finite lattice containing 8 elements
sage: sorted(J2)                                                            #␣
↪needs sage.modules
[(), (0,), (1,), (1, 2), (1, 3), (2,), (3,), (4,)]
```

```
>>> from sage.all import *
>>> J2 = P.order_ideals_lattice(False); J2                                  #␣
↪needs sage.modules
Finite lattice containing 8 elements
>>> sorted(J2)                                                              #␣
↪needs sage.modules
[(), (0,), (1,), (1, 2), (1, 3), (2,), (3,), (4,)]
```

panyushev_complement(*antichain*, *direction='up'*)

alias of *order_ideal_complement_generators()*.

panyushev_orbit_iter(*antichain*, *element_constructor=<class 'set'>*, *stop=True*, *check=True*)

Iterate over the Panyushev orbit of an antichain `antichain` of `self`.

The Panyushev orbit of an antichain is its orbit under Panyushev complementation (see *panyushev_complement()*).

INPUT:

- `antichain` – an antichain of `self`, given as an iterable
- `element_constructor` – a type constructor (default: `set`). Can be `set`, `tuple`, `list`, `frozenset`, `iter`, etc. To be applied to the antichains before they are yielded.
- `stop` – boolean (default: `True`); whether the iterator should stop once it completes its cycle (this happens when it is set to `True`) or go on forever (this happens when it is set to `False`).
- `check` – boolean (default: `True`); whether to check `antichain` for being an antichain

OUTPUT:

- an iterator over the orbit of the antichain `antichain` under Panyushev complementation. This iterator I has the property that `I[0] == antichain` and each i satisfies `self.order_ideal_complement_generators(I[i]) == I[i+1]`, where `I[i+1]` has to be understood as `I[0]` if it is undefined. The entries `I[i]` are sets by default, but depending on the optional keyword variable `element_constructors` they can also be tuples, lists etc.

EXAMPLES:

```
sage: P = Poset( ( [1,2,3], [ [1,3], [2,3] ] ) )
sage: list(P.panyushev_orbit_iter(set([1, 2])))
[{1, 2}, {3}, set()]
sage: list(P.panyushev_orbit_iter([1, 2]))
[{1, 2}, {3}, set()]
sage: list(P.panyushev_orbit_iter([2, 1]))
[{1, 2}, {3}, set()]
sage: list(P.panyushev_orbit_iter(set([1, 2]), element_constructor=list))
[[1, 2], [3], []]
```

(continues on next page)

(continued from previous page)

```
sage: list(P.panyushev_orbit_iter(set([1, 2]), element_
↪constructor=frozenset))
[frozenset({1, 2}), frozenset({3}), frozenset()]
sage: list(P.panyushev_orbit_iter(set([1, 2]), element_constructor=tuple))
[(1, 2), (3,), ()]

sage: P = Poset( {} )
sage: list(P.panyushev_orbit_iter([]))
[set()]

sage: P = Poset({ 1: [2, 3], 2: [4], 3: [4], 4: [] })
sage: Piter = P.panyushev_orbit_iter([2], stop=False)
sage: next(Piter)
{2}
sage: next(Piter)
{3}
sage: next(Piter)
{2}
sage: next(Piter)
{3}
```

```
>>> from sage.all import *
>>> P = Poset( ( [Integer(1),Integer(2),Integer(3)], [ [Integer(1),
↪Integer(3)], [Integer(2),Integer(3)] ] ) )
>>> list(P.panyushev_orbit_iter(set([Integer(1), Integer(2)])))
[{1, 2}, {3}, set()]
>>> list(P.panyushev_orbit_iter([Integer(1), Integer(2)]))
[{1, 2}, {3}, set()]
>>> list(P.panyushev_orbit_iter([Integer(2), Integer(1)]))
[{1, 2}, {3}, set()]
>>> list(P.panyushev_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=list))
[[1, 2], [3], []]
>>> list(P.panyushev_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=frozenset))
[frozenset({1, 2}), frozenset({3}), frozenset()]
>>> list(P.panyushev_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=tuple))
[(1, 2), (3,), ()]

>>> P = Poset( {} )
>>> list(P.panyushev_orbit_iter([]))
[set()]

>>> P = Poset({ Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)], Integer(4): [] })
>>> Piter = P.panyushev_orbit_iter([Integer(2)], stop=False)
>>> next(Piter)
{2}
>>> next(Piter)
{3}
>>> next(Piter)
```

(continues on next page)

(continued from previous page)

```
{2}
>>> next(Piter)
{3}
```

panyushev_orbits(*element_constructor=<class 'set'>*)

Return the Panyushev orbits of antichains in `self`.

The Panyushev orbit of an antichain is its orbit under Panyushev complementation (see *panyushev_complement()*).

INPUT:

- `element_constructor` – (default: `set`) a type constructor (`set`, `tuple`, `list`, `frozenset`, `iter`, etc.) which is to be applied to the antichains before they are returned

OUTPUT:

- the partition of the set of all antichains of `self` into orbits under Panyushev complementation. This is returned as a list of lists `L` such that for each `L` and `i`, cyclically: `self.order_ideal_complement_generators(L[i]) == L[i+1]`. The entries `L[i]` are sets by default, but depending on the optional keyword variable `element_constructors` they can also be tuples, lists etc.

EXAMPLES:

```
sage: # needs sage.modules
sage: P = Poset( ( [1,2,3], [ [1,3], [2,3] ] ) )
sage: orb = P.panyushev_orbits()
sage: sorted(sorted(o) for o in orb)
[[set(), {1, 2}, {3}], [{2}, {1}]]
sage: orb = P.panyushev_orbits(element_constructor=list)
sage: sorted(sorted(o) for o in orb)
[[[], [1, 2], [3]], [[1], [2]]]
sage: orb = P.panyushev_orbits(element_constructor=frozenset)
sage: sorted(sorted(o) for o in orb)
[[frozenset(), frozenset({1, 2}), frozenset({3})],
 [frozenset({2}), frozenset({1})]]
sage: orb = P.panyushev_orbits(element_constructor=tuple)
sage: sorted(sorted(o) for o in orb)
[[(), (1, 2), (3,)], [(1,), (2,)]]
sage: P = Poset( {} )
sage: P.panyushev_orbits()
[[set()]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> P = Poset( ( [Integer(1),Integer(2),Integer(3)], [ [Integer(1),
↪Integer(3)], [Integer(2),Integer(3)] ] ) )
>>> orb = P.panyushev_orbits()
>>> sorted(sorted(o) for o in orb)
[[set(), {1, 2}, {3}], [{2}, {1}]]
>>> orb = P.panyushev_orbits(element_constructor=list)
>>> sorted(sorted(o) for o in orb)
[[[], [1, 2], [3]], [[1], [2]]]
>>> orb = P.panyushev_orbits(element_constructor=frozenset)
>>> sorted(sorted(o) for o in orb)
[[frozenset(), frozenset({1, 2}), frozenset({3})],
```

(continues on next page)

(continued from previous page)

```
 [frozenset({2}), frozenset({1})]]
>>> orb = P.panyushev_orbits(element_constructor=tuple)
>>> sorted(sorted(o) for o in orb)
[[(), (1, 2), (3,)], [(1,), (2,)]]
>>> P = Poset( {} )
>>> P.panyushev_orbits()
[[set()]]
```

rowmotion(*order_ideal*)

The image of the order ideal `order_ideal` under rowmotion in `self`.

Rowmotion on a finite poset P is an automorphism of the set $J(P)$ of all order ideals of P. One way to define it is as follows: Given an order ideal $I \in J(P)$, we let F be the set-theoretic complement of I in P. Furthermore we let A be the antichain consisting of all minimal elements of F. Then, the rowmotion of I is defined to be the order ideal of P generated by the antichain A (that is, the order ideal consisting of each element of P which has some element of A above it).

Rowmotion is related (actually, isomorphic) to Panyushev complementation (*panyushev_complement()*).

INPUT:

- `order_ideal` – an order ideal of `self`, as a set

OUTPUT: the image of `order_ideal` under rowmotion, as a set again

EXAMPLES:

```
sage: P = Poset( {1: [2, 3], 2: [], 3: [], 4: [8],
....:             5: [], 6: [5], 7: [1, 4], 8: []} )
sage: I = Set({2, 6, 1, 7})
sage: P.rowmotion(I)
{1, 3, 4, 5, 6, 7}

sage: P = Poset( {} )
sage: I = Set({})
sage: P.rowmotion(I)
{}
```

```
>>> from sage.all import *
>>> P = Poset( {Integer(1): [Integer(2), Integer(3)], Integer(2): [],␣
↪Integer(3): [], Integer(4): [Integer(8)],
...              Integer(5): [], Integer(6): [Integer(5)], Integer(7):␣
↪[Integer(1), Integer(4)], Integer(8): []} )
>>> I = Set({Integer(2), Integer(6), Integer(1), Integer(7)})
>>> P.rowmotion(I)
{1, 3, 4, 5, 6, 7}

>>> P = Poset( {} )
>>> I = Set({})
>>> P.rowmotion(I)
{}
```

rowmotion_orbit_iter(*oideal*, *element_constructor=<class 'set'>*, *stop=True*, *check=True*)

Iterate over the rowmotion orbit of an order ideal `oideal` of `self`.

The rowmotion orbit of an order ideal is its orbit under rowmotion (see *rowmotion()*).

INPUT:

- `oideal` – an order ideal of `self`, given as an iterable
- `element_constructor` – (defaults to `set`) a type constructor (`set`, `tuple`, `list`, `frozenset`, `iter`, etc.) which is to be applied to the order ideals before they are yielded
- `stop` – boolean (default: `True`); whether the iterator should stop once it completes its cycle (this happens when it is set to `True`) or go on forever (this happens when it is set to `False`)
- `check` – boolean (default: `True`); whether `oideal` should be checked for being an order ideal

OUTPUT:

- an iterator over the orbit of the order ideal `oideal` under rowmotion. This iterator I has the property that `I[0] == oideal` and that every i satisfies `self.rowmotion(I[i]) == I[i+1]`, where `I[i+1]` has to be understood as `I[0]` if it is undefined. The entries `I[i]` are sets by default, but depending on the optional keyword variable `element_constructors` they can also be tuples, lists etc.

EXAMPLES:

```
sage: P = Poset( ( [1,2,3], [ [1,3], [2,3] ] ) )
sage: list(P.rowmotion_orbit_iter(set([1, 2])))
[{1, 2}, {1, 2, 3}, set()]
sage: list(P.rowmotion_orbit_iter([1, 2]))
[{1, 2}, {1, 2, 3}, set()]
sage: list(P.rowmotion_orbit_iter([2, 1]))
[{1, 2}, {1, 2, 3}, set()]
sage: list(P.rowmotion_orbit_iter(set([1, 2]), element_constructor=list))
[[1, 2], [1, 2, 3], []]
sage: list(P.rowmotion_orbit_iter(set([1, 2]), element_
↪constructor=frozenset))
[frozenset({1, 2}), frozenset({1, 2, 3}), frozenset()]
sage: list(P.rowmotion_orbit_iter(set([1, 2]), element_constructor=tuple))
[(1, 2), (1, 2, 3), ()]

sage: P = Poset( {} )
sage: list(P.rowmotion_orbit_iter([]))
[set()]

sage: P = Poset({ 1: [2, 3], 2: [4], 3: [4], 4: [] })
sage: Piter = P.rowmotion_orbit_iter([1, 2, 3], stop=False)
sage: next(Piter)
{1, 2, 3}
sage: next(Piter)
{1, 2, 3, 4}
sage: next(Piter)
set()
sage: next(Piter)
{1}
sage: next(Piter)
{1, 2, 3}

sage: P = Poset({ 1: [4], 2: [4, 5], 3: [5] })
sage: list(P.rowmotion_orbit_iter([1, 2], element_constructor=list))
[[1, 2], [1, 2, 3, 4], [2, 3, 5], [1], [2, 3], [1, 2, 3, 5], [1, 2, 4],␣
↪[3]]
```

```
>>> from sage.all import *
```

(continues on next page)

(continued from previous page)

```
>>> P = Poset( ( [Integer(1),Integer(2),Integer(3)], [ [Integer(1),
↪Integer(3)], [Integer(2),Integer(3)] ] ) )
>>> list(P.rowmotion_orbit_iter(set([Integer(1), Integer(2)])))
[{1, 2}, {1, 2, 3}, set()]
>>> list(P.rowmotion_orbit_iter([Integer(1), Integer(2)]))
[{1, 2}, {1, 2, 3}, set()]
>>> list(P.rowmotion_orbit_iter([Integer(2), Integer(1)]))
[{1, 2}, {1, 2, 3}, set()]
>>> list(P.rowmotion_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=list))
[[1, 2], [1, 2, 3], []]
>>> list(P.rowmotion_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=frozenset))
[frozenset({1, 2}), frozenset({1, 2, 3}), frozenset()]
>>> list(P.rowmotion_orbit_iter(set([Integer(1), Integer(2)]), element_
↪constructor=tuple))
[(1, 2), (1, 2, 3), ()]

>>> P = Poset( {} )
>>> list(P.rowmotion_orbit_iter([]))
[set()]

>>> P = Poset({ Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)], Integer(4): [] })
>>> Piter = P.rowmotion_orbit_iter([Integer(1), Integer(2), Integer(3)],␣
↪stop=False)
>>> next(Piter)
{1, 2, 3}
>>> next(Piter)
{1, 2, 3, 4}
>>> next(Piter)
set()
>>> next(Piter)
{1}
>>> next(Piter)
{1, 2, 3}

>>> P = Poset({ Integer(1): [Integer(4)], Integer(2): [Integer(4),␣
↪Integer(5)], Integer(3): [Integer(5)] })
>>> list(P.rowmotion_orbit_iter([Integer(1), Integer(2)], element_
↪constructor=list))
[[1, 2], [1, 2, 3, 4], [2, 3, 5], [1], [2, 3], [1, 2, 3, 5], [1, 2, 4],␣
↪[3]]
```

rowmotion_orbits(*element_constructor=<class 'set'>*)

Return the rowmotion orbits of order ideals in `self`.

The rowmotion orbit of an order ideal is its orbit under rowmotion (see *rowmotion()*).

INPUT:

- `element_constructor` – (default: `set`) a type constructor (`set`, `tuple`, `list`, `frozenset`, `iter`, etc.) which is to be applied to the antichains before they are returned

OUTPUT:

- the partition of the set of all order ideals of `self` into orbits under rowmotion. This is returned as a list of lists `L` such that for each `L` and `i`, cyclically: `self.rowmotion(L[i]) == L[i+1]`. The entries `L[i]` are sets by default, but depending on the optional keyword variable `element_constructors` they can also be tuples, lists etc.

EXAMPLES:

```
sage: # needs sage.modules
sage: P = Poset( {1: [2, 3], 2: [], 3: [], 4: [2]} )
sage: sorted(len(o) for o in P.rowmotion_orbits())
[3, 5]
sage: orb = P.rowmotion_orbits(element_constructor=list)
sage: sorted(sorted(e) for e in orb)
[[[], [4, 1], [4, 1, 2, 3]], [[1], [1, 3], [4], [4, 1, 2], [4, 1, 3]]]
sage: orb = P.rowmotion_orbits(element_constructor=tuple)
sage: sorted(sorted(e) for e in orb)
[[(), (4, 1), (4, 1, 2, 3)], [(1,), (1, 3), (4,), (4, 1, 2), (4, 1, 3)]]
sage: P = Poset({})
sage: P.rowmotion_orbits(element_constructor=tuple)
[[()]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> P = Poset( {Integer(1): [Integer(2), Integer(3)], Integer(2): [],␣
↪Integer(3): [], Integer(4): [Integer(2)]} )
>>> sorted(len(o) for o in P.rowmotion_orbits())
[3, 5]
>>> orb = P.rowmotion_orbits(element_constructor=list)
>>> sorted(sorted(e) for e in orb)
[[[], [4, 1], [4, 1, 2, 3]], [[1], [1, 3], [4], [4, 1, 2], [4, 1, 3]]]
>>> orb = P.rowmotion_orbits(element_constructor=tuple)
>>> sorted(sorted(e) for e in orb)
[[(), (4, 1), (4, 1, 2, 3)], [(1,), (1, 3), (4,), (4, 1, 2), (4, 1, 3)]]
>>> P = Poset({})
>>> P.rowmotion_orbits(element_constructor=tuple)
[[()]]
```

rowmotion_orbits_plots()

Return plots of the rowmotion orbits of order ideals in `self`.

The rowmotion orbit of an order ideal is its orbit under rowmotion (see *rowmotion()*).

EXAMPLES:

```
sage: # needs sage.modules sage.plot
sage: P = Poset( {1: [2, 3], 2: [], 3: [], 4: [2]} )
sage: P.rowmotion_orbits_plots()
Graphics Array of size 2 x 5
sage: P = Poset({})
sage: P.rowmotion_orbits_plots()
Graphics Array of size 1 x 1
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.plot
>>> P = Poset( {Integer(1): [Integer(2), Integer(3)], Integer(2): [],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3): [], Integer(4): [Integer(2)]} )
>>> P.rowmotion_orbits_plots()
Graphics Array of size 2 x 5
>>> P = Poset({})
>>> P.rowmotion_orbits_plots()
Graphics Array of size 1 x 1
```

toggling_orbit_iter(*vs*, *oideal*, *element_constructor=<class 'set'>*, *stop=True*, *check=True*)

Iterate over the orbit of an order ideal `oideal` of `self` under the operation of toggling the vertices `vs[0], vs[1], ...` in this order.

See *order_ideal_toggle()* for a definition of toggling.

Warning

The orbit is that under the composition of toggles, *not* under the single toggles themselves. Thus, for example, if `vs == [1,2]`, then the orbit has the form $(I, T_2T_1I, T_2T_1T_2T_1I, \ldots)$ (where I denotes `oideal` and T_i means toggling at i) rather than $(I, T_1I, T_2T_1I, T_1T_2T_1I, \ldots)$.

INPUT:

- `vs` – list (or other iterable) of elements of `self` (but since the output depends on the order, sets should not be used as `vs`).
- `oideal` – an order ideal of `self`, given as an iterable
- `element_constructor` – (default: `set`) a type constructor (`set`, `tuple`, `list`, `frozenset`, `iter`, etc.) which is to be applied to the order ideals before they are yielded.
- `stop` – boolean (default: `True`); whether the iterator should stop once it completes its cycle (this happens when it is set to `True`) or go on forever (this happens when it is set to `False`).
- `check` – boolean (default: `True`); whether `oideal` should be checked for being an order ideal.

OUTPUT:

- an iterator over the orbit of the order ideal `oideal` under toggling the vertices in the list `vs` in this order. This iterator I has the property that `I[0] == oideal` and that every i satisfies `self.order_ideal_toggles(I[i], vs) == I[i+1]`, where `I[i+1]` has to be understood as `I[0]` if it is undefined. The entries `I[i]` are sets by default, but depending on the optional keyword variable `element_constructors` they can also be tuples, lists etc.

EXAMPLES:

```
sage: P = Poset( ( [1,2,3], [ [1,3], [2,3] ] ) )
sage: list(P.toggling_orbit_iter([1, 3, 1], set([1, 2])))
[{1, 2}]
sage: list(P.toggling_orbit_iter([1, 2, 3], set([1, 2])))
[{1, 2}, set(), {1, 2, 3}]
sage: list(P.toggling_orbit_iter([3, 2, 1], set([1, 2])))
[{1, 2}, {1, 2, 3}, set()]
sage: list(P.toggling_orbit_iter([3, 2, 1], set([1, 2]),
....:                            element_constructor=list))
[[1, 2], [1, 2, 3], []]
sage: list(P.toggling_orbit_iter([3, 2, 1], set([1, 2]),
....:                            element_constructor=frozenset))
[frozenset({1, 2}), frozenset({1, 2, 3}), frozenset()]
sage: list(P.toggling_orbit_iter([3, 2, 1], set([1, 2]),
....:                            element_constructor=tuple))
```

(continues on next page)

(continued from previous page)

```
[(1, 2), (1, 2, 3), ()]
sage: list(P.toggling_orbit_iter([3, 2, 1], [2, 1],
....:                            element_constructor=tuple))
[(1, 2), (1, 2, 3), ()]

sage: P = Poset( {} )
sage: list(P.toggling_orbit_iter([], []))
[set()]

sage: P = Poset({ 1: [2, 3], 2: [4], 3: [4], 4: [] })
sage: Piter = P.toggling_orbit_iter([1, 2, 4, 3], [1, 2, 3], stop=False)
sage: next(Piter)
{1, 2, 3}
sage: next(Piter)
{1}
sage: next(Piter)
set()
sage: next(Piter)
{1, 2, 3}
sage: next(Piter)
{1}
```

```
>>> from sage.all import *
>>> P = Poset( ( [Integer(1),Integer(2),Integer(3)], [ [Integer(1),
↪Integer(3)], [Integer(2),Integer(3)] ] ) )
>>> list(P.toggling_orbit_iter([Integer(1), Integer(3), Integer(1)],␣
↪set([Integer(1), Integer(2)])))
[{1, 2}]
>>> list(P.toggling_orbit_iter([Integer(1), Integer(2), Integer(3)],␣
↪set([Integer(1), Integer(2)])))
[{1, 2}, set(), {1, 2, 3}]
>>> list(P.toggling_orbit_iter([Integer(3), Integer(2), Integer(1)],␣
↪set([Integer(1), Integer(2)])))
[{1, 2}, {1, 2, 3}, set()]
>>> list(P.toggling_orbit_iter([Integer(3), Integer(2), Integer(1)],␣
↪set([Integer(1), Integer(2)]),
...                            element_constructor=list))
[[1, 2], [1, 2, 3], []]
>>> list(P.toggling_orbit_iter([Integer(3), Integer(2), Integer(1)],␣
↪set([Integer(1), Integer(2)]),
...                            element_constructor=frozenset))
[frozenset({1, 2}), frozenset({1, 2, 3}), frozenset()]
>>> list(P.toggling_orbit_iter([Integer(3), Integer(2), Integer(1)],␣
↪set([Integer(1), Integer(2)]),
...                            element_constructor=tuple))
[(1, 2), (1, 2, 3), ()]
>>> list(P.toggling_orbit_iter([Integer(3), Integer(2), Integer(1)],␣
↪[Integer(2), Integer(1)],
...                            element_constructor=tuple))
[(1, 2), (1, 2, 3), ()]

>>> P = Poset( {} )
```

(continues on next page)

(continued from previous page)

```
>>> list(P.toggling_orbit_iter([], []))
[set()]

>>> P = Poset({ Integer(1): [Integer(2), Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): [Integer(4)], Integer(4): [] })
>>> Piter = P.toggling_orbit_iter([Integer(1), Integer(2), Integer(4),␣
↪Integer(3)], [Integer(1), Integer(2), Integer(3)], stop=False)
>>> next(Piter)
{1, 2, 3}
>>> next(Piter)
{1}
>>> next(Piter)
set()
>>> next(Piter)
{1, 2, 3}
>>> next(Piter)
{1}
```

toggling_orbits(*vs*, *element_constructor=<class 'set'>*)

Return the orbits of order ideals in `self` under the operation of toggling the vertices `vs[0]`, `vs[1]`, ... in this order.

See *order_ideal_toggle()* for a definition of toggling.

Warning

The orbits are those under the composition of toggles, *not* under the single toggles themselves. Thus, for example, if `vs == [1,2]`, then the orbits have the form $(I, T_2T_1I, T_2T_1T_2T_1I, \ldots)$ (where I denotes an order ideal and T_i means toggling at i) rather than $(I, T_1I, T_2T_1I, T_1T_2T_1I, \ldots)$.

INPUT:

- `vs` – a list (or other iterable) of elements of `self` (but since the output depends on the order, sets should not be used as `vs`).

OUTPUT:

A partition of the order ideals of `self`, as a list of sets `L` such that for each `L` and `i`, cyclically: `self.order_ideal_toggles(L[i], vs) == L[i+1]`.

EXAMPLES:

```
sage: P = Poset( {1: [2, 4], 2: [], 3: [4], 4: []} )
sage: sorted(len(o) for o in P.toggling_orbits([1, 2]))                 #␣
↪needs sage.modules
[2, 3, 3]
sage: P = Poset( {1: [3], 2: [1, 4], 3: [], 4: [3]} )
sage: sorted(len(o) for o in P.toggling_orbits((1, 2, 4, 3)))           #␣
↪needs sage.modules
[3, 3]
```

```
>>> from sage.all import *
>>> P = Poset( {Integer(1): [Integer(2), Integer(4)], Integer(2): [],␣
↪Integer(3): [Integer(4)], Integer(4): []} )
```

(continues on next page)

(continued from previous page)

```
>>> sorted(len(o) for o in P.toggling_orbits([Integer(1), Integer(2)]))  ␣
↪              # needs sage.modules
[2, 3, 3]
>>> P = Poset( {Integer(1): [Integer(3)], Integer(2): [Integer(1),␣
↪Integer(4)], Integer(3): [], Integer(4): [Integer(3)]} )
>>> sorted(len(o) for o in P.toggling_orbits((Integer(1), Integer(2),␣
↪Integer(4), Integer(3))))           # needs sage.modules
[3, 3]
```

toggling_orbits_plots(*vs*)

Return plots of the orbits of order ideals in `self` under the operation of toggling the vertices `vs[0]`, `vs[1]`, `...` in this order.

See *toggling_orbits()* for more information.

EXAMPLES:

```
sage: # needs sage.modules sage.plot
sage: P = Poset( {1: [2, 3], 2: [], 3: [], 4: [2]} )
sage: P.toggling_orbits_plots([1,2,3,4])
Graphics Array of size 2 x 5
sage: P = Poset({})
sage: P.toggling_orbits_plots([])
Graphics Array of size 1 x 1
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.plot
>>> P = Poset( {Integer(1): [Integer(2), Integer(3)], Integer(2): [],␣
↪Integer(3): [], Integer(4): [Integer(2)]} )
>>> P.toggling_orbits_plots([Integer(1),Integer(2),Integer(3),Integer(4)])
Graphics Array of size 2 x 5
>>> P = Poset({})
>>> P.toggling_orbits_plots([])
Graphics Array of size 1 x 1
```

4.68 Finite semigroups

class sage.categories.finite_semigroups.**FiniteSemigroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite (multiplicative) semigroups.

A finite semigroup is a *finite set* endowed with an associative binary operation $*$.

Warning

Finite semigroups in Sage used to be automatically endowed with an *enumerated set* structure; the default enumeration is then obtained by iteratively multiplying the semigroup generators. This forced any finite semigroup to either implement an enumeration, or provide semigroup generators; this was often inconvenient.

Instead, finite semigroups that provide a distinguished finite set of generators with *semigroup_generators()* should now explicitly declare themselves in the category of *finitely generated semigroups*:

```
sage: Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
```

```
>>> from sage.all import *
>>> Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
```

This is a backward incompatible change.

EXAMPLES:

```
sage: C = FiniteSemigroups(); C
Category of finite semigroups
sage: C.super_categories()
[Category of semigroups, Category of finite sets]
sage: sorted(C.axioms())
['Associative', 'Finite']
sage: C.example()
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> C = FiniteSemigroups(); C
Category of finite semigroups
>>> C.super_categories()
[Category of semigroups, Category of finite sets]
>>> sorted(C.axioms())
['Associative', 'Finite']
>>> C.example()
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
```

class ParentMethods

Bases: object

idempotents()

Return the idempotents of the semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('x','y'))
sage: sorted(S.idempotents())
['x', 'xy', 'y', 'yx']
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('x','y'))
>>> sorted(S.idempotents())
['x', 'xy', 'y', 'yx']
```

j_classes()

Return the J-classes of the semigroup.

Two elements u and v of a monoid are in the same J-class if u divides v and v divides u.

OUTPUT:

All the J-classes of self, as a list of lists.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
sage: sorted(map(sorted, S.j_classes()))                                    #␣
↪needs sage.graphs
[['a'], ['ab', 'ba'], ['abc', 'acb', 'bac', 'bca', 'cab', 'cba'],
 ['ac', 'ca'], ['b'], ['bc', 'cb'], ['c']]
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
>>> sorted(map(sorted, S.j_classes()))                                      #␣
↪needs sage.graphs
[['a'], ['ab', 'ba'], ['abc', 'acb', 'bac', 'bca', 'cab', 'cba'],
 ['ac', 'ca'], ['b'], ['bc', 'cb'], ['c']]
```

j_classes_of_idempotents()

Return all the idempotents of self, grouped by J-class.

OUTPUT: list of lists

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
sage: sorted(map(sorted, S.j_classes_of_idempotents()))                     #␣
↪needs sage.graphs
[['a'], ['ab', 'ba'], ['abc', 'acb', 'bac', 'bca', 'cab', 'cba'],
 ['ac', 'ca'], ['b'], ['bc', 'cb'], ['c']]
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
>>> sorted(map(sorted, S.j_classes_of_idempotents()))                       #␣
↪needs sage.graphs
[['a'], ['ab', 'ba'], ['abc', 'acb', 'bac', 'bca', 'cab', 'cba'],
 ['ac', 'ca'], ['b'], ['bc', 'cb'], ['c']]
```

j_transversal_of_idempotents()

Return a list of one idempotent per regular J-class.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('a','b', 'c'))
```

The chosen elements depend on the order of each J-class, and that order is random when using Python 3.

```
sage: sorted(S.j_transversal_of_idempotents())  # random                    #␣
↪needs sage.graphs
['a', 'ab', 'abc', 'ac', 'b', 'c', 'cb']
```

```
>>> from sage.all import *
>>> sorted(S.j_transversal_of_idempotents())  # random                    #␣
↪needs sage.graphs
['a', 'ab', 'abc', 'ac', 'b', 'c', 'cb']
```

4.69 Finite sets

class sage.categories.finite_sets.**FiniteSets**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite sets.

EXAMPLES:

```
sage: C = FiniteSets(); C
Category of finite sets
sage: C.super_categories()
[Category of sets]
sage: C.all_super_categories()
[Category of finite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
sage: C.example()
NotImplemented
```

```
>>> from sage.all import *
>>> C = FiniteSets(); C
Category of finite sets
>>> C.super_categories()
[Category of sets]
>>> C.all_super_categories()
[Category of finite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
>>> C.example()
NotImplemented
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

extra_super_categories()

EXAMPLES:

```
sage: FiniteSets().Algebras(QQ).extra_super_categories()
[Category of finite dimensional vector spaces with basis over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> FiniteSets().Algebras(QQ).extra_super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of finite dimensional vector spaces with basis over Rational␣
↪Field]
```

This implements the fact that the algebra of a finite set is finite dimensional:

```
sage: FiniteMonoids().Algebras(QQ).is_subcategory(AlgebrasWithBasis(QQ).
↪FiniteDimensional())
True
```

```
>>> from sage.all import *
>>> FiniteMonoids().Algebras(QQ).is_subcategory(AlgebrasWithBasis(QQ).
↪FiniteDimensional())
True
```

class ParentMethods

Bases: object

is_finite()

Return True since self is finite.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.is_finite()
True
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C.is_finite()
True
```

class SubcategoryMethods

Bases: object

Infinite()

Incompatible axiom.

EXAMPLES:

```
sage: P = Posets().Finite()
sage: P.Infinite()
Traceback (most recent call last):
...
TypeError: incompatible axioms: finite and infinite
```

```
>>> from sage.all import *
>>> P = Posets().Finite()
>>> P.Infinite()
Traceback (most recent call last):
...
TypeError: incompatible axioms: finite and infinite
```

class Subquotients(*category*, **args*)

Bases: *SubquotientsCategory*

extra_super_categories()

EXAMPLES:

```
sage: FiniteSets().Subquotients().extra_super_categories()
[Category of finite sets]
```

```
>>> from sage.all import *
>>> FiniteSets().Subquotients().extra_super_categories()
[Category of finite sets]
```

This implements the fact that a subquotient (and therefore a quotient or subobject) of a finite set is finite:

```
sage: FiniteSets().Subquotients().is_subcategory(FiniteSets())
True
sage: FiniteSets().Quotients    ().is_subcategory(FiniteSets())
True
sage: FiniteSets().Subobjects   ().is_subcategory(FiniteSets())
True
```

```
>>> from sage.all import *
>>> FiniteSets().Subquotients().is_subcategory(FiniteSets())
True
>>> FiniteSets().Quotients    ().is_subcategory(FiniteSets())
True
>>> FiniteSets().Subobjects   ().is_subcategory(FiniteSets())
True
```

4.70 Finite Weyl Groups

class sage.categories.finite_weyl_groups.**FiniteWeylGroups**(*base_category*)

Bases: *CategoryWithAxiom*

The category of finite Weyl groups.

EXAMPLES:

```
sage: C = FiniteWeylGroups()
sage: C
Category of finite Weyl groups
sage: C.super_categories()
[Category of finite Coxeter groups, Category of Weyl groups]
sage: C.example()
The symmetric group on {0, ..., 3}
```

```
>>> from sage.all import *
>>> C = FiniteWeylGroups()
>>> C
Category of finite Weyl groups
>>> C.super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of finite Coxeter groups, Category of Weyl groups]
>>> C.example()
The symmetric group on {0, ..., 3}
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

4.71 Finitely Generated Lambda bracket Algebras

AUTHORS:

- Reimundo Heluani (2020-08-21): Initial implementation.

class sage.categories.finitely_generated_lambda_bracket_algebras.**FinitelyGeneratedLambdaBracketAlgebr**

Bases: *CategoryWithAxiom_over_base_ring*

The category of finitely generated lambda bracket algebras.

EXAMPLES:

```
sage: from sage.categories.lambda_bracket_algebras import LambdaBracketAlgebras
sage: LambdaBracketAlgebras(QQbar).FinitelyGenerated()                          #␣
↪needs sage.rings.number_field
Category of finitely generated lambda bracket algebras over Algebraic Field
```

```
>>> from sage.all import *
>>> from sage.categories.lambda_bracket_algebras import LambdaBracketAlgebras
>>> LambdaBracketAlgebras(QQbar).FinitelyGenerated()                            #␣
↪needs sage.rings.number_field
Category of finitely generated lambda bracket algebras over Algebraic Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded finitely generated Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).FinitelyGenerated().Graded()                  #␣
↪needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).FinitelyGenerated().Graded()                    #␣
↪needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras
 over Algebraic Field
```

class ParentMethods

Bases: object

gen(*i*)

The i-th generator of this Lie conformal algebra.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: V = lie_conformal_algebras.Affine(QQ, 'A1')
sage: V.gens()
(B[alpha[1]], B[alphacheck[1]], B[-alpha[1]], B['K'])
sage: V.gen(0)
B[alpha[1]]
sage: V.1
B[alphacheck[1]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> V = lie_conformal_algebras.Affine(QQ, 'A1')
>>> V.gens()
(B[alpha[1]], B[alphacheck[1]], B[-alpha[1]], B['K'])
>>> V.gen(Integer(0))
B[alpha[1]]
>>> V.gen(1)
B[alphacheck[1]]
```

ngens()

The number of generators of this Lie conformal algebra.

EXAMPLES:

```
sage: Vir = lie_conformal_algebras.Virasoro(QQ)                    #␣
↪needs sage.combinat sage.modules
sage: Vir.ngens()                                                  #␣
↪needs sage.combinat sage.modules
2

sage: V = lie_conformal_algebras.Affine(QQ, 'A2')                  #␣
↪needs sage.combinat sage.modules
sage: V.ngens()                                                    #␣
↪needs sage.combinat sage.modules
9
```

```
>>> from sage.all import *
>>> Vir = lie_conformal_algebras.Virasoro(QQ)                      #␣
↪needs sage.combinat sage.modules
>>> Vir.ngens()                                                    #␣
↪needs sage.combinat sage.modules
2

>>> V = lie_conformal_algebras.Affine(QQ, 'A2')                    #␣
↪needs sage.combinat sage.modules
>>> V.ngens()                                                      #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
9
```

some_elements()

Some elements of this Lie conformal algebra.

This method returns a list with elements containing at least the generators.

EXAMPLES:

```
sage: V = lie_conformal_algebras.Affine(QQ, 'A1',                          #␣
↪needs sage.combinat sage.modules
....:                                     names=('e', 'h', 'f'))
sage: V.some_elements()                                                    #␣
↪needs sage.combinat sage.modules
[e, h, f, K, ...]
sage: all(v.parent() is V for v in V.some_elements())                      #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> V = lie_conformal_algebras.Affine(QQ, 'A1',                            #␣
↪needs sage.combinat sage.modules
...                                     names=('e', 'h', 'f'))
>>> V.some_elements()                                                      #␣
↪needs sage.combinat sage.modules
[e, h, f, K, ...]
>>> all(v.parent() is V for v in V.some_elements())                        #␣
↪needs sage.combinat sage.modules
True
```

4.72 Finitely Generated Lie Conformal Algebras

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.finitely_generated_lie_conformal_algebras.**FinitelyGeneratedLieConformalAlgebras**

Bases: *CategoryWithAxiom_over_base_ring*

The category of finitely generated Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).FinitelyGenerated()                      #␣
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).FinitelyGenerated()                        #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras over Algebraic Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded finitely generated Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).FinitelyGenerated().Graded()                #␣
↪needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).FinitelyGenerated().Graded()                  #␣
↪needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras
 over Algebraic Field
```

class ParentMethods

Bases: object

some_elements()

Some elements of this Lie conformal algebra.

Returns a list with elements containing at least the generators.

EXAMPLES:

```
sage: V = lie_conformal_algebras.Affine(QQ, 'A1',                             #␣
↪needs sage.combinat sage.modules
....:                                   names=('e', 'h', 'f'))
sage: V.some_elements()                                                       #␣
↪needs sage.combinat sage.modules
[e, h, f, K, ...]
sage: all(v.parent() is V for v in V.some_elements())                         #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> V = lie_conformal_algebras.Affine(QQ, 'A1',                               #␣
↪needs sage.combinat sage.modules
...                                   names=('e', 'h', 'f'))
>>> V.some_elements()                                                         #␣
↪needs sage.combinat sage.modules
[e, h, f, K, ...]
>>> all(v.parent() is V for v in V.some_elements())                           #␣
↪needs sage.combinat sage.modules
True
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

The category of super finitely generated Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(AA).FinitelyGenerated().Super()                  #␣
↪needs sage.rings.number_field
Category of super finitely generated Lie conformal algebras
 over Algebraic Real Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(AA).FinitelyGenerated().Super()                    #␣
↪needs sage.rings.number_field
Category of super finitely generated Lie conformal algebras
 over Algebraic Real Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded super finitely generated Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).FinitelyGenerated().Super().Graded()  #␣
↪needs sage.rings.number_field
Category of H-graded super finitely generated Lie conformal algebras
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).FinitelyGenerated().Super().Graded()  #␣
↪needs sage.rings.number_field
Category of H-graded super finitely generated Lie conformal algebras
 over Algebraic Field
```

4.73 Finitely generated magmas

class sage.categories.finitely_generated_magmas.**FinitelyGeneratedMagmas**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finitely generated (multiplicative) magmas.

See *Magmas.SubcategoryMethods.FinitelyGeneratedAsMagma()* for details.

EXAMPLES:

```
sage: C = Magmas().FinitelyGeneratedAsMagma(); C
Category of finitely generated magmas
sage: C.super_categories()
[Category of magmas]
sage: sorted(C.axioms())
['FinitelyGeneratedAsMagma']
```

```
>>> from sage.all import *
>>> C = Magmas().FinitelyGeneratedAsMagma(); C
Category of finitely generated magmas
```

(continues on next page)

(continued from previous page)

```
>>> C.super_categories()
[Category of magmas]
>>> sorted(C.axioms())
['FinitelyGeneratedAsMagma']
```

class ParentMethods

Bases: object

magma_generators()

Return distinguished magma generators for self.

OUTPUT: a finite family

This method should be implemented by all *finitely generated magmas*.

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: S.magma_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S.magma_generators()
Family ('a', 'b', 'c', 'd')
```

4.74 Finitely generated semigroups

class sage.categories.finitely_generated_semigroups.**FinitelyGeneratedSemigroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finitely generated (multiplicative) semigroups.

A *finitely generated semigroup* is a *semigroup* endowed with a distinguished finite set of generators (see *FinitelyGeneratedSemigroups.ParentMethods.semigroup_generators()*). This makes it into an *enumerated set*.

EXAMPLES:

```
sage: C = Semigroups().FinitelyGenerated(); C
Category of finitely generated semigroups
sage: C.super_categories()
[Category of semigroups,
 Category of finitely generated magmas,
 Category of enumerated sets]
sage: sorted(C.axioms())
['Associative', 'Enumerated', 'FinitelyGeneratedAsMagma']
sage: C.example()
An example of a semigroup: the free semigroup generated
by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> C = Semigroups().FinitelyGenerated(); C
Category of finitely generated semigroups
>>> C.super_categories()
[Category of semigroups,
 Category of finitely generated magmas,
 Category of enumerated sets]
>>> sorted(C.axioms())
['Associative', 'Enumerated', 'FinitelyGeneratedAsMagma']
>>> C.example()
An example of a semigroup: the free semigroup generated
by ('a', 'b', 'c', 'd')
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class ParentMethods

Bases: object

some_elements()

Return an iterable containing some elements of the semigroup.

OUTPUT: the ten first elements of the semigroup, if they exist

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('x','y'))
sage: sorted(S.some_elements())
['x', 'xy', 'y', 'yx']
sage: S = FiniteSemigroups().example(alphabet=('x','y','z'))
sage: X = S.some_elements()
sage: len(X)
10
sage: all(x in S for x in X)
True
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('x','y'))
>>> sorted(S.some_elements())
['x', 'xy', 'y', 'yx']
>>> S = FiniteSemigroups().example(alphabet=('x','y','z'))
>>> X = S.some_elements()
>>> len(X)
10
>>> all(x in S for x in X)
True
```

class ParentMethods

Bases: object

ideal(*gens*, *side='twosided'*)

Return the side-sided ideal generated by gens.

This brute force implementation recursively multiplies the elements of gens by the distinguished generators of this semigroup.

See also

semigroup_generators()

INPUT:

- `gens` – list (or iterable) of elements of `self`
- `side` – [default: "twosided"] "left", "right" or "twosided"

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: sorted(S.ideal([S('cab')], side='left'))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
sage: list(S.ideal([S('cab')], side='right'))
['cab', 'cabd']
sage: sorted(S.ideal([S('cab')], side='twosided'))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
sage: sorted(S.ideal([S('cab')]))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> sorted(S.ideal([S('cab')], side='left'))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
>>> list(S.ideal([S('cab')], side='right'))
['cab', 'cabd']
>>> sorted(S.ideal([S('cab')], side='twosided'))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
>>> sorted(S.ideal([S('cab')]))
['abc', 'abcd', 'abdc', 'acb', 'acbd', 'acdb', 'adbc',
 'adcb', 'bac', 'bacd', 'badc', 'bca', 'bcad', 'bcda',
 'bdac', 'bdca', 'cab', 'cabd', 'cadb', 'cba', 'cbad',
 'cbda', 'cdab', 'cdba', 'dabc', 'dacb', 'dbac', 'dbca',
 'dcab', 'dcba']
```

semigroup_generators()

Return distinguished semigroup generators for `self`.

OUTPUT: a finite family

This method should be implemented by all semigroups in *FinitelyGeneratedSemigroups*.

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

succ_generators(*side='twosided'*)

Return the successor function of the `side`-sided Cayley graph of `self`.

This is a function that maps an element of `self` to all the products of `x` by a generator of this semigroup, where the product is taken on the left, right, or both sides.

INPUT:

- `side` – `'left'`, `'right'`, or `'twosided'` (default)

Todo

Design choice:

- find a better name for this method
- should we return a set? a family?

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: S.succ_generators("left" )(S('ca'))
('ac', 'bca', 'ca', 'dca')
sage: S.succ_generators("right")(S('ca'))
('ca', 'cab', 'ca', 'cad')
sage: S.succ_generators("twosided" )(S('ca'))
('ac', 'bca', 'ca', 'dca', 'ca', 'cab', 'ca', 'cad')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S.succ_generators("left" )(S('ca'))
('ac', 'bca', 'ca', 'dca')
>>> S.succ_generators("right")(S('ca'))
('ca', 'cab', 'ca', 'cad')
>>> S.succ_generators("twosided" )(S('ca'))
('ac', 'bca', 'ca', 'dca', 'ca', 'cab', 'ca', 'cad')
```

example()

EXAMPLES:

```
sage: Semigroups().FinitelyGenerated().example()
An example of a semigroup: the free semigroup generated
by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> Semigroups().FinitelyGenerated().example()
An example of a semigroup: the free semigroup generated
by ('a', 'b', 'c', 'd')
```

extra_super_categories()

State that a finitely generated semigroup is endowed with a default enumeration.

EXAMPLES:

```
sage: Semigroups().FinitelyGenerated().extra_super_categories()
[Category of enumerated sets]
```

```
>>> from sage.all import *
>>> Semigroups().FinitelyGenerated().extra_super_categories()
[Category of enumerated sets]
```

4.75 Function fields

class sage.categories.function_fields.**FunctionFields**

Bases: *Category*

The category of function fields.

EXAMPLES:

We create the category of function fields:

```
sage: C = FunctionFields()
sage: C
Category of function fields
```

```
>>> from sage.all import *
>>> C = FunctionFields()
>>> C
Category of function fields
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

super_categories()

Return the Category of which this is a direct sub-Category For a list off all super categories see all_super_categories

EXAMPLES:

```
sage: FunctionFields().super_categories()
[Category of fields]
```

```
>>> from sage.all import *
>>> FunctionFields().super_categories()
[Category of fields]
```

4.76 G-Sets

class sage.categories.g_sets.**GSets**(G)

Bases: *Category*

The category of G-sets, for a group G.

EXAMPLES:

```
sage: S = SymmetricGroup(3)                                                 #␣
↪needs sage.groups
sage: GSets(S)                                                              #␣
↪needs sage.groups
Category of G-sets for Symmetric group of order 3! as a permutation group
```

```
>>> from sage.all import *
>>> S = SymmetricGroup(Integer(3))                                           ␣
↪     # needs sage.groups
>>> GSets(S)                                                               #␣
↪needs sage.groups
Category of G-sets for Symmetric group of order 3! as a permutation group
```

TODO: should this derive from Category_over_base?

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: GSets.an_instance()  # indirect doctest                               #␣
↪needs sage.groups
Category of G-sets for Symmetric group of order 8! as a permutation group
```

```
>>> from sage.all import *
>>> GSets.an_instance()  # indirect doctest                                 #␣
↪needs sage.groups
Category of G-sets for Symmetric group of order 8! as a permutation group
```

super_categories()

EXAMPLES:

```
sage: GSets(SymmetricGroup(8)).super_categories()                           #␣
↪needs sage.groups
[Category of sets]
```

```
>>> from sage.all import *
>>> GSets(SymmetricGroup(Integer(8))).super_categories()
↪     # needs sage.groups
[Category of sets]
```

4.77 Gcd domains

class sage.categories.gcd_domains.**GcdDomains**

Bases: *Category_singleton*

The category of gcd domains domains where gcd can be computed but where there is no guarantee of factorisation into irreducibles

EXAMPLES:

```
sage: GcdDomains()
Category of gcd domains
sage: GcdDomains().super_categories()
[Category of integral domains]
```

```
>>> from sage.all import *
>>> GcdDomains()
Category of gcd domains
>>> GcdDomains().super_categories()
[Category of integral domains]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

additional_structure()

Return None.

Indeed, the category of gcd domains defines no additional structure: a ring morphism between two gcd domains is a gcd domain morphism.

See also

Category.additional_structure()

EXAMPLES:

```
sage: GcdDomains().additional_structure()
```

```
>>> from sage.all import *
>>> GcdDomains().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: GcdDomains().super_categories()
[Category of integral domains]
```

```
>>> from sage.all import *
>>> GcdDomains().super_categories()
[Category of integral domains]
```

4.78 Generalized Coxeter Groups

class sage.categories.generalized_coxeter_groups.**GeneralizedCoxeterGroups**

Bases: *Category_singleton*

The category of generalized Coxeter groups.

A generalized Coxeter group is a group with a presentation of the following form:

$$\langle s_i \mid s_i^{p_i}, s_i s_j \cdots = s_j s_i \cdots \rangle,$$

where $p_i > 1$, $i \in I$, and the factors in the braid relation occur $m_{ij} = m_{ji}$ times for all $i \neq j \in I$.

EXAMPLES:

```
sage: from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
sage: C = GeneralizedCoxeterGroups(); C
Category of generalized Coxeter groups
```

```
>>> from sage.all import *
>>> from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
>>> C = GeneralizedCoxeterGroups(); C
Category of generalized Coxeter groups
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of finite generalized Coxeter groups.

extra_super_categories()

Implement that a finite generalized Coxeter group is a well-generated complex reflection group.

EXAMPLES:

```
sage: from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
sage: from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups

sage: Cat = GeneralizedCoxeterGroups().Finite()
sage: Cat.extra_super_categories()
[Category of well generated finite complex reflection groups]
sage: Cat.is_subcategory(ComplexReflectionGroups().Finite().
↪WellGenerated())
True
```

```
>>> from sage.all import *
>>> from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
>>> from sage.categories.complex_reflection_groups import␣
↪ComplexReflectionGroups

>>> Cat = GeneralizedCoxeterGroups().Finite()
>>> Cat.extra_super_categories()
[Category of well generated finite complex reflection groups]
>>> Cat.is_subcategory(ComplexReflectionGroups().Finite().WellGenerated())
True
```

additional_structure()

Return `None`.

Indeed, all the structure generalized Coxeter groups have in addition to groups (simple reflections, …) is already defined in the super category.

See also

Category.additional_structure()

EXAMPLES:

```
sage: from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
sage: GeneralizedCoxeterGroups().additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
>>> GeneralizedCoxeterGroups().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
sage: GeneralizedCoxeterGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

```
>>> from sage.all import *
>>> from sage.categories.generalized_coxeter_groups import␣
↪GeneralizedCoxeterGroups
>>> GeneralizedCoxeterGroups().super_categories()
[Category of complex reflection or generalized Coxeter groups]
```

4.79 Graded Algebras

class sage.categories.graded_algebras.**GradedAlgebras**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded algebras.

EXAMPLES:

```
sage: GradedAlgebras(ZZ)
Category of graded algebras over Integer Ring
sage: GradedAlgebras(ZZ).super_categories()
[Category of filtered algebras over Integer Ring,
 Category of graded modules over Integer Ring]
```

```
>>> from sage.all import *
>>> GradedAlgebras(ZZ)
Category of graded algebras over Integer Ring
>>> GradedAlgebras(ZZ).super_categories()
[Category of filtered algebras over Integer Ring,
 Category of graded modules over Integer Ring]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

graded_algebra()

Return the associated graded algebra to self.

Since self is already graded, this just returns self.

EXAMPLES:

```
sage: m = SymmetricFunctions(QQ).m()                                    #␣
↪needs sage.combinat sage.modules
sage: m.graded_algebra() is m                                           #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> m = SymmetricFunctions(QQ).m()                                      #␣
↪needs sage.combinat sage.modules
>>> m.graded_algebra() is m                                             #␣
↪needs sage.combinat sage.modules
True
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

extra_super_categories()

EXAMPLES:

```
sage: Algebras(QQ).Graded().SignedTensorProducts().extra_super_
↪categories()
[Category of graded algebras over Rational Field]
sage: Algebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded algebras over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).Graded().SignedTensorProducts().extra_super_categories()
[Category of graded algebras over Rational Field]
>>> Algebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded algebras over Rational Field]
```

Meaning: a signed tensor product of algebras is an algebra

class SubcategoryMethods

Bases: object

SignedTensorProducts()

Return the full subcategory of objects of self constructed as signed tensor products.

See also

- *SignedTensorProductsCategory*
- *CovariantFunctorialConstruction*

EXAMPLES:

```
sage: AlgebrasWithBasis(QQ).Graded().SignedTensorProducts()
Category of signed tensor products of graded algebras with basis
 over Rational Field
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).Graded().SignedTensorProducts()
Category of signed tensor products of graded algebras with basis
 over Rational Field
```

4.80 Graded algebras with basis

class sage.categories.graded_algebras_with_basis.**GradedAlgebrasWithBasis**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded algebras with a distinguished basis.

EXAMPLES:

```
sage: C = GradedAlgebrasWithBasis(ZZ); C
Category of graded algebras with basis over Integer Ring
sage: sorted(C.super_categories(), key=str)
[Category of filtered algebras with basis over Integer Ring,
 Category of graded algebras over Integer Ring,
 Category of graded modules with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> C = GradedAlgebrasWithBasis(ZZ); C
Category of graded algebras with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of filtered algebras with basis over Integer Ring,
 Category of graded algebras over Integer Ring,
 Category of graded modules with basis over Integer Ring]
```

class ElementMethods

Bases: object

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: object

top_degree()

Return the top degree of the finite dimensional graded algebra.

EXAMPLES:

```
sage: ch = matroids.Uniform(4,6).chow_ring(QQ, False)
sage: ch.top_degree()
3
sage: ch = matroids.Wheel(3).chow_ring(QQ, True, 'atom-free')
sage: ch.top_degree()
3
```

```
>>> from sage.all import *
>>> ch = matroids.Uniform(Integer(4),Integer(6)).chow_ring(QQ, False)
>>> ch.top_degree()
3
>>> ch = matroids.Wheel(Integer(3)).chow_ring(QQ, True, 'atom-free')
>>> ch.top_degree()
3
```

class ParentMethods

Bases: object

completion()

alias of *formal_series_ring()*.

formal_series_ring()

Return the completion of all formal linear combinations of `self` with finite linear combinations in each homogeneous degree (computed lazily).

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: NCSF = NonCommutativeSymmetricFunctions(QQ)
sage: S = NCSF.Complete()
sage: L = S.formal_series_ring()
sage: L
```

(continues on next page)

(continued from previous page)

```
Lazy completion of Non-Commutative Symmetric Functions over
 the Rational Field in the Complete basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> NCSF = NonCommutativeSymmetricFunctions(QQ)
>>> S = NCSF.Complete()
>>> L = S.formal_series_ring()
>>> L
Lazy completion of Non-Commutative Symmetric Functions over
 the Rational Field in the Complete basis
```

free_graded_module(*generator_degrees*, *names=None*)

Create a finitely generated free graded module over `self`.

INPUT:

- `generator_degrees` – tuple of integers defining the number of generators of the module and their degrees
- `names` – (optional) the names of the generators. If `names` is a comma-separated string like `'a, b, c'`, then those will be the names. Otherwise, for example if `names` is `abc`, then the names will be `abc[d,i]`.

By default, if all generators are in distinct degrees, then the `names` of the generators will have the form `g[d]` where `d` is the degree of the generator. If the degrees are not distinct, then the generators will be called `g[d,i]` where `d` is the degree and `i` is its index in the list of generators in that degree.

See `sage.modules.fp_graded.free_module` for more examples and details.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: Q = QuadraticForm(QQ, 3, [1,2,3,4,5,6])
sage: Cl = CliffordAlgebra(Q)
sage: M = Cl.free_graded_module((0, 2, 3))
sage: M.gens()
(g[0], g[2], g[3])
sage: N.<xy, z> = Cl.free_graded_module((1, 2))
sage: N.generators()
(xy, z)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Q = QuadraticForm(QQ, Integer(3), [Integer(1),Integer(2),Integer(3),
↪Integer(4),Integer(5),Integer(6)])
>>> Cl = CliffordAlgebra(Q)
>>> M = Cl.free_graded_module((Integer(0), Integer(2), Integer(3)))
>>> M.gens()
(g[0], g[2], g[3])
>>> N = Cl.free_graded_module((Integer(1), Integer(2)), names=('xy', 'z',
↪)); (xy, z,) = N._first_ngens(2)
>>> N.generators()
(xy, z)
```

graded_algebra()

Return the associated graded algebra to `self`.

This is `self`, because `self` is already graded. See *graded_algebra()* for the general behavior of this method, and see `AssociatedGradedAlgebra` for the definition and properties of associated graded algebras.

EXAMPLES:

```
sage: m = SymmetricFunctions(QQ).m()                                    #␣
↪needs sage.combinat sage.modules
sage: m.graded_algebra() is m                                           #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> m = SymmetricFunctions(QQ).m()                                      #␣
↪needs sage.combinat sage.modules
>>> m.graded_algebra() is m                                             #␣
↪needs sage.combinat sage.modules
True
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

The category of algebras with basis constructed by signed tensor product of algebras with basis.

class ParentMethods

Bases: `object`

Implement operations on tensor products of super algebras with basis.

one_basis()

Return the index of the one of this signed tensor product of algebras, as per `AlgebrasWithBasis.ParentMethods.one_basis`.

It is the tuple whose operands are the indices of the ones of the operands, as returned by their *one_basis()* methods.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A.<x,y> = ExteriorAlgebra(QQ)
sage: A.one_basis()
0
sage: B = tensor((A, A, A))
sage: B.one_basis()
(0, 0, 0)
sage: B.one()
1 # 1 # 1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = A._first_
↪ngens(2)
>>> A.one_basis()
0
>>> B = tensor((A, A, A))
>>> B.one_basis()
```

(continues on next page)

(continued from previous page)

```
(0, 0, 0)
>>> B.one()
1 # 1 # 1
```

product_on_basis(*t0*, *t1*)

The product of the algebra on the basis, as per `AlgebrasWithBasis.ParentMethods.product_on_basis`.

EXAMPLES:

Test the sign in the super tensor product:

```
sage: # needs sage.combinat sage.modules
sage: A = SteenrodAlgebra(3)
sage: x = A.Q(0)
sage: y = x.coproduct()
sage: y^2
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = SteenrodAlgebra(Integer(3))
>>> x = A.Q(Integer(0))
>>> y = x.coproduct()
>>> y**Integer(2)
0
```

TODO: optimize this implementation!

extra_super_categories()

EXAMPLES:

```
sage: Cat = AlgebrasWithBasis(QQ).Graded()
sage: Cat.SignedTensorProducts().extra_super_categories()
[Category of graded algebras with basis over Rational Field]
sage: Cat.SignedTensorProducts().super_categories()
[Category of graded algebras with basis over Rational Field,
 Category of signed tensor products of graded algebras over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> Cat = AlgebrasWithBasis(QQ).Graded()
>>> Cat.SignedTensorProducts().extra_super_categories()
[Category of graded algebras with basis over Rational Field]
>>> Cat.SignedTensorProducts().super_categories()
[Category of graded algebras with basis over Rational Field,
 Category of signed tensor products of graded algebras over Rational␣
↪Field]
```

4.81 Graded bialgebras

sage.categories.graded_bialgebras.**GradedBialgebras**(*base_ring*)

The category of graded bialgebras.

EXAMPLES:

```
sage: C = GradedBialgebras(QQ); C
Join of Category of graded algebras over Rational Field
    and Category of bialgebras over Rational Field
    and Category of graded coalgebras over Rational Field
sage: C is Bialgebras(QQ).Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedBialgebras(QQ); C
Join of Category of graded algebras over Rational Field
    and Category of bialgebras over Rational Field
    and Category of graded coalgebras over Rational Field
>>> C is Bialgebras(QQ).Graded()
True
```

4.82 Graded bialgebras with basis

sage.categories.graded_bialgebras_with_basis.**GradedBialgebrasWithBasis**(*base_ring*)

The category of graded bialgebras with a distinguished basis.

EXAMPLES:

```
sage: C = GradedBialgebrasWithBasis(QQ); C
Join of Category of ...
sage: sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of graded algebras with basis over Rational Field,
 Category of graded coalgebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = GradedBialgebrasWithBasis(QQ); C
Join of Category of ...
>>> sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of graded algebras with basis over Rational Field,
 Category of graded coalgebras with basis over Rational Field]
```

4.83 Graded Coalgebras

class sage.categories.graded_coalgebras.**GradedCoalgebras**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded coalgebras.

EXAMPLES:

```
sage: C = GradedCoalgebras(QQ); C
Category of graded coalgebras over Rational Field
sage: C is Coalgebras(QQ).Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedCoalgebras(QQ); C
Category of graded coalgebras over Rational Field
>>> C is Coalgebras(QQ).Graded()
True
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

extra_super_categories()

EXAMPLES:

```
sage: Coalgebras(QQ).Graded().SignedTensorProducts().extra_super_
↪categories()
[Category of graded coalgebras over Rational Field]
sage: Coalgebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> Coalgebras(QQ).Graded().SignedTensorProducts().extra_super_
↪categories()
[Category of graded coalgebras over Rational Field]
>>> Coalgebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded coalgebras over Rational Field]
```

Meaning: a signed tensor product of coalgebras is a coalgebra

class SubcategoryMethods

Bases: `object`

SignedTensorProducts()

Return the full subcategory of objects of `self` constructed as signed tensor products.

See also

- *SignedTensorProductsCategory*
- *CovariantFunctorialConstruction*

EXAMPLES:

```
sage: CoalgebrasWithBasis(QQ).Graded().SignedTensorProducts()
Category of signed tensor products of graded coalgebras with basis
 over Rational Field
```

```
>>> from sage.all import *
>>> CoalgebrasWithBasis(QQ).Graded().SignedTensorProducts()
```

(continues on next page)

(continued from previous page)

```
Category of signed tensor products of graded coalgebras with basis
 over Rational Field
```

4.84 Graded coalgebras with basis

class sage.categories.graded_coalgebras_with_basis.**GradedCoalgebrasWithBasis**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded coalgebras with a distinguished basis.

EXAMPLES:

```
sage: C = GradedCoalgebrasWithBasis(QQ); C
Category of graded coalgebras with basis over Rational Field
sage: C is Coalgebras(QQ).WithBasis().Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedCoalgebrasWithBasis(QQ); C
Category of graded coalgebras with basis over Rational Field
>>> C is Coalgebras(QQ).WithBasis().Graded()
True
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

The category of coalgebras with basis constructed by signed tensor product of coalgebras with basis.

extra_super_categories()

EXAMPLES:

```
sage: Cat = CoalgebrasWithBasis(QQ).Graded()
sage: Cat.SignedTensorProducts().extra_super_categories()
[Category of graded coalgebras with basis over Rational Field]
sage: Cat.SignedTensorProducts().super_categories()
[Category of graded coalgebras with basis over Rational Field,
 Category of signed tensor products of graded coalgebras over Rational␣
↪Field]
```

```
>>> from sage.all import *
>>> Cat = CoalgebrasWithBasis(QQ).Graded()
>>> Cat.SignedTensorProducts().extra_super_categories()
[Category of graded coalgebras with basis over Rational Field]
>>> Cat.SignedTensorProducts().super_categories()
[Category of graded coalgebras with basis over Rational Field,
 Category of signed tensor products of graded coalgebras over Rational␣
↪Field]
```

4.85 Graded Hopf algebras

sage.categories.graded_hopf_algebras.**GradedHopfAlgebras**(*base_ring*)

The category of graded Hopf algebras.

EXAMPLES:

```
sage: C = GradedHopfAlgebras(QQ); C
Join of Category of Hopf algebras over Rational Field
    and Category of graded algebras over Rational Field
    and Category of graded coalgebras over Rational Field
sage: C is HopfAlgebras(QQ).Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedHopfAlgebras(QQ); C
Join of Category of Hopf algebras over Rational Field
    and Category of graded algebras over Rational Field
    and Category of graded coalgebras over Rational Field
>>> C is HopfAlgebras(QQ).Graded()
True
```

Note

This is not a graded Hopf algebra as is typically defined in algebraic topology as the product in the tensor square $(x \otimes y)(a \otimes b) = (xa) \otimes (yb)$ does not carry an additional sign. For this, instead use *super Hopf algebras*.

4.86 Graded Hopf algebras with basis

class sage.categories.graded_hopf_algebras_with_basis.**GradedHopfAlgebrasWithBasis**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded Hopf algebras with a distinguished basis.

EXAMPLES:

```
sage: C = GradedHopfAlgebrasWithBasis(ZZ); C
Category of graded Hopf algebras with basis over Integer Ring
sage: C.super_categories()
[Category of filtered Hopf algebras with basis over Integer Ring,
 Category of graded algebras with basis over Integer Ring,
 Category of graded coalgebras with basis over Integer Ring]

sage: C is HopfAlgebras(ZZ).WithBasis().Graded()
True
sage: C is HopfAlgebras(ZZ).Graded().WithBasis()
False
```

```
>>> from sage.all import *
>>> C = GradedHopfAlgebrasWithBasis(ZZ); C
```

(continues on next page)

(continued from previous page)

```
Category of graded Hopf algebras with basis over Integer Ring
>>> C.super_categories()
[Category of filtered Hopf algebras with basis over Integer Ring,
 Category of graded algebras with basis over Integer Ring,
 Category of graded coalgebras with basis over Integer Ring]

>>> C is HopfAlgebras(ZZ).WithBasis().Graded()
True
>>> C is HopfAlgebras(ZZ).Graded().WithBasis()
False
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ElementMethods

Bases: object

class ParentMethods

Bases: object

antipode_on_basis(*index*)

The antipode on the basis element indexed by index.

INPUT:

- index – an element of the index set

For a graded connected Hopf algebra, we can define an antipode recursively by

$$S(x) := -\sum_{x^L \neq x} S(x^L) \times x^R$$

when $|x| > 0$, and by $S(x) = x$ when $|x| = 0$.

counit_on_basis(*i*)

The default counit of a graded connected Hopf algebra.

INPUT:

- i – an element of the index set

OUTPUT: an element of the base ring

$$c(i) := \begin{cases} 1 & \text{if } i \text{ indexes the 1 of the algebra} \\ 0 & \text{otherwise.} \end{cases}$$

EXAMPLES:

```
sage: H = GradedHopfAlgebrasWithBasis(QQ).Connected().example()      #␣
↪needs sage.modules
sage: H.monomial(4).counit()  # indirect doctest                    #␣
↪needs sage.modules
0
sage: H.monomial(0).counit()  # indirect doctest                    #␣
↪needs sage.modules
1
```

```
>>> from sage.all import *
>>> H = GradedHopfAlgebrasWithBasis(QQ).Connected().example()      #␣
↪needs sage.modules
>>> H.monomial(Integer(4)).counit()  # indirect doctest                ␣
↪    # needs sage.modules
0
>>> H.monomial(Integer(0)).counit()  # indirect doctest                ␣
↪    # needs sage.modules
1
```

example()

Return an example of a graded connected Hopf algebra with a distinguished basis.

class ElementMethods

Bases: object

class ParentMethods

Bases: object

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

super_categories()

EXAMPLES:

```
sage: GradedHopfAlgebrasWithBasis(QQ).WithRealizations().super_
↪categories()
[Join of Category of Hopf algebras over Rational Field
 and Category of graded algebras over Rational Field
 and Category of graded coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> GradedHopfAlgebrasWithBasis(QQ).WithRealizations().super_categories()
[Join of Category of Hopf algebras over Rational Field
 and Category of graded algebras over Rational Field
 and Category of graded coalgebras over Rational Field]
```

example()

Return an example of a graded Hopf algebra with a distinguished basis.

4.87 Graded Lie Algebras

AUTHORS:

- Eero Hakavuori (2018-08-16): initial version

class sage.categories.graded_lie_algebras.**GradedLieAlgebras**(*base_category*)

Bases: *GradedModulesCategory*

Category of graded Lie algebras.

class Stratified(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of stratified Lie algebras.

A graded Lie algebra $L = \bigoplus_{k=1}^{M} L_k$ (where possibly $M = \infty$) is called *stratified* if it is generated by L_1; in other words, we have $L_{k+1} = [L_1, L_k]$.

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional stratified Lie algebras.

EXAMPLES:

```
sage: LieAlgebras(QQ).Graded().Stratified().FiniteDimensional()
Category of finite dimensional stratified Lie algebras over Rational Field
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).Graded().Stratified().FiniteDimensional()
Category of finite dimensional stratified Lie algebras over Rational Field
```

extra_super_categories()

Implement the fact that a finite dimensional stratified Lie algebra is nilpotent.

EXAMPLES:

```
sage: C = LieAlgebras(QQ).Graded().Stratified().FiniteDimensional()
sage: C.extra_super_categories()
[Category of nilpotent Lie algebras over Rational Field]
sage: C is C.Nilpotent()
True
sage: C.is_subcategory(LieAlgebras(QQ).Nilpotent())
True
```

```
>>> from sage.all import *
>>> C = LieAlgebras(QQ).Graded().Stratified().FiniteDimensional()
>>> C.extra_super_categories()
[Category of nilpotent Lie algebras over Rational Field]
>>> C is C.Nilpotent()
True
>>> C.is_subcategory(LieAlgebras(QQ).Nilpotent())
True
```

class SubcategoryMethods

Bases: `object`

Stratified()

Return the full subcategory of stratified objects of `self`.

A Lie algebra is stratified if it is graded and generated as a Lie algebra by its component of degree one.

EXAMPLES:

```
sage: LieAlgebras(QQ).Graded().Stratified()
Category of stratified Lie algebras over Rational Field
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).Graded().Stratified()
Category of stratified Lie algebras over Rational Field
```

4.88 Graded Lie Algebras With Basis

class sage.categories.graded_lie_algebras_with_basis.**GradedLieAlgebrasWithBasis**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded Lie algebras with a distinguished basis.

EXAMPLES:

```
sage: C = LieAlgebras(ZZ).WithBasis().Graded(); C
Category of graded Lie algebras with basis over Integer Ring
sage: C.super_categories()
[Category of graded modules with basis over Integer Ring,
 Category of Lie algebras with basis over Integer Ring,
 Category of graded Lie algebras over Integer Ring]

sage: C is LieAlgebras(ZZ).WithBasis().Graded()
True
sage: C is LieAlgebras(ZZ).Graded().WithBasis()
False
```

```
>>> from sage.all import *
>>> C = LieAlgebras(ZZ).WithBasis().Graded(); C
Category of graded Lie algebras with basis over Integer Ring
>>> C.super_categories()
[Category of graded modules with basis over Integer Ring,
 Category of Lie algebras with basis over Integer Ring,
 Category of graded Lie algebras over Integer Ring]

>>> C is LieAlgebras(ZZ).WithBasis().Graded()
True
>>> C is LieAlgebras(ZZ).Graded().WithBasis()
False
```

FiniteDimensional

alias of *FiniteDimensionalGradedLieAlgebrasWithBasis*

4.89 Graded Lie Conformal Algebras

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.graded_lie_conformal_algebras.**GradedLieConformalAlgebras**(*base_category*)

Bases: *GradedLieConformalAlgebrasCategory*

The category of graded Lie conformal algebras.

EXAMPLES:

```
sage: C = LieConformalAlgebras(QQbar).Graded(); C                              #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
Category of H-graded Lie conformal algebras over Algebraic Field

sage: CS = LieConformalAlgebras(QQ).Graded().Super(); CS
Category of H-graded super Lie conformal algebras over Rational Field
sage: CS is LieConformalAlgebras(QQ).Super().Graded()
True
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQbar).Graded(); C                                #␣
↪needs sage.rings.number_field
Category of H-graded Lie conformal algebras over Algebraic Field

>>> CS = LieConformalAlgebras(QQ).Graded().Super(); CS
Category of H-graded super Lie conformal algebras over Rational Field
>>> CS is LieConformalAlgebras(QQ).Super().Graded()
True
```

class sage.categories.graded_lie_conformal_algebras.**GradedLieConformalAlgebrasCategory**(*base_category*)

Bases: *GradedModulesCategory*

Super(*base_ring=None*)

Return the super-analogue category of `self`.

INPUT:

- `base_ring` – this is ignored

EXAMPLES:

```
sage: # needs sage.rings.number_field
sage: C = LieConformalAlgebras(QQbar)
sage: C.Graded().Super() is C.Super().Graded()
True
sage: Cp = C.WithBasis()
sage: Cp.Graded().Super() is Cp.Super().Graded()
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> C = LieConformalAlgebras(QQbar)
>>> C.Graded().Super() is C.Super().Graded()
True
>>> Cp = C.WithBasis()
>>> Cp.Graded().Super() is Cp.Super().Graded()
True
```

4.90 Graded modules

class sage.categories.graded_modules.**GradedModules**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded modules.

We consider every graded module $M = \bigoplus_i M_i$ as a filtered module under the (natural) filtration given by

$$F_i = \bigoplus_{j<i} M_j.$$

EXAMPLES:

```
sage: GradedModules(ZZ)
Category of graded modules over Integer Ring
sage: GradedModules(ZZ).super_categories()
[Category of filtered modules over Integer Ring]
```

```
>>> from sage.all import *
>>> GradedModules(ZZ)
Category of graded modules over Integer Ring
>>> GradedModules(ZZ).super_categories()
[Category of filtered modules over Integer Ring]
```

The category of graded modules defines the graded structure which shall be preserved by morphisms:

```
sage: Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
```

```
>>> from sage.all import *
>>> Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

class sage.categories.graded_modules.**GradedModulesCategory**(*base_category*)

Bases: *RegressiveCovariantConstructionCategory*, *Category_over_base_ring*

EXAMPLES:

```
sage: C = GradedAlgebras(QQ)
sage: C
Category of graded algebras over Rational Field
sage: C.base_category()
Category of algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of filtered algebras over Rational Field,
 Category of graded vector spaces over Rational Field]

sage: AlgebrasWithBasis(QQ).Graded().base_ring()
Rational Field
sage: GradedHopfAlgebrasWithBasis(QQ).base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> C = GradedAlgebras(QQ)
>>> C
Category of graded algebras over Rational Field
>>> C.base_category()
Category of algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of filtered algebras over Rational Field,
 Category of graded vector spaces over Rational Field]

>>> AlgebrasWithBasis(QQ).Graded().base_ring()
Rational Field
>>> GradedHopfAlgebrasWithBasis(QQ).base_ring()
Rational Field
```

classmethod default_super_categories(*category*, **args*)

Return the default super categories of `category.Graded()`.

Mathematical meaning: every graded object (module, algebra, etc.) is a filtered object with the (implicit) filtration defined by $F_i = \bigoplus_{j \leq i} G_j$.

INPUT:

- `cls` – the class `GradedModulesCategory`
- `category` – a category

OUTPUT: a (join) category

In practice, this returns `category.Filtered()`, joined together with the result of the method *`RegressiveCovariantConstructionCategory.default_super_categories()`* (that is the join of `category.Filtered()` and `cat` for each `cat` in the super categories of `category`).

EXAMPLES:

Consider `category=Algebras()`, which has `cat=Modules()` as super category. Then, a grading of an algebra G is also a filtration of G:

```
sage: Algebras(QQ).Graded().super_categories()
[Category of filtered algebras over Rational Field,
 Category of graded vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).Graded().super_categories()
[Category of filtered algebras over Rational Field,
 Category of graded vector spaces over Rational Field]
```

This resulted from the following call:

```
sage: sage.categories.graded_modules.GradedModulesCategory.default_super_
↪categories(Algebras(QQ))
Join of Category of filtered algebras over Rational Field
 and Category of graded vector spaces over Rational Field
```

```
>>> from sage.all import *
>>> sage.categories.graded_modules.GradedModulesCategory.default_super_
```

(continues on next page)

(continued from previous page)

```
↪categories(Algebras(QQ))
Join of Category of filtered algebras over Rational Field
 and Category of graded vector spaces over Rational Field
```

4.91 Graded modules with basis

class sage.categories.graded_modules_with_basis.**GradedModulesWithBasis**(*base_category*)

Bases: *GradedModulesCategory*

The category of graded modules with a distinguished basis.

EXAMPLES:

```
sage: C = GradedModulesWithBasis(ZZ); C
Category of graded modules with basis over Integer Ring
sage: sorted(C.super_categories(), key=str)
[Category of filtered modules with basis over Integer Ring,
 Category of graded modules over Integer Ring]
sage: C is ModulesWithBasis(ZZ).Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedModulesWithBasis(ZZ); C
Category of graded modules with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of filtered modules with basis over Integer Ring,
 Category of graded modules over Integer Ring]
>>> C is ModulesWithBasis(ZZ).Graded()
True
```

class ElementMethods

Bases: object

degree_negation()

Return the image of `self` under the degree negation automorphism of the graded module to which `self` belongs.

The degree negation is the module automorphism which scales every homogeneous element of degree k by $(-1)^k$ (for all k). This assumes that the module to which `self` belongs (that is, the module `self.parent()`) is $\mathbf{Z}$-graded.

EXAMPLES:

```
sage: E.<a,b> = ExteriorAlgebra(QQ)                                    #␣
↪needs sage.modules
sage: ((1 + a) * (1 + b)).degree_negation()                            #␣
↪needs sage.modules
a*b - a - b + 1
sage: E.zero().degree_negation()                                       #␣
↪needs sage.modules
0
```

(continues on next page)

(continued from previous page)

```
sage: P = GradedModulesWithBasis(ZZ).example(); P                              #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Integer Ring
sage: pbp = lambda x: P.basis()[Partition(list(x))]
sage: p = pbp([3,1]) - 2 * pbp([2]) + 4 * pbp([1])                             #␣
↪needs sage.combinat sage.modules
sage: p.degree_negation()                                                      #␣
↪needs sage.combinat sage.modules
-4*P[1] - 2*P[2] + P[3, 1]
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('a', 'b',)); (a, b,) = E._first_
↪ngens(2)# needs sage.modules
>>> ((Integer(1) + a) * (Integer(1) + b)).degree_negation()               ␣
↪              # needs sage.modules
a*b - a - b + 1
>>> E.zero().degree_negation()                                             #␣
↪needs sage.modules
0

>>> P = GradedModulesWithBasis(ZZ).example(); P                            #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Integer Ring
>>> pbp = lambda x: P.basis()[Partition(list(x))]
>>> p = pbp([Integer(3),Integer(1)]) - Integer(2) * pbp([Integer(2)]) +␣
↪Integer(4) * pbp([Integer(1)])                          # needs sage.
↪combinat sage.modules
>>> p.degree_negation()                                                    #␣
↪needs sage.combinat sage.modules
-4*P[1] - 2*P[2] + P[3, 1]
```

class ParentMethods

Bases: object

degree_negation(*element*)

Return the image of `element` under the degree negation automorphism of the graded module `self`.

The degree negation is the module automorphism which scales every homogeneous element of degree k by $(-1)^k$ (for all k). This assumes that the module `self` is $\mathbf{Z}$-graded.

INPUT:

- `element` – element of the module `self`

EXAMPLES:

```
sage: E.<a,b> = ExteriorAlgebra(QQ)                                            #␣
↪needs sage.modules
sage: E.degree_negation((1 + a) * (1 + b))                                     #␣
↪needs sage.modules
a*b - a - b + 1
sage: E.degree_negation(E.zero())                                              #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
0

sage: P = GradedModulesWithBasis(ZZ).example(); P                              #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Integer Ring
sage: pbp = lambda x: P.basis()[Partition(list(x))]
sage: p = pbp([3,1]) - 2 * pbp([2]) + 4 * pbp([1])                             #␣
↪needs sage.combinat sage.modules
sage: P.degree_negation(p)                                                     #␣
↪needs sage.combinat sage.modules
-4*P[1] - 2*P[2] + P[3, 1]
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('a', 'b',)); (a, b,) = E._first_
↪ngens(2)# needs sage.modules
>>> E.degree_negation((Integer(1) + a) * (Integer(1) + b))                     ␣
↪              # needs sage.modules
a*b - a - b + 1
>>> E.degree_negation(E.zero())                                               #␣
↪needs sage.modules
0

>>> P = GradedModulesWithBasis(ZZ).example(); P                               #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Integer Ring
>>> pbp = lambda x: P.basis()[Partition(list(x))]
>>> p = pbp([Integer(3),Integer(1)]) - Integer(2) * pbp([Integer(2)]) +␣
↪Integer(4) * pbp([Integer(1)])                          # needs sage.
↪combinat sage.modules
>>> P.degree_negation(p)                                                      #␣
↪needs sage.combinat sage.modules
-4*P[1] - 2*P[2] + P[3, 1]
```

quotient_module(*submodule*, *check=True*, *already_echelonized=False*, *category=None*)

Construct the quotient module `self` / `submodule`.

INPUT:

- `submodule` – a submodule with basis of `self`, or something that can be turned into one via `self.submodule(submodule)`
- `check`, `already_echelonized` – passed down to *submodule()*
- `category` – (optional) the category of the quotient module

Warning

At this point, this only supports quotients by free submodules admitting a basis in unitriangular echelon form. In this case, the quotient is also a free module, with a basis consisting of the retract of a subset of the basis of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z, y])
sage: Q = E.quotient_module(S)
sage: Q.category()
Join of
 Category of quotients of graded modules with basis over Rational Field␣
↪and
 Category of graded vector spaces with basis over Rational Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z, y])
>>> Q = E.quotient_module(S)
>>> Q.category()
Join of
 Category of quotients of graded modules with basis over Rational Field␣
↪and
 Category of graded vector spaces with basis over Rational Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
```

See also

- *submodule()*
- *Rings.ParentMethods.quotient()*
- sage.modules.with_basis.subquotient.QuotientModuleWithBasis

submodule(*gens*, *check=True*, *already_echelonized=False*, *unitriangular=False*, *support_order=None*, *category=None*, **args*, ***opts*)

Return the submodule spanned by a finite set of elements.

INPUT:

- `gens` – list or family of elements of `self`
- `check` – boolean (default: `True`); whether to verify that the elements of `gens` are in `self`
- **`already_echelonized` – boolean (default: `False`); whether** the elements of `gens` are already in (not necessarily reduced) echelon form
- `unitriangular` – boolean (default: `False`); whether the lift morphism is unitriangular
- `support_order` – (optional) either something that can be converted into a tuple or a key function
- `category` – (optional) the category of the submodule

If `already_echelonized` is `False`, then the generators are put in reduced echelon form using `echelonize()`, and reindexed by $0, 1, \ldots$.

Warning

At this point, this method only works for finite dimensional submodules and if matrices can be echelonized over the base ring.

If in addition `unitriangular` is `True`, then the generators are made such that the coefficients of the pivots are 1, so that lifting map is unitriangular.

The basis of the submodule uses the same index set as the generators, and the lifting map sends y_i to $gens[i]$.

See also

- *quotient_module()*
- sage.modules.with_basis.subquotient.SubmoduleWithBasis

EXAMPLES:

A graded submodule of a graded module generated by homogeneous elements is naturally graded:

```
sage: # needs sage.combinat sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z])
sage: S.category()
Join of
 Category of graded vector spaces with basis over Rational Field and
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
sage: S.basis()[0].degree()
1
sage: S.basis()[1].degree()
2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z])
>>> S.category()
Join of
 Category of graded vector spaces with basis over Rational Field and
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
>>> S.basis()[Integer(0)].degree()
1
>>> S.basis()[Integer(1)].degree()
2
```

We check on the echelonized basis:

```
sage: Sp = E.submodule([1, x + y + 5, x*y - y*z + x + y - 2])           #␣
↪needs sage.combinat sage.modules
sage: Sp.category()                                                     #␣
↪needs sage.combinat sage.modules
Join of
 Category of graded vector spaces with basis over Rational Field and
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> Sp = E.submodule([Integer(1), x + y + Integer(5), x*y - y*z + x + y -␣
↪Integer(2)])           # needs sage.combinat sage.modules
>>> Sp.category()                                                       #␣
↪needs sage.combinat sage.modules
Join of
 Category of graded vector spaces with basis over Rational Field and
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field
```

If it is generated by inhomogeneous elements, then it is filtered by default:

```
sage: F = E.submodule([x + y*z, x*z + y*x])                             #␣
↪needs sage.combinat sage.modules
sage: F.category()                                                      #␣
↪needs sage.combinat sage.modules
Join of
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field and
 Category of filtered vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> F = E.submodule([x + y*z, x*z + y*x])                               #␣
↪needs sage.combinat sage.modules
>>> F.category()                                                        #␣
↪needs sage.combinat sage.modules
Join of
 Category of subobjects of filtered modules with basis over Rational␣
↪Field and
 Category of finite dimensional filtered modules with basis over Rational␣
↪Field and
 Category of filtered vector spaces with basis over Rational Field
```

If `category` is specified, then it does not give any extra structure to the submodule (we can think of this as applying the forgetful functor):

```
sage: # needs sage.combinat sage.modules
sage: SM = E.submodule([x + y, x*y - y*z],
....:                  category=ModulesWithBasis(QQ))
sage: SM.category()
Join of
 Category of finite dimensional vector spaces with basis over Rational␣
↪Field and
 Category of subobjects of sets
sage: FM = E.submodule([x + 1, x*y - x*y*z],
....:                  category=ModulesWithBasis(QQ))
sage: FM.category()
Join of
 Category of finite dimensional vector spaces with basis over Rational␣
↪Field and
 Category of subobjects of sets
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> SM = E.submodule([x + y, x*y - y*z],
...                  category=ModulesWithBasis(QQ))
>>> SM.category()
Join of
 Category of finite dimensional vector spaces with basis over Rational␣
↪Field and
 Category of subobjects of sets
>>> FM = E.submodule([x + Integer(1), x*y - x*y*z],
...                  category=ModulesWithBasis(QQ))
>>> FM.category()
Join of
 Category of finite dimensional vector spaces with basis over Rational␣
↪Field and
 Category of subobjects of sets
```

If we have specified that this is a graded submodule of a graded module, then the echelonized elements must be homogeneous:

```
sage: Cat = ModulesWithBasis(QQ).Graded().Subobjects()
sage: E.submodule([x + y, x*y - 1], category=Cat)                    #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: all of the generators must be homogeneous
sage: E.submodule([x + y, x*y - x - y], category=Cat)                #␣
↪needs sage.combinat sage.modules
Free module generated by {0, 1} over Rational Field
```

```
>>> from sage.all import *
>>> Cat = ModulesWithBasis(QQ).Graded().Subobjects()
>>> E.submodule([x + y, x*y - Integer(1)], category=Cat)                ␣
↪     # needs sage.combinat sage.modules
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: all of the generators must be homogeneous
>>> E.submodule([x + y, x*y - x - y], category=Cat)                    #␣
↪needs sage.combinat sage.modules
Free module generated by {0, 1} over Rational Field
```

class Quotients(*category*, **args*)

Bases: *QuotientsCategory*

class ElementMethods

Bases: object

degree()

Return the degree of self.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z, y])
sage: Q = E.quotient_module(S)
sage: B = Q.basis()
sage: [B[i].lift() for i in Q.indices()]
[1, z, x*z, y*z, x*y*z]
sage: [B[i].degree() for i in Q.indices()]
[0, 1, 2, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z, y])
>>> Q = E.quotient_module(S)
>>> B = Q.basis()
>>> [B[i].lift() for i in Q.indices()]
[1, z, x*z, y*z, x*y*z]
>>> [B[i].degree() for i in Q.indices()]
[0, 1, 2, 2, 3]
```

class ParentMethods

Bases: object

degree_on_basis(*m*)

Return the degree of the basis element indexed by m in self.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: E.<x,y,z> = ExteriorAlgebra(QQ)
sage: S = E.submodule([x + y, x*y - y*z, y])
sage: Q = E.quotient_module(S)
sage: B = Q.basis()
sage: [B[i].lift() for i in Q.indices()]
[1, z, x*z, y*z, x*y*z]
```

(continues on next page)

(continued from previous page)

```
sage: [Q.degree_on_basis(i) for i in Q.indices()]
[0, 1, 2, 2, 3]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = E._
↪first_ngens(3)
>>> S = E.submodule([x + y, x*y - y*z, y])
>>> Q = E.quotient_module(S)
>>> B = Q.basis()
>>> [B[i].lift() for i in Q.indices()]
[1, z, x*z, y*z, x*y*z]
>>> [Q.degree_on_basis(i) for i in Q.indices()]
[0, 1, 2, 2, 3]
```

4.92 Graphs

class sage.categories.graphs.**Graphs**

Bases: *Category_singleton*

The category of graphs.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs(); C
Category of graphs
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs(); C
Category of graphs
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom*

The category of connected graphs.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().Connected()
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().Connected()
>>> TestSuite(C).run()
```

extra_super_categories()

Return the extra super categories of `self`.

A connected graph is also a metric space.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: Graphs().Connected().super_categories() # indirect doctest
[Category of connected topological spaces,
 Category of connected simplicial complexes,
 Category of graphs,
 Category of metric spaces]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> Graphs().Connected().super_categories() # indirect doctest
[Category of connected topological spaces,
 Category of connected simplicial complexes,
 Category of graphs,
 Category of metric spaces]
```

class ParentMethods

Bases: object

dimension()

Return the dimension of self as a CW complex.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.dimension()
1
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.dimension()
1
```

edges()

Return the edges of self.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.edges()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.edges()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

faces()

Return the faces of self.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: sorted(C.faces(), key=lambda x: (x.dimension(), x.value))
[0, 1, 2, 3, 4, (0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> sorted(C.faces(), key=lambda x: (x.dimension(), x.value))
[0, 1, 2, 3, 4, (0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

facets()

Return the facets of `self`.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.facets()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.facets()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

vertices()

Return the vertices of `self`.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.vertices()
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.vertices()
[0, 1, 2, 3, 4]
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: Graphs().super_categories()
[Category of simplicial complexes]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> Graphs().super_categories()
[Category of simplicial complexes]
```

4.93 Group Algebras

This module implements the category of group algebras for arbitrary groups over arbitrary commutative rings. For details, see *sage.categories.algebra_functor*.

AUTHOR:

- David Loeffler (2008-08-24): initial version
- Martin Raum (2009-08): update to use new coercion model – see Issue #6670.
- John Palmieri (2011-07): more updates to coercion, categories, etc., group algebras constructed using CombinatorialFreeModule – see Issue #6670.
- Nicolas M. Thiéry (2010-2017), Travis Scrimshaw (2017): generalization to a covariant functorial construction for monoid algebras, and beyond – see e.g. Issue #18700.

class sage.categories.group_algebras.**GroupAlgebras**(*category*, **args*)

Bases: *AlgebrasCategory*

The category of group algebras over a given base ring.

EXAMPLES:

```
sage: C = Groups().Algebras(ZZ); C
Category of group algebras over Integer Ring
sage: C.super_categories()
[Category of Hopf algebras with basis over Integer Ring,
 Category of monoid algebras over Integer Ring]
```

```
>>> from sage.all import *
>>> C = Groups().Algebras(ZZ); C
Category of group algebras over Integer Ring
>>> C.super_categories()
[Category of Hopf algebras with basis over Integer Ring,
 Category of monoid algebras over Integer Ring]
```

We can also construct this category with:

```
sage: C is GroupAlgebras(ZZ)
True
```

```
>>> from sage.all import *
>>> C is GroupAlgebras(ZZ)
True
```

Here is how to create the group algebra of a group G:

```
sage: G = DihedralGroup(5)                                                  #␣
↪needs sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: QG = G.algebra(QQ); QG                                                    #␣
↪needs sage.groups sage.modules
Algebra of
 Dihedral group of order 10 as a permutation group over Rational Field
```

```
>>> from sage.all import *
>>> G = DihedralGroup(Integer(5))                                               ␣
↪     # needs sage.groups
>>> QG = G.algebra(QQ); QG                                                     #␣
↪needs sage.groups sage.modules
Algebra of
 Dihedral group of order 10 as a permutation group over Rational Field
```

and an example of computation:

```
sage: g = G.an_element(); g                                                     #␣
↪needs sage.groups sage.modules
(1,4)(2,3)
sage: (QG.term(g) + 1)**3                                                       #␣
↪needs sage.groups sage.modules
4*() + 4*(1,4)(2,3)
```

```
>>> from sage.all import *
>>> g = G.an_element(); g                                                      #␣
↪needs sage.groups sage.modules
(1,4)(2,3)
>>> (QG.term(g) + Integer(1))**Integer(3)                                       ␣
↪               # needs sage.groups sage.modules
4*() + 4*(1,4)(2,3)
```

Todo

- Check which methods would be better located in `Monoid.Algebras` or `Groups.Finite.Algebras`.

class ElementMethods

Bases: `object`

central_form()

Return `self` expressed in the canonical basis of the center of the group algebra.

INPUT:

- `self` – an element of the center of the group algebra

OUTPUT:

- A formal linear combination of the conjugacy class representatives representing its coordinates in the canonical basis of the center. See *sage.categories.group_algebras.GroupAlgebras.ParentMethods.center_basis()* for details.

Warning

- This method requires the underlying group to have a method `conjugacy_classes_rep-`

resentatives (every permutation group has one, thanks GAP!).

- This method does not check that the element is indeed central. Use the method *Monoids.Algebras.ElementMethods.is_central()* for this purpose.
- This function has a complexity linear in the number of conjugacy classes of the group. One could easily implement a function whose complexity is linear in the size of the support of self.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: QS3 = SymmetricGroup(3).algebra(QQ)
sage: A = QS3([2,3,1]) + QS3([3,1,2])
sage: A.central_form()
B[(1,2,3)]
sage: QS4 = SymmetricGroup(4).algebra(QQ)
sage: B = sum(len(s.cycle_type()) * QS4(s) for s in Permutations(4))
sage: B.central_form()
4*B[()] + 3*B[(1,2)] + 2*B[(1,2)(3,4)] + 2*B[(1,2,3)] + B[(1,2,3,4)]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> QS3 = SymmetricGroup(Integer(3)).algebra(QQ)
>>> A = QS3([Integer(2),Integer(3),Integer(1)]) + QS3([Integer(3),
...Integer(1),Integer(2)])
>>> A.central_form()
B[(1,2,3)]
>>> QS4 = SymmetricGroup(Integer(4)).algebra(QQ)
>>> B = sum(len(s.cycle_type()) * QS4(s) for s in␣
...Permutations(Integer(4)))
>>> B.central_form()
4*B[()] + 3*B[(1,2)] + 2*B[(1,2)(3,4)] + 2*B[(1,2,3)] + B[(1,2,3,4)]
```

The following test fails due to a bug involving combinatorial free modules and the coercion system (see Issue #28544):

```
sage: # needs sage.groups sage.modules
sage: G = PermutationGroup([[(1,2,3),(4,5)], [(3,4)]])
sage: QG = GroupAlgebras(QQ).example(G)
sage: s = sum(QG.basis())
sage: s.central_form()          # not tested
B[()] + B[(4,5)] + B[(3,4,5)] + B[(2,3)(4,5)]
+ B[(2,3,4,5)] + B[(1,2)(3,4,5)] + B[(1,2,3,4,5)]
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> G = PermutationGroup([[(Integer(1),Integer(2),Integer(3)),(Integer(4),
...Integer(5))], [(Integer(3),Integer(4))]])
>>> QG = GroupAlgebras(QQ).example(G)
>>> s = sum(QG.basis())
>>> s.central_form()          # not tested
B[()] + B[(4,5)] + B[(3,4,5)] + B[(2,3)(4,5)]
```

(continues on next page)

(continued from previous page)

```
+ B[(2,3,4,5)] + B[(1,2)(3,4,5)] + B[(1,2,3,4,5)]
```

See also

- *sage.categories.group_algebras.GroupAlgebras.ParentMethods.center_basis()*
- *Monoids.Algebras.ElementMethods.is_central()*

class ParentMethods

Bases: object

antipode_on_basis(*g*)

Return the antipode of the element g of the basis.

Each basis element g is group-like, and so has antipode g^{-1}. This method is used to compute the antipode of any element.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = CyclicPermutationGroup(6).algebra(ZZ); A
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
sage: g = CyclicPermutationGroup(6).an_element(); g
(1,2,3,4,5,6)
sage: A.antipode_on_basis(g)
(1,6,5,4,3,2)
sage: a = A.an_element(); a
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
sage: a.antipode()
() + 3*(1,5,3)(2,6,4) + 3*(1,6,5,4,3,2)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = CyclicPermutationGroup(Integer(6)).algebra(ZZ); A
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
>>> g = CyclicPermutationGroup(Integer(6)).an_element(); g
(1,2,3,4,5,6)
>>> A.antipode_on_basis(g)
(1,6,5,4,3,2)
>>> a = A.an_element(); a
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
>>> a.antipode()
() + 3*(1,5,3)(2,6,4) + 3*(1,6,5,4,3,2)
```

center_basis()

Return a basis of the center of the group algebra.

The canonical basis of the center of the group algebra is the family $(f_\sigma)_{\sigma \in C}$, where C is any collection of representatives of the conjugacy classes of the group, and f_σ is the sum of the elements in the conjugacy class of σ.

OUTPUT: tuple of elements of `self`

Warning

- This method requires the underlying group to have a method `conjugacy_classes` (every permutation group has one, thanks GAP!).

EXAMPLES:

```
sage: SymmetricGroup(3).algebra(QQ).center_basis()                          #␣
↪needs sage.combinat sage.groups sage.modules
((), (2,3) + (1,2) + (1,3), (1,2,3) + (1,3,2))
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(3)).algebra(QQ).center_basis()                   ␣
↪     # needs sage.combinat sage.groups sage.modules
((), (2,3) + (1,2) + (1,3), (1,2,3) + (1,3,2))
```

See also

- *sage.categories.group_algebras.GroupAlgebras.ElementMethods.central_form()*
- *Monoids.Algebras.ElementMethods.is_central()*

coproduct_on_basis(*g*)

Return the coproduct of the element `g` of the basis.

Each basis element `g` is group-like. This method is used to compute the coproduct of any element.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: A = CyclicPermutationGroup(6).algebra(ZZ); A
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
sage: g = CyclicPermutationGroup(6).an_element(); g
(1,2,3,4,5,6)
sage: A.coproduct_on_basis(g)
(1,2,3,4,5,6) # (1,2,3,4,5,6)
sage: a = A.an_element(); a
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
sage: a.coproduct()
() # () + 3*(1,2,3,4,5,6) # (1,2,3,4,5,6) + 3*(1,3,5)(2,4,6) # (1,3,5)(2,
↪4,6)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> A = CyclicPermutationGroup(Integer(6)).algebra(ZZ); A
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
```

(continues on next page)

(continued from previous page)

```
>>> g = CyclicPermutationGroup(Integer(6)).an_element(); g
(1,2,3,4,5,6)
>>> A.coproduct_on_basis(g)
(1,2,3,4,5,6) # (1,2,3,4,5,6)
>>> a = A.an_element(); a
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
>>> a.coproduct()
() # () + 3*(1,2,3,4,5,6) # (1,2,3,4,5,6) + 3*(1,3,5)(2,4,6) # (1,3,5)(2,
↪4,6)
```

counit(*x*)

Return the counit of the element `x` of the group algebra.

This is the sum of all coefficients of `x` with respect to the standard basis of the group algebra.

EXAMPLES:

```
sage: A = CyclicPermutationGroup(6).algebra(ZZ); A                    #␣
↪needs sage.groups sage.modules
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
sage: a = A.an_element(); a                                           #␣
↪needs sage.groups sage.modules
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
sage: a.counit()                                                      #␣
↪needs sage.groups sage.modules
7
```

```
>>> from sage.all import *
>>> A = CyclicPermutationGroup(Integer(6)).algebra(ZZ); A                ␣
↪     # needs sage.groups sage.modules
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
>>> a = A.an_element(); a                                             #␣
↪needs sage.groups sage.modules
() + 3*(1,2,3,4,5,6) + 3*(1,3,5)(2,4,6)
>>> a.counit()                                                        #␣
↪needs sage.groups sage.modules
7
```

counit_on_basis(*g*)

Return the counit of the element `g` of the basis.

Each basis element `g` is group-like, and so has counit 1. This method is used to compute the counit of any element.

EXAMPLES:

```
sage: A = CyclicPermutationGroup(6).algebra(ZZ); A                    #␣
↪needs sage.groups sage.modules
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
sage: g = CyclicPermutationGroup(6).an_element(); g                   #␣
↪needs sage.groups sage.modules
```

(continues on next page)

(continued from previous page)

```
(1,2,3,4,5,6)
sage: A.counit_on_basis(g)                                                  #␣
↪needs sage.groups sage.modules
1
```

```
>>> from sage.all import *
>>> A = CyclicPermutationGroup(Integer(6)).algebra(ZZ); A                   ␣
↪     # needs sage.groups sage.modules
Algebra of
 Cyclic group of order 6 as a permutation group over Integer Ring
>>> g = CyclicPermutationGroup(Integer(6)).an_element(); g                  ␣
↪     # needs sage.groups sage.modules
(1,2,3,4,5,6)
>>> A.counit_on_basis(g)                                                  #␣
↪needs sage.groups sage.modules
1
```

group()

Return the underlying group of the group algebra.

EXAMPLES:

```
sage: GroupAlgebras(QQ).example(GL(3, GF(11))).group()                      #␣
↪needs sage.groups sage.modules
General Linear Group of degree 3 over Finite Field of size 11
sage: SymmetricGroup(10).algebra(QQ).group()                                #␣
↪needs sage.combinat sage.groups sage.modules
Symmetric group of order 10! as a permutation group
```

```
>>> from sage.all import *
>>> GroupAlgebras(QQ).example(GL(Integer(3), GF(Integer(11)))).group()      ␣
↪               # needs sage.groups sage.modules
General Linear Group of degree 3 over Finite Field of size 11
>>> SymmetricGroup(Integer(10)).algebra(QQ).group()                         ␣
↪     # needs sage.combinat sage.groups sage.modules
Symmetric group of order 10! as a permutation group
```

is_integral_domain(*proof=True*)

Return `True` if `self` is an integral domain.

This is false unless `self.base_ring()` is an integral domain, and even then it is false unless `self.group()` has no nontrivial elements of finite order. I don't know if this condition suffices, but it obviously does if the group is abelian and finitely generated.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S2 = SymmetricGroup(2)
sage: GroupAlgebra(S2).is_integral_domain()                                 #␣
↪needs sage.combinat
False
sage: S1 = SymmetricGroup(1)
sage: GroupAlgebra(S1).is_integral_domain()                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
True
sage: GroupAlgebra(S1, IntegerModRing(4)).is_integral_domain()           #␣
↪needs sage.combinat
False
sage: GroupAlgebra(AbelianGroup(1)).is_integral_domain()
True
sage: GroupAlgebra(AbelianGroup(2, [0,2])).is_integral_domain()
False
sage: GroupAlgebra(GL(2, ZZ)).is_integral_domain()       # not implemented
False
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S2 = SymmetricGroup(Integer(2))
>>> GroupAlgebra(S2).is_integral_domain()                                #␣
↪needs sage.combinat
False
>>> S1 = SymmetricGroup(Integer(1))
>>> GroupAlgebra(S1).is_integral_domain()                                #␣
↪needs sage.combinat
True
>>> GroupAlgebra(S1, IntegerModRing(Integer(4))).is_integral_domain()    ␣
↪     # needs sage.combinat
False
>>> GroupAlgebra(AbelianGroup(Integer(1))).is_integral_domain()
True
>>> GroupAlgebra(AbelianGroup(Integer(2), [Integer(0),Integer(2)])).is_
↪integral_domain()
False
>>> GroupAlgebra(GL(Integer(2), ZZ)).is_integral_domain()       # not␣
↪implemented
False
```

example(*G=None*)

Return an example of group algebra.

EXAMPLES:

```
sage: GroupAlgebras(QQ['x']).example()                                   #␣
↪needs sage.groups sage.modules
Algebra of Dihedral group of order 8 as a permutation group
 over Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> GroupAlgebras(QQ['x']).example()                                     #␣
↪needs sage.groups sage.modules
Algebra of Dihedral group of order 8 as a permutation group
 over Univariate Polynomial Ring in x over Rational Field
```

An other group can be specified as optional argument:

```
sage: GroupAlgebras(QQ).example(AlternatingGroup(4))                    #␣
↪needs sage.groups sage.modules
Algebra of
 Alternating group of order 4!/2 as a permutation group over Rational Field
```

```
>>> from sage.all import *
>>> GroupAlgebras(QQ).example(AlternatingGroup(Integer(4)))                ␣
↪     # needs sage.groups sage.modules
Algebra of
 Alternating group of order 4!/2 as a permutation group over Rational Field
```

extra_super_categories()

Implement the fact that the algebra of a group is a Hopf algebra.

EXAMPLES:

```
sage: C = Groups().Algebras(QQ)
sage: C.extra_super_categories()
[Category of Hopf algebras over Rational Field]
sage: sorted(C.super_categories(), key=str)
[Category of Hopf algebras with basis over Rational Field,
 Category of monoid algebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = Groups().Algebras(QQ)
>>> C.extra_super_categories()
[Category of Hopf algebras over Rational Field]
>>> sorted(C.super_categories(), key=str)
[Category of Hopf algebras with basis over Rational Field,
 Category of monoid algebras over Rational Field]
```

4.94 Groupoid

class sage.categories.groupoid.**Groupoid**(*G=None*)

Bases: *CategoryWithParameters*

The category of groupoids, for a set (usually a group) G.

FIXME:

- Groupoid or Groupoids ?
- definition and link with Wikipedia article Groupoid
- Should Groupoid inherit from Category_over_base?

EXAMPLES:

```
sage: Groupoid(DihedralGroup(3))
Groupoid with underlying set Dihedral group of order 6 as a permutation group
```

```
>>> from sage.all import *
>>> Groupoid(DihedralGroup(Integer(3)))
Groupoid with underlying set Dihedral group of order 6 as a permutation group
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: Groupoid.an_instance() # indirect doctest
Groupoid with underlying set Symmetric group of order 8! as a permutation␣
↪group
```

```
>>> from sage.all import *
>>> Groupoid.an_instance() # indirect doctest
Groupoid with underlying set Symmetric group of order 8! as a permutation␣
↪group
```

super_categories()

EXAMPLES:

```
sage: Groupoid(DihedralGroup(3)).super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Groupoid(DihedralGroup(Integer(3))).super_categories()
[Category of sets]
```

4.95 Groups

class sage.categories.groups.**Groups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of (multiplicative) groups, i.e. monoids with inverses.

EXAMPLES:

```
sage: Groups()
Category of groups
sage: Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
```

```
>>> from sage.all import *
>>> Groups()
Category of groups
>>> Groups().super_categories()
[Category of monoids, Category of inverse unital magmas]
```

Algebras

alias of *GroupAlgebras*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of groups constructed as Cartesian products of groups.

This construction gives the direct product of groups. See Wikipedia article Direct_product and Wikipedia article Direct_product_of_groups for more information.

class ParentMethods

Bases: object

group_generators()

Return the group generators of self.

EXAMPLES:

```
sage: # needs sage.groups
sage: C5 = CyclicPermutationGroup(5)
sage: C4 = CyclicPermutationGroup(4)
sage: S4 = SymmetricGroup(3)
sage: C = cartesian_product([C5, C4, S4])
sage: C.group_generators()
Family (((1,2,3,4,5), (), ()),
        ((), (1,2,3,4), ()),
        ((), (), (1,2)),
        ((), (), (2,3)))
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> C5 = CyclicPermutationGroup(Integer(5))
>>> C4 = CyclicPermutationGroup(Integer(4))
>>> S4 = SymmetricGroup(Integer(3))
>>> C = cartesian_product([C5, C4, S4])
>>> C.group_generators()
Family (((1,2,3,4,5), (), ()),
        ((), (1,2,3,4), ()),
        ((), (), (1,2)),
        ((), (), (2,3)))
```

We check the other portion of Issue #16718 is fixed:

```
sage: len(C.j_classes())                                                    #␣
↪needs sage.graphs sage.groups
1
```

```
>>> from sage.all import *
>>> len(C.j_classes())                                                    #␣
↪needs sage.graphs sage.groups
1
```

An example with an infinitely generated group (a better output is needed):

```
sage: # needs sage.combinat sage.groups
sage: G = Groups.free([1,2])
sage: H = Groups.free(ZZ)
sage: C = cartesian_product([G, H])
sage: C.monoid_generators()
Lazy family (gen(i))_{i in The Cartesian product of (...)}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
```

(continues on next page)

(continued from previous page)

```
>>> G = Groups.free([Integer(1),Integer(2)])
>>> H = Groups.free(ZZ)
>>> C = cartesian_product([G, H])
>>> C.monoid_generators()
Lazy family (gen(i))_{i in The Cartesian product of (...)}
```

order()

Return the cardinality of `self`.

EXAMPLES:

```
sage: C = cartesian_product([SymmetricGroup(10), SL(2, GF(3))])      #␣
↪needs sage.groups sage.modules
sage: C.order()                                                      #␣
↪needs sage.groups sage.modules
87091200
```

```
>>> from sage.all import *
>>> C = cartesian_product([SymmetricGroup(Integer(10)), SL(Integer(2),␣
↪GF(Integer(3)))])      # needs sage.groups sage.modules
>>> C.order()                                                        #␣
↪needs sage.groups sage.modules
87091200
```

Todo

this method is just here to prevent `FiniteGroups.ParentMethods` to call `_cardinality_from_iterator`.

extra_super_categories()

A Cartesian product of groups is endowed with a natural group structure.

EXAMPLES:

```
sage: C = Groups().CartesianProducts()
sage: C.extra_super_categories()
[Category of groups]
sage: sorted(C.super_categories(), key=str)
[Category of Cartesian products of inverse unital magmas,
 Category of Cartesian products of monoids,
 Category of groups]
```

```
>>> from sage.all import *
>>> C = Groups().CartesianProducts()
>>> C.extra_super_categories()
[Category of groups]
>>> sorted(C.super_categories(), key=str)
[Category of Cartesian products of inverse unital magmas,
 Category of Cartesian products of monoids,
 Category of groups]
```

class Commutative(*base_category*)

Bases: *CategoryWithAxiom*

Category of commutative (abelian) groups.

A group G is *commutative* if $xy = yx$ for all $x, y \in G$.

static free(*index_set=None*, *names=None*, ***kwds*)

Return the free commutative group.

INPUT:

- `index_set` – (optional) an index set for the generators; if an integer, then this represents $\{0, 1, \ldots, n-1\}$
- `names` – string or list/tuple/iterable of strings (default: `'x'`); the generator names or name prefix

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: Groups.Commutative.free(index_set=ZZ)
Free abelian group indexed by Integer Ring
sage: Groups().Commutative().free(ZZ)
Free abelian group indexed by Integer Ring
sage: Groups().Commutative().free(5)
Multiplicative Abelian group isomorphic to Z x Z x Z x Z x Z
sage: F.<x,y,z> = Groups().Commutative().free(); F
Multiplicative Abelian group isomorphic to Z x Z x Z
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> Groups.Commutative.free(index_set=ZZ)
Free abelian group indexed by Integer Ring
>>> Groups().Commutative().free(ZZ)
Free abelian group indexed by Integer Ring
>>> Groups().Commutative().free(Integer(5))
Multiplicative Abelian group isomorphic to Z x Z x Z x Z x Z
>>> F = Groups().Commutative().free(names=('x', 'y', 'z',)); (x, y, z,) =␣
↪F._first_ngens(3); F
Multiplicative Abelian group isomorphic to Z x Z x Z
```

class ElementMethods

Bases: `object`

conjugacy_class()

Return the conjugacy class of `self`.

EXAMPLES:

```
sage: D = DihedralGroup(5)                                                  #␣
↪needs sage.groups
sage: g = D((1,3,5,2,4))                                                    #␣
↪needs sage.groups
sage: g.conjugacy_class()                                                   #␣
↪needs sage.groups
Conjugacy class of (1,3,5,2,4)
 in Dihedral group of order 10 as a permutation group
```

(continues on next page)

(continued from previous page)

```
sage: H = MatrixGroup([matrix(GF(5), 2, [1,2, -1,1]),                    #␣
↪needs sage.modules
....:                  matrix(GF(5), 2, [1,1, 0,1])])
sage: h = H(matrix(GF(5), 2, [1,2, -1,1]))                               #␣
↪needs sage.modules
sage: h.conjugacy_class()                                                #␣
↪needs sage.groups sage.modules
Conjugacy class of [1 2]
                   [4 1]
in Matrix group over Finite Field of size 5 with 2 generators (
[1 2]  [1 1]
[4 1], [0 1]
)

sage: G = SL(2, GF(2))                                                   #␣
↪needs sage.modules
sage: g = G.gens()[0]                                                    #␣
↪needs sage.groups sage.modules
sage: g.conjugacy_class()                                                #␣
↪needs sage.groups sage.modules
Conjugacy class of [1 1]
[0 1] in Special Linear Group of degree 2 over Finite Field of size 2

sage: G = SL(2, QQ)                                                      #␣
↪needs sage.modules
sage: g = G([[1,1], [0,1]])                                              #␣
↪needs sage.modules
sage: g.conjugacy_class()                                                #␣
↪needs sage.groups sage.modules
Conjugacy class of [1 1]
[0 1] in Special Linear Group of degree 2 over Rational Field
```

```
>>> from sage.all import *
>>> D = DihedralGroup(Integer(5))                                        ␣
↪     # needs sage.groups
>>> g = D((Integer(1),Integer(3),Integer(5),Integer(2),Integer(4)))      ␣
↪                                          # needs sage.groups
>>> g.conjugacy_class()                                                 #␣
↪needs sage.groups
Conjugacy class of (1,3,5,2,4)
 in Dihedral group of order 10 as a permutation group

>>> H = MatrixGroup([matrix(GF(Integer(5)), Integer(2), [Integer(1),
↪Integer(2), -Integer(1),Integer(1)]),                    # needs sage.
↪modules
...                   matrix(GF(Integer(5)), Integer(2), [Integer(1),
↪Integer(1), Integer(0),Integer(1)])])
>>> h = H(matrix(GF(Integer(5)), Integer(2), [Integer(1),Integer(2), -
↪Integer(1),Integer(1)]))                                  # needs sage.
↪modules
>>> h.conjugacy_class()                                                 #␣
↪needs sage.groups sage.modules
```

(continues on next page)

(continued from previous page)

```
Conjugacy class of [1 2]
                   [4 1]
in Matrix group over Finite Field of size 5 with 2 generators (
[1 2]  [1 1]
[4 1], [0 1]
)

>>> G = SL(Integer(2), GF(Integer(2)))                                    ␣
↪              # needs sage.modules
>>> g = G.gens()[Integer(0)]                                              ␣
↪     # needs sage.groups sage.modules
>>> g.conjugacy_class()                                                  #␣
↪needs sage.groups sage.modules
Conjugacy class of [1 1]
[0 1] in Special Linear Group of degree 2 over Finite Field of size 2

>>> G = SL(Integer(2), QQ)                                                ␣
↪     # needs sage.modules
>>> g = G([[Integer(1),Integer(1)], [Integer(0),Integer(1)]])             ␣
↪                                 # needs sage.modules
>>> g.conjugacy_class()                                                  #␣
↪needs sage.groups sage.modules
Conjugacy class of [1 1]
[0 1] in Special Linear Group of degree 2 over Rational Field
```

Finite

alias of *FiniteGroups*

Lie

alias of *LieGroups*

class ParentMethods

Bases: object

cayley_table(*names='letters'*, *elements=None*)

Return the "multiplication" table of this multiplicative group, which is also known as the "Cayley table".

> **Note**
>
> The order of the elements in the row and column headings is equal to the order given by the table's column_keys() method. The association between the actual elements and the names/symbols used in the table can also be retrieved as a dictionary with the translation() method.

For groups, this routine should behave identically to the *multiplication_table()* method for magmas, which applies in greater generality.

INPUT:

- names – the type of names used, values are:
  - 'letters' – lowercase ASCII letters are used for a base 26 representation of the elements' positions in the list given by *list()*, padded to a common width with leading 'a's.
  - 'digits' – base 10 representation of the elements' positions in the list given by column_keys(), padded to a common width with leading zeros.
  - 'elements' – the string representations of the elements themselves.

– a list - a list of strings, where the length of the list equals the number of elements.

- `elements` – (default: `None`) a list of elements of the group, in forms that can be coerced into the structure, eg. their string representations. This may be used to impose an alternate ordering on the elements, perhaps when this is used in the context of a particular structure. The default is to use whatever ordering is provided by the the group, which is reported by the `column_keys()` method. Or the `elements` can be a subset which is closed under the operation. In particular, this can be used when the base set is infinite.

OUTPUT:

An object representing the multiplication table. This is an `OperationTable` object and even more documentation can be found there.

EXAMPLES:

Permutation groups, matrix groups and abelian groups can all compute their multiplication tables.

```
sage: # needs sage.groups
sage: G = DiCyclicGroup(3)
sage: T = G.cayley_table()
sage: T.column_keys()
((), (5,6,7), ..., (1,4,2,3)(5,7))
sage: T
*  a b c d e f g h i j k l
 +----------------------
a| a b c d e f g h i j k l
b| b c a e f d i g h l j k
c| c a b f d e h i g k l j
d| d e f a b c j k l g h i
e| e f d b c a l j k i g h
f| f d e c a b k l j h i g
g| g h i j k l d e f a b c
h| h i g k l j f d e c a b
i| i g h l j k e f d b c a
j| j k l g h i a b c d e f
k| k l j h i g c a b f d e
l| l j k i g h b c a e f d
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = DiCyclicGroup(Integer(3))
>>> T = G.cayley_table()
>>> T.column_keys()
((), (5,6,7), ..., (1,4,2,3)(5,7))
>>> T
*  a b c d e f g h i j k l
 +----------------------
a| a b c d e f g h i j k l
b| b c a e f d i g h l j k
c| c a b f d e h i g k l j
d| d e f a b c j k l g h i
e| e f d b c a l j k i g h
f| f d e c a b k l j h i g
g| g h i j k l d e f a b c
h| h i g k l j f d e c a b
i| i g h l j k e f d b c a
```

(continues on next page)

(continued from previous page)

```
j| j k l g h i a b c d e f
k| k l j h i g c a b f d e
l| l j k i g h b c a e f d
```

```
sage: M = SL(2, 2)                                                          #␣
↪needs sage.modules
sage: M.cayley_table()                                                      #␣
↪needs sage.libs.gap sage.modules
*  a b c d e f
 +------------
a| a b c d e f
b| b a d c f e
c| c e a f b d
d| d f b e a c
e| e c f a d b
f| f d e b c a
```

```
>>> from sage.all import *
>>> M = SL(Integer(2), Integer(2))                                          ␣
↪              # needs sage.modules
>>> M.cayley_table()                                                       #␣
↪needs sage.libs.gap sage.modules
*  a b c d e f
 +------------
a| a b c d e f
b| b a d c f e
c| c e a f b d
d| d f b e a c
e| e c f a d b
f| f d e b c a
<BLANKLINE>
```

```
sage: A = AbelianGroup([2, 3])                                              #␣
↪needs sage.groups
sage: A.cayley_table()                                                      #␣
↪needs sage.groups
*  a b c d e f
 +------------
a| a b c d e f
b| b c a e f d
c| c a b f d e
d| d e f a b c
e| e f d b c a
f| f d e c a b
```

```
>>> from sage.all import *
>>> A = AbelianGroup([Integer(2), Integer(3)])                              ␣
↪              # needs sage.groups
>>> A.cayley_table()                                                       #␣
↪needs sage.groups
*  a b c d e f
```

(continues on next page)

(continued from previous page)

```
 +------------
a| a b c d e f
b| b c a e f d
c| c a b f d e
d| d e f a b c
e| e f d b c a
f| f d e c a b
```

Lowercase ASCII letters are the default symbols used for the table, but you can also specify the use of decimal digit strings, or provide your own strings (in the proper order if they have meaning). Also, if the elements themselves are not too complex, you can choose to just use the string representations of the elements themselves.

```
sage: C = CyclicPermutationGroup(11)                                    #␣
↪needs sage.groups
sage: C.cayley_table(names='digits')                                    #␣
↪needs sage.groups
 *  00 01 02 03 04 05 06 07 08 09 10
  +--------------------------------
00| 00 01 02 03 04 05 06 07 08 09 10
01| 01 02 03 04 05 06 07 08 09 10 00
02| 02 03 04 05 06 07 08 09 10 00 01
03| 03 04 05 06 07 08 09 10 00 01 02
04| 04 05 06 07 08 09 10 00 01 02 03
05| 05 06 07 08 09 10 00 01 02 03 04
06| 06 07 08 09 10 00 01 02 03 04 05
07| 07 08 09 10 00 01 02 03 04 05 06
08| 08 09 10 00 01 02 03 04 05 06 07
09| 09 10 00 01 02 03 04 05 06 07 08
10| 10 00 01 02 03 04 05 06 07 08 09
```

```
>>> from sage.all import *
>>> C = CyclicPermutationGroup(Integer(11))                                 ␣
↪     # needs sage.groups
>>> C.cayley_table(names='digits')                                        #␣
↪needs sage.groups
 *  00 01 02 03 04 05 06 07 08 09 10
  +--------------------------------
00| 00 01 02 03 04 05 06 07 08 09 10
01| 01 02 03 04 05 06 07 08 09 10 00
02| 02 03 04 05 06 07 08 09 10 00 01
03| 03 04 05 06 07 08 09 10 00 01 02
04| 04 05 06 07 08 09 10 00 01 02 03
05| 05 06 07 08 09 10 00 01 02 03 04
06| 06 07 08 09 10 00 01 02 03 04 05
07| 07 08 09 10 00 01 02 03 04 05 06
08| 08 09 10 00 01 02 03 04 05 06 07
09| 09 10 00 01 02 03 04 05 06 07 08
10| 10 00 01 02 03 04 05 06 07 08 09
```

```
sage: G = QuaternionGroup()                                             #␣
↪needs sage.groups
```

(continues on next page)

(continued from previous page)

```
sage: names = ['1', 'I', '-1', '-I', 'J', '-K', '-J', 'K']
sage: G.cayley_table(names=names)                                              #␣
↪needs sage.groups
 *   1  I -1 -I  J -K -J  K
  +-----------------------
 1|  1  I -1 -I  J -K -J  K
 I|  I -1 -I  1  K  J -K -J
-1| -1 -I  1  I -J  K  J -K
-I| -I  1  I -1 -K -J  K  J
 J|  J -K -J  K -1 -I  1  I
-K| -K -J  K  J  I -1 -I  1
-J| -J  K  J -K  1  I -1 -I
 K|  K  J -K -J -I  1  I -1
```

```
>>> from sage.all import *
>>> G = QuaternionGroup()                                                      #␣
↪needs sage.groups
>>> names = ['1', 'I', '-1', '-I', 'J', '-K', '-J', 'K']
>>> G.cayley_table(names=names)                                                #␣
↪needs sage.groups
 *   1  I -1 -I  J -K -J  K
  +-----------------------
 1|  1  I -1 -I  J -K -J  K
 I|  I -1 -I  1  K  J -K -J
-1| -1 -I  1  I -J  K  J -K
-I| -I  1  I -1 -K -J  K  J
 J|  J -K -J  K -1 -I  1  I
-K| -K -J  K  J  I -1 -I  1
-J| -J  K  J -K  1  I -1 -I
 K|  K  J -K -J -I  1  I -1
```

```
sage: A = AbelianGroup([2, 2])                                                 #␣
↪needs sage.groups
sage: A.cayley_table(names='elements')                                         #␣
↪needs sage.groups
    *      1    f1    f0 f0*f1
     +-----------------------
    1|      1    f1    f0 f0*f1
   f1|     f1     1 f0*f1    f0
   f0|     f0 f0*f1     1    f1
f0*f1| f0*f1    f0    f1     1
```

```
>>> from sage.all import *
>>> A = AbelianGroup([Integer(2), Integer(2)])                                 ␣
↪              # needs sage.groups
>>> A.cayley_table(names='elements')                                           #␣
↪needs sage.groups
    *      1    f1    f0 f0*f1
     +-----------------------
    1|      1    f1    f0 f0*f1
   f1|     f1     1 f0*f1    f0
```

(continues on next page)

(continued from previous page)

```
   f0|    f0 f0*f1     1    f1
f0*f1| f0*f1    f0    f1     1
```

The `change_names()` routine behaves similarly, but changes an existing table "in-place."

```
sage: # needs sage.groups
sage: G = AlternatingGroup(3)
sage: T = G.cayley_table()
sage: T.change_names('digits')
sage: T
*  0 1 2
 +------
0| 0 1 2
1| 1 2 0
2| 2 0 1
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = AlternatingGroup(Integer(3))
>>> T = G.cayley_table()
>>> T.change_names('digits')
>>> T
*  0 1 2
 +------
0| 0 1 2
1| 1 2 0
2| 2 0 1
```

For an infinite group, you can still work with finite sets of elements, provided the set is closed under multiplication. Elements will be coerced into the group as part of setting up the table.

```
sage: # needs sage.modules
sage: G = SL(2,ZZ); G
Special Linear Group of degree 2 over Integer Ring
sage: identity = matrix(ZZ, [[1,0], [0,1]])
sage: G.cayley_table(elements=[identity, -identity])
*  a b
 +----
a| a b
b| b a
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> G = SL(Integer(2),ZZ); G
Special Linear Group of degree 2 over Integer Ring
>>> identity = matrix(ZZ, [[Integer(1),Integer(0)], [Integer(0),
↪Integer(1)]])
>>> G.cayley_table(elements=[identity, -identity])
*  a b
 +----
a| a b
b| b a
```

The `OperationTable` class provides even greater flexibility, including changing the operation. Here is one such example, illustrating the computation of commutators. `commutator` is defined as a function of two variables, before being used to build the table. From this, the commutator subgroup seems obvious, and creating a Cayley table with just these three elements confirms that they form a closed subset in the group.

```
sage: # needs sage.groups sage.modules
sage: from sage.matrix.operation_table import OperationTable
sage: G = DiCyclicGroup(3)
sage: commutator = lambda x, y: x*y*x^-1*y^-1
sage: T = OperationTable(G, commutator); T
.  a b c d e f g h i j k l
 +------------------------
a| a a a a a a a a a a a a
b| a a a a a a c c c c c c
c| a a a a a a b b b b b b
d| a a a a a a a a a a a a
e| a a a a a a c c c c c c
f| a a a a a a b b b b b b
g| a b c a b c a c b a c b
h| a b c a b c b a c b a c
i| a b c a b c c b a c b a
j| a b c a b c a c b a c b
k| a b c a b c b a c b a c
l| a b c a b c c b a c b a
sage: trans = T.translation()
sage: comm = [trans['a'], trans['b'], trans['c']]
sage: comm
[(), (5,6,7), (5,7,6)]
sage: P = G.cayley_table(elements=comm)
sage: P
*  a b c
 +------
a| a b c
b| b c a
c| c a b
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> from sage.matrix.operation_table import OperationTable
>>> G = DiCyclicGroup(Integer(3))
>>> commutator = lambda x, y: x*y*x**-Integer(1)*y**-Integer(1)
>>> T = OperationTable(G, commutator); T
.  a b c d e f g h i j k l
 +------------------------
a| a a a a a a a a a a a a
b| a a a a a a c c c c c c
c| a a a a a a b b b b b b
d| a a a a a a a a a a a a
e| a a a a a a c c c c c c
f| a a a a a a b b b b b b
g| a b c a b c a c b a c b
h| a b c a b c b a c b a c
```

(continues on next page)

(continued from previous page)

```
i| a b c a b c c b a c b a
j| a b c a b c a c b a c b
k| a b c a b c b a c b a c
l| a b c a b c c b a c b a
>>> trans = T.translation()
>>> comm = [trans['a'], trans['b'], trans['c']]
>>> comm
[(), (5,6,7), (5,7,6)]
>>> P = G.cayley_table(elements=comm)
>>> P
*  a b c
 +------
a| a b c
b| b c a
c| c a b
```

Todo

Arrange an ordering of elements into cosets of a normal subgroup close to size $\sqrt{n}$. Then the quotient group structure is often apparent in the table. See comments on Issue #7555.

AUTHOR:

- Rob Beezer (2010-03-15)

conjugacy_class(*g*)

Return the conjugacy class of the element `g`.

This is a fall-back method for groups not defined over GAP.

EXAMPLES:

```
sage: A = AbelianGroup([2, 2])                                                  #␣
↪needs sage.groups
sage: c = A.conjugacy_class(A.an_element())                                     #␣
↪needs sage.groups
sage: type(c)                                                                   #␣
↪needs sage.groups
<class 'sage.groups.conjugacy_classes.ConjugacyClass_with_category'>
```

```
>>> from sage.all import *
>>> A = AbelianGroup([Integer(2), Integer(2)])                                  ␣
↪              # needs sage.groups
>>> c = A.conjugacy_class(A.an_element())                                      #␣
↪needs sage.groups
>>> type(c)                                                                    #␣
↪needs sage.groups
<class 'sage.groups.conjugacy_classes.ConjugacyClass_with_category'>
```

group_generators()

Return group generators for `self`.

This default implementation calls `gens`, for backward compatibility.

EXAMPLES:

```
sage: A = AlternatingGroup(4)                                                 #␣
↪needs sage.groups
sage: A.group_generators()                                                    #␣
↪needs sage.groups
Family ((1,2,3), (2,3,4))
```

```
>>> from sage.all import *
>>> A = AlternatingGroup(Integer(4))                                          ␣
↪     # needs sage.groups
>>> A.group_generators()                                                     #␣
↪needs sage.groups
Family ((1,2,3), (2,3,4))
```

holomorph()

The holomorph of a group.

The holomorph of a group G is the semidirect product $G \rtimes_{id} Aut(G)$, where id is the identity function on $Aut(G)$, the automorphism group of G.

See Wikipedia article Holomorph (mathematics)

EXAMPLES:

```
sage: G = Groups().example()                                                  #␣
↪needs sage.modules
sage: G.holomorph()                                                           #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: holomorph of General Linear Group of degree 4
over Rational Field not yet implemented
```

```
>>> from sage.all import *
>>> G = Groups().example()                                                   #␣
↪needs sage.modules
>>> G.holomorph()                                                            #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: holomorph of General Linear Group of degree 4
over Rational Field not yet implemented
```

monoid_generators()

Return the generators of `self` as a monoid.

Let G be a group with generating set X. In general, the generating set of G as a monoid is given by $X \cup X^{-1}$, where X^{-1} is the set of inverses of X. If G is a finite group, then the generating set as a monoid is X.

EXAMPLES:

```
sage: # needs sage.groups
sage: A = AlternatingGroup(4)
```

(continues on next page)

(continued from previous page)

```
sage: A.monoid_generators()
Family ((1,2,3), (2,3,4))
sage: F.<x,y> = FreeGroup()
sage: F.monoid_generators()
Family (x, y, x^-1, y^-1)
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> A = AlternatingGroup(Integer(4))
>>> A.monoid_generators()
Family ((1,2,3), (2,3,4))
>>> F = FreeGroup(names=('x', 'y',)); (x, y,) = F._first_ngens(2)
>>> F.monoid_generators()
Family (x, y, x^-1, y^-1)
```

semidirect_product(*N*, *mapping*, *check=True*)

The semi-direct product of two groups.

EXAMPLES:

```
sage: G = Groups().example()                                                #␣
↪needs sage.modules
sage: G.semidirect_product(G, Morphism(G, G))                               #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: semidirect product of General Linear Group of degree␣
↪4
over Rational Field and General Linear Group of degree 4 over Rational␣
↪Field
not yet implemented
```

```
>>> from sage.all import *
>>> G = Groups().example()                                                  #␣
↪needs sage.modules
>>> G.semidirect_product(G, Morphism(G, G))                                 #␣
↪needs sage.modules
Traceback (most recent call last):
...
NotImplementedError: semidirect product of General Linear Group of degree␣
↪4
over Rational Field and General Linear Group of degree 4 over Rational␣
↪Field
not yet implemented
```

class Topological(*category*, **args*)

Bases: *TopologicalSpacesCategory*

Category of topological groups.

A topological group G is a group which has a topology such that multiplication and taking inverses are continuous functions.

REFERENCES:

- Wikipedia article Topological_group

example()

EXAMPLES:

```
sage: Groups().example()                                                    #␣
↪needs sage.modules
General Linear Group of degree 4 over Rational Field
```

```
>>> from sage.all import *
>>> Groups().example()                                                      #␣
↪needs sage.modules
General Linear Group of degree 4 over Rational Field
```

static free(*index_set=None*, *names=None*, ***kwds*)

Return the free group.

INPUT:

- `index_set` – (optional) an index set for the generators; if an integer, then this represents $\{0, 1, \ldots, n-1\}$
- `names` – string or list/tuple/iterable of strings (default: `'x'`); the generator names or name prefix

When the index set is an integer or only variable names are given, this returns `FreeGroup_class`, which currently has more features due to the interface with GAP than `IndexedFreeGroup`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: Groups.free(index_set=ZZ)
Free group indexed by Integer Ring
sage: Groups().free(ZZ)
Free group indexed by Integer Ring
sage: Groups().free(5)
Free Group on generators {x0, x1, x2, x3, x4}
sage: F.<x,y,z> = Groups().free(); F
Free Group on generators {x, y, z}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> Groups.free(index_set=ZZ)
Free group indexed by Integer Ring
>>> Groups().free(ZZ)
Free group indexed by Integer Ring
>>> Groups().free(Integer(5))
Free Group on generators {x0, x1, x2, x3, x4}
>>> F = Groups().free(names=('x', 'y', 'z',)); (x, y, z,) = F._first_ngens(3);
↪ F
Free Group on generators {x, y, z}
```

4.96 Hecke modules

class sage.categories.hecke_modules.**HeckeModules**(R)

Bases: *Category_module*

The category of Hecke modules.

A Hecke module is a module M over the emph{anemic} Hecke algebra, i.e., the Hecke algebra generated by Hecke operators T_n with n coprime to the level of M. (Every Hecke module defines a level function, which is a positive integer.) The reason we require that M only be a module over the anemic Hecke algebra is that many natural maps, e.g., degeneracy maps, Atkin-Lehner operators, etc., are **T**-module homomorphisms; but they are homomorphisms over the anemic Hecke algebra.

EXAMPLES:

We create the category of Hecke modules over **Q**:

```
sage: C = HeckeModules(RationalField()); C
Category of Hecke modules over Rational Field
```

```
>>> from sage.all import *
>>> C = HeckeModules(RationalField()); C
Category of Hecke modules over Rational Field
```

TODO: check that this is what we want:

```
sage: C.super_categories()
[Category of vector spaces with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C.super_categories()
[Category of vector spaces with basis over Rational Field]
```

[Category of vector spaces over Rational Field]

Note that the base ring can be an arbitrary commutative ring:

```
sage: HeckeModules(IntegerRing())
Category of Hecke modules over Integer Ring
sage: HeckeModules(FiniteField(5))
Category of Hecke modules over Finite Field of size 5
```

```
>>> from sage.all import *
>>> HeckeModules(IntegerRing())
Category of Hecke modules over Integer Ring
>>> HeckeModules(FiniteField(Integer(5)))
Category of Hecke modules over Finite Field of size 5
```

The base ring doesn't have to be a principal ideal domain:

```
sage: HeckeModules(PolynomialRing(IntegerRing(), 'x'))
Category of Hecke modules over Univariate Polynomial Ring in x over Integer Ring
```

```
>>> from sage.all import *
>>> HeckeModules(PolynomialRing(IntegerRing(), 'x'))
Category of Hecke modules over Univariate Polynomial Ring in x over Integer Ring
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class ParentMethods

Bases: object

extra_super_categories()

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: HeckeModules(QQ).super_categories()
[Category of vector spaces with basis over Rational Field]
```

```
>>> from sage.all import *
>>> HeckeModules(QQ).super_categories()
[Category of vector spaces with basis over Rational Field]
```

4.97 Highest Weight Crystals

class sage.categories.highest_weight_crystals.**HighestWeightCrystalHomset**(*X*, *Y*, *category=None*)

Bases: *CrystalHomset*

The set of crystal morphisms from a highest weight crystal to another crystal.

See also

See *sage.categories.crystals.CrystalHomset* for more information.

Element

alias of *HighestWeightCrystalMorphism*

class sage.categories.highest_weight_crystals.**HighestWeightCrystalMorphism**(*parent*, *on_gens*, *cartan_type=None*, *virtualization=None*, *scaling_factors=None*, *gens=None*, *check=True*)

Bases: *CrystalMorphismByGenerators*

A virtual crystal morphism whose domain is a highest weight crystal.

INPUT:

- `parent` – a homset
- `on_gens` – a function or list that determines the image of the generators (if given a list, then this uses the order of the generators of the domain) of the domain under `self`
- `cartan_type` – (optional) a Cartan type; the default is the Cartan type of the domain
- `virtualization` – (optional) a dictionary whose keys are in the index set of the domain and whose values are lists of entries in the index set of the codomain
- `scaling_factors` – (optional) a dictionary whose keys are in the index set of the domain and whose values are scaling factors for the weight, ε and φ
- `gens` – (optional) a list of generators to define the morphism; the default is to use the highest weight vectors of the crystal
- `check` – boolean (default: `True`); check if the crystal morphism is valid

class sage.categories.highest_weight_crystals.**HighestWeightCrystals**

Bases: *Category_singleton*

The category of highest weight crystals.

A crystal is highest weight if it is acyclic; in particular, every connected component has a unique highest weight element, and that element generate the component.

EXAMPLES:

```
sage: C = HighestWeightCrystals()
sage: C
Category of highest weight crystals
sage: C.super_categories()
[Category of crystals]
sage: C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> C = HighestWeightCrystals()
>>> C
Category of highest weight crystals
>>> C.super_categories()
[Category of crystals]
>>> C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

class ElementMethods

Bases: `object`

string_parameters(*word=None*)

Return the string parameters of `self` corresponding to the reduced word `word`.

Given a reduced expression $w = s_{i_1} \cdots s_{i_k}$, the string parameters of $b \in B$ corresponding to w are $(a_1, \ldots, a_k)$ such that

$$e_{i_m}^{a_m} \cdots e_{i_1}^{a_1} b \neq 0$$
$$e_{i_m}^{a_m+1} \cdots e_{i_1}^{a_1} b = 0$$

for all $1 \leq m \leq k$.

For connected components isomorphic to $B(\lambda)$ or $B(\infty)$, if $w = w_0$ is the longest element of the Weyl group, then the path determined by the string parametrization terminates at the highest weight vector.

INPUT:

- `word` – a word in the alphabet of the index set; if not specified and we are in finite type, then this will be some reduced expression for the long element determined by the Weyl group

EXAMPLES:

```
sage: B = crystals.infinity.NakajimaMonomials(['A',3])
sage: mg = B.highest_weight_vector()
sage: w0 = [1,2,1,3,2,1]
sage: mg.string_parameters(w0)
[0, 0, 0, 0, 0, 0]
sage: mg.f_string([1]).string_parameters(w0)
[1, 0, 0, 0, 0, 0]
sage: mg.f_string([1,1,1]).string_parameters(w0)
[3, 0, 0, 0, 0, 0]
sage: mg.f_string([1,1,1,2,2]).string_parameters(w0)
[1, 2, 2, 0, 0, 0]
sage: mg.f_string([1,1,1,2,2]) == mg.f_string([1,1,2,2,1])
True
sage: x = mg.f_string([1,1,1,2,2,1,3,3,2,1,1,1])
sage: x.string_parameters(w0)
[4, 1, 1, 2, 2, 2]
sage: x.string_parameters([3,2,1,3,2,3])
[2, 3, 7, 0, 0, 0]
sage: x == mg.f_string([1]*7 + [2]*3 + [3]*2)
True
```

```
>>> from sage.all import *
>>> B = crystals.infinity.NakajimaMonomials(['A',Integer(3)])
>>> mg = B.highest_weight_vector()
>>> w0 = [Integer(1),Integer(2),Integer(1),Integer(3),Integer(2),
...Integer(1)]
>>> mg.string_parameters(w0)
[0, 0, 0, 0, 0, 0]
>>> mg.f_string([Integer(1)]).string_parameters(w0)
[1, 0, 0, 0, 0, 0]
>>> mg.f_string([Integer(1),Integer(1),Integer(1)]).string_parameters(w0)
[3, 0, 0, 0, 0, 0]
>>> mg.f_string([Integer(1),Integer(1),Integer(1),Integer(2),Integer(2)]).
...string_parameters(w0)
[1, 2, 2, 0, 0, 0]
>>> mg.f_string([Integer(1),Integer(1),Integer(1),Integer(2),Integer(2)])␣
...== mg.f_string([Integer(1),Integer(1),Integer(2),Integer(2),Integer(1)])
True
>>> x = mg.f_string([Integer(1),Integer(1),Integer(1),Integer(2),
...Integer(2),Integer(1),Integer(3),Integer(3),Integer(2),Integer(1),
...Integer(1),Integer(1)])
>>> x.string_parameters(w0)
[4, 1, 1, 2, 2, 2]
>>> x.string_parameters([Integer(3),Integer(2),Integer(1),Integer(3),
...Integer(2),Integer(3)])
[2, 3, 7, 0, 0, 0]
```

(continues on next page)

(continued from previous page)

```
>>> x == mg.f_string([Integer(1)]*Integer(7) + [Integer(2)]*Integer(3) +␣
↪[Integer(3)]*Integer(2))
True
```

```
sage: B = crystals.infinity.Tableaux("A5")
sage: b = B(rows=[[1,1,1,1,1,1,1,1,1,1,1,1,1,1,3,6,6,6,6,6,6],
....:             [2,2,2,2,2,2,2,2,2,4,5,5,5,6],
....:             [3,3,3,3,3,3,3,5],
....:             [4,4,4,6,6,6],
....:             [5,6]])
sage: b.string_parameters([1,2,1,3,2,1,4,3,2,1,5,4,3,2,1])
[0, 1, 1, 1, 1, 0, 4, 4, 3, 0, 11, 10, 7, 7, 6]

sage: B = crystals.infinity.Tableaux("G2")
sage: b = B(rows=[[1,1,1,1,1,3,3,0,-3,-3,-2,-2,-1,-1,-1,-1],[2,3,3,3]])
sage: b.string_parameters([2,1,2,1,2,1])
[5, 13, 11, 15, 4, 4]
sage: b.string_parameters([1,2,1,2,1,2])
[7, 12, 15, 8, 10, 0]
```

```
>>> from sage.all import *
>>> B = crystals.infinity.Tableaux("A5")
>>> b = B(rows=[[Integer(1),Integer(1),Integer(1),Integer(1),Integer(1),
↪Integer(1),Integer(1),Integer(1),Integer(1),Integer(1),Integer(1),
↪Integer(1),Integer(1),Integer(1),Integer(1),Integer(3),Integer(6),
↪Integer(6),Integer(6),Integer(6),Integer(6),Integer(6)],
...              [Integer(2),Integer(2),Integer(2),Integer(2),Integer(2),
↪Integer(2),Integer(2),Integer(2),Integer(2),Integer(4),Integer(5),
↪Integer(5),Integer(5),Integer(6)],
...              [Integer(3),Integer(3),Integer(3),Integer(3),Integer(3),
↪Integer(3),Integer(3),Integer(5)],
...              [Integer(4),Integer(4),Integer(4),Integer(6),Integer(6),
↪Integer(6)],
...              [Integer(5),Integer(6)]])
>>> b.string_parameters([Integer(1),Integer(2),Integer(1),Integer(3),
↪Integer(2),Integer(1),Integer(4),Integer(3),Integer(2),Integer(1),
↪Integer(5),Integer(4),Integer(3),Integer(2),Integer(1)])
[0, 1, 1, 1, 1, 0, 4, 4, 3, 0, 11, 10, 7, 7, 6]

>>> B = crystals.infinity.Tableaux("G2")
>>> b = B(rows=[[Integer(1),Integer(1),Integer(1),Integer(1),Integer(1),
↪Integer(3),Integer(3),Integer(0),-Integer(3),-Integer(3),-Integer(2),-
↪Integer(2),-Integer(1),-Integer(1),-Integer(1),-Integer(1)],[Integer(2),
↪Integer(3),Integer(3),Integer(3)]])
>>> b.string_parameters([Integer(2),Integer(1),Integer(2),Integer(1),
↪Integer(2),Integer(1)])
[5, 13, 11, 15, 4, 4]
>>> b.string_parameters([Integer(1),Integer(2),Integer(1),Integer(2),
↪Integer(1),Integer(2)])
[7, 12, 15, 8, 10, 0]
```

```
sage: C = crystals.Tableaux(['C',2], shape=[2,1])
sage: mg = C.highest_weight_vector()
sage: lw = C.lowest_weight_vectors()[0]
sage: lw.string_parameters([1,2,1,2])
[1, 2, 3, 1]
sage: lw.string_parameters([2,1,2,1])
[1, 3, 2, 1]
sage: lw.e_string([2,1,1,1,2,2,1]) == mg
True
sage: lw.e_string([1,2,2,1,1,1,2]) == mg
True
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['C',Integer(2)], shape=[Integer(2),Integer(1)])
>>> mg = C.highest_weight_vector()
>>> lw = C.lowest_weight_vectors()[Integer(0)]
>>> lw.string_parameters([Integer(1),Integer(2),Integer(1),Integer(2)])
[1, 2, 3, 1]
>>> lw.string_parameters([Integer(2),Integer(1),Integer(2),Integer(1)])
[1, 3, 2, 1]
>>> lw.e_string([Integer(2),Integer(1),Integer(1),Integer(1),Integer(2),
↪Integer(2),Integer(1)]) == mg
True
>>> lw.e_string([Integer(1),Integer(2),Integer(2),Integer(1),Integer(1),
↪Integer(1),Integer(2)]) == mg
True
```

class ParentMethods

Bases: object

connected_components_generators()

alias of *highest_weight_vectors()*.

digraph(*subset=None*, *index_set=None*, *depth=None*)

Return the DiGraph associated to self.

INPUT:

- subset – (optional) a subset of vertices for which the digraph should be constructed
- index_set – (optional) the index set to draw arrows
- depth – the depth to draw; optional only for finite crystals

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: T.digraph()
Digraph on 8 vertices
sage: S = T.subcrystal(max_depth=2)
sage: len(S)
5
sage: G = T.digraph(subset=list(S))
sage: G.is_isomorphic(T.digraph(depth=2), edge_labels=True)
True
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> T.digraph()
Digraph on 8 vertices
>>> S = T.subcrystal(max_depth=Integer(2))
>>> len(S)
5
>>> G = T.digraph(subset=list(S))
>>> G.is_isomorphic(T.digraph(depth=Integer(2)), edge_labels=True)
True
```

highest_weight_vector()

Return the highest weight vector if there is a single one; otherwise, raises an error.

Caveat: this assumes that *highest_weight_vectors()* returns a list or tuple.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C.highest_weight_vector()
1
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C.highest_weight_vector()
1
```

highest_weight_vectors()

Return the highest weight vectors of `self`.

This default implementation selects among the module generators those that are highest weight, and caches the result. A crystal element b is highest weight if $e_i(b) = 0$ for all i in the index set.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C.highest_weight_vectors()
(1,)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C.highest_weight_vectors()
(1,)
```

```
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(C, C, C, generators=[[C(2),C(1),C(1)],
....:                                                 [C(1),C(2),C(1)]])
sage: T.highest_weight_vectors()
([2, 1, 1], [1, 2, 1])
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(C, C, C, generators=[[C(Integer(2)),
```

(continues on next page)

(continued from previous page)

```
↪C(Integer(1)),C(Integer(1))],
...                                                    [C(Integer(1)),
↪C(Integer(2)),C(Integer(1))]])
>>> T.highest_weight_vectors()
([2, 1, 1], [1, 2, 1])
```

lowest_weight_vectors()

Return the lowest weight vectors of `self`.

This default implementation selects among all elements of the crystal those that are lowest weight, and cache the result. A crystal element b is lowest weight if $f_i(b) = 0$ for all i in the index set.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C.lowest_weight_vectors()
(6,)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C.lowest_weight_vectors()
(6,)
```

```
sage: C = crystals.Letters(['A',2])
sage: T = crystals.TensorProduct(C, C, C,generators=[[C(2),C(1),C(1)],
....:                                                [C(1),C(2),C(1)]])
sage: T.lowest_weight_vectors()
([3, 2, 3], [3, 3, 2])
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(2)])
>>> T = crystals.TensorProduct(C, C, C,generators=[[C(Integer(2)),
↪C(Integer(1)),C(Integer(1))],
...                                                [C(Integer(1)),
↪C(Integer(2)),C(Integer(1))]])
>>> T.lowest_weight_vectors()
([3, 2, 3], [3, 3, 2])
```

q_dimension(*q=None*, *prec=None*, *use_product=False*)

Return the q-dimension of `self`.

Let $B(\lambda)$ denote a highest weight crystal. Recall that the degree of the μ-weight space of $B(\lambda)$ (under the principal gradation) is equal to $\langle \rho^\vee, \lambda - \mu \rangle$ where $\langle \rho^\vee, \alpha_i \rangle = 1$ for all $i \in I$ (in particular, take $\rho^\vee = \sum_{i \in I} h_i$).

The q-dimension of a highest weight crystal $B(\lambda)$ is defined as

$$\dim_q B(\lambda) := \sum_{j \geq 0} \dim(B_j) q^j,$$

where B_j denotes the degree j portion of $B(\lambda)$. This can be expressed as the product

$$\dim_q B(\lambda) = \prod_{\alpha^\vee \in \Delta_+^\vee} \left(\frac{1 - q^{\langle \lambda + \rho, \alpha^\vee \rangle}}{1 - q^{\langle \rho, \alpha^\vee \rangle}} \right)^{\operatorname{mult} \alpha},$$

where $\Delta_+^\vee$ denotes the set of positive coroots. Taking the limit as $q \to 1$ gives the dimension of $B(\lambda)$. For more information, see [Ka1990] Section 10.10.

INPUT:

- `q` – the (generic) parameter q
- `prec` – (default: `None`) the precision of the power series ring to use if the crystal is not known to be finite (i.e. the number of terms returned). If `None`, then the result is returned as a lazy power series.
- `use_product` – boolean (default: `False`); if we have a finite crystal and `True`, use the product formula

EXAMPLES:

```
sage: C = crystals.Tableaux(['A',2], shape=[2,1])
sage: qdim = C.q_dimension(); qdim
q^4 + 2*q^3 + 2*q^2 + 2*q + 1
sage: qdim(1)
8
sage: len(C) == qdim(1)
True
sage: C.q_dimension(use_product=True) == qdim
True
sage: C.q_dimension(prec=20)
q^4 + 2*q^3 + 2*q^2 + 2*q + 1
sage: C.q_dimension(prec=2)
2*q + 1

sage: R.<t> = QQ[]
sage: C.q_dimension(q=t^2)
t^8 + 2*t^6 + 2*t^4 + 2*t^2 + 1

sage: C = crystals.Tableaux(['A',2], shape=[5,2])
sage: C.q_dimension()
q^10 + 2*q^9 + 4*q^8 + 5*q^7 + 6*q^6 + 6*q^5
 + 6*q^4 + 5*q^3 + 4*q^2 + 2*q + 1

sage: C = crystals.Tableaux(['B',2], shape=[2,1])
sage: qdim = C.q_dimension(); qdim
q^10 + 2*q^9 + 3*q^8 + 4*q^7 + 5*q^6 + 5*q^5
 + 5*q^4 + 4*q^3 + 3*q^2 + 2*q + 1
sage: qdim == C.q_dimension(use_product=True)
True

sage: C = crystals.Tableaux(['D',4], shape=[2,1])
sage: C.q_dimension()
q^16 + 2*q^15 + 4*q^14 + 7*q^13 + 10*q^12 + 13*q^11
 + 16*q^10 + 18*q^9 + 18*q^8 + 18*q^7 + 16*q^6 + 13*q^5
 + 10*q^4 + 7*q^3 + 4*q^2 + 2*q + 1
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> qdim = C.q_dimension(); qdim
q^4 + 2*q^3 + 2*q^2 + 2*q + 1
>>> qdim(Integer(1))
8
>>> len(C) == qdim(Integer(1))
```

(continues on next page)

(continued from previous page)

```
True
>>> C.q_dimension(use_product=True) == qdim
True
>>> C.q_dimension(prec=Integer(20))
q^4 + 2*q^3 + 2*q^2 + 2*q + 1
>>> C.q_dimension(prec=Integer(2))
2*q + 1

>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> C.q_dimension(q=t**Integer(2))
t^8 + 2*t^6 + 2*t^4 + 2*t^2 + 1

>>> C = crystals.Tableaux(['A',Integer(2)], shape=[Integer(5),Integer(2)])
>>> C.q_dimension()
q^10 + 2*q^9 + 4*q^8 + 5*q^7 + 6*q^6 + 6*q^5
 + 6*q^4 + 5*q^3 + 4*q^2 + 2*q + 1

>>> C = crystals.Tableaux(['B',Integer(2)], shape=[Integer(2),Integer(1)])
>>> qdim = C.q_dimension(); qdim
q^10 + 2*q^9 + 3*q^8 + 4*q^7 + 5*q^6 + 5*q^5
 + 5*q^4 + 4*q^3 + 3*q^2 + 2*q + 1
>>> qdim == C.q_dimension(use_product=True)
True

>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2),Integer(1)])
>>> C.q_dimension()
q^16 + 2*q^15 + 4*q^14 + 7*q^13 + 10*q^12 + 13*q^11
 + 16*q^10 + 18*q^9 + 18*q^8 + 18*q^7 + 16*q^6 + 13*q^5
 + 10*q^4 + 7*q^3 + 4*q^2 + 2*q + 1
```

We check with a finite tensor product:

```
sage: TP = crystals.TensorProduct(C, C)
sage: TP.cardinality()
25600
sage: qdim = TP.q_dimension(use_product=True); qdim # long time
q^32 + 2*q^31 + 8*q^30 + 15*q^29 + 34*q^28 + 63*q^27 + 110*q^26
 + 175*q^25 + 276*q^24 + 389*q^23 + 550*q^22 + 725*q^21
 + 930*q^20 + 1131*q^19 + 1362*q^18 + 1548*q^17 + 1736*q^16
 + 1858*q^15 + 1947*q^14 + 1944*q^13 + 1918*q^12 + 1777*q^11
 + 1628*q^10 + 1407*q^9 + 1186*q^8 + 928*q^7 + 720*q^6
 + 498*q^5 + 342*q^4 + 201*q^3 + 117*q^2 + 48*q + 26
sage: qdim(1) # long time
25600
sage: TP.q_dimension() == qdim # long time
True
```

```
>>> from sage.all import *
>>> TP = crystals.TensorProduct(C, C)
>>> TP.cardinality()
25600
>>> qdim = TP.q_dimension(use_product=True); qdim # long time
```

(continues on next page)

(continued from previous page)

```
q^32 + 2*q^31 + 8*q^30 + 15*q^29 + 34*q^28 + 63*q^27 + 110*q^26
 + 175*q^25 + 276*q^24 + 389*q^23 + 550*q^22 + 725*q^21
 + 930*q^20 + 1131*q^19 + 1362*q^18 + 1548*q^17 + 1736*q^16
 + 1858*q^15 + 1947*q^14 + 1944*q^13 + 1918*q^12 + 1777*q^11
 + 1628*q^10 + 1407*q^9 + 1186*q^8 + 928*q^7 + 720*q^6
 + 498*q^5 + 342*q^4 + 201*q^3 + 117*q^2 + 48*q + 26
>>> qdim(Integer(1)) # long time
25600
>>> TP.q_dimension() == qdim # long time
True
```

The q-dimensions of infinite crystals are returned as formal power series:

```
sage: C = crystals.LSPaths(['A',2,1], [1,0,0])
sage: C.q_dimension(prec=5)
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + O(q^5)
sage: C.q_dimension(prec=10)
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + 5*q^5 + 7*q^6
 + 9*q^7 + 13*q^8 + 16*q^9 + O(q^10)
sage: qdim = C.q_dimension(); qdim
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + 5*q^5 + 7*q^6 + O(q^7)
sage: qdim[:16]
[1, 1, 2, 2, 4, 5, 7, 9, 13, 16, 22, 27, 36, 44, 57, 70]
```

```
>>> from sage.all import *
>>> C = crystals.LSPaths(['A',Integer(2),Integer(1)], [Integer(1),
↪Integer(0),Integer(0)])
>>> C.q_dimension(prec=Integer(5))
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + O(q^5)
>>> C.q_dimension(prec=Integer(10))
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + 5*q^5 + 7*q^6
 + 9*q^7 + 13*q^8 + 16*q^9 + O(q^10)
>>> qdim = C.q_dimension(); qdim
1 + q + 2*q^2 + 2*q^3 + 4*q^4 + 5*q^5 + 7*q^6 + O(q^7)
>>> qdim[:Integer(16)]
[1, 1, 2, 2, 4, 5, 7, 9, 13, 16, 22, 27, 36, 44, 57, 70]
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of highest weight crystals constructed by tensor product of highest weight crystals.

class ParentMethods

Bases: object

Implement operations on tensor products of crystals.

highest_weight_vectors()

Return the highest weight vectors of self.

This works by using a backtracing algorithm since if $b_2 \otimes b_1$ is highest weight then b_1 is highest weight.

EXAMPLES:

```
sage: C = crystals.Tableaux(['D',4], shape=[2,2])
sage: D = crystals.Tableaux(['D',4], shape=[1])
sage: T = crystals.TensorProduct(D, C)
sage: T.highest_weight_vectors()
([[[1]], [[1, 1], [2, 2]]],
 [[[3]], [[1, 1], [2, 2]]],
 [[[-2]], [[1, 1], [2, 2]]])
sage: L = filter(lambda x: x.is_highest_weight(), T)
sage: tuple(L) == T.highest_weight_vectors()
True
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2),
↪Integer(2)])
>>> D = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1)])
>>> T = crystals.TensorProduct(D, C)
>>> T.highest_weight_vectors()
([[[1]], [[1, 1], [2, 2]]],
 [[[3]], [[1, 1], [2, 2]]],
 [[[-2]], [[1, 1], [2, 2]]])
>>> L = filter(lambda x: x.is_highest_weight(), T)
>>> tuple(L) == T.highest_weight_vectors()
True
```

highest_weight_vectors_iterator()

Iterate over the highest weight vectors of `self`.

This works by using a backtracing algorithm since if $b_2 \otimes b_1$ is highest weight then b_1 is highest weight.

EXAMPLES:

```
sage: C = crystals.Tableaux(['D',4], shape=[2,2])
sage: D = crystals.Tableaux(['D',4], shape=[1])
sage: T = crystals.TensorProduct(D, C)
sage: tuple(T.highest_weight_vectors_iterator())
([[[1]], [[1, 1], [2, 2]]],
 [[[3]], [[1, 1], [2, 2]]],
 [[[-2]], [[1, 1], [2, 2]]])
sage: L = filter(lambda x: x.is_highest_weight(), T)
sage: tuple(L) == tuple(T.highest_weight_vectors_iterator())
True
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['D',Integer(4)], shape=[Integer(2),
↪Integer(2)])
>>> D = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1)])
>>> T = crystals.TensorProduct(D, C)
>>> tuple(T.highest_weight_vectors_iterator())
([[[1]], [[1, 1], [2, 2]]],
 [[[3]], [[1, 1], [2, 2]]],
 [[[-2]], [[1, 1], [2, 2]]])
>>> L = filter(lambda x: x.is_highest_weight(), T)
```

(continues on next page)

(continued from previous page)

```
>>> tuple(L) == tuple(T.highest_weight_vectors_iterator())
True
```

extra_super_categories()

EXAMPLES:

```
sage: HighestWeightCrystals().TensorProducts().extra_super_categories()
[Category of highest weight crystals]
```

```
>>> from sage.all import *
>>> HighestWeightCrystals().TensorProducts().extra_super_categories()
[Category of highest weight crystals]
```

additional_structure()

Return `None`.

Indeed, the category of highest weight crystals defines no additional structure: it only guarantees the existence of a unique highest weight element in each component.

See also

additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: HighestWeightCrystals().additional_structure()
```

```
>>> from sage.all import *
>>> HighestWeightCrystals().additional_structure()
```

example()

Return an example of highest weight crystals, as per *Category.example()*.

EXAMPLES:

```
sage: B = HighestWeightCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> B = HighestWeightCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

super_categories()

EXAMPLES:

```
sage: HighestWeightCrystals().super_categories()
[Category of crystals]
```

```
>>> from sage.all import *
>>> HighestWeightCrystals().super_categories()
[Category of crystals]
```

4.98 Hopf algebras

class sage.categories.hopf_algebras.**HopfAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Hopf algebras.

EXAMPLES:

```
sage: HopfAlgebras(QQ)
Category of Hopf algebras over Rational Field
sage: HopfAlgebras(QQ).super_categories()
[Category of bialgebras over Rational Field]
```

```
>>> from sage.all import *
>>> HopfAlgebras(QQ)
Category of Hopf algebras over Rational Field
>>> HopfAlgebras(QQ).super_categories()
[Category of bialgebras over Rational Field]
```

class DualCategory(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Hopf algebras constructed as dual of a Hopf algebra

class ParentMethods

Bases: object

class ElementMethods

Bases: object

antipode()

Return the antipode of `self`.

EXAMPLES:

```
sage: # needs sage.groups
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Rational Field
sage: [a,b] = A.algebra_generators()
sage: a, a.antipode()
(B[(1,2,3)], B[(1,3,2)])
sage: b, b.antipode()
(B[(1,3)], B[(1,3)])
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Rational Field
>>> [a,b] = A.algebra_generators()
>>> a, a.antipode()
(B[(1,2,3)], B[(1,3,2)])
>>> b, b.antipode()
(B[(1,3)], B[(1,3)])
```

class Morphism

Bases: *Category*

The category of Hopf algebra morphisms.

class ParentMethods

Bases: object

class Realizations(*category*, **args*)

Bases: *RealizationsCategory*

class ParentMethods

Bases: object

antipode_by_coercion(*x*)

Return the image of x by the antipode.

This default implementation coerces to the default realization, computes the antipode there, and coerces the result back.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: N = NonCommutativeSymmetricFunctions(QQ)
sage: R = N.ribbon()
sage: R.antipode_by_coercion.__module__
'sage.categories.hopf_algebras'
sage: R.antipode_by_coercion(R[1,3,1])
-R[2, 1, 2]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> N = NonCommutativeSymmetricFunctions(QQ)
>>> R = N.ribbon()
>>> R.antipode_by_coercion.__module__
'sage.categories.hopf_algebras'
>>> R.antipode_by_coercion(R[Integer(1),Integer(3),Integer(1)])
-R[2, 1, 2]
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

The category of super Hopf algebras.

> **Note**
>
> A super Hopf algebra is *not* simply a Hopf algebra with a $\mathbf{Z}/2\mathbf{Z}$ grading due to the signed bialgebra compatibility conditions.

class ElementMethods

Bases: object

antipode()

Return the antipode of self.

EXAMPLES:

```
sage: A = SteenrodAlgebra(3)                                          #␣
↪needs sage.combinat sage.modules
sage: a = A.an_element()                                              #␣
↪needs sage.combinat sage.modules
sage: a, a.antipode()                                                 #␣
↪needs sage.combinat sage.modules
(2 Q_1 Q_3 P(2,1), Q_1 Q_3 P(2,1))
```

```
>>> from sage.all import *
>>> A = SteenrodAlgebra(Integer(3))                                    ␣
↪    # needs sage.combinat sage.modules
>>> a = A.an_element()                                               #␣
↪needs sage.combinat sage.modules
>>> a, a.antipode()                                                  #␣
↪needs sage.combinat sage.modules
(2 Q_1 Q_3 P(2,1), Q_1 Q_3 P(2,1))
```

dual()

Return the dual category.

EXAMPLES:

The category of super Hopf algebras over any field is self dual:

```
sage: C = HopfAlgebras(QQ).Super()
sage: C.dual()
Category of super Hopf algebras over Rational Field
```

```
>>> from sage.all import *
>>> C = HopfAlgebras(QQ).Super()
>>> C.dual()
Category of super Hopf algebras over Rational Field
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of Hopf algebras constructed by tensor product of Hopf algebras

class ElementMethods

Bases: object

class ParentMethods

Bases: object

extra_super_categories()

EXAMPLES:

```
sage: C = HopfAlgebras(QQ).TensorProducts()
sage: C.extra_super_categories()
[Category of Hopf algebras over Rational Field]
sage: sorted(C.super_categories(), key=str)
[Category of Hopf algebras over Rational Field,
 Category of tensor products of algebras over Rational Field,
 Category of tensor products of coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = HopfAlgebras(QQ).TensorProducts()
>>> C.extra_super_categories()
[Category of Hopf algebras over Rational Field]
>>> sorted(C.super_categories(), key=str)
[Category of Hopf algebras over Rational Field,
 Category of tensor products of algebras over Rational Field,
 Category of tensor products of coalgebras over Rational Field]
```

WithBasis

alias of *HopfAlgebrasWithBasis*

dual()

Return the dual category.

EXAMPLES:

The category of Hopf algebras over any field is self dual:

```
sage: C = HopfAlgebras(QQ)
sage: C.dual()
Category of Hopf algebras over Rational Field
```

```
>>> from sage.all import *
>>> C = HopfAlgebras(QQ)
>>> C.dual()
Category of Hopf algebras over Rational Field
```

super_categories()

EXAMPLES:

```
sage: HopfAlgebras(QQ).super_categories()
[Category of bialgebras over Rational Field]
```

```
>>> from sage.all import *
>>> HopfAlgebras(QQ).super_categories()
[Category of bialgebras over Rational Field]
```

4.99 Hopf algebras with basis

class sage.categories.hopf_algebras_with_basis.**HopfAlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of Hopf algebras with a distinguished basis.

EXAMPLES:

```
sage: C = HopfAlgebrasWithBasis(QQ)
sage: C
Category of Hopf algebras with basis over Rational Field
sage: C.super_categories()
[Category of Hopf algebras over Rational Field,
 Category of bialgebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = HopfAlgebrasWithBasis(QQ)
>>> C
Category of Hopf algebras with basis over Rational Field
>>> C.super_categories()
[Category of Hopf algebras over Rational Field,
 Category of bialgebras with basis over Rational Field]
```

We now show how to use a simple Hopf algebra, namely the group algebra of the dihedral group (see also AlgebrasWithBasis):

```
sage: A = C.example(); A                                                    #␣
↪needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Rational Field
sage: A.rename('A')                                                         #␣
↪needs sage.groups
sage: A.category()                                                          #␣
↪needs sage.groups
Category of finite dimensional Hopf algebras with basis over Rational Field

sage: A.one_basis()                                                         #␣
↪needs sage.groups
()
sage: A.one()                                                               #␣
↪needs sage.groups
B[()]

sage: A.base_ring()                                                         #␣
↪needs sage.groups
Rational Field
sage: A.basis().keys()                                                      #␣
↪needs sage.groups
Dihedral group of order 6 as a permutation group

sage: # needs sage.groups
sage: [a,b] = A.algebra_generators()
sage: a, b
```

(continues on next page)

(continued from previous page)

```
(B[(1,2,3)], B[(1,3)])
sage: a^3, b^2
(B[()], B[()])
sage: a*b
B[(1,2)]

sage: A.product           # todo: not quite ...                                #␣
↪needs sage.groups
<bound method MagmaticAlgebras.WithBasis.ParentMethods._product_from_product_on_
↪basis_multiply of A>
sage: A.product(b, b)                                                        #␣
↪needs sage.groups
B[()]

sage: A.zero().coproduct()                                                   #␣
↪needs sage.groups
0
sage: A.zero().coproduct().parent()                                          #␣
↪needs sage.groups
A # A
sage: a.coproduct()                                                          #␣
↪needs sage.groups
B[(1,2,3)] # B[(1,2,3)]

sage: TestSuite(A).run(verbose=True)                                         #␣
↪needs sage.groups
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_antipode() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_characteristic() . . . pass
running ._test_construction() . . . pass
running ._test_distributivity() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_monomial_coefficients() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
sage: A.__class__                                                           #␣
↪needs sage.groups
<class 'sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra_with_
↪category'>
sage: A.element_class                                                       #␣
↪needs sage.groups
<class 'sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra_with_
↪category.element_class'>
```

```
>>> from sage.all import *
>>> A = C.example(); A                                                      #␣
↪needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Rational Field
>>> A.rename('A')                                                           #␣
↪needs sage.groups
>>> A.category()                                                            #␣
↪needs sage.groups
Category of finite dimensional Hopf algebras with basis over Rational Field

>>> A.one_basis()                                                           #␣
↪needs sage.groups
()
>>> A.one()                                                                 #␣
↪needs sage.groups
B[()]

>>> A.base_ring()                                                           #␣
↪needs sage.groups
Rational Field
>>> A.basis().keys()                                                        #␣
↪needs sage.groups
Dihedral group of order 6 as a permutation group

>>> # needs sage.groups
>>> [a,b] = A.algebra_generators()
>>> a, b
(B[(1,2,3)], B[(1,3)])
>>> a**Integer(3), b**Integer(2)
(B[()], B[()])
>>> a*b
B[(1,2)]

>>> A.product           # todo: not quite ...                               #␣
↪needs sage.groups
<bound method MagmaticAlgebras.WithBasis.ParentMethods._product_from_product_on_
↪basis_multiply of A>
```

(continues on next page)

(continued from previous page)

```
>>> A.product(b, b)                                                    #␣
↪needs sage.groups
B[()]

>>> A.zero().coproduct()                                               #␣
↪needs sage.groups
0
>>> A.zero().coproduct().parent()                                      #␣
↪needs sage.groups
A # A
>>> a.coproduct()                                                      #␣
↪needs sage.groups
B[(1,2,3)] # B[(1,2,3)]

>>> TestSuite(A).run(verbose=True)                                     #␣
↪needs sage.groups
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_antipode() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_characteristic() . . . pass
running ._test_construction() . . . pass
running ._test_distributivity() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_monomial_coefficients() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
>>> A.__class__                                                        #␣
↪needs sage.groups
<class 'sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra_with_
↪category'>
>>> A.element_class                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
<class 'sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra_with_
↪category.element_class'>
```

Let us look at the code for implementing A:

```
sage: A??                                   # not implemented                    #␣
↪needs sage.groups
```

```
>>> from sage.all import *
>>> A??                                     # not implemented                    #␣
↪needs sage.groups
```

class ElementMethods

Bases: object

Filtered

alias of *FilteredHopfAlgebrasWithBasis*

FiniteDimensional

alias of *FiniteDimensionalHopfAlgebrasWithBasis*

Graded

alias of *GradedHopfAlgebrasWithBasis*

class ParentMethods

Bases: object

antipode()

The antipode of this Hopf algebra.

If *antipode_on_basis()* is available, this constructs the antipode morphism from `self` to `self` by extending it by linearity. Otherwise, *antipode_by_coercion()* is used, if available.

EXAMPLES:

```
sage: # needs sage.groups
sage: A = HopfAlgebrasWithBasis(ZZ).example(); A
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Integer Ring
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: [a,b] = A.algebra_generators()
sage: a, A.antipode(a)
(B[(1,2,3)], B[(1,3,2)])
sage: b, A.antipode(b)
(B[(1,3)], B[(1,3)])
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> A = HopfAlgebrasWithBasis(ZZ).example(); A
An example of Hopf algebra with basis: the group algebra of the
 Dihedral group of order 6 as a permutation group over Integer Ring
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> [a,b] = A.algebra_generators()
```

(continues on next page)

(continued from previous page)

```
>>> a, A.antipode(a)
(B[(1,2,3)], B[(1,3,2)])
>>> b, A.antipode(b)
(B[(1,3)], B[(1,3)])
```

antipode_on_basis(*x*)

The antipode of the Hopf algebra on the basis (optional).

INPUT:

- `x` – an index of an element of the basis of `self`

Returns the antipode of the basis element indexed by `x`.

If this method is implemented, then *antipode()* is defined from this by linearity.

EXAMPLES:

```
sage: # needs sage.groups
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: W = A.basis().keys(); W
Dihedral group of order 6 as a permutation group
sage: w = W.gen(0); w
(1,2,3)
sage: A.antipode_on_basis(w)
B[(1,3,2)]
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> W = A.basis().keys(); W
Dihedral group of order 6 as a permutation group
>>> w = W.gen(Integer(0)); w
(1,2,3)
>>> A.antipode_on_basis(w)
B[(1,3,2)]
```

Super

alias of *SuperHopfAlgebrasWithBasis*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of Hopf algebras with basis constructed by tensor product of Hopf algebras with basis

class ElementMethods

Bases: `object`

class ParentMethods

Bases: `object`

extra_super_categories()

EXAMPLES:

```
sage: C = HopfAlgebrasWithBasis(QQ).TensorProducts()
sage: C.extra_super_categories()
[Category of Hopf algebras with basis over Rational Field]
```

(continues on next page)

(continued from previous page)

```
sage: sorted(C.super_categories(), key=str)
[Category of Hopf algebras with basis over Rational Field,
 Category of tensor products of Hopf algebras over Rational Field,
 Category of tensor products of algebras with basis over Rational Field]
```

```
>>> from sage.all import *
>>> C = HopfAlgebrasWithBasis(QQ).TensorProducts()
>>> C.extra_super_categories()
[Category of Hopf algebras with basis over Rational Field]
>>> sorted(C.super_categories(), key=str)
[Category of Hopf algebras with basis over Rational Field,
 Category of tensor products of Hopf algebras over Rational Field,
 Category of tensor products of algebras with basis over Rational Field]
```

example(*G=None*)

Return an example of algebra with basis:

```
sage: HopfAlgebrasWithBasis(QQ['x']).example()                                  #␣
↪needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
Dihedral group of order 6 as a permutation group
over Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> HopfAlgebrasWithBasis(QQ['x']).example()                                    #␣
↪needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
Dihedral group of order 6 as a permutation group
over Univariate Polynomial Ring in x over Rational Field
```

An other group can be specified as optional argument:

```
sage: HopfAlgebrasWithBasis(QQ).example(SymmetricGroup(4))                      #␣
↪needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
Symmetric group of order 4! as a permutation group over Rational Field
```

```
>>> from sage.all import *
>>> HopfAlgebrasWithBasis(QQ).example(SymmetricGroup(Integer(4)))               ␣
↪     # needs sage.groups
An example of Hopf algebra with basis: the group algebra of the
Symmetric group of order 4! as a permutation group over Rational Field
```

4.100 H-trivial semigroups

class sage.categories.h_trivial_semigroups.**HTrivialSemigroups**(*base_category*)

Bases: *CategoryWithAxiom*

Finite_extra_super_categories()

Implement the fact that a finite H-trivial is aperiodic.

EXAMPLES:

```
sage: Semigroups().HTrivial().Finite_extra_super_categories()
[Category of aperiodic semigroups]
sage: Semigroups().HTrivial().Finite() is Semigroups().Aperiodic().Finite()
True
```

```
>>> from sage.all import *
>>> Semigroups().HTrivial().Finite_extra_super_categories()
[Category of aperiodic semigroups]
>>> Semigroups().HTrivial().Finite() is Semigroups().Aperiodic().Finite()
True
```

Inverse_extra_super_categories()

Implement the fact that an H-trivial inverse semigroup is J-trivial.

Todo

Generalization for inverse semigroups.

Recall that there are two invertibility axioms for a semigroup S:

- One stating the existence, for all x, of a local inverse y satisfying $x = xyx$ and $y = yxy$;
- One stating the existence, for all x, of a global inverse y satisfying $xy = yx = 1$, where 1 is the unit of S (which must of course exist).

It is sufficient to have local inverses for H-triviality to imply J-triviality. However, at this stage, only the second axiom is implemented in Sage (see *Magmas.Unital.SubcategoryMethods.Inverse()*). Therefore this fact is only implemented for semigroups with global inverses, that is groups. However the trivial group is the unique H-trivial group, so this is rather boring.

EXAMPLES:

```
sage: Semigroups().HTrivial().Inverse_extra_super_categories()
[Category of j trivial semigroups]
sage: Monoids().HTrivial().Inverse()
Category of h trivial groups
```

```
>>> from sage.all import *
>>> Semigroups().HTrivial().Inverse_extra_super_categories()
[Category of j trivial semigroups]
>>> Monoids().HTrivial().Inverse()
Category of h trivial groups
```

4.101 Infinite Enumerated Sets

AUTHORS:

- Florent Hivert (2009-11): initial revision.

class sage.categories.infinite_enumerated_sets.**InfiniteEnumeratedSets**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of infinite enumerated sets.

An infinite enumerated sets is a countable set together with a canonical enumeration of its elements.

EXAMPLES:

```
sage: InfiniteEnumeratedSets()
Category of infinite enumerated sets
sage: InfiniteEnumeratedSets().super_categories()
[Category of enumerated sets, Category of infinite sets]
sage: InfiniteEnumeratedSets().all_super_categories()
[Category of infinite enumerated sets,
 Category of enumerated sets,
 Category of infinite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> InfiniteEnumeratedSets()
Category of infinite enumerated sets
>>> InfiniteEnumeratedSets().super_categories()
[Category of enumerated sets, Category of infinite sets]
>>> InfiniteEnumeratedSets().all_super_categories()
[Category of infinite enumerated sets,
 Category of enumerated sets,
 Category of infinite sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ParentMethods

Bases: object

list()

Raise an error because `self` is an infinite enumerated set.

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN.list()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN.list()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

random_element()

Raise an error because `self` is an infinite enumerated set.

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN.random_element()
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN.random_element()
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

TODO: should this be an optional abstract_method instead?

tuple()

Raise an error because `self` is an infinite enumerated set.

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN.tuple()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN.tuple()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
```

4.102 Integral domains

class sage.categories.integral_domains.**IntegralDomains**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of integral domains.

An integral domain is commutative ring with no zero divisors, or equivalently a commutative domain.

EXAMPLES:

```
sage: C = IntegralDomains(); C
Category of integral domains
sage: sorted(C.super_categories(), key=str)
[Category of commutative rings, Category of domains]
sage: C is Domains().Commutative()
True
sage: C is Rings().Commutative().NoZeroDivisors()
True
```

```
>>> from sage.all import *
>>> C = IntegralDomains(); C
Category of integral domains
>>> sorted(C.super_categories(), key=str)
[Category of commutative rings, Category of domains]
>>> C is Domains().Commutative()
True
>>> C is Rings().Commutative().NoZeroDivisors()
True
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

fraction_field()

Return the fraction field of self.

EXAMPLES:

```
sage: R = GF(61)['x,y']
sage: Frac(R)
Fraction Field of Multivariate Polynomial Ring in x, y over Finite Field␣
↪of size 61
sage: R.fraction_field()
Fraction Field of Multivariate Polynomial Ring in x, y over Finite Field␣
↪of size 61
```

```
>>> from sage.all import *
>>> R = GF(Integer(61))['x,y']
>>> Frac(R)
Fraction Field of Multivariate Polynomial Ring in x, y over Finite Field␣
↪of size 61
>>> R.fraction_field()
Fraction Field of Multivariate Polynomial Ring in x, y over Finite Field␣
↪of size 61
```

is_field(*proof=True*)

Return True if this ring is a field.

EXAMPLES:

```
sage: ZZ['x'].is_field()
False
```

```
>>> from sage.all import *
>>> ZZ['x'].is_field()
False
```

is_integral_domain(*proof=True*)

Return True, since this in an object of the category of integral domains.

EXAMPLES:

```
sage: ZZ.is_integral_domain()
True
sage: QQ.is_integral_domain()
True
sage: Parent(QQ, category=IntegralDomains()).is_integral_domain()
True

sage: L.<z> = LazyLaurentSeriesRing(QQ)                                    #␣
↪needs sage.combinat
sage: L.is_integral_domain()                                               #␣
↪needs sage.combinat
True
sage: L.is_integral_domain(proof=True)                                     #␣
↪needs sage.combinat
True

sage: ZZ['x'].is_integral_domain()
True
```

```
>>> from sage.all import *
>>> ZZ.is_integral_domain()
True
>>> QQ.is_integral_domain()
True
>>> Parent(QQ, category=IntegralDomains()).is_integral_domain()
True

>>> L = LazyLaurentSeriesRing(QQ, names=('z',)); (z,) = L._first_ngens(1)
↪# needs sage.combinat
>>> L.is_integral_domain()                                                 #␣
↪needs sage.combinat
True
>>> L.is_integral_domain(proof=True)                                       #␣
↪needs sage.combinat
True

>>> ZZ['x'].is_integral_domain()
True
```

localization(*additional_units*, *names=None*, *normalize=True*, *category=None*)

Return the localization of `self` at the given additional units.

EXAMPLES:

```
sage: R.<x, y> = GF(3)[]
sage: R.localization((x*y, x**2 + y**2))                                   ␣
↪  # needs sage.rings.finite_rings
Multivariate Polynomial Ring in x, y over Finite Field of size 3
 localized at (y, x, x^2 + y^2)
sage: ~y in _                                                              ␣
↪  # needs sage.rings.finite_rings
True
```

```
>>> from sage.all import *
>>> R = GF(Integer(3))['x, y']; (x, y,) = R._first_ngens(2)
>>> R.localization((x*y, x**Integer(2) + y**Integer(2)))
↪                  # needs sage.rings.finite_rings
Multivariate Polynomial Ring in x, y over Finite Field of size 3
 localized at (y, x, x^2 + y^2)
>>> ~y in _
↪# needs sage.rings.finite_rings
True
```

4.103 J-trivial semigroups

class sage.categories.j_trivial_semigroups.**JTrivialSemigroups**(*base_category*)

Bases: *CategoryWithAxiom*

extra_super_categories()

Implement the fact that a J-trivial semigroup is L and R-trivial.

EXAMPLES:

```
sage: Semigroups().JTrivial().extra_super_categories()
[Category of l trivial semigroups, Category of r trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().JTrivial().extra_super_categories()
[Category of l trivial semigroups, Category of r trivial semigroups]
```

4.104 Kac-Moody Algebras

AUTHORS:

- Travis Scrimshaw (07-15-2017): Initial implementation

class sage.categories.kac_moody_algebras.**KacMoodyAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

Category of Kac-Moody algebras.

class ParentMethods

Bases: object

cartan_type()

Return the Cartan type of `self`.

EXAMPLES:

```
sage: L = LieAlgebra(QQ, cartan_type=['A', 2])                  #␣
↪needs sage.combinat sage.modules
sage: L.cartan_type()                                           #␣
↪needs sage.combinat sage.modules
['A', 2]
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, cartan_type=['A', Integer(2)])
↪     # needs sage.combinat sage.modules
>>> L.cartan_type()
↪needs sage.combinat sage.modules
['A', 2]
```

weyl_group()

Return the Weyl group of `self`.

EXAMPLES:

```
sage: L = LieAlgebra(QQ, cartan_type=['A', 2])
↪needs sage.combinat sage.modules
sage: L.weyl_group()
↪needs sage.combinat sage.modules
Weyl Group of type ['A', 2] (as a matrix group acting on the ambient
↪space)
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, cartan_type=['A', Integer(2)])
↪     # needs sage.combinat sage.modules
>>> L.weyl_group()
↪needs sage.combinat sage.modules
Weyl Group of type ['A', 2] (as a matrix group acting on the ambient
↪space)
```

example(*n=2*)

Return an example of a Kac-Moody algebra as per *Category.example*.

EXAMPLES:

```
sage: from sage.categories.kac_moody_algebras import KacMoodyAlgebras
sage: KacMoodyAlgebras(QQ).example()
↪needs sage.combinat sage.modules
Lie algebra of ['A', 2] in the Chevalley basis
```

```
>>> from sage.all import *
>>> from sage.categories.kac_moody_algebras import KacMoodyAlgebras
>>> KacMoodyAlgebras(QQ).example()
↪needs sage.combinat sage.modules
Lie algebra of ['A', 2] in the Chevalley basis
```

We can specify the rank of the example:

```
sage: KacMoodyAlgebras(QQ).example(4)
↪needs sage.combinat sage.modules
Lie algebra of ['A', 4] in the Chevalley basis
```

```
>>> from sage.all import *
>>> KacMoodyAlgebras(QQ).example(Integer(4))
↪     # needs sage.combinat sage.modules
Lie algebra of ['A', 4] in the Chevalley basis
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.kac_moody_algebras import KacMoodyAlgebras
sage: KacMoodyAlgebras(QQ).super_categories()
[Category of Lie algebras over Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.kac_moody_algebras import KacMoodyAlgebras
>>> KacMoodyAlgebras(QQ).super_categories()
[Category of Lie algebras over Rational Field]
```

4.105 Kähler Algebras

AUTHORS:

- Shriya M

class sage.categories.kahler_algebras.**KahlerAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of graded algebras satisfying the Kähler package.

A finite-dimensional graded algebra $\bigoplus_{k=1}^{r} A^k$ satisfies the *Kähler package* if the following properties hold:

- Poincaré duality: There exists a perfect $\mathbf{Z}$-bilinear pairing given by

$$A^k \times A^{r-k} \longrightarrow \mathbf{Z}$$
$$(a, b) \mapsto \deg(a \cdot b).$$

- Hard-Lefschetz Theorem: The graded algebra contains *Lefschetz elements* $\omega \in A^1_{\mathbf{R}}$ such that multiplication by ω is an injection from $A^k_{\mathbf{R}} \longrightarrow A^{k+1}_{\mathbf{R}}$ for all $k < \frac{r}{2}$.
- Hodge-Riemann-Minikowski Relations: Every Lefchetz element ω, define quadratic forms on $A^k_{\mathbf{R}}$ given by

$$a \mapsto (-1)^k \deg(a \cdot \omega^{r-2k} \cdot a)$$

This quadratic form becomes positive definite upon restriction to the kernel of the following map

$$A^k_{\mathbf{R}} \longrightarrow A^{r-k+1}_{\mathbf{R}}$$
$$a \mapsto a \cdot \omega^{r-2k+1}.$$

REFERENCES:

- [ANR2023]

class ParentMethods

Bases: object

hodge_riemann_relations(*k*)

Return the quadratic form for the corresponding k ($< \frac{r}{2}$) for the Kähler algebra, where r is the top degree.

EXAMPLES:

```
sage: ch = matroids.Uniform(4, 6).chow_ring(QQ, False)
sage: ch.hodge_riemann_relations(1)
Quadratic form in 36 variables over Rational Field with coefficients:
[ 3 -1 -1 3 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 -1 3 3 ]
[ * 3 -1 3 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 3 -1 -1 -1 -1 -1 -1 -1 3 ]
[ * * 3 3 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 3 -1 -1 -1 3 ]
[ * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * 3 -1 3 -1 3 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 3 -1 -1␣
↪-1 -1 -1 -1 -1 -1 -1 -1 3 ]
[ * * * * * 3 3 -1 -1 3 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 3 -1 -1 -1 -1 -1 -1 3 ]
[ * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * 3 3 3 -1 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 3 -1 -1 3 ]
[ * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * 3 -1 3 -1 -1 3 -1 -1 3 -1 -1 -1 3 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * 3 3 -1 3 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * 3 3 3 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 3␣
↪-1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * 3 3 3 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 3 -1 3 ]
[ * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * 3 -1 3 -1 -1 3 -1 -1 -1 3 -1 -1␣
↪-1 3 -1 3 ]
[ * * * * * * * * * * * * * * * * * * * * * 3 3 -1 3 -1 -1 -1 3 -1 -1 -1␣
↪3 -1 -1 3 ]
[ * * * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * * * * 3 3 3 -1 3 -1 -1 -1 3 -1 -
↪1 -1 3 ]
[ * * * * * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 ]
```

(continues on next page)

(continued from previous page)

```
[ * * *
```

(continued from previous page)

```
↪-1 -1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * * 3 3 -1 3 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 3 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * 3 3 3 -1 -1 -1 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 3␣
↪-1 -1 -1 -1 -1 3 ]
[ * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * 3 3 3 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 3 -1 3 ]
[ * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1␣
↪-1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * 3 -1 3 -1 -1 3 -1 -1 -1 3 -1 -1␣
↪-1 3 -1 3 ]
[ * * * * * * * * * * * * * * * * 3 3 -1 3 -1 -1 -1 3 -1 -1 -1␣
↪3 -1 -1 3 ]
[ * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * 3 3 3 -1 3 -1 -1 -1 3 -1 -
↪1 -1 3 ]
[ * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -1 -
↪1 -1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * 3 3 3 -1 -1 -1 -1␣
↪3 3 ]
[ * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -1 -
↪1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1 -
↪1 -1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -1 -1␣
↪-1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * * 3 3 3 3 3 3␣
↪]
[ * * * * * * * * * * * * * * * * * * * * * * 3 -1 -1 -
↪1 -1 ]
[ * * * * * * * * * * * * * * * * * * * * * * 3 -1 -1␣
↪-1 ]
[ * * * * * * * * * * * * * * * * * * * * * * * 3 -1 -
↪1 ]
[ * * * * * * * * * * * * * * * * * * * * * * * * 3 -
↪1 ]
[ * * * * * * * * * * * * * * * * * * * * * * * * 3␣
↪]
>>> ch.hodge_riemann_relations(Integer(3))
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: k must be less than r/2 < 2
```

lefschetz_element()

Return one Lefschetz element of the given Kähler algebra.

EXAMPLES:

```
sage: U46 = matroids.Uniform(4, 6)
sage: C = U46.chow_ring(QQ, False)
sage: w = C.lefschetz_element(); w
-2*A01 - 2*A02 - 2*A03 - 2*A04 - 2*A05 - 2*A12 - 2*A13 - 2*A14
- 2*A15 - 2*A23 - 2*A24 - 2*A25 - 2*A34 - 2*A35 - 2*A45 - 6*A012
- 6*A013 - 6*A014 - 6*A015 - 6*A023 - 6*A024 - 6*A025 - 6*A034
- 6*A035 - 6*A045 - 6*A123 - 6*A124 - 6*A125 - 6*A134 - 6*A135
- 6*A145 - 6*A234 - 6*A235 - 6*A245 - 6*A345 - 30*A012345
sage: basis_deg = {}
sage: for b in C.basis():
....:     deg = b.homogeneous_degree()
....:     if deg not in basis_deg:
....:         basis_deg[deg] = []
....:     basis_deg[deg].append(b)
sage: m = max(basis_deg); m
3
sage: len(basis_deg[1]) == len(basis_deg[2])
True
sage: matrix([(w*b).to_vector() for b in basis_deg[1]]).rank()
36
sage: len(basis_deg[2])
36
```

```
>>> from sage.all import *
>>> U46 = matroids.Uniform(Integer(4), Integer(6))
>>> C = U46.chow_ring(QQ, False)
>>> w = C.lefschetz_element(); w
-2*A01 - 2*A02 - 2*A03 - 2*A04 - 2*A05 - 2*A12 - 2*A13 - 2*A14
- 2*A15 - 2*A23 - 2*A24 - 2*A25 - 2*A34 - 2*A35 - 2*A45 - 6*A012
- 6*A013 - 6*A014 - 6*A015 - 6*A023 - 6*A024 - 6*A025 - 6*A034
- 6*A035 - 6*A045 - 6*A123 - 6*A124 - 6*A125 - 6*A134 - 6*A135
- 6*A145 - 6*A234 - 6*A235 - 6*A245 - 6*A345 - 30*A012345
>>> basis_deg = {}
>>> for b in C.basis():
...     deg = b.homogeneous_degree()
...     if deg not in basis_deg:
...         basis_deg[deg] = []
...     basis_deg[deg].append(b)
>>> m = max(basis_deg); m
3
>>> len(basis_deg[Integer(1)]) == len(basis_deg[Integer(2)])
True
>>> matrix([(w*b).to_vector() for b in basis_deg[Integer(1)]]).rank()
```

(continues on next page)

(continued from previous page)

```
36
>>> len(basis_deg[Integer(2)])
36
```

poincare_pairing(*a*, *b*)

Return the Poincaré pairing of two elements of the Kähler algebra.

EXAMPLES:

```
sage: ch = matroids.catalog.Fano().chow_ring(QQ, True, 'fy')
sage: Ba, Bb, Bc, Bd, Be, Bf, Bg, Babf, Bace, Badg, Bbcd, Bbeg, Bcfg,␣
↪Bdef, Babcdefg = ch.gens()[8:]
sage: u = ch(-Babf^2 + Bcfg^2 - 8/7*Bc*Babcdefg + 1/2*Bd*Babcdefg -␣
↪Bf*Babcdefg - Bg*Babcdefg); u
-Babf^2 + Bcfg^2 - 8/7*Bc*Babcdefg + 1/2*Bd*Babcdefg - Bf*Babcdefg -␣
↪Bg*Babcdefg
sage: v = ch(Bg - 2/37*Babf + Badg + Bbeg + Bcfg + Babcdefg); v
Bg - 2/37*Babf + Badg + Bbeg + Bcfg + Babcdefg
sage: ch.poincare_pairing(v, u)
3
```

```
>>> from sage.all import *
>>> ch = matroids.catalog.Fano().chow_ring(QQ, True, 'fy')
>>> Ba, Bb, Bc, Bd, Be, Bf, Bg, Babf, Bace, Badg, Bbcd, Bbeg, Bcfg, Bdef,␣
↪Babcdefg = ch.gens()[Integer(8):]
>>> u = ch(-Babf**Integer(2) + Bcfg**Integer(2) - Integer(8)/
↪Integer(7)*Bc*Babcdefg + Integer(1)/Integer(2)*Bd*Babcdefg -␣
↪Bf*Babcdefg - Bg*Babcdefg); u
-Babf^2 + Bcfg^2 - 8/7*Bc*Babcdefg + 1/2*Bd*Babcdefg - Bf*Babcdefg -␣
↪Bg*Babcdefg
>>> v = ch(Bg - Integer(2)/Integer(37)*Babf + Badg + Bbeg + Bcfg +␣
↪Babcdefg); v
Bg - 2/37*Babf + Badg + Bbeg + Bcfg + Babcdefg
>>> ch.poincare_pairing(v, u)
3
```

super_categories()

Return the super categories of `self`.

EXAMPLES:

```
sage: from sage.categories.kahler_algebras import KahlerAlgebras

sage: C = KahlerAlgebras(QQ); C
Category of kahler algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of finite dimensional graded algebras with basis over
Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.kahler_algebras import KahlerAlgebras
```

(continues on next page)

(continued from previous page)

```
>>> C = KahlerAlgebras(QQ); C
Category of kahler algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of finite dimensional graded algebras with basis over
Rational Field]
```

4.106 Lambda Bracket Algebras

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.lambda_bracket_algebras.**LambdaBracketAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Lambda bracket algebras.

This is an abstract base category for Lie conformal algebras and super Lie conformal algebras.

class ElementMethods

Bases: object

T(*n=1*)

The n-th derivative of self.

INPUT:

- n – integer (default: 1); how many times to apply T to this element

OUTPUT:

$T^n a$ where a is this element. Notice that we use the *divided powers* notation $T^{(j)} = \frac{T^j}{j!}$.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: Vir = lie_conformal_algebras.Virasoro(QQ)
sage: Vir.inject_variables()
Defining L, C
sage: L.T()
TL
sage: L.T(3)
6*T^(3)L
sage: C.T()
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Vir = lie_conformal_algebras.Virasoro(QQ)
>>> Vir.inject_variables()
Defining L, C
>>> L.T()
TL
>>> L.T(Integer(3))
6*T^(3)L
>>> C.T()
0
```

bracket (*rhs*)

The λ-bracket of these two elements.

EXAMPLES:

The brackets of the Virasoro Lie conformal algebra:

```
sage: Vir = lie_conformal_algebras.Virasoro(QQ); L = Vir.0                #␣
↪needs sage.combinat sage.modules
sage: L.bracket(L)                                                        #␣
↪needs sage.combinat sage.modules
{0: TL, 1: 2*L, 3: 1/2*C}
sage: L.bracket(L.T())                                                    #␣
↪needs sage.combinat sage.modules
{0: 2*T^(2)L, 1: 3*TL, 2: 4*L, 4: 2*C}
```

```
>>> from sage.all import *
>>> Vir = lie_conformal_algebras.Virasoro(QQ); L = Vir.gen(0)              ␣
↪ # needs sage.combinat sage.modules
>>> L.bracket(L)                                                          #␣
↪needs sage.combinat sage.modules
{0: TL, 1: 2*L, 3: 1/2*C}
>>> L.bracket(L.T())                                                      #␣
↪needs sage.combinat sage.modules
{0: 2*T^(2)L, 1: 3*TL, 2: 4*L, 4: 2*C}
```

Now with a current algebra:

```
sage: # needs sage.combinat sage.modules
sage: V = lie_conformal_algebras.Affine(QQ, 'A1')
sage: V.gens()
(B[alpha[1]], B[alphacheck[1]], B[-alpha[1]], B['K'])
sage: E = V.0; H = V.1; F = V.2
sage: H.bracket(H)
{1: 2*B['K']}
sage: E.bracket(F)
{0: B[alphacheck[1]], 1: B['K']}
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> V = lie_conformal_algebras.Affine(QQ, 'A1')
>>> V.gens()
(B[alpha[1]], B[alphacheck[1]], B[-alpha[1]], B['K'])
>>> E = V.gen(0); H = V.gen(1); F = V.gen(2)
>>> H.bracket(H)
{1: 2*B['K']}
>>> E.bracket(F)
{0: B[alphacheck[1]], 1: B['K']}
```

nproduct (*rhs*, *n*)

The n-th product of these two elements.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: Vir = lie_conformal_algebras.Virasoro(QQ); L = Vir.0
sage: L.nproduct(L, 3)
1/2*C
sage: L.nproduct(L.T(), 0)
2*T^(2)L
sage: V = lie_conformal_algebras.Affine(QQ, 'A1')
sage: E = V.0; H = V.1; F = V.2
sage: E.nproduct(H, 0) == - 2*E
True
sage: E.nproduct(F, 1)
B['K']
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Vir = lie_conformal_algebras.Virasoro(QQ); L = Vir.gen(0)
>>> L.nproduct(L, Integer(3))
1/2*C
>>> L.nproduct(L.T(), Integer(0))
2*T^(2)L
>>> V = lie_conformal_algebras.Affine(QQ, 'A1')
>>> E = V.gen(0); H = V.gen(1); F = V.gen(2)
>>> E.nproduct(H, Integer(0)) == - Integer(2)*E
True
>>> E.nproduct(F, Integer(1))
B['K']
```

FinitelyGeneratedAsLambdaBracketAlgebra

alias of *FinitelyGeneratedLambdaBracketAlgebras*

class ParentMethods

Bases: object

ideal(*gens*, ***kwds*)

The ideal of this Lambda bracket algebra generated by gens.

Todo

Ideals of Lie Conformal Algebras are not implemented yet.

EXAMPLES:

```
sage: Vir = lie_conformal_algebras.Virasoro(QQ)                    #␣
↪needs sage.combinat sage.modules
sage: Vir.ideal()                                                  #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
NotImplementedError: ideals of Lie Conformal algebras are not implemented␣
↪yet
```

```
>>> from sage.all import *
>>> Vir = lie_conformal_algebras.Virasoro(QQ)                                  #␣
↪needs sage.combinat sage.modules
>>> Vir.ideal()                                                               #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
NotImplementedError: ideals of Lie Conformal algebras are not implemented␣
↪yet
```

class SubcategoryMethods

Bases: object

FinitelyGenerated()

The category of finitely generated Lambda bracket algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQ).FinitelyGenerated()
Category of finitely generated Lie conformal algebras over Rational Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQ).FinitelyGenerated()
Category of finitely generated Lie conformal algebras over Rational Field
```

FinitelyGeneratedAsLambdaBracketAlgebra()

The category of finitely generated Lambda bracket algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(QQ).FinitelyGenerated()
Category of finitely generated Lie conformal algebras over Rational Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQ).FinitelyGenerated()
Category of finitely generated Lie conformal algebras over Rational Field
```

WithBasis

alias of *LambdaBracketAlgebrasWithBasis*

super_categories()

The list of super categories of this category.

EXAMPLES:

```
sage: from sage.categories.lambda_bracket_algebras import␣
↪LambdaBracketAlgebras
sage: LambdaBracketAlgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.lambda_bracket_algebras import LambdaBracketAlgebras
>>> LambdaBracketAlgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

4.107 Lambda Bracket Algebras With Basis

AUTHORS:

- Reimundo Heluani (2020-08-21): Initial implementation.

class sage.categories.lambda_bracket_algebras_with_basis.**LambdaBracketAlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of Lambda bracket algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).WithBasis()                                  #␣
↪needs sage.rings.number_field
Category of Lie conformal algebras with basis over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).WithBasis()                                    #␣
↪needs sage.rings.number_field
Category of Lie conformal algebras with basis over Algebraic Field
```

class ElementMethods

Bases: object

index()

The index of this basis element.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: V = lie_conformal_algebras.NeveuSchwarz(QQ)
sage: V.inject_variables()
Defining L, G, C
sage: G.T(3).index()
('G', 3)
sage: v = V.an_element(); v
L + G + C
sage: v.index()
Traceback (most recent call last):
...
ValueError: index can only be computed for monomials, got L + G + C
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> V = lie_conformal_algebras.NeveuSchwarz(QQ)
>>> V.inject_variables()
Defining L, G, C
>>> G.T(Integer(3)).index()
('G', 3)
>>> v = V.an_element(); v
L + G + C
>>> v.index()
```

(continues on next page)

(continued from previous page)

```
Traceback (most recent call last):
...
ValueError: index can only be computed for monomials, got L + G + C
```

class FinitelyGeneratedAsLambdaBracketAlgebra(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of finitely generated lambda bracket algebras with basis.

EXAMPLES:

```
sage: # needs sage.rings.number_field
sage: C = LieConformalAlgebras(QQbar)
sage: C1 = C.WithBasis().FinitelyGenerated(); C1
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
sage: C2 = C.FinitelyGenerated().WithBasis(); C2
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
sage: C1 is C2
True
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> C = LieConformalAlgebras(QQbar)
>>> C1 = C.WithBasis().FinitelyGenerated(); C1
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
>>> C2 = C.FinitelyGenerated().WithBasis(); C2
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
>>> C1 is C2
True
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded finitely generated lambda bracket algebras with basis.

EXAMPLES:

```
sage: C = LieConformalAlgebras(QQbar)                                   #␣
↪needs sage.rings.number_field
sage: C.WithBasis().FinitelyGenerated().Graded()                        #␣
↪needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras
 with basis over Algebraic Field
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQbar)                                     #␣
↪needs sage.rings.number_field
>>> C.WithBasis().FinitelyGenerated().Graded()                          #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
Category of H-graded finitely generated Lie conformal algebras
 with basis over Algebraic Field
```

class ParentMethods

Bases: object

degree_on_basis(*m*)

Return the degree of the basis element indexed by `m` in `self`.

EXAMPLES:

```
sage: V = lie_conformal_algebras.Virasoro(QQ)                    #␣
↪needs sage.combinat sage.modules
sage: V.degree_on_basis(('L', 2))                                #␣
↪needs sage.combinat sage.modules
4
```

```
>>> from sage.all import *
>>> V = lie_conformal_algebras.Virasoro(QQ)                    #␣
↪needs sage.combinat sage.modules
>>> V.degree_on_basis(('L', Integer(2)))                            ␣
↪  # needs sage.combinat sage.modules
4
```

4.108 Lattice posets

class sage.categories.lattice_posets.**DistributiveLattices**(*base_category*)

Bases: *CategoryWithAxiom*

The category of distributive lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().Distributive(); cat
Category of finite distributive lattices

sage: cat.super_categories()
[Category of finite lattice posets,
 Category of distributive lattices]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().Distributive(); cat
Category of finite distributive lattices

>>> cat.super_categories()
[Category of finite lattice posets,
 Category of distributive lattices]
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: object

is_distributive()

Return whether self is a distributive lattice.

EXAMPLES:

```
sage: P = posets.Crown(4).order_ideals_lattice()
sage: P.is_distributive()
True
```

```
>>> from sage.all import *
>>> P = posets.Crown(Integer(4)).order_ideals_lattice()
>>> P.is_distributive()
True
```

extra_super_categories()

Return a list of the super categories of self.

These encode implications between properties.

EXAMPLES:

```
sage: LatticePosets().Distributive().super_categories()
[Category of congruence uniform lattice posets,
 Category of trim lattice posets,
 Category of chain graded lattice posets]
```

```
>>> from sage.all import *
>>> LatticePosets().Distributive().super_categories()
[Category of congruence uniform lattice posets,
 Category of trim lattice posets,
 Category of chain graded lattice posets]
```

class sage.categories.lattice_posets.**LatticePosets**

Bases: *Category*

The category of lattices, i.e. partially ordered sets in which any two elements have a unique supremum (the elements' least upper bound; called their *join*) and a unique infimum (greatest lower bound; called their *meet*).

EXAMPLES:

```
sage: LatticePosets()
Category of lattice posets
sage: LatticePosets().super_categories()
[Category of posets]
sage: LatticePosets().example()
NotImplemented
```

```
>>> from sage.all import *
>>> LatticePosets()
Category of lattice posets
>>> LatticePosets().super_categories()
[Category of posets]
```

(continues on next page)

(continued from previous page)

```
>>> LatticePosets().example()
NotImplemented
```

See also

- *Posets*
- *FiniteLatticePosets*, LatticePoset()

class ChainGraded(*base_category*)

Bases: *CategoryWithAxiom*

The category of graded lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().ChainGraded(); cat
Category of finite chain graded lattice posets

sage: cat.super_categories()
[Category of finite lattice posets,
 Category of chain graded lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().ChainGraded(); cat
Category of finite chain graded lattice posets

>>> cat.super_categories()
[Category of finite lattice posets,
 Category of chain graded lattice posets]
```

class ParentMethods

Bases: object

is_graded()

Return whether self is a graded lattice.

EXAMPLES:

```
sage: posets.DivisorLattice(12).is_graded()
True
```

```
>>> from sage.all import *
>>> posets.DivisorLattice(Integer(12)).is_graded()
True
```

class CongruenceUniform(*base_category*)

Bases: *CategoryWithAxiom*

The category of congruence uniform lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().CongruenceUniform(); cat
Category of finite congruence uniform lattice posets
sage: cat.super_categories()
[Category of finite lattice posets,
 Category of congruence uniform lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().CongruenceUniform(); cat
Category of finite congruence uniform lattice posets
>>> cat.super_categories()
[Category of finite lattice posets,
 Category of congruence uniform lattice posets]
```

class ParentMethods

Bases: object

brick_coloring()

Return the coloring of edges by join-irreducible elements.

This is defined for all congruence uniform lattices in the article [Muhle2017].

See also

core_label_order()

EXAMPLES:

```
sage: P = posets.PentagonPoset()
sage: P.brick_coloring()
{(0, 1): 1, (0, 2): 2, (1, 4): 2, (2, 3): 3, (3, 4): 1}
```

```
>>> from sage.all import *
>>> P = posets.PentagonPoset()
>>> P.brick_coloring()
{(0, 1): 1, (0, 2): 2, (1, 4): 2, (2, 3): 3, (3, 4): 1}
```

core_label_order()

Return the core-label order of this lattice.

This is a partial order on the same underlying set whose relations are determined by the canonical brick-coloring of the edges of the Hasse diagram. This is defined for all congruence uniform lattices in the article [Muhle2017].

This is also known as the shard intersection order, in the terminology of Reading.

See also

brick_coloring()

EXAMPLES:

```
sage: P = posets.PentagonPoset()
sage: Q = P.core_label_order(); Q
Finite poset containing 5 elements
sage: Q.relations_number()
12
sage: Q.is_graded()
True

sage: P = posets.TamariLattice(4)
sage: PQ = posets.NoncrossingPartitions(SymmetricGroup(4))
sage: P.core_label_order().is_isomorphic(PQ)
True
```

```
>>> from sage.all import *
>>> P = posets.PentagonPoset()
>>> Q = P.core_label_order(); Q
Finite poset containing 5 elements
>>> Q.relations_number()
12
>>> Q.is_graded()
True

>>> P = posets.TamariLattice(Integer(4))
>>> PQ = posets.NoncrossingPartitions(SymmetricGroup(Integer(4)))
>>> P.core_label_order().is_isomorphic(PQ)
True
```

is_congruence_uniform()

Return whether `self` is a congruence uniform lattice.

EXAMPLES:

```
sage: posets.TamariLattice(4).is_congruence_uniform()
True
```

```
>>> from sage.all import *
>>> posets.TamariLattice(Integer(4)).is_congruence_uniform()
True
```

extra_super_categories()

Return a list of the super categories of `self`.

These encode implications between properties.

EXAMPLES:

```
sage: FiniteLatticePosets().CongruenceUniform().super_categories()
[Category of finite lattice posets,
 Category of congruence uniform lattice posets]
```

```
>>> from sage.all import *
>>> FiniteLatticePosets().CongruenceUniform().super_categories()
[Category of finite lattice posets,
 Category of congruence uniform lattice posets]
```

class Extremal(*base_category*)

Bases: *CategoryWithAxiom*

The category of extremal uniform lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().Extremal(); cat
Category of finite extremal lattice posets

sage: cat.super_categories()
[Category of finite lattice posets,
 Category of extremal lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().Extremal(); cat
Category of finite extremal lattice posets

>>> cat.super_categories()
[Category of finite lattice posets,
 Category of extremal lattice posets]
```

class ParentMethods

Bases: object

is_extremal()

Return whether self is an extremal lattice.

EXAMPLES:

```
sage: posets.TamariLattice(4).is_extremal()
True
```

```
>>> from sage.all import *
>>> posets.TamariLattice(Integer(4)).is_extremal()
True
```

Finite

alias of *FiniteLatticePosets*

class ParentMethods

Bases: object

join(*x*, *y*)

Return the join of x and y in this lattice.

INPUT:

- x, y – elements of self

EXAMPLES:

```
sage: D = LatticePoset((divisors(60), attrcall("divides")))                #␣
↪needs sage.graphs sage.modules
sage: D.join( D(6), D(10) )                                                #␣
↪needs sage.graphs sage.modules
30
```

```
>>> from sage.all import *
>>> D = LatticePoset((divisors(Integer(60)), attrcall("divides")))
↪     # needs sage.graphs sage.modules
>>> D.join( D(Integer(6)), D(Integer(10)) )
↪              # needs sage.graphs sage.modules
30
```

meet(*x*, *y*)

Return the meet of x and y in this lattice.

INPUT:

- `x`, `y` – elements of `self`

EXAMPLES:

```
sage: D = LatticePoset((divisors(30), attrcall("divides")))
↪needs sage.graphs sage.modules
sage: D.meet( D(6), D(15) )
↪needs sage.graphs sage.modules
3
```

```
>>> from sage.all import *
>>> D = LatticePoset((divisors(Integer(30)), attrcall("divides")))
↪     # needs sage.graphs sage.modules
>>> D.meet( D(Integer(6)), D(Integer(15)) )
↪              # needs sage.graphs sage.modules
3
```

class Semidistributive(*base_category*)

Bases: *CategoryWithAxiom*

The category of semidistributive lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().Semidistributive(); cat
Category of finite semidistributive lattice posets

sage: cat.super_categories()
[Category of finite lattice posets,
 Category of semidistributive lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().Semidistributive(); cat
Category of finite semidistributive lattice posets

>>> cat.super_categories()
[Category of finite lattice posets,
 Category of semidistributive lattice posets]
```

class ParentMethods

Bases: `object`

is_semidistributive()

Return whether `self` is a semidistributive lattice.

EXAMPLES:

```
sage: posets.TamariLattice(4).is_semidistributive()
True
```

```
>>> from sage.all import *
>>> posets.TamariLattice(Integer(4)).is_semidistributive()
True
```

kappa(a)

Return the maximum element greater than the element covered by `a` but not greater than `a`.

Define $\kappa(a)$ as the maximum element of $(\uparrow a_*) \setminus (\uparrow a)$, where a_* is the element covered by a. It is always a meet-irreducible element, if it exists.

INPUT:

- `a` – a join-irreducible element of the lattice

Warning

Element `a` is expected to be join-irreducible, and this is *not* checked.

OUTPUT:

the element $\kappa(a)$

This will raise a `ValueError` if there is not a unique greatest element with given constraints.

EXAMPLES:

```
sage: V = ['b', 0, 1, 2, 3, 4, 't']
sage: C = [['b', 0], ['b', 1], [0, 2], [2, 3], [2, 4], [3, 't'], [1,␣
↪4], [4, 't']]
sage: L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
sage: [(a, L.kappa(a)) for a in L.join_irreducibles()]
[(0, 1), (2, 0), (3, 4), (1, 3)]
```

```
>>> from sage.all import *
>>> V = ['b', Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(4), 't']
>>> C = [['b', Integer(0)], ['b', Integer(1)], [Integer(0),␣
↪Integer(2)], [Integer(2), Integer(3)], [Integer(2), Integer(4)],␣
↪[Integer(3), 't'], [Integer(1), Integer(4)], [Integer(4), 't']]
>>> L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
>>> [(a, L.kappa(a)) for a in L.join_irreducibles()]
[(0, 1), (2, 0), (3, 4), (1, 3)]
```

kappa_dual(a)

Return the minimum element smaller than the element covering `a` but not smaller than `a`.

Define $\kappa^*(a)$ as the minimum element of $(\downarrow a_*) \setminus (\downarrow a)$, where a_* is the element covering a. It is always a join-irreducible element, if it exists.

INPUT:

- a – a meet-irreducible element of the lattice

Warning

Element a is expected to be meet-irreducible, and this is *not* checked.

OUTPUT:

the element $\kappa^*(a)$

This will raise a ValueError if there is not a unique greatest element with given constraints.

EXAMPLES:

```
sage: V = ['b', 0, 1, 2, 3, 4, 't']
sage: C = [['b', 0], ['b', 1], [0, 2], [2, 3], [2, 4], [3, 't'], [1,␣
↪4], [4, 't']]
sage: L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
sage: [(a, L.kappa_dual(a)) for a in L.meet_irreducibles()]
[(0, 2), (3, 1), (1, 0), (4, 3)]
```

```
>>> from sage.all import *
>>> V = ['b', Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(4), 't']
>>> C = [['b', Integer(0)], ['b', Integer(1)], [Integer(0),␣
↪Integer(2)], [Integer(2), Integer(3)], [Integer(2), Integer(4)],␣
↪[Integer(3), 't'], [Integer(1), Integer(4)], [Integer(4), 't']]
>>> L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
>>> [(a, L.kappa_dual(a)) for a in L.meet_irreducibles()]
[(0, 2), (3, 1), (1, 0), (4, 3)]
```

rowmotion_semidistributive(a)

Return the image of the element a under semidistributive rowmotion in self.

Classical rowmotion is usually defined as an automorphism on the set of order ideals $J(P)$ of a finite poset P. It is a special case of semidistributive rowmotion because every distributive lattice is isomorphic to $J(P)$ for some P by Birkhoff's representation theorem.

See also

If the image of rowmotion of several elements is needed, semidistributive_rowmotion() from sage.dynamics.finite_dynamical_system_catalog is much more efficient.

EXAMPLES:

```
sage: V = ['b', 0, 1, 2, 3, 4, 't']
sage: C = [['b', 0], ['b', 1], [0, 2], [2, 3], [2, 4], [3, 't'], [1,␣
↪4], [4, 't']]
sage: L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
sage: L.rowmotion_semidistributive(0)
```

(continues on next page)

(continued from previous page)

```
2

sage: L = posets.TamariLattice(3)
sage: row = L.rowmotion_semidistributive
sage: DS = DiscreteDynamicalSystem(L, row)
sage: sorted([sorted([DyckWord(x[:-1]) for x in c]) for c in DS.
↪cycles()])
[[[1, 0, 1, 0, 1, 0], [1, 1, 1, 0, 0, 0]],
 [[1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [1, 1, 0, 1, 0, 0]]]
sage: L = posets.TamariLattice(4)
sage: L.rowmotion_semidistributive((1,1,0,1,1,0,0,0,0))
(1, 0, 1, 1, 0, 0, 1, 0, 0)
```

```
>>> from sage.all import *
>>> V = ['b', Integer(0), Integer(1), Integer(2), Integer(3),␣
↪Integer(4), 't']
>>> C = [['b', Integer(0)], ['b', Integer(1)], [Integer(0),␣
↪Integer(2)], [Integer(2), Integer(3)], [Integer(2), Integer(4)],␣
↪[Integer(3), 't'], [Integer(1), Integer(4)], [Integer(4), 't']]
>>> L = LatticePoset([V, C], category=LatticePosets().Finite().
↪Semidistributive())
>>> L.rowmotion_semidistributive(Integer(0))
2

>>> L = posets.TamariLattice(Integer(3))
>>> row = L.rowmotion_semidistributive
>>> DS = DiscreteDynamicalSystem(L, row)
>>> sorted([sorted([DyckWord(x[:-Integer(1)]) for x in c]) for c in DS.
↪cycles()])
[[[1, 0, 1, 0, 1, 0], [1, 1, 1, 0, 0, 0]],
 [[1, 0, 1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [1, 1, 0, 1, 0, 0]]]
>>> L = posets.TamariLattice(Integer(4))
>>> L.rowmotion_semidistributive((Integer(1),Integer(1),Integer(0),
↪Integer(1),Integer(1),Integer(0),Integer(0),Integer(0),Integer(0)))
(1, 0, 1, 1, 0, 0, 1, 0, 0)
```

Check that classical rowmotion is a special case of semidistributive rowmotion:

```
sage: T = posets.TamariLattice(3)
sage: L = T.order_ideals_lattice()
sage: all(L.rowmotion_semidistributive(a) == T.rowmotion(a) for a in L)
True

sage: P = posets.UpDownPoset(10)
sage: L = T.order_ideals_lattice()
sage: all(L.rowmotion_semidistributive(a) == T.rowmotion(a) for a in L)
True
```

```
>>> from sage.all import *
>>> T = posets.TamariLattice(Integer(3))
>>> L = T.order_ideals_lattice()
>>> all(L.rowmotion_semidistributive(a) == T.rowmotion(a) for a in L)
```

(continues on next page)

(continued from previous page)

```
True

>>> P = posets.UpDownPoset(Integer(10))
>>> L = T.order_ideals_lattice()
>>> all(L.rowmotion_semidistributive(a) == T.rowmotion(a) for a in L)
True
```

spine()

Return the spine of `self`.

For a semidistributive lattice L, the *spine* of L is the distributive lattice constructed as the subposet on the union of longest maximal chains.

EXAMPLES:

```
sage: P = posets.TamariLattice(4)
sage: S = P.spine(); S
Finite lattice containing 8 elements
sage: S.category()
Category of facade finite enumerated distributive lattices
```

```
>>> from sage.all import *
>>> P = posets.TamariLattice(Integer(4))
>>> S = P.spine(); S
Finite lattice containing 8 elements
>>> S.category()
Category of facade finite enumerated distributive lattices
```

class Stone(*base_category*)

Bases: *CategoryWithAxiom*

The category of Stone lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().Stone(); cat
Category of finite stone distributive lattices

sage: cat.super_categories()
[Category of finite distributive lattices,
 Category of stone lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().Stone(); cat
Category of finite stone distributive lattices

>>> cat.super_categories()
[Category of finite distributive lattices,
 Category of stone lattice posets]
```

class ParentMethods

Bases: `object`

is_stone()

Return whether `self` is a Stone lattice.

EXAMPLES:

```
sage: posets.DivisorLattice(12).is_stone()
True
```

```
>>> from sage.all import *
>>> posets.DivisorLattice(Integer(12)).is_stone()
True
```

extra_super_categories()

Return a list of the super categories of `self`.

These encode implications between properties.

EXAMPLES:

```
sage: FiniteLatticePosets().Stone().super_categories()
[Category of finite distributive lattices,
 Category of stone lattice posets]
```

```
>>> from sage.all import *
>>> FiniteLatticePosets().Stone().super_categories()
[Category of finite distributive lattices,
 Category of stone lattice posets]
```

class SubcategoryMethods

Bases: `object`

ChainGraded()

A lattice is graded if all maximal chains have the same length.

To avoid possible confusion, the name of the axiom is `ChainGraded`.

EXAMPLES:

```
sage: P = posets.DivisorLattice(24)
sage: P in FiniteLatticePosets().ChainGraded()
True
```

```
>>> from sage.all import *
>>> P = posets.DivisorLattice(Integer(24))
>>> P in FiniteLatticePosets().ChainGraded()
True
```

CongruenceUniform()

A finite lattice $(L, \vee, \wedge)$ is congruence uniform if it can be constructed by a sequence of interval doublings starting with the lattice with one element.

EXAMPLES:

```
sage: P = posets.TamariLattice(2)
sage: P in FiniteLatticePosets().CongruenceUniform()
True
```

```
>>> from sage.all import *
>>> P = posets.TamariLattice(Integer(2))
>>> P in FiniteLatticePosets().CongruenceUniform()
True
```

Distributive()

A lattice $(L, \vee, \wedge)$ is distributive if meet distributes over join: $x \wedge (y \vee z) = (x \wedge y) \vee (x \wedge z)$ for every $x, y, z \in L$.

From duality in lattices, it follows that then also join distributes over meet.

A distributive lattice is always graded.

See Wikipedia article Distributive lattice.

EXAMPLES:

```
sage: P = posets.ChainPoset(2).order_ideals_lattice()
sage: P in FiniteLatticePosets().Distributive()
True
```

```
>>> from sage.all import *
>>> P = posets.ChainPoset(Integer(2)).order_ideals_lattice()
>>> P in FiniteLatticePosets().Distributive()
True
```

Extremal()

A finite lattice $(L, \vee, \wedge)$ is extremal if if it has a chain of length n (containing $n + 1$ elements) and exactly n join-irreducibles and n meet-irreducibles.

This notion was defined by George Markowsky.

EXAMPLES:

```
sage: P = posets.TamariLattice(2)
sage: P in FiniteLatticePosets().Extremal()
True
```

```
>>> from sage.all import *
>>> P = posets.TamariLattice(Integer(2))
>>> P in FiniteLatticePosets().Extremal()
True
```

Semidistributive()

A finite lattice $(L, \vee, \wedge)$ is semidistributive if it is both join-semidistributive and meet-semidistributive.

A finite lattice is join-semidistributive if for all elements e, x, y in the lattice we have

$$e \vee x = e \vee y \implies e \vee x = e \vee (x \wedge y)$$

Meet-semidistributivity is the dual property.

EXAMPLES:

```
sage: P = posets.TamariLattice(2)
sage: P in FiniteLatticePosets().Semidistributive()
True
```

```
>>> from sage.all import *
>>> P = posets.TamariLattice(Integer(2))
>>> P in FiniteLatticePosets().Semidistributive()
True
```

Stone()

A Stone lattice $(L, \vee, \wedge)$ is a pseudo-complemented distributive lattice such that $a^* \vee a^{**} = 1$.

See Wikipedia article Stone algebra.

EXAMPLES:

```
sage: P = posets.DivisorLattice(24)
sage: P in FiniteLatticePosets().Stone()
True
```

```
>>> from sage.all import *
>>> P = posets.DivisorLattice(Integer(24))
>>> P in FiniteLatticePosets().Stone()
True
```

Trim()

A finite lattice $(L, \vee, \wedge)$ is trim if it is extremal and left modular.

This notion is defined in [**?**].

EXAMPLES:

```
sage: P = posets.TamariLattice(2)
sage: P in FiniteLatticePosets().Trim()
True
```

```
>>> from sage.all import *
>>> P = posets.TamariLattice(Integer(2))
>>> P in FiniteLatticePosets().Trim()
True
```

class Trim(*base_category*)

Bases: *CategoryWithAxiom*

The category of trim uniform lattices.

EXAMPLES:

```
sage: cat = FiniteLatticePosets().Trim(); cat
Category of finite trim lattice posets
sage: cat.super_categories()
[Category of finite lattice posets,
 Category of trim lattice posets]
```

```
>>> from sage.all import *
>>> cat = FiniteLatticePosets().Trim(); cat
Category of finite trim lattice posets
>>> cat.super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of finite lattice posets,
 Category of trim lattice posets]
```

ChainGraded

alias of *DistributiveLattices*

class ParentMethods

Bases: object

is_trim()

Return whether self is a trim lattice.

EXAMPLES:

```
sage: posets.TamariLattice(4).is_trim()
True
```

```
>>> from sage.all import *
>>> posets.TamariLattice(Integer(4)).is_trim()
True
```

extra_super_categories()

Return a list of the super categories of self.

These encode implications between properties.

EXAMPLES:

```
sage: FiniteLatticePosets().Trim().super_categories()
[Category of finite lattice posets,
 Category of trim lattice posets]
```

```
>>> from sage.all import *
>>> FiniteLatticePosets().Trim().super_categories()
[Category of finite lattice posets,
 Category of trim lattice posets]
```

super_categories()

Return a list of the (immediate) super categories of self, as per *Category.super_categories()*.

EXAMPLES:

```
sage: LatticePosets().super_categories()
[Category of posets]
```

```
>>> from sage.all import *
>>> LatticePosets().super_categories()
[Category of posets]
```

4.109 Left modules

class sage.categories.left_modules.**LeftModules**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of left modules left modules over an rng (ring not necessarily with unit), i.e. an abelian group with left multiplication by elements of the rng

EXAMPLES:

```
sage: LeftModules(ZZ)
Category of left modules over Integer Ring
sage: LeftModules(ZZ).super_categories()
[Category of commutative additive groups]
```

```
>>> from sage.all import *
>>> LeftModules(ZZ)
Category of left modules over Integer Ring
>>> LeftModules(ZZ).super_categories()
[Category of commutative additive groups]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: LeftModules(QQ).super_categories()
[Category of commutative additive groups]
```

```
>>> from sage.all import *
>>> LeftModules(QQ).super_categories()
[Category of commutative additive groups]
```

4.110 Lie Algebras

AUTHORS:

- Travis Scrimshaw (07-15-2013): Initial implementation

class sage.categories.lie_algebras.**LieAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Lie algebras.

EXAMPLES:

```
sage: C = LieAlgebras(QQ); C
Category of Lie algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> C = LieAlgebras(QQ); C
Category of Lie algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of vector spaces over Rational Field]
```

We construct a typical parent in this category, and do some computations with it:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: A = C.example(); A
An example of a Lie algebra: the Lie algebra from the associative
 algebra Symmetric group algebra of order 3 over Rational Field
 generated by ([2, 1, 3], [2, 3, 1])
sage: A.category()
Category of Lie algebras over Rational Field
sage: A.base_ring()
Rational Field
sage: a, b = A.lie_algebra_generators()
sage: a.bracket(b)
-[1, 3, 2] + [3, 2, 1]
sage: b.bracket(2*a + b)
2*[1, 3, 2] - 2*[3, 2, 1]
sage: A.bracket(a, b)
-[1, 3, 2] + [3, 2, 1]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> A = C.example(); A
An example of a Lie algebra: the Lie algebra from the associative
 algebra Symmetric group algebra of order 3 over Rational Field
 generated by ([2, 1, 3], [2, 3, 1])
>>> A.category()
Category of Lie algebras over Rational Field
>>> A.base_ring()
Rational Field
>>> a, b = A.lie_algebra_generators()
>>> a.bracket(b)
-[1, 3, 2] + [3, 2, 1]
>>> b.bracket(Integer(2)*a + b)
2*[1, 3, 2] - 2*[3, 2, 1]
>>> A.bracket(a, b)
-[1, 3, 2] + [3, 2, 1]
```

Please see the source code of A (with `A??`) for how to implement other Lie algebras.

Todo

Many of these tests should use Lie algebras that are not the minimal example and need to be added after Issue #16820 (and Issue #16823).

class ElementMethods

Bases: `object`

bracket(*rhs*)

Return the Lie bracket `[self, rhs]`.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: L = LieAlgebras(QQ).example()
sage: x,y = L.lie_algebra_generators()
sage: x.bracket(y)
-[1, 3, 2] + [3, 2, 1]
sage: x.bracket(0)
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> L = LieAlgebras(QQ).example()
>>> x,y = L.lie_algebra_generators()
>>> x.bracket(y)
-[1, 3, 2] + [3, 2, 1]
>>> x.bracket(Integer(0))
0
```

exp(*lie_group=None*)

Return the exponential of `self` in `lie_group`.

INPUT:

- `lie_group` – (optional) the Lie group to map into; If `lie_group` is not given, the Lie group associated to the parent Lie algebra of `self` is used.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules sage.symbolic
sage: L.<X,Y,Z> = LieAlgebra(QQ, 2, step=2)
sage: g = (X + Y + Z).exp(); g
exp(X + Y + Z)
sage: h = X.exp(); h
exp(X)
sage: g.parent()
Lie group G of Free Nilpotent Lie algebra of rank 2 and step 2
 over Rational Field
sage: g.parent() is h.parent()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules sage.symbolic
>>> L = LieAlgebra(QQ, Integer(2), step=Integer(2), names=('X', 'Y', 'Z',
↪)); (X, Y, Z,) = L._first_ngens(3)
>>> g = (X + Y + Z).exp(); g
exp(X + Y + Z)
>>> h = X.exp(); h
exp(X)
>>> g.parent()
Lie group G of Free Nilpotent Lie algebra of rank 2 and step 2
 over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> g.parent() is h.parent()
True
```

The Lie group can be specified explicitly:

```
sage: # needs sage.combinat sage.modules sage.symbolic
sage: H = L.lie_group('H')
sage: k = Z.exp(lie_group=H); k
exp(Z)
sage: k.parent()
Lie group H of Free Nilpotent Lie algebra of rank 2 and step 2
 over Rational Field
sage: g.parent() == k.parent()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules sage.symbolic
>>> H = L.lie_group('H')
>>> k = Z.exp(lie_group=H); k
exp(Z)
>>> k.parent()
Lie group H of Free Nilpotent Lie algebra of rank 2 and step 2
 over Rational Field
>>> g.parent() == k.parent()
False
```

killing_form(*x*)

Return the Killing form of `self` and `x`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
sage: a.killing_form(b)                                                  #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> a.killing_form(b)                                                  #␣
↪needs sage.modules
0
```

lift()

Return the image of `self` under the canonical lift from the Lie algebra to its universal enveloping algebra.

EXAMPLES:

```
sage: # needs sage.combinat sage.libs.singularsage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: elt = 3*a + b - c
sage: elt.lift()
3*b0 + b1 - b2
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.libs.singularsage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> elt = Integer(3)*a + b - c
>>> elt.lift()
3*b0 + b1 - b2
```

```
sage: L.<x,y> = LieAlgebra(QQ, abelian=True)                                #␣
↪needs sage.combinat sage.modules
sage: x.lift()                                                              #␣
↪needs sage.combinat sage.modules
x
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, abelian=True, names=('x', 'y',)); (x, y,) = L._
↪first_ngens(2)# needs sage.combinat sage.modules
>>> x.lift()                                                                #␣
↪needs sage.combinat sage.modules
x
```

to_vector(*order=None*)

Return the vector in `g.module()` corresponding to the element `self` of `g` (where `g` is the parent of `self`).

Implement this if you implement `g.module()`. See *module()* for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()         #␣
↪needs sage.modules
sage: u = L((1, 0, 0)).to_vector(); u                                       #␣
↪needs sage.modules
(1, 0, 0)
sage: parent(u)                                                             #␣
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()        #␣
↪needs sage.modules
>>> u = L((Integer(1), Integer(0), Integer(0))).to_vector(); u             ␣
↪                        # needs sage.modules
(1, 0, 0)
>>> parent(u)                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

WithBasis

alias of *FiniteDimensionalLieAlgebrasWithBasis*

extra_super_categories()

Implement the fact that a finite dimensional Lie algebra over a finite ring is finite.

EXAMPLES:

```
sage: LieAlgebras(IntegerModRing(4)).FiniteDimensional().extra_super_
↪categories()
[Category of finite sets]
sage: LieAlgebras(ZZ).FiniteDimensional().extra_super_categories()
[]
sage: C = LieAlgebras(GF(5)).FiniteDimensional()
sage: C.is_subcategory(Sets().Finite())
True
sage: C = LieAlgebras(ZZ).FiniteDimensional()
sage: C.is_subcategory(Sets().Finite())
False
sage: C = LieAlgebras(GF(5)).WithBasis().FiniteDimensional()
sage: C.is_subcategory(Sets().Finite())
True
```

```
>>> from sage.all import *
>>> LieAlgebras(IntegerModRing(Integer(4))).FiniteDimensional().extra_
↪super_categories()
[Category of finite sets]
>>> LieAlgebras(ZZ).FiniteDimensional().extra_super_categories()
[]
>>> C = LieAlgebras(GF(Integer(5))).FiniteDimensional()
>>> C.is_subcategory(Sets().Finite())
True
>>> C = LieAlgebras(ZZ).FiniteDimensional()
>>> C.is_subcategory(Sets().Finite())
False
>>> C = LieAlgebras(GF(Integer(5))).WithBasis().FiniteDimensional()
>>> C.is_subcategory(Sets().Finite())
True
```

Graded

alias of *GradedLieAlgebras*

class Nilpotent(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of nilpotent Lie algebras.

class ParentMethods

Bases: object

is_nilpotent()

Return True since self is nilpotent.

EXAMPLES:

```
sage: h = lie_algebras.Heisenberg(ZZ, oo)                                   #␣
↪needs sage.combinat sage.modules
sage: h.is_nilpotent()                                                      #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> h = lie_algebras.Heisenberg(ZZ, oo)                                     #␣
↪needs sage.combinat sage.modules
>>> h.is_nilpotent()                                                        #␣
↪needs sage.combinat sage.modules
True
```

step()

Return the nilpotency step of self.

EXAMPLES:

```
sage: h = lie_algebras.Heisenberg(ZZ, oo)                                   #␣
↪needs sage.combinat sage.modules
sage: h.step()                                                              #␣
↪needs sage.combinat sage.modules
2
```

```
>>> from sage.all import *
>>> h = lie_algebras.Heisenberg(ZZ, oo)                                     #␣
↪needs sage.combinat sage.modules
>>> h.step()                                                                #␣
↪needs sage.combinat sage.modules
2
```

class ParentMethods

Bases: object

baker_campbell_hausdorff(*X*, *Y*, *prec=None*)

alias of *bch()*.

bch(*X*, *Y*, *prec=None*)

Return the element $\log(\exp(X)\exp(Y))$.

The BCH formula is an expression for $\log(\exp(X)\exp(Y))$ as a sum of Lie brackets of X and Y with rational coefficients. It is only defined if the base ring of self has a coercion from the rationals.

INPUT:

- X – an element of self
- Y – an element of self
- prec – integer; the maximum length of Lie brackets to be considered in the formula

EXAMPLES:

The BCH formula for the generators of a free nilpotent Lie algebra of step 4:

```
sage: L = LieAlgebra(QQ, 2, step=4)                                    #␣
↪needs sage.combinat sage.modules
sage: L.inject_variables()                                             #␣
↪needs sage.combinat sage.modules
Defining X_1, X_2, X_12, X_112, X_122, X_1112, X_1122, X_1222
sage: L.bch(X_1, X_2)                                                  #␣
↪needs sage.combinat sage.modules
X_1 + X_2 + 1/2*X_12 + 1/12*X_112 + 1/12*X_122 + 1/24*X_1122
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, Integer(2), step=Integer(4))                     ␣
↪              # needs sage.combinat sage.modules
>>> L.inject_variables()                                              #␣
↪needs sage.combinat sage.modules
Defining X_1, X_2, X_12, X_112, X_122, X_1112, X_1122, X_1222
>>> L.bch(X_1, X_2)                                                   #␣
↪needs sage.combinat sage.modules
X_1 + X_2 + 1/2*X_12 + 1/12*X_112 + 1/12*X_122 + 1/24*X_1122
```

An example of the BCH formula in a quotient:

```
sage: Q = L.quotient(X_112 + X_122)                                    #␣
↪needs sage.combinat sage.modules
sage: x, y = Q.basis().list()[:2]                                      #␣
↪needs sage.combinat sage.modules
sage: Q.bch(x, y)                                                      #␣
↪needs sage.combinat sage.modules
X_1 + X_2 + 1/2*X_12 - 1/24*X_1112
```

```
>>> from sage.all import *
>>> Q = L.quotient(X_112 + X_122)                                     #␣
↪needs sage.combinat sage.modules
>>> x, y = Q.basis().list()[:Integer(2)]                                ␣
↪     # needs sage.combinat sage.modules
>>> Q.bch(x, y)                                                       #␣
↪needs sage.combinat sage.modules
X_1 + X_2 + 1/2*X_12 - 1/24*X_1112
```

The BCH formula for a non-nilpotent Lie algebra requires the precision to be explicitly stated:

```
sage: L.<X,Y> = LieAlgebra(QQ)                                         #␣
↪needs sage.combinat sage.modules
sage: L.bch(X, Y)                                                      #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: the Lie algebra is not known to be nilpotent,
 so you must specify the precision
sage: L.bch(X, Y, 4)                                                   #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
X + 1/12*[X, [X, Y]] + 1/24*[X, [[X, Y], Y]]
+ 1/2*[X, Y] + 1/12*[[X, Y], Y] + Y
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, names=('X', 'Y',)); (X, Y,) = L._first_ngens(2)#␣
↪needs sage.combinat sage.modules
>>> L.bch(X, Y)                                                          #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
ValueError: the Lie algebra is not known to be nilpotent,
 so you must specify the precision
>>> L.bch(X, Y, Integer(4))                                               ␣
↪     # needs sage.combinat sage.modules
X + 1/12*[X, [X, Y]] + 1/24*[X, [[X, Y], Y]]
+ 1/2*[X, Y] + 1/12*[[X, Y], Y] + Y
```

The BCH formula requires a coercion from the rationals:

```
sage: L.<X,Y,Z> = LieAlgebra(ZZ, 2, step=2)                              #␣
↪needs sage.combinat sage.modules
sage: L.bch(X, Y)                                                        #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
TypeError: the BCH formula is not well defined
since Integer Ring has no coercion from Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebra(ZZ, Integer(2), step=Integer(2), names=('X', 'Y', 'Z',
↪)); (X, Y, Z,) = L._first_ngens(3)# needs sage.combinat sage.modules
>>> L.bch(X, Y)                                                          #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
...
TypeError: the BCH formula is not well defined
since Integer Ring has no coercion from Rational Field
```

We use the free nilpotent Lie algebra to approximate the BCH formula in the nilpotent completion of the free Lie algebra, where the BCH formula can be treated as a formal group law (we thank Greg Kuperberg for this example):

```
sage: L = LieAlgebra(QQ, 2, step=4)
sage: L.options.display = "brackets"
sage: a, b = L.basis(1)
sage: L.bch(a, b)
X_1 + X_2 + 1/2*[X_1, X_2] + 1/12*[X_1, [X_1, X_2]]
 + 1/12*[[X_1, X_2], X_2] + 1/24*[X_1, [[X_1, X_2], X_2]]
sage: L.bch(L.bch(L.bch(a, b), -a), -b)  # the commutator
[X_1, X_2] + 1/2*[X_1, [X_1, X_2]] - 1/2*[[X_1, X_2], X_2]
 + 1/6*[X_1, [X_1, [X_1, X_2]]] - 1/4*[X_1, [[X_1, X_2], X_2]]
 + 1/6*[[[X_1, X_2], X_2], X_2]
```

(continues on next page)

(continued from previous page)

```
sage: L.options._reset()  # reset the printing options
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, Integer(2), step=Integer(4))
>>> L.options.display = "brackets"
>>> a, b = L.basis(Integer(1))
>>> L.bch(a, b)
X_1 + X_2 + 1/2*[X_1, X_2] + 1/12*[X_1, [X_1, X_2]]
 + 1/12*[[X_1, X_2], X_2] + 1/24*[X_1, [[X_1, X_2], X_2]]
>>> L.bch(L.bch(L.bch(a, b), -a), -b)  # the commutator
[X_1, X_2] + 1/2*[X_1, [X_1, X_2]] - 1/2*[[X_1, X_2], X_2]
 + 1/6*[X_1, [X_1, [X_1, X_2]]] - 1/4*[X_1, [[X_1, X_2], X_2]]
 + 1/6*[[[X_1, X_2], X_2], X_2]
>>> L.options._reset()  # reset the printing options
```

bracket(*lhs*, *rhs*)

Return the Lie bracket `[lhs, rhs]` after coercing `lhs` and `rhs` into elements of `self`.

If `lhs` and `rhs` are Lie algebras, then this constructs the product space, and if only one of them is a Lie algebra, then it constructs the corresponding ideal.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: L = LieAlgebras(QQ).example()
sage: x, y = L.lie_algebra_generators()
sage: L.bracket(x, x + y)
-[1, 3, 2] + [3, 2, 1]
sage: L.bracket(x, 0)
0
sage: L.bracket(0, x)
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> L = LieAlgebras(QQ).example()
>>> x, y = L.lie_algebra_generators()
>>> L.bracket(x, x + y)
-[1, 3, 2] + [3, 2, 1]
>>> L.bracket(x, Integer(0))
0
>>> L.bracket(Integer(0), x)
0
```

Constructing the product space:

```
sage: L = lie_algebras.Heisenberg(QQ, 1)                                    #␣
↪needs sage.combinat sage.modules
sage: Z = L.bracket(L, L); Z                                                #␣
↪needs sage.combinat sage.modules
Ideal (z) of Heisenberg algebra of rank 1 over Rational Field
sage: L.bracket(L, Z)                                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
Ideal () of Heisenberg algebra of rank 1 over Rational Field
```

```
>>> from sage.all import *
>>> L = lie_algebras.Heisenberg(QQ, Integer(1))                              ␣
↪     # needs sage.combinat sage.modules
>>> Z = L.bracket(L, L); Z                                                  #␣
↪needs sage.combinat sage.modules
Ideal (z) of Heisenberg algebra of rank 1 over Rational Field
>>> L.bracket(L, Z)                                                         #␣
↪needs sage.combinat sage.modules
Ideal () of Heisenberg algebra of rank 1 over Rational Field
```

Constructing ideals:

```
sage: p, q, z = L.basis(); p, q, z                                          #␣
↪needs sage.combinat sage.modules
(p1, q1, z)
sage: L.bracket(3*p, L)                                                     #␣
↪needs sage.combinat sage.modules
Ideal (3*p1) of Heisenberg algebra of rank 1 over Rational Field
sage: L.bracket(L, q + p)                                                   #␣
↪needs sage.combinat sage.modules
Ideal (p1 + q1) of Heisenberg algebra of rank 1 over Rational Field
```

```
>>> from sage.all import *
>>> p, q, z = L.basis(); p, q, z                                            #␣
↪needs sage.combinat sage.modules
(p1, q1, z)
>>> L.bracket(Integer(3)*p, L)                                              ␣
↪     # needs sage.combinat sage.modules
Ideal (3*p1) of Heisenberg algebra of rank 1 over Rational Field
>>> L.bracket(L, q + p)                                                     #␣
↪needs sage.combinat sage.modules
Ideal (p1 + q1) of Heisenberg algebra of rank 1 over Rational Field
```

center_universal_enveloping_algebra(*UEA=None*)

Return the center of the universal enveloping algebra of `self`.

EXAMPLES:

```
sage: L = LieAlgebra(QQ, 3, 'x', abelian=True)
sage: L.center_universal_enveloping_algebra()
Center of Universal enveloping algebra of Abelian Lie algebra on 3␣
↪generators (x0, x1, x2)
 over Rational Field in the Poincare-Birkhoff-Witt basis
sage: PBW = L.pbw_basis()
sage: L.center_universal_enveloping_algebra(PBW)
Center of Universal enveloping algebra of Abelian Lie algebra on 3␣
↪generators (x0, x1, x2)
 over Rational Field in the Poincare-Birkhoff-Witt basis
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, Integer(3), 'x', abelian=True)
>>> L.center_universal_enveloping_algebra()
Center of Universal enveloping algebra of Abelian Lie algebra on 3␣
↪generators (x0, x1, x2)
 over Rational Field in the Poincare-Birkhoff-Witt basis
>>> PBW = L.pbw_basis()
>>> L.center_universal_enveloping_algebra(PBW)
Center of Universal enveloping algebra of Abelian Lie algebra on 3␣
↪generators (x0, x1, x2)
 over Rational Field in the Poincare-Birkhoff-Witt basis
```

from_vector(*v*, *order=None*, *coerce=False*)

Return the element of `self` corresponding to the vector `v` in `self.module()`.

Implement this if you implement *module()*; see the documentation of the latter for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: u = L.from_vector(vector(QQ, (1, 0, 0))); u                        #␣
↪needs sage.modules
(1, 0, 0)
sage: parent(u) is L                                                     #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> u = L.from_vector(vector(QQ, (Integer(1), Integer(0), Integer(0))));␣
↪u                          # needs sage.modules
(1, 0, 0)
>>> parent(u) is L                                                     #␣
↪needs sage.modules
True
```

ideal(**gens*, ***kwds*)

Return the ideal of `self` generated by `gens`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
sage: L.ideal([2*a - c, b + c])                                          #␣
↪needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 2-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
```

(continues on next page)

(continued from previous page)

```
[   1    0 -1/2]
[   0    1    1]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> L.ideal([Integer(2)*a - c, b + c])                                   ␣
↪      # needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 2-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

```
sage: # needs sage.combinat sage.groups sage.modules
sage: L = LieAlgebras(QQ).example()
sage: x, y = L.lie_algebra_generators()
sage: L.ideal([x + y])
Traceback (most recent call last):
...
NotImplementedError: ideals not yet implemented: see #16824
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> L = LieAlgebras(QQ).example()
>>> x, y = L.lie_algebra_generators()
>>> L.ideal([x + y])
Traceback (most recent call last):
...
NotImplementedError: ideals not yet implemented: see #16824
```

is_abelian()

Return `True` if this Lie algebra is abelian.

A Lie algebra $\mathfrak{g}$ is abelian if $[x, y] = 0$ for all $x, y \in \mathfrak{g}$.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.groups
sage: L.is_abelian()                                                   #␣
↪needs sage.groups
False
sage: R = QQ['x,y']
sage: L = LieAlgebras(QQ).example(R.gens())
sage: L.is_abelian()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.groups
>>> L.is_abelian()                                                   #␣
↪needs sage.groups
False
>>> R = QQ['x,y']
>>> L = LieAlgebras(QQ).example(R.gens())
>>> L.is_abelian()
True
```

```
sage: # not implemented, needs sage.combinat sage.modules
sage: L.<x> = LieAlgebra(QQ, 1)
sage: L.is_abelian()
True
sage: L.<x,y> = LieAlgebra(QQ, 2)
sage: L.is_abelian()
False
```

```
>>> from sage.all import *
>>> # not implemented, needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, Integer(1), names=('x',)); (x,) = L._first_ngens(1)
>>> L.is_abelian()
True
>>> L = LieAlgebra(QQ, Integer(2), names=('x', 'y',)); (x, y,) = L._first_
↪ngens(2)
>>> L.is_abelian()
False
```

is_commutative()

Return if `self` is commutative.

This is equivalent to `self` being abelian.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).example()                                  #␣
↪needs sage.combinat sage.groups sage.modules
sage: L.is_commutative()                                             #␣
↪needs sage.combinat sage.groups sage.modules
False
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.combinat sage.groups sage.modules
>>> L.is_commutative()                                               #␣
↪needs sage.combinat sage.groups sage.modules
False
```

```
sage: L.<x> = LieAlgebra(QQ, 1)          # not implemented           #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
sage: L.is_commutative()                # not implemented                #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, Integer(1), names=('x',)); (x,) = L._first_ngens(1)
↪# not implemented                # needs sage.combinat sage.modules
>>> L.is_commutative()                # not implemented                #␣
↪needs sage.combinat sage.modules
True
```

is_ideal(*A*)

Return if `self` is an ideal of `A`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).example()                                     #␣
↪needs sage.combinat sage.groups sage.modules
sage: L.is_ideal(L)                                                     #␣
↪needs sage.combinat sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).example()                                     #␣
↪needs sage.combinat sage.groups sage.modules
>>> L.is_ideal(L)                                                     #␣
↪needs sage.combinat sage.groups sage.modules
True
```

is_nilpotent()

Return if `self` is a nilpotent Lie algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()     #␣
↪needs sage.modules
sage: L.is_nilpotent()                                                  #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()     #␣
↪needs sage.modules
>>> L.is_nilpotent()                                                  #␣
↪needs sage.modules
True
```

is_solvable()

Return if `self` is a solvable Lie algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.is_solvable()                                                    #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L.is_solvable()                                                    #␣
↪needs sage.modules
True
```

killing_form(*x*, *y*)

Return the Killing form of `x` and `y`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
sage: L.killing_form(a, b + c)                                           #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> L.killing_form(a, b + c)                                           #␣
↪needs sage.modules
0
```

lie_group(*name='G'*, ***kwds*)

Return the simply connected Lie group related to `self`.

INPUT:

- `name` – string (default: `'G'`); the name (symbol) given to the Lie group

EXAMPLES:

```
sage: L = lie_algebras.Heisenberg(QQ, 1)                                 #␣
↪needs sage.combinat sage.modules
sage: G = L.lie_group('G'); G                                            #␣
↪needs sage.combinat sage.modules sage.symbolic
Lie group G of Heisenberg algebra of rank 1 over Rational Field
```

```
>>> from sage.all import *
>>> L = lie_algebras.Heisenberg(QQ, Integer(1))                          ␣
↪     # needs sage.combinat sage.modules
>>> G = L.lie_group('G'); G                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules sage.symbolic
Lie group G of Heisenberg algebra of rank 1 over Rational Field
```

lift()

Construct the lift morphism from `self` to the universal enveloping algebra of `self` (the latter is implemented as *universal_enveloping_algebra()*).

This is a Lie algebra homomorphism. It is injective if `self` is a free module over its base ring, or if the base ring is a **Q**-algebra.

EXAMPLES:

```
sage: # needs sage.combinat sage.libs.singular sage.modules
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: lifted = L.lift(2*a + b - c); lifted
2*b0 + b1 - b2
sage: lifted.parent() is L.universal_enveloping_algebra()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.libs.singular sage.modules
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> lifted = L.lift(Integer(2)*a + b - c); lifted
2*b0 + b1 - b2
>>> lifted.parent() is L.universal_enveloping_algebra()
True
```

module()

Return an R-module which is isomorphic to the underlying R-module of `self`.

The rationale behind this method is to enable linear algebraic functionality on `self` (such as computing the span of a list of vectors in `self`) via an isomorphism from `self` to an R-module (typically, although not always, an R-module of the form R^n for an $n \in \mathbf{N}$) on which such functionality already exists. For this method to be of any use, it should return an R-module which has linear algebraic functionality that `self` does not have.

For instance, if `self` has ordered basis (e, f, h), then `self.module()` will be the R-module R^3, and the elements e, f and h of `self` will correspond to the basis vectors $(1, 0, 0)$, $(0, 1, 0)$ and $(0, 0, 1)$ of `self.module()`.

This method *module()* needs to be set whenever a finite-dimensional Lie algebra with basis is intended to support linear algebra (which is, e.g., used in the computation of centralizers and lower central series). One then needs to also implement the R-module isomorphism from `self` to `self.module()` in both directions; that is, implement:

- a `to_vector` ElementMethod which sends every element of `self` to the corresponding element of `self.module()`;
- a `from_vector` ParentMethod which sends every element of `self.module()` to an element of `self`.

The `from_vector` method will automatically serve as an element constructor of `self` (that is, `self(v)` for any `v` in `self.module()` will return `self.from_vector(v)`).

Todo

Ensure that this is actually so.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.module()                                                         #␣
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L.module()                                                         #␣
↪needs sage.modules
Vector space of dimension 3 over Rational Field
```

representation(*f=None*, *index_set=None*, *on_basis=False*, ***kwargs*)

Return a representation of `self`.

If no arguments are given, then this returns the trivial representation.

Currently the only implemented method of constructing a representation is by explicitly specifying the action of

- the elements of `self` by matrices;
- the basis elements of `self` using a `dict` or a `Family()`;
- a function on basis elements (either passed as `on_basis` or setting `on_basis=True`).

INPUT:

- `f` – the function that defines the action
- `index_set` – the index set of the representation
- `on_basis` – (optional) see above

See also

`RepresentationByMorphism`

EXAMPLES:

```
sage: L.<x,y> = LieAlgebra(QQ, {('x','y'): {'y':1}})
sage: f = {x: Matrix([[1,0],[0,0]]), y: Matrix([[0,1],[0,0]])}
sage: L.representation(f)
Representation of Lie algebra on 2 generators (x, y) over Rational Field␣
↪defined by:
       [1 0]
x |--> [0 0]
       [0 1]
y |--> [0 0]
sage: L.representation()
Trivial representation of Lie algebra on 2 generators (x, y) over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, {('x','y'): {'y':Integer(1)}}, names=('x', 'y',));␣
↪(x, y,) = L._first_ngens(2)
>>> f = {x: Matrix([[Integer(1),Integer(0)],[Integer(0),Integer(0)]]), y:␣
↪Matrix([[Integer(0),Integer(1)],[Integer(0),Integer(0)]])}
>>> L.representation(f)
Representation of Lie algebra on 2 generators (x, y) over Rational Field␣
↪defined by:
       [1 0]
x |--> [0 0]
       [0 1]
y |--> [0 0]
>>> L.representation()
Trivial representation of Lie algebra on 2 generators (x, y) over␣
↪Rational Field
```

subalgebra(*gens*, *names=None*, *index_set=None*, *category=None*)

Return the subalgebra of `self` generated by `gens`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
sage: L.subalgebra([2*a - c, b + c])                                     #␣
↪needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 2-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> a, b, c = L.lie_algebra_generators()                               #␣
↪needs sage.modules
>>> L.subalgebra([Integer(2)*a - c, b + c])                              ␣
↪     # needs sage.modules
An example of a finite dimensional Lie algebra with basis:
 the 2-dimensional abelian Lie algebra over Rational Field
 with basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

```
sage: # needs sage.combinat sage.groups sage.modules
sage: L = LieAlgebras(QQ).example()
sage: x,y = L.lie_algebra_generators()
sage: L.subalgebra([x + y])
Traceback (most recent call last):
...
NotImplementedError: subalgebras not yet implemented: see #17416
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> L = LieAlgebras(QQ).example()
>>> x,y = L.lie_algebra_generators()
>>> L.subalgebra([x + y])
Traceback (most recent call last):
...
NotImplementedError: subalgebras not yet implemented: see #17416
```

trivial_representation()

Return the trivial representation of `self`.

EXAMPLES:

```
sage: L = lie_algebras.strictly_upper_triangular_matrices(QQ, 4)
sage: L.trivial_representation()
Trivial representation of Lie algebra of 4-dimensional
 strictly upper triangular matrices over Rational Field
```

```
>>> from sage.all import *
>>> L = lie_algebras.strictly_upper_triangular_matrices(QQ, Integer(4))
>>> L.trivial_representation()
Trivial representation of Lie algebra of 4-dimensional
 strictly upper triangular matrices over Rational Field
```

universal_enveloping_algebra()

Return the universal enveloping algebra of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.universal_enveloping_algebra()                                   #␣
↪needs sage.combinat sage.libs.singular sage.modules
Noncommutative Multivariate Polynomial Ring in b0, b1, b2
 over Rational Field, nc-relations: {}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L.universal_enveloping_algebra()                                   #␣
↪needs sage.combinat sage.libs.singular sage.modules
Noncommutative Multivariate Polynomial Ring in b0, b1, b2
 over Rational Field, nc-relations: {}
```

```
sage: L = LieAlgebra(QQ, 3, 'x', abelian=True)                           #␣
↪needs sage.combinat sage.modules
sage: L.universal_enveloping_algebra()                                   #␣
↪needs sage.combinat sage.libs.singular sage.modules
Multivariate Polynomial Ring in x0, x1, x2 over Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, Integer(3), 'x', abelian=True)                    ␣
```

(continues on next page)

(continued from previous page)

```
↪      # needs sage.combinat sage.modules
>>> L.universal_enveloping_algebra()                                    #␣
↪needs sage.combinat sage.libs.singular sage.modules
Multivariate Polynomial Ring in x0, x1, x2 over Rational Field
```

See also

lift()

class SubcategoryMethods

Bases: object

Nilpotent()

Return the full subcategory of nilpotent objects of self.

A Lie algebra L is nilpotent if there exist an integer s such that all iterated brackets of L of length more than s vanish. The integer s is called the nilpotency step. For instance any abelian Lie algebra is nilpotent of step 1.

EXAMPLES:

```
sage: LieAlgebras(QQ).Nilpotent()
Category of nilpotent Lie algebras over Rational Field
sage: LieAlgebras(QQ).WithBasis().Nilpotent()
Category of nilpotent Lie algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).Nilpotent()
Category of nilpotent Lie algebras over Rational Field
>>> LieAlgebras(QQ).WithBasis().Nilpotent()
Category of nilpotent Lie algebras with basis over Rational Field
```

WithBasis

alias of *LieAlgebrasWithBasis*

example(*gens=None*)

Return an example of a Lie algebra as per *Category.example*.

EXAMPLES:

```
sage: LieAlgebras(QQ).example()                                         #␣
↪needs sage.combinat sage.groups sage.modules
An example of a Lie algebra: the Lie algebra from the associative algebra
 Symmetric group algebra of order 3 over Rational Field
 generated by ([2, 1, 3], [2, 3, 1])
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).example()                                           #␣
↪needs sage.combinat sage.groups sage.modules
An example of a Lie algebra: the Lie algebra from the associative algebra
 Symmetric group algebra of order 3 over Rational Field
 generated by ([2, 1, 3], [2, 3, 1])
```

Another set of generators can be specified as an optional argument:

```
sage: F.<x,y,z> = FreeAlgebra(QQ)                                              #␣
↪needs sage.combinat sage.modules
sage: LieAlgebras(QQ).example(F.gens())                                        #␣
↪needs sage.combinat sage.modules
An example of a Lie algebra: the Lie algebra from the associative algebra
 Free Algebra on 3 generators (x, y, z) over Rational Field
 generated by (x, y, z)
```

```
>>> from sage.all import *
>>> F = FreeAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = F._first_
↪ngens(3)# needs sage.combinat sage.modules
>>> LieAlgebras(QQ).example(F.gens())                                          #␣
↪needs sage.combinat sage.modules
An example of a Lie algebra: the Lie algebra from the associative algebra
 Free Algebra on 3 generators (x, y, z) over Rational Field
 generated by (x, y, z)
```

super_categories()

EXAMPLES:

```
sage: LieAlgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

class sage.categories.lie_algebras.**LiftMorphism**(*domain*, *codomain*)

Bases: *Morphism*

The natural lifting morphism from a Lie algebra to its enveloping algebra.

4.111 Lie Algebras With Basis

AUTHORS:

- Travis Scrimshaw (07-15-2013): Initial implementation

class sage.categories.lie_algebras_with_basis.**LieAlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of Lie algebras with a basis.

class ElementMethods

Bases: object

lift()

Lift self to the universal enveloping algebra.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: S = SymmetricGroup(3).algebra(QQ)
sage: L = LieAlgebra(associative=S)
sage: x = L.gen(3)
sage: y = L.gen(1)
sage: x.lift()
b3
sage: y.lift()
b1
sage: x * y
b1*b3 + b4 - b5
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> S = SymmetricGroup(Integer(3)).algebra(QQ)
>>> L = LieAlgebra(associative=S)
>>> x = L.gen(Integer(3))
>>> y = L.gen(Integer(1))
>>> x.lift()
b3
>>> y.lift()
b1
>>> x * y
b1*b3 + b4 - b5
```

to_vector(*order=None*)

Return the vector in `g.module()` corresponding to the element `self` of `g` (where `g` is the parent of `self`).

Implement this if you implement `g.module()`. See *module()* for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.an_element().to_vector()                                         #␣
↪needs sage.modules
(0, 0, 0)
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> L.an_element().to_vector()                                         #␣
↪needs sage.modules
(0, 0, 0)
```

Todo

Doctest this implementation on an example not overshadowed.

Graded

alias of *GradedLieAlgebrasWithBasis*

class ParentMethods

Bases: object

bracket_on_basis(*x*, *y*)

Return the bracket of basis elements indexed by `x` and `y` where `x < y`. If this is not implemented, then the method `_bracket_()` for the elements must be overwritten.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()                          #␣
↪needs sage.combinat sage.modules
sage: L.bracket_on_basis(Partition([3,1]), Partition([2,2,1,1]))         #␣
↪needs sage.combinat sage.modules
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()                            #␣
↪needs sage.combinat sage.modules
>>> L.bracket_on_basis(Partition([Integer(3),Integer(1)]),␣
↪Partition([Integer(2),Integer(2),Integer(1),Integer(1)]))          #␣
↪needs sage.combinat sage.modules
0
```

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: L.dimension()                                                      #␣
↪needs sage.modules
3
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()       #␣
↪needs sage.modules
>>> L.dimension()                                                       #␣
↪needs sage.modules
3
```

```
sage: L = LieAlgebra(QQ, 'x,y', {('x','y'): {'x':1}})                    #␣
↪needs sage.combinat sage.modules
sage: L.dimension()                                                      #␣
↪needs sage.combinat sage.modules
2
```

```
>>> from sage.all import *
>>> L = LieAlgebra(QQ, 'x,y', {('x','y'): {'x':Integer(1)}})             ␣
↪     # needs sage.combinat sage.modules
>>> L.dimension()                                                       #␣
↪needs sage.combinat sage.modules
2
```

from_vector(*v*, *order=None*, *coerce=False*)

Return the element of `self` corresponding to the vector `v` in `self.module()`.

Implement this if you implement *module()*; see the documentation of *module()* for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
sage: u = L.from_vector(vector(QQ, (1, 0, 0))); u                        #␣
↪needs sage.modules
(1, 0, 0)
sage: parent(u) is L                                                     #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()      #␣
↪needs sage.modules
>>> u = L.from_vector(vector(QQ, (Integer(1), Integer(0), Integer(0))));␣
↪u                          # needs sage.modules
(1, 0, 0)
>>> parent(u) is L                                                     #␣
↪needs sage.modules
True
```

module()

Return an R-module which is isomorphic to the underlying R-module of `self`.

See *module()* for an explanation.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()                          #␣
↪needs sage.combinat sage.modules
sage: L.module()                                                         #␣
↪needs sage.combinat sage.modules
Free module generated by Partitions over Rational Field
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()                          #␣
↪needs sage.combinat sage.modules
>>> L.module()                                                         #␣
↪needs sage.combinat sage.modules
Free module generated by Partitions over Rational Field
```

pbw_basis(*basis_key=None*, ***kwds*)

Return the Poincare-Birkhoff-Witt basis of the universal enveloping algebra corresponding to `self`.

EXAMPLES:

```
sage: L = lie_algebras.sl(QQ, 2)                                         #␣
↪needs sage.combinat sage.modules
sage: PBW = L.pbw_basis()                                                #␣
↪needs sage.combinat sage.modules
```

```
>>> from sage.all import *
>>> L = lie_algebras.sl(QQ, Integer(2))
↪      # needs sage.combinat sage.modules
>>> PBW = L.pbw_basis()
↪needs sage.combinat sage.modules
```

poincare_birkhoff_witt_basis(*basis_key=None*, ***kwds*)

alias of *pbw_basis()*.

example(*gens=None*)

Return an example of a Lie algebra as per *Category.example*.

EXAMPLES:

```
sage: LieAlgebras(QQ).WithBasis().example()
↪needs sage.combinat sage.modules
An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Partitions over Rational Field
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).WithBasis().example()
↪needs sage.combinat sage.modules
An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Partitions over Rational Field
```

Another set of generators can be specified as an optional argument:

```
sage: LieAlgebras(QQ).WithBasis().example(Compositions())
↪needs sage.combinat sage.modules
An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Compositions of nonnegative integers
 over Rational Field
```

```
>>> from sage.all import *
>>> LieAlgebras(QQ).WithBasis().example(Compositions())
↪needs sage.combinat sage.modules
An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Compositions of nonnegative integers
 over Rational Field
```

4.112 Lie Conformal Algebras

Let R be a commutative ring, a *super Lie conformal algebra* [Kac1997] over R (also known as a *vertex Lie algebra*) is an $R[T]$ super module L together with a $\mathbb{Z}/2\mathbb{Z}$-graded R-bilinear operation (called the λ-bracket) $L \otimes L \to L[\lambda]$ (polynomials in λ with coefficients in L), $a \otimes b \mapsto [a_\lambda b]$ satisfying

1. Sesquilinearity:

$$[Ta_\lambda b] = -\lambda[a_\lambda b], \qquad [a_\lambda Tb] = (\lambda + T)[a_\lambda b].$$

2. Skew-Symmetry:

$$[a_\lambda b] = -(-1)^{p(a)p(b)}[b_{-\lambda - T}a],$$

where $p(a)$ is 0 if a is *even* and 1 if a is *odd*. The bracket in the RHS is computed as follows. First we evaluate $[b_\mu a]$ with the formal parameter μ to the *left*, then replace each appearance of the formal variable μ by $-\lambda - T$. Finally apply T to the coefficients in L.

3. Jacobi identity:

$$[a_\lambda [b_\mu c]] = [[a_{\lambda+\mu} b]_\mu c] + (-1)^{p(a)p(b)} [b_\mu [a_\lambda c]],$$

which is understood as an equality in $L[\lambda, \mu]$.

T is usually called the *translation operation* or the *derivative*. For an element $a \in L$ we will say that Ta is the *derivative of* a. We define the *n-th products* $a_{(n)}b$ for $a, b \in L$ by

$$[a_\lambda b] = \sum_{n \geq 0} \frac{\lambda^n}{n!} a_{(n)} b.$$

A Lie conformal algebra is called *H-Graded* [DSK2006] if there exists a decomposition $L = \oplus L_n$ such that the λ-bracket becomes graded of degree -1, that is:

$$a_{(n)}b \in L_{p+q-n-1} \qquad a \in L_p,\ b \in L_q,\ n \geq 0.$$

In particular this implies that the action of T increases degree by 1.

Note

In the literature arbitrary gradings are allowed. In this implementation we only support nonnegative rational gradings.

EXAMPLES:

1. The **Virasoro** Lie conformal algebra Vir over a ring R where 12 is invertible has two generators L, C as an $R[T]$-module. It is the direct sum of a free module of rank 1 generated by L, and a free rank one R module generated by C satisfying $TC = 0$. C is central (the λ-bracket of C with any other vector vanishes). The remaining λ-bracket is given by

$$[L_\lambda L] = TL + 2\lambda L + \frac{\lambda^3}{12} C.$$

2. The **affine** or current Lie conformal algebra $L(\mathfrak{g})$ associated to a finite dimensional Lie algebra $\mathfrak{g}$ with non-degenerate, invariant R-bilinear form $(,)$ is given as a central extension of the free $R[T]$ module generated by $\mathfrak{g}$ by a central element K. The λ-bracket of generators is given by

$$[a_\lambda b] = [a, b] + \lambda(a, b)K, \qquad a, b \in \mathfrak{g}$$

3. The **Weyl** Lie conformal algebra, or $\beta - \gamma$ system is given as the central extension of a free $R[T]$ module with two generators β and γ, by a central element K. The only non-trivial brackets among generators are

$$[\beta_\lambda \gamma] = -[\gamma_\lambda \beta] = K$$

4. The **Neveu-Schwarz** super Lie conformal algebra is a super Lie conformal algebra which is an extension of the Virasoro Lie conformal algebra. It consists of a Virasoro generator L as in example 1 above and an *odd* generator G. The remaining brackets are given by:

$$[L_\lambda G] = \left(T + \frac{3}{2}\lambda\right) G \qquad [G_\lambda G] = 2L + \frac{\lambda^2}{3} C$$

See also

- sage.algebras.lie_conformal_algebras.lie_conformal_algebra
- sage.algebras.lie_conformal_algebras.examples

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.lie_conformal_algebras.**LieConformalAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Lie conformal algebras.

This is the base category for all Lie conformal algebras. Subcategories with axioms are `FinitelyGenerated` and `WithBasis`. A *finitely generated* Lie conformal algebra is a Lie conformal algebra over R which is finitely generated as an $R[T]$-module. A Lie conformal algebra *with basis* is one with a preferred basis as an R-module.

EXAMPLES:

The base category:

```
sage: C = LieConformalAlgebras(QQ); C
Category of Lie conformal algebras over Rational Field
sage: C.is_subcategory(VectorSpaces(QQ))
True
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQ); C
Category of Lie conformal algebras over Rational Field
>>> C.is_subcategory(VectorSpaces(QQ))
True
```

Some subcategories:

```
sage: LieConformalAlgebras(QQbar).FinitelyGenerated().WithBasis()                #␣
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).FinitelyGenerated().WithBasis()                #␣
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
```

In addition we support functorial constructions `Graded` and `Super`. These functors commute:

```
sage: CGS = LieConformalAlgebras(AA).Graded().Super(); CGS                       #␣
↪needs sage.rings.number_field
Category of H-graded super Lie conformal algebras over Algebraic Real Field
sage: CGS is LieConformalAlgebras(AA).Super().Graded()                           #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> CGS = LieConformalAlgebras(AA).Graded().Super(); CGS                    #␣
↪needs sage.rings.number_field
Category of H-graded super Lie conformal algebras over Algebraic Real Field
>>> CGS is LieConformalAlgebras(AA).Super().Graded()                        #␣
↪needs sage.rings.number_field
True
```

That is, we only consider gradings on super Lie conformal algebras that are compatible with the $\mathbf{Z}/2\mathbf{Z}$ grading.

The base ring needs to be a commutative ring:

```
sage: LieConformalAlgebras(QuaternionAlgebra(2))                            #␣
↪needs sage.combinat sage.modules
Traceback (most recent call last):
ValueError: base must be a commutative ring
got Quaternion Algebra (-1, -1) with base ring Rational Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QuaternionAlgebra(Integer(2)))                     ␣
↪     # needs sage.combinat sage.modules
Traceback (most recent call last):
ValueError: base must be a commutative ring
got Quaternion Algebra (-1, -1) with base ring Rational Field
```

class ElementMethods

Bases: object

is_even_odd()

Return 0 if this element is *even* and 1 if it is *odd*.

> **Note**
>
> This method returns 0 by default since every Lie conformal algebra can be thought as a purely even Lie conformal algebra. In order to implement a super Lie conformal algebra, the user needs to implement this method.

EXAMPLES:

```
sage: R = lie_conformal_algebras.NeveuSchwarz(QQ)                           #␣
↪needs sage.combinat sage.modules
sage: R.inject_variables()                                                  #␣
↪needs sage.combinat sage.modules
Defining L, G, C
sage: G.is_even_odd()                                                       #␣
↪needs sage.combinat sage.modules
1
```

```
>>> from sage.all import *
>>> R = lie_conformal_algebras.NeveuSchwarz(QQ)                             #␣
↪needs sage.combinat sage.modules
>>> R.inject_variables()                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
Defining L, G, C
>>> G.is_even_odd()                                                    #␣
↪needs sage.combinat sage.modules
1
```

FinitelyGeneratedAsLambdaBracketAlgebra

alias of *FinitelyGeneratedLieConformalAlgebras*

Graded

alias of *GradedLieConformalAlgebras*

class ParentMethods

Bases: object

Super

alias of *SuperLieConformalAlgebras*

WithBasis

alias of *LieConformalAlgebrasWithBasis*

example()

An example of parent in this category.

EXAMPLES:

```
sage: LieConformalAlgebras(QQ).example()                                #␣
↪needs sage.combinat sage.modules
The Virasoro Lie conformal algebra over Rational Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQ).example()                                  #␣
↪needs sage.combinat sage.modules
The Virasoro Lie conformal algebra over Rational Field
```

super_categories()

The list of super categories of this category.

EXAMPLES:

```
sage: C = LieConformalAlgebras(QQ)
sage: C.super_categories()
[Category of Lambda bracket algebras over Rational Field]
sage: C = LieConformalAlgebras(QQ).FinitelyGenerated(); C
Category of finitely generated Lie conformal algebras over Rational Field
sage: C.super_categories()
[Category of finitely generated lambda bracket algebras over Rational Field,
 Category of Lie conformal algebras over Rational Field]
sage: C.all_super_categories()
[Category of finitely generated Lie conformal algebras over Rational Field,
 Category of finitely generated lambda bracket algebras over Rational Field,
 Category of Lie conformal algebras over Rational Field,
 Category of Lambda bracket algebras over Rational Field,
 Category of vector spaces over Rational Field,
```

(continues on next page)

(continued from previous page)

```
 Category of modules over Rational Field,
 Category of bimodules over Rational Field on the left and Rational Field on␣
↪the right,
 Category of right modules over Rational Field,
 Category of left modules over Rational Field,
 Category of commutative additive groups,
 Category of additive groups,
 Category of additive inverse additive unital additive magmas,
 Category of commutative additive monoids,
 Category of additive monoids,
 Category of additive unital additive magmas,
 Category of commutative additive semigroups,
 Category of additive commutative additive magmas,
 Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQ)
>>> C.super_categories()
[Category of Lambda bracket algebras over Rational Field]
>>> C = LieConformalAlgebras(QQ).FinitelyGenerated(); C
Category of finitely generated Lie conformal algebras over Rational Field
>>> C.super_categories()
[Category of finitely generated lambda bracket algebras over Rational Field,
 Category of Lie conformal algebras over Rational Field]
>>> C.all_super_categories()
[Category of finitely generated Lie conformal algebras over Rational Field,
 Category of finitely generated lambda bracket algebras over Rational Field,
 Category of Lie conformal algebras over Rational Field,
 Category of Lambda bracket algebras over Rational Field,
 Category of vector spaces over Rational Field,
 Category of modules over Rational Field,
 Category of bimodules over Rational Field on the left and Rational Field on␣
↪the right,
 Category of right modules over Rational Field,
 Category of left modules over Rational Field,
 Category of commutative additive groups,
 Category of additive groups,
 Category of additive inverse additive unital additive magmas,
 Category of commutative additive monoids,
 Category of additive monoids,
 Category of additive unital additive magmas,
 Category of commutative additive semigroups,
 Category of additive commutative additive magmas,
 Category of additive semigroups,
 Category of additive magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

4.113 Lie Conformal Algebras With Basis

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.lie_conformal_algebras_with_basis.**LieConformalAlgebrasWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).WithBasis()                                  #␣
↪needs sage.rings.number_field
Category of Lie conformal algebras with basis over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).WithBasis()                                    #␣
↪needs sage.rings.number_field
Category of Lie conformal algebras with basis over Algebraic Field
```

class FinitelyGeneratedAsLambdaBracketAlgebra(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of finitely generated Lie conformal algebras with basis.

EXAMPLES:

```
sage: C = LieConformalAlgebras(QQbar)                                          #␣
↪needs sage.rings.number_field
sage: CWF = C.WithBasis().FinitelyGenerated(); CWF                             #␣
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
sage: CWF is C.FinitelyGenerated().WithBasis()                                 #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQbar)                                            #␣
↪needs sage.rings.number_field
>>> CWF = C.WithBasis().FinitelyGenerated(); CWF                               #␣
↪needs sage.rings.number_field
Category of finitely generated Lie conformal algebras with basis
 over Algebraic Field
>>> CWF is C.FinitelyGenerated().WithBasis()                                   #␣
↪needs sage.rings.number_field
True
```

class Graded(*base_category*)

Bases: *GradedLieConformalAlgebrasCategory*

The category of H-graded finitely generated Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).WithBasis().FinitelyGenerated().
↪Graded()           # needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras with basis
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).WithBasis().FinitelyGenerated().Graded() ␣
↪        # needs sage.rings.number_field
Category of H-graded finitely generated Lie conformal algebras with basis
 over Algebraic Field
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

The category of super finitely generated Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(AA).WithBasis().FinitelyGenerated().Super()  #␣
↪needs sage.rings.number_field
Category of super finitely generated Lie conformal algebras with basis
 over Algebraic Real Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(AA).WithBasis().FinitelyGenerated().Super()  #␣
↪needs sage.rings.number_field
Category of super finitely generated Lie conformal algebras with basis
 over Algebraic Real Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded super finitely generated Lie conformal algebras with basis.

EXAMPLES:

```
sage: C = LieConformalAlgebras(QQbar).WithBasis().FinitelyGenerated() ␣
↪         # needs sage.rings.number_field
sage: C.Graded().Super()                                               ␣
↪         # needs sage.rings.number_field
Category of H-graded super finitely generated Lie conformal algebras
 with basis over Algebraic Field
sage: C.Graded().Super() is C.Super().Graded()                         ␣
↪         # needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> C = LieConformalAlgebras(QQbar).WithBasis().FinitelyGenerated()   ␣
↪       # needs sage.rings.number_field
>>> C.Graded().Super()                                                ␣
↪       # needs sage.rings.number_field
Category of H-graded super finitely generated Lie conformal algebras
```

(continues on next page)

(continued from previous page)

```
 with basis over Algebraic Field
>>> C.Graded().Super() is C.Super().Graded()                                ␣
↪       # needs sage.rings.number_field
True
```

class Graded(*base_category*)

Bases: *GradedLieConformalAlgebrasCategory*

The category of H-graded Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).WithBasis().Graded()                    #␣
↪needs sage.rings.number_field
Category of H-graded Lie conformal algebras with basis over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).WithBasis().Graded()                      #␣
↪needs sage.rings.number_field
Category of H-graded Lie conformal algebras with basis over Algebraic Field
```

class Super(*base_category*)

Bases: *SuperModulesCategory*

The category of super Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(AA).WithBasis().Super()                        #␣
↪needs sage.rings.number_field
Category of super Lie conformal algebras with basis
 over Algebraic Real Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(AA).WithBasis().Super()                          #␣
↪needs sage.rings.number_field
Category of super Lie conformal algebras with basis
 over Algebraic Real Field
```

class Graded(*base_category*)

Bases: *GradedLieConformalAlgebrasCategory*

The category of H-graded super Lie conformal algebras with basis.

EXAMPLES:

```
sage: LieConformalAlgebras(QQbar).WithBasis().Super().Graded()            #␣
↪needs sage.rings.number_field
Category of H-graded super Lie conformal algebras with basis
 over Algebraic Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQbar).WithBasis().Super().Graded()              #␣
↪needs sage.rings.number_field
```

(continues on next page)

(continued from previous page)

```
Category of H-graded super Lie conformal algebras with basis
 over Algebraic Field
```

class ParentMethods

Bases: object

4.114 Lie Groups

class sage.categories.lie_groups.**LieGroups**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of Lie groups.

A Lie group is a topological group with a smooth manifold structure.

EXAMPLES:

```
sage: from sage.categories.lie_groups import LieGroups
sage: C = LieGroups(QQ); C
Category of Lie groups over Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.lie_groups import LieGroups
>>> C = LieGroups(QQ); C
Category of Lie groups over Rational Field
```

additional_structure()

Return None.

Indeed, the category of Lie groups defines no new structure: a morphism of topological spaces and of smooth manifolds is a morphism as Lie groups.

See also

Category.additional_structure()

EXAMPLES:

```
sage: from sage.categories.lie_groups import LieGroups
sage: LieGroups(QQ).additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.lie_groups import LieGroups
>>> LieGroups(QQ).additional_structure()
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.lie_groups import LieGroups
sage: LieGroups(QQ).super_categories()
[Category of topological groups,
 Category of smooth manifolds over Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.lie_groups import LieGroups
>>> LieGroups(QQ).super_categories()
[Category of topological groups,
 Category of smooth manifolds over Rational Field]
```

4.115 Loop Crystals

class sage.categories.loop_crystals.**KirillovReshetikhinCrystals**

Bases: *Category_singleton*

Category of Kirillov-Reshetikhin crystals.

class ElementMethods

Bases: object

energy_function()

Return the energy function of self.

Let B be a KR crystal. Let $b^\sharp$ denote the unique element such that $\varphi(b^\sharp) = \ell\Lambda_0$ with $\ell = \min\{\langle c, \varphi(b) \mid b \in B\}$. Let u_B denote the maximal element of B. The *energy* of $b \in B$ is given by

$$D(b) = H(b \otimes b^\sharp) - H(u_B \otimes b^\sharp),$$

where H is the *local energy function*.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['D',4,1], 2,1)
sage: for x in K.classically_highest_weight_vectors():
....:     x, x.energy_function()
([], 1)
([[1], [2]], 0)

sage: K = crystals.KirillovReshetikhin(['D',4,3], 1,2)
sage: for x in K.classically_highest_weight_vectors():
....:     x, x.energy_function()
([], 2)
([[1]], 1)
([[1, 1]], 0)
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2),Integer(1))
>>> for x in K.classically_highest_weight_vectors():
...     x, x.energy_function()
([], 1)
([[1], [2]], 0)

>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(3)],␣
↪Integer(1),Integer(2))
>>> for x in K.classically_highest_weight_vectors():
...     x, x.energy_function()
```

(continues on next page)

(continued from previous page)

```
([], 2)
([[1]], 1)
([[1, 1]], 0)
```

lusztig_involution()

Return the result of the classical Lusztig involution on `self`.

EXAMPLES:

```
sage: KRT = crystals.KirillovReshetikhin(['D',4,1], 2, 3, model='KR')
sage: mg = KRT.module_generators[1]
sage: mg.lusztig_involution()
[[-2, -2, 1], [-1, -1, 2]]
sage: elt = mg.f_string([2,1,3,2]); elt
[[3, -2, 1], [4, -1, 2]]
sage: elt.lusztig_involution()
[[-4, -2, 1], [-3, -1, 2]]
```

```
>>> from sage.all import *
>>> KRT = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2), Integer(3), model='KR')
>>> mg = KRT.module_generators[Integer(1)]
>>> mg.lusztig_involution()
[[-2, -2, 1], [-1, -1, 2]]
>>> elt = mg.f_string([Integer(2),Integer(1),Integer(3),Integer(2)]); elt
[[3, -2, 1], [4, -1, 2]]
>>> elt.lusztig_involution()
[[-4, -2, 1], [-3, -1, 2]]
```

class ParentMethods

Bases: `object`

R_matrix(K)

Return the combinatorial R-matrix of `self` to `K`.

The *combinatorial R-matrix* is the affine crystal isomorphism $R : L \otimes K \to K \otimes L$ which maps $u_L \otimes u_K$ to $u_K \otimes u_L$, where u_K is the unique element in $K = B^{r,s}$ of weight $s\Lambda_r - sc\Lambda_0$ (see *maximal_vector()*).

INPUT:

- `self` – a crystal L
- `K` – a Kirillov-Reshetikhin crystal of the same type as L

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: L = crystals.KirillovReshetikhin(['A',2,1],1,2)
sage: f = K.R_matrix(L)
sage: [[b,f(b)] for b in crystals.TensorProduct(K,L)]
[[[[1]], [[1, 1]]], [[[1, 1]], [[1]]]],
 [[[1]], [[1, 2]]], [[[1, 1]], [[2]]]],
 [[[1]], [[2, 2]]], [[[1, 2]], [[2]]]],
 [[[1]], [[1, 3]]], [[[1, 1]], [[3]]]],
 [[[1]], [[2, 3]]], [[[1, 2]], [[3]]]],
```

(continues on next page)

(continued from previous page)

```
 [[[[1]], [[3, 3]]], [[[1, 3]], [[3]]]],
 [[[[2]], [[1, 1]]], [[[1, 2]], [[1]]]],
 [[[[2]], [[1, 2]]], [[[2, 2]], [[1]]]],
 [[[[2]], [[2, 2]]], [[[2, 2]], [[2]]]],
 [[[[2]], [[1, 3]]], [[[2, 3]], [[1]]]],
 [[[[2]], [[2, 3]]], [[[2, 2]], [[3]]]],
 [[[[2]], [[3, 3]]], [[[2, 3]], [[3]]]],
 [[[[3]], [[1, 1]]], [[[1, 3]], [[1]]]],
 [[[[3]], [[1, 2]]], [[[1, 3]], [[2]]]],
 [[[[3]], [[2, 2]]], [[[2, 3]], [[2]]]],
 [[[[3]], [[1, 3]]], [[[3, 3]], [[1]]]],
 [[[[3]], [[2, 3]]], [[[3, 3]], [[2]]]],
 [[[[3]], [[3, 3]]], [[[3, 3]], [[3]]]]]

sage: K = crystals.KirillovReshetikhin(['D',4,1],1,1)
sage: L = crystals.KirillovReshetikhin(['D',4,1],2,1)
sage: f = K.R_matrix(L)
sage: T = crystals.TensorProduct(K,L)
sage: b = T( K(rows=[[1]]), L(rows=[]) )
sage: f(b)
[[[2], [-2]], [[1]]]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> L = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(2))
>>> f = K.R_matrix(L)
>>> [[b,f(b)] for b in crystals.TensorProduct(K,L)]
[[[[[1]], [[1, 1]]], [[[1, 1]], [[1]]]],
 [[[[1]], [[1, 2]]], [[[1, 1]], [[2]]]],
 [[[[1]], [[2, 2]]], [[[1, 2]], [[2]]]],
 [[[[1]], [[1, 3]]], [[[1, 1]], [[3]]]],
 [[[[1]], [[2, 3]]], [[[1, 2]], [[3]]]],
 [[[[1]], [[3, 3]]], [[[1, 3]], [[3]]]],
 [[[[2]], [[1, 1]]], [[[1, 2]], [[1]]]],
 [[[[2]], [[1, 2]]], [[[2, 2]], [[1]]]],
 [[[[2]], [[2, 2]]], [[[2, 2]], [[2]]]],
 [[[[2]], [[1, 3]]], [[[2, 3]], [[1]]]],
 [[[[2]], [[2, 3]]], [[[2, 2]], [[3]]]],
 [[[[2]], [[3, 3]]], [[[2, 3]], [[3]]]],
 [[[[3]], [[1, 1]]], [[[1, 3]], [[1]]]],
 [[[[3]], [[1, 2]]], [[[1, 3]], [[2]]]],
 [[[[3]], [[2, 2]]], [[[2, 3]], [[2]]]],
 [[[[3]], [[1, 3]]], [[[3, 3]], [[1]]]],
 [[[[3]], [[2, 3]]], [[[3, 3]], [[2]]]],
 [[[[3]], [[3, 3]]], [[[3, 3]], [[3]]]]]

>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],
↪Integer(1),Integer(1))
>>> L = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],
↪Integer(2),Integer(1))
```

(continues on next page)

(continued from previous page)

```
>>> f = K.R_matrix(L)
>>> T = crystals.TensorProduct(K,L)
>>> b = T( K(rows=[[Integer(1)]]), L(rows=[]) )
>>> f(b)
[[[2], [-2]], [[1]]]
```

Alternatively, one can compute the combinatorial R-matrix using the isomorphism method of digraphs:

```
sage: K1 = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: K2 = crystals.KirillovReshetikhin(['A',2,1],2,1)
sage: T1 = crystals.TensorProduct(K1,K2)
sage: T2 = crystals.TensorProduct(K2,K1)
sage: T1.digraph().is_isomorphic(T2.digraph(), edge_labels=True,  # todo:␣
↪not implemented (see #10904 and #10549)
....:                                certificate=True)
(True, {[[[1]], [[2], [3]]]: [[[1], [3]], [[2]]],
        [[[3]], [[2], [3]]]: [[[2], [3]], [[3]]],
        [[[3]], [[1], [3]]]: [[[1], [3]], [[3]]],
        [[[1]], [[1], [3]]]: [[[1], [3]], [[1]]], [[[1]],
        [[1], [2]]]: [[[1], [2]], [[1]]],
        [[[2]], [[1], [2]]]: [[[1], [2]], [[2]]], [[[3]],
        [[1], [2]]]: [[[2], [3]], [[1]]],
        [[[2]], [[1], [3]]]: [[[1], [2]], [[3]]],
        [[[2]], [[2], [3]]]: [[[2], [3]], [[2]]]})
```

```
>>> from sage.all import *
>>> K1 = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> K2 = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(2),Integer(1))
>>> T1 = crystals.TensorProduct(K1,K2)
>>> T2 = crystals.TensorProduct(K2,K1)
>>> T1.digraph().is_isomorphic(T2.digraph(), edge_labels=True,  # todo:␣
↪not implemented (see #10904 and #10549)
...                               certificate=True)
(True, {[[[1]], [[2], [3]]]: [[[1], [3]], [[2]]],
        [[[3]], [[2], [3]]]: [[[2], [3]], [[3]]],
        [[[3]], [[1], [3]]]: [[[1], [3]], [[3]]],
        [[[1]], [[1], [3]]]: [[[1], [3]], [[1]]], [[[1]],
        [[1], [2]]]: [[[1], [2]], [[1]]],
        [[[2]], [[1], [2]]]: [[[1], [2]], [[2]]], [[[3]],
        [[1], [2]]]: [[[2], [3]], [[1]]],
        [[[2]], [[1], [3]]]: [[[1], [2]], [[3]]],
        [[[2]], [[2], [3]]]: [[[2], [3]], [[2]]]})
```

affinization()

Return the corresponding affinization crystal of `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: K.affinization()
```

(continues on next page)

(continued from previous page)

```
Affinization of Kirillov-Reshetikhin crystal of type ['A', 2, 1] with (r,
↪s)=(1,1)

sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1, model='KR')
sage: K.affinization()
Affinization of Kirillov-Reshetikhin tableaux of type ['A', 2, 1] and␣
↪shape (1, 1)
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.affinization()
Affinization of Kirillov-Reshetikhin crystal of type ['A', 2, 1] with (r,
↪s)=(1,1)

>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1), model='KR')
>>> K.affinization()
Affinization of Kirillov-Reshetikhin tableaux of type ['A', 2, 1] and␣
↪shape (1, 1)
```

b_sharp()

Return the element $b^\sharp$ of `self`.

Let B be a KR crystal. The element $b^\sharp$ is the unique element such that $\varphi(b^\sharp) = \ell\Lambda_0$ with $\ell = \min\{\langle c, \varphi(b)\rangle \mid b \in B\}$.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',6,2], 2,1)
sage: K.b_sharp()
[]
sage: K.b_sharp().Phi()
Lambda[0]

sage: K = crystals.KirillovReshetikhin(['C',3,1], 1,3)
sage: K.b_sharp()
[[-1]]
sage: K.b_sharp().Phi()
2*Lambda[0]

sage: K = crystals.KirillovReshetikhin(['D',6,2], 2,2)
sage: K.b_sharp() # long time
[]
sage: K.b_sharp().Phi() # long time
2*Lambda[0]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(6),Integer(2)],␣
↪Integer(2),Integer(1))
>>> K.b_sharp()
[]
```

(continues on next page)

(continued from previous page)

```
>>> K.b_sharp().Phi()
Lambda[0]

>>> K = crystals.KirillovReshetikhin(['C',Integer(3),Integer(1)],␣
↪Integer(1),Integer(3))
>>> K.b_sharp()
[[-1]]
>>> K.b_sharp().Phi()
2*Lambda[0]

>>> K = crystals.KirillovReshetikhin(['D',Integer(6),Integer(2)],␣
↪Integer(2),Integer(2))
>>> K.b_sharp() # long time
[]
>>> K.b_sharp().Phi() # long time
2*Lambda[0]
```

cardinality()

Return the cardinality of `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['E',6,1], 1,1)
sage: K.cardinality()
27
sage: K = crystals.KirillovReshetikhin(['C',6,1], 4,3)
sage: K.cardinality()
4736732
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['E',Integer(6),Integer(1)],␣
↪Integer(1),Integer(1))
>>> K.cardinality()
27
>>> K = crystals.KirillovReshetikhin(['C',Integer(6),Integer(1)],␣
↪Integer(4),Integer(3))
>>> K.cardinality()
4736732
```

classical_decomposition()

Return the classical decomposition of `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',3,1], 2,2)
sage: K.classical_decomposition()
The crystal of tableaux of type ['A', 3] and shape(s) [[2, 2]]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],␣
↪Integer(2),Integer(2))
>>> K.classical_decomposition()
The crystal of tableaux of type ['A', 3] and shape(s) [[2, 2]]
```

classically_highest_weight_vectors()

Return the classically highest weight elements of `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['E',6,1],1,1)
sage: K.classically_highest_weight_vectors()
([(1,)],)
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['E',Integer(6),Integer(1)],
↪Integer(1),Integer(1))
>>> K.classically_highest_weight_vectors()
([(1,)],)
```

is_perfect(*ell=None*)

Check if `self` is a perfect crystal of level `ell`.

A crystal $\mathcal{B}$ is perfect of level ℓ if:

1. $\mathcal{B}$ is isomorphic to the crystal graph of a finite-dimensional $U_q'(\mathfrak{g})$-module.
2. $\mathcal{B} \otimes \mathcal{B}$ is connected.
3. There exists a $\lambda \in X$, such that $\mathrm{wt}(\mathcal{B}) \subset \lambda + \sum_{i \in I} \mathbf{Z}_{\leq 0} \alpha_i$ and there is a unique element in $\mathcal{B}$ of classical weight λ.
4. For all $b \in \mathcal{B}$, $\mathrm{level}(\varepsilon(b)) \geq \ell$.
5. For all Λ dominant weights of level ℓ, there exist unique elements $b_\Lambda, b^\Lambda \in \mathcal{B}$, such that $\varepsilon(b_\Lambda) = \Lambda = \varphi(b^\Lambda)$.

Points (1)-(3) are known to hold. This method checks points (4) and (5).

If `self` is the Kirillov-Reshetikhin crystal $B^{r,s}$, then it was proven for non-exceptional types in [FOS2010] that it is perfect if and only if s/c_r is an integer (where c_r is a constant related to the type of the crystal).

It is conjectured this is true for all affine types.

INPUT:

- `ell` – (default: s/c_r) integer; the level

REFERENCES:

[FOS2010]

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: K.is_perfect()
True

sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 1)
sage: K.is_perfect()
False

sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 2)
sage: K.is_perfect()
True

sage: K = crystals.KirillovReshetikhin(['E',6,1], 1, 3)
sage: K.is_perfect()
True
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.is_perfect()
True

>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.is_perfect()
False

>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(2))
>>> K.is_perfect()
True

>>> K = crystals.KirillovReshetikhin(['E',Integer(6),Integer(1)],␣
↪Integer(1), Integer(3))
>>> K.is_perfect()
True
```

Todo

Implement a version for tensor products of KR crystals.

level()

Return the level of `self` when `self` is a perfect crystal.

See also

is_perfect()

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: K.level()
1
sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 2)
sage: K.level()
1
sage: K = crystals.KirillovReshetikhin(['D',4,1], 1, 3)
sage: K.level()
3

sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 1)
sage: K.level()
Traceback (most recent call last):
...
ValueError: this crystal is not perfect
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.level()
1
>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(2))
>>> K.level()
1
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(1), Integer(3))
>>> K.level()
3

>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.level()
Traceback (most recent call last):
...
ValueError: this crystal is not perfect
```

local_energy_function(*B*)

Return the local energy function of `self` and `B`.

See *LocalEnergyFunction* for a definition.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',6,2], 2,1)
sage: Kp = crystals.KirillovReshetikhin(['A',6,2], 1,1)
sage: H = K.local_energy_function(Kp); H
Local energy function of
 Kirillov-Reshetikhin crystal of type ['BC', 3, 2] with (r,s)=(2,1)
tensor
 Kirillov-Reshetikhin crystal of type ['BC', 3, 2] with (r,s)=(1,1)
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(6),Integer(2)],␣
↪Integer(2),Integer(1))
>>> Kp = crystals.KirillovReshetikhin(['A',Integer(6),Integer(2)],␣
↪Integer(1),Integer(1))
>>> H = K.local_energy_function(Kp); H
Local energy function of
 Kirillov-Reshetikhin crystal of type ['BC', 3, 2] with (r,s)=(2,1)
tensor
 Kirillov-Reshetikhin crystal of type ['BC', 3, 2] with (r,s)=(1,1)
```

maximal_vector()

Return the unique element of classical weight $s\Lambda_r$ in `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['C',2,1],1,2)
sage: K.maximal_vector()
```

(continues on next page)

(continued from previous page)

```
[[1, 1]]
sage: K = crystals.KirillovReshetikhin(['E',6,1],1,1)
sage: K.maximal_vector()
[(1,)]

sage: K = crystals.KirillovReshetikhin(['D',4,1],2,1)
sage: K.maximal_vector()
[[1], [2]]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],
↪Integer(1),Integer(2))
>>> K.maximal_vector()
[[1, 1]]
>>> K = crystals.KirillovReshetikhin(['E',Integer(6),Integer(1)],
↪Integer(1),Integer(1))
>>> K.maximal_vector()
[(1,)]

>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],
↪Integer(2),Integer(1))
>>> K.maximal_vector()
[[1], [2]]
```

module_generator()

Return the unique module generator of classical weight $s\Lambda_r$ of the Kirillov-Reshetikhin crystal $B^{r,s}$.

EXAMPLES:

```
sage: La = RootSystem(['G',2,1]).weight_space().fundamental_weights()
sage: K = crystals.ProjectedLevelZeroLSPaths(La[1])
sage: K.module_generator()
(-Lambda[0] + Lambda[1],)
```

```
>>> from sage.all import *
>>> La = RootSystem(['G',Integer(2),Integer(1)]).weight_space().
↪fundamental_weights()
>>> K = crystals.ProjectedLevelZeroLSPaths(La[Integer(1)])
>>> K.module_generator()
(-Lambda[0] + Lambda[1],)
```

q_dimension(*q=None*, *prec=None*, *use_product=False*)

Return the q-dimension of `self`.

The q-dimension of a KR crystal is defined as the q-dimension of the underlying classical crystal.

EXAMPLES:

```
sage: KRC = crystals.KirillovReshetikhin(['A',2,1], 2,2)
sage: KRC.q_dimension()
q^4 + q^3 + 2*q^2 + q + 1
sage: KRC = crystals.KirillovReshetikhin(['D',4,1], 2,1)
```

(continues on next page)

(continued from previous page)

```
sage: KRC.q_dimension()
q^10 + q^9 + 3*q^8 + 3*q^7 + 4*q^6 + 4*q^5 + 4*q^4 + 3*q^3 + 3*q^2 + q + 2
```

```
>>> from sage.all import *
>>> KRC = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(2),Integer(2))
>>> KRC.q_dimension()
q^4 + q^3 + 2*q^2 + q + 1
>>> KRC = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2),Integer(1))
>>> KRC.q_dimension()
q^10 + q^9 + 3*q^8 + 3*q^7 + 4*q^6 + 4*q^5 + 4*q^4 + 3*q^3 + 3*q^2 + q + 2
```

r()

Return the value r in `self` written as $B^{r,s}$.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',3,1], 2,4)
sage: K.r()
2
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],␣
↪Integer(2),Integer(4))
>>> K.r()
2
```

s()

Return the value s in `self` written as $B^{r,s}$.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',3,1], 2,4)
sage: K.s()
4
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(3),Integer(1)],␣
↪Integer(2),Integer(4))
>>> K.s()
4
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of tensor products of Kirillov-Reshetikhin crystals.

class ElementMethods

Bases: `object`

affine_grading()

Return the affine grading of `self`.

The affine grading is calculated by finding a path from `self` to a ground state path (using the helper method *e_string_to_ground_state()*) and counting the number of affine Kashiwara operators e_0 applied on the way.

OUTPUT: integer

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K)
sage: t = T.module_generators[0]
sage: t.affine_grading()
1

sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K,K)
sage: hw = T.classically_highest_weight_vectors()
sage: for b in hw:
....:     print("{} {}".format(b, b.affine_grading()))
[[[1]], [[1]], [[1]]] 3
[[[2]], [[1]], [[1]]] 2
[[[1]], [[2]], [[1]]] 1
[[[3]], [[2]], [[1]]] 0

sage: K = crystals.KirillovReshetikhin(['C',2,1],1,1)
sage: T = crystals.TensorProduct(K,K,K)
sage: hw = T.classically_highest_weight_vectors()
sage: for b in hw:
....:     print("{} {}".format(b, b.affine_grading()))
[[[1]], [[1]], [[1]]] 2
[[[2]], [[1]], [[1]]] 1
[[[-1]], [[1]], [[1]]] 1
[[[1]], [[2]], [[1]]] 1
[[[-2]], [[2]], [[1]]] 0
[[[1]], [[-1]], [[1]]] 0
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K)
>>> t = T.module_generators[Integer(0)]
>>> t.affine_grading()
1

>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K,K)
>>> hw = T.classically_highest_weight_vectors()
>>> for b in hw:
...     print("{} {}".format(b, b.affine_grading()))
[[[1]], [[1]], [[1]]] 3
[[[2]], [[1]], [[1]]] 2
[[[1]], [[2]], [[1]]] 1
[[[3]], [[2]], [[1]]] 0
```

(continues on next page)

(continued from previous page)

```
>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K,K)
>>> hw = T.classically_highest_weight_vectors()
>>> for b in hw:
...     print("{} {}".format(b, b.affine_grading()))
[[[1]], [[1]], [[1]]] 2
[[[2]], [[1]], [[1]]] 1
[[[-1]], [[1]], [[1]]] 1
[[[1]], [[2]], [[1]]] 1
[[[-2]], [[2]], [[1]]] 0
[[[1]], [[-1]], [[1]]] 0
```

e_string_to_ground_state()

Return a string of integers in the index set $(i_1, \ldots, i_k)$ such that $e_{i_k} \cdots e_{i_1}$ of `self` is the ground state.

This method calculates a path from `self` to a ground state path using Demazure arrows as defined in Lemma 7.3 in [ST2011].

OUTPUT: a tuple of integers $(i_1, \ldots, i_k)$

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K)
sage: t = T.module_generators[0]
sage: t.e_string_to_ground_state()
(0, 2)

sage: K = crystals.KirillovReshetikhin(['C',2,1],1,1)
sage: T = crystals.TensorProduct(K,K)
sage: t = T.module_generators[0]; t
[[[1]], [[1]]]
sage: t.e_string_to_ground_state()
(0,)
sage: x = t.e(0)
sage: x.e_string_to_ground_state()
()
sage: y = t.f_string([1,2,1,1,0]); y
[[[2]], [[1]]]
sage: y.e_string_to_ground_state()
()
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K)
>>> t = T.module_generators[Integer(0)]
>>> t.e_string_to_ground_state()
(0, 2)
```

(continues on next page)

(continued from previous page)

```
>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K)
>>> t = T.module_generators[Integer(0)]; t
[[[1]], [[1]]]
>>> t.e_string_to_ground_state()
(0,)
>>> x = t.e(Integer(0))
>>> x.e_string_to_ground_state()
()
>>> y = t.f_string([Integer(1),Integer(2),Integer(1),Integer(1),
↪Integer(0)]); y
[[[2]], [[1]]]
>>> y.e_string_to_ground_state()
()
```

energy_function(*algorithm=None*)

Return the energy function of `self`.

ALGORITHM:

definition

Let T be a tensor product of Kirillov-Reshetikhin crystals. Let R_i and H_i be the combinatorial R-matrix and local energy functions, respectively, acting on the i and $i+1$ factors. Let D_B be the energy function of a single Kirillov-Reshetikhin crystal. The *energy function* is given by

$$D = \sum_{j>i} H_i R_{i+1} R_{i+2} \cdots R_{j-1} + \sum_j D_B R_1 R_2 \cdots R_{j-1},$$

where D_B acts on the rightmost factor.

grading

If `self` is an element of T, a tensor product of perfect crystals of the same level, then use the affine grading to determine the energy. Specifically, let g denote the affine grading of `self` and d the affine grading of the maximal vector in T. Then the energy of `self` is given by $d - g$.

For more details, see Theorem 7.5 in [ST2011].

INPUT:

- `algorithm` – (default: `None`) use one of the following algorithms to determine the energy function:
  - `'definition'` – use the definition of the energy function;
  - `'grading'` – use the affine grading;

  if not specified, then this uses `'grading'` if all factors are perfect of the same level and otherwise this uses `'definition'`

OUTPUT: integer

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: T = crystals.TensorProduct(K,K,K)
sage: hw = T.classically_highest_weight_vectors()
sage: for b in hw:
```

(continues on next page)

(continued from previous page)

```
....:     print("{} {}".format(b, b.energy_function()))
[[[1]], [[1]], [[1]]] 0
[[[2]], [[1]], [[1]]] 1
[[[1]], [[2]], [[1]]] 2
[[[3]], [[2]], [[1]]] 3

sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 2)
sage: T = crystals.TensorProduct(K,K)
sage: hw = T.classically_highest_weight_vectors()
sage: for b in hw:
....:     print("{} {}".format(b, b.energy_function()))
[[], []] 4
[[[1, 1]], []] 3
[[], [[1, 1]]] 1
[[[1, 1]], [[1, 1]]] 0
[[[1, 2]], [[1, 1]]] 1
[[[2, 2]], [[1, 1]]] 2
[[[-1, -1]], [[1, 1]]] 2
[[[1, -1]], [[1, 1]]] 2
[[[2, -1]], [[1, 1]]] 2

sage: K = crystals.KirillovReshetikhin(['C',2,1], 1, 1)
sage: T = crystals.TensorProduct(K)
sage: t = T.module_generators[0]
sage: t.energy_function('grading')
Traceback (most recent call last):
...
NotImplementedError: all crystals in the tensor product
 need to be perfect of the same level
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> T = crystals.TensorProduct(K,K,K)
>>> hw = T.classically_highest_weight_vectors()
>>> for b in hw:
...     print("{} {}".format(b, b.energy_function()))
[[[1]], [[1]], [[1]]] 0
[[[2]], [[1]], [[1]]] 1
[[[1]], [[2]], [[1]]] 2
[[[3]], [[2]], [[1]]] 3

>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(2))
>>> T = crystals.TensorProduct(K,K)
>>> hw = T.classically_highest_weight_vectors()
>>> for b in hw:
...     print("{} {}".format(b, b.energy_function()))
[[], []] 4
[[[1, 1]], []] 3
[[], [[1, 1]]] 1
[[[1, 1]], [[1, 1]]] 0
```

(continues on next page)

(continued from previous page)

```
[[[1, 2]], [[1, 1]]] 1
[[[2, 2]], [[1, 1]]] 2
[[[-1, -1]], [[1, 1]]] 2
[[[1, -1]], [[1, 1]]] 2
[[[2, -1]], [[1, 1]]] 2

>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> T = crystals.TensorProduct(K)
>>> t = T.module_generators[Integer(0)]
>>> t.energy_function('grading')
Traceback (most recent call last):
...
NotImplementedError: all crystals in the tensor product
 need to be perfect of the same level
```

class ParentMethods

Bases: object

cardinality()

Return the cardinality of self.

EXAMPLES:

```
sage: RC = RiggedConfigurations(['A', 3, 1], [[3, 2], [1, 2]])
sage: RC.cardinality()
100
sage: len(RC.list())
100

sage: RC = RiggedConfigurations(['E', 7, 1], [[1,1]])
sage: RC.cardinality()
134
sage: len(RC.list())
134

sage: RC = RiggedConfigurations(['B', 3, 1], [[2,2],[1,2]])
sage: RC.cardinality()
5130
```

```
>>> from sage.all import *
>>> RC = RiggedConfigurations(['A', Integer(3), Integer(1)],␣
↪[[Integer(3), Integer(2)], [Integer(1), Integer(2)]])
>>> RC.cardinality()
100
>>> len(RC.list())
100

>>> RC = RiggedConfigurations(['E', Integer(7), Integer(1)],␣
↪[[Integer(1),Integer(1)]])
>>> RC.cardinality()
134
```

(continues on next page)

(continued from previous page)

```
>>> len(RC.list())
134

>>> RC = RiggedConfigurations(['B', Integer(3), Integer(1)],␣
↪[[Integer(2),Integer(2)],[Integer(1),Integer(2)]])
>>> RC.cardinality()
5130
```

classically_highest_weight_vectors()

Return the classically highest weight elements of `self`.

This works by using a backtracking algorithm since if $b_2 \otimes b_1$ is classically highest weight then b_1 is classically highest weight.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K,K)
sage: T.classically_highest_weight_vectors()
([[[1]], [[1]], [[1]]],
 [[[2]], [[1]], [[1]]],
 [[[1]], [[2]], [[1]]],
 [[[3]], [[2]], [[1]]])
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K,K)
>>> T.classically_highest_weight_vectors()
([[[1]], [[1]], [[1]]],
 [[[2]], [[1]], [[1]]],
 [[[1]], [[2]], [[1]]],
 [[[3]], [[2]], [[1]]])
```

maximal_vector()

Return the maximal vector of `self`.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K,K)
sage: T.maximal_vector()
[[[1]], [[1]], [[1]]]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K,K)
>>> T.maximal_vector()
[[[1]], [[1]], [[1]]]
```

one_dimensional_configuration_sum(*q=None*, *group_components=True*)

Compute the one-dimensional configuration sum of `self`.

INPUT:

- `q` – (default: `None`) a variable or `None`; if `None`, a variable q is set in the code
- `group_components` – boolean (default: `True`); if `True`, then the terms are grouped by classical component

The one-dimensional configuration sum is the sum of the weights of all elements in the crystal weighted by the energy function.

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['A',2,1],1,1)
sage: T = crystals.TensorProduct(K,K)
sage: T.one_dimensional_configuration_sum()
B[-2*Lambda[1] + 2*Lambda[2]] + (q+1)*B[-Lambda[1]]
 + (q+1)*B[Lambda[1] - Lambda[2]] + B[2*Lambda[1]]
 + B[-2*Lambda[2]] + (q+1)*B[Lambda[2]]
sage: R.<t> = ZZ[]
sage: T.one_dimensional_configuration_sum(t, False)
B[-2*Lambda[1] + 2*Lambda[2]] + (t+1)*B[-Lambda[1]]
 + (t+1)*B[Lambda[1] - Lambda[2]] + B[2*Lambda[1]]
 + B[-2*Lambda[2]] + (t+1)*B[Lambda[2]]

sage: R = RootSystem(['A',2,1])
sage: La = R.weight_space().basis()
sage: LS = crystals.ProjectedLevelZeroLSPaths(2*La[1])
sage: (LS.one_dimensional_configuration_sum()  # long time
....:  == T.one_dimensional_configuration_sum())
True
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(1),Integer(1))
>>> T = crystals.TensorProduct(K,K)
>>> T.one_dimensional_configuration_sum()
B[-2*Lambda[1] + 2*Lambda[2]] + (q+1)*B[-Lambda[1]]
 + (q+1)*B[Lambda[1] - Lambda[2]] + B[2*Lambda[1]]
 + B[-2*Lambda[2]] + (q+1)*B[Lambda[2]]
>>> R = ZZ['t']; (t,) = R._first_ngens(1)
>>> T.one_dimensional_configuration_sum(t, False)
B[-2*Lambda[1] + 2*Lambda[2]] + (t+1)*B[-Lambda[1]]
 + (t+1)*B[Lambda[1] - Lambda[2]] + B[2*Lambda[1]]
 + B[-2*Lambda[2]] + (t+1)*B[Lambda[2]]

>>> R = RootSystem(['A',Integer(2),Integer(1)])
>>> La = R.weight_space().basis()
>>> LS = crystals.ProjectedLevelZeroLSPaths(Integer(2)*La[Integer(1)])
>>> (LS.one_dimensional_configuration_sum()  # long time
...  == T.one_dimensional_configuration_sum())
True
```

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.loop_crystals import␣
↪KirillovReshetikhinCrystals
```

(continues on next page)

(continued from previous page)

```
sage: KirillovReshetikhinCrystals().TensorProducts().extra_super_
↪categories()
[Category of finite regular loop crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import KirillovReshetikhinCrystals
>>> KirillovReshetikhinCrystals().TensorProducts().extra_super_
↪categories()
[Category of finite regular loop crystals]
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.loop_crystals import KirillovReshetikhinCrystals
sage: KirillovReshetikhinCrystals().super_categories()
[Category of finite regular loop crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import KirillovReshetikhinCrystals
>>> KirillovReshetikhinCrystals().super_categories()
[Category of finite regular loop crystals]
```

class sage.categories.loop_crystals.**LocalEnergyFunction**(*B*, *Bp*, *normalization=0*)

Bases: *Map*

The local energy function.

Let B and B' be Kirillov-Reshetikhin crystals with maximal vectors u_B and $u_{B'}$ respectively. The *local energy function* $H : B \otimes B' \to \mathbf{Z}$ is the function which satisfies

$$H(e_0(b \otimes b')) = H(b \otimes b') + \begin{cases} 1 & \text{if } i = 0 \text{ and LL}, \\ -1 & \text{if } i = 0 \text{ and RR}, \\ 0 & \text{otherwise}, \end{cases}$$

where LL (resp. RR) denote e_0 acts on the left (resp. right) on both $b \otimes b'$ and $R(b \otimes b')$, and normalized by $H(u_B \otimes u_{B'}) = 0$.

INPUT:

- `B` – a Kirillov-Reshetikhin crystal
- `Bp` – a Kirillov-Reshetikhin crystal
- `normalization` – (default: 0) the normalization value

EXAMPLES:

```
sage: K = crystals.KirillovReshetikhin(['C',2,1], 1,2)
sage: K2 = crystals.KirillovReshetikhin(['C',2,1], 2,1)
sage: H = K.local_energy_function(K2)
sage: T = tensor([K, K2])
sage: hw = T.classically_highest_weight_vectors()
sage: for b in hw:
....:     b, H(b)
([[], [[1], [2]]], 1)
```

(continues on next page)

(continued from previous page)

```
([[1, 1]], [[1], [2]]], 0)
([[2, -2]], [[1], [2]]], 1)
([[1, -2]], [[1], [2]]], 1)
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)], Integer(1),
↪Integer(2))
>>> K2 = crystals.KirillovReshetikhin(['C',Integer(2),Integer(1)], Integer(2),
↪Integer(1))
>>> H = K.local_energy_function(K2)
>>> T = tensor([K, K2])
>>> hw = T.classically_highest_weight_vectors()
>>> for b in hw:
...     b, H(b)
([], [[1], [2]]], 1)
([[1, 1]], [[1], [2]]], 0)
([[2, -2]], [[1], [2]]], 1)
([[1, -2]], [[1], [2]]], 1)
```

REFERENCES:

[KKMMNN1992]

class sage.categories.loop_crystals.**LoopCrystals**

Bases: *Category_singleton*

The category of $U_q'(\mathfrak{g})$-crystals, where $\mathfrak{g}$ is of affine type.

The category is called loop crystals as we can also consider them as crystals corresponding to the loop algebra $\mathfrak{g}_0[t]$, where $\mathfrak{g}_0$ is the corresponding classical type.

EXAMPLES:

```
sage: from sage.categories.loop_crystals import LoopCrystals
sage: C = LoopCrystals()
sage: C
Category of loop crystals
sage: C.super_categories()
[Category of crystals]
sage: C.example()
Kirillov-Reshetikhin crystal of type ['A', 3, 1] with (r,s)=(1,1)
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import LoopCrystals
>>> C = LoopCrystals()
>>> C
Category of loop crystals
>>> C.super_categories()
[Category of crystals]
>>> C.example()
Kirillov-Reshetikhin crystal of type ['A', 3, 1] with (r,s)=(1,1)
```

class ParentMethods

Bases: object

digraph(*subset=None*, *index_set=None*)

Return the `DiGraph` associated to `self`.

INPUT:

- `subset` – (optional) a subset of vertices for which the digraph should be constructed
- `index_set` – (optional) the index set to draw arrows

See also

sage.categories.crystals.Crystals.ParentMethods.digraph()

EXAMPLES:

```
sage: C = crystals.KirillovReshetikhin(['D',4,1], 2, 1)
sage: G = C.digraph()
sage: G.latex_options()  # optional - dot2tex
LaTeX options for Digraph on 29 vertices:
{...'edge_options': <function ... at ...>...}
sage: view(G, tightpage=True)  # optional - dot2tex graphviz, not tested␣
↪(opens external window)
```

```
>>> from sage.all import *
>>> C = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2), Integer(1))
>>> G = C.digraph()
>>> G.latex_options()  # optional - dot2tex
LaTeX options for Digraph on 29 vertices:
{...'edge_options': <function ... at ...>...}
>>> view(G, tightpage=True)  # optional - dot2tex graphviz, not tested␣
↪(opens external window)
```

weight_lattice_realization()

Return the weight lattice realization used to express weights of elements in `self`.

The default is to use the non-extended affine weight lattice.

EXAMPLES:

```
sage: C = crystals.Letters(['A', 5])
sage: C.weight_lattice_realization()
Ambient space of the Root system of type ['A', 5]
sage: K = crystals.KirillovReshetikhin(['A',2,1], 1, 1)
sage: K.weight_lattice_realization()
Weight lattice of the Root system of type ['A', 2, 1]
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A', Integer(5)])
>>> C.weight_lattice_realization()
Ambient space of the Root system of type ['A', 5]
>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],␣
↪Integer(1), Integer(1))
>>> K.weight_lattice_realization()
Weight lattice of the Root system of type ['A', 2, 1]
```

example(*n=3*)

Return an example of Kirillov-Reshetikhin crystals, as per *Category.example()*.

EXAMPLES:

```
sage: from sage.categories.loop_crystals import LoopCrystals
sage: B = LoopCrystals().example(); B
Kirillov-Reshetikhin crystal of type ['A', 3, 1] with (r,s)=(1,1)
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import LoopCrystals
>>> B = LoopCrystals().example(); B
Kirillov-Reshetikhin crystal of type ['A', 3, 1] with (r,s)=(1,1)
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.loop_crystals import LoopCrystals
sage: LoopCrystals().super_categories()
[Category of crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import LoopCrystals
>>> LoopCrystals().super_categories()
[Category of crystals]
```

class sage.categories.loop_crystals.**RegularLoopCrystals**

Bases: *Category_singleton*

The category of regular $U_q'(\mathfrak{g})$-crystals, where $\mathfrak{g}$ is of affine type.

class ElementMethods

Bases: object

classical_weight()

Return the classical weight of `self`.

EXAMPLES:

```
sage: R = RootSystem(['A',2,1])
sage: La = R.weight_space().basis()
sage: LS = crystals.ProjectedLevelZeroLSPaths(2*La[1])
sage: hw = LS.classically_highest_weight_vectors()
sage: [(v.weight(), v.classical_weight()) for v in hw]
[(-2*Lambda[0] + 2*Lambda[1], (2, 0, 0)),
 (-Lambda[0] + Lambda[2], (1, 1, 0))]
```

```
>>> from sage.all import *
>>> R = RootSystem(['A',Integer(2),Integer(1)])
>>> La = R.weight_space().basis()
>>> LS = crystals.ProjectedLevelZeroLSPaths(Integer(2)*La[Integer(1)])
>>> hw = LS.classically_highest_weight_vectors()
>>> [(v.weight(), v.classical_weight()) for v in hw]
[(-2*Lambda[0] + 2*Lambda[1], (2, 0, 0)),
 (-Lambda[0] + Lambda[2], (1, 1, 0))]
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.loop_crystals import RegularLoopCrystals
sage: RegularLoopCrystals().super_categories()
[Category of regular crystals,
 Category of loop crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.loop_crystals import RegularLoopCrystals
>>> RegularLoopCrystals().super_categories()
[Category of regular crystals,
 Category of loop crystals]
```

4.116 L-trivial semigroups

class sage.categories.l_trivial_semigroups.**LTrivialSemigroups**(*base_category*)

Bases: *CategoryWithAxiom*

Commutative_extra_super_categories()

Implement the fact that a commutative R-trivial semigroup is J-trivial.

EXAMPLES:

```
sage: Semigroups().LTrivial().Commutative_extra_super_categories()
[Category of j trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().LTrivial().Commutative_extra_super_categories()
[Category of j trivial semigroups]
```

RTrivial_extra_super_categories()

Implement the fact that an L-trivial and R-trivial semigroup is J-trivial.

EXAMPLES:

```
sage: Semigroups().LTrivial().RTrivial_extra_super_categories()
[Category of j trivial magmas]
```

```
>>> from sage.all import *
>>> Semigroups().LTrivial().RTrivial_extra_super_categories()
[Category of j trivial magmas]
```

extra_super_categories()

Implement the fact that a L-trivial semigroup is H-trivial.

EXAMPLES:

```
sage: Semigroups().LTrivial().extra_super_categories()
[Category of h trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().LTrivial().extra_super_categories()
[Category of h trivial semigroups]
```

4.117 Magmas

Magmas.ParentMethods.**__init_extra__**()

EXAMPLES:

```
sage: S = Semigroups().example("free")
sage: S('a') * S('b') # indirect doctest
'ab'
sage: S('a').__class__._mul_ == S('a').__class__._mul_parent
True
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free")
>>> S('a') * S('b') # indirect doctest
'ab'
>>> S('a').__class__._mul_ == S('a').__class__._mul_parent
True
```

class sage.categories.magmas.**Magmas**

Bases: *Category_singleton*

The category of (multiplicative) magmas.

A magma is a set with a binary operation $*$.

EXAMPLES:

```
sage: Magmas()
Category of magmas
sage: Magmas().super_categories()
[Category of sets]
sage: Magmas().all_super_categories()
[Category of magmas, Category of sets,
 Category of sets with partial maps, Category of objects]
```

```
>>> from sage.all import *
>>> Magmas()
Category of magmas
>>> Magmas().super_categories()
[Category of sets]
>>> Magmas().all_super_categories()
[Category of magmas, Category of sets,
 Category of sets with partial maps, Category of objects]
```

The following axioms are defined by this category:

```
sage: Magmas().Associative()
Category of semigroups
```

(continues on next page)

(continued from previous page)

```
sage: Magmas().Unital()
Category of unital magmas
sage: Magmas().Commutative()
Category of commutative magmas
sage: Magmas().Unital().Inverse()
Category of inverse unital magmas
sage: Magmas().Associative()
Category of semigroups
sage: Magmas().Associative().Unital()
Category of monoids
sage: Magmas().Associative().Unital().Inverse()
Category of groups
```

```
>>> from sage.all import *
>>> Magmas().Associative()
Category of semigroups
>>> Magmas().Unital()
Category of unital magmas
>>> Magmas().Commutative()
Category of commutative magmas
>>> Magmas().Unital().Inverse()
Category of inverse unital magmas
>>> Magmas().Associative()
Category of semigroups
>>> Magmas().Associative().Unital()
Category of monoids
>>> Magmas().Associative().Unital().Inverse()
Category of groups
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

is_field(*proof=True*)

Return True if self is a field.

For a magma algebra RS this is always false unless S is trivial and the base ring R is a field.

EXAMPLES:

```
sage: SymmetricGroup(1).algebra(QQ).is_field()                          #␣
↪needs sage.combinat sage.groups
True
sage: SymmetricGroup(1).algebra(ZZ).is_field()                          #␣
↪needs sage.combinat sage.groups
False
sage: SymmetricGroup(2).algebra(QQ).is_field()                          #␣
↪needs sage.combinat sage.groups
False
sage: Magmas().example().algebra(QQ).is_field()
False
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(1)).algebra(QQ).is_field()
↪    # needs sage.combinat sage.groups
True
>>> SymmetricGroup(Integer(1)).algebra(ZZ).is_field()
↪    # needs sage.combinat sage.groups
False
>>> SymmetricGroup(Integer(2)).algebra(QQ).is_field()
↪    # needs sage.combinat sage.groups
False
>>> Magmas().example().algebra(QQ).is_field()
False
```

extra_super_categories()

EXAMPLES:

```
sage: MCA = Magmas().Commutative().Algebras(QQ)
sage: MCA.extra_super_categories()
[Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA = Magmas().Commutative().Algebras(QQ)
>>> MCA.extra_super_categories()
[Category of commutative magmas]
```

This implements the fact that the algebra of a commutative magma is commutative:

```
sage: MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

In particular, commutative monoid algebras are commutative algebras:

```
sage: MoCA = Monoids().Commutative().Algebras(QQ)
sage: MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

```
>>> from sage.all import *
>>> MoCA = Monoids().Commutative().Algebras(QQ)
>>> MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

Associative

alias of *Semigroups*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ParentMethods

Bases: object

product(*left*, *right*)

EXAMPLES:

```
sage: C = Magmas().CartesianProducts().example(); C
The Cartesian product of (Rational Field, Integer Ring, Integer Ring)
sage: x = C.an_element(); x
(1/2, 1, 1)
sage: x * x
(1/4, 1, 1)

sage: # needs sage.groups sage.modules
sage: A = SymmetricGroupAlgebra(QQ, 3)
sage: x = cartesian_product([A([1,3,2]), A([2,3,1])])
sage: y = cartesian_product([A([1,3,2]), A([2,3,1])])
sage: cartesian_product([A,A]).product(x,y)
B[(0, [1, 2, 3])] + B[(1, [3, 1, 2])]
sage: x*y
B[(0, [1, 2, 3])] + B[(1, [3, 1, 2])]
```

```
>>> from sage.all import *
>>> C = Magmas().CartesianProducts().example(); C
The Cartesian product of (Rational Field, Integer Ring, Integer Ring)
>>> x = C.an_element(); x
(1/2, 1, 1)
>>> x * x
(1/4, 1, 1)

>>> # needs sage.groups sage.modules
>>> A = SymmetricGroupAlgebra(QQ, Integer(3))
>>> x = cartesian_product([A([Integer(1),Integer(3),Integer(2)]),␣
↪A([Integer(2),Integer(3),Integer(1)])])
>>> y = cartesian_product([A([Integer(1),Integer(3),Integer(2)]),␣
↪A([Integer(2),Integer(3),Integer(1)])])
>>> cartesian_product([A,A]).product(x,y)
B[(0, [1, 2, 3])] + B[(1, [3, 1, 2])]
>>> x*y
B[(0, [1, 2, 3])] + B[(1, [3, 1, 2])]
```

example()

Return an example of Cartesian product of magmas.

EXAMPLES:

```
sage: C = Magmas().CartesianProducts().example(); C
The Cartesian product of (Rational Field, Integer Ring, Integer Ring)
sage: C.category()
Join of Category of Cartesian products of commutative rings and
Category of Cartesian products of metric spaces
sage: sorted(C.category().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Commutative',
```

(continues on next page)

(continued from previous page)

```
 'Distributive', 'Unital']

sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> C = Magmas().CartesianProducts().example(); C
The Cartesian product of (Rational Field, Integer Ring, Integer Ring)
>>> C.category()
Join of Category of Cartesian products of commutative rings and
Category of Cartesian products of metric spaces
>>> sorted(C.category().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Commutative',
 'Distributive', 'Unital']

>>> TestSuite(C).run()
```

extra_super_categories()

This implements the fact that a subquotient (and therefore a quotient or subobject) of a finite set is finite.

EXAMPLES:

```
sage: Semigroups().CartesianProducts().extra_super_categories()
[Category of semigroups]
sage: Semigroups().CartesianProducts().super_categories()
[Category of semigroups, Category of Cartesian products of magmas]
```

```
>>> from sage.all import *
>>> Semigroups().CartesianProducts().extra_super_categories()
[Category of semigroups]
>>> Semigroups().CartesianProducts().super_categories()
[Category of semigroups, Category of Cartesian products of magmas]
```

class Commutative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

extra_super_categories()

EXAMPLES:

```
sage: MCA = Magmas().Commutative().Algebras(QQ)
sage: MCA.extra_super_categories()
[Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA = Magmas().Commutative().Algebras(QQ)
>>> MCA.extra_super_categories()
[Category of commutative magmas]
```

This implements the fact that the algebra of a commutative magma is commutative:

```
sage: MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

In particular, commutative monoid algebras are commutative algebras:

```
sage: MoCA = Monoids().Commutative().Algebras(QQ)
sage: MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

```
>>> from sage.all import *
>>> MoCA = Monoids().Commutative().Algebras(QQ)
>>> MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of commutative additive magmas is still a commutative additive magmas.

EXAMPLES:

```
sage: C = Magmas().Commutative().CartesianProducts()
sage: C.extra_super_categories()
[Category of commutative magmas]
sage: C.axioms()
frozenset({'Commutative'})
```

```
>>> from sage.all import *
>>> C = Magmas().Commutative().CartesianProducts()
>>> C.extra_super_categories()
[Category of commutative magmas]
>>> C.axioms()
frozenset({'Commutative'})
```

class ParentMethods

Bases: object

is_commutative()

Return True, since commutative magmas are commutative.

EXAMPLES:

```
sage: Parent(QQ, category=CommutativeRings()).is_commutative()
True
```

```
>>> from sage.all import *
>>> Parent(QQ, category=CommutativeRings()).is_commutative()
True
```

class ElementMethods

Bases: object

is_idempotent()

Test whether self is idempotent.

EXAMPLES:

```
sage: S = Semigroups().example("free"); S
An example of a semigroup:
 the free semigroup generated by ('a', 'b', 'c', 'd')
sage: a = S('a')
sage: a^2
'aa'
sage: a.is_idempotent()
False
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free"); S
An example of a semigroup:
 the free semigroup generated by ('a', 'b', 'c', 'd')
>>> a = S('a')
>>> a**Integer(2)
'aa'
>>> a.is_idempotent()
False
```

```
sage: L = Semigroups().example("leftzero"); L
An example of a semigroup: the left zero semigroup
sage: x = L('x')
sage: x^2
'x'
sage: x.is_idempotent()
True
```

```
>>> from sage.all import *
>>> L = Semigroups().example("leftzero"); L
An example of a semigroup: the left zero semigroup
>>> x = L('x')
>>> x**Integer(2)
'x'
>>> x.is_idempotent()
True
```

FinitelyGeneratedAsMagma

alias of *FinitelyGeneratedMagmas*

class JTrivial(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: object

multiplication_table(*names='letters'*, *elements=None*)

Return a table describing the multiplication operation.

> **Note**
>
> The order of the elements in the row and column headings is equal to the order given by the table's column_keys() method. The association can also be retrieved with the translation() method.

INPUT:

- names – the type of names used
  - 'letters' – lowercase ASCII letters are used for a base 26 representation of the elements' positions in the list given by column_keys(), padded to a common width with leading 'a's.
  - 'digits' – base 10 representation of the elements' positions in the list given by column_keys(), padded to a common width with leading zeros.
  - 'elements' – the string representations of the elements themselves.
  - a list - a list of strings, where the length of the list equals the number of elements.
- elements – (default: None) a list of elements of the magma, in forms that can be coerced into the structure, eg. their string representations. This may be used to impose an alternate ordering on the elements, perhaps when this is used in the context of a particular structure. The default is to use whatever ordering the S.list method returns. Or the elements can be a subset which is closed under the operation. In particular, this can be used when the base set is infinite.

OUTPUT:

The multiplication table as an object of the class OperationTable which defines several methods for manipulating and displaying the table. See the documentation there for full details to supplement the documentation here.

EXAMPLES:

The default is to represent elements as lowercase ASCII letters.

```
sage: G = CyclicPermutationGroup(5)                                    #␣
↪needs sage.groups
sage: G.multiplication_table()                                         #␣
↪needs sage.groups
*  a b c d e
 +----------
a| a b c d e
b| b c d e a
c| c d e a b
d| d e a b c
e| e a b c d
```

```
>>> from sage.all import *
>>> G = CyclicPermutationGroup(Integer(5))                             ␣
↪     # needs sage.groups
>>> G.multiplication_table()                                          #␣
↪needs sage.groups
*  a b c d e
 +----------
a| a b c d e
```

(continues on next page)

(continued from previous page)

```
b| b c d e a
c| c d e a b
d| d e a b c
e| e a b c d
```

All that is required is that an algebraic structure has a multiplication defined. A *LeftRegularBand* is an example of a finite semigroup. The `names` argument allows displaying the elements in different ways.

```
sage: from sage.categories.examples.finite_semigroups import␣
↪LeftRegularBand
sage: L = LeftRegularBand(('a', 'b'))
sage: T = L.multiplication_table(names='digits')                        #␣
↪needs sage.modules
sage: T.column_keys()                                                   #␣
↪needs sage.modules
('a', 'ab', 'b', 'ba')
sage: T                                                                 #␣
↪needs sage.modules
*  0 1 2 3
 +--------
0| 0 1 1 1
1| 1 1 1 1
2| 3 3 2 3
3| 3 3 3 3
```

```
>>> from sage.all import *
>>> from sage.categories.examples.finite_semigroups import LeftRegularBand
>>> L = LeftRegularBand(('a', 'b'))
>>> T = L.multiplication_table(names='digits')                          #␣
↪needs sage.modules
>>> T.column_keys()                                                    #␣
↪needs sage.modules
('a', 'ab', 'b', 'ba')
>>> T                                                                  #␣
↪needs sage.modules
*  0 1 2 3
 +--------
0| 0 1 1 1
1| 1 1 1 1
2| 3 3 2 3
3| 3 3 3 3
```

Specifying the elements in an alternative order can provide more insight into how the operation behaves.

```
sage: L = LeftRegularBand(('a', 'b', 'c'))
sage: elts = sorted(L.list())
sage: L.multiplication_table(elements=elts)                             #␣
↪needs sage.modules
*  a b c d e f g h i j k l m n o
 +-------------------------------
a| a b c d e b b c c c d d e e e
b| b b c c c b b c c c c c c c c
```

(continues on next page)

(continued from previous page)

```
c| c c c c c c c c c c c c c c c
d| d e e d e e e e e e d d e e e
e| e e e e e e e e e e e e e e e
f| g g h h h f g h i j i j j i j
g| g g h h h g g h h h h h h h h
h| h h h h h h h h h h h h h h h
i| j j j j j i j j i j i j j i j
j| j j j j j j j j j j j j j j j
k| l m m l m n o o n o k l m n o
l| l m m l m m m m m m l l m m m
m| m m m m m m m m m m m m m m m
n| o o o o o n o o n o n o o n o
o| o o o o o o o o o o o o o o o
```

```
>>> from sage.all import *
>>> L = LeftRegularBand(('a', 'b', 'c'))
>>> elts = sorted(L.list())
>>> L.multiplication_table(elements=elts)                                    #␣
↪needs sage.modules
*  a b c d e f g h i j k l m n o
 +------------------------------
a| a b c d e b b c c c d d e e e
b| b b c c c b b c c c c c c c c
c| c c c c c c c c c c c c c c c
d| d e e d e e e e e e d d e e e
e| e e e e e e e e e e e e e e e
f| g g h h h f g h i j i j j i j
g| g g h h h g g h h h h h h h h
h| h h h h h h h h h h h h h h h
i| j j j j j i j j i j i j j i j
j| j j j j j j j j j j j j j j j
k| l m m l m n o o n o k l m n o
l| l m m l m m m m m m l l m m m
m| m m m m m m m m m m m m m m m
n| o o o o o n o o n o n o o n o
o| o o o o o o o o o o o o o o o
```

The `elements` argument can be used to provide a subset of the elements of the structure. The subset must be closed under the operation. Elements need only be in a form that can be coerced into the set. The `names` argument can also be used to request that the elements be represented with their usual string representation.

```
sage: L = LeftRegularBand(('a','b','c'))
sage: elts=['a', 'c', 'ac', 'ca']
sage: L.multiplication_table(names='elements', elements=elts)             #␣
↪needs sage.modules
   *   'a'  'c' 'ac' 'ca'
    +--------------------
 'a'|  'a' 'ac' 'ac' 'ac'
 'c'| 'ca'  'c' 'ca' 'ca'
'ac'| 'ac' 'ac' 'ac' 'ac'
'ca'| 'ca' 'ca' 'ca' 'ca'
```

```
>>> from sage.all import *
>>> L = LeftRegularBand(('a','b','c'))
>>> elts=['a', 'c', 'ac', 'ca']
>>> L.multiplication_table(names='elements', elements=elts)           #␣
↪needs sage.modules
   *   'a'  'c' 'ac' 'ca'
    +--------------------
 'a'|  'a' 'ac' 'ac' 'ac'
 'c'| 'ca'  'c' 'ca' 'ca'
'ac'| 'ac' 'ac' 'ac' 'ac'
'ca'| 'ca' 'ca' 'ca' 'ca'
```

The table returned can be manipulated in various ways. See the documentation for `OperationTable` for more comprehensive documentation.

```
sage: # needs sage.groups sage.modules
sage: G = AlternatingGroup(3)
sage: T = G.multiplication_table()
sage: T.column_keys()
((), (1,2,3), (1,3,2))
sage: T.translation()
{'a': (), 'b': (1,2,3), 'c': (1,3,2)}
sage: T.change_names(['x', 'y', 'z'])
sage: T.translation()
{'x': (), 'y': (1,2,3), 'z': (1,3,2)}
sage: T
*  x y z
 +------
x| x y z
y| y z x
z| z x y
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> G = AlternatingGroup(Integer(3))
>>> T = G.multiplication_table()
>>> T.column_keys()
((), (1,2,3), (1,3,2))
>>> T.translation()
{'a': (), 'b': (1,2,3), 'c': (1,3,2)}
>>> T.change_names(['x', 'y', 'z'])
>>> T.translation()
{'x': (), 'y': (1,2,3), 'z': (1,3,2)}
>>> T
*  x y z
 +------
x| x y z
y| y z x
z| z x y
```

product(*x*, *y*)

The binary multiplication of the magma.

INPUT:

- x, y – elements of this magma

OUTPUT:

- an element of the magma (the product of x and y)

EXAMPLES:

```
sage: S = Semigroups().example("free")
sage: x = S('a'); y = S('b')
sage: S.product(x, y)
'ab'
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free")
>>> x = S('a'); y = S('b')
>>> S.product(x, y)
'ab'
```

A parent in `Magmas()` must either implement *`product()`* in the parent class or `_mul_` in the element class. By default, the addition method on elements `x._mul_(y)` calls `S.product(x,y)`, and reciprocally.

As a bonus, `S.product` models the binary function from `S` to `S`:

```
sage: bin = S.product
sage: bin(x,y)
'ab'
```

```
>>> from sage.all import *
>>> bin = S.product
>>> bin(x,y)
'ab'
```

Currently, `S.product` is just a bound method:

```
sage: bin
<bound method FreeSemigroup.product of An example of a semigroup:
 the free semigroup generated by ('a', 'b', 'c', 'd')>
```

```
>>> from sage.all import *
>>> bin
<bound method FreeSemigroup.product of An example of a semigroup:
 the free semigroup generated by ('a', 'b', 'c', 'd')>
```

When Sage will support multivariate morphisms, it will be possible, and in fact recommended, to enrich `S.product` with extra mathematical structure. This will typically be implemented using lazy attributes.:

```
sage: bin                 # todo: not implemented
Generic binary morphism:
From: (S x S)
To:   S
```

```
>>> from sage.all import *
>>> bin                 # todo: not implemented
Generic binary morphism:
```

(continues on next page)

(continued from previous page)

```
From: (S x S)
To:   S
```

product_from_element_class_mul(*x*, *y*)

alias of *product()*.

class Realizations(*category*, **args*)

Bases: *RealizationsCategory*

class ParentMethods

Bases: object

product_by_coercion(*left*, *right*)

Default implementation of product for realizations.

This method coerces to the realization specified by `self.realization_of().a_realization()`, computes the product in that realization, and then coerces back.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: Out = Sets().WithRealizations().example().Out(); Out
The subset algebra of {1, 2, 3} over Rational Field
 in the Out basis
sage: Out.product
<bound method Magmas.Realizations.ParentMethods.product_by_coercion
 of The subset algebra of {1, 2, 3} over Rational Field
  in the Out basis>
sage: Out.product.__module__
'sage.categories.magmas'
sage: x = Out.an_element()
sage: y = Out.an_element()
sage: Out.product(x, y)
Out[{}] + 4*Out[{1}] + 9*Out[{2}] + Out[{1, 2}]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Out = Sets().WithRealizations().example().Out(); Out
The subset algebra of {1, 2, 3} over Rational Field
 in the Out basis
>>> Out.product
<bound method Magmas.Realizations.ParentMethods.product_by_coercion
 of The subset algebra of {1, 2, 3} over Rational Field
  in the Out basis>
>>> Out.product.__module__
'sage.categories.magmas'
>>> x = Out.an_element()
>>> y = Out.an_element()
>>> Out.product(x, y)
Out[{}] + 4*Out[{1}] + 9*Out[{2}] + Out[{1, 2}]
```

class SubcategoryMethods

Bases: object

Associative()

Return the full subcategory of the associative objects of `self`.

A (multiplicative) *magma* M is *associative* if, for all $x, y, z \in M$,

$$x * (y * z) = (x * y) * z$$

See also

Wikipedia article Associative_property

EXAMPLES:

```
sage: Magmas().Associative()
Category of semigroups
```

```
>>> from sage.all import *
>>> Magmas().Associative()
Category of semigroups
```

Commutative()

Return the full subcategory of the commutative objects of `self`.

A (multiplicative) *magma* M is *commutative* if, for all $x, y \in M$,

$$x * y = y * x$$

See also

Wikipedia article Commutative_property

EXAMPLES:

```
sage: Magmas().Commutative()
Category of commutative magmas
sage: Monoids().Commutative()
Category of commutative monoids
```

```
>>> from sage.all import *
>>> Magmas().Commutative()
Category of commutative magmas
>>> Monoids().Commutative()
Category of commutative monoids
```

Distributive()

Return the full subcategory of the objects of `self` where $*$ is distributive on $+$.

INPUT:

- `self` – a subcategory of *Magmas* and *AdditiveMagmas*

Given that Sage does not yet know that the category *MagmasAndAdditiveMagmas* is the intersection of the categories *Magmas* and *AdditiveMagmas*, the method *Distributive()* is not available, as would be desirable, for this intersection.

This method is a workaround. It checks that `self` is a subcategory of both *Magmas* and *AdditiveMagmas* and upgrades it to a subcategory of *MagmasAndAdditiveMagmas* before applying the axiom. It complains otherwise, since the `Distributive` axiom does not make sense for a plain magma.

EXAMPLES:

```
sage: (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
sage: (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings

sage: Magmas().Distributive()
Traceback (most recent call last):
...
ValueError: The distributive axiom only makes sense on a magma
which is simultaneously an additive magma
sage: Semigroups().Distributive()
Traceback (most recent call last):
...
ValueError: The distributive axiom only makes sense on a magma
which is simultaneously an additive magma
```

```
>>> from sage.all import *
>>> (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
>>> (Monoids() & CommutativeAdditiveGroups()).Distributive()
Category of rings

>>> Magmas().Distributive()
Traceback (most recent call last):
...
ValueError: The distributive axiom only makes sense on a magma
which is simultaneously an additive magma
>>> Semigroups().Distributive()
Traceback (most recent call last):
...
ValueError: The distributive axiom only makes sense on a magma
which is simultaneously an additive magma
```

FinitelyGenerated()

Return the subcategory of the objects of `self` that are endowed with a distinguished finite set of (multiplicative) magma generators.

EXAMPLES:

This is a shorthand for *FinitelyGeneratedAsMagma()*, which see:

```
sage: Magmas().FinitelyGenerated()
Category of finitely generated magmas
sage: Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
```

(continues on next page)

(continued from previous page)

```
sage: Groups().FinitelyGenerated()
Category of finitely generated enumerated groups
```

```
>>> from sage.all import *
>>> Magmas().FinitelyGenerated()
Category of finitely generated magmas
>>> Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
>>> Groups().FinitelyGenerated()
Category of finitely generated enumerated groups
```

An error is raised if this is ambiguous:

```
sage: (Magmas() & AdditiveMagmas()).FinitelyGenerated()
Traceback (most recent call last):
...
ValueError: FinitelyGenerated is ambiguous for
Join of Category of magmas and Category of additive magmas.
Please use explicitly one of the FinitelyGeneratedAsXXX methods
```

```
>>> from sage.all import *
>>> (Magmas() & AdditiveMagmas()).FinitelyGenerated()
Traceback (most recent call last):
...
ValueError: FinitelyGenerated is ambiguous for
Join of Category of magmas and Category of additive magmas.
Please use explicitly one of the FinitelyGeneratedAsXXX methods
```

Note

Checking that there is no ambiguity currently assumes that all the other "finitely generated" axioms involve an additive structure. As of Sage 6.4, this is correct.

The use of this shorthand should be reserved for casual interactive use or when there is no risk of ambiguity.

FinitelyGeneratedAsMagma()

Return the subcategory of the objects of `self` that are endowed with a distinguished finite set of (multiplicative) magma generators.

A set S of elements of a multiplicative magma form a *set of generators* if any element of the magma can be expressed recursively from elements of S and products thereof.

It is not imposed that morphisms shall preserve the distinguished set of generators; hence this is a full subcategory.

See also

Wikipedia article Unital_magma#unital

EXAMPLES:

```
sage: Magmas().FinitelyGeneratedAsMagma()
Category of finitely generated magmas
```

```
>>> from sage.all import *
>>> Magmas().FinitelyGeneratedAsMagma()
Category of finitely generated magmas
```

Being finitely generated does depend on the structure: for a ring, being finitely generated as a magma, as an additive magma, or as a ring are different concepts. Hence the name of this axiom is explicit:

```
sage: Rings().FinitelyGeneratedAsMagma()
Category of finitely generated as magma enumerated rings
```

```
>>> from sage.all import *
>>> Rings().FinitelyGeneratedAsMagma()
Category of finitely generated as magma enumerated rings
```

On the other hand, it does not depend on the multiplicative structure: for example a group is finitely generated if and only if it is finitely generated as a magma. A short hand is provided when there is no ambiguity, and the output tries to reflect that:

```
sage: Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
sage: Groups().FinitelyGenerated()
Category of finitely generated enumerated groups

sage: Semigroups().FinitelyGenerated().axioms()
frozenset({'Associative', 'Enumerated', 'FinitelyGeneratedAsMagma'})
```

```
>>> from sage.all import *
>>> Semigroups().FinitelyGenerated()
Category of finitely generated semigroups
>>> Groups().FinitelyGenerated()
Category of finitely generated enumerated groups

>>> Semigroups().FinitelyGenerated().axioms()
frozenset({'Associative', 'Enumerated', 'FinitelyGeneratedAsMagma'})
```

Note that the set of generators may depend on the actual category; for example, in a group, one can often use less generators since it is allowed to take inverses.

JTrivial()

Return the full subcategory of the J-trivial objects of `self`.

This axiom is in fact only meaningful for *semigroups*. This stub definition is here as a workaround for Issue #20515, in order to define the J-trivial axiom as the intersection of the L and R-trivial axioms.

See also

Semigroups.SubcategoryMethods.JTrivial()

Unital()

Return the subcategory of the unital objects of `self`.

A (multiplicative) *magma* M is *unital* if it admits an element 1, called *unit*, such that for all $x \in M$,

$$1 * x = x * 1 = x$$

This element is necessarily unique, and should be provided as `M.one()`.

See also

Wikipedia article Unital_magma#unital

EXAMPLES:

```
sage: Magmas().Unital()
Category of unital magmas
sage: Semigroups().Unital()
Category of monoids
sage: Monoids().Unital()
Category of monoids
sage: from sage.categories.associative_algebras import AssociativeAlgebras
sage: AssociativeAlgebras(QQ).Unital()
Category of algebras over Rational Field
```

```
>>> from sage.all import *
>>> Magmas().Unital()
Category of unital magmas
>>> Semigroups().Unital()
Category of monoids
>>> Monoids().Unital()
Category of monoids
>>> from sage.categories.associative_algebras import AssociativeAlgebras
>>> AssociativeAlgebras(QQ).Unital()
Category of algebras over Rational Field
```

class Subquotients(*category*, **args*)

Bases: *SubquotientsCategory*

The category of subquotient magmas.

See *Sets.SubcategoryMethods.Subquotients()* for the general setup for subquotients. In the case of a subquotient magma S of a magma G, the condition that r be a morphism in `As` can be rewritten as follows:

- for any two $a, b \in S$ the identity $a \times_S b = r(l(a) \times_G l(b))$ holds.

This is used by this category to implement the product $\times_S$ of S from l and r and the product of G.

EXAMPLES:

```
sage: Semigroups().Subquotients().all_super_categories()
[Category of subquotients of semigroups, Category of semigroups,
 Category of subquotients of magmas, Category of magmas,
 Category of subquotients of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Semigroups().Subquotients().all_super_categories()
[Category of subquotients of semigroups, Category of semigroups,
 Category of subquotients of magmas, Category of magmas,
 Category of subquotients of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ParentMethods

Bases: object

product(*x*, *y*)

Return the product of two elements of self.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S
An example of a (sub)quotient semigroup:
a quotient of the left zero semigroup
sage: S.product(S(19), S(3))
19
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S
An example of a (sub)quotient semigroup:
a quotient of the left zero semigroup
>>> S.product(S(Integer(19)), S(Integer(3)))
19
```

Here is a more elaborate example involving a sub algebra:

```
sage: Z = SymmetricGroup(5).algebra(QQ).center()                    #␣
↪needs sage.combinat sage.groups
sage: B = Z.basis()                                                 #␣
↪needs sage.combinat sage.groups
sage: B[3] * B[2]                                                   #␣
↪needs sage.combinat sage.groups
4*B[2] + 6*B[3] + 5*B[6]
```

```
>>> from sage.all import *
>>> Z = SymmetricGroup(Integer(5)).algebra(QQ).center()                ␣
↪    # needs sage.combinat sage.groups
>>> B = Z.basis()                                                  #␣
↪needs sage.combinat sage.groups
>>> B[Integer(3)] * B[Integer(2)]                                      ␣
↪             # needs sage.combinat sage.groups
4*B[2] + 6*B[3] + 5*B[6]
```

class Unital(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

extra_super_categories()

EXAMPLES:

```
sage: MCA = Magmas().Commutative().Algebras(QQ)
sage: MCA.extra_super_categories()
[Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA = Magmas().Commutative().Algebras(QQ)
>>> MCA.extra_super_categories()
[Category of commutative magmas]
```

This implements the fact that the algebra of a commutative magma is commutative:

```
sage: MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

```
>>> from sage.all import *
>>> MCA.super_categories()
[Category of magma algebras over Rational Field,
 Category of commutative magmas]
```

In particular, commutative monoid algebras are commutative algebras:

```
sage: MoCA = Monoids().Commutative().Algebras(QQ)
sage: MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

```
>>> from sage.all import *
>>> MoCA = Monoids().Commutative().Algebras(QQ)
>>> MoCA.is_subcategory(Algebras(QQ).Commutative())
True
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ElementMethods

Bases: object

class ParentMethods

Bases: object

one()

Return the unit of this Cartesian product.

It is built from the units for the Cartesian factors of `self`.

EXAMPLES:

```
sage: cartesian_product([QQ, ZZ, RR]).one()                    #␣
↪needs sage.rings.real_mpfr
(1, 1, 1.00000000000000)
```

```
>>> from sage.all import *
>>> cartesian_product([QQ, ZZ, RR]).one()                          #␣
↪needs sage.rings.real_mpfr
(1, 1, 1.00000000000000)
```

extra_super_categories()

Implement the fact that a Cartesian product of unital magmas is a unital magma

EXAMPLES:

```
sage: C = Magmas().Unital().CartesianProducts()
sage: C.extra_super_categories()
[Category of unital magmas]
sage: C.axioms()
frozenset({'Unital'})

sage: Monoids().CartesianProducts().is_subcategory(Monoids())
True
```

```
>>> from sage.all import *
>>> C = Magmas().Unital().CartesianProducts()
>>> C.extra_super_categories()
[Category of unital magmas]
>>> C.axioms()
frozenset({'Unital'})

>>> Monoids().CartesianProducts().is_subcategory(Monoids())
True
```

class ElementMethods

Bases: object

class Inverse(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of magmas with inverses is a magma with inverse.

EXAMPLES:

```
sage: C = Magmas().Unital().Inverse().CartesianProducts()
sage: C.extra_super_categories()
[Category of inverse unital magmas]
sage: sorted(C.axioms())
['Inverse', 'Unital']
```

```
>>> from sage.all import *
>>> C = Magmas().Unital().Inverse().CartesianProducts()
>>> C.extra_super_categories()
[Category of inverse unital magmas]
```

(continues on next page)

(continued from previous page)

```
>>> sorted(C.axioms())
['Inverse', 'Unital']
```

class ParentMethods

Bases: object

is_empty()

Return whether self is empty.

Since this set is a unital magma it is not empty and this method always return False.

EXAMPLES:

```
sage: S = SymmetricGroup(2)                                             #␣
↪needs sage.groups
sage: S.is_empty()                                                      #␣
↪needs sage.groups
False

sage: M = Monoids().example()
sage: M.is_empty()
False
```

```
>>> from sage.all import *
>>> S = SymmetricGroup(Integer(2))                                        ␣
↪    # needs sage.groups
>>> S.is_empty()                                                       #␣
↪needs sage.groups
False

>>> M = Monoids().example()
>>> M.is_empty()
False
```

one()

Return the unit of the monoid, that is the unique neutral element for $*$.

Note

The default implementation is to coerce 1 into self. It is recommended to override this method because the coercion from the integers:

- is not always meaningful (except for 1);
- often uses self.one().

EXAMPLES:

```
sage: M = Monoids().example(); M
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
sage: M.one()
''
```

```
>>> from sage.all import *
>>> M = Monoids().example(); M
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
>>> M.one()
''
```

class Realizations(*category*, **args*)

Bases: *RealizationsCategory*

class ParentMethods

Bases: object

one()

Return the unit element of self.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: from sage.combinat.root_system.extended_affine_weyl_group␣
↪import ExtendedAffineWeylGroup
sage: PvW0 = ExtendedAffineWeylGroup(['A',2,1]).PvW0()
sage: PvW0 in Magmas().Unital().Realizations()
True
sage: PvW0.one()
1
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> from sage.combinat.root_system.extended_affine_weyl_group import␣
↪ExtendedAffineWeylGroup
>>> PvW0 = ExtendedAffineWeylGroup(['A',Integer(2),Integer(1)]).
↪PvW0()
>>> PvW0 in Magmas().Unital().Realizations()
True
>>> PvW0.one()
1
```

class SubcategoryMethods

Bases: object

Inverse()

Return the full subcategory of the inverse objects of self.

An inverse :class:` (multiplicative) magma <Magmas>` is a *unital magma* such that every element admits both an inverse on the left and on the right. Such a magma is also called a *loop*.

See also

Wikipedia article Inverse_element, Wikipedia article Quasigroup

EXAMPLES:

```
sage: Magmas().Unital().Inverse()
Category of inverse unital magmas
sage: Monoids().Inverse()
Category of groups
```

```
>>> from sage.all import *
>>> Magmas().Unital().Inverse()
Category of inverse unital magmas
>>> Monoids().Inverse()
Category of groups
```

additional_structure()

Return `self`.

Indeed, the category of unital magmas defines an additional structure, namely the unit of the magma which shall be preserved by morphisms.

See also

additional_structure()

EXAMPLES:

```
sage: Magmas().Unital().additional_structure()
Category of unital magmas
```

```
>>> from sage.all import *
>>> Magmas().Unital().additional_structure()
Category of unital magmas
```

super_categories()

EXAMPLES:

```
sage: Magmas().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Magmas().super_categories()
[Category of sets]
```

4.118 Magmas and Additive Magmas

class sage.categories.magmas_and_additive_magmas.**MagmasAndAdditiveMagmas**

Bases: *Category_singleton*

The category of sets $(S, +, *)$ with an additive operation '+' and a multiplicative operation $*$

EXAMPLES:

```
sage: from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
```

(continues on next page)

(continued from previous page)

```
sage: C = MagmasAndAdditiveMagmas(); C
Category of magmas and additive magmas
```

```
>>> from sage.all import *
>>> from sage.categories.magmas_and_additive_magmas import MagmasAndAdditiveMagmas
>>> C = MagmasAndAdditiveMagmas(); C
Category of magmas and additive magmas
```

This is the base category for the categories of rings and their variants:

```
sage: C.Distributive()
Category of distributive magmas and additive magmas
sage: C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().AdditiveInverse()
Category of rngs
sage: C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().Unital()
Category of semirings
sage: C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().AdditiveInverse().Unital()
Category of rings
```

```
>>> from sage.all import *
>>> C.Distributive()
Category of distributive magmas and additive magmas
>>> C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().AdditiveInverse()
Category of rngs
>>> C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().Unital()
Category of semirings
>>> C.Distributive().Associative().AdditiveAssociative().AdditiveCommutative().
↪AdditiveUnital().AdditiveInverse().Unital()
Category of rings
```

This category is really meant to represent the intersection of the categories of *Magmas* and *AdditiveMagmas*; however Sage's infrastructure does not allow yet to model this:

```
sage: Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas

sage: Magmas() & AdditiveMagmas()        # todo: not implemented
Category of magmas and additive magmas
```

```
>>> from sage.all import *
>>> Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas

>>> Magmas() & AdditiveMagmas()        # todo: not implemented
Category of magmas and additive magmas
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that this structure is stable under Cartesian products.

Distributive

alias of *DistributiveMagmasAndAdditiveMagmas*

class SubcategoryMethods

Bases: object

Distributive()

Return the full subcategory of the objects of `self` where $*$ is distributive on $+$.

A *magma* and *additive magma* M is *distributive* if, for all $x, y, z \in M$,

$$x * (y + z) = x * y + x * z \text{ and } (x + y) * z = x * z + y * z$$

EXAMPLES:

```
sage: from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
sage: C = MagmasAndAdditiveMagmas().Distributive(); C
Category of distributive magmas and additive magmas
```

```
>>> from sage.all import *
>>> from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
>>> C = MagmasAndAdditiveMagmas().Distributive(); C
Category of distributive magmas and additive magmas
```

Note

Given that Sage does not know that *MagmasAndAdditiveMagmas* is the intersection of *Magmas* and *AdditiveMagmas*, this method is not available for:

```
sage: Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas
```

```
>>> from sage.all import *
>>> Magmas() & AdditiveMagmas()
Join of Category of magmas and Category of additive magmas
```

Still, the natural syntax works:

```
sage: (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
```

```
>>> from sage.all import *
>>> (Magmas() & AdditiveMagmas()).Distributive()
Category of distributive magmas and additive magmas
```

thanks to a workaround implemented in *Magmas.SubcategoryMethods.Distributive()*:

```
sage: (Magmas() & AdditiveMagmas()).Distributive.__module__
'sage.categories.magmas'
```

```
>>> from sage.all import *
>>> (Magmas() & AdditiveMagmas()).Distributive.__module__
'sage.categories.magmas'
```

additional_structure()

Return `None`.

Indeed, this category is meant to represent the join of *AdditiveMagmas* and *Magmas*. As such, it defines no additional structure.

See also

Category.additional_structure()

EXAMPLES:

```
sage: from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
sage: MagmasAndAdditiveMagmas().additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
>>> MagmasAndAdditiveMagmas().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
sage: MagmasAndAdditiveMagmas().super_categories()
[Category of magmas, Category of additive magmas]
```

```
>>> from sage.all import *
>>> from sage.categories.magmas_and_additive_magmas import␣
↪MagmasAndAdditiveMagmas
>>> MagmasAndAdditiveMagmas().super_categories()
[Category of magmas, Category of additive magmas]
```

4.119 Non-unital non-associative algebras

class sage.categories.magmatic_algebras.**MagmaticAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of algebras over a given base ring.

An algebra over a ring R is a module over R endowed with a bilinear multiplication.

Warning

MagmaticAlgebras will eventually replace the current *Algebras* for consistency with e.g. Wikipedia article Algebras which assumes neither associativity nor the existence of a unit (see Issue #15043).

EXAMPLES:

```
sage: from sage.categories.magmatic_algebras import MagmaticAlgebras
sage: C = MagmaticAlgebras(ZZ); C
Category of magmatic algebras over Integer Ring
sage: C.super_categories()
[Category of additive commutative additive associative additive
  unital distributive magmas and additive magmas,
 Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> from sage.categories.magmatic_algebras import MagmaticAlgebras
>>> C = MagmaticAlgebras(ZZ); C
Category of magmatic algebras over Integer Ring
>>> C.super_categories()
[Category of additive commutative additive associative additive
  unital distributive magmas and additive magmas,
 Category of modules over Integer Ring]
```

Associative

alias of *AssociativeAlgebras*

class ParentMethods

Bases: object

algebra_generators()

Return a family of generators of this algebra.

EXAMPLES:

```
sage: F = AlgebrasWithBasis(QQ).example(); F                                #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
sage: F.algebra_generators()                                                #␣
↪needs sage.combinat sage.modules
Family (B[word: a], B[word: b], B[word: c])
```

```
>>> from sage.all import *
>>> F = AlgebrasWithBasis(QQ).example(); F                                  #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
>>> F.algebra_generators()                                                  #␣
↪needs sage.combinat sage.modules
Family (B[word: a], B[word: b], B[word: c])
```

Unital

alias of *UnitalAlgebras*

class WithBasis(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: object

derivations_basis()

Return a basis for the Lie algebra of derivations of `self` as matrices.

A derivation D of an algebra is an endomorphism of A such that

$$D(ab) = D(a)b + aD(b)$$

for all $a, b \in A$. The set of all derivations form a Lie algebra.

EXAMPLES:

We construct the Heisenberg Lie algebra as a multiplicative algebra:

```
sage: # needs sage.combinat sage.modules
sage: p_mult = matrix([[0,0,0], [0,0,-1], [0,0,0]])
sage: q_mult = matrix([[0,0,1], [0,0,0], [0,0,0]])
sage: A = algebras.FiniteDimensional(QQ,
....:           [p_mult, q_mult, matrix(QQ, 3, 3)], 'p,q,z')
sage: A.inject_variables()
Defining p, q, z
sage: p * q
z
sage: q * p
-z
sage: A.derivations_basis()
(
[1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]  [0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]  [1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]
[0 0 1], [0 0 0], [0 0 0], [0 0 1], [1 0 0], [0 1 0]
)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> p_mult = matrix([[Integer(0),Integer(0),Integer(0)], [Integer(0),
↪Integer(0),-Integer(1)], [Integer(0),Integer(0),Integer(0)]])
>>> q_mult = matrix([[Integer(0),Integer(0),Integer(1)], [Integer(0),
↪Integer(0),Integer(0)], [Integer(0),Integer(0),Integer(0)]])
>>> A = algebras.FiniteDimensional(QQ,
...           [p_mult, q_mult, matrix(QQ, Integer(3), Integer(3))],
↪'p,q,z')
>>> A.inject_variables()
Defining p, q, z
>>> p * q
z
>>> q * p
-z
```

(continues on next page)

(continued from previous page)

```
>>> A.derivations_basis()
(
[1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]  [0 0 0]  [0 0 0]
[0 0 0]  [0 0 0]  [1 0 0]  [0 1 0]  [0 0 0]  [0 0 0]
[0 0 1], [0 0 0], [0 0 0], [0 0 1], [1 0 0], [0 1 0]
)
```

We construct another example using the exterior algebra and verify we obtain a derivation:

```
sage: # needs sage.combinat sage.modules
sage: A = algebras.Exterior(QQ, 1)
sage: A.derivations_basis()
(
[0 0]
[0 1]
)
sage: D = A.module_morphism(matrix=A.derivations_basis()[0],
....:                       codomain=A)
sage: one, e = A.basis()
sage: all(D(a*b) == D(a) * b + a * D(b)
....:     for a in A.basis() for b in A.basis())
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = algebras.Exterior(QQ, Integer(1))
>>> A.derivations_basis()
(
[0 0]
[0 1]
)
>>> D = A.module_morphism(matrix=A.derivations_basis()[Integer(0)],
...                       codomain=A)
>>> one, e = A.basis()
>>> all(D(a*b) == D(a) * b + a * D(b)
...     for a in A.basis() for b in A.basis())
True
```

REFERENCES:

Wikipedia article Derivation_(differential_algebra)

to_finite_dimensional_algebra(*names='e'*, *assume_associative=True*, *assume_unital=True*)

Return `self` as a `FiniteDimensionalAlgebra`.

This forgets the indexing of the basis, flattening the elements into vectors. The name of the vectors can be passed via the `names` parameter.

To convert an element v of `self` into the `FiniteDimensionalAlgebra F` returned by this method, use `F(v.to_vector())`. For the inverse direction, use `self.from_vector(v.vector())`.

INPUT:

- `names` – string (default: `'e'`); names for the basis elements

- `assume_associative` – boolean (default: `False`); if `True`, then the category is set to `category.Associative()` and methods requiring associativity assume this
- `assume_unital` – boolean (default: `False`); if `True`, then the category is set to `category.Unital()` and methods requiring unitality assume this

EXAMPLES:

```
sage: B = DescentAlgebra(QQ,3).B()
sage: list(B.basis())
[B[1, 1, 1], B[1, 2], B[2, 1], B[3]]
sage: B_fda = B.to_finite_dimensional_algebra(); B_fda
Finite-dimensional algebra of degree 4 over Rational Field
sage: e = B_fda.basis(); e
Finite family {0: e0, 1: e1, 2: e2, 3: e3}
sage: x = e[1] * e[2]; x
e0 + e1
sage: y = B[1,2] * B[2,1]; y
B[1, 1, 1] + B[1, 2]
sage: B_fda(y.to_vector()) == x
True
sage: B.from_vector(x.vector()) == y
True
```

```
>>> from sage.all import *
>>> B = DescentAlgebra(QQ,Integer(3)).B()
>>> list(B.basis())
[B[1, 1, 1], B[1, 2], B[2, 1], B[3]]
>>> B_fda = B.to_finite_dimensional_algebra(); B_fda
Finite-dimensional algebra of degree 4 over Rational Field
>>> e = B_fda.basis(); e
Finite family {0: e0, 1: e1, 2: e2, 3: e3}
>>> x = e[Integer(1)] * e[Integer(2)]; x
e0 + e1
>>> y = B[Integer(1),Integer(2)] * B[Integer(2),Integer(1)]; y
B[1, 1, 1] + B[1, 2]
>>> B_fda(y.to_vector()) == x
True
>>> B.from_vector(x.vector()) == y
True
```

class ParentMethods

Bases: `object`

algebra_generators()

Return generators for this algebra.

This default implementation returns the basis of this algebra.

OUTPUT: a family

See also

- *basis()*
- *MagmaticAlgebras.ParentMethods.algebra_generators()*

EXAMPLES:

```
sage: D4 = DescentAlgebra(QQ, 4).B()                                    #␣
↪needs sage.combinat sage.groups sage.modules
sage: D4.algebra_generators()                                           #␣
↪needs sage.combinat sage.groups sage.modules
Lazy family (...)_{i in Compositions of 4}

sage: R.<x> = ZZ[]
sage: P = PartitionAlgebra(1, x, R)                                     #␣
↪needs sage.combinat sage.modules
sage: P.algebra_generators()                                            #␣
↪needs sage.combinat sage.modules
Lazy family (Term map
 from Partition diagrams of order 1
   to Partition Algebra of rank 1 with parameter x
        over Univariate Polynomial Ring in x
         over Integer Ring(i))_{i in Partition diagrams of order 1}
```

```
>>> from sage.all import *
>>> D4 = DescentAlgebra(QQ, Integer(4)).B()                                 ␣
↪    # needs sage.combinat sage.groups sage.modules
>>> D4.algebra_generators()                                               #␣
↪needs sage.combinat sage.groups sage.modules
Lazy family (...)_{i in Compositions of 4}

>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> P = PartitionAlgebra(Integer(1), x, R)                                  ␣
↪    # needs sage.combinat sage.modules
>>> P.algebra_generators()                                                #␣
↪needs sage.combinat sage.modules
Lazy family (Term map
 from Partition diagrams of order 1
   to Partition Algebra of rank 1 with parameter x
        over Univariate Polynomial Ring in x
         over Integer Ring(i))_{i in Partition diagrams of order 1}
```

product()

The product of the algebra, as per *Magmas.ParentMethods.product()*

By default, this is implemented using one of the following methods, in the specified order:

- *product_on_basis()*
- *product_by_coercion()*

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                               #␣
↪needs sage.combinat sage.modules
sage: a, b, c = A.algebra_generators()                                  #␣
↪needs sage.combinat sage.modules
sage: A.product(a + 2*b, 3*c)                                           #␣
↪needs sage.combinat sage.modules
3*B[word: ac] + 6*B[word: bc]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                    #␣
↪needs sage.combinat sage.modules
>>> a, b, c = A.algebra_generators()                                       #␣
↪needs sage.combinat sage.modules
>>> A.product(a + Integer(2)*b, Integer(3)*c)                                ␣
↪              # needs sage.combinat sage.modules
3*B[word: ac] + 6*B[word: bc]
```

product_on_basis(*i*, *j*)

The product of the algebra on the basis (optional).

INPUT:

- `i`, `j` – the indices of two elements of the basis of `self`

Return the product of the two corresponding basis elements indexed by `i` and `j`.

If implemented, *product()* is defined from it by bilinearity.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                                  #␣
↪needs sage.combinat sage.modules
sage: Word = A.basis().keys()                                              #␣
↪needs sage.combinat sage.modules
sage: A.product_on_basis(Word("abc"), Word("cba"))                         #␣
↪needs sage.combinat sage.modules
B[word: abccba]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                    #␣
↪needs sage.combinat sage.modules
>>> Word = A.basis().keys()                                               #␣
↪needs sage.combinat sage.modules
>>> A.product_on_basis(Word("abc"), Word("cba"))                          #␣
↪needs sage.combinat sage.modules
B[word: abccba]
```

additional_structure()

Return `None`.

Indeed, the category of (magmatic) algebras defines no new structure: a morphism of modules and of magmas between two (magmatic) algebras is a (magmatic) algebra morphism.

See also

Category.additional_structure()

Todo

This category should be a *CategoryWithAxiom*, the axiom specifying the compatibility between the magma and module structure.

EXAMPLES:

```
sage: from sage.categories.magmatic_algebras import MagmaticAlgebras
sage: MagmaticAlgebras(ZZ).additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.magmatic_algebras import MagmaticAlgebras
>>> MagmaticAlgebras(ZZ).additional_structure()
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.magmatic_algebras import MagmaticAlgebras
sage: MA = MagmaticAlgebras(ZZ)
sage: MA.super_categories()
[Category of additive commutative additive associative additive
  unital distributive magmas and additive magmas,
 Category of modules over Integer Ring]

sage: from sage.categories.additive_semigroups import AdditiveSemigroups
sage: MA.is_subcategory((AdditiveSemigroups() & Magmas()).Distributive())
True
```

```
>>> from sage.all import *
>>> from sage.categories.magmatic_algebras import MagmaticAlgebras
>>> MA = MagmaticAlgebras(ZZ)
>>> MA.super_categories()
[Category of additive commutative additive associative additive
  unital distributive magmas and additive magmas,
 Category of modules over Integer Ring]

>>> from sage.categories.additive_semigroups import AdditiveSemigroups
>>> MA.is_subcategory((AdditiveSemigroups() & Magmas()).Distributive())
True
```

4.120 Manifolds

class sage.categories.manifolds.**ComplexManifolds**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of complex manifolds.

A d-dimensional complex manifold is a manifold whose underlying vector space is $\mathbf{C}^d$ and has a holomorphic atlas.

super_categories()

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
```

(continues on next page)

(continued from previous page)

```
>>> Manifolds(RR).super_categories()
[Category of topological spaces]
```

class sage.categories.manifolds.**Manifolds**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of manifolds over any topological field.

Let k be a topological field. A d-dimensional *k-manifold* M is a second countable Hausdorff space such that the neighborhood of any point $x \in M$ is homeomorphic to k^d.

EXAMPLES:

```
sage: # needs sage.rings.real_mpfr
sage: from sage.categories.manifolds import Manifolds
sage: C = Manifolds(RR); C
Category of manifolds over Real Field with 53 bits of precision
sage: C.super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> # needs sage.rings.real_mpfr
>>> from sage.categories.manifolds import Manifolds
>>> C = Manifolds(RR); C
Category of manifolds over Real Field with 53 bits of precision
>>> C.super_categories()
[Category of topological spaces]
```

class AlmostComplex(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of almost complex manifolds.

An *almost complex manifold* M is a manifold with a smooth tensor field J of rank $(1, 1)$ such that $J^2 = -1$ when regarded as a vector bundle isomorphism $J : TM \to TM$ on the tangent bundle. The tensor field J is called the *almost complex structure* of M.

extra_super_categories()

Return the extra super categories of `self`.

An almost complex manifold is smooth.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).AlmostComplex().super_categories()  # indirect␣
↪doctest          # needs sage.rings.real_mpfr
[Category of smooth manifolds
 over Real Field with 53 bits of precision]
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).AlmostComplex().super_categories()  # indirect doctest ␣
↪        # needs sage.rings.real_mpfr
[Category of smooth manifolds
 over Real Field with 53 bits of precision]
```

class Analytic(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of complex manifolds.

An analytic manifold is a manifold with an analytic atlas.

extra_super_categories()

Return the extra super categories of `self`.

An analytic manifold is smooth.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Analytic().super_categories()  # indirect doctest   #␣
↪needs sage.rings.real_mpfr
[Category of smooth manifolds
 over Real Field with 53 bits of precision]
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Analytic().super_categories()  # indirect doctest   #␣
↪needs sage.rings.real_mpfr
[Category of smooth manifolds
 over Real Field with 53 bits of precision]
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of connected manifolds.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: C = Manifolds(RR).Connected()
sage: TestSuite(C).run(skip='_test_category_over_bases')
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> C = Manifolds(RR).Connected()
>>> TestSuite(C).run(skip='_test_category_over_bases')
```

class Differentiable(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of differentiable manifolds.

A differentiable manifold is a manifold with a differentiable atlas.

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional manifolds.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: C = Manifolds(RR).FiniteDimensional()
sage: TestSuite(C).run(skip='_test_category_over_bases')
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> C = Manifolds(RR).FiniteDimensional()
>>> TestSuite(C).run(skip='_test_category_over_bases')
```

class ParentMethods

Bases: object

dimension()

Return the dimension of self.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: M = Manifolds(RR).example()
sage: M.dimension()
3
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> M = Manifolds(RR).example()
>>> M.dimension()
3
```

class Smooth(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of smooth manifolds.

A smooth manifold is a manifold with a smooth atlas.

extra_super_categories()

Return the extra super categories of self.

A smooth manifold is differentiable.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Smooth().super_categories()  # indirect doctest     #␣
↪needs sage.rings.real_mpfr
[Category of differentiable manifolds
 over Real Field with 53 bits of precision]
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Smooth().super_categories()  # indirect doctest     #␣
↪needs sage.rings.real_mpfr
[Category of differentiable manifolds
 over Real Field with 53 bits of precision]
```

class SubcategoryMethods

Bases: object

AlmostComplex()

Return the subcategory of the almost complex objects of self.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).AlmostComplex()                                    #␣
↪needs sage.rings.real_mpfr
Category of almost complex manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).AlmostComplex()                                      #␣
↪needs sage.rings.real_mpfr
Category of almost complex manifolds
 over Real Field with 53 bits of precision
```

Analytic()

Return the subcategory of the analytic objects of self.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Analytic()                                         #␣
↪needs sage.rings.real_mpfr
Category of analytic manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Analytic()                                           #␣
↪needs sage.rings.real_mpfr
Category of analytic manifolds
 over Real Field with 53 bits of precision
```

Complex()

Return the subcategory of manifolds over $\mathbf{C}$ of self.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(CC).Complex()                                          #␣
↪needs sage.rings.real_mpfr
Category of complex manifolds over
 Complex Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(CC).Complex()                                            #␣
↪needs sage.rings.real_mpfr
```

(continues on next page)

(continued from previous page)

```
Category of complex manifolds over
 Complex Field with 53 bits of precision
```

Connected()

Return the full subcategory of the connected objects of `self`.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Connected()                                             #␣
↪needs sage.rings.real_mpfr
Category of connected manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Connected()                                               #␣
↪needs sage.rings.real_mpfr
Category of connected manifolds
 over Real Field with 53 bits of precision
```

Differentiable()

Return the subcategory of the differentiable objects of `self`.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Differentiable()                                        #␣
↪needs sage.rings.real_mpfr
Category of differentiable manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Differentiable()                                          #␣
↪needs sage.rings.real_mpfr
Category of differentiable manifolds
 over Real Field with 53 bits of precision
```

FiniteDimensional()

Return the full subcategory of the finite dimensional objects of `self`.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: C = Manifolds(RR).Connected().FiniteDimensional(); C                  #␣
↪needs sage.rings.real_mpfr
Category of finite dimensional connected manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
```

(continues on next page)

(continued from previous page)

```
>>> C = Manifolds(RR).Connected().FiniteDimensional(); C                    #␣
↪needs sage.rings.real_mpfr
Category of finite dimensional connected manifolds
 over Real Field with 53 bits of precision
```

Smooth()

Return the subcategory of the smooth objects of `self`.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).Smooth()                                                #␣
↪needs sage.rings.real_mpfr
Category of smooth manifolds
 over Real Field with 53 bits of precision
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).Smooth()                                                  #␣
↪needs sage.rings.real_mpfr
Category of smooth manifolds
 over Real Field with 53 bits of precision
```

additional_structure()

Return `None`.

Indeed, the category of manifolds defines no new structure: a morphism of topological spaces between manifolds is a manifold morphism.

See also

Category.additional_structure()

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).additional_structure()
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> Manifolds(RR).additional_structure()
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: Manifolds(RR).super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
```

(continues on next page)

(continued from previous page)

```
>>> Manifolds(RR).super_categories()
[Category of topological spaces]
```

4.121 Matrix algebras

class sage.categories.matrix_algebras.**MatrixAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of matrix algebras over a field.

EXAMPLES:

```
sage: MatrixAlgebras(RationalField())
Category of matrix algebras over Rational Field
```

```
>>> from sage.all import *
>>> MatrixAlgebras(RationalField())
Category of matrix algebras over Rational Field
```

super_categories()

EXAMPLES:

```
sage: MatrixAlgebras(QQ).super_categories()
[Category of algebras over Rational Field]
```

```
>>> from sage.all import *
>>> MatrixAlgebras(QQ).super_categories()
[Category of algebras over Rational Field]
```

4.122 Metric Spaces

class sage.categories.metric_spaces.**MetricSpaces**(*category*, **args*)

Bases: *MetricSpacesCategory*

The category of metric spaces.

A *metric* on a set S is a function $d : S \times S \to \mathbf{R}$ such that:

- $d(a, b) \geq 0$,
- $d(a, b) = 0$ if and only if $a = b$.

A metric space is a set S with a distinguished metric.

Implementation

Objects in this category must implement either a `dist` on the parent or the elements or `metric` on the parent; otherwise this will cause an infinite recursion.

Todo

- Implement a general geodesics class.
- Implement a category for metric additive groups and move the generic distance $d(a, b) = |a - b|$ there.
- Incorporate the length of a geodesic as part of the default distance cycle.

EXAMPLES:

```
sage: from sage.categories.metric_spaces import MetricSpaces
sage: C = MetricSpaces()
sage: C
Category of metric spaces
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> from sage.categories.metric_spaces import MetricSpaces
>>> C = MetricSpaces()
>>> C
Category of metric spaces
>>> TestSuite(C).run()
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

class ParentMethods

Bases: object

dist(*a*, *b*)

Return the distance between a and b in self.

It is defined as the maximum of the distances within the Cartesian factors.

EXAMPLES:

```
sage: from sage.categories.metric_spaces import MetricSpaces
sage: Q2 = QQ.cartesian_product(QQ)
sage: Q2.category()
Join of
Category of Cartesian products of commutative rings and
Category of Cartesian products of metric spaces
sage: Q2 in MetricSpaces()
True
sage: Q2.dist((0, 0), (2, 3))
3
```

```
>>> from sage.all import *
>>> from sage.categories.metric_spaces import MetricSpaces
>>> Q2 = QQ.cartesian_product(QQ)
>>> Q2.category()
Join of
Category of Cartesian products of commutative rings and
Category of Cartesian products of metric spaces
>>> Q2 in MetricSpaces()
True
```

(continues on next page)

(continued from previous page)

```
>>> Q2.dist((Integer(0), Integer(0)), (Integer(2), Integer(3)))
3
```

extra_super_categories()

Implement the fact that a (finite) Cartesian product of metric spaces is a metric space.

EXAMPLES:

```
sage: from sage.categories.metric_spaces import MetricSpaces
sage: C = MetricSpaces().CartesianProducts()
sage: C.extra_super_categories()
[Category of metric spaces]
sage: C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of metric spaces]
sage: C.axioms()
frozenset()
```

```
>>> from sage.all import *
>>> from sage.categories.metric_spaces import MetricSpaces
>>> C = MetricSpaces().CartesianProducts()
>>> C.extra_super_categories()
[Category of metric spaces]
>>> C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of metric spaces]
>>> C.axioms()
frozenset()
```

class Complete(*base_category*)

Bases: *CategoryWithAxiom*

The category of complete metric spaces.

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a (finite) Cartesian product of complete metric spaces is a complete metric space.

EXAMPLES:

```
sage: from sage.categories.metric_spaces import MetricSpaces
sage: C = MetricSpaces().Complete().CartesianProducts()
sage: C.extra_super_categories()
[Category of complete metric spaces]
sage: C.super_categories()
[Category of Cartesian products of metric spaces,
 Category of complete metric spaces]
sage: C.axioms()
frozenset({'Complete'})

sage: R2 = RR.cartesian_product(RR)
```

(continues on next page)

(continued from previous page)

```
sage: R2 in MetricSpaces()
True
sage: R2 in MetricSpaces().Complete()
True

sage: QR = QQ.cartesian_product(RR)
sage: QR in MetricSpaces()
True
sage: QR in MetricSpaces().Complete()
False
```

```
>>> from sage.all import *
>>> from sage.categories.metric_spaces import MetricSpaces
>>> C = MetricSpaces().Complete().CartesianProducts()
>>> C.extra_super_categories()
[Category of complete metric spaces]
>>> C.super_categories()
[Category of Cartesian products of metric spaces,
 Category of complete metric spaces]
>>> C.axioms()
frozenset({'Complete'})

>>> R2 = RR.cartesian_product(RR)
>>> R2 in MetricSpaces()
True
>>> R2 in MetricSpaces().Complete()
True

>>> QR = QQ.cartesian_product(RR)
>>> QR in MetricSpaces()
True
>>> QR in MetricSpaces().Complete()
False
```

class ElementMethods

Bases: `object`

abs()

Return the absolute value of `self`.

EXAMPLES:

```
sage: CC(I).abs()                                                           #␣
↪needs sage.rings.real_mpfr sage.symbolic
1.00000000000000
```

```
>>> from sage.all import *
>>> CC(I).abs()                                                             #␣
↪needs sage.rings.real_mpfr sage.symbolic
1.00000000000000
```

dist(*b*)

Return the distance between `self` and `other`.

EXAMPLES:

```
sage: # needs sage.symbolic
sage: UHP = HyperbolicPlane().UHP()
sage: p1 = UHP.get_point(5 + 7*I)
sage: p2 = UHP.get_point(1 + I)
sage: p1.dist(p2)
arccosh(33/7)
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> UHP = HyperbolicPlane().UHP()
>>> p1 = UHP.get_point(Integer(5) + Integer(7)*I)
>>> p2 = UHP.get_point(Integer(1) + I)
>>> p1.dist(p2)
arccosh(33/7)
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

The category of homsets of metric spaces.

It consists of the metric maps, that is, the Lipschitz functions with Lipschitz constant 1.

class ElementMethods

Bases: object

class ParentMethods

Bases: object

dist(*a*, *b*)

Return the distance between a and b in self.

EXAMPLES:

```
sage: # needs sage.symbolic
sage: UHP = HyperbolicPlane().UHP()
sage: p1 = UHP.get_point(5 + 7*I)
sage: p2 = UHP.get_point(1.0 + I)
sage: UHP.dist(p1, p2)
2.23230104635820

sage: PD = HyperbolicPlane().PD()                                    #␣
↪needs sage.symbolic
sage: PD.dist(PD.get_point(0), PD.get_point(I/2))                    #␣
↪needs sage.symbolic
arccosh(5/3)
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> UHP = HyperbolicPlane().UHP()
>>> p1 = UHP.get_point(Integer(5) + Integer(7)*I)
>>> p2 = UHP.get_point(RealNumber('1.0') + I)
>>> UHP.dist(p1, p2)
2.23230104635820
```

(continues on next page)

(continued from previous page)

```
>>> PD = HyperbolicPlane().PD()                                    #␣
↪needs sage.symbolic
>>> PD.dist(PD.get_point(Integer(0)), PD.get_point(I/Integer(2)))        ␣
↪              # needs sage.symbolic
arccosh(5/3)
```

metric_function()

Return the metric function of `self`.

EXAMPLES:

```
sage: # needs sage.symbolic
sage: UHP = HyperbolicPlane().UHP()
sage: m = UHP.metric_function()
sage: p1 = UHP.get_point(5 + 7*I)
sage: p2 = UHP.get_point(1.0 + I)
sage: m(p1, p2)
2.23230104635820
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> UHP = HyperbolicPlane().UHP()
>>> m = UHP.metric_function()
>>> p1 = UHP.get_point(Integer(5) + Integer(7)*I)
>>> p2 = UHP.get_point(RealNumber('1.0') + I)
>>> m(p1, p2)
2.23230104635820
```

class SubcategoryMethods

Bases: `object`

Complete()

Return the full subcategory of the complete objects of `self`.

EXAMPLES:

```
sage: Sets().Metric().Complete()
Category of complete metric spaces
```

```
>>> from sage.all import *
>>> Sets().Metric().Complete()
Category of complete metric spaces
```

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

class ParentMethods

Bases: `object`

dist(*a*, *b*)

Return the distance between `a` and `b` by converting them to a realization of `self` and doing the computation.

EXAMPLES:

```
sage: # needs sage.symbolic
sage: H = HyperbolicPlane()
sage: PD = H.PD()
sage: p1 = PD.get_point(0)
sage: p2 = PD.get_point(I/2)
sage: H.dist(p1, p2)
arccosh(5/3)
```

```
>>> from sage.all import *
>>> # needs sage.symbolic
>>> H = HyperbolicPlane()
>>> PD = H.PD()
>>> p1 = PD.get_point(Integer(0))
>>> p2 = PD.get_point(I/Integer(2))
>>> H.dist(p1, p2)
arccosh(5/3)
```

class sage.categories.metric_spaces.**MetricSpacesCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

classmethod default_super_categories(*category*)

Return the default super categories of `category.Metric()`.

Mathematical meaning: if A is a metric space in the category C, then A is also a topological space.

INPUT:

- `cls` – the class `MetricSpaces`
- `category` – a category Cat

OUTPUT:

A (join) category

In practice, this returns `category.Metric()`, joined together with the result of the method *RegressiveCovariantConstructionCategory.default_super_categories()* (that is the join of `category` and `cat.Metric()` for each `cat` in the super categories of `category`).

EXAMPLES:

Consider `category=Groups()`. Then, a group G with a metric is simultaneously a topological group by itself, and a metric space:

```
sage: Groups().Metric().super_categories()
[Category of topological groups, Category of metric spaces]
```

```
>>> from sage.all import *
>>> Groups().Metric().super_categories()
[Category of topological groups, Category of metric spaces]
```

This resulted from the following call:

```
sage: sage.categories.metric_spaces.MetricSpacesCategory.default_super_
↪categories(Groups())
Join of Category of topological groups and Category of metric spaces
```

```
>>> from sage.all import *
>>> sage.categories.metric_spaces.MetricSpacesCategory.default_super_
↪categories(Groups())
Join of Category of topological groups and Category of metric spaces
```

4.123 Modular abelian varieties

class sage.categories.modular_abelian_varieties.**ModularAbelianVarieties**(*Y*)

Bases: *Category_over_base*

The category of modular abelian varieties over a given field.

EXAMPLES:

```
sage: ModularAbelianVarieties(QQ)
Category of modular abelian varieties over Rational Field
```

```
>>> from sage.all import *
>>> ModularAbelianVarieties(QQ)
Category of modular abelian varieties over Rational Field
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class Endset(*base_category*)

Bases: *CategoryWithAxiom*

extra_super_categories()

Implement the fact that an endset of modular abelian variety is a ring.

EXAMPLES:

```
sage: ModularAbelianVarieties(QQ).Endsets().extra_super_categories()
[Category of rings]
```

```
>>> from sage.all import *
>>> ModularAbelianVarieties(QQ).Endsets().extra_super_categories()
[Category of rings]
```

base_field()

EXAMPLES:

```
sage: ModularAbelianVarieties(QQ).base_field()
Rational Field
```

```
>>> from sage.all import *
>>> ModularAbelianVarieties(QQ).base_field()
Rational Field
```

super_categories()

EXAMPLES:

```
sage: ModularAbelianVarieties(QQ).super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> ModularAbelianVarieties(QQ).super_categories()
[Category of sets]
```

4.124 Modules

class sage.categories.modules.**Modules**(*base*, *name=None*)

Bases: *Category_module*

The category of all modules over a base ring R.

An R-module M is a left and right R-module over a commutative ring R such that:

$$r * (x * s) = (r * x) * s \qquad \forall r, s \in R \text{ and } x \in M$$

INPUT:

- `base_ring` – a ring R or subcategory of `Rings()`
- `dispatch` – boolean (for internal use; default: `True`)

When the base ring is a field, the category of vector spaces is returned instead (unless `dispatch == False`).

Warning

Outside of the context of symmetric modules over a commutative ring, the specifications of this category are fuzzy and not yet set in stone (see below). The code in this category and its subcategories is therefore prone to bugs or arbitrary limitations in this case.

EXAMPLES:

```
sage: Modules(ZZ)
Category of modules over Integer Ring
sage: Modules(QQ)
Category of vector spaces over Rational Field

sage: Modules(Rings())
Category of modules over rings
sage: Modules(FiniteFields())
Category of vector spaces over finite enumerated fields

sage: Modules(Integers(9))
Category of modules over Ring of integers modulo 9

sage: Modules(Integers(9)).super_categories()
[Category of bimodules over Ring of integers modulo 9 on the left
                        and Ring of integers modulo 9 on the right]

sage: Modules(ZZ).super_categories()
[Category of bimodules over Integer Ring on the left
```

(continues on next page)

(continued from previous page)

```
                        and Integer Ring on the right]

sage: Modules == RingModules
True

sage: Modules(ZZ['x']).is_abelian()    # see #6081
True
```

```
>>> from sage.all import *
>>> Modules(ZZ)
Category of modules over Integer Ring
>>> Modules(QQ)
Category of vector spaces over Rational Field

>>> Modules(Rings())
Category of modules over rings
>>> Modules(FiniteFields())
Category of vector spaces over finite enumerated fields

>>> Modules(Integers(Integer(9)))
Category of modules over Ring of integers modulo 9

>>> Modules(Integers(Integer(9))).super_categories()
[Category of bimodules over Ring of integers modulo 9 on the left
                        and Ring of integers modulo 9 on the right]

>>> Modules(ZZ).super_categories()
[Category of bimodules over Integer Ring on the left
                        and Integer Ring on the right]

>>> Modules == RingModules
True

>>> Modules(ZZ['x']).is_abelian()    # see #6081
True
```

Todo

- Clarify the distinction, if any, with `BiModules(R, R)`. In particular, if R is a commutative ring (e.g. a field), some pieces of the code possibly assume that M is a *symmetric `R`-`R`-bimodule*:

$$r * x = x * r \qquad \forall r \in R \text{ and } x \in M$$

- Make sure that non symmetric modules are properly supported by all the code, and advertise it.
- Make sure that non commutative rings are properly supported by all the code, and advertise it.
- Add support for base semirings.
- Implement a `FreeModules(R)` category, when so prompted by a concrete use case: e.g. modeling a free module with several bases (using *`Realizations()`*) or with an atlas of local maps (see e.g. Issue #15916).

class CartesianProducts(*category*, **args*)

Bases: *`CartesianProductsCategory`*

The category of modules constructed as Cartesian products of modules.

This construction gives the direct product of modules. The implementation is based on the following resources:

- http://groups.google.fr/group/sage-devel/browse_thread/thread/35a72b1d0a2fc77a/348f42ae77a66d16#348f42ae77a66d16
- Wikipedia article Direct_product

class ElementMethods

Bases: `object`

class ParentMethods

Bases: `object`

extra_super_categories()

A Cartesian product of modules is endowed with a natural module structure.

EXAMPLES:

```
sage: Modules(ZZ).CartesianProducts().extra_super_categories()
[Category of modules over Integer Ring]
sage: Modules(ZZ).CartesianProducts().super_categories()
[Category of Cartesian products of commutative additive groups,
 Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).CartesianProducts().extra_super_categories()
[Category of modules over Integer Ring]
>>> Modules(ZZ).CartesianProducts().super_categories()
[Category of Cartesian products of commutative additive groups,
 Category of modules over Integer Ring]
```

class ElementMethods

Bases: `object`

Filtered

alias of *FilteredModules*

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

Implement the fact that a (finite) tensor product of finite dimensional modules is a finite dimensional module.

EXAMPLES:

```
sage: Modules(ZZ).FiniteDimensional().TensorProducts().extra_super_
↪categories()
[Category of finite dimensional modules over Integer Ring]
sage: Modules(QQ).FiniteDimensional().TensorProducts().
↪FiniteDimensional()
Category of tensor products of finite dimensional vector spaces
 over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).FiniteDimensional().TensorProducts().extra_super_
↪categories()
[Category of finite dimensional modules over Integer Ring]
>>> Modules(QQ).FiniteDimensional().TensorProducts().
↪FiniteDimensional()
Category of tensor products of finite dimensional vector spaces
 over Rational Field
```

extra_super_categories()

Implement the fact that a finite dimensional module over a finite ring is finite.

EXAMPLES:

```
sage: Modules(IntegerModRing(4)).FiniteDimensional().extra_super_
↪categories()
[Category of finite sets]
sage: Modules(ZZ).FiniteDimensional().extra_super_categories()
[]
sage: Modules(GF(5)).FiniteDimensional().is_subcategory(Sets().Finite())
True
sage: Modules(ZZ).FiniteDimensional().is_subcategory(Sets().Finite())
False

sage: Modules(Rings().Finite()).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
sage: Modules(Rings()).FiniteDimensional().is_subcategory(Sets().Finite())
False
```

```
>>> from sage.all import *
>>> Modules(IntegerModRing(Integer(4))).FiniteDimensional().extra_super_
↪categories()
[Category of finite sets]
>>> Modules(ZZ).FiniteDimensional().extra_super_categories()
[]
>>> Modules(GF(Integer(5))).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
>>> Modules(ZZ).FiniteDimensional().is_subcategory(Sets().Finite())
False

>>> Modules(Rings().Finite()).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
>>> Modules(Rings()).FiniteDimensional().is_subcategory(Sets().Finite())
False
```

class FinitelyPresented(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

extra_super_categories()

Implement the fact that a finitely presented module over a finite ring is finite.

EXAMPLES:

```
sage: Modules(IntegerModRing(4)).FiniteDimensional().extra_super_
↪categories()
[Category of finite sets]
sage: Modules(ZZ).FiniteDimensional().extra_super_categories()
[]
sage: Modules(GF(5)).FiniteDimensional().is_subcategory(Sets().Finite())
True
sage: Modules(ZZ).FiniteDimensional().is_subcategory(Sets().Finite())
False

sage: Modules(Rings().Finite()).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
sage: Modules(Rings()).FiniteDimensional().is_subcategory(Sets().Finite())
False
```

```
>>> from sage.all import *
>>> Modules(IntegerModRing(Integer(4))).FiniteDimensional().extra_super_
↪categories()
[Category of finite sets]
>>> Modules(ZZ).FiniteDimensional().extra_super_categories()
[]
>>> Modules(GF(Integer(5))).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
>>> Modules(ZZ).FiniteDimensional().is_subcategory(Sets().Finite())
False
```

(continues on next page)

(continued from previous page)

```
>>> Modules(Rings().Finite()).FiniteDimensional().is_subcategory(Sets().
↪Finite())
True
>>> Modules(Rings()).FiniteDimensional().is_subcategory(Sets().Finite())
False
```

Graded

alias of *GradedModules*

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

The category of homomorphism sets $\hom(X, Y)$ for X, Y modules.

class Endset(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of endomorphism sets $End(X)$ for X a module (this is not used yet)

extra_super_categories()

Implement the fact that the endomorphism set of a module is an algebra.

See also

extra_super_categories()

EXAMPLES:

```
sage: Modules(ZZ).Endsets().extra_super_categories()
[Category of magmatic algebras over Integer Ring]

sage: End(ZZ^3) in Algebras(ZZ)                                    #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> Modules(ZZ).Endsets().extra_super_categories()
[Category of magmatic algebras over Integer Ring]

>>> End(ZZ**Integer(3)) in Algebras(ZZ)                             ␣
↪     # needs sage.modules
True
```

class ParentMethods

Bases: `object`

base_ring()

Return the base ring of `self`.

EXAMPLES:

```
sage: # needs sage.modules
sage: E = CombinatorialFreeModule(ZZ, [1,2,3])
sage: F = CombinatorialFreeModule(ZZ, [2,3,4])
sage: H = Hom(E, F)
sage: H.base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> E = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2),Integer(3)])
>>> F = CombinatorialFreeModule(ZZ, [Integer(2),Integer(3),Integer(4)])
>>> H = Hom(E, F)
>>> H.base_ring()
Integer Ring
```

This `base_ring` method is actually overridden by `sage.structure.category_object.CategoryObject.base_ring()`:

```
sage: H.base_ring.__module__                                                #␣
↪needs sage.modules
'sage.structure.category_object'
```

```
>>> from sage.all import *
>>> H.base_ring.__module__                                                  #␣
↪needs sage.modules
'sage.structure.category_object'
```

Here we call it directly:

```
sage: method = H.category().parent_class.base_ring                          #␣
↪needs sage.modules
sage: method.__get__(H)()                                                   #␣
↪needs sage.modules
Integer Ring
```

```
>>> from sage.all import *
>>> method = H.category().parent_class.base_ring                            #␣
↪needs sage.modules
>>> method.__get__(H)()                                                     #␣
↪needs sage.modules
Integer Ring
```

zero()

EXAMPLES:

```
sage: # needs sage.modules
sage: E = CombinatorialFreeModule(ZZ, [1,2,3])
sage: F = CombinatorialFreeModule(ZZ, [2,3,4])
sage: H = Hom(E, F)
sage: f = H.zero()
sage: f
Generic morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Free module generated by {1, 2, 3} over Integer Ring
  To:   Free module generated by {2, 3, 4} over Integer Ring
sage: f(E.monomial(2))
0
sage: f(E.monomial(3)) == F.zero()
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> E = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2),Integer(3)])
>>> F = CombinatorialFreeModule(ZZ, [Integer(2),Integer(3),Integer(4)])
>>> H = Hom(E, F)
>>> f = H.zero()
>>> f
Generic morphism:
  From: Free module generated by {1, 2, 3} over Integer Ring
  To:   Free module generated by {2, 3, 4} over Integer Ring
>>> f(E.monomial(Integer(2)))
0
>>> f(E.monomial(Integer(3))) == F.zero()
True
```

base_ring()

EXAMPLES:

```
sage: Modules(ZZ).Homsets().base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> Modules(ZZ).Homsets().base_ring()
Integer Ring
```

Todo

Generalize this so that any homset category of a full subcategory of modules over a base ring is a category over this base ring.

extra_super_categories()

EXAMPLES:

```
sage: Modules(ZZ).Homsets().extra_super_categories()
[Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).Homsets().extra_super_categories()
[Category of modules over Integer Ring]
```

class ParentMethods

Bases: object

linear_combination(*iter_of_elements_coeff*, *factor_on_left=True*)

Return the linear combination $\lambda_1 v_1 + \cdots + \lambda_k v_k$ (resp. the linear combination $v_1\lambda_1 + \cdots + v_k\lambda_k$) where `iter_of_elements_coeff` iterates through the sequence $((\lambda_1, v_1), ..., (\lambda_k, v_k))$.

INPUT:

- `iter_of_elements_coeff` – iterator of pairs `(element, coeff)` with `element` in `self` and `coeff` in `self.base_ring()`
- `factor_on_left` – (optional) if `True`, the coefficients are multiplied from the left; if `False`, the coefficients are multiplied from the right

EXAMPLES:

```
sage: m = matrix([[0,1], [1,1]])                                              #␣
↪needs sage.modules
sage: J.<a,b,c> = JordanAlgebra(m)                                             #␣
↪needs sage.combinat sage.modules
sage: J.linear_combination(((a+b, 1), (-2*b + c, -1)))                        #␣
↪needs sage.combinat sage.modules
1 + (3, -1)
```

```
>>> from sage.all import *
>>> m = matrix([[Integer(0),Integer(1)], [Integer(1),Integer(1)]])       ␣
↪                                  # needs sage.modules
>>> J = JordanAlgebra(m, names=('a', 'b', 'c',)); (a, b, c,) = J._first_
↪ngens(3)# needs sage.combinat sage.modules
>>> J.linear_combination(((a+b, Integer(1)), (-Integer(2)*b + c, -
↪Integer(1))))                  # needs sage.combinat sage.modules
1 + (3, -1)
```

module_morphism(*function*, *category*, *codomain*, ***keywords*)

Construct a module morphism from `self` to `codomain`.

Let `self` be a module X over a ring R. This constructs a morphism $f : X \to Y$.

INPUT:

- `self` – a parent X in `Modules(R)`
- `function` – a function f from X to Y
- `codomain` – the codomain Y of the morphism (default: `f.codomain()` if it's defined; otherwise it must be specified)
- `category` – a category or `None` (default: `None`)

EXAMPLES:

```
sage: # needs sage.modules
sage: V = FiniteRankFreeModule(QQ, 2)
sage: e = V.basis('e'); e
Basis (e_0,e_1) on the
 2-dimensional vector space over the Rational Field
sage: neg = V.module_morphism(function=operator.neg, codomain=V); neg
Generic endomorphism of
 2-dimensional vector space over the Rational Field
sage: neg(e[0])
Element -e_0 of the 2-dimensional vector space over the Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> V = FiniteRankFreeModule(QQ, Integer(2))
>>> e = V.basis('e'); e
Basis (e_0,e_1) on the
 2-dimensional vector space over the Rational Field
>>> neg = V.module_morphism(function=operator.neg, codomain=V); neg
Generic endomorphism of
 2-dimensional vector space over the Rational Field
>>> neg(e[Integer(0)])
Element -e_0 of the 2-dimensional vector space over the Rational Field
```

quotient(*submodule*, *check=True*, ***kwds*)

Construct the quotient module `self` / `submodule`.

INPUT:

- `submodule` – a submodule with basis of `self`, or something that can be turned into one via `self.submodule(submodule)`
- `check`, other keyword arguments – passed on to `quotient_module`.

This method just delegates to `quotient_module`. Classes implementing modules should override that method.

Parents in categories with additional structure may override *quotient()*. For example, in algebras, *quotient()* will be the same as *quotient_ring()*.

EXAMPLES:

```
sage: C = CombinatorialFreeModule(QQ, ['a','b','c'])                    #␣
↪needs sage.modules
sage: TA = TensorAlgebra(C)                                             #␣
↪needs sage.combinat sage.modules
sage: TA.quotient                                                       #␣
↪needs sage.combinat sage.modules
<bound method Rings.ParentMethods.quotient of
 Tensor Algebra of Free module generated by {'a', 'b', 'c'}
 over Rational Field>
```

```
>>> from sage.all import *
>>> C = CombinatorialFreeModule(QQ, ['a','b','c'])                      #␣
↪needs sage.modules
>>> TA = TensorAlgebra(C)                                               #␣
↪needs sage.combinat sage.modules
>>> TA.quotient                                                         #␣
↪needs sage.combinat sage.modules
<bound method Rings.ParentMethods.quotient of
 Tensor Algebra of Free module generated by {'a', 'b', 'c'}
 over Rational Field>
```

tensor_square()

Return the tensor square of `self`.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()                           #␣
↪needs sage.groups sage.modules
sage: A.tensor_square()                                                 #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
 as a permutation group over Rational Field # An example
 of Hopf algebra with basis: the group algebra of the Dihedral
 group of order 6 as a permutation group over Rational Field
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()                    #␣
↪needs sage.groups sage.modules
>>> A.tensor_square()                                          #␣
↪needs sage.groups sage.modules
An example of Hopf algebra with basis:
 the group algebra of the Dihedral group of order 6
 as a permutation group over Rational Field # An example
 of Hopf algebra with basis: the group algebra of the Dihedral
 group of order 6 as a permutation group over Rational Field
```

class SubcategoryMethods

Bases: `object`

DualObjects()

Return the category of spaces constructed as duals of spaces of `self`.

The *dual* of a vector space V is the space consisting of all linear functionals on V (see Wikipedia article Dual_space). Additional structure on V can endow its dual with additional structure; for example, if V is a finite dimensional algebra, then its dual is a coalgebra.

This returns the category of spaces constructed as dual of spaces in `self`, endowed with the appropriate additional structure.

Warning

- This semantic of `dual` and `DualObject` is imposed on all subcategories, in particular to make `dual` a covariant functorial construction.

  A subcategory that defines a different notion of dual needs to use a different name.

- Typically, the category of graded modules should define a separate `graded_dual` construction (see Issue #15647). For now the two constructions are not distinguished which is an oversimplified model.

See also

- *dual.DualObjectsCategory*
- *CovariantFunctorialConstruction*.

EXAMPLES:

```
sage: VectorSpaces(QQ).DualObjects()
Category of duals of vector spaces over Rational Field
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).DualObjects()
Category of duals of vector spaces over Rational Field
```

The dual of a vector space is a vector space:

```
sage: VectorSpaces(QQ).DualObjects().super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).DualObjects().super_categories()
[Category of vector spaces over Rational Field]
```

The dual of an algebra is a coalgebra:

```
sage: sorted(Algebras(QQ).DualObjects().super_categories(), key=str)
[Category of coalgebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> sorted(Algebras(QQ).DualObjects().super_categories(), key=str)
[Category of coalgebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

The dual of a coalgebra is an algebra:

```
sage: sorted(Coalgebras(QQ).DualObjects().super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> sorted(Coalgebras(QQ).DualObjects().super_categories(), key=str)
[Category of algebras over Rational Field,
 Category of duals of vector spaces over Rational Field]
```

As a shorthand, this category can be accessed with the *dual()* method:

```
sage: VectorSpaces(QQ).dual()
Category of duals of vector spaces over Rational Field
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).dual()
Category of duals of vector spaces over Rational Field
```

Filtered(*base_ring=None*)

Return the subcategory of the filtered objects of `self`.

INPUT:

- `base_ring` – this is ignored

EXAMPLES:

```
sage: Modules(ZZ).Filtered()
Category of filtered modules over Integer Ring

sage: Coalgebras(QQ).Filtered()
Category of filtered coalgebras over Rational Field

sage: AlgebrasWithBasis(QQ).Filtered()
Category of filtered algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).Filtered()
Category of filtered modules over Integer Ring

>>> Coalgebras(QQ).Filtered()
Category of filtered coalgebras over Rational Field

>>> AlgebrasWithBasis(QQ).Filtered()
Category of filtered algebras with basis over Rational Field
```

Todo

- Explain why this does not commute with *WithBasis()*
- Improve the support for covariant functorial constructions categories over a base ring so as to get rid of the `base_ring` argument.

FiniteDimensional()

Return the full subcategory of the finite dimensional objects of `self`.

EXAMPLES:

```
sage: Modules(ZZ).FiniteDimensional()
Category of finite dimensional modules over Integer Ring
sage: Coalgebras(QQ).FiniteDimensional()
Category of finite dimensional coalgebras over Rational Field
sage: AlgebrasWithBasis(QQ).FiniteDimensional()
Category of finite dimensional algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).FiniteDimensional()
Category of finite dimensional modules over Integer Ring
>>> Coalgebras(QQ).FiniteDimensional()
Category of finite dimensional coalgebras over Rational Field
>>> AlgebrasWithBasis(QQ).FiniteDimensional()
Category of finite dimensional algebras with basis over Rational Field
```

FinitelyPresented()

Return the full subcategory of the finitely presented objects of `self`.

EXAMPLES:

```
sage: Modules(ZZ).FinitelyPresented()
Category of finitely presented modules over Integer Ring
sage: A = SteenrodAlgebra(2)                                                #␣
↪needs sage.combinat sage.modules
sage: from sage.modules.fp_graded.module import FPModule                    #␣
↪needs sage.combinat sage.modules
sage: FPModule(A, [0, 1], [[Sq(2), Sq(1)]]).category()                      #␣
↪needs sage.combinat sage.modules
Category of finitely presented graded modules
 over mod 2 Steenrod algebra, milnor basis
```

```
>>> from sage.all import *
>>> Modules(ZZ).FinitelyPresented()
Category of finitely presented modules over Integer Ring
>>> A = SteenrodAlgebra(Integer(2))                                          ␣
↪     # needs sage.combinat sage.modules
>>> from sage.modules.fp_graded.module import FPModule                      #␣
↪needs sage.combinat sage.modules
>>> FPModule(A, [Integer(0), Integer(1)], [[Sq(Integer(2)),␣
↪Sq(Integer(1))]]).category()                    # needs sage.combinat␣
↪sage.modules
Category of finitely presented graded modules
 over mod 2 Steenrod algebra, milnor basis
```

Graded(*base_ring=None*)

Return the subcategory of the graded objects of `self`.

INPUT:

- `base_ring` – this is ignored

EXAMPLES:

```
sage: Modules(ZZ).Graded()
Category of graded modules over Integer Ring

sage: Coalgebras(QQ).Graded()
Category of graded coalgebras over Rational Field

sage: AlgebrasWithBasis(QQ).Graded()
Category of graded algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).Graded()
Category of graded modules over Integer Ring

>>> Coalgebras(QQ).Graded()
Category of graded coalgebras over Rational Field

>>> AlgebrasWithBasis(QQ).Graded()
Category of graded algebras with basis over Rational Field
```

Todo

- Explain why this does not commute with *WithBasis()*
- Improve the support for covariant functorial constructions categories over a base ring so as to get rid of the `base_ring` argument.

Super(*base_ring=None*)

Return the super-analogue category of `self`.

INPUT:

- `base_ring` – this is ignored

EXAMPLES:

```
sage: Modules(ZZ).Super()
Category of super modules over Integer Ring

sage: Coalgebras(QQ).Super()
Category of super coalgebras over Rational Field

sage: AlgebrasWithBasis(QQ).Super()
Category of super algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).Super()
Category of super modules over Integer Ring

>>> Coalgebras(QQ).Super()
Category of super coalgebras over Rational Field

>>> AlgebrasWithBasis(QQ).Super()
Category of super algebras with basis over Rational Field
```

Todo

- Explain why this does not commute with *WithBasis()*
- Improve the support for covariant functorial constructions categories over a base ring so as to get rid of the `base_ring` argument.

TensorProducts()

Return the full subcategory of objects of `self` constructed as tensor products.

See also

- *tensor.TensorProductsCategory*
- *RegressiveCovariantConstructionCategory*.

EXAMPLES:

```
sage: ModulesWithBasis(QQ).TensorProducts()
Category of tensor products of vector spaces with basis over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).TensorProducts()
Category of tensor products of vector spaces with basis over Rational␣
↪Field
```

WithBasis()

Return the full subcategory of the objects of `self` with a distinguished basis.

EXAMPLES:

```
sage: Modules(ZZ).WithBasis()
Category of modules with basis over Integer Ring
sage: Coalgebras(QQ).WithBasis()
Category of coalgebras with basis over Rational Field
sage: AlgebrasWithBasis(QQ).WithBasis()
Category of algebras with basis over Rational Field
```

```
>>> from sage.all import *
>>> Modules(ZZ).WithBasis()
Category of modules with basis over Integer Ring
>>> Coalgebras(QQ).WithBasis()
Category of coalgebras with basis over Rational Field
>>> AlgebrasWithBasis(QQ).WithBasis()
Category of algebras with basis over Rational Field
```

base_ring()

Return the base ring (category) for `self`.

This implements a `base_ring` method for all subcategories of `Modules(K)`.

EXAMPLES:

```
sage: C = Modules(QQ) & Semigroups(); C
Join of Category of semigroups
    and Category of vector spaces over Rational Field
sage: C.base_ring()
Rational Field
sage: C.base_ring.__module__
'sage.categories.modules'

sage: C2 = Modules(Rings()) & Semigroups(); C2
Join of Category of semigroups and Category of modules over rings
sage: C2.base_ring()
Category of rings
sage: C2.base_ring.__module__
'sage.categories.modules'

sage: # needs sage.combinat sage.groups sage.modules
sage: C3 = DescentAlgebra(QQ,3).B().category()
sage: C3.base_ring.__module__
```

(continues on next page)

(continued from previous page)

```
'sage.categories.modules'
sage: C3.base_ring()
Rational Field

sage: # needs sage.combinat sage.modules
sage: C4 = QuasiSymmetricFunctions(QQ).F().category()
sage: C4.base_ring.__module__
'sage.categories.modules'
sage: C4.base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> C = Modules(QQ) & Semigroups(); C
Join of Category of semigroups
    and Category of vector spaces over Rational Field
>>> C.base_ring()
Rational Field
>>> C.base_ring.__module__
'sage.categories.modules'

>>> C2 = Modules(Rings()) & Semigroups(); C2
Join of Category of semigroups and Category of modules over rings
>>> C2.base_ring()
Category of rings
>>> C2.base_ring.__module__
'sage.categories.modules'

>>> # needs sage.combinat sage.groups sage.modules
>>> C3 = DescentAlgebra(QQ,Integer(3)).B().category()
>>> C3.base_ring.__module__
'sage.categories.modules'
>>> C3.base_ring()
Rational Field

>>> # needs sage.combinat sage.modules
>>> C4 = QuasiSymmetricFunctions(QQ).F().category()
>>> C4.base_ring.__module__
'sage.categories.modules'
>>> C4.base_ring()
Rational Field
```

dual()

alias of *DualObjects()*.

Super

alias of *SuperModules*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of modules constructed by tensor product of modules.

class ParentMethods

Bases: object

Implement operations on tensor products of modules.

construction()

Return the construction of `self`.

EXAMPLES:

```
sage: A = algebras.Free(QQ, 2)                                          #␣
↪needs sage.combinat sage.modules
sage: T = A.tensor(A)                                                   #␣
↪needs sage.combinat sage.modules
sage: T.construction()                                                  #␣
↪needs sage.combinat sage.modules
(The tensor functorial construction,
 (Free Algebra on 2 generators (None0, None1) over Rational Field,
  Free Algebra on 2 generators (None0, None1) over Rational Field))
```

```
>>> from sage.all import *
>>> A = algebras.Free(QQ, Integer(2))                                       ␣
↪    # needs sage.combinat sage.modules
>>> T = A.tensor(A)                                                      #␣
↪needs sage.combinat sage.modules
>>> T.construction()                                                     #␣
↪needs sage.combinat sage.modules
(The tensor functorial construction,
 (Free Algebra on 2 generators (None0, None1) over Rational Field,
  Free Algebra on 2 generators (None0, None1) over Rational Field))
```

tensor_factors()

Return the tensor factors of this tensor product.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(ZZ, [1,2])
sage: F.rename('F')
sage: G = CombinatorialFreeModule(ZZ, [3,4])
sage: G.rename('G')
sage: T = tensor([F, G]); T
F # G
sage: T.tensor_factors()
(F, G)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2)])
>>> F.rename('F')
>>> G = CombinatorialFreeModule(ZZ, [Integer(3),Integer(4)])
>>> G.rename('G')
>>> T = tensor([F, G]); T
F # G
>>> T.tensor_factors()
(F, G)
```

extra_super_categories()

EXAMPLES:

```
sage: Modules(ZZ).TensorProducts().extra_super_categories()
[Category of modules over Integer Ring]
sage: Modules(ZZ).TensorProducts().super_categories()
[Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).TensorProducts().extra_super_categories()
[Category of modules over Integer Ring]
>>> Modules(ZZ).TensorProducts().super_categories()
[Category of modules over Integer Ring]
```

WithBasis

alias of *ModulesWithBasis*

additional_structure()

Return None.

Indeed, the category of modules defines no additional structure: a bimodule morphism between two modules is a module morphism.

See also

Category.additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: Modules(ZZ).additional_structure()
```

```
>>> from sage.all import *
>>> Modules(ZZ).additional_structure()
```

super_categories()

EXAMPLES:

```
sage: Modules(ZZ).super_categories()
[Category of bimodules over Integer Ring on the left
                        and Integer Ring on the right]
```

```
>>> from sage.all import *
>>> Modules(ZZ).super_categories()
[Category of bimodules over Integer Ring on the left
                        and Integer Ring on the right]
```

Nota bene:

```
sage: Modules(QQ)
Category of vector spaces over Rational Field
sage: Modules(QQ).super_categories()
[Category of modules over Rational Field]
```

```
>>> from sage.all import *
>>> Modules(QQ)
Category of vector spaces over Rational Field
>>> Modules(QQ).super_categories()
[Category of modules over Rational Field]
```

4.125 Modules With Basis

AUTHORS:

- Nicolas M. Thiery (2008-2014): initial revision, axiomatization
- Jason Bandlow and Florent Hivert (2010): Triangular Morphisms
- Christian Stump (2010): Issue #9648 module_morphism's to a wider class of codomains

class sage.categories.modules_with_basis.**ModulesWithBasis**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of modules with a distinguished basis.

The elements are represented by expanding them in the distinguished basis. The morphisms are not required to respect the distinguished basis.

EXAMPLES:

```
sage: ModulesWithBasis(ZZ)
Category of modules with basis over Integer Ring
sage: ModulesWithBasis(ZZ).super_categories()
[Category of modules over Integer Ring]
```

```
>>> from sage.all import *
>>> ModulesWithBasis(ZZ)
Category of modules with basis over Integer Ring
>>> ModulesWithBasis(ZZ).super_categories()
[Category of modules over Integer Ring]
```

If the base ring is actually a field, this constructs instead the category of vector spaces with basis:

```
sage: ModulesWithBasis(QQ)
Category of vector spaces with basis over Rational Field

sage: ModulesWithBasis(QQ).super_categories()
[Category of modules with basis over Rational Field,
 Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ)
Category of vector spaces with basis over Rational Field
```

(continues on next page)

(continued from previous page)

```
>>> ModulesWithBasis(QQ).super_categories()
[Category of modules with basis over Rational Field,
 Category of vector spaces over Rational Field]
```

Let X and Y be two modules with basis. We can build $Hom(X, Y)$:

```
sage: X = CombinatorialFreeModule(QQ, [1,2]); X.rename('X')                    #␣
↪needs sage.modules
sage: Y = CombinatorialFreeModule(QQ, [3,4]); Y.rename('Y')                    #␣
↪needs sage.modules
sage: H = Hom(X, Y); H                                                         #␣
↪needs sage.modules
Set of Morphisms from X to Y
 in Category of finite dimensional vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> X = CombinatorialFreeModule(QQ, [Integer(1),Integer(2)]); X.rename('X')      ␣
↪              # needs sage.modules
>>> Y = CombinatorialFreeModule(QQ, [Integer(3),Integer(4)]); Y.rename('Y')      ␣
↪              # needs sage.modules
>>> H = Hom(X, Y); H                                                          #␣
↪needs sage.modules
Set of Morphisms from X to Y
 in Category of finite dimensional vector spaces with basis over Rational Field
```

The simplest morphism is the zero map:

```
sage: H.zero()      # todo: move this test into module once we have an example   #␣
↪needs sage.modules
Generic morphism:
  From: X
  To:   Y
```

```
>>> from sage.all import *
>>> H.zero()      # todo: move this test into module once we have an example   #␣
↪needs sage.modules
Generic morphism:
  From: X
  To:   Y
```

which we can apply to elements of X:

```
sage: x = X.monomial(1) + 3 * X.monomial(2)                                    #␣
↪needs sage.modules
sage: H.zero()(x)                                                              #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> x = X.monomial(Integer(1)) + Integer(3) * X.monomial(Integer(2))            ␣
↪                      # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> H.zero()(x)                                                                #␣
↪needs sage.modules
0
```

EXAMPLES:

We now construct a more interesting morphism by extending a function by linearity:

```
sage: phi = H(on_basis=lambda i: Y.monomial(i + 2)); phi                       #␣
↪needs sage.modules
Generic morphism:
  From: X
  To:   Y
sage: phi(x)                                                                   #␣
↪needs sage.modules
B[3] + 3*B[4]
```

```
>>> from sage.all import *
>>> phi = H(on_basis=lambda i: Y.monomial(i + Integer(2))); phi                 ␣
↪     # needs sage.modules
Generic morphism:
  From: X
  To:   Y
>>> phi(x)                                                                     #␣
↪needs sage.modules
B[3] + 3*B[4]
```

We can retrieve the function acting on indices of the basis:

```
sage: f = phi.on_basis()                                                       #␣
↪needs sage.modules
sage: f(1), f(2)                                                               #␣
↪needs sage.modules
(B[3], B[4])
```

```
>>> from sage.all import *
>>> f = phi.on_basis()                                                         #␣
↪needs sage.modules
>>> f(Integer(1)), f(Integer(2))                                               ␣
↪              # needs sage.modules
(B[3], B[4])
```

$Hom(X, Y)$ has a natural module structure (except for the zero, the operations are not yet implemented though). However since the dimension is not necessarily finite, it is not a module with basis; but see *FiniteDimensionalModulesWithBasis* and *GradedModulesWithBasis*:

```
sage: H in ModulesWithBasis(QQ), H in Modules(QQ)                              #␣
↪needs sage.modules
(False, True)
```

```
>>> from sage.all import *
>>> H in ModulesWithBasis(QQ), H in Modules(QQ)                                #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
(False, True)
```

Some more playing around with categories and higher order homsets:

```
sage: H.category()                                                          #␣
↪needs sage.modules
Category of homsets of finite dimensional modules with basis over Rational Field
sage: Hom(H, H).category()                                                  #␣
↪needs sage.modules
Category of endsets of
 homsets of finite dimensional modules with basis over Rational Field
```

```
>>> from sage.all import *
>>> H.category()                                                            #␣
↪needs sage.modules
Category of homsets of finite dimensional modules with basis over Rational Field
>>> Hom(H, H).category()                                                    #␣
↪needs sage.modules
Category of endsets of
 homsets of finite dimensional modules with basis over Rational Field
```

Todo

`End(X)` is an algebra.

Note

This category currently requires an implementation of an element method `support`. Once Issue #18066 is merged, an implementation of an `items` method will be required.

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of modules with basis constructed by Cartesian products of modules with basis.

class ParentMethods

Bases: `object`

extra_super_categories()

EXAMPLES:

```
sage: ModulesWithBasis(QQ).CartesianProducts().extra_super_categories()
[Category of vector spaces with basis over Rational Field]
sage: ModulesWithBasis(QQ).CartesianProducts().super_categories()
[Category of Cartesian products of modules with basis over Rational Field,
 Category of vector spaces with basis over Rational Field,
 Category of Cartesian products of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).CartesianProducts().extra_super_categories()
[Category of vector spaces with basis over Rational Field]
>>> ModulesWithBasis(QQ).CartesianProducts().super_categories()
[Category of Cartesian products of modules with basis over Rational Field,
 Category of vector spaces with basis over Rational Field,
 Category of Cartesian products of vector spaces over Rational Field]
```

class DualObjects(*category*, **args*)

Bases: *DualObjectsCategory*

extra_super_categories()

EXAMPLES:

```
sage: ModulesWithBasis(ZZ).DualObjects().extra_super_categories()
[Category of modules over Integer Ring]
sage: ModulesWithBasis(QQ).DualObjects().super_categories()
[Category of duals of vector spaces over Rational Field,
 Category of duals of modules with basis over Rational Field]
```

```
>>> from sage.all import *
>>> ModulesWithBasis(ZZ).DualObjects().extra_super_categories()
[Category of modules over Integer Ring]
>>> ModulesWithBasis(QQ).DualObjects().super_categories()
[Category of duals of vector spaces over Rational Field,
 Category of duals of modules with basis over Rational Field]
```

class ElementMethods

Bases: object

change_ring(*R*)

Return this element with its coefficients converted to R.

INPUT:

- R – the new base ring

EXAMPLES:

```
sage: F = CombinatorialFreeModule(QQ, ['a', 'b'])
sage: B = F.basis()
sage: x = B['a'] + 2*B['b']
sage: y = x.change_ring(ZZ)
sage: y
B['a'] + 2*B['b']
sage: y.parent().base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a', 'b'])
>>> B = F.basis()
>>> x = B['a'] + Integer(2)*B['b']
>>> y = x.change_ring(ZZ)
>>> y
B['a'] + 2*B['b']
```

(continues on next page)

(continued from previous page)

```
>>> y.parent().base_ring()
Integer Ring
```

If the base ring is unchanged, return the same element:

```
sage: x.change_ring(QQ) is x
True
```

```
>>> from sage.all import *
>>> x.change_ring(QQ) is x
True
```

Coefficients that become zero are removed:

```
sage: (7*B['a'] + B['b']).change_ring(GF(7))                                #␣
↪needs sage.rings.finite_rings
B['b']
```

```
>>> from sage.all import *
>>> (Integer(7)*B['a'] + B['b']).change_ring(GF(Integer(7)))                ␣
↪              # needs sage.rings.finite_rings
B['b']
```

The coefficients must be convertible to the new base ring:

```
sage: (1/2*B['a']).change_ring(ZZ)
Traceback (most recent call last):
...
TypeError: no conversion of this rational to integer
```

```
>>> from sage.all import *
>>> (Integer(1)/Integer(2)*B['a']).change_ring(ZZ)
Traceback (most recent call last):
...
TypeError: no conversion of this rational to integer
```

coefficient(*m*)

Return the coefficient of `m` in `self` and raise an error if `m` is not in the basis indexing set.

INPUT:

- `m` – a basis index of the parent of `self`

OUTPUT:

The `B[m]`-coordinate of `self` with respect to the basis `B`. Here, `B` denotes the given basis of the parent of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: s = CombinatorialFreeModule(QQ, Partitions())
sage: z = s([4]) - 2*s([2,1]) + s([1,1,1]) + s([1])
sage: z.coefficient([4])
1
```

(continues on next page)

(continued from previous page)

```
sage: z.coefficient([2,1])
-2
sage: z.coefficient(Partition([2,1]))
-2
sage: z.coefficient([1,2])
Traceback (most recent call last):
...
AssertionError: [1, 2] should be an element of Partitions
sage: z.coefficient(Composition([2,1]))
Traceback (most recent call last):
...
AssertionError: [2, 1] should be an element of Partitions
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> s = CombinatorialFreeModule(QQ, Partitions())
>>> z = s([Integer(4)]) - Integer(2)*s([Integer(2),Integer(1)]) +␣
↪s([Integer(1),Integer(1),Integer(1)]) + s([Integer(1)])
>>> z.coefficient([Integer(4)])
1
>>> z.coefficient([Integer(2),Integer(1)])
-2
>>> z.coefficient(Partition([Integer(2),Integer(1)]))
-2
>>> z.coefficient([Integer(1),Integer(2)])
Traceback (most recent call last):
...
AssertionError: [1, 2] should be an element of Partitions
>>> z.coefficient(Composition([Integer(2),Integer(1)]))
Traceback (most recent call last):
...
AssertionError: [2, 1] should be an element of Partitions
```

Test that `coefficient` also works for those parents that do not have an `element_class`:

```
sage: # needs sage.modules sage.rings.padics
sage: H = pAdicWeightSpace(3)
sage: F = CombinatorialFreeModule(QQ, H)
sage: hasattr(H, "element_class")
False
sage: h = H.an_element()
sage: (2*F.monomial(h)).coefficient(h)
2
```

```
>>> from sage.all import *
>>> # needs sage.modules sage.rings.padics
>>> H = pAdicWeightSpace(Integer(3))
>>> F = CombinatorialFreeModule(QQ, H)
>>> hasattr(H, "element_class")
False
>>> h = H.an_element()
>>> (Integer(2)*F.monomial(h)).coefficient(h)
```

(continues on next page)

(continued from previous page)

```
2
```

coefficients(*sort=True*)

Return a list of the (nonzero) coefficients appearing on the basis elements in `self` (in an arbitrary order).

INPUT:

- `sort` – boolean (default: `True`); to sort the coefficients based upon the default ordering of the indexing set

See also

dense_coefficient_list()

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] - 3*B['c']
sage: f.coefficients()
[1, -3]
sage: f = B['c'] - 3*B['a']
sage: f.coefficients()
[-3, 1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] - Integer(3)*B['c']
>>> f.coefficients()
[1, -3]
>>> f = B['c'] - Integer(3)*B['a']
>>> f.coefficients()
[-3, 1]
```

```
sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: z = s([4]) + s([2,1]) + s([1,1,1]) + s([1])                           #␣
↪needs sage.combinat sage.modules
sage: z.coefficients()                                                      #␣
↪needs sage.combinat sage.modules
[1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> z = s([Integer(4)]) + s([Integer(2),Integer(1)]) + s([Integer(1),
↪Integer(1),Integer(1)]) + s([Integer(1)])                                  # needs␣
↪sage.combinat sage.modules
>>> z.coefficients()                                                        #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
[1, 1, 1, 1]
```

is_zero()

Return `True` if and only if `self == 0`.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] - 3*B['c']
sage: f.is_zero()
False
sage: F.zero().is_zero()
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] - Integer(3)*B['c']
>>> f.is_zero()
False
>>> F.zero().is_zero()
True
```

```
sage: # needs sage.combinat sage.modules
sage: s = SymmetricFunctions(QQ).schur()
sage: s([2,1]).is_zero()
False
sage: s(0).is_zero()
True
sage: (s([2,1]) - s([2,1])).is_zero()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> s = SymmetricFunctions(QQ).schur()
>>> s([Integer(2),Integer(1)]).is_zero()
False
>>> s(Integer(0)).is_zero()
True
>>> (s([Integer(2),Integer(1)]) - s([Integer(2),Integer(1)])).is_zero()
True
```

items()

Return a list of pairs `(i, c)`, where `c` is the `i`-th coefficient of `i` in the standard basis.

EXAMPLES:

```
sage: # needs sage.algebras
sage: B = FiniteDimensionalAlgebra(QQ, [Matrix([[1,0], [0,1]]),
....:                                   Matrix([[0,1], [-1,0]])])
sage: elt = B(Matrix([[1,2], [-2,1]]))
sage: elt.items()
dict_items([(0, 1), (1, 2)])
```

```
>>> from sage.all import *
>>> # needs sage.algebras
>>> B = FiniteDimensionalAlgebra(QQ, [Matrix([[Integer(1),Integer(0)],␣
↪[Integer(0),Integer(1)]]),
...                                   Matrix([[Integer(0),Integer(1)], [-
↪Integer(1),Integer(0)]])])
>>> elt = B(Matrix([[Integer(1),Integer(2)], [-Integer(2),Integer(1)]]))
>>> elt.items()
dict_items([(0, 1), (1, 2)])
```

```
sage: # needs sage.combinat sage.modules
sage: h = SymmetricFunctions(QQ).h()
sage: (h[2]+3*h[3]).items()
dict_items([([2], 1), ([3], 3)])
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> h = SymmetricFunctions(QQ).h()
>>> (h[Integer(2)]+Integer(3)*h[Integer(3)]).items()
dict_items([([2], 1), ([3], 3)])
```

leading_coefficient(**args*, ***kwds*)

Return the leading coefficient of `self`.

This is the coefficient of the term whose corresponding basis element is maximal. Note that this may not be the term which actually appears first when `self` is printed.

If the default term ordering is not what is desired, a comparison key, `key(x,y)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.leading_coefficient()
1
sage: def key(x): return -x
sage: x.leading_coefficient(key=key)
3

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.leading_coefficient()                                               #␣
↪needs sage.combinat sage.modules
-5
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
>>> x.leading_coefficient()
1
>>> def key(x): return -x
>>> x.leading_coefficient(key=key)
3

>>> s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.leading_coefficient()                                               #␣
↪needs sage.combinat sage.modules
-5
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_coefficient()
1
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_coefficient()
3
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_coefficient()
2
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_coefficient()
1
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_coefficient()
3
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_coefficient()
2
```

leading_item(**args*, ***kwds*)

Return the pair `(k, c)` where

$$c \cdot (\text{the basis element indexed by } k)$$

is the leading term of `self`.

Here 'leading term' means that the corresponding basis element is maximal. Note that this may not be the term which actually appears first when `self` is printed.

If the default term ordering is not what is desired, a comparison function, `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + 4*X.monomial(3)
sage: x.leading_item()
(3, 4)
sage: def key(x): return -x
sage: x.leading_item(key=key)
(1, 3)

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.leading_item()                                                      #␣
↪needs sage.combinat sage.modules
([3], -5)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + Integer(4)*X.monomial(Integer(3))
>>> x.leading_item()
(3, 4)
>>> def key(x): return -x
>>> x.leading_item(key=key)
(1, 3)

>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.leading_item()                                                        #␣
↪needs sage.combinat sage.modules
([3], -5)
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_item()
((0, 4, 0), 1)
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_item()
((1, 2, 0), 3)
```

(continues on next page)

(continued from previous page)

```
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_item()
((0, 1, 3), 2)
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_item()
((0, 4, 0), 1)
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_item()
((1, 2, 0), 3)
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_item()
((0, 1, 3), 2)
```

leading_monomial(**args*, ***kwds*)

Return the leading monomial of `self`.

This is the monomial whose corresponding basis element is maximal. Note that this may not be the term which actually appears first when `self` is printed.

If the default term ordering is not what is desired, a comparison key, `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.leading_monomial()
B[3]
sage: def key(x): return -x
sage: x.leading_monomial(key=key)
B[1]

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.leading_monomial()                                                  #␣
↪needs sage.combinat sage.modules
s[3]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> x.leading_monomial()
B[3]
>>> def key(x): return -x
>>> x.leading_monomial(key=key)
B[1]

>>> s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                        # needs␣
↪sage.combinat sage.modules
>>> f.leading_monomial()                                                  #␣
↪needs sage.combinat sage.modules
s[3]
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_monomial()
y^4
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_monomial()
x*y^2
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_monomial()
y*z^3
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_monomial()
y^4
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_monomial()
x*y^2
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_monomial()
y*z^3
```

leading_support(**args*, ***kwds*)

Return the maximal element of the support of `self`.

Note that this may not be the term which actually appears first when `self` is printed.

If the default ordering of the basis elements is not what is desired, a comparison key, `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3])
sage: X.rename('X'); x = X.basis()
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + 4*X.monomial(3)
sage: x.leading_support()
3
sage: def key(x): return -x
sage: x.leading_support(key=key)
1

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.leading_support()                                                   #␣
↪needs sage.combinat sage.modules
[3]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)])
>>> X.rename('X'); x = X.basis()
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + Integer(4)*X.monomial(Integer(3))
>>> x.leading_support()
3
>>> def key(x): return -x
>>> x.leading_support(key=key)
1

>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.leading_support()                                                     #␣
↪needs sage.combinat sage.modules
[3]
```

leading_term(**args*, ***kwds*)

Return the leading term of `self`.

This is the term whose corresponding basis element is maximal. Note that this may not be the term which actually appears first when `self` is printed.

If the default term ordering is not what is desired, a comparison key, `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.leading_term()
B[3]
```

(continues on next page)

(continued from previous page)

```
sage: def key(x): return -x
sage: x.leading_term(key=key)
3*B[1]

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.leading_term()                                                      #␣
↪needs sage.combinat sage.modules
-5*s[3]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
>>> x.leading_term()
B[3]
>>> def key(x): return -x
>>> x.leading_term(key=key)
3*B[1]

>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.leading_term()                                                        #␣
↪needs sage.combinat sage.modules
-5*s[3]
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_term()
y^4
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_term()
3*x*y^2
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).leading_term()
2*y*z^3
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_term()
y^4
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_term()
3*x*y^2
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).leading_term()
2*y*z^3
```

length()

Return the number of basis elements whose coefficients in `self` are nonzero.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] - 3*B['c']
sage: f.length()
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] - Integer(3)*B['c']
>>> f.length()
2
```

```
sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: z = s([4]) + s([2,1]) + s([1,1,1]) + s([1])                           #␣
↪needs sage.combinat sage.modules
sage: z.length()                                                            #␣
↪needs sage.combinat sage.modules
4
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> z = s([Integer(4)]) + s([Integer(2),Integer(1)]) + s([Integer(1),
↪Integer(1),Integer(1)]) + s([Integer(1)])                                  # needs␣
↪sage.combinat sage.modules
>>> z.length()                                                              #␣
↪needs sage.combinat sage.modules
4
```

map_coefficients(*f*, *new_base_ring=None*)

Return the element obtained by applying `f` to the nonzero coefficients of `self`.

If `f` is a *sage.categories.map.Map*, then the resulting polynomial will be defined over the codomain of `f`. Otherwise, the resulting polynomial will be over the same ring as `self`. Set `new_base_ring` to override this behaviour.

An error is raised if the coefficients cannot be converted to the new base ring.

INPUT:

- `f` – a callable that will be applied to the coefficients of `self`
- `new_base_ring` – (optional) if given, the resulting element will be defined over this ring

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] - 3*B['c']
sage: f.map_coefficients(lambda x: x + 5)
6*B['a'] + 2*B['c']
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] - Integer(3)*B['c']
>>> f.map_coefficients(lambda x: x + Integer(5))
6*B['a'] + 2*B['c']
```

Killed coefficients are handled properly:

```
sage: f.map_coefficients(lambda x: 0)                                       #␣
↪needs sage.modules
0
sage: list(f.map_coefficients(lambda x: 0))                                 #␣
↪needs sage.modules
[]
```

```
>>> from sage.all import *
>>> f.map_coefficients(lambda x: Integer(0))                                ␣
↪     # needs sage.modules
0
>>> list(f.map_coefficients(lambda x: Integer(0)))                          ␣
↪     # needs sage.modules
[]
```

```
sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: a = s([2,1]) + 2*s([3,2])                                             #␣
↪needs sage.combinat sage.modules
sage: a.map_coefficients(lambda x: x * 2)                                   #␣
↪needs sage.combinat sage.modules
2*s[2, 1] + 4*s[3, 2]
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> a = s([Integer(2),Integer(1)]) + Integer(2)*s([Integer(3),
↪Integer(2)])                                                 # needs sage.
↪combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> a.map_coefficients(lambda x: x * Integer(2))                              ␣
↪     # needs sage.combinat sage.modules
2*s[2, 1] + 4*s[3, 2]
```

We can map into a different base ring:

```
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: a = 1/2*(B['a'] + 3*B['c']); a
1/2*B['a'] + 3/2*B['c']
sage: b = a.map_coefficients(lambda c: 2*c, ZZ); b
B['a'] + 3*B['c']
sage: b.parent()
Free module generated by {'a', 'b', 'c'} over Integer Ring
sage: b.map_coefficients(lambda c: 1/2*c, ZZ)
Traceback (most recent call last):
...
TypeError: no conversion of this rational to integer
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> a = Integer(1)/Integer(2)*(B['a'] + Integer(3)*B['c']); a
1/2*B['a'] + 3/2*B['c']
>>> b = a.map_coefficients(lambda c: Integer(2)*c, ZZ); b
B['a'] + 3*B['c']
>>> b.parent()
Free module generated by {'a', 'b', 'c'} over Integer Ring
>>> b.map_coefficients(lambda c: Integer(1)/Integer(2)*c, ZZ)
Traceback (most recent call last):
...
TypeError: no conversion of this rational to integer
```

Coefficients are converted to the new base ring after applying the map:

```
sage: B['a'].map_coefficients(lambda c: 2*c, GF(2))
0
sage: B['a'].map_coefficients(lambda c: GF(2)(c), QQ)
B['a']
```

```
>>> from sage.all import *
>>> B['a'].map_coefficients(lambda c: Integer(2)*c, GF(Integer(2)))
0
>>> B['a'].map_coefficients(lambda c: GF(Integer(2))(c), QQ)
B['a']
```

map_item(*f*)

Mapping a function on items.

INPUT:

- `f` – a function mapping pairs `(index, coeff)` to other such pairs

Return a new element of `self.parent()` obtained by applying the function f to all items `(index, coeff)` of `self`.

EXAMPLES:

```
sage: B = CombinatorialFreeModule(ZZ, [-1, 0, 1])                              #␣
↪needs sage.modules
sage: x = B.an_element(); x                                                    #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
sage: x.map_item(lambda i, c: (-i, 2*c))                                       #␣
↪needs sage.modules
6*B[-1] + 4*B[0] + 4*B[1]
```

```
>>> from sage.all import *
>>> B = CombinatorialFreeModule(ZZ, [-Integer(1), Integer(0),␣
↪Integer(1)])                          # needs sage.modules
>>> x = B.an_element(); x                                                      #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
>>> x.map_item(lambda i, c: (-i, Integer(2)*c))                                ␣
↪     # needs sage.modules
6*B[-1] + 4*B[0] + 4*B[1]
```

`f` needs not be injective:

```
sage: x.map_item(lambda i, c: (1, 2*c))                                        #␣
↪needs sage.modules
14*B[1]

sage: s = SymmetricFunctions(QQ).schur()                                       #␣
↪needs sage.combinat sage.modules
sage: f = lambda m, c: (m.conjugate(), 2 * c)
sage: a = s([2,1]) + s([1,1,1])                                                #␣
↪needs sage.combinat sage.modules
sage: a.map_item(f)                                                            #␣
↪needs sage.combinat sage.modules
2*s[2, 1] + 2*s[3]
```

```
>>> from sage.all import *
>>> x.map_item(lambda i, c: (Integer(1), Integer(2)*c))                        ␣
↪               # needs sage.modules
14*B[1]

>>> s = SymmetricFunctions(QQ).schur()                                         #␣
↪needs sage.combinat sage.modules
>>> f = lambda m, c: (m.conjugate(), Integer(2) * c)
>>> a = s([Integer(2),Integer(1)]) + s([Integer(1),Integer(1),
↪Integer(1)])                                          # needs sage.
↪combinat sage.modules
>>> a.map_item(f)                                                              #␣
↪needs sage.combinat sage.modules
2*s[2, 1] + 2*s[3]
```

map_support(*f*)

Mapping a function on the support.

INPUT:

- `f` – an endofunction on the indices of the free module

Return a new element of `self.parent()` obtained by applying the function `f` to all of the objects indexing the basis elements.

EXAMPLES:

```
sage: B = CombinatorialFreeModule(ZZ, [-1, 0, 1])                              #␣
↪needs sage.modules
sage: x = B.an_element(); x                                                    #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
sage: x.map_support(lambda i: -i)                                              #␣
↪needs sage.modules
3*B[-1] + 2*B[0] + 2*B[1]
```

```
>>> from sage.all import *
>>> B = CombinatorialFreeModule(ZZ, [-Integer(1), Integer(0),␣
↪Integer(1)])                          # needs sage.modules
>>> x = B.an_element(); x                                                      #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
>>> x.map_support(lambda i: -i)                                                #␣
↪needs sage.modules
3*B[-1] + 2*B[0] + 2*B[1]
```

`f` needs not be injective:

```
sage: x.map_support(lambda i: 1)                                               #␣
↪needs sage.modules
7*B[1]

sage: s = SymmetricFunctions(QQ).schur()                                       #␣
↪needs sage.combinat sage.modules
sage: a = s([2,1]) + 2*s([3,2])                                                #␣
↪needs sage.combinat sage.modules
sage: a.map_support(lambda x: x.conjugate())                                   #␣
↪needs sage.combinat sage.modules
s[2, 1] + 2*s[2, 2, 1]
```

```
>>> from sage.all import *
>>> x.map_support(lambda i: Integer(1))                                         ␣
↪     # needs sage.modules
7*B[1]

>>> s = SymmetricFunctions(QQ).schur()                                         #␣
↪needs sage.combinat sage.modules
>>> a = s([Integer(2),Integer(1)]) + Integer(2)*s([Integer(3),
↪Integer(2)])                                          # needs sage.
↪combinat sage.modules
>>> a.map_support(lambda x: x.conjugate())                                     #␣
↪needs sage.combinat sage.modules
s[2, 1] + 2*s[2, 2, 1]
```

map_support_skip_none(*f*)

Mapping a function on the support.

INPUT:

- `f` – an endofunction on the indices of the free module

Returns a new element of `self.parent()` obtained by applying the function f to all of the objects indexing the basis elements.

EXAMPLES:

```
sage: B = CombinatorialFreeModule(ZZ, [-1, 0, 1])                    #␣
↪needs sage.modules
sage: x = B.an_element(); x                                          #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
sage: x.map_support_skip_none(lambda i: -i if i else None)           #␣
↪needs sage.modules
3*B[-1] + 2*B[1]
```

```
>>> from sage.all import *
>>> B = CombinatorialFreeModule(ZZ, [-Integer(1), Integer(0),␣
↪Integer(1)])                          # needs sage.modules
>>> x = B.an_element(); x                                            #␣
↪needs sage.modules
2*B[-1] + 2*B[0] + 3*B[1]
>>> x.map_support_skip_none(lambda i: -i if i else None)             #␣
↪needs sage.modules
3*B[-1] + 2*B[1]
```

`f` needs not be injective:

```
sage: x.map_support_skip_none(lambda i: 1 if i else None)            #␣
↪needs sage.modules
5*B[1]
```

```
>>> from sage.all import *
>>> x.map_support_skip_none(lambda i: Integer(1) if i else None)      ␣
↪     # needs sage.modules
5*B[1]
```

monomial_coefficients(*copy=True*)

Return a dictionary whose keys are indices of basis elements in the support of `self` and whose values are the corresponding coefficients.

INPUT:

- `copy` – boolean (default: `True`); if `self` is internally represented by a dictionary `d`, then make a copy of `d`; if `False`, then this can cause undesired behavior by mutating `d`

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] + 3*B['c']
sage: d = f.monomial_coefficients()
```

(continues on next page)

(continued from previous page)

```
sage: d['a']
1
sage: d['c']
3
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] + Integer(3)*B['c']
>>> d = f.monomial_coefficients()
>>> d['a']
1
>>> d['c']
3
```

monomials()

Return a list of the monomials of `self` (in an arbitrary order).

The monomials of an element a are defined to be the basis elements whose corresponding coefficients of a are nonzero.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] + 2*B['c']
sage: f.monomials()
[B['a'], B['c']]

sage: (F.zero()).monomials()                                                #␣
↪needs sage.modules
[]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] + Integer(2)*B['c']
>>> f.monomials()
[B['a'], B['c']]

>>> (F.zero()).monomials()                                                  #␣
↪needs sage.modules
[]
```

support()

Return an iterable of the objects indexing the basis of `self.parent()` whose corresponding coefficients of `self` are nonzero.

This method returns these objects in an arbitrary order.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] - 3*B['c']
sage: sorted(f.support())
['a', 'c']
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] - Integer(3)*B['c']
>>> sorted(f.support())
['a', 'c']
```

```
sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: z = s([4]) + s([2,1]) + s([1,1,1]) + s([1])                           #␣
↪needs sage.combinat sage.modules
sage: sorted(z.support())                                                   #␣
↪needs sage.combinat sage.modules
[[1], [1, 1, 1], [2, 1], [4]]
```

```
>>> from sage.all import *
>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> z = s([Integer(4)]) + s([Integer(2),Integer(1)]) + s([Integer(1),
↪Integer(1),Integer(1)]) + s([Integer(1)])                                  # needs␣
↪sage.combinat sage.modules
>>> sorted(z.support())                                                     #␣
↪needs sage.combinat sage.modules
[[1], [1, 1, 1], [2, 1], [4]]
```

support_of_term()

Return the support of `self`, where `self` is a monomial (possibly with coefficient).

EXAMPLES:

```
sage: X = CombinatorialFreeModule(QQ, [1,2,3,4]); X.rename('X')             #␣
↪needs sage.modules
sage: X.monomial(2).support_of_term()                                       #␣
↪needs sage.modules
2
sage: X.term(3, 2).support_of_term()                                        #␣
↪needs sage.modules
3
```

```
>>> from sage.all import *
>>> X = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3),
↪Integer(4)]); X.rename('X')          # needs sage.modules
>>> X.monomial(Integer(2)).support_of_term()                                ␣
↪     # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
2
>>> X.term(Integer(3), Integer(2)).support_of_term()                    ␣
↪              # needs sage.modules
3
```

An exception is raised if `self` has more than one term:

```
sage: (X.monomial(2) + X.monomial(3)).support_of_term()                #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: B[2] + B[3] is not a single term
```

```
>>> from sage.all import *
>>> (X.monomial(Integer(2)) + X.monomial(Integer(3))).support_of_term()  ␣
↪              # needs sage.modules
Traceback (most recent call last):
...
ValueError: B[2] + B[3] is not a single term
```

tensor(**elements*)

Return the tensor product of its arguments, as an element of the tensor product of the parents of those elements.

EXAMPLES:

```
sage: C = AlgebrasWithBasis(QQ)
sage: A = C.example()                                                  #␣
↪needs sage.combinat sage.modules
sage: a, b, c = A.algebra_generators()                                 #␣
↪needs sage.combinat sage.modules
sage: a.tensor(b, c)                                                   #␣
↪needs sage.combinat sage.modules
B[word: a] # B[word: b] # B[word: c]
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(QQ)
>>> A = C.example()                                                  #␣
↪needs sage.combinat sage.modules
>>> a, b, c = A.algebra_generators()                                 #␣
↪needs sage.combinat sage.modules
>>> a.tensor(b, c)                                                   #␣
↪needs sage.combinat sage.modules
B[word: a] # B[word: b] # B[word: c]
```

FIXME: is this a policy that we want to enforce on all parents?

terms()

Return a list of the (nonzero) terms of `self` (in an arbitrary order).

See also

monomials()

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: B = F.basis()
sage: f = B['a'] + 2*B['c']
sage: f.terms()
[B['a'], 2*B['c']]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> B = F.basis()
>>> f = B['a'] + Integer(2)*B['c']
>>> f.terms()
[B['a'], 2*B['c']]
```

trailing_coefficient(**args*, ***kwds*)

Return the trailing coefficient of `self`.

This is the coefficient of the monomial whose corresponding basis element is minimal. Note that this may not be the term which actually appears last when `self` is printed.

If the default term ordering is not what is desired, a comparison key `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.trailing_coefficient()
3
sage: def key(x): return -x
sage: x.trailing_coefficient(key=key)
1

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.trailing_coefficient()                                              #␣
↪needs sage.combinat sage.modules
2
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
```

(continues on next page)

(continued from previous page)

```
>>> x.trailing_coefficient()
3
>>> def key(x): return -x
>>> x.trailing_coefficient(key=key)
1

>>> s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.trailing_coefficient()                                              #␣
↪needs sage.combinat sage.modules
2
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_coefficient()
4
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_coefficient()
2
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_coefficient()
3
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_coefficient()
4
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_coefficient()
2
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_coefficient()
3
```

trailing_item(**args*, ***kwds*)

Return the pair `(c, k)` where `c*self.parent().monomial(k)` is the trailing term of `self`.

This is the monomial whose corresponding basis element is minimal. Note that this may not be the term which actually appears last when `self` is printed.

If the default term ordering is not what is desired, a comparison key `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.trailing_item()
(1, 3)
sage: def key(x): return -x
sage: x.trailing_item(key=key)
(3, 1)

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.trailing_item()                                                     #␣
↪needs sage.combinat sage.modules
([1], 2)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
>>> x.trailing_item()
(1, 3)
>>> def key(x): return -x
>>> x.trailing_item(key=key)
(3, 1)

>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.trailing_item()                                                       #␣
↪needs sage.combinat sage.modules
([1], 2)
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_item()
((1, 1, 1), 4)
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_item()
((0, 1, 3), 2)
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_item()
((1, 2, 0), 3)
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_item()
((1, 1, 1), 4)
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_item()
((0, 1, 3), 2)
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_item()
((1, 2, 0), 3)
```

trailing_monomial(**args*, ***kwds*)

Return the trailing monomial of `self`.

This is the monomial whose corresponding basis element is minimal. Note that this may not be the term which actually appears last when `self` is printed.

If the default term ordering is not what is desired, a comparison key `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.trailing_monomial()
B[1]
sage: def key(x): return -x
sage: x.trailing_monomial(key=key)
B[3]

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.trailing_monomial()                                                 #␣
↪needs sage.combinat sage.modules
s[1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
>>> x.trailing_monomial()
B[1]
>>> def key(x): return -x
>>> x.trailing_monomial(key=key)
B[3]
```

(continues on next page)

(continued from previous page)

```
>>> s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.trailing_monomial()                                                 #␣
↪needs sage.combinat sage.modules
s[1]
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_monomial()
x*y*z
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_monomial()
y*z^3
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_monomial()
x*y^2
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_monomial()
x*y*z
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_monomial()
y*z^3
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_monomial()
x*y^2
```

trailing_support(**args*, ***kwds*)

Return the minimal element of the support of `self`. Note that this may not be the term which actually appears last when `self` is printed.

If the default ordering of the basis elements is not what is desired, a comparison key, `key(x)`, can be provided.

EXAMPLES:

```
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')          #␣
↪needs sage.modules
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + 4*X.monomial(3)            #␣
↪needs sage.modules
sage: x.trailing_support()                                               #␣
↪needs sage.modules
1
```

(continues on next page)

(continued from previous page)

```
sage: def key(x): return -x
sage: x.trailing_support(key=key)                                           #␣
↪needs sage.modules
3

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.trailing_support()                                                  #␣
↪needs sage.combinat sage.modules
[1]
```

```
>>> from sage.all import *
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')          # needs sage.modules
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + Integer(4)*X.monomial(Integer(3))            #␣
↪needs sage.modules
>>> x.trailing_support()                                                     #␣
↪needs sage.modules
1

>>> def key(x): return -x
>>> x.trailing_support(key=key)                                              #␣
↪needs sage.modules
3

>>> s = SymmetricFunctions(QQ).schur()                                       #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.trailing_support()                                                     #␣
↪needs sage.combinat sage.modules
[1]
```

trailing_term(**args*, ***kwds*)

Return the trailing term of `self`.

This is the term whose corresponding basis element is minimal. Note that this may not be the term which actually appears last when `self` is printed.

If the default term ordering is not what is desired, a comparison key `key(x)`, can be provided.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1, 2, 3]); X.rename('X')
sage: x = 3*X.monomial(1) + 2*X.monomial(2) + X.monomial(3)
sage: x.trailing_term()
3*B[1]
```

(continues on next page)

(continued from previous page)

```
sage: def key(x): return -x
sage: x.trailing_term(key=key)
B[3]

sage: s = SymmetricFunctions(QQ).schur()                                    #␣
↪needs sage.combinat sage.modules
sage: f = 2*s[1] + 3*s[2,1] - 5*s[3]                                        #␣
↪needs sage.combinat sage.modules
sage: f.trailing_term()                                                     #␣
↪needs sage.combinat sage.modules
2*s[1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)]);
↪ X.rename('X')
>>> x = Integer(3)*X.monomial(Integer(1)) + Integer(2)*X.
↪monomial(Integer(2)) + X.monomial(Integer(3))
>>> x.trailing_term()
3*B[1]
>>> def key(x): return -x
>>> x.trailing_term(key=key)
B[3]

>>> s = SymmetricFunctions(QQ).schur()                                      #␣
↪needs sage.combinat sage.modules
>>> f = Integer(2)*s[Integer(1)] + Integer(3)*s[Integer(2),Integer(1)] -␣
↪Integer(5)*s[Integer(3)]                                          # needs␣
↪sage.combinat sage.modules
>>> f.trailing_term()                                                       #␣
↪needs sage.combinat sage.modules
2*s[1]
```

The term ordering of polynomial rings is taken into account:

```
sage: R.<x,y,z> = QQ[]
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_term()
4*x*y*z
sage: R.<x,y,z> = PolynomialRing(QQ, order='lex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_term()
2*y*z^3
sage: R.<x,y,z> = PolynomialRing(QQ, order='invlex')
sage: (3*x*y^2 + 2*y*z^3 + y^4 + 4*x*y*z).trailing_term()
3*x*y^2
```

```
>>> from sage.all import *
>>> R = QQ['x, y, z']; (x, y, z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_term()
4*x*y*z
>>> R = PolynomialRing(QQ, order='lex', names=('x', 'y', 'z',)); (x, y, z,
↪) = R._first_ngens(3)
```

(continues on next page)

(continued from previous page)

```
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_term()
2*y*z^3
>>> R = PolynomialRing(QQ, order='invlex', names=('x', 'y', 'z',)); (x, y,
↪ z,) = R._first_ngens(3)
>>> (Integer(3)*x*y**Integer(2) + Integer(2)*y*z**Integer(3) +␣
↪y**Integer(4) + Integer(4)*x*y*z).trailing_term()
3*x*y^2
```

Filtered

alias of *FilteredModulesWithBasis*

FiniteDimensional

alias of *FiniteDimensionalModulesWithBasis*

Graded

alias of *GradedModulesWithBasis*

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class ParentMethods

Bases: object

class MorphismMethods

Bases: object

on_basis()

Return the action of this morphism on basis elements.

OUTPUT:

- a function from the indices of the basis of the domain to the codomain

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1,2,3]);   X.rename('X')
sage: Y = CombinatorialFreeModule(QQ, [1,2,3,4]); Y.rename('Y')
sage: H = Hom(X, Y)
sage: x = X.basis()
sage: f = H(lambda x: Y.zero()).on_basis()
sage: f(2)
0
sage: f = lambda i: Y.monomial(i) + 2*Y.monomial(i+1)
sage: g = H(on_basis=f).on_basis()
sage: g(2)
B[2] + 2*B[3]
sage: g == f
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3)]); ␣
↪ X.rename('X')
>>> Y = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3),
```

(continues on next page)

(continued from previous page)

```
↪Integer(4)]); Y.rename('Y')
>>> H = Hom(X, Y)
>>> x = X.basis()
>>> f = H(lambda x: Y.zero()).on_basis()
>>> f(Integer(2))
0
>>> f = lambda i: Y.monomial(i) + Integer(2)*Y.monomial(i+Integer(1))
>>> g = H(on_basis=f).on_basis()
>>> g(Integer(2))
B[2] + 2*B[3]
>>> g == f
True
```

class ParentMethods

Bases: object

basis()

Return the basis of self.

EXAMPLES:

```
sage: F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                #␣
↪needs sage.modules
sage: F.basis()                                                       #␣
↪needs sage.modules
Finite family {'a': B['a'], 'b': B['b'], 'c': B['c']}
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                  #␣
↪needs sage.modules
>>> F.basis()                                                         #␣
↪needs sage.modules
Finite family {'a': B['a'], 'b': B['b'], 'c': B['c']}
```

```
sage: QS3 = SymmetricGroupAlgebra(QQ, 3)                              #␣
↪needs sage.combinat sage.groups sage.modules
sage: list(QS3.basis())                                               #␣
↪needs sage.combinat sage.groups sage.modules
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

```
>>> from sage.all import *
>>> QS3 = SymmetricGroupAlgebra(QQ, Integer(3))                       ␣
↪     # needs sage.combinat sage.groups sage.modules
>>> list(QS3.basis())                                                 #␣
↪needs sage.combinat sage.groups sage.modules
[[1, 2, 3], [1, 3, 2], [2, 1, 3], [2, 3, 1], [3, 1, 2], [3, 2, 1]]
```

cardinality()

Return the cardinality of self.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: S = SymmetricGroupAlgebra(QQ, 4)
sage: S.cardinality()
+Infinity
sage: S = SymmetricGroupAlgebra(GF(2), 4)
sage: S.cardinality()
16777216
sage: S.cardinality().factor()
2^24

sage: # needs sage.modules
sage: E.<x,y> = ExteriorAlgebra(QQ)
sage: E.cardinality()
+Infinity
sage: E.<x,y> = ExteriorAlgebra(GF(3))
sage: E.cardinality()
81

sage: s = SymmetricFunctions(GF(2)).s()                                    #␣
↪needs sage.combinat sage.modules
sage: s.cardinality()                                                      #␣
↪needs sage.combinat sage.modules
+Infinity

sage: M = CombinatorialFreeModule(QQ, [])
sage: M.dimension()
0
sage: M.cardinality()
1
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S = SymmetricGroupAlgebra(QQ, Integer(4))
>>> S.cardinality()
+Infinity
>>> S = SymmetricGroupAlgebra(GF(Integer(2)), Integer(4))
>>> S.cardinality()
16777216
>>> S.cardinality().factor()
2^24

>>> # needs sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = E._first_
↪ngens(2)
>>> E.cardinality()
+Infinity
>>> E = ExteriorAlgebra(GF(Integer(3)), names=('x', 'y',)); (x, y,) = E._
↪first_ngens(2)
>>> E.cardinality()
81

>>> s = SymmetricFunctions(GF(Integer(2))).s()                             ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.combinat sage.modules
>>> s.cardinality()                                                    #␣
↪needs sage.combinat sage.modules
+Infinity

>>> M = CombinatorialFreeModule(QQ, [])
>>> M.dimension()
0
>>> M.cardinality()
1
```

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: A.<x,y> = algebras.DifferentialWeyl(QQ)                          #␣
↪needs sage.modules
sage: A.dimension()                                                    #␣
↪needs sage.modules
+Infinity
```

```
>>> from sage.all import *
>>> A = algebras.DifferentialWeyl(QQ, names=('x', 'y',)); (x, y,) = A._
↪first_ngens(2)# needs sage.modules
>>> A.dimension()                                                      #␣
↪needs sage.modules
+Infinity
```

echelon_form(*elements*, *row_reduced=False*, *order=None*)

Return a basis in echelon form of the subspace spanned by a finite set of elements.

INPUT:

- `elements` – list or finite iterable of elements of `self`
- `row_reduced` – boolean (default: `False`); whether to compute the basis for the row reduced echelon form
- `order` – (optional) either something that can be converted into a tuple or a key function

OUTPUT:

A list of elements of `self` whose expressions as vectors form a matrix in echelon form. If `base_ring` is specified, then the calculation is achieved in this base ring.

EXAMPLES:

```
sage: R.<x,y> = QQ[]
sage: C = CombinatorialFreeModule(R, ZZ, prefix='z')                   #␣
↪needs sage.modules
sage: z = C.basis()                                                    #␣
↪needs sage.modules
sage: C.echelon_form([z[0] - z[1], 2*z[1] - 2*z[2], z[0] - z[2]])      #␣
↪needs sage.libs.singular sage.modules
[z[0] - z[2], z[1] - z[2]]
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> C = CombinatorialFreeModule(R, ZZ, prefix='z')                    #␣
↪needs sage.modules
>>> z = C.basis()                                                     #␣
↪needs sage.modules
>>> C.echelon_form([z[Integer(0)] - z[Integer(1)],␣
↪Integer(2)*z[Integer(1)] - Integer(2)*z[Integer(2)], z[Integer(0)] -␣
↪z[Integer(2)]])       # needs sage.libs.singular sage.modules
[z[0] - z[2], z[1] - z[2]]
```

intersection(*other*)

Return the intersection of `self` with `other`.

EXAMPLES:

```
sage: X = CombinatorialFreeModule(QQ, range(4)); x = X.basis()
sage: U = X.submodule([x[0]-x[1], x[1]-x[2], x[2]-x[3]])
sage: F = CombinatorialFreeModule(QQ, ['a','b','c','d'])
sage: G = F.submodule([F.basis()['a']])
sage: X.intersection(X) is X
True
sage: X.intersection(U) is U
True
sage: X.intersection(F)
Traceback (most recent call last):
...
TypeError: other must be a submodule
sage: X.intersection(G)
Traceback (most recent call last):
...
ArithmeticError: this module must be the ambient
```

```
>>> from sage.all import *
>>> X = CombinatorialFreeModule(QQ, range(Integer(4))); x = X.basis()
>>> U = X.submodule([x[Integer(0)]-x[Integer(1)], x[Integer(1)]-
↪x[Integer(2)], x[Integer(2)]-x[Integer(3)]])
>>> F = CombinatorialFreeModule(QQ, ['a','b','c','d'])
>>> G = F.submodule([F.basis()['a']])
>>> X.intersection(X) is X
True
>>> X.intersection(U) is U
True
>>> X.intersection(F)
Traceback (most recent call last):
...
TypeError: other must be a submodule
>>> X.intersection(G)
Traceback (most recent call last):
...
ArithmeticError: this module must be the ambient
```

is_finite()

Return whether `self` is finite.

This is true if and only if `self.basis().keys()` and `self.base_ring()` are both finite.

EXAMPLES:

```
sage: GroupAlgebra(SymmetricGroup(2), IntegerModRing(10)).is_finite()    #␣
↪needs sage.combinat sage.groups sage.modules
True
sage: GroupAlgebra(SymmetricGroup(2)).is_finite()                         #␣
↪needs sage.combinat sage.groups sage.modules
False
sage: GroupAlgebra(AbelianGroup(1), IntegerModRing(10)).is_finite()       #␣
↪needs sage.groups sage.modules
False
```

```
>>> from sage.all import *
>>> GroupAlgebra(SymmetricGroup(Integer(2)), IntegerModRing(Integer(10))).
↪is_finite()    # needs sage.combinat sage.groups sage.modules
True
>>> GroupAlgebra(SymmetricGroup(Integer(2))).is_finite()                  ␣
↪      # needs sage.combinat sage.groups sage.modules
False
>>> GroupAlgebra(AbelianGroup(Integer(1)), IntegerModRing(Integer(10))).
↪is_finite()      # needs sage.groups sage.modules
False
```

module_morphism(*on_basis=None*, *matrix=None*, *function=None*, *diagonal=None*, *triangular=None*, *unitriangular=False*, ***keywords*)

Construct a module morphism from `self` to `codomain`.

Let `self` be a module X with a basis indexed by I. This constructs a morphism $f : X \to Y$ by linearity from a map $I \to Y$ which is to be its restriction to the basis $(x_i)_{i \in I}$ of X. Some variants are possible too.

INPUT:

- `self` – a parent X in `ModulesWithBasis(R)` with basis $x = (x_i)_{i \in I}$

Exactly one of the four following options must be specified in order to define the morphism:

- `on_basis` – a function f from I to Y
- `diagonal` – a function d from I to R
- `function` – a function f from X to Y
- `matrix` – a matrix of size $\dim Y \times \dim X$ (if the keyword `side` is set to `'left'`) or $\dim Y \times \dim X$ (if this keyword is `'right'`)

Further options include:

- `codomain` – the codomain Y of the morphism (default: `f.codomain()` if it's defined; otherwise it must be specified)
- `category` – a category or `None` (default: `None`)
- `zero` – the zero of the codomain (default: `codomain.zero()`); can be used (with care) to define affine maps. Only meaningful with `on_basis`.
- `position` – nonnegative integer specifying which positional argument is used as the input of the function f (default: 0); this is currently only used with `on_basis`.
- `triangular` – (default: `None`) `'upper'` or `'lower'` or `None`:
  - `'upper'` – if the *`leading_support()`* of the image of the basis vector x_i is i, or
  - `'lower'` – if the *`trailing_support()`* of the image of the basis vector x_i is i.
- `unitriangular` – boolean (default: `False`); only meaningful for a triangular morphism. As a shorthand, one may use `unitriangular="lower"` for `triangular='lower', unitriangular=True`.

- `side` – `'left'` or `'right'` (default: `'left'`); only meaningful for a morphism built from a matrix

EXAMPLES:

With the `on_basis` option, this returns a function g obtained by extending f by linearity on the `position`-th positional argument. For example, for `position == 1` and a ternary function f, one has:

$$g\left(a, \sum_i \lambda_i x_i, c\right) = \sum_i \lambda_i f(a, i, c).$$

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [1,2,3]);   X.rename('X')
sage: Y = CombinatorialFreeModule(QQ, [1,2,3,4]); Y.rename('Y')
sage: def f(i):
....:     return Y.monomial(i) + 2*Y.monomial(i+1)
sage: phi = X.module_morphism(f, codomain=Y)
sage: x = X.basis(); y = Y.basis()
sage: phi(x[1] + x[3])
B[1] + 2*B[2] + B[3] + 2*B[4]
sage: phi
Generic morphism:
From: X
To:   Y
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3)]); ␣
↪ X.rename('X')
>>> Y = CombinatorialFreeModule(QQ, [Integer(1),Integer(2),Integer(3),
↪Integer(4)]); Y.rename('Y')
>>> def f(i):
...     return Y.monomial(i) + Integer(2)*Y.monomial(i+Integer(1))
>>> phi = X.module_morphism(f, codomain=Y)
>>> x = X.basis(); y = Y.basis()
>>> phi(x[Integer(1)] + x[Integer(3)])
B[1] + 2*B[2] + B[3] + 2*B[4]
>>> phi
Generic morphism:
From: X
To:   Y
```

By default, the category is the first of `Modules(R).WithBasis().FiniteDimensional()`, `Modules(R).WithBasis()`, `Modules(R)`, and `CommutativeAdditiveMonoids()` that contains both the domain and the codomain:

```
sage: phi.category_for()                                                    #␣
↪needs sage.modules
Category of finite dimensional vector spaces with basis
 over Rational Field
```

```
>>> from sage.all import *
>>> phi.category_for()                                                      #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
Category of finite dimensional vector spaces with basis
 over Rational Field
```

With the `zero` argument, one can define affine morphisms:

```
sage: def f(i):
....:     return Y.monomial(i) + 2*Y.monomial(i+1)
sage: phi = X.module_morphism(f, codomain=Y, zero=10*y[1])              #␣
↪needs sage.modules
sage: phi(x[1] + x[3])                                                  #␣
↪needs sage.modules
11*B[1] + 2*B[2] + B[3] + 2*B[4]
```

```
>>> from sage.all import *
>>> def f(i):
...     return Y.monomial(i) + Integer(2)*Y.monomial(i+Integer(1))
>>> phi = X.module_morphism(f, codomain=Y,␣
↪zero=Integer(10)*y[Integer(1)])              # needs sage.modules
>>> phi(x[Integer(1)] + x[Integer(3)])                                  ␣
↪              # needs sage.modules
11*B[1] + 2*B[2] + B[3] + 2*B[4]
```

In this special case, the default category is `Sets()`:

```
sage: phi.category_for()                                                #␣
↪needs sage.modules
Category of sets
```

```
>>> from sage.all import *
>>> phi.category_for()                                                 #␣
↪needs sage.modules
Category of sets
```

One can construct morphisms with the base ring as codomain:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(ZZ, [1, -1])
sage: phi = X.module_morphism(on_basis=lambda i: i, codomain=ZZ)
sage: phi(2 * X.monomial(1) + 3 * X.monomial(-1))
-1
sage: phi.category_for()
Category of commutative additive semigroups
sage: phi.category_for()        # not implemented
Category of modules over Integer Ring
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(ZZ, [Integer(1), -Integer(1)])
>>> phi = X.module_morphism(on_basis=lambda i: i, codomain=ZZ)
>>> phi(Integer(2) * X.monomial(Integer(1)) + Integer(3) * X.monomial(-
↪Integer(1)))
-1
```

(continues on next page)

(continued from previous page)

```
>>> phi.category_for()
Category of commutative additive semigroups
>>> phi.category_for()        # not implemented
Category of modules over Integer Ring
```

Or more generally any ring admitting a coercion map from the base ring:

```
sage: # needs sage.modules
sage: phi = X.module_morphism(on_basis=lambda i: i, codomain=RR)
sage: phi(2 * X.monomial(1) + 3 * X.monomial(-1))
-1.00000000000000
sage: phi.category_for()
Category of commutative additive semigroups
sage: phi.category_for()        # not implemented
Category of modules over Integer Ring

sage: phi = X.module_morphism(on_basis=lambda i: i, codomain=Zmod(4))    #␣
↪needs sage.modules
sage: phi(2 * X.monomial(1) + 3 * X.monomial(-1))                       #␣
↪needs sage.modules
3

sage: phi = Y.module_morphism(on_basis=lambda i: i, codomain=Zmod(4))    #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: codomain(=Ring of integers modulo 4) should be a module
over the base ring of the domain(=Y)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> phi = X.module_morphism(on_basis=lambda i: i, codomain=RR)
>>> phi(Integer(2) * X.monomial(Integer(1)) + Integer(3) * X.monomial(-
↪Integer(1)))
-1.00000000000000
>>> phi.category_for()
Category of commutative additive semigroups
>>> phi.category_for()        # not implemented
Category of modules over Integer Ring

>>> phi = X.module_morphism(on_basis=lambda i: i,␣
↪codomain=Zmod(Integer(4)))    # needs sage.modules
>>> phi(Integer(2) * X.monomial(Integer(1)) + Integer(3) * X.monomial(-
↪Integer(1)))                        # needs sage.modules
3

>>> phi = Y.module_morphism(on_basis=lambda i: i,␣
↪codomain=Zmod(Integer(4)))    # needs sage.modules
Traceback (most recent call last):
...
ValueError: codomain(=Ring of integers modulo 4) should be a module
over the base ring of the domain(=Y)
```

On can also define module morphisms between free modules over different base rings; here we implement the natural map from $X = \mathbf{R}^2$ to $Y = \mathbf{C}$:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(RR, ['x', 'y'])
sage: Y = CombinatorialFreeModule(CC, ['z'])
sage: x = X.monomial('x')
sage: y = X.monomial('y')
sage: z = Y.monomial('z')
sage: def on_basis(a):
....:     if a == 'x':
....:         return CC(1) * z
....:     elif a == 'y':
....:         return CC(I) * z
sage: phi = X.module_morphism(on_basis=on_basis, codomain=Y)
sage: v = 3 * x + 2 * y; v
3.00000000000000*B['x'] + 2.00000000000000*B['y']
sage: phi(v)                                                                #␣
↪needs sage.symbolic
(3.00000000000000+2.00000000000000*I)*B['z']
sage: phi.category_for()
Category of commutative additive semigroups
sage: phi.category_for()        # not implemented
Category of vector spaces over Real Field with 53 bits of precision

sage: # needs sage.modules
sage: Y = CombinatorialFreeModule(CC['q'], ['z'])
sage: z = Y.monomial('z')
sage: phi = X.module_morphism(on_basis=on_basis, codomain=Y)
sage: phi(v)                                                                #␣
↪needs sage.symbolic
(3.00000000000000+2.00000000000000*I)*B['z']
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(RR, ['x', 'y'])
>>> Y = CombinatorialFreeModule(CC, ['z'])
>>> x = X.monomial('x')
>>> y = X.monomial('y')
>>> z = Y.monomial('z')
>>> def on_basis(a):
...     if a == 'x':
...         return CC(Integer(1)) * z
...     elif a == 'y':
...         return CC(I) * z
>>> phi = X.module_morphism(on_basis=on_basis, codomain=Y)
>>> v = Integer(3) * x + Integer(2) * y; v
3.00000000000000*B['x'] + 2.00000000000000*B['y']
>>> phi(v)                                                                  #␣
↪needs sage.symbolic
(3.00000000000000+2.00000000000000*I)*B['z']
>>> phi.category_for()
Category of commutative additive semigroups
```

(continues on next page)

(continued from previous page)

```
>>> phi.category_for()        # not implemented
Category of vector spaces over Real Field with 53 bits of precision

>>> # needs sage.modules
>>> Y = CombinatorialFreeModule(CC['q'], ['z'])
>>> z = Y.monomial('z')
>>> phi = X.module_morphism(on_basis=on_basis, codomain=Y)
>>> phi(v)                                                              #␣
↪needs sage.symbolic
(3.00000000000000+2.00000000000000*I)*B['z']
```

Of course, there should be a coercion between the respective base rings of the domain and the codomain for this to be meaningful:

```
sage: Y = CombinatorialFreeModule(QQ, ['z'])                            #␣
↪needs sage.modules
sage: phi = X.module_morphism(on_basis=on_basis, codomain=Y)            #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: codomain(=Free module generated by {'z'} over Rational Field)
should be a module over the base ring of the domain(=Free module
generated by {'x', 'y'} over Real Field with 53 bits of precision)

sage: Y = CombinatorialFreeModule(RR['q'], ['z'])                       #␣
↪needs sage.modules
sage: phi = Y.module_morphism(on_basis=on_basis, codomain=X)            #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: codomain(=Free module generated by {'x', 'y'}
over Real Field with 53 bits of precision) should be a module over
the base ring of the domain(=Free module generated by {'z'} over
Univariate Polynomial Ring in q over Real Field with 53 bits of precision)
```

```
>>> from sage.all import *
>>> Y = CombinatorialFreeModule(QQ, ['z'])                              #␣
↪needs sage.modules
>>> phi = X.module_morphism(on_basis=on_basis, codomain=Y)              #␣
↪needs sage.modules
Traceback (most recent call last):
...
ValueError: codomain(=Free module generated by {'z'} over Rational Field)
should be a module over the base ring of the domain(=Free module
generated by {'x', 'y'} over Real Field with 53 bits of precision)

>>> Y = CombinatorialFreeModule(RR['q'], ['z'])                         #␣
↪needs sage.modules
>>> phi = Y.module_morphism(on_basis=on_basis, codomain=X)              #␣
↪needs sage.modules
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: codomain(=Free module generated by {'x', 'y'}
over Real Field with 53 bits of precision) should be a module over
the base ring of the domain(=Free module generated by {'z'} over
Univariate Polynomial Ring in q over Real Field with 53 bits of precision)
```

With the `diagonal=d` argument, this constructs the module morphism g such that

$$`g(x_i) = d(i)y_i`.$$

This assumes that the respective bases x and y of X and Y have the same index set I:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(ZZ, [1, 2, 3]); X.rename('X')
sage: from sage.arith.misc import factorial
sage: phi = X.module_morphism(diagonal=factorial, codomain=X)
sage: x = X.basis()
sage: phi(x[1]), phi(x[2]), phi(x[3])
(B[1], 2*B[2], 6*B[3])
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(ZZ, [Integer(1), Integer(2), Integer(3)]);
... X.rename('X')
>>> from sage.arith.misc import factorial
>>> phi = X.module_morphism(diagonal=factorial, codomain=X)
>>> x = X.basis()
>>> phi(x[Integer(1)]), phi(x[Integer(2)]), phi(x[Integer(3)])
(B[1], 2*B[2], 6*B[3])
```

See also: `sage.modules.with_basis.morphism.DiagonalModuleMorphism`.

With the `matrix=m` argument, this constructs the module morphism whose matrix in the distinguished basis of X and Y is m:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(ZZ, [1,2,3]); X.rename('X')
sage: x = X.basis()
sage: Y = CombinatorialFreeModule(ZZ, [3,4]); Y.rename('Y')
sage: y = Y.basis()
sage: m = matrix([[0,1,2], [3,5,0]])
sage: phi = X.module_morphism(matrix=m, codomain=Y)
sage: phi(x[1])
3*B[4]
sage: phi(x[2])
B[3] + 5*B[4]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2),Integer(3)]);␣
...X.rename('X')
>>> x = X.basis()
>>> Y = CombinatorialFreeModule(ZZ, [Integer(3),Integer(4)]); Y.rename('Y
...')
```

(continues on next page)

(continued from previous page)

```
>>> y = Y.basis()
>>> m = matrix([[Integer(0),Integer(1),Integer(2)], [Integer(3),
↪Integer(5),Integer(0)]])
>>> phi = X.module_morphism(matrix=m, codomain=Y)
>>> phi(x[Integer(1)])
3*B[4]
>>> phi(x[Integer(2)])
B[3] + 5*B[4]
```

See also: `sage.modules.with_basis.morphism.ModuleMorphismFromMatrix`.

With `triangular="upper"`, the constructed module morphism is assumed to be upper triangular; that is its matrix in the distinguished basis of X and Y would be upper triangular with invertible elements on its diagonal. This is used to compute preimages and to invert the morphism:

```
sage: # needs sage.modules
sage: I = list(range(1, 200))
sage: X = CombinatorialFreeModule(QQ, I); X.rename('X'); x = X.basis()
sage: Y = CombinatorialFreeModule(QQ, I); Y.rename('Y'); y = Y.basis()
sage: f = Y.sum_of_monomials * divisors
sage: phi = X.module_morphism(f, triangular='upper', codomain=Y)
sage: phi(x[2])
B[1] + B[2]
sage: phi(x[6])
B[1] + B[2] + B[3] + B[6]
sage: phi(x[30])
B[1] + B[2] + B[3] + B[5] + B[6] + B[10] + B[15] + B[30]
sage: phi.preimage(y[2])
-B[1] + B[2]
sage: phi.preimage(y[6])
B[1] - B[2] - B[3] + B[6]
sage: phi.preimage(y[30])
-B[1] + B[2] + B[3] + B[5] - B[6] - B[10] - B[15] + B[30]
sage: (phi^-1)(y[30])
-B[1] + B[2] + B[3] + B[5] - B[6] - B[10] - B[15] + B[30]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> I = list(range(Integer(1), Integer(200)))
>>> X = CombinatorialFreeModule(QQ, I); X.rename('X'); x = X.basis()
>>> Y = CombinatorialFreeModule(QQ, I); Y.rename('Y'); y = Y.basis()
>>> f = Y.sum_of_monomials * divisors
>>> phi = X.module_morphism(f, triangular='upper', codomain=Y)
>>> phi(x[Integer(2)])
B[1] + B[2]
>>> phi(x[Integer(6)])
B[1] + B[2] + B[3] + B[6]
>>> phi(x[Integer(30)])
B[1] + B[2] + B[3] + B[5] + B[6] + B[10] + B[15] + B[30]
>>> phi.preimage(y[Integer(2)])
-B[1] + B[2]
>>> phi.preimage(y[Integer(6)])
```

(continues on next page)

(continued from previous page)

```
B[1] - B[2] - B[3] + B[6]
>>> phi.preimage(y[Integer(30)])
-B[1] + B[2] + B[3] + B[5] - B[6] - B[10] - B[15] + B[30]
>>> (phi**-Integer(1))(y[Integer(30)])
-B[1] + B[2] + B[3] + B[5] - B[6] - B[10] - B[15] + B[30]
```

Since Issue #8678, one can also define a triangular morphism from a function:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, [0,1,2,3,4]); x = X.basis()
sage: from sage.modules.with_basis.morphism import␣
↪TriangularModuleMorphismFromFunction
sage: def f(x): return x + X.term(0, sum(x.coefficients()))
sage: phi = X.module_morphism(function=f, codomain=X,
....:                         triangular='upper')
sage: phi(x[2] + 3*x[4])
4*B[0] + B[2] + 3*B[4]
sage: phi.preimage(_)
B[2] + 3*B[4]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, [Integer(0),Integer(1),Integer(2),
↪Integer(3),Integer(4)]); x = X.basis()
>>> from sage.modules.with_basis.morphism import␣
↪TriangularModuleMorphismFromFunction
>>> def f(x): return x + X.term(Integer(0), sum(x.coefficients()))
>>> phi = X.module_morphism(function=f, codomain=X,
...                         triangular='upper')
>>> phi(x[Integer(2)] + Integer(3)*x[Integer(4)])
4*B[0] + B[2] + 3*B[4]
>>> phi.preimage(_)
B[2] + 3*B[4]
```

For details and further optional arguments, see `sage.modules.with_basis.morphism.TriangularModuleMorphism`.

Warning

As a temporary measure, until multivariate morphisms are implemented, the constructed morphism is in `Hom(codomain, domain, category)`. This is only correct for unary functions.

Todo

- Should codomain be `self` by default in the diagonal, triangular, and matrix cases?
- Support for diagonal morphisms between modules not sharing the same index set

monomial(*i*)

Return the basis element indexed by `i`.

INPUT:

- `i` – an element of the index set

EXAMPLES:

```
sage: F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                #␣
↪needs sage.modules
sage: F.monomial('a')                                                 #␣
↪needs sage.modules
B['a']
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                  #␣
↪needs sage.modules
>>> F.monomial('a')                                                   #␣
↪needs sage.modules
B['a']
```

`F.monomial` is in fact (almost) a map:

```
sage: F.monomial                                                      #␣
↪needs sage.modules
Term map from {'a', 'b', 'c'}
 to Free module generated by {'a', 'b', 'c'} over Rational Field
```

```
>>> from sage.all import *
>>> F.monomial                                                        #␣
↪needs sage.modules
Term map from {'a', 'b', 'c'}
 to Free module generated by {'a', 'b', 'c'} over Rational Field
```

monomial_or_zero_if_none(*i*)

EXAMPLES:

```
sage: F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                #␣
↪needs sage.modules
sage: F.monomial_or_zero_if_none('a')                                 #␣
↪needs sage.modules
B['a']
sage: F.monomial_or_zero_if_none(None)                                #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                  #␣
↪needs sage.modules
>>> F.monomial_or_zero_if_none('a')                                   #␣
↪needs sage.modules
B['a']
>>> F.monomial_or_zero_if_none(None)                                  #␣
↪needs sage.modules
0
```

quotient_module(*submodule*, *check=True*, *already_echelonized=False*, *category=None*)

Construct the quotient module `self` / `submodule`.

INPUT:

- `submodule` – a submodule with basis of `self`, or something that can be turned into one via `self.submodule(submodule)`
- `check`, `already_echelonized` – passed down to *submodule()*

Warning

At this point, this only supports quotients by free submodules admitting a basis in unitriangular echelon form. In this case, the quotient is also a free module, with a basis consisting of the retract of a subset of the basis of `self`.

EXAMPLES:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, range(3), prefix='x')
sage: x = X.basis()
sage: Y = X.quotient_module([x[0] - x[1], x[1] - x[2]],
....:                       already_echelonized=True)
sage: Y.print_options(prefix='y'); Y
Free module generated by {2} over Rational Field
sage: y = Y.basis()
sage: y[2]
y[2]
sage: y[2].lift()
x[2]
sage: Y.retract(x[0] + 2*x[1])
3*y[2]

sage: # needs sage.modules
sage: R.<a,b> = QQ[]
sage: C = CombinatorialFreeModule(R, range(3), prefix='x')
sage: x = C.basis()
sage: gens = [x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]]
sage: Y = C.quotient_module(gens)
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, range(Integer(3)), prefix='x')
>>> x = X.basis()
>>> Y = X.quotient_module([x[Integer(0)] - x[Integer(1)], x[Integer(1)] -␣
↪x[Integer(2)]],
...                       already_echelonized=True)
>>> Y.print_options(prefix='y'); Y
Free module generated by {2} over Rational Field
>>> y = Y.basis()
>>> y[Integer(2)]
y[2]
>>> y[Integer(2)].lift()
x[2]
>>> Y.retract(x[Integer(0)] + Integer(2)*x[Integer(1)])
3*y[2]

>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> R = QQ['a, b']; (a, b,) = R._first_ngens(2)
>>> C = CombinatorialFreeModule(R, range(Integer(3)), prefix='x')
>>> x = C.basis()
>>> gens = [x[Integer(0)] - x[Integer(1)], Integer(2)*x[Integer(1)] -␣
↪Integer(2)*x[Integer(2)], x[Integer(0)] - x[Integer(2)]]
>>> Y = C.quotient_module(gens)
```

See also

- *submodule()*
- *Rings.ParentMethods.quotient()*
- sage.modules.with_basis.subquotient.QuotientModuleWithBasis

random_element(*n=2*)

Return a 'random' element of `self`.

INPUT:

- `n` – integer (default: 2); number of summands

ALGORITHM:

Return a sum of n terms, each of which is formed by multiplying a random element of the base ring by a random element of the group.

EXAMPLES:

```
sage: x = DihedralGroup(6).algebra(QQ).random_element()                    #␣
↪needs sage.groups sage.modules
sage: x.parent() is DihedralGroup(6).algebra(QQ)                           #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> x = DihedralGroup(Integer(6)).algebra(QQ).random_element()              ␣
↪     # needs sage.groups sage.modules
>>> x.parent() is DihedralGroup(Integer(6)).algebra(QQ)                     ␣
↪     # needs sage.groups sage.modules
True
```

Note, this result can depend on the PRNG state in libgap in a way that depends on which packages are loaded, so we must re-seed GAP to ensure a consistent result for this example:

```
sage: libgap.set_seed(0)                                                   #␣
↪needs sage.libs.gap
0
sage: m = SU(2, 13).algebra(QQ).random_element(1)                          #␣
↪needs sage.groups sage.libs.pari sage.modules
sage: m.parent() is SU(2, 13).algebra(QQ)                                  #␣
↪needs sage.groups sage.libs.pari sage.modules
True
sage: p = CombinatorialFreeModule(ZZ, Partitions(4)).random_element()     #␣
↪needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
sage: p.parent() is CombinatorialFreeModule(ZZ, Partitions(4))          #␣
↪needs sage.combinat sage.modules
True
```

```
>>> from sage.all import *
>>> libgap.set_seed(Integer(0))                                          ␣
↪     # needs sage.libs.gap
0
>>> m = SU(Integer(2), Integer(13)).algebra(QQ).random_
↪element(Integer(1))                          # needs sage.groups sage.libs.
↪pari sage.modules
>>> m.parent() is SU(Integer(2), Integer(13)).algebra(QQ)                ␣
↪              # needs sage.groups sage.libs.pari sage.modules
True
>>> p = CombinatorialFreeModule(ZZ, Partitions(Integer(4))).random_
↪element()    # needs sage.combinat sage.modules
>>> p.parent() is CombinatorialFreeModule(ZZ, Partitions(Integer(4)))    ␣
↪     # needs sage.combinat sage.modules
True
```

rank()

Return the rank of `self`.

Since there is a (distinguished) basis, the rank of `self` is equal to the cardinality of the basis (which equals the *dimension()* of `self`).

EXAMPLES:

```
sage: A.<x,y> = algebras.DifferentialWeyl(QQ)                           #␣
↪needs sage.modules
sage: A.rank()                                                          #␣
↪needs sage.modules
+Infinity

sage: R.<x,y> = QQ[]
sage: R.rank()
+Infinity

sage: F = CombinatorialFreeModule(QQ, ['a','b','c'])
sage: F.rank()
3
```

```
>>> from sage.all import *
>>> A = algebras.DifferentialWeyl(QQ, names=('x', 'y',)); (x, y,) = A._
↪first_ngens(2)# needs sage.modules
>>> A.rank()                                                             #␣
↪needs sage.modules
+Infinity

>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.rank()
+Infinity
```

(continues on next page)

(continued from previous page)

```
>>> F = CombinatorialFreeModule(QQ, ['a','b','c'])
>>> F.rank()
3
```

submodule(*gens*, *check=True*, *already_echelonized=False*, *unitriangular=False*, *support_order=None*, *category=None*, *submodule_class=None*, **args*, ***opts*)

The submodule spanned by a finite set of elements.

INPUT:

- `gens` – list or family of elements of `self`
- `check` – boolean (default: `True`); whether to verify that the elements of `gens` are in `self`
- **`already_echelonized` – boolean (default: `False`); whether** the elements of `gens` are already in (not necessarily reduced) echelon form
- `unitriangular` – boolean (default: `False`); whether the lift morphism is unitriangular
- `support_order` – (optional) either something that can be converted into a tuple or a key function
- `category` – (optional) the category of the submodule
- `submodule_class` – (optional) the class of the submodule to return

If `already_echelonized` is `False`, then the generators are put in reduced echelon form using `echelonize()`, and reindexed by $0, 1, \ldots$.

Warning

At this point, this method only works for finite dimensional submodules and if matrices can be echelonized over the base ring.

If in addition `unitriangular` is `True`, then the generators are made such that the coefficients of the pivots are 1, so that lifting map is unitriangular.

The basis of the submodule uses the same index set as the generators, and the lifting map sends y_i to $gens[i]$.

See also

- *quotient_module()*
- `sage.modules.with_basis.subquotient.SubmoduleWithBasis`

EXAMPLES:

We construct a submodule of the free $\mathbf{Q}$-module generated by x_0, x_1, x_2. The submodule is spanned by $y_0 = x_0 - x_1$ and $y_1 - x_1 - x_2$, and its basis elements are indexed by 0 and 1:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, range(3), prefix='x')
sage: x = X.basis()
sage: gens = [x[0] - x[1], x[1] - x[2]]; gens
[x[0] - x[1], x[1] - x[2]]
sage: Y = X.submodule(gens, already_echelonized=True)
sage: Y.print_options(prefix='y'); Y
Free module generated by {0, 1} over Rational Field
```

(continues on next page)

(continued from previous page)

```
sage: y = Y.basis()
sage: y[1]
y[1]
sage: y[1].lift()
x[1] - x[2]
sage: Y.retract(x[0] - x[2])
y[0] + y[1]
sage: Y.retract(x[0])
Traceback (most recent call last):
...
ValueError: x[0] is not in the image
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, range(Integer(3)), prefix='x')
>>> x = X.basis()
>>> gens = [x[Integer(0)] - x[Integer(1)], x[Integer(1)] - x[Integer(2)]];
↪ gens
[x[0] - x[1], x[1] - x[2]]
>>> Y = X.submodule(gens, already_echelonized=True)
>>> Y.print_options(prefix='y'); Y
Free module generated by {0, 1} over Rational Field
>>> y = Y.basis()
>>> y[Integer(1)]
y[1]
>>> y[Integer(1)].lift()
x[1] - x[2]
>>> Y.retract(x[Integer(0)] - x[Integer(2)])
y[0] + y[1]
>>> Y.retract(x[Integer(0)])
Traceback (most recent call last):
...
ValueError: x[0] is not in the image
```

By using a family to specify a basis of the submodule, we obtain a submodule whose index set coincides with the index set of the family:

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, range(3), prefix='x')
sage: x = X.basis()
sage: gens = Family({1: x[0] - x[1], 3: x[1] - x[2]}); gens
Finite family {1: x[0] - x[1], 3: x[1] - x[2]}
sage: Y = X.submodule(gens, already_echelonized=True)
sage: Y.print_options(prefix='y'); Y
Free module generated by {1, 3} over Rational Field
sage: y = Y.basis()
sage: y[1]
y[1]
sage: y[1].lift()
x[0] - x[1]
sage: y[3].lift()
x[1] - x[2]
```

(continues on next page)

(continued from previous page)

```
sage: Y.retract(x[0] - x[2])
y[1] + y[3]
sage: Y.retract(x[0])
Traceback (most recent call last):
...
ValueError: x[0] is not in the image
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, range(Integer(3)), prefix='x')
>>> x = X.basis()
>>> gens = Family({Integer(1): x[Integer(0)] - x[Integer(1)], Integer(3):␣
↪x[Integer(1)] - x[Integer(2)]}); gens
Finite family {1: x[0] - x[1], 3: x[1] - x[2]}
>>> Y = X.submodule(gens, already_echelonized=True)
>>> Y.print_options(prefix='y'); Y
Free module generated by {1, 3} over Rational Field
>>> y = Y.basis()
>>> y[Integer(1)]
y[1]
>>> y[Integer(1)].lift()
x[0] - x[1]
>>> y[Integer(3)].lift()
x[1] - x[2]
>>> Y.retract(x[Integer(0)] - x[Integer(2)])
y[1] + y[3]
>>> Y.retract(x[Integer(0)])
Traceback (most recent call last):
...
ValueError: x[0] is not in the image
```

It is not necessary that the generators of the submodule form a basis (an explicit basis will be computed):

```
sage: # needs sage.modules
sage: X = CombinatorialFreeModule(QQ, range(3), prefix='x')
sage: x = X.basis()
sage: gens = [x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]]; gens
[x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]]
sage: Y = X.submodule(gens, already_echelonized=False)
sage: Y.print_options(prefix='y')
sage: Y
Free module generated by {0, 1} over Rational Field
sage: [b.lift() for b in Y.basis()]
[x[0] - x[2], x[1] - x[2]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> X = CombinatorialFreeModule(QQ, range(Integer(3)), prefix='x')
>>> x = X.basis()
>>> gens = [x[Integer(0)] - x[Integer(1)], Integer(2)*x[Integer(1)] -␣
↪Integer(2)*x[Integer(2)], x[Integer(0)] - x[Integer(2)]]; gens
[x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]]
```

(continues on next page)

(continued from previous page)

```
>>> Y = X.submodule(gens, already_echelonized=False)
>>> Y.print_options(prefix='y')
>>> Y
Free module generated by {0, 1} over Rational Field
>>> [b.lift() for b in Y.basis()]
[x[0] - x[2], x[1] - x[2]]
```

We now implement by hand the center of the algebra of the symmetric group S_3:

```
sage: # needs sage.combinat sage.groups sage.modules
sage: S3 = SymmetricGroup(3)
sage: S3A = S3.algebra(QQ)
sage: basis = S3A.annihilator_basis(S3A.algebra_generators(),
....:                               S3A.bracket)
sage: basis
((), (1,2,3) + (1,3,2), (2,3) + (1,2) + (1,3))
sage: center = S3A.submodule(basis,
....:     category=AlgebrasWithBasis(QQ).Subobjects(),
....:     already_echelonized=True)
sage: center
Free module generated by {0, 1, 2} over Rational Field
sage: center in Algebras
True
sage: center.print_options(prefix='c')
sage: c = center.basis()
sage: c[1].lift()
(1,2,3) + (1,3,2)
sage: c[0]^2
c[0]
sage: e = 1/6 * (c[0]+c[1]+c[2])
sage: e.is_idempotent()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups sage.modules
>>> S3 = SymmetricGroup(Integer(3))
>>> S3A = S3.algebra(QQ)
>>> basis = S3A.annihilator_basis(S3A.algebra_generators(),
...                               S3A.bracket)
>>> basis
((), (1,2,3) + (1,3,2), (2,3) + (1,2) + (1,3))
>>> center = S3A.submodule(basis,
...     category=AlgebrasWithBasis(QQ).Subobjects(),
...     already_echelonized=True)
>>> center
Free module generated by {0, 1, 2} over Rational Field
>>> center in Algebras
True
>>> center.print_options(prefix='c')
>>> c = center.basis()
>>> c[Integer(1)].lift()
(1,2,3) + (1,3,2)
```

(continues on next page)

(continued from previous page)

```
>>> c[Integer(0)]**Integer(2)
c[0]
>>> e = Integer(1)/Integer(6) *␣
↪(c[Integer(0)]+c[Integer(1)]+c[Integer(2)])
>>> e.is_idempotent()
True
```

Of course, this center is best constructed using:

```
sage: center = S3A.center()                                                    #␣
↪needs sage.combinat sage.groups sage.modules
```

```
>>> from sage.all import *
>>> center = S3A.center()                                                      #␣
↪needs sage.combinat sage.groups sage.modules
```

We can also automatically construct a basis such that the lift morphism is (lower) unitriangular:

```
sage: # needs sage.modules
sage: R.<a,b> = QQ[]
sage: C = CombinatorialFreeModule(R, range(3), prefix='x')
sage: x = C.basis()
sage: gens = [x[0] - x[1], 2*x[1] - 2*x[2], x[0] - x[2]]
sage: Y = C.submodule(gens, unitriangular=True)
sage: Y.lift.matrix()
[ 1  0]
[ 0  1]
[-1 -1]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> R = QQ['a, b']; (a, b,) = R._first_ngens(2)
>>> C = CombinatorialFreeModule(R, range(Integer(3)), prefix='x')
>>> x = C.basis()
>>> gens = [x[Integer(0)] - x[Integer(1)], Integer(2)*x[Integer(1)] -␣
↪Integer(2)*x[Integer(2)], x[Integer(0)] - x[Integer(2)]]
>>> Y = C.submodule(gens, unitriangular=True)
>>> Y.lift.matrix()
[ 1  0]
[ 0  1]
[-1 -1]
```

We now construct a (finite-dimensional) submodule of an infinite dimensional free module:

```
sage: # needs sage.modules
sage: C = CombinatorialFreeModule(QQ, ZZ, prefix='z')
sage: z = C.basis()
sage: gens = [z[0] - z[1], 2*z[1] - 2*z[2], z[0] - z[2]]
sage: Y = C.submodule(gens)
sage: [Y.lift(b) for b in Y.basis()]
[z[0] - z[2], z[1] - z[2]]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> C = CombinatorialFreeModule(QQ, ZZ, prefix='z')
>>> z = C.basis()
>>> gens = [z[Integer(0)] - z[Integer(1)], Integer(2)*z[Integer(1)] -␣
↪Integer(2)*z[Integer(2)], z[Integer(0)] - z[Integer(2)]]
>>> Y = C.submodule(gens)
>>> [Y.lift(b) for b in Y.basis()]
[z[0] - z[2], z[1] - z[2]]
```

sum_of_monomials()

Return the sum of the basis elements with indices in `indices`.

INPUT:

- `indices` – list (or iterable) of indices of basis elements

EXAMPLES:

```
sage: F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                #␣
↪needs sage.modules
sage: F.sum_of_monomials(['a', 'b'])                                  #␣
↪needs sage.modules
B['a'] + B['b']

sage: F.sum_of_monomials(['a', 'b', 'a'])                             #␣
↪needs sage.modules
2*B['a'] + B['b']
```

```
>>> from sage.all import *
>>> F = CombinatorialFreeModule(QQ, ['a', 'b', 'c'])                #␣
↪needs sage.modules
>>> F.sum_of_monomials(['a', 'b'])                                  #␣
↪needs sage.modules
B['a'] + B['b']

>>> F.sum_of_monomials(['a', 'b', 'a'])                             #␣
↪needs sage.modules
2*B['a'] + B['b']
```

`F.sum_of_monomials` is in fact (almost) a map:

```
sage: F.sum_of_monomials                                              #␣
↪needs sage.modules
A map to Free module generated by {'a', 'b', 'c'} over Rational Field
```

```
>>> from sage.all import *
>>> F.sum_of_monomials                                               #␣
↪needs sage.modules
A map to Free module generated by {'a', 'b', 'c'} over Rational Field
```

sum_of_terms(*terms*)

Construct a sum of terms of `self`.

INPUT:

- `terms` – list (or iterable) of pairs `(index, coeff)`

OUTPUT:

Sum of `coeff * B[index]` over all `(index, coeff)` in `terms`, where `B` is the basis of `self`.

EXAMPLES:

```
sage: m = matrix([[0,1], [1,1]])                                    #␣
↪needs sage.modules
sage: J.<a,b,c> = JordanAlgebra(m)                                   #␣
↪needs sage.combinat sage.modules
sage: J.sum_of_terms([(0, 2), (2, -3)])                             #␣
↪needs sage.combinat sage.modules
2 + (0, -3)
```

```
>>> from sage.all import *
>>> m = matrix([[Integer(0),Integer(1)], [Integer(1),Integer(1)]])  ␣
↪                                  # needs sage.modules
>>> J = JordanAlgebra(m, names=('a', 'b', 'c',)); (a, b, c,) = J._first_
↪ngens(3)# needs sage.combinat sage.modules
>>> J.sum_of_terms([(Integer(0), Integer(2)), (Integer(2), -Integer(3))])␣
↪                                  # needs sage.combinat sage.modules
2 + (0, -3)
```

tensor(**parents*, ***kwargs*)

Return the tensor product of the parents.

EXAMPLES:

```
sage: C = AlgebrasWithBasis(QQ)
sage: A = C.example(); A.rename('A')                                 #␣
↪needs sage.combinat sage.modules
sage: A.tensor(A, A)                                                 #␣
↪needs sage.combinat sage.modules
A # A # A
sage: A.rename(None)                                                 #␣
↪needs sage.combinat sage.modules
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(QQ)
>>> A = C.example(); A.rename('A')                                   #␣
↪needs sage.combinat sage.modules
>>> A.tensor(A, A)                                                   #␣
↪needs sage.combinat sage.modules
A # A # A
>>> A.rename(None)                                                   #␣
↪needs sage.combinat sage.modules
```

term(*index*, *coeff=None*)

Construct a term in `self`.

INPUT:

- `index` – the index of a basis element
- `coeff` – an element of the coefficient ring (default: one)

OUTPUT: `coeff * B[index]`, where `B` is the basis of `self`

EXAMPLES:

```
sage: m = matrix([[0,1], [1,1]])                                              #␣
↪needs sage.modules
sage: J.<a,b,c> = JordanAlgebra(m)                                            #␣
↪needs sage.combinat sage.modules
sage: J.term(1, -2)                                                           #␣
↪needs sage.combinat sage.modules
0 + (-2, 0)
```

```
>>> from sage.all import *
>>> m = matrix([[Integer(0),Integer(1)], [Integer(1),Integer(1)]])       ␣
↪                                  # needs sage.modules
>>> J = JordanAlgebra(m, names=('a', 'b', 'c',)); (a, b, c,) = J._first_
↪ngens(3)# needs sage.combinat sage.modules
>>> J.term(Integer(1), -Integer(2))                                      ␣
↪              # needs sage.combinat sage.modules
0 + (-2, 0)
```

Design: should this do coercion on the coefficient ring?

Super

alias of *SuperModulesWithBasis*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of modules with basis constructed by tensor product of modules with basis.

class ElementMethods

Bases: object

Implement operations on elements of tensor products of modules with basis.

apply_multilinear_morphism(*f*, *codomain=None*)

Return the result of applying the morphism induced by `f` to `self`.

INPUT:

- `f` – a multilinear morphism from the component modules of the parent tensor product to any module
- `codomain` – the codomain of `f` (optional)

By the universal property of the tensor product, `f` induces a linear morphism from $self.parent()$ to the target module. Returns the result of applying that morphism to `self`.

The codomain is used for optimizations purposes only. If it's not provided, it's recovered by calling `f` on the zero input.

EXAMPLES:

We start with simple (admittedly not so interesting) examples, with two modules A and B:

```
sage: # needs sage.modules
sage: A = CombinatorialFreeModule(ZZ, [1,2], prefix='A')
sage: A.rename('A')
sage: B = CombinatorialFreeModule(ZZ, [3,4], prefix='B')
sage: B.rename('B')
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = CombinatorialFreeModule(ZZ, [Integer(1),Integer(2)], prefix='A
↪')
>>> A.rename('A')
>>> B = CombinatorialFreeModule(ZZ, [Integer(3),Integer(4)], prefix='B
↪')
>>> B.rename('B')
```

and f the bilinear morphism $(a, b) \mapsto b \otimes a$ from $A \times B$ to $B \otimes A$:

```
sage: def f(a, b):
....:     return tensor([b,a])
```

```
>>> from sage.all import *
>>> def f(a, b):
...     return tensor([b,a])
```

Now, calling applying f on $a \otimes b$ returns the same as $f(a, b)$:

```
sage: # needs sage.modules
sage: a = A.monomial(1) + 2 * A.monomial(2); a
A[1] + 2*A[2]
sage: b = B.monomial(3) - 2 * B.monomial(4); b
B[3] - 2*B[4]
sage: f(a, b)
B[3] # A[1] + 2*B[3] # A[2] - 2*B[4] # A[1] - 4*B[4] # A[2]
sage: tensor([a, b]).apply_multilinear_morphism(f)
B[3] # A[1] + 2*B[3] # A[2] - 2*B[4] # A[1] - 4*B[4] # A[2]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> a = A.monomial(Integer(1)) + Integer(2) * A.monomial(Integer(2)); a
A[1] + 2*A[2]
>>> b = B.monomial(Integer(3)) - Integer(2) * B.monomial(Integer(4)); b
B[3] - 2*B[4]
>>> f(a, b)
B[3] # A[1] + 2*B[3] # A[2] - 2*B[4] # A[1] - 4*B[4] # A[2]
>>> tensor([a, b]).apply_multilinear_morphism(f)
B[3] # A[1] + 2*B[3] # A[2] - 2*B[4] # A[1] - 4*B[4] # A[2]
```

f may be a bilinear morphism to any module over the base ring of A and B. Here the codomain is $\mathbf{Z}$:

```
sage: def f(a, b):
....:     return sum(a.coefficients(), 0) * sum(b.coefficients(), 0)
sage: f(a, b)                                                       #␣
↪needs sage.modules
-3
sage: tensor([a, b]).apply_multilinear_morphism(f)                  #␣
↪needs sage.modules
-3
```

```
>>> from sage.all import *
>>> def f(a, b):
...     return sum(a.coefficients(), Integer(0)) * sum(b.
↪coefficients(), Integer(0))
>>> f(a, b)                                                    #␣
↪needs sage.modules
-3
>>> tensor([a, b]).apply_multilinear_morphism(f)               #␣
↪needs sage.modules
-3
```

Mind the 0 in the sums above; otherwise f would not return 0 in $\mathbf{Z}$:

```
sage: def f(a, b):
....:     return sum(a.coefficients()) * sum(b.coefficients())
sage: type(f(A.zero(), B.zero()))                                #␣
↪needs sage.modules
<... 'int'>
```

```
>>> from sage.all import *
>>> def f(a, b):
...     return sum(a.coefficients()) * sum(b.coefficients())
>>> type(f(A.zero(), B.zero()))                                 #␣
↪needs sage.modules
<... 'int'>
```

Which would be wrong and break this method:

```
sage: tensor([a, b]).apply_multilinear_morphism(f)               #␣
↪needs sage.modules
Traceback (most recent call last):
...
AttributeError: 'int' object has no attribute 'parent'...
```

```
>>> from sage.all import *
>>> tensor([a, b]).apply_multilinear_morphism(f)               #␣
↪needs sage.modules
Traceback (most recent call last):
...
AttributeError: 'int' object has no attribute 'parent'...
```

Here we consider an example where the codomain is a module with basis with a different base ring:

```
   sage: # needs sage.modules
   sage: C = CombinatorialFreeModule(QQ, [(1,3),(2,4)], prefix='C')
   sage: C.rename('C')
   sage: def f(a, b):
   ....:     return C.sum_of_terms([((1,3), QQ(a[1]*b[3])),
   ....:                            ((2,4), QQ(a[2]*b[4]))])
   sage: f(a,b)
   C[(1, 3)] - 4*C[(2, 4)]
   sage: tensor([a, b]).apply_multilinear_morphism(f)
   C[(1, 3)] - 4*C[(2, 4)]
```

(continues on next page)

(continued from previous page)

```
We conclude with a real life application, where we
check that the antipode of the Hopf algebra of
Symmetric functions on the Schur basis satisfies its
defining formula::

   sage: # needs lrcalc_python sage.combinat sage.modules
   sage: Sym = SymmetricFunctions(QQ)
   sage: s = Sym.schur()
   sage: def f(a, b): return a * b.antipode()
   sage: x = 4 * s.an_element(); x
   8*s[] + 8*s[1] + 12*s[2]
   sage: x.coproduct().apply_multilinear_morphism(f)
   8*s[]
   sage: x.coproduct().apply_multilinear_morphism(f) == x.counit()
   True
```

We recover the constant term of x, as desired.

Todo

Extract a method to linearize a multilinear morphism, and delegate the work there.

class ParentMethods

Bases: object

Implement operations on tensor products of modules with basis.

extra_super_categories()

EXAMPLES:

```
sage: ModulesWithBasis(QQ).TensorProducts().extra_super_categories()
[Category of vector spaces with basis over Rational Field]
sage: ModulesWithBasis(QQ).TensorProducts().super_categories()
[Category of tensor products of modules with basis over Rational Field,
 Category of vector spaces with basis over Rational Field,
 Category of tensor products of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).TensorProducts().extra_super_categories()
[Category of vector spaces with basis over Rational Field]
>>> ModulesWithBasis(QQ).TensorProducts().super_categories()
[Category of tensor products of modules with basis over Rational Field,
 Category of vector spaces with basis over Rational Field,
 Category of tensor products of vector spaces over Rational Field]
```

is_abelian()

Return whether this category is abelian.

This is the case if and only if the base ring is a field.

EXAMPLES:

```
sage: ModulesWithBasis(QQ).is_abelian()
True
sage: ModulesWithBasis(ZZ).is_abelian()
False
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).is_abelian()
True
>>> ModulesWithBasis(ZZ).is_abelian()
False
```

class sage.categories.modules_with_basis.**ModulesWithBasisHomset**(*X: DomainElementT*, *Y: CodomainElementT*, *category:* Category | *None* = *None*, *base: Ring* | *None* = *None*, *check: bool* = *True*)

Bases: *Homset*

The homset of morphisms between two modules with a distinguished basis.

This behaves like a generic *Homset*, except that a matrix is interpreted as the matrix of the morphism in the distinguished bases of the domain and codomain, by delegating to *module_morphism()*.

This makes `linear_transformation()` (and `Hom(X, Y)(some_matrix)`) work for vector spaces built from a combinatorial basis.

Note

The matrix is interpreted with the same convention as `linear_transformation()` and `VectorSpaceHomspace`: the rows of the matrix are the images of the basis of the domain (`side='left'` there). This corresponds to `side='right'` of *module_morphism()*. A different convention can be requested by passing `side` explicitly.

EXAMPLES:

```
sage: # needs sage.modules
sage: from sage.categories.modules_with_basis import ModulesWithBasisHomset
sage: X = CombinatorialFreeModule(QQ, [1, 2])
sage: Y = CombinatorialFreeModule(QQ, [1, 2, 3])
sage: H = Hom(X, Y)
sage: isinstance(H, ModulesWithBasisHomset)
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> from sage.categories.modules_with_basis import ModulesWithBasisHomset
>>> X = CombinatorialFreeModule(QQ, [Integer(1), Integer(2)])
>>> Y = CombinatorialFreeModule(QQ, [Integer(1), Integer(2), Integer(3)])
>>> H = Hom(X, Y)
>>> isinstance(H, ModulesWithBasisHomset)
True
```

4.126 Monoid algebras

sage.categories.monoid_algebras.**MonoidAlgebras**(*base_ring*)

The category of monoid algebras over `base_ring`.

EXAMPLES:

```
sage: C = MonoidAlgebras(QQ); C
Category of monoid algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of semigroup algebras over Rational Field,
 Category of unital magma algebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = MonoidAlgebras(QQ); C
Category of monoid algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of bialgebras with basis over Rational Field,
 Category of semigroup algebras over Rational Field,
 Category of unital magma algebras over Rational Field]
```

This is just an alias for:

```
sage: C is Monoids().Algebras(QQ)
True
```

```
>>> from sage.all import *
>>> C is Monoids().Algebras(QQ)
True
```

4.127 Monoids

class sage.categories.monoids.**Monoids**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of (multiplicative) monoids.

A *monoid* is a unital *semigroup*, that is a set endowed with a multiplicative binary operation $*$ which is associative and admits a unit (see Wikipedia article Monoid).

EXAMPLES:

```
sage: Monoids()
Category of monoids
sage: Monoids().super_categories()
[Category of semigroups, Category of unital magmas]
sage: Monoids().all_super_categories()
[Category of monoids,
 Category of semigroups,
 Category of unital magmas, Category of magmas,
 Category of sets,
 Category of sets with partial maps,
```

(continues on next page)

(continued from previous page)

```
 Category of objects]

sage: Monoids().axioms()
frozenset({'Associative', 'Unital'})
sage: Semigroups().Unital()
Category of monoids

sage: Monoids().example()
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> Monoids()
Category of monoids
>>> Monoids().super_categories()
[Category of semigroups, Category of unital magmas]
>>> Monoids().all_super_categories()
[Category of monoids,
 Category of semigroups,
 Category of unital magmas, Category of magmas,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

>>> Monoids().axioms()
frozenset({'Associative', 'Unital'})
>>> Semigroups().Unital()
Category of monoids

>>> Monoids().example()
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ElementMethods

Bases: object

is_central()

Return whether the element `self` is central.

EXAMPLES:

```
sage: SG4 = SymmetricGroupAlgebra(ZZ,4)                                 #␣
↪needs sage.groups sage.modules
sage: SG4(1).is_central()                                               #␣
↪needs sage.groups sage.modules
True
sage: SG4(Permutation([1,3,2,4])).is_central()                          #␣
↪needs sage.groups sage.modules
False

sage: A = GroupAlgebras(QQ).example(); A                                #␣
↪needs sage.groups sage.modules
```

(continues on next page)

(continued from previous page)

```
Algebra of Dihedral group of order 8
 as a permutation group over Rational Field
sage: sum(A.basis()).is_central()                                   #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> SG4 = SymmetricGroupAlgebra(ZZ,Integer(4))                      ␣
↪    # needs sage.groups sage.modules
>>> SG4(Integer(1)).is_central()                                    ␣
↪    # needs sage.groups sage.modules
True
>>> SG4(Permutation([Integer(1),Integer(3),Integer(2),Integer(4)])).is_
↪central()                      # needs sage.groups sage.modules
False

>>> A = GroupAlgebras(QQ).example(); A                              #␣
↪needs sage.groups sage.modules
Algebra of Dihedral group of order 8
 as a permutation group over Rational Field
>>> sum(A.basis()).is_central()                                     #␣
↪needs sage.groups sage.modules
True
```

class ParentMethods

Bases: object

algebra_generators()

Return generators for this algebra.

For a monoid algebra, the algebra generators are built from the monoid generators if available and from the semigroup generators otherwise.

See also

- *Semigroups.Algebras.ParentMethods.algebra_generators()*
- *MagmaticAlgebras.ParentMethods.algebra_generators()*.

EXAMPLES:

```
sage: M = Monoids().example(); M
An example of a monoid:
the free monoid generated by ('a', 'b', 'c', 'd')
sage: M.monoid_generators()
Finite family {'a': 'a', 'b': 'b', 'c': 'c', 'd': 'd'}
sage: M.algebra(ZZ).algebra_generators()                            #␣
↪needs sage.modules
Finite family {'a': B['a'], 'b': B['b'], 'c': B['c'], 'd': B['d']}

sage: Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid:
```

(continues on next page)

(continued from previous page)

```
the integers modulo 12
sage: Z12.monoid_generators()
Traceback (most recent call last):
...
AttributeError: 'IntegerModMonoid_with_category' object
has no attribute 'monoid_generators'...
sage: Z12.semigroup_generators()
Family (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
sage: Z12.algebra(QQ).algebra_generators()                                  #␣
↪needs sage.modules
Family (B[0], B[1], B[2], B[3], B[4], B[5], B[6], B[7], B[8], B[9],␣
↪B[10], B[11])


sage: A10 = AlternatingGroup(10)                                            #␣
↪needs sage.groups
sage: GroupAlgebras(QQ).example(A10).algebra_generators()                   #␣
↪needs sage.groups sage.modules
Family ((1,2,3,4,5,6,7,8,9), (8,9,10))

sage: A = DihedralGroup(3).algebra(QQ); A                                   #␣
↪needs sage.groups sage.modules
Algebra of Dihedral group of order 6 as a permutation group
 over Rational Field
sage: A.algebra_generators()                                                #␣
↪needs sage.groups sage.modules
Family ((1,2,3), (1,3))
```

```
>>> from sage.all import *
>>> M = Monoids().example(); M
An example of a monoid:
the free monoid generated by ('a', 'b', 'c', 'd')
>>> M.monoid_generators()
Finite family {'a': 'a', 'b': 'b', 'c': 'c', 'd': 'd'}
>>> M.algebra(ZZ).algebra_generators()                                      #␣
↪needs sage.modules
Finite family {'a': B['a'], 'b': B['b'], 'c': B['c'], 'd': B['d']}

>>> Z12 = Monoids().Finite().example(); Z12
An example of a finite multiplicative monoid:
the integers modulo 12
>>> Z12.monoid_generators()
Traceback (most recent call last):
...
AttributeError: 'IntegerModMonoid_with_category' object
has no attribute 'monoid_generators'...
>>> Z12.semigroup_generators()
Family (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
>>> Z12.algebra(QQ).algebra_generators()                                    #␣
↪needs sage.modules
Family (B[0], B[1], B[2], B[3], B[4], B[5], B[6], B[7], B[8], B[9],␣
↪B[10], B[11])
```

(continues on next page)

(continued from previous page)

```
>>> A10 = AlternatingGroup(Integer(10))                                    ␣
↪    # needs sage.groups
>>> GroupAlgebras(QQ).example(A10).algebra_generators()                  #␣
↪needs sage.groups sage.modules
Family ((1,2,3,4,5,6,7,8,9), (8,9,10))

>>> A = DihedralGroup(Integer(3)).algebra(QQ); A                           ␣
↪    # needs sage.groups sage.modules
Algebra of Dihedral group of order 6 as a permutation group
 over Rational Field
>>> A.algebra_generators()                                               #␣
↪needs sage.groups sage.modules
Family ((1,2,3), (1,3))
```

is_field(*proof=True*)

Return `True` if `self` is a field.

For a monoid algebra RS this is always false unless S is trivial and the base ring R is a field.

EXAMPLES:

```
sage: SymmetricGroup(1).algebra(QQ).is_field()                             #␣
↪needs sage.combinat sage.groups
True
sage: SymmetricGroup(1).algebra(ZZ).is_field()                             #␣
↪needs sage.combinat sage.groups
False
sage: SymmetricGroup(2).algebra(QQ).is_field()                             #␣
↪needs sage.combinat sage.groups
False
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(1)).algebra(QQ).is_field()                      ␣
↪    # needs sage.combinat sage.groups
True
>>> SymmetricGroup(Integer(1)).algebra(ZZ).is_field()                      ␣
↪    # needs sage.combinat sage.groups
False
>>> SymmetricGroup(Integer(2)).algebra(QQ).is_field()                      ␣
↪    # needs sage.combinat sage.groups
False
```

one_basis()

Return the unit of the monoid, which indexes the unit of this algebra, as per *one_basis()*.

EXAMPLES:

```
sage: # needs sage.modules
sage: A = Monoids().example().algebra(ZZ)
sage: A.one_basis()
''
```

(continues on next page)

(continued from previous page)

```
sage: A.one()
B['']
sage: A(3)
3*B['']
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = Monoids().example().algebra(ZZ)
>>> A.one_basis()
''
>>> A.one()
B['']
>>> A(Integer(3))
3*B['']
```

extra_super_categories()

The algebra of a monoid is a bialgebra and a monoid.

EXAMPLES:

```
sage: C = Monoids().Algebras(QQ)
sage: C.extra_super_categories()
[Category of bialgebras over Rational Field,
 Category of monoids]
sage: Monoids().Algebras(QQ).super_categories()
[Category of bialgebras with basis over Rational Field,
 Category of semigroup algebras over Rational Field,
 Category of unital magma algebras over Rational Field]
```

```
>>> from sage.all import *
>>> C = Monoids().Algebras(QQ)
>>> C.extra_super_categories()
[Category of bialgebras over Rational Field,
 Category of monoids]
>>> Monoids().Algebras(QQ).super_categories()
[Category of bialgebras with basis over Rational Field,
 Category of semigroup algebras over Rational Field,
 Category of unital magma algebras over Rational Field]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of monoids constructed as Cartesian products of monoids.

This construction gives the direct product of monoids. See Wikipedia article Direct_product for more information.

class ElementMethods

Bases: object

multiplicative_order()

Return the multiplicative order of this element.

EXAMPLES:

```
sage: # needs sage.groups sage.modules
sage: G1 = SymmetricGroup(3)
sage: G2 = SL(2, 3)
sage: G = cartesian_product([G1, G2])
sage: G((G1.gen(0), G2.gen(1))).multiplicative_order()
12
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> G1 = SymmetricGroup(Integer(3))
>>> G2 = SL(Integer(2), Integer(3))
>>> G = cartesian_product([G1, G2])
>>> G((G1.gen(Integer(0)), G2.gen(Integer(1)))).multiplicative_order()
12
```

class ParentMethods

Bases: object

monoid_generators()

Return the generators of self.

EXAMPLES:

```
sage: # needs sage.combinat sage.groups
sage: M = Monoids.free([1, 2, 3])
sage: N = Monoids.free(['a', 'b'])
sage: C = cartesian_product([M, N])
sage: C.monoid_generators()
Family ((F[1], 1), (F[2], 1), (F[3], 1),
        (1, F['a']), (1, F['b']))
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.groups
>>> M = Monoids.free([Integer(1), Integer(2), Integer(3)])
>>> N = Monoids.free(['a', 'b'])
>>> C = cartesian_product([M, N])
>>> C.monoid_generators()
Family ((F[1], 1), (F[2], 1), (F[3], 1),
        (1, F['a']), (1, F['b']))
```

An example with an infinitely generated group (a better output is needed):

```
sage: N = Monoids.free(ZZ)                                                  #␣
↪needs sage.combinat
sage: C = cartesian_product([M, N])                                         #␣
↪needs sage.combinat sage.groups
sage: C.monoid_generators()                                                 #␣
↪needs sage.combinat sage.groups
Lazy family (gen(i))_{i in The Cartesian product of (...)}
```

```
>>> from sage.all import *
>>> N = Monoids.free(ZZ)                                                    #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
>>> C = cartesian_product([M, N])                                    #␣
↪needs sage.combinat sage.groups
>>> C.monoid_generators()                                            #␣
↪needs sage.combinat sage.groups
Lazy family (gen(i))_{i in The Cartesian product of (...)}
```

extra_super_categories()

A Cartesian product of monoids is endowed with a natural group structure.

EXAMPLES:

```
sage: C = Monoids().CartesianProducts()
sage: C.extra_super_categories()
[Category of monoids]
sage: sorted(C.super_categories(), key=str)
[Category of Cartesian products of semigroups,
 Category of Cartesian products of unital magmas,
 Category of monoids]
```

```
>>> from sage.all import *
>>> C = Monoids().CartesianProducts()
>>> C.extra_super_categories()
[Category of monoids]
>>> sorted(C.super_categories(), key=str)
[Category of Cartesian products of semigroups,
 Category of Cartesian products of unital magmas,
 Category of monoids]
```

class Commutative(*base_category*)

Bases: *CategoryWithAxiom_singleton*

Category of commutative (abelian) monoids.

A monoid M is *commutative* if $xy = yx$ for all $x, y \in M$.

static free(*index_set=None*, *names=None*, ***kwds*)

Return a free abelian monoid on n generators or with the generators indexed by a set I.

A free monoid is constructed by specifying either:

- the number of generators and/or the names of the generators, or
- the indexing set for the generators.

INPUT:

- `index_set` – (optional) an index set for the generators; if an integer, then this represents $\{0, 1, \ldots, n-1\}$
- `names` – string or list/tuple/iterable of strings (default: `'x'`); the generator names or name prefix

EXAMPLES:

```
sage: Monoids.Commutative.free(index_set=ZZ)                           #␣
↪needs sage.combinat
Free abelian monoid indexed by Integer Ring
sage: Monoids().Commutative().free(ZZ)                                 #␣
↪needs sage.combinat
Free abelian monoid indexed by Integer Ring
sage: F.<x,y,z> = Monoids().Commutative().free(); F                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
Free abelian monoid indexed by {'x', 'y', 'z'}
```

```
>>> from sage.all import *
>>> Monoids.Commutative.free(index_set=ZZ)                                  #␣
↪needs sage.combinat
Free abelian monoid indexed by Integer Ring
>>> Monoids().Commutative().free(ZZ)                                        #␣
↪needs sage.combinat
Free abelian monoid indexed by Integer Ring
>>> F = Monoids().Commutative().free(names=('x', 'y', 'z',)); (x, y, z,)␣
↪= F._first_ngens(3); F                        # needs sage.combinat
Free abelian monoid indexed by {'x', 'y', 'z'}
```

class ElementMethods

Bases: object

inverse()

Return the multiplicative inverse of self.

This is an alias for inversion, which can also be invoked by ~x for an element x.

Nota Bene: Element classes should implement __invert__ only.

EXAMPLES:

```
sage: AA(sqrt(~2)).inverse()                                                #␣
↪needs sage.rings.number_field sage.symbolic
1.414213562373095?
```

```
>>> from sage.all import *
>>> AA(sqrt(~Integer(2))).inverse()                                          ␣
↪     # needs sage.rings.number_field sage.symbolic
1.414213562373095?
```

is_one()

Return whether self is the one of the monoid.

The default implementation is to compare with self.one().

powers(*n*)

Return the list $[x^0, x^1, \ldots, x^{n-1}]$.

EXAMPLES:

```
sage: A = Matrix([[1, 1], [-1, 0]])                                         #␣
↪needs sage.modules
sage: A.powers(6)                                                           #␣
↪needs sage.modules
[
[1 0]  [ 1  1]  [ 0  1]  [-1  0]  [-1 -1]  [ 0 -1]
[0 1], [-1  0], [-1 -1], [ 0 -1], [ 1  0], [ 1  1]
]
```

```
>>> from sage.all import *
>>> A = Matrix([[Integer(1), Integer(1)], [-Integer(1), Integer(0)]])
↪                                  # needs sage.modules
>>> A.powers(Integer(6))
↪      # needs sage.modules
[
[1 0]  [ 1  1]  [ 0  1]  [-1  0]  [-1 -1]  [ 0 -1]
[0 1], [-1  0], [-1 -1], [ 0 -1], [ 1  0], [ 1  1]
]
```

Finite

alias of *FiniteMonoids*

Inverse

alias of *Groups*

class ParentMethods

Bases: object

prod(*args*)

n-ary product of elements of self.

INPUT:

- args – list (or iterable) of elements of self

Returns the product of the elements in args, as an element of self.

EXAMPLES:

```
sage: S = Monoids().example()
sage: S.prod([S('a'), S('b')])
'ab'
```

```
>>> from sage.all import *
>>> S = Monoids().example()
>>> S.prod([S('a'), S('b')])
'ab'
```

semigroup_generators()

Return the generators of self as a semigroup.

The generators of a monoid M as a semigroup are the generators of M as a monoid and the unit.

EXAMPLES:

```
sage: M = Monoids().free([1,2,3])                                       #␣
↪needs sage.combinat
sage: M.semigroup_generators()                                          #␣
↪needs sage.combinat
Family (1, F[1], F[2], F[3])
```

```
>>> from sage.all import *
>>> M = Monoids().free([Integer(1),Integer(2),Integer(3)])
↪                        # needs sage.combinat
>>> M.semigroup_generators()                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat
Family (1, F[1], F[2], F[3])
```

submonoid(*generators*, *category=None*)

Return the multiplicative submonoid generated by `generators`.

INPUT:

- `generators` – a finite family of elements of `self`, or a list, iterable, … that can be converted into one (see `Family()`).
- `category` – a category

This is a shorthand for *Semigroups.ParentMethods.subsemigroup()* that specifies that this is a submonoid, and in particular that the unit is `self.one()`.

EXAMPLES:

```
sage: R = IntegerModRing(15)
sage: M = R.submonoid([R(3), R(5)]); M                                  #␣
↪needs sage.combinat
A submonoid of (Ring of integers modulo 15) with 2 generators
sage: M.list()                                                          #␣
↪needs sage.combinat
[1, 3, 5, 9, 0, 10, 12, 6]
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(15))
>>> M = R.submonoid([R(Integer(3)), R(Integer(5))]); M                  ␣
↪              # needs sage.combinat
A submonoid of (Ring of integers modulo 15) with 2 generators
>>> M.list()                                                           #␣
↪needs sage.combinat
[1, 3, 5, 9, 0, 10, 12, 6]
```

Not the presence of the unit, unlike in:

```
sage: S = R.subsemigroup([R(3), R(5)]); S                               #␣
↪needs sage.combinat
A subsemigroup of (Ring of integers modulo 15) with 2 generators
sage: S.list()                                                          #␣
↪needs sage.combinat
[3, 5, 9, 0, 10, 12, 6]
```

```
>>> from sage.all import *
>>> S = R.subsemigroup([R(Integer(3)), R(Integer(5))]); S               ␣
↪              # needs sage.combinat
A subsemigroup of (Ring of integers modulo 15) with 2 generators
>>> S.list()                                                           #␣
↪needs sage.combinat
[3, 5, 9, 0, 10, 12, 6]
```

This method is really a shorthand for subsemigroup:

```
sage: M2 = R.subsemigroup([R(3), R(5)], one=R.one())                    #␣
↪needs sage.combinat
```

(continues on next page)

(continued from previous page)

```
sage: M2 is M                                                                  #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> M2 = R.subsemigroup([R(Integer(3)), R(Integer(5))], one=R.one())      ␣
↪              # needs sage.combinat
>>> M2 is M                                                                 #␣
↪needs sage.combinat
True
```

class Subquotients(*category*, **args*)

Bases: *SubquotientsCategory*

class ParentMethods

Bases: object

one()

Return the multiplicative unit of this monoid, obtained by retracting that of the ambient monoid.

EXAMPLES:

```
sage: S = Monoids().Subquotients().example()  # todo: not implemented
sage: S.one()                                 # todo: not implemented
```

```
>>> from sage.all import *
>>> S = Monoids().Subquotients().example()  # todo: not implemented
>>> S.one()                                 # todo: not implemented
```

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

class ParentMethods

Bases: object

one()

Return the unit of this monoid.

This default implementation returns the unit of the realization of `self` given by *a_realization()*.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                               #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.one.__module__                                                         #␣
↪needs sage.modules
'sage.categories.monoids'
sage: A.one()                                                                  #␣
↪needs sage.modules
F[{}]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                                  #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.one.__module__                                                            #␣
↪needs sage.modules
'sage.categories.monoids'
>>> A.one()                                                                     #␣
↪needs sage.modules
F[{}]
```

static free(*index_set=None*, *names=None*, ***kwds*)

Return a free monoid on n generators or with the generators indexed by a set I.

A free monoid is constructed by specifying either:

- the number of generators and/or the names of the generators
- the indexing set for the generators

INPUT:

- `index_set` – (optional) an index set for the generators; if an integer, then this represents $\{0, 1, \ldots, n-1\}$
- `names` – string or list/tuple/iterable of strings (default: `'x'`); the generator names or name prefix

EXAMPLES:

```
sage: Monoids.free(index_set=ZZ)                                                #␣
↪needs sage.combinat
Free monoid indexed by Integer Ring
sage: Monoids().free(ZZ)                                                        #␣
↪needs sage.combinat
Free monoid indexed by Integer Ring
sage: F.<x,y,z> = Monoids().free(); F                                           #␣
↪needs sage.combinat
Free monoid indexed by {'x', 'y', 'z'}
```

```
>>> from sage.all import *
>>> Monoids.free(index_set=ZZ)                                                  #␣
↪needs sage.combinat
Free monoid indexed by Integer Ring
>>> Monoids().free(ZZ)                                                          #␣
↪needs sage.combinat
Free monoid indexed by Integer Ring
>>> F = Monoids().free(names=('x', 'y', 'z',)); (x, y, z,) = F._first_
↪ngens(3); F                                          # needs sage.combinat
Free monoid indexed by {'x', 'y', 'z'}
```

4.128 Noetherian rings

EXAMPLES:

```
sage: from sage.categories.noetherian_rings import NoetherianRings
sage: GF(4, "a") in NoetherianRings()                                    # needs␣
↪sage.rings.finite_rings
True
sage: QQ in NoetherianRings()
True
sage: ZZ in NoetherianRings()
True
sage: IntegerModRing(4) in NoetherianRings()
True
sage: IntegerModRing(5) in NoetherianRings()
True
```

```
>>> from sage.all import *
>>> from sage.categories.noetherian_rings import NoetherianRings
>>> GF(Integer(4), "a") in NoetherianRings()                             #␣
↪needs sage.rings.finite_rings
True
>>> QQ in NoetherianRings()
True
>>> ZZ in NoetherianRings()
True
>>> IntegerModRing(Integer(4)) in NoetherianRings()
True
>>> IntegerModRing(Integer(5)) in NoetherianRings()
True
```

class sage.categories.noetherian_rings.**NoetherianRings**

Bases: *Category*

The category of Noetherian rings.

A Noetherian ring is a commutative ring in which every ideal is finitely generated.

See Wikipedia article Noetherian ring

EXAMPLES:

```
sage: from sage.categories.noetherian_rings import NoetherianRings
sage: C = NoetherianRings(); C
Category of noetherian rings
sage: sorted(C.super_categories(), key=str)
[Category of commutative rings]
```

```
>>> from sage.all import *
>>> from sage.categories.noetherian_rings import NoetherianRings
>>> C = NoetherianRings(); C
Category of noetherian rings
>>> sorted(C.super_categories(), key=str)
[Category of commutative rings]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

is_noetherian(*proof=True*)

Return True, since this in an object of the category of Noetherian rings.

EXAMPLES:

```
sage: ZZ.is_noetherian()
True
sage: QQ.is_noetherian()
True
sage: ZZ['x'].is_noetherian()
True
sage: R.<x> = PolynomialRing(QQ)
sage: R.is_noetherian()
True

sage: L.<z> = LazyLaurentSeriesRing(QQ)                                    #␣
↪needs sage.combinat
sage: L.is_noetherian()                                                    #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> ZZ.is_noetherian()
True
>>> QQ.is_noetherian()
True
>>> ZZ['x'].is_noetherian()
True
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> R.is_noetherian()
True

>>> L = LazyLaurentSeriesRing(QQ, names=('z',)); (z,) = L._first_ngens(1)
↪# needs sage.combinat
>>> L.is_noetherian()                                                      # needs␣
↪sage.combinat
True
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.noetherian_rings import NoetherianRings
sage: NoetherianRings().super_categories()
[Category of commutative rings]
```

```
>>> from sage.all import *
>>> from sage.categories.noetherian_rings import NoetherianRings
>>> NoetherianRings().super_categories()
[Category of commutative rings]
```

4.129 Number fields

class sage.categories.number_fields.**NumberFields**

Bases: *Category_singleton*

The category of number fields.

EXAMPLES:

We create the category of number fields:

```
sage: C = NumberFields()
sage: C
Category of number fields
```

```
>>> from sage.all import *
>>> C = NumberFields()
>>> C
Category of number fields
```

By definition, it is infinite:

```
sage: NumberFields().Infinite() is NumberFields()
True
```

```
>>> from sage.all import *
>>> NumberFields().Infinite() is NumberFields()
True
```

Notice that the rational numbers $\mathbf{Q}$ *are* considered as an object in this category:

```
sage: RationalField() in C
True
```

```
>>> from sage.all import *
>>> RationalField() in C
True
```

However, we can define a degree 1 extension of $\mathbf{Q}$, which is of course also in this category:

```
sage: x = PolynomialRing(RationalField(), 'x').gen()
sage: K = NumberField(x - 1, 'a'); K                                    #␣
↪needs sage.rings.number_field
Number Field in a with defining polynomial x - 1
sage: K in C                                                            #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> x = PolynomialRing(RationalField(), 'x').gen()
>>> K = NumberField(x - Integer(1), 'a'); K                              ␣
↪     # needs sage.rings.number_field
Number Field in a with defining polynomial x - 1
>>> K in C                                                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.number_field
True
```

Number fields all lie in this category, regardless of the name of the variable:

```
sage: K = NumberField(x^2 + 1, 'a')                                    #␣
↪needs sage.rings.number_field
sage: K in C                                                           #␣
↪needs sage.rings.number_field
True
```

```
>>> from sage.all import *
>>> K = NumberField(x**Integer(2) + Integer(1), 'a')                    ␣
↪               # needs sage.rings.number_field
>>> K in C                                                            #␣
↪needs sage.rings.number_field
True
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

zeta_function(*prec=53*, *max_imaginary_part=0*, *algorithm='pari'*)

Return the Dedekind zeta function of this number field.

Actually, this returns an interface for computing with the Dedekind zeta function $\zeta_F(s)$ of the number field F.

INPUT:

- prec – integer (default: 53); bits of precision
- max_imaginary_part – real (default: 0)
- algorithm – ignored

OUTPUT: the zeta function of this number field

This returns an interface to Pari's own general implementation of L-functions.

EXAMPLES:

```
sage: K.<a> = NumberField(ZZ['x'].0^2 + ZZ['x'].0 - 1)                 #␣
↪needs sage.rings.number_field
sage: Z = K.zeta_function(); Z                                         #␣
↪needs sage.rings.number_field sage.symbolic
PARI zeta function associated to Number Field in a
 with defining polynomial x^2 + x - 1
sage: Z(-1)                                                            #␣
↪needs sage.rings.number_field sage.symbolic
0.0333333333333333

sage: x = polygen(QQ, 'x')
sage: L.<a, b, c> = NumberField([x^2 - 5, x^2 + 3, x^2 + 1])           #␣
↪needs sage.rings.number_field
sage: Z = L.zeta_function()                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.rings.number_field sage.symbolic
sage: Z(5)                                                                  #␣
↪needs sage.rings.number_field sage.symbolic
1.00199015670185
```

```
>>> from sage.all import *
>>> K = NumberField(ZZ['x'].gen(0)**Integer(2) + ZZ['x'].gen(0) -␣
↪Integer(1), names=('a',)); (a,) = K._first_ngens(1)# needs sage.rings.
↪number_field
>>> Z = K.zeta_function(); Z                                                #␣
↪needs sage.rings.number_field sage.symbolic
PARI zeta function associated to Number Field in a
 with defining polynomial x^2 + x - 1
>>> Z(-Integer(1))                                                          ␣
↪     # needs sage.rings.number_field sage.symbolic
0.0333333333333333

>>> x = polygen(QQ, 'x')
>>> L = NumberField([x**Integer(2) - Integer(5), x**Integer(2) +␣
↪Integer(3), x**Integer(2) + Integer(1)], names=('a', 'b', 'c',)); (a, b,
↪ c,) = L._first_ngens(3)# needs sage.rings.number_field
>>> Z = L.zeta_function()                                                   #␣
↪needs sage.rings.number_field sage.symbolic
>>> Z(Integer(5))                                                           ␣
↪     # needs sage.rings.number_field sage.symbolic
1.00199015670185
```

super_categories()

EXAMPLES:

```
sage: NumberFields().super_categories()
[Category of infinite fields]
```

```
>>> from sage.all import *
>>> NumberFields().super_categories()
[Category of infinite fields]
```

4.130 Objects

class sage.categories.objects.**Objects**

Bases: *Category_singleton*

The category of all objects the basic category

EXAMPLES:

```
sage: Objects()
Category of objects
sage: Objects().super_categories()
[]
```

```
>>> from sage.all import *
>>> Objects()
Category of objects
>>> Objects().super_categories()
[]
```

class ParentMethods

Bases: object

Methods for all category objects

class SubcategoryMethods

Bases: object

Endsets()

Return the category of endsets between objects of this category.

EXAMPLES:

```
sage: Sets().Endsets()
Category of endsets of sets

sage: Rings().Endsets()
Category of endsets of unital magmas and additive unital additive magmas
```

```
>>> from sage.all import *
>>> Sets().Endsets()
Category of endsets of sets

>>> Rings().Endsets()
Category of endsets of unital magmas and additive unital additive magmas
```

See also

- *Homsets()*

Homsets()

Return the category of homsets between objects of this category.

EXAMPLES:

```
sage: Sets().Homsets()
Category of homsets of sets

sage: Rings().Homsets()
Category of homsets of unital magmas and additive unital additive magmas
```

```
>>> from sage.all import *
>>> Sets().Homsets()
Category of homsets of sets

>>> Rings().Homsets()
Category of homsets of unital magmas and additive unital additive magmas
```

Note

Background

Information, code, documentation, and tests about the category of homsets of a category `Cs` should go in the nested class `Cs.Homsets`. They will then be made available to homsets of any subcategory of `Cs`.

Assume, for example, that homsets of `Cs` are `Cs` themselves. This information can be implemented in the method `Cs.Homsets.extra_super_categories` to make `Cs.Homsets()` a subcategory of `Cs()`.

Methods about the homsets themselves should go in the nested class `Cs.Homsets.ParentMethods`.

Methods about the morphisms can go in the nested class `Cs.Homsets.ElementMethods`. However it's generally preferable to put them in the nested class `Cs.MorphimMethods`; indeed they will then apply to morphisms of all subcategories of `Cs`, and not only full subcategories.

See also

FunctorialConstructionCategory

Todo

- Design a mechanism to specify that an axiom is compatible with taking subsets. Examples: `Finite`, `Associative`, `Commutative` (when meaningful), but not `Infinite` nor `Unital`.
- Design a mechanism to specify that, when B is a subcategory of A, a B-homset is a subset of the corresponding A homset. And use it to recover all the relevant axioms from homsets in super categories.
- For instances of redundant code due to this missing feature, see:
  - *extra_super_categories()*
  - *extra_super_categories()* (slightly different nature)
  - plus plenty of spots where this is not implemented.

additional_structure()

Return `None`.

Indeed, by convention, the category of objects defines no additional structure.

See also

Category.additional_structure()

EXAMPLES:

```
sage: Objects().additional_structure()
```

```
>>> from sage.all import *
>>> Objects().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: Objects().super_categories()
[]
```

```
>>> from sage.all import *
>>> Objects().super_categories()
[]
```

4.131 Ore modules

AUTHORS:

- Xavier Caruso (2024-09): initial implementation

class sage.categories.ore_modules.**OreModules**(*ore*)

Bases: *Category_over_base_ring*

Category of Ore modules.

ore_ring(*var='x'*)

Return the underlying Ore polynomial ring.

INPUT:

- `var` (default; `x`) – the variable name

EXAMPLES:

```
sage: from sage.categories.ore_modules import OreModules
sage: K.<a> = GF(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: cat = OreModules(K, Frob)
sage: cat.ore_ring()
Ore Polynomial Ring in x over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5

sage: cat.ore_ring('y')
Ore Polynomial Ring in y over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

```
>>> from sage.all import *
>>> from sage.categories.ore_modules import OreModules
>>> K = GF(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> cat = OreModules(K, Frob)
>>> cat.ore_ring()
Ore Polynomial Ring in x over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

(continues on next page)

(continued from previous page)

```
>>> cat.ore_ring('y')
Ore Polynomial Ring in y over Finite Field in a of size 5^3 twisted by a |-->␣
↪a^5
```

super_categories()

Return the immediate super categories of this category.

EXAMPLES:

```
sage: from sage.categories.ore_modules import OreModules
sage: K.<a> = GF(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: cat = OreModules(K, Frob)
sage: cat.super_categories()
[Category of vector spaces over Finite Field in a of size 5^3]
```

```
>>> from sage.all import *
>>> from sage.categories.ore_modules import OreModules
>>> K = GF(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> cat = OreModules(K, Frob)
>>> cat.super_categories()
[Category of vector spaces over Finite Field in a of size 5^3]
```

twisting_derivation()

Return the underlying twisting derivation.

EXAMPLES:

```
sage: from sage.categories.ore_modules import OreModules
sage: R.<t> = QQ[]
sage: d = R.derivation()
sage: cat = OreModules(R, d)
sage: cat.twisting_derivation()
d/dt
```

```
>>> from sage.all import *
>>> from sage.categories.ore_modules import OreModules
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> d = R.derivation()
>>> cat = OreModules(R, d)
>>> cat.twisting_derivation()
d/dt
```

If the twising derivation is zero, nothing is returned:

```
sage: K.<a> = GF(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: cat = OreModules(K, Frob)
sage: cat.twisting_derivation()
```

```
>>> from sage.all import *
>>> K = GF(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> cat = OreModules(K, Frob)
>>> cat.twisting_derivation()
```

twisting_morphism()

Return the underlying twisting morphism.

EXAMPLES:

```
sage: from sage.categories.ore_modules import OreModules
sage: K.<a> = GF(5^3)
sage: Frob = K.frobenius_endomorphism()
sage: cat = OreModules(K, Frob)
sage: cat.twisting_morphism()
Frobenius endomorphism a |--> a^5 on Finite Field in a of size 5^3
```

```
>>> from sage.all import *
>>> from sage.categories.ore_modules import OreModules
>>> K = GF(Integer(5)**Integer(3), names=('a',)); (a,) = K._first_ngens(1)
>>> Frob = K.frobenius_endomorphism()
>>> cat = OreModules(K, Frob)
>>> cat.twisting_morphism()
Frobenius endomorphism a |--> a^5 on Finite Field in a of size 5^3
```

If the twising morphism is the identity, nothing is returned:

```
sage: R.<t> = QQ[]
sage: d = R.derivation()
sage: cat = OreModules(R, d)
sage: cat.twisting_morphism()
```

```
>>> from sage.all import *
>>> R = QQ['t']; (t,) = R._first_ngens(1)
>>> d = R.derivation()
>>> cat = OreModules(R, d)
>>> cat.twisting_morphism()
```

4.132 Partially ordered monoids

class sage.categories.partially_ordered_monoids.**PartiallyOrderedMonoids**

Bases: *Category_singleton*

The category of partially ordered monoids, that is partially ordered sets which are also monoids, and such that multiplication preserves the ordering: $x \leq y$ implies $x * z < y * z$ and $z * x < z * y$.

See Wikipedia article Ordered_monoid

EXAMPLES:

```
sage: PartiallyOrderedMonoids()
Category of partially ordered monoids
```

(continues on next page)

(continued from previous page)

```
sage: PartiallyOrderedMonoids().super_categories()
[Category of posets, Category of monoids]
```

```
>>> from sage.all import *
>>> PartiallyOrderedMonoids()
Category of partially ordered monoids
>>> PartiallyOrderedMonoids().super_categories()
[Category of posets, Category of monoids]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: PartiallyOrderedMonoids().super_categories()
[Category of posets, Category of monoids]
```

```
>>> from sage.all import *
>>> PartiallyOrderedMonoids().super_categories()
[Category of posets, Category of monoids]
```

4.133 Permutation groups

class sage.categories.permutation_groups.**PermutationGroups**

Bases: *Category*

The category of permutation groups.

A *permutation group* is a group whose elements are concretely represented by permutations of some set. In other words, the group comes endowed with a distinguished action on some set.

This distinguished action should be preserved by permutation group morphisms. For details, see Wikipedia article Permutation_group#Permutation_isomorphic_groups.

Todo

shall we accept only permutations with finite support or not?

EXAMPLES:

```
sage: PermutationGroups()
Category of permutation groups
sage: PermutationGroups().super_categories()
[Category of groups]
```

```
>>> from sage.all import *
>>> PermutationGroups()
Category of permutation groups
>>> PermutationGroups().super_categories()
[Category of groups]
```

The category of permutation groups defines additional structure that should be preserved by morphisms, namely the distinguished action:

```
sage: PermutationGroups().additional_structure()
Category of permutation groups
```

```
>>> from sage.all import *
>>> PermutationGroups().additional_structure()
Category of permutation groups
```

Finite

alias of *FinitePermutationGroups*

super_categories()

Return a list of the immediate super categories of `self`.

EXAMPLES:

```
sage: PermutationGroups().super_categories()
[Category of groups]
```

```
>>> from sage.all import *
>>> PermutationGroups().super_categories()
[Category of groups]
```

4.134 Pointed sets

class sage.categories.pointed_sets.**PointedSets**

Bases: *Category_singleton*

The category of pointed sets.

EXAMPLES:

```
sage: PointedSets()
Category of pointed sets
```

```
>>> from sage.all import *
>>> PointedSets()
Category of pointed sets
```

super_categories()

EXAMPLES:

```
sage: PointedSets().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> PointedSets().super_categories()
[Category of sets]
```

4.135 Polyhedral subsets of free ZZ, QQ or RR-modules.

class sage.categories.polyhedra.**PolyhedralSets**(R)

Bases: *Category_over_base_ring*

The category of polyhedra over a ring.

EXAMPLES:

We create the category of polyhedra over **Q**:

```
sage: PolyhedralSets(QQ)
Category of polyhedral sets over Rational Field
```

```
>>> from sage.all import *
>>> PolyhedralSets(QQ)
Category of polyhedral sets over Rational Field
```

super_categories()

EXAMPLES:

```
sage: PolyhedralSets(QQ).super_categories()
[Category of commutative magmas, Category of additive monoids]
```

```
>>> from sage.all import *
>>> PolyhedralSets(QQ).super_categories()
[Category of commutative magmas, Category of additive monoids]
```

4.136 Posets

class sage.categories.posets.**Posets**

Bases: *Category*

The category of posets i.e. sets with a partial order structure.

EXAMPLES:

```
sage: Posets()
Category of posets
sage: Posets().super_categories()
[Category of sets]
sage: P = Posets().example(); P
An example of a poset: sets ordered by inclusion
```

```
>>> from sage.all import *
>>> Posets()
Category of posets
```

(continues on next page)

(continued from previous page)

```
>>> Posets().super_categories()
[Category of sets]
>>> P = Posets().example(); P
An example of a poset: sets ordered by inclusion
```

The partial order is implemented by the mandatory method *le()*:

```
sage: x = P(Set([1,3])); y = P(Set([1,2,3]))
sage: x, y
({1, 3}, {1, 2, 3})
sage: P.le(x, y)
True
sage: P.le(x, x)
True
sage: P.le(y, x)
False
```

```
>>> from sage.all import *
>>> x = P(Set([Integer(1),Integer(3)])); y = P(Set([Integer(1),Integer(2),
↪Integer(3)]))
>>> x, y
({1, 3}, {1, 2, 3})
>>> P.le(x, y)
True
>>> P.le(x, x)
True
>>> P.le(y, x)
False
```

The other comparison methods are called *lt()*, *ge()*, *gt()*, following Python's naming convention in `operator`. Default implementations are provided:

```
sage: P.lt(x, x)
False
sage: P.ge(y, x)
True
```

```
>>> from sage.all import *
>>> P.lt(x, x)
False
>>> P.ge(y, x)
True
```

Unless the poset is a facade (see *Sets.Facade*), one can compare directly its elements using the usual Python operators:

```
sage: D = Poset((divisors(30), attrcall("divides")), facade = False)
sage: D(3) <= D(6)
True
sage: D(3) <= D(3)
True
sage: D(3) <= D(5)
```

(continues on next page)

(continued from previous page)

```
False
sage: D(3) < D(3)
False
sage: D(10) >= D(5)
True
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")), facade = False)
>>> D(Integer(3)) <= D(Integer(6))
True
>>> D(Integer(3)) <= D(Integer(3))
True
>>> D(Integer(3)) <= D(Integer(5))
False
>>> D(Integer(3)) < D(Integer(3))
False
>>> D(Integer(10)) >= D(Integer(5))
True
```

At this point, this has to be implemented by hand. Once Issue #10130 will be resolved, this will be automatically provided by this category:

```
sage: # not implemented
sage: x < y
True
sage: x < x
False
sage: x <= x
True
sage: y >= x
True
```

```
>>> from sage.all import *
>>> # not implemented
>>> x < y
True
>>> x < x
False
>>> x <= x
True
>>> y >= x
True
```

See also

`Poset()`, *FinitePosets*, *LatticePosets*

class Bounded(*base_category*)

Bases: *CategoryWithAxiom*

The category of bounded posets.

EXAMPLES:

```
sage: cat = Posets().Bounded(); cat
Category of bounded posets

sage: cat.super_categories()
[Category of posets]
```

```
>>> from sage.all import *
>>> cat = Posets().Bounded(); cat
Category of bounded posets

>>> cat.super_categories()
[Category of posets]
```

class ParentMethods

Bases: object

is_bounded()

Return whether self is a bounded poset.

EXAMPLES:

```
sage: posets.TamariLattice(4).is_bounded()
True
```

```
>>> from sage.all import *
>>> posets.TamariLattice(Integer(4)).is_bounded()
True
```

class ElementMethods

Bases: object

Finite

alias of *FinitePosets*

class ParentMethods

Bases: object

CartesianProduct

alias of CartesianProductPoset

directed_subset(*elements*, *direction*)

Return the order filter or the order ideal generated by a list of elements.

If direction is 'up', the order filter (upper set) is being returned.

If direction is 'down', the order ideal (lower set) is being returned.

INPUT:

- elements – list of elements
- direction – 'up' or 'down'

EXAMPLES:

```
sage: B = posets.BooleanLattice(4)
sage: B.directed_subset([3, 8], 'up')
```

(continues on next page)

(continued from previous page)

```
[3, 7, 8, 9, 10, 11, 12, 13, 14, 15]
sage: B.directed_subset([7, 10], 'down')
[0, 1, 2, 3, 4, 5, 6, 7, 8, 10]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(4))
>>> B.directed_subset([Integer(3), Integer(8)], 'up')
[3, 7, 8, 9, 10, 11, 12, 13, 14, 15]
>>> B.directed_subset([Integer(7), Integer(10)], 'down')
[0, 1, 2, 3, 4, 5, 6, 7, 8, 10]
```

ge (x, y)

Return whether $x \geq y$ in the poset `self`.

INPUT:

- `x`, `y` – elements of `self`

This default implementation delegates the work to *le()*.

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.ge( 6, 3 )
True
sage: D.ge( 3, 3 )
True
sage: D.ge( 3, 5 )
False
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.ge( Integer(6), Integer(3) )
True
>>> D.ge( Integer(3), Integer(3) )
True
>>> D.ge( Integer(3), Integer(5) )
False
```

gt (x, y)

Return whether $x > y$ in the poset `self`.

INPUT:

- `x`, `y` – elements of `self`

This default implementation delegates the work to *lt()*.

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.gt( 3, 6 )
False
sage: D.gt( 3, 3 )
False
sage: D.gt( 3, 5 )
False
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.gt( Integer(3), Integer(6) )
False
>>> D.gt( Integer(3), Integer(3) )
False
>>> D.gt( Integer(3), Integer(5) )
False
```

is_antichain_of_poset(*o*)

Return whether an iterable `o` is an antichain of `self`.

INPUT:

- `o` – an iterable (e. g., list, set, or tuple) containing some elements of `self`

OUTPUT:

`True` if the subset of `self` consisting of the entries of `o` is an antichain of `self`, and `False` otherwise.

EXAMPLES:

```
sage: P = Poset((divisors(12), attrcall("divides")),
....:           facade=True, linear_extension=True)
sage: sorted(P.list())
[1, 2, 3, 4, 6, 12]
sage: P.is_antichain_of_poset([1, 3])
False
sage: P.is_antichain_of_poset([3, 1])
False
sage: P.is_antichain_of_poset([1, 1, 3])
False
sage: P.is_antichain_of_poset([])
True
sage: P.is_antichain_of_poset([1])
True
sage: P.is_antichain_of_poset([1, 1])
True
sage: P.is_antichain_of_poset([3, 4])
True
sage: P.is_antichain_of_poset([3, 4, 12])
False
sage: P.is_antichain_of_poset([6, 4])
True
sage: P.is_antichain_of_poset(i for i in divisors(12)
....:                         if (2 < i and i < 6))
True
sage: P.is_antichain_of_poset(i for i in divisors(12)
....:                         if (2 <= i and i < 6))
False

sage: Q = Poset({2: [3, 1], 3: [4], 1: [4]})
sage: Q.is_antichain_of_poset((1, 2))
False
sage: Q.is_antichain_of_poset((2, 4))
False
```

(continues on next page)

(continued from previous page)

```
sage: Q.is_antichain_of_poset((4, 2))
False
sage: Q.is_antichain_of_poset((2, 2))
True
sage: Q.is_antichain_of_poset((3, 4))
False
sage: Q.is_antichain_of_poset((3, 1))
True
sage: Q.is_antichain_of_poset((1, ))
True
sage: Q.is_antichain_of_poset(())
True
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")),
...           facade=True, linear_extension=True)
>>> sorted(P.list())
[1, 2, 3, 4, 6, 12]
>>> P.is_antichain_of_poset([Integer(1), Integer(3)])
False
>>> P.is_antichain_of_poset([Integer(3), Integer(1)])
False
>>> P.is_antichain_of_poset([Integer(1), Integer(1), Integer(3)])
False
>>> P.is_antichain_of_poset([])
True
>>> P.is_antichain_of_poset([Integer(1)])
True
>>> P.is_antichain_of_poset([Integer(1), Integer(1)])
True
>>> P.is_antichain_of_poset([Integer(3), Integer(4)])
True
>>> P.is_antichain_of_poset([Integer(3), Integer(4), Integer(12)])
False
>>> P.is_antichain_of_poset([Integer(6), Integer(4)])
True
>>> P.is_antichain_of_poset(i for i in divisors(Integer(12))
...                         if (Integer(2) < i and i < Integer(6)))
True
>>> P.is_antichain_of_poset(i for i in divisors(Integer(12))
...                         if (Integer(2) <= i and i < Integer(6)))
False

>>> Q = Poset({Integer(2): [Integer(3), Integer(1)], Integer(3):␣
↪[Integer(4)], Integer(1): [Integer(4)]})
>>> Q.is_antichain_of_poset((Integer(1), Integer(2)))
False
>>> Q.is_antichain_of_poset((Integer(2), Integer(4)))
False
>>> Q.is_antichain_of_poset((Integer(4), Integer(2)))
False
>>> Q.is_antichain_of_poset((Integer(2), Integer(2)))
```

(continues on next page)

(continued from previous page)

```
True
>>> Q.is_antichain_of_poset((Integer(3), Integer(4)))
False
>>> Q.is_antichain_of_poset((Integer(3), Integer(1)))
True
>>> Q.is_antichain_of_poset((Integer(1), ))
True
>>> Q.is_antichain_of_poset(())
True
```

An infinite poset:

```
sage: from sage.categories.examples.posets import␣
↪FiniteSetsOrderedByInclusion
sage: R = FiniteSetsOrderedByInclusion()
sage: R.is_antichain_of_poset([R(set([3, 1, 2])),
....:                          R(set([1, 4])), R(set([4, 5]))])
True
sage: R.is_antichain_of_poset([R(set([3, 1, 2, 4])),
....:                          R(set([1, 4])), R(set([4, 5]))])
False
```

```
>>> from sage.all import *
>>> from sage.categories.examples.posets import␣
↪FiniteSetsOrderedByInclusion
>>> R = FiniteSetsOrderedByInclusion()
>>> R.is_antichain_of_poset([R(set([Integer(3), Integer(1), Integer(2)])),
...                          R(set([Integer(1), Integer(4)])),␣
↪R(set([Integer(4), Integer(5)]))])
True
>>> R.is_antichain_of_poset([R(set([Integer(3), Integer(1), Integer(2),␣
↪Integer(4)])),
...                          R(set([Integer(1), Integer(4)])),␣
↪R(set([Integer(4), Integer(5)]))])
False
```

is_chain_of_poset(*o*, *ordered=False*)

Return whether an iterable `o` is a chain of `self`, including a check for `o` being ordered from smallest to largest element if the keyword `ordered` is set to `True`.

INPUT:

- `o` – an iterable (e. g., list, set, or tuple) containing some elements of `self`
- `ordered` – a Boolean (default: `False`); decides whether the notion of a chain includes being ordered

OUTPUT:

If `ordered` is set to `False`, the truth value of the following assertion is returned: The subset of `self` formed by the elements of `o` is a chain in `self`.

If `ordered` is set to `True`, the truth value of the following assertion is returned: Every element of the list `o` is (strictly!) smaller than its successor in `self`. (This makes no sense if `ordered` is a set.)

EXAMPLES:

```
sage: P = Poset((divisors(12), attrcall("divides")),
....:           facade=True, linear_extension=True)
sage: sorted(P.list())
[1, 2, 3, 4, 6, 12]
sage: P.is_chain_of_poset([1, 3])
True
sage: P.is_chain_of_poset([3, 1])
True
sage: P.is_chain_of_poset([1, 3], ordered=True)
True
sage: P.is_chain_of_poset([3, 1], ordered=True)
False
sage: P.is_chain_of_poset([])
True
sage: P.is_chain_of_poset([], ordered=True)
True
sage: P.is_chain_of_poset((2, 12, 6))
True
sage: P.is_chain_of_poset((2, 6, 12), ordered=True)
True
sage: P.is_chain_of_poset((2, 12, 6), ordered=True)
False
sage: P.is_chain_of_poset((2, 12, 6, 3))
False
sage: P.is_chain_of_poset((2, 3))
False

sage: Q = Poset({2: [3, 1], 3: [4], 1: [4]})
sage: Q.is_chain_of_poset([1, 2], ordered=True)
False
sage: Q.is_chain_of_poset([1, 2])
True
sage: Q.is_chain_of_poset([2, 1], ordered=True)
True
sage: Q.is_chain_of_poset([2, 1, 1], ordered=True)
False
sage: Q.is_chain_of_poset([3])
True
sage: Q.is_chain_of_poset([4, 2, 3])
True
sage: Q.is_chain_of_poset([4, 2, 3], ordered=True)
False
sage: Q.is_chain_of_poset([2, 3, 4], ordered=True)
True
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")),
...           facade=True, linear_extension=True)
>>> sorted(P.list())
[1, 2, 3, 4, 6, 12]
>>> P.is_chain_of_poset([Integer(1), Integer(3)])
True
```

(continues on next page)

(continued from previous page)

```
>>> P.is_chain_of_poset([Integer(3), Integer(1)])
True
>>> P.is_chain_of_poset([Integer(1), Integer(3)], ordered=True)
True
>>> P.is_chain_of_poset([Integer(3), Integer(1)], ordered=True)
False
>>> P.is_chain_of_poset([])
True
>>> P.is_chain_of_poset([], ordered=True)
True
>>> P.is_chain_of_poset((Integer(2), Integer(12), Integer(6)))
True
>>> P.is_chain_of_poset((Integer(2), Integer(6), Integer(12)),␣
↪ordered=True)
True
>>> P.is_chain_of_poset((Integer(2), Integer(12), Integer(6)),␣
↪ordered=True)
False
>>> P.is_chain_of_poset((Integer(2), Integer(12), Integer(6), Integer(3)))
False
>>> P.is_chain_of_poset((Integer(2), Integer(3)))
False

>>> Q = Poset({Integer(2): [Integer(3), Integer(1)], Integer(3):␣
↪[Integer(4)], Integer(1): [Integer(4)]})
>>> Q.is_chain_of_poset([Integer(1), Integer(2)], ordered=True)
False
>>> Q.is_chain_of_poset([Integer(1), Integer(2)])
True
>>> Q.is_chain_of_poset([Integer(2), Integer(1)], ordered=True)
True
>>> Q.is_chain_of_poset([Integer(2), Integer(1), Integer(1)],␣
↪ordered=True)
False
>>> Q.is_chain_of_poset([Integer(3)])
True
>>> Q.is_chain_of_poset([Integer(4), Integer(2), Integer(3)])
True
>>> Q.is_chain_of_poset([Integer(4), Integer(2), Integer(3)],␣
↪ordered=True)
False
>>> Q.is_chain_of_poset([Integer(2), Integer(3), Integer(4)],␣
↪ordered=True)
True
```

Examples with infinite posets:

```
sage: from sage.categories.examples.posets import␣
↪FiniteSetsOrderedByInclusion
sage: R = FiniteSetsOrderedByInclusion()
sage: R.is_chain_of_poset([R(set([3, 1, 2])),
....:                      R(set([1, 4])),
```

(continues on next page)

(continued from previous page)

```
....:                       R(set([4, 5]))])
False
sage: R.is_chain_of_poset([R(set([3, 1, 2])),
....:                       R(set([1, 2])),
....:                       R(set([1]))], ordered=True)
False
sage: R.is_chain_of_poset([R(set([3, 1, 2])),
....:                       R(set([1, 2])), R(set([1]))])
True

sage: from sage.categories.examples.posets import␣
↪PositiveIntegersOrderedByDivisibilityFacade
sage: T = PositiveIntegersOrderedByDivisibilityFacade()
sage: T.is_chain_of_poset((T(3), T(4), T(7)))
False
sage: T.is_chain_of_poset((T(3), T(6), T(3)))
True
sage: T.is_chain_of_poset((T(3), T(6), T(3)), ordered=True)
False
sage: T.is_chain_of_poset((T(3), T(3), T(6)))
True
sage: T.is_chain_of_poset((T(3), T(3), T(6)), ordered=True)
False
sage: T.is_chain_of_poset((T(3), T(6)), ordered=True)
True
sage: T.is_chain_of_poset((), ordered=True)
True
sage: T.is_chain_of_poset((T(3),), ordered=True)
True
sage: T.is_chain_of_poset((T(q) for q in divisors(27)))
True
sage: T.is_chain_of_poset((T(q) for q in divisors(18)))
False
```

```
>>> from sage.all import *
>>> from sage.categories.examples.posets import␣
↪FiniteSetsOrderedByInclusion
>>> R = FiniteSetsOrderedByInclusion()
>>> R.is_chain_of_poset([R(set([Integer(3), Integer(1), Integer(2)])),
...                       R(set([Integer(1), Integer(4)])),
...                       R(set([Integer(4), Integer(5)]))])
False
>>> R.is_chain_of_poset([R(set([Integer(3), Integer(1), Integer(2)])),
...                       R(set([Integer(1), Integer(2)])),
...                       R(set([Integer(1)]))], ordered=True)
False
>>> R.is_chain_of_poset([R(set([Integer(3), Integer(1), Integer(2)])),
...                       R(set([Integer(1), Integer(2)])),␣
↪R(set([Integer(1)]))])
True

>>> from sage.categories.examples.posets import␣
```

(continues on next page)

(continued from previous page)

```
↪PositiveIntegersOrderedByDivisibilityFacade
>>> T = PositiveIntegersOrderedByDivisibilityFacade()
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(4)), T(Integer(7))))
False
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(6)), T(Integer(3))))
True
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(6)), T(Integer(3))),␣
↪ordered=True)
False
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(3)), T(Integer(6))))
True
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(3)), T(Integer(6))),␣
↪ordered=True)
False
>>> T.is_chain_of_poset((T(Integer(3)), T(Integer(6))), ordered=True)
True
>>> T.is_chain_of_poset((), ordered=True)
True
>>> T.is_chain_of_poset((T(Integer(3)),), ordered=True)
True
>>> T.is_chain_of_poset((T(q) for q in divisors(Integer(27))))
True
>>> T.is_chain_of_poset((T(q) for q in divisors(Integer(18))))
False
```

is_order_filter(*o*)

Return whether `o` is an order filter of `self`, assuming `self` has no infinite ascending path.

INPUT:

- `o` – list (or set, or tuple) containing some elements of `self`

EXAMPLES:

```
sage: P = Poset((divisors(12), attrcall("divides")),
....:           facade=True, linear_extension=True)
sage: sorted(P.list())
[1, 2, 3, 4, 6, 12]
sage: P.is_order_filter([4, 12])
True
sage: P.is_order_filter([])
True
sage: P.is_order_filter({3, 4, 12})
False
sage: P.is_order_filter({3, 6, 12})
True
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")),
...           facade=True, linear_extension=True)
>>> sorted(P.list())
[1, 2, 3, 4, 6, 12]
>>> P.is_order_filter([Integer(4), Integer(12)])
True
```

(continues on next page)

(continued from previous page)

```
>>> P.is_order_filter([])
True
>>> P.is_order_filter({Integer(3), Integer(4), Integer(12)})
False
>>> P.is_order_filter({Integer(3), Integer(6), Integer(12)})
True
```

is_order_ideal(*o*)

Return whether `o` is an order ideal of `self`, assuming `self` has no infinite descending path.

INPUT:

- `o` – list (or set, or tuple) containing some elements of `self`

EXAMPLES:

```
sage: P = Poset((divisors(12), attrcall("divides")),
....:           facade=True, linear_extension=True)
sage: sorted(P.list())
[1, 2, 3, 4, 6, 12]
sage: P.is_order_ideal([1, 3])
True
sage: P.is_order_ideal([])
True
sage: P.is_order_ideal({1, 3})
True
sage: P.is_order_ideal([1, 3, 4])
False
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")),
...           facade=True, linear_extension=True)
>>> sorted(P.list())
[1, 2, 3, 4, 6, 12]
>>> P.is_order_ideal([Integer(1), Integer(3)])
True
>>> P.is_order_ideal([])
True
>>> P.is_order_ideal({Integer(1), Integer(3)})
True
>>> P.is_order_ideal([Integer(1), Integer(3), Integer(4)])
False
```

le(*x*, *y*)

Return whether $x \leq y$ in the poset `self`.

INPUT:

- `x`, `y` – elements of `self`

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.le( 3, 6 )
True
sage: D.le( 3, 3 )
True
```

(continues on next page)

(continued from previous page)

```
sage: D.le( 3, 5 )
False
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.le( Integer(3), Integer(6) )
True
>>> D.le( Integer(3), Integer(3) )
True
>>> D.le( Integer(3), Integer(5) )
False
```

lower_covers(x)

Return the lower covers of x, that is, the elements y such that $y < x$ and there exists no z such that $y < z < x$.

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.lower_covers(15)
[3, 5]
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.lower_covers(Integer(15))
[3, 5]
```

lt(x, y)

Return whether $x < y$ in the poset `self`.

INPUT:

- `x`, `y` – elements of `self`

This default implementation delegates the work to *le()*.

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.lt( 3, 6 )
True
sage: D.lt( 3, 3 )
False
sage: D.lt( 3, 5 )
False
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.lt( Integer(3), Integer(6) )
True
>>> D.lt( Integer(3), Integer(3) )
False
>>> D.lt( Integer(3), Integer(5) )
False
```

order_filter(*elements*)

Return the order filter generated by a list of elements.

A subset I of a poset is said to be an order filter if, for any x in I and y such that $y \geq x$, then y is in I.

This is also called the upper set generated by these elements.

EXAMPLES:

```
sage: B = posets.BooleanLattice(4)
sage: B.order_filter([3,8])
[3, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(4))
>>> B.order_filter([Integer(3),Integer(8)])
[3, 7, 8, 9, 10, 11, 12, 13, 14, 15]
```

order_ideal(*elements*)

Return the order ideal in `self` generated by the elements of an iterable `elements`.

A subset I of a poset is said to be an order ideal if, for any x in I and y such that $y \leq x$, then y is in I.

This is also called the lower set generated by these elements.

EXAMPLES:

```
sage: B = posets.BooleanLattice(4)
sage: B.order_ideal([7,10])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 10]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(4))
>>> B.order_ideal([Integer(7),Integer(10)])
[0, 1, 2, 3, 4, 5, 6, 7, 8, 10]
```

order_ideal_toggle(I, v)

Return the result of toggling the element `v` in the order ideal `I`.

If v is an element of a poset P, then toggling the element v is an automorphism of the set $J(P)$ of all order ideals of P. It is defined as follows: If I is an order ideal of P, then the image of I under toggling the element v is

- the set $I \cup \{v\}$, if $v \notin I$ but every element of P smaller than v is in I;
- the set $I \setminus \{v\}$, if $v \in I$ but no element of P greater than v is in I;
- I otherwise.

This image always is an order ideal of P.

EXAMPLES:

```
sage: P = Poset({1: [2,3], 2: [4], 3: []})
sage: I = Set({1, 2})
sage: I in P.order_ideals_lattice()                                    #␣
↪needs sage.modules
True
sage: P.order_ideal_toggle(I, 1)
{1, 2}
```

(continues on next page)

(continued from previous page)

```
sage: P.order_ideal_toggle(I, 2)
{1}
sage: P.order_ideal_toggle(I, 3)
{1, 2, 3}
sage: P.order_ideal_toggle(I, 4)
{1, 2, 4}
sage: P4 = Posets(4)
sage: all(all(all(P.order_ideal_toggle(P.order_ideal_toggle(I, i), i) ==␣
↪I          # needs sage.modules
....:                 for i in range(4))
....:               for I in P.order_ideals_lattice(facade=True))
....:         for P in P4)
True
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(2),Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): []})
>>> I = Set({Integer(1), Integer(2)})
>>> I in P.order_ideals_lattice()                                    #␣
↪needs sage.modules
True
>>> P.order_ideal_toggle(I, Integer(1))
{1, 2}
>>> P.order_ideal_toggle(I, Integer(2))
{1}
>>> P.order_ideal_toggle(I, Integer(3))
{1, 2, 3}
>>> P.order_ideal_toggle(I, Integer(4))
{1, 2, 4}
>>> P4 = Posets(Integer(4))
>>> all(all(all(P.order_ideal_toggle(P.order_ideal_toggle(I, i), i) == I ␣
↪         # needs sage.modules
...                 for i in range(Integer(4)))
...               for I in P.order_ideals_lattice(facade=True))
...         for P in P4)
True
```

order_ideal_toggles(*I*, *vs*)

Return the result of toggling the elements of the list (or iterable) `vs` (one by one, from left to right) in the order ideal `I`.

See *order_ideal_toggle()* for a definition of toggling.

EXAMPLES:

```
sage: P = Poset({1: [2,3], 2: [4], 3: []})
sage: I = Set({1, 2})
sage: P.order_ideal_toggles(I, [1,2,3,4])
{1, 3}
sage: P.order_ideal_toggles(I, (1,2,3,4))
{1, 3}
```

```
>>> from sage.all import *
>>> P = Poset({Integer(1): [Integer(2),Integer(3)], Integer(2):␣
↪[Integer(4)], Integer(3): []})
>>> I = Set({Integer(1), Integer(2)})
>>> P.order_ideal_toggles(I, [Integer(1),Integer(2),Integer(3),
↪Integer(4)])
{1, 3}
>>> P.order_ideal_toggles(I, (Integer(1),Integer(2),Integer(3),
↪Integer(4)))
{1, 3}
```

principal_lower_set(*x*)

alias of *principal_order_ideal()*.

principal_order_filter(*x*)

Return the order filter generated by an element `x`.

This is also called the upper set generated by this element.

EXAMPLES:

```
sage: B = posets.BooleanLattice(4)
sage: B.principal_order_filter(2)
[2, 3, 6, 7, 10, 11, 14, 15]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(4))
>>> B.principal_order_filter(Integer(2))
[2, 3, 6, 7, 10, 11, 14, 15]
```

principal_order_ideal(*x*)

Return the order ideal generated by an element `x`.

This is also called the lower set generated by this element.

EXAMPLES:

```
sage: B = posets.BooleanLattice(4)
sage: B.principal_order_ideal(6)
[0, 2, 4, 6]
```

```
>>> from sage.all import *
>>> B = posets.BooleanLattice(Integer(4))
>>> B.principal_order_ideal(Integer(6))
[0, 2, 4, 6]
```

principal_upper_set(*x*)

alias of *principal_order_filter()*.

upper_covers(*x*)

Return the upper covers of x, that is, the elements y such that $x < y$ and there exists no z such that $x < z < y$.

EXAMPLES:

```
sage: D = Poset((divisors(30), attrcall("divides")))
sage: D.upper_covers(3)
[6, 15]
```

```
>>> from sage.all import *
>>> D = Poset((divisors(Integer(30)), attrcall("divides")))
>>> D.upper_covers(Integer(3))
[6, 15]
```

class SubcategoryMethods

Bases: object

Bounded()

A bounded poset is a poset with a unique maximal element and a unique minimal element.

EXAMPLES:

```
sage: P = posets.DivisorLattice(24)
sage: P in Posets().Bounded()
True
```

```
>>> from sage.all import *
>>> P = posets.DivisorLattice(Integer(24))
>>> P in Posets().Bounded()
True
```

example(*choice=None*)

Return examples of objects of Posets(), as per *Category.example()*.

EXAMPLES:

```
sage: Posets().example()
An example of a poset: sets ordered by inclusion

sage: Posets().example("facade")
An example of a facade poset:
 the positive integers ordered by divisibility
```

```
>>> from sage.all import *
>>> Posets().example()
An example of a poset: sets ordered by inclusion

>>> Posets().example("facade")
An example of a facade poset:
 the positive integers ordered by divisibility
```

super_categories()

Return a list of the (immediate) super categories of self, as per *Category.super_categories()*.

EXAMPLES:

```
sage: Posets().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Posets().super_categories()
[Category of sets]
```

4.137 Principal ideal domains

class sage.categories.principal_ideal_domains.**PrincipalIdealDomains**

Bases: *Category_singleton*

The category of (constructive) principal ideal domains.

By constructive, we mean that a single generator can be constructively found for any ideal given by a finite set of generators. Note that this constructive definition only implies that finitely generated ideals are principal. It is not clear what we would mean by an infinitely generated ideal.

EXAMPLES:

```
sage: PrincipalIdealDomains()
Category of principal ideal domains
sage: PrincipalIdealDomains().super_categories()
[Category of unique factorization domains, Category of Dedekind domains]
```

```
>>> from sage.all import *
>>> PrincipalIdealDomains()
Category of principal ideal domains
>>> PrincipalIdealDomains().super_categories()
[Category of unique factorization domains, Category of Dedekind domains]
```

See also Wikipedia article Principal_ideal_domain

class ElementMethods

Bases: object

class ParentMethods

Bases: object

class_group()

Return the trivial group, since the class group of a PID is trivial.

EXAMPLES:

```
sage: QQ.class_group()                                                  ␣
↪  # needs sage.groups
Trivial Abelian group
```

```
>>> from sage.all import *
>>> QQ.class_group()
↪# needs sage.groups
Trivial Abelian group
```

content(*x*, *y*, *coerce=True*)

Return the content of x and y.

This is the unique element c of `self` such that x/c and y/c are coprime and integral.

EXAMPLES:

```
sage: QQ.content(ZZ(42), ZZ(48)); type(QQ.content(ZZ(42), ZZ(48)))
6
<class 'sage.rings.rational.Rational'>
sage: QQ.content(1/2, 1/3)
1/6
sage: factor(1/2); factor(1/3); factor(1/6)
2^-1
3^-1
2^-1 * 3^-1
sage: a = (2*3)/(7*11); b = (13*17)/(19*23)
sage: factor(a); factor(b); factor(QQ.content(a,b))
2 * 3 * 7^-1 * 11^-1
13 * 17 * 19^-1 * 23^-1
7^-1 * 11^-1 * 19^-1 * 23^-1
```

```
>>> from sage.all import *
>>> QQ.content(ZZ(Integer(42)), ZZ(Integer(48))); type(QQ.
↪content(ZZ(Integer(42)), ZZ(Integer(48))))
6
<class 'sage.rings.rational.Rational'>
>>> QQ.content(Integer(1)/Integer(2), Integer(1)/Integer(3))
1/6
>>> factor(Integer(1)/Integer(2)); factor(Integer(1)/Integer(3));␣
↪factor(Integer(1)/Integer(6))
2^-1
3^-1
2^-1 * 3^-1
>>> a = (Integer(2)*Integer(3))/(Integer(7)*Integer(11)); b =␣
↪(Integer(13)*Integer(17))/(Integer(19)*Integer(23))
>>> factor(a); factor(b); factor(QQ.content(a,b))
2 * 3 * 7^-1 * 11^-1
13 * 17 * 19^-1 * 23^-1
7^-1 * 11^-1 * 19^-1 * 23^-1
```

Note the changes to the second entry:

```
sage: c = (2*3)/(7*11); d = (13*17)/(7*19*23)
sage: factor(c); factor(d); factor(QQ.content(c,d))
2 * 3 * 7^-1 * 11^-1
7^-1 * 13 * 17 * 19^-1 * 23^-1
7^-1 * 11^-1 * 19^-1 * 23^-1
sage: e = (2*3)/(7*11); f = (13*17)/(7^3*19*23)
sage: factor(e); factor(f); factor(QQ.content(e,f))
2 * 3 * 7^-1 * 11^-1
7^-3 * 13 * 17 * 19^-1 * 23^-1
7^-3 * 11^-1 * 19^-1 * 23^-1
```

```
>>> from sage.all import *
>>> c = (Integer(2)*Integer(3))/(Integer(7)*Integer(11)); d =␣
↪(Integer(13)*Integer(17))/(Integer(7)*Integer(19)*Integer(23))
>>> factor(c); factor(d); factor(QQ.content(c,d))
2 * 3 * 7^-1 * 11^-1
```

(continues on next page)

(continued from previous page)

```
7^-1 * 13 * 17 * 19^-1 * 23^-1
7^-1 * 11^-1 * 19^-1 * 23^-1
>>> e = (Integer(2)*Integer(3))/(Integer(7)*Integer(11)); f =␣
↪(Integer(13)*Integer(17))/
↪(Integer(7)**Integer(3)*Integer(19)*Integer(23))
>>> factor(e); factor(f); factor(QQ.content(e,f))
2 * 3 * 7^-1 * 11^-1
7^-3 * 13 * 17 * 19^-1 * 23^-1
7^-3 * 11^-1 * 19^-1 * 23^-1
```

gcd(*x*, *y*, *coerce=True*)

Return the greatest common divisor of `x` and `y`, as elements of `self`.

EXAMPLES:

The integers are a principal ideal domain and hence a GCD domain:

```
sage: ZZ.gcd(42, 48)
6
sage: 42.factor(); 48.factor()
2 * 3 * 7
2^4 * 3
sage: ZZ.gcd(2^4*7^2*11, 2^3*11*13)
88
sage: 88.factor()
2^3 * 11
```

```
>>> from sage.all import *
>>> ZZ.gcd(Integer(42), Integer(48))
6
>>> Integer(42).factor(); Integer(48).factor()
2 * 3 * 7
2^4 * 3
>>> ZZ.gcd(Integer(2)**Integer(4)*Integer(7)**Integer(2)*Integer(11),␣
↪Integer(2)**Integer(3)*Integer(11)*Integer(13))
88
>>> Integer(88).factor()
2^3 * 11
```

In a field, any nonzero element is a GCD of any nonempty set of nonzero elements. In previous versions, Sage used to return 1 in the case of the rational field. However, since Issue #10771, the rational field is considered as the *fraction field* of the integer ring. For the fraction field of an integral domain that provides both GCD and LCM, it is possible to pick a GCD that is compatible with the GCD of the base ring:

```
sage: QQ.gcd(ZZ(42), ZZ(48)); type(QQ.gcd(ZZ(42), ZZ(48)))
6
<class 'sage.rings.rational.Rational'>
sage: QQ.gcd(1/2, 1/3)
1/6
```

```
>>> from sage.all import *
>>> QQ.gcd(ZZ(Integer(42)), ZZ(Integer(48))); type(QQ.gcd(ZZ(Integer(42)),
```

(continues on next page)

(continued from previous page)

```
↪ ZZ(Integer(48))))
6
<class 'sage.rings.rational.Rational'>
>>> QQ.gcd(Integer(1)/Integer(2), Integer(1)/Integer(3))
1/6
```

Polynomial rings over fields are GCD domains as well. Here is a simple example over the ring of polynomials over the rationals as well as over an extension ring. Note that `gcd` requires x and y to be coercible:

```
sage: # needs sage.rings.number_field
sage: R.<x> = PolynomialRing(QQ)
sage: S.<a> = NumberField(x^2 - 2, 'a')
sage: f = (x - a)*(x + a); g = (x - a)*(x^2 - 2)
sage: print(f); print(g)
x^2 - 2
x^3 - a*x^2 - 2*x + 2*a
sage: f in R
True
sage: g in R
False
sage: R.gcd(f, g)
Traceback (most recent call last):
...
TypeError: Unable to coerce 2*a to a rational
sage: R.base_extend(S).gcd(f,g)
x^2 - 2
sage: R.base_extend(S).gcd(f, (x - a)*(x^2 - 3))
x - a
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> S = NumberField(x**Integer(2) - Integer(2), 'a', names=('a',)); (a,)␣
↪= S._first_ngens(1)
>>> f = (x - a)*(x + a); g = (x - a)*(x**Integer(2) - Integer(2))
>>> print(f); print(g)
x^2 - 2
x^3 - a*x^2 - 2*x + 2*a
>>> f in R
True
>>> g in R
False
>>> R.gcd(f, g)
Traceback (most recent call last):
...
TypeError: Unable to coerce 2*a to a rational
>>> R.base_extend(S).gcd(f,g)
x^2 - 2
>>> R.base_extend(S).gcd(f, (x - a)*(x**Integer(2) - Integer(3)))
x - a
```

is_noetherian()

Every principal ideal domain is Noetherian, so we return `True`.

EXAMPLES:

```
sage: Zp(5).is_noetherian()
↪  # needs sage.rings.padics
True
```

```
>>> from sage.all import *
>>> Zp(Integer(5)).is_noetherian()
↪          # needs sage.rings.padics
True
```

additional_structure()

Return `None`.

Indeed, the category of principal ideal domains defines no additional structure: a ring morphism between two principal ideal domains is a principal ideal domain morphism.

EXAMPLES:

```
sage: PrincipalIdealDomains().additional_structure()
```

```
>>> from sage.all import *
>>> PrincipalIdealDomains().additional_structure()
```

super_categories()

EXAMPLES:

PIDs are Dedekind domains (see Issue #42800):

```
sage: PolynomialRing(Zmod(17), names='x') in DedekindDomains()
True
```

```
>>> from sage.all import *
>>> PolynomialRing(Zmod(Integer(17)), names='x') in DedekindDomains()
True
```

4.138 Quotient fields

class sage.categories.quotient_fields.**QuotientFields**

Bases: *Category_singleton*

The category of quotient fields over an integral domain.

EXAMPLES:

```
sage: QuotientFields()
Category of quotient fields
sage: QuotientFields().super_categories()
[Category of fields]
```

```
>>> from sage.all import *
>>> QuotientFields()
Category of quotient fields
```

(continues on next page)

(continued from previous page)

```
>>> QuotientFields().super_categories()
[Category of fields]
```

class ElementMethods

Bases: object

denominator()

Constructor for abstract methods.

EXAMPLES:

```
sage: def f(x):
....:     "doc of f"
....:     return 1
sage: x = abstract_method(f); x
<abstract method f at ...>
sage: x.__doc__
'doc of f'
sage: x.__name__
'f'
sage: x.__module__
'__main__'
```

```
>>> from sage.all import *
>>> def f(x):
...     "doc of f"
...     return Integer(1)
>>> x = abstract_method(f); x
<abstract method f at ...>
>>> x.__doc__
'doc of f'
>>> x.__name__
'f'
>>> x.__module__
'__main__'
```

derivative(**args*)

The derivative of this rational function, with respect to variables supplied in args.

Multiple variables and iteration counts may be supplied; see documentation for the global derivative() function for more details.

See also

_derivative

EXAMPLES:

```
sage: F.<x> = Frac(QQ['x'])
sage: (1/x).derivative()
-1/x^2
```

```
>>> from sage.all import *
>>> F = Frac(QQ['x'], names=('x',)); (x,) = F._first_ngens(1)
>>> (Integer(1)/x).derivative()
-1/x^2
```

```
sage: (x+1/x).derivative(x, 2)
2/x^3
```

```
>>> from sage.all import *
>>> (x+Integer(1)/x).derivative(x, Integer(2))
2/x^3
```

```
sage: F.<x,y> = Frac(QQ['x,y'])
sage: (1/(x+y)).derivative(x,y)
2/(x^3 + 3*x^2*y + 3*x*y^2 + y^3)
```

```
>>> from sage.all import *
>>> F = Frac(QQ['x,y'], names=('x', 'y',)); (x, y,) = F._first_ngens(2)
>>> (Integer(1)/(x+y)).derivative(x,y)
2/(x^3 + 3*x^2*y + 3*x*y^2 + y^3)
```

factor(**args*, ***kwds*)

Return the factorization of `self` over the base ring.

INPUT:

- `*args` – arbitrary arguments suitable over the base ring
- `**kwds` – arbitrary keyword arguments suitable over the base ring

OUTPUT: factorization of `self` over the base ring

EXAMPLES:

```
sage: K.<x> = QQ[]
sage: f = (x^3+x)/(x-3)
sage: f.factor()                                                            #␣
↪needs sage.libs.pari
(x - 3)^-1 * x * (x^2 + 1)
```

```
>>> from sage.all import *
>>> K = QQ['x']; (x,) = K._first_ngens(1)
>>> f = (x**Integer(3)+x)/(x-Integer(3))
>>> f.factor()                                                              #␣
↪needs sage.libs.pari
(x - 3)^-1 * x * (x^2 + 1)
```

Here is an example to show that Issue #7868 has been resolved:

```
sage: R.<x,y> = GF(2)[]
sage: f = x*y/(x+y)
sage: f.factor()                                                            #␣
↪needs sage.rings.finite_rings
(x + y)^-1 * y * x
```

```
>>> from sage.all import *
>>> R = GF(Integer(2))['x, y']; (x, y,) = R._first_ngens(2)
>>> f = x*y/(x+y)
>>> f.factor()                                                    #␣
↪needs sage.rings.finite_rings
(x + y)^-1 * y * x
```

gcd(*other*)

Greatest common divisor.

Note

In a field, the greatest common divisor is not very informative, as it is only determined up to a unit. But in the fraction field of an integral domain that provides both gcd and lcm, it is possible to be a bit more specific and define the gcd uniquely up to a unit of the base ring (rather than in the fraction field).

AUTHOR:

- Simon King (2011-02): See Issue #10771

EXAMPLES:

```
sage: # needs sage.libs.pari
sage: R.<x> = QQ['x']
sage: p = (1+x)^3*(1+2*x^2)/(1-x^5)
sage: q = (1+x)^2*(1+3*x^2)/(1-x^4)
sage: factor(p)
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
sage: factor(q)
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
sage: gcd(p, q)
(x + 1)/(x^7 + x^5 - x^2 - 1)
sage: factor(gcd(p, q))
(x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^4 + x^3 + x^2 + x + 1)^-1
sage: factor(gcd(p, 1 + x))
(x - 1)^-1 * (x + 1) * (x^4 + x^3 + x^2 + x + 1)^-1
sage: factor(gcd(1 + x, q))
(x - 1)^-1 * (x + 1) * (x^2 + 1)^-1
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> p = (Integer(1)+x)**Integer(3)*(Integer(1)+Integer(2)*x**Integer(2))/
↪(Integer(1)-x**Integer(5))
>>> q = (Integer(1)+x)**Integer(2)*(Integer(1)+Integer(3)*x**Integer(2))/
↪(Integer(1)-x**Integer(4))
>>> factor(p)
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
>>> factor(q)
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
>>> gcd(p, q)
(x + 1)/(x^7 + x^5 - x^2 - 1)
```

(continues on next page)

(continued from previous page)

```
>>> factor(gcd(p, q))
(x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^4 + x^3 + x^2 + x + 1)^-1
>>> factor(gcd(p, Integer(1) + x))
(x - 1)^-1 * (x + 1) * (x^4 + x^3 + x^2 + x + 1)^-1
>>> factor(gcd(Integer(1) + x, q))
(x - 1)^-1 * (x + 1) * (x^2 + 1)^-1
```

lcm(*other*)

Least common multiple.

In a field, the least common multiple is not very informative, as it is only determined up to a unit. But in the fraction field of an integral domain that provides both gcd and lcm, it is reasonable to be a bit more specific and to define the least common multiple so that it restricts to the usual least common multiple in the base ring and is unique up to a unit of the base ring (rather than up to a unit of the fraction field).

The least common multiple is easily described in terms of the prime decomposition. A rational number can be written as a product of primes with integer (positive or negative) powers in a unique way. The least common multiple of two rational numbers x and y can then be defined by specifying that the exponent of every prime p in $lcm(x, y)$ is the supremum of the exponents of p in x, and the exponent of p in y (where the primes that does not appear in the decomposition of x or y are considered to have exponent zero).

AUTHOR:

- Simon King (2011-02): See Issue #10771

EXAMPLES:

```
sage: lcm(2/3, 1/5)
2
```

```
>>> from sage.all import *
>>> lcm(Integer(2)/Integer(3), Integer(1)/Integer(5))
2
```

Indeed $2/3 = 2^1 3^{-1} 5^0$ and $1/5 = 2^0 3^0 5^{-1}$, so $lcm(2/3, 1/5) = 2^1 3^0 5^0 = 2$.

sage: lcm(1/3, 1/5) 1 sage: lcm(1/3, 1/6) 1/3

Some more involved examples:

```
sage: # needs sage.libs.pari
sage: R.<x> = QQ[]
sage: p = (1+x)^3*(1+2*x^2)/(1-x^5)
sage: q = (1+x)^2*(1+3*x^2)/(1-x^4)
sage: factor(p)
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
sage: factor(q)
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
sage: factor(lcm(p, q))
(x - 1)^-1 * (x + 1)^3 * (x^2 + 1/3) * (x^2 + 1/2)
sage: factor(lcm(p, 1 + x))
(x + 1)^3 * (x^2 + 1/2)
sage: factor(lcm(1 + x, q))
(x + 1) * (x^2 + 1/3)
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> p = (Integer(1)+x)**Integer(3)*(Integer(1)+Integer(2)*x**Integer(2))/
↪(Integer(1)-x**Integer(5))
>>> q = (Integer(1)+x)**Integer(2)*(Integer(1)+Integer(3)*x**Integer(2))/
↪(Integer(1)-x**Integer(4))
>>> factor(p)
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
>>> factor(q)
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
>>> factor(lcm(p, q))
(x - 1)^-1 * (x + 1)^3 * (x^2 + 1/3) * (x^2 + 1/2)
>>> factor(lcm(p, Integer(1) + x))
(x + 1)^3 * (x^2 + 1/2)
>>> factor(lcm(Integer(1) + x, q))
(x + 1) * (x^2 + 1/3)
```

numerator()

Constructor for abstract methods.

EXAMPLES:

```
sage: def f(x):
....:     "doc of f"
....:     return 1
sage: x = abstract_method(f); x
<abstract method f at ...>
sage: x.__doc__
'doc of f'
sage: x.__name__
'f'
sage: x.__module__
'__main__'
```

```
>>> from sage.all import *
>>> def f(x):
...     "doc of f"
...     return Integer(1)
>>> x = abstract_method(f); x
<abstract method f at ...>
>>> x.__doc__
'doc of f'
>>> x.__name__
'f'
>>> x.__module__
'__main__'
```

partial_fraction_decomposition(*decompose_powers=True*)

Decompose fraction field element into a whole part and a list of fraction field elements over prime power denominators.

The sum will be equal to the original fraction.

INPUT:

- `decompose_powers` – boolean (default: `True`); whether to decompose prime power denominators as opposed to having a single term for each irreducible factor of the denominator

OUTPUT: partial fraction decomposition of `self` over the base ring

AUTHORS:

- Robert Bradshaw (2007-05-31)

EXAMPLES:

```
sage: # needs sage.libs.pari
sage: S.<t> = QQ[]
sage: q = 1/(t+1) + 2/(t+2) + 3/(t-3); q
(6*t^2 + 4*t - 6)/(t^3 - 7*t - 6)
sage: whole, parts = q.partial_fraction_decomposition(); parts
[3/(t - 3), 1/(t + 1), 2/(t + 2)]
sage: sum(parts) == q
True
sage: q = 1/(t^3+1) + 2/(t^2+2) + 3/(t-3)^5
sage: whole, parts = q.partial_fraction_decomposition(); parts
[1/3/(t + 1), 3/(t^5 - 15*t^4 + 90*t^3 - 270*t^2 + 405*t - 243),
 (-1/3*t + 2/3)/(t^2 - t + 1), 2/(t^2 + 2)]
sage: sum(parts) == q
True
sage: q = 2*t / (t + 3)^2
sage: q.partial_fraction_decomposition()
(0, [2/(t + 3), -6/(t^2 + 6*t + 9)])
sage: for p in q.partial_fraction_decomposition()[1]:
....:     print(p.factor())
(2) * (t + 3)^-1
(-6) * (t + 3)^-2
sage: q.partial_fraction_decomposition(decompose_powers=False)
(0, [2*t/(t^2 + 6*t + 9)])
```

```
>>> from sage.all import *
>>> # needs sage.libs.pari
>>> S = QQ['t']; (t,) = S._first_ngens(1)
>>> q = Integer(1)/(t+Integer(1)) + Integer(2)/(t+Integer(2)) +␣
↪Integer(3)/(t-Integer(3)); q
(6*t^2 + 4*t - 6)/(t^3 - 7*t - 6)
>>> whole, parts = q.partial_fraction_decomposition(); parts
[3/(t - 3), 1/(t + 1), 2/(t + 2)]
>>> sum(parts) == q
True
>>> q = Integer(1)/(t**Integer(3)+Integer(1)) + Integer(2)/
↪(t**Integer(2)+Integer(2)) + Integer(3)/(t-Integer(3))**Integer(5)
>>> whole, parts = q.partial_fraction_decomposition(); parts
[1/3/(t + 1), 3/(t^5 - 15*t^4 + 90*t^3 - 270*t^2 + 405*t - 243),
 (-1/3*t + 2/3)/(t^2 - t + 1), 2/(t^2 + 2)]
>>> sum(parts) == q
True
>>> q = Integer(2)*t / (t + Integer(3))**Integer(2)
>>> q.partial_fraction_decomposition()
(0, [2/(t + 3), -6/(t^2 + 6*t + 9)])
>>> for p in q.partial_fraction_decomposition()[Integer(1)]:
...     print(p.factor())
```

(continues on next page)

(continued from previous page)

```
(2) * (t + 3)^-1
(-6) * (t + 3)^-2
>>> q.partial_fraction_decomposition(decompose_powers=False)
(0, [2*t/(t^2 + 6*t + 9)])
```

We can decompose over a given algebraic extension:

```
sage: R.<x> = QQ[sqrt(2)][]                                                 #␣
↪needs sage.rings.number_field sage.symbolic
sage: r = 1/(x^4+1)                                                         #␣
↪needs sage.rings.number_field sage.symbolic
sage: r.partial_fraction_decomposition()                                    #␣
↪needs sage.rings.number_field sage.symbolic
(0,
 [(-1/4*sqrt2*x + 1/2)/(x^2 - sqrt2*x + 1),
  (1/4*sqrt2*x + 1/2)/(x^2 + sqrt2*x + 1)])

sage: R.<x> = QQ[I][]  # of QQ[sqrt(-1)]                                    #␣
↪needs sage.rings.number_field sage.symbolic
sage: r =  1/(x^4+1)                                                        #␣
↪needs sage.rings.number_field sage.symbolic
sage: r.partial_fraction_decomposition()                                    #␣
↪needs sage.rings.number_field sage.symbolic
(0, [(-1/2*I)/(x^2 - I), 1/2*I/(x^2 + I)])
```

```
>>> from sage.all import *
>>> R = QQ[sqrt(Integer(2))]['x']; (x,) = R._first_ngens(1)# needs sage.
↪rings.number_field sage.symbolic
>>> r = Integer(1)/(x**Integer(4)+Integer(1))                               ␣
↪                       # needs sage.rings.number_field sage.symbolic
>>> r.partial_fraction_decomposition()                                      #␣
↪needs sage.rings.number_field sage.symbolic
(0,
 [(-1/4*sqrt2*x + 1/2)/(x^2 - sqrt2*x + 1),
  (1/4*sqrt2*x + 1/2)/(x^2 + sqrt2*x + 1)])

>>> R = QQ[I]['x']; (x,) = R._first_ngens(1)# of QQ[sqrt(-1)]               ␣
↪                     # needs sage.rings.number_field sage.symbolic
>>> r =  Integer(1)/(x**Integer(4)+Integer(1))                              ␣
↪                       # needs sage.rings.number_field sage.symbolic
>>> r.partial_fraction_decomposition()                                      #␣
↪needs sage.rings.number_field sage.symbolic
(0, [(-1/2*I)/(x^2 - I), 1/2*I/(x^2 + I)])
```

We can also ask Sage to find the least extension where the denominator factors in linear terms:

```
sage: # needs sage.rings.number_field
sage: R.<x> = QQ[]
sage: r = 1/(x^4+2)
sage: N = r.denominator().splitting_field('a'); N
Number Field in a with defining polynomial x^8 - 8*x^6 + 28*x^4 + 16*x^2␣
↪+ 36
```

(continues on next page)

(continued from previous page)

```
sage: R1.<x1> = N[]
sage: r1 = 1/(x1^4+2)
sage: r1.partial_fraction_decomposition()
(0,
 [(-1/224*a^6 + 13/448*a^4 - 5/56*a^2 - 25/224)/(x1 - 1/28*a^6 + 13/56*a^
↪4 - 5/7*a^2 - 25/28),
  (1/224*a^6 - 13/448*a^4 + 5/56*a^2 + 25/224)/(x1 + 1/28*a^6 - 13/56*a^4␣
↪+ 5/7*a^2 + 25/28),
  (-5/1344*a^7 + 43/1344*a^5 - 85/672*a^3 - 31/672*a)/(x1 - 5/168*a^7 +␣
↪43/168*a^5 - 85/84*a^3 - 31/84*a),
  (5/1344*a^7 - 43/1344*a^5 + 85/672*a^3 + 31/672*a)/(x1 + 5/168*a^7 - 43/
↪168*a^5 + 85/84*a^3 + 31/84*a)])
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> r = Integer(1)/(x**Integer(4)+Integer(2))
>>> N = r.denominator().splitting_field('a'); N
Number Field in a with defining polynomial x^8 - 8*x^6 + 28*x^4 + 16*x^2␣
↪+ 36
>>> R1 = N['x1']; (x1,) = R1._first_ngens(1)
>>> r1 = Integer(1)/(x1**Integer(4)+Integer(2))
>>> r1.partial_fraction_decomposition()
(0,
 [(-1/224*a^6 + 13/448*a^4 - 5/56*a^2 - 25/224)/(x1 - 1/28*a^6 + 13/56*a^
↪4 - 5/7*a^2 - 25/28),
  (1/224*a^6 - 13/448*a^4 + 5/56*a^2 + 25/224)/(x1 + 1/28*a^6 - 13/56*a^4␣
↪+ 5/7*a^2 + 25/28),
  (-5/1344*a^7 + 43/1344*a^5 - 85/672*a^3 - 31/672*a)/(x1 - 5/168*a^7 +␣
↪43/168*a^5 - 85/84*a^3 - 31/84*a),
  (5/1344*a^7 - 43/1344*a^5 + 85/672*a^3 + 31/672*a)/(x1 + 5/168*a^7 - 43/
↪168*a^5 + 85/84*a^3 + 31/84*a)])
```

Or we may work directly over an algebraically closed field:

```
sage: R.<x> = QQbar[]                                                       #␣
↪needs sage.rings.number_field
sage: r =  1/(x^4+1)                                                        #␣
↪needs sage.rings.number_field
sage: r.partial_fraction_decomposition()                                    #␣
↪needs sage.rings.number_field
(0,
 [(-0.1767766952966369? - 0.1767766952966369?*I)/(x - 0.7071067811865475?␣
↪- 0.7071067811865475?*I),
  (-0.1767766952966369? + 0.1767766952966369?*I)/(x - 0.7071067811865475?␣
↪+ 0.7071067811865475?*I),
  (0.1767766952966369? - 0.1767766952966369?*I)/(x + 0.7071067811865475? -
↪ 0.7071067811865475?*I),
  (0.1767766952966369? + 0.1767766952966369?*I)/(x + 0.7071067811865475?␣
↪+ 0.7071067811865475?*I)])
```

```
>>> from sage.all import *
>>> R = QQbar['x']; (x,) = R._first_ngens(1)# needs sage.rings.number_
↪field
>>> r =  Integer(1)/(x**Integer(4)+Integer(1))                              ␣
↪                          # needs sage.rings.number_field
>>> r.partial_fraction_decomposition()                                      #␣
↪needs sage.rings.number_field
(0,
 [(-0.1767766952966369? - 0.1767766952966369?*I)/(x - 0.7071067811865475?␣
↪- 0.7071067811865475?*I),
  (-0.1767766952966369? + 0.1767766952966369?*I)/(x - 0.7071067811865475?␣
↪+ 0.7071067811865475?*I),
  (0.1767766952966369? - 0.1767766952966369?*I)/(x + 0.7071067811865475? -
↪ 0.7071067811865475?*I),
  (0.1767766952966369? + 0.1767766952966369?*I)/(x + 0.7071067811865475?␣
↪+ 0.7071067811865475?*I)])
```

We do the best we can over inexact fields:

```
sage: # needs sage.rings.number_field sage.rings.real_mpfr
sage: R.<x> = RealField(20)[]
sage: q = 1/(x^2 + x + 2)^2 + 1/(x-1); q
(x^4 + 2.0000*x^3
  + 5.0000*x^2 + 5.0000*x + 3.0000)/(x^5 + x^4 + 3.0000*x^3 - x^2 - 4.
↪0000)
sage: whole, parts = q.partial_fraction_decomposition(); parts
[1.0000/(x - 1.0000),
 1.0000/(x^4 + 2.0000*x^3 + 5.0000*x^2 + 4.0000*x + 4.0000)]
sage: sum(parts)
(x^4 + 2.0000*x^3
  + 5.0000*x^2 + 5.0000*x + 3.0000)/(x^5 + x^4 + 3.0000*x^3 - x^2 - 4.
↪0000)
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field sage.rings.real_mpfr
>>> R = RealField(Integer(20))['x']; (x,) = R._first_ngens(1)
>>> q = Integer(1)/(x**Integer(2) + x + Integer(2))**Integer(2) +␣
↪Integer(1)/(x-Integer(1)); q
(x^4 + 2.0000*x^3
  + 5.0000*x^2 + 5.0000*x + 3.0000)/(x^5 + x^4 + 3.0000*x^3 - x^2 - 4.
↪0000)
>>> whole, parts = q.partial_fraction_decomposition(); parts
[1.0000/(x - 1.0000),
 1.0000/(x^4 + 2.0000*x^3 + 5.0000*x^2 + 4.0000*x + 4.0000)]
>>> sum(parts)
(x^4 + 2.0000*x^3
  + 5.0000*x^2 + 5.0000*x + 3.0000)/(x^5 + x^4 + 3.0000*x^3 - x^2 - 4.
↪0000)
```

xgcd(*other*)

Return a triple `(g,s,t)` of elements of that field such that `g` is the greatest common divisor of `self` and `other` and `g = s*self + t*other`.

Note

In a field, the greatest common divisor is not very informative, as it is only determined up to a unit. But in the fraction field of an integral domain that provides both xgcd and lcm, it is possible to be a bit more specific and define the gcd uniquely up to a unit of the base ring (rather than in the fraction field).

EXAMPLES:

```
sage: QQ(3).xgcd(QQ(2))
(1, 1, -1)
sage: QQ(3).xgcd(QQ(1/2))
(1/2, 0, 1)
sage: QQ(1/3).xgcd(QQ(2))
(1/3, 1, 0)
sage: QQ(3/2).xgcd(QQ(5/2))
(1/2, 2, -1)

sage: R.<x> = QQ['x']
sage: p = (1+x)^3*(1+2*x^2)/(1-x^5)
sage: q = (1+x)^2*(1+3*x^2)/(1-x^4)
sage: factor(p)                                                             #␣
↪needs sage.libs.pari
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
sage: factor(q)                                                             #␣
↪needs sage.libs.pari
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
sage: g, s, t = xgcd(p, q)
sage: g
(x + 1)/(x^7 + x^5 - x^2 - 1)
sage: g == s*p + t*q
True
```

```
>>> from sage.all import *
>>> QQ(Integer(3)).xgcd(QQ(Integer(2)))
(1, 1, -1)
>>> QQ(Integer(3)).xgcd(QQ(Integer(1)/Integer(2)))
(1/2, 0, 1)
>>> QQ(Integer(1)/Integer(3)).xgcd(QQ(Integer(2)))
(1/3, 1, 0)
>>> QQ(Integer(3)/Integer(2)).xgcd(QQ(Integer(5)/Integer(2)))
(1/2, 2, -1)

>>> R = QQ['x']; (x,) = R._first_ngens(1)
>>> p = (Integer(1)+x)**Integer(3)*(Integer(1)+Integer(2)*x**Integer(2))/
↪(Integer(1)-x**Integer(5))
>>> q = (Integer(1)+x)**Integer(2)*(Integer(1)+Integer(3)*x**Integer(2))/
↪(Integer(1)-x**Integer(4))
>>> factor(p)                                                               #␣
↪needs sage.libs.pari
(-2) * (x - 1)^-1 * (x + 1)^3 * (x^2 + 1/2) * (x^4 + x^3 + x^2 + x + 1)^-1
>>> factor(q)                                                               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.pari
(-3) * (x - 1)^-1 * (x + 1) * (x^2 + 1)^-1 * (x^2 + 1/3)
>>> g, s, t = xgcd(p, q)
>>> g
(x + 1)/(x^7 + x^5 - x^2 - 1)
>>> g == s*p + t*q
True
```

An example without a well defined gcd or xgcd on its base ring:

```
sage: # needs sage.rings.number_field
sage: K = QuadraticField(5)
sage: O = K.maximal_order()
sage: R = PolynomialRing(O, 'x')
sage: F = R.fraction_field()
sage: x = F.gen(0)
sage: x.gcd(x+1)
1
sage: x.xgcd(x+1)
(1, 1/x, 0)
sage: zero = F.zero()
sage: zero.gcd(x)
1
sage: zero.xgcd(x)
(1, 0, 1/x)
sage: zero.xgcd(zero)
(0, 0, 0)
```

```
>>> from sage.all import *
>>> # needs sage.rings.number_field
>>> K = QuadraticField(Integer(5))
>>> O = K.maximal_order()
>>> R = PolynomialRing(O, 'x')
>>> F = R.fraction_field()
>>> x = F.gen(Integer(0))
>>> x.gcd(x+Integer(1))
1
>>> x.xgcd(x+Integer(1))
(1, 1/x, 0)
>>> zero = F.zero()
>>> zero.gcd(x)
1
>>> zero.xgcd(x)
(1, 0, 1/x)
>>> zero.xgcd(zero)
(0, 0, 0)
```

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: QuotientFields().super_categories()
[Category of fields]
```

```
>>> from sage.all import *
>>> QuotientFields().super_categories()
[Category of fields]
```

4.139 Quantum Group Representations

AUTHORS:

- Travis Scrimshaw (2018): initial version

class sage.categories.quantum_group_representations.**QuantumGroupRepresentations**(*base*, *name=None*)

Bases: *Category_module*

The category of quantum group representations.

class ParentMethods

Bases: object

cartan_type()

Return the Cartan type of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
sage: C = crystals.Tableaux(['C',4], shape=[1])
sage: R = ZZ['q'].fraction_field()
sage: V = MinusculeRepresentation(R, C)
sage: V.cartan_type()
['C', 4]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
>>> C = crystals.Tableaux(['C',Integer(4)], shape=[Integer(1)])
>>> R = ZZ['q'].fraction_field()
>>> V = MinusculeRepresentation(R, C)
>>> V.cartan_type()
['C', 4]
```

index_set()

Return the index set of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
```

(continues on next page)

(continued from previous page)

```
sage: C = crystals.Tableaux(['C',4], shape=[1])
sage: R = ZZ['q'].fraction_field()
sage: V = MinusculeRepresentation(R, C)
sage: V.index_set()
(1, 2, 3, 4)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
>>> C = crystals.Tableaux(['C',Integer(4)], shape=[Integer(1)])
>>> R = ZZ['q'].fraction_field()
>>> V = MinusculeRepresentation(R, C)
>>> V.index_set()
(1, 2, 3, 4)
```

q()

Return the quantum parameter q of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
sage: C = crystals.Tableaux(['C',4], shape=[1])
sage: R = ZZ['q'].fraction_field()
sage: V = MinusculeRepresentation(R, C)
sage: V.q()
q
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
>>> C = crystals.Tableaux(['C',Integer(4)], shape=[Integer(1)])
>>> R = ZZ['q'].fraction_field()
>>> V = MinusculeRepresentation(R, C)
>>> V.q()
q
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of quantum group representations constructed by tensor product of quantum group representations.

Warning

We use the reversed coproduct in order to match the tensor product rule on crystals.

class ParentMethods

Bases: object

cartan_type()

Return the Cartan type of `self`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
sage: C = crystals.Tableaux(['C',2], shape=[1])
sage: R = ZZ['q'].fraction_field()
sage: V = MinusculeRepresentation(R, C)
sage: T = tensor([V,V])
sage: T.cartan_type()
['C', 2]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪MinusculeRepresentation
>>> C = crystals.Tableaux(['C',Integer(2)], shape=[Integer(1)])
>>> R = ZZ['q'].fraction_field()
>>> V = MinusculeRepresentation(R, C)
>>> T = tensor([V,V])
>>> T.cartan_type()
['C', 2]
```

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
sage: Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
sage: Cat.TensorProducts().extra_super_categories()
[Category of quantum group representations over
 Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

```
>>> from sage.all import *
>>> from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
>>> Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
>>> Cat.TensorProducts().extra_super_categories()
[Category of quantum group representations over
 Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

class WithBasis(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of quantum group representations with a distinguished basis.

class ElementMethods

Bases: `object`

K(*i*, *power=1*)

Return the action of K_i on `self` to the power `power`.

INPUT:

- `i` – an element of the index set
- `power` – (default: 1) the power of K_i

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
sage: K = crystals.KirillovReshetikhin(['D',4,2], 1,1)
sage: R = ZZ['q'].fraction_field()
sage: V = AdjointRepresentation(R, K)
sage: v = V.an_element(); v
2*B[[]] + 2*B[[[1]]] + 3*B[[[2]]]
sage: v.K(0)
2*B[[]] + 2/q^2*B[[[1]]] + 3*B[[[2]]]
sage: v.K(1)
2*B[[]] + 2*q^2*B[[[1]]] + 3/q^2*B[[[2]]]
sage: v.K(1, 2)
2*B[[]] + 2*q^4*B[[[1]]] + 3/q^4*B[[[2]]]
sage: v.K(1, -1)
2*B[[]] + 2/q^2*B[[[1]]] + 3*q^2*B[[[2]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(2)],␣
↪Integer(1),Integer(1))
>>> R = ZZ['q'].fraction_field()
>>> V = AdjointRepresentation(R, K)
>>> v = V.an_element(); v
2*B[[]] + 2*B[[[1]]] + 3*B[[[2]]]
>>> v.K(Integer(0))
2*B[[]] + 2/q^2*B[[[1]]] + 3*B[[[2]]]
>>> v.K(Integer(1))
2*B[[]] + 2*q^2*B[[[1]]] + 3/q^2*B[[[2]]]
>>> v.K(Integer(1), Integer(2))
2*B[[]] + 2*q^4*B[[[1]]] + 3/q^4*B[[[2]]]
>>> v.K(Integer(1), -Integer(1))
2*B[[]] + 2/q^2*B[[[1]]] + 3*q^2*B[[[2]]]
```

e(i)

Return the action of e_i on `self`.

INPUT:

- `i` – an element of the index set

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
sage: C = crystals.Tableaux(['G',2], shape=[1,1])
sage: R = ZZ['q'].fraction_field()
sage: V = AdjointRepresentation(R, C)
```

(continues on next page)

(continued from previous page)

```
sage: v = V.an_element(); v
2*B[[1], [2]]] + 2*B[[1], [3]]] + 3*B[[2], [3]]]
sage: v.e(1)
((3*q^4+3*q^2+3)/q^2)*B[[1], [3]]]
sage: v.e(2)
2*B[[1], [2]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
>>> C = crystals.Tableaux(['G',Integer(2)], shape=[Integer(1),
↪Integer(1)])
>>> R = ZZ['q'].fraction_field()
>>> V = AdjointRepresentation(R, C)
>>> v = V.an_element(); v
2*B[[1], [2]]] + 2*B[[1], [3]]] + 3*B[[2], [3]]]
>>> v.e(Integer(1))
((3*q^4+3*q^2+3)/q^2)*B[[1], [3]]]
>>> v.e(Integer(2))
2*B[[1], [2]]]
```

f(*i*)

Return the action of f_i on `self`.

INPUT:

- `i` – an element of the index set

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
sage: K = crystals.KirillovReshetikhin(['D',4,1], 2,1)
sage: R = ZZ['q'].fraction_field()
sage: V = AdjointRepresentation(R, K)
sage: v = V.an_element(); v
2*B[[]] + 2*B[[1], [2]]] + 3*B[[1], [3]]]
sage: v.f(0)
((2*q^2+2)/q)*B[[1], [2]]]
sage: v.f(1)
3*B[[2], [3]]]
sage: v.f(2)
2*B[[1], [3]]]
sage: v.f(3)
3*B[[1], [4]]]
sage: v.f(4)
3*B[[1], [-4]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import␣
↪AdjointRepresentation
```

(continues on next page)

(continued from previous page)

```
>>> K = crystals.KirillovReshetikhin(['D',Integer(4),Integer(1)],␣
↪Integer(2),Integer(1))
>>> R = ZZ['q'].fraction_field()
>>> V = AdjointRepresentation(R, K)
>>> v = V.an_element(); v
2*B[[]] + 2*B[[[1], [2]]] + 3*B[[[1], [3]]]
>>> v.f(Integer(0))
((2*q^2+2)/q)*B[[[1], [2]]]
>>> v.f(Integer(1))
3*B[[[2], [3]]]
>>> v.f(Integer(2))
2*B[[[1], [3]]]
>>> v.f(Integer(3))
3*B[[[1], [4]]]
>>> v.f(Integer(4))
3*B[[[1], [-4]]]
```

class ParentMethods

Bases: object

tensor(**factors*)

Return the tensor product of `self` with the representations `factors`.

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import (
....:     MinusculeRepresentation, AdjointRepresentation)
sage: R = ZZ['q'].fraction_field()
sage: CM = crystals.Tableaux(['D',4], shape=[1])
sage: CA = crystals.Tableaux(['D',4], shape=[1,1])
sage: V = MinusculeRepresentation(R, CM)
sage: V.tensor(V, V)
V((1, 0, 0, 0)) # V((1, 0, 0, 0)) # V((1, 0, 0, 0))
sage: A = MinusculeRepresentation(R, CA)
sage: V.tensor(A)
V((1, 0, 0, 0)) # V((1, 1, 0, 0))
sage: B = crystals.Tableaux(['A',2], shape=[1])
sage: W = MinusculeRepresentation(R, B)
sage: tensor([W,V])
Traceback (most recent call last):
...
ValueError: all factors must be of the same Cartan type
sage: tensor([V,A,W])
Traceback (most recent call last):
...
ValueError: all factors must be of the same Cartan type
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import (
...     MinusculeRepresentation, AdjointRepresentation)
```

(continues on next page)

(continued from previous page)

```
>>> R = ZZ['q'].fraction_field()
>>> CM = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1)])
>>> CA = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1),
↪Integer(1)])
>>> V = MinusculeRepresentation(R, CM)
>>> V.tensor(V, V)
V((1, 0, 0, 0)) # V((1, 0, 0, 0)) # V((1, 0, 0, 0))
>>> A = MinusculeRepresentation(R, CA)
>>> V.tensor(A)
V((1, 0, 0, 0)) # V((1, 1, 0, 0))
>>> B = crystals.Tableaux(['A',Integer(2)], shape=[Integer(1)])
>>> W = MinusculeRepresentation(R, B)
>>> tensor([W,V])
Traceback (most recent call last):
...
ValueError: all factors must be of the same Cartan type
>>> tensor([V,A,W])
Traceback (most recent call last):
...
ValueError: all factors must be of the same Cartan type
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of quantum group representations with a distinguished basis constructed by tensor product of quantum group representations with a distinguished basis.

class ParentMethods

Bases: object

K_on_basis(*i*, *b*, *power=1*)

Return the action of K_i on the basis element indexed by `b` to the power `power`.

INPUT:

- `i` – an element of the index set
- `b` – an element of basis keys
- `power` – (default: 1) the power of K_i

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import (
....:     MinusculeRepresentation, AdjointRepresentation)
sage: R = ZZ['q'].fraction_field()
sage: CM = crystals.Tableaux(['A',2], shape=[1])
sage: VM = MinusculeRepresentation(R, CM)
sage: CA = crystals.Tableaux(['A',2], shape=[2,1])
sage: VA = AdjointRepresentation(R, CA)
sage: v = tensor([sum(VM.basis()), VA.module_generator()]); v
B[[[1]]] # B[[[1, 1], [2]]]
 + B[[[2]]] # B[[[1, 1], [2]]]
 + B[[[3]]] # B[[[1, 1], [2]]]
sage: v.K(1)  # indirect doctest
q^2*B[[[1]]] # B[[[1, 1], [2]]]
 + B[[[2]]] # B[[[1, 1], [2]]]
```

(continues on next page)

(continued from previous page)

```
 + q*B[[[3]]] # B[[[1, 1], [2]]]
sage: v.K(2, -1)  # indirect doctest
1/q*B[[[1]]] # B[[[1, 1], [2]]]
 + 1/q^2*B[[[2]]] # B[[[1, 1], [2]]]
 + B[[[3]]] # B[[[1, 1], [2]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import (
...     MinusculeRepresentation, AdjointRepresentation)
>>> R = ZZ['q'].fraction_field()
>>> CM = crystals.Tableaux(['A',Integer(2)], shape=[Integer(1)])
>>> VM = MinusculeRepresentation(R, CM)
>>> CA = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),
↪Integer(1)])
>>> VA = AdjointRepresentation(R, CA)
>>> v = tensor([sum(VM.basis()), VA.module_generator()]); v
B[[[1]]] # B[[[1, 1], [2]]]
 + B[[[2]]] # B[[[1, 1], [2]]]
 + B[[[3]]] # B[[[1, 1], [2]]]
>>> v.K(Integer(1))  # indirect doctest
q^2*B[[[1]]] # B[[[1, 1], [2]]]
 + B[[[2]]] # B[[[1, 1], [2]]]
 + q*B[[[3]]] # B[[[1, 1], [2]]]
>>> v.K(Integer(2), -Integer(1))  # indirect doctest
1/q*B[[[1]]] # B[[[1, 1], [2]]]
 + 1/q^2*B[[[2]]] # B[[[1, 1], [2]]]
 + B[[[3]]] # B[[[1, 1], [2]]]
```

e_on_basis(*i*, *b*)

Return the action of e_i on the basis element indexed by `b`.

INPUT:

- `i` – an element of the index set
- `b` – an element of basis keys

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import (
....:     MinusculeRepresentation, AdjointRepresentation)
sage: R = ZZ['q'].fraction_field()
sage: CM = crystals.Tableaux(['D',4], shape=[1])
sage: VM = MinusculeRepresentation(R, CM)
sage: CA = crystals.Tableaux(['D',4], shape=[1,1])
sage: VA = AdjointRepresentation(R, CA)
sage: v = tensor([VM.an_element(), VA.an_element()]); v
4*B[[[1]]] # B[[[1], [2]]] + 4*B[[[1]]] # B[[[1], [3]]]
 + 6*B[[[1]]] # B[[[2], [3]]] + 4*B[[[2]]] # B[[[1], [2]]]
 + 4*B[[[2]]] # B[[[1], [3]]] + 6*B[[[2]]] # B[[[2], [3]]]
 + 6*B[[[3]]] # B[[[1], [2]]] + 6*B[[[3]]] # B[[[1], [3]]]
 + 9*B[[[3]]] # B[[[2], [3]]]
sage: v.e(1)  # indirect doctest
```

(continues on next page)

(continued from previous page)

```
4*B[[[1]]] # B[[[1], [2]]]
 + ((4*q+6)/q)*B[[[1]]] # B[[[1], [3]]]
 + 6*B[[[1]]] # B[[[2], [3]]]
 + 6*q*B[[[2]]] # B[[[1], [3]]]
 + 9*B[[[3]]] # B[[[1], [3]]]
sage: v.e(2)  # indirect doctest
4*B[[[1]]] # B[[[1], [2]]]
 + ((6*q+4)/q)*B[[[2]]] # B[[[1], [2]]]
 + 6*B[[[2]]] # B[[[1], [3]]]
 + 9*B[[[2]]] # B[[[2], [3]]]
 + 6*q*B[[[3]]] # B[[[1], [2]]]
sage: v.e(3)  # indirect doctest
0
sage: v.e(4)  # indirect doctest
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import (
...     MinusculeRepresentation, AdjointRepresentation)
>>> R = ZZ['q'].fraction_field()
>>> CM = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1)])
>>> VM = MinusculeRepresentation(R, CM)
>>> CA = crystals.Tableaux(['D',Integer(4)], shape=[Integer(1),
↪Integer(1)])
>>> VA = AdjointRepresentation(R, CA)
>>> v = tensor([VM.an_element(), VA.an_element()]); v
4*B[[[1]]] # B[[[1], [2]]] + 4*B[[[1]]] # B[[[1], [3]]]
 + 6*B[[[1]]] # B[[[2], [3]]] + 4*B[[[2]]] # B[[[1], [2]]]
 + 4*B[[[2]]] # B[[[1], [3]]] + 6*B[[[2]]] # B[[[2], [3]]]
 + 6*B[[[3]]] # B[[[1], [2]]] + 6*B[[[3]]] # B[[[1], [3]]]
 + 9*B[[[3]]] # B[[[2], [3]]]
>>> v.e(Integer(1))  # indirect doctest
4*B[[[1]]] # B[[[1], [2]]]
 + ((4*q+6)/q)*B[[[1]]] # B[[[1], [3]]]
 + 6*B[[[1]]] # B[[[2], [3]]]
 + 6*q*B[[[2]]] # B[[[1], [3]]]
 + 9*B[[[3]]] # B[[[1], [3]]]
>>> v.e(Integer(2))  # indirect doctest
4*B[[[1]]] # B[[[1], [2]]]
 + ((6*q+4)/q)*B[[[2]]] # B[[[1], [2]]]
 + 6*B[[[2]]] # B[[[1], [3]]]
 + 9*B[[[2]]] # B[[[2], [3]]]
 + 6*q*B[[[3]]] # B[[[1], [2]]]
>>> v.e(Integer(3))  # indirect doctest
0
>>> v.e(Integer(4))  # indirect doctest
0
```

f_on_basis(*i*, *b*)

Return the action of f_i on the basis element indexed by `b`.

INPUT:

- `i` – an element of the index set
- `b` – an element of basis keys

EXAMPLES:

```
sage: # needs sage.combinat sage.graphs sage.modules
sage: from sage.algebras.quantum_groups.representations import (
....:     MinusculeRepresentation, AdjointRepresentation)
sage: R = ZZ['q'].fraction_field()
sage: KM = crystals.KirillovReshetikhin(['B',3,1], 3,1)
sage: VM = MinusculeRepresentation(R, KM)
sage: KA = crystals.KirillovReshetikhin(['B',3,1], 2,1)
sage: VA = AdjointRepresentation(R, KA)
sage: v = tensor([VM.an_element(), VA.an_element()]); v
4*B[[+++, []]] # B[[]] + 4*B[[+++, []]] # B[[[1], [2]]]
 + 6*B[[+++, []]] # B[[[1], [3]]] + 4*B[[++-, []]] # B[[]]
 + 4*B[[++-, []]] # B[[[1], [2]]]
 + 6*B[[++-, []]] # B[[[1], [3]]] + 6*B[[+-+, []]] # B[[]]
 + 6*B[[+-+, []]] # B[[[1], [2]]]
 + 9*B[[+-+, []]] # B[[[1], [3]]]
sage: v.f(0)  # indirect doctest
((4*q^4+4)/q^2)*B[[+++, []]] # B[[[1], [2]]]
 + ((4*q^4+4)/q^2)*B[[++-, []]] # B[[[1], [2]]]
 + ((6*q^4+6)/q^2)*B[[+-+, []]] # B[[[1], [2]]]
sage: v.f(1)  # indirect doctest
6*B[[+++, []]] # B[[[2], [3]]]
 + 6*B[[++-, []]] # B[[[2], [3]]]
 + 9*B[[+-+, []]] # B[[[2], [3]]]
 + 6*B[[-++, []]] # B[[]]
 + 6*B[[-++, []]] # B[[[1], [2]]]
 + 9*q^2*B[[-++, []]] # B[[[1], [3]]]
sage: v.f(2)  # indirect doctest
4*B[[+++, []]] # B[[[1], [3]]]
 + 4*B[[++-, []]] # B[[[1], [3]]]
 + 4*B[[+-+, []]] # B[[]]
 + 4*q^2*B[[+-+, []]] # B[[[1], [2]]]
 + ((6*q^2+6)/q^2)*B[[+-+, []]] # B[[[1], [3]]]
sage: v.f(3)  # indirect doctest
6*B[[+++, []]] # B[[[1], [0]]]
 + 4*B[[++-, []]] # B[[]]
 + 4*B[[++-, []]] # B[[[1], [2]]]
 + 6*q^2*B[[++-, []]] # B[[[1], [3]]]
 + 6*B[[++-, []]] # B[[[1], [0]]]
 + 9*B[[+-+, []]] # B[[[1], [0]]]
 + 6*B[[+--, []]] # B[[]]
 + 6*B[[+--, []]] # B[[[1], [2]]]
 + 9*q^2*B[[+--, []]] # B[[[1], [3]]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs sage.modules
>>> from sage.algebras.quantum_groups.representations import (
...     MinusculeRepresentation, AdjointRepresentation)
>>> R = ZZ['q'].fraction_field()
>>> KM = crystals.KirillovReshetikhin(['B',Integer(3),Integer(1)],␣
```

(continues on next page)

(continued from previous page)

```
↪Integer(3),Integer(1))
>>> VM = MinusculeRepresentation(R, KM)
>>> KA = crystals.KirillovReshetikhin(['B',Integer(3),Integer(1)],␣
↪Integer(2),Integer(1))
>>> VA = AdjointRepresentation(R, KA)
>>> v = tensor([VM.an_element(), VA.an_element()]); v
4*B[[+++, []]] # B[[]] + 4*B[[+++, []]] # B[[[1], [2]]]
 + 6*B[[+++, []]] # B[[[1], [3]]] + 4*B[[++-, []]] # B[[]]
 + 4*B[[++-, []]] # B[[[1], [2]]]
 + 6*B[[++-, []]] # B[[[1], [3]]] + 6*B[[+-+, []]] # B[[]]
 + 6*B[[+-+, []]] # B[[[1], [2]]]
 + 9*B[[+-+, []]] # B[[[1], [3]]]
>>> v.f(Integer(0))  # indirect doctest
((4*q^4+4)/q^2)*B[[+++, []]] # B[[[1], [2]]]
 + ((4*q^4+4)/q^2)*B[[++-, []]] # B[[[1], [2]]]
 + ((6*q^4+6)/q^2)*B[[+-+, []]] # B[[[1], [2]]]
>>> v.f(Integer(1))  # indirect doctest
6*B[[+++, []]] # B[[[2], [3]]]
 + 6*B[[++-, []]] # B[[[2], [3]]]
 + 9*B[[+-+, []]] # B[[[2], [3]]]
 + 6*B[[-++, []]] # B[[]]
 + 6*B[[-++, []]] # B[[[1], [2]]]
 + 9*q^2*B[[-++, []]] # B[[[1], [3]]]
>>> v.f(Integer(2))  # indirect doctest
4*B[[+++, []]] # B[[[1], [3]]]
 + 4*B[[++-, []]] # B[[[1], [3]]]
 + 4*B[[+-+, []]] # B[[]]
 + 4*q^2*B[[+-+, []]] # B[[[1], [2]]]
 + ((6*q^2+6)/q^2)*B[[+-+, []]] # B[[[1], [3]]]
>>> v.f(Integer(3))  # indirect doctest
6*B[[+++, []]] # B[[[1], [0]]]
 + 4*B[[++-, []]] # B[[]]
 + 4*B[[++-, []]] # B[[[1], [2]]]
 + 6*q^2*B[[++-, []]] # B[[[1], [3]]]
 + 6*B[[++-, []]] # B[[[1], [0]]]
 + 9*B[[+-+, []]] # B[[[1], [0]]]
 + 6*B[[+--, []]] # B[[]]
 + 6*B[[+--, []]] # B[[[1], [2]]]
 + 9*q^2*B[[+--, []]] # B[[[1], [3]]]
```

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
sage: Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
sage: Cat.WithBasis().TensorProducts().extra_super_categories()
[Category of quantum group representations with basis over
  Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

```
>>> from sage.all import *
>>> from sage.categories.quantum_group_representations import␣
```

(continues on next page)

(continued from previous page)

```
↪QuantumGroupRepresentations
>>> Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
>>> Cat.WithBasis().TensorProducts().extra_super_categories()
[Category of quantum group representations with basis over
  Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

example()

Return an example of a quantum group representation as per *Category.example*.

EXAMPLES:

```
sage: from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
sage: Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
sage: Cat.example()                                                    #␣
↪needs sage.combinat sage.graphs sage.modules
V((2, 1, 0))
```

```
>>> from sage.all import *
>>> from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
>>> Cat = QuantumGroupRepresentations(ZZ['q'].fraction_field())
>>> Cat.example()                                                      #␣
↪needs sage.combinat sage.graphs sage.modules
V((2, 1, 0))
```

super_categories()

Return the super categories of `self`.

EXAMPLES:

```
sage: from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
sage: QuantumGroupRepresentations(ZZ['q'].fraction_field()).super_categories()
[Category of vector spaces over
  Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

```
>>> from sage.all import *
>>> from sage.categories.quantum_group_representations import␣
↪QuantumGroupRepresentations
>>> QuantumGroupRepresentations(ZZ['q'].fraction_field()).super_categories()
[Category of vector spaces over
  Fraction Field of Univariate Polynomial Ring in q over Integer Ring]
```

4.140 Regular Crystals

class sage.categories.regular_crystals.**RegularCrystals**

Bases: *Category_singleton*

The category of regular crystals.

A crystal is called *regular* if every vertex b satisfies

$$\varepsilon_i(b) = \max\{k \mid e_i^k(b) \neq 0\} \quad \text{and} \quad \varphi_i(b) = \max\{k \mid f_i^k(b) \neq 0\}.$$

Note

Regular crystals are sometimes referred to as *normal*. When only one of the conditions (on either φ_i or ε_i) holds, these crystals are sometimes called *seminormal* or *semiregular*.

EXAMPLES:

```
sage: C = RegularCrystals()
sage: C
Category of regular crystals
sage: C.super_categories()
[Category of crystals]
sage: C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> C = RegularCrystals()
>>> C
Category of regular crystals
>>> C.super_categories()
[Category of crystals]
>>> C.example()
Highest weight crystal of type A_3 of highest weight omega_1
```

class ElementMethods

Bases: object

demazure_operator_simple(*i*, *ring=None*)

Return the Demazure operator D_i applied to `self`.

INPUT:

- `i` – an element of the index set of the underlying crystal
- `ring` – (default: `QQ`) a ring

OUTPUT:

An element of the `ring`-free module indexed by the underlying crystal.

Let $r = \langle \mathrm{wt}(b), \alpha_i^\vee \rangle$, then $D_i(b)$ is defined as follows:

- If $r \geq 0$, this returns the sum of the elements obtained from `self` by application of f_i^k for $0 \leq k \leq r$.
- If $r < 0$, this returns the opposite of the sum of the elements obtained by application of e_i^k for $0 < k < -r$.

REFERENCES:

- [Li1995]
- [Ka1993]

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,2],[2]])
```

(continues on next page)

(continued from previous page)

```
sage: t.demazure_operator_simple(2)
B[[1, 2], [2]]] + B[[1, 3], [2]]] + B[[1, 3], [3]]]
sage: t.demazure_operator_simple(2).parent()
Algebra of The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]]
        over Integer Ring

sage: t.demazure_operator_simple(1)
0

sage: K = crystals.KirillovReshetikhin(['A',2,1],2,1)
sage: t = K(rows=[[3],[2]])
sage: t.demazure_operator_simple(0)
B[[1, 2]]] + B[[2, 3]]]
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(2)],[Integer(2)]])
>>> t.demazure_operator_simple(Integer(2))
B[[1, 2], [2]]] + B[[1, 3], [2]]] + B[[1, 3], [3]]]
>>> t.demazure_operator_simple(Integer(2)).parent()
Algebra of The crystal of tableaux of type ['A', 2] and shape(s) [[2, 1]]
        over Integer Ring

>>> t.demazure_operator_simple(Integer(1))
0

>>> K = crystals.KirillovReshetikhin(['A',Integer(2),Integer(1)],
↪Integer(2),Integer(1))
>>> t = K(rows=[[Integer(3)],[Integer(2)]])
>>> t.demazure_operator_simple(Integer(0))
B[[1, 2]]] + B[[2, 3]]]
```

dual_equivalence_class(*index_set=None*)

Return the dual equivalence class indexed by `index_set` of `self`.

The dual equivalence class of an element $b \in B$ is the set of all elements of B reachable from b via sequences of i-elementary dual equivalence relations (i.e., i-elementary dual equivalence transformations and their inverses) for i in the index set of B.

For this to be well-defined, the element b has to be of weight 0 with respect to I; that is, we need to have $\varepsilon_j(b) = \varphi_j(b)$ for all $j \in I$.

See [As2008]. See also *dual_equivalence_graph()* for a definition of i-elementary dual equivalence transformations.

INPUT:

- `index_set` – (optional) the index set I (default: the whole index set of the crystal); this has to be a subset of the index set of the crystal (as a list or tuple)

OUTPUT:

The dual equivalence class of `self` indexed by the subset `index_set`. This class is returned as an undirected edge-colored multigraph. The color of an edge is the index i of the dual equivalence relation it encodes.

See also

- *dual_equivalence_graph()*
- sage.combinat.partition.Partition.dual_equivalence_graph()

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',3], shape=[2,2])
sage: G = T(2,1,4,3).dual_equivalence_class()
sage: G.edges(sort=True)
[([[1, 3], [2, 4]], [[1, 2], [3, 4]], 2),
 ([[1, 3], [2, 4]], [[1, 2], [3, 4]], 3)]
sage: T = crystals.Tableaux(['A',4], shape=[3,2])
sage: G = T(2,1,4,3,5).dual_equivalence_class()
sage: G.edges(sort=True)
[([[1, 3, 4], [2, 5]], [[1, 3, 5], [2, 4]], 4),
 ([[1, 3, 4], [2, 5]], [[1, 2, 4], [3, 5]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 4)]
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(3)], shape=[Integer(2),Integer(2)])
>>> G = T(Integer(2),Integer(1),Integer(4),Integer(3)).dual_equivalence_
↪class()
>>> G.edges(sort=True)
[([[1, 3], [2, 4]], [[1, 2], [3, 4]], 2),
 ([[1, 3], [2, 4]], [[1, 2], [3, 4]], 3)]
>>> T = crystals.Tableaux(['A',Integer(4)], shape=[Integer(3),Integer(2)])
>>> G = T(Integer(2),Integer(1),Integer(4),Integer(3),Integer(5)).dual_
↪equivalence_class()
>>> G.edges(sort=True)
[([[1, 3, 4], [2, 5]], [[1, 3, 5], [2, 4]], 4),
 ([[1, 3, 4], [2, 5]], [[1, 2, 4], [3, 5]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 4)]
```

epsilon(i)

Return ε_i of `self`.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).epsilon(1)
0
sage: C(2).epsilon(1)
1
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).epsilon(Integer(1))
0
>>> C(Integer(2)).epsilon(Integer(1))
1
```

phi(*i*)

Return φ_i of `self`.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).phi(1)
1
sage: C(2).phi(1)
0
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).phi(Integer(1))
1
>>> C(Integer(2)).phi(Integer(1))
0
```

stembridgeDel_depth(*i*, *j*)

Return the difference in the j-depth of `self` and f_i of `self`, where i and j are in the index set of the underlying crystal. This function is useful for checking the Stembridge local axioms for crystal bases.

The i-depth of a crystal node x is $\varepsilon_i(x)$.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,1],[2]])
sage: t.stembridgeDel_depth(1,2)
0
sage: s = T(rows=[[1,3],[3]])
sage: s.stembridgeDel_depth(1,2)
-1
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(1)],[Integer(2)]])
>>> t.stembridgeDel_depth(Integer(1),Integer(2))
0
>>> s = T(rows=[[Integer(1),Integer(3)],[Integer(3)]])
>>> s.stembridgeDel_depth(Integer(1),Integer(2))
-1
```

stembridgeDel_rise(*i*, *j*)

Return the difference in the j-rise of `self` and f_i of `self`, where i and j are in the index set of the underlying crystal. This function is useful for checking the Stembridge local axioms for crystal bases.

The i-rise of a crystal node x is $\varphi_i(x)$.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,1],[2]])
sage: t.stembridgeDel_rise(1,2)
-1
sage: s = T(rows=[[1,3],[3]])
sage: s.stembridgeDel_rise(1,2)
0
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(1)],[Integer(2)]])
>>> t.stembridgeDel_rise(Integer(1),Integer(2))
-1
>>> s = T(rows=[[Integer(1),Integer(3)],[Integer(3)]])
>>> s.stembridgeDel_rise(Integer(1),Integer(2))
0
```

stembridgeDelta_depth(i, j)

Return the difference in the j-depth of `self` and e_i of `self`, where i and j are in the index set of the underlying crystal. This function is useful for checking the Stembridge local axioms for crystal bases.

The i-depth of a crystal node x is $-\varepsilon_i(x)$.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,2],[2]])
sage: t.stembridgeDelta_depth(1,2)
0
sage: s = T(rows=[[2,3],[3]])
sage: s.stembridgeDelta_depth(1,2)
-1
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(2)],[Integer(2)]])
>>> t.stembridgeDelta_depth(Integer(1),Integer(2))
0
>>> s = T(rows=[[Integer(2),Integer(3)],[Integer(3)]])
>>> s.stembridgeDelta_depth(Integer(1),Integer(2))
-1
```

stembridgeDelta_rise(i, j)

Return the difference in the j-rise of `self` and e_i of `self`, where i and j are in the index set of the underlying crystal. This function is useful for checking the Stembridge local axioms for crystal bases.

The i-rise of a crystal node x is $\varphi_i(x)$.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,2],[2]])
sage: t.stembridgeDelta_rise(1,2)
```

(continues on next page)

(continued from previous page)

```
-1
sage: s = T(rows=[[2,3],[3]])
sage: s.stembridgeDelta_rise(1,2)
0
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(2)],[Integer(2)]])
>>> t.stembridgeDelta_rise(Integer(1),Integer(2))
-1
>>> s = T(rows=[[Integer(2),Integer(3)],[Integer(3)]])
>>> s.stembridgeDelta_rise(Integer(1),Integer(2))
0
```

stembridgeTriple(i, j)

Let A be the Cartan matrix of the crystal, x a crystal element, and let i and j be in the index set of the crystal. Further, set `b=stembridgeDelta_depth(x,i,j)`, and `c=stembridgeDelta_rise(x,i,j))`. If `x.e(i)` is non-empty, this function returns the triple (A_{ij}, b, c); otherwise it returns `None`. By the Stembridge local characterization of crystal bases, one should have $A_{ij} = b + c$.

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T(rows=[[1,1],[2]])
sage: t.stembridgeTriple(1,2)
sage: s = T(rows=[[1,2],[2]])
sage: s.stembridgeTriple(1,2)
(-1, 0, -1)

sage: T = crystals.Tableaux(['B',2], shape=[2,1])
sage: t = T(rows=[[1,2],[2]])
sage: t.stembridgeTriple(1,2)
(-2, 0, -2)
sage: s = T(rows=[[-1,-1],[0]])
sage: s.stembridgeTriple(1,2)
(-2, -2, 0)
sage: u = T(rows=[[0,2],[1]])
sage: u.stembridgeTriple(1,2)
(-2, -1, -1)
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(1)],[Integer(2)]])
>>> t.stembridgeTriple(Integer(1),Integer(2))
>>> s = T(rows=[[Integer(1),Integer(2)],[Integer(2)]])
>>> s.stembridgeTriple(Integer(1),Integer(2))
(-1, 0, -1)

>>> T = crystals.Tableaux(['B',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T(rows=[[Integer(1),Integer(2)],[Integer(2)]])
>>> t.stembridgeTriple(Integer(1),Integer(2))
(-2, 0, -2)
```

(continues on next page)

(continued from previous page)

```
>>> s = T(rows=[[-Integer(1),-Integer(1)],[Integer(0)]])
>>> s.stembridgeTriple(Integer(1),Integer(2))
(-2, -2, 0)
>>> u = T(rows=[[Integer(0),Integer(2)],[Integer(1)]])
>>> u.stembridgeTriple(Integer(1),Integer(2))
(-2, -1, -1)
```

weight()

Return the weight of this crystal element.

EXAMPLES:

```
sage: C = crystals.Letters(['A',5])
sage: C(1).weight()
(1, 0, 0, 0, 0, 0)
```

```
>>> from sage.all import *
>>> C = crystals.Letters(['A',Integer(5)])
>>> C(Integer(1)).weight()
(1, 0, 0, 0, 0, 0)
```

class MorphismMethods

Bases: object

is_isomorphism()

Check if self is a crystal isomorphism, which is true if and only if this is a strict embedding with the same number of connected components.

EXAMPLES:

```
sage: A21 = RootSystem(['A',2,1])
sage: La = A21.weight_space(extended=True).fundamental_weights()
sage: B = crystals.LSPaths(La[0])
sage: La = A21.weight_lattice(extended=True).fundamental_weights()
sage: C = crystals.GeneralizedYoungWalls(2, La[0])
sage: H = Hom(B, C)
sage: from sage.categories.highest_weight_crystals import␣
↪HighestWeightCrystalMorphism
sage: class Psi(HighestWeightCrystalMorphism):
....:     def is_strict(self):
....:         return True
sage: psi = Psi(H, C.module_generators); psi
['A', 2, 1] Crystal morphism:
  From: The crystal of LS paths of type ['A', 2, 1] and weight Lambda[0]
  To:   Highest weight crystal of generalized Young walls
        of Cartan type ['A', 2, 1] and highest weight Lambda[0]
  Defn: (Lambda[0],) |--> []
sage: psi.is_isomorphism()
True
```

```
>>> from sage.all import *
>>> A21 = RootSystem(['A',Integer(2),Integer(1)])
>>> La = A21.weight_space(extended=True).fundamental_weights()
```

(continues on next page)

(continued from previous page)

```
>>> B = crystals.LSPaths(La[Integer(0)])
>>> La = A21.weight_lattice(extended=True).fundamental_weights()
>>> C = crystals.GeneralizedYoungWalls(Integer(2), La[Integer(0)])
>>> H = Hom(B, C)
>>> from sage.categories.highest_weight_crystals import␣
↪HighestWeightCrystalMorphism
>>> class Psi(HighestWeightCrystalMorphism):
...     def is_strict(self):
...         return True
>>> psi = Psi(H, C.module_generators); psi
['A', 2, 1] Crystal morphism:
  From: The crystal of LS paths of type ['A', 2, 1] and weight Lambda[0]
  To:   Highest weight crystal of generalized Young walls
        of Cartan type ['A', 2, 1] and highest weight Lambda[0]
  Defn: (Lambda[0],) |--> []
>>> psi.is_isomorphism()
True
```

class ParentMethods

Bases: `object`

demazure_operator(*element*, *reduced_word*)

Return the application of Demazure operators D_i for i from `reduced_word` on `element`.

INPUT:

- `element` – an element of a free module indexed by the underlying crystal
- `reduced_word` – a reduced word of the Weyl group of the same type as the underlying crystal

OUTPUT: an element of the free module indexed by the underlying crystal

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: C = CombinatorialFreeModule(QQ, T)                                #␣
↪needs sage.modules
sage: t = T.highest_weight_vector()
sage: b = 2*C(t)                                                        #␣
↪needs sage.modules
sage: T.demazure_operator(b,[1,2,1])                                    #␣
↪needs sage.modules
2*B[[[1, 1], [2]]] + 2*B[[[1, 2], [2]]] + 2*B[[[1, 3], [2]]]
+ 2*B[[[1, 1], [3]]] + 2*B[[[1, 2], [3]]] + 2*B[[[1, 3], [3]]]
+ 2*B[[[2, 2], [3]]] + 2*B[[[2, 3], [3]]]
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> C = CombinatorialFreeModule(QQ, T)                                  #␣
↪needs sage.modules
>>> t = T.highest_weight_vector()
>>> b = Integer(2)*C(t)                                                 ␣
↪     # needs sage.modules
>>> T.demazure_operator(b,[Integer(1),Integer(2),Integer(1)])           ␣
↪                        # needs sage.modules
2*B[[[1, 1], [2]]] + 2*B[[[1, 2], [2]]] + 2*B[[[1, 3], [2]]]
```

(continues on next page)

(continued from previous page)

```
+ 2*B[[[1, 1], [3]]] + 2*B[[[1, 2], [3]]] + 2*B[[[1, 3], [3]]]
+ 2*B[[[2, 2], [3]]] + 2*B[[[2, 3], [3]]]
```

The Demazure operator is idempotent:

```
sage: # needs sage.modules
sage: T = crystals.Tableaux("A1", shape=[4])
sage: C = CombinatorialFreeModule(QQ, T)
sage: b = C(T.module_generators[0]); b
B[[[1, 1, 1, 1]]]
sage: e = T.demazure_operator(b,[1]); e
B[[[1, 1, 1, 1]]] + B[[[1, 1, 1, 2]]] + B[[[1, 1, 2, 2]]]
+ B[[[1, 2, 2, 2]]] + B[[[2, 2, 2, 2]]]
sage: e == T.demazure_operator(e,[1])
True
sage: all(T.demazure_operator(T.demazure_operator(C(t),[1]),[1])
....:       == T.demazure_operator(C(t),[1]) for t in T)
True
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> T = crystals.Tableaux("A1", shape=[Integer(4)])
>>> C = CombinatorialFreeModule(QQ, T)
>>> b = C(T.module_generators[Integer(0)]); b
B[[[1, 1, 1, 1]]]
>>> e = T.demazure_operator(b,[Integer(1)]); e
B[[[1, 1, 1, 1]]] + B[[[1, 1, 1, 2]]] + B[[[1, 1, 2, 2]]]
+ B[[[1, 2, 2, 2]]] + B[[[2, 2, 2, 2]]]
>>> e == T.demazure_operator(e,[Integer(1)])
True
>>> all(T.demazure_operator(T.demazure_operator(C(t),[Integer(1)]),
↪[Integer(1)])
...       == T.demazure_operator(C(t),[Integer(1)]) for t in T)
True
```

demazure_subcrystal(*element*, *reduced_word*, *only_support=True*)

Return the subcrystal corresponding to the application of Demazure operators D_i for i from `reduced_word` on `element`.

INPUT:

- `element` – an element of a free module indexed by the underlying crystal
- `reduced_word` – a reduced word of the Weyl group of the same type as the underlying crystal
- `only_support` – boolean (default: `True`); only include arrows corresponding to the support of `reduced_word`

OUTPUT: the Demazure subcrystal

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',2], shape=[2,1])
sage: t = T.highest_weight_vector()
sage: S = T.demazure_subcrystal(t, [1,2])
sage: list(S)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
```

(continues on next page)

(continued from previous page)

```
 [[1, 2], [3]], [[2, 2], [3]]]
sage: S = T.demazure_subcrystal(t, [2,1])
sage: list(S)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
 [[1, 3], [2]], [[1, 3], [3]]]
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(2)], shape=[Integer(2),Integer(1)])
>>> t = T.highest_weight_vector()
>>> S = T.demazure_subcrystal(t, [Integer(1),Integer(2)])
>>> list(S)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
 [[1, 2], [3]], [[2, 2], [3]]]
>>> S = T.demazure_subcrystal(t, [Integer(2),Integer(1)])
>>> list(S)
[[[1, 1], [2]], [[1, 2], [2]], [[1, 1], [3]],
 [[1, 3], [2]], [[1, 3], [3]]]
```

We construct an example where we don't only want the arrows indicated by the support of the reduced word:

```
sage: K = crystals.KirillovReshetikhin(['A',1,1], 1, 2)
sage: mg = K.module_generator()
sage: S = K.demazure_subcrystal(mg, [1])
sage: S.digraph().edges(sort=True)
[([[1, 1]], [[1, 2]], 1), ([[1, 2]], [[2, 2]], 1)]
sage: S = K.demazure_subcrystal(mg, [1], only_support=False)
sage: S.digraph().edges(sort=True)
[([[1, 1]], [[1, 2]], 1),
 ([[1, 2]], [[1, 1]], 0),
 ([[1, 2]], [[2, 2]], 1),
 ([[2, 2]], [[1, 2]], 0)]
```

```
>>> from sage.all import *
>>> K = crystals.KirillovReshetikhin(['A',Integer(1),Integer(1)],␣
↪Integer(1), Integer(2))
>>> mg = K.module_generator()
>>> S = K.demazure_subcrystal(mg, [Integer(1)])
>>> S.digraph().edges(sort=True)
[([[1, 1]], [[1, 2]], 1), ([[1, 2]], [[2, 2]], 1)]
>>> S = K.demazure_subcrystal(mg, [Integer(1)], only_support=False)
>>> S.digraph().edges(sort=True)
[([[1, 1]], [[1, 2]], 1),
 ([[1, 2]], [[1, 1]], 0),
 ([[1, 2]], [[2, 2]], 1),
 ([[2, 2]], [[1, 2]], 0)]
```

dual_equivalence_graph(*X=None*, *index_set=None*, *directed=True*)

Return the dual equivalence graph indexed by `index_set` on the subset `X` of `self`.

Let $b \in B$ be an element of weight 0, so $\varepsilon_j(b) = \varphi_j(b)$ for all $j \in I$, where I is the indexing set. We say b' is an i-elementary dual equivalence transformation of b (where $i \in I$) if

- $\varepsilon_i(b) = 1$ and $\varepsilon_{i-1}(b) = 0$, and

- $b' = f_{i-1} f_i e_{i-1} e_i b$.

We can do the inverse procedure by interchanging i and $i-1$ above.

Note

If the index set is not an ordered interval, we let $i-1$ mean the index appearing before i in I.

This definition comes from [As2008] Section 4 (where our $\varphi_j(b)$ and $\varepsilon_j(b)$ are denoted by $\epsilon(b,j)$ and $-\delta(b,j)$, respectively).

The dual equivalence graph of B is defined to be the colored graph whose vertices are the elements of B of weight 0, and whose edges of color i (for $i \in I$) connect pairs $\{b, b'\}$ such that b' is an i-elementary dual equivalence transformation of b.

Note

This dual equivalence graph is a generalization of $\mathcal{G}(\mathcal{X})$ in [As2008] Section 4 except we do not require $\varepsilon_i(b) = 0, 1$ for all i.

This definition can be generalized by choosing a subset X of the set of all vertices of B of weight 0, and restricting the dual equivalence graph to the vertex set X.

INPUT:

- `X` – (optional) the vertex set X (default: the whole set of vertices of `self` of weight 0)
- `index_set` – (optional) the index set I (default: the whole index set of `self`); this has to be a subset of the index set of `self` (as a list or tuple)
- `directed` – boolean (default: `True`); whether to have the dual equivalence graph be directed, where the head of an edge $b - b'$ is b and the tail is $b' = f_{i-1} f_i e_{i-1} e_i b$)

See also

`sage.combinat.partition.Partition.dual_equivalence_graph()`

EXAMPLES:

```
sage: T = crystals.Tableaux(['A',3], shape=[2,2])
sage: G = T.dual_equivalence_graph()
sage: G.edges(sort=True)
[([[1, 3], [2, 4]], [[1, 2], [3, 4]], 2),
 ([[1, 2], [3, 4]], [[1, 3], [2, 4]], 3)]
sage: T = crystals.Tableaux(['A',4], shape=[3,2])
sage: G = T.dual_equivalence_graph()
sage: G.edges(sort=True)
[([[1, 3, 5], [2, 4]], [[1, 3, 4], [2, 5]], 4),
 ([[1, 3, 5], [2, 4]], [[1, 2, 5], [3, 4]], 2),
 ([[1, 3, 4], [2, 5]], [[1, 2, 4], [3, 5]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 3),
 ([[1, 2, 4], [3, 5]], [[1, 2, 3], [4, 5]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 4)]

sage: T = crystals.Tableaux(['A',4], shape=[3,1])
sage: G = T.dual_equivalence_graph(index_set=[1,2,3])
```

(continues on next page)

(continued from previous page)

```
sage: G.vertices(sort=True)
[[[1, 3, 4], [2]], [[1, 2, 4], [3]], [[1, 2, 3], [4]]]
sage: G.edges(sort=True)
[([[1, 3, 4], [2]], [[1, 2, 4], [3]], 2),
 ([[1, 2, 4], [3]], [[1, 2, 3], [4]], 3)]
```

```
>>> from sage.all import *
>>> T = crystals.Tableaux(['A',Integer(3)], shape=[Integer(2),Integer(2)])
>>> G = T.dual_equivalence_graph()
>>> G.edges(sort=True)
[([[1, 3], [2, 4]], [[1, 2], [3, 4]], 2),
 ([[1, 2], [3, 4]], [[1, 3], [2, 4]], 3)]
>>> T = crystals.Tableaux(['A',Integer(4)], shape=[Integer(3),Integer(2)])
>>> G = T.dual_equivalence_graph()
>>> G.edges(sort=True)
[([[1, 3, 5], [2, 4]], [[1, 3, 4], [2, 5]], 4),
 ([[1, 3, 5], [2, 4]], [[1, 2, 5], [3, 4]], 2),
 ([[1, 3, 4], [2, 5]], [[1, 2, 4], [3, 5]], 2),
 ([[1, 2, 5], [3, 4]], [[1, 3, 5], [2, 4]], 3),
 ([[1, 2, 4], [3, 5]], [[1, 2, 3], [4, 5]], 3),
 ([[1, 2, 3], [4, 5]], [[1, 2, 4], [3, 5]], 4)]

>>> T = crystals.Tableaux(['A',Integer(4)], shape=[Integer(3),Integer(1)])
>>> G = T.dual_equivalence_graph(index_set=[Integer(1),Integer(2),
↪Integer(3)])
>>> G.vertices(sort=True)
[[[1, 3, 4], [2]], [[1, 2, 4], [3]], [[1, 2, 3], [4]]]
>>> G.edges(sort=True)
[([[1, 3, 4], [2]], [[1, 2, 4], [3]], 2),
 ([[1, 2, 4], [3]], [[1, 2, 3], [4]], 3)]
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of regular crystals constructed by tensor product of regular crystals.

extra_super_categories()

EXAMPLES:

```
sage: RegularCrystals().TensorProducts().extra_super_categories()
[Category of regular crystals]
```

```
>>> from sage.all import *
>>> RegularCrystals().TensorProducts().extra_super_categories()
[Category of regular crystals]
```

additional_structure()

Return `None`.

Indeed, the category of regular crystals defines no new structure: it only relates ε_a and φ_a to e_a and f_a respectively.

See also

additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: RegularCrystals().additional_structure()
```

```
>>> from sage.all import *
>>> RegularCrystals().additional_structure()
```

example(*n=3*)

Return an example of highest weight crystals, as per *Category.example()*.

EXAMPLES:

```
sage: B = RegularCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

```
>>> from sage.all import *
>>> B = RegularCrystals().example(); B
Highest weight crystal of type A_3 of highest weight omega_1
```

super_categories()

EXAMPLES:

```
sage: RegularCrystals().super_categories()
[Category of crystals]
```

```
>>> from sage.all import *
>>> RegularCrystals().super_categories()
[Category of crystals]
```

4.141 Regular Supercrystals

class sage.categories.regular_supercrystals.**RegularSuperCrystals**

Bases: *Category_singleton*

The category of crystals for super Lie algebras.

EXAMPLES:

```
sage: from sage.categories.regular_supercrystals import RegularSuperCrystals
sage: C = RegularSuperCrystals()
sage: C
Category of regular super crystals
```

(continues on next page)

(continued from previous page)

```
sage: C.super_categories()
[Category of finite super crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.regular_supercrystals import RegularSuperCrystals
>>> C = RegularSuperCrystals()
>>> C
Category of regular super crystals
>>> C.super_categories()
[Category of finite super crystals]
```

Parents in this category should implement the following methods:

- either an attribute `_cartan_type` or a method `cartan_type`
- `module_generators` – a list (or container) of distinct elements that generate the crystal using f_i and e_i

Furthermore, their elements `x` should implement the following methods:

- `x.e(i)` (returning $e_i(x)$)
- `x.f(i)` (returning $f_i(x)$)
- `x.weight()` (returning $\mathrm{wt}(x)$)

EXAMPLES:

```
sage: from sage.misc.abstract_method import abstract_methods_of_class
sage: from sage.categories.regular_supercrystals import RegularSuperCrystals
sage: abstract_methods_of_class(RegularSuperCrystals().element_class)
{'optional': [], 'required': ['e', 'f', 'weight']}
```

```
>>> from sage.all import *
>>> from sage.misc.abstract_method import abstract_methods_of_class
>>> from sage.categories.regular_supercrystals import RegularSuperCrystals
>>> abstract_methods_of_class(RegularSuperCrystals().element_class)
{'optional': [], 'required': ['e', 'f', 'weight']}
```

class ElementMethods

Bases: `object`

epsilon(i)

Return ε_i of `self`.

EXAMPLES:

```
sage: C = crystals.Tableaux(['A',[1,2]], shape=[2,1])
sage: c = C.an_element(); c
[[-2, -2], [-1]]
sage: c.epsilon(2)
0
sage: c.epsilon(0)
0
sage: c.epsilon(-1)
0
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['A',[Integer(1),Integer(2)]],␣
↪shape=[Integer(2),Integer(1)])
>>> c = C.an_element(); c
[[-2, -2], [-1]]
>>> c.epsilon(Integer(2))
0
>>> c.epsilon(Integer(0))
0
>>> c.epsilon(-Integer(1))
0
```

phi(*i*)

Return φ_i of `self`.

EXAMPLES:

```
sage: C = crystals.Tableaux(['A',[1,2]], shape=[2,1])
sage: c = C.an_element(); c
[[-2, -2], [-1]]
sage: c.phi(1)
0
sage: c.phi(2)
0
sage: c.phi(0)
1
```

```
>>> from sage.all import *
>>> C = crystals.Tableaux(['A',[Integer(1),Integer(2)]],␣
↪shape=[Integer(2),Integer(1)])
>>> c = C.an_element(); c
[[-2, -2], [-1]]
>>> c.phi(Integer(1))
0
>>> c.phi(Integer(2))
0
>>> c.phi(Integer(0))
1
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of regular crystals constructed by tensor product of regular crystals.

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.regular_supercrystals import␣
↪RegularSuperCrystals
sage: RegularSuperCrystals().TensorProducts().extra_super_categories()
[Category of regular super crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.regular_supercrystals import RegularSuperCrystals
>>> RegularSuperCrystals().TensorProducts().extra_super_categories()
[Category of regular super crystals]
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.regular_supercrystals import RegularSuperCrystals
sage: C = RegularSuperCrystals()
sage: C.super_categories()
[Category of finite super crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.regular_supercrystals import RegularSuperCrystals
>>> C = RegularSuperCrystals()
>>> C.super_categories()
[Category of finite super crystals]
```

4.142 Right modules

class sage.categories.right_modules.**RightModules**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of right modules right modules over an rng (ring not necessarily with unit), i.e. an abelian group with right multiplication by elements of the rng

EXAMPLES:

```
sage: RightModules(QQ)
Category of right modules over Rational Field
sage: RightModules(QQ).super_categories()
[Category of commutative additive groups]
```

```
>>> from sage.all import *
>>> RightModules(QQ)
Category of right modules over Rational Field
>>> RightModules(QQ).super_categories()
[Category of commutative additive groups]
```

class ElementMethods

Bases: object

class ParentMethods

Bases: object

super_categories()

EXAMPLES:

```
sage: RightModules(QQ).super_categories()
[Category of commutative additive groups]
```

```
>>> from sage.all import *
>>> RightModules(QQ).super_categories()
[Category of commutative additive groups]
```

4.143 Ring ideals

class sage.categories.ring_ideals.**RingIdeals**(R)

Bases: *Category_ideal*

The category of two-sided ideals in a fixed ring.

EXAMPLES:

```
sage: Ideals(Integers(200))
Category of ring ideals in Ring of integers modulo 200
sage: C = Ideals(IntegerRing()); C
Category of ring ideals in Integer Ring
sage: I = C([8,12,18])
sage: I
Principal ideal (2) of Integer Ring
```

```
>>> from sage.all import *
>>> Ideals(Integers(Integer(200)))
Category of ring ideals in Ring of integers modulo 200
>>> C = Ideals(IntegerRing()); C
Category of ring ideals in Integer Ring
>>> I = C([Integer(8),Integer(12),Integer(18)])
>>> I
Principal ideal (2) of Integer Ring
```

See also: *CommutativeRingIdeals*.

Todo

- If useful, implement `RingLeftIdeals` and `RingRightIdeals` of which `RingIdeals` would be a subcategory.
- Make `RingIdeals(R)`, return `CommutativeRingIdeals(R)` when `R` is commutative.

super_categories()

EXAMPLES:

```
sage: RingIdeals(ZZ).super_categories()
[Category of modules over Integer Ring]
sage: RingIdeals(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> RingIdeals(ZZ).super_categories()
[Category of modules over Integer Ring]
```

(continues on next page)

(continued from previous page)

```
>>> RingIdeals(QQ).super_categories()
[Category of vector spaces over Rational Field]
```

4.144 Rings

class sage.categories.rings.**Rings**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of rings.

Associative rings with unit, not necessarily commutative

EXAMPLES:

```
sage: Rings()
Category of rings
sage: sorted(Rings().super_categories(), key=str)
[Category of rngs, Category of semirings]

sage: sorted(Rings().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Unital']

sage: Rings() is (CommutativeAdditiveGroups() & Monoids()).Distributive()
True
sage: Rings() is Rngs().Unital()
True
sage: Rings() is Semirings().AdditiveInverse()
True
```

```
>>> from sage.all import *
>>> Rings()
Category of rings
>>> sorted(Rings().super_categories(), key=str)
[Category of rngs, Category of semirings]

>>> sorted(Rings().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive', 'Unital']

>>> Rings() is (CommutativeAdditiveGroups() & Monoids()).Distributive()
True
>>> Rings() is Rngs().Unital()
True
>>> Rings() is Semirings().AdditiveInverse()
True
```

Todo

(see Issue #sage_trac/wiki/CategoriesRoadMap)

- Make Rings() into a subcategory or alias of Algebras(ZZ);

- A parent P in the category `Rings()` should automatically be in the category `Algebras(P)`.

Commutative

alias of *CommutativeRings*

Division

alias of *DivisionRings*

class ElementMethods

Bases: `object`

inverse_of_unit()

Return the inverse of this element if it is a unit.

OUTPUT: an element in the same ring as this element

EXAMPLES:

```
sage: R.<x> = ZZ[]
sage: S = R.quo(x^2 + x + 1)                                                #␣
↪needs sage.libs.pari
sage: S(1).inverse_of_unit()                                                #␣
↪needs sage.libs.pari
1
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> S = R.quo(x**Integer(2) + x + Integer(1))                               ␣
↪               # needs sage.libs.pari
>>> S(Integer(1)).inverse_of_unit()                                         ␣
↪     # needs sage.libs.pari
1
```

This method fails when the element is not a unit:

```
sage: 2.inverse_of_unit()
Traceback (most recent call last):
...
ArithmeticError: inverse does not exist
```

```
>>> from sage.all import *
>>> Integer(2).inverse_of_unit()
Traceback (most recent call last):
...
ArithmeticError: inverse does not exist
```

The inverse returned is in the same ring as this element:

```
sage: a = -1
sage: a.parent()
Integer Ring
sage: a.inverse_of_unit().parent()
Integer Ring
```

```
>>> from sage.all import *
>>> a = -Integer(1)
>>> a.parent()
Integer Ring
>>> a.inverse_of_unit().parent()
Integer Ring
```

Note that this is often not the case when computing inverses in other ways:

```
sage: (~a).parent()
Rational Field
sage: (1/a).parent()
Rational Field
```

```
>>> from sage.all import *
>>> (~a).parent()
Rational Field
>>> (Integer(1)/a).parent()
Rational Field
```

is_unit()

Return whether this element is a unit in the ring.

Note

This is a generic implementation for (non-commutative) rings which only works for the one element, its additive inverse, and the zero element. Most rings should provide a more specialized implementation.

EXAMPLES:

```
sage: # needs sage.modules
sage: MS = MatrixSpace(ZZ, 2)
sage: MS.one().is_unit()
True
sage: MS.zero().is_unit()
False
sage: MS([1,2,3,4]).is_unit()
False
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> MS = MatrixSpace(ZZ, Integer(2))
>>> MS.one().is_unit()
True
>>> MS.zero().is_unit()
False
>>> MS([Integer(1),Integer(2),Integer(3),Integer(4)]).is_unit()
False
```

class MorphismMethods

Bases: object

extend_to_fraction_field()

Return the extension of this morphism to fraction fields of the domain and the codomain.

EXAMPLES:

```
sage: S.<x> = QQ[]
sage: f = S.hom([x + 1]); f
Ring endomorphism of Univariate Polynomial Ring in x over Rational Field
    Defn: x |--> x + 1

sage: g = f.extend_to_fraction_field(); g                                   #␣
↪needs sage.libs.singular
Ring endomorphism of Fraction Field of Univariate Polynomial Ring in x
 over Rational Field
    Defn: x |--> x + 1
sage: g(x)                                                                  #␣
↪needs sage.libs.singular
x + 1
sage: g(1/x)                                                                #␣
↪needs sage.libs.singular
1/(x + 1)
```

```
>>> from sage.all import *
>>> S = QQ['x']; (x,) = S._first_ngens(1)
>>> f = S.hom([x + Integer(1)]); f
Ring endomorphism of Univariate Polynomial Ring in x over Rational Field
    Defn: x |--> x + 1

>>> g = f.extend_to_fraction_field(); g                                     #␣
↪needs sage.libs.singular
Ring endomorphism of Fraction Field of Univariate Polynomial Ring in x
 over Rational Field
    Defn: x |--> x + 1
>>> g(x)                                                                    #␣
↪needs sage.libs.singular
x + 1
>>> g(Integer(1)/x)                                                          ␣
↪     # needs sage.libs.singular
1/(x + 1)
```

If this morphism is not injective, it does not extend to the fraction field and an error is raised:

```
sage: f = GF(5).coerce_map_from(ZZ)
sage: f.extend_to_fraction_field()
Traceback (most recent call last):
...
ValueError: the morphism is not injective
```

```
>>> from sage.all import *
>>> f = GF(Integer(5)).coerce_map_from(ZZ)
>>> f.extend_to_fraction_field()
Traceback (most recent call last):
...
ValueError: the morphism is not injective
```

is_injective()

Return whether or not this morphism is injective.

EXAMPLES:

```
sage: # needs sage.libs.singular
sage: R.<x,y> = QQ[]
sage: R.hom([x, y^2], R).is_injective()
True
sage: R.hom([x, x^2], R).is_injective()
False
sage: S.<u,v> = R.quotient(x^3*y)
sage: R.hom([v, u], S).is_injective()
False
sage: S.hom([-u, v], S).is_injective()
True
sage: S.cover().is_injective()
False
```

```
>>> from sage.all import *
>>> # needs sage.libs.singular
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.hom([x, y**Integer(2)], R).is_injective()
True
>>> R.hom([x, x**Integer(2)], R).is_injective()
False
>>> S = R.quotient(x**Integer(3)*y, names=('u', 'v',)); (u, v,) = S._
↪first_ngens(2)
>>> R.hom([v, u], S).is_injective()
False
>>> S.hom([-u, v], S).is_injective()
True
>>> S.cover().is_injective()
False
```

If the domain is a field, the homomorphism is injective:

```
sage: K.<x> = FunctionField(QQ)
sage: L.<y> = FunctionField(QQ)
sage: f = K.hom([y]); f
Function Field morphism:
  From: Rational function field in x over Rational Field
  To:   Rational function field in y over Rational Field
  Defn: x |--> y
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> K = FunctionField(QQ, names=('x',)); (x,) = K._first_ngens(1)
>>> L = FunctionField(QQ, names=('y',)); (y,) = L._first_ngens(1)
>>> f = K.hom([y]); f
Function Field morphism:
  From: Rational function field in x over Rational Field
```

(continues on next page)

(continued from previous page)

```
  To:   Rational function field in y over Rational Field
  Defn: x |--> y
>>> f.is_injective()
True
```

Unless the codomain is the zero ring:

```
sage: codomain = Integers(1)
sage: f = QQ.hom([Zmod(1)(0)], check=False)
sage: f.is_injective()
False
```

```
>>> from sage.all import *
>>> codomain = Integers(Integer(1))
>>> f = QQ.hom([Zmod(Integer(1))(Integer(0))], check=False)
>>> f.is_injective()
False
```

Homomorphism from rings of characteristic zero to rings of positive characteristic can not be injective:

```
sage: R.<x> = ZZ[]
sage: f = R.hom([GF(3)(1)]); f
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Finite Field of size 3
  Defn: x |--> 1
sage: f.is_injective()
False
```

```
>>> from sage.all import *
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> f = R.hom([GF(Integer(3))(Integer(1))]); f
Ring morphism:
  From: Univariate Polynomial Ring in x over Integer Ring
  To:   Finite Field of size 3
  Defn: x |--> 1
>>> f.is_injective()
False
```

A morphism whose domain is an order in a number field is injective if the codomain has characteristic zero:

```
sage: K.<x> = FunctionField(QQ)
sage: f = ZZ.hom(K); f
Composite map:
  From: Integer Ring
  To:   Rational function field in x over Rational Field
  Defn:   Conversion via FractionFieldElement_1poly_field map:
          From: Integer Ring
          To:   Fraction Field of Univariate Polynomial Ring in x
                over Rational Field
        then
```

(continues on next page)

(continued from previous page)

```
          Isomorphism:
          From: Fraction Field of Univariate Polynomial Ring in x
                over Rational Field
          To:   Rational function field in x over Rational Field
sage: f.is_injective()
True
```

```
>>> from sage.all import *
>>> K = FunctionField(QQ, names=('x',)); (x,) = K._first_ngens(1)
>>> f = ZZ.hom(K); f
Composite map:
  From: Integer Ring
  To:   Rational function field in x over Rational Field
  Defn:   Conversion via FractionFieldElement_1poly_field map:
          From: Integer Ring
          To:   Fraction Field of Univariate Polynomial Ring in x
                over Rational Field
        then
          Isomorphism:
          From: Fraction Field of Univariate Polynomial Ring in x
                over Rational Field
          To:   Rational function field in x over Rational Field
>>> f.is_injective()
True
```

A coercion to the fraction field is injective:

```
sage: R = ZpFM(3)                                                           #␣
↪needs sage.rings.padics
sage: R.fraction_field().coerce_map_from(R).is_injective()
True
```

```
>>> from sage.all import *
>>> R = ZpFM(Integer(3))                                                    ␣
↪     # needs sage.rings.padics
>>> R.fraction_field().coerce_map_from(R).is_injective()
True
```

NoZeroDivisors

alias of *Domains*

class ParentMethods

Bases: `object`

bracket(*x*, *y*)

Return the Lie bracket $[x, y] = xy - yx$ of x and y.

INPUT:

- `x`, `y` – elements of `self`

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: F = AlgebrasWithBasis(QQ).example()
```

(continues on next page)

(continued from previous page)

```
sage: F
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
sage: a, b, c = F.algebra_generators()
sage: F.bracket(a, b)
B[word: ab] - B[word: ba]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> F = AlgebrasWithBasis(QQ).example()
>>> F
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
>>> a, b, c = F.algebra_generators()
>>> F.bracket(a, b)
B[word: ab] - B[word: ba]
```

This measures the default of commutation between x and y. F endowed with the bracket operation is a Lie algebra; in particular, it satisfies Jacobi's identity:

```
sage: (F.bracket(F.bracket(a,b), c) + F.bracket(F.bracket(b,c), a)       #␣
↪needs sage.combinat sage.modules
....:  + F.bracket(F.bracket(c,a), b))
0
```

```
>>> from sage.all import *
>>> (F.bracket(F.bracket(a,b), c) + F.bracket(F.bracket(b,c), a)       #␣
↪needs sage.combinat sage.modules
...  + F.bracket(F.bracket(c,a), b))
0
```

characteristic()

Return the characteristic of this ring.

EXAMPLES:

```
sage: QQ.characteristic()
0
sage: GF(19).characteristic()
19
sage: Integers(8).characteristic()
8
sage: Zp(5).characteristic()                                              #␣
↪needs sage.rings.padics
0
```

```
>>> from sage.all import *
>>> QQ.characteristic()
0
>>> GF(Integer(19)).characteristic()
19
>>> Integers(Integer(8)).characteristic()
```

(continues on next page)

(continued from previous page)

```
8
>>> Zp(Integer(5)).characteristic()                                    ␣
↪     # needs sage.rings.padics
0
```

epsilon()

Return the precision error of elements in this ring.

Note

This is not used anywhere inside the code base.

EXAMPLES:

```
sage: RDF.epsilon()
2.220446049250313e-16
sage: ComplexField(53).epsilon()                                       ␣
↪  # needs sage.rings.real_mpfr
2.22044604925031e-16
sage: RealField(10).epsilon()                                          ␣
↪  # needs sage.rings.real_mpfr
0.0020
```

```
>>> from sage.all import *
>>> RDF.epsilon()
2.220446049250313e-16
>>> ComplexField(Integer(53)).epsilon()                                ␣
↪         # needs sage.rings.real_mpfr
2.22044604925031e-16
>>> RealField(Integer(10)).epsilon()                                   ␣
↪         # needs sage.rings.real_mpfr
0.0020
```

For exact rings, zero is returned:

```
sage: ZZ.epsilon()
0
```

```
>>> from sage.all import *
>>> ZZ.epsilon()
0
```

This also works over derived rings:

```
sage: RR['x'].epsilon()                                                ␣
↪  # needs sage.rings.real_mpfr
2.22044604925031e-16
sage: QQ['x'].epsilon()
0
```

```
>>> from sage.all import *
>>> RR['x'].epsilon()
↪# needs sage.rings.real_mpfr
2.22044604925031e-16
>>> QQ['x'].epsilon()
0
```

For the symbolic ring, there is no reasonable answer:

```
sage: SR.epsilon()                                                          ␣
↪  # needs sage.symbolic
Traceback (most recent call last):
...
NotImplementedError
```

```
>>> from sage.all import *
>>> SR.epsilon()
↪# needs sage.symbolic
Traceback (most recent call last):
...
NotImplementedError
```

free_module(*base=None*, *basis=None*, *map=True*)

Return a free module V over the specified subring together with maps to and from V.

The default implementation only supports the case that the base ring is the ring itself.

INPUT:

- `base` – a subring R so that this ring is isomorphic to a finite-rank free R-module V
- `basis` – (optional) a basis for this ring over the base
- `map` – boolean (default: `True`); whether to return R-linear maps to and from V

OUTPUT: a finite-rank free R-module V

- An R-module isomorphism from V to this ring (only included if `map` is `True`)
- An R-module isomorphism from this ring to V (only included if `map` is `True`)

EXAMPLES:

```
sage: # needs sage.modules
sage: R.<x> = QQ[[]]
sage: V, from_V, to_V = R.free_module(R)
sage: v = to_V(1 + x); v
(1 + x)
sage: from_V(v)
1 + x
sage: W, from_W, to_W = R.free_module(R, basis=(1 - x))
sage: W is V
True
sage: w = to_W(1 + x); w
(1 - x^2)
sage: from_W(w)
1 + x + O(x^20)
```

```
>>> from sage.all import *
>>> # needs sage.modules
```

(continues on next page)

(continued from previous page)

```
>>> R = QQ[['x']]; (x,) = R._first_ngens(1)
>>> V, from_V, to_V = R.free_module(R)
>>> v = to_V(Integer(1) + x); v
(1 + x)
>>> from_V(v)
1 + x
>>> W, from_W, to_W = R.free_module(R, basis=(Integer(1) - x))
>>> W is V
True
>>> w = to_W(Integer(1) + x); w
(1 - x^2)
>>> from_W(w)
1 + x + O(x^20)
```

ideal(**args*, ***kwds*)

Create an ideal of this ring.

INPUT:

- an element or a list/tuple/sequence of elements, the generators
- `coerce` – boolean (default: `True`); whether to first coerce the elements into this ring. This must be a keyword argument. Only set it to `False` if you are certain that each generator is already in the ring.
- `ideal_class` – callable (default: `self._ideal_class_()`); this must be a keyword argument. A constructor for ideals, taking the ring as the first argument and then the generators. Usually a subclass of `Ideal_generic` or `Ideal_nc`.
- Further named arguments (such as `side` in the case of non-commutative rings) are forwarded to the ideal class.

The keyword `side` can be one of `'twosided'`, `'left'`, `'right'`. It determines whether the resulting ideal is twosided, a left ideal or a right ideal.

EXAMPLES:

Matrix rings:

```
sage: # needs sage.modules
sage: MS = MatrixSpace(QQ, 2, 2)
sage: MS.ideal(2)
Twosided Ideal
(
  [2 0]
  [0 2]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
sage: MS.ideal([MS.0, MS.1], side='right')
Right Ideal
(
  [1 0]
  [0 0],

  [0 1]
  [0 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> MS = MatrixSpace(QQ, Integer(2), Integer(2))
>>> MS.ideal(Integer(2))
Twosided Ideal
(
  [2 0]
  [0 2]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
>>> MS.ideal([MS.gen(0), MS.gen(1)], side='right')
Right Ideal
(
  [1 0]
  [0 0],
<BLANKLINE>
  [0 1]
  [0 0]
)
 of Full MatrixSpace of 2 by 2 dense matrices over Rational Field
```

Polynomial rings:

```
sage: R.<x,y> = QQ[]
sage: R.ideal(x,y)
Ideal (x, y) of Multivariate Polynomial Ring in x, y over Rational Field
sage: R.ideal(x+y^2)
Ideal (y^2 + x) of Multivariate Polynomial Ring in x, y over Rational␣
↪Field
sage: R.ideal( [x^3,y^3+x^3] )
Ideal (x^3, x^3 + y^3) of Multivariate Polynomial Ring in x, y over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> R = QQ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.ideal(x,y)
Ideal (x, y) of Multivariate Polynomial Ring in x, y over Rational Field
>>> R.ideal(x+y**Integer(2))
Ideal (y^2 + x) of Multivariate Polynomial Ring in x, y over Rational␣
↪Field
>>> R.ideal( [x**Integer(3),y**Integer(3)+x**Integer(3)] )
Ideal (x^3, x^3 + y^3) of Multivariate Polynomial Ring in x, y over␣
↪Rational Field
```

Non-commutative rings:

```
sage: A = SteenrodAlgebra(2)                                              ␣
↪  # needs sage.combinat sage.modules
sage: A.ideal(A.1, A.2^2)                                                 ␣
↪  # needs sage.combinat sage.modules
Twosided Ideal (Sq(2), Sq(2,2)) of mod 2 Steenrod algebra, milnor basis
sage: A.ideal(A.1, A.2^2, side='left')                                    ␣
↪  # needs sage.combinat sage.modules
```

(continues on next page)

(continued from previous page)

```
Left Ideal (Sq(2), Sq(2,2)) of mod 2 Steenrod algebra, milnor basis
```

```
>>> from sage.all import *
>>> A = SteenrodAlgebra(Integer(2))
↪         # needs sage.combinat sage.modules
>>> A.ideal(A.gen(1), A.gen(2)**Integer(2))
↪                    # needs sage.combinat sage.modules
Twosided Ideal (Sq(2), Sq(2,2)) of mod 2 Steenrod algebra, milnor basis
>>> A.ideal(A.gen(1), A.gen(2)**Integer(2), side='left')
↪                    # needs sage.combinat sage.modules
Left Ideal (Sq(2), Sq(2,2)) of mod 2 Steenrod algebra, milnor basis
```

is_commutative()

Return whether the ring is commutative.

The answer is `True` only if the category is a sub-category of `CommutativeRings`.

It is recommended to use instead `R in Rings().Commutative()`.

EXAMPLES:

```
sage: Q.<i,j,k> = QuaternionAlgebra(QQ, -1, -1)
↪  # needs sage.combinat sage.modules
sage: Q.is_commutative()
↪  # needs sage.combinat sage.modules
False
```

```
>>> from sage.all import *
>>> Q = QuaternionAlgebra(QQ, -Integer(1), -Integer(1), names=('i', 'j',
↪'k',)); (i, j, k,) = Q._first_ngens(3)# needs sage.combinat sage.modules
>>> Q.is_commutative()
↪# needs sage.combinat sage.modules
False
```

is_field(*proof=True*)

Return `True` if this ring is a field.

INPUT:

- `proof` – boolean (default: `True`); determines what to do in unknown cases

ALGORITHM:

If the parameter `proof` is set to `True`, the returned value is correct but the method might throw an error. Otherwise, if it is set to `False`, the method returns `True` if it can establish that `self` is a field and `False` otherwise.

EXAMPLES:

```
sage: QQ.is_field()
True
sage: GF(9, 'a').is_field()
↪  # needs sage.rings.finite_rings
True
sage: ZZ.is_field()
False
```

(continues on next page)

(continued from previous page)

```
sage: QQ['x'].is_field()
False
sage: Frac(QQ['x']).is_field()
True
```

```
>>> from sage.all import *
>>> QQ.is_field()
True
>>> GF(Integer(9), 'a').is_field()
↪          # needs sage.rings.finite_rings
True
>>> ZZ.is_field()
False
>>> QQ['x'].is_field()
False
>>> Frac(QQ['x']).is_field()
True
```

This illustrates the use of the `proof` parameter:

```
sage: R.<a,b> = QQ[]
sage: S.<x,y> = R.quo((b^3))
↪  # needs sage.libs.singular
sage: S.is_field(proof=True)
↪  # needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError
sage: S.is_field(proof=False)
↪  # needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> R = QQ['a, b']; (a, b,) = R._first_ngens(2)
>>> S = R.quo((b**Integer(3)), names=('x', 'y',)); (x, y,) = S._first_
↪ngens(2)# needs sage.libs.singular
>>> S.is_field(proof=True)
↪# needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError
>>> S.is_field(proof=False)
↪# needs sage.libs.singular
False
```

is_integral_domain(*proof=True*)

Return `True` if this ring is an integral domain.

INPUT:

- `proof` – boolean (default: `True`); determine what to do in unknown cases

ALGORITHM:

If the parameter `proof` is set to `True`, the returned value is correct but the method might throw an

error. Otherwise, if it is set to `False`, the method returns `True` if it can establish that `self` is an integral domain and `False` otherwise.

EXAMPLES:

```
sage: QQ.is_integral_domain()
True
sage: ZZ.is_integral_domain()
True
sage: ZZ['x,y,z'].is_integral_domain()
True
sage: Integers(8).is_integral_domain()
False
sage: Zp(7).is_integral_domain()                                    ␣
↪  # needs sage.rings.padics
True
sage: Qp(7).is_integral_domain()                                    ␣
↪  # needs sage.rings.padics
True
sage: R.<a,b> = QQ[]
sage: S.<x,y> = R.quo((b^3))                                        ␣
↪  # needs sage.libs.singular
sage: S.is_integral_domain()                                        ␣
↪  # needs sage.libs.singular
False
sage: R = ZZ.quotient(ZZ.ideal(10)); R.is_integral_domain()
False
```

```
>>> from sage.all import *
>>> QQ.is_integral_domain()
True
>>> ZZ.is_integral_domain()
True
>>> ZZ['x,y,z'].is_integral_domain()
True
>>> Integers(Integer(8)).is_integral_domain()
False
>>> Zp(Integer(7)).is_integral_domain()                             ␣
↪          # needs sage.rings.padics
True
>>> Qp(Integer(7)).is_integral_domain()                             ␣
↪          # needs sage.rings.padics
True
>>> R = QQ['a, b']; (a, b,) = R._first_ngens(2)
>>> S = R.quo((b**Integer(3)), names=('x', 'y',)); (x, y,) = S._first_
↪ngens(2)# needs sage.libs.singular
>>> S.is_integral_domain()
↪# needs sage.libs.singular
False
>>> R = ZZ.quotient(ZZ.ideal(Integer(10))); R.is_integral_domain()
False
```

This illustrates the use of the `proof` parameter:

```
sage: R.<a,b> = ZZ[]
sage: S.<x,y> = R.quo((b^3))
↪  # needs sage.libs.singular
sage: S.is_integral_domain(proof=True)
↪  # needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError
sage: S.is_integral_domain(proof=False)
↪  # needs sage.libs.singular
False
```

```
>>> from sage.all import *
>>> R = ZZ['a, b']; (a, b,) = R._first_ngens(2)
>>> S = R.quo((b**Integer(3)), names=('x', 'y',)); (x, y,) = S._first_
↪ngens(2)# needs sage.libs.singular
>>> S.is_integral_domain(proof=True)
↪# needs sage.libs.singular
Traceback (most recent call last):
...
NotImplementedError
>>> S.is_integral_domain(proof=False)
↪# needs sage.libs.singular
False
```

is_integrally_closed()

Return whether this ring is integrally closed.

This is the default implementation that raises a NotImplementedError.

EXAMPLES:

```
sage: x = polygen(ZZ, 'x')
sage: K.<a> = NumberField(x^2 + 189*x + 394)
sage: R = K.order(2*a)
sage: R.is_integrally_closed()
False
sage: R
Order of conductor 2 generated by 2*a in Number Field in a with defining
↪polynomial x^2 + 189*x + 394
sage: S = K.maximal_order(); S
Maximal Order generated by a in Number Field in a with defining
↪polynomial x^2 + 189*x + 394
sage: S.is_integrally_closed()
True
```

```
>>> from sage.all import *
>>> x = polygen(ZZ, 'x')
>>> K = NumberField(x**Integer(2) + Integer(189)*x + Integer(394), names=(
↪'a',)); (a,) = K._first_ngens(1)
>>> R = K.order(Integer(2)*a)
>>> R.is_integrally_closed()
False
```

(continues on next page)

(continued from previous page)

```
>>> R
Order of conductor 2 generated by 2*a in Number Field in a with defining␣
↪polynomial x^2 + 189*x + 394
>>> S = K.maximal_order(); S
Maximal Order generated by a in Number Field in a with defining␣
↪polynomial x^2 + 189*x + 394
>>> S.is_integrally_closed()
True
```

is_noetherian()

Return `True` if this ring is Noetherian.

EXAMPLES:

```
sage: QQ.is_noetherian()
True
sage: ZZ.is_noetherian()
True
```

```
>>> from sage.all import *
>>> QQ.is_noetherian()
True
>>> ZZ.is_noetherian()
True
```

is_prime_field()

Return `True` if this ring is one of the prime fields $\mathbf{Q}$ or $\mathbf{F}_p$.

EXAMPLES:

```
sage: QQ.is_prime_field()
True
sage: GF(3).is_prime_field()
True
sage: GF(9, 'a').is_prime_field()                                              ␣
↪  # needs sage.rings.finite_rings
False
sage: ZZ.is_prime_field()
False
sage: QQ['x'].is_prime_field()
False
sage: Qp(19).is_prime_field()                                                  ␣
↪  # needs sage.rings.padics
False
sage: R.<x> = PolynomialRing(QQ)
sage: S = R.quotient(x + 16)
sage: S.is_prime_field()
True
sage: S = R.quotient(x^2 - 2)
sage: S.is_prime_field()
False
sage: R.<x> = PolynomialRing(GF(5))
sage: S = R.quotient(x^2 + x + 1)
```

(continues on next page)

(continued from previous page)

```
sage: S.is_field()
True
sage: S.is_prime_field()
False
sage: T = R.quotient(x - 2)
sage: T.is_prime_field()
True
```

```
>>> from sage.all import *
>>> QQ.is_prime_field()
True
>>> GF(Integer(3)).is_prime_field()
True
>>> GF(Integer(9), 'a').is_prime_field()
↪          # needs sage.rings.finite_rings
False
>>> ZZ.is_prime_field()
False
>>> QQ['x'].is_prime_field()
False
>>> Qp(Integer(19)).is_prime_field()
↪          # needs sage.rings.padics
False
>>> R = PolynomialRing(QQ, names=('x',)); (x,) = R._first_ngens(1)
>>> S = R.quotient(x + Integer(16))
>>> S.is_prime_field()
True
>>> S = R.quotient(x**Integer(2) - Integer(2))
>>> S.is_prime_field()
False
>>> R = PolynomialRing(GF(Integer(5)), names=('x',)); (x,) = R._first_
↪ngens(1)
>>> S = R.quotient(x**Integer(2) + x + Integer(1))
>>> S.is_field()
True
>>> S.is_prime_field()
False
>>> T = R.quotient(x - Integer(2))
>>> T.is_prime_field()
True
```

We check that bug Issue #40426 is fixed:

```
sage: K = GF(4)
sage: A.<x> = K[]
sage: L = K.extension(x+1)
sage: L.is_prime_field()
False
```

```
>>> from sage.all import *
>>> K = GF(Integer(4))
>>> A = K['x']; (x,) = A._first_ngens(1)
```

(continues on next page)

(continued from previous page)

```
>>> L = K.extension(x+Integer(1))
>>> L.is_prime_field()
False
```

is_ring()

Return `True`, since this in an object of the category of rings.

EXAMPLES:

```
sage: Parent(QQ,category=Rings()).is_ring()
True
```

```
>>> from sage.all import *
>>> Parent(QQ,category=Rings()).is_ring()
True
```

is_subring(*other*)

Return `True` if the canonical map from `self` to `other` is injective.

This raises a `NotImplementedError` if not known.

EXAMPLES:

```
sage: ZZ.is_subring(QQ)
True
sage: ZZ.is_subring(GF(19))
False
```

```
>>> from sage.all import *
>>> ZZ.is_subring(QQ)
True
>>> ZZ.is_subring(GF(Integer(19)))
False
```

is_zero()

Return `True` if this is the zero ring.

EXAMPLES:

```
sage: Integers(1).is_zero()
True
sage: Integers(2).is_zero()
False
sage: QQ.is_zero()
False
sage: R.<x> = ZZ[]
sage: R.quo(1).is_zero()
True
sage: R.<x> = GF(101)[]
sage: R.quo(77).is_zero()
True
sage: R.quo(x^2 + 1).is_zero()                                              #␣
↪needs sage.libs.pari
False
```

```
>>> from sage.all import *
>>> Integers(Integer(1)).is_zero()
True
>>> Integers(Integer(2)).is_zero()
False
>>> QQ.is_zero()
False
>>> R = ZZ['x']; (x,) = R._first_ngens(1)
>>> R.quo(Integer(1)).is_zero()
True
>>> R = GF(Integer(101))['x']; (x,) = R._first_ngens(1)
>>> R.quo(Integer(77)).is_zero()
True
>>> R.quo(x**Integer(2) + Integer(1)).is_zero()
↪               # needs sage.libs.pari
False
```

localization(**args*, ***kwds*)

Return the localization of `self`.

This only works for integral domains.

EXAMPLES:

```
sage: R = Zmod(6)
sage: R.localization((4))
Traceback (most recent call last):
...
TypeError: self must be an integral domain
```

```
>>> from sage.all import *
>>> R = Zmod(Integer(6))
>>> R.localization((Integer(4)))
Traceback (most recent call last):
...
TypeError: self must be an integral domain
```

nilradical()

Return the nilradical of this ring.

EXAMPLES:

```
sage: QQ['x,y'].nilradical()
Ideal (0) of Multivariate Polynomial Ring in x, y over Rational Field
```

```
>>> from sage.all import *
>>> QQ['x,y'].nilradical()
Ideal (0) of Multivariate Polynomial Ring in x, y over Rational Field
```

See also

nilradical()

quo(*I*, *names=None*, ***kwds*)

Quotient of a ring by a two-sided ideal.

> **Note**
>
> This is a synonym for *quotient()*.

EXAMPLES:

```
sage: MS = MatrixSpace(QQ, 2)                                                  #␣
↪needs sage.modules
sage: I = MS * MS.gens() * MS                                                  #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(QQ, Integer(2))                                            ␣
↪     # needs sage.modules
>>> I = MS * MS.gens() * MS                                                   #␣
↪needs sage.modules
```

`MS` is not an instance of `Ring`.

However it is an instance of the parent class of the category of rings. The quotient method is inherited from there:

```
sage: isinstance(MS, sage.rings.ring.Ring)                                     #␣
↪needs sage.modules
False
sage: isinstance(MS, Rings().parent_class)                                     #␣
↪needs sage.modules
True
sage: MS.quo(I, names=['a','b','c','d'])                                       #␣
↪needs sage.modules
Quotient of Full MatrixSpace of 2 by 2 dense matrices
 over Rational Field by the ideal
(
  [1 0]
  [0 0],

  [0 1]
  [0 0],

  [0 0]
  [1 0],

  [0 0]
  [0 1]
)
```

```
>>> from sage.all import *
>>> isinstance(MS, sage.rings.ring.Ring)                                      #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
False
>>> isinstance(MS, Rings().parent_class)                                    #␣
↪needs sage.modules
True
>>> MS.quo(I, names=['a','b','c','d'])                                      #␣
↪needs sage.modules
Quotient of Full MatrixSpace of 2 by 2 dense matrices
 over Rational Field by the ideal
(
  [1 0]
  [0 0],
<BLANKLINE>
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 0],
<BLANKLINE>
  [0 0]
  [0 1]
)
```

A test with a subclass of `Ring`:

```
sage: # needs sage.libs.singular
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = R.quo((x^2, y))
sage: S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
sage: S.gens()
(a, 0)
sage: a == b
False
```

```
>>> from sage.all import *
>>> # needs sage.libs.singular
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._
↪first_ngens(2)
>>> S = R.quo((x**Integer(2), y), names=('a', 'b',)); (a, b,) = S._first_
↪ngens(2)
>>> S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
>>> S.gens()
(a, 0)
>>> a == b
False
```

quotient(*I*, *names=None*, ***kwds*)

Quotient of a ring by a two-sided ideal.

INPUT:

- `I` – a twosided ideal of this ring
- `names` – (optional) names of the generators of the quotient (if there are multiple generators, you can specify a single character string and the generators are named in sequence starting with 0).
- further named arguments that may be passed to the quotient ring constructor.

EXAMPLES:

Usually, a ring inherits a method *quotient_ring()*. So, we need a bit of effort to make the following example work with the category framework:

```
sage: # needs sage.combinat sage.modules
sage: F.<x,y,z> = FreeAlgebra(QQ)
sage: from sage.rings.noncommutative_ideals import Ideal_nc
sage: from itertools import product
sage: class PowerIdeal(Ideal_nc):
....:  def __init__(self, R, n):
....:      self._power = n
....:      Ideal_nc.__init__(self, R, [R.prod(m)
....:                                  for m in product(R.gens(),␣
↪repeat=n)])
....:  def reduce(self, x):
....:      R = self.ring()
....:      return add([c*R(m) for m, c in x
....:                  if len(m) < self._power], R(0))
sage: I = PowerIdeal(F, 3)
sage: Q = Rings().parent_class.quotient(F, I); Q
Quotient of Free Algebra on 3 generators (x, y, z) over Rational Field
 by the ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x,
               x*z*y, x*z^2, y*x^2, y*x*y, y*x*z, y^2*x, y^3,
               y^2*z, y*z*x, y*z*y, y*z^2, z*x^2, z*x*y, z*x*z,
               z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3)
sage: Q.0
xbar
sage: Q.1
ybar
sage: Q.2
zbar
sage: Q.0*Q.1
xbar*ybar
sage: Q.0*Q.1*Q.0
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> F = FreeAlgebra(QQ, names=('x', 'y', 'z',)); (x, y, z,) = F._first_
↪ngens(3)
>>> from sage.rings.noncommutative_ideals import Ideal_nc
>>> from itertools import product
>>> class PowerIdeal(Ideal_nc):
...  def __init__(self, R, n):
...      self._power = n
...      Ideal_nc.__init__(self, R, [R.prod(m)
...                                  for m in product(R.gens(),␣
↪repeat=n)])
...  def reduce(self, x):
```

(continues on next page)

(continued from previous page)

```
...       R = self.ring()
...       return add([c*R(m) for m, c in x
...                   if len(m) < self._power], R(Integer(0)))
>>> I = PowerIdeal(F, Integer(3))
>>> Q = Rings().parent_class.quotient(F, I); Q
Quotient of Free Algebra on 3 generators (x, y, z) over Rational Field
 by the ideal (x^3, x^2*y, x^2*z, x*y*x, x*y^2, x*y*z, x*z*x,
               x*z*y, x*z^2, y*x^2, y*x*y, y*x*z, y^2*x, y^3,
               y^2*z, y*z*x, y*z*y, y*z^2, z*x^2, z*x*y, z*x*z,
               z*y*x, z*y^2, z*y*z, z^2*x, z^2*y, z^3)
>>> Q.gen(0)
xbar
>>> Q.gen(1)
ybar
>>> Q.gen(2)
zbar
>>> Q.gen(0)*Q.gen(1)
xbar*ybar
>>> Q.gen(0)*Q.gen(1)*Q.gen(0)
0
```

An example with polynomial rings:

```
sage: R.<x> = PolynomialRing(ZZ)
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: S = R.quotient(I, 'a')
sage: S.gens()
(a,)

sage: # needs sage.libs.singular
sage: R.<x,y> = PolynomialRing(QQ, 2)
sage: S.<a,b> = R.quotient((x^2, y))
sage: S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
sage: S.gens()
(a, 0)
sage: a == b
False
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
>>> S = R.quotient(I, 'a')
>>> S.gens()
(a,)

>>> # needs sage.libs.singular
>>> R = PolynomialRing(QQ, Integer(2), names=('x', 'y',)); (x, y,) = R._
↪first_ngens(2)
>>> S = R.quotient((x**Integer(2), y), names=('a', 'b',)); (a, b,) = S._
```

(continues on next page)

(continued from previous page)

```
↪first_ngens(2)
>>> S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
>>> S.gens()
(a, 0)
>>> a == b
False
```

quotient_ring(*I*, *names=None*, ***kwds*)

Quotient of a ring by a two-sided ideal.

Note

This is a synonym for *quotient()*.

INPUT:

- `I` – an ideal of R
- `names` – (optional) names of the generators of the quotient. (If there are multiple generators, you can specify a single character string and the generators are named in sequence starting with 0.)
- further named arguments that may be passed to the quotient ring constructor.

OUTPUT: `R/I` – the quotient ring of R by the ideal I

EXAMPLES:

```
sage: MS = MatrixSpace(QQ, 2)                                                  #␣
↪needs sage.modules
sage: I = MS * MS.gens() * MS                                                  #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> MS = MatrixSpace(QQ, Integer(2))                                            ␣
↪     # needs sage.modules
>>> I = MS * MS.gens() * MS                                                  #␣
↪needs sage.modules
```

`MS` is not an instance of `Ring`, but it is an instance of the parent class of the category of rings. The quotient method is inherited from there:

```
sage: isinstance(MS, sage.rings.ring.Ring)                                     #␣
↪needs sage.modules
False
sage: isinstance(MS, Rings().parent_class)                                     #␣
↪needs sage.modules
True
sage: MS.quotient_ring(I, names=['a','b','c','d'])                             #␣
↪needs sage.modules
Quotient of Full MatrixSpace of 2 by 2 dense matrices
 over Rational Field by the ideal
(
  [1 0]
```

(continues on next page)

(continued from previous page)

```
  [0 0],

  [0 1]
  [0 0],

  [0 0]
  [1 0],

  [0 0]
  [0 1]
)
```

```
>>> from sage.all import *
>>> isinstance(MS, sage.rings.ring.Ring)                                    #␣
↪needs sage.modules
False
>>> isinstance(MS, Rings().parent_class)                                    #␣
↪needs sage.modules
True
>>> MS.quotient_ring(I, names=['a','b','c','d'])                            #␣
↪needs sage.modules
Quotient of Full MatrixSpace of 2 by 2 dense matrices
 over Rational Field by the ideal
(
  [1 0]
  [0 0],
<BLANKLINE>
  [0 1]
  [0 0],
<BLANKLINE>
  [0 0]
  [1 0],
<BLANKLINE>
  [0 0]
  [0 1]
)
```

A test with a subclass of `Ring`:

```
sage: R.<x> = PolynomialRing(ZZ)
sage: I = R.ideal([4 + 3*x + x^2, 1 + x^2])
sage: S = R.quotient_ring(I, 'a')
sage: S.gens()
(a,)

sage: # needs sage.libs.singular
sage: R.<x,y> = PolynomialRing(QQ,2)
sage: S.<a,b> = R.quotient_ring((x^2, y))
sage: S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
sage: S.gens()
```

(continues on next page)

(continued from previous page)

```
(a, 0)
sage: a == b
False
```

```
>>> from sage.all import *
>>> R = PolynomialRing(ZZ, names=('x',)); (x,) = R._first_ngens(1)
>>> I = R.ideal([Integer(4) + Integer(3)*x + x**Integer(2), Integer(1) +␣
↪x**Integer(2)])
>>> S = R.quotient_ring(I, 'a')
>>> S.gens()
(a,)

>>> # needs sage.libs.singular
>>> R = PolynomialRing(QQ,Integer(2), names=('x', 'y',)); (x, y,) = R._
↪first_ngens(2)
>>> S = R.quotient_ring((x**Integer(2), y), names=('a', 'b',)); (a, b,) =␣
↪S._first_ngens(2)
>>> S
Quotient of Multivariate Polynomial Ring in x, y over Rational Field
 by the ideal (x^2, y)
>>> S.gens()
(a, 0)
>>> a == b
False
```

random_element(**args*)

Return a random integer coerced into this ring.

INPUT:

- either no integer, one integer or two integers

The integer is chosen uniformly from the closed interval `[-2,2]`, `[-a,a]` or `[a,b]` according to the length of the input.

ALGORITHM:

This uses Python's `randint`.

EXAMPLES:

```
sage: -8 <= ZZ.random_element(8) <= 8
True
sage: -8 <= QQ.random_element(8) <= 8
True
sage: 4 <= ZZ.random_element(4,12) <= 12
True
```

```
>>> from sage.all import *
>>> -Integer(8) <= ZZ.random_element(Integer(8)) <= Integer(8)
True
>>> -Integer(8) <= QQ.random_element(Integer(8)) <= Integer(8)
True
>>> Integer(4) <= ZZ.random_element(Integer(4),Integer(12)) <= Integer(12)
True
```

unit_ideal()

Return the unit ideal of this ring.

EXAMPLES:

```
sage: Zp(7).unit_ideal()
↪  # needs sage.rings.padics
Principal ideal (1 + O(7^20)) of 7-adic Ring with capped relative
↪precision 20
```

```
>>> from sage.all import *
>>> Zp(Integer(7)).unit_ideal()
↪          # needs sage.rings.padics
Principal ideal (1 + O(7^20)) of 7-adic Ring with capped relative
↪precision 20
```

zeta(*n=2*, *all=False*)

Return a primitive `n`-th root of unity in `self` if there is one, or raise a `ValueError` otherwise.

INPUT:

- `n` – positive integer
- `all` – boolean (default: `False`); whether to return a list of all primitive n-th roots of unity. If `True`, raise a `ValueError` if `self` is not an integral domain.

OUTPUT: element of `self` of finite order

EXAMPLES:

```
sage: QQ.zeta()
-1
sage: QQ.zeta(1)
1
sage: CyclotomicField(6).zeta(6)
↪  # needs sage.rings.number_field
zeta6
sage: CyclotomicField(3).zeta(3)
↪  # needs sage.rings.number_field
zeta3
sage: CyclotomicField(3).zeta(3).multiplicative_order()
↪  # needs sage.rings.number_field
3

sage: # needs sage.rings.finite_rings
sage: a = GF(7).zeta(); a
3
sage: a.multiplicative_order()
6
sage: a = GF(49,'z').zeta(); a
z
sage: a.multiplicative_order()
48
sage: a = GF(49,'z').zeta(2); a
6
sage: a.multiplicative_order()
2
```

(continues on next page)

(continued from previous page)

```
sage: QQ.zeta(3)
Traceback (most recent call last):
...
ValueError: no n-th root of unity in rational field
sage: Zp(7, prec=8).zeta()                                                  ␣
↪  # needs sage.rings.padics
3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + 6*7^6 + 2*7^7 + O(7^8)
```

```
>>> from sage.all import *
>>> QQ.zeta()
-1
>>> QQ.zeta(Integer(1))
1
>>> CyclotomicField(Integer(6)).zeta(Integer(6))                            ␣
↪                  # needs sage.rings.number_field
zeta6
>>> CyclotomicField(Integer(3)).zeta(Integer(3))                            ␣
↪                  # needs sage.rings.number_field
zeta3
>>> CyclotomicField(Integer(3)).zeta(Integer(3)).multiplicative_order()     ␣
↪                  # needs sage.rings.number_field
3

>>> # needs sage.rings.finite_rings
>>> a = GF(Integer(7)).zeta(); a
3
>>> a.multiplicative_order()
6
>>> a = GF(Integer(49),'z').zeta(); a
z
>>> a.multiplicative_order()
48
>>> a = GF(Integer(49),'z').zeta(Integer(2)); a
6
>>> a.multiplicative_order()
2

>>> QQ.zeta(Integer(3))
Traceback (most recent call last):
...
ValueError: no n-th root of unity in rational field
>>> Zp(Integer(7), prec=Integer(8)).zeta()                                  ␣
↪                  # needs sage.rings.padics
3 + 4*7 + 6*7^2 + 3*7^3 + 2*7^5 + 6*7^6 + 2*7^7 + O(7^8)
```

zeta_order()

Return the order of the distinguished root of unity in `self`.

EXAMPLES:

```
sage: CyclotomicField(19).zeta_order()                                      ␣
```

(continues on next page)

(continued from previous page)

```
↪  # needs sage.rings.number_field
38
sage: GF(19).zeta_order()
18
sage: GF(5^3,'a').zeta_order()                                              ␣
↪  # needs sage.rings.finite_rings
124
sage: Zp(7, prec=8).zeta_order()                                            ␣
↪  # needs sage.rings.padics
6
```

```
>>> from sage.all import *
>>> CyclotomicField(Integer(19)).zeta_order()                               ␣
↪         # needs sage.rings.number_field
38
>>> GF(Integer(19)).zeta_order()
18
>>> GF(Integer(5)**Integer(3),'a').zeta_order()                             ␣
↪                    # needs sage.rings.finite_rings
124
>>> Zp(Integer(7), prec=Integer(8)).zeta_order()                            ␣
↪                   # needs sage.rings.padics
6
```

class SubcategoryMethods

Bases: object

Division()

Return the full subcategory of the division objects of self.

A ring satisfies the *division axiom* if all nonzero elements have multiplicative inverses.

EXAMPLES:

```
sage: Rings().Division()
Category of division rings
sage: Rings().Commutative().Division()
Category of fields
```

```
>>> from sage.all import *
>>> Rings().Division()
Category of division rings
>>> Rings().Commutative().Division()
Category of fields
```

NoZeroDivisors()

Return the full subcategory of the objects of self having no nonzero zero divisors.

A *zero divisor* in a ring R is an element $x \in R$ such that there exists a nonzero element $y \in R$ such that $x \cdot y = 0$ or $y \cdot x = 0$ (see Wikipedia article Zero_divisor).

EXAMPLES:

```
sage: Rings().NoZeroDivisors()
Category of domains
```

```
>>> from sage.all import *
>>> Rings().NoZeroDivisors()
Category of domains
```

4.145 Rngs

class sage.categories.rngs.**Rngs**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of rngs.

An *rng* $(S, +, *)$ is similar to a ring but not necessarily unital. In other words, it is a combination of a commutative additive group $(S, +)$ and a multiplicative semigroup $(S, *)$, where $*$ distributes over $+$.

EXAMPLES:

```
sage: C = Rngs(); C
Category of rngs
sage: sorted(C.super_categories(), key=str)
[Category of associative additive commutative additive associative additive␣
↪unital distributive magmas and additive magmas,
 Category of commutative additive groups]

sage: sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive']

sage: C is (CommutativeAdditiveGroups() & Semigroups()).Distributive()
True
sage: C.Unital()
Category of rings
```

```
>>> from sage.all import *
>>> C = Rngs(); C
Category of rngs
>>> sorted(C.super_categories(), key=str)
[Category of associative additive commutative additive associative additive␣
↪unital distributive magmas and additive magmas,
 Category of commutative additive groups]

>>> sorted(C.axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveInverse',
 'AdditiveUnital', 'Associative', 'Distributive']

>>> C is (CommutativeAdditiveGroups() & Semigroups()).Distributive()
True
>>> C.Unital()
Category of rings
```

class ParentMethods

Bases: object

ideal_monoid()

The monoid of the ideals of this ring.

> **Note**
>
> The code is copied from the base class of rings. This is since there are rings that do not inherit from that class, such as matrix algebras. See Issue #7797.

EXAMPLES:

```
sage: # needs sage.modules
sage: MS = MatrixSpace(QQ, 2, 2)
sage: isinstance(MS, Ring)
False
sage: MS in Rings()
True
sage: MS.ideal_monoid()
Monoid of ideals of Full MatrixSpace of 2 by 2 dense matrices
over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> MS = MatrixSpace(QQ, Integer(2), Integer(2))
>>> isinstance(MS, Ring)
False
>>> MS in Rings()
True
>>> MS.ideal_monoid()
Monoid of ideals of Full MatrixSpace of 2 by 2 dense matrices
over Rational Field
```

Note that the monoid is cached:

```
sage: MS.ideal_monoid() is MS.ideal_monoid()                                #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> MS.ideal_monoid() is MS.ideal_monoid()                                  #␣
↪needs sage.modules
True
```

principal_ideal(*gen*, *coerce=True*)

Return the principal ideal generated by gen.

EXAMPLES:

```
sage: R.<x,y> = ZZ[]
sage: R.principal_ideal(x+2*y)
Ideal (x + 2*y) of Multivariate Polynomial Ring in x, y over Integer Ring
```

```
>>> from sage.all import *
>>> R = ZZ['x, y']; (x, y,) = R._first_ngens(2)
>>> R.principal_ideal(x+Integer(2)*y)
Ideal (x + 2*y) of Multivariate Polynomial Ring in x, y over Integer Ring
```

zero_ideal()

Return the zero ideal of this ring (cached).

EXAMPLES:

```
sage: ZZ.zero_ideal()
Principal ideal (0) of Integer Ring
sage: QQ.zero_ideal()
Principal ideal (0) of Rational Field
sage: QQ['x'].zero_ideal()
Principal ideal (0) of Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> ZZ.zero_ideal()
Principal ideal (0) of Integer Ring
>>> QQ.zero_ideal()
Principal ideal (0) of Rational Field
>>> QQ['x'].zero_ideal()
Principal ideal (0) of Univariate Polynomial Ring in x over Rational Field
```

The result is cached:

```
sage: ZZ.zero_ideal() is ZZ.zero_ideal()
True
```

```
>>> from sage.all import *
>>> ZZ.zero_ideal() is ZZ.zero_ideal()
True
```

Unital

alias of *Rings*

4.146 R-trivial semigroups

class sage.categories.r_trivial_semigroups.**RTrivialSemigroups**(*base_category*)

Bases: *CategoryWithAxiom*

Commutative_extra_super_categories()

Implement the fact that a commutative R-trivial semigroup is J-trivial.

EXAMPLES:

```
sage: Semigroups().RTrivial().Commutative_extra_super_categories()
[Category of j trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().RTrivial().Commutative_extra_super_categories()
[Category of j trivial semigroups]
```

extra_super_categories()

Implement the fact that a R-trivial semigroup is H-trivial.

EXAMPLES:

```
sage: Semigroups().RTrivial().extra_super_categories()
[Category of h trivial semigroups]
```

```
>>> from sage.all import *
>>> Semigroups().RTrivial().extra_super_categories()
[Category of h trivial semigroups]
```

4.147 Schemes

class sage.categories.schemes.**AbelianVarieties**(*base*)

Bases: *Schemes_over_base*

The category of abelian varieties over a given field.

EXAMPLES:

```
sage: AbelianVarieties(QQ)
Category of abelian varieties over Rational Field
sage: AbelianVarieties(ZZ)
Traceback (most recent call last):
...
ValueError: category of abelian varieties is only defined over fields
```

```
>>> from sage.all import *
>>> AbelianVarieties(QQ)
Category of abelian varieties over Rational Field
>>> AbelianVarieties(ZZ)
Traceback (most recent call last):
...
ValueError: category of abelian varieties is only defined over fields
```

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

Overloaded `Homsets` class to register the homset as an additive abelian group.

EXAMPLES:

```
sage: AbelianVarieties(QQ).Homsets().is_
↪subcategory(CommutativeAdditiveGroups())
True
```

```
>>> from sage.all import *
>>> AbelianVarieties(QQ).Homsets().is_subcategory(CommutativeAdditiveGroups())
True
```

class Endset(*base_category*)

Bases: *CategoryWithAxiom*

Overloaded `Endset` class to register the endset as a ring.

sage: AbelianVarieties(QQ).Endsets().is_subcategory(Rings()) True

extra_super_categories()

Register the endset as a ring.

EXAMPLES:

```
sage: End(EllipticCurve(j=1)) in Rings()
True
```

```
>>> from sage.all import *
>>> End(EllipticCurve(j=Integer(1))) in Rings()
True
```

extra_super_categories()

Register the homset as an additive abelian group.

EXAMPLES:

```
sage: Hom(EllipticCurve(j=1), EllipticCurve(j=2)) in␣
↪CommutativeAdditiveGroups()
True
```

```
>>> from sage.all import *
>>> Hom(EllipticCurve(j=Integer(1)), EllipticCurve(j=Integer(2))) in␣
↪CommutativeAdditiveGroups()
True
```

base_scheme()

EXAMPLES:

```
sage: Schemes(Spec(ZZ)).base_scheme()
Spectrum of Integer Ring
```

```
>>> from sage.all import *
>>> Schemes(Spec(ZZ)).base_scheme()
Spectrum of Integer Ring
```

super_categories()

EXAMPLES:

```
sage: AbelianVarieties(QQ).super_categories()
[Category of schemes over Rational Field,
 Category of commutative additive groups]
```

```
>>> from sage.all import *
>>> AbelianVarieties(QQ).super_categories()
[Category of schemes over Rational Field,
 Category of commutative additive groups]
```

class sage.categories.schemes.**Jacobians**(*base*)

Bases: *Schemes_over_base*

The category of Jacobians attached to curves or function fields.

EXAMPLES:

```
sage: Jacobians(QQ)
Category of Jacobians over Rational Field
```

```
>>> from sage.all import *
>>> Jacobians(QQ)
Category of Jacobians over Rational Field
```

class ParentMethods

Bases: object

base_curve()

Return the curve to which this Jacobian is attached.

EXAMPLES:

```
sage: # needs sage.rings.function_field
sage: K.<x> = FunctionField(GF(2))
sage: J = K.jacobian()
sage: J.base_curve()
Rational function field in x over Finite Field of size 2
```

```
>>> from sage.all import *
>>> # needs sage.rings.function_field
>>> K = FunctionField(GF(Integer(2)), names=('x',)); (x,) = K._first_
↪ngens(1)
>>> J = K.jacobian()
>>> J.base_curve()
Rational function field in x over Finite Field of size 2
```

base_scheme()

Return the base scheme of this Jacobians category.

EXAMPLES:

```
sage: Jacobians(QQ).base_scheme()
Spectrum of Rational Field
```

```
>>> from sage.all import *
>>> Jacobians(QQ).base_scheme()
Spectrum of Rational Field
```

super_categories()

Return the super categories of this Jacobians category.

EXAMPLES:

```
sage: Jacobians(QQ).super_categories()
[Category of abelian varieties over Rational Field]
```

```
>>> from sage.all import *
>>> Jacobians(QQ).super_categories()
[Category of abelian varieties over Rational Field]
```

class sage.categories.schemes.**Schemes**

Bases: *Category*

The category of all schemes.

EXAMPLES:

```
sage: Schemes()
Category of schemes
```

```
>>> from sage.all import *
>>> Schemes()
Category of schemes
```

`Schemes` can also be used to construct the category of schemes over a given base:

```
sage: Schemes(Spec(ZZ))
Category of schemes over Integer Ring

sage: Schemes(ZZ)
Category of schemes over Integer Ring
```

```
>>> from sage.all import *
>>> Schemes(Spec(ZZ))
Category of schemes over Integer Ring

>>> Schemes(ZZ)
Category of schemes over Integer Ring
```

Todo

Make `Schemes()` a singleton category (and remove *Schemes* from the workaround in *_test_category_over_bases()*).

This is currently incompatible with the dispatching below.

super_categories()

EXAMPLES:

```
sage: Schemes().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Schemes().super_categories()
[Category of sets]
```

class sage.categories.schemes.**Schemes_over_base**(*base*, *name=None*)

Bases: *Category_over_base*

The category of schemes over a given base scheme.

EXAMPLES:

```
sage: Schemes(Spec(ZZ))
Category of schemes over Integer Ring
```

```
>>> from sage.all import *
>>> Schemes(Spec(ZZ))
Category of schemes over Integer Ring
```

base_scheme()

EXAMPLES:

```
sage: Schemes(Spec(ZZ)).base_scheme()
Spectrum of Integer Ring
```

```
>>> from sage.all import *
>>> Schemes(Spec(ZZ)).base_scheme()
Spectrum of Integer Ring
```

super_categories()

EXAMPLES:

```
sage: Schemes(Spec(ZZ)).super_categories()
[Category of schemes]
```

```
>>> from sage.all import *
>>> Schemes(Spec(ZZ)).super_categories()
[Category of schemes]
```

4.148 Semigroups

class sage.categories.semigroups.**Semigroups**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of (multiplicative) semigroups.

A *semigroup* is an associative *magma*, that is a set endowed with a multiplicative binary operation $*$ which is associative (see Wikipedia article Semigroup).

The operation $*$ is not required to have a neutral element. A semigroup for which such an element exists is a *monoid*.

EXAMPLES:

```
sage: C = Semigroups(); C
Category of semigroups
sage: C.super_categories()
[Category of magmas]
sage: C.all_super_categories()
[Category of semigroups, Category of magmas,
 Category of sets, Category of sets with partial maps, Category of objects]
```

(continues on next page)

(continued from previous page)

```
sage: C.axioms()
frozenset({'Associative'})
sage: C.example()
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> C = Semigroups(); C
Category of semigroups
>>> C.super_categories()
[Category of magmas]
>>> C.all_super_categories()
[Category of semigroups, Category of magmas,
 Category of sets, Category of sets with partial maps, Category of objects]
>>> C.axioms()
frozenset({'Associative'})
>>> C.example()
An example of a semigroup: the left zero semigroup
```

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

algebra_generators()

The generators of this algebra, as per *MagmaticAlgebras.ParentMethods.algebra_generators()*.

They correspond to the generators of the semigroup.

EXAMPLES:

```
sage: M = FiniteSemigroups().example(); M
An example of a finite semigroup:
the left regular band generated by ('a', 'b', 'c', 'd')
sage: M.semigroup_generators()
Family ('a', 'b', 'c', 'd')
sage: M.algebra(ZZ).algebra_generators()                                #␣
↪needs sage.modules
Family (B['a'], B['b'], B['c'], B['d'])
```

```
>>> from sage.all import *
>>> M = FiniteSemigroups().example(); M
An example of a finite semigroup:
the left regular band generated by ('a', 'b', 'c', 'd')
>>> M.semigroup_generators()
Family ('a', 'b', 'c', 'd')
>>> M.algebra(ZZ).algebra_generators()                                  #␣
↪needs sage.modules
Family (B['a'], B['b'], B['c'], B['d'])
```

gen(*i=0*)

Return the `i`-th generator of `self`.

EXAMPLES:

```
sage: A = GL(3, GF(7)).algebra(ZZ)                                          #␣
↪needs sage.modules
sage: A.gen(0)                                                              #␣
↪needs sage.groups sage.libs.pari sage.modules
[3 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> A = GL(Integer(3), GF(Integer(7))).algebra(ZZ)                          ␣
↪             # needs sage.modules
>>> A.gen(Integer(0))                                                       ␣
↪    # needs sage.groups sage.libs.pari sage.modules
[3 0 0]
[0 1 0]
[0 0 1]
```

gens()

Return the generators of `self`.

EXAMPLES:

```
sage: a, b = SL2Z.algebra(ZZ).gens(); a, b                                  #␣
↪needs sage.groups sage.modular sage.modules
([ 0 -1]
 [ 1  0],
 [1 1]
 [0 1])
sage: 2*a + b                                                               #␣
↪needs sage.groups sage.modular sage.modules
2*[ 0 -1]
  [ 1  0]
+
[1 1]
[0 1]
```

```
>>> from sage.all import *
>>> a, b = SL2Z.algebra(ZZ).gens(); a, b                                    #␣
↪needs sage.groups sage.modular sage.modules
([ 0 -1]
 [ 1  0],
 [1 1]
 [0 1])
>>> Integer(2)*a + b                                                        ␣
↪    # needs sage.groups sage.modular sage.modules
2*[ 0 -1]
  [ 1  0]
+
[1 1]
[0 1]
```

ngens()

Return the number of generators of `self`.

EXAMPLES:

```
sage: SL2Z.algebra(ZZ).ngens()                                          #␣
↪needs sage.groups sage.modular sage.modules
2
sage: DihedralGroup(4).algebra(RR).ngens()                              #␣
↪needs sage.groups sage.modules
2
```

```
>>> from sage.all import *
>>> SL2Z.algebra(ZZ).ngens()                                            #␣
↪needs sage.groups sage.modular sage.modules
2
>>> DihedralGroup(Integer(4)).algebra(RR).ngens()                          ␣
↪    # needs sage.groups sage.modules
2
```

product_on_basis(*g1*, *g2*)

Product, on basis elements, as per *MagmaticAlgebras.WithBasis.ParentMethods.product_on_basis()*.

The product of two basis elements is induced by the product of the corresponding elements of the group.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(); S
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: a, b, c, d = A.algebra_generators()                               #␣
↪needs sage.modules
sage: a * b + b * d * c * d                                             #␣
↪needs sage.modules
B['ab'] + B['bdc']
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(); S
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> a, b, c, d = A.algebra_generators()                                 #␣
↪needs sage.modules
>>> a * b + b * d * c * d                                               #␣
↪needs sage.modules
B['ab'] + B['bdc']
```

regular_representation(*side='left'*)

Return the regular representation of `self`.

INPUT:

- `side` – (default: `'left'`) whether this is the `'left'` or `'right'` regular representation

EXAMPLES:

```
sage: # needs sage.groups
sage: G = groups.permutation.Dihedral(4)
sage: A = G.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: V = A.regular_representation()                                    #␣
↪needs sage.modules
sage: V == G.regular_representation(QQ)                                 #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = groups.permutation.Dihedral(Integer(4))
>>> A = G.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> V = A.regular_representation()                                      #␣
↪needs sage.modules
>>> V == G.regular_representation(QQ)                                   #␣
↪needs sage.modules
True
```

representation(*module*, *on_basis*, *side='left'*, **args*, ***kwargs*)

Return a representation of `self` on `module` with the action of the semigroup given by `on_basis`.

INPUT:

- `module` – a module with a basis
- `on_basis` – function which takes as input `g`, `m`, where `g` is an element of the semigroup and `m` is an element of the indexing set for the basis, and returns the result of `g` acting on `m`
- `side` – (default: `'left'`) whether this is a `'left'` or `'right'` representation

EXAMPLES:

```
sage: # needs sage.groups
sage: G = groups.permutation.Dihedral(5)
sage: CFM = CombinatorialFreeModule(GF(2), [1, 2, 3, 4, 5])
sage: A = G.algebra(GF(2))
sage: R = A.representation(CFM, lambda g, i: CFM.basis()[g(i)], side=
↪'right')
sage: R
Representation of Dihedral group of order 10 as a permutation
 group indexed by {1, 2, 3, 4, 5} over Finite Field of size 2
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = groups.permutation.Dihedral(Integer(5))
>>> CFM = CombinatorialFreeModule(GF(Integer(2)), [Integer(1),␣
↪Integer(2), Integer(3), Integer(4), Integer(5)])
>>> A = G.algebra(GF(Integer(2)))
>>> R = A.representation(CFM, lambda g, i: CFM.basis()[g(i)], side=
↪'right')
>>> R
```

(continues on next page)

(continued from previous page)

```
Representation of Dihedral group of order 10 as a permutation
 group indexed by {1, 2, 3, 4, 5} over Finite Field of size 2
```

trivial_representation(*side='twosided'*)

Return the trivial representation of `self`.

INPUT:

- `side` – ignored

EXAMPLES:

```
sage: # needs sage.groups
sage: G = groups.permutation.Dihedral(4)
sage: A = G.algebra(QQ)                                                 #␣
↪needs sage.modules
sage: V = A.trivial_representation()                                    #␣
↪needs sage.modules
sage: V == G.trivial_representation(QQ)                                 #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = groups.permutation.Dihedral(Integer(4))
>>> A = G.algebra(QQ)                                                   #␣
↪needs sage.modules
>>> V = A.trivial_representation()                                      #␣
↪needs sage.modules
>>> V == G.trivial_representation(QQ)                                   #␣
↪needs sage.modules
True
```

extra_super_categories()

Implement the fact that the algebra of a semigroup is indeed a (not necessarily unital) algebra.

EXAMPLES:

```
sage: Semigroups().Algebras(QQ).extra_super_categories()
[Category of semigroups]
sage: Semigroups().Algebras(QQ).super_categories()
[Category of associative algebras over Rational Field,
 Category of magma algebras over Rational Field]
```

```
>>> from sage.all import *
>>> Semigroups().Algebras(QQ).extra_super_categories()
[Category of semigroups]
>>> Semigroups().Algebras(QQ).super_categories()
[Category of associative algebras over Rational Field,
 Category of magma algebras over Rational Field]
```

Aperiodic

alias of *AperiodicSemigroups*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a Cartesian product of semigroups is a semigroup.

EXAMPLES:

```
sage: Semigroups().CartesianProducts().extra_super_categories()
[Category of semigroups]
sage: Semigroups().CartesianProducts().super_categories()
[Category of semigroups, Category of Cartesian products of magmas]
```

```
>>> from sage.all import *
>>> Semigroups().CartesianProducts().extra_super_categories()
[Category of semigroups]
>>> Semigroups().CartesianProducts().super_categories()
[Category of semigroups, Category of Cartesian products of magmas]
```

class ElementMethods

Bases: object

Finite

alias of *FiniteSemigroups*

FinitelyGeneratedAsMagma

alias of *FinitelyGeneratedSemigroups*

HTrivial

alias of *HTrivialSemigroups*

JTrivial

alias of *JTrivialSemigroups*

LTrivial

alias of *LTrivialSemigroups*

class ParentMethods

Bases: object

cayley_graph(*side='right'*, *simple=False*, *elements=None*, *generators=None*, *connecting_set=None*)

Return the Cayley graph for this finite semigroup.

INPUT:

- `side` – `'left'`, `'right'`, or `'twosided'`: the side on which the generators act (default: `'right'`)
- `simple` – boolean (default: `False`); if `True`, returns a simple graph (no loops, no labels, no multiple edges)
- `generators` – list; tuple, or family of elements of `self` (default: `self.semigroup_generators()`)
- `connecting_set` – alias for `generators`; deprecated
- `elements` – list (or iterable) of elements of `self`

OUTPUT: DiGraph

EXAMPLES:

We start with the (right) Cayley graphs of some classical groups:

```
sage: # needs sage.graphs sage.groups
sage: D4 = DihedralGroup(4); D4
Dihedral group of order 8 as a permutation group
sage: G = D4.cayley_graph()
sage: show(G, color_by_label=True, edge_labels=True)                    #␣
↪needs sage.plot
sage: A5 = AlternatingGroup(5); A5
Alternating group of order 5!/2 as a permutation group
sage: G = A5.cayley_graph()
sage: G.show3d(color_by_label=True, edge_size=0.01,                     #␣
↪needs sage.plot
....:          edge_size2=0.02, vertex_size=0.03)
sage: G.show3d(vertex_size=0.03,        # long time (less than a minute),␣
↪needs sage.plot
....:          edge_size=0.01, edge_size2=0.02,
....:          vertex_colors={(1,1,1): G.vertices(sort=True)},
....:          bgcolor=(0,0,0), color_by_label=True,
....:          xres=700, yres=700, iterations=200)
sage: G.n_edges()
120

sage: # needs sage.combinat sage.graphs sage.groups
sage: w = WeylGroup(['A', 3])
sage: d = w.cayley_graph(); d
Digraph on 24 vertices
sage: d.show3d(color_by_label=True, edge_size=0.01, vertex_size=0.03)   #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> D4 = DihedralGroup(Integer(4)); D4
Dihedral group of order 8 as a permutation group
>>> G = D4.cayley_graph()
>>> show(G, color_by_label=True, edge_labels=True)                      #␣
↪needs sage.plot
>>> A5 = AlternatingGroup(Integer(5)); A5
Alternating group of order 5!/2 as a permutation group
>>> G = A5.cayley_graph()
>>> G.show3d(color_by_label=True, edge_size=RealNumber('0.01'),          ␣
↪          # needs sage.plot
...          edge_size2=RealNumber('0.02'), vertex_size=RealNumber('0.03
↪'))
>>> G.show3d(vertex_size=RealNumber('0.03'),        # long time (less␣
↪than a minute), needs sage.plot
...          edge_size=RealNumber('0.01'), edge_size2=RealNumber('0.02'),
...          vertex_colors={(Integer(1),Integer(1),Integer(1)): G.
↪vertices(sort=True)},
...          bgcolor=(Integer(0),Integer(0),Integer(0)), color_by_
↪label=True,
...          xres=Integer(700), yres=Integer(700),␣
↪iterations=Integer(200))
>>> G.n_edges()
```

(continues on next page)

(continued from previous page)

```
120

>>> # needs sage.combinat sage.graphs sage.groups
>>> w = WeylGroup(['A', Integer(3)])
>>> d = w.cayley_graph(); d
Digraph on 24 vertices
>>> d.show3d(color_by_label=True, edge_size=RealNumber('0.01'), vertex_
↪size=RealNumber('0.03'))    # needs sage.plot
```

Alternative generators may be specified:

```
sage: # needs sage.graphs sage.groups
sage: G = A5.cayley_graph(generators=[A5.gens()[0]])
sage: G.n_edges()
60
sage: g = PermutationGroup([(i + 1, j + 1)
....:                       for i in range(5)
....:                       for j in range(5) if j != i])
sage: g.cayley_graph(generators=[(1,2), (2,3)])
Digraph on 120 vertices
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> G = A5.cayley_graph(generators=[A5.gens()[Integer(0)]])
>>> G.n_edges()
60
>>> g = PermutationGroup([(i + Integer(1), j + Integer(1))
...                       for i in range(Integer(5))
...                       for j in range(Integer(5)) if j != i])
>>> g.cayley_graph(generators=[(Integer(1),Integer(2)), (Integer(2),
↪Integer(3))])
Digraph on 120 vertices
```

If `elements` is specified, then only the subgraph induced and those elements is returned. Here we use it to display the Cayley graph of the free monoid truncated on the elements of length at most 3:

```
sage: # needs sage.combinat sage.graphs
sage: M = Monoids().example(); M
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
sage: elements = [M.prod(w)
....:             for w in sum((list(Words(M.semigroup_generators(), k))
....:                           for k in range(4)), [])]
sage: G = M.cayley_graph(elements=elements)
sage: G.n_vertices(), G.n_edges()
(85, 84)
sage: G.show3d(color_by_label=True, edge_size=0.001, vertex_size=0.01)  #␣
↪needs sage.plot
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.graphs
>>> M = Monoids().example(); M
```

(continues on next page)

(continued from previous page)

```
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
>>> elements = [M.prod(w)
...             for w in sum((list(Words(M.semigroup_generators(), k))
...                           for k in range(Integer(4))), [])]
>>> G = M.cayley_graph(elements=elements)
>>> G.n_vertices(), G.n_edges()
(85, 84)
>>> G.show3d(color_by_label=True, edge_size=RealNumber('0.001'), vertex_
↪size=RealNumber('0.01'))  # needs sage.plot
```

We now illustrate the `side` and `simple` options on a semigroup:

```
sage: S = FiniteSemigroups().example(alphabet=('a', 'b'))
sage: g = S.cayley_graph(simple=True)                                    #␣
↪needs sage.graphs
sage: g.vertices(sort=True)                                              #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
sage: g.edges(sort=True)                                                 #␣
↪needs sage.graphs
[('a', 'ab', None), ('b', 'ba', None)]
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('a', 'b'))
>>> g = S.cayley_graph(simple=True)                                      #␣
↪needs sage.graphs
>>> g.vertices(sort=True)                                                #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
>>> g.edges(sort=True)                                                   #␣
↪needs sage.graphs
[('a', 'ab', None), ('b', 'ba', None)]
```

```
sage: g = S.cayley_graph(side='left', simple=True)                       #␣
↪needs sage.graphs
sage: g.vertices(sort=True)                                              #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
sage: g.edges(sort=True)                                                 #␣
↪needs sage.graphs
[('a', 'ba', None), ('ab', 'ba', None), ('b', 'ab', None),
('ba', 'ab', None)]
```

```
>>> from sage.all import *
>>> g = S.cayley_graph(side='left', simple=True)                         #␣
↪needs sage.graphs
>>> g.vertices(sort=True)                                                #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
>>> g.edges(sort=True)                                                   #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.graphs
[('a', 'ba', None), ('ab', 'ba', None), ('b', 'ab', None),
('ba', 'ab', None)]
```

```
sage: g = S.cayley_graph(side='twosided', simple=True)                    #␣
↪needs sage.graphs
sage: g.vertices(sort=True)                                               #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
sage: g.edges(sort=True)                                                  #␣
↪needs sage.graphs
[('a', 'ab', None), ('a', 'ba', None), ('ab', 'ba', None),
('b', 'ab', None), ('b', 'ba', None), ('ba', 'ab', None)]
```

```
>>> from sage.all import *
>>> g = S.cayley_graph(side='twosided', simple=True)                      #␣
↪needs sage.graphs
>>> g.vertices(sort=True)                                                 #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
>>> g.edges(sort=True)                                                    #␣
↪needs sage.graphs
[('a', 'ab', None), ('a', 'ba', None), ('ab', 'ba', None),
('b', 'ab', None), ('b', 'ba', None), ('ba', 'ab', None)]
```

```
sage: g = S.cayley_graph(side='twosided')                                 #␣
↪needs sage.graphs
sage: g.vertices(sort=True)                                               #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
sage: g.edges(sort=True)                                                  #␣
↪needs sage.graphs
[('a', 'a', (0, 'left')), ('a', 'a', (0, 'right')), ('a', 'ab', (1, 'right
↪')), ('a', 'ba', (1, 'left')), ('ab', 'ab', (0, 'left')), ('ab', 'ab',␣
↪(0, 'right')), ('ab', 'ab', (1, 'right')), ('ab', 'ba', (1, 'left')), (
↪'b', 'ab', (0, 'left')), ('b', 'b', (1, 'left')), ('b', 'b', (1, 'right
↪')), ('b', 'ba', (0, 'right')), ('ba', 'ab', (0, 'left')), ('ba', 'ba',␣
↪(0, 'right')), ('ba', 'ba', (1, 'left')), ('ba', 'ba', (1, 'right'))]
```

```
>>> from sage.all import *
>>> g = S.cayley_graph(side='twosided')                                   #␣
↪needs sage.graphs
>>> g.vertices(sort=True)                                                 #␣
↪needs sage.graphs
['a', 'ab', 'b', 'ba']
>>> g.edges(sort=True)                                                    #␣
↪needs sage.graphs
[('a', 'a', (0, 'left')), ('a', 'a', (0, 'right')), ('a', 'ab', (1, 'right
↪')), ('a', 'ba', (1, 'left')), ('ab', 'ab', (0, 'left')), ('ab', 'ab',␣
↪(0, 'right')), ('ab', 'ab', (1, 'right')), ('ab', 'ba', (1, 'left')), (
↪'b', 'ab', (0, 'left')), ('b', 'b', (1, 'left')), ('b', 'b', (1, 'right
```

(continues on next page)

(continued from previous page)

```
↪')), ('b', 'ba', (0, 'right')), ('ba', 'ab', (0, 'left')), ('ba', 'ba',␣
↪(0, 'right')), ('ba', 'ba', (1, 'left')), ('ba', 'ba', (1, 'right'))]
```

```
sage: s1 = SymmetricGroup(1); s = s1.cayley_graph()                    #␣
↪needs sage.graphs sage.groups
sage: s.vertices(sort=False)                                           #␣
↪needs sage.graphs sage.groups
[()]
```

```
>>> from sage.all import *
>>> s1 = SymmetricGroup(Integer(1)); s = s1.cayley_graph()              ␣
↪     # needs sage.graphs sage.groups
>>> s.vertices(sort=False)                                            #␣
↪needs sage.graphs sage.groups
[()]
```

Todo

- Add more options for constructing subgraphs of the Cayley graph, handling the standard use cases when exploring large/infinite semigroups (a predicate, generators of an ideal, a maximal length in term of the generators)
- Specify good default layout/plot/latex options in the graph
- Generalize to combinatorial modules with module generators / operators

AUTHORS:

- Bobby Moretti (2007-08-10)
- Robert Miller (2008-05-01): editing
- Nicolas M. Thiery (2008-12): extension to semigroups, `side`, `simple`, and `elements` options, …

magma_generators()

An alias for *semigroup_generators()*.

EXAMPLES:

```
sage: S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c',
↪ 'd')
sage: S.magma_generators()
Family ('a', 'b', 'c', 'd')
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c',
↪ 'd')
>>> S.magma_generators()
Family ('a', 'b', 'c', 'd')
>>> S.semigroup_generators()
```

(continues on next page)

(continued from previous page)

```
Family ('a', 'b', 'c', 'd')
```

prod(*args*)

Return the product of the list of elements `args` inside `self`.

EXAMPLES:

```
sage: S = Semigroups().example("free")
sage: S.prod([S('a'), S('b'), S('c')])
'abc'
sage: S.prod([])
Traceback (most recent call last):
...
AssertionError: Cannot compute an empty product in a semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free")
>>> S.prod([S('a'), S('b'), S('c')])
'abc'
>>> S.prod([])
Traceback (most recent call last):
...
AssertionError: Cannot compute an empty product in a semigroup
```

regular_representation(*base_ring=None*, *side='left'*)

Return the regular representation of `self` over `base_ring`.

- `side` – (default: `'left'`) whether this is the `'left'` or `'right'` regular representation

EXAMPLES:

```
sage: G = groups.permutation.Dihedral(4)                                    #␣
↪needs sage.groups
sage: G.regular_representation()                                            #␣
↪needs sage.groups
Left Regular Representation of Dihedral group of order 8
 as a permutation group over Integer Ring
```

```
>>> from sage.all import *
>>> G = groups.permutation.Dihedral(Integer(4))                              ␣
↪     # needs sage.groups
>>> G.regular_representation()                                             #␣
↪needs sage.groups
Left Regular Representation of Dihedral group of order 8
 as a permutation group over Integer Ring
```

representation(*module*, *on_basis*, *side='left'*, **args*, ***kwargs*)

Return a representation of `self` on `module` with the action given by `on_basis`.

INPUT:

- `module` – a module with a basis
- `on_basis` – function which takes as input `g`, `m`, where `g` is an element of the semigroup and `m` is an element of the indexing set for the basis, and returns the result of `g` acting on `m`
- `side` – (default: `'left'`) whether this is a `'left'` or `'right'` representation

EXAMPLES:

```
sage: # needs sage.groups
sage: G = CyclicPermutationGroup(3)
sage: M = algebras.Exterior(QQ, 'x', 3)
sage: def on_basis(g, m):  # cyclically permute generators
....:     return M.prod([M.monomial(FrozenBitset([g(j+1)-1])) for j in m])
sage: from sage.categories.algebras import Algebras
sage: R = G.representation(M, on_basis, category=Algebras(QQ).WithBasis().
↪FiniteDimensional())
sage: R
Representation of Cyclic group of order 3 as a permutation group
 indexed by Subsets of {0,1,...,2} over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.groups
>>> G = CyclicPermutationGroup(Integer(3))
>>> M = algebras.Exterior(QQ, 'x', Integer(3))
>>> def on_basis(g, m):  # cyclically permute generators
...     return M.prod([M.monomial(FrozenBitset([g(j+Integer(1))-
↪Integer(1)])) for j in m])
>>> from sage.categories.algebras import Algebras
>>> R = G.representation(M, on_basis, category=Algebras(QQ).WithBasis().
↪FiniteDimensional())
>>> R
Representation of Cyclic group of order 3 as a permutation group
 indexed by Subsets of {0,1,...,2} over Rational Field
```

semigroup_generators()

Return distinguished semigroup generators for `self`.

OUTPUT: a family

This method is optional.

EXAMPLES:

```
sage: S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c',
↪ 'd')
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c',
↪ 'd')
>>> S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

subsemigroup(*generators*, *one=None*, *category=None*)

Return the multiplicative subsemigroup generated by `generators`.

INPUT:

- `generators` – a finite family of elements of `self`, or a list, iterable, … that can be converted into one (see `Family()`).
- `one` – a unit for the subsemigroup, or `None`

- `category` – a category

This implementation lazily constructs all the elements of the semigroup, and the right Cayley graph relations between them, and uses the latter as an automaton.

See `AutomaticSemigroup` for details.

EXAMPLES:

```
sage: R = IntegerModRing(15)
sage: M = R.subsemigroup([R(3), R(5)]); M                                   #␣
↪needs sage.combinat
A subsemigroup of (Ring of integers modulo 15) with 2 generators
sage: M.list()                                                              #␣
↪needs sage.combinat
[3, 5, 9, 0, 10, 12, 6]
```

```
>>> from sage.all import *
>>> R = IntegerModRing(Integer(15))
>>> M = R.subsemigroup([R(Integer(3)), R(Integer(5))]); M                   ␣
↪              # needs sage.combinat
A subsemigroup of (Ring of integers modulo 15) with 2 generators
>>> M.list()                                                                #␣
↪needs sage.combinat
[3, 5, 9, 0, 10, 12, 6]
```

By default, M is just in the category of subsemigroups:

```
sage: M in Semigroups().Subobjects()                                        #␣
↪needs sage.combinat
True
```

```
>>> from sage.all import *
>>> M in Semigroups().Subobjects()                                          #␣
↪needs sage.combinat
True
```

In the following example, we specify that M is a submonoid of the finite monoid R (it shares the same unit), and a group by itself:

```
sage: M = R.subsemigroup([R(-1)],                                           #␣
↪needs sage.combinat
....:     category=Monoids().Finite().Subobjects() & Groups()); M
A submonoid of (Ring of integers modulo 15) with 1 generators
sage: M.list()                                                              #␣
↪needs sage.combinat
[1, 14]
sage: M.one()                                                               #␣
↪needs sage.combinat
1
```

```
>>> from sage.all import *
>>> M = R.subsemigroup([R(-Integer(1))],                                    ␣
↪     # needs sage.combinat
...     category=Monoids().Finite().Subobjects() & Groups()); M
```

(continues on next page)

(continued from previous page)

```
A submonoid of (Ring of integers modulo 15) with 1 generators
>>> M.list()                                                              #␣
↪needs sage.combinat
[1, 14]
>>> M.one()                                                               #␣
↪needs sage.combinat
1
```

In the following example, M is a group; however, its unit does not coincide with that of R, so M is only a subsemigroup, and we need to specify its unit explicitly:

```
sage: M = R.subsemigroup([R(5)],                                          #␣
↪needs sage.combinat
....:     category=Semigroups().Finite().Subobjects() & Groups()); M
Traceback (most recent call last):
...
ValueError: For a monoid which is just a subsemigroup,
the unit should be specified

sage: # needs sage.combinat sage.groups
sage: M = R.subsemigroup([R(5)], one=R(10),
....:     category=Semigroups().Finite().Subobjects() & Groups()); M
A subsemigroup of (Ring of integers modulo 15) with 1 generators
sage: M in Groups()
True
sage: M.list()
[10, 5]
sage: M.one()
10
```

```
>>> from sage.all import *
>>> M = R.subsemigroup([R(Integer(5))],                                   ␣
↪     # needs sage.combinat
...     category=Semigroups().Finite().Subobjects() & Groups()); M
Traceback (most recent call last):
...
ValueError: For a monoid which is just a subsemigroup,
the unit should be specified

>>> # needs sage.combinat sage.groups
>>> M = R.subsemigroup([R(Integer(5))], one=R(Integer(10)),
...     category=Semigroups().Finite().Subobjects() & Groups()); M
A subsemigroup of (Ring of integers modulo 15) with 1 generators
>>> M in Groups()
True
>>> M.list()
[10, 5]
>>> M.one()
10
```

trivial_representation(*base_ring=None*, *side='twosided'*)

Return the trivial representation of `self` over `base_ring`.

INPUT:

- `base_ring` – (optional) the base ring; the default is **Z**
- `side` – ignored

EXAMPLES:

```
sage: G = groups.permutation.Dihedral(4)                                #␣
↪needs sage.groups
sage: G.trivial_representation()                                        #␣
↪needs sage.groups
Trivial representation of Dihedral group of order 8
 as a permutation group over Integer Ring
```

```
>>> from sage.all import *
>>> G = groups.permutation.Dihedral(Integer(4))                          ␣
↪     # needs sage.groups
>>> G.trivial_representation()                                         #␣
↪needs sage.groups
Trivial representation of Dihedral group of order 8
 as a permutation group over Integer Ring
```

class Quotients(*category*, **args*)

Bases: *QuotientsCategory*

class ParentMethods

Bases: `object`

semigroup_generators()

Return semigroup generators for `self` by retracting the semigroup generators of the ambient semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().Quotients().example().semigroup_
↪generators() # todo: not implemented
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().Quotients().example().semigroup_
↪generators() # todo: not implemented
```

example()

Return an example of quotient of a semigroup, as per *Category.example()*.

EXAMPLES:

```
sage: Semigroups().Quotients().example()
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
```

```
>>> from sage.all import *
>>> Semigroups().Quotients().example()
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
```

RTrivial

alias of *RTrivialSemigroups*

class SubcategoryMethods

Bases: object

Aperiodic()

Return the full subcategory of the aperiodic objects of self.

A (multiplicative) *semigroup* S is *aperiodic* if for any element $s \in S$, the sequence $s, s^2, s^3, \ldots$ eventually stabilizes.

In terms of variety, this can be described by the equation $s^\omega s = s$.

EXAMPLES:

```
sage: Semigroups().Aperiodic()
Category of aperiodic semigroups
```

```
>>> from sage.all import *
>>> Semigroups().Aperiodic()
Category of aperiodic semigroups
```

An aperiodic semigroup is H-trivial:

```
sage: Semigroups().Aperiodic().axioms()
frozenset({'Aperiodic', 'Associative', 'HTrivial'})
```

```
>>> from sage.all import *
>>> Semigroups().Aperiodic().axioms()
frozenset({'Aperiodic', 'Associative', 'HTrivial'})
```

In the finite case, the two notions coincide:

```
sage: Semigroups().Aperiodic().Finite() is Semigroups().HTrivial().
↪Finite()
True
```

```
>>> from sage.all import *
>>> Semigroups().Aperiodic().Finite() is Semigroups().HTrivial().Finite()
True
```

See also

- Wikipedia article Aperiodic_semigroup
- *Semigroups.SubcategoryMethods.RTrivial*
- *Semigroups.SubcategoryMethods.LTrivial*
- *Semigroups.SubcategoryMethods.JTrivial*
- *Semigroups.SubcategoryMethods.Aperiodic*

HTrivial()

Return the full subcategory of the H-trivial objects of self.

Let S be (multiplicative) *semigroup*. Two elements of S are in the same H-class if they are in the same L-class and in the same R-class.

The semigroup S is *H-trivial* if all its H-classes are trivial (that is of cardinality 1).

EXAMPLES:

```
sage: C = Semigroups().HTrivial(); C
Category of h trivial semigroups
sage: Semigroups().HTrivial().Finite().example()
NotImplemented
```

```
>>> from sage.all import *
>>> C = Semigroups().HTrivial(); C
Category of h trivial semigroups
>>> Semigroups().HTrivial().Finite().example()
NotImplemented
```

See also

- Wikipedia article Green%27s_relations
- *Semigroups.SubcategoryMethods.RTrivial*
- *Semigroups.SubcategoryMethods.LTrivial*
- *Semigroups.SubcategoryMethods.JTrivial*
- *Semigroups.SubcategoryMethods.Aperiodic*

JTrivial()

Return the full subcategory of the J-trivial objects of `self`.

Let S be (multiplicative) *semigroup*. The *J-preorder* $\leq_J$ on S is defined by:

$$x \leq_J y \qquad \Longleftrightarrow \qquad x \in SyS$$

The *J-classes* are the equivalence classes for the associated equivalence relation. The semigroup S is *J-trivial* if all its J-classes are trivial (that is of cardinality 1), or equivalently if the J-preorder is in fact a partial order.

EXAMPLES:

```
sage: C = Semigroups().JTrivial(); C
Category of j trivial semigroups
```

```
>>> from sage.all import *
>>> C = Semigroups().JTrivial(); C
Category of j trivial semigroups
```

A semigroup is J-trivial if and only if it is L-trivial and R-trivial:

```
sage: sorted(C.axioms())
['Associative', 'HTrivial', 'JTrivial', 'LTrivial', 'RTrivial']
sage: Semigroups().LTrivial().RTrivial()
Category of j trivial semigroups
```

```
>>> from sage.all import *
>>> sorted(C.axioms())
['Associative', 'HTrivial', 'JTrivial', 'LTrivial', 'RTrivial']
>>> Semigroups().LTrivial().RTrivial()
Category of j trivial semigroups
```

For a commutative semigroup, all three axioms are equivalent:

```
sage: Semigroups().Commutative().LTrivial()
Category of commutative j trivial semigroups
sage: Semigroups().Commutative().RTrivial()
Category of commutative j trivial semigroups
```

```
>>> from sage.all import *
>>> Semigroups().Commutative().LTrivial()
Category of commutative j trivial semigroups
>>> Semigroups().Commutative().RTrivial()
Category of commutative j trivial semigroups
```

See also

- Wikipedia article Green%27s_relations
- *Semigroups.SubcategoryMethods.LTrivial*
- *Semigroups.SubcategoryMethods.RTrivial*
- *Semigroups.SubcategoryMethods.HTrivial*

LTrivial()

Return the full subcategory of the L-trivial objects of `self`.

Let S be (multiplicative) *semigroup*. The *L-preorder* $\leq_L$ on S is defined by:

$$x \leq_L y \qquad \Longleftrightarrow \qquad x \in Sy$$

The *L-classes* are the equivalence classes for the associated equivalence relation. The semigroup S is *L-trivial* if all its L-classes are trivial (that is of cardinality 1), or equivalently if the L-preorder is in fact a partial order.

EXAMPLES:

```
sage: C = Semigroups().LTrivial(); C
Category of l trivial semigroups
```

```
>>> from sage.all import *
>>> C = Semigroups().LTrivial(); C
Category of l trivial semigroups
```

A L-trivial semigroup is H-trivial:

```
sage: sorted(C.axioms())
['Associative', 'HTrivial', 'LTrivial']
```

```
>>> from sage.all import *
>>> sorted(C.axioms())
['Associative', 'HTrivial', 'LTrivial']
```

See also

- Wikipedia article Green%27s_relations
- *Semigroups.SubcategoryMethods.RTrivial*
- *Semigroups.SubcategoryMethods.JTrivial*
- *Semigroups.SubcategoryMethods.HTrivial*

RTrivial()

Return the full subcategory of the R-trivial objects of `self`.

Let S be (multiplicative) *semigroup*. The *R-preorder* $\leq_R$ on S is defined by:

$$x \leq_R y \qquad \Longleftrightarrow \qquad x \in yS$$

The *R-classes* are the equivalence classes for the associated equivalence relation. The semigroup S is *R-trivial* if all its R-classes are trivial (that is of cardinality 1), or equivalently if the R-preorder is in fact a partial order.

EXAMPLES:

```
sage: C = Semigroups().RTrivial(); C
Category of r trivial semigroups
```

```
>>> from sage.all import *
>>> C = Semigroups().RTrivial(); C
Category of r trivial semigroups
```

An R-trivial semigroup is H-trivial:

```
sage: sorted(C.axioms())
['Associative', 'HTrivial', 'RTrivial']
```

```
>>> from sage.all import *
>>> sorted(C.axioms())
['Associative', 'HTrivial', 'RTrivial']
```

See also

- Wikipedia article Green%27s_relations
- *Semigroups.SubcategoryMethods.LTrivial*
- *Semigroups.SubcategoryMethods.JTrivial*
- *Semigroups.SubcategoryMethods.HTrivial*

class Subquotients(*category*, **args*)

Bases: *SubquotientsCategory*

The category of subquotient semi-groups.

EXAMPLES:

```
sage: Semigroups().Subquotients().all_super_categories()
[Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

[Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Semigroups().Subquotients().all_super_categories()
[Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]

[Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

example()

Return an example of subquotient of a semigroup, as per *Category.example()*.

EXAMPLES:

```
sage: Semigroups().Subquotients().example()
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
```

```
>>> from sage.all import *
>>> Semigroups().Subquotients().example()
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
```

Unital

alias of *Monoids*

example(*choice='leftzero'*, ***kwds*)

Return an example of a semigroup, as per *Category.example()*.

INPUT:

- `choice` – string (default: `'leftzero'`); can be either 'leftzero' for the left zero semigroup, or 'free' for the free semigroup
- `**kwds` – keyword arguments passed onto the constructor for the chosen semigroup

EXAMPLES:

```
sage: Semigroups().example(choice='leftzero')
An example of a semigroup: the left zero semigroup
sage: Semigroups().example(choice='free')
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c', 'd
↪')
sage: Semigroups().example(choice='free', alphabet=('a','b'))
An example of a semigroup: the free semigroup generated by ('a', 'b')
```

```
>>> from sage.all import *
>>> Semigroups().example(choice='leftzero')
An example of a semigroup: the left zero semigroup
>>> Semigroups().example(choice='free')
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c', 'd
↪')
>>> Semigroups().example(choice='free', alphabet=('a','b'))
An example of a semigroup: the free semigroup generated by ('a', 'b')
```

4.149 Semirings

class sage.categories.semirings.**Semirings**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of semirings.

A semiring $(S, +, *)$ is similar to a ring, but without the requirement that each element must have an additive inverse. In other words, it is a combination of a commutative additive monoid $(S, +)$ and a multiplicative monoid $(S, *)$, where $*$ distributes over $+$.

See also

Wikipedia article Semiring

EXAMPLES:

```
sage: Semirings()
Category of semirings
sage: Semirings().super_categories()
[Category of associative additive commutative additive
 associative additive unital distributive magmas and additive magmas,
 Category of monoids]

sage: sorted(Semirings().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveUnital',
 'Associative', 'Distributive', 'Unital']

sage: Semirings() is (CommutativeAdditiveMonoids() & Monoids()).Distributive()
True

sage: Semirings().AdditiveInverse()
Category of rings
```

```
>>> from sage.all import *
>>> Semirings()
Category of semirings
>>> Semirings().super_categories()
[Category of associative additive commutative additive
 associative additive unital distributive magmas and additive magmas,
 Category of monoids]

>>> sorted(Semirings().axioms())
['AdditiveAssociative', 'AdditiveCommutative', 'AdditiveUnital',
 'Associative', 'Distributive', 'Unital']

>>> Semirings() is (CommutativeAdditiveMonoids() & Monoids()).Distributive()
True

>>> Semirings().AdditiveInverse()
Category of rings
```

4.150 Semisimple Algebras

class sage.categories.semisimple_algebras.**SemisimpleAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

The category of semisimple algebras over a given base ring.

EXAMPLES:

```
sage: from sage.categories.semisimple_algebras import SemisimpleAlgebras
sage: C = SemisimpleAlgebras(QQ); C
Category of semisimple algebras over Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.semisimple_algebras import SemisimpleAlgebras
>>> C = SemisimpleAlgebras(QQ); C
Category of semisimple algebras over Rational Field
```

This category is best constructed as:

```
sage: D = Algebras(QQ).Semisimple(); D
Category of semisimple algebras over Rational Field
sage: D is C
True

sage: C.super_categories()
[Category of algebras over Rational Field]
```

```
>>> from sage.all import *
>>> D = Algebras(QQ).Semisimple(); D
Category of semisimple algebras over Rational Field
>>> D is C
True

>>> C.super_categories()
[Category of algebras over Rational Field]
```

Typically, finite group algebras are semisimple:

```
sage: DihedralGroup(5).algebra(QQ) in SemisimpleAlgebras                    #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> DihedralGroup(Integer(5)).algebra(QQ) in SemisimpleAlgebras                ␣
↪     # needs sage.groups
True
```

Unless the characteristic of the field divides the order of the group:

```
sage: DihedralGroup(5).algebra(IntegerModRing(5)) in SemisimpleAlgebras     #␣
↪needs sage.groups
False

sage: DihedralGroup(5).algebra(IntegerModRing(7)) in SemisimpleAlgebras     #␣
↪needs sage.groups
True
```

```
>>> from sage.all import *
>>> DihedralGroup(Integer(5)).algebra(IntegerModRing(Integer(5))) in␣
↪SemisimpleAlgebras          # needs sage.groups
False

>>> DihedralGroup(Integer(5)).algebra(IntegerModRing(Integer(7))) in␣
↪SemisimpleAlgebras          # needs sage.groups
True
```

See also

Wikipedia article Semisimple_algebra

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

WithBasis

alias of *FiniteDimensionalSemisimpleAlgebrasWithBasis*

class ParentMethods

Bases: object

radical_basis(***keywords*)

Return a basis of the Jacobson radical of this algebra.

- keywords – for compatibility; ignored

OUTPUT: the empty list since this algebra is semisimple

EXAMPLES:

```
sage: A = SymmetricGroup(4).algebra(QQ)                                 #␣
↪needs sage.combinat sage.groups
sage: A.radical_basis()                                                 #␣
↪needs sage.combinat sage.groups
()
```

```
>>> from sage.all import *
>>> A = SymmetricGroup(Integer(4)).algebra(QQ)                              ␣
↪     # needs sage.combinat sage.groups
>>> A.radical_basis()                                                    #␣
↪needs sage.combinat sage.groups
()
```

super_categories()

EXAMPLES:

```
sage: Algebras(QQ).Semisimple().super_categories()
[Category of algebras over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).Semisimple().super_categories()
[Category of algebras over Rational Field]
```

4.151 Sets

Sets.ParentMethods.**_test_elements_eq_symmetric**(***options*)

Run generic tests on the equality of elements.

This tests that == is symmetric.

See also: TestSuite.

EXAMPLES:

```
sage: C = Sets().example()
sage: C._test_elements_eq_symmetric()
```

```
>>> from sage.all import *
>>> C = Sets().example()
>>> C._test_elements_eq_symmetric()
```

We test a non symmetric equality:

```
sage: P = Sets().example("wrapper")
sage: P._test_elements_eq_symmetric()                                    # needs␣
↪sage.libs.pari
sage: eq = P.element_class.__eq__

sage: def non_sym_eq(x, y):
....:     if not y in P:                        return False
....:     elif eq(x, P(47)) and eq(y, P(53)): return True
....:     else:                                 return eq(x, y)
sage: P.element_class.__eq__ = non_sym_eq
sage: P._test_elements_eq_symmetric()                                    # needs␣
↪sage.libs.pari
Traceback (most recent call last):
...
AssertionError: non symmetric equality: 47 == 53 but 53 != 47
```

```
>>> from sage.all import *
>>> P = Sets().example("wrapper")
>>> P._test_elements_eq_symmetric()                                      # needs␣
↪sage.libs.pari
>>> eq = P.element_class.__eq__

>>> def non_sym_eq(x, y):
...     if not y in P:                        return False
...     elif eq(x, P(Integer(47))) and eq(y, P(Integer(53))): return True
...     else:                                 return eq(x, y)
>>> P.element_class.__eq__ = non_sym_eq
>>> P._test_elements_eq_symmetric()                                      # needs␣
↪sage.libs.pari
Traceback (most recent call last):
...
AssertionError: non symmetric equality: 47 == 53 but 53 != 47
```

We restore `P.element_class` in a proper state for further tests:

```
sage: P.element_class.__eq__ = eq
```

```
>>> from sage.all import *
>>> P.element_class.__eq__ = eq
```

`Sets.ParentMethods.`**`_test_elements_eq_transitive`**`(`*`**options`*`)`

Run generic tests on the equality of elements.

Test that == is transitive.

See also: `TestSuite`.

EXAMPLES:

```
sage: C = Sets().example()
sage: C._test_elements_eq_transitive()
```

```
>>> from sage.all import *
>>> C = Sets().example()
>>> C._test_elements_eq_transitive()
```

We test a non transitive equality:

```
sage: R = Zp(3)
↪  # needs sage.rings.padics
sage: test = raw_getattr(Sets().ParentMethods, "_test_elements_eq_transitive")
sage: test(R, elements=[R(3,2), R(3,1), R(0)])
↪  # needs sage.rings.padics
Traceback (most recent call last):
...
AssertionError: non transitive equality:
3 + O(3^2) == O(3) and O(3) == 0 but 3 + O(3^2) != 0
```

```
>>> from sage.all import *
>>> R = Zp(Integer(3))
↪         # needs sage.rings.padics
>>> test = raw_getattr(Sets().ParentMethods, "_test_elements_eq_transitive")
>>> test(R, elements=[R(Integer(3),Integer(2)), R(Integer(3),Integer(1)),
↪R(Integer(0))])                                    # needs sage.rings.padics
Traceback (most recent call last):
...
AssertionError: non transitive equality:
3 + O(3^2) == O(3) and O(3) == 0 but 3 + O(3^2) != 0
```

exception sage.categories.sets_cat.**EmptySetError**

Bases: ValueError

Exception raised when some operation can't be performed on the empty set.

EXAMPLES:

```
sage: def first_element(st):
....:  if not st: raise EmptySetError("no elements")
....:  else: return st[0]
sage: first_element(Set((1,2,3)))
1
sage: first_element(Set([]))
Traceback (most recent call last):
...
EmptySetError: no elements
```

```
>>> from sage.all import *
>>> def first_element(st):
...  if not st: raise EmptySetError("no elements")
...  else: return st[Integer(0)]
>>> first_element(Set((Integer(1),Integer(2),Integer(3))))
1
```

(continues on next page)

(continued from previous page)

```
>>> first_element(Set([]))
Traceback (most recent call last):
...
EmptySetError: no elements
```

class sage.categories.sets_cat.**Sets**

Bases: *Category_singleton*

The category of sets.

The base category for collections of elements with = (equality).

This is also the category whose objects are all parents.

EXAMPLES:

```
sage: Sets()
Category of sets
sage: Sets().super_categories()
[Category of sets with partial maps]
sage: Sets().all_super_categories()
[Category of sets, Category of sets with partial maps, Category of objects]
```

```
>>> from sage.all import *
>>> Sets()
Category of sets
>>> Sets().super_categories()
[Category of sets with partial maps]
>>> Sets().all_super_categories()
[Category of sets, Category of sets with partial maps, Category of objects]
```

Let us consider an example of set:

```
sage: P = Sets().example("inherits")
sage: P
Set of prime numbers
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> P
Set of prime numbers
```

See `P??` for the code.

P is in the category of sets:

```
sage: P.category()
Category of sets
```

```
>>> from sage.all import *
>>> P.category()
Category of sets
```

and therefore gets its methods from the following classes:

```
sage: for cl in P.__class__.mro(): print(cl)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Abstract'>
<class 'sage.structure.unique_representation.UniqueRepresentation'>
<class 'sage.misc.fast_methods.WithEqualityById'>
<class 'sage.structure.unique_representation.CachedRepresentation'>
<class 'sage.structure.unique_representation.WithPicklingByInitArgs'>
<class 'sage.structure.parent.Parent'>
<class 'sage.structure.category_object.CategoryObject'>
<class 'sage.structure.sage_object.SageObject'>
<class 'sage.categories.sets_cat.Sets.parent_class'>
<class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.parent_class'>
<class 'sage.categories.objects.Objects.parent_class'>
<class 'object'>
```

```
>>> from sage.all import *
>>> for cl in P.__class__.mro(): print(cl)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Abstract'>
<class 'sage.structure.unique_representation.UniqueRepresentation'>
<class 'sage.misc.fast_methods.WithEqualityById'>
<class 'sage.structure.unique_representation.CachedRepresentation'>
<class 'sage.structure.unique_representation.WithPicklingByInitArgs'>
<class 'sage.structure.parent.Parent'>
<class 'sage.structure.category_object.CategoryObject'>
<class 'sage.structure.sage_object.SageObject'>
<class 'sage.categories.sets_cat.Sets.parent_class'>
<class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.parent_class'>
<class 'sage.categories.objects.Objects.parent_class'>
<class 'object'>
```

We run some generic checks on P:

```
sage: TestSuite(P).run(verbose=True)                                    #␣
↪needs sage.libs.pari
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(P).run(verbose=True)                                    #␣
↪needs sage.libs.pari
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

Now, we manipulate some elements of P:

```
sage: P.an_element()
47
sage: x = P(3)
sage: x.parent()
Set of prime numbers
sage: x in P, 4 in P
(True, False)
sage: x.is_prime()
True
```

```
>>> from sage.all import *
>>> P.an_element()
47
>>> x = P(Integer(3))
>>> x.parent()
Set of prime numbers
>>> x in P, Integer(4) in P
(True, False)
>>> x.is_prime()
```

(continues on next page)

(continued from previous page)

```
True
```

They get their methods from the following classes:

```
sage: for cl in x.__class__.mro(): print(cl)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits.Element'>
<class 'sage.rings.integer.IntegerWrapper'>
<class 'sage.rings.integer.Integer'>
<class 'sage.structure.element.EuclideanDomainElement'>
<class 'sage.structure.element.PrincipalIdealDomainElement'>
<class 'sage.structure.element.DedekindDomainElement'>
<class 'sage.structure.element.IntegralDomainElement'>
<class 'sage.structure.element.CommutativeRingElement'>
<class 'sage.structure.element.RingElement'>
<class 'sage.structure.element.ModuleElement'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Abstract.Element'>
<class 'sage.structure.element.Element'>
<class 'sage.structure.sage_object.SageObject'>
<class 'sage.categories.sets_cat.Sets.element_class'>
<class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.element_class'>
<class 'sage.categories.objects.Objects.element_class'>
<... 'object'>
```

```
>>> from sage.all import *
>>> for cl in x.__class__.mro(): print(cl)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits.Element'>
<class 'sage.rings.integer.IntegerWrapper'>
<class 'sage.rings.integer.Integer'>
<class 'sage.structure.element.EuclideanDomainElement'>
<class 'sage.structure.element.PrincipalIdealDomainElement'>
<class 'sage.structure.element.DedekindDomainElement'>
<class 'sage.structure.element.IntegralDomainElement'>
<class 'sage.structure.element.CommutativeRingElement'>
<class 'sage.structure.element.RingElement'>
<class 'sage.structure.element.ModuleElement'>
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Abstract.Element'>
<class 'sage.structure.element.Element'>
<class 'sage.structure.sage_object.SageObject'>
<class 'sage.categories.sets_cat.Sets.element_class'>
<class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.element_class'>
<class 'sage.categories.objects.Objects.element_class'>
<... 'object'>
```

FIXME: Objects.element_class is not very meaningful …

class Algebras(*category*, **args*)

Bases: *AlgebrasCategory*

class ParentMethods

Bases: object

construction()

Return the functorial construction of self.

EXAMPLES:

```
sage: A = GroupAlgebra(KleinFourGroup(), QQ)                                #␣
↪needs sage.groups sage.modules
sage: F, arg = A.construction(); F, arg                                     #␣
↪needs sage.groups sage.modules
(GroupAlgebraFunctor, Rational Field)
sage: F(arg) is A                                                           #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> A = GroupAlgebra(KleinFourGroup(), QQ)                                  #␣
↪needs sage.groups sage.modules
>>> F, arg = A.construction(); F, arg                                       #␣
↪needs sage.groups sage.modules
(GroupAlgebraFunctor, Rational Field)
>>> F(arg) is A                                                             #␣
↪needs sage.groups sage.modules
True
```

This also works for structures such as monoid algebras (see Issue #27937):

```
sage: A = FreeAbelianMonoid('x,y').algebra(QQ)                              #␣
↪needs sage.combinat sage.modules
sage: F, arg = A.construction(); F, arg                                     #␣
↪needs sage.groups sage.modules
(The algebra functorial construction,
 Free abelian monoid on 2 generators (x, y))
sage: F(arg) is A                                                           #␣
↪needs sage.groups sage.modules
True
```

```
>>> from sage.all import *
>>> A = FreeAbelianMonoid('x,y').algebra(QQ)                                #␣
↪needs sage.combinat sage.modules
>>> F, arg = A.construction(); F, arg                                       #␣
↪needs sage.groups sage.modules
(The algebra functorial construction,
 Free abelian monoid on 2 generators (x, y))
>>> F(arg) is A                                                             #␣
↪needs sage.groups sage.modules
True
```

extra_super_categories()

EXAMPLES:

```
sage: Sets().Algebras(ZZ).super_categories()
[Category of modules with basis over Integer Ring]
```

(continues on next page)

(continued from previous page)

```
sage: Sets().Algebras(QQ).extra_super_categories()
[Category of vector spaces with basis over Rational Field]

sage: Sets().example().algebra(ZZ).categories()                             #␣
↪needs sage.modules
[Category of set algebras over Integer Ring,
 Category of modules with basis over Integer Ring,
 ...
 Category of objects]
```

```
>>> from sage.all import *
>>> Sets().Algebras(ZZ).super_categories()
[Category of modules with basis over Integer Ring]

>>> Sets().Algebras(QQ).extra_super_categories()
[Category of vector spaces with basis over Rational Field]

>>> Sets().example().algebra(ZZ).categories()                               #␣
↪needs sage.modules
[Category of set algebras over Integer Ring,
 Category of modules with basis over Integer Ring,
 ...
 Category of objects]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

EXAMPLES:

```
sage: C = Sets().CartesianProducts().example()
sage: C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
sage: C.category()
Category of Cartesian products of sets
sage: C.categories()
[Category of Cartesian products of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> C = Sets().CartesianProducts().example()
>>> C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
>>> C.category()
Category of Cartesian products of sets
>>> C.categories()
```

(continues on next page)

(continued from previous page)

```
[Category of Cartesian products of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
>>> TestSuite(C).run()
```

class ElementMethods

Bases: object

cartesian_factors()

Return the Cartesian factors of self.

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(ZZ, [4,5]); F.rename('F')
sage: G = CombinatorialFreeModule(ZZ, [4,6]); G.rename('G')
sage: H = CombinatorialFreeModule(ZZ, [4,7]); H.rename('H')
sage: S = cartesian_product([F, G, H])
sage: x = (S.monomial((0,4)) + 2 * S.monomial((0,5))
....:      + 3 * S.monomial((1,6)) + 4 * S.monomial((2,4))
....:      + 5 * S.monomial((2,7)))
sage: x.cartesian_factors()
(B[4] + 2*B[5], 3*B[6], 4*B[4] + 5*B[7])
sage: [s.parent() for s in x.cartesian_factors()]
[F, G, H]
sage: S.zero().cartesian_factors()
(0, 0, 0)
sage: [s.parent() for s in S.zero().cartesian_factors()]
[F, G, H]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(ZZ, [Integer(4),Integer(5)]); F.rename(
↪'F')
>>> G = CombinatorialFreeModule(ZZ, [Integer(4),Integer(6)]); G.rename(
↪'G')
>>> H = CombinatorialFreeModule(ZZ, [Integer(4),Integer(7)]); H.rename(
↪'H')
>>> S = cartesian_product([F, G, H])
>>> x = (S.monomial((Integer(0),Integer(4))) + Integer(2) * S.
↪monomial((Integer(0),Integer(5)))
...      + Integer(3) * S.monomial((Integer(1),Integer(6))) +␣
↪Integer(4) * S.monomial((Integer(2),Integer(4)))
...      + Integer(5) * S.monomial((Integer(2),Integer(7))))
>>> x.cartesian_factors()
(B[4] + 2*B[5], 3*B[6], 4*B[4] + 5*B[7])
>>> [s.parent() for s in x.cartesian_factors()]
[F, G, H]
>>> S.zero().cartesian_factors()
(0, 0, 0)
>>> [s.parent() for s in S.zero().cartesian_factors()]
[F, G, H]
```

cartesian_projection(*i*)

Return the projection of `self` onto the i-th factor of the Cartesian product.

INPUT:

- `i` – the index of a factor of the Cartesian product

EXAMPLES:

```
sage: # needs sage.modules
sage: F = CombinatorialFreeModule(ZZ, [4,5]); F.rename('F')
sage: G = CombinatorialFreeModule(ZZ, [4,6]); G.rename('G')
sage: S = cartesian_product([F, G])
sage: x = (S.monomial((0,4)) + 2 * S.monomial((0,5))
....:      + 3 * S.monomial((1,6)))
sage: x.cartesian_projection(0)
B[4] + 2*B[5]
sage: x.cartesian_projection(1)
3*B[6]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = CombinatorialFreeModule(ZZ, [Integer(4),Integer(5)]); F.rename(
↪'F')
>>> G = CombinatorialFreeModule(ZZ, [Integer(4),Integer(6)]); G.rename(
↪'G')
>>> S = cartesian_product([F, G])
>>> x = (S.monomial((Integer(0),Integer(4))) + Integer(2) * S.
↪monomial((Integer(0),Integer(5)))
...      + Integer(3) * S.monomial((Integer(1),Integer(6))))
>>> x.cartesian_projection(Integer(0))
B[4] + 2*B[5]
>>> x.cartesian_projection(Integer(1))
3*B[6]
```

class ParentMethods

Bases: `object`

an_element()

EXAMPLES:

```
sage: C = Sets().CartesianProducts().example(); C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
sage: C.an_element()
(47, 42, 1)
```

```
>>> from sage.all import *
>>> C = Sets().CartesianProducts().example(); C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
>>> C.an_element()
(47, 42, 1)
```

cardinality()

Return the cardinality of `self`.

EXAMPLES:

```
sage: E = FiniteEnumeratedSet([1,2,3])
sage: C = cartesian_product([E, SymmetricGroup(4)])                    #␣
↪needs sage.groups
sage: C.cardinality()                                                  #␣
↪needs sage.groups
72

sage: E = FiniteEnumeratedSet([])
sage: C = cartesian_product([E, ZZ, QQ])
sage: C.cardinality()
0

sage: C = cartesian_product([ZZ, QQ])
sage: C.cardinality()
+Infinity

sage: cartesian_product([GF(5), Permutations(10)]).cardinality()
18144000
sage: cartesian_product([GF(71)]*20).cardinality() == 71**20
True
```

```
>>> from sage.all import *
>>> E = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> C = cartesian_product([E, SymmetricGroup(Integer(4))])              ␣
↪    # needs sage.groups
>>> C.cardinality()                                                    #␣
↪needs sage.groups
72

>>> E = FiniteEnumeratedSet([])
>>> C = cartesian_product([E, ZZ, QQ])
>>> C.cardinality()
0

>>> C = cartesian_product([ZZ, QQ])
>>> C.cardinality()
+Infinity

>>> cartesian_product([GF(Integer(5)), Permutations(Integer(10))]).
↪cardinality()
18144000
>>> cartesian_product([GF(Integer(71))]*Integer(20)).cardinality() ==␣
↪Integer(71)**Integer(20)
True
```

cartesian_factors()

Return the Cartesian factors of `self`.

EXAMPLES:

```
sage: cartesian_product([QQ, ZZ, ZZ]).cartesian_factors()
(Rational Field, Integer Ring, Integer Ring)
```

```
>>> from sage.all import *
>>> cartesian_product([QQ, ZZ, ZZ]).cartesian_factors()
(Rational Field, Integer Ring, Integer Ring)
```

cartesian_projection(*i*)

Return the natural projection onto the i-th Cartesian factor of `self`.

INPUT:

• `i` – the index of a Cartesian factor of `self`

EXAMPLES:

```
sage: C = Sets().CartesianProducts().example(); C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
sage: x = C.an_element(); x
(47, 42, 1)
sage: pi = C.cartesian_projection(1)
sage: pi(x)
42
```

```
>>> from sage.all import *
>>> C = Sets().CartesianProducts().example(); C
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
>>> x = C.an_element(); x
(47, 42, 1)
>>> pi = C.cartesian_projection(Integer(1))
>>> pi(x)
42
```

construction()

The construction functor and the list of Cartesian factors.

EXAMPLES:

```
sage: C = cartesian_product([QQ, ZZ, ZZ])
sage: C.construction()
(The cartesian_product functorial construction,
(Rational Field, Integer Ring, Integer Ring))
```

```
>>> from sage.all import *
>>> C = cartesian_product([QQ, ZZ, ZZ])
>>> C.construction()
(The cartesian_product functorial construction,
(Rational Field, Integer Ring, Integer Ring))
```

is_empty()

Return whether this set is empty.

EXAMPLES:

```
sage: S1 = FiniteEnumeratedSet([1,2,3])
sage: S2 = Set([])
sage: cartesian_product([S1,ZZ]).is_empty()
False
sage: cartesian_product([S1,S2,S1]).is_empty()
True
```

```
>>> from sage.all import *
>>> S1 = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> S2 = Set([])
>>> cartesian_product([S1,ZZ]).is_empty()
False
>>> cartesian_product([S1,S2,S1]).is_empty()
True
```

Even when some parent did not implement `is_empty`, as long as one element is nonempty, the result can be determined:

```
sage: C = ConditionSet(QQ, lambda x: x > 0)
sage: C.is_empty()
Traceback (most recent call last):
...
AttributeError...
sage: cartesian_product([C,[]]).is_empty()
True
sage: cartesian_product([C,C]).is_empty()
Traceback (most recent call last):
...
NotImplementedError...
```

```
>>> from sage.all import *
>>> C = ConditionSet(QQ, lambda x: x > Integer(0))
>>> C.is_empty()
Traceback (most recent call last):
...
AttributeError...
>>> cartesian_product([C,[]]).is_empty()
True
>>> cartesian_product([C,C]).is_empty()
Traceback (most recent call last):
...
NotImplementedError...
```

is_finite()

Return whether this set is finite.

EXAMPLES:

```
sage: E = FiniteEnumeratedSet([1,2,3])
sage: C = cartesian_product([E, SymmetricGroup(4)])                    #␣
↪needs sage.groups
sage: C.is_finite()                                                    #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.groups
True

sage: cartesian_product([ZZ,ZZ]).is_finite()
False
sage: cartesian_product([ZZ, Set(), ZZ]).is_finite()
True
```

```
>>> from sage.all import *
>>> E = FiniteEnumeratedSet([Integer(1),Integer(2),Integer(3)])
>>> C = cartesian_product([E, SymmetricGroup(Integer(4))])
↪    # needs sage.groups
>>> C.is_finite()
↪needs sage.groups
True

>>> cartesian_product([ZZ,ZZ]).is_finite()
False
>>> cartesian_product([ZZ, Set(), ZZ]).is_finite()
True
```

random_element(**args*)

Return a random element of this Cartesian product.

The extra arguments are passed down to each of the factors of the Cartesian product.

EXAMPLES:

```
sage: C = cartesian_product([Permutations(10)]*5)
sage: C.random_element()            # random
([2, 9, 4, 7, 1, 8, 6, 10, 5, 3],
 [8, 6, 5, 7, 1, 4, 9, 3, 10, 2],
 [5, 10, 3, 8, 2, 9, 1, 4, 7, 6],
 [9, 6, 10, 3, 2, 1, 5, 8, 7, 4],
 [8, 5, 2, 9, 10, 3, 7, 1, 4, 6])

sage: C = cartesian_product([ZZ]*10)
sage: c1 = C.random_element()
sage: c1                    # random
(3, 1, 4, 1, 1, -3, 0, -4, -17, 2)
sage: c2 = C.random_element(4,7)
sage: c2                    # random
(6, 5, 6, 4, 5, 6, 6, 4, 5, 5)
sage: all(4 <= i <= 7 for i in c2)
True
```

```
>>> from sage.all import *
>>> C = cartesian_product([Permutations(Integer(10))]*Integer(5))
>>> C.random_element()            # random
([2, 9, 4, 7, 1, 8, 6, 10, 5, 3],
 [8, 6, 5, 7, 1, 4, 9, 3, 10, 2],
 [5, 10, 3, 8, 2, 9, 1, 4, 7, 6],
```

(continues on next page)

(continued from previous page)

```
 [9, 6, 10, 3, 2, 1, 5, 8, 7, 4],
 [8, 5, 2, 9, 10, 3, 7, 1, 4, 6])

>>> C = cartesian_product([ZZ]*Integer(10))
>>> c1 = C.random_element()
>>> c1                    # random
(3, 1, 4, 1, 1, -3, 0, -4, -17, 2)
>>> c2 = C.random_element(Integer(4),Integer(7))
>>> c2                    # random
(6, 5, 6, 4, 5, 6, 6, 4, 5, 5)
>>> all(Integer(4) <= i <= Integer(7) for i in c2)
True
```

example()

EXAMPLES:

```
sage: Sets().CartesianProducts().example()
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
```

```
>>> from sage.all import *
>>> Sets().CartesianProducts().example()
The Cartesian product of (Set of prime numbers (basic implementation),
 An example of an infinite enumerated set: the nonnegative integers,
 An example of a finite enumerated set: {1,2,3})
```

extra_super_categories()

A Cartesian product of sets is a set.

EXAMPLES:

```
sage: Sets().CartesianProducts().extra_super_categories()
[Category of sets]
sage: Sets().CartesianProducts().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Sets().CartesianProducts().extra_super_categories()
[Category of sets]
>>> Sets().CartesianProducts().super_categories()
[Category of sets]
```

class ElementMethods

Bases: object

cartesian_product(**elements*)

Return the Cartesian product of its arguments, as an element of the Cartesian product of the parents of those elements.

EXAMPLES:

```
sage: C = AlgebrasWithBasis(QQ)
sage: A = C.example()                                                    #␣
↪needs sage.combinat sage.modules
sage: a, b, c = A.algebra_generators()                                   #␣
↪needs sage.combinat sage.modules
sage: a.cartesian_product(b, c)                                          #␣
↪needs sage.combinat sage.modules
B[(0, word: a)] + B[(1, word: b)] + B[(2, word: c)]
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(QQ)
>>> A = C.example()                                                      #␣
↪needs sage.combinat sage.modules
>>> a, b, c = A.algebra_generators()                                     #␣
↪needs sage.combinat sage.modules
>>> a.cartesian_product(b, c)                                            #␣
↪needs sage.combinat sage.modules
B[(0, word: a)] + B[(1, word: b)] + B[(2, word: c)]
```

FIXME: is this a policy that we want to enforce on all parents?

Enumerated

alias of *EnumeratedSets*

Facade

alias of *FacadeSets*

Finite

alias of *FiniteSets*

class Infinite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class ParentMethods

Bases: object

cardinality()

Count the elements of the enumerated set.

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN.cardinality()
+Infinity
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN.cardinality()
+Infinity
```

is_empty()

Return whether this set is empty.

Since this set is infinite this always returns `False`.

EXAMPLES:

```
sage: C = InfiniteEnumeratedSets().example()
sage: C.is_empty()
False
```

```
>>> from sage.all import *
>>> C = InfiniteEnumeratedSets().example()
>>> C.is_empty()
False
```

is_finite()

Return whether this set is finite.

Since this set is infinite this always returns `False`.

EXAMPLES:

```
sage: C = InfiniteEnumeratedSets().example()
sage: C.is_finite()
False
```

```
>>> from sage.all import *
>>> C = InfiniteEnumeratedSets().example()
>>> C.is_finite()
False
```

class SubcategoryMethods

Bases: `object`

Finite()

Incompatible axiom.

EXAMPLES:

```
sage: C = NN.category()
sage: C.Finite()
Traceback (most recent call last):
...
TypeError: incompatible axioms: finite and infinite
```

```
>>> from sage.all import *
>>> C = NN.category()
>>> C.Finite()
Traceback (most recent call last):
...
TypeError: incompatible axioms: finite and infinite
```

class IsomorphicObjects(*category*, **args*)

Bases: *IsomorphicObjectsCategory*

A category for isomorphic objects of sets.

EXAMPLES:

```
sage: Sets().IsomorphicObjects()
Category of isomorphic objects of sets
sage: Sets().IsomorphicObjects().all_super_categories()
[Category of isomorphic objects of sets,
 Category of subobjects of sets, Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Sets().IsomorphicObjects()
Category of isomorphic objects of sets
>>> Sets().IsomorphicObjects().all_super_categories()
[Category of isomorphic objects of sets,
 Category of subobjects of sets, Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ParentMethods

Bases: object

Metric

alias of *MetricSpaces*

class MorphismMethods

Bases: object

image(*domain_subset=None*)

Return the image of the domain or of domain_subset.

EXAMPLES:

```
sage: # needs sage.combinat
sage: P = Partitions(6)
sage: H = Hom(P, ZZ)
sage: f = H(ZZ.sum)
sage: X = f.image()                                                    #␣
↪needs sage.libs.flint
sage: list(X)                                                          #␣
↪needs sage.libs.flint
[6]
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> P = Partitions(Integer(6))
>>> H = Hom(P, ZZ)
>>> f = H(ZZ.sum)
>>> X = f.image()                                                      #␣
↪needs sage.libs.flint
>>> list(X)                                                            #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.libs.flint
[6]
```

is_injective()

Return whether this map is injective.

EXAMPLES:

```
sage: f = ZZ.hom(GF(3)); f
Natural morphism:
  From: Integer Ring
  To:   Finite Field of size 3
sage: f.is_injective()
False
```

```
>>> from sage.all import *
>>> f = ZZ.hom(GF(Integer(3))); f
Natural morphism:
  From: Integer Ring
  To:   Finite Field of size 3
>>> f.is_injective()
False
```

class ParentMethods

Bases: object

CartesianProduct

alias of CartesianProduct

algebra(*base_ring*, *category=None*, ***kwds*)

Return the algebra of `self` over `base_ring`.

INPUT:

- `self` – a parent S
- `base_ring` – a ring K
- `category` – a super category of the category of S, or `None`

This returns the space of formal linear combinations of elements of S with coefficients in K, endowed with whatever structure can be induced from that of S. See the documentation of *sage.categories.algebra_functor* for details.

EXAMPLES:

If S is a *group*, the result is its group algebra KS:

```
sage: # needs sage.groups sage.modules
sage: S = DihedralGroup(4); S
Dihedral group of order 8 as a permutation group
sage: A = S.algebra(QQ); A
Algebra of Dihedral group of order 8 as a permutation group
 over Rational Field
sage: A.category()
Category of finite group algebras over Rational Field
sage: a = A.an_element(); a
() + (1,3) + 2*(1,3)(2,4) + 3*(1,4,3,2)
```

```
>>> from sage.all import *
>>> # needs sage.groups sage.modules
>>> S = DihedralGroup(Integer(4)); S
Dihedral group of order 8 as a permutation group
>>> A = S.algebra(QQ); A
Algebra of Dihedral group of order 8 as a permutation group
 over Rational Field
>>> A.category()
Category of finite group algebras over Rational Field
>>> a = A.an_element(); a
() + (1,3) + 2*(1,3)(2,4) + 3*(1,4,3,2)
```

This space is endowed with an algebra structure, obtained by extending by bilinearity the multiplication of G to a multiplication on RG:

```
sage: a * a                                                                 #␣
↪needs sage.groups sage.modules
6*() + 4*(2,4) + 3*(1,2)(3,4) + 12*(1,2,3,4) + 2*(1,3)
 + 13*(1,3)(2,4) + 6*(1,4,3,2) + 3*(1,4)(2,3)
```

```
>>> from sage.all import *
>>> a * a                                                                   #␣
↪needs sage.groups sage.modules
6*() + 4*(2,4) + 3*(1,2)(3,4) + 12*(1,2,3,4) + 2*(1,3)
 + 13*(1,3)(2,4) + 6*(1,4,3,2) + 3*(1,4)(2,3)
```

If S is a *monoid*, the result is its monoid algebra KS:

```
sage: S = Monoids().example(); S
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ); A                                                  #␣
↪needs sage.modules
Algebra of
 An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
 over Rational Field
sage: A.category()                                                          #␣
↪needs sage.modules
Category of monoid algebras over Rational Field
```

```
>>> from sage.all import *
>>> S = Monoids().example(); S
An example of a monoid:
 the free monoid generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ); A                                                    #␣
↪needs sage.modules
Algebra of
 An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
 over Rational Field
>>> A.category()                                                            #␣
↪needs sage.modules
Category of monoid algebras over Rational Field
```

Similarly, we can construct algebras for additive magmas, monoids, and groups.

One may specify for which category one takes the algebra; here we build the algebra of the additive group GF_3:

```
sage: # needs sage.modules
sage: from sage.categories.additive_groups import AdditiveGroups
sage: S = GF(7)
sage: A = S.algebra(QQ, category=AdditiveGroups()); A
Algebra of Finite Field of size 7 over Rational Field
sage: A.category()
Category of finite dimensional additive group algebras
         over Rational Field
sage: a = A(S(1))
sage: a
1
sage: 1 + a * a * a
0 + 3
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> from sage.categories.additive_groups import AdditiveGroups
>>> S = GF(Integer(7))
>>> A = S.algebra(QQ, category=AdditiveGroups()); A
Algebra of Finite Field of size 7 over Rational Field
>>> A.category()
Category of finite dimensional additive group algebras
         over Rational Field
>>> a = A(S(Integer(1)))
>>> a
1
>>> Integer(1) + a * a * a
0 + 3
```

Note that the `category` keyword needs to be fed with the structure on S to be used, not the induced structure on the result.

an_element()

Return a (preferably typical) element of this parent.

This is used both for illustration and testing purposes. If the set `self` is empty, *an_element()* should raise the exception *EmptySetError*.

This default implementation calls `_an_element_` and caches the result. Any parent should implement either *an_element()* or `_an_element_`.

EXAMPLES:

```
sage: CDF.an_element()
↪# needs sage.rings.complex_double
1.0*I
sage: ZZ[['t']].an_element()
t
```

```
>>> from sage.all import *
>>> CDF.an_element()                                                    #␣
↪needs sage.rings.complex_double
1.0*I
>>> ZZ[['t']].an_element()
t
```

cartesian_product(**parents*, ***kwargs*)

Return the Cartesian product of the parents.

INPUT:

- `parents` – list (or other iterable) of parents
- `category` – (default: `None`) the category the Cartesian product belongs to. If `None` is passed, then *`category_from_parents()`* is used to determine the category.
- `extra_category` – (default: `None`) a category that is added to the Cartesian product in addition to the categories obtained from the parents.
- other keyword arguments will passed on to the class used for this Cartesian product (see also `CartesianProduct`).

OUTPUT: the Cartesian product

EXAMPLES:

```
sage: C = AlgebrasWithBasis(QQ)
sage: A = C.example(); A.rename('A')                                    #␣
↪needs sage.combinat sage.modules
sage: A.cartesian_product(A, A)                                         #␣
↪needs sage.combinat sage.modules
A (+) A (+) A
sage: ZZ.cartesian_product(GF(2), FiniteEnumeratedSet([1,2,3]))
The Cartesian product of (Integer Ring,
                          Finite Field of size 2, {1, 2, 3})

sage: C = ZZ.cartesian_product(A); C                                    #␣
↪needs sage.combinat sage.modules
The Cartesian product of (Integer Ring, A)
```

```
>>> from sage.all import *
>>> C = AlgebrasWithBasis(QQ)
>>> A = C.example(); A.rename('A')                                      #␣
↪needs sage.combinat sage.modules
>>> A.cartesian_product(A, A)                                           #␣
↪needs sage.combinat sage.modules
A (+) A (+) A
>>> ZZ.cartesian_product(GF(Integer(2)), FiniteEnumeratedSet([Integer(1),
↪Integer(2),Integer(3)]))
The Cartesian product of (Integer Ring,
                          Finite Field of size 2, {1, 2, 3})

>>> C = ZZ.cartesian_product(A); C                                      #␣
↪needs sage.combinat sage.modules
The Cartesian product of (Integer Ring, A)
```

construction()

Return a pair `(functor, parent)` such that `functor(parent)` returns `self`. If `self` does not

have a functorial construction, return `None`.

EXAMPLES:

```
sage: QQ.construction()
(FractionField, Integer Ring)
sage: f, R = QQ['x'].construction()
sage: f
Poly[x]
sage: R
Rational Field
sage: f(R)
Univariate Polynomial Ring in x over Rational Field
```

```
>>> from sage.all import *
>>> QQ.construction()
(FractionField, Integer Ring)
>>> f, R = QQ['x'].construction()
>>> f
Poly[x]
>>> R
Rational Field
>>> f(R)
Univariate Polynomial Ring in x over Rational Field
```

is_parent_of(*element*)

Return whether `self` is the parent of `element`.

INPUT:

- `element` – any object

EXAMPLES:

```
sage: S = ZZ
sage: S.is_parent_of(1)
True
sage: S.is_parent_of(2/1)
False
```

```
>>> from sage.all import *
>>> S = ZZ
>>> S.is_parent_of(Integer(1))
True
>>> S.is_parent_of(Integer(2)/Integer(1))
False
```

This method differs from `__contains__()` because it does not attempt any coercion:

```
sage: 2/1 in S, S.is_parent_of(2/1)
(True, False)
sage: int(1) in S, S.is_parent_of(int(1))
(True, False)
```

```
>>> from sage.all import *
>>> Integer(2)/Integer(1) in S, S.is_parent_of(Integer(2)/Integer(1))
```

(continues on next page)

(continued from previous page)

```
(True, False)
>>> int(Integer(1)) in S, S.is_parent_of(int(Integer(1)))
(True, False)
```

some_elements()

Return a list (or iterable) of elements of `self`.

This is typically used for running generic tests (see `TestSuite`).

This default implementation calls *an_element()*.

EXAMPLES:

```
sage: S = Sets().example(); S
Set of prime numbers (basic implementation)
sage: S.an_element()
47
sage: S.some_elements()
[47]
sage: S = Set([])
sage: list(S.some_elements())
[]
```

```
>>> from sage.all import *
>>> S = Sets().example(); S
Set of prime numbers (basic implementation)
>>> S.an_element()
47
>>> S.some_elements()
[47]
>>> S = Set([])
>>> list(S.some_elements())
[]
```

This method should return an iterable, *not* an iterator.

class Quotients(*category*, **args*)

Bases: *QuotientsCategory*

A category for quotients of sets.

See also

Quotients()

EXAMPLES:

```
sage: Sets().Quotients()
Category of quotients of sets
sage: Sets().Quotients().all_super_categories()
[Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
```

(continues on next page)

(continued from previous page)

```
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Sets().Quotients()
Category of quotients of sets
>>> Sets().Quotients().all_super_categories()
[Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ParentMethods

Bases: object

class Realizations(*category*, **args*)

Bases: *RealizationsCategory*

class ParentMethods

Bases: object

realization_of()

Return the parent this is a realization of.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: In = A.In(); In                                               #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field in the In basis
sage: In.realization_of()                                           #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> In = A.In(); In                                                 #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field in the In basis
>>> In.realization_of()                                             #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

class SubcategoryMethods

Bases: object

Algebras(*base_ring*)

Return the category of objects constructed as algebras of objects of `self` over `base_ring`.

INPUT:

- `base_ring` – a ring

See *Sets.ParentMethods.algebra()* for the precise meaning in Sage of the *algebra of an object*.

EXAMPLES:

```
sage: Monoids().Algebras(QQ)
Category of monoid algebras over Rational Field

sage: Groups().Algebras(QQ)
Category of group algebras over Rational Field

sage: AdditiveMagmas().AdditiveAssociative().Algebras(QQ)
Category of additive semigroup algebras over Rational Field

sage: Monoids().Algebras(Rings())
Category of monoid algebras over Category of rings
```

```
>>> from sage.all import *
>>> Monoids().Algebras(QQ)
Category of monoid algebras over Rational Field

>>> Groups().Algebras(QQ)
Category of group algebras over Rational Field

>>> AdditiveMagmas().AdditiveAssociative().Algebras(QQ)
Category of additive semigroup algebras over Rational Field

>>> Monoids().Algebras(Rings())
Category of monoid algebras over Category of rings
```

See also

- *algebra_functor.AlgebrasCategory*
- *CovariantFunctorialConstruction*

CartesianProducts()

Return the full subcategory of the objects of `self` constructed as Cartesian products.

See also

- *cartesian_product.CartesianProductFunctor*
- *RegressiveCovariantConstructionCategory*

EXAMPLES:

```
sage: Sets().CartesianProducts()
Category of Cartesian products of sets
sage: Semigroups().CartesianProducts()
Category of Cartesian products of semigroups
```

(continues on next page)

(continued from previous page)

```
sage: EuclideanDomains().CartesianProducts()
Category of Cartesian products of commutative rings
```

```
>>> from sage.all import *
>>> Sets().CartesianProducts()
Category of Cartesian products of sets
>>> Semigroups().CartesianProducts()
Category of Cartesian products of semigroups
>>> EuclideanDomains().CartesianProducts()
Category of Cartesian products of commutative rings
```

Enumerated()

Return the full subcategory of the enumerated objects of `self`.

An enumerated object can be iterated to get its elements.

EXAMPLES:

```
sage: Sets().Enumerated()
Category of enumerated sets
sage: Rings().Finite().Enumerated()
Category of finite enumerated rings
sage: Rings().Infinite().Enumerated()
Category of infinite enumerated rings
```

```
>>> from sage.all import *
>>> Sets().Enumerated()
Category of enumerated sets
>>> Rings().Finite().Enumerated()
Category of finite enumerated rings
>>> Rings().Infinite().Enumerated()
Category of infinite enumerated rings
```

Facade()

Return the full subcategory of the facade objects of `self`.

What is a facade set?

Recall that, in Sage, *sets are modelled by *parents**, and their elements know which distinguished set they belong to. For example, the ring of integers **Z** is modelled by the parent `ZZ`, and integers know that they belong to this set:

```
sage: ZZ
Integer Ring
sage: 42.parent()
Integer Ring
```

```
>>> from sage.all import *
>>> ZZ
Integer Ring
>>> Integer(42).parent()
Integer Ring
```

Sometimes, it is convenient to represent the elements of a parent `P` by elements of some other parent. For example, the elements of the set of prime numbers are represented by plain integers:

```
sage: Primes()
Set of all prime numbers: 2, 3, 5, 7, ...
sage: p = Primes().an_element(); p
43
sage: p.parent()
Integer Ring
```

```
>>> from sage.all import *
>>> Primes()
Set of all prime numbers: 2, 3, 5, 7, ...
>>> p = Primes().an_element(); p
43
>>> p.parent()
Integer Ring
```

In this case, `P` is called a *facade set*.

This feature is advertised through the category of P:

```
sage: Primes().category()
Category of facade infinite enumerated sets
sage: Sets().Facade()
Category of facade sets
```

```
>>> from sage.all import *
>>> Primes().category()
Category of facade infinite enumerated sets
>>> Sets().Facade()
Category of facade sets
```

Typical use cases include modeling a subset of an existing parent:

```
sage: Set([4,6,9])                    # random
{4, 6, 9}
sage: Sets().Facade().example()
An example of facade set: the monoid of positive integers
```

```
>>> from sage.all import *
>>> Set([Integer(4),Integer(6),Integer(9)])                    # random
{4, 6, 9}
>>> Sets().Facade().example()
An example of facade set: the monoid of positive integers
```

or the union of several parents:

```
sage: Sets().Facade().example("union")
An example of a facade set: the integers completed by +-infinity
```

```
>>> from sage.all import *
>>> Sets().Facade().example("union")
An example of a facade set: the integers completed by +-infinity
```

or endowing an existing parent with more (or less!) structure:

```
sage: Posets().example("facade")
An example of a facade poset: the positive integers ordered by␣
↪divisibility
```

```
>>> from sage.all import *
>>> Posets().example("facade")
An example of a facade poset: the positive integers ordered by␣
↪divisibility
```

Let us investigate in detail a close variant of this last example: let P be set of divisors of 12 partially ordered by divisibility. There are two options for representing its elements:

1. as plain integers:

```
sage: P = Poset((divisors(12), attrcall("divides")), facade=True)     ␣
↪ # needs sage.graphs
```

```
>>> from sage.all import *
>>> P = Poset((divisors(Integer(12)), attrcall("divides")),␣
↪facade=True)       # needs sage.graphs
```

2. as integers, modified to be aware that their parent is P:

```
sage: Q = Poset((divisors(12), attrcall("divides")), facade=False)    ␣
↪ # needs sage.graphs
```

```
>>> from sage.all import *
>>> Q = Poset((divisors(Integer(12)), attrcall("divides")),␣
↪facade=False)       # needs sage.graphs
```

The advantage of option 1. is that one needs not do conversions back and forth between P and $\mathbf{Z}$. The disadvantage is that this introduces an ambiguity when writing $2 < 3$: does this compare 2 and 3 w.r.t. the natural order on integers or w.r.t. divisibility?:

```
sage: 2 < 3
True
```

```
>>> from sage.all import *
>>> Integer(2) < Integer(3)
True
```

To raise this ambiguity, one needs to explicitly specify the underlying poset as in $2 <_P 3$:

```
sage: P = Posets().example("facade")
sage: P.lt(2,3)
False
```

```
>>> from sage.all import *
>>> P = Posets().example("facade")
>>> P.lt(Integer(2),Integer(3))
False
```

On the other hand, with option 2. and once constructed, the elements know unambiguously how to compare themselves:

```
sage: Q(2) < Q(3)                                                    #␣
↪needs sage.graphs
False
sage: Q(2) < Q(6)                                                    #␣
↪needs sage.graphs
True
```

```
>>> from sage.all import *
>>> Q(Integer(2)) < Q(Integer(3))                                    ␣
↪              # needs sage.graphs
False
>>> Q(Integer(2)) < Q(Integer(6))                                    ␣
↪              # needs sage.graphs
True
```

Beware that `P(2)` is still the integer 2. Therefore `P(2) < P(3)` still compares 2 and 3 as integers!:

```
sage: P(2) < P(3)
True
```

```
>>> from sage.all import *
>>> P(Integer(2)) < P(Integer(3))
True
```

In short P being a facade parent is one of the programmatic counterparts (with e.g. coercions) of the usual mathematical idiom: "for ease of notation, we identify an element of P with the corresponding integer". Too many identifications lead to confusion; the lack thereof leads to heavy, if not obfuscated, notations. Finding the right balance is an art, and even though there are common guidelines, it is ultimately up to the writer to choose which identifications to do. This is no different in code.

See also

The following examples illustrate various ways to implement subsets like the set of prime numbers; look at their code for details:

```
sage: Sets().example("facade")
Set of prime numbers (facade implementation)
sage: Sets().example("inherits")
Set of prime numbers
sage: Sets().example("wrapper")
Set of prime numbers (wrapper implementation)
```

```
>>> from sage.all import *
>>> Sets().example("facade")
Set of prime numbers (facade implementation)
>>> Sets().example("inherits")
Set of prime numbers
>>> Sets().example("wrapper")
Set of prime numbers (wrapper implementation)
```

Specifications

A parent which is a facade must either:

- call `Parent` initialization using the `facade` parameter to specify a parent, or tuple thereof.
- overload the method *`facade_for()`*.

Note

The concept of facade parents was originally introduced in the computer algebra system MuPAD.

`Finite``()`

Return the full subcategory of the finite objects of `self`.

EXAMPLES:

```
sage: Sets().Finite()
Category of finite sets
sage: Rings().Finite()
Category of finite rings
```

```
>>> from sage.all import *
>>> Sets().Finite()
Category of finite sets
>>> Rings().Finite()
Category of finite rings
```

`Infinite``()`

Return the full subcategory of the infinite objects of `self`.

EXAMPLES:

```
sage: Sets().Infinite()
Category of infinite sets
sage: Rings().Infinite()
Category of infinite rings
```

```
>>> from sage.all import *
>>> Sets().Infinite()
Category of infinite sets
>>> Rings().Infinite()
Category of infinite rings
```

`IsomorphicObjects``()`

Return the full subcategory of the objects of `self` constructed by isomorphism.

Given a concrete category `As()` (i.e. a subcategory of `Sets()`), `As().IsomorphicObjects()` returns the category of objects of `As()` endowed with a distinguished description as the image of some other object of `As()` by an isomorphism in this category.

See *`Subquotients()`* for background.

EXAMPLES:

In the following example, A is defined as the image by $x \mapsto x^2$ of the finite set $B = \{1, 2, 3\}$:

```
sage: A = FiniteEnumeratedSets().IsomorphicObjects().example(); A
The image by some isomorphism of An example of a finite enumerated set:
↪{1,2,3}
```

```
>>> from sage.all import *
>>> A = FiniteEnumeratedSets().IsomorphicObjects().example(); A
The image by some isomorphism of An example of a finite enumerated set:
↪{1,2,3}
```

Since B is a finite enumerated set, so is A:

```
sage: A in FiniteEnumeratedSets()
True
sage: A.cardinality()
3
sage: A.list()
[1, 4, 9]
```

```
>>> from sage.all import *
>>> A in FiniteEnumeratedSets()
True
>>> A.cardinality()
3
>>> A.list()
[1, 4, 9]
```

The isomorphism from B to A is available as:

```
sage: A.retract(3)
9
```

```
>>> from sage.all import *
>>> A.retract(Integer(3))
9
```

and its inverse as:

```
sage: A.lift(9)
3
```

```
>>> from sage.all import *
>>> A.lift(Integer(9))
3
```

It often is natural to declare those morphisms as coercions so that one can do `A(b)` and `B(a)` to go back and forth between A and B (TODO: refer to a category example where the maps are declared as a coercion). This is not done by default. Indeed, in many cases one only wants to transport part of the structure of B to A. Assume for example, that one wants to construct the set of integers $B = ZZ$, endowed with `max` as addition, and `+` as multiplication instead of the usual `+` and `*`. One can construct A as isomorphic to B as an infinite enumerated set. However A is *not* isomorphic to B as a ring; for example, for $a \in A$ and $a \in B$, the expressions $a + A(b)$ and $B(a) + b$ give completely different results; hence we would not want the expression $a + b$ to be implicitly resolved to any one of above two, as the coercion mechanism would do.

Coercions also cannot be used with facade parents (see *Sets.Facade*) like in the example above.

We now look at a category of isomorphic objects:

```
sage: C = Sets().IsomorphicObjects(); C
Category of isomorphic objects of sets

sage: C.super_categories()
[Category of subobjects of sets, Category of quotients of sets]

sage: C.all_super_categories()
[Category of isomorphic objects of sets,
 Category of subobjects of sets,
 Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = Sets().IsomorphicObjects(); C
Category of isomorphic objects of sets

>>> C.super_categories()
[Category of subobjects of sets, Category of quotients of sets]

>>> C.all_super_categories()
[Category of isomorphic objects of sets,
 Category of subobjects of sets,
 Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

Unless something specific about isomorphic objects is implemented for this category, one actually get an optimized super category:

```
sage: C = Semigroups().IsomorphicObjects(); C
Join of Category of quotients of semigroups
    and Category of isomorphic objects of sets
```

```
>>> from sage.all import *
>>> C = Semigroups().IsomorphicObjects(); C
Join of Category of quotients of semigroups
    and Category of isomorphic objects of sets
```

See also

- *Subquotients()* for background
- *isomorphic_objects.IsomorphicObjectsCategory*
- *RegressiveCovariantConstructionCategory*

Metric()

Return the subcategory of the metric objects of `self`.

Quotients()

Return the full subcategory of the objects of `self` constructed as quotients.

Given a concrete category `As()` (i.e. a subcategory of `Sets()`), `As().Quotients()` returns the category of objects of `As()` endowed with a distinguished description as quotient (in fact homomorphic image) of some other object of `As()`.

Implementing an object of `As().Quotients()` is done in the same way as for `As().Subquotients()`; namely by providing an ambient space and a lift and a retract map. See *Subquotients()* for detailed instructions.

See also

- *Subquotients()* for background
- *quotients.QuotientsCategory*
- *RegressiveCovariantConstructionCategory*

EXAMPLES:

```
sage: C = Semigroups().Quotients(); C
Category of quotients of semigroups
sage: C.super_categories()
[Category of subquotients of semigroups, Category of quotients of sets]
sage: C.all_super_categories()
[Category of quotients of semigroups,
 Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
 Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = Semigroups().Quotients(); C
Category of quotients of semigroups
>>> C.super_categories()
[Category of subquotients of semigroups, Category of quotients of sets]
>>> C.all_super_categories()
[Category of quotients of semigroups,
 Category of subquotients of semigroups,
 Category of semigroups,
 Category of subquotients of magmas,
 Category of magmas,
```

(continues on next page)

(continued from previous page)

```
 Category of quotients of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

The caller is responsible for checking that the given category admits a well defined category of quotients:

```
sage: EuclideanDomains().Quotients()
Join of Category of euclidean domains
    and Category of subquotients of monoids
    and Category of quotients of semigroups
```

```
>>> from sage.all import *
>>> EuclideanDomains().Quotients()
Join of Category of euclidean domains
    and Category of subquotients of monoids
    and Category of quotients of semigroups
```

Subobjects()

Return the full subcategory of the objects of `self` constructed as subobjects.

Given a concrete category `As()` (i.e. a subcategory of `Sets()`), `As().Subobjects()` returns the category of objects of `As()` endowed with a distinguished embedding into some other object of `As()`.

Implementing an object of `As().Subobjects()` is done in the same way as for `As().Subquotients()`; namely by providing an ambient space and a lift and a retract map. In the case of a trivial embedding, the two maps will typically be identity maps that just change the parent of their argument. See *Subquotients()* for detailed instructions.

See also

- *Subquotients()* for background
- *subobjects.SubobjectsCategory*
- *RegressiveCovariantConstructionCategory*

EXAMPLES:

```
sage: C = Sets().Subobjects(); C
Category of subobjects of sets

sage: C.super_categories()
[Category of subquotients of sets]

sage: C.all_super_categories()
[Category of subobjects of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C = Sets().Subobjects(); C
Category of subobjects of sets

>>> C.super_categories()
[Category of subquotients of sets]

>>> C.all_super_categories()
[Category of subobjects of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

Unless something specific about subobjects is implemented for this category, one actually gets an optimized super category:

```
sage: C = Semigroups().Subobjects(); C
Join of Category of subquotients of semigroups
    and Category of subobjects of sets
```

```
>>> from sage.all import *
>>> C = Semigroups().Subobjects(); C
Join of Category of subquotients of semigroups
    and Category of subobjects of sets
```

The caller is responsible for checking that the given category admits a well defined category of subobjects.

Subquotients()

Return the full subcategory of the objects of `self` constructed as subquotients.

Given a concrete category `self == As()` (i.e. a subcategory of `Sets()`), `As().Subquotients()` returns the category of objects of `As()` endowed with a distinguished description as subquotient of some other object of `As()`.

EXAMPLES:

```
sage: Monoids().Subquotients()
Category of subquotients of monoids
```

```
>>> from sage.all import *
>>> Monoids().Subquotients()
Category of subquotients of monoids
```

A parent A in `As()` is further in `As().Subquotients()` if there is a distinguished parent B in `As()`, called the *ambient set*, a subobject B' of B, and a pair of maps:

$$l : A \to B' \text{ and } r : B' \to A$$

called respectively the *lifting map* and *retract map* such that $r \circ l$ is the identity of A and r is a morphism in `As()`.

Todo

Draw the typical commutative diagram.

It follows that, for each operation op of the category, we have some property like:

$$op_A(e) = r(op_B(l(e))), \text{ for all } e \in A$$

This allows for implementing the operations on A from those on B.

The two most common use cases are:

- *homomorphic images* (or *quotients*), when $B' = B$, r is an homomorphism from B to A (typically a canonical quotient map), and l a section of it (not necessarily a homomorphism); see `Quotients()`;
- *subobjects* (up to an isomorphism), when l is an embedding from A into B; in this case, B' is typically isomorphic to A through the inverse isomorphisms r and l; see `Subobjects()`;

Note

- The usual definition of "subquotient" (Wikipedia article Subquotient) does not involve the lifting map l. This map is required in Sage's context to make the definition constructive. It is only used in computations and does not affect their results. This is relatively harmless since the category is a concrete category (i.e., its objects are sets and its morphisms are set maps).
- In mathematics, especially in the context of quotients, the retract map r is often referred to as a *projection map* instead.
- Since B' is not specified explicitly, it is possible to abuse the framework with situations where B' is not quite a subobject and r not quite a morphism, as long as the lifting and retract maps can be used as above to compute all the operations in A. Use at your own risk!

Assumptions:

- For any category `As()`, `As().Subquotients()` is a subcategory of `As()`.

  Example: a subquotient of a group is a group (e.g., a left or right quotient of a group by a non-normal subgroup is not in this category).
- This construction is covariant: if `As()` is a subcategory of `Bs()`, then `As().Subquotients()` is a subcategory of `Bs().Subquotients()`.

  Example: if A is a subquotient of B in the category of groups, then it is also a subquotient of B in the category of monoids.
- If the user (or a program) calls `As().Subquotients()`, then it is assumed that subquotients are well defined in this category. This is not checked, and probably never will be. Note that, if a category `As()` does not specify anything about its subquotients, then its subquotient category looks like this:

```
sage: EuclideanDomains().Subquotients()
Join of Category of euclidean domains
    and Category of subquotients of monoids
```

```
>>> from sage.all import *
>>> EuclideanDomains().Subquotients()
Join of Category of euclidean domains
    and Category of subquotients of monoids
```

Interface: the ambient set B of A is given by `A.ambient()`. The subset B' needs not be specified, so the retract map is handled as a partial map from B to A.

The lifting and retract map are implemented respectively as methods `A.lift(a)` and `A.retract(b)`. As a shorthand for the former, one can use alternatively `a.lift()`:

```
sage: S = Semigroups().Subquotients().example(); S
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
sage: S.ambient()
An example of a semigroup: the left zero semigroup
sage: S(3).lift().parent()
An example of a semigroup: the left zero semigroup
sage: S(3) * S(1) == S.retract( S(3).lift() * S(1).lift() )
True
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example(); S
An example of a (sub)quotient semigroup: a quotient of the left zero␣
↪semigroup
>>> S.ambient()
An example of a semigroup: the left zero semigroup
>>> S(Integer(3)).lift().parent()
An example of a semigroup: the left zero semigroup
>>> S(Integer(3)) * S(Integer(1)) == S.retract( S(Integer(3)).lift() *␣
↪S(Integer(1)).lift() )
True
```

See `S?` for more.

Todo

use a more interesting example, like $\mathbf{Z}/n\mathbf{Z}$.

See also

- *Quotients()*, *Subobjects()*, *IsomorphicObjects()*
- *subquotients.SubquotientsCategory*
- *RegressiveCovariantConstructionCategory*

Topological()

Return the subcategory of the topological objects of `self`.

class Subobjects(*category*, **args*)

Bases: *SubobjectsCategory*

A category for subobjects of sets.

See also

Subobjects()

EXAMPLES:

```
sage: Sets().Subobjects()
Category of subobjects of sets
sage: Sets().Subobjects().all_super_categories()
[Category of subobjects of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Sets().Subobjects()
Category of subobjects of sets
>>> Sets().Subobjects().all_super_categories()
[Category of subobjects of sets,
 Category of subquotients of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ParentMethods

Bases: object

class Subquotients(*category*, **args*)

Bases: *SubquotientsCategory*

A category for subquotients of sets.

See also

Subquotients()

EXAMPLES:

```
sage: Sets().Subquotients()
Category of subquotients of sets
sage: Sets().Subquotients().all_super_categories()
[Category of subquotients of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> Sets().Subquotients()
Category of subquotients of sets
>>> Sets().Subquotients().all_super_categories()
[Category of subquotients of sets, Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

class ElementMethods

Bases: object

lift()

Lift `self` to the ambient space for its parent.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: s = S.an_element()
sage: s, s.parent()
(42, An example of a (sub)quotient semigroup:
      a quotient of the left zero semigroup)
sage: S.lift(s), S.lift(s).parent()
(42, An example of a semigroup: the left zero semigroup)
sage: s.lift(), s.lift().parent()
(42, An example of a semigroup: the left zero semigroup)
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> s = S.an_element()
>>> s, s.parent()
(42, An example of a (sub)quotient semigroup:
      a quotient of the left zero semigroup)
>>> S.lift(s), S.lift(s).parent()
(42, An example of a semigroup: the left zero semigroup)
>>> s.lift(), s.lift().parent()
(42, An example of a semigroup: the left zero semigroup)
```

class ParentMethods

Bases: object

ambient()

Return the ambient space for self.

EXAMPLES:

```
sage: Semigroups().Subquotients().example().ambient()
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> Semigroups().Subquotients().example().ambient()
An example of a semigroup: the left zero semigroup
```

See also

Sets.SubcategoryMethods.Subquotients() for the specifications and *lift()* and *retract()*.

lift(x)

Lift x to the ambient space for self.

INPUT:

- x – an element of self

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: s = S.an_element()
sage: s, s.parent()
```

(continues on next page)

(continued from previous page)

```
(42, An example of a (sub)quotient semigroup:
      a quotient of the left zero semigroup)
sage: S.lift(s), S.lift(s).parent()
(42, An example of a semigroup: the left zero semigroup)
sage: s.lift(), s.lift().parent()
(42, An example of a semigroup: the left zero semigroup)
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> s = S.an_element()
>>> s, s.parent()
(42, An example of a (sub)quotient semigroup:
      a quotient of the left zero semigroup)
>>> S.lift(s), S.lift(s).parent()
(42, An example of a semigroup: the left zero semigroup)
>>> s.lift(), s.lift().parent()
(42, An example of a semigroup: the left zero semigroup)
```

See also

Sets.SubcategoryMethods.Subquotients for the specifications, *ambient()*, *retract()*, and also *lift()*.

retract(*x*)

Retract `x` to `self`.

INPUT:

- `x` – an element of the ambient space for `self`

See also

Sets.SubcategoryMethods.Subquotients for the specifications, *ambient()*, *lift()*, and the element method *lift()*.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: s = S.ambient().an_element()
sage: s, s.parent()
(42, An example of a semigroup: the left zero semigroup)
sage: S.retract(s), S.retract(s).parent()
(42, An example of a (sub)quotient semigroup:
      a quotient of the left zero semigroup)
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> s = S.ambient().an_element()
>>> s, s.parent()
(42, An example of a semigroup: the left zero semigroup)
>>> S.retract(s), S.retract(s).parent()
```

(continues on next page)

(continued from previous page)

```
(42, An example of a (sub)quotient semigroup:
     a quotient of the left zero semigroup)
```

Topological

alias of *TopologicalSpaces*

class WithRealizations(*category*, **args*)

Bases: *WithRealizationsCategory*

class ParentMethods

Bases: object

class Realizations(*parent_with_realization*)

Bases: *Category_realization_of_parent*

super_categories()

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.Realizations().super_categories()                           #␣
↪needs sage.modules
[Category of realizations of sets]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.Realizations().super_categories()                             #␣
↪needs sage.modules
[Category of realizations of sets]
```

a_realization()

Return a realization of `self`.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.a_realization()                                             #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
 in the Fundamental basis
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.a_realization()                                               #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
 in the Fundamental basis
```

facade_for()

Return the parents `self` is a facade for, that is the realizations of `self`

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.facade_for()                                                #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]

sage: # needs sage.combinat sage.modules
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: f = A.F().an_element(); f
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
sage: i = A.In().an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
sage: o = A.Out().an_element(); o
Out[{}] + 2*Out[{1}] + 3*Out[{2}] + Out[{1, 2}]
sage: f in A, i in A, o in A
(True, True, True)
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.facade_for()                                                  #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]

>>> # needs sage.combinat sage.modules
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> f = A.F().an_element(); f
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
>>> i = A.In().an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
>>> o = A.Out().an_element(); o
Out[{}] + 2*Out[{1}] + 3*Out[{2}] + Out[{1, 2}]
>>> f in A, i in A, o in A
(True, True, True)
```

inject_shorthands(*shorthands=None*, *verbose=True*)

Import standard shorthands into the global namespace.

INPUT:

- `shorthands` – list (or iterable) of strings (default: `self._shorthands`) or `'all'` (for `self._shorthands_all`)
- **`verbose` – boolean (default: `True`);**
  whether to print the defined shorthands

EXAMPLES:

When computing with a set with multiple realizations, like `SymmetricFunctions` or *SubsetAlgebra*, it is convenient to define shorthands for the various realizations, but cumbersome to do it by hand:

```
sage: S = SymmetricFunctions(ZZ); S                                         #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring
sage: s = S.s(); s                                                          #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the Schur basis
sage: e = S.e(); e                                                          #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the elementary basis
```

```
>>> from sage.all import *
>>> S = SymmetricFunctions(ZZ); S                                           #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring
>>> s = S.s(); s                                                            #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the Schur basis
>>> e = S.e(); e                                                            #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the elementary basis
```

This method automates the process:

```
sage: # needs sage.combinat sage.modules
sage: S.inject_shorthands()
Defining e as shorthand for
 Symmetric Functions over Integer Ring in the elementary basis
Defining f as shorthand for
 Symmetric Functions over Integer Ring in the forgotten basis
Defining h as shorthand for
 Symmetric Functions over Integer Ring in the homogeneous basis
Defining m as shorthand for
 Symmetric Functions over Integer Ring in the monomial basis
Defining p as shorthand for
 Symmetric Functions over Integer Ring in the powersum basis
Defining s as shorthand for
 Symmetric Functions over Integer Ring in the Schur basis
sage: s[1] + e[2] * p[1,1] + 2*h[3] + m[2,1]
s[1] - 2*s[1, 1, 1] + s[1, 1, 1, 1] + s[2, 1]
+ 2*s[2, 1, 1] + s[2, 2] + 2*s[3] + s[3, 1]
sage: e
Symmetric Functions over Integer Ring in the elementary basis
sage: p
Symmetric Functions over Integer Ring in the powersum basis
```

(continues on next page)

(continued from previous page)

```
sage: s
Symmetric Functions over Integer Ring in the Schur basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> S.inject_shorthands()
Defining e as shorthand for
 Symmetric Functions over Integer Ring in the elementary basis
Defining f as shorthand for
 Symmetric Functions over Integer Ring in the forgotten basis
Defining h as shorthand for
 Symmetric Functions over Integer Ring in the homogeneous basis
Defining m as shorthand for
 Symmetric Functions over Integer Ring in the monomial basis
Defining p as shorthand for
 Symmetric Functions over Integer Ring in the powersum basis
Defining s as shorthand for
 Symmetric Functions over Integer Ring in the Schur basis
>>> s[Integer(1)] + e[Integer(2)] * p[Integer(1),Integer(1)] +␣
↪Integer(2)*h[Integer(3)] + m[Integer(2),Integer(1)]
s[1] - 2*s[1, 1, 1] + s[1, 1, 1, 1] + s[2, 1]
+ 2*s[2, 1, 1] + s[2, 2] + 2*s[3] + s[3, 1]
>>> e
Symmetric Functions over Integer Ring in the elementary basis
>>> p
Symmetric Functions over Integer Ring in the powersum basis
>>> s
Symmetric Functions over Integer Ring in the Schur basis
```

Sometimes, like for symmetric functions, one can request for all shorthands to be defined, including less common ones:

```
sage: S.inject_shorthands("all")                                        #␣
↪needs lrcalc_python sage.combinat sage.modules
Defining e as shorthand for
 Symmetric Functions over Integer Ring in the elementary basis
Defining f as shorthand for
 Symmetric Functions over Integer Ring in the forgotten basis
Defining h as shorthand for
 Symmetric Functions over Integer Ring in the homogeneous basis
Defining ht as shorthand for
 Symmetric Functions over Integer Ring in the
  induced trivial symmetric group character basis
Defining m as shorthand for
 Symmetric Functions over Integer Ring in the monomial basis
Defining o as shorthand for
 Symmetric Functions over Integer Ring in the orthogonal basis
Defining p as shorthand for
 Symmetric Functions over Integer Ring in the powersum basis
Defining s as shorthand for
 Symmetric Functions over Integer Ring in the Schur basis
Defining sp as shorthand for
```

(continues on next page)

(continued from previous page)

```
 Symmetric Functions over Integer Ring in the symplectic basis
Defining st as shorthand for
 Symmetric Functions over Integer Ring in the
  irreducible symmetric group character basis
Defining w as shorthand for
 Symmetric Functions over Integer Ring in the Witt basis
Defining xt as shorthand for
 Symmetric Functions over Integer Ring in the irreducible rook monoid␣
↪character basis
```

```
>>> from sage.all import *
>>> S.inject_shorthands("all")                                          #␣
↪needs lrcalc_python sage.combinat sage.modules
Defining e as shorthand for
 Symmetric Functions over Integer Ring in the elementary basis
Defining f as shorthand for
 Symmetric Functions over Integer Ring in the forgotten basis
Defining h as shorthand for
 Symmetric Functions over Integer Ring in the homogeneous basis
Defining ht as shorthand for
 Symmetric Functions over Integer Ring in the
  induced trivial symmetric group character basis
Defining m as shorthand for
 Symmetric Functions over Integer Ring in the monomial basis
Defining o as shorthand for
 Symmetric Functions over Integer Ring in the orthogonal basis
Defining p as shorthand for
 Symmetric Functions over Integer Ring in the powersum basis
Defining s as shorthand for
 Symmetric Functions over Integer Ring in the Schur basis
Defining sp as shorthand for
 Symmetric Functions over Integer Ring in the symplectic basis
Defining st as shorthand for
 Symmetric Functions over Integer Ring in the
  irreducible symmetric group character basis
Defining w as shorthand for
 Symmetric Functions over Integer Ring in the Witt basis
Defining xt as shorthand for
 Symmetric Functions over Integer Ring in the irreducible rook monoid␣
↪character basis
```

The messages can be silenced by setting `verbose=False`:

```
sage: # needs sage.combinat sage.modules
sage: Q = QuasiSymmetricFunctions(ZZ)
sage: Q.inject_shorthands(verbose=False)
sage: F[1,2,1] + 5*M[1,3] + F[2]^2
5*F[1, 1, 1, 1] - 5*F[1, 1, 2] - 3*F[1, 2, 1] + 6*F[1, 3] +
2*F[2, 2] + F[3, 1] + F[4]
sage: F
Quasisymmetric functions over the Integer Ring in the
 Fundamental basis
```

(continues on next page)

(continued from previous page)

```
sage: M
Quasisymmetric functions over the Integer Ring in the
 Monomial basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> Q = QuasiSymmetricFunctions(ZZ)
>>> Q.inject_shorthands(verbose=False)
>>> F[Integer(1),Integer(2),Integer(1)] + Integer(5)*M[Integer(1),
↪Integer(3)] + F[Integer(2)]**Integer(2)
5*F[1, 1, 1, 1] - 5*F[1, 1, 2] - 3*F[1, 2, 1] + 6*F[1, 3] +
2*F[2, 2] + F[3, 1] + F[4]
>>> F
Quasisymmetric functions over the Integer Ring in the
 Fundamental basis
>>> M
Quasisymmetric functions over the Integer Ring in the
 Monomial basis
```

One can also just import a subset of the shorthands:

```
sage: # needs sage.combinat sage.modules
sage: SQ = SymmetricFunctions(QQ)
sage: SQ.inject_shorthands(['p', 's'], verbose=False)
sage: p
Symmetric Functions over Rational Field in the powersum basis
sage: s
Symmetric Functions over Rational Field in the Schur basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> SQ = SymmetricFunctions(QQ)
>>> SQ.inject_shorthands(['p', 's'], verbose=False)
>>> p
Symmetric Functions over Rational Field in the powersum basis
>>> s
Symmetric Functions over Rational Field in the Schur basis
```

Note that `e` is left unchanged:

```
sage: e                                                                    #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the elementary basis
```

```
>>> from sage.all import *
>>> e                                                                      #␣
↪needs sage.combinat sage.modules
Symmetric Functions over Integer Ring in the elementary basis
```

realizations()

Return all the realizations of `self` that `self` is aware of.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                    #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.realizations()                                              #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                      #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.realizations()                                                #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the␣
↪Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

Note

Constructing a parent `P` in the category `A.Realizations()` automatically adds `P` to this list by calling `A._register_realization(A)`

example(*base_ring=None*, *set=None*)

Return an example of set with multiple realizations, as per *Category.example()*.

EXAMPLES:

```
sage: Sets().WithRealizations().example()                           #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field

sage: Sets().WithRealizations().example(ZZ, Set([1,2]))             #␣
↪needs sage.modules
The subset algebra of {1, 2} over Integer Ring
```

```
>>> from sage.all import *
>>> Sets().WithRealizations().example()                             #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field

>>> Sets().WithRealizations().example(ZZ, Set([Integer(1),Integer(2)]))  ␣
↪              # needs sage.modules
The subset algebra of {1, 2} over Integer Ring
```

extra_super_categories()

A set with multiple realizations is a facade parent.

EXAMPLES:

```
sage: Sets().WithRealizations().extra_super_categories()
[Category of facade sets]
sage: Sets().WithRealizations().super_categories()
[Category of facade sets]
```

```
>>> from sage.all import *
>>> Sets().WithRealizations().extra_super_categories()
[Category of facade sets]
>>> Sets().WithRealizations().super_categories()
[Category of facade sets]
```

example(*choice=None*)

Return examples of objects of `Sets()`, as per *Category.example()*.

EXAMPLES:

```
sage: Sets().example()
Set of prime numbers (basic implementation)

sage: Sets().example("inherits")
Set of prime numbers

sage: Sets().example("facade")
Set of prime numbers (facade implementation)

sage: Sets().example("wrapper")
Set of prime numbers (wrapper implementation)
```

```
>>> from sage.all import *
>>> Sets().example()
Set of prime numbers (basic implementation)

>>> Sets().example("inherits")
Set of prime numbers

>>> Sets().example("facade")
Set of prime numbers (facade implementation)

>>> Sets().example("wrapper")
Set of prime numbers (wrapper implementation)
```

super_categories()

We include SetsWithPartialMaps between Sets and Objects so that we can define morphisms between sets that are only partially defined. This is also to have the Homset constructor not complain that SetsWithPartialMaps is not a supercategory of Fields, for example.

EXAMPLES:

```
sage: Sets().super_categories()
[Category of sets with partial maps]
```

```
>>> from sage.all import *
>>> Sets().super_categories()
```

(continues on next page)

(continued from previous page)

```
[Category of sets with partial maps]
```

sage.categories.sets_cat.**print_compare**(*x*, *y*)

Helper method used in *Sets.ParentMethods._test_elements_eq_symmetric()*, *Sets.ParentMethods._test_elements_eq_transitive()*.

INPUT:

- `x` – an element
- `y` – an element

EXAMPLES:

```
sage: from sage.categories.sets_cat import print_compare
sage: print_compare(1,2)
1 != 2
sage: print_compare(1,1)
1 == 1
```

```
>>> from sage.all import *
>>> from sage.categories.sets_cat import print_compare
>>> print_compare(Integer(1),Integer(2))
1 != 2
>>> print_compare(Integer(1),Integer(1))
1 == 1
```

4.152 Sets With a Grading

SetsWithGrading.ParentMethods.**_test_graded_components**(***options*)

Test that some graded components of `self` are parent with initialized category and that the parent has a properly implemented `grading()` method.

EXAMPLES:

```
sage: SetsWithGrading().example()._test_graded_components()
```

```
>>> from sage.all import *
>>> SetsWithGrading().example()._test_graded_components()
```

class sage.categories.sets_with_grading.**SetsWithGrading**

Bases: *Category*

The category of sets with a grading.

A *set with a grading* is a set S equipped with a grading by some other set I (by default the set $\mathbf{N}$ of the nonnegative integers):

$$S = \biguplus_{i \in I} S_i$$

where the *graded components* S_i are (usually finite) sets. The *grading* function maps each element s of S to its *grade* i, so that $s \in S_i$.

From implementation point of view, if the graded set is enumerated then each graded component should be enumerated (there is a check in the method *_test_graded_components()*). The contrary needs not be true.

To implement this category, a parent must either implement *graded_component()* or *subset()*. If only *subset()* is implemented, the first argument must be the grading for compatibility with *graded_component()*. Additionally either the parent must implement *grading()* or its elements must implement a method `grade()`. See the example *sage.categories.examples.sets_with_grading.NonNegativeIntegers*.

Finally, if the graded set is enumerated (see *EnumeratedSets*) then each graded component should be enumerated. The contrary needs not be true.

EXAMPLES:

A typical example of a set with a grading is the set of nonnegative integers graded by themselves:

```
sage: N = SetsWithGrading().example(); N
Non negative integers
sage: N.category()
Category of facade infinite sets with grading
sage: N.grading_set()
Non negative integers
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example(); N
Non negative integers
>>> N.category()
Category of facade infinite sets with grading
>>> N.grading_set()
Non negative integers
```

The *grading function* is given by `N.grading`:

```
sage: N.grading(4)
4
```

```
>>> from sage.all import *
>>> N.grading(Integer(4))
4
```

The graded component N_i is the set with one element i:

```
sage: N.graded_component(grade=5)
{5}
sage: N.graded_component(grade=42)
{42}
```

```
>>> from sage.all import *
>>> N.graded_component(grade=Integer(5))
{5}
>>> N.graded_component(grade=Integer(42))
{42}
```

Here are some information about this category:

```
sage: SetsWithGrading()
Category of sets with grading
sage: SetsWithGrading().super_categories()
[Category of sets]
```

(continues on next page)

(continued from previous page)

```
sage: SetsWithGrading().all_super_categories()
[Category of sets with grading,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> SetsWithGrading()
Category of sets with grading
>>> SetsWithGrading().super_categories()
[Category of sets]
>>> SetsWithGrading().all_super_categories()
[Category of sets with grading,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

Todo

- This should be moved to `Sets().WithGrading()`.
- Should the grading set be a parameter for this category?
- Does the enumeration need to be compatible with the grading? Be careful that the fact that graded components are allowed to be finite or infinite make the answer complicated.

class ParentMethods

Bases: `object`

generating_series()

Default implementation for generating series.

OUTPUT: a series, indexed by the grading set

EXAMPLES:

```
   sage: N = SetsWithGrading().example(); N
   Non negative integers
   sage: N.generating_series()
   1/(-z + 1)

   sage: Permutations().generating_series()
↪ # needs sage.combinat
   1 + z + 2*z^2 + 6*z^3 + 24*z^4 + 120*z^5 + 720*z^6 + O(z^7)

.. TODO::

    - Very likely, this should always return a lazy power series.
```

graded_component(*grade*)

Return the graded component of `self` with grade `grade`.

The default implementation just calls the method *`subset()`* with the first argument `grade`.

EXAMPLES:

```
sage: N = SetsWithGrading().example(); N
Non negative integers
sage: N.graded_component(3)
{3}
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example(); N
Non negative integers
>>> N.graded_component(Integer(3))
{3}
```

grading(*elt*)

Return the grading of the element `elt` of `self`.

This default implementation calls `elt.grade()`.

EXAMPLES:

```
sage: N = SetsWithGrading().example(); N
Non negative integers
sage: N.grading(4)
4
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example(); N
Non negative integers
>>> N.grading(Integer(4))
4
```

grading_set()

Return the set `self` is graded by. By default, this is the set of nonnegative integers.

EXAMPLES:

```
sage: SetsWithGrading().example().grading_set()
Non negative integers
```

```
>>> from sage.all import *
>>> SetsWithGrading().example().grading_set()
Non negative integers
```

subset(**args*, ***options*)

Return the subset of `self` described by the given parameters.

See also

-graded_component()

EXAMPLES:

```
sage: W = WeightedIntegerVectors([3,2,1]); W                                #␣
↪needs sage.combinat
Integer vectors weighted by [3, 2, 1]
sage: W.subset(4)                                                           #␣
↪needs sage.combinat
Integer vectors of 4 weighted by [3, 2, 1]
```

```
>>> from sage.all import *
>>> W = WeightedIntegerVectors([Integer(3),Integer(2),Integer(1)]); W    ␣
↪                        # needs sage.combinat
Integer vectors weighted by [3, 2, 1]
>>> W.subset(Integer(4))                                                  ␣
↪     # needs sage.combinat
Integer vectors of 4 weighted by [3, 2, 1]
```

super_categories()

EXAMPLES:

```
sage: SetsWithGrading().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> SetsWithGrading().super_categories()
[Category of sets]
```

4.153 SetsWithPartialMaps

class sage.categories.sets_with_partial_maps.**SetsWithPartialMaps**

Bases: *Category_singleton*

The category whose objects are sets and whose morphisms are maps that are allowed to raise a `ValueError` on some inputs.

This category is equivalent to the category of pointed sets, via the equivalence sending an object X to X union {error}, a morphism f to the morphism of pointed sets that sends x to f(x) if f does not raise an error on x, or to error if it does.

EXAMPLES:

```
sage: SetsWithPartialMaps()
Category of sets with partial maps

sage: SetsWithPartialMaps().super_categories()
[Category of objects]
```

```
>>> from sage.all import *
>>> SetsWithPartialMaps()
Category of sets with partial maps

>>> SetsWithPartialMaps().super_categories()
[Category of objects]
```

super_categories()

EXAMPLES:

```
sage: SetsWithPartialMaps().super_categories()
[Category of objects]
```

```
>>> from sage.all import *
>>> SetsWithPartialMaps().super_categories()
[Category of objects]
```

4.154 Shephard Groups

class sage.categories.shephard_groups.**ShephardGroups**

Bases: *Category_singleton*

The category of Shephard groups.

EXAMPLES:

```
sage: from sage.categories.shephard_groups import ShephardGroups
sage: C = ShephardGroups(); C
Category of shephard groups
```

```
>>> from sage.all import *
>>> from sage.categories.shephard_groups import ShephardGroups
>>> C = ShephardGroups(); C
Category of shephard groups
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.shephard_groups import ShephardGroups
sage: ShephardGroups().super_categories()
[Category of finite generalized Coxeter groups]
```

```
>>> from sage.all import *
>>> from sage.categories.shephard_groups import ShephardGroups
>>> ShephardGroups().super_categories()
[Category of finite generalized Coxeter groups]
```

4.155 Simplicial Complexes

class sage.categories.simplicial_complexes.**SimplicialComplexes**

Bases: *Category_singleton*

The category of abstract simplicial complexes.

An abstract simplicial complex A is a collection of sets X such that:

- $\emptyset \in A$,
- if $X \subset Y \in A$, then $X \in A$.

Todo

Implement the category of simplicial complexes considered as *CW complexes* and rename this to the category of `AbstractSimplicialComplexes` with appropriate functors.

EXAMPLES:

```
sage: from sage.categories.simplicial_complexes import SimplicialComplexes
sage: C = SimplicialComplexes(); C
Category of simplicial complexes
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_complexes import SimplicialComplexes
>>> C = SimplicialComplexes(); C
Category of simplicial complexes
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom*

The category of connected simplicial complexes.

EXAMPLES:

```
sage: from sage.categories.simplicial_complexes import SimplicialComplexes
sage: C = SimplicialComplexes().Connected()
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_complexes import SimplicialComplexes
>>> C = SimplicialComplexes().Connected()
>>> TestSuite(C).run()
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

Category of finite simplicial complexes.

class ParentMethods

Bases: `object`

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: S = SimplicialComplex([[1,3,4], [1,2],[2,5],[4,5]])                    #␣
↪needs sage.graphs
sage: S.dimension()                                                          #␣
↪needs sage.graphs
2
```

```
>>> from sage.all import *
>>> S = SimplicialComplex([[Integer(1),Integer(3),Integer(4)],␣
↪[Integer(1),Integer(2)],[Integer(2),Integer(5)],[Integer(4),
```

(continues on next page)

(continued from previous page)

```
↪Integer(5)]])           # needs sage.graphs
>>> S.dimension()                                                    #␣
↪needs sage.graphs
2
```

class ParentMethods

Bases: object

faces()

Return the faces of self.

EXAMPLES:

```
sage: S = SimplicialComplex([[1,3,4], [1,2],[2,5],[4,5]])                #␣
↪needs sage.graphs
sage: S.faces()                                                          #␣
↪needs sage.graphs
{-1: {()},
 0: {(1,), (2,), (3,), (4,), (5,)},
 1: {(1, 2), (1, 3), (1, 4), (2, 5), (3, 4), (4, 5)},
 2: {(1, 3, 4)}}
```

```
>>> from sage.all import *
>>> S = SimplicialComplex([[Integer(1),Integer(3),Integer(4)],␣
↪[Integer(1),Integer(2)],[Integer(2),Integer(5)],[Integer(4),
↪Integer(5)]])               # needs sage.graphs
>>> S.faces()                                                            #␣
↪needs sage.graphs
{-1: {()},
 0: {(1,), (2,), (3,), (4,), (5,)},
 1: {(1, 2), (1, 3), (1, 4), (2, 5), (3, 4), (4, 5)},
 2: {(1, 3, 4)}}
```

facets()

Return the facets of self.

EXAMPLES:

```
sage: S = SimplicialComplex([[1,3,4], [1,2],[2,5],[4,5]])                #␣
↪needs sage.graphs
sage: sorted(S.facets())                                                 #␣
↪needs sage.graphs
[(1, 2), (1, 3, 4), (2, 5), (4, 5)]
```

```
>>> from sage.all import *
>>> S = SimplicialComplex([[Integer(1),Integer(3),Integer(4)],␣
↪[Integer(1),Integer(2)],[Integer(2),Integer(5)],[Integer(4),
↪Integer(5)]])               # needs sage.graphs
>>> sorted(S.facets())                                                   #␣
↪needs sage.graphs
[(1, 2), (1, 3, 4), (2, 5), (4, 5)]
```

class SubcategoryMethods

Bases: object

Connected()

Return the full subcategory of the connected objects of self.

EXAMPLES:

```
sage: from sage.categories.simplicial_complexes import SimplicialComplexes
sage: SimplicialComplexes().Connected()
Category of connected simplicial complexes
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_complexes import SimplicialComplexes
>>> SimplicialComplexes().Connected()
Category of connected simplicial complexes
```

super_categories()

Return the super categories of self.

EXAMPLES:

```
sage: from sage.categories.simplicial_complexes import SimplicialComplexes
sage: SimplicialComplexes().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_complexes import SimplicialComplexes
>>> SimplicialComplexes().super_categories()
[Category of sets]
```

4.156 Simplicial Sets

class sage.categories.simplicial_sets.**SimplicialSets**

Bases: *Category_singleton*

The category of simplicial sets.

A simplicial set X is a collection of sets X_i, indexed by the nonnegative integers, together with maps

$$d_i : X_n \to X_{n-1}, \quad 0 \le i \le n \quad \text{(face maps)}$$
$$s_j : X_n \to X_{n+1}, \quad 0 \le j \le n \quad \text{(degeneracy maps)}$$

satisfying the *simplicial identities*:

$$d_i d_j = d_{j-1} d_i \quad \text{if } i < j$$
$$d_i s_j = s_{j-1} d_i \quad \text{if } i < j$$
$$d_j s_j = 1 = d_{j+1} s_j$$
$$d_i s_j = s_j d_{i-1} \quad \text{if } i > j+1$$
$$s_i s_j = s_{j+1} s_i \quad \text{if } i \le j$$

Morphisms are sequences of maps $f_i : X_i \to Y_i$ which commute with the face and degeneracy maps.

EXAMPLES:

```
sage: from sage.categories.simplicial_sets import SimplicialSets
sage: C = SimplicialSets(); C
Category of simplicial sets
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_sets import SimplicialSets
>>> C = SimplicialSets(); C
Category of simplicial sets
```

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

Category of finite simplicial sets.

The objects are simplicial sets with finitely many non-degenerate simplices.

class Homsets(*category*, **args*)

Bases: *HomsetsCategory*

class Endset(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: object

one()

Return the identity morphism in $\mathrm{Hom}(S, S)$.

EXAMPLES:

```
sage: T = simplicial_sets.Torus()                                    #␣
↪needs sage.graphs
sage: Hom(T, T).identity()                                           #␣
↪needs sage.graphs
Simplicial set endomorphism of Torus
  Defn: Identity map
```

```
>>> from sage.all import *
>>> T = simplicial_sets.Torus()                                      #␣
↪needs sage.graphs
>>> Hom(T, T).identity()                                             #␣
↪needs sage.graphs
Simplicial set endomorphism of Torus
  Defn: Identity map
```

class ParentMethods

Bases: object

is_finite()

Return `True` if this simplicial set is finite, i.e., has a finite number of nondegenerate simplices.

EXAMPLES:

```
sage: simplicial_sets.Torus().is_finite()                            #␣
↪needs sage.graphs
```

(continues on next page)

(continued from previous page)

```
True
sage: C5 = groups.misc.MultiplicativeAbelian([5])                          #␣
↪needs sage.graphs sage.groups
sage: simplicial_sets.ClassifyingSpace(C5).is_finite()                     #␣
↪needs sage.graphs sage.groups
False
```

```
>>> from sage.all import *
>>> simplicial_sets.Torus().is_finite()                                    #␣
↪needs sage.graphs
True
>>> C5 = groups.misc.MultiplicativeAbelian([Integer(5)])                    ␣
↪     # needs sage.graphs sage.groups
>>> simplicial_sets.ClassifyingSpace(C5).is_finite()                       #␣
↪needs sage.graphs sage.groups
False
```

is_pointed()

Return `True` if this simplicial set is pointed, i.e., has a base point.

EXAMPLES:

```
sage: # needs sage.graphs
sage: from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
sage: v = AbstractSimplex(0)
sage: w = AbstractSimplex(0)
sage: e = AbstractSimplex(1)
sage: X = SimplicialSet({e: (v, w)})
sage: Y = SimplicialSet({e: (v, w)}, base_point=w)
sage: X.is_pointed()
False
sage: Y.is_pointed()
True
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
>>> v = AbstractSimplex(Integer(0))
>>> w = AbstractSimplex(Integer(0))
>>> e = AbstractSimplex(Integer(1))
>>> X = SimplicialSet({e: (v, w)})
>>> Y = SimplicialSet({e: (v, w)}, base_point=w)
>>> X.is_pointed()
False
>>> Y.is_pointed()
True
```

set_base_point(*point*)

Return a copy of this simplicial set in which the base point is set to `point`.

INPUT:

- `point` – a 0-simplex in this simplicial set

EXAMPLES:

```
sage: # needs sage.graphs
sage: from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
sage: v = AbstractSimplex(0, name='v_0')
sage: w = AbstractSimplex(0, name='w_0')
sage: e = AbstractSimplex(1)
sage: X = SimplicialSet({e: (v, w)})
sage: Y = SimplicialSet({e: (v, w)}, base_point=w)
sage: Y.base_point()
w_0
sage: X_star = X.set_base_point(w)
sage: X_star.base_point()
w_0
sage: Y_star = Y.set_base_point(v)
sage: Y_star.base_point()
v_0
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
>>> v = AbstractSimplex(Integer(0), name='v_0')
>>> w = AbstractSimplex(Integer(0), name='w_0')
>>> e = AbstractSimplex(Integer(1))
>>> X = SimplicialSet({e: (v, w)})
>>> Y = SimplicialSet({e: (v, w)}, base_point=w)
>>> Y.base_point()
w_0
>>> X_star = X.set_base_point(w)
>>> X_star.base_point()
w_0
>>> Y_star = Y.set_base_point(v)
>>> Y_star.base_point()
v_0
```

class Pointed(*base_category*)

Bases: *CategoryWithAxiom*

class Finite(*base_category*)

Bases: *CategoryWithAxiom*

class ParentMethods

Bases: `object`

fat_wedge(*n*)

Return the n-th fat wedge of this pointed simplicial set.

This is the subcomplex of the n-fold product X^n consisting of those points in which at least one factor is the base point. Thus when $n = 2$, this is the wedge of the simplicial set with itself, but when n is larger, the fat wedge is larger than the n-fold wedge.

EXAMPLES:

```
sage: # needs sage.graphs
sage: S1 = simplicial_sets.Sphere(1)
sage: S1.fat_wedge(0)
Point
sage: S1.fat_wedge(1)
S^1
sage: S1.fat_wedge(2).fundamental_group()                                   #␣
↪needs sage.groups
Finitely presented group < e0, e1 |  >
sage: S1.fat_wedge(4).homology()                                            #␣
↪needs sage.modules
{0: 0, 1: Z x Z x Z x Z, 2: Z^6, 3: Z x Z x Z x Z}
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> S1 = simplicial_sets.Sphere(Integer(1))
>>> S1.fat_wedge(Integer(0))
Point
>>> S1.fat_wedge(Integer(1))
S^1
>>> S1.fat_wedge(Integer(2)).fundamental_group()                                ␣
↪  # needs sage.groups
Finitely presented group < e0, e1 |  >
>>> S1.fat_wedge(Integer(4)).homology()                                         ␣
↪  # needs sage.modules
{0: 0, 1: Z x Z x Z x Z, 2: Z^6, 3: Z x Z x Z x Z}
```

smash_product(**others*)

Return the smash product of this simplicial set with `others`.

INPUT:

- `others` – one or several simplicial sets

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: S1 = simplicial_sets.Sphere(1)
sage: RP2 = simplicial_sets.RealProjectiveSpace(2)
sage: X = S1.smash_product(RP2)
sage: X.homology(base_ring=GF(2))                                           #␣
↪needs sage.modules
{0: Vector space of dimension 0 over Finite Field of size 2,
 1: Vector space of dimension 0 over Finite Field of size 2,
 2: Vector space of dimension 1 over Finite Field of size 2,
 3: Vector space of dimension 1 over Finite Field of size 2}

sage: T = S1.product(S1)                                                    #␣
↪needs sage.graphs sage.groups
sage: X = T.smash_product(S1)                                               #␣
↪needs sage.graphs sage.groups
sage: X.homology(reduced=False)                                             #␣
↪needs sage.graphs sage.groups sage.modules
{0: Z, 1: 0, 2: Z x Z, 3: Z}
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> S1 = simplicial_sets.Sphere(Integer(1))
>>> RP2 = simplicial_sets.RealProjectiveSpace(Integer(2))
>>> X = S1.smash_product(RP2)
>>> X.homology(base_ring=GF(Integer(2)))
↪  # needs sage.modules
{0: Vector space of dimension 0 over Finite Field of size 2,
 1: Vector space of dimension 0 over Finite Field of size 2,
 2: Vector space of dimension 1 over Finite Field of size 2,
 3: Vector space of dimension 1 over Finite Field of size 2}

>>> T = S1.product(S1)                                          #␣
↪needs sage.graphs sage.groups
>>> X = T.smash_product(S1)                                     #␣
↪needs sage.graphs sage.groups
>>> X.homology(reduced=False)                                   #␣
↪needs sage.graphs sage.groups sage.modules
{0: Z, 1: 0, 2: Z x Z, 3: Z}
```

unset_base_point()

Return a copy of this simplicial set in which the base point has been forgotten.

EXAMPLES:

```
sage: # needs sage.graphs
sage: from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
sage: v = AbstractSimplex(0, name='v_0')
sage: w = AbstractSimplex(0, name='w_0')
sage: e = AbstractSimplex(1)
sage: Y = SimplicialSet({e: (v, w)}, base_point=w)
sage: Y.is_pointed()
True
sage: Y.base_point()
w_0
sage: Z = Y.unset_base_point()
sage: Z.is_pointed()
False
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
>>> v = AbstractSimplex(Integer(0), name='v_0')
>>> w = AbstractSimplex(Integer(0), name='w_0')
>>> e = AbstractSimplex(Integer(1))
>>> Y = SimplicialSet({e: (v, w)}, base_point=w)
>>> Y.is_pointed()
True
>>> Y.base_point()
w_0
>>> Z = Y.unset_base_point()
```

(continues on next page)

(continued from previous page)

```
>>> Z.is_pointed()
False
```

class ParentMethods

Bases: object

base_point()

Return this simplicial set's base point.

EXAMPLES:

```
sage: # needs sage.graphs
sage: from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
sage: v = AbstractSimplex(0, name='*')
sage: e = AbstractSimplex(1)
sage: S1 = SimplicialSet({e: (v, v)}, base_point=v)
sage: S1.is_pointed()
True
sage: S1.base_point()
*
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> from sage.topology.simplicial_set import AbstractSimplex,␣
↪SimplicialSet
>>> v = AbstractSimplex(Integer(0), name='*')
>>> e = AbstractSimplex(Integer(1))
>>> S1 = SimplicialSet({e: (v, v)}, base_point=v)
>>> S1.is_pointed()
True
>>> S1.base_point()
*
```

base_point_map(*domain=None*)

Return a map from a one-point space to this one, with image the base point.

This raises an error if this simplicial set does not have a base point.

INPUT:

- `domain` – (default: `None`) use this to specify a particular one-point space as the domain. The default behavior is to use the `sage.topology.simplicial_set_examples.Point()` function to use a standard one-point space.

EXAMPLES:

```
sage: # needs sage.graphs
sage: T = simplicial_sets.Torus()
sage: f = T.base_point_map(); f
Simplicial set morphism:
  From: Point
  To:   Torus
  Defn: Constant map at (v_0, v_0)
sage: S3 = simplicial_sets.Sphere(3)
```

(continues on next page)

(continued from previous page)

```
sage: g = S3.base_point_map()
sage: f.domain() == g.domain()
True
sage: RP3 = simplicial_sets.RealProjectiveSpace(3)                    #␣
↪needs sage.groups
sage: temp = simplicial_sets.Simplex(0)
sage: pt = temp.set_base_point(temp.n_cells(0)[0])
sage: h = RP3.base_point_map(domain=pt)                                #␣
↪needs sage.groups
sage: f.domain() == h.domain()                                         #␣
↪needs sage.groups
False

sage: C5 = groups.misc.MultiplicativeAbelian([5])                      #␣
↪needs sage.graphs sage.groups
sage: BC5 = simplicial_sets.ClassifyingSpace(C5)                       #␣
↪needs sage.graphs sage.groups
sage: BC5.base_point_map()                                             #␣
↪needs sage.graphs sage.groups
Simplicial set morphism:
  From: Point
  To:   Classifying space of Multiplicative Abelian group isomorphic␣
↪to C5
  Defn: Constant map at 1
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> T = simplicial_sets.Torus()
>>> f = T.base_point_map(); f
Simplicial set morphism:
  From: Point
  To:   Torus
  Defn: Constant map at (v_0, v_0)
>>> S3 = simplicial_sets.Sphere(Integer(3))
>>> g = S3.base_point_map()
>>> f.domain() == g.domain()
True
>>> RP3 = simplicial_sets.RealProjectiveSpace(Integer(3))              ␣
↪    # needs sage.groups
>>> temp = simplicial_sets.Simplex(Integer(0))
>>> pt = temp.set_base_point(temp.n_cells(Integer(0))[Integer(0)])
>>> h = RP3.base_point_map(domain=pt)                                  #␣
↪needs sage.groups
>>> f.domain() == h.domain()                                           #␣
↪needs sage.groups
False

>>> C5 = groups.misc.MultiplicativeAbelian([Integer(5)])               ␣
↪    # needs sage.graphs sage.groups
>>> BC5 = simplicial_sets.ClassifyingSpace(C5)                         #␣
↪needs sage.graphs sage.groups
>>> BC5.base_point_map()                                               #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.graphs sage.groups
Simplicial set morphism:
  From: Point
  To:   Classifying space of Multiplicative Abelian group isomorphic␣
↪to C5
  Defn: Constant map at 1
```

connectivity(*max_dim=None*)

Return the connectivity of this pointed simplicial set.

INPUT:

- `max_dim` – specify a maximum dimension through which to check. This is required if this simplicial set is simply connected and not finite.

The dimension of the first nonzero homotopy group. If simply connected, this is the same as the dimension of the first nonzero homology group.

Warning

See the warning for the *is_simply_connected()* method.

The connectivity of a contractible space is `+Infinity`.

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: simplicial_sets.Sphere(3).connectivity()
2
sage: simplicial_sets.Sphere(0).connectivity()
-1
sage: K = simplicial_sets.Simplex(4)
sage: K = K.set_base_point(K.n_cells(0)[0])
sage: K.connectivity()
+Infinity
sage: X = simplicial_sets.Torus().suspension(2)
sage: X.connectivity()
2

sage: C2 = groups.misc.MultiplicativeAbelian([2])                    #␣
↪needs sage.graphs sage.groups
sage: BC2 = simplicial_sets.ClassifyingSpace(C2)                     #␣
↪needs sage.graphs sage.groups
sage: BC2.connectivity()                                             #␣
↪needs sage.graphs sage.groups
0
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> simplicial_sets.Sphere(Integer(3)).connectivity()
2
>>> simplicial_sets.Sphere(Integer(0)).connectivity()
-1
>>> K = simplicial_sets.Simplex(Integer(4))
```

(continues on next page)

(continued from previous page)

```
>>> K = K.set_base_point(K.n_cells(Integer(0))[Integer(0)])
>>> K.connectivity()
+Infinity
>>> X = simplicial_sets.Torus().suspension(Integer(2))
>>> X.connectivity()
2

>>> C2 = groups.misc.MultiplicativeAbelian([Integer(2)])                ␣
↪    # needs sage.graphs sage.groups
>>> BC2 = simplicial_sets.ClassifyingSpace(C2)                          #␣
↪needs sage.graphs sage.groups
>>> BC2.connectivity()                                                  #␣
↪needs sage.graphs sage.groups
0
```

cover(*character*)

Return the cover of the simplicial set associated to a character of the fundamental group.

The character is represented by a dictionary, that assigns an element of a finite group to each non-degenerate 1-dimensional cell. It should correspond to an epimorphism from the fundamental group.

INPUT:

- `character` – dictionary

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: S1 = simplicial_sets.Sphere(1)
sage: S1_ = simplicial_sets.Sphere(1)
sage: S1_.n_cells(1)[0].rename("sigma_1'")
sage: W = S1.wedge(S1_)
sage: G = CyclicPermutationGroup(3)
sage: a, b = W.n_cells(1)
sage: C = W.cover({a : G.gen(0), b : G.gen(0)^2})
sage: C.face_data()
{(*, ()): None,
 (*, (1,2,3)): None,
 (*, (1,3,2)): None,
 (sigma_1', ()): ((*, (1,3,2)), (*, ())),
 (sigma_1', (1,2,3)): ((*, ()), (*, (1,2,3))),
 (sigma_1', (1,3,2)): ((*, (1,2,3)), (*, (1,3,2))),
 (sigma_1, ()): ((*, (1,2,3)), (*, ())),
 (sigma_1, (1,2,3)): ((*, (1,3,2)), (*, (1,2,3))),
 (sigma_1, (1,3,2)): ((*, ()), (*, (1,3,2)))}
sage: C.homology(1)                                                    #␣
↪needs sage.modules
Z x Z x Z x Z
sage: C.fundamental_group()
Finitely presented group < e0, e1, e2, e3 |  >
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> S1 = simplicial_sets.Sphere(Integer(1))
>>> S1_ = simplicial_sets.Sphere(Integer(1))
```

(continues on next page)

(continued from previous page)

```
>>> S1_.n_cells(Integer(1))[Integer(0)].rename("sigma_1'")
>>> W = S1.wedge(S1_)
>>> G = CyclicPermutationGroup(Integer(3))
>>> a, b = W.n_cells(Integer(1))
>>> C = W.cover({a : G.gen(Integer(0)), b : G.
↪gen(Integer(0))**Integer(2)})
>>> C.face_data()
{(*, ()): None,
 (*, (1,2,3)): None,
 (*, (1,3,2)): None,
 (sigma_1', ()): ((*, (1,3,2)), (*, ())),
 (sigma_1', (1,2,3)): ((*, ()), (*, (1,2,3))),
 (sigma_1', (1,3,2)): ((*, (1,2,3)), (*, (1,3,2))),
 (sigma_1, ()): ((*, (1,2,3)), (*, ())),
 (sigma_1, (1,2,3)): ((*, (1,3,2)), (*, (1,2,3))),
 (sigma_1, (1,3,2)): ((*, ()), (*, (1,3,2)))}
>>> C.homology(Integer(1))
↪    # needs sage.modules
Z x Z x Z x Z
>>> C.fundamental_group()
Finitely presented group < e0, e1, e2, e3 |  >
```

covering_map(*character*)

Return the covering map associated to a character.

The character is represented by a dictionary that assigns an element of a finite group to each non-degenerate 1-dimensional cell. It should correspond to an epimorphism from the fundamental group.

INPUT:

- `character` – dictionary

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: S1 = simplicial_sets.Sphere(1)
sage: S1_ = simplicial_sets.Sphere(1)
sage: S1_.n_cells(1)[0].rename("sigma_1'")
sage: W = S1.wedge(S1_)
sage: G = CyclicPermutationGroup(3)
sage: a, b = W.n_cells(1)
sage: C = W.covering_map({a : G.gen(0), b : G.one()}); C
Simplicial set morphism:
  From: Simplicial set with 9 non-degenerate simplices
  To:   Wedge: (S^1 v S^1)
  Defn: [(*, ()), (*, (1,2,3)), (*, (1,3,2)), (sigma_1', ()),
         (sigma_1', (1,2,3)), (sigma_1', (1,3,2)), (sigma_1, ()),
         (sigma_1, (1,2,3)), (sigma_1, (1,3,2))]
        --> [*, *, *, sigma_1', sigma_1', sigma_1', sigma_1, sigma_1,
↪sigma_1]
sage: C.domain()
Simplicial set with 9 non-degenerate simplices
sage: C.domain().face_data()
{(*, ()): None,
 (*, (1,2,3)): None,
```

(continues on next page)

(continued from previous page)

```
 (*, (1,3,2)): None,
 (sigma_1', ()): ((*, ()), (*, ())),
 (sigma_1', (1,2,3)): ((*, (1,2,3)), (*, (1,2,3))),
 (sigma_1', (1,3,2)): ((*, (1,3,2)), (*, (1,3,2))),
 (sigma_1, ()): ((*, (1,2,3)), (*, ())),
 (sigma_1, (1,2,3)): ((*, (1,3,2)), (*, (1,2,3))),
 (sigma_1, (1,3,2)): ((*, ()), (*, (1,3,2)))}
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> S1 = simplicial_sets.Sphere(Integer(1))
>>> S1_ = simplicial_sets.Sphere(Integer(1))
>>> S1_.n_cells(Integer(1))[Integer(0)].rename("sigma_1'")
>>> W = S1.wedge(S1_)
>>> G = CyclicPermutationGroup(Integer(3))
>>> a, b = W.n_cells(Integer(1))
>>> C = W.covering_map({a : G.gen(Integer(0)), b : G.one()}); C
Simplicial set morphism:
  From: Simplicial set with 9 non-degenerate simplices
  To:   Wedge: (S^1 v S^1)
  Defn: [(*, ()), (*, (1,2,3)), (*, (1,3,2)), (sigma_1', ()),
         (sigma_1', (1,2,3)), (sigma_1', (1,3,2)), (sigma_1, ()),
         (sigma_1, (1,2,3)), (sigma_1, (1,3,2))]
        --> [*, *, *, sigma_1', sigma_1', sigma_1', sigma_1, sigma_1,␣
↪sigma_1]
>>> C.domain()
Simplicial set with 9 non-degenerate simplices
>>> C.domain().face_data()
{(*, ()): None,
 (*, (1,2,3)): None,
 (*, (1,3,2)): None,
 (sigma_1', ()): ((*, ()), (*, ())),
 (sigma_1', (1,2,3)): ((*, (1,2,3)), (*, (1,2,3))),
 (sigma_1', (1,3,2)): ((*, (1,3,2)), (*, (1,3,2))),
 (sigma_1, ()): ((*, (1,2,3)), (*, ())),
 (sigma_1, (1,2,3)): ((*, (1,3,2)), (*, (1,2,3))),
 (sigma_1, (1,3,2)): ((*, ()), (*, (1,3,2)))}
```

fundamental_group(*simplify=True*)

Return the fundamental group of this pointed simplicial set.

INPUT:

- `simplify` – boolean (default: `True`); if `False`, then return a presentation of the group in terms of generators and relations. If `True`, the default, simplify as much as GAP is able to.

Algorithm: we compute the edge-path group – see Section 19 of [Kan1958] and Wikipedia article Fundamental_group. Choose a spanning tree for the connected component of the 1-skeleton containing the base point, and then the group's generators are given by the non-degenerate edges. There are two types of relations: $e = 1$ if e is in the spanning tree, and for every 2-simplex, if its faces are e_0, e_1, and e_2, then we impose the relation $e_0 e_1^{-1} e_2 = 1$, where we first set $e_i = 1$ if e_i is degenerate.

EXAMPLES:

```
sage: S1 = simplicial_sets.Sphere(1)                                    #␣
↪needs sage.graphs
sage: eight = S1.wedge(S1)                                              #␣
↪needs sage.graphs
sage: eight.fundamental_group()  # free group on 2 generators           #␣
↪needs sage.graphs sage.groups
Finitely presented group < e0, e1 |  >
```

```
>>> from sage.all import *
>>> S1 = simplicial_sets.Sphere(Integer(1))                                 ␣
↪    # needs sage.graphs
>>> eight = S1.wedge(S1)                                                #␣
↪needs sage.graphs
>>> eight.fundamental_group()  # free group on 2 generators           #␣
↪needs sage.graphs sage.groups
Finitely presented group < e0, e1 |  >
```

The fundamental group of a disjoint union of course depends on the choice of base point:

```
sage: T = simplicial_sets.Torus()                                       #␣
↪needs sage.graphs
sage: K = simplicial_sets.KleinBottle()                                 #␣
↪needs sage.graphs
sage: X = T.disjoint_union(K)                                           #␣
↪needs sage.graphs

sage: # needs sage.graphs
sage: X_0 = X.set_base_point(X.n_cells(0)[0])
sage: X_0.fundamental_group().is_abelian()                              #␣
↪needs sage.groups
True
sage: X_1 = X.set_base_point(X.n_cells(0)[1])
sage: X_1.fundamental_group().is_abelian()                              #␣
↪needs sage.groups
False

sage: RP3 = simplicial_sets.RealProjectiveSpace(3)                      #␣
↪needs sage.graphs sage.groups
sage: RP3.fundamental_group()                                           #␣
↪needs sage.graphs sage.groups
Finitely presented group < e | e^2 >
```

```
>>> from sage.all import *
>>> T = simplicial_sets.Torus()                                        #␣
↪needs sage.graphs
>>> K = simplicial_sets.KleinBottle()                                  #␣
↪needs sage.graphs
>>> X = T.disjoint_union(K)                                            #␣
↪needs sage.graphs

>>> # needs sage.graphs
>>> X_0 = X.set_base_point(X.n_cells(Integer(0))[Integer(0)])
```

(continues on next page)

(continued from previous page)

```
>>> X_0.fundamental_group().is_abelian()                                #␣
↪needs sage.groups
True
>>> X_1 = X.set_base_point(X.n_cells(Integer(0))[Integer(1)])
>>> X_1.fundamental_group().is_abelian()                                #␣
↪needs sage.groups
False

>>> RP3 = simplicial_sets.RealProjectiveSpace(Integer(3))              ␣
↪    # needs sage.graphs sage.groups
>>> RP3.fundamental_group()                                            #␣
↪needs sage.graphs sage.groups
Finitely presented group < e | e^2 >
```

Compute the fundamental group of some classifying spaces:

```
sage: C5 = groups.misc.MultiplicativeAbelian([5])                      #␣
↪needs sage.graphs sage.groups
sage: BC5 = C5.nerve()                                                 #␣
↪needs sage.graphs sage.groups
sage: BC5.fundamental_group()                                          #␣
↪needs sage.graphs sage.groups
Finitely presented group < e0 | e0^5 >

sage: # needs sage.graphs sage.groups
sage: Sigma3 = groups.permutation.Symmetric(3)
sage: BSigma3 = Sigma3.nerve()
sage: pi = BSigma3.fundamental_group(); pi
Finitely presented group < e1, e2 | e2^2, e1^3, (e2*e1)^2 >
sage: pi.order()
6
sage: pi.is_abelian()
False
```

```
>>> from sage.all import *
>>> C5 = groups.misc.MultiplicativeAbelian([Integer(5)])              ␣
↪    # needs sage.graphs sage.groups
>>> BC5 = C5.nerve()                                                  #␣
↪needs sage.graphs sage.groups
>>> BC5.fundamental_group()                                           #␣
↪needs sage.graphs sage.groups
Finitely presented group < e0 | e0^5 >

>>> # needs sage.graphs sage.groups
>>> Sigma3 = groups.permutation.Symmetric(Integer(3))
>>> BSigma3 = Sigma3.nerve()
>>> pi = BSigma3.fundamental_group(); pi
Finitely presented group < e1, e2 | e2^2, e1^3, (e2*e1)^2 >
>>> pi.order()
6
>>> pi.is_abelian()
False
```

The sphere has a trivial fundamental group:

```
sage: S2 = simplicial_sets.Sphere(2)                                    #␣
↪needs sage.graphs
sage: S2.fundamental_group()                                            #␣
↪needs sage.graphs sage.groups
Finitely presented group <  |  >
```

```
>>> from sage.all import *
>>> S2 = simplicial_sets.Sphere(Integer(2))                                ␣
↪    # needs sage.graphs
>>> S2.fundamental_group()                                              #␣
↪needs sage.graphs sage.groups
Finitely presented group <  |  >
```

is_simply_connected()

Return `True` if this pointed simplicial set is simply connected.

Warning

Determining simple connectivity is not always possible, because it requires determining when a group, as given by generators and relations, is trivial. So this conceivably may give a false negative in some cases.

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: T = simplicial_sets.Torus()
sage: T.is_simply_connected()
False
sage: T.suspension().is_simply_connected()
True
sage: simplicial_sets.KleinBottle().is_simply_connected()
False

sage: # needs sage.graphs
sage: S2 = simplicial_sets.Sphere(2)
sage: S3 = simplicial_sets.Sphere(3)
sage: (S2.wedge(S3)).is_simply_connected()                              #␣
↪needs sage.groups
True
sage: X = S2.disjoint_union(S3)
sage: X = X.set_base_point(X.n_cells(0)[0])
sage: X.is_simply_connected()
False

sage: C3 = groups.misc.MultiplicativeAbelian([3])                       #␣
↪needs sage.graphs sage.groups
sage: BC3 = simplicial_sets.ClassifyingSpace(C3)                        #␣
↪needs sage.graphs sage.groups
sage: BC3.is_simply_connected()                                         #␣
↪needs sage.graphs sage.groups
```

(continues on next page)

(continued from previous page)

```
False
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> T = simplicial_sets.Torus()
>>> T.is_simply_connected()
False
>>> T.suspension().is_simply_connected()
True
>>> simplicial_sets.KleinBottle().is_simply_connected()
False

>>> # needs sage.graphs
>>> S2 = simplicial_sets.Sphere(Integer(2))
>>> S3 = simplicial_sets.Sphere(Integer(3))
>>> (S2.wedge(S3)).is_simply_connected()                                #␣
↪needs sage.groups
True
>>> X = S2.disjoint_union(S3)
>>> X = X.set_base_point(X.n_cells(Integer(0))[Integer(0)])
>>> X.is_simply_connected()
False

>>> C3 = groups.misc.MultiplicativeAbelian([Integer(3)])                ␣
↪    # needs sage.graphs sage.groups
>>> BC3 = simplicial_sets.ClassifyingSpace(C3)                          #␣
↪needs sage.graphs sage.groups
>>> BC3.is_simply_connected()                                           #␣
↪needs sage.graphs sage.groups
False
```

twisted_chain_complex(*twisting_operator=None*, *dimensions=None*, *augmented=False*, *cochain=False*, *verbose=False*, *subcomplex=None*, *check=False*)

Return the normalized chain complex twisted by some operator.

A twisting operator is a map from the set of simplices to some algebra. The differentials are then twisted by this operator.

INPUT:

- `twisting_operator` – dictionary, associating the twist of each simplex. If it is not given, the canonical one (associated to the laurent polynomial ring abelianization of the fundamental group, ignoring torsion) is used.
- `dimensions` – if `None`, compute the chain complex in all dimensions. If a list or tuple of integers, compute the chain complex in those dimensions, setting the chain groups in all other dimensions to zero.
- `augmented` – boolean (default: `False`); if `True`, return the augmented chain complex (that is, include a class in dimension -1 corresponding to the empty cell).
- `cochain` – boolean (default: `False`); if `True`, return the cochain complex (that is, the dual of the chain complex).
- `verbose` – boolean (default: `False`); ignored
- `subcomplex` – (default: `None`) if present, compute the chain complex relative to this subcomplex
- `check` – boolean (default: `False`); if `True`, make sure that the chain complex is actually a chain complex: the differentials are composable and their product is zero.

The normalized chain complex of a simplicial set is isomorphic to the chain complex obtained by modding out by degenerate simplices, and the latter is what is actually constructed here.

EXAMPLES:

```
sage: # needs sage.graphs
sage: W = simplicial_sets.Sphere(1).wedge(simplicial_sets.Sphere(2))
sage: W.nondegenerate_simplices()
[*, sigma_1, sigma_2]
sage: s1 = W.nondegenerate_simplices()[1]
sage: L.<t> = LaurentPolynomialRing(QQ)
sage: tw = {s1:t}
sage: ChC = W.twisted_chain_complex(tw)
sage: ChC.differential(1)
[-1 + t]
sage: ChC.differential(2)
[0]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> W = simplicial_sets.Sphere(Integer(1)).wedge(simplicial_sets.
↪Sphere(Integer(2)))
>>> W.nondegenerate_simplices()
[*, sigma_1, sigma_2]
>>> s1 = W.nondegenerate_simplices()[Integer(1)]
>>> L = LaurentPolynomialRing(QQ, names=('t',)); (t,) = L._first_
↪ngens(1)
>>> tw = {s1:t}
>>> ChC = W.twisted_chain_complex(tw)
>>> ChC.differential(Integer(1))
[-1 + t]
>>> ChC.differential(Integer(2))
[0]
```

```
sage: # needs sage.graphs
sage: X = simplicial_sets.Torus()
sage: C = X.twisted_chain_complex()
sage: C.differential(1)
[      f2 - 1 f1*f2^-1 - 1       f1 - 1]
sage: C.differential(2)
[       1 f1*f2^-1]
[      f2        1]
[      -1       -1]
sage: C.differential(3)
[]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> X = simplicial_sets.Torus()
>>> C = X.twisted_chain_complex()
>>> C.differential(Integer(1))
[      f2 - 1 f1*f2^-1 - 1       f1 - 1]
>>> C.differential(Integer(2))
```

(continues on next page)

(continued from previous page)

```
[       1 f1*f2^-1]
[      f2        1]
[      -1       -1]
>>> C.differential(Integer(3))
[]
```

```
sage: # needs sage.graphs
sage: Y = simplicial_sets.RealProjectiveSpace(2)
sage: C = Y.twisted_chain_complex()
sage: C.differential(1)
[-1 + F1]
sage: C.differential(2)
[1 + F1]
sage: C.differential(3)
[]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> Y = simplicial_sets.RealProjectiveSpace(Integer(2))
>>> C = Y.twisted_chain_complex()
>>> C.differential(Integer(1))
[-1 + F1]
>>> C.differential(Integer(2))
[1 + F1]
>>> C.differential(Integer(3))
[]
```

twisted_homology(*n*, *reduced=False*)

The n-th twisted homology module of the simplicial set with respect to the abelianization of the fundamental_group.

It is a module over a polynomial ring, including relations to make some variables the multiplicative inverses of others.

INPUT:

- `n` – positive integer
- `reduced` – boolean (default: `False`); if set to True, the presentation matrix will be reduced

EXAMPLES:

```
sage: # needs sage.graphs
sage: X = simplicial_sets.Sphere(1).wedge(simplicial_sets.Sphere(2))
sage: X.twisted_homology(1)
Quotient module by Submodule of Ambient free module of rank 0 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv over␣
↪Integer Ring
Generated by the rows of the matrix:
[]
sage: X.twisted_homology(2)
Quotient module by Submodule of Ambient free module of rank 1 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv over␣
↪Integer Ring
Generated by the rows of the matrix:
[f1*f1inv - 1]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> X = simplicial_sets.Sphere(Integer(1)).wedge(simplicial_sets.
↪Sphere(Integer(2)))
>>> X.twisted_homology(Integer(1))
Quotient module by Submodule of Ambient free module of rank 0 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv over␣
↪Integer Ring
Generated by the rows of the matrix:
[]
>>> X.twisted_homology(Integer(2))
Quotient module by Submodule of Ambient free module of rank 1 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv over␣
↪Integer Ring
Generated by the rows of the matrix:
[f1*f1inv - 1]
```

```
sage: # needs sage.graphs
sage: Y = simplicial_sets.Torus()
sage: Y.twisted_homology(1)
Quotient module by Submodule of Ambient free module of rank 5 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[           1            0            0            0            0]
[           0            1            0            0            0]
[           0            0            1            0            0]
[           0            0            0            1            0]
[           0            0            0            0            1]
[f1*f1inv - 1            0            0            0            0]
[           0 f1*f1inv - 1            0            0            0]
[           0            0 f1*f1inv - 1            0            0]
[           0            0            0 f1*f1inv - 1            0]
[           0            0            0            0 f1*f1inv - 1]
[f2*f2inv - 1            0            0            0            0]
[           0 f2*f2inv - 1            0            0            0]
[           0            0 f2*f2inv - 1            0            0]
[           0            0            0 f2*f2inv - 1            0]
[           0            0            0            0 f2*f2inv - 1]
sage: Y.twisted_homology(2)
Quotient module by Submodule of Ambient free module of rank 0 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[]
sage: Y.twisted_homology(1, reduced=True)
Quotient module by Submodule of Ambient free module of rank 5 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[1 0 0 0 0]
[0 1 0 0 0]
```

(continues on next page)

(continued from previous page)

```
[0 0 1 0 0]
[0 0 0 1 0]
[0 0 0 0 1]
```

```
>>> from sage.all import *
>>> # needs sage.graphs
>>> Y = simplicial_sets.Torus()
>>> Y.twisted_homology(Integer(1))
Quotient module by Submodule of Ambient free module of rank 5 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[           1            0            0            0            0]
[           0            1            0            0            0]
[           0            0            1            0            0]
[           0            0            0            1            0]
[           0            0            0            0            1]
[f1*f1inv - 1            0            0            0            0]
[           0 f1*f1inv - 1            0            0            0]
[           0            0 f1*f1inv - 1            0            0]
[           0            0            0 f1*f1inv - 1            0]
[           0            0            0            0 f1*f1inv - 1]
[f2*f2inv - 1            0            0            0            0]
[           0 f2*f2inv - 1            0            0            0]
[           0            0 f2*f2inv - 1            0            0]
[           0            0            0 f2*f2inv - 1            0]
[           0            0            0            0 f2*f2inv - 1]
>>> Y.twisted_homology(Integer(2))
Quotient module by Submodule of Ambient free module of rank 0 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[]
>>> Y.twisted_homology(Integer(1), reduced=True)
Quotient module by Submodule of Ambient free module of rank 5 over the␣
↪integral domain Multivariate Polynomial Ring in f1, f1inv, f2, f2inv␣
↪over Integer Ring
Generated by the rows of the matrix:
[1 0 0 0 0]
[0 1 0 0 0]
[0 0 1 0 0]
[0 0 0 1 0]
[0 0 0 0 1]
```

universal_cover()

Return the universal cover of the simplicial set. The fundamental group must be finite in order to ensure that the universal cover is a simplicial set of finite type.

EXAMPLES:

```
sage: # needs sage.graphs sage.groups
sage: RP3 = simplicial_sets.RealProjectiveSpace(3)
```

(continues on next page)

(continued from previous page)

```
sage: C = RP3.universal_cover(); C
Simplicial set with 8 non-degenerate simplices
sage: len(C.face_data())  # needs gap_package_polenta
8
sage: C.fundamental_group()
Finitely presented group <  |  >
sage: C.betti() == simplicial_sets.Sphere(3).betti()
True
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.groups
>>> RP3 = simplicial_sets.RealProjectiveSpace(Integer(3))
>>> C = RP3.universal_cover(); C
Simplicial set with 8 non-degenerate simplices
>>> len(C.face_data())  # needs gap_package_polenta
8
>>> C.fundamental_group()
Finitely presented group <  |  >
>>> C.betti() == simplicial_sets.Sphere(Integer(3)).betti()
True
```

universal_cover_map()

Return the universal covering map of the simplicial set.

It requires the fundamental group to be finite.

EXAMPLES:

```
sage: RP2 = simplicial_sets.RealProjectiveSpace(2)                    #␣
↪needs sage.graphs sage.groups
sage: phi = RP2.universal_cover_map(); phi                            #␣
↪needs sage.graphs sage.groups gap_package_polenta
Simplicial set morphism:
  From: Simplicial set with 6 non-degenerate simplices
  To:   RP^2
  Defn: [(1, 1), (1, e), (f, 1), (f, e), (f * f, 1), (f * f, e)]
        --> [1, 1, f, f, f * f, f * f]
sage: phi.domain().face_data()                                        #␣
↪needs sage.graphs sage.groups gap_package_polenta
    {(1, 1): None,
     (1, e): None,
     (f, 1): ((1, e), (1, 1)),
     (f, e): ((1, 1), (1, e)),
     (f * f, 1): ((f, e), s_0 (1, 1), (f, 1)),
     (f * f, e): ((f, 1), s_0 (1, e), (f, e))}
```

```
>>> from sage.all import *
>>> RP2 = simplicial_sets.RealProjectiveSpace(Integer(2))              ␣
↪    # needs sage.graphs sage.groups
>>> phi = RP2.universal_cover_map(); phi                              #␣
↪needs sage.graphs sage.groups gap_package_polenta
Simplicial set morphism:
```

(continues on next page)

(continued from previous page)

```
  From: Simplicial set with 6 non-degenerate simplices
  To:   RP^2
  Defn: [(1, 1), (1, e), (f, 1), (f, e), (f * f, 1), (f * f, e)]
        --> [1, 1, f, f, f * f, f * f]
>>> phi.domain().face_data()                                    #␣
↪needs sage.graphs sage.groups gap_package_polenta
    {(1, 1): None,
     (1, e): None,
     (f, 1): ((1, e), (1, 1)),
     (f, e): ((1, 1), (1, e)),
     (f * f, 1): ((f, e), s_0 (1, 1), (f, 1)),
     (f * f, e): ((f, 1), s_0 (1, e), (f, e))}
```

class SubcategoryMethods

Bases: object

Pointed()

A simplicial set is *pointed* if it has a distinguished base point.

EXAMPLES:

```
sage: from sage.categories.simplicial_sets import SimplicialSets
sage: SimplicialSets().Pointed().Finite()
Category of finite pointed simplicial sets
sage: SimplicialSets().Finite().Pointed()
Category of finite pointed simplicial sets
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_sets import SimplicialSets
>>> SimplicialSets().Pointed().Finite()
Category of finite pointed simplicial sets
>>> SimplicialSets().Finite().Pointed()
Category of finite pointed simplicial sets
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.simplicial_sets import SimplicialSets
sage: SimplicialSets().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> from sage.categories.simplicial_sets import SimplicialSets
>>> SimplicialSets().super_categories()
[Category of sets]
```

4.157 Super Algebras

class sage.categories.super_algebras.**SuperAlgebras**(*base_category*)

Bases: *SuperModulesCategory*

The category of super algebras.

An *R-super algebra* is an R-super module A endowed with an R-algebra structure satisfying

$$A_0A_0 \subseteq A_0, \qquad A_0A_1 \subseteq A_1, \qquad A_1A_0 \subseteq A_1, \qquad A_1A_1 \subseteq A_0$$

and $1 \in A_0$.

EXAMPLES:

```
sage: Algebras(ZZ).Super()
Category of super algebras over Integer Ring
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Super()
Category of super algebras over Integer Ring
```

class ParentMethods

Bases: object

graded_algebra()

Return the associated graded algebra to self.

> **Warning**
>
> Because a super module M is naturally $\mathbf{Z}/2\mathbf{Z}$-graded, and graded modules have a natural filtration induced by the grading, if M has a different filtration, then the associated graded module $\operatorname{gr} M \neq M$. This is most apparent with super algebras, such as the differential Weyl algebra, and the multiplication may not coincide.

tensor(**parents*, ***kwargs*)

Return the tensor product of the parents.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A.<x,y,z> = ExteriorAlgebra(ZZ); A.rename('A')
sage: T = A.tensor(A,A); T
A # A # A
sage: T in Algebras(ZZ).Graded().SignedTensorProducts()
True
sage: T in Algebras(ZZ).Graded().TensorProducts()
False
sage: A.rename(None)
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = ExteriorAlgebra(ZZ, names=('x', 'y', 'z',)); (x, y, z,) = A._
↪first_ngens(3); A.rename('A')
>>> T = A.tensor(A,A); T
A # A # A
>>> T in Algebras(ZZ).Graded().SignedTensorProducts()
True
>>> T in Algebras(ZZ).Graded().TensorProducts()
False
>>> A.rename(None)
```

This also works when the other elements do not have a signed tensor product (Issue #31266):

```
sage: # needs sage.combinat sage.modules
sage: a = SteenrodAlgebra(3).an_element()
sage: M = CombinatorialFreeModule(GF(3), ['s', 't', 'u'])
sage: s = M.basis()['s']
sage: tensor([a, s])                                                    #␣
↪needs sage.rings.finite_rings
2*Q_1 Q_3 P(2,1) # B['s']
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> a = SteenrodAlgebra(Integer(3)).an_element()
>>> M = CombinatorialFreeModule(GF(Integer(3)), ['s', 't', 'u'])
>>> s = M.basis()['s']
>>> tensor([a, s])                                                      #␣
↪needs sage.rings.finite_rings
2*Q_1 Q_3 P(2,1) # B['s']
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

extra_super_categories()

EXAMPLES:

```
sage: Coalgebras(QQ).Graded().SignedTensorProducts().extra_super_
↪categories()
[Category of graded coalgebras over Rational Field]
sage: Coalgebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> Coalgebras(QQ).Graded().SignedTensorProducts().extra_super_
↪categories()
[Category of graded coalgebras over Rational Field]
>>> Coalgebras(QQ).Graded().SignedTensorProducts().super_categories()
[Category of graded coalgebras over Rational Field]
```

Meaning: a signed tensor product of coalgebras is a coalgebra

class SubcategoryMethods

Bases: `object`

Supercommutative()

Return the full subcategory of the supercommutative objects of `self`.

A super algebra M is *supercommutative* if, for all homogeneous $x, y \in M$,

$$x \cdot y = (-1)^{|x||y|} y \cdot x.$$

REFERENCES:

Wikipedia article Supercommutative_algebra

EXAMPLES:

```
sage: Algebras(ZZ).Super().Supercommutative()
Category of supercommutative algebras over Integer Ring
sage: Algebras(ZZ).Super().WithBasis().Supercommutative()
Category of supercommutative algebras with basis over Integer Ring
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Super().Supercommutative()
Category of supercommutative algebras over Integer Ring
>>> Algebras(ZZ).Super().WithBasis().Supercommutative()
Category of supercommutative algebras with basis over Integer Ring
```

Supercommutative

alias of *SupercommutativeAlgebras*

extra_super_categories()

EXAMPLES:

```
sage: Algebras(ZZ).Super().super_categories() # indirect doctest
[Category of graded algebras over Integer Ring,
 Category of super modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Super().super_categories() # indirect doctest
[Category of graded algebras over Integer Ring,
 Category of super modules over Integer Ring]
```

4.158 Super algebras with basis

class sage.categories.super_algebras_with_basis.**SuperAlgebrasWithBasis**(*base_category*)

Bases: *SuperModulesCategory*

The category of super algebras with a distinguished basis.

EXAMPLES:

```
sage: C = Algebras(ZZ).WithBasis().Super(); C
Category of super algebras with basis over Integer Ring
```

```
>>> from sage.all import *
>>> C = Algebras(ZZ).WithBasis().Super(); C
Category of super algebras with basis over Integer Ring
```

class ElementMethods

Bases: object

supercommutator(*x*)

Return the supercommutator of `self` and `x`.

Let A be a superalgebra. The *supercommutator* of homogeneous elements $x, y \in A$ is defined by

$$[x, y\} = xy - (-1)^{|x||y|}yx$$

and extended to all elements by linearity.

EXAMPLES:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(ZZ, 3, [1,2,3,4,5,6])
sage: Cl.<x,y,z> = CliffordAlgebra(Q)
sage: a = x*y - z
sage: b = x - y + y*z
sage: a.supercommutator(b)
-5*x*y + 8*x*z - 2*y*z - 6*x + 12*y - 5*z
sage: a.supercommutator(Cl.one())
0
sage: Cl.one().supercommutator(a)
0
sage: Cl.zero().supercommutator(a)
0
sage: a.supercommutator(Cl.zero())
0

sage: # needs sage.modules
sage: Q = QuadraticForm(ZZ, 2, [-1,1,-3])
sage: Cl.<x,y> = CliffordAlgebra(Q)
sage: [a.supercommutator(b) for a in Cl.basis() for b in Cl.basis()]
[0, 0, 0, 0, 0, -2, 1, -x - 2*y, 0, 1,
 -6, 6*x + y, 0, x + 2*y, -6*x - y, 0]
sage: [a*b-b*a for a in Cl.basis() for b in Cl.basis()]
[0, 0, 0, 0, 0, 0, 2*x*y - 1, -x - 2*y, 0,
 -2*x*y + 1, 0, 6*x + y, 0, x + 2*y, -6*x - y, 0]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(ZZ, Integer(3), [Integer(1),Integer(2),Integer(3),
↪Integer(4),Integer(5),Integer(6)])
>>> Cl = CliffordAlgebra(Q, names=('x', 'y', 'z',)); (x, y, z,) = Cl._
↪first_ngens(3)
>>> a = x*y - z
>>> b = x - y + y*z
>>> a.supercommutator(b)
-5*x*y + 8*x*z - 2*y*z - 6*x + 12*y - 5*z
>>> a.supercommutator(Cl.one())
0
>>> Cl.one().supercommutator(a)
0
>>> Cl.zero().supercommutator(a)
0
>>> a.supercommutator(Cl.zero())
0

>>> # needs sage.modules
>>> Q = QuadraticForm(ZZ, Integer(2), [-Integer(1),Integer(1),-
↪Integer(3)])
>>> Cl = CliffordAlgebra(Q, names=('x', 'y',)); (x, y,) = Cl._first_
↪ngens(2)
>>> [a.supercommutator(b) for a in Cl.basis() for b in Cl.basis()]
[0, 0, 0, 0, 0, -2, 1, -x - 2*y, 0, 1,
```

(continues on next page)

(continued from previous page)

```
 -6, 6*x + y, 0, x + 2*y, -6*x - y, 0]
>>> [a*b-b*a for a in Cl.basis() for b in Cl.basis()]
[0, 0, 0, 0, 0, 0, 2*x*y - 1, -x - 2*y, 0,
 -2*x*y + 1, 0, 6*x + y, 0, x + 2*y, -6*x - y, 0]
```

Exterior algebras inherit from Clifford algebras, so supercommutators work as well. We verify the exterior algebra is supercommutative:

```
sage: E.<x,y,z,w> = ExteriorAlgebra(QQ)                                    #␣
↪needs sage.modules
sage: all(b1.supercommutator(b2) == 0                                      #␣
↪needs sage.modules
....:     for b1 in E.basis() for b2 in E.basis())
True
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('x', 'y', 'z', 'w',)); (x, y, z, w,) =␣
↪E._first_ngens(4)# needs sage.modules
>>> all(b1.supercommutator(b2) == Integer(0)                                ␣
↪     # needs sage.modules
...     for b1 in E.basis() for b2 in E.basis())
True
```

class ParentMethods

Bases: `object`

graded_algebra()

Return the associated graded module to `self`.

See `AssociatedGradedAlgebra` for the definition and the properties of this.

See also

graded_algebra()

EXAMPLES:

```
sage: W.<x,y> = algebras.DifferentialWeyl(QQ)                              #␣
↪needs sage.modules
sage: W.graded_algebra()                                                   #␣
↪needs sage.combinat sage.modules
Graded Algebra of Differential Weyl algebra of
 polynomials in x, y over Rational Field
```

```
>>> from sage.all import *
>>> W = algebras.DifferentialWeyl(QQ, names=('x', 'y',)); (x, y,) = W._
↪first_ngens(2)# needs sage.modules
>>> W.graded_algebra()                                                      #␣
↪needs sage.combinat sage.modules
Graded Algebra of Differential Weyl algebra of
 polynomials in x, y over Rational Field
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

The category of super algebras with basis constructed by tensor product of super algebras with basis.

extra_super_categories()

EXAMPLES:

```
sage: Algebras(QQ).Super().SignedTensorProducts().extra_super_categories()
[Category of super algebras over Rational Field]
sage: Algebras(QQ).Super().SignedTensorProducts().super_categories()
[Category of signed tensor products of graded algebras over Rational␣
↪Field,
 Category of super algebras over Rational Field]
```

```
>>> from sage.all import *
>>> Algebras(QQ).Super().SignedTensorProducts().extra_super_categories()
[Category of super algebras over Rational Field]
>>> Algebras(QQ).Super().SignedTensorProducts().super_categories()
[Category of signed tensor products of graded algebras over Rational␣
↪Field,
 Category of super algebras over Rational Field]
```

Meaning: a signed tensor product of super algebras is a super algebra

extra_super_categories()

EXAMPLES:

```
sage: C = Algebras(ZZ).WithBasis().Super()
sage: sorted(C.super_categories(), key=str)  # indirect doctest
[Category of graded algebras with basis over Integer Ring,
 Category of super algebras over Integer Ring,
 Category of super modules with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> C = Algebras(ZZ).WithBasis().Super()
>>> sorted(C.super_categories(), key=str)  # indirect doctest
[Category of graded algebras with basis over Integer Ring,
 Category of super algebras over Integer Ring,
 Category of super modules with basis over Integer Ring]
```

4.159 Super Hopf algebras with basis

class sage.categories.super_hopf_algebras_with_basis.**SuperHopfAlgebrasWithBasis**(*base_category*)

Bases: *SuperModulesCategory*

The category of super Hopf algebras with a distinguished basis.

EXAMPLES:

```
sage: C = HopfAlgebras(ZZ).WithBasis().Super(); C
Category of super Hopf algebras with basis over Integer Ring
```

(continues on next page)

(continued from previous page)

```
sage: sorted(C.super_categories(), key=str)
[Category of super Hopf algebras over Integer Ring,
 Category of super algebras with basis over Integer Ring,
 Category of super coalgebras with basis over Integer Ring]
```

```
>>> from sage.all import *
>>> C = HopfAlgebras(ZZ).WithBasis().Super(); C
Category of super Hopf algebras with basis over Integer Ring
>>> sorted(C.super_categories(), key=str)
[Category of super Hopf algebras over Integer Ring,
 Category of super algebras with basis over Integer Ring,
 Category of super coalgebras with basis over Integer Ring]
```

class ParentMethods

Bases: `object`

antipode()

The antipode of this Hopf algebra.

If *antipode_on_basis()* is available, this constructs the antipode morphism from `self` to `self` by extending it by linearity. Otherwise, *antipode_by_coercion()* is used, if available.

EXAMPLES:

```
sage: A = SteenrodAlgebra(7)                                                #␣
↪needs sage.combinat sage.modules
sage: a = A.an_element()                                                    #␣
↪needs sage.combinat sage.modules
sage: a, A.antipode(a)                                                      #␣
↪needs sage.combinat sage.modules
(6 Q_1 Q_3 P(2,1), Q_1 Q_3 P(2,1))
```

```
>>> from sage.all import *
>>> A = SteenrodAlgebra(Integer(7))                                          ␣
↪     # needs sage.combinat sage.modules
>>> a = A.an_element()                                                     #␣
↪needs sage.combinat sage.modules
>>> a, A.antipode(a)                                                       #␣
↪needs sage.combinat sage.modules
(6 Q_1 Q_3 P(2,1), Q_1 Q_3 P(2,1))
```

4.160 Super Lie Conformal Algebras

AUTHORS:

- Reimundo Heluani (2019-10-05): Initial implementation.

class sage.categories.super_lie_conformal_algebras.**SuperLieConformalAlgebras**(*base_category*)

Bases: *SuperModulesCategory*

The category of super Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(AA).Super()                                    #␣
↪needs sage.rings.number_field
Category of super Lie conformal algebras over Algebraic Real Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(AA).Super()                                      #␣
↪needs sage.rings.number_field
Category of super Lie conformal algebras over Algebraic Real Field
```

Notice that we can force to have a *purely even* super Lie conformal algebra:

```
sage: bosondict = {('a','a'): {1:{('K',0):1}}}
sage: R = LieConformalAlgebra(QQ, bosondict, names=('a',),                #␣
↪needs sage.combinat sage.modules
....:                         central_elements=('K',), super=True)
sage: [g.is_even_odd() for g in R.gens()]                                 #␣
↪needs sage.combinat sage.modules
[0, 0]
```

```
>>> from sage.all import *
>>> bosondict = {('a','a'): {Integer(1):{('K',Integer(0)):Integer(1)}}}
>>> R = LieConformalAlgebra(QQ, bosondict, names=('a',),                  #␣
↪needs sage.combinat sage.modules
...                         central_elements=('K',), super=True)
>>> [g.is_even_odd() for g in R.gens()]                                   #␣
↪needs sage.combinat sage.modules
[0, 0]
```

class ElementMethods

Bases: object

is_even_odd()

Return 0 if this element is *even* and 1 if it is *odd*.

EXAMPLES:

```
sage: R = lie_conformal_algebras.NeveuSchwarz(QQ)                         #␣
↪needs sage.combinat sage.modules
sage: R.inject_variables()                                                #␣
↪needs sage.combinat sage.modules
Defining L, G, C
sage: G.is_even_odd()                                                     #␣
↪needs sage.combinat sage.modules
1
```

```
>>> from sage.all import *
>>> R = lie_conformal_algebras.NeveuSchwarz(QQ)                           #␣
↪needs sage.combinat sage.modules
>>> R.inject_variables()                                                  #␣
↪needs sage.combinat sage.modules
Defining L, G, C
>>> G.is_even_odd()                                                       #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.combinat sage.modules
1
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

The category of H-graded super Lie conformal algebras.

EXAMPLES:

```
sage: LieConformalAlgebras(AA).Super().Graded()                                 #␣
↪needs sage.rings.number_field
Category of H-graded super Lie conformal algebras over Algebraic Real Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(AA).Super().Graded()                                   #␣
↪needs sage.rings.number_field
Category of H-graded super Lie conformal algebras over Algebraic Real Field
```

class ParentMethods

Bases: object

example()

An example parent in this category.

EXAMPLES:

```
sage: LieConformalAlgebras(QQ).Super().example()                                #␣
↪needs sage.combinat sage.modules
The Neveu-Schwarz super Lie conformal algebra over Rational Field
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQ).Super().example()                                  #␣
↪needs sage.combinat sage.modules
The Neveu-Schwarz super Lie conformal algebra over Rational Field
```

extra_super_categories()

The extra super categories of `self`.

EXAMPLES:

```
sage: LieConformalAlgebras(QQ).Super().super_categories()
[Category of super modules over Rational Field,
 Category of Lambda bracket algebras over Rational Field]
```

```
>>> from sage.all import *
>>> LieConformalAlgebras(QQ).Super().super_categories()
[Category of super modules over Rational Field,
 Category of Lambda bracket algebras over Rational Field]
```

4.161 Super modules

class sage.categories.super_modules.**SuperModules**(*base_category*)

Bases: *SuperModulesCategory*

The category of super modules.

An *R-super module* (where R is a ring) is an R-module M equipped with a decomposition $M = M_0 \oplus M_1$ into two R-submodules M_0 and M_1 (called the *even part* and the *odd part* of M, respectively).

Thus, an R-super module automatically becomes a $\mathbf{Z}/2\mathbf{Z}$-graded R-module, with M_0 being the degree-0 component and M_1 being the degree-1 component.

EXAMPLES:

```
sage: Modules(ZZ).Super()
Category of super modules over Integer Ring
sage: Modules(ZZ).Super().super_categories()
[Category of graded modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).Super()
Category of super modules over Integer Ring
>>> Modules(ZZ).Super().super_categories()
[Category of graded modules over Integer Ring]
```

The category of super modules defines the super structure which shall be preserved by morphisms:

```
sage: Modules(ZZ).Super().additional_structure()
Category of super modules over Integer Ring
```

```
>>> from sage.all import *
>>> Modules(ZZ).Super().additional_structure()
Category of super modules over Integer Ring
```

class ElementMethods

Bases: object

is_even()

Return if `self` is an even element.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: cat = Algebras(QQ).WithBasis().Super()
sage: C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
sage: C.degree_on_basis = sum
sage: C.basis()[2,2,1].is_even()
False
sage: C.basis()[2,2].is_even()
True
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> cat = Algebras(QQ).WithBasis().Super()
```

(continues on next page)

(continued from previous page)

```
>>> C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
>>> C.degree_on_basis = sum
>>> C.basis()[Integer(2),Integer(2),Integer(1)].is_even()
False
>>> C.basis()[Integer(2),Integer(2)].is_even()
True
```

is_even_odd()

Return `0` if `self` is an even element or `1` if an odd element.

Note

The default implementation assumes that the even/odd is determined by the parity of *degree()*.

Overwrite this method if the even/odd behavior is desired to be independent.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: cat = Algebras(QQ).WithBasis().Super()
sage: C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
sage: C.degree_on_basis = sum
sage: C.basis()[2,2,1].is_even_odd()
1
sage: C.basis()[2,2].is_even_odd()
0
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> cat = Algebras(QQ).WithBasis().Super()
>>> C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
>>> C.degree_on_basis = sum
>>> C.basis()[Integer(2),Integer(2),Integer(1)].is_even_odd()
1
>>> C.basis()[Integer(2),Integer(2)].is_even_odd()
0
```

is_odd()

Return if `self` is an odd element.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: cat = Algebras(QQ).WithBasis().Super()
sage: C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
sage: C.degree_on_basis = sum
sage: C.basis()[2,2,1].is_odd()
True
sage: C.basis()[2,2].is_odd()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> cat = Algebras(QQ).WithBasis().Super()
>>> C = CombinatorialFreeModule(QQ, Partitions(), category=cat)
>>> C.degree_on_basis = sum
>>> C.basis()[Integer(2),Integer(2),Integer(1)].is_odd()
True
>>> C.basis()[Integer(2),Integer(2)].is_odd()
False
```

class ParentMethods

Bases: object

extra_super_categories()

Add *VectorSpaces* to the super categories of `self` if the base ring is a field.

EXAMPLES:

```
sage: Modules(QQ).Super().extra_super_categories()
[Category of vector spaces over Rational Field]
sage: Modules(ZZ).Super().extra_super_categories()
[]
```

```
>>> from sage.all import *
>>> Modules(QQ).Super().extra_super_categories()
[Category of vector spaces over Rational Field]
>>> Modules(ZZ).Super().extra_super_categories()
[]
```

This makes sure that `Modules(QQ).Super()` returns an instance of *SuperModules* and not a join category of an instance of this class and of `VectorSpaces(QQ)`:

```
sage: type(Modules(QQ).Super())
<class 'sage.categories.super_modules.SuperModules_with_category'>
```

```
>>> from sage.all import *
>>> type(Modules(QQ).Super())
<class 'sage.categories.super_modules.SuperModules_with_category'>
```

Todo

Get rid of this workaround once there is a more systematic approach for the alias `Modules(QQ)` -> `VectorSpaces(QQ)`. Probably the latter should be a category with axiom, and covariant constructions should play well with axioms.

super_categories()

EXAMPLES:

```
sage: Modules(ZZ).Super().super_categories()
[Category of graded modules over Integer Ring]
```

```
>>> from sage.all import *
>>> Modules(ZZ).Super().super_categories()
[Category of graded modules over Integer Ring]
```

Nota bene:

```
sage: Modules(QQ).Super()
Category of super modules over Rational Field
sage: Modules(QQ).Super().super_categories()
[Category of graded modules over Rational Field]
```

```
>>> from sage.all import *
>>> Modules(QQ).Super()
Category of super modules over Rational Field
>>> Modules(QQ).Super().super_categories()
[Category of graded modules over Rational Field]
```

class sage.categories.super_modules.**SuperModulesCategory**(*base_category*)

Bases: *CovariantConstructionCategory*, *Category_over_base_ring*

EXAMPLES:

```
sage: C = Algebras(QQ).Super()
sage: C
Category of super algebras over Rational Field
sage: C.base_category()
Category of algebras over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of graded algebras over Rational Field,
 Category of super modules over Rational Field]

sage: AlgebrasWithBasis(QQ).Super().base_ring()
Rational Field
sage: HopfAlgebrasWithBasis(QQ).Super().base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> C = Algebras(QQ).Super()
>>> C
Category of super algebras over Rational Field
>>> C.base_category()
Category of algebras over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of graded algebras over Rational Field,
 Category of super modules over Rational Field]

>>> AlgebrasWithBasis(QQ).Super().base_ring()
Rational Field
>>> HopfAlgebrasWithBasis(QQ).Super().base_ring()
Rational Field
```

classmethod default_super_categories(*category*, **args*)

Return the default super categories of $F_{Cat}(A, B, ...)$ for $A, B, ...$ parents in Cat.

INPUT:

- `cls` – the category class for the functor F
- `category` – a category Cat
- `*args` – further arguments for the functor

OUTPUT: a join category

This implements the property that subcategories constructed by the set of whitelisted axioms is a subcategory.

EXAMPLES:

```
sage: HopfAlgebras(ZZ).WithBasis().FiniteDimensional().Super()  # indirect␣
↪doctest
Category of finite dimensional super Hopf algebras with basis over Integer␣
↪Ring
```

```
>>> from sage.all import *
>>> HopfAlgebras(ZZ).WithBasis().FiniteDimensional().Super()  # indirect␣
↪doctest
Category of finite dimensional super Hopf algebras with basis over Integer␣
↪Ring
```

4.162 Super modules with basis

class sage.categories.super_modules_with_basis.**SuperModulesWithBasis**(*base_category*)

Bases: *SuperModulesCategory*

The category of super modules with a distinguished basis.

An *R-super module with a distinguished basis* is an R-super module equipped with an R-module basis whose elements are homogeneous.

EXAMPLES:

```
sage: C = GradedModulesWithBasis(QQ); C
Category of graded vector spaces with basis over Rational Field
sage: sorted(C.super_categories(), key=str)
[Category of filtered vector spaces with basis over Rational Field,
 Category of graded modules with basis over Rational Field,
 Category of graded vector spaces over Rational Field]
sage: C is ModulesWithBasis(QQ).Graded()
True
```

```
>>> from sage.all import *
>>> C = GradedModulesWithBasis(QQ); C
Category of graded vector spaces with basis over Rational Field
>>> sorted(C.super_categories(), key=str)
[Category of filtered vector spaces with basis over Rational Field,
 Category of graded modules with basis over Rational Field,
 Category of graded vector spaces over Rational Field]
>>> C is ModulesWithBasis(QQ).Graded()
True
```

class ElementMethods

Bases: object

even_component()

Return the even component of self.

EXAMPLES:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(QQ, 2, [1,2,3])
sage: C.<x,y> = CliffordAlgebra(Q)
sage: a = x*y + x - 3*y + 4
sage: a.even_component()
x*y + 4
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(QQ, Integer(2), [Integer(1),Integer(2),Integer(3)])
>>> C = CliffordAlgebra(Q, names=('x', 'y',)); (x, y,) = C._first_ngens(2)
>>> a = x*y + x - Integer(3)*y + Integer(4)
>>> a.even_component()
x*y + 4
```

is_even_odd()

Return 0 if self is an even element and 1 if self is an odd element.

EXAMPLES:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(QQ, 2, [1,2,3])
sage: C.<x,y> = CliffordAlgebra(Q)
sage: a = x + y
sage: a.is_even_odd()
1
sage: a = x*y + 4
sage: a.is_even_odd()
0
sage: a = x + 4
sage: a.is_even_odd()
Traceback (most recent call last):
...
ValueError: element is not homogeneous

sage: E.<x,y> = ExteriorAlgebra(QQ)                                    #␣
↪needs sage.modules
sage: (x*y).is_even_odd()                                              #␣
↪needs sage.modules
0
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(QQ, Integer(2), [Integer(1),Integer(2),Integer(3)])
>>> C = CliffordAlgebra(Q, names=('x', 'y',)); (x, y,) = C._first_ngens(2)
>>> a = x + y
```

(continues on next page)

(continued from previous page)

```
>>> a.is_even_odd()
1
>>> a = x*y + Integer(4)
>>> a.is_even_odd()
0
>>> a = x + Integer(4)
>>> a.is_even_odd()
Traceback (most recent call last):
...
ValueError: element is not homogeneous

>>> E = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = E._first_
↪ngens(2)# needs sage.modules
>>> (x*y).is_even_odd()                                              #␣
↪needs sage.modules
0
```

is_super_homogeneous()

Return whether this element is homogeneous, in the sense of a super module (i.e., is even or odd).

EXAMPLES:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(QQ, 2, [1,2,3])
sage: C.<x,y> = CliffordAlgebra(Q)
sage: a = x + y
sage: a.is_super_homogeneous()
True
sage: a = x*y + 4
sage: a.is_super_homogeneous()
True
sage: a = x*y + x - 3*y + 4
sage: a.is_super_homogeneous()
False
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(QQ, Integer(2), [Integer(1),Integer(2),Integer(3)])
>>> C = CliffordAlgebra(Q, names=('x', 'y',)); (x, y,) = C._first_ngens(2)
>>> a = x + y
>>> a.is_super_homogeneous()
True
>>> a = x*y + Integer(4)
>>> a.is_super_homogeneous()
True
>>> a = x*y + x - Integer(3)*y + Integer(4)
>>> a.is_super_homogeneous()
False
```

The exterior algebra has a $\mathbf{Z}$ grading, which induces the $\mathbf{Z}/2\mathbf{Z}$ grading. However the definition of homogeneous elements differs because of the different gradings:

```
sage: # needs sage.combinat sage.modules
sage: E.<x,y> = ExteriorAlgebra(QQ)
sage: a = x*y + 4
sage: a.is_super_homogeneous()
True
sage: a.is_homogeneous()
False
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> E = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = E._first_
↪ngens(2)
>>> a = x*y + Integer(4)
>>> a.is_super_homogeneous()
True
>>> a.is_homogeneous()
False
```

odd_component()

Return the odd component of `self`.

EXAMPLES:

```
sage: # needs sage.modules
sage: Q = QuadraticForm(QQ, 2, [1,2,3])
sage: C.<x,y> = CliffordAlgebra(Q)
sage: a = x*y + x - 3*y + 4
sage: a.odd_component()
x - 3*y
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> Q = QuadraticForm(QQ, Integer(2), [Integer(1),Integer(2),Integer(3)])
>>> C = CliffordAlgebra(Q, names=('x', 'y',)); (x, y,) = C._first_ngens(2)
>>> a = x*y + x - Integer(3)*y + Integer(4)
>>> a.odd_component()
x - 3*y
```

class ParentMethods

Bases: `object`

4.163 Supercommutative Algebras

class sage.categories.supercommutative_algebras.**SupercommutativeAlgebras**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of supercommutative algebras.

An *R-supercommutative algebra* is an R-super algebra $A = A_0 \oplus A_1$ endowed with an R-super algebra structure satisfying:

$$x_0 x_0' = x_0' x_0, \qquad x_1 x_1' = -x_1' x_1, \qquad x_0 x_1 = x_1 x_0,$$

for all $x_0, x_0' \in A_0$ and $x_1, x_1' \in A_1$.

EXAMPLES:

```
sage: Algebras(ZZ).Supercommutative()
Category of supercommutative algebras over Integer Ring
```

```
>>> from sage.all import *
>>> Algebras(ZZ).Supercommutative()
Category of supercommutative algebras over Integer Ring
```

class SignedTensorProducts(*category*, **args*)

Bases: *SignedTensorProductsCategory*

extra_super_categories()

Return the extra super categories of `self`.

A signed tensor product of supercommutative algebras is a supercommutative algebra.

EXAMPLES:

```
sage: C = Algebras(ZZ).Supercommutative().SignedTensorProducts()
sage: C.extra_super_categories()
[Category of supercommutative algebras over Integer Ring]
```

```
>>> from sage.all import *
>>> C = Algebras(ZZ).Supercommutative().SignedTensorProducts()
>>> C.extra_super_categories()
[Category of supercommutative algebras over Integer Ring]
```

class WithBasis(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: `object`

4.164 Supercrystals

class sage.categories.supercrystals.**SuperCrystals**

Bases: *Category_singleton*

class Finite(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class ElementMethods

Bases: `object`

is_genuine_highest_weight(*index_set=None*)

Return whether `self` is a genuine highest weight element.

INPUT:

- `index_set` – (optional) the index set of the (sub)crystal on which to check

EXAMPLES:

```
sage: B = crystals.Tableaux(['A', [1,1]], shape=[3,2,1])
sage: for b in B.highest_weight_vectors():
```

(continues on next page)

(continued from previous page)

```
....:     print("{} {}".format(b, b.is_genuine_highest_weight()))
[[-2, -2, -2], [-1, -1], [1]] True
[[-2, -2, -2], [-1, 2], [1]] False
[[-2, -2, 2], [-1, -1], [1]] False
sage: [b for b in B if b.is_genuine_highest_weight([-1,0])]
[[[-2, -2, -2], [-1, -1], [1]],
 [[-2, -2, -2], [-1, -1], [2]],
 [[-2, -2, -2], [-1, 2], [2]],
 [[-2, -2, 2], [-1, -1], [2]],
 [[-2, -2, 2], [-1, 2], [2]],
 [[-2, -2, -2], [-1, 2], [1]],
 [[-2, -2, 2], [-1, -1], [1]],
 [[-2, -2, 2], [-1, 2], [1]]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A', [Integer(1),Integer(1)]],␣
↪shape=[Integer(3),Integer(2),Integer(1)])
>>> for b in B.highest_weight_vectors():
...     print("{} {}".format(b, b.is_genuine_highest_weight()))
[[-2, -2, -2], [-1, -1], [1]] True
[[-2, -2, -2], [-1, 2], [1]] False
[[-2, -2, 2], [-1, -1], [1]] False
>>> [b for b in B if b.is_genuine_highest_weight([-Integer(1),
↪Integer(0)])]
[[[-2, -2, -2], [-1, -1], [1]],
 [[-2, -2, -2], [-1, -1], [2]],
 [[-2, -2, -2], [-1, 2], [2]],
 [[-2, -2, 2], [-1, -1], [2]],
 [[-2, -2, 2], [-1, 2], [2]],
 [[-2, -2, -2], [-1, 2], [1]],
 [[-2, -2, 2], [-1, -1], [1]],
 [[-2, -2, 2], [-1, 2], [1]]]
```

is_genuine_lowest_weight(*index_set=None*)

Return whether `self` is a genuine lowest weight element.

INPUT:

- `index_set` – (optional) the index set of the (sub)crystal on which to check

EXAMPLES:

```
sage: B = crystals.Tableaux(['A', [1,1]], shape=[3,2,1])
sage: for b in sorted(B.lowest_weight_vectors()):
....:     print("{} {}".format(b, b.is_genuine_lowest_weight()))
[[-2, 1, 2], [-1, 2], [1]] False
[[-2, 1, 2], [-1, 2], [2]] False
[[-1, 1, 2], [1, 2], [2]] True
sage: [b for b in B if b.is_genuine_lowest_weight([-1,0])]
[[[-2, -1, 1], [-1, 1], [1]],
 [[-2, -1, 1], [-1, 1], [2]],
 [[-2, 1, 2], [-1, 1], [2]],
 [[-2, 1, 2], [-1, 1], [1]],
 [[-1, -1, 1], [1, 2], [2]],
```

(continues on next page)

(continued from previous page)

```
 [[-1, -1, 1], [1, 2], [1]],
 [[-1, 1, 2], [1, 2], [2]],
 [[-1, 1, 2], [1, 2], [1]]]
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A', [Integer(1),Integer(1)]],␣
↪shape=[Integer(3),Integer(2),Integer(1)])
>>> for b in sorted(B.lowest_weight_vectors()):
...     print("{} {}".format(b, b.is_genuine_lowest_weight()))
[[-2, 1, 2], [-1, 2], [1]] False
[[-2, 1, 2], [-1, 2], [2]] False
[[-1, 1, 2], [1, 2], [2]] True
>>> [b for b in B if b.is_genuine_lowest_weight([-Integer(1),
↪Integer(0)])]
[[[-2, -1, 1], [-1, 1], [1]],
 [[-2, -1, 1], [-1, 1], [2]],
 [[-2, 1, 2], [-1, 1], [2]],
 [[-2, 1, 2], [-1, 1], [1]],
 [[-1, -1, 1], [1, 2], [2]],
 [[-1, -1, 1], [1, 2], [1]],
 [[-1, 1, 2], [1, 2], [2]],
 [[-1, 1, 2], [1, 2], [1]]]
```

class ParentMethods

Bases: object

character()

Return the character of self.

Todo

Once the $WeylCharacterRing$ is implemented, make this consistent with the implementation in *sage.categories.classical_crystals.ClassicalCrystals.ParentMethods.character()*.

EXAMPLES:

```
sage: B = crystals.Letters(['A',[1,2]])
sage: B.character()
B[(1, 0, 0, 0, 0)] + B[(0, 1, 0, 0, 0)] + B[(0, 0, 1, 0, 0)]
 + B[(0, 0, 0, 1, 0)] + B[(0, 0, 0, 0, 1)]
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A',[Integer(1),Integer(2)]])
>>> B.character()
B[(1, 0, 0, 0, 0)] + B[(0, 1, 0, 0, 0)] + B[(0, 0, 1, 0, 0)]
 + B[(0, 0, 0, 1, 0)] + B[(0, 0, 0, 0, 1)]
```

connected_components()

Return the connected components of self as subcrystals.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,2]])
sage: B.connected_components()
[Subcrystal of The crystal of letters for type ['A', [1, 2]]]

sage: T = B.tensor(B)
sage: T.connected_components()
[Subcrystal of Full tensor product of the crystals
  [The crystal of letters for type ['A', [1, 2]],
   The crystal of letters for type ['A', [1, 2]]],
 Subcrystal of Full tensor product of the crystals
  [The crystal of letters for type ['A', [1, 2]],
   The crystal of letters for type ['A', [1, 2]]]]
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(2)]])
>>> B.connected_components()
[Subcrystal of The crystal of letters for type ['A', [1, 2]]]

>>> T = B.tensor(B)
>>> T.connected_components()
[Subcrystal of Full tensor product of the crystals
  [The crystal of letters for type ['A', [1, 2]],
   The crystal of letters for type ['A', [1, 2]]],
 Subcrystal of Full tensor product of the crystals
  [The crystal of letters for type ['A', [1, 2]],
   The crystal of letters for type ['A', [1, 2]]]]
```

connected_components_generators()

alias of *genuine_highest_weight_vectors()*.

digraph(*index_set=None*)

Return the DiGraph associated to self.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,3]])
sage: G = B.digraph(); G
Multi-digraph on 6 vertices
sage: Q = crystals.Letters(['Q',3])
sage: G = Q.digraph(); G
Multi-digraph on 3 vertices
sage: G.edges(sort=True)
[(1, 2, -1), (1, 2, 1), (2, 3, -2), (2, 3, 2)]
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(3)]])
>>> G = B.digraph(); G
Multi-digraph on 6 vertices
>>> Q = crystals.Letters(['Q',Integer(3)])
>>> G = Q.digraph(); G
Multi-digraph on 3 vertices
>>> G.edges(sort=True)
[(1, 2, -1), (1, 2, 1), (2, 3, -2), (2, 3, 2)]
```

The edges of the crystal graph are by default colored using blue for edge 1, red for edge 2, green for edge 3, and dashed with the corresponding color for barred edges. Edge 0 is dotted black:

```
sage: view(G)  # optional - dot2tex graphviz, not tested (opens␣
↪external window)
```

```
>>> from sage.all import *
>>> view(G)  # optional - dot2tex graphviz, not tested (opens external␣
↪window)
```

genuine_highest_weight_vectors()

Return the tuple of genuine highest weight elements of `self`.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,2]])
sage: B.genuine_highest_weight_vectors()
(-2,)

sage: T = B.tensor(B)
sage: T.genuine_highest_weight_vectors()
([-2, -1], [-2, -2])
sage: s1, s2 = T.connected_components()
sage: s = s1 + s2
sage: s.genuine_highest_weight_vectors()
([-2, -1], [-2, -2])
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(2)]])
>>> B.genuine_highest_weight_vectors()
(-2,)

>>> T = B.tensor(B)
>>> T.genuine_highest_weight_vectors()
([-2, -1], [-2, -2])
>>> s1, s2 = T.connected_components()
>>> s = s1 + s2
>>> s.genuine_highest_weight_vectors()
([-2, -1], [-2, -2])
```

genuine_lowest_weight_vectors()

Return the tuple of genuine lowest weight elements of `self`.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,2]])
sage: B.genuine_lowest_weight_vectors()
(3,)

sage: T = B.tensor(B)
sage: T.genuine_lowest_weight_vectors()
([3, 3], [3, 2])
sage: s1, s2 = T.connected_components()
sage: s = s1 + s2
```

(continues on next page)

(continued from previous page)

```
sage: s.genuine_lowest_weight_vectors()
([3, 3], [3, 2])
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(2)]])
>>> B.genuine_lowest_weight_vectors()
(3,)

>>> T = B.tensor(B)
>>> T.genuine_lowest_weight_vectors()
([3, 3], [3, 2])
>>> s1, s2 = T.connected_components()
>>> s = s1 + s2
>>> s.genuine_lowest_weight_vectors()
([3, 3], [3, 2])
```

highest_weight_vectors()

Return the highest weight vectors of `self`.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,2]])
sage: B.highest_weight_vectors()
(-2,)

sage: T = B.tensor(B)
sage: T.highest_weight_vectors()
([-2, -2], [-2, -1])
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(2)]])
>>> B.highest_weight_vectors()
(-2,)

>>> T = B.tensor(B)
>>> T.highest_weight_vectors()
([-2, -2], [-2, -1])
```

We give an example from [BKK2000] that has fake highest weight vectors:

```
sage: B = crystals.Tableaux(['A', [1,1]], shape=[3,2,1])
sage: B.highest_weight_vectors()
([[-2, -2, -2], [-1, -1], [1]],
 [[-2, -2, -2], [-1, 2], [1]],
 [[-2, -2, 2], [-1, -1], [1]])
sage: B.genuine_highest_weight_vectors()
([[-2, -2, -2], [-1, -1], [1]],)
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A', [Integer(1),Integer(1)]],␣
↪shape=[Integer(3),Integer(2),Integer(1)])
>>> B.highest_weight_vectors()
```

(continues on next page)

(continued from previous page)

```
([[-2, -2, -2], [-1, -1], [1]],
 [[-2, -2, -2], [-1, 2], [1]],
 [[-2, -2, 2], [-1, -1], [1]])
>>> B.genuine_highest_weight_vectors()
([[-2, -2, -2], [-1, -1], [1]],)
```

lowest_weight_vectors()

Return the lowest weight vectors of `self`.

EXAMPLES:

```
sage: B = crystals.Letters(['A', [1,2]])
sage: B.lowest_weight_vectors()
(3,)

sage: T = B.tensor(B)
sage: sorted(T.lowest_weight_vectors())
[[3, 2], [3, 3]]
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A', [Integer(1),Integer(2)]])
>>> B.lowest_weight_vectors()
(3,)

>>> T = B.tensor(B)
>>> sorted(T.lowest_weight_vectors())
[[3, 2], [3, 3]]
```

We give an example from [BKK2000] that has fake lowest weight vectors:

```
sage: B = crystals.Tableaux(['A', [1,1]], shape=[3,2,1])
sage: sorted(B.lowest_weight_vectors())
[[[-2, 1, 2], [-1, 2], [1]],
 [[-2, 1, 2], [-1, 2], [2]],
 [[-1, 1, 2], [1, 2], [2]]]
sage: B.genuine_lowest_weight_vectors()
([[-1, 1, 2], [1, 2], [2]],)
```

```
>>> from sage.all import *
>>> B = crystals.Tableaux(['A', [Integer(1),Integer(1)]],␣
↪shape=[Integer(3),Integer(2),Integer(1)])
>>> sorted(B.lowest_weight_vectors())
[[[-2, 1, 2], [-1, 2], [1]],
 [[-2, 1, 2], [-1, 2], [2]],
 [[-1, 1, 2], [1, 2], [2]]]
>>> B.genuine_lowest_weight_vectors()
([[-1, 1, 2], [1, 2], [2]],)
```

class ParentMethods

Bases: `object`

tensor(**crystals*, ***options*)

Return the tensor product of `self` with the crystals `B`.

EXAMPLES:

```
sage: B = crystals.Letters(['A',[1,2]])
sage: C = crystals.Tableaux(['A',[1,2]], shape = [2,1])
sage: T = C.tensor(B); T
Full tensor product of the crystals
 [Crystal of BKK tableaux of shape [2, 1] of gl(2|3),
  The crystal of letters for type ['A', [1, 2]]]
sage: S = B.tensor(C); S
Full tensor product of the crystals
 [The crystal of letters for type ['A', [1, 2]],
  Crystal of BKK tableaux of shape [2, 1] of gl(2|3)]
sage: G = T.digraph()
sage: H = S.digraph()
sage: G.is_isomorphic(H, edge_labels=True)
True
```

```
>>> from sage.all import *
>>> B = crystals.Letters(['A',[Integer(1),Integer(2)]])
>>> C = crystals.Tableaux(['A',[Integer(1),Integer(2)]], shape =␣
↪[Integer(2),Integer(1)])
>>> T = C.tensor(B); T
Full tensor product of the crystals
 [Crystal of BKK tableaux of shape [2, 1] of gl(2|3),
  The crystal of letters for type ['A', [1, 2]]]
>>> S = B.tensor(C); S
Full tensor product of the crystals
 [The crystal of letters for type ['A', [1, 2]],
  Crystal of BKK tableaux of shape [2, 1] of gl(2|3)]
>>> G = T.digraph()
>>> H = S.digraph()
>>> G.is_isomorphic(H, edge_labels=True)
True
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

The category of regular crystals constructed by tensor product of regular crystals.

extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.supercrystals import SuperCrystals
sage: SuperCrystals().TensorProducts().extra_super_categories()
[Category of super crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.supercrystals import SuperCrystals
>>> SuperCrystals().TensorProducts().extra_super_categories()
[Category of super crystals]
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.supercrystals import SuperCrystals
sage: C = SuperCrystals()
sage: C.super_categories()
[Category of crystals]
```

```
>>> from sage.all import *
>>> from sage.categories.supercrystals import SuperCrystals
>>> C = SuperCrystals()
>>> C.super_categories()
[Category of crystals]
```

4.165 Topological Spaces

class sage.categories.topological_spaces.**TopologicalSpaces**(*category*, **args*)

Bases: *TopologicalSpacesCategory*

The category of topological spaces.

EXAMPLES:

```
sage: Sets().Topological()
Category of topological spaces
sage: Sets().Topological().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Sets().Topological()
Category of topological spaces
>>> Sets().Topological().super_categories()
[Category of sets]
```

The category of topological spaces defines the topological structure, which shall be preserved by morphisms:

```
sage: Sets().Topological().additional_structure()
Category of topological spaces
```

```
>>> from sage.all import *
>>> Sets().Topological().additional_structure()
Category of topological spaces
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a (finite) Cartesian product of topological spaces is a topological space.

EXAMPLES:

```
sage: from sage.categories.topological_spaces import TopologicalSpaces
sage: C = TopologicalSpaces().CartesianProducts()
sage: C.extra_super_categories()
[Category of topological spaces]
```

(continues on next page)

(continued from previous page)

```
sage: C.super_categories()
[Category of Cartesian products of sets, Category of topological spaces]
sage: C.axioms()
frozenset()
```

```
>>> from sage.all import *
>>> from sage.categories.topological_spaces import TopologicalSpaces
>>> C = TopologicalSpaces().CartesianProducts()
>>> C.extra_super_categories()
[Category of topological spaces]
>>> C.super_categories()
[Category of Cartesian products of sets, Category of topological spaces]
>>> C.axioms()
frozenset()
```

class Compact(*base_category*)

Bases: *CategoryWithAxiom*

The category of compact topological spaces.

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a (finite) Cartesian product of compact topological spaces is compact.

EXAMPLES:

```
sage: from sage.categories.topological_spaces import TopologicalSpaces
sage: C = TopologicalSpaces().Compact().CartesianProducts()
sage: C.extra_super_categories()
[Category of compact topological spaces]
sage: C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of compact topological spaces]
sage: C.axioms()
frozenset({'Compact'})
```

```
>>> from sage.all import *
>>> from sage.categories.topological_spaces import TopologicalSpaces
>>> C = TopologicalSpaces().Compact().CartesianProducts()
>>> C.extra_super_categories()
[Category of compact topological spaces]
>>> C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of compact topological spaces]
>>> C.axioms()
frozenset({'Compact'})
```

class Connected(*base_category*)

Bases: *CategoryWithAxiom*

The category of connected topological spaces.

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

Implement the fact that a (finite) Cartesian product of connected topological spaces is connected.

EXAMPLES:

```
sage: from sage.categories.topological_spaces import TopologicalSpaces
sage: C = TopologicalSpaces().Connected().CartesianProducts()
sage: C.extra_super_categories()
[Category of connected topological spaces]
sage: C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of connected topological spaces]
sage: C.axioms()
frozenset({'Connected'})
```

```
>>> from sage.all import *
>>> from sage.categories.topological_spaces import TopologicalSpaces
>>> C = TopologicalSpaces().Connected().CartesianProducts()
>>> C.extra_super_categories()
[Category of connected topological spaces]
>>> C.super_categories()
[Category of Cartesian products of topological spaces,
 Category of connected topological spaces]
>>> C.axioms()
frozenset({'Connected'})
```

class SubcategoryMethods

Bases: object

Compact()

Return the subcategory of the compact objects of `self`.

EXAMPLES:

```
sage: Sets().Topological().Compact()
Category of compact topological spaces
```

```
>>> from sage.all import *
>>> Sets().Topological().Compact()
Category of compact topological spaces
```

Connected()

Return the full subcategory of the connected objects of `self`.

EXAMPLES:

```
sage: Sets().Topological().Connected()
Category of connected topological spaces
```

```
>>> from sage.all import *
>>> Sets().Topological().Connected()
Category of connected topological spaces
```

class sage.categories.topological_spaces.**TopologicalSpacesCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

4.166 Kac-Moody Algebras With Triangular Decomposition Basis

AUTHORS:

- Travis Scrimshaw (07-15-2017): Initial implementation

class sage.categories.triangular_kac_moody_algebras.**TriangularKacMoodyAlgebras**(*base*, *name=None*)

Bases: *Category_over_base_ring*

Category of Kac-Moody algebras with a distinguished basis that respects the triangular decomposition.

We require that the grading group is the root lattice of the appropriate Cartan type.

class ElementMethods

Bases: object

part()

Return whether the element `v` is in the lower, zero, or upper part of `self`.

OUTPUT:

-1 if `v` is in the lower part, 0 if in the zero part, or 1 if in the upper part

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = LieAlgebra(QQ, cartan_type='F4')
sage: L.inject_variables()
Defining e1, e2, e3, e4, f1, f2, f3, f4, h1, h2, h3, h4
sage: e1.part()
1
sage: f4.part()
-1
sage: (h2 + h3).part()
0
sage: (f1.bracket(f2) + 4*f4).part()
-1
sage: (e1 + f1).part()
Traceback (most recent call last):
...
ValueError: element is not in one part
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = LieAlgebra(QQ, cartan_type='F4')
>>> L.inject_variables()
Defining e1, e2, e3, e4, f1, f2, f3, f4, h1, h2, h3, h4
>>> e1.part()
1
>>> f4.part()
-1
>>> (h2 + h3).part()
```

(continues on next page)

(continued from previous page)

```
0
>>> (f1.bracket(f2) + Integer(4)*f4).part()
-1
>>> (e1 + f1).part()
Traceback (most recent call last):
...
ValueError: element is not in one part
```

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

Category of finite dimensional Kac-Moody algebras (which correspond to semisimple Lie algebras) with a distinguished basis that respects the triangular decomposition.

class ElementMethods

Bases: object

transpose()

Return the transpose of self.

The transpose τ is the map that sends the root basis elements $e_\alpha \leftrightarrow e_{-\alpha}$ and fixes the Cartan subalgebra h_α. It is an anti-involution in the sense $[\tau(a), \tau(b)] = \tau([b, a])$.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: g = LieAlgebra(QQ, cartan_type=['G', 2])
sage: for b in g.basis():
....:     for bp in g.basis():
....:         a = g[b.transpose(), bp.transpose()]
....:         ap = g[bp, b].transpose()
....:         assert a == ap
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> g = LieAlgebra(QQ, cartan_type=['G', Integer(2)])
>>> for b in g.basis():
...     for bp in g.basis():
...         a = g[b.transpose(), bp.transpose()]
...         ap = g[bp, b].transpose()
...         assert a == ap
```

class ParentMethods

Bases: object

transpose()

The transpose map of self.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: g = LieAlgebra(QQ, cartan_type=['B', 2])
sage: g.transpose
Generic endomorphism of Lie algebra of ['B', 2] in the Chevalley basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> g = LieAlgebra(QQ, cartan_type=['B', Integer(2)])
>>> g.transpose
Generic endomorphism of Lie algebra of ['B', 2] in the Chevalley basis
```

class ParentMethods

Bases: object

e(*i=None*)

Return the generators e of `self`.

INPUT:

- `i` – (optional) if specified, return just the generator e_i

EXAMPLES:

```
sage: L = lie_algebras.so(QQ, 5)                                        #␣
↪needs sage.combinat sage.modules
sage: L.e()                                                             #␣
↪needs sage.combinat sage.modules
Finite family {1: E[alpha[1]], 2: E[alpha[2]]}
sage: L.e(1)                                                            #␣
↪needs sage.combinat sage.modules
E[alpha[1]]
```

```
>>> from sage.all import *
>>> L = lie_algebras.so(QQ, Integer(5))                                  ␣
↪     # needs sage.combinat sage.modules
>>> L.e()                                                              #␣
↪needs sage.combinat sage.modules
Finite family {1: E[alpha[1]], 2: E[alpha[2]]}
>>> L.e(Integer(1))                                                      ␣
↪     # needs sage.combinat sage.modules
E[alpha[1]]
```

f(*i=None*)

Return the generators f of `self`.

INPUT:

- `i` – (optional) if specified, return just the generator f_i

EXAMPLES:

```
sage: L = lie_algebras.so(QQ, 5)                                        #␣
↪needs sage.combinat sage.modules
sage: L.f()                                                             #␣
↪needs sage.combinat sage.modules
Finite family {1: E[-alpha[1]], 2: E[-alpha[2]]}
sage: L.f(1)                                                            #␣
↪needs sage.combinat sage.modules
E[-alpha[1]]
```

```
>>> from sage.all import *
>>> L = lie_algebras.so(QQ, Integer(5))                                  ␣
```

(continues on next page)

(continued from previous page)

```
↪     # needs sage.combinat sage.modules
>>> L.f()                                                                    #␣
↪needs sage.combinat sage.modules
Finite family {1: E[-alpha[1]], 2: E[-alpha[2]]}
>>> L.f(Integer(1))                                                          ␣
↪     # needs sage.combinat sage.modules
E[-alpha[1]]
```

simple_module(*la*, *basis_key=None*, ***kwds*)

Return the simple module with highest weight `la` over `self`.

INPUT:

- `la` – the weight
- `basis_key` – (optional) a key function for the indexing set of the basis elements of `self`

OUTPUT:

If `la` is `Verma antidominant`, then this returns the `VermaModule` of highest weight `la` (which is simple and more efficient). Otherwise this returns a `SimpleModule`.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: g = lie_algebras.sl(QQ, 3)
sage: P = g.cartan_type().root_system().weight_lattice()
sage: La = P.fundamental_weights()
sage: L = g.simple_module(La[1] + La[2])
sage: L
Simple module with highest weight Lambda[1] + Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
sage: L = g.simple_module(-La[1] - La[2])
sage: L
Verma module with highest weight -Lambda[1] - Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> g = lie_algebras.sl(QQ, Integer(3))
>>> P = g.cartan_type().root_system().weight_lattice()
>>> La = P.fundamental_weights()
>>> L = g.simple_module(La[Integer(1)] + La[Integer(2)])
>>> L
Simple module with highest weight Lambda[1] + Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
>>> L = g.simple_module(-La[Integer(1)] - La[Integer(2)])
>>> L
Verma module with highest weight -Lambda[1] - Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
```

verma_module(*la*, *basis_key=None*, ***kwds*)

Return the Verma module with highest weight `la` over `self`.

INPUT:

- `basis_key` – (optional) a key function for the indexing set of the basis elements of `self`

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: L = lie_algebras.sl(QQ, 3)
sage: P = L.cartan_type().root_system().weight_lattice()
sage: La = P.fundamental_weights()
sage: M = L.verma_module(La[1] + La[2])
sage: M
Verma module with highest weight Lambda[1] + Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> L = lie_algebras.sl(QQ, Integer(3))
>>> P = L.cartan_type().root_system().weight_lattice()
>>> La = P.fundamental_weights()
>>> M = L.verma_module(La[Integer(1)] + La[Integer(2)])
>>> M
Verma module with highest weight Lambda[1] + Lambda[2]
 of Lie algebra of ['A', 2] in the Chevalley basis
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.triangular_kac_moody_algebras import␣
↪TriangularKacMoodyAlgebras
sage: TriangularKacMoodyAlgebras(QQ).super_categories()
[Join of Category of graded Lie algebras with basis over Rational Field
     and Category of kac moody algebras over Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.triangular_kac_moody_algebras import␣
↪TriangularKacMoodyAlgebras
>>> TriangularKacMoodyAlgebras(QQ).super_categories()
[Join of Category of graded Lie algebras with basis over Rational Field
     and Category of kac moody algebras over Rational Field]
```

4.167 Unique factorization domains

class sage.categories.unique_factorization_domains.**UniqueFactorizationDomains**

Bases: *Category_singleton*

The category of (constructive) unique factorization domains.

In a constructive unique factorization domain we can constructively factor members into a product of a finite number of irreducible elements.

EXAMPLES:

```
sage: UniqueFactorizationDomains()
Category of unique factorization domains
sage: UniqueFactorizationDomains().super_categories()
[Category of gcd domains]
```

```
>>> from sage.all import *
>>> UniqueFactorizationDomains()
Category of unique factorization domains
>>> UniqueFactorizationDomains().super_categories()
[Category of gcd domains]
```

class ElementMethods

Bases: object

radical(**args*, ***kwds*)

Return the radical of this element, i.e. the product of its irreducible factors.

This default implementation calls `squarefree_decomposition` if available, and `factor` otherwise.

See also

squarefree_part()

EXAMPLES:

```
sage: Pol.<x> = QQ[]
sage: (x^2*(x-1)^3).radical()
x^2 - x
sage: pol = 37 * (x-1)^3 * (x-2)^2 * (x-1/3)^7 * (x-3/7)
sage: pol.radical()
37*x^4 - 2923/21*x^3 + 1147/7*x^2 - 1517/21*x + 74/7

sage: Integer(10).radical()
10
sage: Integer(-100).radical()
10
sage: Integer(0).radical()
0
```

```
>>> from sage.all import *
>>> Pol = QQ['x']; (x,) = Pol._first_ngens(1)
>>> (x**Integer(2)*(x-Integer(1))**Integer(3)).radical()
x^2 - x
>>> pol = Integer(37) * (x-Integer(1))**Integer(3) * (x-
↪Integer(2))**Integer(2) * (x-Integer(1)/Integer(3))**Integer(7) * (x-
↪Integer(3)/Integer(7))
>>> pol.radical()
37*x^4 - 2923/21*x^3 + 1147/7*x^2 - 1517/21*x + 74/7

>>> Integer(Integer(10)).radical()
10
>>> Integer(-Integer(100)).radical()
10
>>> Integer(Integer(0)).radical()
0
```

The next example shows how to compute the radical of a number, assuming no prime > 100000 has exponent > 1 in the factorization:

```
sage: n = 2^1000-1; n / radical(n, limit=100000)
125
```

```
>>> from sage.all import *
>>> n = Integer(2)**Integer(1000)-Integer(1); n / radical(n,␣
↪limit=Integer(100000))
125
```

squarefree_part()

Return the square-free part of this element, i.e. the product of its irreducible factors appearing with odd multiplicity.

This default implementation calls `squarefree_decomposition`.

See also

radical()

EXAMPLES:

```
sage: Pol.<x> = QQ[]
sage: (x^2*(x-1)^3).squarefree_part()
x - 1
sage: pol = 37 * (x-1)^3 * (x-2)^2 * (x-1/3)^7 * (x-3/7)
sage: pol.squarefree_part()
37*x^3 - 1369/21*x^2 + 703/21*x - 37/7
```

```
>>> from sage.all import *
>>> Pol = QQ['x']; (x,) = Pol._first_ngens(1)
>>> (x**Integer(2)*(x-Integer(1))**Integer(3)).squarefree_part()
x - 1
>>> pol = Integer(37) * (x-Integer(1))**Integer(3) * (x-
↪Integer(2))**Integer(2) * (x-Integer(1)/Integer(3))**Integer(7) * (x-
↪Integer(3)/Integer(7))
>>> pol.squarefree_part()
37*x^3 - 1369/21*x^2 + 703/21*x - 37/7
```

class ParentMethods

Bases: `object`

is_unique_factorization_domain(*proof=True*)

Return `True`, since this in an object of the category of unique factorization domains.

EXAMPLES:

```
sage: UFD = UniqueFactorizationDomains()
sage: Parent(QQ, category=UFD).is_unique_factorization_domain()
True
```

```
>>> from sage.all import *
>>> UFD = UniqueFactorizationDomains()
>>> Parent(QQ, category=UFD).is_unique_factorization_domain()
True
```

additional_structure()

Return whether `self` is a structure category.

> **See also**
>
> *Category.additional_structure()*

The category of unique factorization domains does not define additional structure: a ring morphism between unique factorization domains is a unique factorization domain morphism.

EXAMPLES:

```
sage: UniqueFactorizationDomains().additional_structure()
```

```
>>> from sage.all import *
>>> UniqueFactorizationDomains().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: UniqueFactorizationDomains().super_categories()
[Category of gcd domains]
```

```
>>> from sage.all import *
>>> UniqueFactorizationDomains().super_categories()
[Category of gcd domains]
```

4.168 Unital algebras

class sage.categories.unital_algebras.**UnitalAlgebras**(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of non-associative algebras over a given base ring.

A non-associative algebra over a ring R is a module over R which is also a unital magma.

> **Warning**
>
> Until Issue #15043 is implemented, *Algebras* is the category of associative unital algebras; thus, unlike the name suggests, *UnitalAlgebras* is not a subcategory of *Algebras* but of *MagmaticAlgebras*.

EXAMPLES:

```
sage: from sage.categories.unital_algebras import UnitalAlgebras
sage: C = UnitalAlgebras(ZZ); C
Category of unital algebras over Integer Ring
```

```
>>> from sage.all import *
>>> from sage.categories.unital_algebras import UnitalAlgebras
>>> C = UnitalAlgebras(ZZ); C
Category of unital algebras over Integer Ring
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

The category of unital algebras constructed as Cartesian products of unital algebras.

This construction gives the direct product of algebras. See discussion on:

- http://groups.google.fr/group/sage-devel/browse_thread/thread/35a72b1d0a2fc77a/348f42ae77a66d16#348f42ae77a66d16
- Wikipedia article Direct_product

class ParentMethods

Bases: object

one()

Return the multiplicative unit element.

EXAMPLES:

```
sage: # needs sage.graphs sage.modules
sage: S2 = simplicial_complexes.Sphere(2)
sage: H = S2.cohomology_ring(QQ)
sage: C = cartesian_product([H, H])
sage: one = C.one()
sage: one
B[(0, (0, 0))] + B[(1, (0, 0))]
sage: one == one * one
True
sage: all(b == b * one for b in C.basis())
True
```

```
>>> from sage.all import *
>>> # needs sage.graphs sage.modules
>>> S2 = simplicial_complexes.Sphere(Integer(2))
>>> H = S2.cohomology_ring(QQ)
>>> C = cartesian_product([H, H])
>>> one = C.one()
>>> one
B[(0, (0, 0))] + B[(1, (0, 0))]
>>> one == one * one
True
>>> all(b == b * one for b in C.basis())
True
```

extra_super_categories()

A Cartesian product of algebras is endowed with a natural unital algebra structure.

EXAMPLES:

```
sage: from sage.categories.unital_algebras import UnitalAlgebras
sage: C = UnitalAlgebras(QQ).CartesianProducts()
sage: C.extra_super_categories()
[Category of unital algebras over Rational Field]
sage: sorted(C.super_categories(), key=str)
[Category of Cartesian products of distributive magmas and additive␣
```

(continues on next page)

(continued from previous page)

```
↪magmas,
 Category of Cartesian products of unital magmas,
 Category of Cartesian products of vector spaces over Rational Field,
 Category of unital algebras over Rational Field]
```

```
>>> from sage.all import *
>>> from sage.categories.unital_algebras import UnitalAlgebras
>>> C = UnitalAlgebras(QQ).CartesianProducts()
>>> C.extra_super_categories()
[Category of unital algebras over Rational Field]
>>> sorted(C.super_categories(), key=str)
[Category of Cartesian products of distributive magmas and additive␣
↪magmas,
 Category of Cartesian products of unital magmas,
 Category of Cartesian products of vector spaces over Rational Field,
 Category of unital algebras over Rational Field]
```

class ParentMethods

Bases: object

from_base_ring(*r*)

Return the canonical embedding of `r` into `self`.

INPUT:

- `r` – an element of `self.base_ring()`

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example(); A                                #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
sage: A.from_base_ring(1)                                                   #␣
↪needs sage.combinat sage.modules
B[word: ]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example(); A                                  #␣
↪needs sage.combinat sage.modules
An example of an algebra with basis:
 the free algebra on the generators ('a', 'b', 'c') over Rational Field
>>> A.from_base_ring(Integer(1))                                             ␣
↪     # needs sage.combinat sage.modules
B[word: ]
```

class WithBasis(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class ParentMethods

Bases: object

from_base_ring()

from_base_ring_from_one_basis(*r*)

Implement the canonical embedding from the ground ring.

INPUT:

- `r` – an element of the coefficient ring

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example()
sage: A.from_base_ring_from_one_basis(3)
3*B[word: ]
sage: A.from_base_ring(3)
3*B[word: ]
sage: A(3)
3*B[word: ]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example()
>>> A.from_base_ring_from_one_basis(Integer(3))
3*B[word: ]
>>> A.from_base_ring(Integer(3))
3*B[word: ]
>>> A(Integer(3))
3*B[word: ]
```

one()

Return the multiplicative unit element.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                                   #␣
↪needs sage.combinat sage.modules
sage: A.one_basis()                                                         #␣
↪needs sage.combinat sage.modules
word:
sage: A.one()                                                               #␣
↪needs sage.combinat sage.modules
B[word: ]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                     #␣
↪needs sage.combinat sage.modules
>>> A.one_basis()                                                           #␣
↪needs sage.combinat sage.modules
word:
>>> A.one()                                                                 #␣
↪needs sage.combinat sage.modules
B[word: ]
```

one_basis()

When the one of an algebra with basis is an element of this basis, this optional method can return the index of this element. This is used to provide a default implementation of *one()*, and an optimized default implementation of *from_base_ring()*.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example()
sage: A.one_basis()
word:
sage: A.one()
B[word: ]
sage: A.from_base_ring(4)
4*B[word: ]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example()
>>> A.one_basis()
word:
>>> A.one()
B[word: ]
>>> A.from_base_ring(Integer(4))
4*B[word: ]
```

one_from_one_basis()

Return the one of the algebra, as per *Magmas.Unital.ParentMethods.one()*

By default, this is implemented from *one_basis()*, if available.

EXAMPLES:

```
sage: # needs sage.combinat sage.modules
sage: A = AlgebrasWithBasis(QQ).example()
sage: A.one_basis()
word:
sage: A.one_from_one_basis()
B[word: ]
sage: A.one()
B[word: ]
```

```
>>> from sage.all import *
>>> # needs sage.combinat sage.modules
>>> A = AlgebrasWithBasis(QQ).example()
>>> A.one_basis()
word:
>>> A.one_from_one_basis()
B[word: ]
>>> A.one()
B[word: ]
```

4.169 Vector Bundles

class sage.categories.vector_bundles.**VectorBundles**(*base_space*, *base_field*, *name=None*)

Bases: *Category_over_base_ring*

The category of vector bundles over any base space and base field.

See also

TopologicalVectorBundle

EXAMPLES:

```
sage: M = Manifold(2, 'M', structure='top')
sage: from sage.categories.vector_bundles import VectorBundles
sage: C = VectorBundles(M, RR); C
Category of vector bundles over Real Field with 53 bits of precision
 with base space 2-dimensional topological manifold M
sage: C.super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> M = Manifold(Integer(2), 'M', structure='top')
>>> from sage.categories.vector_bundles import VectorBundles
>>> C = VectorBundles(M, RR); C
Category of vector bundles over Real Field with 53 bits of precision
 with base space 2-dimensional topological manifold M
>>> C.super_categories()
[Category of topological spaces]
```

class Differentiable(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of differentiable vector bundles.

A differentiable vector bundle is a differentiable manifold with differentiable surjective projection on a differentiable base space.

class Smooth(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

The category of smooth vector bundles.

A smooth vector bundle is a smooth manifold with smooth surjective projection on a smooth base space.

class SubcategoryMethods

Bases: object

Differentiable()

Return the subcategory of the differentiable objects of `self`.

EXAMPLES:

```
sage: M = Manifold(2, 'M')
sage: from sage.categories.vector_bundles import VectorBundles
sage: VectorBundles(M, RR).Differentiable()
Category of differentiable vector bundles over Real Field with
 53 bits of precision with base space 2-dimensional
 differentiable manifold M
```

```
>>> from sage.all import *
>>> M = Manifold(Integer(2), 'M')
```

(continues on next page)

(continued from previous page)

```
>>> from sage.categories.vector_bundles import VectorBundles
>>> VectorBundles(M, RR).Differentiable()
Category of differentiable vector bundles over Real Field with
 53 bits of precision with base space 2-dimensional
 differentiable manifold M
```

Smooth()

Return the subcategory of the smooth objects of `self`.

EXAMPLES:

```
sage: M = Manifold(2, 'M')
sage: from sage.categories.vector_bundles import VectorBundles
sage: VectorBundles(M, RR).Smooth()
Category of smooth vector bundles over Real Field with 53 bits
 of precision with base space 2-dimensional differentiable
 manifold M
```

```
>>> from sage.all import *
>>> M = Manifold(Integer(2), 'M')
>>> from sage.categories.vector_bundles import VectorBundles
>>> VectorBundles(M, RR).Smooth()
Category of smooth vector bundles over Real Field with 53 bits
 of precision with base space 2-dimensional differentiable
 manifold M
```

base_space()

Return the base space of this category.

EXAMPLES:

```
sage: M = Manifold(2, 'M', structure='top')
sage: from sage.categories.vector_bundles import VectorBundles
sage: VectorBundles(M, RR).base_space()
2-dimensional topological manifold M
```

```
>>> from sage.all import *
>>> M = Manifold(Integer(2), 'M', structure='top')
>>> from sage.categories.vector_bundles import VectorBundles
>>> VectorBundles(M, RR).base_space()
2-dimensional topological manifold M
```

super_categories()

EXAMPLES:

```
sage: M = Manifold(2, 'M')
sage: from sage.categories.vector_bundles import VectorBundles
sage: VectorBundles(M, RR).super_categories()
[Category of topological spaces]
```

```
>>> from sage.all import *
>>> M = Manifold(Integer(2), 'M')
```

(continues on next page)

(continued from previous page)

```
>>> from sage.categories.vector_bundles import VectorBundles
>>> VectorBundles(M, RR).super_categories()
[Category of topological spaces]
```

4.170 Vector Spaces

class sage.categories.vector_spaces.**VectorSpaces**(*K*)

Bases: *Category_module*

The category of (abstract) vector spaces over a given field.

??? with an embedding in an ambient vector space ???

EXAMPLES:

```
sage: VectorSpaces(QQ)
Category of vector spaces over Rational Field
sage: VectorSpaces(QQ).super_categories()
[Category of modules over Rational Field]
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ)
Category of vector spaces over Rational Field
>>> VectorSpaces(QQ).super_categories()
[Category of modules over Rational Field]
```

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

The category of vector spaces is closed under Cartesian products:

```
sage: C = VectorSpaces(QQ)
sage: C.CartesianProducts()
Category of Cartesian products of vector spaces over Rational Field
sage: C in C.CartesianProducts().super_categories()
True
```

```
>>> from sage.all import *
>>> C = VectorSpaces(QQ)
>>> C.CartesianProducts()
Category of Cartesian products of vector spaces over Rational Field
>>> C in C.CartesianProducts().super_categories()
True
```

class DualObjects(*category*, **args*)

Bases: *DualObjectsCategory*

extra_super_categories()

Return the dual category.

EXAMPLES:

The category of algebras over the Rational Field is dual to the category of coalgebras over the same field:

```
sage: C = VectorSpaces(QQ)
sage: C.dual()
Category of duals of vector spaces over Rational Field
sage: C.dual().super_categories() # indirect doctest
[Category of vector spaces over Rational Field]
```

```
>>> from sage.all import *
>>> C = VectorSpaces(QQ)
>>> C.dual()
Category of duals of vector spaces over Rational Field
>>> C.dual().super_categories() # indirect doctest
[Category of vector spaces over Rational Field]
```

class ElementMethods

Bases: object

class Filtered(*base_category*)

Bases: *FilteredModulesCategory*

Category of filtered vector spaces.

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

Implement the fact that a (finite) tensor product of finite dimensional vector spaces is a finite dimensional vector space.

EXAMPLES:

```
sage: VectorSpaces(QQ).FiniteDimensional().TensorProducts().extra_
↪super_categories()
[Category of finite dimensional vector spaces over Rational Field]
sage: VectorSpaces(QQ).FiniteDimensional().TensorProducts().
↪FiniteDimensional()
Category of tensor products of finite dimensional vector spaces over␣
↪Rational Field
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).FiniteDimensional().TensorProducts().extra_super_
↪categories()
[Category of finite dimensional vector spaces over Rational Field]
>>> VectorSpaces(QQ).FiniteDimensional().TensorProducts().
↪FiniteDimensional()
Category of tensor products of finite dimensional vector spaces over␣
↪Rational Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

Category of graded vector spaces.

class ParentMethods

Bases: object

dimension()

Return the dimension of this vector space.

EXAMPLES:

```
sage: M = FreeModule(FiniteField(19), 100)                                  #␣
↪needs sage.modules
sage: W = M.submodule([M.gen(50)])                                          #␣
↪needs sage.modules
sage: W.dimension()                                                         #␣
↪needs sage.modules
1

sage: M = FiniteRankFreeModule(QQ, 3)                                       #␣
↪needs sage.modules
sage: M.dimension()                                                         #␣
↪needs sage.modules
3
sage: M.tensor_module(1, 2).dimension()                                     #␣
↪needs sage.modules
27
```

```
>>> from sage.all import *
>>> M = FreeModule(FiniteField(Integer(19)), Integer(100))                  ␣
↪              # needs sage.modules
>>> W = M.submodule([M.gen(Integer(50))])                                   ␣
↪     # needs sage.modules
>>> W.dimension()                                                          #␣
↪needs sage.modules
1

>>> M = FiniteRankFreeModule(QQ, Integer(3))                                ␣
↪     # needs sage.modules
>>> M.dimension()                                                          #␣
↪needs sage.modules
3
>>> M.tensor_module(Integer(1), Integer(2)).dimension()                     ␣
↪              # needs sage.modules
27
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

The category of vector spaces is closed under tensor products:

```
sage: C = VectorSpaces(QQ)
sage: C.TensorProducts()
Category of tensor products of vector spaces over Rational Field
sage: C in C.TensorProducts().super_categories()
True
```

```
>>> from sage.all import *
>>> C = VectorSpaces(QQ)
>>> C.TensorProducts()
Category of tensor products of vector spaces over Rational Field
>>> C in C.TensorProducts().super_categories()
True
```

class WithBasis(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class CartesianProducts(*category*, **args*)

Bases: *CartesianProductsCategory*

extra_super_categories()

The category of vector spaces with basis is closed under Cartesian products:

```
sage: C = VectorSpaces(QQ).WithBasis()
sage: C.CartesianProducts()
Category of Cartesian products of vector spaces with basis over␣
↪Rational Field
sage: C in C.CartesianProducts().super_categories()
True
```

```
>>> from sage.all import *
>>> C = VectorSpaces(QQ).WithBasis()
>>> C.CartesianProducts()
Category of Cartesian products of vector spaces with basis over␣
↪Rational Field
>>> C in C.CartesianProducts().super_categories()
True
```

class Filtered(*base_category*)

Bases: *FilteredModulesCategory*

Category of filtered vector spaces with basis.

example(*base_ring=None*)

Return an example of a graded vector space with basis, as per *Category.example()*.

EXAMPLES:

```
sage: Modules(QQ).WithBasis().Graded().example()                    #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Rational Field
```

```
>>> from sage.all import *
>>> Modules(QQ).WithBasis().Graded().example()                    #␣
↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Rational Field
```

class FiniteDimensional(*base_category*)

Bases: *CategoryWithAxiom_over_base_ring*

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

Implement the fact that a (finite) tensor product of finite dimensional vector spaces is a finite dimensional vector space.

EXAMPLES:

```
sage: VectorSpaces(QQ).WithBasis().FiniteDimensional().
 ↪TensorProducts().extra_super_categories()
[Category of finite dimensional vector spaces with basis over␣
 ↪Rational Field]
sage: VectorSpaces(QQ).WithBasis().FiniteDimensional().
 ↪TensorProducts().FiniteDimensional()
Category of tensor products of finite dimensional vector spaces with␣
 ↪basis over Rational Field
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).WithBasis().FiniteDimensional().
 ↪TensorProducts().extra_super_categories()
[Category of finite dimensional vector spaces with basis over␣
 ↪Rational Field]
>>> VectorSpaces(QQ).WithBasis().FiniteDimensional().
 ↪TensorProducts().FiniteDimensional()
Category of tensor products of finite dimensional vector spaces with␣
 ↪basis over Rational Field
```

class Graded(*base_category*)

Bases: *GradedModulesCategory*

Category of graded vector spaces with basis.

example(*base_ring=None*)

Return an example of a graded vector space with basis, as per *Category.example()*.

EXAMPLES:

```
sage: Modules(QQ).WithBasis().Graded().example()                    #␣
 ↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Rational Field
```

```
>>> from sage.all import *
>>> Modules(QQ).WithBasis().Graded().example()                    #␣
 ↪needs sage.combinat sage.modules
An example of a graded module with basis:
 the free module on partitions over Rational Field
```

class TensorProducts(*category*, **args*)

Bases: *TensorProductsCategory*

extra_super_categories()

The category of vector spaces with basis is closed under tensor products:

```
sage: C = VectorSpaces(QQ).WithBasis()
sage: C.TensorProducts()
Category of tensor products of vector spaces with basis over Rational␣
↪Field
sage: C in C.TensorProducts().super_categories()
True
```

```
>>> from sage.all import *
>>> C = VectorSpaces(QQ).WithBasis()
>>> C.TensorProducts()
Category of tensor products of vector spaces with basis over Rational␣
↪Field
>>> C in C.TensorProducts().super_categories()
True
```

is_abelian()

Return whether this category is abelian.

This is always `True` since the base ring is a field.

EXAMPLES:

```
sage: VectorSpaces(QQ).WithBasis().is_abelian()
True
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).WithBasis().is_abelian()
True
```

additional_structure()

Return `None`.

Indeed, the category of vector spaces defines no additional structure: a bimodule morphism between two vector spaces is a vector space morphism.

See also

additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: VectorSpaces(QQ).additional_structure()
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).additional_structure()
```

base_field()

Return the base field over which the vector spaces of this category are all defined.

EXAMPLES:

```
sage: VectorSpaces(QQ).base_field()
Rational Field
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).base_field()
Rational Field
```

super_categories()

EXAMPLES:

```
sage: VectorSpaces(QQ).super_categories()
[Category of modules over Rational Field]
```

```
>>> from sage.all import *
>>> VectorSpaces(QQ).super_categories()
[Category of modules over Rational Field]
```

4.171 Weyl Groups

class sage.categories.weyl_groups.**WeylGroups**

Bases: *Category_singleton*

The category of Weyl groups.

See the Wikipedia page of Weyl Groups.

EXAMPLES:

```
sage: WeylGroups()
Category of Weyl groups
sage: WeylGroups().super_categories()
[Category of Coxeter groups]
```

```
>>> from sage.all import *
>>> WeylGroups()
Category of Weyl groups
>>> WeylGroups().super_categories()
[Category of Coxeter groups]
```

Here are some examples:

```
sage: WeylGroups().example()            # todo: not implemented
sage: FiniteWeylGroups().example()
The symmetric group on {0, ..., 3}
sage: AffineWeylGroups().example()      # todo: not implemented
sage: WeylGroup(["B", 3])
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)
```

```
>>> from sage.all import *
>>> WeylGroups().example()            # todo: not implemented
>>> FiniteWeylGroups().example()
The symmetric group on {0, ..., 3}
>>> AffineWeylGroups().example()      # todo: not implemented
>>> WeylGroup(["B", Integer(3)])
Weyl Group of type ['B', 3] (as a matrix group acting on the ambient space)
```

This one will eventually be also in this category:

```
sage: SymmetricGroup(4)
Symmetric group of order 4! as a permutation group
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(4))
Symmetric group of order 4! as a permutation group
```

class ElementMethods

Bases: object

bruhat_lower_covers_coroots()

Return all 2-tuples (v, α) where v is covered by self and α is the positive coroot such that self = v s_α where s_α is the reflection orthogonal to α.

ALGORITHM:

See *bruhat_lower_covers()* and *bruhat_lower_covers_reflections()* for Coxeter groups.

EXAMPLES:

```
sage: W = WeylGroup(['A',3], prefix='s')
sage: w = W.from_reduced_word([3,1,2,1])
sage: w.bruhat_lower_covers_coroots()
[(s1*s2*s1, alphacheck[1] + alphacheck[2] + alphacheck[3]),
 (s3*s2*s1, alphacheck[2]), (s3*s1*s2, alphacheck[1])]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)], prefix='s')
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(1)])
>>> w.bruhat_lower_covers_coroots()
[(s1*s2*s1, alphacheck[1] + alphacheck[2] + alphacheck[3]),
 (s3*s2*s1, alphacheck[2]), (s3*s1*s2, alphacheck[1])]
```

bruhat_upper_covers_coroots()

Return all 2-tuples (v, α) where v is covers self and α is the positive coroot such that self = v s_α where s_α is the reflection orthogonal to α.

ALGORITHM:

See *bruhat_upper_covers()* and *bruhat_upper_covers_reflections()* for Coxeter groups.

EXAMPLES:

```
sage: W = WeylGroup(['A',4], prefix='s')
sage: w = W.from_reduced_word([3,1,2,1])
```

(continues on next page)

(continued from previous page)

```
sage: w.bruhat_upper_covers_coroots()
[(s1*s2*s3*s2*s1, alphacheck[3]),
 (s2*s3*s1*s2*s1, alphacheck[2] + alphacheck[3]),
 (s3*s4*s1*s2*s1, alphacheck[4]),
 (s4*s3*s1*s2*s1, alphacheck[1] + alphacheck[2] + alphacheck[3] +␣
↪alphacheck[4])]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(4)], prefix='s')
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(1)])
>>> w.bruhat_upper_covers_coroots()
[(s1*s2*s3*s2*s1, alphacheck[3]),
 (s2*s3*s1*s2*s1, alphacheck[2] + alphacheck[3]),
 (s3*s4*s1*s2*s1, alphacheck[4]),
 (s4*s3*s1*s2*s1, alphacheck[1] + alphacheck[2] + alphacheck[3] +␣
↪alphacheck[4])]
```

inversion_arrangement(*side='right'*)

Return the inversion hyperplane arrangement of `self`.

INPUT:

- `side` – `'right'` (default) or `'left'`

OUTPUT:

A (central) hyperplane arrangement whose hyperplanes correspond to the inversions of `self` given as roots.

The `side` parameter determines on which side to compute the inversions.

EXAMPLES:

```
sage: W = WeylGroup(['A',3])
sage: w = W.from_reduced_word([1, 2, 3, 1, 2])
sage: A = w.inversion_arrangement(); A
Arrangement of 5 hyperplanes of dimension 3 and rank 3
sage: A.hyperplanes()
(Hyperplane 0*a1 + 0*a2 + a3 + 0,
 Hyperplane 0*a1 + a2 + 0*a3 + 0,
 Hyperplane 0*a1 + a2 + a3 + 0,
 Hyperplane a1 + a2 + 0*a3 + 0,
 Hyperplane a1 + a2 + a3 + 0)
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)])
>>> w = W.from_reduced_word([Integer(1), Integer(2), Integer(3),␣
↪Integer(1), Integer(2)])
>>> A = w.inversion_arrangement(); A
Arrangement of 5 hyperplanes of dimension 3 and rank 3
>>> A.hyperplanes()
(Hyperplane 0*a1 + 0*a2 + a3 + 0,
 Hyperplane 0*a1 + a2 + 0*a3 + 0,
 Hyperplane 0*a1 + a2 + a3 + 0,
 Hyperplane a1 + a2 + 0*a3 + 0,
 Hyperplane a1 + a2 + a3 + 0)
```

The identity element gives the empty arrangement:

```
sage: W = WeylGroup(['A',3])
sage: W.one().inversion_arrangement()
Empty hyperplane arrangement of dimension 3
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)])
>>> W.one().inversion_arrangement()
Empty hyperplane arrangement of dimension 3
```

inversions(*side='right'*, *inversion_type='reflections'*)

Return the set of inversions of `self`.

INPUT:

- `side` – 'right' (default) or 'left'
- `inversion_type` – 'reflections' (default), 'roots', or 'coroots'

OUTPUT:

For reflections, the set of reflections r in the Weyl group such that `self r < self`. For (co)roots, the set of positive (co)roots that are sent by `self` to negative (co)roots; their associated reflections are described above.

If `side` is 'left', the inverse Weyl group element is used.

EXAMPLES:

```
sage: W = WeylGroup(['C',2], prefix='s')
sage: w = W.from_reduced_word([1,2])
sage: w.inversions()
[s2, s2*s1*s2]
sage: w.inversions(inversion_type = 'reflections')
[s2, s2*s1*s2]
sage: w.inversions(inversion_type = 'roots')
[alpha[2], alpha[1] + alpha[2]]
sage: w.inversions(inversion_type = 'coroots')
[alphacheck[2], alphacheck[1] + 2*alphacheck[2]]
sage: w.inversions(side = 'left')
[s1, s1*s2*s1]
sage: w.inversions(side = 'left', inversion_type = 'roots')
[alpha[1], 2*alpha[1] + alpha[2]]
sage: w.inversions(side = 'left', inversion_type = 'coroots')
[alphacheck[1], alphacheck[1] + alphacheck[2]]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['C',Integer(2)], prefix='s')
>>> w = W.from_reduced_word([Integer(1),Integer(2)])
>>> w.inversions()
[s2, s2*s1*s2]
>>> w.inversions(inversion_type = 'reflections')
[s2, s2*s1*s2]
>>> w.inversions(inversion_type = 'roots')
[alpha[2], alpha[1] + alpha[2]]
>>> w.inversions(inversion_type = 'coroots')
[alphacheck[2], alphacheck[1] + 2*alphacheck[2]]
```

(continues on next page)

(continued from previous page)

```
>>> w.inversions(side = 'left')
[s1, s1*s2*s1]
>>> w.inversions(side = 'left', inversion_type = 'roots')
[alpha[1], 2*alpha[1] + alpha[2]]
>>> w.inversions(side = 'left', inversion_type = 'coroots')
[alphacheck[1], alphacheck[1] + alphacheck[2]]
```

is_pieri_factor()

Return whether `self` is a Pieri factor, as used for computing Stanley symmetric functions.

See also

- *stanley_symmetric_function()*
- *WeylGroups.ParentMethods.pieri_factors()*

EXAMPLES:

```
sage: W = WeylGroup(['A',5,1])
sage: W.from_reduced_word([3,2,5]).is_pieri_factor()
True
sage: W.from_reduced_word([3,2,4,5]).is_pieri_factor()
False

sage: W = WeylGroup(['C',4,1])
sage: W.from_reduced_word([0,2,1]).is_pieri_factor()
True
sage: W.from_reduced_word([0,2,1,0]).is_pieri_factor()
False

sage: W = WeylGroup(['B',3])
sage: W.from_reduced_word([3,2,3]).is_pieri_factor()
False
sage: W.from_reduced_word([2,1,2]).is_pieri_factor()
True
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(5),Integer(1)])
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(5)]).is_pieri_
↪factor()
True
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(4),Integer(5)]).is_
↪pieri_factor()
False

>>> W = WeylGroup(['C',Integer(4),Integer(1)])
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(1)]).is_pieri_
↪factor()
True
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)]).is_
↪pieri_factor()
```

(continues on next page)

(continued from previous page)

```
False

>>> W = WeylGroup(['B',Integer(3)])
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(3)]).is_pieri_
↪factor()
False
>>> W.from_reduced_word([Integer(2),Integer(1),Integer(2)]).is_pieri_
↪factor()
True
```

left_pieri_factorizations(*max_length=None*)

Return all factorizations of `self` as uv, where u is a Pieri factor and v is an element of the Weyl group.

See also

- *WeylGroups.ParentMethods.pieri_factors()*
- sage.combinat.root_system.pieri_factors

EXAMPLES:

If we take $w = w_0$ the maximal element of a strict parabolic subgroup of type $A_{n_1} \times \cdots \times A_{n_k}$, then the Pieri factorizations are in correspondence with all Pieri factors, and there are $\prod 2^{n_i}$ of them:

```
sage: W = WeylGroup(['A', 4, 1])
sage: W.from_reduced_word([]).left_pieri_factorizations().cardinality()
1
sage: W.from_reduced_word([1]).left_pieri_factorizations().cardinality()
2
sage: W.from_reduced_word([1,2,1]).left_pieri_factorizations().
↪cardinality()
4
sage: W.from_reduced_word([1,2,3,1,2,1]).left_pieri_factorizations().
↪cardinality()
8

sage: W.from_reduced_word([1,3]).left_pieri_factorizations().cardinality()
4
sage: W.from_reduced_word([1,3,4,3]).left_pieri_factorizations().
↪cardinality()
8

sage: W.from_reduced_word([2,1]).left_pieri_factorizations().cardinality()
3
sage: W.from_reduced_word([1,2]).left_pieri_factorizations().cardinality()
2
sage: [W.from_reduced_word([1,2]).left_pieri_factorizations(max_length=i).
↪cardinality()
....:  for i in [-1, 0, 1, 2]]
[0, 1, 2, 2]

sage: W = WeylGroup(['C',4,1])
```

(continues on next page)

(continued from previous page)

```
sage: w = W.from_reduced_word([0,3,2,1,0])
sage: w.left_pieri_factorizations().cardinality()
7
sage: [(u.reduced_word(),v.reduced_word())
....:  for (u,v) in w.left_pieri_factorizations()]
[([], [3, 2, 0, 1, 0]),
([0], [3, 2, 1, 0]),
([3], [2, 0, 1, 0]),
([3, 0], [2, 1, 0]),
([3, 2], [0, 1, 0]),
([3, 2, 0], [1, 0]),
([3, 2, 0, 1], [0])]

sage: W = WeylGroup(['B',4,1])
sage: W.from_reduced_word([0,2,1,0]).left_pieri_factorizations().
↪cardinality()
6
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(4), Integer(1)])
>>> W.from_reduced_word([]).left_pieri_factorizations().cardinality()
1
>>> W.from_reduced_word([Integer(1)]).left_pieri_factorizations().
↪cardinality()
2
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1)]).left_pieri_
↪factorizations().cardinality()
4
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(1),
↪Integer(2),Integer(1)]).left_pieri_factorizations().cardinality()
8

>>> W.from_reduced_word([Integer(1),Integer(3)]).left_pieri_
↪factorizations().cardinality()
4
>>> W.from_reduced_word([Integer(1),Integer(3),Integer(4),Integer(3)]).
↪left_pieri_factorizations().cardinality()
8

>>> W.from_reduced_word([Integer(2),Integer(1)]).left_pieri_
↪factorizations().cardinality()
3
>>> W.from_reduced_word([Integer(1),Integer(2)]).left_pieri_
↪factorizations().cardinality()
2
>>> [W.from_reduced_word([Integer(1),Integer(2)]).left_pieri_
↪factorizations(max_length=i).cardinality()
...  for i in [-Integer(1), Integer(0), Integer(1), Integer(2)]]
[0, 1, 2, 2]

>>> W = WeylGroup(['C',Integer(4),Integer(1)])
>>> w = W.from_reduced_word([Integer(0),Integer(3),Integer(2),Integer(1),
```

(continues on next page)

(continued from previous page)

```
↪Integer(0)])
>>> w.left_pieri_factorizations().cardinality()
7
>>> [(u.reduced_word(),v.reduced_word())
...  for (u,v) in w.left_pieri_factorizations()]
[([], [3, 2, 0, 1, 0]),
([0], [3, 2, 1, 0]),
([3], [2, 0, 1, 0]),
([3, 0], [2, 1, 0]),
([3, 2], [0, 1, 0]),
([3, 2, 0], [1, 0]),
([3, 2, 0, 1], [0])]

>>> W = WeylGroup(['B',Integer(4),Integer(1)])
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)]).
↪left_pieri_factorizations().cardinality()
6
```

quantum_bruhat_successors(*index_set=None*, *roots=False*, *quantum_only=False*)

Return the successors of `self` in the quantum Bruhat graph on the parabolic quotient of the Weyl group determined by the subset of Dynkin nodes `index_set`.

INPUT:

- `self` – a Weyl group element, which is assumed to be of minimum length in its coset with respect to the parabolic subgroup
- `index_set` – (default: `None`) indicates the set of simple reflections used to generate the parabolic subgroup; the default value indicates that the subgroup is the identity
- `roots` – boolean (default: `False`); if `True`, returns the list of 2-tuples (`w`, α) where `w` is a successor and α is the positive root associated with the successor relation
- `quantum_only` – boolean (default: `False`); if `True`, returns only the quantum successors

EXAMPLES:

```
sage: W = WeylGroup(['A',3], prefix='s')
sage: w = W.from_reduced_word([3,1,2])
sage: w.quantum_bruhat_successors([1], roots = True)
[(s3, alpha[2]), (s1*s2*s3*s2, alpha[3]),
 (s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3])]
sage: w.quantum_bruhat_successors([1,3])
[1, s2*s3*s1*s2]
sage: w.quantum_bruhat_successors(roots = True)
[(s3*s1*s2*s1, alpha[1]),
 (s3*s1, alpha[2]),
 (s1*s2*s3*s2, alpha[3]),
 (s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3])]
sage: w.quantum_bruhat_successors()
[s3*s1*s2*s1, s3*s1, s1*s2*s3*s2, s2*s3*s1*s2]
sage: w.quantum_bruhat_successors(quantum_only = True)
[s3*s1]
sage: w = W.from_reduced_word([2,3])
sage: w.quantum_bruhat_successors([1,3])
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: s2*s3 is not of minimum length in its coset
of the parabolic subgroup generated by the reflections (1, 3)
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)], prefix='s')
>>> w = W.from_reduced_word([Integer(3),Integer(1),Integer(2)])
>>> w.quantum_bruhat_successors([Integer(1)], roots = True)
[(s3, alpha[2]), (s1*s2*s3*s2, alpha[3]),
 (s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3])]
>>> w.quantum_bruhat_successors([Integer(1),Integer(3)])
[1, s2*s3*s1*s2]
>>> w.quantum_bruhat_successors(roots = True)
[(s3*s1*s2*s1, alpha[1]),
 (s3*s1, alpha[2]),
 (s1*s2*s3*s2, alpha[3]),
 (s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3])]
>>> w.quantum_bruhat_successors()
[s3*s1*s2*s1, s3*s1, s1*s2*s3*s2, s2*s3*s1*s2]
>>> w.quantum_bruhat_successors(quantum_only = True)
[s3*s1]
>>> w = W.from_reduced_word([Integer(2),Integer(3)])
>>> w.quantum_bruhat_successors([Integer(1),Integer(3)])
Traceback (most recent call last):
...
ValueError: s2*s3 is not of minimum length in its coset
of the parabolic subgroup generated by the reflections (1, 3)
```

reflection_to_coroot()

Return the coroot associated with the reflection `self`.

EXAMPLES:

```
sage: W = WeylGroup(['C',2], prefix='s')
sage: W.from_reduced_word([1,2,1]).reflection_to_coroot()
alphacheck[1] + alphacheck[2]
sage: W.from_reduced_word([1,2]).reflection_to_coroot()
Traceback (most recent call last):
...
ValueError: s1*s2 is not a reflection
sage: W.long_element().reflection_to_coroot()
Traceback (most recent call last):
...
ValueError: s2*s1*s2*s1 is not a reflection
```

```
>>> from sage.all import *
>>> W = WeylGroup(['C',Integer(2)], prefix='s')
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1)]).reflection_to_
↪coroot()
alphacheck[1] + alphacheck[2]
>>> W.from_reduced_word([Integer(1),Integer(2)]).reflection_to_coroot()
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
ValueError: s1*s2 is not a reflection
>>> W.long_element().reflection_to_coroot()
Traceback (most recent call last):
...
ValueError: s2*s1*s2*s1 is not a reflection
```

reflection_to_root()

Return the root associated with the reflection `self`.

EXAMPLES:

```
sage: W = WeylGroup(['C',2], prefix='s')
sage: W.from_reduced_word([1,2,1]).reflection_to_root()
2*alpha[1] + alpha[2]
sage: W.from_reduced_word([1,2]).reflection_to_root()
Traceback (most recent call last):
...
ValueError: s1*s2 is not a reflection
sage: W.long_element().reflection_to_root()
Traceback (most recent call last):
...
ValueError: s2*s1*s2*s1 is not a reflection
```

```
>>> from sage.all import *
>>> W = WeylGroup(['C',Integer(2)], prefix='s')
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1)]).reflection_to_
↪root()
2*alpha[1] + alpha[2]
>>> W.from_reduced_word([Integer(1),Integer(2)]).reflection_to_root()
Traceback (most recent call last):
...
ValueError: s1*s2 is not a reflection
>>> W.long_element().reflection_to_root()
Traceback (most recent call last):
...
ValueError: s2*s1*s2*s1 is not a reflection
```

stanley_symmetric_function()

Return the affine Stanley symmetric function indexed by `self`.

INPUT:

- `self` – an element w of a Weyl group

Returns the affine Stanley symmetric function indexed by w. Stanley symmetric functions are defined as generating series of the factorizations of w into Pieri factors and weighted by a statistic on Pieri factors.

See also

- *stanley_symmetric_function_as_polynomial()*
- *WeylGroups.ParentMethods.pieri_factors()*
- sage.combinat.root_system.pieri_factors

EXAMPLES:

```
sage: W = WeylGroup(['A', 3, 1])
sage: W.from_reduced_word([3,1,2,0,3,1,0]).stanley_symmetric_function()
8*m[1, 1, 1, 1, 1, 1, 1] + 4*m[2, 1, 1, 1, 1, 1]
+ 2*m[2, 2, 1, 1, 1] + m[2, 2, 2, 1]
sage: A = AffinePermutationGroup(['A',3,1])
sage: A.from_reduced_word([3,1,2,0,3,1,0]).stanley_symmetric_function()
8*m[1, 1, 1, 1, 1, 1, 1] + 4*m[2, 1, 1, 1, 1, 1]
+ 2*m[2, 2, 1, 1, 1] + m[2, 2, 2, 1]

sage: W = WeylGroup(['C',3,1])
sage: W.from_reduced_word([0,2,1,0]).stanley_symmetric_function()
32*m[1, 1, 1, 1] + 16*m[2, 1, 1] + 8*m[2, 2] + 4*m[3, 1]

sage: W = WeylGroup(['B',3,1])
sage: W.from_reduced_word([3,2,1]).stanley_symmetric_function()
2*m[1, 1, 1] + m[2, 1] + 1/2*m[3]

sage: W = WeylGroup(['B',4])
sage: w = W.from_reduced_word([3,2,3,1])
sage: w.stanley_symmetric_function()  # long time (6s on sage.math, 2011)
48*m[1, 1, 1, 1] + 24*m[2, 1, 1] + 12*m[2, 2] + 8*m[3, 1] + 2*m[4]

sage: A = AffinePermutationGroup(['A',4,1])
sage: a = A([-2,0,1,4,12])
sage: a.stanley_symmetric_function()
6*m[1, 1, 1, 1, 1, 1, 1, 1] + 5*m[2, 1, 1, 1, 1, 1, 1]
+ 4*m[2, 2, 1, 1, 1, 1] + 3*m[2, 2, 2, 1, 1] + 2*m[2, 2, 2, 2]
+ 4*m[3, 1, 1, 1, 1, 1] + 3*m[3, 2, 1, 1, 1] + 2*m[3, 2, 2, 1]
+ 2*m[3, 3, 1, 1] + m[3, 3, 2] + 3*m[4, 1, 1, 1, 1]
+ 2*m[4, 2, 1, 1] + m[4, 2, 2] + m[4, 3, 1]
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3), Integer(1)])
>>> W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(0),
↪Integer(3),Integer(1),Integer(0)]).stanley_symmetric_function()
8*m[1, 1, 1, 1, 1, 1, 1] + 4*m[2, 1, 1, 1, 1, 1]
+ 2*m[2, 2, 1, 1, 1] + m[2, 2, 2, 1]
>>> A = AffinePermutationGroup(['A',Integer(3),Integer(1)])
>>> A.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(0),
↪Integer(3),Integer(1),Integer(0)]).stanley_symmetric_function()
8*m[1, 1, 1, 1, 1, 1, 1] + 4*m[2, 1, 1, 1, 1, 1]
+ 2*m[2, 2, 1, 1, 1] + m[2, 2, 2, 1]

>>> W = WeylGroup(['C',Integer(3),Integer(1)])
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)]).
↪stanley_symmetric_function()
32*m[1, 1, 1, 1] + 16*m[2, 1, 1] + 8*m[2, 2] + 4*m[3, 1]

>>> W = WeylGroup(['B',Integer(3),Integer(1)])
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(1)]).stanley_
↪symmetric_function()
```

(continues on next page)

(continued from previous page)

```
2*m[1, 1, 1] + m[2, 1] + 1/2*m[3]

>>> W = WeylGroup(['B',Integer(4)])
>>> w = W.from_reduced_word([Integer(3),Integer(2),Integer(3),Integer(1)])
>>> w.stanley_symmetric_function()  # long time (6s on sage.math, 2011)
48*m[1, 1, 1, 1] + 24*m[2, 1, 1] + 12*m[2, 2] + 8*m[3, 1] + 2*m[4]

>>> A = AffinePermutationGroup(['A',Integer(4),Integer(1)])
>>> a = A([-Integer(2),Integer(0),Integer(1),Integer(4),Integer(12)])
>>> a.stanley_symmetric_function()
6*m[1, 1, 1, 1, 1, 1, 1, 1] + 5*m[2, 1, 1, 1, 1, 1, 1]
+ 4*m[2, 2, 1, 1, 1, 1] + 3*m[2, 2, 2, 1, 1] + 2*m[2, 2, 2, 2]
+ 4*m[3, 1, 1, 1, 1, 1] + 3*m[3, 2, 1, 1, 1] + 2*m[3, 2, 2, 1]
+ 2*m[3, 3, 1, 1] + m[3, 3, 2] + 3*m[4, 1, 1, 1, 1]
+ 2*m[4, 2, 1, 1] + m[4, 2, 2] + m[4, 3, 1]
```

One more example (Issue #14095):

```
sage: G = SymmetricGroup(4)
sage: w = G.from_reduced_word([3,2,3,1])
sage: w.stanley_symmetric_function()
3*m[1, 1, 1, 1] + 2*m[2, 1, 1] + m[2, 2] + m[3, 1]
```

```
>>> from sage.all import *
>>> G = SymmetricGroup(Integer(4))
>>> w = G.from_reduced_word([Integer(3),Integer(2),Integer(3),Integer(1)])
>>> w.stanley_symmetric_function()
3*m[1, 1, 1, 1] + 2*m[2, 1, 1] + m[2, 2] + m[3, 1]
```

REFERENCES:

- [BH1994]
- [Lam2008]
- [LSS2009]
- [Pon2010]

stanley_symmetric_function_as_polynomial(*max_length=None*)

Return a multivariate generating function for the number of factorizations of a Weyl group element into Pieri factors of decreasing length, weighted by a statistic on Pieri factors.

See also

- *stanley_symmetric_function()*
- *WeylGroups.ParentMethods.pieri_factors()*
- sage.combinat.root_system.pieri_factors

INPUT:

- `self` – an element w of a Weyl group W
- `max_length` – nonnegative integer or infinity (default: infinity)

Returns the generating series for the Pieri factorizations $w = u_1 \cdots u_k$, where u_i is a Pieri factor for all i, $l(w) = \sum_{i=1}^{k} l(u_i)$ and `max_length` $\geq l(u_1) \geq \cdots \geq l(u_k)$.

A factorization $u_1 \cdots u_k$ contributes a monomial of the form $\prod_i x_{l(u_i)}$, with coefficient given by $\prod_i 2^{c(u_i)}$, where c is a type-dependent statistic on Pieri factors, as returned by the method `u[i].stanley_symm_poly_weight()`.

EXAMPLES:

```
sage: W = WeylGroup(['A', 3, 1])
sage: W.from_reduced_word([]).stanley_symmetric_function_as_polynomial()
1
sage: W.from_reduced_word([1]).stanley_symmetric_function_as_polynomial()
x1
sage: W.from_reduced_word([1,2]).stanley_symmetric_function_as_
↪polynomial()
x1^2
sage: W.from_reduced_word([2,1]).stanley_symmetric_function_as_
↪polynomial()
x1^2 + x2
sage: W.from_reduced_word([1,2,1]).stanley_symmetric_function_as_
↪polynomial()
2*x1^3 + x1*x2
sage: W.from_reduced_word([1,2,1,0]).stanley_symmetric_function_as_
↪polynomial()
3*x1^4 + 2*x1^2*x2 + x2^2 + x1*x3
sage: x = W.from_reduced_word([1,2,3,1,2,1,0])
sage: x.stanley_symmetric_function_as_polynomial()  # long time
22*x1^7 + 11*x1^5*x2 + 5*x1^3*x2^2 + 3*x1^4*x3 + 2*x1*x2^3 + x1^2*x2*x3
sage: y = W.from_reduced_word([3,1,2,0,3,1,0])
sage: y.stanley_symmetric_function_as_polynomial()  # long time
8*x1^7 + 4*x1^5*x2 + 2*x1^3*x2^2 + x1*x2^3

sage: W = WeylGroup(['C',3,1])
sage: W.from_reduced_word([0,2,1,0]).stanley_symmetric_function_as_
↪polynomial()
32*x1^4 + 16*x1^2*x2 + 8*x2^2 + 4*x1*x3

sage: W = WeylGroup(['B',3,1])
sage: W.from_reduced_word([3,2,1]).stanley_symmetric_function_as_
↪polynomial()
2*x1^3 + x1*x2 + 1/2*x3
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A', Integer(3), Integer(1)])
>>> W.from_reduced_word([]).stanley_symmetric_function_as_polynomial()
1
>>> W.from_reduced_word([Integer(1)]).stanley_symmetric_function_as_
↪polynomial()
x1
>>> W.from_reduced_word([Integer(1),Integer(2)]).stanley_symmetric_
↪function_as_polynomial()
x1^2
>>> W.from_reduced_word([Integer(2),Integer(1)]).stanley_symmetric_
↪function_as_polynomial()
x1^2 + x2
```

(continues on next page)

(continued from previous page)

```
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1)]).stanley_
↪symmetric_function_as_polynomial()
2*x1^3 + x1*x2
>>> W.from_reduced_word([Integer(1),Integer(2),Integer(1),Integer(0)]).
↪stanley_symmetric_function_as_polynomial()
3*x1^4 + 2*x1^2*x2 + x2^2 + x1*x3
>>> x = W.from_reduced_word([Integer(1),Integer(2),Integer(3),Integer(1),
↪Integer(2),Integer(1),Integer(0)])
>>> x.stanley_symmetric_function_as_polynomial()  # long time
22*x1^7 + 11*x1^5*x2 + 5*x1^3*x2^2 + 3*x1^4*x3 + 2*x1*x2^3 + x1^2*x2*x3
>>> y = W.from_reduced_word([Integer(3),Integer(1),Integer(2),Integer(0),
↪Integer(3),Integer(1),Integer(0)])
>>> y.stanley_symmetric_function_as_polynomial()  # long time
8*x1^7 + 4*x1^5*x2 + 2*x1^3*x2^2 + x1*x2^3

>>> W = WeylGroup(['C',Integer(3),Integer(1)])
>>> W.from_reduced_word([Integer(0),Integer(2),Integer(1),Integer(0)]).
↪stanley_symmetric_function_as_polynomial()
32*x1^4 + 16*x1^2*x2 + 8*x2^2 + 4*x1*x3

>>> W = WeylGroup(['B',Integer(3),Integer(1)])
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(1)]).stanley_
↪symmetric_function_as_polynomial()
2*x1^3 + x1*x2 + 1/2*x3
```

Algorithm: Induction on the left Pieri factors. Note that this induction preserves subsets of W which are stable by taking right factors, and in particular Grassmanian elements.

Finite

alias of *FiniteWeylGroups*

class ParentMethods

Bases: object

bruhat_cone(*x*, *y*, *side='upper'*, *backend='cdd'*)

Return the (upper or lower) Bruhat cone associated to the interval `[x,y]`.

To a cover relation $v \prec w$ in strong Bruhat order you can assign a positive root β given by the unique reflection s_β such that $s_\beta v = w$.

The upper Bruhat cone of the interval $[x, y]$ is the non-empty, polyhedral cone generated by the roots corresponding to $x \prec a$ for all atoms a in the interval. The lower Bruhat cone of the interval $[x, y]$ is the non-empty, polyhedral cone generated by the roots corresponding to $c \prec y$ for all coatoms c in the interval.

INPUT:

- `x` – an element in the group W
- `y` – an element in the group W
- `side` – (default: `'upper'`) must be one of the following:
  - `'upper'` – return the upper Bruhat cone of the interval [`x`, `y`]
  - `'lower'` – return the lower Bruhat cone of the interval [`x`, `y`]
- `backend` – string (default: `'cdd'`); the backend to use to create the polyhedron

EXAMPLES:

```
sage: W = WeylGroup(['A',2])
sage: x = W.from_reduced_word([1])
sage: y = W.w0
sage: W.bruhat_cone(x, y)
A 2-dimensional polyhedron in QQ^3
 defined as the convex hull of 1 vertex and 2 rays

sage: W = WeylGroup(['E',6])
sage: x = W.one()
sage: y = W.w0
sage: W.bruhat_cone(x, y, side='lower')
A 6-dimensional polyhedron in QQ^8
 defined as the convex hull of 1 vertex and 6 rays
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(2)])
>>> x = W.from_reduced_word([Integer(1)])
>>> y = W.w0
>>> W.bruhat_cone(x, y)
A 2-dimensional polyhedron in QQ^3
 defined as the convex hull of 1 vertex and 2 rays

>>> W = WeylGroup(['E',Integer(6)])
>>> x = W.one()
>>> y = W.w0
>>> W.bruhat_cone(x, y, side='lower')
A 6-dimensional polyhedron in QQ^8
 defined as the convex hull of 1 vertex and 6 rays
```

REFERENCES:

- [Dy1994]
- [JS2021]

coxeter_matrix()

Return the Coxeter matrix associated to `self`.

EXAMPLES:

```
sage: G = WeylGroup(['A',3])
sage: G.coxeter_matrix()
[1 3 2]
[3 1 3]
[2 3 1]
```

```
>>> from sage.all import *
>>> G = WeylGroup(['A',Integer(3)])
>>> G.coxeter_matrix()
[1 3 2]
[3 1 3]
[2 3 1]
```

pieri_factors(**args*, ***keywords*)

Return the set of Pieri factors in this Weyl group.

For any type, the set of Pieri factors forms a lower ideal in Bruhat order, generated by all the conjugates of some special element of the Weyl group. In type A_n, this special element is $s_n \cdots s_1$, and the conjugates are obtained by rotating around this reduced word.

These are used to compute Stanley symmetric functions.

See also

- *WeylGroups.ElementMethods.stanley_symmetric_function()*
- sage.combinat.root_system.pieri_factors

EXAMPLES:

```
sage: W = WeylGroup(['A',5,1])
sage: PF = W.pieri_factors()
sage: PF.cardinality()
63

sage: W = WeylGroup(['B',3])
sage: PF = W.pieri_factors()
sage: sorted([w.reduced_word() for w in PF])
[[],
 [1],
 [1, 2],
 [1, 2, 1],
 [1, 2, 3],
 [1, 2, 3, 1],
 [1, 2, 3, 2],
 [1, 2, 3, 2, 1],
 [2],
 [2, 1],
 [2, 3],
 [2, 3, 1],
 [2, 3, 2],
 [2, 3, 2, 1],
 [3],
 [3, 1],
 [3, 1, 2],
 [3, 1, 2, 1],
 [3, 2],
 [3, 2, 1]]
sage: W = WeylGroup(['C',4,1])
sage: PF = W.pieri_factors()
sage: W.from_reduced_word([3,2,0]) in PF
True
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(5),Integer(1)])
>>> PF = W.pieri_factors()
>>> PF.cardinality()
63
```

(continues on next page)

(continued from previous page)

```
>>> W = WeylGroup(['B',Integer(3)])
>>> PF = W.pieri_factors()
>>> sorted([w.reduced_word() for w in PF])
[[],
 [1],
 [1, 2],
 [1, 2, 1],
 [1, 2, 3],
 [1, 2, 3, 1],
 [1, 2, 3, 2],
 [1, 2, 3, 2, 1],
 [2],
 [2, 1],
 [2, 3],
 [2, 3, 1],
 [2, 3, 2],
 [2, 3, 2, 1],
 [3],
 [3, 1],
 [3, 1, 2],
 [3, 1, 2, 1],
 [3, 2],
 [3, 2, 1]]
>>> W = WeylGroup(['C',Integer(4),Integer(1)])
>>> PF = W.pieri_factors()
>>> W.from_reduced_word([Integer(3),Integer(2),Integer(0)]) in PF
True
```

quantum_bruhat_graph(*index_set=()*)

Return the quantum Bruhat graph of the quotient of the Weyl group by a parabolic subgroup W_J.

INPUT:

- `index_set` – (default: ()) a tuple J of nodes of the Dynkin diagram

By default, the value for `index_set` indicates that the subgroup is trivial and the quotient is the full Weyl group.

EXAMPLES:

```
sage: W = WeylGroup(['A',3], prefix='s')
sage: g = W.quantum_bruhat_graph((1,3))
sage: g
Parabolic Quantum Bruhat Graph of Weyl Group of type ['A', 3]
 (as a matrix group acting on the ambient space)
 for nodes (1, 3): Digraph on 6 vertices
sage: g.vertices(sort=True)
[s2*s3*s1*s2, s3*s1*s2, s1*s2, s3*s2, s2, 1]
sage: g.edges(sort=True)
[(s2*s3*s1*s2, s2, alpha[2]),
 (s3*s1*s2, s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3]),
 (s3*s1*s2, 1, alpha[2]),
 (s1*s2, s3*s1*s2, alpha[2] + alpha[3]),
 (s3*s2, s3*s1*s2, alpha[1] + alpha[2]),
 (s2, s1*s2, alpha[1] + alpha[2]),
```

(continues on next page)

(continued from previous page)

```
 (s2, s3*s2, alpha[2] + alpha[3]),
 (1, s2, alpha[2])]
sage: W = WeylGroup(['A',3,1], prefix='s')
sage: g = W.quantum_bruhat_graph()
Traceback (most recent call last):
...
ValueError: the Cartan type ['A', 3, 1] is not finite
```

```
>>> from sage.all import *
>>> W = WeylGroup(['A',Integer(3)], prefix='s')
>>> g = W.quantum_bruhat_graph((Integer(1),Integer(3)))
>>> g
Parabolic Quantum Bruhat Graph of Weyl Group of type ['A', 3]
 (as a matrix group acting on the ambient space)
 for nodes (1, 3): Digraph on 6 vertices
>>> g.vertices(sort=True)
[s2*s3*s1*s2, s3*s1*s2, s1*s2, s3*s2, s2, 1]
>>> g.edges(sort=True)
[(s2*s3*s1*s2, s2, alpha[2]),
 (s3*s1*s2, s2*s3*s1*s2, alpha[1] + alpha[2] + alpha[3]),
 (s3*s1*s2, 1, alpha[2]),
 (s1*s2, s3*s1*s2, alpha[2] + alpha[3]),
 (s3*s2, s3*s1*s2, alpha[1] + alpha[2]),
 (s2, s1*s2, alpha[1] + alpha[2]),
 (s2, s3*s2, alpha[2] + alpha[3]),
 (1, s2, alpha[2])]
>>> W = WeylGroup(['A',Integer(3),Integer(1)], prefix='s')
>>> g = W.quantum_bruhat_graph()
Traceback (most recent call last):
...
ValueError: the Cartan type ['A', 3, 1] is not finite
```

additional_structure()

Return None.

Indeed, the category of Weyl groups defines no additional structure: Weyl groups are a special class of Coxeter groups.

See also

additional_structure()

Todo

Should this category be a *CategoryWithAxiom*?

EXAMPLES:

```
sage: WeylGroups().additional_structure()
```

```
>>> from sage.all import *
>>> WeylGroups().additional_structure()
```

super_categories()

EXAMPLES:

```
sage: WeylGroups().super_categories()
[Category of Coxeter groups]
```

```
>>> from sage.all import *
>>> WeylGroups().super_categories()
[Category of Coxeter groups]
```

4.172 Technical Categories

4.172.1 Facade Sets

For background, see *What is a facade set?*.

class sage.categories.facade_sets.**FacadeSets**(*base_category*)

Bases: *CategoryWithAxiom_singleton*

class ParentMethods

Bases: object

facade_for()

Return the parents this set is a facade for.

This default implementation assumes that `self` has an attribute `_facade_for`, typically initialized by `Parent`. If the attribute is not present, the method raises a `NotImplementedError`.

EXAMPLES:

```
sage: S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
sage: S.facade_for()
(Integer Ring,)
```

```
>>> from sage.all import *
>>> S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
>>> S.facade_for()
(Integer Ring,)
```

Check that Issue #13801 is corrected:

```
sage: class A(Parent):
....:     def __init__(self):
....:         Parent.__init__(self, category=Sets(), facade=True)
sage: a = A()
sage: a.facade_for()
Traceback (most recent call last):
...
```

(continues on next page)

(continued from previous page)

```
NotImplementedError: this parent did not specify which parents it is a␣
↪facade for
```

```
>>> from sage.all import *
>>> class A(Parent):
...     def __init__(self):
...         Parent.__init__(self, category=Sets(), facade=True)
>>> a = A()
>>> a.facade_for()
Traceback (most recent call last):
...
NotImplementedError: this parent did not specify which parents it is a␣
↪facade for
```

is_parent_of(*element*)

Return whether `self` is the parent of `element`.

INPUT:

- `element` – any object

Since `self` is a facade domain, this actually tests whether the parent of `element` is any of the parent `self` is a facade for.

EXAMPLES:

```
sage: S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
sage: S.is_parent_of(1)
True
sage: S.is_parent_of(1/2)
False
```

```
>>> from sage.all import *
>>> S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
>>> S.is_parent_of(Integer(1))
True
>>> S.is_parent_of(Integer(1)/Integer(2))
False
```

This method differs from `__contains__()` in two ways. First, this does not take into account the fact that `self` may be a strict subset of the parent(s) it is a facade for:

```
sage: -1 in S, S.is_parent_of(-1)
(False, True)
```

```
>>> from sage.all import *
>>> -Integer(1) in S, S.is_parent_of(-Integer(1))
(False, True)
```

Furthermore, there is no coercion attempted:

```
sage: int(1) in S, S.is_parent_of(int(1))
(True, False)
```

```
>>> from sage.all import *
>>> int(Integer(1)) in S, S.is_parent_of(int(Integer(1)))
(True, False)
```

Warning

this implementation does not handle facade parents of facade parents. Is this a feature we want generically?

example(*choice='subset'*)

Return an example of facade set, as per *Category.example()*.

INPUT:

- `choice` – 'union' or 'subset' (default: `'subset'`)

EXAMPLES:

```
sage: Sets().Facade().example()
An example of facade set: the monoid of positive integers
sage: Sets().Facade().example(choice='union')
An example of a facade set: the integers completed by +-infinity
sage: Sets().Facade().example(choice='subset')
An example of facade set: the monoid of positive integers
```

```
>>> from sage.all import *
>>> Sets().Facade().example()
An example of facade set: the monoid of positive integers
>>> Sets().Facade().example(choice='union')
An example of a facade set: the integers completed by +-infinity
>>> Sets().Facade().example(choice='subset')
An example of facade set: the monoid of positive integers
```

CHAPTER

FIVE

FUNCTORIAL CONSTRUCTIONS

5.1 Covariant Functorial Constructions

A *functorial construction* is a collection of functors $(F_{Cat})_{Cat}$ (indexed by a collection of categories) which associate to a sequence of parents $(A, B, ...)$ in a category Cat a parent $F_{Cat}(A, B, ...)$. Typical examples of functorial constructions are *CartesianProductFunctor* and *tensor*.

The category of $F_{Cat}(A, B, ...)$, which only depends on Cat, is called the (functorial) construction category.

A functorial construction is *(category)-covariant* if for every categories Cat and $SuperCat$, the category of $F_{Cat}(A, B, ...)$ is a subcategory of the category of $F_{SuperCat}(A, B, ...)$ whenever Cat is a subcategory of $SuperCat$. A functorial construction is *(category)-regressive* if the category of $F_{Cat}(A, B, ...)$ is a subcategory of Cat.

The goal of this module is to provide generic support for covariant functorial constructions. In particular, given some parents A, B, ..., in respective categories Cat_A, Cat_B, ..., it provides tools for calculating the best known category for the parent $F(A, B, ...)$. For examples, knowing that Cartesian products of semigroups (resp. monoids, groups) have a semigroup (resp. monoid, group) structure, and given a group B and two monoids A and C it can calculate that $A \times B \times C$ is naturally endowed with a monoid structure.

See *CovariantFunctorialConstruction*, *CovariantConstructionCategory* and *RegressiveCovariantConstructionCategory* for more details.

CovariantConstructionCategory.**extra_super_categories**()

Return the extra super categories of a construction category.

Default implementation which returns `[]`.

EXAMPLES:

```
sage: Sets().Subquotients().extra_super_categories()
[]
sage: Semigroups().Quotients().extra_super_categories()
[]
```

```
>>> from sage.all import *
>>> Sets().Subquotients().extra_super_categories()
[]
>>> Semigroups().Quotients().extra_super_categories()
[]
```

AUTHORS:

- Nicolas M. Thiery (2010): initial revision

class sage.categories.covariant_functorial_construction.**CovariantConstructionCategory**(*category*, **args*)

Bases: *FunctorialConstructionCategory*

Abstract class for categories F_{Cat} obtained through a covariant functorial construction

additional_structure()

Return the additional structure defined by `self`.

By default, a functorial construction category `A.F()` defines additional structure if and only if A is the category defining F. The rationale is that, for a subcategory B of A, the fact that $B.F()$ morphisms shall preserve the F-specific structure is already imposed by $A.F()$.

See also

- *Category.additional_structure()*.
- *is_construction_defined_by_base()*.

EXAMPLES:

```
sage: Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
sage: Algebras(ZZ).Graded().additional_structure()
```

```
>>> from sage.all import *
>>> Modules(ZZ).Graded().additional_structure()
Category of graded modules over Integer Ring
>>> Algebras(ZZ).Graded().additional_structure()
```

classmethod default_super_categories(*category*, **args*)

Return the default super categories of $F_{Cat}(A, B, ...)$ for $A, B, ...$ parents in Cat.

INPUT:

- `cls` – the category class for the functor F
- `category` – a category Cat
- `*args` – further arguments for the functor

OUTPUT: a (join) category

The default implementation is to return the join of the categories of $F(A, B, ...)$ for $A, B, ...$ in turn in each of the super categories of `category`.

This is implemented as a class method, in order to be able to reconstruct the functorial category associated to each of the super categories of `category`.

EXAMPLES:

Bialgebras are both algebras and coalgebras:

```
sage: Bialgebras(QQ).super_categories()
[Category of algebras over Rational Field,
 Category of coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> Bialgebras(QQ).super_categories()
[Category of algebras over Rational Field,
 Category of coalgebras over Rational Field]
```

Hence tensor products of bialgebras are tensor products of algebras and tensor products of coalgebras:

```
sage: Bialgebras(QQ).TensorProducts().super_categories()
[Category of tensor products of algebras over Rational Field,
 Category of tensor products of coalgebras over Rational Field]
```

```
>>> from sage.all import *
>>> Bialgebras(QQ).TensorProducts().super_categories()
[Category of tensor products of algebras over Rational Field,
 Category of tensor products of coalgebras over Rational Field]
```

Here is how *default_super_categories()* was called internally:

```
sage: C = sage.categories.tensor.TensorProductsCategory
sage: C.default_super_categories(Bialgebras(QQ))
Join of Category of tensor products of algebras over Rational Field
    and Category of tensor products of coalgebras over Rational Field
```

```
>>> from sage.all import *
>>> C = sage.categories.tensor.TensorProductsCategory
>>> C.default_super_categories(Bialgebras(QQ))
Join of Category of tensor products of algebras over Rational Field
    and Category of tensor products of coalgebras over Rational Field
```

We now show a similar example, with the `Algebra` functor which takes a parameter **Q**:

```
sage: FiniteMonoids().super_categories()
[Category of monoids, Category of finite semigroups]
sage: sorted(FiniteMonoids().Algebras(QQ).super_categories(), key=str)
[Category of finite dimensional algebras with basis over Rational Field,
 Category of finite set algebras over Rational Field,
 Category of monoid algebras over Rational Field]
```

```
>>> from sage.all import *
>>> FiniteMonoids().super_categories()
[Category of monoids, Category of finite semigroups]
>>> sorted(FiniteMonoids().Algebras(QQ).super_categories(), key=str)
[Category of finite dimensional algebras with basis over Rational Field,
 Category of finite set algebras over Rational Field,
 Category of monoid algebras over Rational Field]
```

Note that neither the category of *finite* semigroup algebras nor that of monoid algebras appear in the result; this is because there is currently nothing specific implemented about them.

Here is how *default_super_categories()* was called internally:

```
sage: C = sage.categories.algebra_functor.AlgebrasCategory
sage: C.default_super_categories(FiniteMonoids(), QQ)
```

(continues on next page)

(continued from previous page)

```
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of monoid algebras over Rational Field
    and Category of finite set algebras over Rational Field
```

```
>>> from sage.all import *
>>> C = sage.categories.algebra_functor.AlgebrasCategory
>>> C.default_super_categories(FiniteMonoids(), QQ)
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of monoid algebras over Rational Field
    and Category of finite set algebras over Rational Field
```

is_construction_defined_by_base()

Return whether the construction is defined by the base of `self`.

EXAMPLES:

The graded functorial construction is defined by the modules category. Hence this method returns `True` for graded modules and `False` for other graded xxx categories:

```
sage: Modules(ZZ).Graded().is_construction_defined_by_base()
True
sage: Algebras(QQ).Graded().is_construction_defined_by_base()
False
sage: Modules(ZZ).WithBasis().Graded().is_construction_defined_by_base()
False
```

```
>>> from sage.all import *
>>> Modules(ZZ).Graded().is_construction_defined_by_base()
True
>>> Algebras(QQ).Graded().is_construction_defined_by_base()
False
>>> Modules(ZZ).WithBasis().Graded().is_construction_defined_by_base()
False
```

This is implemented as follows: given the base category A and the construction F of `self`, that is `self=A.F()`, check whether no super category of A has F defined.

Note

Recall that, when A does not implement the construction `F`, a join category is returned. Therefore, in such cases, this method is not available:

```
sage: Bialgebras(QQ).Graded().is_construction_defined_by_base()
Traceback (most recent call last):
...
AttributeError: 'JoinCategory_with_category' object has
no attribute 'is_construction_defined_by_base'
```

```
>>> from sage.all import *
>>> Bialgebras(QQ).Graded().is_construction_defined_by_base()
Traceback (most recent call last):
...
AttributeError: 'JoinCategory_with_category' object has
no attribute 'is_construction_defined_by_base'
```

class
sage.categories.covariant_functorial_construction.**CovariantFunctorialConstruction**

Bases: UniqueRepresentation, SageObject

An abstract class for construction functors F (eg F = Cartesian product, tensor product, **Q**-algebra, ...) such that:

- Each category Cat (eg Cat = `Groups()`) can provide a category F_{Cat} for parents constructed via this functor (e.g. F_{Cat} = `CartesianProductsOf(Groups())`).
- For every category Cat, F_{Cat} is a subcategory of $F_{SuperCat}$ for every super category $SuperCat$ of Cat (the functorial construction is (category)-covariant).
- For parents A, B, ..., respectively in the categories Cat_A, Cat_B, ..., the category of $F(A, B, ...)$ is F_{Cat} where Cat is the meet of the categories Cat_A, Cat_B, ...,.

This covers two slightly different use cases:

- In the first use case, one uses directly the construction functor to create new parents:

```
sage: tensor()  # todo: not implemented (add an example)
```

```
>>> from sage.all import *
>>> tensor()  # todo: not implemented (add an example)
```

  or even new elements, which indirectly constructs the corresponding parent:

```
sage: tensor(...) # todo: not implemented
```

```
>>> from sage.all import *
>>> tensor(Ellipsis) # todo: not implemented
```

- In the second use case, one implements a parent, and then put it in the category F_{Cat} to specify supplementary mathematical information about that parent.

The main purpose of this class is to handle automatically the trivial part of the category hierarchy. For example, `CartesianProductsOf(Groups())` is set automatically as a subcategory of `CartesianProductsOf(Monoids())`.

In practice, each subclass of this class should provide the following attributes:

- `_functor_category` – string which should match the name of the nested category class to be used in each category to specify information and generic operations for elements of this category
- `_functor_name` – string which specifies the name of the functor, and also (when relevant) of the method on parents and elements used for calling the construction

TODO: What syntax do we want for F_{Cat}? For example, for the tensor product construction, which one do we want among (see chat on IRC, on 07/12/2009):

- `tensor(Cat)`
- `tensor((Cat, Cat))`
- `tensor.of((Cat, Cat))`
- `tensor.category_from_categories((Cat, Cat, Cat))`
- `Cat.TensorProducts()`

The syntax `Cat.TensorProducts()` does not supports well multivariate constructions like `tensor.of([Algebras(), HopfAlgebras(), ...])`. Also it forces every category to be (somehow) aware of all the tensorial construction that could apply to it, even those which are only induced from super categories.

Note: for each functorial construction, there probably is one (or several) largest categories on which it applies. For example, the *CartesianProductsCategory* construction makes only sense for concrete categories, that is subcategories of `Sets()`. Maybe we want to model this one way or the other.

category_from_categories(*categories*)

Return the category of $F(A, B, ...)$ for $A, B, ...$ parents in the given categories.

INPUT:

- `self` – a functor F
- `categories` – a non empty tuple of categories

EXAMPLES:

```
sage: Cat1 = Rings()
sage: Cat2 = Groups()
sage: cartesian_product.category_from_categories((Cat1, Cat1, Cat1))
Join of Category of rings and ...
    and Category of Cartesian products of monoids
    and Category of Cartesian products of commutative additive groups

sage: cartesian_product.category_from_categories((Cat1, Cat2))
Category of Cartesian products of monoids
```

```
>>> from sage.all import *
>>> Cat1 = Rings()
>>> Cat2 = Groups()
>>> cartesian_product.category_from_categories((Cat1, Cat1, Cat1))
Join of Category of rings and ...
    and Category of Cartesian products of monoids
    and Category of Cartesian products of commutative additive groups

>>> cartesian_product.category_from_categories((Cat1, Cat2))
Category of Cartesian products of monoids
```

category_from_category(*category*)

Return the category of $F(A, B, ...)$ for $A, B, ...$ parents in `category`.

INPUT:

- `self` – a functor F
- `category` – a category

EXAMPLES:

```
sage: tensor.category_from_category(ModulesWithBasis(QQ))
Category of tensor products of vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> tensor.category_from_category(ModulesWithBasis(QQ))
Category of tensor products of vector spaces with basis over Rational Field
```

TODO: add support for parametrized functors

category_from_parents(*parents*)

Return the category of $F(A, B, ...)$ for $A, B, ...$ parents.

INPUT:

- `self` – a functor F
- `parents` – a list (or iterable) of parents

EXAMPLES:

```
sage: E = CombinatorialFreeModule(QQ, ["a", "b", "c"])                    #␣
↪needs sage.modules
sage: tensor.category_from_parents((E, E, E))                             #␣
↪needs sage.modules
Category of tensor products of
 finite dimensional vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> E = CombinatorialFreeModule(QQ, ["a", "b", "c"])                      #␣
↪needs sage.modules
>>> tensor.category_from_parents((E, E, E))                               #␣
↪needs sage.modules
Category of tensor products of
 finite dimensional vector spaces with basis over Rational Field
```

class sage.categories.covariant_functorial_construction.**FunctorialConstructionCategory**(*category*, **args*)

Bases: *Category*

Abstract class for categories F_{Cat} obtained through a functorial construction

base_category()

Return the base category of the category `self`.

For any category `B` = F_{Cat} obtained through a functorial construction F, the call `B.base_category()` returns the category Cat.

EXAMPLES:

```
sage: Semigroups().Quotients().base_category()
Category of semigroups
```

```
>>> from sage.all import *
>>> Semigroups().Quotients().base_category()
Category of semigroups
```

classmethod category_of(*category*, **args*)

Return the image category of the functor F_{Cat}.

This is the main entry point for constructing the category F_{Cat} of parents $F(A, B, ...)$ constructed from parents $A, B, ...$ in Cat.

INPUT:

- `cls` – the category class for the functorial construction F
- `category` – a category Cat
- `*args` – further arguments for the functor

EXAMPLES:

```
sage: C = sage.categories.tensor.TensorProductsCategory
sage: C.category_of(ModulesWithBasis(QQ))
Category of tensor products of vector spaces with basis over Rational Field

sage: C = sage.categories.algebra_functor.AlgebrasCategory
sage: C.category_of(FiniteMonoids(), QQ)
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of monoid algebras over Rational Field
    and Category of finite set algebras over Rational Field
```

```
>>> from sage.all import *
>>> C = sage.categories.tensor.TensorProductsCategory
>>> C.category_of(ModulesWithBasis(QQ))
Category of tensor products of vector spaces with basis over Rational Field

>>> C = sage.categories.algebra_functor.AlgebrasCategory
>>> C.category_of(FiniteMonoids(), QQ)
Join of Category of finite dimensional algebras with basis over Rational Field
    and Category of monoid algebras over Rational Field
    and Category of finite set algebras over Rational Field
```

extra_super_categories()

Return the extra super categories of a construction category.

Default implementation which returns `[]`.

EXAMPLES:

```
sage: Sets().Subquotients().extra_super_categories()
[]
sage: Semigroups().Quotients().extra_super_categories()
[]
```

```
>>> from sage.all import *
>>> Sets().Subquotients().extra_super_categories()
[]
>>> Semigroups().Quotients().extra_super_categories()
[]
```

super_categories()

Return the super categories of a construction category.

EXAMPLES:

```
sage: Sets().Subquotients().super_categories()
[Category of sets]
sage: Semigroups().Quotients().super_categories()
[Category of subquotients of semigroups, Category of quotients of sets]
```

```
>>> from sage.all import *
>>> Sets().Subquotients().super_categories()
[Category of sets]
>>> Semigroups().Quotients().super_categories()
[Category of subquotients of semigroups, Category of quotients of sets]
```

class sage.categories.covariant_functorial_construction.**RegressiveCovariantConstructionCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*

Abstract class for categories F_{Cat} obtained through a regressive covariant functorial construction

classmethod default_super_categories(*category*, **args*)

Return the default super categories of $F_{Cat}(A, B, ...)$ for $A, B, ...$ parents in Cat.

INPUT:

- `cls` – the category class for the functor F
- `category` – a category Cat
- `*args` – further arguments for the functor

OUTPUT: a join category

This implements the property that an induced subcategory is a subcategory.

EXAMPLES:

A subquotient of a monoid is a monoid, and a subquotient of semigroup:

```
sage: Monoids().Subquotients().super_categories()
[Category of monoids, Category of subquotients of semigroups]
```

```
>>> from sage.all import *
>>> Monoids().Subquotients().super_categories()
[Category of monoids, Category of subquotients of semigroups]
```

5.2 Cartesian Product Functorial Construction

AUTHORS:

- Nicolas M. Thiery (2008-2010): initial revision and refactorization

class sage.categories.cartesian_product.**CartesianProductFunctor**(*category=None*)

Bases: *CovariantFunctorialConstruction*, *MultivariateConstructionFunctor*

The Cartesian product functor.

EXAMPLES:

```
sage: cartesian_product
The cartesian_product functorial construction
```

```
>>> from sage.all import *
>>> cartesian_product
The cartesian_product functorial construction
```

`cartesian_product` takes a finite collection of sets, and constructs the Cartesian product of those sets:

```
sage: A = FiniteEnumeratedSet(['a','b','c'])
sage: B = FiniteEnumeratedSet([1,2])
sage: C = cartesian_product([A, B]); C
The Cartesian product of ({'a', 'b', 'c'}, {1, 2})
sage: C.an_element()
('a', 1)
sage: C.list()          # todo: not implemented
[['a', 1], ['a', 2], ['b', 1], ['b', 2], ['c', 1], ['c', 2]]
```

```
>>> from sage.all import *
>>> A = FiniteEnumeratedSet(['a','b','c'])
>>> B = FiniteEnumeratedSet([Integer(1),Integer(2)])
>>> C = cartesian_product([A, B]); C
The Cartesian product of ({'a', 'b', 'c'}, {1, 2})
>>> C.an_element()
('a', 1)
>>> C.list()          # todo: not implemented
[['a', 1], ['a', 2], ['b', 1], ['b', 2], ['c', 1], ['c', 2]]
```

If those sets are endowed with more structure, say they are monoids (hence in the category `Monoids()`), then the result is automatically endowed with its natural monoid structure:

```
sage: M = Monoids().example()
sage: M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
sage: M.rename('M')
sage: C = cartesian_product([M, ZZ, QQ])
sage: C
The Cartesian product of (M, Integer Ring, Rational Field)
sage: C.an_element()
('abcd', 1, 1/2)
sage: C.an_element()^2
('abcdabcd', 1, 1/4)
sage: C.category()
Category of Cartesian products of monoids

sage: Monoids().CartesianProducts()
Category of Cartesian products of monoids
```

```
>>> from sage.all import *
>>> M = Monoids().example()
>>> M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
>>> M.rename('M')
>>> C = cartesian_product([M, ZZ, QQ])
>>> C
The Cartesian product of (M, Integer Ring, Rational Field)
```

(continues on next page)

(continued from previous page)

```
>>> C.an_element()
('abcd', 1, 1/2)
>>> C.an_element()**Integer(2)
('abcdabcd', 1, 1/4)
>>> C.category()
Category of Cartesian products of monoids

>>> Monoids().CartesianProducts()
Category of Cartesian products of monoids
```

The Cartesian product functor is covariant: if `A` is a subcategory of `B`, then `A.CartesianProducts()` is a subcategory of `B.CartesianProducts()` (see also *CovariantFunctorialConstruction*):

```
sage: C.categories()
[Category of Cartesian products of monoids,
 Category of monoids,
 Category of Cartesian products of semigroups,
 Category of semigroups,
 Category of Cartesian products of unital magmas,
 Category of Cartesian products of magmas,
 Category of unital magmas,
 Category of magmas,
 Category of Cartesian products of sets,
 Category of sets, ...]

[Category of Cartesian products of monoids,
 Category of monoids,
 Category of Cartesian products of semigroups,
 Category of semigroups,
 Category of Cartesian products of magmas,
 Category of unital magmas,
 Category of magmas,
 Category of Cartesian products of sets,
 Category of sets,
 Category of sets with partial maps,
 Category of objects]
```

```
>>> from sage.all import *
>>> C.categories()
[Category of Cartesian products of monoids,
 Category of monoids,
 Category of Cartesian products of semigroups,
 Category of semigroups,
 Category of Cartesian products of unital magmas,
 Category of Cartesian products of magmas,
 Category of unital magmas,
 Category of magmas,
 Category of Cartesian products of sets,
 Category of sets, ...]

[Category of Cartesian products of monoids,
 Category of monoids,
```

(continues on next page)

(continued from previous page)

```
Category of Cartesian products of semigroups,
Category of semigroups,
Category of Cartesian products of magmas,
Category of unital magmas,
Category of magmas,
Category of Cartesian products of sets,
Category of sets,
Category of sets with partial maps,
Category of objects]
```

Hence, the role of `Monoids().CartesianProducts()` is solely to provide mathematical information and algorithms which are relevant to Cartesian product of monoids. For example, it specifies that the result is again a monoid, and that its multiplicative unit is the Cartesian product of the units of the underlying sets:

```
sage: C.one()
('', 1, 1)
```

```
>>> from sage.all import *
>>> C.one()
('', 1, 1)
```

Those are implemented in the nested class *Monoids.CartesianProducts* of `Monoids(QQ)`. This nested class is itself a subclass of *CartesianProductsCategory*.

symbol = ' (+) '

class sage.categories.cartesian_product.**CartesianProductsCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*

An abstract base class for all `CartesianProducts` categories.

CartesianProducts()

Return the category of (finite) Cartesian products of objects of `self`.

By associativity of Cartesian products, this is `self` (a Cartesian product of Cartesian products of A's is a Cartesian product of A's).

EXAMPLES:

```
sage: ModulesWithBasis(QQ).CartesianProducts().CartesianProducts()
Category of Cartesian products of vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).CartesianProducts().CartesianProducts()
Category of Cartesian products of vector spaces with basis over Rational Field
```

base_ring()

The base ring of a Cartesian product is the base ring of the underlying category.

EXAMPLES:

```
sage: Algebras(ZZ).CartesianProducts().base_ring()
Integer Ring
```

```
>>> from sage.all import *
>>> Algebras(ZZ).CartesianProducts().base_ring()
Integer Ring
```

5.3 Tensor Product Functorial Construction

AUTHORS:

- Nicolas M. Thiéry (2008-2010): initial revision and refactorization

class sage.categories.tensor.**TensorProductFunctor**

Bases: *CovariantFunctorialConstruction*

A singleton class for the tensor functor.

This functor takes a collection of vector spaces (or modules with basis), and constructs the tensor product of those vector spaces. If this vector space is in a subcategory, say that of `Algebras(QQ)`, it is automatically endowed with its natural algebra structure, thanks to the category `Algebras(QQ).TensorProducts()` of tensor products of algebras. For elements, it constructs the natural tensor product element in the corresponding tensor product of their parents.

The tensor functor is covariant: if `A` is a subcategory of `B`, then `A.TensorProducts()` is a subcategory of `B.TensorProducts()` (see also *CovariantFunctorialConstruction*). Hence, the role of `Algebras(QQ).TensorProducts()` is solely to provide mathematical information and algorithms which are relevant to tensor product of algebras.

Those are implemented in the nested class *TensorProducts* of `Algebras(QQ)`. This nested class is itself a subclass of *TensorProductsCategory*.

symbol = ' # '

unicode_symbol = ' ⍰ '

class sage.categories.tensor.**TensorProductsCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*

An abstract base class for all TensorProducts's categories.

TensorProducts()

Return the category of tensor products of objects of `self`.

By associativity of tensor products, this is `self` (a tensor product of tensor products of Cat's is a tensor product of Cat's)

EXAMPLES:

```
sage: ModulesWithBasis(QQ).TensorProducts().TensorProducts()
Category of tensor products of vector spaces with basis over Rational Field
```

```
>>> from sage.all import *
>>> ModulesWithBasis(QQ).TensorProducts().TensorProducts()
Category of tensor products of vector spaces with basis over Rational Field
```

base()

The base of a tensor product is the base (usually a ring) of the underlying category.

EXAMPLES:

```
sage: ModulesWithBasis(ZZ).TensorProducts().base()
Integer Ring
```

```
>>> from sage.all import *
>>> ModulesWithBasis(ZZ).TensorProducts().base()
Integer Ring
```

sage.categories.tensor.**tensor = The tensor functorial construction**

The tensor product functorial construction

See *TensorProductFunctor* for more information

EXAMPLES:

```
sage: tensor
The tensor functorial construction
```

```
>>> from sage.all import *
>>> tensor
The tensor functorial construction
```

5.4 Signed Tensor Product Functorial Construction

AUTHORS:

- Travis Scrimshaw (2019-07): initial version

class sage.categories.signed_tensor.**SignedTensorProductFunctor**

Bases: *CovariantFunctorialConstruction*

A singleton class for the signed tensor functor.

This functor takes a collection of graded algebras (possibly with basis) and constructs the signed tensor product of those algebras. If this algebra is in a subcategory, say that of `Algebras(QQ).Graded()`, it is automatically endowed with its natural algebra structure, thanks to the category `Algebras(QQ).Graded().SignedTensorProducts()` of signed tensor products of graded algebras. For elements, it constructs the natural tensor product element in the corresponding tensor product of their parents.

The signed tensor functor is covariant: if `A` is a subcategory of `B`, then `A.SignedTensorProducts()` is a subcategory of `B.SignedTensorProducts()` (see also *CovariantFunctorialConstruction*). Hence, the role of `Algebras(QQ).Graded().SignedTensorProducts()` is solely to provide mathematical information and algorithms which are relevant to signed tensor product of graded algebras.

Those are implemented in the nested class *SignedTensorProducts* of `Algebras(QQ).Graded()`. This nested class is itself a subclass of *SignedTensorProductsCategory*.

EXAMPLES:

```
sage: tensor_signed
The signed tensor functorial construction
```

```
>>> from sage.all import *
>>> tensor_signed
The signed tensor functorial construction
```

symbol = ' # '

unicode_symbol = ' ⊡ '

class sage.categories.signed_tensor.**SignedTensorProductsCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*

An abstract base class for all SignedTensorProducts's categories.

SignedTensorProducts()

Return the category of signed tensor products of objects of `self`.

By associativity of signed tensor products, this is `self` (a tensor product of signed tensor products of Cat's is a tensor product of Cat's with the same twisting morphism)

EXAMPLES:

```
sage: AlgebrasWithBasis(QQ).Graded().SignedTensorProducts().
↪SignedTensorProducts()
Category of signed tensor products of graded algebras with basis
 over Rational Field
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(QQ).Graded().SignedTensorProducts().
↪SignedTensorProducts()
Category of signed tensor products of graded algebras with basis
 over Rational Field
```

base()

The base of a signed tensor product is the base (usually a ring) of the underlying category.

EXAMPLES:

```
sage: AlgebrasWithBasis(ZZ).Graded().SignedTensorProducts().base()
Integer Ring
```

```
>>> from sage.all import *
>>> AlgebrasWithBasis(ZZ).Graded().SignedTensorProducts().base()
Integer Ring
```

5.5 Dual functorial construction

AUTHORS:

- Nicolas M. Thiery (2009-2010): initial revision

class sage.categories.dual.**DualFunctor**

Bases: *CovariantFunctorialConstruction*

A singleton class for the dual functor

symbol = '^*'

class sage.categories.dual.**DualObjectsCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*

5.6 Group algebras and beyond: the Algebra functorial construction

5.6.1 Introduction: group algebras

Let G be a group and R be a ring. For example:

```
sage: G = DihedralGroup(3)
sage: R = QQ
```

```
>>> from sage.all import *
>>> G = DihedralGroup(Integer(3))
>>> R = QQ
```

The *group algebra* $A = RG$ of G over R is the space of formal linear combinations of elements of $group$ with coefficients in R:

```
sage: A = G.algebra(R); A
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
sage: a = A.an_element(); a
() + (1,2) + 3*(1,2,3) + 2*(1,3,2)
```

```
>>> from sage.all import *
>>> A = G.algebra(R); A
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
>>> a = A.an_element(); a
() + (1,2) + 3*(1,2,3) + 2*(1,3,2)
```

This space is endowed with an algebra structure, obtained by extending by bilinearity the multiplication of G to a multiplication on RG:

```
sage: A in Algebras
True
sage: a * a
14*() + 5*(2,3) + 2*(1,2) + 10*(1,2,3) + 13*(1,3,2) + 5*(1,3)
```

```
>>> from sage.all import *
>>> A in Algebras
True
>>> a * a
14*() + 5*(2,3) + 2*(1,2) + 10*(1,2,3) + 13*(1,3,2) + 5*(1,3)
```

In particular, the product of two basis elements is induced by the product of the corresponding elements of the group, and the unit of the group algebra is indexed by the unit of the group:

```
sage: s, t = A.algebra_generators()
sage: s*t
(1,2)
sage: A.one_basis()
()
sage: A.one()
()
```

```
>>> from sage.all import *
>>> s, t = A.algebra_generators()
>>> s*t
(1,2)
>>> A.one_basis()
()
>>> A.one()
()
```

For the user convenience and backward compatibility, the group algebra can also be constructed with:

```
sage: GroupAlgebra(G, R)
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
```

```
>>> from sage.all import *
>>> GroupAlgebra(G, R)
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
```

Since Issue #18700, both constructions are strictly equivalent:

```
sage: GroupAlgebra(G, R) is G.algebra(R)
True
```

```
>>> from sage.all import *
>>> GroupAlgebra(G, R) is G.algebra(R)
True
```

Group algebras are further endowed with a Hopf algebra structure; see below.

5.6.2 Generalizations

The above construction extends to weaker multiplicative structures than groups: magmas, semigroups, monoids. For a monoid S, we obtain the monoid algebra RS, which is defined exactly as above:

```
sage: S = Monoids().example(); S
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
sage: A = S.algebra(QQ); A
Algebra of An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
        over Rational Field
sage: A.category()
Category of monoid algebras over Rational Field
```

```
>>> from sage.all import *
>>> S = Monoids().example(); S
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
>>> A = S.algebra(QQ); A
Algebra of An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
        over Rational Field
>>> A.category()
Category of monoid algebras over Rational Field
```

This construction also extends to additive structures: magmas, semigroups, monoids, or groups:

```
sage: S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid:
 the free commutative monoid generated by ('a', 'b', 'c', 'd')
sage: U = S.algebra(QQ); U
Algebra of An example of a commutative monoid:
           the free commutative monoid generated by ('a', 'b', 'c', 'd')
        over Rational Field
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid:
 the free commutative monoid generated by ('a', 'b', 'c', 'd')
>>> U = S.algebra(QQ); U
Algebra of An example of a commutative monoid:
           the free commutative monoid generated by ('a', 'b', 'c', 'd')
        over Rational Field
```

Despite saying "free module", this is really an algebra, whose multiplication is induced by the addition of elements of S:

```
sage: U in Algebras(QQ)
True
sage: (a,b,c,d) = S.additive_semigroup_generators()
sage: U(a) * U(b)
B[a + b]
```

```
>>> from sage.all import *
>>> U in Algebras(QQ)
True
>>> (a,b,c,d) = S.additive_semigroup_generators()
>>> U(a) * U(b)
B[a + b]
```

To catter uniformly for the use cases above and some others, for S a set and K a ring, we define in Sage the *algebra of `S`* as the K-free module with basis indexed by S, endowed with whatever algebraic structure can be induced from that of S.

Warning

In most use cases, the result is actually an algebra, hence the name of this construction. In other cases this name is misleading:

```
sage: A = Sets().example().algebra(QQ); A
Algebra of Set of prime numbers (basic implementation)
        over Rational Field
sage: A.category()
Category of set algebras over Rational Field
sage: A in Algebras(QQ)
False
```

```
>>> from sage.all import *
>>> A = Sets().example().algebra(QQ); A
Algebra of Set of prime numbers (basic implementation)
        over Rational Field
>>> A.category()
Category of set algebras over Rational Field
>>> A in Algebras(QQ)
False
```

Suggestions for a uniform, meaningful, and non misleading name are welcome!

To achieve this flexibility, the features are implemented as a *Covariant Functorial Constructions* that is essentially a hierarchy of categories each providing the relevant additional features:

```
sage: A = DihedralGroup(3).algebra(QQ)
sage: A.categories()
[Category of finite group algebras over Rational Field,
 ...
 Category of group algebras over Rational Field,
 ...
 Category of monoid algebras over Rational Field,
 ...
 Category of semigroup algebras over Rational Field,
 ...
 Category of unital magma algebras over Rational Field,
 Category of magma algebras over Rational Field,
 ...
 Category of set algebras over Rational Field,
 ...]
```

```
>>> from sage.all import *
>>> A = DihedralGroup(Integer(3)).algebra(QQ)
>>> A.categories()
[Category of finite group algebras over Rational Field,
 ...
 Category of group algebras over Rational Field,
 ...
 Category of monoid algebras over Rational Field,
 ...
 Category of semigroup algebras over Rational Field,
 ...
 Category of unital magma algebras over Rational Field,
 Category of magma algebras over Rational Field,
 ...
 Category of set algebras over Rational Field,
 ...]
```

5.6.3 Specifying the algebraic structure

Constructing the algebra of a set endowed with both an additive and a multiplicative structure is ambiguous:

```
sage: Z3 = IntegerModRing(3)
sage: A = Z3.algebra(QQ)
Traceback (most recent call last):
...
TypeError:  `S = Ring of integers modulo 3` is both
 an additive and a multiplicative semigroup.
 Constructing its algebra is ambiguous.
 Please use, e.g., S.algebra(QQ, category=Semigroups())
```

```
>>> from sage.all import *
>>> Z3 = IntegerModRing(Integer(3))
>>> A = Z3.algebra(QQ)
Traceback (most recent call last):
...
TypeError:  `S = Ring of integers modulo 3` is both
 an additive and a multiplicative semigroup.
 Constructing its algebra is ambiguous.
 Please use, e.g., S.algebra(QQ, category=Semigroups())
```

This ambiguity can be resolved using the `category` argument of the construction:

```
sage: A = Z3.algebra(QQ, category=Monoids()); A
Algebra of Ring of integers modulo 3 over Rational Field
sage: A.category()
Category of finite dimensional monoid algebras over Rational Field

sage: A = Z3.algebra(QQ, category=CommutativeAdditiveGroups()); A
Algebra of Ring of integers modulo 3 over Rational Field
sage: A.category()
Category of finite dimensional commutative additive group algebras
 over Rational Field
```

```
>>> from sage.all import *
>>> A = Z3.algebra(QQ, category=Monoids()); A
Algebra of Ring of integers modulo 3 over Rational Field
>>> A.category()
Category of finite dimensional monoid algebras over Rational Field

>>> A = Z3.algebra(QQ, category=CommutativeAdditiveGroups()); A
Algebra of Ring of integers modulo 3 over Rational Field
>>> A.category()
Category of finite dimensional commutative additive group algebras
 over Rational Field
```

In general, the `category` argument can be used to specify which structure of S shall be extended to KS.

5.6.4 Group algebras, continued

Let us come back to the case of a group algebra $A = RG$. It is endowed with more structure and in particular that of a *Hopf algebra*:

```
sage: G = DihedralGroup(3)
sage: A = G.algebra(R); A
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
sage: A in HopfAlgebras(R).FiniteDimensional().WithBasis()
True
```

```
>>> from sage.all import *
>>> G = DihedralGroup(Integer(3))
>>> A = G.algebra(R); A
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
>>> A in HopfAlgebras(R).FiniteDimensional().WithBasis()
True
```

The basis elements are *group-like* for the coproduct: $\Delta(g) = g \otimes g$:

```
sage: s
(1,2,3)
sage: s.coproduct()
(1,2,3) # (1,2,3)
```

```
>>> from sage.all import *
>>> s
(1,2,3)
>>> s.coproduct()
(1,2,3) # (1,2,3)
```

The counit is the constant function 1 on the basis elements:

```
sage: A = GroupAlgebra(DihedralGroup(6), QQ)
sage: [A.counit(g) for g in A.basis()]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

```
>>> from sage.all import *
>>> A = GroupAlgebra(DihedralGroup(Integer(6)), QQ)
>>> [A.counit(g) for g in A.basis()]
[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1]
```

The antipode is given on basis elements by $\chi(g) = g^{-1}$:

```
sage: A = GroupAlgebra(DihedralGroup(3), QQ)
sage: s
(1,2,3)
sage: s.antipode()
(1,3,2)
```

```
>>> from sage.all import *
>>> A = GroupAlgebra(DihedralGroup(Integer(3)), QQ)
>>> s
(1,2,3)
>>> s.antipode()
(1,3,2)
```

By Maschke's theorem, for a finite group whose cardinality does not divide the characteristic of the base field, the algebra is semisimple:

```
sage: SymmetricGroup(5).algebra(QQ) in Algebras(QQ).Semisimple()                    #␣
↪needs sage.combinat
True
sage: CyclicPermutationGroup(10).algebra(FiniteField(7)) in Algebras.Semisimple
True
sage: CyclicPermutationGroup(10).algebra(FiniteField(5)) in Algebras.Semisimple
False
```

```
>>> from sage.all import *
>>> SymmetricGroup(Integer(5)).algebra(QQ) in Algebras(QQ).Semisimple()              ␣
↪     # needs sage.combinat
True
>>> CyclicPermutationGroup(Integer(10)).algebra(FiniteField(Integer(7))) in Algebras.
↪Semisimple
True
>>> CyclicPermutationGroup(Integer(10)).algebra(FiniteField(Integer(5))) in Algebras.
↪Semisimple
False
```

5.6.5 Coercions

Let RS be the algebra of some structure S. Then RS admits the natural coercion from any other algebra $R'S'$ of some structure S', as long as R' coerces into R and S' coerces into S.

For example, since there is a natural inclusion from the dihedral group D_2 of order 4 into the symmetric group S_4 of order 4!, and since there is a natural map from the integers to the rationals, there is a natural map from $\mathbf{Z}[D_2]$ to $\mathbf{Q}[S_4]$:

```
sage: # needs sage.combinat
sage: A = DihedralGroup(2).algebra(ZZ)
sage: B = SymmetricGroup(4).algebra(QQ)
sage: a = A.an_element(); a
() + 2*(3,4) + 3*(1,2) + (1,2)(3,4)
sage: b = B.an_element(); b
() + (2,3,4) + 2*(1,3)(2,4) + 3*(1,4)(2,3)
sage: B(a)
() + 2*(3,4) + 3*(1,2) + (1,2)(3,4)
sage: a * b  # a is automatically converted to an element of B
() + 2*(3,4) + 2*(2,3) + (2,3,4) + 3*(1,2) + (1,2)(3,4) + (1,3,2)
 + 3*(1,3,4,2) + 5*(1,3)(2,4) + 13*(1,3,2,4) + 12*(1,4,2,3) + 5*(1,4)(2,3)
sage: parent(a * b)
Symmetric group algebra of order 4 over Rational Field
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> A = DihedralGroup(Integer(2)).algebra(ZZ)
>>> B = SymmetricGroup(Integer(4)).algebra(QQ)
>>> a = A.an_element(); a
() + 2*(3,4) + 3*(1,2) + (1,2)(3,4)
>>> b = B.an_element(); b
() + (2,3,4) + 2*(1,3)(2,4) + 3*(1,4)(2,3)
>>> B(a)
() + 2*(3,4) + 3*(1,2) + (1,2)(3,4)
>>> a * b  # a is automatically converted to an element of B
() + 2*(3,4) + 2*(2,3) + (2,3,4) + 3*(1,2) + (1,2)(3,4) + (1,3,2)
 + 3*(1,3,4,2) + 5*(1,3)(2,4) + 13*(1,3,2,4) + 12*(1,4,2,3) + 5*(1,4)(2,3)
>>> parent(a * b)
Symmetric group algebra of order 4 over Rational Field
```

There is no obvious map in the other direction, though:

```
sage: A(b)
Traceback (most recent call last):
...
TypeError: do not know how to make x (= () + (2,3,4) + 2*(1,3)(2,4) + 3*(1,4)(2,3))
 an element of self
 (=Algebra of Dihedral group of order 4 as a permutation group over Integer Ring)
```

```
>>> from sage.all import *
>>> A(b)
Traceback (most recent call last):
...
TypeError: do not know how to make x (= () + (2,3,4) + 2*(1,3)(2,4) + 3*(1,4)(2,3))
 an element of self
 (=Algebra of Dihedral group of order 4 as a permutation group over Integer Ring)
```

If S is a unital (additive) magma, then RS is a unital algebra, and thus admits a coercion from its base ring R and any ring that coerces into R.

```
sage: G = DihedralGroup(2)
sage: A = G.algebra(ZZ)
sage: A(2)
2*()
```

```
>>> from sage.all import *
>>> G = DihedralGroup(Integer(2))
>>> A = G.algebra(ZZ)
>>> A(Integer(2))
2*()
```

If S is a multiplicative group, then RS admits a coercion from S and from any group which coerce into S:

```
sage: g = DihedralGroup(2).gen(0); g
(3,4)
sage: A(g)
(3,4)
```

(continues on next page)

(continued from previous page)

```
sage: A(2) * g
2*(3,4)
```

```
>>> from sage.all import *
>>> g = DihedralGroup(Integer(2)).gen(Integer(0)); g
(3,4)
>>> A(g)
(3,4)
>>> A(Integer(2)) * g
2*(3,4)
```

Note that there is an ambiguity if S' is a group which coerces into both R and S. For example) if S is the additive group $(\mathbf{Z}, +)$, and $A = RS$ is its group algebra, then the integer 2 can be coerced into A in two ways – via S, or via the base ring R – and *the answers are different*. It that case the coercion to R takes precedence. In particular, if $\mathbf{Z}$ is the ring (or group) of integers, then $\mathbf{Z}$ will coerce to any RS, by sending $\mathbf{Z}$ to R. In generic code, it is therefore recommended to always explicitly use `A.monomial(g)` to convert an element of the group into A.

AUTHORS:

- David Loeffler (2008-08-24): initial version
- Martin Raum (2009-08): update to use new coercion model – see Issue #6670.
- John Palmieri (2011-07): more updates to coercion, categories, etc., group algebras constructed using CombinatorialFreeModule – see Issue #6670.
- Nicolas M. Thiéry (2010-2017), Travis Scrimshaw (2017): generalization to a covariant functorial construction for monoid algebras, and beyond – see e.g. Issue #18700.

class sage.categories.algebra_functor.**AlgebraFunctor**(*base_ring*)

Bases: *CovariantFunctorialConstruction*

For a fixed ring, a functor sending a group/… to the corresponding group/… algebra.

EXAMPLES:

```
sage: from sage.categories.algebra_functor import AlgebraFunctor
sage: F = AlgebraFunctor(QQ); F
The algebra functorial construction
sage: F(DihedralGroup(3))
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.algebra_functor import AlgebraFunctor
>>> F = AlgebraFunctor(QQ); F
The algebra functorial construction
>>> F(DihedralGroup(Integer(3)))
Algebra of Dihedral group of order 6 as a permutation group
        over Rational Field
```

base_ring()

Return the base ring for this functor.

EXAMPLES:

```
sage: from sage.categories.algebra_functor import AlgebraFunctor
sage: AlgebraFunctor(QQ).base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.algebra_functor import AlgebraFunctor
>>> AlgebraFunctor(QQ).base_ring()
Rational Field
```

class sage.categories.algebra_functor.**AlgebrasCategory**(*category*, **args*)

Bases: *CovariantConstructionCategory*, *Category_over_base_ring*

An abstract base class for categories of monoid algebras, groups algebras, and the like.

See also

- *Sets.ParentMethods.algebra()*
- *Sets.SubcategoryMethods.Algebras()*
- *CovariantFunctorialConstruction*

INPUT:

- `base_ring` – a ring

EXAMPLES:

```
sage: C = Groups().Algebras(QQ); C
Category of group algebras over Rational Field
sage: C = Monoids().Algebras(QQ); C
Category of monoid algebras over Rational Field

sage: C._short_name()
'Algebras'
sage: latex(C) # todo: improve that
\mathbf{Algebras}(\mathbf{Monoids})
```

```
>>> from sage.all import *
>>> C = Groups().Algebras(QQ); C
Category of group algebras over Rational Field
>>> C = Monoids().Algebras(QQ); C
Category of monoid algebras over Rational Field

>>> C._short_name()
'Algebras'
>>> latex(C) # todo: improve that
\mathbf{Algebras}(\mathbf{Monoids})
```

class ParentMethods

Bases: `object`

coproduct_on_basis(*g*)

Return the coproduct of the element `g` of the basis.

Each basis element `g` is group-like. This method is used to compute the coproduct of any element.

EXAMPLES:

```
sage: # needs sage.combinat
sage: PF = NonDecreasingParkingFunctions(4)
sage: A = PF.algebra(ZZ); A
Algebra of Non-decreasing parking functions of size 4 over Integer Ring
sage: g = PF.an_element(); g
[1, 1, 1, 1]
sage: A.coproduct_on_basis(g)
B[[1, 1, 1, 1]] # B[[1, 1, 1, 1]]
sage: a = A.an_element(); a
2*B[[1, 1, 1, 1]] + 2*B[[1, 1, 1, 2]] + 3*B[[1, 1, 1, 3]]
sage: a.coproduct()
2*B[[1, 1, 1, 1]] # B[[1, 1, 1, 1]] +
2*B[[1, 1, 1, 2]] # B[[1, 1, 1, 2]] +
3*B[[1, 1, 1, 3]] # B[[1, 1, 1, 3]]
```

```
>>> from sage.all import *
>>> # needs sage.combinat
>>> PF = NonDecreasingParkingFunctions(Integer(4))
>>> A = PF.algebra(ZZ); A
Algebra of Non-decreasing parking functions of size 4 over Integer Ring
>>> g = PF.an_element(); g
[1, 1, 1, 1]
>>> A.coproduct_on_basis(g)
B[[1, 1, 1, 1]] # B[[1, 1, 1, 1]]
>>> a = A.an_element(); a
2*B[[1, 1, 1, 1]] + 2*B[[1, 1, 1, 2]] + 3*B[[1, 1, 1, 3]]
>>> a.coproduct()
2*B[[1, 1, 1, 1]] # B[[1, 1, 1, 1]] +
2*B[[1, 1, 1, 2]] # B[[1, 1, 1, 2]] +
3*B[[1, 1, 1, 3]] # B[[1, 1, 1, 3]]
```

class sage.categories.algebra_functor.**GroupAlgebraFunctor**(*group*)

Bases: *ConstructionFunctor*

For a fixed group, a functor sending a commutative ring to the corresponding group algebra.

INPUT:

- `group` – the group associated to each group algebra under consideration

EXAMPLES:

```
sage: from sage.categories.algebra_functor import GroupAlgebraFunctor
sage: F = GroupAlgebraFunctor(KleinFourGroup()); F
GroupAlgebraFunctor
sage: A = F(QQ); A
Algebra of The Klein 4 group of order 4, as a permutation group over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> from sage.categories.algebra_functor import GroupAlgebraFunctor
```

(continues on next page)

(continued from previous page)

```
>>> F = GroupAlgebraFunctor(KleinFourGroup()); F
GroupAlgebraFunctor
>>> A = F(QQ); A
Algebra of The Klein 4 group of order 4, as a permutation group over Rational␣
↪Field
```

group()

Return the group which is associated to this functor.

EXAMPLES:

```
sage: from sage.categories.algebra_functor import GroupAlgebraFunctor
sage: GroupAlgebraFunctor(CyclicPermutationGroup(17)).group() ==␣
↪CyclicPermutationGroup(17)
True
```

```
>>> from sage.all import *
>>> from sage.categories.algebra_functor import GroupAlgebraFunctor
>>> GroupAlgebraFunctor(CyclicPermutationGroup(Integer(17))).group() ==␣
↪CyclicPermutationGroup(Integer(17))
True
```

5.7 Subquotient Functorial Construction

AUTHORS:

- Nicolas M. Thiery (2010): initial revision

class sage.categories.subquotients.**SubquotientsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

5.8 Quotients Functorial Construction

AUTHORS:

- Nicolas M. Thiery (2010): initial revision

class sage.categories.quotients.**QuotientsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

classmethod default_super_categories(*category*)

Return the default super categories of `category.Quotients()`.

Mathematical meaning: if A is a quotient of B in the category C, then A is also a subquotient of B in the category C.

INPUT:

- `cls` – the class `QuotientsCategory`
- `category` – a category Cat

OUTPUT: a (join) category

In practice, this returns `category.Subquotients()`, joined together with the result of the method *RegressiveCovariantConstructionCategory.default_super_categories()* (that is the join of `category` and `cat.Quotients()` for each `cat` in the super categories of `category`).

EXAMPLES:

Consider `category=Groups()`, which has `cat=Monoids()` as super category. Then, a subgroup of a group G is simultaneously a subquotient of G, a group by itself, and a quotient monoid of `G`:

```
sage: Groups().Quotients().super_categories()
[Category of groups, Category of subquotients of monoids, Category of␣
↪quotients of semigroups]
```

```
>>> from sage.all import *
>>> Groups().Quotients().super_categories()
[Category of groups, Category of subquotients of monoids, Category of␣
↪quotients of semigroups]
```

Mind the last item above: there is indeed currently nothing implemented about quotient monoids.

This resulted from the following call:

```
sage: sage.categories.quotients.QuotientsCategory.default_super_
↪categories(Groups())
Join of Category of groups and Category of subquotients of monoids and␣
↪Category of quotients of semigroups
```

```
>>> from sage.all import *
>>> sage.categories.quotients.QuotientsCategory.default_super_
↪categories(Groups())
Join of Category of groups and Category of subquotients of monoids and␣
↪Category of quotients of semigroups
```

5.9 Subobjects Functorial Construction

AUTHORS:

- Nicolas M. Thiery (2010): initial revision

class `sage.categories.subobjects.`**SubobjectsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

classmethod default_super_categories(*category*)

Return the default super categories of `category.Subobjects()`.

Mathematical meaning: if A is a subobject of B in the category C, then A is also a subquotient of B in the category C.

INPUT:

- `cls` – the class `SubobjectsCategory`
- `category` – a category Cat

OUTPUT: a (join) category

In practice, this returns `category.Subquotients()`, joined together with the result of the method *RegressiveCovariantConstructionCategory.default_super_categories()* (that is the join of `category` and `cat.Subobjects()` for each `cat` in the super categories of `category`).

EXAMPLES:

Consider `category=Groups()`, which has `cat=Monoids()` as super category. Then, a subgroup of a group G is simultaneously a subquotient of G, a group by itself, and a submonoid of G:

```
sage: Groups().Subobjects().super_categories()
[Category of groups, Category of subquotients of monoids, Category of␣
↪subobjects of sets]
```

```
>>> from sage.all import *
>>> Groups().Subobjects().super_categories()
[Category of groups, Category of subquotients of monoids, Category of␣
↪subobjects of sets]
```

Mind the last item above: there is indeed currently nothing implemented about submonoids.

This resulted from the following call:

```
sage: sage.categories.subobjects.SubobjectsCategory.default_super_
↪categories(Groups())
Join of Category of groups and Category of subquotients of monoids and␣
↪Category of subobjects of sets
```

```
>>> from sage.all import *
>>> sage.categories.subobjects.SubobjectsCategory.default_super_
↪categories(Groups())
Join of Category of groups and Category of subquotients of monoids and␣
↪Category of subobjects of sets
```

5.10 Isomorphic Objects Functorial Construction

AUTHORS:

- Nicolas M. Thiery (2010): initial revision

class sage.categories.isomorphic_objects.**IsomorphicObjectsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

classmethod default_super_categories(*category*)

Return the default super categories of `category.IsomorphicObjects()`.

Mathematical meaning: if A is the image of B by an isomorphism in the category C, then A is both a subobject of B and a quotient of B in the category C.

INPUT:

- `cls` – the class `IsomorphicObjectsCategory`
- `category` – a category Cat

OUTPUT: a (join) category

In practice, this returns `category.Subobjects()` and `category.Quotients()`, joined together with the result of the method *RegressiveCovariantConstructionCategory.default_super_categories()* (that is the join of `category` and `cat.IsomorphicObjects()` for each `cat` in the super categories of `category`).

EXAMPLES:

Consider `category=Groups()`, which has `cat=Monoids()` as super category. Then, the image of a group G' by a group isomorphism is simultaneously a subgroup of G, a subquotient of G, a group by itself, and the image of G by a monoid isomorphism:

```
sage: Groups().IsomorphicObjects().super_categories()
[Category of groups,
 Category of subquotients of monoids,
 Category of quotients of semigroups,
 Category of isomorphic objects of sets]
```

```
>>> from sage.all import *
>>> Groups().IsomorphicObjects().super_categories()
[Category of groups,
 Category of subquotients of monoids,
 Category of quotients of semigroups,
 Category of isomorphic objects of sets]
```

Mind the last item above: there is indeed currently nothing implemented about isomorphic objects of monoids.

This resulted from the following call:

```
sage: sage.categories.isomorphic_objects.IsomorphicObjectsCategory.default_
↪super_categories(Groups())
Join of Category of groups and
Category of subquotients of monoids and
Category of quotients of semigroups and
Category of isomorphic objects of sets
```

```
>>> from sage.all import *
>>> sage.categories.isomorphic_objects.IsomorphicObjectsCategory.default_
↪super_categories(Groups())
Join of Category of groups and
Category of subquotients of monoids and
Category of quotients of semigroups and
Category of isomorphic objects of sets
```

5.11 Homset categories

`HomsetsCategory.`**`_test_homsets_category`**`(`*`**options`*`)`

Run generic tests on this homsets category.

See also

`TestSuite`.

EXAMPLES:

```
sage: Sets().Homsets()._test_homsets_category()
```

```
>>> from sage.all import *
>>> Sets().Homsets()._test_homsets_category()
```

class sage.categories.homsets.**Homsets**

Bases: *Category_singleton*

The category of all homsets.

EXAMPLES:

```
sage: from sage.categories.homsets import Homsets
sage: Homsets()
Category of homsets
```

```
>>> from sage.all import *
>>> from sage.categories.homsets import Homsets
>>> Homsets()
Category of homsets
```

This is a subcategory of `Sets()`:

```
sage: Homsets().super_categories()
[Category of sets]
```

```
>>> from sage.all import *
>>> Homsets().super_categories()
[Category of sets]
```

By this, we assume that all homsets implemented in Sage are sets, or equivalently that we only implement locally small categories. See Wikipedia article Category_(mathematics).

Issue #17364: every homset category shall be a subcategory of the category of all homsets:

```
sage: Schemes().Homsets().is_subcategory(Homsets())
True
sage: AdditiveMagmas().Homsets().is_subcategory(Homsets())
True
sage: AdditiveMagmas().AdditiveUnital().Homsets().is_subcategory(Homsets())
True
```

```
>>> from sage.all import *
>>> Schemes().Homsets().is_subcategory(Homsets())
True
>>> AdditiveMagmas().Homsets().is_subcategory(Homsets())
True
>>> AdditiveMagmas().AdditiveUnital().Homsets().is_subcategory(Homsets())
True
```

This is tested in *_test_homsets_category()*.

class Endset(*base_category*)

Bases: *CategoryWithAxiom_singleton*

The category of all endomorphism sets.

This category serves too purposes: making sure that the `Endset` axiom is implemented in the category where it's defined, namely `Homsets`, and specifying that `Endsets` are monoids.

EXAMPLES:

```
sage: from sage.categories.homsets import Homsets
sage: Homsets().Endset()
Category of endsets
```

```
>>> from sage.all import *
>>> from sage.categories.homsets import Homsets
>>> Homsets().Endset()
Category of endsets
```

class ParentMethods

Bases: `object`

is_endomorphism_set()

Return `True` as `self` is in the category of `Endsets`.

EXAMPLES:

```
sage: P.<t> = ZZ[]
sage: E = End(P)
sage: E.is_endomorphism_set()
True
```

```
>>> from sage.all import *
>>> P = ZZ['t']; (t,) = P._first_ngens(1)
>>> E = End(P)
>>> E.is_endomorphism_set()
True
```

extra_super_categories()

Implement the fact that endsets are monoids.

See also

CategoryWithAxiom.extra_super_categories()

EXAMPLES:

```
sage: from sage.categories.homsets import Homsets
sage: Homsets().Endset().extra_super_categories()
[Category of monoids]
```

```
>>> from sage.all import *
>>> from sage.categories.homsets import Homsets
>>> Homsets().Endset().extra_super_categories()
[Category of monoids]
```

class ParentMethods

Bases: object

is_endomorphism_set()

Return True if the domain and codomain of self are the same object.

EXAMPLES:

```
sage: P.<t> = ZZ[]
sage: f = P.hom([1/2*t])
sage: f.parent().is_endomorphism_set()
False
sage: g = P.hom([2*t])
sage: g.parent().is_endomorphism_set()
True
```

```
>>> from sage.all import *
>>> P = ZZ['t']; (t,) = P._first_ngens(1)
>>> f = P.hom([Integer(1)/Integer(2)*t])
>>> f.parent().is_endomorphism_set()
False
>>> g = P.hom([Integer(2)*t])
>>> g.parent().is_endomorphism_set()
True
```

class SubcategoryMethods

Bases: object

Endset()

Return the subcategory of the homsets of self that are endomorphism sets.

EXAMPLES:

```
sage: Sets().Homsets().Endset()
Category of endsets of sets

sage: Posets().Homsets().Endset()
Category of endsets of posets
```

```
>>> from sage.all import *
>>> Sets().Homsets().Endset()
Category of endsets of sets

>>> Posets().Homsets().Endset()
Category of endsets of posets
```

super_categories()

Return the super categories of self.

EXAMPLES:

```
sage: from sage.categories.homsets import Homsets
sage: Homsets()
Category of homsets
```

```
>>> from sage.all import *
>>> from sage.categories.homsets import Homsets
>>> Homsets()
Category of homsets
```

class sage.categories.homsets.**HomsetsCategory**(*category*, **args*)

Bases: *FunctorialConstructionCategory*, *CategoryWithParameters*

base()

If this homsets category is subcategory of a category with a base, return that base.

Todo

Is this really useful?

EXAMPLES:

```
sage: ModulesWithBasis(ZZ).Homsets().base()
Integer Ring
```

```
>>> from sage.all import *
>>> ModulesWithBasis(ZZ).Homsets().base()
Integer Ring
```

classmethod default_super_categories(*category*)

Return the default super categories of `category.Homsets()`.

INPUT:

- `cls` – the category class for the functor F
- `category` – a category Cat

OUTPUT: a category

As for the other functorial constructions, if `category` implements a nested `Homsets` class, this method is used in combination with `category.Homsets().extra_super_categories()` to compute the super categories of `category.Homsets()`.

EXAMPLES:

If `category` has one or more full super categories, then the join of their respective homsets category is returned. In this example, this join consists of a single category:

```
sage: from sage.categories.homsets import HomsetsCategory
sage: from sage.categories.additive_groups import AdditiveGroups

sage: C = AdditiveGroups()
sage: C.full_super_categories()
[Category of additive inverse additive unital additive magmas,
 Category of additive monoids]
sage: H = HomsetsCategory.default_super_categories(C); H
Category of homsets of additive monoids
sage: type(H)
```

(continues on next page)

(continued from previous page)

```
<class 'sage.categories.additive_monoids.AdditiveMonoids.Homsets_with_category
↪'>
```

```
>>> from sage.all import *
>>> from sage.categories.homsets import HomsetsCategory
>>> from sage.categories.additive_groups import AdditiveGroups

>>> C = AdditiveGroups()
>>> C.full_super_categories()
[Category of additive inverse additive unital additive magmas,
 Category of additive monoids]
>>> H = HomsetsCategory.default_super_categories(C); H
Category of homsets of additive monoids
>>> type(H)
<class 'sage.categories.additive_monoids.AdditiveMonoids.Homsets_with_category
↪'>
```

and, given that nothing specific is currently implemented for homsets of additive groups, `H` is directly the category thereof:

```
sage: C.Homsets()
Category of homsets of additive monoids
```

```
>>> from sage.all import *
>>> C.Homsets()
Category of homsets of additive monoids
```

Similarly for rings: a ring homset is just a homset of unital magmas and additive magmas:

```
sage: Rings().Homsets()
Category of homsets of unital magmas and additive unital additive magmas
```

```
>>> from sage.all import *
>>> Rings().Homsets()
Category of homsets of unital magmas and additive unital additive magmas
```

Otherwise, if `category` implements a nested class `Homsets`, this method returns the category of all homsets:

```
sage: AdditiveMagmas.Homsets
<class 'sage.categories.additive_magmas.AdditiveMagmas.Homsets'>
sage: HomsetsCategory.default_super_categories(AdditiveMagmas())
Category of homsets
```

```
>>> from sage.all import *
>>> AdditiveMagmas.Homsets
<class 'sage.categories.additive_magmas.AdditiveMagmas.Homsets'>
>>> HomsetsCategory.default_super_categories(AdditiveMagmas())
Category of homsets
```

which gives one of the super categories of `category.Homsets()`:

```
sage: AdditiveMagmas().Homsets().super_categories()
[Category of additive magmas, Category of homsets]
sage: AdditiveMagmas().AdditiveUnital().Homsets().super_categories()
[Category of additive unital additive magmas, Category of homsets]
```

```
>>> from sage.all import *
>>> AdditiveMagmas().Homsets().super_categories()
[Category of additive magmas, Category of homsets]
>>> AdditiveMagmas().AdditiveUnital().Homsets().super_categories()
[Category of additive unital additive magmas, Category of homsets]
```

the other coming from `category.Homsets().extra_super_categories()`:

```
sage: AdditiveMagmas().Homsets().extra_super_categories()
[Category of additive magmas]
```

```
>>> from sage.all import *
>>> AdditiveMagmas().Homsets().extra_super_categories()
[Category of additive magmas]
```

Finally, as a last resort, this method returns a stub category modelling the homsets of this category:

```
sage: hasattr(Posets, "Homsets")
False
sage: H = HomsetsCategory.default_super_categories(Posets()); H
Category of homsets of posets
sage: type(H)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
sage: Posets().Homsets()
Category of homsets of posets
```

```
>>> from sage.all import *
>>> hasattr(Posets, "Homsets")
False
>>> H = HomsetsCategory.default_super_categories(Posets()); H
Category of homsets of posets
>>> type(H)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
>>> Posets().Homsets()
Category of homsets of posets
```

class sage.categories.homsets.**HomsetsOf**(*category*, **args*)

Bases: *HomsetsCategory*

Default class for homsets of a category.

This is used when a category C defines some additional structure but not a homset category of its own. Indeed, unlike for covariant functorial constructions, we cannot represent the homset category of C by just the join of the homset categories of its super categories.

EXAMPLES:

```
sage: C = (Magmas() & Posets()).Homsets(); C
Category of homsets of magmas and posets
```

(continues on next page)

(continued from previous page)

```
sage: type(C)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
```

```
>>> from sage.all import *
>>> C = (Magmas() & Posets()).Homsets(); C
Category of homsets of magmas and posets
>>> type(C)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
```

super_categories()

Return the super categories of `self`.

A stub homset category admits a single super category, namely the category of all homsets.

EXAMPLES:

```
sage: C = (Magmas() & Posets()).Homsets(); C
Category of homsets of magmas and posets
sage: type(C)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
sage: C.super_categories()
[Category of homsets]
```

```
>>> from sage.all import *
>>> C = (Magmas() & Posets()).Homsets(); C
Category of homsets of magmas and posets
>>> type(C)
<class 'sage.categories.homsets.HomsetsOf_with_category'>
>>> C.super_categories()
[Category of homsets]
```

5.12 Realizations Covariant Functorial Construction

See also

- *Sets().WithRealizations* for an introduction to *realizations* and *with realizations*.
- *sage.categories.covariant_functorial_construction* for an introduction to covariant functorial constructions.
- *sage.categories.examples.with_realizations* for an example.

class sage.categories.realizations.**Category_realization_of_parent**(*parent_with_realization*)

Bases: *Category_over_base*, BindableClass

An abstract base class for categories of all realizations of a given parent.

INPUT:

- `parent_with_realization` – a parent

See also

Sets().WithRealizations

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A                                #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                                  #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

The role of this base class is to implement some technical goodies, like the binding `A.Realizations()` when a subclass `Realizations` is implemented as a nested class in `A` (see the *code of the example*):

```
sage: C = A.Realizations(); C                                                   #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> C = A.Realizations(); C                                                     #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

as well as the name for that category.

`sage.categories.realizations.`**`Realizations`**(*self*)

Return the category of realizations of the parent `self` or of objects of the category `self`

INPUT:

- `self` – a parent or a concrete category

Note

this *function* is actually inserted as a *method* in the class *Category* (see *Realizations()*). It is defined here for code locality reasons.

EXAMPLES:

The category of realizations of some algebra:

```
sage: Algebras(QQ).Realizations()
Join of Category of algebras over Rational Field
    and Category of realizations of unital magmas
```

```
>>> from sage.all import *
>>> Algebras(QQ).Realizations()
```

(continues on next page)

(continued from previous page)

```
Join of Category of algebras over Rational Field
    and Category of realizations of unital magmas
```

The category of realizations of a given algebra:

```
sage: A = Sets().WithRealizations().example(); A                                #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
sage: A.Realizations()                                                          #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field

sage: C = GradedHopfAlgebrasWithBasis(QQ).Realizations(); C
Join of Category of graded Hopf algebras with basis over Rational Field
    and Category of realizations of Hopf algebras over Rational Field
sage: C.super_categories()
[Category of graded Hopf algebras with basis over Rational Field,
 Category of realizations of Hopf algebras over Rational Field]

sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                                  #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
>>> A.Realizations()                                                            #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field

>>> C = GradedHopfAlgebrasWithBasis(QQ).Realizations(); C
Join of Category of graded Hopf algebras with basis over Rational Field
    and Category of realizations of Hopf algebras over Rational Field
>>> C.super_categories()
[Category of graded Hopf algebras with basis over Rational Field,
 Category of realizations of Hopf algebras over Rational Field]

>>> TestSuite(C).run()
```

See also

- *Sets().WithRealizations*
- ClasscallMetaclass

Todo

Add an optional argument to allow for:

```
sage: Realizations(A, category=Blahs()) # todo: not implemented
```

```
>>> from sage.all import *
>>> Realizations(A, category=Blahs()) # todo: not implemented
```

class sage.categories.realizations.**RealizationsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

An abstract base class for all categories of realizations category.

Realization are implemented as *RegressiveCovariantConstructionCategory*. See there for the documentation of how the various bindings such as `Sets().Realizations()` and `P.Realizations()`, where `P` is a parent, work.

See also

Sets().WithRealizations

5.13 With Realizations Covariant Functorial Construction

See also

- *Sets().WithRealizations* for an introduction to *realizations* and *with realizations*.
- *sage.categories.covariant_functorial_construction* for an introduction to covariant functorial constructions.

sage.categories.with_realizations.**WithRealizations**(*self*)

Return the category of parents in `self` endowed with multiple realizations.

INPUT:

- `self` – a category

See also

- The documentation and code (*sage.categories.examples.with_realizations*) of `Sets().WithRealizations().example()` for more on how to use and implement a parent with several realizations.
- Various use cases:
  - `SymmetricFunctions`
  - `QuasiSymmetricFunctions`
  - `NonCommutativeSymmetricFunctions`
  - `SymmetricFunctionsNonCommutingVariables`
  - `DescentAlgebra`
  - `MoebiusAlgebra`
  - `IwahoriHeckeAlgebra`
  - `ExtendedAffineWeylGroup()`

- The Implementing Algebraic Structures thematic tutorial.
- *sage.categories.realizations*

Note

this *function* is actually inserted as a *method* in the class *Category* (see *WithRealizations()*). It is defined here for code locality reasons.

EXAMPLES:

```
sage: Sets().WithRealizations()
Category of sets with realizations
```

```
>>> from sage.all import *
>>> Sets().WithRealizations()
Category of sets with realizations
```

Parent with realizations

Let us now explain the concept of realizations. A *parent with realizations* is a facade parent (see *Sets.Facade*) admitting multiple concrete realizations where its elements are represented. Consider for example an algebra A which admits several natural bases:

```
sage: A = Sets().WithRealizations().example(); A                                #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A                                  #␣
↪needs sage.modules
The subset algebra of {1, 2, 3} over Rational Field
```

For each such basis B one implements a parent P_B which realizes A with its elements represented by expanding them on the basis B:

```
sage: # needs sage.modules
sage: A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
sage: A.Out()
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
sage: A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
sage: A.an_element()
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
>>> A.Out()
```

(continues on next page)

(continued from previous page)

```
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
>>> A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
>>> A.an_element()
F[{}] + 2*F[{1}] + 3*F[{2}] + F[{1, 2}]
```

If B and B' are two bases, then the change of basis from B to B' is implemented by a canonical coercion between P_B and $P_{B'}$:

```
sage: # needs sage.modules
sage: F = A.F(); In = A.In(); Out = A.Out()
sage: i = In.an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
sage: F(i)
7*F[{}] + 3*F[{1}] + 4*F[{2}] + F[{1, 2}]
sage: F.coerce_map_from(Out)
Generic morphism:
  From: The subset algebra of {1, 2, 3} over Rational Field in the Out basis
  To:   The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> F = A.F(); In = A.In(); Out = A.Out()
>>> i = In.an_element(); i
In[{}] + 2*In[{1}] + 3*In[{2}] + In[{1, 2}]
>>> F(i)
7*F[{}] + 3*F[{1}] + 4*F[{2}] + F[{1, 2}]
>>> F.coerce_map_from(Out)
Generic morphism:
  From: The subset algebra of {1, 2, 3} over Rational Field in the Out basis
  To:   The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
```

allowing for mixed arithmetic:

```
sage: (1 + Out.from_set(1)) * In.from_set(2,3)                                #␣
↪needs sage.modules
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}]
+ 2*Out[{1, 3}] + 4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> (Integer(1) + Out.from_set(Integer(1))) * In.from_set(Integer(2),Integer(3)) ␣
↪                                  # needs sage.modules
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}]
+ 2*Out[{1, 3}] + 4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

In our example, there are three realizations:

```
sage: A.realizations()                                                        #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis,
```

(continues on next page)

(continued from previous page)

```
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

```
>>> from sage.all import *
>>> A.realizations()                                                    #␣
↪needs sage.modules
[The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis,
 The subset algebra of {1, 2, 3} over Rational Field in the In basis,
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis]
```

Instead of manually defining the shorthands `F`, `In`, and `Out`, as above one can just do:

```
sage: A.inject_shorthands()                                               #␣
↪needs sage.modules
Defining F as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
Defining In as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the In basis
Defining Out as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

```
>>> from sage.all import *
>>> A.inject_shorthands()                                               #␣
↪needs sage.modules
Defining F as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
Defining In as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the In basis
Defining Out as shorthand for
 The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

Rationale

Besides some goodies described below, the role of A is threefold:

- To provide, as illustrated above, a single entry point for the algebra as a whole: documentation, access to its properties and different realizations, etc.
- To provide a natural location for the initialization of the bases and the coercions between, and other methods that are common to all bases.
- To let other objects refer to A while allowing elements to be represented in any of the realizations.

We now illustrate this second point by defining the polynomial ring with coefficients in A:

```
sage: P = A['x']; P                                                       #␣
↪needs sage.modules
Univariate Polynomial Ring in x over
 The subset algebra of {1, 2, 3} over Rational Field
sage: x = P.gen()                                                         #␣
↪needs sage.modules
```

```
>>> from sage.all import *
>>> P = A['x']; P                                                          #␣
↪needs sage.modules
Univariate Polynomial Ring in x over
 The subset algebra of {1, 2, 3} over Rational Field
>>> x = P.gen()                                                            #␣
↪needs sage.modules
```

In the following examples, the coefficients turn out to be all represented in the F basis:

```
sage: P.one()                                                              #␣
↪needs sage.modules
F[{}]
sage: (P.an_element() + 1)^2                                               #␣
↪needs sage.modules
F[{}]*x^2 + 2*F[{}]*x + F[{}]
```

```
>>> from sage.all import *
>>> P.one()                                                                #␣
↪needs sage.modules
F[{}]
>>> (P.an_element() + Integer(1))**Integer(2)                                ␣
↪               # needs sage.modules
F[{}]*x^2 + 2*F[{}]*x + F[{}]
```

However we can create a polynomial with mixed coefficients, and compute with it:

```
sage: p = P([1, In[{1}], Out[{2}] ]); p                                    #␣
↪needs sage.modules
Out[{2}]*x^2 + In[{1}]*x + F[{}]
sage: p^2                                                                  #␣
↪needs sage.modules
Out[{2}]*x^4
+ (-8*In[{}] + 4*In[{1}] + 8*In[{2}] + 4*In[{3}]
   - 4*In[{1, 2}] - 2*In[{1, 3}] - 4*In[{2, 3}] + 2*In[{1, 2, 3}])*x^3
+ (F[{}] + 3*F[{1}] + 2*F[{2}] - 2*F[{1, 2}] - 2*F[{2, 3}] + 2*F[{1, 2, 3}])*x^2
+ (2*F[{}] + 2*F[{1}])*x
+ F[{}]
```

```
>>> from sage.all import *
>>> p = P([Integer(1), In[{Integer(1)}], Out[{Integer(2)}] ]); p             ␣
↪                      # needs sage.modules
Out[{2}]*x^2 + In[{1}]*x + F[{}]
>>> p**Integer(2)                                                           ␣
↪       # needs sage.modules
Out[{2}]*x^4
+ (-8*In[{}] + 4*In[{1}] + 8*In[{2}] + 4*In[{3}]
   - 4*In[{1, 2}] - 2*In[{1, 3}] - 4*In[{2, 3}] + 2*In[{1, 2, 3}])*x^3
+ (F[{}] + 3*F[{1}] + 2*F[{2}] - 2*F[{1, 2}] - 2*F[{2, 3}] + 2*F[{1, 2, 3}])*x^2
+ (2*F[{}] + 2*F[{1}])*x
+ F[{}]
```

Note how each coefficient involves a single basis which need not be that of the other coefficients. Which basis is

used depends on how coercion happened during mixed arithmetic and needs not be deterministic.

One can easily coerce all coefficient to a given basis with:

```
sage: p.map_coefficients(In)                                                #␣
↪needs sage.modules
(-4*In[{}] + 2*In[{1}] + 4*In[{2}] + 2*In[{3}]
 - 2*In[{1, 2}] - In[{1, 3}] - 2*In[{2, 3}] + In[{1, 2, 3}])*x^2
+ In[{1}]*x + In[{}]
```

```
>>> from sage.all import *
>>> p.map_coefficients(In)                                                  #␣
↪needs sage.modules
(-4*In[{}] + 2*In[{1}] + 4*In[{2}] + 2*In[{3}]
 - 2*In[{1, 2}] - In[{1, 3}] - 2*In[{2, 3}] + In[{1, 2, 3}])*x^2
+ In[{1}]*x + In[{}]
```

Alas, the natural notation for constructing such polynomials does not yet work:

```
sage: In[{1}] * x                                                           #␣
↪needs sage.modules
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
'The subset algebra of {1, 2, 3} over Rational Field in the In basis'
and 'Univariate Polynomial Ring in x over
The subset algebra of {1, 2, 3} over Rational Field'
```

```
>>> from sage.all import *
>>> In[{Integer(1)}] * x                                                    ␣
↪     # needs sage.modules
Traceback (most recent call last):
...
TypeError: unsupported operand parent(s) for *:
'The subset algebra of {1, 2, 3} over Rational Field in the In basis'
and 'Univariate Polynomial Ring in x over
The subset algebra of {1, 2, 3} over Rational Field'
```

The category of realizations of A

The set of all realizations of A, together with the coercion morphisms is a category (whose class inherits from *Category_realization_of_parent*):

```
sage: A.Realizations()                                                      #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

```
>>> from sage.all import *
>>> A.Realizations()                                                        #␣
↪needs sage.modules
Category of realizations of
 The subset algebra of {1, 2, 3} over Rational Field
```

The various parent realizing A belong to this category:

```
sage: A.F() in A.Realizations()                                                 #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> A.F() in A.Realizations()                                                   #␣
↪needs sage.modules
True
```

A itself is in the category of algebras with realizations:

```
sage: A in Algebras(QQ).WithRealizations()                                      #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Algebras(QQ).WithRealizations()                                        #␣
↪needs sage.modules
True
```

The (mostly technical) `WithRealizations` categories are the analogs of the `*WithSeveralBases` categories in MuPAD-Combinat. They provide support tools for handling the different realizations and the morphisms between them.

Typically, `VectorSpaces(QQ).FiniteDimensional().WithRealizations()` will eventually be in charge, whenever a coercion $\phi : A \mapsto B$ is registered, to register ϕ^{-1} as coercion $B \mapsto A$ if there is none defined yet. To achieve this, `FiniteDimensionalVectorSpaces` would provide a nested class `WithRealizations` implementing the appropriate logic.

`WithRealizations` is a *regressive covariant functorial construction*. On our example, this simply means that A is automatically in the category of rings with realizations (covariance):

```
sage: A in Rings().WithRealizations()                                           #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Rings().WithRealizations()                                             #␣
↪needs sage.modules
True
```

and in the category of algebras (regressiveness):

```
sage: A in Algebras(QQ)                                                         #␣
↪needs sage.modules
True
```

```
>>> from sage.all import *
>>> A in Algebras(QQ)                                                           #␣
↪needs sage.modules
True
```

Note

For `C` a category, `C.WithRealizations()` in fact calls `sage.categories.with_realizations.WithRealizations(C)`. The later is responsible for building the hierarchy of the categories with realizations in parallel to that of their base categories, optimizing away those categories that do not provide a `WithRealizations` nested class. See *sage.categories.covariant_functorial_construction* for the technical details.

Note

Design question: currently `WithRealizations` is a regressive construction. That is `self.WithRealizations()` is a subcategory of `self` by default:

```
sage: Algebras(QQ).WithRealizations().super_categories()
[Category of algebras over Rational Field,
 Category of monoids with realizations,
 Category of additive unital additive magmas with realizations]
```

```
>>> from sage.all import *
>>> Algebras(QQ).WithRealizations().super_categories()
[Category of algebras over Rational Field,
 Category of monoids with realizations,
 Category of additive unital additive magmas with realizations]
```

Is this always desirable? For example, `AlgebrasWithBasis(QQ).WithRealizations()` should certainly be a subcategory of `Algebras(QQ)`, but not of `AlgebrasWithBasis(QQ)`. This is because `AlgebrasWithBasis(QQ)` is specifying something about the concrete realization.

class sage.categories.with_realizations.**WithRealizationsCategory**(*category*, **args*)

Bases: *RegressiveCovariantConstructionCategory*

An abstract base class for all categories of parents with multiple realizations.

See also

Sets().WithRealizations

The role of this base class is to implement some technical goodies, such as the name for that category.

CHAPTER

SIX

EXAMPLES OF PARENTS USING CATEGORIES

6.1 Examples of algebras with basis

sage.categories.examples.algebras_with_basis.**Example**

alias of *FreeAlgebra*

class sage.categories.examples.algebras_with_basis.**FreeAlgebra**(*R*, *alphabet=('a', 'b', 'c')*)

Bases: CombinatorialFreeModule

An example of an algebra with basis: the free algebra.

This class illustrates a minimal implementation of an algebra with basis.

algebra_generators()

Return the generators of this algebra, as per *algebra_generators()*.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example(); A                                    #␣
↪needs sage.modules
An example of an algebra with basis: the free algebra on the generators ('a',
↪'b', 'c') over Rational Field
sage: A.algebra_generators()                                                    #␣
↪needs sage.modules
Family (B[word: a], B[word: b], B[word: c])
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example(); A                                      #␣
↪needs sage.modules
An example of an algebra with basis: the free algebra on the generators ('a',
↪'b', 'c') over Rational Field
>>> A.algebra_generators()                                                      #␣
↪needs sage.modules
Family (B[word: a], B[word: b], B[word: c])
```

one_basis()

Return the empty word, which index the one of this algebra, as per *one_basis()*.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                                       #␣
↪needs sage.modules
sage: A.one_basis()                                                             #␣
```

(continues on next page)

(continued from previous page)

```
↪needs sage.modules
word:
sage: A.one()                                                              #␣
↪needs sage.modules
B[word: ]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                    #␣
↪needs sage.modules
>>> A.one_basis()                                                          #␣
↪needs sage.modules
word:
>>> A.one()                                                                #␣
↪needs sage.modules
B[word: ]
```

product_on_basis(*w1*, *w2*)

Product of basis elements, as per *product_on_basis()*.

EXAMPLES:

```
sage: # needs sage.modules
sage: A = AlgebrasWithBasis(QQ).example()
sage: Words = A.basis().keys()
sage: A.product_on_basis(Words("acb"), Words("cba"))
B[word: acbcba]
sage: a, b, c = A.algebra_generators()
sage: a * (1-b)^2 * c
B[word: abbc] - 2*B[word: abc] + B[word: ac]
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = AlgebrasWithBasis(QQ).example()
>>> Words = A.basis().keys()
>>> A.product_on_basis(Words("acb"), Words("cba"))
B[word: acbcba]
>>> a, b, c = A.algebra_generators()
>>> a * (Integer(1)-b)**Integer(2) * c
B[word: abbc] - 2*B[word: abc] + B[word: ac]
```

6.2 Examples of commutative additive monoids

sage.categories.examples.commutative_additive_monoids.**Example**

alias of *FreeCommutativeAdditiveMonoid*

class sage.categories.examples.commutative_additive_monoids.**FreeCommutativeAdditiveMonoid**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: *FreeCommutativeAdditiveSemigroup*

An example of a commutative additive monoid: the free commutative monoid.

This class illustrates a minimal implementation of a commutative monoid.

EXAMPLES:

```
sage: S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid: the free commutative monoid generated by ('a',
↪ 'b', 'c', 'd')

sage: S.category()
Category of commutative additive monoids
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveMonoids().example(); S
An example of a commutative monoid: the free commutative monoid generated by ('a',
↪ 'b', 'c', 'd')

>>> S.category()
Category of commutative additive monoids
```

This is the free semigroup generated by:

```
sage: S.additive_semigroup_generators()
Family (a, b, c, d)
```

```
>>> from sage.all import *
>>> S.additive_semigroup_generators()
Family (a, b, c, d)
```

with product rule given by $a \times b = a$ for all a, b:

```
sage: a, b, c, d = S.additive_semigroup_generators()
```

```
>>> from sage.all import *
>>> a, b, c, d = S.additive_semigroup_generators()
```

We conclude by running systematic tests on this commutative monoid:

```
sage: TestSuite(S).run(verbose = True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
```

(continues on next page)

(continued from previous page)

```
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(S).run(verbose = True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
running ._test_zero() . . . pass
```

class Element(*parent*, *iterable*)

Bases: *Element*

zero()

Return the zero of this additive monoid, as per *zero()*.

EXAMPLES:

```
sage: M = CommutativeAdditiveMonoids().example(); M
An example of a commutative monoid: the free commutative monoid generated by (
↪'a', 'b', 'c', 'd')
sage: M.zero()
0
```

```
>>> from sage.all import *
>>> M = CommutativeAdditiveMonoids().example(); M
An example of a commutative monoid: the free commutative monoid generated by (
↪'a', 'b', 'c', 'd')
>>> M.zero()
0
```

6.3 Examples of commutative additive semigroups

sage.categories.examples.commutative_additive_semigroups.**Example**

alias of *FreeCommutativeAdditiveSemigroup*

class sage.categories.examples.commutative_additive_semigroups.**FreeCommutativeAdditiveSemigroup**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: UniqueRepresentation, Parent

An example of a commutative additive monoid: the free commutative monoid.

This class illustrates a minimal implementation of a commutative additive monoid.

EXAMPLES:

```
sage: S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup: the free commutative semigroup generated␣
↪by ('a', 'b', 'c', 'd')

sage: S.category()
Category of commutative additive semigroups
```

```
>>> from sage.all import *
>>> S = CommutativeAdditiveSemigroups().example(); S
An example of a commutative semigroup: the free commutative semigroup generated␣
↪by ('a', 'b', 'c', 'd')

>>> S.category()
Category of commutative additive semigroups
```

This is the free semigroup generated by:

```
sage: S.additive_semigroup_generators()
Family (a, b, c, d)
```

```
>>> from sage.all import *
>>> S.additive_semigroup_generators()
Family (a, b, c, d)
```

with product rule given by $a \times b = a$ for all a, b:

```
sage: a, b, c, d = S.additive_semigroup_generators()
```

```
>>> from sage.all import *
>>> a, b, c, d = S.additive_semigroup_generators()
```

We conclude by running systematic tests on this commutative monoid:

```
sage: TestSuite(S).run(verbose = True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(S).run(verbose = True)
running ._test_additive_associativity() . . . pass
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

class Element(*parent*, *iterable*)

Bases: ElementWrapper

EXAMPLES:

```
sage: F = CommutativeAdditiveSemigroups().example()
sage: x = F.element_class(F, (('a',4), ('b', 0), ('a', 2), ('c', 1), ('d',␣
↪5)))
sage: x
2*a + c + 5*d
sage: x.value
{'a': 2, 'b': 0, 'c': 1, 'd': 5}
sage: x.parent()
An example of a commutative semigroup: the free commutative semigroup␣
↪generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> F = CommutativeAdditiveSemigroups().example()
>>> x = F.element_class(F, (('a',Integer(4)), ('b', Integer(0)), ('a',␣
↪Integer(2)), ('c', Integer(1)), ('d', Integer(5))))
>>> x
2*a + c + 5*d
>>> x.value
{'a': 2, 'b': 0, 'c': 1, 'd': 5}
>>> x.parent()
An example of a commutative semigroup: the free commutative semigroup␣
↪generated by ('a', 'b', 'c', 'd')
```

Internally, elements are represented as dense dictionaries which associate to each generator of the monoid its multiplicity. In order to get an element, we wrap the dictionary into an element via ElementWrapper:

```
sage: x.value
{'a': 2, 'b': 0, 'c': 1, 'd': 5}
```

```
>>> from sage.all import *
>>> x.value
{'a': 2, 'b': 0, 'c': 1, 'd': 5}
```

additive_semigroup_generators()

Return the generators of the semigroup.

EXAMPLES:

```
sage: F = CommutativeAdditiveSemigroups().example()
sage: F.additive_semigroup_generators()
Family (a, b, c, d)
```

```
>>> from sage.all import *
>>> F = CommutativeAdditiveSemigroups().example()
>>> F.additive_semigroup_generators()
Family (a, b, c, d)
```

an_element()

Return an element of the semigroup.

EXAMPLES:

```
sage: F = CommutativeAdditiveSemigroups().example()
sage: F.an_element()
a + 2*b + 3*c + 4*d
```

```
>>> from sage.all import *
>>> F = CommutativeAdditiveSemigroups().example()
>>> F.an_element()
a + 2*b + 3*c + 4*d
```

summation(*x*, *y*)

Return the product of `x` and `y` in the semigroup, as per *summation()*.

EXAMPLES:

```
sage: F = CommutativeAdditiveSemigroups().example()
sage: a, b, c, d = F.additive_semigroup_generators()
sage: F.summation(a,b)
a + b
sage: (a+b) + (a+c)
2*a + b + c
```

```
>>> from sage.all import *
>>> F = CommutativeAdditiveSemigroups().example()
>>> a, b, c, d = F.additive_semigroup_generators()
>>> F.summation(a,b)
a + b
>>> (a+b) + (a+c)
2*a + b + c
```

6.4 Examples of Coxeter groups

6.5 Example of a crystal

class sage.categories.examples.crystals.**HighestWeightCrystalOfTypeA**(*n=3*)

Bases: `UniqueRepresentation`, `Parent`

An example of a crystal: the highest weight crystal of type A_n of highest weight ω_1.

The purpose of this class is to provide a minimal template for implementing crystals. See `CrystalOfLetters()` for a full featured and optimized implementation.

EXAMPLES:

```
sage: C = Crystals().example()
sage: C
Highest weight crystal of type A_3 of highest weight omega_1
sage: C.category()
Category of classical crystals
```

```
>>> from sage.all import *
>>> C = Crystals().example()
>>> C
Highest weight crystal of type A_3 of highest weight omega_1
>>> C.category()
Category of classical crystals
```

The elements of this crystal are in the set $\{1, \ldots, n+1\}$:

```
sage: C.list()
[1, 2, 3,  4]
sage: C.module_generators[0]
1
```

```
>>> from sage.all import *
>>> C.list()
[1, 2, 3,  4]
>>> C.module_generators[Integer(0)]
1
```

The crystal operators themselves correspond to the elementary transpositions:

```
sage: b = C.module_generators[0]
sage: b.f(1)
2
sage: b.f(1).e(1) == b
True
```

```
>>> from sage.all import *
>>> b = C.module_generators[Integer(0)]
>>> b.f(Integer(1))
2
>>> b.f(Integer(1)).e(Integer(1)) == b
True
```

Only the following basic operations are implemented:

- *cartan_type()* or an attribute _cartan_type
- an attribute module_generators
- *Element.e()*
- *Element.f()*

All the other usual crystal operations are inherited from the categories; for example:

```
sage: C.cardinality()
4
```

```
>>> from sage.all import *
>>> C.cardinality()
4
```

class Element

Bases: ElementWrapper

e(i)

Return the action of e_i on self.

EXAMPLES:

```
sage: C = Crystals().example(4)
sage: [[c,i,c.e(i)] for i in C.index_set() for c in C if c.e(i) is not␣
↪None]
[[2, 1, 1], [3, 2, 2], [4, 3, 3], [5, 4, 4]]
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(4))
>>> [[c,i,c.e(i)] for i in C.index_set() for c in C if c.e(i) is not None]
[[2, 1, 1], [3, 2, 2], [4, 3, 3], [5, 4, 4]]
```

f(i)

Return the action of f_i on self.

EXAMPLES:

```
sage: C = Crystals().example(4)
sage: [[c,i,c.f(i)] for i in C.index_set() for c in C if c.f(i) is not␣
↪None]
[[1, 1, 2], [2, 2, 3], [3, 3, 4], [4, 4, 5]]
```

```
>>> from sage.all import *
>>> C = Crystals().example(Integer(4))
>>> [[c,i,c.f(i)] for i in C.index_set() for c in C if c.f(i) is not None]
[[1, 1, 2], [2, 2, 3], [3, 3, 4], [4, 4, 5]]
```

class sage.categories.examples.crystals.**NaiveCrystal**

Bases: UniqueRepresentation, Parent

This is an example of a "crystal" which does not come from any kind of representation, designed primarily to test the Stembridge local rules with. The crystal has vertices labeled 0 through 5, with 0 the highest weight.

The code here could also possibly be generalized to create a class that automatically builds a crystal from an edge-colored digraph, if someone feels adventurous.

Currently, only the methods *highest_weight_vector()*, *e()*, and *f()* are guaranteed to work.

EXAMPLES:

```
sage: C = Crystals().example(choice='naive')
sage: C.highest_weight_vector()
0
```

```
>>> from sage.all import *
>>> C = Crystals().example(choice='naive')
>>> C.highest_weight_vector()
0
```

class Element

Bases: ElementWrapper

e(*i*)

Return the action of e_i on self.

EXAMPLES:

```
sage: C = Crystals().example(choice='naive')
sage: [[c,i,c.e(i)] for i in C.index_set() for c in [C(j) for j in [0..
↪5]] if c.e(i) is not None]
[[1, 1, 0], [2, 1, 1], [3, 1, 2], [5, 1, 3], [4, 2, 0], [5, 2, 4]]
```

```
>>> from sage.all import *
>>> C = Crystals().example(choice='naive')
>>> [[c,i,c.e(i)] for i in C.index_set() for c in [C(j) for j in␣
↪(ellipsis_range(Integer(0),Ellipsis,Integer(5)))] if c.e(i) is not None]
[[1, 1, 0], [2, 1, 1], [3, 1, 2], [5, 1, 3], [4, 2, 0], [5, 2, 4]]
```

f(*i*)

Return the action of f_i on self.

EXAMPLES:

```
sage: C = Crystals().example(choice='naive')
sage: [[c,i,c.f(i)] for i in C.index_set() for c in [C(j) for j in [0..
↪5]] if c.f(i) is not None]
[[0, 1, 1], [1, 1, 2], [2, 1, 3], [3, 1, 5], [0, 2, 4], [4, 2, 5]]
```

```
>>> from sage.all import *
>>> C = Crystals().example(choice='naive')
>>> [[c,i,c.f(i)] for i in C.index_set() for c in [C(j) for j in␣
↪(ellipsis_range(Integer(0),Ellipsis,Integer(5)))] if c.f(i) is not None]
[[0, 1, 1], [1, 1, 2], [2, 1, 3], [3, 1, 5], [0, 2, 4], [4, 2, 5]]
```

6.6 Examples of CW complexes

sage.categories.examples.cw_complexes.**Example**

alias of *Surface*

class sage.categories.examples.cw_complexes.**Surface**(*bdy=(1, 2, 1, 2)*)

Bases: UniqueRepresentation, Parent

An example of a CW complex: a (2-dimensional) surface.

This class illustrates a minimal implementation of a CW complex.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example(); X
An example of a CW complex: the surface given by the boundary map (1, 2, 1, 2)

sage: X.category()
Category of finite finite dimensional CW complexes
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example(); X
An example of a CW complex: the surface given by the boundary map (1, 2, 1, 2)

>>> X.category()
Category of finite finite dimensional CW complexes
```

We conclude by running systematic tests on this manifold:

```
sage: TestSuite(X).run()
```

```
>>> from sage.all import *
>>> TestSuite(X).run()
```

class Element(*parent*, *dim*, *name*)

Bases: `Element`

A cell in a CW complex.

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: f = X.an_element()
sage: f.dimension()
2
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> f = X.an_element()
>>> f.dimension()
2
```

an_element()

Return an element of the CW complex, as per *Sets.ParentMethods.an_element()*.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: X.an_element()
2-cell f
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> X.an_element()
2-cell f
```

cells()

Return the cells of `self`.

EXAMPLES:

```
sage: from sage.categories.cw_complexes import CWComplexes
sage: X = CWComplexes().example()
sage: C = X.cells()
sage: sorted((d, C[d]) for d in C.keys())
[(0, (0-cell v,)),
 (1, (0-cell e1, 0-cell e2)),
 (2, (2-cell f,))]
```

```
>>> from sage.all import *
>>> from sage.categories.cw_complexes import CWComplexes
>>> X = CWComplexes().example()
>>> C = X.cells()
>>> sorted((d, C[d]) for d in C.keys())
[(0, (0-cell v,)),
 (1, (0-cell e1, 0-cell e2)),
 (2, (2-cell f,))]
```

6.7 Example of facade set

class sage.categories.examples.facade_sets.**IntegersCompletion**

Bases: `UniqueRepresentation`, `Parent`

An example of a facade parent: the set of integers completed with $+\infty$ $-\infty$

This class illustrates a minimal implementation of a facade parent that models the union of several other parents.

EXAMPLES:

```
sage: S = Sets().Facade().example("union"); S
An example of a facade set: the integers completed by +-infinity
```

```
>>> from sage.all import *
>>> S = Sets().Facade().example("union"); S
An example of a facade set: the integers completed by +-infinity
```

class sage.categories.examples.facade_sets.**PositiveIntegerMonoid**

Bases: `UniqueRepresentation`, `Parent`

An example of a facade parent: the positive integers viewed as a multiplicative monoid

This class illustrates a minimal implementation of a facade parent which models a subset of a set.

EXAMPLES:

```
sage: S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
```

```
>>> from sage.all import *
>>> S = Sets().Facade().example(); S
An example of facade set: the monoid of positive integers
```

6.8 Examples of finite Coxeter groups

class sage.categories.examples.finite_coxeter_groups.**DihedralGroup**(*n=5*)

Bases: UniqueRepresentation, Parent

An example of finite Coxeter group: the n-th dihedral group of order $2n$.

The purpose of this class is to provide a minimal template for implementing finite Coxeter groups. See DihedralGroup for a full featured and optimized implementation.

EXAMPLES:

```
sage: G = FiniteCoxeterGroups().example()
```

```
>>> from sage.all import *
>>> G = FiniteCoxeterGroups().example()
```

This group is generated by two simple reflections s_1 and s_2 subject to the relation $(s_1 s_2)^n = 1$:

```
sage: G.simple_reflections()
Finite family {1: (1,), 2: (2,)}

sage: s1, s2 = G.simple_reflections()
sage: (s1*s2)^5 == G.one()
True
```

```
>>> from sage.all import *
>>> G.simple_reflections()
Finite family {1: (1,), 2: (2,)}

>>> s1, s2 = G.simple_reflections()
>>> (s1*s2)**Integer(5) == G.one()
True
```

An element is represented by its reduced word (a tuple of elements of $self.index_set()$):

```
sage: G.an_element()
(1, 2)

sage: list(G)
[(),
 (1,),
 (2,),
 (1, 2),
 (2, 1),
```

(continues on next page)

(continued from previous page)

```
 (1, 2, 1),
 (2, 1, 2),
 (1, 2, 1, 2),
 (2, 1, 2, 1),
 (1, 2, 1, 2, 1)]
```

```
>>> from sage.all import *
>>> G.an_element()
(1, 2)

>>> list(G)
[(),
 (1,),
 (2,),
 (1, 2),
 (2, 1),
 (1, 2, 1),
 (2, 1, 2),
 (1, 2, 1, 2),
 (2, 1, 2, 1),
 (1, 2, 1, 2, 1)]
```

This reduced word is unique, except for the longest element where the chosen reduced word is $(1, 2, 1, 2 \ldots)$:

```
sage: G.long_element()
(1, 2, 1, 2, 1)
```

```
>>> from sage.all import *
>>> G.long_element()
(1, 2, 1, 2, 1)
```

class Element

Bases: ElementWrapper

apply_simple_reflection_right(i)

Implement *sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.apply_simple_reflection()*.

EXAMPLES:

```
sage: D5 = FiniteCoxeterGroups().example(5)
sage: [i^2 for i in D5]  # indirect doctest
[(), (), (), (1, 2, 1, 2), (2, 1, 2, 1), (), (), (2, 1), (1, 2), ()]
sage: [i^5 for i in D5]  # indirect doctest
[(), (1,), (2,), (), (), (1, 2, 1), (2, 1, 2), (), (), (1, 2, 1, 2, 1)]
```

```
>>> from sage.all import *
>>> D5 = FiniteCoxeterGroups().example(Integer(5))
>>> [i**Integer(2) for i in D5]  # indirect doctest
[(), (), (), (1, 2, 1, 2), (2, 1, 2, 1), (), (), (2, 1), (1, 2), ()]
>>> [i**Integer(5) for i in D5]  # indirect doctest
[(), (1,), (2,), (), (), (1, 2, 1), (2, 1, 2), (), (), (1, 2, 1, 2, 1)]
```

has_right_descent(*i*, *positive=False*, *side='right'*)

Implement *has_right_descent()*.

EXAMPLES:

```
sage: D6 = FiniteCoxeterGroups().example(6)
sage: s = D6.simple_reflections()
sage: s[1].has_descent(1)
True
sage: s[1].has_descent(1)
True
sage: s[1].has_descent(2)
False
sage: D6.one().has_descent(1)
False
sage: D6.one().has_descent(2)
False
sage: D6.long_element().has_descent(1)
True
sage: D6.long_element().has_descent(2)
True
```

```
>>> from sage.all import *
>>> D6 = FiniteCoxeterGroups().example(Integer(6))
>>> s = D6.simple_reflections()
>>> s[Integer(1)].has_descent(Integer(1))
True
>>> s[Integer(1)].has_descent(Integer(1))
True
>>> s[Integer(1)].has_descent(Integer(2))
False
>>> D6.one().has_descent(Integer(1))
False
>>> D6.one().has_descent(Integer(2))
False
>>> D6.long_element().has_descent(Integer(1))
True
>>> D6.long_element().has_descent(Integer(2))
True
```

wrapped_class

alias of `tuple`

coxeter_matrix()

Return the Coxeter matrix of `self`.

EXAMPLES:

```
sage: FiniteCoxeterGroups().example(6).coxeter_matrix()
[1 6]
[6 1]
```

```
>>> from sage.all import *
>>> FiniteCoxeterGroups().example(Integer(6)).coxeter_matrix()
```

(continues on next page)

(continued from previous page)

```
[1 6]
[6 1]
```

degrees()

Return the degrees of `self`.

EXAMPLES:

```
sage: FiniteCoxeterGroups().example(6).degrees()
(2, 6)
```

```
>>> from sage.all import *
>>> FiniteCoxeterGroups().example(Integer(6)).degrees()
(2, 6)
```

index_set()

Implement *CoxeterGroups.ParentMethods.index_set()*.

EXAMPLES:

```
sage: D4 = FiniteCoxeterGroups().example(4)
sage: D4.index_set()
(1, 2)
```

```
>>> from sage.all import *
>>> D4 = FiniteCoxeterGroups().example(Integer(4))
>>> D4.index_set()
(1, 2)
```

one()

Implement *sage.categories.magmas.Magmas.Unital.ParentMethods.one()*.

EXAMPLES:

```
sage: D6 = FiniteCoxeterGroups().example(6)
sage: D6.one()
()
```

```
>>> from sage.all import *
>>> D6 = FiniteCoxeterGroups().example(Integer(6))
>>> D6.one()
()
```

sage.categories.examples.finite_coxeter_groups.**Example**

alias of *DihedralGroup*

6.9 Example of a finite dimensional algebra with basis

sage.categories.examples.finite_dimensional_algebras_with_basis.**Example**

alias of *KroneckerQuiverPathAlgebra*

class sage.categories.examples.finite_dimensional_algebras_with_basis.**KroneckerQuiverPathAlgebra**(*base*

Bases: CombinatorialFreeModule

An example of a finite dimensional algebra with basis: the path algebra of the Kronecker quiver.

This class illustrates a minimal implementation of a finite dimensional algebra with basis. See sage.quivers.algebra.PathAlgebra for a full-featured implementation of path algebras.

algebra_generators()

Return algebra generators for this algebra.

See also

sage.categories.magmatic_algebras.MagmaticAlgebras.ParentMethods.algebra_generators().

EXAMPLES:

```
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
sage: A.algebra_generators()
Finite family {'x': x, 'y': y, 'a': a, 'b': b}
```

```
>>> from sage.all import *
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example(); A
An example of a finite dimensional algebra with basis:
the path algebra of the Kronecker quiver
(containing the arrows a:x->y and b:x->y) over Rational Field
>>> A.algebra_generators()
Finite family {'x': x, 'y': y, 'a': a, 'b': b}
```

one()

Return the unit of this algebra.

See also

one_basis()

EXAMPLES:

```
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example()
sage: A.one()
x + y
```

```
>>> from sage.all import *
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example()
>>> A.one()
x + y
```

product_on_basis(*w1*, *w2*)

Return the product of the two basis elements indexed by w1 and w2.

See also

product_on_basis()

EXAMPLES:

```
sage: A = FiniteDimensionalAlgebrasWithBasis(QQ).example()
```

```
>>> from sage.all import *
>>> A = FiniteDimensionalAlgebrasWithBasis(QQ).example()
```

Here is the multiplication table for the algebra:

```
sage: matrix([[p*q for q in A.basis()] for p in A.basis()])
[x 0 a b]
[0 y 0 0]
[0 a 0 0]
[0 b 0 0]
```

```
>>> from sage.all import *
>>> matrix([[p*q for q in A.basis()] for p in A.basis()])
[x 0 a b]
[0 y 0 0]
[0 a 0 0]
[0 b 0 0]
```

Here we take some products of linear combinations of basis elements:

```
sage: x, y, a, b = A.basis()
sage: a * (1-b)^2 * x
0
sage: x*a + b*y
a + b
sage: x*x
x
sage: x*y
0
sage: x*a*y
a
```

```
>>> from sage.all import *
>>> x, y, a, b = A.basis()
>>> a * (Integer(1)-b)**Integer(2) * x
0
>>> x*a + b*y
a + b
>>> x*x
x
>>> x*y
0
>>> x*a*y
a
```

6.10 Examples of a finite dimensional Lie algebra with basis

class sage.categories.examples.finite_dimensional_lie_algebras_with_basis.**AbelianLieAlgebra**(*R*, *n=None*, *M=None*, *ambient=None*)

Bases: `Parent`, `UniqueRepresentation`

An example of a finite dimensional Lie algebra with basis: the abelian Lie algebra.

Let R be a commutative ring, and M an R-module. The *abelian Lie algebra* on M is the R-Lie algebra obtained by endowing M with the trivial Lie bracket ($[a, b] = 0$ for all $a, b \in M$).

This class illustrates a minimal implementation of a finite dimensional Lie algebra with basis.

INPUT:

- `R` – base ring
- `n` – (optional) a nonnegative integer (default: `None`)
- `M` – an R-module (default: the free R-module of rank `n`) to serve as the ground space for the Lie algebra
- `ambient` – (optional) a Lie algebra; if this is set, then the resulting Lie algebra is declared a Lie subalgebra of `ambient`

OUTPUT:

The abelian Lie algebra on M.

class Element(*parent*, *value*)

Bases: *Element*

Initialize `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: TestSuite(a).run()
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> TestSuite(a).run()
```

lift()

Return the lift of `self` to the universal enveloping algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: elt = 2*a + 2*b + 3*c
sage: elt.lift()                                                    #␣
↪needs sage.combinat sage.libs.singular
2*b0 + 2*b1 + 3*b2
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> elt = Integer(2)*a + Integer(2)*b + Integer(3)*c
>>> elt.lift()                                                    #␣
↪needs sage.combinat sage.libs.singular
2*b0 + 2*b1 + 3*b2
```

monomial_coefficients(*copy=True*)

Return the monomial coefficients of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: elt = 2*a + 2*b + 3*c
sage: elt.monomial_coefficients()
{0: 2, 1: 2, 2: 3}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> elt = Integer(2)*a + Integer(2)*b + Integer(3)*c
>>> elt.monomial_coefficients()
{0: 2, 1: 2, 2: 3}
```

to_vector(*order=None*, *sparse=False*)

Return `self` as a vector in `self.parent().module()`.

See the docstring of the latter method for the meaning of this.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: elt = 2*a + 2*b + 3*c
sage: elt.to_vector()
(2, 2, 3)
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> elt = Integer(2)*a + Integer(2)*b + Integer(3)*c
>>> elt.to_vector()
(2, 2, 3)
```

ambient()

Return the ambient Lie algebra of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: S = L.subalgebra([2*a+b, b + c])
```

(continues on next page)

(continued from previous page)

```
sage: S.ambient() == L
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> S = L.subalgebra([Integer(2)*a+b, b + c])
>>> S.ambient() == L
True
```

basis()

Return the basis of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.basis()
Finite family {0: (1, 0, 0), 1: (0, 1, 0), 2: (0, 0, 1)}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.basis()
Finite family {0: (1, 0, 0), 1: (0, 1, 0), 2: (0, 0, 1)}
```

basis_matrix()

Return the basis matrix of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.basis_matrix()
[1 0 0]
[0 1 0]
[0 0 1]
```

from_vector(*v*, *order=None*)

Return the element of `self` corresponding to the vector `v` in `self.module()`.

Implement this if you implement *module()*; see the documentation of *module()* for how this is to be done.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: u = L.from_vector(vector(QQ, (1, 0, 0))); u
(1, 0, 0)
sage: parent(u) is L
True
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> u = L.from_vector(vector(QQ, (Integer(1), Integer(0), Integer(0)))); u
(1, 0, 0)
>>> parent(u) is L
True
```

gens()

Return the generators of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.gens()
((1, 0, 0), (0, 1, 0), (0, 0, 1))
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.gens()
((1, 0, 0), (0, 1, 0), (0, 0, 1))
```

ideal(*gens*)

alias of *subalgebra()*.

is_ideal(*A*)

Return if `self` is an ideal of the ambient space `A`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: L.is_ideal(L)
True
sage: S1 = L.subalgebra([2*a+b, b + c])
sage: S1.is_ideal(L)
True
sage: S2 = L.subalgebra([2*a+b])
sage: S2.is_ideal(S1)
True
sage: S1.is_ideal(S2)
False
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> L.is_ideal(L)
True
>>> S1 = L.subalgebra([Integer(2)*a+b, b + c])
>>> S1.is_ideal(L)
True
>>> S2 = L.subalgebra([Integer(2)*a+b])
>>> S2.is_ideal(S1)
True
```

(continues on next page)

(continued from previous page)

```
>>> S1.is_ideal(S2)
False
```

leading_monomials()

Return the set of leading monomials of the basis of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: I = L.ideal([2*a + b, b + c])
sage: I.leading_monomials()
((1, 0, 0), (0, 1, 0))
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> I = L.ideal([Integer(2)*a + b, b + c])
>>> I.leading_monomials()
((1, 0, 0), (0, 1, 0))
```

lie_algebra_generators()

alias of *basis()*.

lift(*x*)

Return the lift of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.gens()
sage: L.lift(a)
b0
sage: L.lift(b).parent() is L.universal_enveloping_algebra()
True

sage: I = L.ideal([a + 2*b, b + 3*c])
sage: I.lift(I.basis()[0])
(1, 0, -6)
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.gens()
>>> L.lift(a)
b0
>>> L.lift(b).parent() is L.universal_enveloping_algebra()
True

>>> I = L.ideal([a + Integer(2)*b, b + Integer(3)*c])
>>> I.lift(I.basis()[Integer(0)])
(1, 0, -6)
```

module()

Return an R-module which is isomorphic to the underlying R-module of `self`.

See *module()* for an explanation.

In this particular example, this returns the module M that was used to construct `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.module()
Vector space of dimension 3 over Rational Field

sage: a, b, c = L.lie_algebra_generators()
sage: S = L.subalgebra([2*a+b, b + c])
sage: S.module()
Vector space of degree 3 and dimension 2 over Rational Field
Basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.module()
Vector space of dimension 3 over Rational Field

>>> a, b, c = L.lie_algebra_generators()
>>> S = L.subalgebra([Integer(2)*a+b, b + c])
>>> S.module()
Vector space of degree 3 and dimension 2 over Rational Field
Basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

subalgebra(*gens*)

Return the Lie subalgebra of `self` generated by the elements of the iterable `gens`.

This currently requires the ground ring R to be a field.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: a, b, c = L.lie_algebra_generators()
sage: L.subalgebra([2*a+b, b + c])
An example of a finite dimensional Lie algebra with basis:
 the 2-dimensional abelian Lie algebra over Rational Field with
 basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> a, b, c = L.lie_algebra_generators()
>>> L.subalgebra([Integer(2)*a+b, b + c])
An example of a finite dimensional Lie algebra with basis:
```

(continues on next page)

(continued from previous page)

```
 the 2-dimensional abelian Lie algebra over Rational Field with
 basis matrix:
[   1    0 -1/2]
[   0    1    1]
```

universal_enveloping_algebra()

Return the universal enveloping algebra of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.universal_enveloping_algebra()
Noncommutative Multivariate Polynomial Ring in b0, b1, b2
 over Rational Field, nc-relations: {}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.universal_enveloping_algebra()
Noncommutative Multivariate Polynomial Ring in b0, b1, b2
 over Rational Field, nc-relations: {}
```

zero()

Return the zero element.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
sage: L.zero()
(0, 0, 0)
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).FiniteDimensional().WithBasis().example()
>>> L.zero()
(0, 0, 0)
```

sage.categories.examples.finite_dimensional_lie_algebras_with_basis.**Example**

alias of *AbelianLieAlgebra*

6.11 Examples of finite enumerated sets

class sage.categories.examples.finite_enumerated_sets.**Example**

Bases: `UniqueRepresentation`, `Parent`

An example of a finite enumerated set: $\{1, 2, 3\}$.

This class provides a minimal implementation of a finite enumerated set.

See `FiniteEnumeratedSet` for a full featured implementation.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().example()
sage: C.cardinality()
```

(continues on next page)

(continued from previous page)

```
3
sage: C.list()
[1, 2, 3]
sage: C.an_element()
1
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().example()
>>> C.cardinality()
3
>>> C.list()
[1, 2, 3]
>>> C.an_element()
1
```

This checks that the different methods of the enumerated set C return consistent results:

```
sage: TestSuite(C).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(C).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

class sage.categories.examples.finite_enumerated_sets.**IsomorphicObjectOfFiniteEnumeratedSet**(*ambient=An example of a finite enumerated set: {1, 2, 3}*)

Bases: `UniqueRepresentation`, `Parent`

ambient()

Return the ambient space for `self`, as per *Sets.Subquotients.ParentMethods.ambient()*.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
sage: C.ambient()
An example of a finite enumerated set: {1,2,3}
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
>>> C.ambient()
An example of a finite enumerated set: {1,2,3}
```

lift(*x*)

INPUT:

- `x` – an element of `self`

Lifts `x` to the ambient space for `self`, as per *Sets.Subquotients.ParentMethods.lift()*.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
sage: C.lift(9)
3
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
>>> C.lift(Integer(9))
3
```

retract(*x*)

INPUT:

- `x` – an element of the ambient space for `self`

Retracts `x` from the ambient space to `self`, as per *Sets.Subquotients.ParentMethods.retract()*.

EXAMPLES:

```
sage: C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
sage: C.retract(3)
9
```

```
>>> from sage.all import *
>>> C = FiniteEnumeratedSets().IsomorphicObjects().example(); C
The image by some isomorphism of An example of a finite enumerated set: {1,2,
↪3}
>>> C.retract(Integer(3))
9
```

6.12 Examples of finite monoids

sage.categories.examples.finite_monoids.**Example**

alias of *IntegerModMonoid*

class sage.categories.examples.finite_monoids.**IntegerModMonoid**(*n=12*)

Bases: UniqueRepresentation, Parent

An example of a finite monoid: the integers mod n.

This class illustrates a minimal implementation of a finite monoid.

EXAMPLES:

```
sage: S = FiniteMonoids().example(); S
An example of a finite multiplicative monoid: the integers modulo 12

sage: S.category()
Category of finitely generated finite enumerated monoids
```

```
>>> from sage.all import *
>>> S = FiniteMonoids().example(); S
An example of a finite multiplicative monoid: the integers modulo 12

>>> S.category()
Category of finitely generated finite enumerated monoids
```

We conclude by running systematic tests on this monoid:

```
sage: TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_prod() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

class Element

Bases: ElementWrapper

wrapped_class

alias of Integer

an_element()

Return an element of the monoid, as per *Sets.ParentMethods.an_element()*.

EXAMPLES:

```
sage: M = FiniteMonoids().example()
sage: M.an_element()
6
```

```
>>> from sage.all import *
>>> M = FiniteMonoids().example()
>>> M.an_element()
6
```

one()

Return the one of the monoid, as per *sage.categories.magmas.Magmas.Unital.ParentMethods.one()*.

EXAMPLES:

```
sage: M = FiniteMonoids().example()
sage: M.one()
1
```

```
>>> from sage.all import *
>>> M = FiniteMonoids().example()
>>> M.one()
1
```

product(x, y)

Return the product of two elements x and y of the monoid, as per *sage.categories.magmas.Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: M = FiniteMonoids().example()
sage: M.product(M(3), M(5))
3
```

```
>>> from sage.all import *
>>> M = FiniteMonoids().example()
>>> M.product(M(Integer(3)), M(Integer(5)))
3
```

semigroup_generators()

Return a set of generators for `self`, as per *Semigroups.ParentMethods.semigroup_generators()*.

Currently this returns all integers mod n, which is of course far from optimal!

EXAMPLES:

```
sage: M = FiniteMonoids().example()
sage: M.semigroup_generators()
Family (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
```

```
>>> from sage.all import *
>>> M = FiniteMonoids().example()
>>> M.semigroup_generators()
Family (0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11)
```

6.13 Examples of finite semigroups

sage.categories.examples.finite_semigroups.**Example**

alias of *LeftRegularBand*

class sage.categories.examples.finite_semigroups.**LeftRegularBand**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: UniqueRepresentation, Parent

An example of a finite semigroup.

This class provides a minimal implementation of a finite semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(); S
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(); S
An example of a finite semigroup:
 the left regular band generated by ('a', 'b', 'c', 'd')
```

This is the semigroup generated by:

```
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

such that $x^2 = x$ and $xyx = xy$ for any x and y in S:

```
sage: S('dab')
'dab'
sage: S('dab') * S('acb')
'dabc'
```

```
>>> from sage.all import *
>>> S('dab')
'dab'
>>> S('dab') * S('acb')
'dabc'
```

It follows that the elements of S are strings without repetitions over the alphabet a, b, c, d:

```
sage: sorted(S.list())
['a', 'ab', 'abc', 'abcd', 'abd', 'abdc', 'ac', 'acb', 'acbd', 'acd',
 'acdb', 'ad', 'adb', 'adbc', 'adc', 'adcb', 'b', 'ba', 'bac',
 'bacd', 'bad', 'badc', 'bc', 'bca', 'bcad', 'bcd', 'bcda', 'bd',
 'bda', 'bdac', 'bdc', 'bdca', 'c', 'ca', 'cab', 'cabd', 'cad',
 'cadb', 'cb', 'cba', 'cbad', 'cbd', 'cbda', 'cd', 'cda', 'cdab',
 'cdb', 'cdba', 'd', 'da', 'dab', 'dabc', 'dac', 'dacb', 'db',
 'dba', 'dbac', 'dbc', 'dbca', 'dc', 'dca', 'dcab', 'dcb', 'dcba']
```

```
>>> from sage.all import *
>>> sorted(S.list())
['a', 'ab', 'abc', 'abcd', 'abd', 'abdc', 'ac', 'acb', 'acbd', 'acd',
 'acdb', 'ad', 'adb', 'adbc', 'adc', 'adcb', 'b', 'ba', 'bac',
 'bacd', 'bad', 'badc', 'bc', 'bca', 'bcad', 'bcd', 'bcda', 'bd',
 'bda', 'bdac', 'bdc', 'bdca', 'c', 'ca', 'cab', 'cabd', 'cad',
 'cadb', 'cb', 'cba', 'cbad', 'cbd', 'cbda', 'cd', 'cda', 'cdab',
 'cdb', 'cdba', 'd', 'da', 'dab', 'dabc', 'dac', 'dacb', 'db',
 'dba', 'dbac', 'dbc', 'dbca', 'dc', 'dca', 'dcab', 'dcb', 'dcba']
```

It also follows that there are finitely many of them:

```
sage: S.cardinality()
64
```

```
>>> from sage.all import *
>>> S.cardinality()
64
```

Indeed:

```
sage: 4 * ( 1 + 3 * (1 + 2 * (1 + 1)))
64
```

```
>>> from sage.all import *
>>> Integer(4) * ( Integer(1) + Integer(3) * (Integer(1) + Integer(2) *␣
↪(Integer(1) + Integer(1))))
64
```

As expected, all the elements of S are idempotents:

```
sage: all( x.is_idempotent() for x in S )
True
```

```
>>> from sage.all import *
>>> all( x.is_idempotent() for x in S )
True
```

Now, let us look at the structure of the semigroup:

```
sage: S = FiniteSemigroups().example(alphabet = ('a','b','c'))
sage: S.cayley_graph(side='left', simple=True).plot()                          #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 60 graphics primitives
sage: S.j_transversal_of_idempotents()  # random (arbitrary choice)            #␣
↪needs sage.graphs
['acb', 'ac', 'ab', 'bc', 'a', 'c', 'b']
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet = ('a','b','c'))
>>> S.cayley_graph(side='left', simple=True).plot()                            #␣
↪needs sage.graphs sage.plot
```

(continues on next page)

(continued from previous page)

```
Graphics object consisting of 60 graphics primitives
>>> S.j_transversal_of_idempotents()  # random (arbitrary choice)              #␣
↪needs sage.graphs
['acb', 'ac', 'ab', 'bc', 'a', 'c', 'b']
```

We conclude by running systematic tests on this semigroup:

```
sage: TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_random() . . . pass
running ._test_rank() . . . pass
running ._test_some_elements() . . . pass
```

class Element

Bases: ElementWrapper

wrapped_class

alias of str

an_element()

Return an element of the semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: S.an_element()
'cdab'

sage: S = FiniteSemigroups().example(("b"))
sage: S.an_element()
'b'
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S.an_element()
'cdab'

>>> S = FiniteSemigroups().example(("b"))
>>> S.an_element()
'b'
```

product(*x*, *y*)

Return the product of two elements of the semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().example()
sage: S('a') * S('b')
'ab'
sage: S('a') * S('b') * S('a')
'ab'
sage: S('a') * S('a')
'a'
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example()
>>> S('a') * S('b')
'ab'
>>> S('a') * S('b') * S('a')
'ab'
>>> S('a') * S('a')
'a'
```

semigroup_generators()

Return the generators of the semigroup.

EXAMPLES:

```
sage: S = FiniteSemigroups().example(alphabet=('x','y'))
sage: S.semigroup_generators()
Family ('x', 'y')
```

```
>>> from sage.all import *
>>> S = FiniteSemigroups().example(alphabet=('x','y'))
>>> S.semigroup_generators()
Family ('x', 'y')
```

6.14 Examples of finite Weyl groups

sage.categories.examples.finite_weyl_groups.**Example**

alias of *SymmetricGroup*

class sage.categories.examples.finite_weyl_groups.**SymmetricGroup**(*n=4*)

Bases: UniqueRepresentation, Parent

An example of finite Weyl group: the symmetric group, with elements in list notation.

The purpose of this class is to provide a minimal template for implementing finite Weyl groups. See SymmetricGroup for a full featured and optimized implementation.

EXAMPLES:

```
sage: S = FiniteWeylGroups().example()
sage: S
The symmetric group on {0, ..., 3}
sage: S.category()
Category of finite irreducible Weyl groups
```

```
>>> from sage.all import *
>>> S = FiniteWeylGroups().example()
>>> S
The symmetric group on {0, ..., 3}
>>> S.category()
Category of finite irreducible Weyl groups
```

The elements of this group are permutations of the set $\{0, \ldots, 3\}$:

```
sage: S.one()
(0, 1, 2, 3)
sage: S.an_element()
(1, 2, 3, 0)
```

```
>>> from sage.all import *
>>> S.one()
(0, 1, 2, 3)
>>> S.an_element()
(1, 2, 3, 0)
```

The group itself is generated by the elementary transpositions:

```
sage: S.simple_reflections()
Finite family {0: (1, 0, 2, 3), 1: (0, 2, 1, 3), 2: (0, 1, 3, 2)}
```

```
>>> from sage.all import *
>>> S.simple_reflections()
Finite family {0: (1, 0, 2, 3), 1: (0, 2, 1, 3), 2: (0, 1, 3, 2)}
```

Only the following basic operations are implemented:

- *one()*
- *product()*
- *simple_reflection()*
- *cartan_type()*
- *Element.has_right_descent()*.

All the other usual Weyl group operations are inherited from the categories:

```
sage: S.cardinality()
24
sage: S.long_element()
(3, 2, 1, 0)
sage: S.cayley_graph(side='left').plot()                                    #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 120 graphics primitives
```

```
>>> from sage.all import *
>>> S.cardinality()
24
>>> S.long_element()
(3, 2, 1, 0)
>>> S.cayley_graph(side='left').plot()                                    #␣
↪needs sage.graphs sage.plot
Graphics object consisting of 120 graphics primitives
```

Alternatively, one could have implemented *sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods.apply_simple_reflection()* instead of *simple_reflection()* and *product()*. See `CoxeterGroups().example()`.

class Element

Bases: ElementWrapper

has_right_descent(*i*)

Implement *CoxeterGroups.ElementMethods.has_right_descent()*.

EXAMPLES:

```
sage: S = FiniteWeylGroups().example()
sage: s = S.simple_reflections()
sage: (s[1] * s[2]).has_descent(2)
True
sage: S._test_has_descent()
```

```
>>> from sage.all import *
>>> S = FiniteWeylGroups().example()
>>> s = S.simple_reflections()
>>> (s[Integer(1)] * s[Integer(2)]).has_descent(Integer(2))
True
>>> S._test_has_descent()
```

cartan_type()

Return the Cartan type of self.

EXAMPLES:

```
sage: FiniteWeylGroups().example().cartan_type()                                #␣
↪needs sage.modules
['A', 3] relabelled by {1: 0, 2: 1, 3: 2}
```

```
>>> from sage.all import *
>>> FiniteWeylGroups().example().cartan_type()                                  #␣
↪needs sage.modules
['A', 3] relabelled by {1: 0, 2: 1, 3: 2}
```

degrees()

Return the degrees of self.

EXAMPLES:

```
sage: W = FiniteWeylGroups().example()
sage: W.degrees()
(2, 3, 4)
```

```
>>> from sage.all import *
>>> W = FiniteWeylGroups().example()
>>> W.degrees()
(2, 3, 4)
```

index_set()

Implement *CoxeterGroups.ParentMethods.index_set()*.

EXAMPLES:

```
sage: FiniteWeylGroups().example().index_set()
[0, 1, 2]
```

```
>>> from sage.all import *
>>> FiniteWeylGroups().example().index_set()
[0, 1, 2]
```

one()

Implement *sage.categories.magmas.Magmas.Unital.ParentMethods.one()*.

EXAMPLES:

```
sage: FiniteWeylGroups().example().one()
(0, 1, 2, 3)
```

```
>>> from sage.all import *
>>> FiniteWeylGroups().example().one()
(0, 1, 2, 3)
```

product(x, y)

Implement *sage.categories.magmas.Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: s = FiniteWeylGroups().example().simple_reflections()
sage: s[1] * s[2]
(0, 2, 3, 1)
```

```
>>> from sage.all import *
>>> s = FiniteWeylGroups().example().simple_reflections()
>>> s[Integer(1)] * s[Integer(2)]
(0, 2, 3, 1)
```

simple_reflection(i)

Implement *sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods.simple_reflection()* by returning the transposition $(i, i+1)$.

EXAMPLES:

```
sage: FiniteWeylGroups().example().simple_reflection(2)
(0, 1, 3, 2)
```

```
>>> from sage.all import *
>>> FiniteWeylGroups().example().simple_reflection(Integer(2))
(0, 1, 3, 2)
```

6.15 Examples of graded connected Hopf algebras with basis

sage.categories.examples.graded_connected_hopf_algebras_with_basis.**Example**

alias of *GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator*

class sage.categories.examples.graded_connected_hopf_algebras_with_basis.**GradedConnectedCombinatorial**

Bases: CombinatorialFreeModule

This class illustrates an implementation of a graded Hopf algebra with basis that has one primitive generator of degree 1 and basis elements indexed by nonnegative integers.

This Hopf algebra example differs from what topologists refer to as a graded Hopf algebra because the twist operation in the tensor rule satisfies

$$(\mu \otimes \mu) \circ (id \otimes \tau \otimes id) \circ (\Delta \otimes \Delta) = \Delta \circ \mu$$

where $\tau(x \otimes y) = y \otimes x$.

coproduct_on_basis(i)

The coproduct of a basis element.

$$\Delta(P_i) = \sum_{j=0}^{i} P_{i-j} \otimes P_j$$

INPUT:

- `i` – nonnegative integer

OUTPUT: an element of the tensor square of `self`

degree_on_basis(i)

The degree of a nonnegative integer is itself.

INPUT:

- `i` – nonnegative integer

OUTPUT: nonnegative integer

one_basis()

Return 0, which index the unit of the Hopf algebra.

OUTPUT: the nonnegative integer 0

EXAMPLES:

```
sage: H = GradedHopfAlgebrasWithBasis(QQ).Connected().example()
sage: H.one_basis()
0
sage: H.one()
P0
```

```
>>> from sage.all import *
>>> H = GradedHopfAlgebrasWithBasis(QQ).Connected().example()
>>> H.one_basis()
0
>>> H.one()
P0
```

product_on_basis(i, j)

The product of two basis elements.

The product of elements of degree `i` and `j` is an element of degree `i+j`.

INPUT:

- i, j – nonnegative integers

OUTPUT: a basis element indexed by i+j

6.16 Examples of graded modules with basis

sage.categories.examples.graded_modules_with_basis.**Example**

alias of *GradedPartitionModule*

class sage.categories.examples.graded_modules_with_basis.**GradedPartitionModule**(*base_ring*)

Bases: CombinatorialFreeModule

This class illustrates an implementation of a graded module with basis: the free module over partitions.

INPUT:

- R – base ring

The implementation involves the following:

- A choice of how to represent elements. In this case, the basis elements are partitions. The algebra is constructed as a CombinatorialFreeModule on the set of partitions, so it inherits all of the methods for such objects, and has operations like addition already defined.

```
sage: A = GradedModulesWithBasis(QQ).example()
↪# needs sage.modules
```

```
>>> from sage.all import *
>>> A = GradedModulesWithBasis(QQ).example()                                #␣
↪needs sage.modules
```

- A basis function - this module is graded by the nonnegative integers, so there is a function defined in this module, creatively called *basis()*, which takes an integer d as input and returns a family of partitions representing a basis for the algebra in degree d.

```
sage: A.basis(2)
↪# needs sage.modules
Lazy family (Term map from Partitions to An example of a graded module with␣
↪basis: the free module on partitions over Rational Field(i))_{i in␣
↪Partitions of the integer 2}
sage: A.basis(6)[Partition([3,2,1])]
↪# needs sage.modules
P[3, 2, 1]
```

```
>>> from sage.all import *
>>> A.basis(Integer(2))                                                     ␣
↪       # needs sage.modules
Lazy family (Term map from Partitions to An example of a graded module with␣
↪basis: the free module on partitions over Rational Field(i))_{i in␣
↪Partitions of the integer 2}
>>> A.basis(Integer(6))[Partition([Integer(3),Integer(2),Integer(1)])]      ␣
↪                                   # needs sage.modules
P[3, 2, 1]
```

- If the algebra is called `A`, then its basis function is stored as `A.basis`. Thus the function can be used to find a basis for the degree d piece: essentially, just call `A.basis(d)`. More precisely, call `x` for each `x` in `A.basis(d)`.

```
sage: [m for m in A.basis(4)]
↪# needs sage.modules
[P[4], P[3, 1], P[2, 2], P[2, 1, 1], P[1, 1, 1, 1]]
```

```
>>> from sage.all import *
>>> [m for m in A.basis(Integer(4))]
↪       # needs sage.modules
[P[4], P[3, 1], P[2, 2], P[2, 1, 1], P[1, 1, 1, 1]]
```

- For dealing with basis elements: *degree_on_basis()*, and `_repr_term()`. The first of these defines the degree of any monomial, and then the *degree* method for elements – see the next item – uses it to compute the degree for a linear combination of monomials. The last of these determines the print representation for monomials, which automatically produces the print representation for general elements.

```
sage: A.degree_on_basis(Partition([4,3]))
↪# needs sage.modules
7
sage: A._repr_term(Partition([4,3]))
↪# needs sage.modules
'P[4, 3]'
```

```
>>> from sage.all import *
>>> A.degree_on_basis(Partition([Integer(4),Integer(3)]))
↪                # needs sage.modules
7
>>> A._repr_term(Partition([Integer(4),Integer(3)]))
↪                # needs sage.modules
'P[4, 3]'
```

- There is a class for elements, which inherits from `IndexedFreeModuleElement`. An element is determined by a dictionary whose keys are partitions and whose corresponding values are the coefficients. The class implements two things: an *is_homogeneous* method and a *degree* method.

```
sage: p = A.monomial(Partition([3,2,1])); p
↪# needs sage.modules
P[3, 2, 1]
sage: p.is_homogeneous()
↪# needs sage.modules
True
sage: p.degree()
↪# needs sage.modules
6
```

```
>>> from sage.all import *
>>> p = A.monomial(Partition([Integer(3),Integer(2),Integer(1)])); p
↪                          # needs sage.modules
P[3, 2, 1]
>>> p.is_homogeneous()                                                #␣
↪needs sage.modules
```

(continues on next page)

(continued from previous page)

```
True
>>> p.degree()                                                              #␣
↪needs sage.modules
6
```

basis(*d=None*)

Return the basis for (the `d`-th homogeneous component of) `self`.

INPUT:

- `d` – (default: `None`) nonnegative integer or `None`

OUTPUT:

If `d` is `None`, returns the basis of the module. Otherwise, returns the basis of the homogeneous component of degree `d` (i.e., the subfamily of the basis of the whole module which consists only of the basis vectors lying in $F_d \setminus \bigcup_{i<d} F_i$).

The basis is always returned as a family.

EXAMPLES:

```
sage: A = ModulesWithBasis(ZZ).Filtered().example()
sage: A.basis(4)
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions of the integer 4}
```

```
>>> from sage.all import *
>>> A = ModulesWithBasis(ZZ).Filtered().example()
>>> A.basis(Integer(4))
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions of the integer 4}
```

Without arguments, the full basis is returned:

```
sage: A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
sage: A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
```

```
>>> from sage.all import *
>>> A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
>>> A.basis()
Lazy family (Term map from Partitions to An example of a
 filtered module with basis: the free module on partitions
 over Integer Ring(i))_{i in Partitions}
```

Checking this method on a filtered algebra. Note that this will typically raise a NotImplementedError when this feature is not implemented.

```
sage: A = AlgebrasWithBasis(ZZ).Filtered().example()
sage: A.basis(4)
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(ZZ).Filtered().example()
>>> A.basis(Integer(4))
Traceback (most recent call last):
...
NotImplementedError: infinite set
```

Without arguments, the full basis is returned:

```
sage: A.basis()
Lazy family (Term map from Free abelian monoid indexed by
 {'x', 'y', 'z'} to An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Integer Ring(i))_{i in
 Free abelian monoid indexed by {'x', 'y', 'z'}}
```

```
>>> from sage.all import *
>>> A.basis()
Lazy family (Term map from Free abelian monoid indexed by
 {'x', 'y', 'z'} to An example of a filtered algebra with
 basis: the universal enveloping algebra of Lie algebra
 of RR^3 with cross product over Integer Ring(i))_{i in
 Free abelian monoid indexed by {'x', 'y', 'z'}}
```

An example with a graded algebra:

```
sage: E.<x,y> = ExteriorAlgebra(QQ)
sage: E.basis()
Lazy family (Term map from Subsets of {0,1} to
 The exterior algebra of rank 2 over Rational Field(i))_{i in
 Subsets of {0,1}}
```

```
>>> from sage.all import *
>>> E = ExteriorAlgebra(QQ, names=('x', 'y',)); (x, y,) = E._first_ngens(2)
>>> E.basis()
Lazy family (Term map from Subsets of {0,1} to
 The exterior algebra of rank 2 over Rational Field(i))_{i in
 Subsets of {0,1}}
```

degree_on_basis(*t*)

The degree of the element determined by the partition t in this graded module.

INPUT:

- t – the index of an element of the basis of this module, i.e. a partition

OUTPUT: integer, the degree of the corresponding basis element

EXAMPLES:

```
sage: # needs sage.modules
sage: A = GradedModulesWithBasis(QQ).example()
sage: A.degree_on_basis(Partition((2,1)))
3
sage: A.degree_on_basis(Partition((4,2,1,1,1,1)))
10
sage: type(A.degree_on_basis(Partition((1,1))))
<class 'sage.rings.integer.Integer'>
```

```
>>> from sage.all import *
>>> # needs sage.modules
>>> A = GradedModulesWithBasis(QQ).example()
>>> A.degree_on_basis(Partition((Integer(2),Integer(1))))
3
>>> A.degree_on_basis(Partition((Integer(4),Integer(2),Integer(1),Integer(1),
↪Integer(1),Integer(1))))
10
>>> type(A.degree_on_basis(Partition((Integer(1),Integer(1)))))
<class 'sage.rings.integer.Integer'>
```

6.17 Examples of graphs

class sage.categories.examples.graphs.**Cycle**(*n=5*)

Bases: UniqueRepresentation, Parent

An example of a graph: the cycle of length n.

This class illustrates a minimal implementation of a graph.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example(); C
An example of a graph: the 5-cycle

sage: C.category()
Category of graphs
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example(); C
An example of a graph: the 5-cycle

>>> C.category()
Category of graphs
```

We conclude by running systematic tests on this graph:

```
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> TestSuite(C).run()
```

class Element

Bases: ElementWrapper

dimension()

Return the dimension of self.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: e = C.edges()[0]
sage: e.dimension()
2
sage: v = C.vertices()[0]
sage: v.dimension()
1
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> e = C.edges()[Integer(0)]
>>> e.dimension()
2
>>> v = C.vertices()[Integer(0)]
>>> v.dimension()
1
```

an_element()

Return an element of the graph, as per *Sets.ParentMethods.an_element()*.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.an_element()
0
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.an_element()
0
```

edges()

Return the edges of self.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
```

(continues on next page)

(continued from previous page)

```
sage: C.edges()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.edges()
[(0, 1), (1, 2), (2, 3), (3, 4), (4, 0)]
```

vertices()

Return the vertices of `self`.

EXAMPLES:

```
sage: from sage.categories.graphs import Graphs
sage: C = Graphs().example()
sage: C.vertices()
[0, 1, 2, 3, 4]
```

```
>>> from sage.all import *
>>> from sage.categories.graphs import Graphs
>>> C = Graphs().example()
>>> C.vertices()
[0, 1, 2, 3, 4]
```

sage.categories.examples.graphs.**Example**

alias of *Cycle*

6.18 Examples of Hopf algebras with basis

class sage.categories.examples.hopf_algebras_with_basis.**MyGroupAlgebra**(R, G)

Bases: CombinatorialFreeModule

An example of a Hopf algebra with basis: the group algebra of a group.

This class illustrates a minimal implementation of a Hopf algebra with basis.

algebra_generators()

Return the generators of this algebra, as per *algebra_generators()*.

They correspond to the generators of the group.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis: the group algebra of the Dihedral␣
↪group of order 6 as a permutation group over Rational Field
sage: A.algebra_generators()
Finite family {(1,2,3): B[(1,2,3)], (1,3): B[(1,3)]}
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example(); A
An example of Hopf algebra with basis: the group algebra of the Dihedral␣
```

(continues on next page)

(continued from previous page)

```
↪group of order 6 as a permutation group over Rational Field
>>> A.algebra_generators()
Finite family {(1,2,3): B[(1,2,3)], (1,3): B[(1,3)]}
```

antipode_on_basis(*g*)

Antipode, on basis elements, as per *HopfAlgebrasWithBasis.ParentMethods.antipode_on_basis()*.

It is given, on basis elements, by $\nu(g) = g^{-1}$

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: a, b = A._group.gens()
sage: A.antipode_on_basis(a)
B[(1,3,2)]
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> a, b = A._group.gens()
>>> A.antipode_on_basis(a)
B[(1,3,2)]
```

coproduct_on_basis(*g*)

Coproduct, on basis elements, as per *sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods.coproduct_on_basis()*.

The basis elements are group like: $\Delta(g) = g \otimes g$.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: a, b = A._group.gens()
sage: A.coproduct_on_basis(a)
B[(1,2,3)] # B[(1,2,3)]
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> a, b = A._group.gens()
>>> A.coproduct_on_basis(a)
B[(1,2,3)] # B[(1,2,3)]
```

counit_on_basis(*g*)

Counit, on basis elements, as per *sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods.counit_on_basis()*.

The counit on the basis elements is 1.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: a, b = A._group.gens()
sage: A.counit_on_basis(a)
1
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> a, b = A._group.gens()
>>> A.counit_on_basis(a)
1
```

one_basis()

Return the one of the group, which index the one of this algebra, as per *one_basis()*.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: A.one_basis()
()
sage: A.one()
B[()]
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> A.one_basis()
()
>>> A.one()
B[()]
```

product_on_basis(*g1*, *g2*)

Product, on basis elements, as per *product_on_basis()*.

The product of two basis elements is induced by the product of the corresponding elements of the group.

EXAMPLES:

```
sage: A = HopfAlgebrasWithBasis(QQ).example()
sage: a, b = A._group.gens()
sage: a*b
(1,2)
sage: A.product_on_basis(a, b)
B[(1,2)]
```

```
>>> from sage.all import *
>>> A = HopfAlgebrasWithBasis(QQ).example()
>>> a, b = A._group.gens()
>>> a*b
(1,2)
>>> A.product_on_basis(a, b)
B[(1,2)]
```

6.19 Examples of infinite enumerated sets

sage.categories.examples.infinite_enumerated_sets.**Example**

alias of *NonNegativeIntegers*

class sage.categories.examples.infinite_enumerated_sets.**NonNegativeIntegers**

Bases: UniqueRepresentation, Parent

An example of infinite enumerated set: the nonnegative integers.

This class provides a minimal implementation of an infinite enumerated set.

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN
An example of an infinite enumerated set: the nonnegative integers
sage: NN.cardinality()
+Infinity
sage: NN.list()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
sage: NN.element_class
<class 'sage.rings.integer.Integer'>
sage: it = iter(NN)
sage: [next(it), next(it), next(it), next(it), next(it)]
[0, 1, 2, 3, 4]
sage: x = next(it); type(x)
<class 'sage.rings.integer.Integer'>
sage: x.parent()
Integer Ring
sage: x+3
8
sage: NN(15)
15
sage: NN.first()
0
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN
An example of an infinite enumerated set: the nonnegative integers
>>> NN.cardinality()
+Infinity
>>> NN.list()
Traceback (most recent call last):
...
NotImplementedError: cannot list an infinite set
>>> NN.element_class
<class 'sage.rings.integer.Integer'>
>>> it = iter(NN)
>>> [next(it), next(it), next(it), next(it), next(it)]
[0, 1, 2, 3, 4]
>>> x = next(it); type(x)
<class 'sage.rings.integer.Integer'>
>>> x.parent()
Integer Ring
>>> x+Integer(3)
8
>>> NN(Integer(15))
15
```

(continues on next page)

(continued from previous page)

```
>>> NN.first()
0
```

This checks that the different methods of NN return consistent results:

```
sage: TestSuite(NN).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(NN).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_enumerated_set_contains() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_enumerated_set_iter_cardinality() . . . pass
running ._test_enumerated_set_iter_list() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

Element

alias of `Integer`

an_element()

EXAMPLES:

```
sage: InfiniteEnumeratedSets().example().an_element()
42
```

```
>>> from sage.all import *
>>> InfiniteEnumeratedSets().example().an_element()
42
```

next(o)

EXAMPLES:

```
sage: NN = InfiniteEnumeratedSets().example()
sage: NN.next(3)
4
```

```
>>> from sage.all import *
>>> NN = InfiniteEnumeratedSets().example()
>>> NN.next(Integer(3))
4
```

6.20 Examples of a Lie algebra

sage.categories.examples.lie_algebras.**Example**

alias of *LieAlgebraFromAssociative*

class sage.categories.examples.lie_algebras.**LieAlgebraFromAssociative**(*gens*)

Bases: `Parent`, `UniqueRepresentation`

An example of a Lie algebra: a Lie algebra generated by a set of elements of an associative algebra.

This class illustrates a minimal implementation of a Lie algebra.

Let R be a commutative ring, and A an associative R-algebra. The Lie algebra A (sometimes denoted A^-) is defined to be the R-module A with Lie bracket given by the commutator in A: that is, $[a, b] := ab - ba$ for all $a, b \in A$.

What this class implements is not precisely A^-, however; it is the Lie subalgebra of A^- generated by the elements of the iterable `gens`. This specific implementation does not provide a reasonable containment test (i.e., it does not allow you to check if a given element a of A^- belongs to this Lie subalgebra); it, however, allows computing inside it.

INPUT:

- gens – a nonempty iterable consisting of elements of an associative algebra A

OUTPUT:

The Lie subalgebra of A^- generated by the elements of gens

EXAMPLES:

We create a model of $\mathfrak{sl}_2$ using matrices:

```
sage: gens = [matrix([[0,1],[0,0]]), matrix([[0,0],[1,0]]), matrix([[1,0],[0,-
↪1]])]
sage: for g in gens:
....:     g.set_immutable()
sage: L = LieAlgebras(QQ).example(gens)
sage: e,f,h = L.lie_algebra_generators()
sage: e.bracket(f) == h
True
sage: h.bracket(e) == 2*e
True
sage: h.bracket(f) == -2*f
True
```

```
>>> from sage.all import *
>>> gens = [matrix([[Integer(0),Integer(1)],[Integer(0),Integer(0)]]),␣
↪matrix([[Integer(0),Integer(0)],[Integer(1),Integer(0)]]), matrix([[Integer(1),
↪Integer(0)],[Integer(0),-Integer(1)]])]
>>> for g in gens:
...     g.set_immutable()
>>> L = LieAlgebras(QQ).example(gens)
>>> e,f,h = L.lie_algebra_generators()
>>> e.bracket(f) == h
True
>>> h.bracket(e) == Integer(2)*e
True
>>> h.bracket(f) == -Integer(2)*f
True
```

class Element

Bases: ElementWrapper

Wrap an element as a Lie algebra element.

lie_algebra_generators()

Return the generators of self as a Lie algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.combinat sage.groups
sage: L.lie_algebra_generators()                                       #␣
↪needs sage.combinat sage.groups
Family ([2, 1, 3], [2, 3, 1])
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.combinat sage.groups
>>> L.lie_algebra_generators()                                       #␣
↪needs sage.combinat sage.groups
Family ([2, 1, 3], [2, 3, 1])
```

zero()

Return the element 0.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).example()                                  #␣
↪needs sage.combinat sage.groups
sage: L.zero()                                                       #␣
↪needs sage.combinat sage.groups
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).example()                                    #␣
↪needs sage.combinat sage.groups
>>> L.zero()                                                         #␣
↪needs sage.combinat sage.groups
0
```

6.21 Examples of a Lie algebra with basis

class sage.categories.examples.lie_algebras_with_basis.**AbelianLieAlgebra**(*R*, *gens*)

Bases: CombinatorialFreeModule

An example of a Lie algebra: the abelian Lie algebra.

This class illustrates a minimal implementation of a Lie algebra with a distinguished basis.

class Element

Bases: IndexedFreeModuleElement

lift()

Return the lift of `self` to the universal enveloping algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: elt = L.an_element()
sage: elt.lift()
3*P[F[2]] + 2*P[F[1]] + 2*P[F[]]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> elt = L.an_element()
>>> elt.lift()
3*P[F[2]] + 2*P[F[1]] + 2*P[F[]]
```

bracket_on_basis(*x*, *y*)

Return the Lie bracket on basis elements indexed by `x` and `y`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: L.bracket_on_basis(Partition([4,1]), Partition([2,2,1]))
0
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> L.bracket_on_basis(Partition([Integer(4),Integer(1)]),␣
↪Partition([Integer(2),Integer(2),Integer(1)]))
0
```

lie_algebra_generators()

Return the generators of `self` as a Lie algebra.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: L.lie_algebra_generators()
Lazy family (Term map from Partitions to
 An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Partitions over Rational
 Field(i))_{i in Partitions}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> L.lie_algebra_generators()
Lazy family (Term map from Partitions to
 An example of a Lie algebra: the abelian Lie algebra on the
 generators indexed by Partitions over Rational
 Field(i))_{i in Partitions}
```

sage.categories.examples.lie_algebras_with_basis.**Example**

alias of *AbelianLieAlgebra*

class sage.categories.examples.lie_algebras_with_basis.**IndexedPolynomialRing**(*R*, *indices*, ***kwds*)

Bases: `CombinatorialFreeModule`

Polynomial ring whose generators are indexed by an arbitrary set.

Todo

Currently this is just used as the universal enveloping algebra for the example of the abelian Lie algebra. This should be factored out into a more complete class.

algebra_generators()

Return the algebra generators of `self`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: UEA = L.universal_enveloping_algebra()
sage: UEA.algebra_generators()
Lazy family (algebra generator map(i))_{i in Partitions}
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> UEA = L.universal_enveloping_algebra()
>>> UEA.algebra_generators()
Lazy family (algebra generator map(i))_{i in Partitions}
```

one_basis()

Return the index of element 1.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: UEA = L.universal_enveloping_algebra()
sage: UEA.one_basis()
1
sage: UEA.one_basis().parent()
Free abelian monoid indexed by Partitions
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> UEA = L.universal_enveloping_algebra()
>>> UEA.one_basis()
1
>>> UEA.one_basis().parent()
Free abelian monoid indexed by Partitions
```

product_on_basis(*x*, *y*)

Return the product of the monomials indexed by `x` and `y`.

EXAMPLES:

```
sage: L = LieAlgebras(QQ).WithBasis().example()
sage: UEA = L.universal_enveloping_algebra()
sage: I = UEA._indices
sage: UEA.product_on_basis(I.an_element(), I.an_element())
P[F[]^4*F[1]^4*F[2]^6]
```

```
>>> from sage.all import *
>>> L = LieAlgebras(QQ).WithBasis().example()
>>> UEA = L.universal_enveloping_algebra()
>>> I = UEA._indices
>>> UEA.product_on_basis(I.an_element(), I.an_element())
P[F[]^4*F[1]^4*F[2]^6]
```

6.22 Examples of magmas

sage.categories.examples.magmas.**Example**

alias of *FreeMagma*

class sage.categories.examples.magmas.**FreeMagma**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: UniqueRepresentation, Parent

An example of magma.

The purpose of this class is to provide a minimal template for implementing a magma.

EXAMPLES:

```
sage: M = Magmas().example(); M
An example of a magma: the free magma generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> M = Magmas().example(); M
An example of a magma: the free magma generated by ('a', 'b', 'c', 'd')
```

This is the free magma generated by:

```
sage: M.magma_generators()
Family ('a', 'b', 'c', 'd')
sage: a, b, c, d = M.magma_generators()
```

```
>>> from sage.all import *
>>> M.magma_generators()
Family ('a', 'b', 'c', 'd')
>>> a, b, c, d = M.magma_generators()
```

and with a non-associative product given by:

```
sage: a * (b * c) * (d * a * b)
'((a*(b*c))*((d*a)*b))'
sage: a * (b * c) == (a * b) * c
False
```

```
>>> from sage.all import *
>>> a * (b * c) * (d * a * b)
'((a*(b*c))*((d*a)*b))'
>>> a * (b * c) == (a * b) * c
False
```

class Element

Bases: ElementWrapper

The class for elements of the free magma.

wrapped_class

alias of str

an_element()

Return an element of the magma.

EXAMPLES:

```
sage: F = Magmas().example()
sage: F.an_element()
'(((a*b)*c)*d)'
```

```
>>> from sage.all import *
>>> F = Magmas().example()
>>> F.an_element()
'(((a*b)*c)*d)'
```

magma_generators()

Return the generators of the magma.

EXAMPLES:

```
sage: F = Magmas().example()
sage: F.magma_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> F = Magmas().example()
>>> F.magma_generators()
Family ('a', 'b', 'c', 'd')
```

product(*x*, *y*)

Return the product of `x` and `y` in the magma, as per *Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: F = Magmas().example()
sage: F('a') * F.an_element()
'(a*(((a*b)*c)*d))'
```

```
>>> from sage.all import *
>>> F = Magmas().example()
>>> F('a') * F.an_element()
'(a*(((a*b)*c)*d))'
```

6.23 Examples of manifolds

sage.categories.examples.manifolds.**Example**

alias of *Plane*

class sage.categories.examples.manifolds.**Plane**(*n=3*, *base_ring=None*)

Bases: UniqueRepresentation, Parent

An example of a manifold: the n-dimensional plane.

This class illustrates a minimal implementation of a manifold.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: M = Manifolds(QQ).example(); M
```

(continues on next page)

(continued from previous page)

```
An example of a Rational Field manifold: the 3-dimensional plane

sage: M.category()
Category of manifolds over Rational Field
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> M = Manifolds(QQ).example(); M
An example of a Rational Field manifold: the 3-dimensional plane

>>> M.category()
Category of manifolds over Rational Field
```

We conclude by running systematic tests on this manifold:

```
sage: TestSuite(M).run()
```

```
>>> from sage.all import *
>>> TestSuite(M).run()
```

Element

alias of `ElementWrapper`

an_element()

Return an element of the manifold, as per *Sets.ParentMethods.an_element()*.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: M = Manifolds(QQ).example()
sage: M.an_element()
(0, 0, 0)
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> M = Manifolds(QQ).example()
>>> M.an_element()
(0, 0, 0)
```

dimension()

Return the dimension of `self`.

EXAMPLES:

```
sage: from sage.categories.manifolds import Manifolds
sage: M = Manifolds(QQ).example()
sage: M.dimension()
3
```

```
>>> from sage.all import *
>>> from sage.categories.manifolds import Manifolds
>>> M = Manifolds(QQ).example()
```

(continues on next page)

(continued from previous page)

```
>>> M.dimension()
3
```

6.24 Examples of monoids

sage.categories.examples.monoids.**Example**

alias of *FreeMonoid*

class sage.categories.examples.monoids.**FreeMonoid**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: *FreeSemigroup*

An example of a monoid: the free monoid.

This class illustrates a minimal implementation of a monoid. For a full featured implementation of free monoids, see *FreeMonoid()*.

EXAMPLES:

```
sage: S = Monoids().example(); S
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')

sage: S.category()
Category of monoids
```

```
>>> from sage.all import *
>>> S = Monoids().example(); S
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')

>>> S.category()
Category of monoids
```

This is the free semigroup generated by:

```
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

with product rule given by concatenation of words:

```
sage: S('dab') * S('acb')
'dabacb'
```

```
>>> from sage.all import *
>>> S('dab') * S('acb')
'dabacb'
```

and unit given by the empty word:

```
sage: S.one()
''
```

```
>>> from sage.all import *
>>> S.one()
''
```

We conclude by running systematic tests on this monoid:

```
sage: TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
```

(continues on next page)

(continued from previous page)

```
running ._test_not_implemented_methods() . . . pass
running ._test_one() . . . pass
running ._test_pickling() . . . pass
running ._test_prod() . . . pass
running ._test_some_elements() . . . pass
```

class Element

Bases: ElementWrapper

wrapped_class

alias of str

monoid_generators()

Return the generators of this monoid.

EXAMPLES:

```
sage: M = Monoids().example(); M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
sage: M.monoid_generators()
Finite family {'a': 'a', 'b': 'b', 'c': 'c', 'd': 'd'}
sage: a,b,c,d = M.monoid_generators()
sage: a*d*c*b
'adcb'
```

```
>>> from sage.all import *
>>> M = Monoids().example(); M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
>>> M.monoid_generators()
Finite family {'a': 'a', 'b': 'b', 'c': 'c', 'd': 'd'}
>>> a,b,c,d = M.monoid_generators()
>>> a*d*c*b
'adcb'
```

one()

Return the one of the monoid, as per *sage.categories.magmas.Magmas.Unital.ParentMethods.one()*.

EXAMPLES:

```
sage: M = Monoids().example(); M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
sage: M.one()
''
```

```
>>> from sage.all import *
>>> M = Monoids().example(); M
An example of a monoid: the free monoid generated by ('a', 'b', 'c', 'd')
>>> M.one()
''
```

6.25 Examples of posets

class sage.categories.examples.posets.**FiniteSetsOrderedByInclusion**

Bases: UniqueRepresentation, Parent

An example of a poset: finite sets ordered by inclusion.

This class provides a minimal implementation of a poset

EXAMPLES:

```
sage: P = Posets().example(); P
An example of a poset: sets ordered by inclusion
```

```
>>> from sage.all import *
>>> P = Posets().example(); P
An example of a poset: sets ordered by inclusion
```

We conclude by running systematic tests on this poset:

```
sage: TestSuite(P).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> TestSuite(P).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
```

(continues on next page)

(continued from previous page)

```
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

class Element

Bases: ElementWrapper

wrapped_class

alias of Set_object_enumerated

an_element()

Return an element of this poset.

EXAMPLES:

```
sage: B = Posets().example()
sage: B.an_element()
{1, 4, 6}
```

```
>>> from sage.all import *
>>> B = Posets().example()
>>> B.an_element()
{1, 4, 6}
```

le(x, y)

Return whether x is a subset of y.

EXAMPLES:

```
sage: P = Posets().example()
sage: P.le( P(Set([1,3])), P(Set([1,2,3])) )
True
sage: P.le( P(Set([1,3])), P(Set([1,3])) )
True
sage: P.le( P(Set([1,2])), P(Set([1,3])) )
False
```

```
>>> from sage.all import *
>>> P = Posets().example()
>>> P.le( P(Set([Integer(1),Integer(3)])), P(Set([Integer(1),Integer(2),
↪Integer(3)])) )
True
>>> P.le( P(Set([Integer(1),Integer(3)])), P(Set([Integer(1),Integer(3)])) )
True
```

(continues on next page)

(continued from previous page)

```
>>> P.le( P(Set([Integer(1),Integer(2)])), P(Set([Integer(1),Integer(3)])) )
False
```

class sage.categories.examples.posets.**PositiveIntegersOrderedByDivisibilityFacade**

Bases: UniqueRepresentation, Parent

An example of a facade poset: the positive integers ordered by divisibility.

This class provides a minimal implementation of a facade poset

EXAMPLES:

```
sage: P = Posets().example("facade"); P
An example of a facade poset: the positive integers ordered by divisibility

sage: P(5)
5
sage: P(0)
Traceback (most recent call last):
...
ValueError: Can't coerce `0` in any parent `An example of a facade poset: the␣
↪positive integers ordered by divisibility` is a facade for

sage: 3 in P
True
sage: 0 in P
False
```

```
>>> from sage.all import *
>>> P = Posets().example("facade"); P
An example of a facade poset: the positive integers ordered by divisibility

>>> P(Integer(5))
5
>>> P(Integer(0))
Traceback (most recent call last):
...
ValueError: Can't coerce `0` in any parent `An example of a facade poset: the␣
↪positive integers ordered by divisibility` is a facade for

>>> Integer(3) in P
True
>>> Integer(0) in P
False
```

class element_class(*X*, *category=None*)

Bases: Set_object_enumerated, *FiniteEnumeratedSets.parent_class*

A finite enumerated set.

le(*x*, *y*)

Return whether x is divisible by y.

EXAMPLES:

```
sage: P = Posets().example("facade")
sage: P.le(3, 6)
True
sage: P.le(3, 3)
True
sage: P.le(3, 7)
False
```

```
>>> from sage.all import *
>>> P = Posets().example("facade")
>>> P.le(Integer(3), Integer(6))
True
>>> P.le(Integer(3), Integer(3))
True
>>> P.le(Integer(3), Integer(7))
False
```

6.26 Examples of semigroups

class sage.categories.examples.semigroups.**FreeSemigroup**(*alphabet=('a', 'b', 'c', 'd')*)

Bases: UniqueRepresentation, Parent

An example of semigroup.

The purpose of this class is to provide a minimal template for implementing of a semigroup.

EXAMPLES:

```
sage: S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S = Semigroups().example("free"); S
An example of a semigroup: the free semigroup generated by ('a', 'b', 'c', 'd')
```

This is the free semigroup generated by:

```
sage: S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> S.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

and with product given by concatenation:

```
sage: S('dab') * S('acb')
'dabacb'
```

```
>>> from sage.all import *
>>> S('dab') * S('acb')
'dabacb'
```

class Element

Bases: ElementWrapper

The class for elements of the free semigroup.

wrapped_class

alias of str

an_element()

Return an element of the semigroup.

EXAMPLES:

```
sage: F = Semigroups().example('free')
sage: F.an_element()
'abcd'
```

```
>>> from sage.all import *
>>> F = Semigroups().example('free')
>>> F.an_element()
'abcd'
```

product(*x*, *y*)

Return the product of x and y in the semigroup, as per *sage.categories.magmas.Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: F = Semigroups().example('free')
sage: F.an_element() * F('a')^5
'abcdaaaaa'
```

```
>>> from sage.all import *
>>> F = Semigroups().example('free')
>>> F.an_element() * F('a')**Integer(5)
'abcdaaaaa'
```

semigroup_generators()

Return the generators of the semigroup.

EXAMPLES:

```
sage: F = Semigroups().example('free')
sage: F.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

```
>>> from sage.all import *
>>> F = Semigroups().example('free')
>>> F.semigroup_generators()
Family ('a', 'b', 'c', 'd')
```

class sage.categories.examples.semigroups.**IncompleteSubquotientSemigroup**(*category=None*)

Bases: UniqueRepresentation, Parent

An incompletely implemented subquotient semigroup, for testing purposes.

EXAMPLES:

```
sage: S = sage.categories.examples.semigroups.IncompleteSubquotientSemigroup()
sage: S
A subquotient of An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S = sage.categories.examples.semigroups.IncompleteSubquotientSemigroup()
>>> S
A subquotient of An example of a semigroup: the left zero semigroup
```

class Element

Bases: ElementWrapper

ambient()

Return the ambient semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S.ambient()
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S.ambient()
An example of a semigroup: the left zero semigroup
```

class sage.categories.examples.semigroups.**LeftZeroSemigroup**

Bases: UniqueRepresentation, Parent

An example of a semigroup.

This class illustrates a minimal implementation of a semigroup.

EXAMPLES:

```
sage: S = Semigroups().example(); S
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().example(); S
An example of a semigroup: the left zero semigroup
```

This is the semigroup that contains all sorts of objects:

```
sage: S.some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

```
>>> from sage.all import *
>>> S.some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

with product rule given by $a \times b = a$ for all a, b:

```
sage: S('hello') * S('world')
'hello'
sage: S(3)*S(1)*S(2)
3
sage: S(3)^12312321312321
3
```

```
>>> from sage.all import *
>>> S('hello') * S('world')
'hello'
>>> S(Integer(3))*S(Integer(1))*S(Integer(2))
3
>>> S(Integer(3))**Integer(12312321312321)
3
```

class Element

Bases: ElementWrapper

is_idempotent()

Trivial implementation of Semigroups.Element.is_idempotent since all elements of this semigroup are idempotent!

EXAMPLES:

```
sage: S = Semigroups().example()
sage: S.an_element().is_idempotent()
True
sage: S(17).is_idempotent()
True
```

```
>>> from sage.all import *
>>> S = Semigroups().example()
>>> S.an_element().is_idempotent()
True
>>> S(Integer(17)).is_idempotent()
True
```

an_element()

Return an element of the semigroup.

EXAMPLES:

```
sage: Semigroups().example().an_element()
42
```

```
>>> from sage.all import *
>>> Semigroups().example().an_element()
42
```

product(*x*, *y*)

Return the product of x and y in the semigroup, as per *sage.categories.magmas.Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: S = Semigroups().example()
sage: S('hello') * S('world')
'hello'
sage: S(3)*S(1)*S(2)
3
```

```
>>> from sage.all import *
>>> S = Semigroups().example()
>>> S('hello') * S('world')
'hello'
>>> S(Integer(3))*S(Integer(1))*S(Integer(2))
3
```

some_elements()

Return a list of some elements of the semigroup.

EXAMPLES:

```
sage: Semigroups().example().some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

```
>>> from sage.all import *
>>> Semigroups().example().some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

class sage.categories.examples.semigroups.**QuotientOfLeftZeroSemigroup**(*category=None*)

Bases: UniqueRepresentation, Parent

Example of a quotient semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example(); S
An example of a (sub)quotient semigroup: a quotient of the left zero semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example(); S
An example of a (sub)quotient semigroup: a quotient of the left zero semigroup
```

This is the quotient of:

```
sage: S.ambient()
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S.ambient()
An example of a semigroup: the left zero semigroup
```

obtained by setting $x = 42$ for any $x \geq 42$:

```
sage: S(100)
42
sage: S(100) == S(42)
True
```

```
>>> from sage.all import *
>>> S(Integer(100))
42
>>> S(Integer(100)) == S(Integer(42))
True
```

The product is inherited from the ambient semigroup:

```
sage: S(1)*S(2) == S(1)
True
```

```
>>> from sage.all import *
>>> S(Integer(1))*S(Integer(2)) == S(Integer(1))
True
```

class Element

Bases: ElementWrapper

ambient()

Return the ambient semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S.ambient()
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S.ambient()
An example of a semigroup: the left zero semigroup
```

an_element()

Return an element of the semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S.an_element()
42
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S.an_element()
42
```

lift(*x*)

Lift the element `x` into the ambient semigroup.

INPUT:

- `x` – an element of `self`

OUTPUT:

- an element of `self.ambient()`.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: x = S.an_element(); x
42
sage: S.lift(x)
42
sage: S.lift(x) in S.ambient()
True
sage: y = S.ambient()(100); y
100
sage: S.lift(S(y))
42
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> x = S.an_element(); x
42
>>> S.lift(x)
42
>>> S.lift(x) in S.ambient()
True
>>> y = S.ambient()(Integer(100)); y
100
>>> S.lift(S(y))
42
```

retract(*x*)

Return the retract `x` onto an element of this semigroup.

INPUT:

- `x` – an element of the ambient semigroup (`self.ambient()`)

OUTPUT: an element of `self`

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: L = S.ambient()
sage: S.retract(L(17))
17
sage: S.retract(L(42))
42
sage: S.retract(L(171))
42
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> L = S.ambient()
>>> S.retract(L(Integer(17)))
17
>>> S.retract(L(Integer(42)))
42
```

(continues on next page)

(continued from previous page)

```
>>> S.retract(L(Integer(171)))
42
```

some_elements()

Return a list of some elements of the semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S.some_elements()
[1, 2, 3, 8, 42, 42]
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S.some_elements()
[1, 2, 3, 8, 42, 42]
```

the_answer()

Return the Answer to Life, the Universe, and Everything as an element of this semigroup.

EXAMPLES:

```
sage: S = Semigroups().Subquotients().example()
sage: S.the_answer()
42
```

```
>>> from sage.all import *
>>> S = Semigroups().Subquotients().example()
>>> S.the_answer()
42
```

6.27 Examples of semigroups in cython

class sage.categories.examples.semigroups_cython.**IdempotentSemigroups**

Bases: *Category*

class ElementMethods

Bases: object

is_idempotent()

EXAMPLES:

```
sage: from sage.categories.examples.semigroups_cython import␣
↪LeftZeroSemigroup
sage: S = LeftZeroSemigroup()
sage: S(2).is_idempotent()
True
```

```
>>> from sage.all import *
>>> from sage.categories.examples.semigroups_cython import␣
↪LeftZeroSemigroup
```

(continues on next page)

(continued from previous page)

```
>>> S = LeftZeroSemigroup()
>>> S(Integer(2)).is_idempotent()
True
```

super_categories()

EXAMPLES:

```
sage: from sage.categories.examples.semigroups_cython import␣
↪IdempotentSemigroups
sage: IdempotentSemigroups().super_categories()
[Category of semigroups]
```

```
>>> from sage.all import *
>>> from sage.categories.examples.semigroups_cython import␣
↪IdempotentSemigroups
>>> IdempotentSemigroups().super_categories()
[Category of semigroups]
```

class sage.categories.examples.semigroups_cython.**LeftZeroSemigroup**

Bases: *LeftZeroSemigroup*

An example of semigroup.

This class illustrates a minimal implementation of a semi-group where the element class is an extension type, and still gets code from the category. The category itself must be a Python class though.

This is purely a proof of concept. The code obviously needs refactorisation!

Comments:

- one cannot play ugly class surgery tricks (as with _mul_parent). available operations should really be declared to the coercion model!

EXAMPLES:

```
sage: from sage.categories.examples.semigroups_cython import LeftZeroSemigroup
sage: S = LeftZeroSemigroup(); S
An example of a semigroup: the left zero semigroup
```

```
>>> from sage.all import *
>>> from sage.categories.examples.semigroups_cython import LeftZeroSemigroup
>>> S = LeftZeroSemigroup(); S
An example of a semigroup: the left zero semigroup
```

This is the semigroup which contains all sort of objects:

```
sage: S.some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

```
>>> from sage.all import *
>>> S.some_elements()
[3, 42, 'a', 3.4, 'raton laveur']
```

with product rule given by $a \times b = a$ for all a, b.

```
sage: S('hello') * S('world')
'hello'

sage: S(3)*S(1)*S(2)
3

sage: S(3)^12312321312321
3

sage: TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> S('hello') * S('world')
'hello'

>>> S(Integer(3))*S(Integer(1))*S(Integer(2))
3

>>> S(Integer(3))**Integer(12312321312321)
3

>>> TestSuite(S).run(verbose = True)
running ._test_an_element() . . . pass
running ._test_associativity() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
```

(continues on next page)

(continued from previous page)

```
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

That's really the only method which is obtained from the category …

```
sage: S(42).is_idempotent
<bound method IdempotentSemigroups.ElementMethods.is_idempotent of 42>
sage: S(42).is_idempotent()
True

sage: S(42)._pow_int
<bound method IdempotentSemigroups.ElementMethods._pow_int of 42>
sage: S(42)^10
42

sage: S(42).is_idempotent
<bound method IdempotentSemigroups.ElementMethods.is_idempotent of 42>
sage: S(42).is_idempotent()
True
```

```
>>> from sage.all import *
>>> S(Integer(42)).is_idempotent
<bound method IdempotentSemigroups.ElementMethods.is_idempotent of 42>
>>> S(Integer(42)).is_idempotent()
True

>>> S(Integer(42))._pow_int
<bound method IdempotentSemigroups.ElementMethods._pow_int of 42>
>>> S(Integer(42))**Integer(10)
42

>>> S(Integer(42)).is_idempotent
<bound method IdempotentSemigroups.ElementMethods.is_idempotent of 42>
>>> S(Integer(42)).is_idempotent()
True
```

Element

alias of *LeftZeroSemigroupElement*

class sage.categories.examples.semigroups_cython.**LeftZeroSemigroupElement**

Bases: Element

EXAMPLES:

```
sage: from sage.categories.examples.semigroups_cython import LeftZeroSemigroup
sage: S = LeftZeroSemigroup()
sage: x = S(3)
sage: TestSuite(x).run()
```

```
>>> from sage.all import *
>>> from sage.categories.examples.semigroups_cython import LeftZeroSemigroup
>>> S = LeftZeroSemigroup()
>>> x = S(Integer(3))
>>> TestSuite(x).run()
```

6.28 Examples of semirings

sage.categories.examples.semirings.**Example**

alias of *TernaryLogic*

class sage.categories.examples.semirings.**Ternary**(*parent*, *n*)

Bases: Element

Elements of the ternary-logic ring.

The semantic is as follows:

- 0 – the integer 0
- 1 – the integer 1
- 2 – some integer greater than 1

An alternative semantic is:

- 0 – an empty set
- 1 – a connected set
- 2 – a disconnected set

The same semantic works for graphs instead of sets.

class sage.categories.examples.semirings.**TernaryLogic**

Bases: UniqueRepresentation, Parent

An example of a semiring.

This class illustrates a minimal implementation of a semiring.

EXAMPLES:

```
sage: S = Semirings().example(); S
An example of a semiring: the ternary-logic semiring
```

```
>>> from sage.all import *
>>> S = Semirings().example(); S
An example of a semiring: the ternary-logic semiring
```

This is the semiring that contains 3 objects:

```
sage: S.some_elements()
[0, 1, many]
```

```
>>> from sage.all import *
>>> S.some_elements()
[0, 1, many]
```

The product rule is as expected:

```
sage: S(1) * S(1)
1
sage: S(1) + S(1)
many
```

```
>>> from sage.all import *
>>> S(Integer(1)) * S(Integer(1))
1
>>> S(Integer(1)) + S(Integer(1))
many
```

Element

alias of *Ternary*

an_element()

Return an element of the semiring.

EXAMPLES:

```
sage: Semirings().example().an_element()
many
```

```
>>> from sage.all import *
>>> Semirings().example().an_element()
many
```

one()

Return the unit of `self`.

EXAMPLES:

```
sage: S = Semirings().example()
sage: S.one()
1
```

```
>>> from sage.all import *
>>> S = Semirings().example()
>>> S.one()
1
```

product(*x*, *y*)

Return the product of `x` and `y` in the semiring as per *sage.categories.magmas.Magmas.ParentMethods.product()*.

EXAMPLES:

```
sage: S = Semirings().example()
sage: S(1) * S(2)
many
```

```
>>> from sage.all import *
>>> S = Semirings().example()
>>> S(Integer(1)) * S(Integer(2))
many
```

some_elements()

Return a list of some elements of the semiring.

EXAMPLES:

```
sage: Semirings().example().some_elements()
[0, 1, many]
```

```
>>> from sage.all import *
>>> Semirings().example().some_elements()
[0, 1, many]
```

summation(*x*, *y*)

Return the sum of `x` and `y` in the semiring as per *sage.categories.additive_magmas.AdditiveMagmas.ParentMethods.summation()*.

EXAMPLES:

```
sage: S = Semirings().example()
sage: S(1) + S(1)
many
```

```
>>> from sage.all import *
>>> S = Semirings().example()
>>> S(Integer(1)) + S(Integer(1))
many
```

6.29 Examples of sets

class sage.categories.examples.sets_cat.**PrimeNumbers**

Bases: UniqueRepresentation, Parent

An example of parent in the category of sets: the set of prime numbers.

The elements are represented as plain integers in **Z** (facade implementation).

This is a minimal implementations. For more advanced examples of implementations, see also:

```
sage: P = Sets().example("facade")
sage: P = Sets().example("inherits")
sage: P = Sets().example("wrapper")
```

```
>>> from sage.all import *
>>> P = Sets().example("facade")
>>> P = Sets().example("inherits")
>>> P = Sets().example("wrapper")
```

EXAMPLES:

```
sage: P = Sets().example()
sage: P(12)
Traceback (most recent call last):
...
AssertionError: 12 is not a prime number
sage: a = P.an_element()
sage: a.parent()
Integer Ring
sage: x = P(13); x
13
sage: type(x)
<class 'sage.rings.integer.Integer'>
sage: x.parent()
Integer Ring
sage: 13 in P
True
sage: 12 in P
False
sage: y = x+1; y
14
sage: type(y)
<class 'sage.rings.integer.Integer'>

sage: TestSuite(P).run(verbose=True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> P = Sets().example()
>>> P(Integer(12))
Traceback (most recent call last):
...
AssertionError: 12 is not a prime number
>>> a = P.an_element()
>>> a.parent()
Integer Ring
>>> x = P(Integer(13)); x
13
>>> type(x)
<class 'sage.rings.integer.Integer'>
>>> x.parent()
Integer Ring
>>> Integer(13) in P
True
>>> Integer(12) in P
False
>>> y = x+Integer(1); y
14
>>> type(y)
<class 'sage.rings.integer.Integer'>

>>> TestSuite(P).run(verbose=True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_nonzero_equal() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

an_element()

Implement *Sets.ParentMethods.an_element()*.

element_class

alias of Integer

class sage.categories.examples.sets_cat.**PrimeNumbers_Abstract**

Bases: UniqueRepresentation, Parent

This class shows how to write a parent while keeping the choice of the datastructure for the children open. Different class with fixed datastructure will then be constructed by inheriting from *PrimeNumbers_Abstract*.

This is used by:

```
sage: P = Sets().example("facade")
sage: P = Sets().example("inherits")
sage: P = Sets().example("wrapper")
```

```
>>> from sage.all import *
>>> P = Sets().example("facade")
>>> P = Sets().example("inherits")
>>> P = Sets().example("wrapper")
```

class Element

Bases: Element

is_prime()

Return whether `self` is a prime number.

EXAMPLES:

```
sage: P = Sets().example("inherits")
sage: x = P.an_element()
sage: P.an_element().is_prime()
True
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> x = P.an_element()
>>> P.an_element().is_prime()
True
```

next()

Return the next prime number.

EXAMPLES:

```
sage: P = Sets().example("inherits")
sage: p = P.an_element(); p
47
sage: p.next()
53
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> p = P.an_element(); p
47
>>> p.next()
53
```

> **Note**
>
> This method is not meant to implement the protocol iterator.

an_element()

Implement *Sets.ParentMethods.an_element()*.

next(*i*)

Return the next prime number.

EXAMPLES:

```
sage: P = Sets().example("inherits")
sage: x = P.next(P.an_element()); x
53
sage: x.parent()
Set of prime numbers
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> x = P.next(P.an_element()); x
53
>>> x.parent()
Set of prime numbers
```

some_elements()

Return some prime numbers.

EXAMPLES:

```
sage: P = Sets().example("inherits")
sage: P.some_elements()
[47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> P.some_elements()
[47, 53, 59, 61, 67, 71, 73, 79, 83, 89, 97]
```

class sage.categories.examples.sets_cat.**PrimeNumbers_Facade**

Bases: *PrimeNumbers_Abstract*

An example of parent in the category of sets: the set of prime numbers.

In this alternative implementation, the elements are represented as plain integers in **Z** (facade implementation).

EXAMPLES:

```
sage: P = Sets().example("facade")
sage: P(12)
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
sage: a = P.an_element()
```

(continues on next page)

(continued from previous page)

```
sage: a.parent()
Integer Ring
sage: x = P(13); x
13
sage: type(x)
<class 'sage.rings.integer.Integer'>
sage: x.parent()
Integer Ring
sage: 13 in P
True
sage: 12 in P
False
sage: y = x+1; y
14
sage: type(y)
<class 'sage.rings.integer.Integer'>

sage: z = P.next(x); z
17
sage: type(z)
<class 'sage.rings.integer.Integer'>
sage: z.parent()
Integer Ring
```

```
>>> from sage.all import *
>>> P = Sets().example("facade")
>>> P(Integer(12))
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
>>> a = P.an_element()
>>> a.parent()
Integer Ring
>>> x = P(Integer(13)); x
13
>>> type(x)
<class 'sage.rings.integer.Integer'>
>>> x.parent()
Integer Ring
>>> Integer(13) in P
True
>>> Integer(12) in P
False
>>> y = x+Integer(1); y
14
>>> type(y)
<class 'sage.rings.integer.Integer'>

>>> z = P.next(x); z
17
>>> type(z)
<class 'sage.rings.integer.Integer'>
```

(continues on next page)

(continued from previous page)

```
>>> z.parent()
Integer Ring
```

The disadvantage of this implementation is that the elements do not know that they are prime, so that prime testing is slow:

```
sage: pf = Sets().example("facade").an_element()
sage: timeit("pf.is_prime()") #    random
625 loops, best of 3: 4.1 us per loop
```

```
>>> from sage.all import *
>>> pf = Sets().example("facade").an_element()
>>> timeit("pf.is_prime()") #    random
625 loops, best of 3: 4.1 us per loop
```

compared to the other implementations where prime testing is only done if needed during the construction of the element, and later on the elements "know" that they are prime:

```
sage: pw = Sets().example("wrapper").an_element()
sage: timeit("pw.is_prime()")    # random
625 loops, best of 3: 859 ns per loop

sage: pi = Sets().example("inherits").an_element()
sage: timeit("pw.is_prime()")    # random
625 loops, best of 3: 854 ns per loop
```

```
>>> from sage.all import *
>>> pw = Sets().example("wrapper").an_element()
>>> timeit("pw.is_prime()")    # random
625 loops, best of 3: 859 ns per loop

>>> pi = Sets().example("inherits").an_element()
>>> timeit("pw.is_prime()")    # random
625 loops, best of 3: 854 ns per loop
```

Note also that the `next` method for the elements does not exist:

```
sage: pf.next()
Traceback (most recent call last):
...
AttributeError: 'sage.rings.integer.Integer' object has no attribute 'next'...
```

```
>>> from sage.all import *
>>> pf.next()
Traceback (most recent call last):
...
AttributeError: 'sage.rings.integer.Integer' object has no attribute 'next'...
```

unlike in the other implementations:

```
sage: pw.next()
53
```

(continues on next page)

(continued from previous page)

```
sage: pi.next()
53
```

```
>>> from sage.all import *
>>> pw.next()
53
>>> pi.next()
53
```

element_class

alias of `Integer`

class sage.categories.examples.sets_cat.**PrimeNumbers_Inherits**

Bases: *PrimeNumbers_Abstract*

An example of parent in the category of sets: the set of prime numbers. In this implementation, the element are stored as object of a new class which inherits from the class Integer (technically `IntegerWrapper`).

EXAMPLES:

```
sage: P = Sets().example("inherits")
sage: P
Set of prime numbers
sage: P(12)
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
sage: a = P.an_element()
sage: a.parent()
Set of prime numbers
sage: x = P(13); x
13
sage: x.is_prime()
True
sage: type(x)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
sage: x.parent()
Set of prime numbers
sage: P(13) in P
True
sage: y = x+1; y
14
sage: type(y)
<class 'sage.rings.integer.Integer'>
sage: y.parent()
Integer Ring
sage: type(P(13)+P(17))
<class 'sage.rings.integer.Integer'>
sage: type(P(2)+P(3))
<class 'sage.rings.integer.Integer'>

sage: z = P.next(x); z
```

(continues on next page)

(continued from previous page)

```
17
sage: type(z)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
sage: z.parent()
Set of prime numbers

sage: TestSuite(P).run(verbose=True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

```
>>> from sage.all import *
>>> P = Sets().example("inherits")
>>> P
Set of prime numbers
>>> P(Integer(12))
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
>>> a = P.an_element()
>>> a.parent()
Set of prime numbers
>>> x = P(Integer(13)); x
13
>>> x.is_prime()
True
>>> type(x)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
>>> x.parent()
Set of prime numbers
>>> P(Integer(13)) in P
True
```

(continues on next page)

(continued from previous page)

```
>>> y = x+Integer(1); y
14
>>> type(y)
<class 'sage.rings.integer.Integer'>
>>> y.parent()
Integer Ring
>>> type(P(Integer(13))+P(Integer(17)))
<class 'sage.rings.integer.Integer'>
>>> type(P(Integer(2))+P(Integer(3)))
<class 'sage.rings.integer.Integer'>

>>> z = P.next(x); z
17
>>> type(z)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Inherits_with_category.
↪element_class'>
>>> z.parent()
Set of prime numbers

>>> TestSuite(P).run(verbose=True)
running ._test_an_element() . . . pass
running ._test_cardinality() . . . pass
running ._test_category() . . . pass
running ._test_construction() . . . pass
running ._test_elements() . . .
  Running the test suite of self.an_element()
  running ._test_category() . . . pass
  running ._test_eq() . . . pass
  running ._test_new() . . . pass
  running ._test_not_implemented_methods() . . . pass
  running ._test_pickling() . . . pass
  pass
running ._test_elements_eq_reflexive() . . . pass
running ._test_elements_eq_symmetric() . . . pass
running ._test_elements_eq_transitive() . . . pass
running ._test_elements_neq() . . . pass
running ._test_eq() . . . pass
running ._test_new() . . . pass
running ._test_not_implemented_methods() . . . pass
running ._test_pickling() . . . pass
running ._test_some_elements() . . . pass
```

See also:

```
sage: P = Sets().example("facade")
sage: P = Sets().example("inherits")
sage: P = Sets().example("wrapper")
```

```
>>> from sage.all import *
>>> P = Sets().example("facade")
>>> P = Sets().example("inherits")
>>> P = Sets().example("wrapper")
```

class Element(*parent*, *p*)

Bases: IntegerWrapper, *Element*

class sage.categories.examples.sets_cat.**PrimeNumbers_Wrapper**

Bases: *PrimeNumbers_Abstract*

An example of parent in the category of sets: the set of prime numbers.

In this second alternative implementation, the prime integer are stored as a attribute of a sage object by inheriting from *ElementWrapper*. In this case we need to ensure conversion and coercion from this parent and its element to ZZ and Integer.

EXAMPLES:

```
sage: P = Sets().example("wrapper")
sage: P(12)
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
sage: a = P.an_element()
sage: a.parent()
Set of prime numbers (wrapper implementation)
sage: x = P(13); x
13
sage: type(x)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Wrapper_with_category.
↪element_class'>
sage: x.parent()
Set of prime numbers (wrapper implementation)
sage: 13 in P
True
sage: 12 in P
False
sage: y = x+1; y
14
sage: type(y)
<class 'sage.rings.integer.Integer'>

sage: z = P.next(x); z
17
sage: type(z)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Wrapper_with_category.
↪element_class'>
sage: z.parent()
Set of prime numbers (wrapper implementation)
```

```
>>> from sage.all import *
>>> P = Sets().example("wrapper")
>>> P(Integer(12))
Traceback (most recent call last):
...
ValueError: 12 is not a prime number
>>> a = P.an_element()
>>> a.parent()
Set of prime numbers (wrapper implementation)
```

(continues on next page)

(continued from previous page)

```
>>> x = P(Integer(13)); x
13
>>> type(x)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Wrapper_with_category.
↪element_class'>
>>> x.parent()
Set of prime numbers (wrapper implementation)
>>> Integer(13) in P
True
>>> Integer(12) in P
False
>>> y = x+Integer(1); y
14
>>> type(y)
<class 'sage.rings.integer.Integer'>

>>> z = P.next(x); z
17
>>> type(z)
<class 'sage.categories.examples.sets_cat.PrimeNumbers_Wrapper_with_category.
↪element_class'>
>>> z.parent()
Set of prime numbers (wrapper implementation)
```

class Element

Bases: ElementWrapper, *Element*

ElementWrapper

alias of ElementWrapper

6.30 Example of a set with grading

sage.categories.examples.sets_with_grading.**Example**

alias of *NonNegativeIntegers*

class sage.categories.examples.sets_with_grading.**NonNegativeIntegers**

Bases: UniqueRepresentation, Parent

Non negative integers graded by themselves.

EXAMPLES:

```
sage: E = SetsWithGrading().example(); E
Non negative integers
sage: E in Sets().Infinite()
True
sage: E.graded_component(0)
{0}
sage: E.graded_component(100)
{100}
```

```
>>> from sage.all import *
>>> E = SetsWithGrading().example(); E
Non negative integers
>>> E in Sets().Infinite()
True
>>> E.graded_component(Integer(0))
{0}
>>> E.graded_component(Integer(100))
{100}
```

an_element()

Return 0.

EXAMPLES:

```
sage: SetsWithGrading().example().an_element()
0
```

```
>>> from sage.all import *
>>> SetsWithGrading().example().an_element()
0
```

generating_series(*var='z'*)

Return $1/(1-z)$.

EXAMPLES:

```
sage: N = SetsWithGrading().example(); N
Non negative integers
sage: f = N.generating_series(); f
1/(-z + 1)
sage: LaurentSeriesRing(ZZ,'z')(f)
1 + z + z^2 + z^3 + z^4 + z^5 + z^6 + z^7 + z^8 + z^9 + z^10 + z^11 + z^12 +␣
↪z^13 + z^14 + z^15 + z^16 + z^17 + z^18 + z^19 + O(z^20)
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example(); N
Non negative integers
>>> f = N.generating_series(); f
1/(-z + 1)
>>> LaurentSeriesRing(ZZ,'z')(f)
1 + z + z^2 + z^3 + z^4 + z^5 + z^6 + z^7 + z^8 + z^9 + z^10 + z^11 + z^12 +␣
↪z^13 + z^14 + z^15 + z^16 + z^17 + z^18 + z^19 + O(z^20)
```

graded_component(*grade*)

Return the component with grade `grade`.

EXAMPLES:

```
sage: N = SetsWithGrading().example()
sage: N.graded_component(65)
{65}
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example()
>>> N.graded_component(Integer(65))
{65}
```

grading(*elt*)

Return the grade of `elt`.

EXAMPLES:

```
sage: N = SetsWithGrading().example()
sage: N.grading(10)
10
```

```
>>> from sage.all import *
>>> N = SetsWithGrading().example()
>>> N.grading(Integer(10))
10
```

6.31 Examples of parents endowed with multiple realizations

class sage.categories.examples.with_realizations.**SubsetAlgebra**(*R*, *S*)

Bases: `UniqueRepresentation`, `Parent`

An example of parent endowed with several realizations.

We consider an algebra $A(S)$ whose bases are indexed by the subsets s of a given set S. We consider three natural basis of this algebra: `F`, `In`, and `Out`. In the first basis, the product is given by the union of the indexing sets. That is, for any $s, t \subset S$

$$F_s F_t = F_{s \cup t}$$

The `In` basis and `Out` basis are defined respectively by:

$$In_s = \sum_{t \subset s} F_t \qquad \text{and} \qquad F_s = \sum_{t \supset s} Out_t$$

Each such basis gives a realization of A, where the elements are represented by their expansion in this basis.

This parent, and its code, demonstrate how to implement this algebra and its three realizations, with coercions and mixed arithmetic between them.

See also

- *Sets().WithRealizations*
- the Implementing Algebraic Structures thematic tutorial.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: A.base_ring()
Rational Field
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> A.base_ring()
Rational Field
```

The three bases of `A`:

```
sage: F   = A.F()  ; F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
sage: In  = A.In() ; In
The subset algebra of {1, 2, 3} over Rational Field in the In basis
sage: Out = A.Out(); Out
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

```
>>> from sage.all import *
>>> F   = A.F()  ; F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
>>> In  = A.In() ; In
The subset algebra of {1, 2, 3} over Rational Field in the In basis
>>> Out = A.Out(); Out
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

One can quickly define all the bases using the following shortcut:

```
sage: A.inject_shorthands()
Defining F as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the Fundamental basis
Defining In as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the In basis
Defining Out as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the Out basis
```

```
>>> from sage.all import *
>>> A.inject_shorthands()
Defining F as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the Fundamental basis
Defining In as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the In basis
Defining Out as shorthand for The subset algebra of {1, 2, 3} over Rational Field␣
↪in the Out basis
```

Accessing the basis elements is done with the *basis()* method:

```
sage: F.basis().list()
[F[{}], F[{1}], F[{2}], F[{3}], F[{1, 2}], F[{1, 3}], F[{2, 3}], F[{1, 2, 3}]]
```

```
>>> from sage.all import *
>>> F.basis().list()
[F[{}], F[{1}], F[{2}], F[{3}], F[{1, 2}], F[{1, 3}], F[{2, 3}], F[{1, 2, 3}]]
```

To access a particular basis element, you can use the *from_set()* method:

```
sage: F.from_set(2,3)
F[{2, 3}]
sage: In.from_set(1,3)
In[{1, 3}]
```

```
>>> from sage.all import *
>>> F.from_set(Integer(2),Integer(3))
F[{2, 3}]
>>> In.from_set(Integer(1),Integer(3))
In[{1, 3}]
```

or as a convenient shorthand, one can use the following notation:

```
sage: F[2,3]
F[{2, 3}]
sage: In[1,3]
In[{1, 3}]
```

```
>>> from sage.all import *
>>> F[Integer(2),Integer(3)]
F[{2, 3}]
>>> In[Integer(1),Integer(3)]
In[{1, 3}]
```

Some conversions:

```
sage: F(In[2,3])
F[{}] + F[{2}] + F[{3}] + F[{2, 3}]
sage: In(F[2,3])
In[{}] - In[{2}] - In[{3}] + In[{2, 3}]

sage: Out(F[3])
Out[{3}] + Out[{1, 3}] + Out[{2, 3}] + Out[{1, 2, 3}]
sage: F(Out[3])
F[{3}] - F[{1, 3}] - F[{2, 3}] + F[{1, 2, 3}]

sage: Out(In[2,3])
Out[{}] + Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}] + 2*Out[{1, 3}] +␣
↪4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> F(In[Integer(2),Integer(3)])
F[{}] + F[{2}] + F[{3}] + F[{2, 3}]
>>> In(F[Integer(2),Integer(3)])
In[{}] - In[{2}] - In[{3}] + In[{2, 3}]

>>> Out(F[Integer(3)])
Out[{3}] + Out[{1, 3}] + Out[{2, 3}] + Out[{1, 2, 3}]
>>> F(Out[Integer(3)])
F[{3}] - F[{1, 3}] - F[{2, 3}] + F[{1, 2, 3}]

>>> Out(In[Integer(2),Integer(3)])
```

(continues on next page)

(continued from previous page)

```
Out[{}] + Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}] + 2*Out[{1, 3}] +␣
↪4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

We can now mix expressions:

```
sage: (1 + Out[1]) * In[2,3]
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}] + 2*Out[{1, 3}] +␣
↪4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> (Integer(1) + Out[Integer(1)]) * In[Integer(2),Integer(3)]
Out[{}] + 2*Out[{1}] + 2*Out[{2}] + 2*Out[{3}] + 2*Out[{1, 2}] + 2*Out[{1, 3}] +␣
↪4*Out[{2, 3}] + 4*Out[{1, 2, 3}]
```

class Bases(*parent_with_realization*)

Bases: *Category_realization_of_parent*

The category of the realizations of the subset algebra

class ParentMethods

Bases: object

from_set(**args*)

Construct the monomial indexed by the set containing the elements passed as arguments.

EXAMPLES:

```
sage: In = Sets().WithRealizations().example().In(); In
The subset algebra of {1, 2, 3} over Rational Field in the In basis
sage: In.from_set(2,3)
In[{2, 3}]
```

```
>>> from sage.all import *
>>> In = Sets().WithRealizations().example().In(); In
The subset algebra of {1, 2, 3} over Rational Field in the In basis
>>> In.from_set(Integer(2),Integer(3))
In[{2, 3}]
```

As a shorthand, one can construct elements using the following notation:

```
sage: In[2,3]
In[{2, 3}]
```

```
>>> from sage.all import *
>>> In[Integer(2),Integer(3)]
In[{2, 3}]
```

one()

Return the unit of this algebra.

This default implementation takes the unit in the fundamental basis, and coerces it in `self`.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: In = A.In(); Out = A.Out()
sage: In.one()
In[{}]
sage: Out.one()
Out[{}] + Out[{1}] + Out[{2}] + Out[{3}] + Out[{1, 2}] + Out[{1, 3}] +␣
↪Out[{2, 3}] + Out[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> In = A.In(); Out = A.Out()
>>> In.one()
In[{}]
>>> Out.one()
Out[{}] + Out[{1}] + Out[{2}] + Out[{3}] + Out[{1, 2}] + Out[{1, 3}] +␣
↪Out[{2, 3}] + Out[{1, 2, 3}]
```

super_categories()

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: C = A.Bases(); C
Category of bases of The subset algebra of {1, 2, 3} over Rational Field
sage: C.super_categories()
[Category of realizations of The subset algebra of {1, 2, 3} over␣
↪Rational Field,
 Join of Category of algebras with basis over Rational Field and
         Category of commutative algebras over Rational Field and
         Category of realizations of unital magmas]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> C = A.Bases(); C
Category of bases of The subset algebra of {1, 2, 3} over Rational Field
>>> C.super_categories()
[Category of realizations of The subset algebra of {1, 2, 3} over␣
↪Rational Field,
 Join of Category of algebras with basis over Rational Field and
         Category of commutative algebras over Rational Field and
         Category of realizations of unital magmas]
```

F

alias of *Fundamental*

class Fundamental(*A*)

Bases: CombinatorialFreeModule, BindableClass

The Subset algebra, in the fundamental basis.

INPUT:

- A – a parent with realization in *SubsetAlgebra*

EXAMPLES:

```
sage: A = Sets().WithRealizations().example()
sage: A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
sage: A.Fundamental()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example()
>>> A.F()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
>>> A.Fundamental()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
```

one()

Return the multiplicative unit element.

EXAMPLES:

```
sage: A = AlgebrasWithBasis(QQ).example()                                   #␣
↪needs sage.combinat sage.modules
sage: A.one_basis()                                                         #␣
↪needs sage.combinat sage.modules
word:
sage: A.one()                                                               #␣
↪needs sage.combinat sage.modules
B[word: ]
```

```
>>> from sage.all import *
>>> A = AlgebrasWithBasis(QQ).example()                                   # needs␣
↪sage.combinat sage.modules
>>> A.one_basis()                                                         # needs␣
↪sage.combinat sage.modules
word:
>>> A.one()                                                               # needs␣
↪sage.combinat sage.modules
B[word: ]
```

one_basis()

Return the index of the basis element which is equal to '1'.

EXAMPLES:

```
sage: F = Sets().WithRealizations().example().F(); F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
sage: F.one_basis()
{}
sage: F.one()
F[{}]
```

```
>>> from sage.all import *
>>> F = Sets().WithRealizations().example().F(); F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
>>> F.one_basis()
{}
>>> F.one()
F[{}]
```

product_on_basis(*left*, *right*)

Product of basis elements, as per *product_on_basis()*.

INPUT:

- `left`, `right` – sets indexing basis elements

EXAMPLES:

```
sage: F = Sets().WithRealizations().example().F(); F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
sage: S = F.basis().keys(); S
Subsets of {1, 2, 3}
sage: F.product_on_basis(S([]), S([]))
F[{}]
sage: F.product_on_basis(S({1}), S({3}))
F[{1, 3}]
sage: F.product_on_basis(S({1,2}), S({2,3}))
F[{1, 2, 3}]
```

```
>>> from sage.all import *
>>> F = Sets().WithRealizations().example().F(); F
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental␣
↪basis
>>> S = F.basis().keys(); S
Subsets of {1, 2, 3}
>>> F.product_on_basis(S([]), S([]))
F[{}]
>>> F.product_on_basis(S({Integer(1)}), S({Integer(3)}))
F[{1, 3}]
>>> F.product_on_basis(S({Integer(1),Integer(2)}), S({Integer(2),
↪Integer(3)}))
F[{1, 2, 3}]
```

class In(*A*)

Bases: `CombinatorialFreeModule`, `BindableClass`

The Subset Algebra, in the `In` basis.

INPUT:

- `A` – a parent with realization in *SubsetAlgebra*

EXAMPLES:

```
sage: A = Sets().WithRealizations().example()
sage: A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example()
>>> A.In()
The subset algebra of {1, 2, 3} over Rational Field in the In basis
```

class Out(*A*)

Bases: CombinatorialFreeModule, BindableClass

The Subset Algebra, in the Out basis.

INPUT:

- A – a parent with realization in *SubsetAlgebra*

EXAMPLES:

```
sage: A = Sets().WithRealizations().example()
sage: A.Out()
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example()
>>> A.Out()
The subset algebra of {1, 2, 3} over Rational Field in the Out basis
```

a_realization()

Return the default realization of self.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: A.a_realization()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> A.a_realization()
The subset algebra of {1, 2, 3} over Rational Field in the Fundamental basis
```

base_set()

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: A.base_set()
{1, 2, 3}
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> A.base_set()
{1, 2, 3}
```

indices()

The objects that index the basis elements of this algebra.

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: A.indices()
Subsets of {1, 2, 3}
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> A.indices()
Subsets of {1, 2, 3}
```

indices_key(*x*)

A key function on a set which gives a linear extension of the inclusion order.

INPUT:

- `x` – set

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: sorted(A.indices(), key=A.indices_key)
[{}, {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3}]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> sorted(A.indices(), key=A.indices_key)
[{}, {1}, {2}, {3}, {1, 2}, {1, 3}, {2, 3}, {1, 2, 3}]
```

supsets(*set*)

Return all the subsets of S containing `set`.

INPUT:

- `set` – a subset of the base set S of `self`

EXAMPLES:

```
sage: A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
sage: A.supsets(Set((2,)))
[{1, 2, 3}, {2, 3}, {1, 2}, {2}]
```

```
>>> from sage.all import *
>>> A = Sets().WithRealizations().example(); A
The subset algebra of {1, 2, 3} over Rational Field
>>> A.supsets(Set((Integer(2),)))
[{1, 2, 3}, {2, 3}, {1, 2}, {2}]
```

CHAPTER

SEVEN

INTERNALS

7.1 Specific category classes

Category_over_base.**_test_category_over_bases**(***options*)

Run generic tests on this category with parameters.

See also

TestSuite.

EXAMPLES:

```
sage: Modules(QQ)._test_category_over_bases()
```

```
>>> from sage.all import *
>>> Modules(QQ)._test_category_over_bases()
```

This is placed in a separate file from categories.py to avoid circular imports (as morphisms must be very low in the hierarchy with the new coercion model).

class sage.categories.category_types.**AbelianCategory**

Bases: *Category*

is_abelian()

Return True as self is an abelian category.

EXAMPLES:

```
sage: CommutativeAdditiveGroups().is_abelian()
True
```

```
>>> from sage.all import *
>>> CommutativeAdditiveGroups().is_abelian()
True
```

class sage.categories.category_types.**Category_ideal**(*ambient*, *name=None*)

Bases: *Category_in_ambient*

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: AlgebraIdeals.an_instance()
Category of algebra ideals in Univariate Polynomial Ring in x over Rational␣
↪Field
```

```
>>> from sage.all import *
>>> AlgebraIdeals.an_instance()
Category of algebra ideals in Univariate Polynomial Ring in x over Rational␣
↪Field
```

ring()

Return the ambient ring used to describe objects `self`.

EXAMPLES:

```
sage: C = Ideals(IntegerRing())
sage: C.ring()
Integer Ring
```

```
>>> from sage.all import *
>>> C = Ideals(IntegerRing())
>>> C.ring()
Integer Ring
```

class sage.categories.category_types.**Category_in_ambient**(*ambient*, *name=None*)

Bases: *Category*

Initialize `self`.

The parameter `name` is ignored.

EXAMPLES:

```
sage: C = Ideals(IntegerRing())
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> C = Ideals(IntegerRing())
>>> TestSuite(C).run()
```

ambient()

Return the ambient object in which objects of this category are embedded.

EXAMPLES:

```
sage: C = Ideals(IntegerRing())
sage: C.ambient()
Integer Ring
```

```
>>> from sage.all import *
>>> C = Ideals(IntegerRing())
>>> C.ambient()
Integer Ring
```

class sage.categories.category_types.**Category_module**(*base*, *name=None*)

Bases: *AbelianCategory*, *Category_over_base_ring*

class sage.categories.category_types.**Category_over_base**(*base*, *name=None*)

Bases: *CategoryWithParameters*

A base class for categories over some base object.

INPUT:

- `base` – a category C or an object of such a category

Assumption: the classes for the parents, elements, morphisms, of `self` should only depend on C. See Issue #11935 for details.

EXAMPLES:

```
sage: Algebras(GF(2)).element_class is Algebras(GF(3)).element_class
True

sage: C = GF(2).category()
sage: Algebras(GF(2)).parent_class is Algebras(C).parent_class
True

sage: C = ZZ.category()
sage: Algebras(ZZ).element_class is Algebras(C).element_class
True
```

```
>>> from sage.all import *
>>> Algebras(GF(Integer(2))).element_class is Algebras(GF(Integer(3))).element_
↪class
True

>>> C = GF(Integer(2)).category()
>>> Algebras(GF(Integer(2))).parent_class is Algebras(C).parent_class
True

>>> C = ZZ.category()
>>> Algebras(ZZ).element_class is Algebras(C).element_class
True
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: Algebras.an_instance()
Category of algebras over Rational Field
```

```
>>> from sage.all import *
>>> Algebras.an_instance()
Category of algebras over Rational Field
```

base()

Return the base over which elements of this category are defined.

EXAMPLES:

```
sage: C = Algebras(QQ)
sage: C.base()
Rational Field
```

```
>>> from sage.all import *
>>> C = Algebras(QQ)
>>> C.base()
Rational Field
```

class sage.categories.category_types.**Category_over_base_ring**(*base*, *name=None*)

Bases: *Category_over_base*

Initialize `self`.

EXAMPLES:

```
sage: C = Algebras(GF(2)); C
Category of algebras over Finite Field of size 2
sage: TestSuite(C).run()
```

```
>>> from sage.all import *
>>> C = Algebras(GF(Integer(2))); C
Category of algebras over Finite Field of size 2
>>> TestSuite(C).run()
```

base_ring()

Return the base ring over which elements of this category are defined.

EXAMPLES:

```
sage: C = Algebras(GF(2))
sage: C.base_ring()
Finite Field of size 2
```

```
>>> from sage.all import *
>>> C = Algebras(GF(Integer(2)))
>>> C.base_ring()
Finite Field of size 2
```

class sage.categories.category_types.**Elements**(*object*)

Bases: *Category*

The category of all elements of a given parent.

EXAMPLES:

```
sage: a = IntegerRing()(5)
sage: C = a.category(); C
Category of elements of Integer Ring
sage: a in C
True
sage: 2/3 in C
False
sage: loads(C.dumps()) == C
True
```

```
>>> from sage.all import *
>>> a = IntegerRing()(Integer(5))
>>> C = a.category(); C
Category of elements of Integer Ring
>>> a in C
True
>>> Integer(2)/Integer(3) in C
False
>>> loads(C.dumps()) == C
True
```

classmethod an_instance()

Return an instance of this class.

EXAMPLES:

```
sage: Elements.an_instance()
Category of elements of Rational Field
```

```
>>> from sage.all import *
>>> Elements.an_instance()
Category of elements of Rational Field
```

object()

EXAMPLES:

```
sage: Elements(ZZ).object()
Integer Ring
```

```
>>> from sage.all import *
>>> Elements(ZZ).object()
Integer Ring
```

super_categories()

EXAMPLES:

```
sage: Elements(ZZ).super_categories()
[Category of objects]
```

```
>>> from sage.all import *
>>> Elements(ZZ).super_categories()
[Category of objects]
```

Todo

Check that this is what we want.

7.2 Singleton categories

static Category_singleton.**__classcall__**(**args*)

Return `cls()` and cache the result in `cls`.

INPUT:

- `*args` – some constant arguments

Most of the time, `args` is meant to be empty. However some singleton categories, in particular axiom categories of singleton categories, may require a constant argument. `*args` is passed down to `__init__`, and ignored upon later calls.

See also

CategoryWithAxiom_singleton

class sage.categories.category_singleton.**Category_contains_method_by_parent_class**

Bases: `object`

Return whether `x` is an object in this category.

More specifically, returns `True` if and only if `x` has a category which is a subcategory of this one.

EXAMPLES:

```
sage: ZZ in Sets()
True
```

```
>>> from sage.all import *
>>> ZZ in Sets()
True
```

class sage.categories.category_singleton.**Category_singleton**

Bases: *Category*

A base class for implementing singleton category.

A *singleton* category is a category whose class takes no parameters like `Fields()` or `Rings()`. See also the Singleton design pattern.

This is a subclass of *Category*, with a couple optimizations for singleton categories.

The main purpose is to make the idioms:

```
sage: QQ in Fields()
True
sage: ZZ in Fields()
False
```

```
>>> from sage.all import *
>>> QQ in Fields()
True
>>> ZZ in Fields()
False
```

as fast as possible, and in particular competitive to calling a constant Python method, in order to foster its systematic use throughout the Sage library. Such tests are time critical, in particular when creating a lot of polynomial rings over small fields like in the elliptic curve code.

EXAMPLES:

```
sage: from sage.categories.category_singleton import Category_singleton
sage: class MyRings(Category):
....:     def super_categories(self): return Rings().super_categories()
sage: class MyRingsSingleton(Category_singleton):
....:     def super_categories(self): return Rings().super_categories()
```

```
>>> from sage.all import *
>>> from sage.categories.category_singleton import Category_singleton
>>> class MyRings(Category):
...     def super_categories(self): return Rings().super_categories()
>>> class MyRingsSingleton(Category_singleton):
...     def super_categories(self): return Rings().super_categories()
```

We create three rings. One of them is contained in the usual category of rings, one in the category of "my rings" and the third in the category of "my rings singleton":

```
sage: R = QQ['x,y']
sage: R1 = Parent(category = MyRings())
sage: R2 = Parent(category = MyRingsSingleton())
sage: R in MyRings()
False
sage: R1 in MyRings()
True
sage: R1 in MyRingsSingleton()
False
sage: R2 in MyRings()
False
sage: R2 in MyRingsSingleton()
True
```

```
>>> from sage.all import *
>>> R = QQ['x,y']
>>> R1 = Parent(category = MyRings())
>>> R2 = Parent(category = MyRingsSingleton())
>>> R in MyRings()
False
>>> R1 in MyRings()
True
>>> R1 in MyRingsSingleton()
False
>>> R2 in MyRings()
```

(continues on next page)

(continued from previous page)

```
False
>>> R2 in MyRingsSingleton()
True
```

One sees that containment tests for the singleton class is a lot faster than for a usual class:

```
sage: # not tested
sage: timeit("R in MyRings()", number=10000)
10000 loops, best of 3: 7.12 µs per loop
sage: timeit("R1 in MyRings()", number=10000)
10000 loops, best of 3: 6.98 µs per loop
sage: timeit("R in MyRingsSingleton()", number=10000)
10000 loops, best of 3: 3.08 µs per loop
sage: timeit("R2 in MyRingsSingleton()", number=10000)
10000 loops, best of 3: 2.99 µs per loop
```

```
>>> from sage.all import *
>>> # not tested
>>> timeit("R in MyRings()", number=Integer(10000))
10000 loops, best of 3: 7.12 µs per loop
>>> timeit("R1 in MyRings()", number=Integer(10000))
10000 loops, best of 3: 6.98 µs per loop
>>> timeit("R in MyRingsSingleton()", number=Integer(10000))
10000 loops, best of 3: 3.08 µs per loop
>>> timeit("R2 in MyRingsSingleton()", number=Integer(10000))
10000 loops, best of 3: 2.99 µs per loop
```

So this is an improvement, but not yet competitive with a pure Cython method:

```
sage: timeit("R.is_ring()", number=10000)                    # not tested
10000 loops, best of 3: 383 ns per loop
```

```
>>> from sage.all import *
>>> timeit("R.is_ring()", number=Integer(10000))                    # not tested
10000 loops, best of 3: 383 ns per loop
```

However, it is competitive with a Python method. Actually it is faster, if one stores the category in a variable:

```
sage: _Rings = Rings()
sage: R3 = Parent(category = _Rings)
sage: R3.is_ring.__module__
'sage.categories.rings'
sage: timeit("R3.is_ring()", number=10000)                   # not tested
10000 loops, best of 3: 2.64 µs per loop
sage: timeit("R3 in Rings()", number=10000)                  # not tested
10000 loops, best of 3: 3.01 µs per loop
sage: timeit("R3 in _Rings", number=10000)                   # not tested
10000 loops, best of 3: 652 ns per loop
```

```
>>> from sage.all import *
>>> _Rings = Rings()
>>> R3 = Parent(category = _Rings)
```

(continues on next page)

(continued from previous page)

```
>>> R3.is_ring.__module__
'sage.categories.rings'
>>> timeit("R3.is_ring()", number=Integer(10000))                    # not tested
10000 loops, best of 3: 2.64 µs per loop
>>> timeit("R3 in Rings()", number=Integer(10000))                   # not tested
10000 loops, best of 3: 3.01 µs per loop
>>> timeit("R3 in _Rings", number=Integer(10000))                    # not tested
10000 loops, best of 3: 652 ns per loop
```

This might not be easy to further optimize, since the time is consumed in many different spots:

```
sage: timeit("MyRingsSingleton.__classcall__()", number=10000)# not tested
10000 loops, best of 3: 306 ns per loop

sage: X = MyRingsSingleton()
sage: timeit("R in X  ", number=10000)                          # not tested
10000 loops, best of 3: 699 ns per loop

sage: c = MyRingsSingleton().__contains__
sage: timeit("c(R)", number = 10000)                            # not tested
10000 loops, best of 3: 661 ns per loop
```

```
>>> from sage.all import *
>>> timeit("MyRingsSingleton.__classcall__()", number=Integer(10000))# not tested
10000 loops, best of 3: 306 ns per loop

>>> X = MyRingsSingleton()
>>> timeit("R in X  ", number=Integer(10000))                       # not tested
10000 loops, best of 3: 699 ns per loop

>>> c = MyRingsSingleton().__contains__
>>> timeit("c(R)", number = Integer(10000))                         # not tested
10000 loops, best of 3: 661 ns per loop
```

Warning

A singleton concrete class A should not have a subclass B (necessarily concrete). Otherwise, creating an instance a of A and an instance b of B would break the singleton principle: A would have two instances a and b.

With the current implementation only direct subclasses of *Category_singleton* are supported:

```
sage: class MyRingsSingleton(Category_singleton):
....:     def super_categories(self): return Rings().super_categories()
sage: class Disaster(MyRingsSingleton): pass
sage: Disaster()
Traceback (most recent call last):
...
AssertionError: <class '__main__.Disaster'> is not a direct subclass of <class
↪'sage.categories.category_singleton.Category_singleton'>
```

```
>>> from sage.all import *
>>> class MyRingsSingleton(Category_singleton):
...     def super_categories(self): return Rings().super_categories()
>>> class Disaster(MyRingsSingleton): pass
>>> Disaster()
Traceback (most recent call last):
...
AssertionError: <class '__main__.Disaster'> is not a direct subclass of <class
↪'sage.categories.category_singleton.Category_singleton'>
```

However, it is acceptable for a direct subclass R of *Category_singleton* to create its unique instance as an instance of a subclass of itself (in which case, its the subclass of R which is concrete, not R itself). This is used for example to plug in extra category code via a dynamic subclass:

```
sage: from sage.categories.category_singleton import Category_singleton
sage: class R(Category_singleton):
....:     def super_categories(self): return [Sets()]
sage: R()
Category of r
sage: R().__class__
<class '__main__.R_with_category'>
sage: R().__class__.mro()
[<class '__main__.R_with_category'>,
 <class '__main__.R'>,
 <class 'sage.categories.category_singleton.Category_singleton'>,
 <class 'sage.categories.category.Category'>,
 <class 'sage.structure.unique_representation.UniqueRepresentation'>,
 <class 'sage.misc.fast_methods.WithEqualityById'>,
 <class 'sage.structure.unique_representation.CachedRepresentation'>,
 <class 'sage.structure.unique_representation.WithPicklingByInitArgs'>,
 <class 'sage.structure.sage_object.SageObject'>,
 <class '__main__.R.subcategory_class'>,
 <class 'sage.categories.sets_cat.Sets.subcategory_class'>,
 <class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.
↪subcategory_class'>,
 <class 'sage.categories.objects.Objects.subcategory_class'>,
 <class 'object'>]
sage: R() is R()
True
sage: R() is R().__class__()
True
```

```
>>> from sage.all import *
>>> from sage.categories.category_singleton import Category_singleton
>>> class R(Category_singleton):
...     def super_categories(self): return [Sets()]
>>> R()
Category of r
>>> R().__class__
<class '__main__.R_with_category'>
>>> R().__class__.mro()
[<class '__main__.R_with_category'>,
 <class '__main__.R'>,
 <class 'sage.categories.category_singleton.Category_singleton'>,
```

```
 <class 'sage.categories.category.Category'>,
 <class 'sage.structure.unique_representation.UniqueRepresentation'>,
 <class 'sage.misc.fast_methods.WithEqualityById'>,
 <class 'sage.structure.unique_representation.CachedRepresentation'>,
 <class 'sage.structure.unique_representation.WithPicklingByInitArgs'>,
 <class 'sage.structure.sage_object.SageObject'>,
 <class '__main__.R.subcategory_class'>,
 <class 'sage.categories.sets_cat.Sets.subcategory_class'>,
 <class 'sage.categories.sets_with_partial_maps.SetsWithPartialMaps.
↪subcategory_class'>,
 <class 'sage.categories.objects.Objects.subcategory_class'>,
 <class 'object'>]
>>> R() is R()
True
>>> R() is R().__class__()
True
```

In that case, `R` is an abstract class and has a single concrete subclass, so this does not break the Singleton design pattern.

See also

Category.__classcall__(), Category.__init__()

Note

The `_test_category` test is failing because `MyRingsSingleton()` is not a subcategory of the join of its super categories:

```
sage: C = MyRingsSingleton()
sage: C.super_categories()
[Category of rngs, Category of semirings]
sage: Rngs() & Semirings()
Category of rings
sage: C.is_subcategory(Rings())
False
```

```
>>> from sage.all import *
>>> C = MyRingsSingleton()
>>> C.super_categories()
[Category of rngs, Category of semirings]
>>> Rngs() & Semirings()
Category of rings
>>> C.is_subcategory(Rings())
False
```

Oh well; it's not really relevant for those tests.

7.3 Fast functions for the category framework

AUTHOR:

- Simon King (initial version)

class sage.categories.category_cy_helper.**AxiomContainer**

Bases: dict

A fast container for axioms.

This is derived from dict. A key is the name of an axiom. The corresponding value is the "rank" of this axiom, that is used to order the axioms in *canonicalize_axioms()*.

EXAMPLES:

```
sage: all_axioms = sage.categories.category_with_axiom.all_axioms
sage: isinstance(all_axioms, sage.categories.category_with_axiom.AxiomContainer)
True
```

```
>>> from sage.all import *
>>> all_axioms = sage.categories.category_with_axiom.all_axioms
>>> isinstance(all_axioms, sage.categories.category_with_axiom.AxiomContainer)
True
```

add(*axiom*)

Add a new axiom name, of the next rank.

EXAMPLES:

```
sage: all_axioms = sage.categories.category_with_axiom.all_axioms
sage: m = max(all_axioms.values())
sage: all_axioms.add('Awesome')
sage: all_axioms['Awesome'] == m + 1
True
```

```
>>> from sage.all import *
>>> all_axioms = sage.categories.category_with_axiom.all_axioms
>>> m = max(all_axioms.values())
>>> all_axioms.add('Awesome')
>>> all_axioms['Awesome'] == m + Integer(1)
True
```

To avoid side effects, we remove the added axiom:

```
sage: del all_axioms['Awesome']
```

```
>>> from sage.all import *
>>> del all_axioms['Awesome']
```

sage.categories.category_cy_helper.**canonicalize_axioms**(*all_axioms*, *axioms*)

Canonicalize a set of axioms.

INPUT:

- all_axioms – all available axioms
- axioms – set (or iterable) of axioms

> **Note**
>
> *AxiomContainer* provides a fast container for axioms, and the collection of axioms is stored in *sage.categories.category_with_axiom*. In order to avoid circular imports, we expect that the collection of all axioms is provided as an argument to this auxiliary function.

OUTPUT:

A set of axioms as a tuple sorted according to the order of the tuple `all_axioms` in *sage.categories.category_with_axiom*.

EXAMPLES:

```
sage: from sage.categories.category_with_axiom import canonicalize_axioms, all_
↪axioms
sage: canonicalize_axioms(all_axioms, ["Commutative", "Connected", "WithBasis",
↪"Finite"])
('Finite', 'Connected', 'WithBasis', 'Commutative')
sage: canonicalize_axioms(all_axioms, ["Commutative", "Connected", "Commutative",
↪"WithBasis", "Finite"])
('Finite', 'Connected', 'WithBasis', 'Commutative')
```

```
>>> from sage.all import *
>>> from sage.categories.category_with_axiom import canonicalize_axioms, all_
↪axioms
>>> canonicalize_axioms(all_axioms, ["Commutative", "Connected", "WithBasis",
↪"Finite"])
('Finite', 'Connected', 'WithBasis', 'Commutative')
>>> canonicalize_axioms(all_axioms, ["Commutative", "Connected", "Commutative",
↪"WithBasis", "Finite"])
('Finite', 'Connected', 'WithBasis', 'Commutative')
```

sage.categories.category_cy_helper.**category_sort_key**(*category*)

Return `category._cmp_key`.

This helper function is used for sorting lists of categories.

It is semantically equivalent to `operator.attrgetter("_cmp_key")`, but currently faster.

EXAMPLES:

```
sage: from sage.categories.category_cy_helper import category_sort_key
sage: category_sort_key(Rings()) is Rings()._cmp_key
True
```

```
>>> from sage.all import *
>>> from sage.categories.category_cy_helper import category_sort_key
>>> category_sort_key(Rings()) is Rings()._cmp_key
True
```

sage.categories.category_cy_helper.**get_axiom_index**(*all_axioms*, *axiom*)

Helper function: Return the rank of an axiom.

INPUT:

- `all_axioms` – the axiom collection

- `axiom` – string, name of an axiom

EXAMPLES:

```
sage: all_axioms = sage.categories.category_with_axiom.all_axioms
sage: from sage.categories.category_cy_helper import get_axiom_index
sage: get_axiom_index(all_axioms, 'AdditiveCommutative') == all_axioms[
↪'AdditiveCommutative']
True
```

```
>>> from sage.all import *
>>> all_axioms = sage.categories.category_with_axiom.all_axioms
>>> from sage.categories.category_cy_helper import get_axiom_index
>>> get_axiom_index(all_axioms, 'AdditiveCommutative') == all_axioms[
↪'AdditiveCommutative']
True
```

`sage.categories.category_cy_helper.`**`join_as_tuple`**(*categories*, *axioms*, *ignore_axioms*)

Helper for *join()*.

INPUT:

- `categories` – tuple of categories to be joined
- `axioms` – tuple of strings; the names of some supplementary axioms
- `ignore_axioms` – tuple of pairs `(cat, axiom)`, such that `axiom` will not be applied to `cat`, should `cat` occur in the algorithm

EXAMPLES:

```
sage: from sage.categories.category_cy_helper import join_as_tuple
sage: T = (Coalgebras(QQ), Sets().Finite(), Algebras(ZZ), SimplicialComplexes())
sage: join_as_tuple(T,(),())
(Category of algebras over Integer Ring,
 Category of finite monoids,
 Category of finite additive groups,
 Category of coalgebras over Rational Field,
 Category of finite simplicial complexes)
sage: join_as_tuple(T,('WithBasis',),())
(Category of algebras with basis over Integer Ring,
 Category of finite monoids,
 Category of coalgebras with basis over Rational Field,
 Category of finite additive groups,
 Category of finite simplicial complexes)
sage: join_as_tuple(T,(),((Monoids(),'Finite'),))
(Category of algebras over Integer Ring,
 Category of finite additive groups,
 Category of coalgebras over Rational Field,
 Category of finite simplicial complexes)
```

```
>>> from sage.all import *
>>> from sage.categories.category_cy_helper import join_as_tuple
>>> T = (Coalgebras(QQ), Sets().Finite(), Algebras(ZZ), SimplicialComplexes())
>>> join_as_tuple(T,(),())
(Category of algebras over Integer Ring,
```

(continues on next page)

(continued from previous page)

```
 Category of finite monoids,
 Category of finite additive groups,
 Category of coalgebras over Rational Field,
 Category of finite simplicial complexes)
>>> join_as_tuple(T,('WithBasis',),())
(Category of algebras with basis over Integer Ring,
 Category of finite monoids,
 Category of coalgebras with basis over Rational Field,
 Category of finite additive groups,
 Category of finite simplicial complexes)
>>> join_as_tuple(T,(),((Monoids(),'Finite'),))
(Category of algebras over Integer Ring,
 Category of finite additive groups,
 Category of coalgebras over Rational Field,
 Category of finite simplicial complexes)
```

7.4 Coercion methods for categories

The purpose of this Cython module is to hold special coercion methods, which are inserted by their respective categories.

7.5 Poor Man's map

class sage.categories.poor_man_map.**PoorManMap**(*function*, *domain=None*, *codomain=None*, *name=None*)

Bases: SageObject

A class for maps between sets which are not (yet) modeled by parents.

Could possibly disappear when all combinatorial classes / enumerated sets will be parents.

INPUT:

- function – a callable or an iterable of callables. This represents the underlying function used to implement this map. If it is an iterable, then the callables will be composed to implement this map.
- domain – the domain of this map or None if the domain is not known or should remain unspecified
- codomain – the codomain of this map or None if the codomain is not known or should remain unspecified
- name – a name for this map or None if this map has no particular name

EXAMPLES:

```
sage: from sage.categories.poor_man_map import PoorManMap
sage: f = PoorManMap(factorial, domain=(1, 2, 3), codomain=(1, 2, 6))
sage: f
A map from (1, 2, 3) to (1, 2, 6)
sage: f(3)
6
```

```
>>> from sage.all import *
>>> from sage.categories.poor_man_map import PoorManMap
>>> f = PoorManMap(factorial, domain=(Integer(1), Integer(2), Integer(3)),␣
```

(continues on next page)

(continued from previous page)

```
↪codomain=(Integer(1), Integer(2), Integer(6)))
>>> f
A map from (1, 2, 3) to (1, 2, 6)
>>> f(Integer(3))
6
```

The composition of several functions can be created by passing in a tuple of functions:

```
sage: i = PoorManMap((factorial, sqrt), domain=(1, 4, 9), codomain=(1, 2, 6))
```

```
>>> from sage.all import *
>>> i = PoorManMap((factorial, sqrt), domain=(Integer(1), Integer(4), Integer(9)),
↪ codomain=(Integer(1), Integer(2), Integer(6)))
```

However, the same effect can also be achieved by just composing maps:

```
sage: g = PoorManMap(factorial, domain=(1, 2, 3), codomain=(1, 2, 6))
sage: h = PoorManMap(sqrt, domain=(1, 4, 9), codomain=(1, 2, 3))
sage: i == g*h
True
```

```
>>> from sage.all import *
>>> g = PoorManMap(factorial, domain=(Integer(1), Integer(2), Integer(3)),␣
↪codomain=(Integer(1), Integer(2), Integer(6)))
>>> h = PoorManMap(sqrt, domain=(Integer(1), Integer(4), Integer(9)),␣
↪codomain=(Integer(1), Integer(2), Integer(3)))
>>> i == g*h
True
```

codomain()

Return the codomain of `self`.

EXAMPLES:

```
sage: from sage.categories.poor_man_map import PoorManMap
sage: PoorManMap(lambda x: x+1, domain=(1,2,3), codomain=(2,3,4)).codomain()
(2, 3, 4)
```

```
>>> from sage.all import *
>>> from sage.categories.poor_man_map import PoorManMap
>>> PoorManMap(lambda x: x+Integer(1), domain=(Integer(1),Integer(2),
↪Integer(3)), codomain=(Integer(2),Integer(3),Integer(4))).codomain()
(2, 3, 4)
```

domain()

Return the domain of `self`.

EXAMPLES:

```
sage: from sage.categories.poor_man_map import PoorManMap
sage: PoorManMap(lambda x: x+1, domain=(1,2,3), codomain=(2,3,4)).domain()
(1, 2, 3)
```

```
>>> from sage.all import *
>>> from sage.categories.poor_man_map import PoorManMap
>>> PoorManMap(lambda x: x+Integer(1), domain=(Integer(1),Integer(2),
↪Integer(3)), codomain=(Integer(2),Integer(3),Integer(4))).domain()
(1, 2, 3)
```

CHAPTER

EIGHT

INDICES AND TABLES

- Index
- Module Index
- Search Page

PYTHON MODULE INDEX

C

sage.categories.action, 267
sage.categories.additive_groups, 272
sage.categories.additive_magmas, 275
sage.categories.additive_monoids, 293
sage.categories.additive_semigroups, 295
sage.categories.affine_weyl_groups, 298
sage.categories.algebra_functor, 1542
sage.categories.algebra_ideals, 303
sage.categories.algebra_modules, 304
sage.categories.algebras, 305
sage.categories.algebras_with_basis, 311
sage.categories.all, 1
sage.categories.anderson_motives, 320
sage.categories.aperiodic_semigroups, 328
sage.categories.associative_algebras, 328
sage.categories.basic, 44
sage.categories.bialgebras, 329
sage.categories.bialgebras_with_basis, 331
sage.categories.bimodules, 338
sage.categories.cartesian_product, 1535
sage.categories.category, 44
sage.categories.category_cy_helper, 1688
sage.categories.category_singleton, 1682
sage.categories.category_types, 1677
sage.categories.category_with_axiom, 109
sage.categories.chain_complexes, 340
sage.categories.classical_crystals, 346
sage.categories.coalgebras, 352
sage.categories.coalgebras_with_basis, 361
sage.categories.coercion_methods, 1691
sage.categories.commutative_additive_groups, 365
sage.categories.commutative_additive_monoids, 367
sage.categories.commutative_additive_semigroups, 368
sage.categories.commutative_algebra_ideals, 369
sage.categories.commutative_algebras, 369
sage.categories.commutative_ring_ideals, 371
sage.categories.commutative_rings, 371
sage.categories.complete_discrete_valuation, 391
sage.categories.complex_reflection_groups, 398
sage.categories.complex_reflection_or_generalized_coxeter_groups, 402
sage.categories.covariant_functorial_construction, 1527
sage.categories.coxeter_group_algebras, 432
sage.categories.coxeter_groups, 436
sage.categories.crystals, 506
sage.categories.cw_complexes, 547
sage.categories.dedekind_domains, 551
sage.categories.discrete_valuation, 554
sage.categories.distributive_magmas_and_additive_magmas, 559
sage.categories.division_rings, 560
sage.categories.domains, 562
sage.categories.drinfeld_modules, 563
sage.categories.dual, 1541
sage.categories.enumerated_sets, 581
sage.categories.euclidean_domains, 593
sage.categories.examples.algebras_with_basis, 1575
sage.categories.examples.commutative_additive_monoids, 1576
sage.categories.examples.commutative_additive_semigroups, 1579
sage.categories.examples.coxeter_groups, 1582
sage.categories.examples.crystals, 1582
sage.categories.examples.cw_complexes, 1585
sage.categories.examples.facade_sets, 1587
sage.categories.examples.finite_coxeter_groups, 1588
sage.categories.examples.finite_dimensional_algebras_with_basis, 1591
sage.categories.examples.finite_dimensional_lie_algebras_with_basis, 1594
sage.categories.examples.finite_enumer-

ated_sets, 1600
sage.categories.examples.finite_monoids, 1604
sage.categories.examples.finite_semigroups, 1607
sage.categories.examples.finite_weyl_groups, 1611
sage.categories.examples.graded_connected_hopf_algebras_with_basis, 1614
sage.categories.examples.graded_modules_with_basis, 1616
sage.categories.examples.graphs, 1620
sage.categories.examples.hopf_algebras_with_basis, 1622
sage.categories.examples.infinite_enumerated_sets, 1624
sage.categories.examples.lie_algebras, 1627
sage.categories.examples.lie_algebras_with_basis, 1629
sage.categories.examples.magmas, 1632
sage.categories.examples.manifolds, 1633
sage.categories.examples.monoids, 1635
sage.categories.examples.posets, 1638
sage.categories.examples.semigroups, 1641
sage.categories.examples.semigroups_cython, 1648
sage.categories.examples.semirings, 1652
sage.categories.examples.sets_cat, 1654
sage.categories.examples.sets_with_grading, 1665
sage.categories.examples.with_realizations, 1667
sage.categories.facade_sets, 1524
sage.categories.fields, 596
sage.categories.filtered_algebras, 610
sage.categories.filtered_algebras_with_basis, 611
sage.categories.filtered_hopf_algebras_with_basis, 624
sage.categories.filtered_modules, 626
sage.categories.filtered_modules_with_basis, 629
sage.categories.finite_complex_reflection_groups, 657
sage.categories.finite_coxeter_groups, 689
sage.categories.finite_crystals, 712
sage.categories.finite_dimensional_algebras_with_basis, 713
sage.categories.finite_dimensional_bialgebras_with_basis, 753
sage.categories.finite_dimensional_coalgebras_with_basis, 753
sage.categories.finite_dimensional_graded_lie_algebras_with_basis, 754
sage.categories.finite_dimensional_hopf_algebras_with_basis, 756
sage.categories.finite_dimensional_lie_algebras_with_basis, 757
sage.categories.finite_dimensional_modules_with_basis, 806
sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis, 827
sage.categories.finite_dimensional_semisimple_algebras_with_basis, 829
sage.categories.finite_enumerated_sets, 833
sage.categories.finite_fields, 844
sage.categories.finite_groups, 851
sage.categories.finite_lattice_posets, 855
sage.categories.finite_monoids, 861
sage.categories.finite_permutation_groups, 868
sage.categories.finite_posets, 874
sage.categories.finite_semigroups, 910
sage.categories.finite_sets, 913
sage.categories.finite_weyl_groups, 915
sage.categories.finitely_generated_lambda_bracket_algebras, 916
sage.categories.finitely_generated_lie_conformal_algebras, 918
sage.categories.finitely_generated_magmas, 920
sage.categories.finitely_generated_semigroups, 921
sage.categories.function_fields, 925
sage.categories.functor, 157
sage.categories.g_sets, 926
sage.categories.gcd_domains, 927
sage.categories.generalized_coxeter_groups, 928
sage.categories.graded_algebras, 930
sage.categories.graded_algebras_with_basis, 931
sage.categories.graded_bialgebras, 936
sage.categories.graded_bialgebras_with_basis, 936
sage.categories.graded_coalgebras, 936
sage.categories.graded_coalgebras_with_basis, 938
sage.categories.graded_hopf_algebras, 939
sage.categories.graded_hopf_algebras_with_basis, 939
sage.categories.graded_lie_algebras, 941
sage.categories.graded_lie_algebras_with_basis, 943

sage.categories.graded_lie_conformal_algebras, 943
sage.categories.graded_modules, 944
sage.categories.graded_modules_with_basis, 947
sage.categories.graphs, 955
sage.categories.group_algebras, 958
sage.categories.groupoid, 966
sage.categories.groups, 967
sage.categories.h_trivial_semigroups, 1006
sage.categories.hecke_modules, 983
sage.categories.highest_weight_crystals, 984
sage.categories.homset, 188
sage.categories.homsets, 1556
sage.categories.hopf_algebras, 996
sage.categories.hopf_algebras_with_basis, 1000
sage.categories.infinite_enumerated_sets, 1007
sage.categories.integral_domains, 1009
sage.categories.isomorphic_objects, 1555
sage.categories.j_trivial_semigroups, 1012
sage.categories.kac_moody_algebras, 1012
sage.categories.kahler_algebras, 1014
sage.categories.l_trivial_semigroups, 1098
sage.categories.lambda_bracket_algebras, 1020
sage.categories.lambda_bracket_algebras_with_basis, 1024
sage.categories.lattice_posets, 1026
sage.categories.left_modules, 1040
sage.categories.lie_algebras, 1041
sage.categories.lie_algebras_with_basis, 1062
sage.categories.lie_conformal_algebras, 1066
sage.categories.lie_conformal_algebras_with_basis, 1072
sage.categories.lie_groups, 1075
sage.categories.loop_crystals, 1076
sage.categories.magmas, 1099
sage.categories.magmas_and_additive_magmas, 1122
sage.categories.magmatic_algebras, 1125
sage.categories.manifolds, 1132
sage.categories.map, 165
sage.categories.matrix_algebras, 1139
sage.categories.metric_spaces, 1139
sage.categories.modular_abelian_varieties, 1146
sage.categories.modules, 1147
sage.categories.modules_with_basis, 1166
sage.categories.monoid_algebras, 1228
sage.categories.monoids, 1228
sage.categories.morphism, 201
sage.categories.noetherian_rings, 1240
sage.categories.number_fields, 1243
sage.categories.objects, 1245
sage.categories.ore_modules, 1248
sage.categories.partially_ordered_monoids, 1250
sage.categories.permutation_groups, 1251
sage.categories.pointed_sets, 1252
sage.categories.polyhedra, 1253
sage.categories.poor_man_map, 1691
sage.categories.posets, 1253
sage.categories.primer, 3
sage.categories.principal_ideal_domains, 1271
sage.categories.pushout, 209
sage.categories.quantum_group_representations, 1287
sage.categories.quotient_fields, 1275
sage.categories.quotients, 1553
sage.categories.r_trivial_semigroups, 1348
sage.categories.realizations, 1563
sage.categories.regular_crystals, 1298
sage.categories.regular_supercrystals, 1311
sage.categories.right_modules, 1314
sage.categories.ring_ideals, 1315
sage.categories.rings, 1316
sage.categories.rngs, 1346
sage.categories.schemes, 1349
sage.categories.semigroups, 1353
sage.categories.semirings, 1375
sage.categories.semisimple_algebras, 1376
sage.categories.sets_cat, 1378
sage.categories.sets_with_grading, 1428
sage.categories.sets_with_partial_maps, 1432
sage.categories.shephard_groups, 1433
sage.categories.signed_tensor, 1540
sage.categories.simplicial_complexes, 1433
sage.categories.simplicial_sets, 1436
sage.categories.subobjects, 1554
sage.categories.subquotients, 1553
sage.categories.super_algebras, 1457
sage.categories.super_algebras_with_basis, 1460
sage.categories.super_hopf_algebras_with_basis, 1463
sage.categories.super_lie_conformal_algebras, 1464
sage.categories.super_modules, 1467
sage.categories.super_modules_with_basis, 1471
sage.categories.supercommutative_algebras, 1474
sage.categories.supercrystals, 1475

sage.categories.tensor, 1539
sage.categories.topological_spaces, 1483
sage.categories.triangular_kac_moody_algebras, 1486
sage.categories.tutorial, 163
sage.categories.unique_factorization_domains, 1490
sage.categories.unital_algebras, 1493
sage.categories.vector_bundles, 1497
sage.categories.vector_spaces, 1500
sage.categories.weyl_groups, 1506
sage.categories.with_realizations, 1566

INDEX

Non-alphabetical

__and__() (*sage.categories.category.Category method*), 48
__classcall__() (*sage.categories.category_singleton.Category_singleton static method*), 1682
__classcall__() (*sage.categories.category_with_axiom.CategoryWithAxiom static method*), 143
__classcall__() (*sage.categories.category.Category static method*), 68
__classget__() (*sage.categories.category_with_axiom.CategoryWithAxiom static method*), 144
__init__() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 145
__init__() (*sage.categories.category.Category method*), 68
__init_extra__() (*sage.categories.magmas.Magmas.ParentMethods method*), 1099
__iter__() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 581
__or__() (*sage.categories.category.Category method*), 48
_all_super_categories() (*sage.categories.category.Category method*), 58
_all_super_categories_proper() (*sage.categories.category.Category method*), 59
_cardinality_from_iterator() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 833
_make_named_class() (*sage.categories.category.Category method*), 60
_make_named_class() (*sage.categories.category.CategoryWithParameters.Category method*), 102
_make_strong_references() (*sage.categories.map.Map method*), 167
_make_weak_references() (*sage.categories.map.Map method*), 165
mul() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 402
repr() (*sage.categories.category.Category method*), 62
repr() (*sage.categories.category.JoinCategory.Category method*), 105
_repr_object_names() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 145
_repr_object_names() (*sage.categories.category.Category method*), 62
_repr_object_names() (*sage.categories.category.JoinCategory.Category method*), 105
_repr_object_names_static() (*sage.categories.category_with_axiom.CategoryWithAxiom static method*), 146
_set_of_super_categories() (*sage.categories.category.Category method*), 59
_sort() (*sage.categories.category.Category static method*), 66
_sort_uniq() (*sage.categories.category.Category static method*), 67
_super_categories() (*sage.categories.category.Category method*), 57
_super_categories_for_classes() (*sage.categories.category.Category method*), 57
_test_category() (*sage.categories.category.Category method*), 62
_test_category_graph() (*sage.categories.category.Category method*), 47
_test_category_over_bases() (*sage.categories.category_types.Category_over_base method*), 1677
_test_category_with_axiom() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 147
_test_elements_eq_symmetric() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1378
_test_elements_eq_transitive() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1379
_test_enumerated_set_iter_cardinality() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 834
_test_graded_components() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1428

_test_homsets_category() (*sage.categories.homsets.HomsetsCategory method*), 1556
_with_axiom() (*sage.categories.category.Category method*), 64
_with_axiom_as_tuple() (*sage.categories.category.Category method*), 65
_without_axioms() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 147
_without_axioms() (*sage.categories.category.Category method*), 65
_without_axioms() (*sage.categories.category.JoinCategory.Category method*), 105

A

A_field() (*sage.categories.anderson_motives.AndersonMotives method*), 320
A_field() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 321
A_field() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 566
A_field() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 568
a_realization() (*sage.categories.examples.with_realizations.SubsetAlgebra method*), 1674
a_realization() (*sage.categories.sets_cat.Sets.WithRealizations.ParentMethods method*), 1420
AbelianCategory (*class in sage.categories.category_types*), 1677
AbelianLieAlgebra (*class in sage.categories.examples.finite_dimensional_lie_algebras_with_basis*), 1594
AbelianLieAlgebra (*class in sage.categories.examples.lie_algebras_with_basis*), 1629
AbelianLieAlgebra.Element (*class in sage.categories.examples.finite_dimensional_lie_algebras_with_basis*), 1594
AbelianLieAlgebra.Element (*class in sage.categories.examples.lie_algebras_with_basis*), 1629
AbelianVarieties (*class in sage.categories.schemes*), 1349
AbelianVarieties.Homsets (*class in sage.categories.schemes*), 1349
AbelianVarieties.Homsets.Endset (*class in sage.categories.schemes*), 1349
abs() (*sage.categories.metric_spaces.MetricSpaces.ElementMethods method*), 1142
absolute_chain() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 439
absolute_chain_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 439
absolute_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 441
absolute_le() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 442
absolute_length() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 443
absolute_length() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 690
absolute_order_ideal() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible.ParentMethods method*), 662
absolute_poset() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible.ParentMethods method*), 663
act() (*sage.categories.action.Action method*), 268
Action (*class in sage.categories.action*), 268
ActionEndomorphism (*class in sage.categories.action*), 269
actor() (*sage.categories.action.Action method*), 269
add() (*sage.categories.category_cy_helper.AxiomContainer method*), 1688
addition_table() (*sage.categories.additive_magmas.AdditiveMagmas.ParentMethods method*), 286
additional_structure() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital method*), 282
additional_structure() (*sage.categories.affine_weyl_groups.AffineWeylGroups method*), 302
additional_structure() (*sage.categories.bialgebras.Bialgebras method*), 330
additional_structure() (*sage.categories.bimodules.Bimodules method*), 339
additional_structure() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 148
additional_structure() (*sage.categories.category.Category method*), 78
additional_structure() (*sage.categories.category.JoinCategory method*), 106
additional_structure() (*sage.categories.classical_crystals.ClassicalCrystals method*), 351
additional_structure() (*sage.categories.complex_reflection_groups.ComplexReflectionGroups method*), 400
additional_structure() (*sage.categories.covariant_functorial_construction.CovariantConstructionCategory method*), 1528
additional_structure() (*sage.categories.coxeter_groups.CoxeterGroups method*), 505
additional_structure() (*sage.categories.enumer-*

ated_sets.EnumeratedSets method), 593
additional_structure() (*sage.categories.gcd_domains.GcdDomains method*), 927
additional_structure() (*sage.categories.generalized_coxeter_groups.GeneralizedCoxeterGroups method*), 929
additional_structure() (*sage.categories.highest_weight_crystals.HighestWeightCrystals method*), 995
additional_structure() (*sage.categories.lie_groups.LieGroups method*), 1075
additional_structure() (*sage.categories.magmas_and_additive_magmas.MagmasAndAdditiveMagmas method*), 1125
additional_structure() (*sage.categories.magmas.Magmas.Unital method*), 1122
additional_structure() (*sage.categories.magmatic_algebras.MagmaticAlgebras method*), 1131
additional_structure() (*sage.categories.manifolds.Manifolds method*), 1138
additional_structure() (*sage.categories.modules.Modules method*), 1165
additional_structure() (*sage.categories.objects.Objects method*), 1247
additional_structure() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains method*), 1275
additional_structure() (*sage.categories.regular_crystals.RegularCrystals method*), 1310
additional_structure() (*sage.categories.unique_factorization_domains.UniqueFactorizationDomains method*), 1492
additional_structure() (*sage.categories.vector_spaces.VectorSpaces method*), 1505
additional_structure() (*sage.categories.weyl_groups.WeylGroups method*), 1523
additive_order() (*sage.categories.commutative_additive_groups.CommutativeAdditiveGroups.CartesianProducts.ElementMethods method*), 366
additive_semigroup_generators() (*sage.categories.examples.commutative_additive_semigroups.FreeCommutativeAdditiveSemigroup method*), 1581
AdditiveAssociative (*sage.categories.additive_magmas.AdditiveMagmas attribute*), 276
AdditiveAssociative() (*sage.categories.additive_magmas.AdditiveMagmas.SubcategoryMethods method*), 291
AdditiveCommutative (*sage.categories.additive_groups.AdditiveGroups attribute*), 273
AdditiveCommutative (*sage.categories.additive_monoids.AdditiveMonoids attribute*), 293
AdditiveCommutative (*sage.categories.additive_semigroups.AdditiveSemigroups attribute*), 295
AdditiveCommutative() (*sage.categories.additive_magmas.AdditiveMagmas.SubcategoryMethods method*), 291
AdditiveGroups (*class in sage.categories.additive_groups*), 272
AdditiveGroups.Algebras (*class in sage.categories.additive_groups*), 273
AdditiveGroups.Algebras.ParentMethods (*class in sage.categories.additive_groups*), 273
AdditiveGroups.Finite (*class in sage.categories.additive_groups*), 274
AdditiveGroups.Finite.Algebras (*class in sage.categories.additive_groups*), 274
AdditiveGroups.Finite.Algebras.ParentMethods (*class in sage.categories.additive_groups*), 274
AdditiveInverse (*sage.categories.additive_monoids.AdditiveMonoids attribute*), 293
AdditiveInverse (*sage.categories.distributive_magmas_and_additive_magmas.DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital.Associative attribute*), 559
AdditiveInverse() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.SubcategoryMethods method*), 281
AdditiveMagmas (*class in sage.categories.additive_magmas*), 275
AdditiveMagmas.AdditiveCommutative (*class in sage.categories.additive_magmas*), 276
AdditiveMagmas.AdditiveCommutative.Algebras (*class in sage.categories.additive_magmas*), 276
AdditiveMagmas.AdditiveCommutative.CartesianProducts (*class in sage.categories.additive_magmas*), 277
AdditiveMagmas.AdditiveUnital (*class in sage.categories.additive_magmas*), 277
AdditiveMagmas.AdditiveUnital.AdditiveInverse (*class in sage.categories.additive_magmas*), 277
AdditiveMagmas.AdditiveUnital.AdditiveInverse.CartesianProducts (*class in sage.categories.additive_magmas*), 277
AdditiveMagmas.AdditiveUnital.AdditiveInverse.CartesianProducts.ElementMethods (*class in sage.categories.additive_magmas*), 277
AdditiveMagmas.AdditiveUnital.Algebras (*class in sage.categories.additive_magmas*), 278
AdditiveMagmas.AdditiveUnital.Algebras.ParentMethods (*class in sage.cate-*

gories.additive_magmas), 278
AdditiveMagmas.AdditiveUnital.CartesianProducts (*class in sage.categories.additive_magmas*), 279
AdditiveMagmas.AdditiveUnital.CartesianProducts.ParentMethods (*class in sage.categories.additive_magmas*), 279
AdditiveMagmas.AdditiveUnital.ElementMethods (*class in sage.categories.additive_magmas*), 279
AdditiveMagmas.AdditiveUnital.Homsets (*class in sage.categories.additive_magmas*), 279
AdditiveMagmas.AdditiveUnital.Homsets.ParentMethods (*class in sage.categories.additive_magmas*), 279
AdditiveMagmas.AdditiveUnital.ParentMethods (*class in sage.categories.additive_magmas*), 280
AdditiveMagmas.AdditiveUnital.SubcategoryMethods (*class in sage.categories.additive_magmas*), 281
AdditiveMagmas.AdditiveUnital.WithRealizations (*class in sage.categories.additive_magmas*), 282
AdditiveMagmas.AdditiveUnital.WithRealizations.ParentMethods (*class in sage.categories.additive_magmas*), 282
AdditiveMagmas.Algebras (*class in sage.categories.additive_magmas*), 283
AdditiveMagmas.Algebras.ParentMethods (*class in sage.categories.additive_magmas*), 283
AdditiveMagmas.CartesianProducts (*class in sage.categories.additive_magmas*), 285
AdditiveMagmas.CartesianProducts.ElementMethods (*class in sage.categories.additive_magmas*), 285
AdditiveMagmas.ElementMethods (*class in sage.categories.additive_magmas*), 285
AdditiveMagmas.Homsets (*class in sage.categories.additive_magmas*), 285
AdditiveMagmas.ParentMethods (*class in sage.categories.additive_magmas*), 286
AdditiveMagmas.SubcategoryMethods (*class in sage.categories.additive_magmas*), 291
AdditiveMonoids (*class in sage.categories.additive_monoids*), 293
AdditiveMonoids.Homsets (*class in sage.categories.additive_monoids*), 293
AdditiveMonoids.ParentMethods (*class in sage.categories.additive_monoids*), 294
AdditiveSemigroups (*class in sage.categories.additive_semigroups*), 295
AdditiveSemigroups.Algebras (*class in sage.categories.additive_semigroups*), 295
AdditiveSemigroups.Algebras.ParentMethods (*class in sage.categories.additive_semigroups*), 295
AdditiveSemigroups.CartesianProducts (*class in sage.categories.additive_semigroups*), 297
AdditiveSemigroups.Homsets (*class in sage.categories.additive_semigroups*), 297
AdditiveSemigroups.ParentMethods (*class in sage.categories.additive_semigroups*), 298
AdditiveUnital (*sage.categories.additive_semigroups.AdditiveSemigroups attribute*), 295
AdditiveUnital() (*sage.categories.additive_magmas.AdditiveMagmas.SubcategoryMethods method*), 292
adjoint_matrix() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ElementMethods method*), 757
affine_grading() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ElementMethods method*), 1086
affine_grassmannian_elements_of_given_length() (*sage.categories.affine_weyl_groups.AffineWeylGroups.ParentMethods method*), 301
affine_grassmannian_to_core() (*sage.categories.affine_weyl_groups.AffineWeylGroups.ElementMethods method*), 299
affine_grassmannian_to_partition() (*sage.categories.affine_weyl_groups.AffineWeylGroups.ElementMethods method*), 300
AffineWeylGroups (*class in sage.categories.affine_weyl_groups*), 298
AffineWeylGroups.ElementMethods (*class in sage.categories.affine_weyl_groups*), 299
AffineWeylGroups.ParentMethods (*class in sage.categories.affine_weyl_groups*), 301
affinization() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1079
algebra() (*sage.categories.algebra_ideals.AlgebraIdeals method*), 303
algebra() (*sage.categories.algebra_modules.AlgebraModules method*), 304
algebra() (*sage.categories.commutative_algebra_ideals.CommutativeAlgebraIdeals method*), 369
algebra() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1397
algebra_generators() (*sage.categories.additive_magmas.AdditiveMagmas.Algebras.ParentMethods method*), 283
algebra_generators() (*sage.categories.additive_semigroups.AdditiveSemigroups.Algebras.ParentMethods method*), 296

algebra_generators() (*sage.categories.algebras.Algebras.Quotients.ParentMethods method*), 309
algebra_generators() (*sage.categories.examples.algebras_with_basis.FreeAlgebra method*), 1575
algebra_generators() (*sage.categories.examples.finite_dimensional_algebras_with_basis.KroneckerQuiverPathAlgebra method*), 1592
algebra_generators() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1622
algebra_generators() (*sage.categories.examples.lie_algebras_with_basis.IndexedPolynomialRing method*), 1630
algebra_generators() (*sage.categories.magmatic_algebras.MagmaticAlgebras.ParentMethods method*), 1126
algebra_generators() (*sage.categories.magmatic_algebras.MagmaticAlgebras.WithBasis.ParentMethods method*), 1129
algebra_generators() (*sage.categories.monoids.Monoids.Algebras.ParentMethods method*), 1230
algebra_generators() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1354
AlgebraFunctor (*class in sage.categories.algebra_functor*), 1550
algebraic_closure() (*sage.categories.fields.Fields.ParentMethods method*), 603
AlgebraicClosureFunctor (*class in sage.categories.pushout*), 209
AlgebraicExtensionFunctor (*class in sage.categories.pushout*), 210
AlgebraIdeals (*class in sage.categories.algebra_ideals*), 303
AlgebraModules (*class in sage.categories.algebra_modules*), 304
Algebras (*class in sage.categories.algebras*), 305
Algebras (*sage.categories.coxeter_groups.CoxeterGroups attribute*), 439
Algebras (*sage.categories.groups.Groups attribute*), 967
Algebras() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1403
Algebras.CartesianProducts (*class in sage.categories.algebras*), 306
AlgebrasCategory (*class in sage.categories.algebra_functor*), 1551
AlgebrasCategory.ParentMethods (*class in sage.categories.algebra_functor*), 1551
Algebras.DualObjects (*class in sage.categories.algebras*), 307
Algebras.ElementMethods (*class in sage.categories.algebras*), 307
Algebras.ParentMethods (*class in sage.categories.algebras*), 307
Algebras.Quotients (*class in sage.categories.algebras*), 309
Algebras.Quotients.ParentMethods (*class in sage.categories.algebras*), 309
Algebras.SubcategoryMethods (*class in sage.categories.algebras*), 309
Algebras.TensorProducts (*class in sage.categories.algebras*), 310
Algebras.TensorProducts.ElementMethods (*class in sage.categories.algebras*), 310
Algebras.TensorProducts.ParentMethods (*class in sage.categories.algebras*), 310
AlgebrasWithBasis (*class in sage.categories.algebras_with_basis*), 311
AlgebrasWithBasis.CartesianProducts (*class in sage.categories.algebras_with_basis*), 314
AlgebrasWithBasis.CartesianProducts.ParentMethods (*class in sage.categories.algebras_with_basis*), 314
AlgebrasWithBasis.ElementMethods (*class in sage.categories.algebras_with_basis*), 315
AlgebrasWithBasis.ParentMethods (*class in sage.categories.algebras_with_basis*), 316
AlgebrasWithBasis.TensorProducts (*class in sage.categories.algebras_with_basis*), 317
AlgebrasWithBasis.TensorProducts.ElementMethods (*class in sage.categories.algebras_with_basis*), 317
AlgebrasWithBasis.TensorProducts.ParentMethods (*class in sage.categories.algebras_with_basis*), 317
all_paths_to_highest_weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 516
all_super_categories() (*sage.categories.category.Category method*), 81
AlmostComplex() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1136
ambient() (*sage.categories.category_types.Category_in_ambient method*), 1678
ambient() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1595
ambient() (*sage.categories.examples.finite_enumerated_sets.IsomorphicObjectOfFiniteEnumeratedSet method*), 1602
ambient() (*sage.categories.examples.semigroups.IncompleteSubquotientSemigroup method*), 1643
ambient() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1646
ambient() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAl-*

gebras WithBasis.Subobjects.ParentMethods method), 802
ambient() (*sage.categories.sets_cat.Sets.Subquotients.ParentMethods method*), 1418
an_element() (*sage.categories.crystals.Crystals.ParentMethods method*), 528
an_element() (*sage.categories.examples.commutative_additive_semigroups.FreeCommutativeAdditiveSemigroup method*), 1582
an_element() (*sage.categories.examples.cw_complexes.Surface method*), 1586
an_element() (*sage.categories.examples.finite_monoids.IntegerModMonoid method*), 1605
an_element() (*sage.categories.examples.finite_semigroups.LeftRegularBand method*), 1610
an_element() (*sage.categories.examples.graphs.Cycle method*), 1621
an_element() (*sage.categories.examples.infinite_enumerated_sets.NonNegativeIntegers method*), 1627
an_element() (*sage.categories.examples.magmas.FreeMagma method*), 1632
an_element() (*sage.categories.examples.manifolds.Plane method*), 1634
an_element() (*sage.categories.examples.posets.FiniteSetsOrderedByInclusion method*), 1639
an_element() (*sage.categories.examples.semigroups.FreeSemigroup method*), 1642
an_element() (*sage.categories.examples.semigroups.LeftZeroSemigroup method*), 1644
an_element() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1646
an_element() (*sage.categories.examples.semirings.TernaryLogic method*), 1653
an_element() (*sage.categories.examples.sets_cat.PrimeNumbers method*), 1656
an_element() (*sage.categories.examples.sets_cat.PrimeNumbers_Abstract method*), 1658
an_element() (*sage.categories.examples.sets_with_grading.NonNegativeIntegers method*), 1666
an_element() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1388
an_element() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1399
an_embedding() (*sage.categories.fields.Fields.ParentMethods method*), 604
an_instance() (*sage.categories.algebra_modules.AlgebraModules class method*), 305
an_instance() (*sage.categories.bimodules.Bimodules class method*), 339
an_instance() (*sage.categories.category_types.Category_ideal class method*), 1677
an_instance() (*sage.categories.category_types.Category_over_base class method*), 1679
an_instance() (*sage.categories.category_types.Elements class method*), 1681
an_instance() (*sage.categories.category.Category class method*), 83
an_instance() (*sage.categories.g_sets.GSets class method*), 926
an_instance() (*sage.categories.groupoid.Groupoid class method*), 966
Analytic() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1136
AndersonMotives (*class in sage.categories.anderson_motives*), 320
AndersonMotives.ParentMethods (*class in sage.categories.anderson_motives*), 321
annihilator() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 816
annihilator_basis() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 818
antichains() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 874
antipode() (*sage.categories.filtered_hopf_algebras_with_basis.FilteredHopfAlgebrasWithBasis.Connected.ParentMethods method*), 625
antipode() (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis.ParentMethods method*), 1004
antipode() (*sage.categories.hopf_algebras.HopfAlgebras.ElementMethods method*), 996
antipode() (*sage.categories.hopf_algebras.HopfAlgebras.Super.ElementMethods method*), 998
antipode() (*sage.categories.super_hopf_algebras_with_basis.SuperHopfAlgebrasWithBasis.ParentMethods method*), 1464
antipode_by_coercion() (*sage.categories.hopf_algebras.HopfAlgebras.Realizations.ParentMethods method*), 997
antipode_on_basis() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1623
antipode_on_basis() (*sage.categories.filtered_hopf_algebras_with_basis.FilteredHopfAlgebrasWithBasis.Connected.ParentMethods method*), 625
antipode_on_basis() (*sage.categories.graded_hopf_algebras_with_basis.GradedHopfAlgebrasWithBasis.Connected.ParentMethods method*), 940

antipode_on_basis() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 961
antipode_on_basis() (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis.ParentMethods method*), 1005
Aperiodic (*sage.categories.semigroups.Semigroups attribute*), 1358
Aperiodic() (*sage.categories.semigroups.Semigroups.SubcategoryMethods method*), 1370
AperiodicSemigroups (*class in sage.categories.aperiodic_semigroups*), 328
apply_conjugation_by_simple_reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 404
apply_demazure_product() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 444
apply_multilinear_morphism() (*sage.categories.modules_with_basis.ModulesWithBasis.TensorProducts.ElementMethods method*), 1223
apply_reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 404
apply_simple_projection() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 445
apply_simple_reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 406
apply_simple_reflection_left() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 408
apply_simple_reflection_right() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 409
apply_simple_reflection_right() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup.Element method*), 1589
apply_simple_reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 410
as_finite_dimensional_algebra() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 759
Associative (*sage.categories.magmas.Magmas attribute*), 1101
Associative (*sage.categories.magmatic_algebras.MagmaticAlgebras attribute*), 1126
Associative() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1111
AssociativeAlgebras (*class in sage.categories.associative_algebras*), 328
axiom() (*in module sage.categories.category_with_axiom*), 153
axiom_of_nested_class() (*in module sage.categories.category_with_axiom*), 154
AxiomContainer (*class in sage.categories.category_cy_helper*), 1688
axioms() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 149
axioms() (*sage.categories.category.Category method*), 84

B

b_sharp() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1080
baker_campbell_hausdorff() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1047
Bars (*class in sage.categories.category_with_axiom*), 141
base() (*sage.categories.anderson_motives.AndersonMotives method*), 324
base() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 322
base() (*sage.categories.category_types.Category_over_base method*), 1679
base() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 568
base() (*sage.categories.homsets.HomsetsCategory method*), 1560
base() (*sage.categories.signed_tensor.SignedTensorProductsCategory method*), 1541
base() (*sage.categories.tensor.TensorProductsCategory method*), 1539
base_category() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 150
base_category() (*sage.categories.covariant_functorial_construction.FunctorialConstructionCategory method*), 1533
base_category_class_and_axiom() (*in module sage.categories.category_with_axiom*), 154
base_change_matrix() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 668
base_curve() (*sage.categories.schemes.Jacobians.ParentMethods method*), 1351
base_field() (*sage.categories.modular_abelian_varieties.ModularAbelianVarieties method*), 1146

base_field() (*sage.categories.vector_spaces.VectorSpaces method*), 1505
base_morphism() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 575
base_morphism() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 569
base_over_constants_field() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 576
base_over_constants_field() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 570
base_point() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1442
base_point_map() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1442
base_ring() (*sage.categories.algebra_functor.AlgebraFunctor method*), 1550
base_ring() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 322
base_ring() (*sage.categories.cartesian_product.CartesianProductsCategory method*), 1538
base_ring() (*sage.categories.category_types.Category_over_base_ring method*), 1680
base_ring() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 571
base_ring() (*sage.categories.modules.Modules.Homsets method*), 1154
base_ring() (*sage.categories.modules.Modules.Homsets.ParentMethods method*), 1152
base_ring() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1162
base_scheme() (*sage.categories.schemes.AbelianVarieties method*), 1350
base_scheme() (*sage.categories.schemes.Jacobians method*), 1351
base_scheme() (*sage.categories.schemes.Schemes_over_base method*), 1353
base_set() (*sage.categories.examples.with_realizations.SubsetAlgebra method*), 1674
base_space() (*sage.categories.vector_bundles.VectorBundles method*), 1499
basis() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1596
basis() (*sage.categories.examples.graded_modules_with_basis.GradedPartitionModule method*), 1618
basis() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 641
basis() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1199
basis_matrix() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1596
basis_matrix() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.Subobjects.ParentMethods method*), 803
bch() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1047
bhz_poset() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 695
Bialgebras (*class in sage.categories.bialgebras*), 329
Bialgebras.ElementMethods (*class in sage.categories.bialgebras*), 329
Bialgebras.Super (*class in sage.categories.bialgebras*), 330
BialgebrasWithBasis (*class in sage.categories.bialgebras_with_basis*), 331
BialgebrasWithBasis.ElementMethods (*class in sage.categories.bialgebras_with_basis*), 331
BialgebrasWithBasis.ParentMethods (*class in sage.categories.bialgebras_with_basis*), 335
Bimodules (*class in sage.categories.bimodules*), 338
Bimodules.ElementMethods (*class in sage.categories.bimodules*), 339
Bimodules.ParentMethods (*class in sage.categories.bimodules*), 339
binary_factorizations() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 446
birational_free_labelling() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 875
birational_rowmotion() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 881
birational_toggle() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 884
birational_toggles() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 889
BlackBoxConstructionFunctor (*class in sage.categories.pushout*), 216
Blahs (*class in sage.categories.category_with_axiom*), 142
Blahs.Commutative (*class in sage.categories.category_with_axiom*), 142
Blahs.Connected (*class in sage.categories.category_with_axiom*), 142
Blahs.FiniteDimensional (*class in sage.categories.category_with_axiom*), 142
Blahs.Flying (*class in sage.categories.category_with_axiom*), 142

Blahs.SubcategoryMethods (*class in sage.categories.category_with_axiom*), 142
Blahs.Unital (*class in sage.categories.category_with_axiom*), 143
Blahs.Unital.Blue (*class in sage.categories.category_with_axiom*), 143
Blue() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
Blue_extra_super_categories() (*sage.categories.category_with_axiom.Blahs method*), 142
Bounded() (*sage.categories.posets.Posets.SubcategoryMethods method*), 1270
bracket() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.ElementMethods method*), 1020
bracket() (*sage.categories.lie_algebras.LieAlgebras.ElementMethods method*), 1042
bracket() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1050
bracket() (*sage.categories.rings.Rings.ParentMethods method*), 1322
bracket_on_basis() (*sage.categories.examples.lie_algebras_with_basis.AbelianLieAlgebra method*), 1629
bracket_on_basis() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1064
braid_group_as_finitely_presented_group() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 481
braid_orbit() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 482
braid_orbit_iter() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 483
braid_relations() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 484
brick_coloring() (*sage.categories.lattice_posets.LatticePosets.CongruenceUniform.ParentMethods method*), 1029
bruhat_cone() (*sage.categories.weyl_groups.WeylGroups.ParentMethods method*), 1519
bruhat_graph() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 484
bruhat_interval() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 486
bruhat_interval_poset() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 486
bruhat_le() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 447
bruhat_lower_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 448
bruhat_lower_covers_coroots() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1507
bruhat_lower_covers_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 450
bruhat_poset() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 696
bruhat_upper_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 451
bruhat_upper_covers() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 691
bruhat_upper_covers_coroots() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1507
bruhat_upper_covers_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 452

C

CallMorphism (*class in sage.categories.morphism*), 201
cambrian_lattice() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 698
canonical_matrix() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 452
canonical_representation() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 487
canonicalize_axioms() (*in module sage.categories.category_cy_helper*), 1688
cardinality() (*sage.categories.classical_crystals.ClassicalCrystals.ParentMethods method*), 347
cardinality() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 669
cardinality() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts.ParentMethods method*), 835
cardinality() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.IsomorphicObjects.ParentMethods method*), 840
cardinality() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 841
cardinality() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 852

cardinality() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1081
cardinality() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ParentMethods method*), 1091
cardinality() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1199
cardinality() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1388
cardinality() (*sage.categories.sets_cat.Sets.Infinite.ParentMethods method*), 1394
cartan_invariants_matrix() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 723
cartan_type() (*sage.categories.crystals.CrystalMorphism method*), 510
cartan_type() (*sage.categories.crystals.Crystals.ElementMethods method*), 517
cartan_type() (*sage.categories.crystals.Crystals.ParentMethods method*), 528
cartan_type() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1613
cartan_type() (*sage.categories.kac_moody_algebras.KacMoodyAlgebras.ParentMethods method*), 1012
cartan_type() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.ParentMethods method*), 1287
cartan_type() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.TensorProducts.ParentMethods method*), 1288
cartesian_factors() (*sage.categories.sets_cat.Sets.CartesianProducts.ElementMethods method*), 1387
cartesian_factors() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1389
cartesian_product() (*sage.categories.sets_cat.Sets.ElementMethods method*), 1393
cartesian_product() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1400
cartesian_projection() (*sage.categories.sets_cat.Sets.CartesianProducts.ElementMethods method*), 1387
cartesian_projection() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1390
CartesianProduct (*sage.categories.posets.Posets.ParentMethods attribute*), 1256
CartesianProduct (*sage.categories.sets_cat.Sets.ParentMethods attribute*), 1397
CartesianProductFunctor (*class in sage.categories.cartesian_product*), 1535
CartesianProducts() (*sage.categories.cartesian_product.CartesianProductsCategory method*), 1538
CartesianProducts() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1404
CartesianProductsCategory (*class in sage.categories.cartesian_product*), 1538
casimir_element() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 760
catalan_number() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods method*), 679
Category (*class in sage.categories.category*), 49
category() (*sage.categories.category.Category method*), 84
category() (*sage.categories.morphism.Morphism method*), 203
Category_contains_method_by_parent_class (*class in sage.categories.category_singleton*), 1682
category_for() (*sage.categories.map.Map method*), 179
category_from_categories() (*sage.categories.covariant_functorial_construction.CovariantFunctorialConstruction method*), 1532
category_from_category() (*sage.categories.covariant_functorial_construction.CovariantFunctorialConstruction method*), 1532
category_from_parents() (*sage.categories.covariant_functorial_construction.CovariantFunctorialConstruction method*), 1533
category_graph() (*in module sage.categories.category*), 107
category_graph() (*sage.categories.category.Category method*), 84
Category_ideal (*class in sage.categories.category_types*), 1677
Category_in_ambient (*class in sage.categories.category_types*), 1678
Category_module (*class in sage.categories.category_types*), 1678
category_of() (*sage.categories.covariant_functorial_construction.FunctorialConstructionCategory class method*), 1533
Category_over_base (*class in sage.categories.category_types*), 1679
Category_over_base_ring (*class in sage.cate-*

gories.category_types), 1680
Category_realization_of_parent (*class in sage.categories.realizations*), 1563
category_sample() (*in module sage.categories.category*), 108
Category_singleton (*class in sage.categories.category_singleton*), 1682
category_sort_key() (*in module sage.categories.category_cy_helper*), 1689
CategoryWithAxiom (*class in sage.categories.category_with_axiom*), 143
CategoryWithAxiom_over_base_ring (*class in sage.categories.category_with_axiom*), 151
CategoryWithAxiom_singleton (*class in sage.categories.category_with_axiom*), 152
CategoryWithParameters (*class in sage.categories.category*), 102
cayley_graph() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1359
cayley_graph_disabled() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 853
cayley_table() (*sage.categories.groups.Groups.ParentMethods method*), 972
cell_module() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 715
cell_module_indices() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 715
cell_module_indices() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts.ParentMethods method*), 719
cell_poset() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 716
cell_poset() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts.ParentMethods method*), 719
cells() (*sage.categories.cw_complexes.CWComplexes.ParentMethods method*), 549
cells() (*sage.categories.examples.cw_complexes.Surface method*), 1587
cells() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 716
Cellular() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.SubcategoryMethods method*), 752
cellular_basis() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 716
cellular_involution() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ElementMethods method*), 714
cellular_involution() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 717
cellular_involution() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts.ParentMethods method*), 720
center() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 726
center() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 763
center_basis() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 727
center_basis() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 961
center_universal_enveloping_algebra() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1051
central_form() (*sage.categories.group_algebras.GroupAlgebras.ElementMethods method*), 959
central_orthogonal_idempotents() (*sage.categories.finite_dimensional_semisimple_algebras_with_basis.FiniteDimensionalSemisimpleAlgebrasWithBasis.Commutative.ParentMethods method*), 830
central_orthogonal_idempotents() (*sage.categories.finite_dimensional_semisimple_algebras_with_basis.FiniteDimensionalSemisimpleAlgebrasWithBasis.ParentMethods method*), 832
centralizer() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 763
centralizer_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 764

ChainComplexes (*class in sage.categories.chain_complexes*), 340
ChainComplexes.ParentMethods (*class in sage.categories.chain_complexes*), 340
ChainGraded (*sage.categories.lattice_posets.LatticePosets.Trim attribute*), 1040
ChainGraded() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1037
change_ring() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1170
character() (*sage.categories.classical_crystals.ClassicalCrystals.ParentMethods method*), 348
character() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1477
character_value() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ElementMethods method*), 658
characteristic() (*sage.categories.algebras.Algebras.ParentMethods method*), 307
characteristic() (*sage.categories.anderson_motives.AndersonMotives method*), 324
characteristic() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 322
characteristic() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 576
characteristic() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 571
characteristic() (*sage.categories.rings.Rings.ParentMethods method*), 1323
characteristic_polynomial() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 807
charpoly() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 808
chevalley_eilenberg_complex() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 765
class_group() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains.ParentMethods method*), 1271
classical_decomposition() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1081
classical_weight() (*sage.categories.loop_crystals.RegularLoopCrystals.ElementMethods method*), 1097
ClassicalCrystals (*class in sage.categories.classical_crystals*), 346
ClassicalCrystals.ElementMethods (*class in sage.categories.classical_crystals*), 346
ClassicalCrystals.ParentMethods (*class in sage.categories.classical_crystals*), 347
ClassicalCrystals.TensorProducts (*class in sage.categories.classical_crystals*), 350
classically_highest_weight_vectors() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1081
classically_highest_weight_vectors() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ParentMethods method*), 1092
Coalgebras (*class in sage.categories.coalgebras*), 352
Coalgebras.Cocommutative (*class in sage.categories.coalgebras*), 352
Coalgebras.DualObjects (*class in sage.categories.coalgebras*), 352
Coalgebras.ElementMethods (*class in sage.categories.coalgebras*), 352
Coalgebras.Filtered (*class in sage.categories.coalgebras*), 354
Coalgebras.ParentMethods (*class in sage.categories.coalgebras*), 354
Coalgebras.Realizations (*class in sage.categories.coalgebras*), 355
Coalgebras.Realizations.ParentMethods (*class in sage.categories.coalgebras*), 355
Coalgebras.SubcategoryMethods (*class in sage.categories.coalgebras*), 357
Coalgebras.Super (*class in sage.categories.coalgebras*), 357
Coalgebras.Super.SubcategoryMethods (*class in sage.categories.coalgebras*), 357
Coalgebras.Super.Supercocommutative (*class in sage.categories.coalgebras*), 358
Coalgebras.TensorProducts (*class in sage.categories.coalgebras*), 359
Coalgebras.TensorProducts.ElementMethods (*class in sage.categories.coalgebras*), 359
Coalgebras.TensorProducts.ParentMethods (*class in sage.categories.coalgebras*), 359
CoalgebrasWithBasis (*class in sage.categories.coalgebras_with_basis*), 361
CoalgebrasWithBasis.ElementMethods (*class in sage.categories.coalgebras_with_basis*), 361
CoalgebrasWithBasis.Filtered (*class in sage.categories.coalgebras_with_basis*), 362
CoalgebrasWithBasis.ParentMethods (*class in sage.categories.coalgebras_with_basis*), 362
CoalgebrasWithBasis.Super (*class in sage.categories.coalgebras_with_basis*), 364

Coalgebras.WithRealizations (*class in sage.categories.coalgebras*), 359
Coalgebras.WithRealizations.ParentMethods (*class in sage.categories.coalgebras*), 359
Cocommutative() (*sage.categories.coalgebras.Coalgebras.SubcategoryMethods method*), 357
codegrees() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 670
codegrees() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 698
codomain (*sage.categories.map.Map attribute*), 180
codomain() (*sage.categories.action.Action method*), 269
codomain() (*sage.categories.action.InverseAction method*), 271
codomain() (*sage.categories.action.PrecomposedAction method*), 272
codomain() (*sage.categories.functor.Functor method*), 161
codomain() (*sage.categories.homset.Homset method*), 195
codomain() (*sage.categories.poor_man_map.PoorMapMap method*), 1692
coefficient() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1171
coefficients() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1173
coercion_reversed (*sage.categories.pushout.ConstructionFunctor attribute*), 224
coercion_reversed (*sage.categories.pushout.SubspaceFunctor attribute*), 246
cohomology() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 767
common_base() (*sage.categories.pushout.ConstructionFunctor method*), 224
common_base() (*sage.categories.pushout.MultivariateConstructionFunctor method*), 240
Commutative (*sage.categories.algebras.Algebras attribute*), 306
Commutative (*sage.categories.division_rings.DivisionRings attribute*), 560
Commutative (*sage.categories.domains.Domains attribute*), 562
Commutative (*sage.categories.rings.Rings attribute*), 1317
Commutative() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
Commutative() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1112
Commutative_extra_super_categories() (*sage.categories.l_trivial_semigroups.LTrivialSemigroups method*), 1098
Commutative_extra_super_categories() (*sage.categories.r_trivial_semigroups.RTrivialSemigroups method*), 1348
CommutativeAdditiveGroups (*class in sage.categories.commutative_additive_groups*), 365
CommutativeAdditiveGroups.Algebras (*class in sage.categories.commutative_additive_groups*), 366
CommutativeAdditiveGroups.CartesianProducts (*class in sage.categories.commutative_additive_groups*), 366
CommutativeAdditiveGroups.CartesianProducts.ElementMethods (*class in sage.categories.commutative_additive_groups*), 366
CommutativeAdditiveMonoids (*class in sage.categories.commutative_additive_monoids*), 367
CommutativeAdditiveSemigroups (*class in sage.categories.commutative_additive_semigroups*), 368
CommutativeAlgebraIdeals (*class in sage.categories.commutative_algebra_ideals*), 369
CommutativeAlgebras (*class in sage.categories.commutative_algebras*), 369
CommutativeAlgebras.TensorProducts (*class in sage.categories.commutative_algebras*), 370
CommutativeRingIdeals (*class in sage.categories.commutative_ring_ideals*), 371
CommutativeRings (*class in sage.categories.commutative_rings*), 371
CommutativeRings.CartesianProducts (*class in sage.categories.commutative_rings*), 371
CommutativeRings.ElementMethods (*class in sage.categories.commutative_rings*), 372
CommutativeRings.Finite (*class in sage.categories.commutative_rings*), 374
CommutativeRings.Finite.ParentMethods (*class in sage.categories.commutative_rings*), 374
CommutativeRings.ParentMethods (*class in sage.categories.commutative_rings*), 378
commutes() (*sage.categories.pushout.CompletionFunctor method*), 219
commutes() (*sage.categories.pushout.ConstructionFunctor method*), 224
Compact() (*sage.categories.topological_spaces.TopologicalSpaces.SubcategoryMethods method*), 1485
Compact_extra_super_categories() (*sage.categories.cw_complexes.CWComplexes method*), 547
Complete() (*sage.categories.metric_spaces.MetricSpaces.SubcategoryMethods method*), 1144
CompleteDiscreteValuationFields (*class in sage.categories.complete_discrete_valuation*), 391
CompleteDiscreteValuationFields.Element-

Methods (*class in sage.categories.complete_discrete_valuation*), 392
CompleteDiscreteValuationRings (*class in sage.categories.complete_discrete_valuation*), 394
CompleteDiscreteValuationRings.ElementMethods (*class in sage.categories.complete_discrete_valuation*), 395
completion() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.ParentMethods method*), 932
CompletionFunctor (*class in sage.categories.pushout*), 218
Complex() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1136
ComplexManifolds (*class in sage.categories.manifolds*), 1132
ComplexReflectionGroups (*class in sage.categories.complex_reflection_groups*), 398
ComplexReflectionGroups.ParentMethods (*class in sage.categories.complex_reflection_groups*), 400
ComplexReflectionOrGeneralizedCoxeterGroups (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 402
ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 404
ComplexReflectionOrGeneralizedCoxeterGroups.Irreducible (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 412
ComplexReflectionOrGeneralizedCoxeterGroups.Irreducible.ParentMethods (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 412
ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 413
ComplexReflectionOrGeneralizedCoxeterGroups.SubcategoryMethods (*class in sage.categories.complex_reflection_or_generalized_coxeter_groups*), 431
CompositeConstructionFunctor (*class in sage.categories.pushout*), 220
CongruenceUniform() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1037
conjugacy_class() (*sage.categories.groups.Groups.ElementMethods method*), 970
conjugacy_class() (*sage.categories.groups.Groups.ParentMethods method*), 979
conjugacy_classes() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 853
conjugacy_classes_representatives() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 853
Connected() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
Connected() (*sage.categories.cw_complexes.CWComplexes.SubcategoryMethods method*), 550
Connected() (*sage.categories.filtered_modules.FilteredModules.SubcategoryMethods method*), 627
Connected() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1137
Connected() (*sage.categories.simplicial_complexes.SimplicialComplexes.SubcategoryMethods method*), 1436
Connected() (*sage.categories.topological_spaces.TopologicalSpaces.SubcategoryMethods method*), 1485
connected_components() (*sage.categories.crystals.Crystals.ParentMethods method*), 528
connected_components() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1477
connected_components_generators() (*sage.categories.crystals.Crystals.ParentMethods method*), 529
connected_components_generators() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 988
connected_components_generators() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1478
connectivity() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1444
constant_coefficient() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 577
constant_coefficient() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 572
construction() (*sage.categories.modules.Modules.TensorProducts.ParentMethods method*), 1164
construction() (*sage.categories.sets_cat.Sets.Algebras.ParentMethods method*), 1385
construction() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1390
construction() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1400
construction_tower() (*in module sage.categories.pushout*), 250
ConstructionFunctor (*class in sage.cate-*

gories.pushout), 221
content() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains.ParentMethods method*), 1271
convolution_power_of_id() (*sage.categories.bialgebras_with_basis.BialgebrasWithBasis.ElementMethods method*), 331
convolution_product() (*sage.categories.bialgebras_with_basis.BialgebrasWithBasis.ElementMethods method*), 333
convolution_product() (*sage.categories.bialgebras_with_basis.BialgebrasWithBasis.ParentMethods method*), 335
coproduct() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods method*), 362
coproduct() (*sage.categories.coalgebras.Coalgebras.ElementMethods method*), 353
coproduct() (*sage.categories.coalgebras.Coalgebras.ParentMethods method*), 354
coproduct() (*sage.categories.coalgebras.Coalgebras.WithRealizations.ParentMethods method*), 359
coproduct_by_coercion() (*sage.categories.coalgebras.Coalgebras.Realizations.ParentMethods method*), 355
coproduct_iterated() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ElementMethods method*), 361
coproduct_on_basis() (*sage.categories.algebra_functor.AlgebrasCategory.ParentMethods method*), 1551
coproduct_on_basis() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods method*), 363
coproduct_on_basis() (*sage.categories.examples.graded_connected_hopf_algebras_with_basis.GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator method*), 1615
coproduct_on_basis() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1623
coproduct_on_basis() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 962
core_label_order() (*sage.categories.lattice_posets.LatticePosets.CongruenceUniform.ParentMethods method*), 1029
coset_representative() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 453
counit() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods method*), 363
counit() (*sage.categories.coalgebras.Coalgebras.ElementMethods method*), 353
counit() (*sage.categories.coalgebras.Coalgebras.ParentMethods method*), 354
counit() (*sage.categories.coalgebras.Coalgebras.WithRealizations.ParentMethods method*), 360
counit() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 963
counit_by_coercion() (*sage.categories.coalgebras.Coalgebras.Realizations.ParentMethods method*), 356
counit_on_basis() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.ParentMethods method*), 364
counit_on_basis() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1623
counit_on_basis() (*sage.categories.graded_hopf_algebras_with_basis.GradedHopfAlgebrasWithBasis.Connected.ParentMethods method*), 940
counit_on_basis() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 963
CovariantConstructionCategory (*class in sage.categories.covariant_functorial_construction*), 1527
CovariantFunctorialConstruction (*class in sage.categories.covariant_functorial_construction*), 1531
cover() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1445
cover_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 454
covered_reflections_subgroup() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 692
covering_map() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1446
coxeter_complex() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 699
coxeter_diagram() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 488
coxeter_element() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 488
coxeter_element() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.ParentMethods method*), 685
coxeter_elements() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.ParentMethods method*), 686

coxeter_knuth_graph() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 693
coxeter_knuth_neighbor() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 694
coxeter_matrix() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 490
coxeter_matrix() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup method*), 1590
coxeter_matrix() (*sage.categories.weyl_groups.WeylGroups.ParentMethods method*), 1520
coxeter_number() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible.ParentMethods method*), 664
coxeter_number() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods method*), 680
coxeter_poset() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 701
coxeter_sorting_word() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 455
coxeter_type() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 490
CoxeterGroupAlgebras (*class in sage.categories.coxeter_group_algebras*), 432
CoxeterGroupAlgebras.ParentMethods (*class in sage.categories.coxeter_group_algebras*), 432
CoxeterGroups (*class in sage.categories.coxeter_groups*), 436
CoxeterGroups.ElementMethods (*class in sage.categories.coxeter_groups*), 439
CoxeterGroups.ParentMethods (*class in sage.categories.coxeter_groups*), 481
crystal_morphism() (*sage.categories.crystals.Crystals.ParentMethods method*), 529
CrystalHomset (*class in sage.categories.crystals*), 506
CrystalMorphism (*class in sage.categories.crystals*), 510
CrystalMorphismByGenerators (*class in sage.categories.crystals*), 513
Crystals (*class in sage.categories.crystals*), 515
Crystals.ElementMethods (*class in sage.categories.crystals*), 516
Crystals.MorphismMethods (*class in sage.categories.crystals*), 526
Crystals.ParentMethods (*class in sage.categories.crystals*), 527
Crystals.SubcategoryMethods (*class in sage.categories.crystals*), 545
Crystals.TensorProducts (*class in sage.categories.crystals*), 546
CWComplexes (*class in sage.categories.cw_complexes*), 547
CWComplexes.Connected (*class in sage.categories.cw_complexes*), 548
CWComplexes.ElementMethods (*class in sage.categories.cw_complexes*), 548
CWComplexes.Finite (*class in sage.categories.cw_complexes*), 548
CWComplexes.FiniteDimensional (*class in sage.categories.cw_complexes*), 549
CWComplexes.Finite.ParentMethods (*class in sage.categories.cw_complexes*), 548
CWComplexes.ParentMethods (*class in sage.categories.cw_complexes*), 549
CWComplexes.SubcategoryMethods (*class in sage.categories.cw_complexes*), 550
Cycle (*class in sage.categories.examples.graphs*), 1620
cycle_index() (*sage.categories.finite_permutation_groups.FinitePermutationGroups.ParentMethods method*), 868
Cycle.Element (*class in sage.categories.examples.graphs*), 1621
cyclotomic_cosets() (*sage.categories.commutative_rings.CommutativeRings.Finite.ParentMethods method*), 374

D

DedekindDomains (*class in sage.categories.dedekind_domains*), 551
DedekindDomains.ElementMethods (*class in sage.categories.dedekind_domains*), 551
DedekindDomains.ParentMethods (*class in sage.categories.dedekind_domains*), 551
default_super_categories() (*sage.categories.covariant_functorial_construction.CovariantConstructionCategory class method*), 1528
default_super_categories() (*sage.categories.covariant_functorial_construction.RegressiveCovariantConstructionCategory class method*), 1535
default_super_categories() (*sage.categories.graded_modules.GradedModulesCategory class method*), 946
default_super_categories() (*sage.categories.homsets.HomsetsCategory class method*), 1560
default_super_categories() (*sage.categories.isomorphic_objects.IsomorphicObjectsCategory class method*), 1555
default_super_categories() (*sage.categories.metric_spaces.MetricSpacesCategory class method*), 1145
default_super_categories() (*sage.categories.quotients.QuotientsCategory class method*), 1553

default_super_categories() (*sage.categories.subobjects.SubobjectsCategory class method*), 1554
default_super_categories() (*sage.categories.super_modules.SuperModulesCategory class method*), 1470
degree() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 630
degree() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.Subobjects.ElementMethods method*), 654
degree() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.Quotients.ElementMethods method*), 954
degree_negation() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.ElementMethods method*), 947
degree_negation() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.ParentMethods method*), 948
degree_on_basis() (*sage.categories.examples.graded_connected_hopf_algebras_with_basis.GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator method*), 1615
degree_on_basis() (*sage.categories.examples.graded_modules_with_basis.GradedPartitionModule method*), 1619
degree_on_basis() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 630
degree_on_basis() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.Subobjects.ParentMethods method*), 656
degree_on_basis() (*sage.categories.finite_dimensional_graded_lie_algebras_with_basis.FiniteDimensionalGradedLieAlgebrasWithBasis.Stratified.ParentMethods method*), 756
degree_on_basis() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.Quotients.ParentMethods method*), 954
degree_on_basis() (*sage.categories.lambda_bracket_algebras_with_basis.LambdaBracketAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Graded.ParentMethods method*), 1026
degrees() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup method*), 1591
degrees() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1613
degrees() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 671
degrees() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 702
demazure_character() (*sage.categories.classical_crystals.ClassicalCrystals.ParentMethods method*), 349
demazure_lusztig_eigenvectors() (*sage.categories.coxeter_group_algebras.CoxeterGroupAlgebras.ParentMethods method*), 432
demazure_lusztig_operator_on_basis() (*sage.categories.coxeter_group_algebras.CoxeterGroupAlgebras.ParentMethods method*), 433
demazure_lusztig_operators() (*sage.categories.coxeter_group_algebras.CoxeterGroupAlgebras.ParentMethods method*), 435
demazure_operator() (*sage.categories.regular_crystals.RegularCrystals.ParentMethods method*), 1306
demazure_operator_simple() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1299
demazure_product() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 491
demazure_subcrystal() (*sage.categories.regular_crystals.RegularCrystals.ParentMethods method*), 1307
denominator() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationFields.ElementMethods method*), 392
denominator() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationRings.ElementMethods method*), 395
denominator() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1276
dense_coefficient_list() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ElementMethods method*), 806
deodhar_factor_element() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 455
deodhar_lift_down() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 456
deodhar_lift_up() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 457
derivation() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 378
derivation_module() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 380
derivations_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimension-*

alLieAlgebrasWithBasis.ParentMethods method), 769
derivations_basis() (*sage.categories.magmatic_algebras.MagmaticAlgebras.WithBasis.FiniteDimensional.ParentMethods method*), 1127
derivative() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1276
derived_series() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 770
derived_subalgebra() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 772
descents() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 457
det() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 808
determinant() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 808
Differentiable() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1137
Differentiable() (*sage.categories.vector_bundles.VectorBundles.SubcategoryMethods method*), 1498
differential() (*sage.categories.chain_complexes.ChainComplexes.ParentMethods method*), 341
digraph() (*sage.categories.crystals.Crystals.ParentMethods method*), 534
digraph() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 988
digraph() (*sage.categories.loop_crystals.LoopCrystals.ParentMethods method*), 1095
digraph() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1478
DihedralGroup (*class in sage.categories.examples.finite_coxeter_groups*), 1588
DihedralGroup.Element (*class in sage.categories.examples.finite_coxeter_groups*), 1589
dimension() (*sage.categories.cw_complexes.CWComplexes.ElementMethods method*), 548
dimension() (*sage.categories.cw_complexes.CWComplexes.Finite.ParentMethods method*), 548
dimension() (*sage.categories.cw_complexes.CWComplexes.ParentMethods method*), 549
dimension() (*sage.categories.examples.cw_complexes.Surface.Element method*), 1586
dimension() (*sage.categories.examples.graphs.Cycle.Element method*), 1621
dimension() (*sage.categories.examples.manifolds.Plane method*), 1634
dimension() (*sage.categories.graphs.Graphs.ParentMethods method*), 956
dimension() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1064
dimension() (*sage.categories.manifolds.Manifolds.ParentMethods method*), 1135
dimension() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1201
dimension() (*sage.categories.simplicial_complexes.SimplicialComplexes.Finite.ParentMethods method*), 1434
dimension() (*sage.categories.vector_spaces.VectorSpaces.ParentMethods method*), 1502
direct_sum() (*sage.categories.crystals.Crystals.ParentMethods method*), 537
directed_subset() (*sage.categories.posets.Posets.ParentMethods method*), 1256
directed_subsets() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 891
DiscreteValuationFields (*class in sage.categories.discrete_valuation*), 554
DiscreteValuationFields.ElementMethods (*class in sage.categories.discrete_valuation*), 555
DiscreteValuationFields.ParentMethods (*class in sage.categories.discrete_valuation*), 555
DiscreteValuationRings (*class in sage.categories.discrete_valuation*), 556
DiscreteValuationRings.ElementMethods (*class in sage.categories.discrete_valuation*), 556
DiscreteValuationRings.ParentMethods (*class in sage.categories.discrete_valuation*), 558
dist() (*sage.categories.metric_spaces.MetricSpaces.CartesianProducts.ParentMethods method*), 1140
dist() (*sage.categories.metric_spaces.MetricSpaces.ElementMethods method*), 1142
dist() (*sage.categories.metric_spaces.MetricSpaces.ParentMethods method*), 1143
dist() (*sage.categories.metric_spaces.MetricSpaces.WithRealizations.ParentMethods method*), 1144
distinguished_reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 413
distinguished_reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 413
Distributive (*sage.categories.magmas_and_additive_magmas.MagmasAndAdditiveMagmas attribute*), 1124

Distributive() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1038
Distributive() (*sage.categories.magmas_and_additive_magmas.MagmasAndAdditiveMagmas.SubcategoryMethods method*), 1124
Distributive() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1112
DistributiveLattices (*class in sage.categories.lattice_posets*), 1026
DistributiveLattices.Finite (*class in sage.categories.lattice_posets*), 1026
DistributiveLattices.ParentMethods (*class in sage.categories.lattice_posets*), 1026
DistributiveMagmasAndAdditiveMagmas (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital.Associative (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.CartesianProducts (*class in sage.categories.distributive_magmas_and_additive_magmas*), 559
DistributiveMagmasAndAdditiveMagmas.ParentMethods (*class in sage.categories.distributive_magmas_and_additive_magmas*), 560
divides() (*sage.categories.fields.Fields.ParentMethods method*), 605
Division (*sage.categories.rings.Rings attribute*), 1317
Division() (*sage.categories.rings.Rings.SubcategoryMethods method*), 1345
DivisionRings (*class in sage.categories.division_rings*), 560
DivisionRings.ElementMethods (*class in sage.categories.division_rings*), 560
DivisionRings.ParentMethods (*class in sage.categories.division_rings*), 561
divisor() (*sage.categories.anderson_motives.AndersonMotives method*), 325
domain (*sage.categories.map.Map attribute*), 180
domain() (*sage.categories.action.Action method*), 269
domain() (*sage.categories.action.PrecomposedAction method*), 272
domain() (*sage.categories.functor.Functor method*), 162
domain() (*sage.categories.homset.Homset method*), 196
domain() (*sage.categories.poor_man_map.PoorManMap method*), 1692
Domains (*class in sage.categories.domains*), 562
domains() (*sage.categories.map.FormalCompositeMap method*), 171
domains() (*sage.categories.map.Map method*), 180
Domains.ElementMethods (*class in sage.categories.domains*), 562
Domains.ParentMethods (*class in sage.categories.domains*), 562
dot_tex() (*sage.categories.crystals.Crystals.ParentMethods method*), 538
DrinfeldModules (*class in sage.categories.drinfeld_modules*), 563
DrinfeldModules.ParentMethods (*class in sage.categories.drinfeld_modules*), 568
dual() (*sage.categories.hopf_algebras.HopfAlgebras method*), 999
dual() (*sage.categories.hopf_algebras.HopfAlgebras.Super method*), 998
dual() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1163
dual_equivalence_class() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1300
dual_equivalence_graph() (*sage.categories.regular_crystals.RegularCrystals.ParentMethods method*), 1308
DualFunctor (*class in sage.categories.dual*), 1541
DualObjects() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1157
DualObjectsCategory (*class in sage.categories.dual*), 1541
DummyObjectsOverBaseRing (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.Commutative (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.Commutative.Facade (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.Commutative.Finite (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.Commutative.FiniteDimensional (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.FiniteDimensional (*class in sage.categories.category_with_axiom*), 152

DummyObjectsOverBaseRing.FiniteDimensional.Finite (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.FiniteDimensional.Unital (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.FiniteDimensional.Unital.Commutative (*class in sage.categories.category_with_axiom*), 152
DummyObjectsOverBaseRing.Unital (*class in sage.categories.category_with_axiom*), 152

E

e() (*sage.categories.crystals.Crystals.ElementMethods method*), 518
e() (*sage.categories.examples.crystals.HighestWeightCrystalOfTypeA.Element method*), 1584
e() (*sage.categories.examples.crystals.NaiveCrystal.Element method*), 1585
e() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.ElementMethods method*), 1290
e() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.ParentMethods method*), 1488
e_on_basis() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.TensorProducts.ParentMethods method*), 1294
e_string() (*sage.categories.crystals.Crystals.ElementMethods method*), 518
e_string_to_ground_state() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ElementMethods method*), 1088
echelon_form() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 821
echelon_form() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1201
edges() (*sage.categories.examples.graphs.Cycle method*), 1621
edges() (*sage.categories.graphs.Graphs.ParentMethods method*), 956
Element (*sage.categories.crystals.CrystalHomset attribute*), 510
Element (*sage.categories.examples.infinite_enumerated_sets.NonNegativeIntegers attribute*), 1627
Element (*sage.categories.examples.manifolds.Plane attribute*), 1634
Element (*sage.categories.examples.semigroups_cython.LeftZeroSemigroup attribute*), 1651
Element (*sage.categories.examples.semirings.TernaryLogic attribute*), 1653
Element (*sage.categories.highest_weight_crystals.HighestWeightCrystalHomset attribute*), 984
element_class (*sage.categories.examples.sets_cat.PrimeNumbers attribute*), 1656
element_class (*sage.categories.examples.sets_cat.PrimeNumbers_Facade attribute*), 1661
element_class() (*sage.categories.category.Category method*), 85
element_class_set_morphism() (*sage.categories.homset.Homset method*), 196
Elements (*class in sage.categories.category_types*), 1680
elements_of_length() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 492
ElementWrapper (*sage.categories.examples.sets_cat.PrimeNumbers_Wrapper attribute*), 1665
EmptySetError, 1380
End() (*in module sage.categories.homset*), 189
end() (*in module sage.categories.homset*), 200
Endset() (*sage.categories.homsets.Homsets.SubcategoryMethods method*), 1559
Endsets() (*sage.categories.anderson_motives.AndersonMotives method*), 320
Endsets() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 567
Endsets() (*sage.categories.objects.Objects.SubcategoryMethods method*), 1246
energy_function() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ElementMethods method*), 1076
energy_function() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ElementMethods method*), 1089
Enumerated (*sage.categories.sets_cat.Sets attribute*), 1394
Enumerated() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1405
EnumeratedSets (*class in sage.categories.enumerated_sets*), 583
EnumeratedSets.CartesianProducts (*class in sage.categories.enumerated_sets*), 584
EnumeratedSets.CartesianProducts.ParentMethods (*class in sage.categories.enumerated_sets*), 584
EnumeratedSets.ElementMethods (*class in sage.categories.enumerated_sets*), 584
EnumeratedSets.ParentMethods (*class in sage.categories.enumerated_sets*), 585
Epsilon() (*sage.categories.crystals.Crystals.ElementMethods method*), 516
epsilon() (*sage.categories.crystals.Crystals.ElementMethods method*), 518

epsilon() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1301
epsilon() (*sage.categories.regular_supercrystals.RegularSuperCrystals.ElementMethods method*), 1312
epsilon() (*sage.categories.rings.Rings.ParentMethods method*), 1324
EquivariantSubobjectConstructionFunctor (*class in sage.categories.pushout*), 226
euclidean_degree() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 556
euclidean_degree() (*sage.categories.euclidean_domains.EuclideanDomains.ElementMethods method*), 594
euclidean_degree() (*sage.categories.fields.Fields.ElementMethods method*), 597
EuclideanDomains (*class in sage.categories.euclidean_domains*), 594
EuclideanDomains.ElementMethods (*class in sage.categories.euclidean_domains*), 594
EuclideanDomains.ParentMethods (*class in sage.categories.euclidean_domains*), 595
even_component() (*sage.categories.super_modules_with_basis.SuperModulesWithBasis.ElementMethods method*), 1472
Example (*class in sage.categories.examples.finite_enumerated_sets*), 1600
Example (*in module sage.categories.examples.algebras_with_basis*), 1575
Example (*in module sage.categories.examples.commutative_additive_monoids*), 1576
Example (*in module sage.categories.examples.commutative_additive_semigroups*), 1579
Example (*in module sage.categories.examples.cw_complexes*), 1585
Example (*in module sage.categories.examples.finite_coxeter_groups*), 1591
Example (*in module sage.categories.examples.finite_dimensional_algebras_with_basis*), 1591
Example (*in module sage.categories.examples.finite_dimensional_lie_algebras_with_basis*), 1600
Example (*in module sage.categories.examples.finite_monoids*), 1604
Example (*in module sage.categories.examples.finite_semigroups*), 1607
Example (*in module sage.categories.examples.finite_weyl_groups*), 1611
Example (*in module sage.categories.examples.graded_connected_hopf_algebras_with_basis*), 1614
Example (*in module sage.categories.examples.graded_modules_with_basis*), 1616
Example (*in module sage.categories.examples.graphs*), 1622
Example (*in module sage.categories.examples.infinite_enumerated_sets*), 1624
Example (*in module sage.categories.examples.lie_algebras*), 1627
Example (*in module sage.categories.examples.lie_algebras_with_basis*), 1630
Example (*in module sage.categories.examples.magmas*), 1632
Example (*in module sage.categories.examples.manifolds*), 1633
Example (*in module sage.categories.examples.monoids*), 1635
Example (*in module sage.categories.examples.semirings*), 1652
Example (*in module sage.categories.examples.sets_with_grading*), 1665
example() (*sage.categories.algebras_with_basis.AlgebrasWithBasis method*), 319
example() (*sage.categories.category.Category method*), 86
example() (*sage.categories.classical_crystals.ClassicalCrystals method*), 351
example() (*sage.categories.complex_reflection_groups.ComplexReflectionGroups method*), 401
example() (*sage.categories.crystals.Crystals method*), 546
example() (*sage.categories.facade_sets.FacadeSets method*), 1526
example() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups method*), 689
example() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible method*), 668
example() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated method*), 688
example() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible method*), 684
example() (*sage.categories.finite_crystals.FiniteCrystals method*), 712
example() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis method*), 805
example() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.IsomorphicObjects method*), 840
example() (*sage.categories.finite_groups.FiniteGroups method*), 855
example() (*sage.categories.finite_permutation_groups.FinitePermutationGroups method*), 873

example() (*sage.categories.finitely_generated_semigroups.FinitelyGeneratedSemigroups method*), 924
example() (*sage.categories.graded_hopf_algebras_with_basis.GradedHopfAlgebrasWithBasis method*), 941
example() (*sage.categories.graded_hopf_algebras_with_basis.GradedHopfAlgebrasWithBasis.Connected method*), 941
example() (*sage.categories.group_algebras.GroupAlgebras method*), 965
example() (*sage.categories.groups.Groups method*), 982
example() (*sage.categories.highest_weight_crystals.HighestWeightCrystals method*), 995
example() (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis method*), 1006
example() (*sage.categories.kac_moody_algebras.KacMoodyAlgebras method*), 1013
example() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis method*), 1066
example() (*sage.categories.lie_algebras.LieAlgebras method*), 1061
example() (*sage.categories.lie_conformal_algebras.LieConformalAlgebras method*), 1070
example() (*sage.categories.loop_crystals.LoopCrystals method*), 1096
example() (*sage.categories.magmas.Magmas.CartesianProducts method*), 1102
example() (*sage.categories.posets.Posets method*), 1270
example() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations method*), 1298
example() (*sage.categories.regular_crystals.RegularCrystals method*), 1311
example() (*sage.categories.semigroups.Semigroups method*), 1375
example() (*sage.categories.semigroups.Semigroups.Quotients method*), 1369
example() (*sage.categories.semigroups.Semigroups.Subquotients method*), 1374
example() (*sage.categories.sets_cat.Sets method*), 1427
example() (*sage.categories.sets_cat.Sets.CartesianProducts method*), 1393
example() (*sage.categories.sets_cat.Sets.WithRealizations method*), 1426
example() (*sage.categories.super_lie_conformal_algebras.SuperLieConformalAlgebras method*), 1466
example() (*sage.categories.vector_spaces.VectorSpaces.WithBasis.Filtered method*), 1503
example() (*sage.categories.vector_spaces.VectorSpaces.WithBasis.Graded method*), 1504
exp() (*sage.categories.lie_algebras.LieAlgebras.ElementMethods method*), 1043
expand() (*sage.categories.pushout.AlgebraicExtensionFunctor method*), 213
expand() (*sage.categories.pushout.CompositeConstructionFunctor method*), 221
expand() (*sage.categories.pushout.ConstructionFunctor method*), 225
expand() (*sage.categories.pushout.InfinitePolynomialFunctor method*), 233
expand() (*sage.categories.pushout.MultiPolynomialFunctor method*), 239
expand_tower() (*in module sage.categories.pushout*), 251
extend_codomain() (*sage.categories.map.Map method*), 180
extend_domain() (*sage.categories.map.Map method*), 181
extend_to_fraction_field() (*sage.categories.rings.Rings.MorphismMethods method*), 1318
extension() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 383
extra_super_categories() (*sage.categories.additive_groups.AdditiveGroups.Finite.Algebras method*), 274
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveCommutative.Algebras method*), 276
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveCommutative.CartesianProducts method*), 277
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.AdditiveInverse.CartesianProducts method*), 277
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.Algebras method*), 278
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.CartesianProducts method*), 279
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.Homsets method*), 280
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.Algebras method*), 284
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.CartesianProducts method*), 285
extra_super_categories() (*sage.categories.additive_magmas.AdditiveMagmas.Homsets method*), 285
extra_super_categories() (*sage.categories.additive_monoids.AdditiveMonoids.Homsets method*),

293
extra_super_categories() (*sage.categories.additive_semigroups.AdditiveSemigroups.Algebras method*), 297
extra_super_categories() (*sage.categories.additive_semigroups.AdditiveSemigroups.CartesianProducts method*), 297
extra_super_categories() (*sage.categories.additive_semigroups.AdditiveSemigroups.Homsets method*), 297
extra_super_categories() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.CartesianProducts method*), 315
extra_super_categories() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.TensorProducts method*), 318
extra_super_categories() (*sage.categories.algebras.Algebras.CartesianProducts method*), 306
extra_super_categories() (*sage.categories.algebras.Algebras.DualObjects method*), 307
extra_super_categories() (*sage.categories.algebras.Algebras.TensorProducts method*), 311
extra_super_categories() (*sage.categories.aperiodic_semigroups.AperiodicSemigroups method*), 328
extra_super_categories() (*sage.categories.category_with_axiom.Blahs.Flying method*), 142
extra_super_categories() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 151
extra_super_categories() (*sage.categories.classical_crystals.ClassicalCrystals.TensorProducts method*), 350
extra_super_categories() (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis.Super method*), 364
extra_super_categories() (*sage.categories.coalgebras.Coalgebras.DualObjects method*), 352
extra_super_categories() (*sage.categories.coalgebras.Coalgebras.Super method*), 358
extra_super_categories() (*sage.categories.coalgebras.Coalgebras.TensorProducts method*), 359
extra_super_categories() (*sage.categories.commutative_algebras.CommutativeAlgebras.TensorProducts method*), 370
extra_super_categories() (*sage.categories.commutative_rings.CommutativeRings.CartesianProducts method*), 371
extra_super_categories() (*sage.categories.commutative_rings.CommutativeRings.Finite method*), 378
extra_super_categories() (*sage.categories.covariant_functorial_construction.CovariantConstructionCategory method*), 1527
extra_super_categories() (*sage.categories.covariant_functorial_construction.FunctorialConstructionCategory method*), 1534
extra_super_categories() (*sage.categories.crystals.Crystals.TensorProducts method*), 546
extra_super_categories() (*sage.categories.cw_complexes.CWComplexes.Finite method*), 549
extra_super_categories() (*sage.categories.distributive_magmas_and_additive_magmas.DistributiveMagmasAndAdditiveMagmas.CartesianProducts method*), 560
extra_super_categories() (*sage.categories.division_rings.DivisionRings method*), 561
extra_super_categories() (*sage.categories.fields.Fields method*), 610
extra_super_categories() (*sage.categories.filtered_modules.FilteredModules method*), 627
extra_super_categories() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups method*), 711
extra_super_categories() (*sage.categories.finite_crystals.FiniteCrystals method*), 713
extra_super_categories() (*sage.categories.finite_crystals.FiniteCrystals.TensorProducts method*), 712
extra_super_categories() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts method*), 721
extra_super_categories() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.TensorProducts method*), 826
extra_super_categories() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts method*), 840
extra_super_categories() (*sage.categories.finite_fields.FiniteFields method*), 851
extra_super_categories() (*sage.categories.finite_groups.FiniteGroups.Algebras method*), 851
extra_super_categories() (*sage.categories.finite_lattice_posets.FiniteLatticePosets method*), 861
extra_super_categories() (*sage.categories.finite_permutation_groups.FinitePermutationGroups method*), 873
extra_super_categories() (*sage.categories.finite_sets.FiniteSets.Algebras method*), 913
extra_super_categories() (*sage.categories.finite_sets.FiniteSets.Subquotients method*), 915
extra_super_categories() (*sage.cate-*

gories.finitely_generated_semigroups.FinitelyGeneratedSemigroups method), 925
extra_super_categories() (*sage.categories.generalized_coxeter_groups.GeneralizedCoxeterGroups.Finite method*), 928
extra_super_categories() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.SignedTensorProducts method*), 935
extra_super_categories() (*sage.categories.graded_algebras.GradedAlgebras.SignedTensorProducts method*), 930
extra_super_categories() (*sage.categories.graded_coalgebras_with_basis.GradedCoalgebrasWithBasis.SignedTensorProducts method*), 938
extra_super_categories() (*sage.categories.graded_coalgebras.GradedCoalgebras.SignedTensorProducts method*), 937
extra_super_categories() (*sage.categories.graded_lie_algebras.GradedLieAlgebras.Stratified.FiniteDimensional method*), 942
extra_super_categories() (*sage.categories.graphs.Graphs.Connected method*), 955
extra_super_categories() (*sage.categories.group_algebras.GroupAlgebras method*), 966
extra_super_categories() (*sage.categories.groups.Groups.CartesianProducts method*), 969
extra_super_categories() (*sage.categories.hecke_modules.HeckeModules.Homsets method*), 984
extra_super_categories() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.TensorProducts method*), 995
extra_super_categories() (*sage.categories.homsets.Homsets.Endset method*), 1558
extra_super_categories() (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis.TensorProducts method*), 1005
extra_super_categories() (*sage.categories.hopf_algebras.HopfAlgebras.TensorProducts method*), 999
extra_super_categories() (*sage.categories.j_trivial_semigroups.JTrivialSemigroups method*), 1012
extra_super_categories() (*sage.categories.l_trivial_semigroups.LTrivialSemigroups method*), 1098
extra_super_categories() (*sage.categories.lattice_posets.DistributiveLattices method*), 1027
extra_super_categories() (*sage.categories.lattice_posets.LatticePosets.CongruenceUniform method*), 1030
extra_super_categories() (*sage.categories.lattice_posets.LatticePosets.Stone method*), 1037
extra_super_categories() (*sage.categories.lattice_posets.LatticePosets.Trim method*), 1040
extra_super_categories() (*sage.categories.lie_algebras.LieAlgebras.FiniteDimensional method*), 1046
extra_super_categories() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts method*), 1093
extra_super_categories() (*sage.categories.magmas_and_additive_magmas.MagmasAndAdditiveMagmas.CartesianProducts method*), 1124
extra_super_categories() (*sage.categories.magmas.Magmas.Algebras method*), 1101
extra_super_categories() (*sage.categories.magmas.Magmas.CartesianProducts method*), 1103
extra_super_categories() (*sage.categories.magmas.Magmas.Commutative.Algebras method*), 1103
extra_super_categories() (*sage.categories.magmas.Magmas.Commutative.CartesianProducts method*), 1104
extra_super_categories() (*sage.categories.magmas.Magmas.Unital.Algebras method*), 1118
extra_super_categories() (*sage.categories.magmas.Magmas.Unital.CartesianProducts method*), 1119
extra_super_categories() (*sage.categories.magmas.Magmas.Unital.Inverse.CartesianProducts method*), 1119
extra_super_categories() (*sage.categories.manifolds.Manifolds.AlmostComplex method*), 1133
extra_super_categories() (*sage.categories.manifolds.Manifolds.Analytic method*), 1134
extra_super_categories() (*sage.categories.manifolds.Manifolds.Smooth method*), 1135
extra_super_categories() (*sage.categories.metric_spaces.MetricSpaces.CartesianProducts method*), 1141
extra_super_categories() (*sage.categories.metric_spaces.MetricSpaces.Complete.CartesianProducts method*), 1141
extra_super_categories() (*sage.categories.modular_abelian_varieties.ModularAbelianVarieties.Homsets.Endset method*), 1146
extra_super_categories() (*sage.categories.modules_with_basis.ModulesWithBasis.CartesianProducts method*), 1169
extra_super_categories() (*sage.categories.modules_with_basis.ModulesWithBasis.DualObjects*

method), 1170
extra_super_categories() (*sage.categories.modules_with_basis.ModulesWithBasis.TensorProducts method*), 1226
extra_super_categories() (*sage.categories.modules.Modules.CartesianProducts method*), 1149
extra_super_categories() (*sage.categories.modules.Modules.FiniteDimensional method*), 1150
extra_super_categories() (*sage.categories.modules.Modules.FiniteDimensional.TensorProducts method*), 1150
extra_super_categories() (*sage.categories.modules.Modules.FinitelyPresented method*), 1151
extra_super_categories() (*sage.categories.modules.Modules.Homsets method*), 1154
extra_super_categories() (*sage.categories.modules.Modules.Homsets.Endset method*), 1152
extra_super_categories() (*sage.categories.modules.Modules.TensorProducts method*), 1164
extra_super_categories() (*sage.categories.monoids.Monoids.Algebras method*), 1233
extra_super_categories() (*sage.categories.monoids.Monoids.CartesianProducts method*), 1235
extra_super_categories() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.TensorProducts method*), 1289
extra_super_categories() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.TensorProducts method*), 1297
extra_super_categories() (*sage.categories.r_trivial_semigroups.RTrivialSemigroups method*), 1348
extra_super_categories() (*sage.categories.regular_crystals.RegularCrystals.TensorProducts method*), 1310
extra_super_categories() (*sage.categories.regular_supercrystals.RegularSuperCrystals.TensorProducts method*), 1313
extra_super_categories() (*sage.categories.schemes.AbelianVarieties.Homsets method*), 1350
extra_super_categories() (*sage.categories.schemes.AbelianVarieties.Homsets.Endset method*), 1350
extra_super_categories() (*sage.categories.semigroups.Semigroups.Algebras method*), 1358
extra_super_categories() (*sage.categories.semigroups.Semigroups.CartesianProducts method*), 1359
extra_super_categories() (*sage.categories.sets_cat.Sets.Algebras method*), 1385
extra_super_categories() (*sage.categories.sets_cat.Sets.CartesianProducts method*), 1393
extra_super_categories() (*sage.categories.sets_cat.Sets.WithRealizations method*), 1426
extra_super_categories() (*sage.categories.super_algebras_with_basis.SuperAlgebrasWithBasis method*), 1463
extra_super_categories() (*sage.categories.super_algebras_with_basis.SuperAlgebrasWithBasis.SignedTensorProducts method*), 1463
extra_super_categories() (*sage.categories.super_algebras.SuperAlgebras method*), 1460
extra_super_categories() (*sage.categories.super_algebras.SuperAlgebras.SignedTensorProducts method*), 1459
extra_super_categories() (*sage.categories.super_lie_conformal_algebras.SuperLieConformalAlgebras method*), 1466
extra_super_categories() (*sage.categories.super_modules.SuperModules method*), 1469
extra_super_categories() (*sage.categories.supercommutative_algebras.SupercommutativeAlgebras.SignedTensorProducts method*), 1475
extra_super_categories() (*sage.categories.supercrystals.SuperCrystals.TensorProducts method*), 1482
extra_super_categories() (*sage.categories.topological_spaces.TopologicalSpaces.CartesianProducts method*), 1483
extra_super_categories() (*sage.categories.topological_spaces.TopologicalSpaces.Compact.CartesianProducts method*), 1484
extra_super_categories() (*sage.categories.topological_spaces.TopologicalSpaces.Connected.CartesianProducts method*), 1485
extra_super_categories() (*sage.categories.unital_algebras.UnitalAlgebras.CartesianProducts method*), 1494
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.CartesianProducts method*), 1500
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.DualObjects method*), 1500
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.FiniteDimensional.TensorProducts method*), 1501
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.TensorProducts method*), 1502
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.WithBasis.Cartesian-*

Products method), 1503
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.WithBasis.FiniteDimensional.TensorProducts method*), 1504
extra_super_categories() (*sage.categories.vector_spaces.VectorSpaces.WithBasis.TensorProducts method*), 1504
Extremal() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1038

F

F (*sage.categories.examples.with_realizations.SubsetAlgebra attribute*), 1671
f() (*sage.categories.crystals.Crystals.ElementMethods method*), 519
f() (*sage.categories.examples.crystals.HighestWeightCrystalOfTypeA.Element method*), 1584
f() (*sage.categories.examples.crystals.NaiveCrystal.Element method*), 1585
f() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.ElementMethods method*), 1291
f() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.ParentMethods method*), 1488
f_on_basis() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.TensorProducts.ParentMethods method*), 1295
f_string() (*sage.categories.crystals.Crystals.ElementMethods method*), 519
Facade (*sage.categories.sets_cat.Sets attribute*), 1394
Facade() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1405
facade_for() (*sage.categories.facade_sets.FacadeSets.ParentMethods method*), 1524
facade_for() (*sage.categories.sets_cat.Sets.WithRealizations.ParentMethods method*), 1420
FacadeSets (*class in sage.categories.facade_sets*), 1524
FacadeSets.ParentMethods (*class in sage.categories.facade_sets*), 1524
faces() (*sage.categories.graphs.Graphs.ParentMethods method*), 956
faces() (*sage.categories.simplicial_complexes.SimplicialComplexes.ParentMethods method*), 1435
facets() (*sage.categories.graphs.Graphs.ParentMethods method*), 957
facets() (*sage.categories.simplicial_complexes.SimplicialComplexes.ParentMethods method*), 1435
factor() (*sage.categories.fields.Fields.ElementMethods method*), 598
factor() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1277
faithful_representation() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 772
fat_wedge() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.Finite.ParentMethods method*), 1439
fcp() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 809
Fields (*class in sage.categories.fields*), 596
Fields.ElementMethods (*class in sage.categories.fields*), 597
Fields.ParentMethods (*class in sage.categories.fields*), 603
Filtered (*sage.categories.algebras_with_basis.AlgebrasWithBasis attribute*), 315
Filtered (*sage.categories.algebras.Algebras attribute*), 307
Filtered (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis attribute*), 1004
Filtered (*sage.categories.modules_with_basis.ModulesWithBasis attribute*), 1198
Filtered (*sage.categories.modules.Modules attribute*), 1149
Filtered() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1158
FilteredAlgebras (*class in sage.categories.filtered_algebras*), 610
FilteredAlgebras.ParentMethods (*class in sage.categories.filtered_algebras*), 610
FilteredAlgebrasWithBasis (*class in sage.categories.filtered_algebras_with_basis*), 611
FilteredAlgebrasWithBasis.ElementMethods (*class in sage.categories.filtered_algebras_with_basis*), 611
FilteredAlgebrasWithBasis.ParentMethods (*class in sage.categories.filtered_algebras_with_basis*), 612
FilteredHopfAlgebrasWithBasis (*class in sage.categories.filtered_hopf_algebras_with_basis*), 624
FilteredHopfAlgebrasWithBasis.Connected (*class in sage.categories.filtered_hopf_algebras_with_basis*), 625
FilteredHopfAlgebrasWithBasis.Connected.ElementMethods (*class in sage.categories.filtered_hopf_algebras_with_basis*), 625
FilteredHopfAlgebrasWithBasis.Connected.ParentMethods (*class in sage.categories.filtered_hopf_algebras_with_basis*), 625
FilteredHopfAlgebrasWithBasis.WithRealiza-

tions (*class in sage.categories.filtered_hopf_algebras_with_basis*), 626
FilteredModules (*class in sage.categories.filtered_modules*), 626
FilteredModulesCategory (*class in sage.categories.filtered_modules*), 628
FilteredModules.Connected (*class in sage.categories.filtered_modules*), 627
FilteredModules.SubcategoryMethods (*class in sage.categories.filtered_modules*), 627
FilteredModulesWithBasis (*class in sage.categories.filtered_modules_with_basis*), 629
FilteredModulesWithBasis.ElementMethods (*class in sage.categories.filtered_modules_with_basis*), 630
FilteredModulesWithBasis.FiniteDimensional (*class in sage.categories.filtered_modules_with_basis*), 640
FilteredModulesWithBasis.FiniteDimensional.ParentMethods (*class in sage.categories.filtered_modules_with_basis*), 640
FilteredModulesWithBasis.ParentMethods (*class in sage.categories.filtered_modules_with_basis*), 641
FilteredModulesWithBasis.Subobjects (*class in sage.categories.filtered_modules_with_basis*), 654
FilteredModulesWithBasis.Subobjects.ElementMethods (*class in sage.categories.filtered_modules_with_basis*), 654
FilteredModulesWithBasis.Subobjects.ParentMethods (*class in sage.categories.filtered_modules_with_basis*), 656
Finite (*sage.categories.complex_reflection_groups.ComplexReflectionGroups attribute*), 400
Finite (*sage.categories.coxeter_groups.CoxeterGroups attribute*), 481
Finite (*sage.categories.crystals.Crystals attribute*), 526
Finite (*sage.categories.enumerated_sets.EnumeratedSets attribute*), 584
Finite (*sage.categories.fields.Fields attribute*), 603
Finite (*sage.categories.groups.Groups attribute*), 972
Finite (*sage.categories.lattice_posets.LatticePosets attribute*), 1031
Finite (*sage.categories.monoids.Monoids attribute*), 1237
Finite (*sage.categories.permutation_groups.PermutationGroups attribute*), 1252
Finite (*sage.categories.posets.Posets attribute*), 1256
Finite (*sage.categories.semigroups.Semigroups attribute*), 1359
Finite (*sage.categories.sets_cat.Sets attribute*), 1394
Finite (*sage.categories.weyl_groups.WeylGroups attribute*), 1519
Finite() (*sage.categories.sets_cat.Sets.Infinite.SubcategoryMethods method*), 1395
Finite() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1409
Finite_extra_super_categories() (*sage.categories.division_rings.DivisionRings method*), 560
Finite_extra_super_categories() (*sage.categories.h_trivial_semigroups.HTrivialSemigroups method*), 1006
FiniteComplexReflectionGroups (*class in sage.categories.finite_complex_reflection_groups*), 657
FiniteComplexReflectionGroups.ElementMethods (*class in sage.categories.finite_complex_reflection_groups*), 658
FiniteComplexReflectionGroups.Irreducible (*class in sage.categories.finite_complex_reflection_groups*), 662
FiniteComplexReflectionGroups.Irreducible.ParentMethods (*class in sage.categories.finite_complex_reflection_groups*), 662
FiniteComplexReflectionGroups.ParentMethods (*class in sage.categories.finite_complex_reflection_groups*), 668
FiniteComplexReflectionGroups.SubcategoryMethods (*class in sage.categories.finite_complex_reflection_groups*), 677
FiniteComplexReflectionGroups.WellGenerated (*class in sage.categories.finite_complex_reflection_groups*), 678
FiniteComplexReflectionGroups.WellGenerated.Irreducible (*class in sage.categories.finite_complex_reflection_groups*), 678
FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods (*class in sage.categories.finite_complex_reflection_groups*), 679
FiniteComplexReflectionGroups.WellGenerated.ParentMethods (*class in sage.categories.finite_complex_reflection_groups*), 684
FiniteCoxeterGroups (*class in sage.categories.finite_coxeter_groups*), 689
FiniteCoxeterGroups.ElementMethods (*class in sage.categories.finite_coxeter_groups*), 690
FiniteCoxeterGroups.ParentMethods (*class in sage.categories.finite_coxeter_groups*), 695
FiniteCrystals (*class in sage.categories.finite_crystals*), 712
FiniteCrystals.TensorProducts (*class in sage.categories.finite_crystals*), 712
FiniteDimensional (*sage.categories.algebras_with_basis.AlgebrasWithBasis attribute*), 315
FiniteDimensional (*sage.categories.graded_lie_algebras_with_basis.GradedLieAlgebrasWithBasis attribute*), 943

FiniteDimensional (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis attribute*), 1004
FiniteDimensional (*sage.categories.modules_with_basis.ModulesWithBasis attribute*), 1198
FiniteDimensional() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
FiniteDimensional() (*sage.categories.cw_complexes.CWComplexes.SubcategoryMethods method*), 550
FiniteDimensional() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1137
FiniteDimensional() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1159
FiniteDimensionalAlgebrasWithBasis (*class in sage.categories.finite_dimensional_algebras_with_basis*), 713
FiniteDimensionalAlgebrasWithBasis.Cellular (*class in sage.categories.finite_dimensional_algebras_with_basis*), 714
FiniteDimensionalAlgebrasWithBasis.Cellular.ElementMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 714
FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 715
FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts (*class in sage.categories.finite_dimensional_algebras_with_basis*), 719
FiniteDimensionalAlgebrasWithBasis.Cellular.TensorProducts.ParentMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 719
FiniteDimensionalAlgebrasWithBasis.ElementMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 722
FiniteDimensionalAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 723
FiniteDimensionalAlgebrasWithBasis.SubcategoryMethods (*class in sage.categories.finite_dimensional_algebras_with_basis*), 752
FiniteDimensionalBialgebrasWithBasis() (*in module sage.categories.finite_dimensional_bialgebras_with_basis*), 753
FiniteDimensionalCoalgebrasWithBasis() (*in module sage.categories.finite_dimensional_coalgebras_with_basis*), 753
FiniteDimensionalGradedLieAlgebrasWithBasis (*class in sage.categories.finite_dimensional_graded_lie_algebras_with_basis*), 754
FiniteDimensionalGradedLieAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_graded_lie_algebras_with_basis*), 754
FiniteDimensionalGradedLieAlgebrasWithBasis.Stratified (*class in sage.categories.finite_dimensional_graded_lie_algebras_with_basis*), 755
FiniteDimensionalGradedLieAlgebrasWithBasis.Stratified.ParentMethods (*class in sage.categories.finite_dimensional_graded_lie_algebras_with_basis*), 755
FiniteDimensionalHopfAlgebrasWithBasis (*class in sage.categories.finite_dimensional_hopf_algebras_with_basis*), 756
FiniteDimensionalHopfAlgebrasWithBasis.ElementMethods (*class in sage.categories.finite_dimensional_hopf_algebras_with_basis*), 757
FiniteDimensionalHopfAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_hopf_algebras_with_basis*), 757
FiniteDimensionalLieAlgebrasWithBasis (*class in sage.categories.finite_dimensional_lie_algebras_with_basis*), 757
FiniteDimensionalLieAlgebrasWithBasis.ElementMethods (*class in sage.categories.finite_dimensional_lie_algebras_with_basis*), 757
FiniteDimensionalLieAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_lie_algebras_with_basis*), 759
FiniteDimensionalLieAlgebrasWithBasis.Subobjects (*class in sage.categories.finite_dimensional_lie_algebras_with_basis*), 802
FiniteDimensionalLieAlgebrasWithBasis.Subobjects.ParentMethods (*class in sage.categories.finite_dimensional_lie_algebras_with_basis*), 802
FiniteDimensionalModulesWithBasis (*class in sage.categories.finite_dimensional_modules_with_basis*), 806
FiniteDimensionalModulesWithBasis.ElementMethods (*class in sage.categories.finite_dimensional_modules_with_basis*), 806
FiniteDimensionalModulesWithBasis.Homsets (*class in sage.categories.finite_dimensional_modules_with_basis*), 807
FiniteDimensionalModulesWithBasis.Homsets.Endset (*class in sage.categories.finite_dimensional_modules_with_basis*), 807
FiniteDimensionalModulesWithBasis.Hom-

sets.Endset.ElementMethods (*class in sage.categories.finite_dimensional_modules_with_basis*), 807
FiniteDimensionalModulesWithBasis.MorphismMethods (*class in sage.categories.finite_dimensional_modules_with_basis*), 812
FiniteDimensionalModulesWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_modules_with_basis*), 816
FiniteDimensionalModulesWithBasis.TensorProducts (*class in sage.categories.finite_dimensional_modules_with_basis*), 826
FiniteDimensionalNilpotentLieAlgebrasWithBasis (*class in sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis*), 827
FiniteDimensionalNilpotentLieAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis*), 827
FiniteDimensionalSemisimpleAlgebrasWithBasis (*class in sage.categories.finite_dimensional_semisimple_algebras_with_basis*), 829
FiniteDimensionalSemisimpleAlgebrasWithBasis.Commutative (*class in sage.categories.finite_dimensional_semisimple_algebras_with_basis*), 830
FiniteDimensionalSemisimpleAlgebrasWithBasis.Commutative.ParentMethods (*class in sage.categories.finite_dimensional_semisimple_algebras_with_basis*), 830
FiniteDimensionalSemisimpleAlgebrasWithBasis.ParentMethods (*class in sage.categories.finite_dimensional_semisimple_algebras_with_basis*), 832
FiniteEnumeratedSets (*class in sage.categories.finite_enumerated_sets*), 834
FiniteEnumeratedSets.CartesianProducts (*class in sage.categories.finite_enumerated_sets*), 835
FiniteEnumeratedSets.CartesianProducts.ParentMethods (*class in sage.categories.finite_enumerated_sets*), 835
FiniteEnumeratedSets.IsomorphicObjects (*class in sage.categories.finite_enumerated_sets*), 840
FiniteEnumeratedSets.IsomorphicObjects.ParentMethods (*class in sage.categories.finite_enumerated_sets*), 840
FiniteEnumeratedSets.ParentMethods (*class in sage.categories.finite_enumerated_sets*), 841
FiniteFields (*class in sage.categories.finite_fields*), 844
FiniteFields.ElementMethods (*class in sage.categories.finite_fields*), 845
FiniteFields.ParentMethods (*class in sage.categories.finite_fields*), 848
FiniteGroups (*class in sage.categories.finite_groups*), 851
FiniteGroups.Algebras (*class in sage.categories.finite_groups*), 851
FiniteGroups.Algebras.ParentMethods (*class in sage.categories.finite_groups*), 851
FiniteGroups.ElementMethods (*class in sage.categories.finite_groups*), 852
FiniteGroups.ParentMethods (*class in sage.categories.finite_groups*), 852
FiniteLatticePosets (*class in sage.categories.finite_lattice_posets*), 855
FiniteLatticePosets.ParentMethods (*class in sage.categories.finite_lattice_posets*), 856
FinitelyGenerated() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.SubcategoryMethods method*), 1023
FinitelyGenerated() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1113
FinitelyGeneratedAsLambdaBracketAlgebra (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras attribute*), 1022
FinitelyGeneratedAsLambdaBracketAlgebra (*sage.categories.lie_conformal_algebras.LieConformalAlgebras attribute*), 1070
FinitelyGeneratedAsLambdaBracketAlgebra() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.SubcategoryMethods method*), 1023
FinitelyGeneratedAsMagma (*sage.categories.magmas.Magmas attribute*), 1105
FinitelyGeneratedAsMagma (*sage.categories.semigroups.Semigroups attribute*), 1359
FinitelyGeneratedAsMagma() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1114
FinitelyGeneratedLambdaBracketAlgebras (*class in sage.categories.finitely_generated_lambda_bracket_algebras*), 916
FinitelyGeneratedLambdaBracketAlgebras.Graded (*class in sage.categories.finitely_generated_lambda_bracket_algebras*), 916
FinitelyGeneratedLambdaBracketAlgebras.ParentMethods (*class in sage.categories.finitely_generated_lambda_bracket_algebras*), 916
FinitelyGeneratedLieConformalAlgebras (*class in sage.categories.finitely_generated_lie_conformal_algebras*), 918
FinitelyGeneratedLieConformalAlgebras.Graded (*class in sage.categories.finitely_generated_lie_conformal_algebras*), 919
FinitelyGeneratedLieConformalAlge-

bras.ParentMethods (*class in sage.categories.finitely_generated_lie_conformal_algebras*), 919
FinitelyGeneratedLieConformalAlgebras.Super (*class in sage.categories.finitely_generated_lie_conformal_algebras*), 919
FinitelyGeneratedLieConformalAlgebras.Super.Graded (*class in sage.categories.finitely_generated_lie_conformal_algebras*), 920
FinitelyGeneratedMagmas (*class in sage.categories.finitely_generated_magmas*), 920
FinitelyGeneratedMagmas.ParentMethods (*class in sage.categories.finitely_generated_magmas*), 921
FinitelyGeneratedSemigroups (*class in sage.categories.finitely_generated_semigroups*), 921
FinitelyGeneratedSemigroups.Finite (*class in sage.categories.finitely_generated_semigroups*), 922
FinitelyGeneratedSemigroups.Finite.ParentMethods (*class in sage.categories.finitely_generated_semigroups*), 922
FinitelyGeneratedSemigroups.ParentMethods (*class in sage.categories.finitely_generated_semigroups*), 922
FinitelyPresented() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1159
FiniteMonoids (*class in sage.categories.finite_monoids*), 861
FiniteMonoids.ElementMethods (*class in sage.categories.finite_monoids*), 861
FiniteMonoids.ParentMethods (*class in sage.categories.finite_monoids*), 863
FinitePermutationGroups (*class in sage.categories.finite_permutation_groups*), 868
FinitePermutationGroups.ElementMethods (*class in sage.categories.finite_permutation_groups*), 868
FinitePermutationGroups.ParentMethods (*class in sage.categories.finite_permutation_groups*), 868
FinitePosets (*class in sage.categories.finite_posets*), 874
FinitePosets.ParentMethods (*class in sage.categories.finite_posets*), 874
FiniteSemigroups (*class in sage.categories.finite_semigroups*), 910
FiniteSemigroups.ParentMethods (*class in sage.categories.finite_semigroups*), 911
FiniteSets (*class in sage.categories.finite_sets*), 913
FiniteSets.Algebras (*class in sage.categories.finite_sets*), 913
FiniteSetsOrderedByInclusion (*class in sage.categories.examples.posets*), 1638
FiniteSetsOrderedByInclusion.Element (*class in sage.categories.examples.posets*), 1639
FiniteSets.ParentMethods (*class in sage.categories.finite_sets*), 914
FiniteSets.SubcategoryMethods (*class in sage.categories.finite_sets*), 914
FiniteSets.Subquotients (*class in sage.categories.finite_sets*), 914
FiniteWeylGroups (*class in sage.categories.finite_weyl_groups*), 915
FiniteWeylGroups.ElementMethods (*class in sage.categories.finite_weyl_groups*), 916
FiniteWeylGroups.ParentMethods (*class in sage.categories.finite_weyl_groups*), 916
first() (*sage.categories.enumerated_sets.EnumeratedSets.CartesianProducts.ParentMethods method*), 584
first() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 585
first() (*sage.categories.map.FormalCompositeMap method*), 171
first_descent() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 458
Flying() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
ForgetfulFunctor() (*in module sage.categories.functor*), 157
ForgetfulFunctor_generic (*class in sage.categories.functor*), 158
formal_series_ring() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.ParentMethods method*), 932
FormalCoercionMorphism (*class in sage.categories.morphism*), 201
FormalCompositeMap (*class in sage.categories.map*), 169
fraction_field() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 384
fraction_field() (*sage.categories.fields.Fields.ParentMethods method*), 606
fraction_field() (*sage.categories.integral_domains.IntegralDomains.ParentMethods method*), 1010
FractionField (*class in sage.categories.pushout*), 229
free() (*sage.categories.groups.Groups static method*), 982
free() (*sage.categories.groups.Groups.Commutative static method*), 970
free() (*sage.categories.monoids.Monoids static method*), 1240
free() (*sage.categories.monoids.Monoids.Commutative static method*), 1235
free_graded_module() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.Par-*

entMethods method), 933
free_module() (*sage.categories.rings.Rings.ParentMethods method*), 1325
FreeAlgebra (*class in sage.categories.examples.algebras_with_basis*), 1575
FreeCommutativeAdditiveMonoid (*class in sage.categories.examples.commutative_additive_monoids*), 1576
FreeCommutativeAdditiveMonoid.Element (*class in sage.categories.examples.commutative_additive_monoids*), 1578
FreeCommutativeAdditiveSemigroup (*class in sage.categories.examples.commutative_additive_semigroups*), 1579
FreeCommutativeAdditiveSemigroup.Element (*class in sage.categories.examples.commutative_additive_semigroups*), 1581
FreeMagma (*class in sage.categories.examples.magmas*), 1632
FreeMagma.Element (*class in sage.categories.examples.magmas*), 1632
FreeMonoid (*class in sage.categories.examples.monoids*), 1635
FreeMonoid.Element (*class in sage.categories.examples.monoids*), 1637
FreeSemigroup (*class in sage.categories.examples.semigroups*), 1641
FreeSemigroup.Element (*class in sage.categories.examples.semigroups*), 1641
frobenius_endomorphism() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 384
from_base_ring() (*sage.categories.unital_algebras.UnitalAlgebras.ParentMethods method*), 1495
from_base_ring() (*sage.categories.unital_algebras.UnitalAlgebras.WithBasis.ParentMethods method*), 1495
from_base_ring_from_one_basis() (*sage.categories.unital_algebras.UnitalAlgebras.WithBasis.ParentMethods method*), 1495
from_graded_conversion() (*sage.categories.filtered_algebras_with_basis.FilteredAlgebrasWithBasis.ParentMethods method*), 612
from_graded_conversion() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 642
from_reduced_word() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 415
from_set() (*sage.categories.examples.with_realizations.SubsetAlgebra.Bases.ParentMethods method*), 1670
from_vector() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1596
from_vector() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 774
from_vector() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 823
from_vector() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1064
from_vector() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1052
full_super_categories() (*sage.categories.category.Category method*), 87
fully_commutative_elements() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 492
function_ring() (*sage.categories.anderson_motives.AndersonMotives method*), 325
function_ring() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 323
function_ring() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 578
function_ring() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 573
FunctionFields (*class in sage.categories.function_fields*), 925
FunctionFields.ElementMethods (*class in sage.categories.function_fields*), 925
FunctionFields.ParentMethods (*class in sage.categories.function_fields*), 925
Functor (*class in sage.categories.functor*), 158
FunctorialConstructionCategory (*class in sage.categories.covariant_functorial_construction*), 1533
fundamental_group() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1447
fuss_catalan_number() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods method*), 681

G

G (*sage.categories.action.Action attribute*), 268
gcd() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 556

gcd() (*sage.categories.euclidean_domains.EuclideanDomains.ElementMethods method*), 594
gcd() (*sage.categories.fields.Fields.ElementMethods method*), 598
gcd() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains.ParentMethods method*), 1273
gcd() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1278
gcd_free_basis() (*sage.categories.euclidean_domains.EuclideanDomains.ParentMethods method*), 595
GcdDomains (*class in sage.categories.gcd_domains*), 927
GcdDomains.ElementMethods (*class in sage.categories.gcd_domains*), 927
GcdDomains.ParentMethods (*class in sage.categories.gcd_domains*), 927
ge() (*sage.categories.posets.Posets.ParentMethods method*), 1257
gen() (*sage.categories.finitely_generated_lambda_bracket_algebras.FinitelyGeneratedLambdaBracketAlgebras.ParentMethods method*), 917
gen() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1354
generalized_noncrossing_partitions() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible.ParentMethods method*), 665
GeneralizedCoxeterGroups (*class in sage.categories.generalized_coxeter_groups*), 928
GeneralizedCoxeterGroups.Finite (*class in sage.categories.generalized_coxeter_groups*), 928
generating_series() (*sage.categories.examples.sets_with_grading.NonNegativeIntegers method*), 1666
generating_series() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1430
gens() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1597
gens() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 824
gens() (*sage.categories.pushout.PermutationGroupFunctor method*), 241
gens() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1355
genuine_highest_weight_vectors() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1479
genuine_lowest_weight_vectors() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1479
get_axiom_index() (*in module sage.categories.category_cy_helper*), 1689
Graded (*sage.categories.algebras_with_basis.AlgebrasWithBasis attribute*), 315
Graded (*sage.categories.algebras.Algebras attribute*), 307
Graded (*sage.categories.coalgebras_with_basis.CoalgebrasWithBasis attribute*), 362
Graded (*sage.categories.coalgebras.Coalgebras attribute*), 354
Graded (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis attribute*), 1004
Graded (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis attribute*), 1063
Graded (*sage.categories.lie_algebras.LieAlgebras attribute*), 1046
Graded (*sage.categories.lie_conformal_algebras.LieConformalAlgebras attribute*), 1070
Graded (*sage.categories.modules_with_basis.ModulesWithBasis attribute*), 1198
Graded (*sage.categories.modules.Modules attribute*), 1152
Graded() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1160
graded_algebra() (*sage.categories.filtered_algebras_with_basis.FilteredAlgebrasWithBasis.ParentMethods method*), 612
graded_algebra() (*sage.categories.filtered_algebras.FilteredAlgebras.ParentMethods method*), 610
graded_algebra() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 643
graded_algebra() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.ParentMethods method*), 933
graded_algebra() (*sage.categories.graded_algebras.GradedAlgebras.ParentMethods method*), 930
graded_algebra() (*sage.categories.super_algebras_with_basis.SuperAlgebrasWithBasis.ParentMethods method*), 1462
graded_algebra() (*sage.categories.super_algebras.SuperAlgebras.ParentMethods method*), 1458
graded_component() (*sage.categories.examples.sets_with_grading.NonNegativeIntegers method*), 1666
graded_component() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1430
GradedAlgebras (*class in sage.categories.graded_algebras*), 930
GradedAlgebras.ElementMethods (*class in sage.categories.graded_algebras*), 930
GradedAlgebras.ParentMethods (*class in sage.cate-*

gories.graded_algebras), 930
GradedAlgebras.SignedTensorProducts (*class in sage.categories.graded_algebras*), 930
GradedAlgebras.SubcategoryMethods (*class in sage.categories.graded_algebras*), 931
GradedAlgebrasWithBasis (*class in sage.categories.graded_algebras_with_basis*), 931
GradedAlgebrasWithBasis.ElementMethods (*class in sage.categories.graded_algebras_with_basis*), 932
GradedAlgebrasWithBasis.FiniteDimensional (*class in sage.categories.graded_algebras_with_basis*), 932
GradedAlgebrasWithBasis.FiniteDimensional.ParentMethods (*class in sage.categories.graded_algebras_with_basis*), 932
GradedAlgebrasWithBasis.ParentMethods (*class in sage.categories.graded_algebras_with_basis*), 932
GradedAlgebrasWithBasis.SignedTensorProducts (*class in sage.categories.graded_algebras_with_basis*), 934
GradedAlgebrasWithBasis.SignedTensorProducts.ParentMethods (*class in sage.categories.graded_algebras_with_basis*), 934
GradedBialgebras() (*in module sage.categories.graded_bialgebras*), 936
GradedBialgebrasWithBasis() (*in module sage.categories.graded_bialgebras_with_basis*), 936
GradedCoalgebras (*class in sage.categories.graded_coalgebras*), 936
GradedCoalgebras.SignedTensorProducts (*class in sage.categories.graded_coalgebras*), 937
GradedCoalgebras.SubcategoryMethods (*class in sage.categories.graded_coalgebras*), 937
GradedCoalgebrasWithBasis (*class in sage.categories.graded_coalgebras_with_basis*), 938
GradedCoalgebrasWithBasis.SignedTensorProducts (*class in sage.categories.graded_coalgebras_with_basis*), 938
GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator (*class in sage.categories.examples.graded_connected_hopf_algebras_with_basis*), 1614
GradedHopfAlgebras() (*in module sage.categories.graded_hopf_algebras*), 939
GradedHopfAlgebrasWithBasis (*class in sage.categories.graded_hopf_algebras_with_basis*), 939
GradedHopfAlgebrasWithBasis.Connected (*class in sage.categories.graded_hopf_algebras_with_basis*), 940
GradedHopfAlgebrasWithBasis.Connected.ElementMethods (*class in sage.categories.graded_hopf_algebras_with_basis*), 940
GradedHopfAlgebrasWithBasis.Connected.ParentMethods (*class in sage.categories.graded_hopf_algebras_with_basis*), 940
GradedHopfAlgebrasWithBasis.ElementMethods (*class in sage.categories.graded_hopf_algebras_with_basis*), 941
GradedHopfAlgebrasWithBasis.ParentMethods (*class in sage.categories.graded_hopf_algebras_with_basis*), 941
GradedHopfAlgebrasWithBasis.WithRealizations (*class in sage.categories.graded_hopf_algebras_with_basis*), 941
GradedLieAlgebras (*class in sage.categories.graded_lie_algebras*), 941
GradedLieAlgebras.Stratified (*class in sage.categories.graded_lie_algebras*), 941
GradedLieAlgebras.Stratified.FiniteDimensional (*class in sage.categories.graded_lie_algebras*), 942
GradedLieAlgebras.SubcategoryMethods (*class in sage.categories.graded_lie_algebras*), 942
GradedLieAlgebrasWithBasis (*class in sage.categories.graded_lie_algebras_with_basis*), 943
GradedLieConformalAlgebras (*class in sage.categories.graded_lie_conformal_algebras*), 943
GradedLieConformalAlgebrasCategory (*class in sage.categories.graded_lie_conformal_algebras*), 944
GradedModules (*class in sage.categories.graded_modules*), 944
GradedModulesCategory (*class in sage.categories.graded_modules*), 945
GradedModules.ElementMethods (*class in sage.categories.graded_modules*), 945
GradedModules.ParentMethods (*class in sage.categories.graded_modules*), 945
GradedModulesWithBasis (*class in sage.categories.graded_modules_with_basis*), 947
GradedModulesWithBasis.ElementMethods (*class in sage.categories.graded_modules_with_basis*), 947
GradedModulesWithBasis.ParentMethods (*class in sage.categories.graded_modules_with_basis*), 948
GradedModulesWithBasis.Quotients (*class in sage.categories.graded_modules_with_basis*), 954
GradedModulesWithBasis.Quotients.ElementMethods (*class in sage.categories.graded_modules_with_basis*), 954
GradedModulesWithBasis.Quotients.ParentMethods (*class in sage.categories.graded_mod-*

ules_with_basis), 954
GradedPartitionModule (*class in sage.categories.examples.graded_modules_with_basis*), 1616
grading() (*sage.categories.examples.sets_with_grading.NonNegativeIntegers method*), 1667
grading() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1431
grading_set() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1431
Graphs (*class in sage.categories.graphs*), 955
Graphs.Connected (*class in sage.categories.graphs*), 955
Graphs.ParentMethods (*class in sage.categories.graphs*), 956
grassmannian_elements() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 493
group() (*sage.categories.additive_groups.AdditiveGroups.Algebras.ParentMethods method*), 273
group() (*sage.categories.algebra_functor.GroupAlgebraFunctor method*), 1553
group() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 964
group_generators() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 418
group_generators() (*sage.categories.groups.Groups.CartesianProducts.ParentMethods method*), 968
group_generators() (*sage.categories.groups.Groups.ParentMethods method*), 979
GroupAlgebraFunctor (*class in sage.categories.algebra_functor*), 1552
GroupAlgebras (*class in sage.categories.group_algebras*), 958
GroupAlgebras.ElementMethods (*class in sage.categories.group_algebras*), 959
GroupAlgebras.ParentMethods (*class in sage.categories.group_algebras*), 961
Groupoid (*class in sage.categories.groupoid*), 966
Groups (*class in sage.categories.groups*), 967
Groups.CartesianProducts (*class in sage.categories.groups*), 967
Groups.CartesianProducts.ParentMethods (*class in sage.categories.groups*), 967
Groups.Commutative (*class in sage.categories.groups*), 969
Groups.ElementMethods (*class in sage.categories.groups*), 970
Groups.ParentMethods (*class in sage.categories.groups*), 972
Groups.Topological (*class in sage.categories.groups*), 981
GSets (*class in sage.categories.g_sets*), 926
gt() (*sage.categories.posets.Posets.ParentMethods method*), 1257

H

has_descent() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 459
has_full_support() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 459
has_left_descent() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 460
has_right_descent() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 460
has_right_descent() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup.Element method*), 1589
has_right_descent() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup.Element method*), 1613
has_standard_involution() (*sage.categories.algebras.Algebras.ParentMethods method*), 308
HeckeModules (*class in sage.categories.hecke_modules*), 983
HeckeModules.Homsets (*class in sage.categories.hecke_modules*), 984
HeckeModules.Homsets.ParentMethods (*class in sage.categories.hecke_modules*), 984
HeckeModules.ParentMethods (*class in sage.categories.hecke_modules*), 984
highest_weight_vector() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 989
highest_weight_vectors() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 989
highest_weight_vectors() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.TensorProducts.ParentMethods method*), 993
highest_weight_vectors() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1480
highest_weight_vectors_iterator() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.TensorProducts.ParentMethods method*), 994
HighestWeightCrystalHomset (*class in sage.categories.highest_weight_crystals*), 984
HighestWeightCrystalMorphism (*class in sage.categories.highest_weight_crystals*), 984

HighestWeightCrystalOfTypeA (*class in sage.categories.examples.crystals*), 1582
HighestWeightCrystalOfTypeA.Element (*class in sage.categories.examples.crystals*), 1584
HighestWeightCrystals (*class in sage.categories.highest_weight_crystals*), 985
HighestWeightCrystals.ElementMethods (*class in sage.categories.highest_weight_crystals*), 985
HighestWeightCrystals.ParentMethods (*class in sage.categories.highest_weight_crystals*), 988
HighestWeightCrystals.TensorProducts (*class in sage.categories.highest_weight_crystals*), 993
HighestWeightCrystals.TensorProducts.ParentMethods (*class in sage.categories.highest_weight_crystals*), 993
hilbert_series() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.FiniteDimensional.ParentMethods method*), 640
hilbert_series() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 644
hochschild_complex() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.ParentMethods method*), 316
hodge_riemann_relations() (*sage.categories.kahler_algebras.KahlerAlgebras.ParentMethods method*), 1014
holomorph() (*sage.categories.groups.Groups.ParentMethods method*), 980
Hom() (*in module sage.categories.homset*), 190
hom() (*in module sage.categories.homset*), 200
homogeneous_component() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 630
homogeneous_component() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 645
homogeneous_component_as_submodule() (*sage.categories.finite_dimensional_graded_lie_algebras_with_basis.FiniteDimensionalGradedLieAlgebrasWithBasis.ParentMethods method*), 754
homogeneous_component_basis() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 645
homogeneous_degree() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 632
homology() (*sage.categories.chain_complexes.ChainComplexes.ParentMethods method*), 341
homology() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 775
HomologyFunctor (*class in sage.categories.chain_complexes*), 344
Homset (*class in sage.categories.homset*), 194
homset_category() (*sage.categories.homset.Homset method*), 197
Homsets (*class in sage.categories.homsets*), 1557
Homsets() (*sage.categories.anderson_motives.AndersonMotives method*), 321
Homsets() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 567
Homsets() (*sage.categories.objects.Objects.SubcategoryMethods method*), 1246
HomsetsCategory (*class in sage.categories.homsets*), 1560
Homsets.Endset (*class in sage.categories.homsets*), 1557
Homsets.Endset.ParentMethods (*class in sage.categories.homsets*), 1558
HomsetsOf (*class in sage.categories.homsets*), 1562
Homsets.ParentMethods (*class in sage.categories.homsets*), 1558
Homsets.SubcategoryMethods (*class in sage.categories.homsets*), 1559
HomsetWithBase (*class in sage.categories.homset*), 200
HopfAlgebras (*class in sage.categories.hopf_algebras*), 996
HopfAlgebras.DualCategory (*class in sage.categories.hopf_algebras*), 996
HopfAlgebras.DualCategory.ParentMethods (*class in sage.categories.hopf_algebras*), 996
HopfAlgebras.ElementMethods (*class in sage.categories.hopf_algebras*), 996
HopfAlgebras.Morphism (*class in sage.categories.hopf_algebras*), 997
HopfAlgebras.ParentMethods (*class in sage.categories.hopf_algebras*), 997
HopfAlgebras.Realizations (*class in sage.categories.hopf_algebras*), 997
HopfAlgebras.Realizations.ParentMethods (*class in sage.categories.hopf_algebras*), 997
HopfAlgebras.Super (*class in sage.categories.hopf_algebras*), 997
HopfAlgebras.Super.ElementMethods (*class in sage.categories.hopf_algebras*), 998
HopfAlgebras.TensorProducts (*class in sage.categories.hopf_algebras*), 998
HopfAlgebras.TensorProducts.ElementMethods (*class in sage.categories.hopf_algebras*), 998
HopfAlgebras.TensorProducts.ParentMethods (*class in sage.categories.hopf_algebras*), 998
HopfAlgebrasWithBasis (*class in sage.categories.hopf_algebras_with_basis*), 1000
HopfAlgebrasWithBasis.ElementMethods (*class in sage.categories.hopf_algebras_with_basis*), 1004

HopfAlgebrasWithBasis.ParentMethods (*class in sage.categories.hopf_algebras_with_basis*), 1004
HopfAlgebrasWithBasis.TensorProducts (*class in sage.categories.hopf_algebras_with_basis*), 1005
HopfAlgebrasWithBasis.TensorProducts.ElementMethods (*class in sage.categories.hopf_algebras_with_basis*), 1005
HopfAlgebrasWithBasis.TensorProducts.ParentMethods (*class in sage.categories.hopf_algebras_with_basis*), 1005
HTrivial (*sage.categories.semigroups.Semigroups attribute*), 1359
HTrivial() (*sage.categories.semigroups.Semigroups.SubcategoryMethods method*), 1370
HTrivialSemigroups (*class in sage.categories.h_trivial_semigroups*), 1006
hypercenter() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 776
hyperplane_index_set() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 419

I

ideal() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1597
ideal() (*sage.categories.fields.Fields.ParentMethods method*), 606
ideal() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 777
ideal() (*sage.categories.finitely_generated_semigroups.FinitelyGeneratedSemigroups.ParentMethods method*), 922
ideal() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.ParentMethods method*), 1022
ideal() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1052
ideal() (*sage.categories.rings.Rings.ParentMethods method*), 1326
ideal_monoid() (*sage.categories.rngs.Rngs.ParentMethods method*), 1347
ideal_submodule() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 728
idempotent_lift() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 730
idempotents() (*sage.categories.finite_semigroups.FiniteSemigroups.ParentMethods method*), 911
IdempotentSemigroups (*class in sage.categories.examples.semigroups_cython*), 1648
IdempotentSemigroups.ElementMethods (*class in sage.categories.examples.semigroups_cython*), 1648
identity() (*sage.categories.homset.Homset method*), 197
IdentityConstructionFunctor (*class in sage.categories.pushout*), 230
IdentityFunctor() (*in module sage.categories.functor*), 162
IdentityFunctor_generic (*class in sage.categories.functor*), 162
IdentityMorphism (*class in sage.categories.morphism*), 201
im_gens() (*sage.categories.crystals.CrystalMorphismByGenerators method*), 513
image() (*sage.categories.crystals.CrystalMorphismByGenerators method*), 514
image() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.MorphismMethods method*), 812
image() (*sage.categories.sets_cat.Sets.MorphismMethods method*), 1396
image_basis() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.MorphismMethods method*), 812
IncompleteSubquotientSemigroup (*class in sage.categories.examples.semigroups*), 1642
IncompleteSubquotientSemigroup.Element (*class in sage.categories.examples.semigroups*), 1643
index() (*sage.categories.lambda_bracket_algebras_with_basis.LambdaBracketAlgebrasWithBasis.ElementMethods method*), 1024
index_set() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 420
index_set() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 494
index_set() (*sage.categories.crystals.Crystals.ElementMethods method*), 519
index_set() (*sage.categories.crystals.Crystals.ParentMethods method*), 538
index_set() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup method*), 1591
index_set() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1613
index_set() (*sage.categories.quantum_group_represen-*

tations.QuantumGroupRepresentations.ParentMethods method), 1287
IndexedPolynomialRing (*class in sage.categories.examples.lie_algebras_with_basis*), 1630
indices() (*sage.categories.examples.with_realizations.SubsetAlgebra method*), 1675
indices_key() (*sage.categories.examples.with_realizations.SubsetAlgebra method*), 1675
induced_graded_map() (*sage.categories.filtered_algebras_with_basis.FilteredAlgebrasWithBasis.ParentMethods method*), 613
induced_graded_map() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 646
Infinite (*sage.categories.enumerated_sets.EnumeratedSets attribute*), 585
Infinite() (*sage.categories.finite_sets.FiniteSets.SubcategoryMethods method*), 914
Infinite() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1409
InfiniteEnumeratedSets (*class in sage.categories.infinite_enumerated_sets*), 1007
InfiniteEnumeratedSets.ParentMethods (*class in sage.categories.infinite_enumerated_sets*), 1008
InfinitePolynomialFunctor (*class in sage.categories.pushout*), 230
inject_shorthands() (*sage.categories.sets_cat.Sets.WithRealizations.ParentMethods method*), 1421
inner_derivations_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 777
IntegerModMonoid (*class in sage.categories.examples.finite_monoids*), 1604
IntegerModMonoid.Element (*class in sage.categories.examples.finite_monoids*), 1605
IntegersCompletion (*class in sage.categories.examples.facade_sets*), 1587
integral_closure() (*sage.categories.dedekind_domains.DedekindDomains.ParentMethods method*), 551
integral_closure() (*sage.categories.fields.Fields.ParentMethods method*), 607
IntegralDomains (*class in sage.categories.integral_domains*), 1009
IntegralDomains.ElementMethods (*class in sage.categories.integral_domains*), 1010
IntegralDomains.ParentMethods (*class in sage.categories.integral_domains*), 1010
intersection() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1202
invariant_module() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 824
Inverse (*sage.categories.monoids.Monoids attribute*), 1237
Inverse() (*sage.categories.magmas.Magmas.Unital.SubcategoryMethods method*), 1121
inverse() (*sage.categories.map.Section method*), 187
inverse() (*sage.categories.monoids.Monoids.ElementMethods method*), 1236
Inverse_extra_super_categories() (*sage.categories.h_trivial_semigroups.HTrivialSemigroups method*), 1007
inverse_of_unit() (*sage.categories.fields.Fields.ElementMethods method*), 599
inverse_of_unit() (*sage.categories.rings.Rings.ElementMethods method*), 1317
InverseAction (*class in sage.categories.action*), 269
inversion_arrangement() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1508
inversion_sequence() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 703
inversions() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1509
inversions_as_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 461
Irreducible() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.SubcategoryMethods method*), 431
irreducible_component_index_sets() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 421
irreducible_components() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.Irreducible.ParentMethods method*), 412
irreducible_components() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 421
irreducibles_poset() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 856
is_abelian() (*sage.categories.category_types.AbelianCategory method*), 1677
is_abelian() (*sage.categories.category.Category method*), 88

is_abelian() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 778
is_abelian() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1053
is_abelian() (*sage.categories.modules_with_basis.ModulesWithBasis method*), 1226
is_abelian() (*sage.categories.vector_spaces.VectorSpaces.WithBasis method*), 1505
is_affine_grassmannian() (*sage.categories.affine_weyl_groups.AffineWeylGroups.ElementMethods method*), 301
is_antichain_of_poset() (*sage.categories.posets.Posets.ParentMethods method*), 1258
is_bounded() (*sage.categories.posets.Posets.Bounded.ParentMethods method*), 1256
is_central() (*sage.categories.monoids.Monoids.Algebras.ElementMethods method*), 1229
is_chain_of_poset() (*sage.categories.posets.Posets.ParentMethods method*), 1260
is_commutative() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 385
is_commutative() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 494
is_commutative() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 731
is_commutative() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1054
is_commutative() (*sage.categories.magmas.Magmas.Commutative.ParentMethods method*), 1104
is_commutative() (*sage.categories.rings.Rings.ParentMethods method*), 1328
is_congruence_uniform() (*sage.categories.lattice_posets.LatticePosets.CongruenceUniform.ParentMethods method*), 1030
is_connected() (*sage.categories.crystals.Crystals.ParentMethods method*), 538
is_construction_defined_by_base() (*sage.categories.covariant_functorial_construction.CovariantConstructionCategory method*), 1530
is_coxeter_element() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ElementMethods method*), 694
is_coxeter_sortable() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 461
is_distributive() (*sage.categories.lattice_posets.DistributiveLattices.ParentMethods method*), 1027
is_embedding() (*sage.categories.crystals.Crystals.MorphismMethods method*), 526
is_empty() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.ParentMethods method*), 280
is_empty() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 585
is_empty() (*sage.categories.magmas.Magmas.Unital.ParentMethods method*), 1120
is_empty() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1390
is_empty() (*sage.categories.sets_cat.Sets.Infinite.ParentMethods method*), 1394
is_endomorphism() (*sage.categories.morphism.Morphism method*), 204
is_endomorphism_set() (*sage.categories.homsets.Homsets.Endset.ParentMethods method*), 1558
is_endomorphism_set() (*sage.categories.homsets.Homsets.ParentMethods method*), 1559
is_euclidean_domain() (*sage.categories.euclidean_domains.EuclideanDomains.ParentMethods method*), 596
is_even() (*sage.categories.super_modules.SuperModules.ElementMethods method*), 1467
is_even_odd() (*sage.categories.lie_conformal_algebras.LieConformalAlgebras.ElementMethods method*), 1069
is_even_odd() (*sage.categories.super_lie_conformal_algebras.SuperLieConformalAlgebras.ElementMethods method*), 1465
is_even_odd() (*sage.categories.super_modules_with_basis.SuperModulesWithBasis.ElementMethods method*), 1472
is_even_odd() (*sage.categories.super_modules.SuperModules.ElementMethods method*), 1468
is_extremal() (*sage.categories.lattice_posets.LatticePosets.Extremal.ParentMethods method*), 1031
is_field() (*sage.categories.fields.Fields.ParentMethods method*), 607
is_field() (*sage.categories.integral_domains.IntegralDomains.ParentMethods method*), 1010
is_field() (*sage.categories.magmas.Magmas.Algebras.ParentMethods method*), 1100
is_field() (*sage.categories.monoids.Monoids.Algebras.ParentMethods method*), 1232
is_field() (*sage.categories.rings.Rings.ParentMethods method*), 1328
is_finite() (*sage.categories.finite_sets.FiniteSets.ParentMethods method*), 914
is_finite() (*sage.categories.modules_with_basis.Mod-*

ulesWithBasis.ParentMethods method), 1202
is_finite() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1391
is_finite() (*sage.categories.sets_cat.Sets.Infinite.ParentMethods method*), 1395
is_finite() (*sage.categories.simplicial_sets.SimplicialSets.ParentMethods method*), 1437
is_full_subcategory() (*sage.categories.category.Category method*), 89
is_fully_commutative() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 463
is_genuine_highest_weight() (*sage.categories.supercrystals.SuperCrystals.Finite.ElementMethods method*), 1475
is_genuine_lowest_weight() (*sage.categories.supercrystals.SuperCrystals.Finite.ElementMethods method*), 1476
is_graded() (*sage.categories.lattice_posets.LatticePosets.ChainGraded.ParentMethods method*), 1028
is_grassmannian() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 463
is_grouplike() (*sage.categories.bialgebras.Bialgebras.ElementMethods method*), 329
is_highest_weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 520
is_homogeneous() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 634
is_ideal() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1597
is_ideal() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 778
is_ideal() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1055
is_idempotent() (*sage.categories.examples.semigroups_cython.IdempotentSemigroups.ElementMethods method*), 1648
is_idempotent() (*sage.categories.examples.semigroups.LeftZeroSemigroup.Element method*), 1644
is_idempotent() (*sage.categories.magmas.Magmas.ElementMethods method*), 1105
is_identity() (*sage.categories.morphism.IdentityMorphism method*), 201
is_identity() (*sage.categories.morphism.Morphism method*), 204
is_identity_decomposition_into_orthogonal_idempotents() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 731
is_injective() (*sage.categories.crystals.CrystalMorphism method*), 511
is_injective() (*sage.categories.map.FormalCompositeMap method*), 172
is_injective() (*sage.categories.morphism.IdentityMorphism method*), 202
is_injective() (*sage.categories.morphism.SetIsomorphism method*), 207
is_injective() (*sage.categories.rings.Rings.MorphismMethods method*), 1319
is_injective() (*sage.categories.sets_cat.Sets.MorphismMethods method*), 1397
is_integral_domain() (*sage.categories.group_algebras.GroupAlgebras.ParentMethods method*), 964
is_integral_domain() (*sage.categories.integral_domains.IntegralDomains.ParentMethods method*), 1010
is_integral_domain() (*sage.categories.rings.Rings.ParentMethods method*), 1329
is_integrally_closed() (*sage.categories.dedekind_domains.DedekindDomains.ParentMethods method*), 552
is_integrally_closed() (*sage.categories.fields.Fields.ParentMethods method*), 607
is_integrally_closed() (*sage.categories.rings.Rings.ParentMethods method*), 1331
is_irreducible() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 422
is_isomorphism() (*sage.categories.crystals.Crystals.MorphismMethods method*), 526
is_isomorphism() (*sage.categories.regular_crystals.RegularCrystals.MorphismMethods method*), 1305
is_lattice() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 891
is_lattice_morphism() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 856
is_left() (*sage.categories.action.Action method*), 269
is_lowest_weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 520
is_Map() (*in module sage.categories.map*), 187
is_Morphism() (*in module sage.categories.morphism*), 209
is_nilpotent() (*sage.categories.finite_dimen-*

sional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method), 779
is_nilpotent() (*sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis.FiniteDimensionalNilpotentLieAlgebrasWithBasis.ParentMethods method*), 827
is_nilpotent() (*sage.categories.lie_algebras.LieAlgebras.Nilpotent.ParentMethods method*), 1047
is_nilpotent() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1055
is_noetherian() (*sage.categories.dedekind_domains.DedekindDomains.ParentMethods method*), 553
is_noetherian() (*sage.categories.noetherian_rings.NoetherianRings.ParentMethods method*), 1242
is_noetherian() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains.ParentMethods method*), 1274
is_noetherian() (*sage.categories.rings.Rings.ParentMethods method*), 1332
is_odd() (*sage.categories.super_modules.SuperModules.ElementMethods method*), 1468
is_one() (*sage.categories.monoids.Monoids.ElementMethods method*), 1236
is_order_filter() (*sage.categories.posets.Posets.ParentMethods method*), 1264
is_order_ideal() (*sage.categories.posets.Posets.ParentMethods method*), 1265
is_parent_of() (*sage.categories.facade_sets.FacadeSets.ParentMethods method*), 1525
is_parent_of() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1401
is_perfect() (*sage.categories.fields.Fields.ParentMethods method*), 608
is_perfect() (*sage.categories.finite_fields.FiniteFields.ParentMethods method*), 848
is_perfect() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1082
is_pieri_factor() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1510
is_pointed() (*sage.categories.simplicial_sets.SimplicialSets.ParentMethods method*), 1438
is_poset_isomorphism() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 893
is_poset_morphism() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 894
is_prime() (*sage.categories.examples.sets_cat.PrimeNumbers_Abstract.Element method*), 1657
is_prime_field() (*sage.categories.rings.Rings.ParentMethods method*), 1332
is_primitive() (*sage.categories.bialgebras.Bialgebras.ElementMethods method*), 329
is_real() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 671
is_real() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 704
is_reducible() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 422
is_reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 411
is_ring() (*sage.categories.rings.Rings.ParentMethods method*), 1334
is_self_dual() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 895
is_semidistributive() (*sage.categories.lattice_posets.LatticePosets.Semidistributive.ParentMethods method*), 1032
is_semisimple() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 780
is_simply_connected() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1450
is_solvable() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 781
is_solvable() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1055
is_square() (*sage.categories.commutative_rings.CommutativeRings.ElementMethods method*), 372
is_square() (*sage.categories.finite_fields.FiniteFields.ElementMethods method*), 845
is_stone() (*sage.categories.lattice_posets.LatticePosets.Stone.ParentMethods method*), 1036
is_strict() (*sage.categories.crystals.Crystals.MorphismMethods method*), 527
is_subcategory() (*sage.categories.category.Category method*), 90
is_subcategory() (*sage.categories.category.JoinCategory method*), 106
is_subring() (*sage.categories.rings.Rings.ParentMethods method*), 1334
is_super_homogeneous() (*sage.categories.super_modules_with_basis.SuperModulesWithBasis.ElementMethods method*), 1473

is_surjective() (*sage.categories.crystals.CrystalMorphism method*), 511
is_surjective() (*sage.categories.map.FormalCompositeMap method*), 174
is_surjective() (*sage.categories.map.Map method*), 182
is_surjective() (*sage.categories.morphism.IdentityMorphism method*), 202
is_surjective() (*sage.categories.morphism.SetIsomorphism method*), 208
is_trim() (*sage.categories.lattice_posets.LatticePosets.Trim.ParentMethods method*), 1040
is_unique_factorization_domain() (*sage.categories.unique_factorization_domains.UniqueFactorizationDomains.ParentMethods method*), 1492
is_unit() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 556
is_unit() (*sage.categories.fields.Fields.ElementMethods method*), 600
is_unit() (*sage.categories.rings.Rings.ElementMethods method*), 1318
is_well_generated() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 672
is_well_generated() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.ParentMethods method*), 687
is_zero() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1174
is_zero() (*sage.categories.rings.Rings.ParentMethods method*), 1334
IsomorphicObjectOfFiniteEnumeratedSet (*class in sage.categories.examples.finite_enumerated_sets*), 1602
IsomorphicObjects() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1409
IsomorphicObjectsCategory (*class in sage.categories.isomorphic_objects*), 1555
isotypic_projective_modules() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 735
items() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1174
iterator_range() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 585
iterator_range() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 841

J

j_classes() (*sage.categories.finite_semigroups.FiniteSemigroups.ParentMethods method*), 911
j_classes_of_idempotents() (*sage.categories.finite_semigroups.FiniteSemigroups.ParentMethods method*), 912
j_transversal_of_idempotents() (*sage.categories.finite_semigroups.FiniteSemigroups.ParentMethods method*), 912
Jacobians (*class in sage.categories.schemes*), 1350
Jacobians.ParentMethods (*class in sage.categories.schemes*), 1351
join() (*sage.categories.category.Category static method*), 92
join() (*sage.categories.lattice_posets.LatticePosets.ParentMethods method*), 1031
join_as_tuple() (*in module sage.categories.category_cy_helper*), 1690
join_irreducibles() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 859
join_irreducibles_poset() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 859
JoinCategory (*class in sage.categories.category*), 104
JTrivial (*sage.categories.semigroups.Semigroups attribute*), 1359
JTrivial() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1115
JTrivial() (*sage.categories.semigroups.Semigroups.SubcategoryMethods method*), 1371
JTrivialSemigroups (*class in sage.categories.j_trivial_semigroups*), 1012

K

K() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.ElementMethods method*), 1289
K_on_basis() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.TensorProducts.ParentMethods method*), 1293
KacMoodyAlgebras (*class in sage.categories.kac_moody_algebras*), 1012
KacMoodyAlgebras.ParentMethods (*class in sage.categories.kac_moody_algebras*), 1012
KahlerAlgebras (*class in sage.categories.kahler_algebras*), 1014
KahlerAlgebras.ParentMethods (*class in sage.categories.kahler_algebras*), 1014
kappa() (*sage.categories.lattice_posets.LatticePosets.Semidistributive.ParentMethods method*), 1033

kappa_dual() (*sage.categories.lattice_posets.LatticePosets.Semidistributive.ParentMethods method*), 1033
kazhdan_lusztig_cell() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 464
kazhdan_lusztig_cells() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 494
kernel() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.MorphismMethods method*), 812
kernel_basis() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.MorphismMethods method*), 813
killing_form() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 781
killing_form() (*sage.categories.lie_algebras.LieAlgebras.ElementMethods method*), 1044
killing_form() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1056
killing_form_matrix() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 782
killing_matrix() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 782
KirillovReshetikhinCrystals (*class in sage.categories.loop_crystals*), 1076
KirillovReshetikhinCrystals.ElementMethods (*class in sage.categories.loop_crystals*), 1076
KirillovReshetikhinCrystals.ParentMethods (*class in sage.categories.loop_crystals*), 1077
KirillovReshetikhinCrystals.TensorProducts (*class in sage.categories.loop_crystals*), 1086
KirillovReshetikhinCrystals.TensorProducts.ElementMethods (*class in sage.categories.loop_crystals*), 1086
KirillovReshetikhinCrystals.TensorProducts.ParentMethods (*class in sage.categories.loop_crystals*), 1091
KroneckerQuiverPathAlgebra (*class in sage.categories.examples.finite_dimensional_algebras_with_basis*), 1591
krull_dimension() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 386
krull_dimension() (*sage.categories.dedekind_domains.DedekindDomains.ParentMethods method*), 553
krull_dimension() (*sage.categories.fields.Fields.ParentMethods method*), 608

L

Lambda() (*sage.categories.crystals.Crystals.ParentMethods method*), 527
LambdaBracketAlgebras (*class in sage.categories.lambda_bracket_algebras*), 1020
LambdaBracketAlgebras.ElementMethods (*class in sage.categories.lambda_bracket_algebras*), 1020
LambdaBracketAlgebras.ParentMethods (*class in sage.categories.lambda_bracket_algebras*), 1022
LambdaBracketAlgebras.SubcategoryMethods (*class in sage.categories.lambda_bracket_algebras*), 1023
LambdaBracketAlgebrasWithBasis (*class in sage.categories.lambda_bracket_algebras_with_basis*), 1024
LambdaBracketAlgebrasWithBasis.ElementMethods (*class in sage.categories.lambda_bracket_algebras_with_basis*), 1024
LambdaBracketAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra (*class in sage.categories.lambda_bracket_algebras_with_basis*), 1025
LambdaBracketAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Graded (*class in sage.categories.lambda_bracket_algebras_with_basis*), 1025
LambdaBracketAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Graded.ParentMethods (*class in sage.categories.lambda_bracket_algebras_with_basis*), 1026
last() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts.ParentMethods method*), 836
last() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 842
latex() (*sage.categories.crystals.Crystals.ParentMethods method*), 539
latex_file() (*sage.categories.crystals.Crystals.ParentMethods method*), 539
LatticePosets (*class in sage.categories.lattice_posets*), 1027
LatticePosets.ChainGraded (*class in sage.categories.lattice_posets*), 1028
LatticePosets.ChainGraded.ParentMethods (*class in sage.categories.lattice_posets*), 1028
LatticePosets.CongruenceUniform (*class in sage.categories.lattice_posets*), 1028

LatticePosets.CongruenceUniform.ParentMethods (*class in sage.categories.lattice_posets*), 1029
LatticePosets.Extremal (*class in sage.categories.lattice_posets*), 1030
LatticePosets.Extremal.ParentMethods (*class in sage.categories.lattice_posets*), 1031
LatticePosets.ParentMethods (*class in sage.categories.lattice_posets*), 1031
LatticePosets.Semidistributive (*class in sage.categories.lattice_posets*), 1032
LatticePosets.Semidistributive.ParentMethods (*class in sage.categories.lattice_posets*), 1032
LatticePosets.Stone (*class in sage.categories.lattice_posets*), 1036
LatticePosets.Stone.ParentMethods (*class in sage.categories.lattice_posets*), 1036
LatticePosets.SubcategoryMethods (*class in sage.categories.lattice_posets*), 1037
LatticePosets.Trim (*class in sage.categories.lattice_posets*), 1039
LatticePosets.Trim.ParentMethods (*class in sage.categories.lattice_posets*), 1040
LaurentPolynomialFunctor (*class in sage.categories.pushout*), 234
lcm() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 557
lcm() (*sage.categories.fields.Fields.ElementMethods method*), 601
lcm() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1279
le() (*sage.categories.examples.posets.FiniteSetsOrderedByInclusion method*), 1639
le() (*sage.categories.examples.posets.PositiveIntegersOrderedByDivisibilityFacade method*), 1640
le() (*sage.categories.posets.Posets.ParentMethods method*), 1265
leading_coefficient() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1175
leading_item() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1176
leading_monomial() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1178
leading_monomials() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1598
leading_support() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1179
leading_term() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1180
lefschetz_element() (*sage.categories.kahler_algebras.KahlerAlgebras.ParentMethods method*), 1018
left_base_ring() (*sage.categories.bimodules.Bimodules method*), 339
left_domain() (*sage.categories.action.Action method*), 269
left_inversions_as_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 467
left_pieri_factorizations() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1511
left_precomposition (*sage.categories.action.PrecomposedAction attribute*), 272
LeftModules (*class in sage.categories.left_modules*), 1040
LeftModules.ElementMethods (*class in sage.categories.left_modules*), 1041
LeftModules.ParentMethods (*class in sage.categories.left_modules*), 1041
LeftRegularBand (*class in sage.categories.examples.finite_semigroups*), 1607
LeftRegularBand.Element (*class in sage.categories.examples.finite_semigroups*), 1610
LeftZeroSemigroup (*class in sage.categories.examples.semigroups*), 1643
LeftZeroSemigroup (*class in sage.categories.examples.semigroups_cython*), 1649
LeftZeroSemigroup.Element (*class in sage.categories.examples.semigroups*), 1644
LeftZeroSemigroupElement (*class in sage.categories.examples.semigroups_cython*), 1651
length() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 467
length() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1182
level() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1083
Lie (*sage.categories.groups.Groups attribute*), 972
lie_algebra_generators() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1598
lie_algebra_generators() (*sage.categories.examples.lie_algebras_with_basis.AbelianLieAlgebra method*), 1630
lie_algebra_generators() (*sage.categories.examples.lie_algebras.LieAlgebraFromAssociative method*), 1628
lie_group() (*sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis.FiniteDimension-*

alNilpotentLieAlgebrasWithBasis.ParentMethods method), 827
lie_group() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1056
LieAlgebraFromAssociative (*class in sage.categories.examples.lie_algebras*), 1627
LieAlgebraFromAssociative.Element (*class in sage.categories.examples.lie_algebras*), 1628
LieAlgebras (*class in sage.categories.lie_algebras*), 1041
LieAlgebras.ElementMethods (*class in sage.categories.lie_algebras*), 1042
LieAlgebras.FiniteDimensional (*class in sage.categories.lie_algebras*), 1046
LieAlgebras.Nilpotent (*class in sage.categories.lie_algebras*), 1046
LieAlgebras.Nilpotent.ParentMethods (*class in sage.categories.lie_algebras*), 1046
LieAlgebras.ParentMethods (*class in sage.categories.lie_algebras*), 1047
LieAlgebras.SubcategoryMethods (*class in sage.categories.lie_algebras*), 1061
LieAlgebrasWithBasis (*class in sage.categories.lie_algebras_with_basis*), 1062
LieAlgebrasWithBasis.ElementMethods (*class in sage.categories.lie_algebras_with_basis*), 1062
LieAlgebrasWithBasis.ParentMethods (*class in sage.categories.lie_algebras_with_basis*), 1063
LieConformalAlgebras (*class in sage.categories.lie_conformal_algebras*), 1068
LieConformalAlgebras.ElementMethods (*class in sage.categories.lie_conformal_algebras*), 1069
LieConformalAlgebras.ParentMethods (*class in sage.categories.lie_conformal_algebras*), 1070
LieConformalAlgebrasWithBasis (*class in sage.categories.lie_conformal_algebras_with_basis*), 1072
LieConformalAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra (*class in sage.categories.lie_conformal_algebras_with_basis*), 1072
LieConformalAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Graded (*class in sage.categories.lie_conformal_algebras_with_basis*), 1072
LieConformalAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Super (*class in sage.categories.lie_conformal_algebras_with_basis*), 1073
LieConformalAlgebrasWithBasis.FinitelyGeneratedAsLambdaBracketAlgebra.Super.Graded (*class in sage.categories.lie_conformal_algebras_with_basis*), 1073
LieConformalAlgebrasWithBasis.Graded (*class in sage.categories.lie_conformal_algebras_with_basis*), 1074
LieConformalAlgebrasWithBasis.Super (*class in sage.categories.lie_conformal_algebras_with_basis*), 1074
LieConformalAlgebrasWithBasis.Super.Graded (*class in sage.categories.lie_conformal_algebras_with_basis*), 1074
LieConformalAlgebrasWithBasis.Super.ParentMethods (*class in sage.categories.lie_conformal_algebras_with_basis*), 1075
LieGroups (*class in sage.categories.lie_groups*), 1075
lift() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1598
lift() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra.Element method*), 1594
lift() (*sage.categories.examples.finite_enumerated_sets.IsomorphicObjectOfFiniteEnumeratedSet method*), 1603
lift() (*sage.categories.examples.lie_algebras_with_basis.AbelianLieAlgebra.Element method*), 1629
lift() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1646
lift() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ElementMethods method*), 1062
lift() (*sage.categories.lie_algebras.LieAlgebras.ElementMethods method*), 1044
lift() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1057
lift() (*sage.categories.sets_cat.Sets.Subquotients.ElementMethods method*), 1417
lift() (*sage.categories.sets_cat.Sets.Subquotients.ParentMethods method*), 1418
lift_from_homology() (*sage.categories.chain_complexes.ChainComplexes.ParentMethods method*), 343
lift_to_precision() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationRings.ElementMethods method*), 396
LiftMorphism (*class in sage.categories.lie_algebras*), 1062
linear_combination() (*sage.categories.modules.Modules.ParentMethods method*), 1154
list() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 587
list() (*sage.categories.infinite_enumerated_sets.InfiniteEnumeratedSets.ParentMethods method*), 1008
local_energy_function() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1084
LocalEnergyFunction (*class in sage.categories.loop_crystals*), 1094
localization() (*sage.categories.integral_domains.Inte-*

gralDomains.ParentMethods method), 1011
localization() (*sage.categories.rings.Rings.ParentMethods method*), 1335
long_element() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 704
LoopCrystals (*class in sage.categories.loop_crystals*), 1095
LoopCrystals.ParentMethods (*class in sage.categories.loop_crystals*), 1095
lower_central_series() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 783
lower_cover_reflections() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 468
lower_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 469
lower_covers() (*sage.categories.posets.Posets.ParentMethods method*), 1266
lowest_weight_vectors() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 990
lowest_weight_vectors() (*sage.categories.supercrystals.SuperCrystals.Finite.ParentMethods method*), 1481
lt() (*sage.categories.posets.Posets.ParentMethods method*), 1266
LTrivial (*sage.categories.semigroups.Semigroups attribute*), 1359
LTrivial() (*sage.categories.semigroups.Semigroups.SubcategoryMethods method*), 1372
LTrivialSemigroups (*class in sage.categories.l_trivial_semigroups*), 1098
lusztig_involution() (*sage.categories.classical_crystals.ClassicalCrystals.ElementMethods method*), 346
lusztig_involution() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ElementMethods method*), 1077

M

m_cambrian_lattice() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 705
magma_generators() (*sage.categories.examples.magmas.FreeMagma method*), 1633
magma_generators() (*sage.categories.finitely_generated_magmas.FinitelyGeneratedMagmas.ParentMethods method*), 921
magma_generators() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1364
Magmas (*class in sage.categories.magmas*), 1099
Magmas.Algebras (*class in sage.categories.magmas*), 1100
Magmas.Algebras.ParentMethods (*class in sage.categories.magmas*), 1100
MagmasAndAdditiveMagmas (*class in sage.categories.magmas_and_additive_magmas*), 1122
MagmasAndAdditiveMagmas.CartesianProducts (*class in sage.categories.magmas_and_additive_magmas*), 1123
MagmasAndAdditiveMagmas.SubcategoryMethods (*class in sage.categories.magmas_and_additive_magmas*), 1124
Magmas.CartesianProducts (*class in sage.categories.magmas*), 1101
Magmas.CartesianProducts.ParentMethods (*class in sage.categories.magmas*), 1101
Magmas.Commutative (*class in sage.categories.magmas*), 1103
Magmas.Commutative.Algebras (*class in sage.categories.magmas*), 1103
Magmas.Commutative.CartesianProducts (*class in sage.categories.magmas*), 1104
Magmas.Commutative.ParentMethods (*class in sage.categories.magmas*), 1104
Magmas.ElementMethods (*class in sage.categories.magmas*), 1105
Magmas.JTrivial (*class in sage.categories.magmas*), 1105
Magmas.ParentMethods (*class in sage.categories.magmas*), 1105
Magmas.Realizations (*class in sage.categories.magmas*), 1111
Magmas.Realizations.ParentMethods (*class in sage.categories.magmas*), 1111
Magmas.SubcategoryMethods (*class in sage.categories.magmas*), 1111
Magmas.Subquotients (*class in sage.categories.magmas*), 1116
Magmas.Subquotients.ParentMethods (*class in sage.categories.magmas*), 1117
Magmas.Unital (*class in sage.categories.magmas*), 1117
Magmas.Unital.Algebras (*class in sage.categories.magmas*), 1117
Magmas.Unital.CartesianProducts (*class in sage.categories.magmas*), 1118
Magmas.Unital.CartesianProducts.ElementMethods (*class in sage.categories.magmas*), 1118
Magmas.Unital.CartesianProducts.ParentMethods (*class in sage.categories.magmas*), 1118
Magmas.Unital.ElementMethods (*class in sage.categories.magmas*), 1119
Magmas.Unital.Inverse (*class in sage.categories.mag-*

mas), 1119
Magmas.Unital.Inverse.CartesianProducts (*class in sage.categories.magmas*), 1119
Magmas.Unital.ParentMethods (*class in sage.categories.magmas*), 1120
Magmas.Unital.Realizations (*class in sage.categories.magmas*), 1121
Magmas.Unital.Realizations.ParentMethods (*class in sage.categories.magmas*), 1121
Magmas.Unital.SubcategoryMethods (*class in sage.categories.magmas*), 1121
MagmaticAlgebras (*class in sage.categories.magmatic_algebras*), 1125
MagmaticAlgebras.ParentMethods (*class in sage.categories.magmatic_algebras*), 1126
MagmaticAlgebras.WithBasis (*class in sage.categories.magmatic_algebras*), 1127
MagmaticAlgebras.WithBasis.FiniteDimensional (*class in sage.categories.magmatic_algebras*), 1127
MagmaticAlgebras.WithBasis.FiniteDimensional.ParentMethods (*class in sage.categories.magmatic_algebras*), 1127
MagmaticAlgebras.WithBasis.ParentMethods (*class in sage.categories.magmatic_algebras*), 1129
Manifolds (*class in sage.categories.manifolds*), 1133
Manifolds.AlmostComplex (*class in sage.categories.manifolds*), 1133
Manifolds.Analytic (*class in sage.categories.manifolds*), 1133
Manifolds.Connected (*class in sage.categories.manifolds*), 1134
Manifolds.Differentiable (*class in sage.categories.manifolds*), 1134
Manifolds.FiniteDimensional (*class in sage.categories.manifolds*), 1134
Manifolds.ParentMethods (*class in sage.categories.manifolds*), 1135
Manifolds.Smooth (*class in sage.categories.manifolds*), 1135
Manifolds.SubcategoryMethods (*class in sage.categories.manifolds*), 1135
Map (*class in sage.categories.map*), 178
map() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 588
map_coefficients() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1182
map_item() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1184
map_support() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1185
map_support_skip_none() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1186
matrix() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.MorphismMethods method*), 813
MatrixAlgebras (*class in sage.categories.matrix_algebras*), 1139
MatrixFunctor (*class in sage.categories.pushout*), 236
maximal_degree() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 636
maximal_degree() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.Subobjects.ElementMethods method*), 656
maximal_vector() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1084
maximal_vector() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ParentMethods method*), 1092
meet() (*sage.categories.category.Category static method*), 95
meet() (*sage.categories.lattice_posets.LatticePosets.ParentMethods method*), 1032
meet_irreducibles() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 860
meet_irreducibles_poset() (*sage.categories.finite_lattice_posets.FiniteLatticePosets.ParentMethods method*), 860
merge() (*sage.categories.pushout.AlgebraicClosureFunctor method*), 210
merge() (*sage.categories.pushout.AlgebraicExtensionFunctor method*), 214
merge() (*sage.categories.pushout.CompletionFunctor method*), 219
merge() (*sage.categories.pushout.ConstructionFunctor method*), 225
merge() (*sage.categories.pushout.InfinitePolynomialFunctor method*), 233
merge() (*sage.categories.pushout.LaurentPolynomialFunctor method*), 235
merge() (*sage.categories.pushout.MatrixFunctor method*), 237
merge() (*sage.categories.pushout.MultiPolynomialFunctor method*), 240
merge() (*sage.categories.pushout.PermutationGroupFunctor method*), 241
merge() (*sage.categories.pushout.PolynomialFunctor method*), 243
merge() (*sage.categories.pushout.QuotientFunctor method*), 244
merge() (*sage.categories.pushout.SubspaceFunctor*

method), 246
merge() (*sage.categories.pushout.VectorFunctor method*), 247
metapost() (*sage.categories.crystals.Crystals.ParentMethods method*), 540
Metric (*sage.categories.sets_cat.Sets attribute*), 1396
Metric() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1412
metric_function() (*sage.categories.metric_spaces.MetricSpaces.ParentMethods method*), 1144
MetricSpaces (*class in sage.categories.metric_spaces*), 1139
MetricSpaces.CartesianProducts (*class in sage.categories.metric_spaces*), 1140
MetricSpaces.CartesianProducts.ParentMethods (*class in sage.categories.metric_spaces*), 1140
MetricSpacesCategory (*class in sage.categories.metric_spaces*), 1145
MetricSpaces.Complete (*class in sage.categories.metric_spaces*), 1141
MetricSpaces.Complete.CartesianProducts (*class in sage.categories.metric_spaces*), 1141
MetricSpaces.ElementMethods (*class in sage.categories.metric_spaces*), 1142
MetricSpaces.Homsets (*class in sage.categories.metric_spaces*), 1143
MetricSpaces.Homsets.ElementMethods (*class in sage.categories.metric_spaces*), 1143
MetricSpaces.ParentMethods (*class in sage.categories.metric_spaces*), 1143
MetricSpaces.SubcategoryMethods (*class in sage.categories.metric_spaces*), 1144
MetricSpaces.WithRealizations (*class in sage.categories.metric_spaces*), 1144
MetricSpaces.WithRealizations.ParentMethods (*class in sage.categories.metric_spaces*), 1144
milnor_fiber_complex() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.ParentMethods method*), 687
milnor_fiber_poset() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 673
min_demazure_product_greater() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 470
minimal_polynomial() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 809
minpoly() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 811
ModularAbelianVarieties (*class in sage.categories.modular_abelian_varieties*), 1146
ModularAbelianVarieties.Homsets (*class in sage.categories.modular_abelian_varieties*), 1146
ModularAbelianVarieties.Homsets.Endset (*class in sage.categories.modular_abelian_varieties*), 1146
module
    sage.categories.action, 267
    sage.categories.additive_groups, 272
    sage.categories.additive_magmas, 275
    sage.categories.additive_monoids, 293
    sage.categories.additive_semigroups, 295
    sage.categories.affine_weyl_groups, 298
    sage.categories.algebra_functor, 1542
    sage.categories.algebra_ideals, 303
    sage.categories.algebra_modules, 304
    sage.categories.algebras, 305
    sage.categories.algebras_with_basis, 311
    sage.categories.all, 1
    sage.categories.anderson_motives, 320
    sage.categories.aperiodic_semigroups, 328
    sage.categories.associative_algebras, 328
    sage.categories.basic, 44
    sage.categories.bialgebras, 329
    sage.categories.bialgebras_with_basis, 331
    sage.categories.bimodules, 338
    sage.categories.cartesian_product, 1535
    sage.categories.category, 44
    sage.categories.category_cy_helper, 1688
    sage.categories.category_singleton, 1682
    sage.categories.category_types, 1677
    sage.categories.category_with_axiom, 109
    sage.categories.chain_complexes, 340
    sage.categories.classical_crystals, 346
    sage.categories.coalgebras, 352
    sage.categories.coalgebras_with_basis, 361
    sage.categories.coercion_methods, 1691
    sage.categories.commutative_additive_groups, 365
    sage.categories.commutative_additive_monoids, 367
    sage.categories.commutative_additive_semigroups, 368
    sage.categories.commutative_algebra_ideals, 369

sage.categories.commutative_algebras, 369
sage.categories.commutative_ring_ideals, 371
sage.categories.commutative_rings, 371
sage.categories.complete_discrete_valuation, 391
sage.categories.complex_reflection_groups, 398
sage.categories.complex_reflection_or_generalized_coxeter_groups, 402
sage.categories.covariant_functorial_construction, 1527
sage.categories.coxeter_group_algebras, 432
sage.categories.coxeter_groups, 436
sage.categories.crystals, 506
sage.categories.cw_complexes, 547
sage.categories.dedekind_domains, 551
sage.categories.discrete_valuation, 554
sage.categories.distributive_magmas_and_additive_magmas, 559
sage.categories.division_rings, 560
sage.categories.domains, 562
sage.categories.drinfeld_modules, 563
sage.categories.dual, 1541
sage.categories.enumerated_sets, 581
sage.categories.euclidean_domains, 593
sage.categories.examples.algebras_with_basis, 1575
sage.categories.examples.commutative_additive_monoids, 1576
sage.categories.examples.commutative_additive_semigroups, 1579
sage.categories.examples.coxeter_groups, 1582
sage.categories.examples.crystals, 1582
sage.categories.examples.cw_complexes, 1585
sage.categories.examples.facade_sets, 1587
sage.categories.examples.finite_coxeter_groups, 1588
sage.categories.examples.finite_dimensional_algebras_with_basis, 1591
sage.categories.examples.finite_dimensional_lie_algebras_with_basis, 1594
sage.categories.examples.finite_enumerated_sets, 1600
sage.categories.examples.finite_monoids, 1604
sage.categories.examples.finite_semigroups, 1607
sage.categories.examples.finite_weyl_groups, 1611
sage.categories.examples.graded_connected_hopf_algebras_with_basis, 1614
sage.categories.examples.graded_modules_with_basis, 1616
sage.categories.examples.graphs, 1620
sage.categories.examples.hopf_algebras_with_basis, 1622
sage.categories.examples.infinite_enumerated_sets, 1624
sage.categories.examples.lie_algebras, 1627
sage.categories.examples.lie_algebras_with_basis, 1629
sage.categories.examples.magmas, 1632
sage.categories.examples.manifolds, 1633
sage.categories.examples.monoids, 1635
sage.categories.examples.posets, 1638
sage.categories.examples.semigroups, 1641
sage.categories.examples.semigroups_cython, 1648
sage.categories.examples.semirings, 1652
sage.categories.examples.sets_cat, 1654
sage.categories.examples.sets_with_grading, 1665
sage.categories.examples.with_realizations, 1667
sage.categories.facade_sets, 1524
sage.categories.fields, 596
sage.categories.filtered_algebras, 610
sage.categories.filtered_algebras_with_basis, 611
sage.categories.filtered_hopf_algebras_with_basis, 624
sage.categories.filtered_modules, 626
sage.categories.filtered_modules_with_basis, 629
sage.categories.finite_complex_reflection_groups, 657
sage.categories.finite_coxeter_groups, 689
sage.categories.finite_crystals, 712
sage.categories.finite_dimensional_algebras_with_basis, 713
sage.categories.finite_dimensional_bialgebras_with_basis, 753
sage.categories.finite_dimensional_coalgebras_with_basis, 753
sage.categories.finite_dimensional_graded_lie_algebras_with_ba-

sis, 754
sage.categories.finite_dimensional_hopf_algebras_with_basis, 756
sage.categories.finite_dimensional_lie_algebras_with_basis, 757
sage.categories.finite_dimensional_modules_with_basis, 806
sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis, 827
sage.categories.finite_dimensional_semisimple_algebras_with_basis, 829
sage.categories.finite_enumerated_sets, 833
sage.categories.finite_fields, 844
sage.categories.finite_groups, 851
sage.categories.finite_lattice_posets, 855
sage.categories.finite_monoids, 861
sage.categories.finite_permutation_groups, 868
sage.categories.finite_posets, 874
sage.categories.finite_semigroups, 910
sage.categories.finite_sets, 913
sage.categories.finite_weyl_groups, 915
sage.categories.finitely_generated_lambda_bracket_algebras, 916
sage.categories.finitely_generated_lie_conformal_algebras, 918
sage.categories.finitely_generated_magmas, 920
sage.categories.finitely_generated_semigroups, 921
sage.categories.function_fields, 925
sage.categories.functor, 157
sage.categories.g_sets, 926
sage.categories.gcd_domains, 927
sage.categories.generalized_coxeter_groups, 928
sage.categories.graded_algebras, 930
sage.categories.graded_algebras_with_basis, 931
sage.categories.graded_bialgebras, 936
sage.categories.graded_bialgebras_with_basis, 936
sage.categories.graded_coalgebras, 936
sage.categories.graded_coalgebras_with_basis, 938
sage.categories.graded_hopf_algebras, 939
sage.categories.graded_hopf_algebras_with_basis, 939
sage.categories.graded_lie_algebras, 941
sage.categories.graded_lie_algebras_with_basis, 943
sage.categories.graded_lie_conformal_algebras, 943
sage.categories.graded_modules, 944
sage.categories.graded_modules_with_basis, 947
sage.categories.graphs, 955
sage.categories.group_algebras, 958
sage.categories.groupoid, 966
sage.categories.groups, 967
sage.categories.h_trivial_semigroups, 1006
sage.categories.hecke_modules, 983
sage.categories.highest_weight_crystals, 984
sage.categories.homset, 188
sage.categories.homsets, 1556
sage.categories.hopf_algebras, 996
sage.categories.hopf_algebras_with_basis, 1000
sage.categories.infinite_enumerated_sets, 1007
sage.categories.integral_domains, 1009
sage.categories.isomorphic_objects, 1555
sage.categories.j_trivial_semigroups, 1012
sage.categories.kac_moody_algebras, 1012
sage.categories.kahler_algebras, 1014
sage.categories.l_trivial_semigroups, 1098
sage.categories.lambda_bracket_algebras, 1020
sage.categories.lambda_bracket_algebras_with_basis, 1024
sage.categories.lattice_posets, 1026
sage.categories.left_modules, 1040
sage.categories.lie_algebras, 1041
sage.categories.lie_algebras_with_basis, 1062
sage.categories.lie_conformal_algebras, 1066
sage.categories.lie_conformal_algebras_with_basis, 1072
sage.categories.lie_groups, 1075
sage.categories.loop_crystals, 1076
sage.categories.magmas, 1099
sage.categories.magmas_and_additive_magmas, 1122
sage.categories.magmatic_algebras, 1125
sage.categories.manifolds, 1132
sage.categories.map, 165

sage.categories.matrix_algebras, 1139
sage.categories.metric_spaces, 1139
sage.categories.modular_abelian_varieties, 1146
sage.categories.modules, 1147
sage.categories.modules_with_basis, 1166
sage.categories.monoid_algebras, 1228
sage.categories.monoids, 1228
sage.categories.morphism, 201
sage.categories.noetherian_rings, 1240
sage.categories.number_fields, 1243
sage.categories.objects, 1245
sage.categories.ore_modules, 1248
sage.categories.partially_ordered_monoids, 1250
sage.categories.permutation_groups, 1251
sage.categories.pointed_sets, 1252
sage.categories.polyhedra, 1253
sage.categories.poor_man_map, 1691
sage.categories.posets, 1253
sage.categories.primer, 3
sage.categories.principal_ideal_domains, 1271
sage.categories.pushout, 209
sage.categories.quantum_group_representations, 1287
sage.categories.quotient_fields, 1275
sage.categories.quotients, 1553
sage.categories.r_trivial_semigroups, 1348
sage.categories.realizations, 1563
sage.categories.regular_crystals, 1298
sage.categories.regular_supercrystals, 1311
sage.categories.right_modules, 1314
sage.categories.ring_ideals, 1315
sage.categories.rings, 1316
sage.categories.rngs, 1346
sage.categories.schemes, 1349
sage.categories.semigroups, 1353
sage.categories.semirings, 1375
sage.categories.semisimple_algebras, 1376
sage.categories.sets_cat, 1378
sage.categories.sets_with_grading, 1428
sage.categories.sets_with_partial_maps, 1432
sage.categories.shephard_groups, 1433
sage.categories.signed_tensor, 1540
sage.categories.simplicial_complexes, 1433
sage.categories.simplicial_sets, 1436
sage.categories.subobjects, 1554
sage.categories.subquotients, 1553
sage.categories.super_algebras, 1457
sage.categories.super_algebras_with_basis, 1460
sage.categories.super_hopf_algebras_with_basis, 1463
sage.categories.super_lie_conformal_algebras, 1464
sage.categories.super_modules, 1467
sage.categories.super_modules_with_basis, 1471
sage.categories.supercommutative_algebras, 1474
sage.categories.supercrystals, 1475
sage.categories.tensor, 1539
sage.categories.topological_spaces, 1483
sage.categories.triangular_kac_moody_algebras, 1486
sage.categories.tutorial, 163
sage.categories.unique_factorization_domains, 1490
sage.categories.unital_algebras, 1493
sage.categories.vector_bundles, 1497
sage.categories.vector_spaces, 1500
sage.categories.weyl_groups, 1506
sage.categories.with_realizations, 1566
module() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1598
module() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 785
module() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1065
module() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1057
module_generator() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1085
module_morphism() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1203
module_morphism() (*sage.categories.modules.Modules.ParentMethods method*), 1155
Modules (*class in sage.categories.modules*), 1147
Modules.CartesianProducts (*class in sage.categories.modules*), 1149
Modules.CartesianProducts.ElementMethods (*class in sage.categories.modules*), 1149
Modules.CartesianProducts.ParentMethods (*class in sage.categories.modules*), 1149
Modules.ElementMethods (*class in sage.categories.modules*), 1149
Modules.FiniteDimensional (*class in sage.categories.modules*), 1150

Modules.FiniteDimensional.TensorProducts (*class in sage.categories.modules*), 1150
Modules.FinitelyPresented (*class in sage.categories.modules*), 1151
Modules.Homsets (*class in sage.categories.modules*), 1152
Modules.Homsets.Endset (*class in sage.categories.modules*), 1152
Modules.Homsets.ParentMethods (*class in sage.categories.modules*), 1152
Modules.ParentMethods (*class in sage.categories.modules*), 1154
Modules.SubcategoryMethods (*class in sage.categories.modules*), 1157
Modules.TensorProducts (*class in sage.categories.modules*), 1163
Modules.TensorProducts.ParentMethods (*class in sage.categories.modules*), 1163
ModulesWithBasis (*class in sage.categories.modules_with_basis*), 1166
ModulesWithBasis.CartesianProducts (*class in sage.categories.modules_with_basis*), 1169
ModulesWithBasis.CartesianProducts.ParentMethods (*class in sage.categories.modules_with_basis*), 1169
ModulesWithBasis.DualObjects (*class in sage.categories.modules_with_basis*), 1170
ModulesWithBasis.ElementMethods (*class in sage.categories.modules_with_basis*), 1170
ModulesWithBasisHomset (*class in sage.categories.modules_with_basis*), 1227
ModulesWithBasis.Homsets (*class in sage.categories.modules_with_basis*), 1198
ModulesWithBasis.Homsets.ParentMethods (*class in sage.categories.modules_with_basis*), 1198
ModulesWithBasis.MorphismMethods (*class in sage.categories.modules_with_basis*), 1198
ModulesWithBasis.ParentMethods (*class in sage.categories.modules_with_basis*), 1199
ModulesWithBasis.TensorProducts (*class in sage.categories.modules_with_basis*), 1223
ModulesWithBasis.TensorProducts.ElementMethods (*class in sage.categories.modules_with_basis*), 1223
ModulesWithBasis.TensorProducts.ParentMethods (*class in sage.categories.modules_with_basis*), 1226
monoid_generators() (*sage.categories.examples.monoids.FreeMonoid method*), 1637
monoid_generators() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 854
monoid_generators() (*sage.categories.groups.Groups.ParentMethods method*), 980
monoid_generators() (*sage.categories.monoids.Monoids.CartesianProducts.ParentMethods method*), 1234
MonoidAlgebras() (*in module sage.categories.monoid_algebras*), 1228
Monoids (*class in sage.categories.monoids*), 1228
Monoids.Algebras (*class in sage.categories.monoids*), 1229
Monoids.Algebras.ElementMethods (*class in sage.categories.monoids*), 1229
Monoids.Algebras.ParentMethods (*class in sage.categories.monoids*), 1230
Monoids.CartesianProducts (*class in sage.categories.monoids*), 1233
Monoids.CartesianProducts.ElementMethods (*class in sage.categories.monoids*), 1233
Monoids.CartesianProducts.ParentMethods (*class in sage.categories.monoids*), 1234
Monoids.Commutative (*class in sage.categories.monoids*), 1235
Monoids.ElementMethods (*class in sage.categories.monoids*), 1236
Monoids.ParentMethods (*class in sage.categories.monoids*), 1237
Monoids.Subquotients (*class in sage.categories.monoids*), 1239
Monoids.Subquotients.ParentMethods (*class in sage.categories.monoids*), 1239
Monoids.WithRealizations (*class in sage.categories.monoids*), 1239
Monoids.WithRealizations.ParentMethods (*class in sage.categories.monoids*), 1239
monomial() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1211
monomial_coefficients() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra.Element method*), 1595
monomial_coefficients() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1187
monomial_or_zero_if_none() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1212
monomials() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1188
Morphism (*class in sage.categories.morphism*), 203
morphism() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 785
morphism_class() (*sage.categories.category.Category method*), 96
multiplication_table() (*sage.categories.magmas.Magmas.ParentMethods method*), 1106

multiplicative_order() (*sage.categories.monoids.Monoids.CartesianProducts.ElementMethods method*), 1233
MultiPolynomialFunctor (*class in sage.categories.pushout*), 238
MultivariateConstructionFunctor (*class in sage.categories.pushout*), 240
MyGroupAlgebra (*class in sage.categories.examples.hopf_algebras_with_basis*), 1622

N

NaiveCrystal (*class in sage.categories.examples.crystals*), 1584
NaiveCrystal.Element (*class in sage.categories.examples.crystals*), 1585
natural_map() (*sage.categories.homset.Homset method*), 198
nerve() (*sage.categories.finite_monoids.FiniteMonoids.ParentMethods method*), 863
next() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 589
next() (*sage.categories.examples.infinite_enumerated_sets.NonNegativeIntegers method*), 1627
next() (*sage.categories.examples.sets_cat.PrimeNumbers_Abstract method*), 1658
next() (*sage.categories.examples.sets_cat.PrimeNumbers_Abstract.Element method*), 1657
ngens() (*sage.categories.finitely_generated_lambda_bracket_algebras.FinitelyGeneratedLambdaBracketAlgebras.ParentMethods method*), 917
ngens() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1355
Nilpotent (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis attribute*), 759
Nilpotent() (*sage.categories.lie_algebras.LieAlgebras.SubcategoryMethods method*), 1061
nilradical() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 787
nilradical() (*sage.categories.rings.Rings.ParentMethods method*), 1335
nilradical_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 788
NoetherianRings (*class in sage.categories.noetherian_rings*), 1241
NoetherianRings.ElementMethods (*class in sage.categories.noetherian_rings*), 1241
NoetherianRings.ParentMethods (*class in sage.categories.noetherian_rings*), 1241
noncrossing_partition_lattice() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.Irreducible.ParentMethods method*), 666
NonNegativeIntegers (*class in sage.categories.examples.infinite_enumerated_sets*), 1624
NonNegativeIntegers (*class in sage.categories.examples.sets_with_grading*), 1665
normalizer() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 790
normalizer_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 791
NoZeroDivisors (*sage.categories.rings.Rings attribute*), 1322
NoZeroDivisors() (*sage.categories.rings.Rings.SubcategoryMethods method*), 1345
nproduct() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.ElementMethods method*), 1021
number_of_connected_components() (*sage.categories.crystals.Crystals.ParentMethods method*), 541
number_of_irreducible_components() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 423
number_of_reflection_hyperplanes() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 675
number_of_reflections() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 675
number_of_reflections_of_full_support() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods method*), 682
number_of_simple_reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 423
NumberFields (*class in sage.categories.number_fields*), 1243
NumberFields.ElementMethods (*class in sage.categories.number_fields*), 1244
NumberFields.ParentMethods (*class in sage.categories.number_fields*), 1244
numerator() (*sage.categories.complete_discrete_valua-*

tion.CompleteDiscreteValuationFields.ElementMethods method), 393
numerator() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationRings.ElementMethods method*), 397
numerator() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1280

O

object() (*sage.categories.anderson_motives.AndersonMotives method*), 326
object() (*sage.categories.category_types.Elements method*), 1681
object() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 578
Objects (*class in sage.categories.objects*), 1245
Objects.ParentMethods (*class in sage.categories.objects*), 1246
Objects.SubcategoryMethods (*class in sage.categories.objects*), 1246
odd_component() (*sage.categories.super_modules_with_basis.SuperModulesWithBasis.ElementMethods method*), 1474
on_basis() (*sage.categories.modules_with_basis.ModulesWithBasis.MorphismMethods method*), 1198
on_left_matrix() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ElementMethods method*), 722
one() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.CartesianProducts.ParentMethods method*), 314
one() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.ParentMethods method*), 316
one() (*sage.categories.examples.finite_coxeter_groups.DihedralGroup method*), 1591
one() (*sage.categories.examples.finite_dimensional_algebras_with_basis.KroneckerQuiverPathAlgebra method*), 1592
one() (*sage.categories.examples.finite_monoids.IntegerModMonoid method*), 1606
one() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1614
one() (*sage.categories.examples.monoids.FreeMonoid method*), 1637
one() (*sage.categories.examples.semirings.TernaryLogic method*), 1653
one() (*sage.categories.examples.with_realizations.SubsetAlgebra.Bases.ParentMethods method*), 1670
one() (*sage.categories.examples.with_realizations.SubsetAlgebra.Fundamental method*), 1672
one() (*sage.categories.homset.Homset method*), 198
one() (*sage.categories.magmas.Magmas.Unital.CartesianProducts.ParentMethods method*), 1118
one() (*sage.categories.magmas.Magmas.Unital.ParentMethods method*), 1120
one() (*sage.categories.magmas.Magmas.Unital.Realizations.ParentMethods method*), 1121
one() (*sage.categories.monoids.Monoids.SubquotientsParentMethods method*), 1239
one() (*sage.categories.monoids.Monoids.WithRealizations.ParentMethods method*), 1239
one() (*sage.categories.simplicial_sets.SimplicialSets.Homsets.Endset.ParentMethods method*), 1437
one() (*sage.categories.unital_algebras.UnitalAlgebras.CartesianProducts.ParentMethods method*), 1494
one() (*sage.categories.unital_algebras.UnitalAlgebras.WithBasis.ParentMethods method*), 1496
one_basis() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.Algebras.ParentMethods method*), 278
one_basis() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.TensorProducts.ParentMethods method*), 317
one_basis() (*sage.categories.examples.algebras_with_basis.FreeAlgebra method*), 1575
one_basis() (*sage.categories.examples.graded_connected_hopf_algebras_with_basis.GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator method*), 1615
one_basis() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1624
one_basis() (*sage.categories.examples.lie_algebras_with_basis.IndexedPolynomialRing method*), 1631
one_basis() (*sage.categories.examples.with_realizations.SubsetAlgebra.Fundamental method*), 1672
one_basis() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.SignedTensorProducts.ParentMethods method*), 934
one_basis() (*sage.categories.monoids.Monoids.Algebras.ParentMethods method*), 1232
one_basis() (*sage.categories.unital_algebras.UnitalAlgebras.WithBasis.ParentMethods method*), 1496
one_dimensional_configuration_sum() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.TensorProducts.ParentMethods method*), 1092
one_from_cartesian_product_of_one_basis() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.CartesianProducts.ParentMethods method*), 314

one_from_one_basis() (*sage.categories.unital_algebras.UnitalAlgebras.WithBasis.ParentMethods method*), 1497
op (*sage.categories.action.Action attribute*), 269
operation() (*sage.categories.action.Action method*), 269
or_subcategory() (*sage.categories.category.Category method*), 96
order() (*sage.categories.groups.Groups.CartesianProducts.ParentMethods method*), 969
order_filter() (*sage.categories.posets.Posets.ParentMethods method*), 1266
order_filter_generators() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 896
order_ideal() (*sage.categories.posets.Posets.ParentMethods method*), 1267
order_ideal_complement_generators() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 896
order_ideal_generators() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 898
order_ideal_toggle() (*sage.categories.posets.Posets.ParentMethods method*), 1267
order_ideal_toggles() (*sage.categories.posets.Posets.ParentMethods method*), 1268
order_ideals_lattice() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 899
ore_polring() (*sage.categories.anderson_motives.AndersonMotives method*), 327
ore_polring() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 323
ore_polring() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 579
ore_polring() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 574
ore_ring() (*sage.categories.ore_modules.OreModules method*), 1248
ore_variable() (*sage.categories.anderson_motives.AndersonMotives.ParentMethods method*), 323
ore_variable() (*sage.categories.drinfeld_modules.DrinfeldModules.ParentMethods method*), 575
OreModules (*class in sage.categories.ore_modules*), 1248
orthogonal_idempotents_central_mod_radical() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 736
over() (*sage.categories.commutative_rings.CommutativeRings.ParentMethods method*), 387

P

panyushev_complement() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 900
panyushev_orbit_iter() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 900
panyushev_orbits() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 902
parent() (*sage.categories.map.Map method*), 182
parent_class() (*sage.categories.category.Category method*), 97
part() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.ElementMethods method*), 1486
partial_fraction_decomposition() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1280
PartiallyOrderedMonoids (*class in sage.categories.partially_ordered_monoids*), 1250
PartiallyOrderedMonoids.ElementMethods (*class in sage.categories.partially_ordered_monoids*), 1251
PartiallyOrderedMonoids.ParentMethods (*class in sage.categories.partially_ordered_monoids*), 1251
pbw_basis() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1065
peirce_decomposition() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 739
peirce_summand() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 741
permutahedron() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 705
PermutationGroupFunctor (*class in sage.categories.pushout*), 240
PermutationGroups (*class in sage.categories.permutation_groups*), 1251
Phi() (*sage.categories.crystals.Crystals.ElementMethods method*), 516
phi() (*sage.categories.crystals.Crystals.ElementMethods method*), 521
phi() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1302
phi() (*sage.categories.regular_supercrystals.RegularSuperCrystals.ElementMethods method*), 1313
phi_minus_epsilon() (*sage.categories.crystals.Crys-*

tals.ElementMethods method), 521
pieri_factors() (*sage.categories.weyl_groups.WeylGroups.ParentMethods method*), 1520
Plane (*class in sage.categories.examples.manifolds*), 1633
plot() (*sage.categories.crystals.Crystals.ParentMethods method*), 541
plot3d() (*sage.categories.crystals.Crystals.ParentMethods method*), 541
poincare_birkhoff_witt_basis() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ParentMethods method*), 1066
poincare_pairing() (*sage.categories.kahler_algebras.KahlerAlgebras.ParentMethods method*), 1019
Pointed() (*sage.categories.simplicial_sets.SimplicialSets.SubcategoryMethods method*), 1457
PointedSets (*class in sage.categories.pointed_sets*), 1252
PolyhedralSets (*class in sage.categories.polyhedra*), 1253
PolynomialFunctor (*class in sage.categories.pushout*), 242
PoorManMap (*class in sage.categories.poor_man_map*), 1691
Posets (*class in sage.categories.posets*), 1253
Posets.Bounded (*class in sage.categories.posets*), 1255
Posets.Bounded.ParentMethods (*class in sage.categories.posets*), 1256
Posets.ElementMethods (*class in sage.categories.posets*), 1256
Posets.ParentMethods (*class in sage.categories.posets*), 1256
Posets.SubcategoryMethods (*class in sage.categories.posets*), 1270
PositiveIntegerMonoid (*class in sage.categories.examples.facade_sets*), 1587
PositiveIntegersOrderedByDivisibilityFacade (*class in sage.categories.examples.posets*), 1640
PositiveIntegersOrderedByDivisibilityFacade.element_class (*class in sage.categories.examples.posets*), 1640
post_compose() (*sage.categories.map.Map method*), 184
powers() (*sage.categories.monoids.Monoids.ElementMethods method*), 1236
pre_compose() (*sage.categories.map.Map method*), 185
PrecomposedAction (*class in sage.categories.action*), 271
prime_subfield() (*sage.categories.fields.Fields.ParentMethods method*), 609
PrimeNumbers (*class in sage.categories.examples.sets_cat*), 1654
PrimeNumbers_Abstract (*class in sage.categories.examples.sets_cat*), 1656
PrimeNumbers_Abstract.Element (*class in sage.categories.examples.sets_cat*), 1657
PrimeNumbers_Facade (*class in sage.categories.examples.sets_cat*), 1658
PrimeNumbers_Inherits (*class in sage.categories.examples.sets_cat*), 1661
PrimeNumbers_Inherits.Element (*class in sage.categories.examples.sets_cat*), 1663
PrimeNumbers_Wrapper (*class in sage.categories.examples.sets_cat*), 1664
PrimeNumbers_Wrapper.Element (*class in sage.categories.examples.sets_cat*), 1665
principal_ideal() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 742
principal_ideal() (*sage.categories.rngs.Rngs.ParentMethods method*), 1347
principal_lower_set() (*sage.categories.posets.Posets.ParentMethods method*), 1269
principal_order_filter() (*sage.categories.posets.Posets.ParentMethods method*), 1269
principal_order_ideal() (*sage.categories.posets.Posets.ParentMethods method*), 1269
principal_upper_set() (*sage.categories.posets.Posets.ParentMethods method*), 1269
PrincipalIdealDomains (*class in sage.categories.principal_ideal_domains*), 1271
PrincipalIdealDomains.ElementMethods (*class in sage.categories.principal_ideal_domains*), 1271
PrincipalIdealDomains.ParentMethods (*class in sage.categories.principal_ideal_domains*), 1271
print_compare() (*in module sage.categories.sets_cat*), 1428
prod() (*sage.categories.monoids.Monoids.ParentMethods method*), 1237
prod() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1365
product() (*sage.categories.examples.finite_monoids.IntegerModMonoid method*), 1606
product() (*sage.categories.examples.finite_semigroups.LeftRegularBand method*), 1610
product() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1614
product() (*sage.categories.examples.magmas.FreeMagma method*), 1633
product() (*sage.categories.examples.semigroups.FreeSemigroup method*), 1642
product() (*sage.categories.examples.semigroups.LeftZe-*

roSemigroup method), 1644
product() (*sage.categories.examples.semirings.TernaryLogic method*), 1653
product() (*sage.categories.magmas.Magmas.CartesianProducts.ParentMethods method*), 1102
product() (*sage.categories.magmas.Magmas.ParentMethods method*), 1109
product() (*sage.categories.magmas.Magmas.Subquotients.ParentMethods method*), 1117
product() (*sage.categories.magmatic_algebras.MagmaticAlgebras.WithBasis.ParentMethods method*), 1130
product_by_coercion() (*sage.categories.magmas.Magmas.Realizations.ParentMethods method*), 1111
product_from_element_class_mul() (*sage.categories.magmas.Magmas.ParentMethods method*), 1111
product_on_basis() (*sage.categories.additive_magmas.AdditiveMagmas.Algebras.ParentMethods method*), 284
product_on_basis() (*sage.categories.additive_semigroups.AdditiveSemigroups.Algebras.ParentMethods method*), 296
product_on_basis() (*sage.categories.algebras_with_basis.AlgebrasWithBasis.TensorProducts.ParentMethods method*), 318
product_on_basis() (*sage.categories.examples.algebras_with_basis.FreeAlgebra method*), 1576
product_on_basis() (*sage.categories.examples.finite_dimensional_algebras_with_basis.KroneckerQuiverPathAlgebra method*), 1592
product_on_basis() (*sage.categories.examples.graded_connected_hopf_algebras_with_basis.GradedConnectedCombinatorialHopfAlgebraWithPrimitiveGenerator method*), 1615
product_on_basis() (*sage.categories.examples.hopf_algebras_with_basis.MyGroupAlgebra method*), 1624
product_on_basis() (*sage.categories.examples.lie_algebras_with_basis.IndexedPolynomialRing method*), 1631
product_on_basis() (*sage.categories.examples.with_realizations.SubsetAlgebra.Fundamental method*), 1673
product_on_basis() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.SignedTensorProducts.ParentMethods method*), 935
product_on_basis() (*sage.categories.magmatic_algebras.MagmaticAlgebras.WithBasis.ParentMethods method*), 1131
product_on_basis() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1356
product_space() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 792
profile() (*sage.categories.finite_permutation_groups.FinitePermutationGroups.ParentMethods method*), 871
profile_polynomial() (*sage.categories.finite_permutation_groups.FinitePermutationGroups.ParentMethods method*), 872
profile_series() (*sage.categories.finite_permutation_groups.FinitePermutationGroups.ParentMethods method*), 872
projection() (*sage.categories.filtered_algebras_with_basis.FilteredAlgebrasWithBasis.ParentMethods method*), 623
projection() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 653
pseudo_order() (*sage.categories.finite_monoids.FiniteMonoids.ElementMethods method*), 862
pushforward() (*sage.categories.morphism.Morphism method*), 205
pushout() (*in module sage.categories.pushout*), 251
pushout() (*sage.categories.pushout.ConstructionFunctor method*), 226
pushout_lattice() (*in module sage.categories.pushout*), 264

Q

q() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.ParentMethods method*), 1288
q_dimension() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ParentMethods method*), 990
q_dimension() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1085
quadratic_nonresidue() (*sage.categories.finite_fields.FiniteFields.ParentMethods method*), 848
quantum_bruhat_graph() (*sage.categories.weyl_groups.WeylGroups.ParentMethods method*), 1522
quantum_bruhat_successors() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1513
QuantumGroupRepresentations (*class in sage.categories.quantum_group_representations*), 1287
QuantumGroupRepresentations.ParentMethods (*class in sage.categories.quantum_group_representations*), 1287
QuantumGroupRepresentations.TensorProducts

(*class in sage.categories.quantum_group_representations*), 1288
QuantumGroupRepresentations.TensorProducts.ParentMethods (*class in sage.categories.quantum_group_representations*), 1288
QuantumGroupRepresentations.WithBasis (*class in sage.categories.quantum_group_representations*), 1289
QuantumGroupRepresentations.WithBasis.ElementMethods (*class in sage.categories.quantum_group_representations*), 1289
QuantumGroupRepresentations.WithBasis.ParentMethods (*class in sage.categories.quantum_group_representations*), 1292
QuantumGroupRepresentations.WithBasis.TensorProducts (*class in sage.categories.quantum_group_representations*), 1293
QuantumGroupRepresentations.WithBasis.TensorProducts.ParentMethods (*class in sage.categories.quantum_group_representations*), 1293
quo() (*sage.categories.rings.Rings.ParentMethods method*), 1335
quo_rem() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 557
quo_rem() (*sage.categories.euclidean_domains.EuclideanDomains.ElementMethods method*), 595
quo_rem() (*sage.categories.fields.Fields.ElementMethods method*), 602
quotient() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 794
quotient() (*sage.categories.modules.Modules.ParentMethods method*), 1156
quotient() (*sage.categories.rings.Rings.ParentMethods method*), 1337
quotient_module() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.ParentMethods method*), 949
quotient_module() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1212
quotient_ring() (*sage.categories.rings.Rings.ParentMethods method*), 1340
QuotientFields (*class in sage.categories.quotient_fields*), 1275
QuotientFields.ElementMethods (*class in sage.categories.quotient_fields*), 1276
QuotientFields.ParentMethods (*class in sage.categories.quotient_fields*), 1286
QuotientFunctor (*class in sage.categories.pushout*), 243
QuotientOfLeftZeroSemigroup (*class in sage.categories.examples.semigroups*), 1645
QuotientOfLeftZeroSemigroup.Element (*class in sage.categories.examples.semigroups*), 1646
Quotients() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1412
QuotientsCategory (*class in sage.categories.quotients*), 1553

R

r() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1086
R_matrix() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1077
radical() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 745
radical() (*sage.categories.unique_factorization_domains.UniqueFactorizationDomains.ElementMethods method*), 1491
radical_basis() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 746
radical_basis() (*sage.categories.finite_dimensional_semisimple_algebras_with_basis.FiniteDimensionalSemisimpleAlgebrasWithBasis.ParentMethods method*), 833
radical_basis() (*sage.categories.semisimple_algebras.SemisimpleAlgebras.ParentMethods method*), 1378
random_element() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 589
random_element() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts.ParentMethods method*), 837
random_element() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 843
random_element() (*sage.categories.infinite_enumerated_sets.InfiniteEnumeratedSets.ParentMethods method*), 1008
random_element() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1214
random_element() (*sage.categories.rings.Rings.ParentMethods method*), 1342
random_element() (*sage.categories.sets_cat.Sets.CartesianProducts.ParentMethods method*), 1392
random_element_of_length() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 498
random_object() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 579

rank (*sage.categories.pushout.AlgebraicClosureFunctor attribute*), 210
rank (*sage.categories.pushout.AlgebraicExtensionFunctor attribute*), 216
rank (*sage.categories.pushout.BlackBoxConstructionFunctor attribute*), 218
rank (*sage.categories.pushout.CompletionFunctor attribute*), 220
rank (*sage.categories.pushout.FractionField attribute*), 230
rank (*sage.categories.pushout.IdentityConstructionFunctor attribute*), 230
rank (*sage.categories.pushout.InfinitePolynomialFunctor attribute*), 234
rank (*sage.categories.pushout.LaurentPolynomialFunctor attribute*), 236
rank (*sage.categories.pushout.MatrixFunctor attribute*), 238
rank (*sage.categories.pushout.MultiPolynomialFunctor attribute*), 240
rank (*sage.categories.pushout.PermutationGroupFunctor attribute*), 242
rank (*sage.categories.pushout.PolynomialFunctor attribute*), 243
rank (*sage.categories.pushout.QuotientFunctor attribute*), 245
rank (*sage.categories.pushout.SubspaceFunctor attribute*), 247
rank (*sage.categories.pushout.VectorFunctor attribute*), 250
rank() (*sage.categories.complex_reflection_groups.ComplexReflectionGroups.ParentMethods method*), 400
rank() (*sage.categories.enumerated_sets.EnumeratedSets.ElementMethods method*), 584
rank() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 590
rank() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ParentMethods method*), 676
rank() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts.ParentMethods method*), 837
rank() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1215
rational_catalan_number() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.Irreducible.ParentMethods method*), 683
realization_of() (*sage.categories.sets_cat.Sets.Realizations.ParentMethods method*), 1403
Realizations() (*in module sage.categories.realizations*), 1564
Realizations() (*sage.categories.category.Category method*), 69
realizations() (*sage.categories.sets_cat.Sets.WithRealizations.ParentMethods method*), 1425
RealizationsCategory (*class in sage.categories.realizations*), 1566
reduce() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.Subobjects.ParentMethods method*), 803
reduce_to_homology() (*sage.categories.chain_complexes.ChainComplexes.ParentMethods method*), 343
reduced_word() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 471
reduced_word_graph() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 471
reduced_word_reverse_iterator() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 472
reduced_words() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 473
reduced_words_iter() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 475
reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 424
reflection_index_set() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 425
reflection_length() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ElementMethods method*), 411
reflection_length() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 475
reflection_length() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ElementMethods method*), 660
reflection_representation() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 499
reflection_to_coroot() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1514
reflection_to_root() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1515
reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.Par-*

entMethods method), 426
reflections_from_w0() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 707
register_as_coercion() (*sage.categories.morphism.Morphism method*), 205
register_as_conversion() (*sage.categories.morphism.Morphism method*), 206
RegressiveCovariantConstructionCategory (*class in sage.categories.covariant_functorial_construction*), 1535
regular_representation() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1356
regular_representation() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1365
RegularCrystals (*class in sage.categories.regular_crystals*), 1298
RegularCrystals.ElementMethods (*class in sage.categories.regular_crystals*), 1299
RegularCrystals.MorphismMethods (*class in sage.categories.regular_crystals*), 1305
RegularCrystals.ParentMethods (*class in sage.categories.regular_crystals*), 1306
RegularCrystals.TensorProducts (*class in sage.categories.regular_crystals*), 1310
RegularLoopCrystals (*class in sage.categories.loop_crystals*), 1097
RegularLoopCrystals.ElementMethods (*class in sage.categories.loop_crystals*), 1097
RegularSuperCrystals (*class in sage.categories.regular_supercrystals*), 1311
RegularSuperCrystals.ElementMethods (*class in sage.categories.regular_supercrystals*), 1312
RegularSuperCrystals.TensorProducts (*class in sage.categories.regular_supercrystals*), 1313
representation() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1058
representation() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1357
representation() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1365
required_methods() (*sage.categories.category.Category method*), 98
residue_field() (*sage.categories.discrete_valuation.DiscreteValuationFields.ParentMethods method*), 555
residue_field() (*sage.categories.discrete_valuation.DiscreteValuationRings.ParentMethods method*), 558
retract() (*sage.categories.examples.finite_enumerated_sets.IsomorphicObjectOfFiniteEnumeratedSet method*), 1603
retract() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1647
retract() (*sage.categories.sets_cat.Sets.Subquotients.ParentMethods method*), 1419
reversed() (*sage.categories.homset.Homset method*), 199
rhodes_radical_congruence() (*sage.categories.finite_monoids.FiniteMonoids.ParentMethods method*), 866
right_base_ring() (*sage.categories.bimodules.Bimodules method*), 340
right_domain() (*sage.categories.action.Action method*), 269
right_precomposition (*sage.categories.action.PrecomposedAction attribute*), 272
RightModules (*class in sage.categories.right_modules*), 1314
RightModules.ElementMethods (*class in sage.categories.right_modules*), 1314
RightModules.ParentMethods (*class in sage.categories.right_modules*), 1314
ring() (*sage.categories.category_types.Category_ideal method*), 1678
RingIdeals (*class in sage.categories.ring_ideals*), 1315
Rings (*class in sage.categories.rings*), 1316
Rings.ElementMethods (*class in sage.categories.rings*), 1317
Rings.MorphismMethods (*class in sage.categories.rings*), 1318
Rings.ParentMethods (*class in sage.categories.rings*), 1322
Rings.SubcategoryMethods (*class in sage.categories.rings*), 1345
Rngs (*class in sage.categories.rngs*), 1346
Rngs.ParentMethods (*class in sage.categories.rngs*), 1346
rowmotion() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 903
rowmotion_orbit_iter() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 903
rowmotion_orbits() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 905
rowmotion_orbits_plots() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 906
rowmotion_semidistributive() (*sage.categories.lattice_posets.LatticePosets.Semidistributive.ParentMethods method*), 1034
RTrivial (*sage.categories.semigroups.Semigroups attribute*), 1369
RTrivial() (*sage.categories.semigroups.Semigroups.SubcategoryMethods method*), 1373

RTrivial_extra_super_categories() (*sage.categories.l_trivial_semigroups.LTrivialSemigroups method*), 1098
RTrivialSemigroups (*class in sage.categories.r_trivial_semigroups*), 1348

S

s() (*sage.categories.crystals.Crystals.ElementMethods method*), 521
s() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals.ParentMethods method*), 1086
sage.categories.action
    module, 267
sage.categories.additive_groups
    module, 272
sage.categories.additive_magmas
    module, 275
sage.categories.additive_monoids
    module, 293
sage.categories.additive_semigroups
    module, 295
sage.categories.affine_weyl_groups
    module, 298
sage.categories.algebra_functor
    module, 1542
sage.categories.algebra_ideals
    module, 303
sage.categories.algebra_modules
    module, 304
sage.categories.algebras
    module, 305
sage.categories.algebras_with_basis
    module, 311
sage.categories.all
    module, 1
sage.categories.anderson_motives
    module, 320
sage.categories.aperiodic_semigroups
    module, 328
sage.categories.associative_algebras
    module, 328
sage.categories.basic
    module, 44
sage.categories.bialgebras
    module, 329
sage.categories.bialgebras_with_basis
    module, 331
sage.categories.bimodules
    module, 338
sage.categories.cartesian_product
    module, 1535
sage.categories.category
    module, 44
sage.categories.category_cy_helper
    module, 1688
sage.categories.category_singleton
    module, 1682
sage.categories.category_types
    module, 1677
sage.categories.category_with_axiom
    module, 109
sage.categories.chain_complexes
    module, 340
sage.categories.classical_crystals
    module, 346
sage.categories.coalgebras
    module, 352
sage.categories.coalgebras_with_basis
    module, 361
sage.categories.coercion_methods
    module, 1691
sage.categories.commutative_additive_groups
    module, 365
sage.categories.commutative_additive_monoids
    module, 367
sage.categories.commutative_additive_semigroups
    module, 368
sage.categories.commutative_algebra_ideals
    module, 369
sage.categories.commutative_algebras
    module, 369
sage.categories.commutative_ring_ideals
    module, 371
sage.categories.commutative_rings
    module, 371
sage.categories.complete_discrete_valuation
    module, 391
sage.categories.complex_reflection_groups
    module, 398
sage.categories.complex_reflection_or_generalized_coxeter_groups
    module, 402
sage.categories.covariant_functorial_construction
    module, 1527
sage.categories.coxeter_group_algebras
    module, 432
sage.categories.coxeter_groups
    module, 436
sage.categories.crystals
    module, 506
sage.categories.cw_complexes
    module, 547
sage.categories.dedekind_domains

module, 551
sage.categories.discrete_valuation
module, 554
sage.categories.distributive_magmas_and_additive_magmas
module, 559
sage.categories.division_rings
module, 560
sage.categories.domains
module, 562
sage.categories.drinfeld_modules
module, 563
sage.categories.dual
module, 1541
sage.categories.enumerated_sets
module, 581
sage.categories.euclidean_domains
module, 593
sage.categories.examples.algebras_with_basis
module, 1575
sage.categories.examples.commutative_additive_monoids
module, 1576
sage.categories.examples.commutative_additive_semigroups
module, 1579
sage.categories.examples.coxeter_groups
module, 1582
sage.categories.examples.crystals
module, 1582
sage.categories.examples.cw_complexes
module, 1585
sage.categories.examples.facade_sets
module, 1587
sage.categories.examples.finite_coxeter_groups
module, 1588
sage.categories.examples.finite_dimensional_algebras_with_basis
module, 1591
sage.categories.examples.finite_dimensional_lie_algebras_with_basis
module, 1594
sage.categories.examples.finite_enumerated_sets
module, 1600
sage.categories.examples.finite_monoids
module, 1604
sage.categories.examples.finite_semigroups
module, 1607
sage.categories.examples.finite_weyl_groups
module, 1611
sage.categories.examples.graded_connected_hopf_algebras_with_basis
module, 1614
sage.categories.examples.graded_modules_with_basis
module, 1616
sage.categories.examples.graphs
module, 1620
sage.categories.examples.hopf_algebras_with_basis
module, 1622
sage.categories.examples.infinite_enumerated_sets
module, 1624
sage.categories.examples.lie_algebras
module, 1627
sage.categories.examples.lie_algebras_with_basis
module, 1629
sage.categories.examples.magmas
module, 1632
sage.categories.examples.manifolds
module, 1633
sage.categories.examples.monoids
module, 1635
sage.categories.examples.posets
module, 1638
sage.categories.examples.semigroups
module, 1641
sage.categories.examples.semigroups_cython
module, 1648
sage.categories.examples.semirings
module, 1652
sage.categories.examples.sets_cat
module, 1654
sage.categories.examples.sets_with_grading
module, 1665
sage.categories.examples.with_realizations
module, 1667
sage.categories.facade_sets
module, 1524
sage.categories.fields
module, 596
sage.categories.filtered_algebras
module, 610
sage.categories.filtered_algebras_with_basis
module, 611
sage.categories.filtered_hopf_algebras_with_basis
module, 624
sage.categories.filtered_modules
module, 626

sage.categories.filtered_modules_with_basis
    module, 629
sage.categories.finite_complex_reflection_groups
    module, 657
sage.categories.finite_coxeter_groups
    module, 689
sage.categories.finite_crystals
    module, 712
sage.categories.finite_dimensional_algebras_with_basis
    module, 713
sage.categories.finite_dimensional_bialgebras_with_basis
    module, 753
sage.categories.finite_dimensional_coalgebras_with_basis
    module, 753
sage.categories.finite_dimensional_graded_lie_algebras_with_basis
    module, 754
sage.categories.finite_dimensional_hopf_algebras_with_basis
    module, 756
sage.categories.finite_dimensional_lie_algebras_with_basis
    module, 757
sage.categories.finite_dimensional_modules_with_basis
    module, 806
sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis
    module, 827
sage.categories.finite_dimensional_semisimple_algebras_with_basis
    module, 829
sage.categories.finite_enumerated_sets
    module, 833
sage.categories.finite_fields
    module, 844
sage.categories.finite_groups
    module, 851
sage.categories.finite_lattice_posets
    module, 855
sage.categories.finite_monoids
    module, 861
sage.categories.finite_permutation_groups
    module, 868
sage.categories.finite_posets
    module, 874
sage.categories.finite_semigroups
    module, 910
sage.categories.finite_sets
    module, 913
sage.categories.finite_weyl_groups
    module, 915
sage.categories.finitely_generated_lambda_bracket_algebras
    module, 916
sage.categories.finitely_generated_lie_conformal_algebras
    module, 918
sage.categories.finitely_generated_magmas
    module, 920
sage.categories.finitely_generated_semigroups
    module, 921
sage.categories.function_fields
    module, 925
sage.categories.functor
    module, 157
sage.categories.g_sets
    module, 926
sage.categories.gcd_domains
    module, 927
sage.categories.generalized_coxeter_groups
    module, 928
sage.categories.graded_algebras
    module, 930
sage.categories.graded_algebras_with_basis
    module, 931
sage.categories.graded_bialgebras
    module, 936
sage.categories.graded_bialgebras_with_basis
    module, 936
sage.categories.graded_coalgebras
    module, 936
sage.categories.graded_coalgebras_with_basis
    module, 938
sage.categories.graded_hopf_algebras
    module, 939
sage.categories.graded_hopf_algebras_with_basis
    module, 939
sage.categories.graded_lie_algebras
    module, 941
sage.categories.graded_lie_algebras_with_basis
    module, 943
sage.categories.graded_lie_conformal_algebras
    module, 943
sage.categories.graded_modules

module, 944
sage.categories.graded_modules_with_basis
module, 947
sage.categories.graphs
module, 955
sage.categories.group_algebras
module, 958
sage.categories.groupoid
module, 966
sage.categories.groups
module, 967
sage.categories.h_trivial_semigroups
module, 1006
sage.categories.hecke_modules
module, 983
sage.categories.highest_weight_crystals
module, 984
sage.categories.homset
module, 188
sage.categories.homsets
module, 1556
sage.categories.hopf_algebras
module, 996
sage.categories.hopf_algebras_with_basis
module, 1000
sage.categories.infinite_enumerated_sets
module, 1007
sage.categories.integral_domains
module, 1009
sage.categories.isomorphic_objects
module, 1555
sage.categories.j_trivial_semigroups
module, 1012
sage.categories.kac_moody_algebras
module, 1012
sage.categories.kahler_algebras
module, 1014
sage.categories.l_trivial_semigroups
module, 1098
sage.categories.lambda_bracket_algebras
module, 1020
sage.categories.lambda_bracket_algebras_with_basis
module, 1024
sage.categories.lattice_posets
module, 1026
sage.categories.left_modules
module, 1040
sage.categories.lie_algebras
module, 1041
sage.categories.lie_algebras_with_basis
module, 1062
sage.categories.lie_conformal_algebras
module, 1066
sage.categories.lie_conformal_algebras_with_basis
module, 1072
sage.categories.lie_groups
module, 1075
sage.categories.loop_crystals
module, 1076
sage.categories.magmas
module, 1099
sage.categories.magmas_and_additive_magmas
module, 1122
sage.categories.magmatic_algebras
module, 1125
sage.categories.manifolds
module, 1132
sage.categories.map
module, 165
sage.categories.matrix_algebras
module, 1139
sage.categories.metric_spaces
module, 1139
sage.categories.modular_abelian_varieties
module, 1146
sage.categories.modules
module, 1147
sage.categories.modules_with_basis
module, 1166
sage.categories.monoid_algebras
module, 1228
sage.categories.monoids
module, 1228
sage.categories.morphism
module, 201
sage.categories.noetherian_rings
module, 1240
sage.categories.number_fields
module, 1243
sage.categories.objects
module, 1245
sage.categories.ore_modules
module, 1248
sage.categories.partially_ordered_monoids
module, 1250
sage.categories.permutation_groups
module, 1251
sage.categories.pointed_sets
module, 1252
sage.categories.polyhedra
module, 1253
sage.categories.poor_man_map
module, 1691
sage.categories.posets
module, 1253
sage.categories.primer

module, 3
sage.categories.principal_ideal_domains
module, 1271
sage.categories.pushout
module, 209
sage.categories.quantum_group_representations
module, 1287
sage.categories.quotient_fields
module, 1275
sage.categories.quotients
module, 1553
sage.categories.r_trivial_semigroups
module, 1348
sage.categories.realizations
module, 1563
sage.categories.regular_crystals
module, 1298
sage.categories.regular_supercrystals
module, 1311
sage.categories.right_modules
module, 1314
sage.categories.ring_ideals
module, 1315
sage.categories.rings
module, 1316
sage.categories.rngs
module, 1346
sage.categories.schemes
module, 1349
sage.categories.semigroups
module, 1353
sage.categories.semirings
module, 1375
sage.categories.semisimple_algebras
module, 1376
sage.categories.sets_cat
module, 1378
sage.categories.sets_with_grading
module, 1428
sage.categories.sets_with_partial_maps
module, 1432
sage.categories.shephard_groups
module, 1433
sage.categories.signed_tensor
module, 1540
sage.categories.simplicial_complexes
module, 1433
sage.categories.simplicial_sets
module, 1436
sage.categories.subobjects
module, 1554
sage.categories.subquotients
module, 1553
sage.categories.super_algebras
module, 1457
sage.categories.super_algebras_with_basis
module, 1460
sage.categories.super_hopf_algebras_with_basis
module, 1463
sage.categories.super_lie_conformal_algebras
module, 1464
sage.categories.super_modules
module, 1467
sage.categories.super_modules_with_basis
module, 1471
sage.categories.supercommutative_algebras
module, 1474
sage.categories.supercrystals
module, 1475
sage.categories.tensor
module, 1539
sage.categories.topological_spaces
module, 1483
sage.categories.triangular_kac_moody_algebras
module, 1486
sage.categories.tutorial
module, 163
sage.categories.unique_factorization_domains
module, 1490
sage.categories.unital_algebras
module, 1493
sage.categories.vector_bundles
module, 1497
sage.categories.vector_spaces
module, 1500
sage.categories.weyl_groups
module, 1506
sage.categories.with_realizations
module, 1566
scaling_factors() (*sage.categories.crystals.CrystalMorphism method*), 512
Schemes (*class in sage.categories.schemes*), 1352
Schemes_over_base (*class in sage.categories.schemes*), 1352
Section (*class in sage.categories.map*), 186
section() (*sage.categories.map.FormalCompositeMap method*), 175
section() (*sage.categories.map.Map method*), 186
section() (*sage.categories.morphism.IdentityMorphism method*), 202
section() (*sage.categories.morphism.SetIsomorphism method*), 208

semidirect_product() (*sage.categories.groups.Groups.ParentMethods method*), 981
Semidistributive() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1038
semigroup_generators() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 427
semigroup_generators() (*sage.categories.examples.finite_monoids.IntegerModMonoid method*), 1606
semigroup_generators() (*sage.categories.examples.finite_semigroups.LeftRegularBand method*), 1611
semigroup_generators() (*sage.categories.examples.semigroups.FreeSemigroup method*), 1642
semigroup_generators() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 854
semigroup_generators() (*sage.categories.finitely_generated_semigroups.FinitelyGeneratedSemigroups.ParentMethods method*), 923
semigroup_generators() (*sage.categories.monoids.Monoids.ParentMethods method*), 1237
semigroup_generators() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1366
semigroup_generators() (*sage.categories.semigroups.Semigroups.Quotients.ParentMethods method*), 1369
Semigroups (*class in sage.categories.semigroups*), 1353
Semigroups.Algebras (*class in sage.categories.semigroups*), 1354
Semigroups.Algebras.ParentMethods (*class in sage.categories.semigroups*), 1354
Semigroups.CartesianProducts (*class in sage.categories.semigroups*), 1358
Semigroups.ElementMethods (*class in sage.categories.semigroups*), 1359
Semigroups.ParentMethods (*class in sage.categories.semigroups*), 1359
Semigroups.Quotients (*class in sage.categories.semigroups*), 1369
Semigroups.Quotients.ParentMethods (*class in sage.categories.semigroups*), 1369
Semigroups.SubcategoryMethods (*class in sage.categories.semigroups*), 1370
Semigroups.Subquotients (*class in sage.categories.semigroups*), 1373
Semirings (*class in sage.categories.semirings*), 1375
Semisimple (*sage.categories.algebras.Algebras attribute*), 309
Semisimple() (*sage.categories.algebras.Algebras.SubcategoryMethods method*), 309
semisimple_quotient() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 749
SemisimpleAlgebras (*class in sage.categories.semisimple_algebras*), 1376
SemisimpleAlgebras.FiniteDimensional (*class in sage.categories.semisimple_algebras*), 1377
SemisimpleAlgebras.ParentMethods (*class in sage.categories.semisimple_algebras*), 1378
set_base_point() (*sage.categories.simplicial_sets.SimplicialSets.ParentMethods method*), 1438
SetIsomorphism (*class in sage.categories.morphism*), 207
SetMorphism (*class in sage.categories.morphism*), 208
Sets (*class in sage.categories.sets_cat*), 1381
Sets.Algebras (*class in sage.categories.sets_cat*), 1384
Sets.Algebras.ParentMethods (*class in sage.categories.sets_cat*), 1384
Sets.CartesianProducts (*class in sage.categories.sets_cat*), 1386
Sets.CartesianProducts.ElementMethods (*class in sage.categories.sets_cat*), 1387
Sets.CartesianProducts.ParentMethods (*class in sage.categories.sets_cat*), 1388
Sets.ElementMethods (*class in sage.categories.sets_cat*), 1393
Sets.Infinite (*class in sage.categories.sets_cat*), 1394
Sets.Infinite.ParentMethods (*class in sage.categories.sets_cat*), 1394
Sets.Infinite.SubcategoryMethods (*class in sage.categories.sets_cat*), 1395
Sets.IsomorphicObjects (*class in sage.categories.sets_cat*), 1395
Sets.IsomorphicObjects.ParentMethods (*class in sage.categories.sets_cat*), 1396
Sets.MorphismMethods (*class in sage.categories.sets_cat*), 1396
Sets.ParentMethods (*class in sage.categories.sets_cat*), 1397
Sets.Quotients (*class in sage.categories.sets_cat*), 1402
Sets.Quotients.ParentMethods (*class in sage.categories.sets_cat*), 1403
Sets.Realizations (*class in sage.categories.sets_cat*), 1403
Sets.Realizations.ParentMethods (*class in sage.categories.sets_cat*), 1403
Sets.SubcategoryMethods (*class in sage.categories.sets_cat*), 1403

Sets.Subobjects (*class in sage.categories.sets_cat*), 1416
Sets.Subobjects.ParentMethods (*class in sage.categories.sets_cat*), 1417
Sets.Subquotients (*class in sage.categories.sets_cat*), 1417
Sets.Subquotients.ElementMethods (*class in sage.categories.sets_cat*), 1417
Sets.Subquotients.ParentMethods (*class in sage.categories.sets_cat*), 1418
SetsWithGrading (*class in sage.categories.sets_with_grading*), 1428
SetsWithGrading.ParentMethods (*class in sage.categories.sets_with_grading*), 1430
SetsWithPartialMaps (*class in sage.categories.sets_with_partial_maps*), 1432
Sets.WithRealizations (*class in sage.categories.sets_cat*), 1420
Sets.WithRealizations.ParentMethods (*class in sage.categories.sets_cat*), 1420
Sets.WithRealizations.ParentMethods.Realizations (*class in sage.categories.sets_cat*), 1420
shard_poset() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 708
ShephardGroups (*class in sage.categories.shephard_groups*), 1433
sign_representation() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 500
SignedTensorProductFunctor (*class in sage.categories.signed_tensor*), 1540
SignedTensorProducts() (*sage.categories.graded_algebras.GradedAlgebras.SubcategoryMethods method*), 931
SignedTensorProducts() (*sage.categories.graded_coalgebras.GradedCoalgebras.SubcategoryMethods method*), 937
SignedTensorProducts() (*sage.categories.signed_tensor.SignedTensorProductsCategory method*), 1541
SignedTensorProductsCategory (*class in sage.categories.signed_tensor*), 1541
simple_module() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.ParentMethods method*), 1489
simple_module_parameterization() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.Cellular.ParentMethods method*), 718
simple_projection() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 501
simple_projections() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 502
simple_reflection() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 427
simple_reflection() (*sage.categories.examples.finite_weyl_groups.SymmetricGroup method*), 1614
simple_reflection_orders() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 428
simple_reflections() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 429
SimplicialComplexes (*class in sage.categories.simplicial_complexes*), 1433
SimplicialComplexes.Connected (*class in sage.categories.simplicial_complexes*), 1434
SimplicialComplexes.Finite (*class in sage.categories.simplicial_complexes*), 1434
SimplicialComplexes.Finite.ParentMethods (*class in sage.categories.simplicial_complexes*), 1434
SimplicialComplexes.ParentMethods (*class in sage.categories.simplicial_complexes*), 1435
SimplicialComplexes.SubcategoryMethods (*class in sage.categories.simplicial_complexes*), 1435
SimplicialSets (*class in sage.categories.simplicial_sets*), 1436
SimplicialSets.Finite (*class in sage.categories.simplicial_sets*), 1437
SimplicialSets.Homsets (*class in sage.categories.simplicial_sets*), 1437
SimplicialSets.Homsets.Endset (*class in sage.categories.simplicial_sets*), 1437
SimplicialSets.Homsets.Endset.ParentMethods (*class in sage.categories.simplicial_sets*), 1437
SimplicialSets.ParentMethods (*class in sage.categories.simplicial_sets*), 1437
SimplicialSets.Pointed (*class in sage.categories.simplicial_sets*), 1439
SimplicialSets.Pointed.Finite (*class in sage.categories.simplicial_sets*), 1439
SimplicialSets.Pointed.Finite.ParentMethods (*class in sage.categories.simplicial_sets*), 1439
SimplicialSets.Pointed.ParentMethods (*class in sage.categories.simplicial_sets*), 1442
SimplicialSets.SubcategoryMethods (*class in*

sage.categories.simplicial_sets), 1457
smash_product() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.Finite.ParentMethods method*), 1440
Smooth() (*sage.categories.manifolds.Manifolds.SubcategoryMethods method*), 1138
Smooth() (*sage.categories.vector_bundles.VectorBundles.SubcategoryMethods method*), 1499
solvable_radical() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 795
solvable_radical_basis() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 796
some_elements() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups.ParentMethods method*), 430
some_elements() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 590
some_elements() (*sage.categories.examples.semigroups.LeftZeroSemigroup method*), 1645
some_elements() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1648
some_elements() (*sage.categories.examples.semirings.TernaryLogic method*), 1654
some_elements() (*sage.categories.examples.sets_cat.PrimeNumbers_Abstract method*), 1658
some_elements() (*sage.categories.finite_groups.FiniteGroups.ParentMethods method*), 854
some_elements() (*sage.categories.finitely_generated_lambda_bracket_algebras.FinitelyGeneratedLambdaBracketAlgebras.ParentMethods method*), 918
some_elements() (*sage.categories.finitely_generated_lie_conformal_algebras.FinitelyGeneratedLieConformalAlgebras.ParentMethods method*), 919
some_elements() (*sage.categories.finitely_generated_semigroups.FinitelyGeneratedSemigroups.Finite.ParentMethods method*), 922
some_elements() (*sage.categories.sets_cat.Sets.ParentMethods method*), 1402
special_node() (*sage.categories.affine_weyl_groups.AffineWeylGroups.ParentMethods method*), 302
spine() (*sage.categories.lattice_posets.LatticePosets.Semidistributive.ParentMethods method*), 1036
sqrt() (*sage.categories.commutative_rings.CommutativeRings.ElementMethods method*), 373
sqrt() (*sage.categories.finite_fields.FiniteFields.ElementMethods method*), 846
square_root() (*sage.categories.finite_fields.FiniteFields.ElementMethods method*), 848
squarefree_part() (*sage.categories.unique_factorization_domains.UniqueFactorizationDomains.ElementMethods method*), 1492
standard_coxeter_elements() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 503
standard_coxeter_elements() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.WellGenerated.ParentMethods method*), 688
stanley_symmetric_function() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1515
stanley_symmetric_function_as_polynomial() (*sage.categories.weyl_groups.WeylGroups.ElementMethods method*), 1517
stembridgeDel_depth() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1302
stembridgeDel_rise() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1302
stembridgeDelta_depth() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1303
stembridgeDelta_rise() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1303
stembridgeTriple() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1304
step() (*sage.categories.finite_dimensional_nilpotent_lie_algebras_with_basis.FiniteDimensionalNilpotentLieAlgebrasWithBasis.ParentMethods method*), 829
step() (*sage.categories.lie_algebras.LieAlgebras.Nilpotent.ParentMethods method*), 1047
Stone() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1039
Stratified() (*sage.categories.graded_lie_algebras.GradedLieAlgebras.SubcategoryMethods method*), 942
string_parameters() (*sage.categories.highest_weight_crystals.HighestWeightCrystals.ElementMethods method*), 985
structure() (*sage.categories.category.Category method*), 99
structure_coefficients() (*sage.categories.finite_di-*

mensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method), 797
subalgebra() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1599
subalgebra() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ParentMethods method*), 751
subalgebra() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 798
subalgebra() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1059
subcategory_class() (*sage.categories.category.Category method*), 100
subcrystal() (*sage.categories.crystals.Crystals.ElementMethods method*), 522
subcrystal() (*sage.categories.crystals.Crystals.ParentMethods method*), 541
submodule() (*sage.categories.graded_modules_with_basis.GradedModulesWithBasis.ParentMethods method*), 950
submodule() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1216
submonoid() (*sage.categories.monoids.Monoids.ParentMethods method*), 1238
Subobjects() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1413
SubobjectsCategory (*class in sage.categories.subobjects*), 1554
Subquotients() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1414
SubquotientsCategory (*class in sage.categories.subquotients*), 1553
subsemigroup() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1366
subset() (*sage.categories.sets_with_grading.SetsWithGrading.ParentMethods method*), 1431
SubsetAlgebra (*class in sage.categories.examples.with_realizations*), 1667
SubsetAlgebra.Bases (*class in sage.categories.examples.with_realizations*), 1670
SubsetAlgebra.Bases.ParentMethods (*class in sage.categories.examples.with_realizations*), 1670
SubsetAlgebra.Fundamental (*class in sage.categories.examples.with_realizations*), 1671
SubsetAlgebra.In (*class in sage.categories.examples.with_realizations*), 1673
SubsetAlgebra.Out (*class in sage.categories.examples.with_realizations*), 1674
SubspaceFunctor (*class in sage.categories.pushout*), 245
succ_generators() (*sage.categories.finitely_generated_semigroups.FinitelyGeneratedSemigroups.ParentMethods method*), 924
sum() (*sage.categories.additive_monoids.AdditiveMonoids.ParentMethods method*), 294
sum_of_monomials() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1221
sum_of_terms() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1221
summation() (*sage.categories.additive_magmas.AdditiveMagmas.ParentMethods method*), 290
summation() (*sage.categories.examples.commutative_additive_semigroups.FreeCommutativeAdditiveSemigroup method*), 1582
summation() (*sage.categories.examples.semirings.TernaryLogic method*), 1654
summation_from_element_class_add() (*sage.categories.additive_magmas.AdditiveMagmas.ParentMethods method*), 291
Super (*sage.categories.algebras_with_basis.AlgebrasWithBasis attribute*), 317
Super (*sage.categories.algebras.Algebras attribute*), 310
Super (*sage.categories.hopf_algebras_with_basis.HopfAlgebrasWithBasis attribute*), 1005
Super (*sage.categories.lie_conformal_algebras.LieConformalAlgebras attribute*), 1070
Super (*sage.categories.modules_with_basis.ModulesWithBasis attribute*), 1223
Super (*sage.categories.modules.Modules attribute*), 1163
Super() (*sage.categories.graded_lie_conformal_algebras.GradedLieConformalAlgebrasCategory method*), 944
Super() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1161
super_categories() (*sage.categories.additive_magmas.AdditiveMagmas method*), 292
super_categories() (*sage.categories.affine_weyl_groups.AffineWeylGroups method*), 303
super_categories() (*sage.categories.algebra_ideals.AlgebraIdeals method*), 303
super_categories() (*sage.categories.algebra_modules.AlgebraModules method*), 305
super_categories() (*sage.categories.anderson_motives.AndersonMotives method*), 327
super_categories() (*sage.categories.bialgebras.Bialgebras method*), 330
super_categories() (*sage.categories.bimodules.Bimodules method*), 340
super_categories() (*sage.categories.category_types.Elements method*), 1681
super_categories() (*sage.categories.cate-*

gory_with_axiom.Bars method), 141
super_categories() (*sage.categories.category_with_axiom.Blahs method*), 143
super_categories() (*sage.categories.category_with_axiom.CategoryWithAxiom method*), 151
super_categories() (*sage.categories.category_with_axiom.DummyObjectsOverBaseRing method*), 152
super_categories() (*sage.categories.category_with_axiom.TestObjects method*), 153
super_categories() (*sage.categories.category.Category method*), 101
super_categories() (*sage.categories.category.JoinCategory method*), 107
super_categories() (*sage.categories.chain_complexes.ChainComplexes method*), 343
super_categories() (*sage.categories.classical_crystals.ClassicalCrystals method*), 351
super_categories() (*sage.categories.coalgebras.Coalgebras method*), 361
super_categories() (*sage.categories.commutative_algebra_ideals.CommutativeAlgebraIdeals method*), 369
super_categories() (*sage.categories.commutative_ring_ideals.CommutativeRingIdeals method*), 371
super_categories() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationFields method*), 394
super_categories() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationRings method*), 398
super_categories() (*sage.categories.complex_reflection_groups.ComplexReflectionGroups method*), 401
super_categories() (*sage.categories.complex_reflection_or_generalized_coxeter_groups.ComplexReflectionOrGeneralizedCoxeterGroups method*), 432
super_categories() (*sage.categories.covariant_functorial_construction.FunctorialConstructionCategory method*), 1534
super_categories() (*sage.categories.coxeter_groups.CoxeterGroups method*), 505
super_categories() (*sage.categories.crystals.Crystals method*), 547
super_categories() (*sage.categories.cw_complexes.CWComplexes method*), 550
super_categories() (*sage.categories.dedekind_domains.DedekindDomains method*), 554
super_categories() (*sage.categories.discrete_valuation.DiscreteValuationFields method*), 556
super_categories() (*sage.categories.discrete_valuation.DiscreteValuationRings method*), 558
super_categories() (*sage.categories.domains.Domains method*), 562
super_categories() (*sage.categories.drinfeld_modules.DrinfeldModules method*), 580
super_categories() (*sage.categories.enumerated_sets.EnumeratedSets method*), 593
super_categories() (*sage.categories.euclidean_domains.EuclideanDomains method*), 596
super_categories() (*sage.categories.examples.semigroups_cython.IdempotentSemigroups method*), 1649
super_categories() (*sage.categories.examples.with_realizations.SubsetAlgebra.Bases method*), 1671
super_categories() (*sage.categories.filtered_hopf_algebras_with_basis.FilteredHopfAlgebrasWithBasis.WithRealizations method*), 626
super_categories() (*sage.categories.function_fields.FunctionFields method*), 925
super_categories() (*sage.categories.g_sets.GSets method*), 926
super_categories() (*sage.categories.gcd_domains.GcdDomains method*), 927
super_categories() (*sage.categories.generalized_coxeter_groups.GeneralizedCoxeterGroups method*), 929
super_categories() (*sage.categories.graded_hopf_algebras_with_basis.GradedHopfAlgebrasWithBasis.WithRealizations method*), 941
super_categories() (*sage.categories.graphs.Graphs method*), 957
super_categories() (*sage.categories.groupoid.Groupoid method*), 967
super_categories() (*sage.categories.hecke_modules.HeckeModules method*), 984
super_categories() (*sage.categories.highest_weight_crystals.HighestWeightCrystals method*), 995
super_categories() (*sage.categories.homsets.Homsets method*), 1559
super_categories() (*sage.categories.homsets.HomsetsOf method*), 1563
super_categories() (*sage.categories.hopf_algebras.HopfAlgebras method*), 999
super_categories() (*sage.categories.kac_moody_algebras.KacMoodyAlgebras method*), 1013
super_categories() (*sage.categories.kahler_algebras.KahlerAlgebras method*), 1019
super_categories() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras method*), 1023
super_categories() (*sage.categories.lattice_posets.LatticePosets method*), 1040

super_categories() (*sage.categories.left_modules.LeftModules method*), 1041
super_categories() (*sage.categories.lie_algebras.LieAlgebras method*), 1062
super_categories() (*sage.categories.lie_conformal_algebras.LieConformalAlgebras method*), 1070
super_categories() (*sage.categories.lie_groups.LieGroups method*), 1075
super_categories() (*sage.categories.loop_crystals.KirillovReshetikhinCrystals method*), 1094
super_categories() (*sage.categories.loop_crystals.LoopCrystals method*), 1097
super_categories() (*sage.categories.loop_crystals.RegularLoopCrystals method*), 1097
super_categories() (*sage.categories.magmas_and_additive_magmas.MagmasAndAdditiveMagmas method*), 1125
super_categories() (*sage.categories.magmas.Magmas method*), 1122
super_categories() (*sage.categories.magmatic_algebras.MagmaticAlgebras method*), 1132
super_categories() (*sage.categories.manifolds.ComplexManifolds method*), 1132
super_categories() (*sage.categories.manifolds.Manifolds method*), 1138
super_categories() (*sage.categories.matrix_algebras.MatrixAlgebras method*), 1139
super_categories() (*sage.categories.modular_abelian_varieties.ModularAbelianVarieties method*), 1146
super_categories() (*sage.categories.modules.Modules method*), 1165
super_categories() (*sage.categories.noetherian_rings.NoetherianRings method*), 1242
super_categories() (*sage.categories.number_fields.NumberFields method*), 1245
super_categories() (*sage.categories.objects.Objects method*), 1248
super_categories() (*sage.categories.ore_modules.OreModules method*), 1249
super_categories() (*sage.categories.partially_ordered_monoids.PartiallyOrderedMonoids method*), 1251
super_categories() (*sage.categories.permutation_groups.PermutationGroups method*), 1252
super_categories() (*sage.categories.pointed_sets.PointedSets method*), 1252
super_categories() (*sage.categories.polyhedra.PolyhedralSets method*), 1253
super_categories() (*sage.categories.posets.Posets method*), 1270
super_categories() (*sage.categories.principal_ideal_domains.PrincipalIdealDomains method*), 1275
super_categories() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations method*), 1298
super_categories() (*sage.categories.quotient_fields.QuotientFields method*), 1286
super_categories() (*sage.categories.regular_crystals.RegularCrystals method*), 1311
super_categories() (*sage.categories.regular_supercrystals.RegularSuperCrystals method*), 1314
super_categories() (*sage.categories.right_modules.RightModules method*), 1314
super_categories() (*sage.categories.ring_ideals.RingIdeals method*), 1315
super_categories() (*sage.categories.schemes.AbelianVarieties method*), 1350
super_categories() (*sage.categories.schemes.Jacobians method*), 1351
super_categories() (*sage.categories.schemes.Schemes method*), 1352
super_categories() (*sage.categories.schemes.Schemes_over_base method*), 1353
super_categories() (*sage.categories.semisimple_algebras.SemisimpleAlgebras method*), 1378
super_categories() (*sage.categories.sets_cat.Sets method*), 1427
super_categories() (*sage.categories.sets_cat.Sets.WithRealizations.ParentMethods.Realizations method*), 1420
super_categories() (*sage.categories.sets_with_grading.SetsWithGrading method*), 1432
super_categories() (*sage.categories.sets_with_partial_maps.SetsWithPartialMaps method*), 1432
super_categories() (*sage.categories.shephard_groups.ShephardGroups method*), 1433
super_categories() (*sage.categories.simplicial_complexes.SimplicialComplexes method*), 1436
super_categories() (*sage.categories.simplicial_sets.SimplicialSets method*), 1457
super_categories() (*sage.categories.super_modules.SuperModules method*), 1469
super_categories() (*sage.categories.supercrystals.SuperCrystals method*), 1482
super_categories() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras method*), 1490
super_categories() (*sage.categories.unique_factorization_domains.UniqueFactorizationDomains method*), 1493
super_categories() (*sage.categories.vector_bundles.VectorBundles method*), 1499
super_categories() (*sage.categories.vec-*

tor_spaces.VectorSpaces method), 1506
super_categories() (*sage.categories.weyl_groups.WeylGroups method*), 1524
SuperAlgebras (*class in sage.categories.super_algebras*), 1457
SuperAlgebras.ParentMethods (*class in sage.categories.super_algebras*), 1458
SuperAlgebras.SignedTensorProducts (*class in sage.categories.super_algebras*), 1459
SuperAlgebras.SubcategoryMethods (*class in sage.categories.super_algebras*), 1459
SuperAlgebrasWithBasis (*class in sage.categories.super_algebras_with_basis*), 1460
SuperAlgebrasWithBasis.ElementMethods (*class in sage.categories.super_algebras_with_basis*), 1460
SuperAlgebrasWithBasis.ParentMethods (*class in sage.categories.super_algebras_with_basis*), 1462
SuperAlgebrasWithBasis.SignedTensorProducts (*class in sage.categories.super_algebras_with_basis*), 1462
Supercocommutative() (*sage.categories.coalgebras.Coalgebras.Super.SubcategoryMethods method*), 357
Supercommutative (*sage.categories.super_algebras.SuperAlgebras attribute*), 1460
Supercommutative() (*sage.categories.algebras.Algebras.SubcategoryMethods method*), 310
Supercommutative() (*sage.categories.super_algebras.SuperAlgebras.SubcategoryMethods method*), 1459
SupercommutativeAlgebras (*class in sage.categories.supercommutative_algebras*), 1474
SupercommutativeAlgebras.SignedTensorProducts (*class in sage.categories.supercommutative_algebras*), 1475
SupercommutativeAlgebras.WithBasis (*class in sage.categories.supercommutative_algebras*), 1475
SupercommutativeAlgebras.WithBasis.ParentMethods (*class in sage.categories.supercommutative_algebras*), 1475
supercommutator() (*sage.categories.super_algebras_with_basis.SuperAlgebrasWithBasis.ElementMethods method*), 1460
SuperCrystals (*class in sage.categories.supercrystals*), 1475
SuperCrystals.Finite (*class in sage.categories.supercrystals*), 1475
SuperCrystals.Finite.ElementMethods (*class in sage.categories.supercrystals*), 1475
SuperCrystals.Finite.ParentMethods (*class in sage.categories.supercrystals*), 1477
SuperCrystals.ParentMethods (*class in sage.categories.supercrystals*), 1481
SuperCrystals.TensorProducts (*class in sage.categories.supercrystals*), 1482
SuperHopfAlgebrasWithBasis (*class in sage.categories.super_hopf_algebras_with_basis*), 1463
SuperHopfAlgebrasWithBasis.ParentMethods (*class in sage.categories.super_hopf_algebras_with_basis*), 1464
SuperLieConformalAlgebras (*class in sage.categories.super_lie_conformal_algebras*), 1464
SuperLieConformalAlgebras.ElementMethods (*class in sage.categories.super_lie_conformal_algebras*), 1465
SuperLieConformalAlgebras.Graded (*class in sage.categories.super_lie_conformal_algebras*), 1466
SuperLieConformalAlgebras.ParentMethods (*class in sage.categories.super_lie_conformal_algebras*), 1466
SuperModules (*class in sage.categories.super_modules*), 1467
SuperModulesCategory (*class in sage.categories.super_modules*), 1470
SuperModules.ElementMethods (*class in sage.categories.super_modules*), 1467
SuperModules.ParentMethods (*class in sage.categories.super_modules*), 1469
SuperModulesWithBasis (*class in sage.categories.super_modules_with_basis*), 1471
SuperModulesWithBasis.ElementMethods (*class in sage.categories.super_modules_with_basis*), 1471
SuperModulesWithBasis.ParentMethods (*class in sage.categories.super_modules_with_basis*), 1474
support() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 476
support() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1188
support_of_term() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1189
supsets() (*sage.categories.examples.with_realizations.SubsetAlgebra method*), 1675
Surface (*class in sage.categories.examples.cw_complexes*), 1585
Surface.Element (*class in sage.categories.examples.cw_complexes*), 1586
symbol (*sage.categories.cartesian_product.CartesianProductFunctor attribute*), 1538
symbol (*sage.categories.dual.DualFunctor attribute*), 1541
symbol (*sage.categories.signed_tensor.SignedTensorPro-*

ductFunctor attribute), 1540
symbol (*sage.categories.tensor.TensorProductFunctor attribute*), 1539
SymmetricGroup (*class in sage.categories.examples.finite_weyl_groups*), 1611
SymmetricGroup.Element (*class in sage.categories.examples.finite_weyl_groups*), 1612

T

T() (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras.ElementMethods method*), 1020
tensor (*in module sage.categories.tensor*), 1540
tensor() (*sage.categories.crystals.Crystals.ElementMethods method*), 523
tensor() (*sage.categories.crystals.Crystals.ParentMethods method*), 544
tensor() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1190
tensor() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1222
tensor() (*sage.categories.quantum_group_representations.QuantumGroupRepresentations.WithBasis.ParentMethods method*), 1292
tensor() (*sage.categories.super_algebras.SuperAlgebras.ParentMethods method*), 1458
tensor() (*sage.categories.supercrystals.SuperCrystals.ParentMethods method*), 1481
tensor_factors() (*sage.categories.modules.Modules.TensorProducts.ParentMethods method*), 1164
tensor_square() (*sage.categories.modules.Modules.ParentMethods method*), 1156
TensorProductFunctor (*class in sage.categories.tensor*), 1539
TensorProducts() (*sage.categories.crystals.Crystals.SubcategoryMethods method*), 546
TensorProducts() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1161
TensorProducts() (*sage.categories.tensor.TensorProductsCategory method*), 1539
TensorProductsCategory (*class in sage.categories.tensor*), 1539
term() (*sage.categories.modules_with_basis.ModulesWithBasis.ParentMethods method*), 1222
terms() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1190
Ternary (*class in sage.categories.examples.semirings*), 1652
TernaryLogic (*class in sage.categories.examples.semirings*), 1652
TestObjects (*class in sage.categories.category_with_axiom*), 152
TestObjects.Commutative (*class in sage.categories.category_with_axiom*), 152
TestObjects.Commutative.Facade (*class in sage.categories.category_with_axiom*), 152
TestObjects.Commutative.Finite (*class in sage.categories.category_with_axiom*), 153
TestObjects.Commutative.FiniteDimensional (*class in sage.categories.category_with_axiom*), 153
TestObjects.FiniteDimensional (*class in sage.categories.category_with_axiom*), 153
TestObjects.FiniteDimensional.Finite (*class in sage.categories.category_with_axiom*), 153
TestObjects.FiniteDimensional.Unital (*class in sage.categories.category_with_axiom*), 153
TestObjects.FiniteDimensional.Unital.Commutative (*class in sage.categories.category_with_axiom*), 153
TestObjects.Unital (*class in sage.categories.category_with_axiom*), 153
the_answer() (*sage.categories.examples.semigroups.QuotientOfLeftZeroSemigroup method*), 1648
then() (*sage.categories.map.FormalCompositeMap method*), 177
to_finite_dimensional_algebra() (*sage.categories.magmatic_algebras.MagmaticAlgebras.WithBasis.FiniteDimensional.ParentMethods method*), 1128
to_graded_conversion() (*sage.categories.filtered_algebras_with_basis.FilteredAlgebrasWithBasis.ParentMethods method*), 624
to_graded_conversion() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ParentMethods method*), 654
to_highest_weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 523
to_lowest_weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 524
to_matrix() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.ElementMethods method*), 660
to_matrix() (*sage.categories.finite_dimensional_algebras_with_basis.FiniteDimensionalAlgebrasWithBasis.ElementMethods method*), 722
to_module_generator() (*sage.categories.crystals.CrystalMorphismByGenerators method*), 514
to_vector() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra.Element method*), 1595
to_vector() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ElementMethods method*),

759
to_vector() (*sage.categories.lie_algebras_with_basis.LieAlgebrasWithBasis.ElementMethods method*), 1063
to_vector() (*sage.categories.lie_algebras.LieAlgebras.ElementMethods method*), 1045
toggling_orbit_iter() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 907
toggling_orbits() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 909
toggling_orbits_plots() (*sage.categories.finite_posets.FinitePosets.ParentMethods method*), 910
top_degree() (*sage.categories.graded_algebras_with_basis.GradedAlgebrasWithBasis.FiniteDimensional.ParentMethods method*), 932
Topological (*sage.categories.sets_cat.Sets attribute*), 1420
Topological() (*sage.categories.sets_cat.Sets.SubcategoryMethods method*), 1416
TopologicalSpaces (*class in sage.categories.topological_spaces*), 1483
TopologicalSpaces.CartesianProducts (*class in sage.categories.topological_spaces*), 1483
TopologicalSpacesCategory (*class in sage.categories.topological_spaces*), 1485
TopologicalSpaces.Compact (*class in sage.categories.topological_spaces*), 1484
TopologicalSpaces.Compact.CartesianProducts (*class in sage.categories.topological_spaces*), 1484
TopologicalSpaces.Connected (*class in sage.categories.topological_spaces*), 1484
TopologicalSpaces.Connected.CartesianProducts (*class in sage.categories.topological_spaces*), 1484
TopologicalSpaces.SubcategoryMethods (*class in sage.categories.topological_spaces*), 1485
trace() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.Homsets.Endset.ElementMethods method*), 811
trailing_coefficient() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1191
trailing_item() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1192
trailing_monomial() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1194
trailing_support() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1195
trailing_term() (*sage.categories.modules_with_basis.ModulesWithBasis.ElementMethods method*), 1196
transpose() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.FiniteDimensional.ElementMethods method*), 1487
transpose() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.FiniteDimensional.ParentMethods method*), 1487
TriangularKacMoodyAlgebras (*class in sage.categories.triangular_kac_moody_algebras*), 1486
TriangularKacMoodyAlgebras.ElementMethods (*class in sage.categories.triangular_kac_moody_algebras*), 1486
TriangularKacMoodyAlgebras.FiniteDimensional (*class in sage.categories.triangular_kac_moody_algebras*), 1487
TriangularKacMoodyAlgebras.FiniteDimensional.ElementMethods (*class in sage.categories.triangular_kac_moody_algebras*), 1487
TriangularKacMoodyAlgebras.FiniteDimensional.ParentMethods (*class in sage.categories.triangular_kac_moody_algebras*), 1487
TriangularKacMoodyAlgebras.ParentMethods (*class in sage.categories.triangular_kac_moody_algebras*), 1488
Trim() (*sage.categories.lattice_posets.LatticePosets.SubcategoryMethods method*), 1039
trivial_representation() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1060
trivial_representation() (*sage.categories.semigroups.Semigroups.Algebras.ParentMethods method*), 1358
trivial_representation() (*sage.categories.semigroups.Semigroups.ParentMethods method*), 1368
truncate() (*sage.categories.filtered_modules_with_basis.FilteredModulesWithBasis.ElementMethods method*), 638
tuple() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 590
tuple() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 843
tuple() (*sage.categories.infinite_enumerated_sets.InfiniteEnumeratedSets.ParentMethods method*), 1009
twisted_chain_complex() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1451
twisted_homology() (*sage.categories.simpli-*

cial_sets.SimplicialSets.Pointed.ParentMethods method), 1453
twisted_invariant_module() (*sage.categories.finite_dimensional_modules_with_basis.FiniteDimensionalModulesWithBasis.ParentMethods method*), 826
twisting_derivation() (*sage.categories.ore_modules.OreModules method*), 1249
twisting_morphism() (*sage.categories.ore_modules.OreModules method*), 1250
type_to_parent() (*in module sage.categories.pushout*), 265

U

uncamelcase() (*in module sage.categories.category_with_axiom*), 156
unicode_symbol (*sage.categories.signed_tensor.SignedTensorProductFunctor attribute*), 1541
unicode_symbol (*sage.categories.tensor.TensorProductFunctor attribute*), 1539
uniformizer() (*sage.categories.discrete_valuation.DiscreteValuationFields.ParentMethods method*), 555
uniformizer() (*sage.categories.discrete_valuation.DiscreteValuationRings.ParentMethods method*), 558
UniqueFactorizationDomains (*class in sage.categories.unique_factorization_domains*), 1490
UniqueFactorizationDomains.ElementMethods (*class in sage.categories.unique_factorization_domains*), 1491
UniqueFactorizationDomains.ParentMethods (*class in sage.categories.unique_factorization_domains*), 1492
unit_ideal() (*sage.categories.rings.Rings.ParentMethods method*), 1342
Unital (*sage.categories.associative_algebras.AssociativeAlgebras attribute*), 328
Unital (*sage.categories.distributive_magmas_and_additive_magmas.DistributiveMagmasAndAdditiveMagmas.AdditiveAssociative.AdditiveCommutative.AdditiveUnital.Associative attribute*), 559
Unital (*sage.categories.magmatic_algebras.MagmaticAlgebras attribute*), 1126
Unital (*sage.categories.rngs.Rngs attribute*), 1348
Unital (*sage.categories.semigroups.Semigroups attribute*), 1375
Unital() (*sage.categories.category_with_axiom.Blahs.SubcategoryMethods method*), 143
Unital() (*sage.categories.magmas.Magmas.SubcategoryMethods method*), 1115
Unital_extra_super_categories() (*sage.categories.category_with_axiom.Bars method*), 141
UnitalAlgebras (*class in sage.categories.unital_algebras*), 1493
UnitalAlgebras.CartesianProducts (*class in sage.categories.unital_algebras*), 1493
UnitalAlgebras.CartesianProducts.ParentMethods (*class in sage.categories.unital_algebras*), 1494
UnitalAlgebras.ParentMethods (*class in sage.categories.unital_algebras*), 1495
UnitalAlgebras.WithBasis (*class in sage.categories.unital_algebras*), 1495
UnitalAlgebras.WithBasis.ParentMethods (*class in sage.categories.unital_algebras*), 1495
universal_commutative_algebra() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 799
universal_cover() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1455
universal_cover_map() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.ParentMethods method*), 1456
universal_enveloping_algebra() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1600
universal_enveloping_algebra() (*sage.categories.lie_algebras.LieAlgebras.ParentMethods method*), 1060
universal_polynomials() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 800
unpickle_map() (*in module sage.categories.map*), 187
unrank() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 591
unrank() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.CartesianProducts.ParentMethods method*), 839
unrank_range() (*sage.categories.enumerated_sets.EnumeratedSets.ParentMethods method*), 592
unrank_range() (*sage.categories.finite_enumerated_sets.FiniteEnumeratedSets.ParentMethods method*), 844
unset_base_point() (*sage.categories.simplicial_sets.SimplicialSets.Pointed.Finite.ParentMethods method*), 1441
upper_central_series() (*sage.categories.finite_dimensional_lie_algebras_with_basis.FiniteDimensionalLieAlgebrasWithBasis.ParentMethods method*), 801

upper_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 477
upper_covers() (*sage.categories.posets.Posets.ParentMethods method*), 1269

V

valuation() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationFields.ElementMethods method*), 393
valuation() (*sage.categories.complete_discrete_valuation.CompleteDiscreteValuationRings.ElementMethods method*), 398
valuation() (*sage.categories.discrete_valuation.DiscreteValuationFields.ElementMethods method*), 555
valuation() (*sage.categories.discrete_valuation.DiscreteValuationRings.ElementMethods method*), 557
vector_space() (*sage.categories.fields.Fields.ParentMethods method*), 609
VectorBundles (*class in sage.categories.vector_bundles*), 1497
VectorBundles.Differentiable (*class in sage.categories.vector_bundles*), 1498
VectorBundles.Smooth (*class in sage.categories.vector_bundles*), 1498
VectorBundles.SubcategoryMethods (*class in sage.categories.vector_bundles*), 1498
VectorFunctor (*class in sage.categories.pushout*), 247
VectorSpaces (*class in sage.categories.vector_spaces*), 1500
VectorSpaces.CartesianProducts (*class in sage.categories.vector_spaces*), 1500
VectorSpaces.DualObjects (*class in sage.categories.vector_spaces*), 1500
VectorSpaces.ElementMethods (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.Filtered (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.FiniteDimensional (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.FiniteDimensional.TensorProducts (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.Graded (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.ParentMethods (*class in sage.categories.vector_spaces*), 1501
VectorSpaces.TensorProducts (*class in sage.categories.vector_spaces*), 1502
VectorSpaces.WithBasis (*class in sage.categories.vector_spaces*), 1503
VectorSpaces.WithBasis.CartesianProducts (*class in sage.categories.vector_spaces*), 1503
VectorSpaces.WithBasis.Filtered (*class in sage.categories.vector_spaces*), 1503
VectorSpaces.WithBasis.FiniteDimensional (*class in sage.categories.vector_spaces*), 1503
VectorSpaces.WithBasis.FiniteDimensional.TensorProducts (*class in sage.categories.vector_spaces*), 1503
VectorSpaces.WithBasis.Graded (*class in sage.categories.vector_spaces*), 1504
VectorSpaces.WithBasis.TensorProducts (*class in sage.categories.vector_spaces*), 1504
verma_module() (*sage.categories.triangular_kac_moody_algebras.TriangularKacMoodyAlgebras.ParentMethods method*), 1489
vertices() (*sage.categories.examples.graphs.Cycle method*), 1622
vertices() (*sage.categories.graphs.Graphs.ParentMethods method*), 957
virtualization() (*sage.categories.crystals.CrystalMorphism method*), 512

W

w0() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 709
weak_covers() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 478
weak_lattice() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 709
weak_le() (*sage.categories.coxeter_groups.CoxeterGroups.ElementMethods method*), 479
weak_order_ideal() (*sage.categories.coxeter_groups.CoxeterGroups.ParentMethods method*), 503
weak_poset() (*sage.categories.finite_coxeter_groups.FiniteCoxeterGroups.ParentMethods method*), 709
weight() (*sage.categories.crystals.Crystals.ElementMethods method*), 525
weight() (*sage.categories.regular_crystals.RegularCrystals.ElementMethods method*), 1305
weight_lattice_realization() (*sage.categories.crystals.Crystals.ParentMethods method*), 545
weight_lattice_realization() (*sage.categories.loop_crystals.LoopCrystals.ParentMethods method*), 1096
WellGenerated() (*sage.categories.finite_complex_reflection_groups.FiniteComplexReflectionGroups.SubcategoryMethods method*), 677
weyl_group() (*sage.categories.kac_moody_algebras.KacMoodyAlgebras.ParentMethods method*), 1013
WeylGroups (*class in sage.categories.weyl_groups*), 1506

WeylGroups.ElementMethods (*class in sage.categories.weyl_groups*), 1507
WeylGroups.ParentMethods (*class in sage.categories.weyl_groups*), 1519
WithBasis (*sage.categories.algebras.Algebras attribute*), 311
WithBasis (*sage.categories.bialgebras.Bialgebras attribute*), 330
WithBasis (*sage.categories.coalgebras.Coalgebras attribute*), 359
WithBasis (*sage.categories.hopf_algebras.HopfAlgebras attribute*), 999
WithBasis (*sage.categories.lambda_bracket_algebras.LambdaBracketAlgebras attribute*), 1023
WithBasis (*sage.categories.lie_algebras.LieAlgebras attribute*), 1061
WithBasis (*sage.categories.lie_algebras.LieAlgebras.FiniteDimensional attribute*), 1046
WithBasis (*sage.categories.lie_conformal_algebras.LieConformalAlgebras attribute*), 1070
WithBasis (*sage.categories.modules.Modules attribute*), 1165
WithBasis (*sage.categories.semisimple_algebras.SemisimpleAlgebras.FiniteDimensional attribute*), 1378
WithBasis() (*sage.categories.modules.Modules.SubcategoryMethods method*), 1162
WithRealizations() (*in module sage.categories.with_realizations*), 1566
WithRealizations() (*sage.categories.category.Category method*), 71
WithRealizationsCategory (*class in sage.categories.with_realizations*), 1573
wrapped_class (*sage.categories.examples.finite_coxeter_groups.DihedralGroup.Element attribute*), 1590
wrapped_class (*sage.categories.examples.finite_monoids.IntegerModMonoid.Element attribute*), 1605
wrapped_class (*sage.categories.examples.finite_semigroups.LeftRegularBand.Element attribute*), 1610
wrapped_class (*sage.categories.examples.magmas.FreeMagma.Element attribute*), 1632
wrapped_class (*sage.categories.examples.monoids.FreeMonoid.Element attribute*), 1637
wrapped_class (*sage.categories.examples.posets.FiniteSetsOrderedByInclusion.Element attribute*), 1639
wrapped_class (*sage.categories.examples.semigroups.FreeSemigroup.Element attribute*), 1642

X

xgcd() (*sage.categories.fields.Fields.ElementMethods method*), 602
xgcd() (*sage.categories.quotient_fields.QuotientFields.ElementMethods method*), 1284

Z

zero() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.CartesianProducts.ParentMethods method*), 279
zero() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.Homsets.ParentMethods method*), 279
zero() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.ParentMethods method*), 281
zero() (*sage.categories.additive_magmas.AdditiveMagmas.AdditiveUnital.WithRealizations.ParentMethods method*), 282
zero() (*sage.categories.examples.commutative_additive_monoids.FreeCommutativeAdditiveMonoid method*), 1578
zero() (*sage.categories.examples.finite_dimensional_lie_algebras_with_basis.AbelianLieAlgebra method*), 1600
zero() (*sage.categories.examples.lie_algebras.LieAlgebraFromAssociative method*), 1629
zero() (*sage.categories.modules.Modules.Homsets.ParentMethods method*), 1153
zero_ideal() (*sage.categories.rngs.Rngs.ParentMethods method*), 1348
zeta() (*sage.categories.finite_fields.FiniteFields.ParentMethods method*), 848
zeta() (*sage.categories.rings.Rings.ParentMethods method*), 1343
zeta_function() (*sage.categories.number_fields.NumberFields.ParentMethods method*), 1244
zeta_order() (*sage.categories.finite_fields.FiniteFields.ParentMethods method*), 850
zeta_order() (*sage.categories.rings.Rings.ParentMethods method*), 1344